기술과 경제학으로 읽기

법률가를 위한 블록체인

법학자가 고교 수학만으로 끝까지 읽을 수 있게 쓴 블록체인 자습서. 해시와 서명에서 수수료 시장과 스테이블코인까지 기술과 경제학의 말로 풀고, 법의 질문이 생기는 자리를 표시했다.

권보원(대구지방법원 부장판사) 엮음

구성 · 0~6부 30장 + 부록 A~F
인터랙티브 그림 · 12종(화면본. 인쇄본은 같은 자리에 정지 그림)
노트북 · Colab 21권(표준 라이브러리, 25장만 numpy)
수치 기준일 · 2026. 10. 25.

인쇄 · Ctrl+P → PDF로 저장하면 접히는 상자가 모두 펼쳐집니다.
오프라인 · 이 파일과 옆의 fonts 폴더만으로 열립니다. 인터랙티브 그림도 인터넷 없이 움직이고, 링크는 인터넷이 있을 때 열립니다.

이 책은 학술 자습서이며 투자 권유나 법률 자문이 아닙니다. 고지 전문은 판권면(부록 F.4)에 있습니다.

1.0판 · 2026. 11. 14. · 생성형 AI와 함께 엮었습니다. 틀린 곳이 있을 수 있으며 앞으로도 계속 고쳐 나가겠습니다.

차례

1장 블록체인이 푸는 문제 — 믿을 제3자 없이 이중지불을 막기 **11**

학습목표 11

1. 복사가 공짜인 세계 11

2. 세 가지 장부 13

3. 왜 어려운가 — 순서와 다수 15

4. 대가 — 느림, 비용, 그리고 「완료」가 없다는 것 15

5. 이 책의 지도 16

6. 정리 17

이 장의 출처 17

2장 30년 전사(前史) — 전자현금에서 백서까지 **19**

학습목표 19

1. 연표 — 읽기 전에 규칙 둘 20

2. 1976 — 열쇠를 둘로 나누다 22

3. 1982 / 1983 — 차움의 전자현금: 익명성은 얻고 은행은 남다 23

4. 1980 / 1987 — 머클의 트리, 1982 — 비잔틴 장군 24

5. 1991 / 1993 — 하버·스토네타의 사슬: 순서는 고정했으나 24

6. 1997 — 해시캐시: 계산으로 비용을 매기다 25

7. 1998 — b-money: 돈을 그렸으나 합의는 비워 두다 25

8. 2005 — bit gold: 등기부와 타임스탬프 26

9. 2008 — 백서: 조립 26

10. 2013 / 2014 — 이더리움: 원장 위에서 프로그램을 27

11. 2014 ~ 2023 — 백서 뒤의 이정표, 이 책의 뒷장들 27

12. 정리 28

이 장의 출처 28

3장 이 책을 읽는 법 — 표식·상자·위젯·노트북, 세 경로 **33**

학습목표 33

1. 표식 세 등급 34

2. 상자 셋 34

3. 위젯과 노트북 35

4. 세 경로 36

5. 일러두기 — 수치, 기준일, 표시 37

6. 정리 38

이 장의 출처 38

4장 해시 함수 — 지문을 찍는 일방향 함수 **39**

학습목표 39

1. 이 기술은 무엇의 실패에서 나왔나 39
2. 직관 — 지문, 그리고 지문이 하지 못하는 일 40
3. 코드 — 한 글자가 전부를 바꾼다 42
4. [중급] 생일 역설 — 충돌 저항은 비트 수의 절반만큼만 강하다 46
5. SHA-256은 무엇인가 47
6. 정리 — 「변조 불가」의 절반 50
이 장의 출처 50

5장 공개키 서명 — 열쇠 둘로 권한을 나누기 52

학습목표 52
1. 이 기술은 무엇의 실패에서 나왔나 52
2. 직관 — 서명은 승인이고, 암호화가 아니다 53
3. [중급] 서명식을 읽는다 — ECDSA 55
4. 코드 — 장난감 곡선 위에서 전부 해 본다 57
5. 정리 61
이 장의 출처 62

6장 머클 트리 — 하나의 뿌리로 전체를 보증하기 64

학습목표 64
1. 이 기술은 무엇의 실패에서 나왔나 64
2. 직관 — 토너먼트 대진표 65
3. [중급] 증명의 길이 — 왜 $\log_2 n$ 인가 66
4. 비트코인의 머클 트리 — 두 가지 구현 특성과 한 가지 약점 69
5. 정리 — 뿌리가 보증하는 것 71
이 장의 출처 71

7장 블록과 체인 — 변조가 비싸지는 구조 74

학습목표 74
1. 이 기술은 무엇의 실패에서 나왔나 74
2. 블록은 무엇으로 되어 있나 75
3. 변조는 왜 연쇄적으로 드러나는가 76
4. 코드 — 장난감 체인 78
5. 정리 80
이 장의 출처 81

8장 작업증명 — 난이도, 채굴, 가장 긴 체인 83

학습목표 83
1. 이 기술은 무엇의 실패에서 나왔나 83
2. 직관 — 복권 84
3. [중급] 수식 — 성공 확률, 기대 시도 횟수, 난이도 85

4. 코드 — 손 채굴기 86
5. 「가장 긴 체인」과 점유율 q 91
6. 정리 92
이 장의 출처 92

9장 최종성의 확률적 성격 — 「확인 수」의 뜻 95
학습목표 95
1. 공격자는 무엇을 할 수 있고 무엇을 할 수 없나 95
2. [중급] 백서 §11의 식 — 두 단계 96
3. 코드 — 백서 표의 재현과 그 너머 97
4. 「완료」는 확률이다 104
5. 정리 105
이 장의 출처 105

10장 지분증명 — 에너지 대신 담보로 107
학습목표 107
1. 이 기술은 무엇의 실패에서 나왔나 107
2. 직관 — 보증금, 투표, 그리고 처벌 108
3. 현행 규칙 — 숫자는 명세에서 110
4. 코드 — 담보 가중 선택과 슬래싱 산수 111
5. 작업증명과 지분증명 — 비용의 종류가 다르다 114
6. 정리 115
이 장의 출처 115

11장 UTXO와 계정 — 원장이 「잔액」을 적는 두 방식 118
학습목표 118
1. 이 기술은 무엇의 실패에서 나왔나 118
2. UTXO — 쓰지 않은 출력의 더미 119
3. 계정 — 주소마다 상태를 적는다 120
4. [중급] 코드 — 두 상태 전이 함수 121
5. 두 모형의 비교 — 그리고 「착오 이체」의 두 모양 123
6. 정리 124
이 장의 출처 124

12장 진짜 블록 해부 — 제네시스 블록 80바이트를 손으로 풀기 126
학습목표 126
1. 데이터는 어디서 왔나 126
2. 80바이트를 여섯 칸으로 127
3. 해시를 다시 계산해 공개값과 맞춘다 129
4. [중급] 거래를 풀어 본다 — 코인베이스와 그 안의 문장 130

5. 2026년의 블록과 견주면 133
6. 정리 — 1부를 닫으며 133
이 장의 출처 134
13장 이더리움 가상머신과 가스 — 코드를 실행하는 원장 135
학습목표 135
1. 이 기술은 무엇의 실패에서 나왔나 135
2. 직관 — 계정, 코드, 거래, 메시지 136
3. 가스 — 멈추지 않는 프로그램을 멈추는 법 138
4. 읽기용 Solidity — 가장 작은 계약 141
5. 코드 — 장난감 EVM으로 상태 전이와 가스 소진을 흉내 낸다 142
6. 정리 — 「계약」이라는 이름의 프로그램 147
이 장의 출처 147
14장 토큰 표준 — ERC-20·721이 정하는 것과 정하지 않는 것 152
학습목표 152
1. 이 기술은 무엇의 실패에서 나왔나 153
2. 직관 — 코인과 토큰, 그리고 장부 하나 153
3. ERC-20 — 대체 가능 토큰이 약속하는 여섯 함수 155
4. ERC-721 — 하나씩 다른 것의 장부 156
5. 토큰이 가리키는 것 — 온체인 권리와 오프체인 약속 158
6. 코드 — 장부 둘을 흉내 내고, 표준이 적은 번호를 다시 계산한다 159
7. 정리 165
이 장의 출처 166
15장 탈중앙 거래소와 AMM — 호가창 없는 가격 결정 170
학습목표 170
1. 이 기술은 무엇의 실패에서 나왔나 171
2. 직관 — 통 두 개와 곱 하나 172
3. 수식 — 상수곱, 수수료, 실행 가격 173
4. 유동성 공급 — 수수료 수입과 비영구 손실은 같은 저울의 양쪽 174
5. 현물가는 오라클이 아니다 — 16·17장으로 가는 다리 176
6. 코드 — 상수곱 풀, 슬리피지 곡선, 그리고 저울의 양쪽 176
7. 정리 180
이 장의 출처 180
16장 오라클 문제 — 체인 밖 사실을 체인 안으로 183
학습목표 183
1. 이 기술은 무엇의 실패에서 나왔나 183
2. 왜 체인은 바깥을 모르는가 184

3. 이상적인 오라클, 그리고 그것이 불가능한 이유 185
4. 설계의 세 갈래 — 무엇을 믿어야 하는가 186
5. 조작은 어느 층에서 일어나는가 188
6. 정리 189
이 장의 출처 190

17장 MEV와 거래 순서 — 순서를 정하는 권력 193
학습목표 193
1. 이 기술은 무엇의 실패에서 나왔나 194
2. 직관 — 앞에 끼우기, 뒤에 끼우기, 양쪽에 끼우기 195
3. 누가 순서를 정하는가 — 경매와 생산자 196
4. 대응 — 금지가 아니라 분업: 제안자–빌더 분리 197
5. 코드 — 샌드위치를 풀 단위로 돌려 본다 199
6. 정리 203
이 장의 출처 203

18장 확장성 — 레이어 2와 롤업 207
학습목표 207
1. 이 기술은 무엇의 실패에서 나왔나 208
2. 롤업의 구조 — 실행은 밖, 데이터와 분쟁은 안 208
3. 낙관적 롤업 — 틀렸다고 증명될 때까지 옳다 210
4. 영지식 롤업 — 옳다는 증명을 매번 첨부한다 212
5. 두 방식의 비교 — 누가 정직해야 하는가 213
6. 신뢰는 사라지지 않고 옮겨 간다 213
7. 정리 — 2부를 닫으며 214
이 장의 출처 215

19장 화폐의 기능론과 「내재가치 0」 균형 220
학습목표 220
1. 이 장이 답하려는 질문 221
2. 기능론 — 점검표이지 증명이 아니다 221
3. [중급] 교환방정식과 그 한계 222
4. 「내재가치 0」의 균형 — 세 모형 223
5. 코드 — 균형의 열거를 숫자로 227
6. 모형이 말하는 것과 말하지 않는 것 229
7. 정리 230
이 장의 출처 230

20장 보안 예산 — 보상에서 수수료로 236
학습목표 236

1. 이 장이 답하려는 질문 237
2. 보상의 두 조각 — 보조금과 수수료 237
3. 부디시의 세 식 — 보안은 흐름이다 239
4. 공격은 얼마나 걸리고 얼마가 드는가 — 명제 3과 표 II 241
5. 공격 비용 계산기 — 입력은 해시율·단가·시간 244
6. [심화] 보조금이 0이 된 세계 — Carlsten 외 246
7. [심화] 재고가 걸려 있으면 — 전문 자본과 붕괴 248
8. 정리 249
이 장의 출처 249

21장 수수료 시장 — 경매에서 EIP-1559로 254
학습목표 254
1. 이 장이 답하려는 질문 255
2. 1차 가격 경매 — 무엇이 문제였나 255
3. EIP-1559 — 공식으로 정하는 통행료 256
4. 세 가지 유인양립 — 누가 속일 수 있는가 259
5. [심화] 블롭 수수료 시장 — 같은 원리, 다른 자원 263
6. 정리 265
이 장의 출처 265

22장 토큰 설계의 경제학 — 발행 규칙·거버넌스·인센티브 270
학습목표 270
1. 이 장이 답하려는 질문 271
2. 발행 — 얼마나, 어떤 규칙으로 271
3. 분배 — 처음에 누가 받는가 272
4. 거버넌스 — 규칙을 누가 바꾸는가 275
5. 「토큰 경제학」이 마케팅 용어로 쓰이는 점 277
6. 정리 277
이 장의 출처 278

23장 게임이론 — 이기적 채굴과 균형 281
학습목표 281
1. 이 장이 답하려는 질문 282
2. 백서의 논증을 이득표로 282
3. 통념 — 「정직한 채굴은 내시 균형이다」 283
4. 이기적 채굴 — 블록을 숨기는 전략 283
5. 같은 틀의 다른 자리 — 조정 게임, 협조적 균형, 언더컷 288
6. 정리 291
이 장의 출처 291

24장 스테이블코인 — 설계 유형과 뱅크런 구조 295
학습목표 295
1. 이 장이 답하려는 질문 296
2. 네 유형 — 약속을 무엇이 받치는가 296
3. 왜 런이 생기는가 — 「묻지 않고 받음」의 경제학 298
4. [심화] 다이아몬드–딥비그 — 런은 균형이다 299
5. 스테이블코인에 옮겨 놓기 — 같은 것과 다른 것 301
6. 실증 — 두 번의 런 307
7. 정리 309
이 장의 출처 309

25장 버블의 계량검정 — 무엇을 재고 무엇을 못 재는가 315
학습목표 315
1. 이 장이 답하려는 질문 316
2. 버블의 정의가 요구하는 두 가정 316
3. [중급] ADF 회귀와 폭발 계수 317
4. [심화] GSADF, 최소 창, 날짜 찍기, 그리고 임계값 319
5. 기초가치를 무엇으로 두는가 — 세 답 322
6. 문헌의 판정표 — 표본·기간·가정·임계값을 함께 324
7. 말하는 것과 말하지 않는 것 326
8. 정리 327
이 장의 출처 327

26장 네 사건 — Mt.Gox · The DAO · Terra-Luna · FTX 332
학습목표 332
1. 이 장이 읽는 방식 — 진단표와 「블록체인 자체」의 범위 333
2. Mt.Gox (2014) — 장부와 체인의 불일치 335
3. The DAO (2016) — 계약 코드의 결함과 그 뒤의 결정 339
4. Terra-Luna (2022) — 설계의 반사성과 발행자의 진술 341
5. FTX (2022) — 고객 자산의 유용과 내부통제의 부재 342
6. 네 사건을 한 표에 344
7. 「블록체인 자체는 깨지지 않았다」의 범위 344
8. 정리 345
이 장의 출처 345

27장 기술과 법의 다리 (1) — 깊이 다루지 않은 이유, 학설 구도, 법역별 서베이 353
학습목표 353
27.1 이 책이 법을 깊이 다루지 않은 이유 353
27.2 법적 성질의 학설 구도 — 견해와 기술 사실의 대응 354
27.3 법역별 서베이 362

정리 369
이 장의 출처 369
28장 기술과 법의 다리 (2) — 쟁점별 개관, 더 읽을 곳 378
학습목표 378
28.1 쟁점별 개관 378
28.2 더 읽을 곳 388
정리 390
이 장의 출처 390
29장 양자 전환 — 서명은 깨지고 해시는 버틴다, 그리고 취약 코인 394
학습목표 394
1. 두 알고리듬, 두 표적 — 비대칭 395
2. 자원 추정 — 큐비트 둘, 가정 다섯 397
3. 무엇이 노출되어 있나 — 공개키의 세 자리 400
4. 전환 제안 — 출력 유형, 서명, 일몰 405
5. 취약 코인 — 세 갈래와 한 질문 408
6. 「2026-10 기준」 — 이 장에서 낡을 것들 410
7. 정리 410
이 장의 출처 411
30장 명명 문제 — 가상자산·crypto-asset·digital asset·暗号資産·cryptoasset 419
학습목표 419
1. 왜 이름이 문제인가 420
2. 다섯 법역의 정의 조문 — 원문 420
3. 조문을 겹쳐 읽기 — 넣은 것과 뺀 것 426
4. 이름이 가르는 것 — 관할·과세·형사 428
5. 한 법역 안에서 이름이 바뀐 세 사례 430
6. 정리 431
이 장의 출처 431
부록 A 동향 노트 · 색인 (2026-10 기준 — 1~6부·0부 기재분) 434
부록 B 흔한 오해 판정표 440
출처 443
부록 C 기술 용어 ↔ 법령 용어 대역표 444
C.1 다섯 법역의 정의어 (30장 2절) 444
C.2 제외 목록 (30장 3절의 표) 447
C.3 기술 용어는 어느 낱말에 닿는가 447
출처 450
부록 D 연표 — 1976~2023 (클릭 연표 W11의 정지 판) 451

1. 29개 이정표 451
2. 클릭 연표 (화면본) 452
출처 453
부록 E 수학 보충 454
대응표 — 어느 상자가 여기를 가리키는가 454
E.1 머클–담고르 구성 — 압축 함수를 이어 붙이기 454
출처 455
부록 F 용어집 · 인터랙티브 그림 목록 · 노트북 목록 · 판권 457
F.1 용어집 457
F.2 인터랙티브 그림(위젯) 목록 461
F.3 노트북 목록 461
F.4 판권 463
출처 464

0부 지도

1장 블록체인이 푸는 문제 — 믿을 제3자 없이 이중지불을 막기

핵심 TAKEAWAY

디지털 파일은 복사가 공짜입니다. 그래서 디지털 「돈」을 만들려면 **같은 돈을 두 번 쓰는 일**(이중지불)을 막아야 하고, 지금까지의 답은 언제나 「장부를 가진 한 사람」 — 은행, 조폐소, 결제 회사 — 을 믿는 것이었습니다. 블록체인은 그 한 사람 없이 같은 일을 하는 방법입니다. 거래를 모두에게 공개하고, 어느 거래가 먼저인지를 **다수가 계산으로 합의한** 하나의 역사에 적습니다. 그 대가는 느림과 비용입니다. 10분에 한 번 기록되고, 전기와 장비가 들며, 기록은 「완료」되지 않고 되돌릴 확률이 작아질 뿐입니다. 법률가가 이 장에서 가져갈 것은 하나입니다 — 블록체인은 「복사가 공짜인 세계에서 하나뿐인 기록을 여럿이 합의해 유지하는 방법」이고, 그래서 이 책의 모든 법적 질문은 「그 기록이 여럿인데 어느 것이 원본인가」에서 시작합니다.

학습목표

1. 「이중지불」이 왜 디지털 현금의 핵심 난제인지 설명한다.
2. 중앙 원장·분산 원장·블록체인을 「누가 장부를 쓰는가」와 「어느 기록이 먼저인지를 누가 정하는가」로 구별한다.
3. 블록체인이 치르는 대가 — 느림, 비용, 확률적 최종성 — 를 그 해법의 구조에서 끌어낸다.
4. 이 책의 여섯 부가 각각 이 문제의 어느 조각을 다루는지 안다.

[입문]만 읽은 독자가 가져가는 것. 블록체인은 「복사가 공짜인 세계에서 하나뿐인 기록을 여럿이 합의해 유지하는 방법」이고, 그 대가가 느림과 비용입니다.

1. 복사가 공짜인 세계

지폐 한 장을 두 사람에게 동시에 줄 수는 없습니다. 종이는 하나이고 건네면 내 손에서 사라집니다. 파일은 다릅니다. 「100원짜리 파일」을 만들어 A에게 보내도 내 컴퓨터에 그대로 남고, 그것을 B에게 다시 보낼 수 있습니다. 둘 다 진짜입니다 — 비트 하나 다르지 않으니까요. 이것이 **이중지불(double spending)** 문제이고, 디지털 현금을 만들려는 모든 시도가 부딪힌 벽입니다. 1982년 학회에서 차움(David Chaum)이 제안한 은닉 서명 전자현금은 은행이 서명한 전자 지폐를 발행하되 은행이 **누구에게** 발행했는지 모르게 하는 데 성공했습니다. 그런데 그 체계의 지급 절차 열세 단계 가운데 열한 번째는 이렇습니다. 「은행은 지폐를 결제된 지폐의 종합 목록에 추가하고, 그 지폐가 이미 목록에 있으면 멈춘다.」[1] 같은 지폐가 두 번 오는지를 **은행이** 확인합니다. 익명성은 얻었지만 이중지불의 감시자는 여전히 은행 한 곳이었습니다.

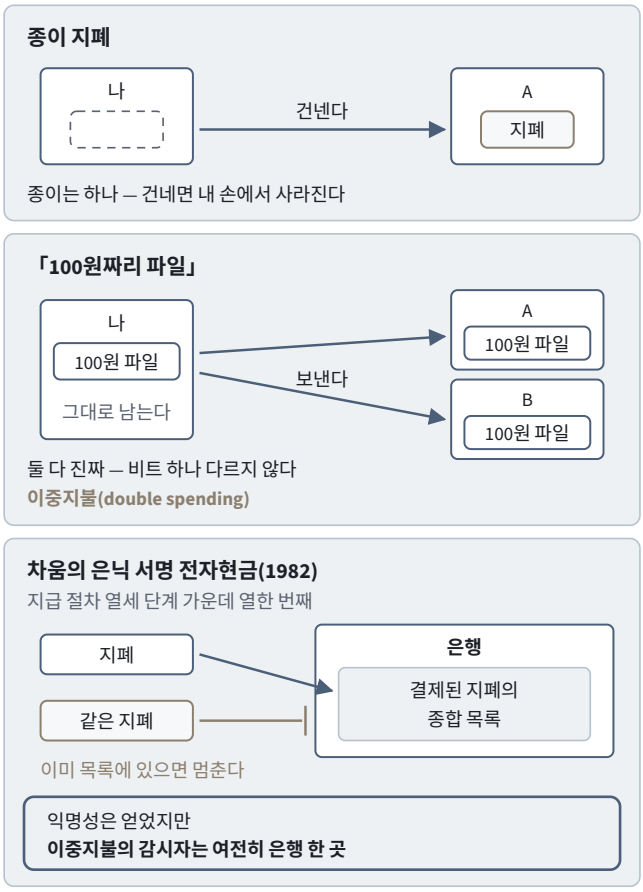


그림 1-1. 종이 지폐는 건네면 손에서 사라지지만 「100원짜리 파일」은 A와 B에게 보내도 둘 다 진짜다. 차움의 체계는 결제된 지폐의 종합 목록을 은행이 들고 같은 지폐가 두 번 오면 멈춘다(열세 단계 가운데 열한 번째).

2008년 백서는 문제를 이렇게 적습니다. 「전자 코인을 디지털 서명의 사슬로 정의한다. … 문제는 물론, 수취인이 이전 소유자들 가운데 누군가가 그 코인을 이중지불하지 않았음을 검증할 수 없다는 것이다. 흔한 해법은 모든 거래를 이중지불에 대해 검사하는 신뢰받는 중앙 기관, 즉 조폐소를 두는 것이다.」 그리고 그 해법의 문제. 「전체 화폐 체계의 운명이 조폐소를 운영하는 회사에 달려 있고, 모든 거래가 그들을 거쳐야 한다 — 꼭 은행처럼.」[2] 차움의 은행이 바로 그 조폐소입니다. 서명은 **누가 승인했는지를** 증명하지만(5장), **같은 돈을 두 번 승인하지 않았는지는** 증명하지 못합니다. 그것은 서명의 문제가 아니라 **순서의** 문제이기 때문입니다.

2. 세 가지 장부

「누가 장부를 쓰는가」와 「어느 거래가 먼저인지를 누가 정하는가」로 세 가지를 가릅니다.

	중앙 원장	분산 원장	블록체인
장부를 쓰는 사람	한 곳(은행·조폐소)	여럿이 각자 사본	여럿이 각자 사본
「먼저」를 정하는 사람	그 한 곳	**정해지지 않음**(사본이 갈리면 누구 말이 맞는가)	가장 많은 계산이 쌓인 사슬 — 참가자 다수가 계산으로 합의
믿어야 하는 것	그 한 곳의 정직과 능력	모든 사본 보관자의 정직	정직한 참가자가 계산력의 다수라는 것
예	차움의 은행(1983); 오늘의 은행 계좌	다이의 b-money 첫 프로토콜(1998)	비트코인(2009~)

가운데 열이 중요합니다. 장부를 여럿이 나눠 가지는 생각 자체는 블록체인 전에 있었습니다. 1998년 다이(Wei Dai)의 b-money 첫 프로토콜은 「모든 참가자가 각 가명에 얼마의 돈이 속하는지를 적은 (별개의) 데이터베이스를 유지한다. 이 계정들이 집합적으로 돈의 소유를 정의한다」고 적습니다.[3] 그런데 사본이 서로 다르면 어떻게 할까요. 앨리스가 「K_B에게 X를 준다」는 메시지를 모두에게 뿌리고 **동시에** 「K_C에게 X를 준다」는 메시지도 뿌리면, 어떤 참가자는 첫 메시지를 먼저 받고 어떤 참가자는 둘째를 먼저 받습니다. 두 장부가 갈리고, 어느 것이 맞는지를 정할 규칙이 첫 프로토콜에는 없습니다. 다이 자신이 그것을 「비실용적」이라 부르며 「동기적이고 방해할 수 없는 익명 방송 채널을 많이 쓴다」는 가정에 기댄다고 적었고, 둘째 프로토콜에서는 장부 관리를 「서버」라 부르는 일부 참가자에게 맡깁니다 — 다시 믿어야 하는 사람이 생깁니다.[4] 2014년 이더리움 백서는 이 역사를 한 문장으로 정리합니다. 「1998년 웨이 다이의 b-money는 계산 퍼즐을 풀어 돈을 만든다는 생각과 탈중앙 합의를 처음 도입한 제안이었으나, 탈중앙 합의를 실제로 어떻게 구현할지에 대한 세부가 부족했다.」[5]

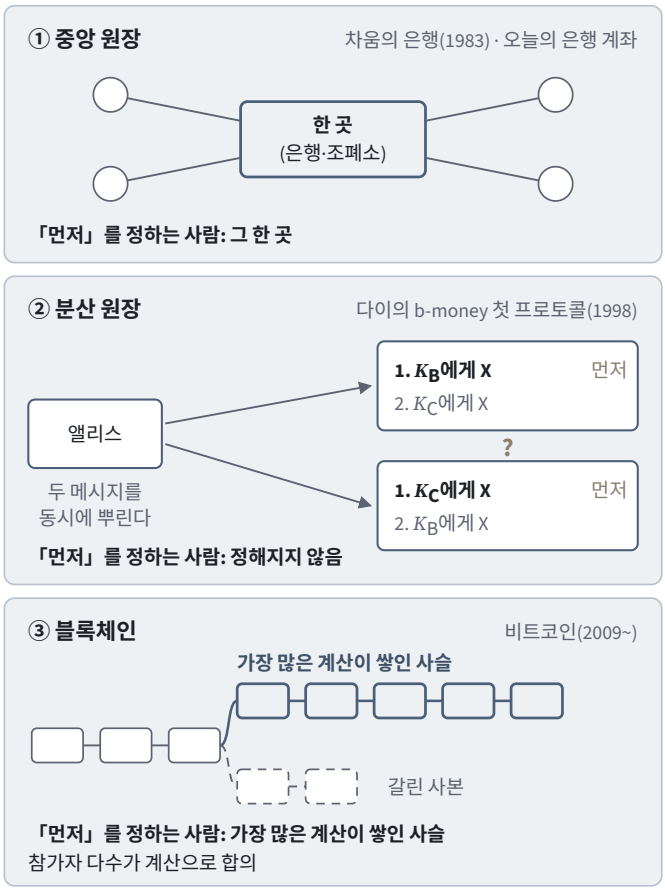


그림 1-2. 위 표의 「먼저」를 정하는 사람 행을 그림으로 옮겼다. 중앙 원장은 한 곳이 정하고, 분산 원장은 앨리스가 두 메시지를 동시에 뿌리면 참가자마다 받는 순서가 달라 정해지지 않으며, 블록체인은 가장 많은 계산이 쌓인 사슬이 정한다.

셋째 열이 그 세부입니다. 백서 §2의 마지막 문단이 문제를 정확히 적습니다. 「우리에게 필요한 것은 수취인이 이전 소유자들이 **더 이른** 거래에 서명하지 않았음을 알 방법이다. 우리 목적에는 가장 이른 거래가 유효한 것이고, 뒤의 이중지불 시도는 상관하지 않는다. 어떤 거래가 없음을 확인하는 유일한 방법은 모든 거래를 아는 것이다. … 신뢰받는 당사자 없이 이를 이루려면 거래가 **공개적으로 알려져야** 하고, 참가자들이 거래가 접수된 순서의 **단일한 역사에** 합의하는 체계가 필요하다. 수취인에게 필요한 증명은, 각 거래의 시점에 노드의 다수가 그것이 처음 접수된 것이라고 동의했다는 증명이다.」[6] 세 요소가 다 들어 있습니다. **공개**(모든 거래를 모두에게), **단일한 역사**(순서 하나), **다수의 동의**(누가 정하는가). 이 책의 1부는 그 세 요소를 만드는 부품 — 해시(4장), 서명(5장), 머클 트리(6장), 사슬(7장), 작업증명(8장) — 을 하나씩 뜯어보고, 9장에서 「다수의 동의」가 왜 확률로만 주어지는지를 봅니다.

3. 왜 어려운가 — 순서와 다수

「다수가 합의한다」는 말이 왜 쉽지 않은지는 1982년의 한 논문이 이미 보였습니다. 램포트(Leslie Lamport)·쇼스택·피즈의 「비잔틴 장군 문제」입니다. 서로 전령으로만 소통하는 장군들이 공동의 작전에 합의해야 하는데 그중 일부가 배신자여서 서로 다른 말을 전할 수 있다면, 충성스러운 장군들이 합의에 이르는 알고리듬이 있는가. 논문의 답은 조건부입니다. 「구두 메시지만 쓸 때 이 문제는 장군의 3분의 2 넘게 충성스러울 때, 그리고 그때에만 풀 수 있고, 따라서 배신자 하나가 충성스러운 둘을 혼란에 빠뜨릴 수 있다. 위조할 수 없는 서면 메시지가 있으면 장군과 배신자의 수에 관계없이 풀 수 있다.」[7] 서명(5장)이 「위조할 수 없는 서면 메시지」를 줍니다. 그러면 문제가 풀린 것 아닌가요. 이더리움 백서는 왜 아닌지를 적습니다. 그 이전의 합의 연구는 「체계의 모든 참가자가 알려져 있다고 가정했고, "N명이 참가하면 체계는 N/4명의 악의적 행위자까지 견딘다"는 형태의 안전 여유를 냈다. 문제는 익명 환경에서 그런 안전 여유는 시빌 공격에 취약하다는 것이다 — 한 공격자가 서버나 봇넷에 가짜 노드 수천 개를 만들어 일방적으로 다수를 차지한다.」[8] 참가자가 누구인지 모르는 열린 네트워크에서는 「한 사람 한 표」를 셀 방법이 없습니다. 사람을 세는 대신 무엇을 셀 것인가 — 백서의 답은 계산입니다. 「작업증명은 본질적으로 CPU 하나당 한 표다.」[9] 그것이 8장이고, 그래서 블록체인에는 전기가 듭니다.

이더리움 백서는 비트코인을 「선착순 등록(first-to-file) 체계」라고 부릅니다. 「어떤 주체가 50 BTC를 가지고 같은 50 BTC를 A와 B에게 동시에 보내면, 먼저 확정된 거래만 처리된다. 두 거래 중 어느 것이 먼저인지를 정하는 내재적 방법은 없고, 이것이 수십 년 동안 탈중앙 디지털 화폐의 개발을 가로막았다. 사토시의 블록체인은 최초의 신뢰할 만한 탈중앙 해법이었다.」[10] 「먼저」를 정하는 **내재적** 방법이 없다는 것 — 그래서 블록체인이 하는 일은 「먼저」를 발견하는 것이 아니라 **만드는** 것입니다. 거래들을 블록에 묶고, 블록을 사슬로 잇고, 그 사슬 가운데 가장 많은 계산이 쌓인 것을 「순서」로 **정의합니다**. 7장이 그 사슬을, 8장이 그 계산을 봅니다.

4. 대가 — 느림, 비용, 그리고 「완료」가 없다는 것

한 사람을 믿지 않는 값은 세 가지로 치릅니다. **느림.** 백서는 블록이 「10분마다 생성된다고 가정하면」이라고 적고, 난이도 조정이 「시간당 평균 블록 수를 목표로」 돌아간다고 적습니다.[11] 은행 서버가 건마다 즉시 처리하는 것과 비교하면, 10분에 한 묶음은 설계된 느림입니다. 모두가 같은 역사에 동의할 시간을 벌기 위해서입니다. **비용.** 「일정량의 새 코인이 꾸준히 추가되는 것은 금 채굴자가 자원을 들여 금을 유통에 더하는 것과 유사하다. 우리 경우에 쓰이는 것은 CPU 시간과 전기다.」[12] 그 전기가 7장의 「변조가 비싸지는 구조」의 재료이고, 20장은 그 비용이 어디서 조달되는지를 봅니다. **「완료」가 없다는 것.** 백서는 공격자가 사슬을 따라잡을 확률이 「블록이 뒤에 쌓일수록 지수적으로 줄어든다」고 적습니다 — 줄어들 뿐 0이 되지는 않습니다.[13] 은행 송금에는 규칙이 정한 「취소 불가 시점」이 있지만, 블록체인에는 그런 순간이 없고 되돌릴 확률이 작아지는 과정만 있습니다. 9장이 그 확률을 계산하고, 28.1(c)가 그것이 법의 「이행」·「완료」 개념과 어떻게 어긋나는지를 적어 둡니다.

> **법률가의 질문**
>
> 블록체인의 장부는 **사본이 수천 개이고**, 그 어느 것도 「원본」이라는 표시를 달고 있지 않습니다. 모두가 같은 바이트를 들고 있을 때는 상관없지만, 어느 노드의 사본과 다른 노드의 사본이 한 블록 다르다면 — 7장이 보이듯 그런 일은 설계상 일어나고 다음 블록들이 하나로 모읍니다 — 그 순간 어느 것이 「그」 장부입니까. 소송에서 당사자가 「체인에 이렇게 기록되어 있다」며 출력물을 제출하면, 그것은 어느 사본의 어느 시점입니까. 증거법의 원본·사본·진정성립 개념은 「원본」이 하나 있고 사본이 그것을 베낀다는 그림을 전제하는 것으로 보이는데, 원본이 **합의로 매 순간 다시 정해지는** 장부에서 그 개념은 무엇을 가리키게 됩니까. 체인이 재편되어 한 블록이 역사에서 빠지면, 그 블록을 담은 출력물은 위조입니까, 한때 참이었던 기록입니까. 이 책은 답하지 않고 5부로 넘깁니다. → 28장

5. 이 책의 지도

여섯 부가 이 문제의 조각을 나눠 맡습니다.

부	묻는 것	장
0부 지도	무엇이 문제이고(1장), 그 해법은 어디서 왔으며(2장), 이 책을 어떻게 읽는가(3장)	1~3
1부 기술 기초	「공개·단일한 역사·다수의 동의」를 만드는 부품 — 해시, 서명, 머클 트리, 사슬, 작업증명, 확률적 최종성, 지분증명, 두 상태 모형, 그리고 진짜 블록 하나	4~12
2부 스마트계약과 응용	그 원장 위에서 **코드가 실행될** 때 생기는 것 — 가상머신과 가스, 토큰, 자동화된 시장, 바깥 사실을 들여오는 문제, 순서를 정하는 권력, 확장	13~18
3부 경제학	그 위에서 돈과 사람이 어떻게 움직이는가 — 화폐론, 보안의 재원, 수수료, 토큰 설계, 게임이론, 스테이블코인, 버블의 계량	19~25
4부 사건으로 읽기	무엇이 무너졌는가 — 네 사건을 같은 진단표에	26
5부 법제도	기술·경제 사실이 법적 판단의 어느 지점에 들어가는가 — 깊이 다루지 않은 이유, 학설 구도, 법역별 서베이, 쟁점별 개관	27~28
6부 열린 문제	서명이 깨지는 날과 남는 코인(29장), 같은 물건의 다섯 이름(30장)	29~30

1·2부의 각 장은 「이 기술은 무엇의 실패에서 나왔나」로 시작합니다. 이 장이 보인 실패 — 믿을 한 사람 없이는 순서를 정할 수 없었다는 것 — 가 그 출발점이고, 2장은 그 실패를 하나씩 메워 온 30년을 봅니다.

6. 정리

디지털 현금의 난제는 복사가 공짜라는 데서 옵니다. 서명은 「누가 승인했는가」를 증명하지만 「같은 돈을 두 번 승인하지 않았는가」는 증명하지 못하고, 뒤의 것은 **순서의** 문제입니다. 지금까지 순서는 장부를 가진 한 사람이 정했습니다. 장부를 여럿이 나눠 가지는 것만으로는 사본이 갈릴 때 누가 맞는지를 정할 수 없고(분산 원장), 참가자가 누구인지 모르는 네트워크에서는 「한 사람 한 표」를 셀 수도 없습니다(시빌 공격). 블록체인은 거래를 모두에게 공개하고, 「먼저」를 **계산으로 정의합니다** — 가장 많은 작업이 쌓인 사슬이 역사입니다. 그 대가가 10분의 느림, 전기와 장비의 비용, 그리고 「완료」 대신 「되돌릴 확률이 작아짐」입니다. 법률가가 이 장에서 가져갈 질문은 하나입니다. 원본이 합의로 매 순간 다시 정해지는 장부에서 「원본」은 무엇인가. 이 책의 나머지는 그 질문에 들어가는 **전제 사실들을** 하나씩 확인하는 일입니다.

이 장의 출처

- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — 이중지불과 조폐소(§2, 2쪽), 작업증명과 「CPU 하나당 한 표」(§4, 3쪽), 유인과 비용(§6, 4쪽), 10분 가정(§7, 4쪽), 확률의 지수적 감소(§4·§11).
- David Chaum, "Blind Signatures for Untraceable Payments," *Advances in Cryptology: Proceedings of Crypto 82*, pp. 199–203 (Springer, 1983) — 지급 절차 13단계와 결제 지폐 목록(202쪽). (스캔본, 이미지로 읽음).
- Wei Dai, "b-money," http://www.weidai.com/bmoney.txt, 1998 — 두 프로토콜. 2026-10-25 접근·축어 대조.
- Leslie Lamport, Robert Shostak & Marshall Pease, "The Byzantine Generals Problem," *ACM TOPLAS* 4(3): 382–401 (1982) — 초록(382쪽).
- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," ethereum.org 공식 PDF (2014; 초고 2013) — 선착순 등록(1쪽), History(4쪽).
- 이 장은 코드·노트북이 없다(설계). 「화폐」라는 말은 기술 명칭으로만 썼고 가치 서술은 없다.

각주

1. David Chaum, "Blind Signatures for Untraceable Payments," in D. Chaum, R. L. Rivest & A. T. Sherman (eds.), *Advances in Cryptology: Proceedings of Crypto 82*, pp. 199–203 (Springer, 1983), 202쪽 「Untraceable Payments System」. 원문: "(11) Bank adds note to comprehensive list of cleared notes and stops if note already on list." 같은 쪽: "The critical concept is that the bank will sign anything with its private key, but anything so signed is worth a fixed amount, say $1." (IACR 아카이브 스캔본, 텍스트 층 없음 — 이미지로 직접 읽음).

2. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), 2쪽 §2 Transactions. 원문: "We define an electronic coin as a chain of digital signatures." / "The problem of course is the payee can't verify that one of the owners did not double-spend the coin. A common solution is to introduce a trusted central authority, or mint, that checks every transaction for double spending. After each transaction, the coin must be returned to the mint to issue a new coin, and only coins issued directly from the mint are trusted not to be double-spent. The problem with this solution is that the fate of the entire money system depends on the company running the mint, with every transaction having to go through them, just like a bank."

3. Wei Dai, "b-money," http://www.weidai.com/bmoney.txt (1998; 2026-10-25 접근·축어 대조). 원문: "In the first protocol, every participant maintains a (seperate) database of how much money belongs to each pseudonym.

These accounts collectively define the ownership of money, and how these accounts are updated is the subject of this protocol." 문서 안에 날짜가 없고, 1998은 나카모토 백서 참고문헌 [1]의 기재다(2장).

4. 같은 글. 원문: "The first one is impractical, because it makes heavy use of a synchronous and unjammable anonymous broadcast channel." / "In the second protocol, the accounts of who has how much money are kept by a subset of the participants (called servers from now on) instead of everyone." / "Since the servers must be trusted to a degree, some mechanism is needed to keep them honest."

5. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform" (ethereum.org 공식 게시본, 2014; 초고 2013), 4쪽 「History」. 원문: "In 1998, Wei Dai's b-money became the first proposal to introduce the idea of creating money through solving computational puzzles as well as decentralized consensus, but the proposal was scant on details as to how decentralized consensus could actually be implemented."

6. Nakamoto (2008), 2쪽 §2. 원문: "We need a way for the payee to know that the previous owners did not sign any earlier transactions. For our purposes, the earliest transaction is the one that counts, so we don't care about later attempts to double-spend. The only way to confirm the absence of a transaction is to be aware of all transactions. In the mint based model, the mint was aware of all transactions and decided which arrived first. To accomplish this without a trusted party, transactions must be publicly announced [1], and we need a system for participants to agree on a single history of the order in which they were received. The payee needs proof that at the time of each transaction, the majority of nodes agreed it was the first received."

7. Leslie Lamport, Robert Shostak & Marshall Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems* 4(3): 382–401 (July 1982), 382쪽 초록. 원문: "It is shown that, using only oral messages, this problem is solvable if and only if more than two-thirds of the generals are loyal; so a single traitor can confound two loyal generals. With unforgeable written messages, the problem is solvable for any number of generals and possible traitors."

8. Buterin (2014), 4쪽 「History」. 원문: "The protocols assumed that all participants in the system were known, and produced security margins of the form "if N parties participate, then the system can tolerate up to N/4 malicious actors". The problem is, however, that in an anonymous setting such security margins are vulnerable to sybil attacks, where a single attacker creates thousands of simulated nodes on a server or botnet and uses these nodes to unilaterally secure a majority share."

9. Nakamoto (2008), 3쪽 §4 Proof-of-Work. 원문: "The proof-of-work also solves the problem of determining representation in majority decision making. If the majority were based on one-IP-address-one-vote, it could be subverted by anyone able to allocate many IPs. Proof-of-work is essentially one-CPU-one-vote. The majority decision is represented by the longest chain, which has the greatest proof-of-work effort invested in it."

10. Buterin (2014), 1쪽. 원문: "Bitcoin as an application can be described as a first-to-file system: if one entity has 50 BTC, and simultaneously sends the same 50 BTC to A and to B, only the transaction that gets confirmed first will process. There is no intrinsic way of determining from two transactions which came earlier, and for decades this stymied the development of decentralized digital currency. Satoshi's blockchain was the first credible decentralized solution."

11. Nakamoto (2008), 4쪽 §7 "If we suppose blocks are generated every 10 minutes ..."; 3쪽 §4 "To compensate for increasing hardware speed and varying interest in running nodes over time, the proof-of-work difficulty is determined by a moving average targeting an average number of blocks per hour. If they're generated too fast, the difficulty increases."

12. Nakamoto (2008), 4쪽 §6 Incentive. 원문: "The steady addition of a constant of amount of new coins is analogous to gold miners expending resources to add gold to circulation. In our case, it is CPU time and electricity that is expended."

13. Nakamoto (2008), 3쪽 §4 "We will show later that the probability of a slower attacker catching up diminishes exponentially as subsequent blocks are added."; 7쪽 §11 "Given our assumption that p > q, the probability drops exponentially as the number of blocks the attacker has to catch up with increases."

0부 지도

2장 30년 전사(前史) — 전자현금에서 백서까지

핵심 TAKEAWAY

비트코인은 발명이라기보다 **조립입니다**. 2008년 백서의 참고문헌 여덟 개가 그 부품 목록에 가깝습니다 — 1980년의 해시 트리, 1991·1993·1997년의 타임스탬프 논문 셋, 1999년의 타임스탬프 서비스 설계, 해시캐시(공지 1997, 논문 2002), 1998년의 b-money, 그리고 교과서의 확률론. 백서가 인용하지 않은 부품 하나는 1976년의 공개키 암호인데, 백서의 「디지털 서명의 사슬」이 그 위에 서 있습니다. 부품마다 **무엇을 풀었고 무엇을 못 풀었는지가** 분명하고, 못 푼 자리가 다음 부품을 불렀습니다. 차움의 전자현금은 익명성을 얻었으나 은행을 남겼고, 하버와 스토네타의 사슬은 순서를 고정했으나 「누가 사슬을 쓰는가」를 남겼으며, 해시캐시는 계산으로 비용을 매겼으나 돈이 아니었고, b-money와 bit gold는 돈을 그렸으나 합의의 세부가 없었습니다. 백서는 그 자리들을 한 체계로 메웠고, 그 뒤의 역사 — 이더리움, The DAO, 자동화된 시장, 두 거래소의 붕괴, 각국의 정의 조문 — 는 이 책의 뒷장들이 맡습니다. 법률가가 가져갈 것은 연도가 아니라 **계보입니다**. 어느 부품이 어디서 왔는지를 알면, 「블록체인」이라는 한 낱말이 서로 다른 열 가지 주장을 한꺼번에 가리킬 때 어느 것을 말하는지 가릴 수 있습니다.

학습목표

1. 전자현금·타임스탬프·해시캐시·b-money·bit gold가 각각 무엇을 해결했고 무엇을 못 했는지 말한다.
2. 2008년 백서가 그 부품들을 어떻게 조립했는지, 백서의 참고문헌을 따라 짚는다.
3. 백서 이후의 이정표(이더리움, The DAO, 자동화된 시장, 포크들, 두 거래소, 정의 조문)가 이 책의 어느 장에서 다뤄지는지 안다.
4. 연도 표기의 규칙 — 학회 연도와 출판 연도, 게시일과 표시일 — 을 알고 연표를 읽는다.

[입문]만 읽은 독자가 가져가는 것. 비트코인은 발명이라기보다 **조립이고**, 조립된 부품 각각은 이미 학술 문헌에 있었습니다.

1. 연표 — 읽기 전에 규칙 둘

아래 표는 이 책이 **원문을 열어 확인한** 이정표만 담습니다. 연도 표기에는 규칙이 둘 있습니다. 첫째, 학회 논문은 학회 연도와 출판 연도가 다를 수 있습니다 — 차움의 논문은 「Crypto 82」에서 발표되어 1983년에 책으로 나왔고, 머클의 트리 서명은 「CRYPTO '87」에서 발표되어 1988년에 나왔습니다. 이 책은 학회명과 함께 쓰면 학회 연도를, 서지로 쓰면 출판 연도를 씁니다. 둘째, 웹 문서는 게시일과 화면 표시일이 다를 수 있습니다 — 사보(Nick Szabo)의 bit gold 글은 2005년 12월 29일에 게시되었고 지금 화면에는 2008년 12월 27일이 찍혀 있습니다(각주에 확인 방법). 확정하지 못한 날짜는 단정하지 않고 그 사실을 적었습니다. 이 표의 정지 판이 부록 D이고, 사건을 누르면 해당 장으로 가는 클릭 연표가 위젯 W11입니다. 2024년 이후의 항목 — 현물 ETF, 스테이블코인 입법, 양자 전환 제안 — 은 낡기 쉬우므로 본문이 아니라 부록 A에 둡니다(3장 일러두기의 갱신 규칙).

연도	이정표	무엇을 풀었나	무엇을 남겼나	장
1976	디피·헬먼, 공개키 암호	열쇠를 나누지 않고 서명과 비밀 통신	「서명자」가 누구인지, 서명한 돈이 두 번 쓰이지 않았는지는 서명 밖의 문제	5
1980 / 1987	머클, 해시 트리	많은 항목을 뿌리 하나로 보증	— (부품)	6
1982	램포트·쇼스택·피즈, 비잔틴 장군	배신자가 섞인 합의의 조건	참가자가 누구인지 아는 닫힌 집단을 전제	1·8
1982 / 1983	차움, 은닉 서명 전자현금	은행이 누구에게 발행했는지 모르는 익명 전자지폐	이중지불 검사는 은행이 한다	1·2
1991 / 1993	하버·스토네타(·베이어), 타임스탬프 사슬과 트리	문서의 선후를 해시 사슬로 고정, 트리로 압축	사슬을 쓰는 서비스를 누가 감시하는가	4·6·7
1997	백, 해시캐시	계산으로 비용을 매기고 한 번의 해시로 검증	돈이 아니라 우표 — 가치의 이전·보존 없음	8
1998	다이, b-money	계산으로 돈을 만들고 모두가 장부를 갖는 그림	장부가 갈릴 때 누구 말이 맞는지	1·8
2005	사보, bit gold	작업증명을 타임스탬프와 소유권 등기로 잇는 그림	등기와 타임스탬프를 「분산」할 방법	8
2008-10-31	나카모토, 백서 공개	위 부품을 한 체계로 — 공개·단일한 역사·계산에 의한 다수	확률적 최종성, 비용, 거버넌스	1·7·8·9
2009-01-03	제네시스 블록	체계가 실제로 돌기 시작	—	12
2014 (초고 2013)	부테린, 이더리움 백서	원장 위에서 임의의 프로그램 실행	멈추지 않는 프로그램의 비용(가스), 코드의 결함	13
2014-02-28 · 2014-04-24	Mt.Gox 민사재생 신청 · 파산절차 개시	—	거래소 장부와 체인의 불일치	26
2016	The DAO 매각·공격·하드포크	—	코드와 거버넌스 결정의 관계	14·22·26

연도	이정표	무엇을 풀었나	무엇을 남겼나	장
2017-07-25	SEC, The DAO 보고서	토큰 매각에 증권법이 적용된다는 판단	어느 토큰이 증권인가	26·27
2019-03-15 · 2020-06-11	東京地裁·東京高裁, Mt.Gox 대표 형사 판결	—	거래소 장부 조작의 형사 평가	26
2020-03	유니스왑 v2 백서	호가창 없는 자동화된 시장	운영자 없는 시장의 수범자	15
2021	런던 포크(EIP-1559) · 탭루트	수수료 시장의 재설계 · 서명 체계 추가	공개키 노출의 재등장(29장)	21·5·29
2022-05 · 2022-09 · 2022-11	Terra 붕괴 · 이더리움 지분증명 전환 · FTX 파산 신청	—	설계의 반사성 · 담보 기반 합의 · 고객 자산 유용	24·26·10
2023-02-03	영국 항소법원 *Tulip Trading*	—	개발자의 신인의무	27·28
2023-06-09	MiCA 관보 게재	EU 단일 정의와 규제	정의가 기술을 따라가는가	27·30



그림 2-1. 클릭 연표 1976~2023. 2장 1절의 표·부록 D 와 같은 29개 이정표. 항목을 누르면 그 사건을 원문으로 다루는 장으로 간다. 날짜는 확인된 정밀도(연·월·일)대로 적고 추정일은 따로 표시한다. 가격 축은 없다. — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 2장 1절).

2. 1976 — 열쇠를 둘로 나누다

디피(Whitfield Diffie)와 헬먼(Martin Hellman)의 1976년 논문은 서론의 첫 문장부터 혁명을 선언합니다. 「우리는 오늘 암호학 혁명의 문턱에 서 있다.」 값싼 디지털 하드웨어가 암호 장치를 「원격 현금 지급기와 컴퓨터 단말기 같은 상업적 응용」에 쓸 수 있게 했고, 그런 응용은 「안전한 열쇠 배포 채널의 필요를 최소화하고 **서면 서명에 상당하는 것을** 제공하는 새로운 종류의 암호 체계」를 요구한다는 것입니다.[1] 「서면 서명에 상당하는 것」 — 이것이 5장의 디지털 서명이고, 1장이 보았듯 백서는 코인을 「디지털 서명의 사슬」로 정의합니다. 그러나 서명은 「누가 승인했는가」까지만 증명합니다. 서명이 두 번 쓰이지 않았는지는 서명 밖의 일이고, 그 밖의 일을 전자현금의 문제로 다룬 것이 이 책이 읽는 자료 가운데 가장 이른 차움입니다.

3. 1982 / 1983 — 차움의 전자현금: 익명성은 얻고 은행은 남다

차움은 1982년 학회 논문의 첫머리에서 전자 결제가 「개인 프라이버시와 지불의 범죄적 이용의 성질·범위 양쪽에 상당한 영향을 미칠 수 있다」고 적고, 두 걱정을 함께 풀 체계를 제안합니다 — 제3자가 지불의 수취인·시각·금액을 알 수 없을 것, 그러면서도 개인이 지불 증명을 내거나 예외적 상황에서 수취인을 밝힐 수 있을 것, 그리고 도난 신고된 지불 수단의 사용을 막을 수 있을 것.[2] 도구가 **은닉 서명(blind signature)** 입니다. 카본지를 댄 봉투에 비유합니다. 「그런 봉투 겉면에 서명을 쓰면 봉투 안의 종이에 서명의 카본 사본이 남는다.」 은행은 봉투 겉에 서명하고, 지불자는 봉투를 열어 서명된 지폐를 꺼내므로, 은행은 자기가 서명한 지폐가 나중에 누구에게서 돌아오는지 연결하지 못합니다.[3] 1장에서 본 대로 그 체계는 이중지불을 막기 위해 은행이 **결제된 지폐의 종합 목록을** 들고 같은 지폐가 두 번 오면 멈춥니다. 익명성은 암호로 얻었고, 이중지불의 감시는 여전히 **한 기관의 장부에** 맡겼습니다. 2014년 이더리움 백서가 이 세대를 두고 「차움 은닉에 주로 기댄 1980~90년대의 익명 전자현금 프로토콜은 높은 프라이버시를 가진 통화를 제공했으나, 중앙화된 중개자에 의존했기 때문에 대체로 자리 잡지 못했다」고 적은 것은 그 자리를 가리킵니다.[4]

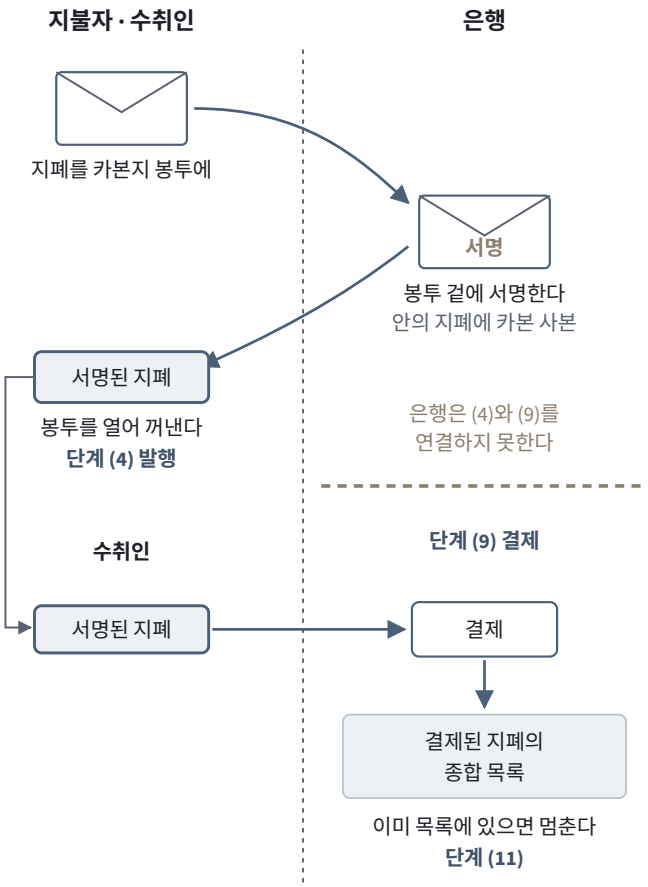


그림 2-2. 카본지 봉투로 비유한 은닉 서명의 흐름. 은행은 봉투 겉에만 서명하므로 단계 (4)에서 발행한 지폐와 단계 (9)에서 결제를 청한 지폐를 연결하지 못하고, 이중지불은 1장에서 본 단계 (11)의 종합 목록 대조로만 막는다. 단계 번호는 차움 논문의 것이고 나머지 단계는 번호 없이 그렸다.

4. 1980 / 1987 — 머클의 트리, 1982 — 비잔틴 장군

두 부품은 돈과 무관한 곳에서 왔습니다. 머클(Ralph Merkle)의 해시 트리는 많은 항목을 해시 하나로 보증하는 장치입니다. 백서는 머클 트리를 쓰며 셋을 인용하는데([7][2][5]), 그 가운데 머클 자신의 것이 1980년 IEEE 보안·프라이버시 심포지엄 논문입니다.[5] 그 논문은 공개키 목록의 한 항목을 목록 전체를 받지 않고 확인하는 방법으로 「트리 인증(tree authentication)」을 적습니다 — 목록을 반으로 나눠 해시하기를 되풀이하면(「분할 정복」) 뿌리 하나와 $\log_2 n$ 개의 중간값만으로 한 항목을 확인할 수 있다는 것입니다. 다만 자세한 논의는 자기의 다른 원고 「A certified digital signature」로 넘깁니다.[6] 베이어·하버·스토네타의 1993년 논문도 트리를 도입하며 「그런 해시 트리는 머클[Merk 80]이 다른 목적으로 — 공개 암호화 키 디렉터리의 인증 증명서를 만들기 위해 — 이전에 썼다」고 적습니다.[7] 머클의 1987년 학회 논문(1988년 출판)은 같은 착상을 서명에 써서 「서명 크기가 서명한 메시지 수의 로그로 증가하는」 체계를 만듭니다 — 6장이 보이는 「증명 길이 $\log_2 n$」의 뿌리입니다.[8] 비잔틴 장군 문제(1982)는 1장에서 보았습니다. 「배신자가 섞인 집단이 합의에 이르는 조건」을 장군의 비유로 정식화했고(엄밀한 증명은 같은 저자들의 1980년 논문으로 넘깁니다), 서명이 있으면 배신자 수와 무관하게 풀린다고 했습니다. 다만 그 결과는 장군이 **누구인지 아는** 집단의 것이었고, 누구나 가짜 장군을 만들어 낼 수 있는 열린 네트워크로 가는 길은 2008년에야 놓입니다.

5. 1991 / 1993 — 하버·스토네타의 사슬: 순서는 고정했으나

7장이 자세히 읽은 논문입니다. 하버(Stuart Haber)와 스토네타(W. Scott Stornetta)는 1991년 논문의 초록에서 문제를 이렇게 세웠습니다. 「문제는 매체가 아니라 **데이터에** 시각을 찍는 것이다. 우리는 … 타임스탬프 서비스가 결탁하더라도 사용자가 자기 문서의 날짜를 앞당기거나 늦추는 것이 불가능하도록 하는, 계산적으로 실용적인 절차를 제안한다.」[9] 서론의 첫 예가 **지식재산권입니다**. 「특허 가능한 아이디어를 발명자가 처음 글로 적은 날짜를 확인해 경쟁하는 주장에 대한 선행성을 세우는 것이 때로 결정적이다.」[10] 해법은 각 인증서가 직전 인증서의 해시를 담는 **연결(linking)** 이었고, 1993년 논문은 그 사슬을 트리로 묶어 「각 타임스탬프 사건에 대해 얻는 공개성을 지수적으로 늘리면서 저장과 검증 계산을 줄이는」 방법을 더했습니다.[11] 그리고 그 논문의 서론 첫 문장 — 「인과가 사건을 시간에 고정한다」 — 가 7장의 열쇠였습니다. 이 사슬이 남긴 것은 「누가 사슬을 쓰는가」입니다. 1991년 논문의 둘째 해법(무작위로 뽑힌 증인 여럿이 서명)은 「중앙 서비스 없이 구현될 수 있다」고 적었지만,[12] 그 증인들이 누구인지는 여전히 알려진 「고객 풀」 안에서 뽑혔습니다. 열린 네트워크에서 「쓰는 사람」을 어떻게 정하는가 — 그 빈칸을 계산이 메웁니다.

6. 1997 — 해시캐시: 계산으로 비용을 매기다

1997년 3월 28일, 애덤 백(Adam Back)은 사이퍼펑크 메일링리스트에 「부분 해시 충돌 기반 우표 체계」를 공지합니다. 「부분 해시를 쓰는 착상은, (원하는 충돌 비트 수를 골라) 계산을 임의로 비싸게 만들 수 있으면서도 **즉시 검증할** 수 있다는 것이다.」[13] 스팸 메일과 익명 리메일러 남용을 막으려는 **우표였습니다**. 보내는 쪽은 해시 앞자리가 0으로 시작하는 입력을 찾느라 CPU를 태우고, 받는 쪽은 해시 한 번으로 확인합니다. 2002년의 정리 논문은 이 비용 함수를 「비대화형이고, 공개적으로 감사 가능하며, 뒷문이 없고, 비용이 확률적으로 무한한」 것으로 분류하고,[14] 백서 §4는 「애덤 백의 해시캐시와 유사한 작업증명 체계」라고 그것을 그대로 가져옵니다. 8장이 그 복권을 다룹니다. 해시캐시가 남긴 것은 분명합니다. 그것은 돈이 아니라 우표였습니다. 한 번 쓰면 끝나고, 가치가 이전되거나 보존되지 않습니다. 2002년 논문 자신이 응용 목록의 끝에 「웨이 다이의 b-money 전자현금 제안의 발행 메커니즘」을 적어 둔 것은, 우표를 돈으로 바꾸려는 다음 부품이 이미 나와 있었다는 뜻입니다.[15]

7. 1998 — b-money: 돈을 그렸으나 합의는 비워 두다

다이의 b-money는 짧은 웹 문서이고 날짜가 적혀 있지 않습니다. 백서의 참고문헌 [1]이 「1998」로 적었고, 백의 2002년 논문은 「Nov 1998」로 적었습니다 — 이 책은 1998만 쓰고 월·일은 쓰지 않습니다. 첫 프로토콜은 1장에서 보았습니다. 돈의 **창조가** 흥미롭습니다. 「누구나 이전에 풀리지 않은 계산 문제의 해를 방송함으로써 돈을 만들 수 있다. 조건은 그 문제를 푸는 데 얼마나 많은 계산 노력이 들었는지를 쉽게 판정할 수 있어야 하고, 해가 그 밖에는 실용적으로도 지적으로도 아무 가치가 없어야 한다는 것뿐이다.」[16] 해시캐시의 우표를 돈의 발행으로 바꾼 셈입니다. 그리고 다이는 자기 제안의 가장 문제적인 부분을 스스로 짚습니다. 「이 프로토콜은 모든 장부 관리자가 특정 계산의 비용에 대해 결정하고 합의할 것을 요구한다. 불행히도 계산 기술은 빠르게, 그리고 항상 공개적으로는 아니게 발전하므로 이 정보는 얻을 수 없거나 부정확하거나 낡을 수 있다.」[17] 부록에서 경매로 그것을 보완하지만, 「장부가 갈릴 때 누구 말이 맞는가」와 「장부 관리자가 누구인가」는 여전히 비어 있습니다. 백서가 b-money에서 가져간 것은 「거래를 공개적으로 알려야 한다」는 한 가지이고, 바로 그 자리에 [1]을 달았습니다.

8. 2005 — bit gold: 등기부와 타임스탬프

사보의 bit gold는 2005년 12월 29일 블로그 글입니다. 문제 제기부터 다릅니다. 「우리의 돈은 현재 그 가치를 제3자에 대한 신뢰에 의존한다. 20세기의 많은 인플레이션과 초인플레이션 사건이 보여 주듯 이것은 이상적인 상태가 아니다.」 사보는 귀금속이 「창조의 비용 때문에 위조할 수 없는 희소성」을 가져 「제3자에 대부분 의존하지 않는 가치」를 주었다고 보지만, 「금속으로는 온라인으로 지불할 수 없다.」 그래서 「최소한의 제3자 신뢰로 온라인에서 위조할 수 없이 비싼 비트를 만들고, 비슷하게 최소한의 신뢰로 안전하게 보관·이전·감정할 수 있는 프로토콜」을 그립니다.[18] 일곱 단계 가운데 셋이 그 뒤의 체계를 예고합니다. 「(3) 작업증명이 안전하게 타임스탬프된다. 이것은 분산된 방식으로, 여러 다른 타임스탬프 서비스로 이루어져야 하며 특정 서비스에 실질적으로 의존하지 않아야 한다. (4) 앨리스는 … bit gold의 분산된 재산권 등기부에 추가한다. 여기서도 어느 한 서버에도 등기부 운영을 실질적으로 의존하지 않는다. (5) 마지막으로 만들어진 bit gold 문자열이 다음 문자열의 도전 비트를 제공한다.」[19] 작업증명(해시캐시)과 타임스탬프 사슬(하버·스토네타)과 **소유권 등기가** 한 그림에 들어왔고, 「마지막 것이 다음 것의 입력」이라는 사슬도 있습니다. 비어 있는 것은 (3)과 (4)의 「분산된 방식」이 실제로 어떻게 작동하느냐이고, 사보 자신이 「이 체계의 보안의 주된 한계는 단계 (3)과 (4)에서 신뢰가 얼마나 잘 분산될 수 있는가」라고 적었습니다.[20] 같은 글은 피니(Hal Finney)가 구현한 변형 RPOW(재사용 가능한 작업증명)를 소개하는데, 이더리움 백서는 그것을 「신뢰 컴퓨팅을 백엔드로 삼아 다시 이상에 못 미쳤다」고 평합니다.[21]

9. 2008 — 백서: 조립

2008년 10월 31일 금요일 14시 10분(미국 동부), 암호학 메일링리스트에 글 하나가 올라옵니다. 「저는 신뢰받는 제3자 없이 완전히 P2P로 돌아가는 새 전자현금 체계를 만들어 왔습니다.」 다섯 줄의 「주요 성질」이 따릅니다. 「이중지불은 P2P 네트워크로 방지된다. 조폐소나 다른 신뢰받는 당사자가 없다. 참가자는 익명일 수 있다. 새 코인은 해시캐시식 작업증명으로 만들어진다. 새 코인 생성을 위한 작업증명이 동시에 이중지불을 막는 네트워크에 동력을 공급한다.」[22] 다섯 줄 가운데 앞의 넷은 이 장이 지나온 부품들의 이름입니다 — 차움이 남긴 조폐소를 없애고, 다이의 「공개된 거래」를 받고, 백의 해시캐시로 코인을 만듭니다. 새것은 다섯째 줄입니다. 돈을 만드는 계산과 순서를 정하는 계산을 **같은 계산으로** 두어, 1장의 「누가 장부를 쓰는가」와 다이가 비워 둔 「장부 관리자가 누구인가」를 한꺼번에 답합니다. 쓰는 사람은 계산을 한 사람이고, 계산을 한 사람은 코인을 받습니다. 백서의 참고문헌 여덟 개가 조립도입니다 — [1] 다이의 b-money(공개 선언), [2]~[5] 타임스탬프 문헌(하버·스토네타 1991·1997, 베이어 외 1993, 마시아스 외 1999), [6] 백의 해시캐시, [7] 머클 1980, [8] 펠러의 확률론 교과서(9장의 도박꾼의 파산).[23] 체계가 실제로 돈 것은 2009년 1월 3일입니다. 12장이 그 첫 블록 80바이트를 손으로 풀었고, 그 안의 시각 1231006505는 2009년 1월 3일 18시 15분 5초(UTC)이며, 코인베이스에는 「The Times 03/Jan/2009 Chancellor on brink of second bailout for banks」라는 문장이 적혀 있습니다.[24]

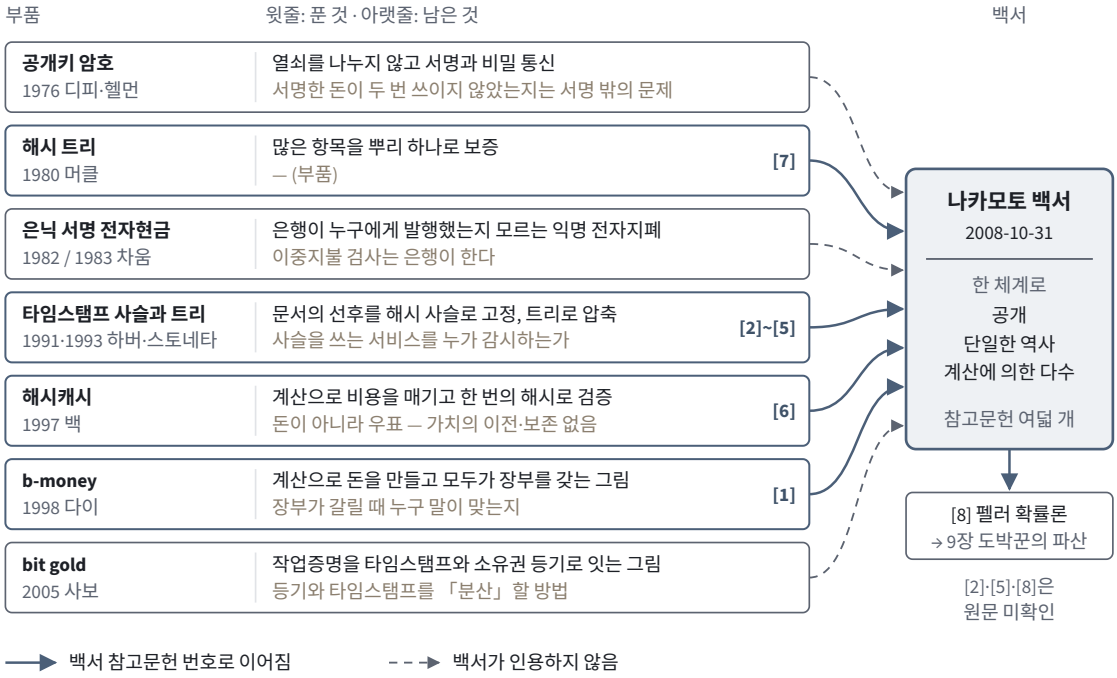


그림 2-3. 백서로 모인 부품들과 각 부품이 풀거나 남긴 것(1절 표). 실선 옆 번호는 백서의 참고문헌 번호이고, 점선은 백서가 인용하지 않은 것이다. [2]·[5]·[8]은 이 책이 원문을 열지 않았다.

10. 2013 / 2014 — 이더리움: 원장 위에서 프로그램을

백서가 나온 지 5년 뒤, 부테린(Vitalik Buterin)의 이더리움 백서는 비트코인을 두 개의 발명으로 나눠 읽습니다. 「첫째는 "비트코인", 어떤 뒷받침도 내재가치도 중앙 발행자도 없이 가치를 유지하는 탈중앙 P2P 온라인 통화다. … 그러나 사토시의 거대한 실험에는 똑같이 중요한 다른 부분이 있다. 거래 순서에 대한 공개적 합의를 가능하게 하는 작업증명 기반 블록체인이라는 개념이다.」 그리고 「블록체인 개념이 돈 이상의 것에 어떻게 쓰일 수 있는가」로 주의를 돌립니다.[25] 백서의 「History」 절은 이 장이 지나온 계보를 그대로 적습니다 — 차움, 다이, 피니, 비잔틴 합의와 시빌 공격, 그리고 「사토시가 제공한 혁신은 … 매우 단순한 탈중앙 합의 프로토콜과 작업증명을 결합한 착상」이라는 평가.[26] 이더리움이 더한 것은 「내장된 완전한 튜링 완전 프로그래밍 언어를 가진 블록체인」이고, 그 결과 — 멈추지 않는 프로그램에 값을 매기는 가스, 열 줄짜리 「계약」, 그 코드의 결함 — 는 13장과 26장이 봅니다. 연도는 「2014(초고 2013)」로 씁니다. 공식 PDF의 표지는 「(2014)」이고 본문에 「2014년 4월 현재」라는 구절이 있으며, 초고의 웹 아카이브 첫 캡처는 2013년 12월 19일입니다.[27]

11. 2014 ~ 2023 — 백서 뒤의 이정표, 이 책의 뒷장들

이 뒤의 역사는 이 책의 뒷장들이 각각 원문으로 다룹니다. 여기서는 어느 장이 맡는지만 적습니다.

- **2014년 2월, Mt.Gox.** 거래소가 민사재생을 신청했고, 체인 밖 장부와 체인의 불일치가 문제의 자리였습니다. 형사 1심(2019)·항소심(2020) 판결까지 26장이 읽습니다.
- **2016년, The DAO.** 토큰 매각(4월 30일~5월 28일), 코드 결함을 이용한 자산 이동(6월 17일), 하드포크(7월 20일). 미국 증권거래위원회의 2017년 7월 25일 보고서는 「DAO 토큰은 증권」이라 판단하고 집행 조치는 하지 않았으며, 「이 기술, "스마트계약" 또는 컴퓨터 코드를 통한 특정 기능의 자동화는 행위를 미국 연방 증권법의 적용 범위에서 벗어나게 하지 않는다」고 적었습니다.[28] 이것이 이 책이

「ICO 국면」을 적는 방식입니다 — 규모와 가격이 아니라 **제도의 판단으로**. 14장·22장·26장·27장.

- **2020년 3월, 유니스왑 v2 백서.** 백서는 v1을 「상수곱 공식에 기반한 자동화된 유동성 프로토콜」이라 부르고 v2를 「같은 공식에 기반한 새 구현」이라 적습니다 — 호가창 없는 시장. 15장이 그 식을, 17장이 그 위의 순서 문제를 봅니다.[29]
- **2021년.** 이더리움의 런던 포크(수수료 시장 재설계, 21장)와 비트코인의 탭루트 활성화(슈노르 서명, 5장·부록 A-8). 탭루트가 공개키 노출을 되살렸다는 점은 29장에서 다시 나옵니다.
- **2022년.** 5월 Terra의 붕괴(24장·26장), 9월 이더리움의 지분증명 전환(10장·부록 A-7), 11월 FTX의 파산 신청(26장).
- **2023년 2월 3일, 영국 항소법원 *Tulip Trading*.** 비트코인 개발자가 소유자에게 신인의무를 지는지가 「심리할 만한 쟁점」이라고 한 판결. 27장이 다룹니다.
- **2023년 6월 9일, MiCA 관보 게재.** EU의 단일 정의 「crypto-asset」과 규제. 적용은 2024년 12월 30일부터. 30장이 그 정의 조문을 다른 네 법역의 것과 나란히 놓습니다.

2024년 이후 — 현물 상장지수펀드, 미국·영국·일본의 입법, 비트코인의 양자 전환 제안, NIST의 양자내성 표준 — 는 부록 A 동향 노트에 기준일과 함께 두었습니다. 그것들은 아직 「역사」가 아니라 「현황」이기 때문입니다.

12. 정리

부품마다 풀린 것과 남은 것이 있었습니다. 1976년의 공개키는 서명을 주었으나 이중지불은 서명 밖의 일이었고, 1983년의 전자현금은 익명성을 주었으나 은행을 남겼으며, 1991·1993년의 사슬과 트리는 순서를 고정했으나 쓰는 사람을 남겼고, 1997년의 해시캐시는 계산으로 비용을 매겼으나 돈이 아니었으며, 1998년의 b-money와 2005년의 bit gold는 돈과 등기부를 그렸으나 「분산된 방식」의 세부를 남겼습니다. 2008년의 백서는 돈을 만드는 계산과 순서를 정하는 계산을 같은 계산으로 두어 그 자리들을 한꺼번에 메웠고, 그 조립도가 참고문헌 여덟 개입니다. 그 뒤의 역사는 이 책의 뒷장들이 원문으로 읽습니다. 법률가가 이 장에서 가져갈 것은 하나입니다 — 「블록체인」이라는 낱말 뒤에는 서로 다른 시대의 서로 다른 발명 열 개가 겹쳐 있고, 어느 사건·어느 조문이 그중 **어느 것을** 가리키는지를 가려 읽어야 한다는 것. 다음 3장은 이 책을 읽는 법입니다.

이 장의 출처

- Whitfield Diffie & Martin E. Hellman, "New Directions in Cryptography," *IEEE Trans. Inform. Theory* IT-22(6): 644–654 (1976) — 644쪽.
- David Chaum, "Blind Signatures for Untraceable Payments," *Advances in Cryptology: Proceedings of Crypto 82*, pp. 199–203 (Springer, 1983) — 199~203쪽(스캔본, 이미지로 읽음).
- Ralph C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," *CRYPTO '87*, LNCS 293, pp. 369–378 (Springer, 1988) — 369쪽.
- Ralph C. Merkle, "Protocols for Public Key Cryptosystems," *Proc. 1980 IEEE Symposium on Security and Privacy*, pp. 122–134 (DOI 10.1109/SP.1980.10006) — 126·127·133쪽. 저자 사이트 사본(스캔본, 텍스트 층은 OCR — 인용은 쪽 이미지로 대조했다).
- Leslie Lamport, Robert Shostak & Marshall Pease, "The Byzantine Generals Problem," *ACM*

TOPLAS 4(3): 382–401 (1982).

- Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *J. Cryptology* 3(2): 99–111 (1991) — 99·110쪽.
- Dave Bayer, Stuart Haber & W. Scott Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping," *Sequences II*, pp. 329–334 (Springer, 1993) — 저자 조판본 1~2쪽.
- Adam Back, "[ANNOUNCE] hash cash postage implementation," cypherpunks, 1997-03-28 (hashcash.org/papers/announce.txt); Adam Back, "Hashcash — A Denial of Service Counter-Measure," 2002-08-01 — 1·3·7·9·10쪽.
- Wei Dai, "b-money," http://www.weidai.com/bmoney.txt (1998). 2026-10-25 접근.
- Nick Szabo, "Bit gold," *Unenumerated*, 2005-12-29 게시(Wayback 2006-03-29 스냅숏으로 확정; 현 화면 표시일 2008-12-27). 2026-10-25 접근.
- Satoshi Nakamoto, "Bitcoin P2P e-cash paper," cryptography@metzdowd.com, 2008-10-31; Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008) — §2·§4·§7·References.
- 이 책의 동봉 데이터 `data/genesis_block.meta.json`(기준일 2026-10-25) — 제네시스 nTime·코인베이스 문장.
- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform" (ethereum.org 공식 PDF, 2014; 초고 2013) — 1·4쪽.
- SEC, Release No. 81207 (2017-07-25) — 1~2쪽.
- Hayden Adams, Noah Zinsmeister & Dan Robinson, "Uniswap v2 Core" (March 2020) — 1쪽.
- 그 밖의 이정표(Mt.Gox·The DAO 경과·런던 포크·탭루트·Terra·Merge·FTX·MiCA)의 1차 자료는 각 담당 장(5·10·21·24·26·30장)과 부록 A의 것이다. 이 장은 코드·노트북이 없다(설계). ICO·ETF·스테이블코인 입법은 「제도 변화」로만 적고 규모·가격을 쓰지 않았다. 2024년 이후 항목은 부록 A.

각주

1. Whitfield Diffie & Martin E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory* IT-22(6): 644–654 (November 1976), 644쪽. 원문: "WE STAND TODAY on the brink of a revolution in cryptography. The development of cheap digital hardware has freed it from the design limitations of mechanical computing and brought the cost of high grade cryptographic devices down to where they can be used in such commercial applications as remote cash dispensers and computer terminals. In turn, such applications create a need for new types of cryptographic systems which minimize the necessity of secure key distribution channels and supply the equivalent of a written signature."

2. David Chaum, "Blind Signatures for Untraceable Payments," in D. Chaum, R. L. Rivest & A. T. Sherman (eds.), *Advances in Cryptology: Proceedings of Crypto 82*, pp. 199–203 (Springer, 1983), 199~200쪽. 원문: "The ultimate structure of the new electronic payments system may have a substantial impact on personal privacy as well as on the nature and extent of criminal use of payments." / "(1) Inability of third parties to determine payee, time or amount of payments made by an individual. (2) Ability of individuals to provide proof of payment, or to determine the identity of the payee under exceptional circumstances. (3) Ability to stop use of payments media reported stolen." (IACR 스캔본, 텍스트 층 없음 — 이미지로 직접 읽음). 학회 1982, 출판 1983.

3. Chaum (1983), 200쪽 「Basic Idea」. 원문: "The paper analog of a blind signature can be implemented with carbon paper lined envelopes. Writing a signature on the outside of such an envelope leaves a carbon copy of the signature on a slip of paper within the envelope." 203쪽: "Notice that by the blind signature property above, when the bank receives a note to be cleared from the payee in step (9) the bank does not know which payer the note was originally issued to in step (4)."

4. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform" (ethereum.org 공식 게시본, 2014; 초고 2013), 4쪽 「History」. 원문: "The anonymous e-cash protocols of the 1980s and the 1990s, mostly reliant on a cryptographic primitive known as Chaumian blinding, provided a currency with a high degree of privacy, but the protocols largely failed to gain traction because of their reliance on a centralized intermediary."

5. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), 4쪽 §7: "transactions are hashed in a Merkle Tree [7][2][5], with only the root included in the block's hash." 참고문헌 [7]: "R.C. Merkle, "Protocols for public key cryptosystems," In Proc. 1980 Symposium on Security and Privacy, IEEE Computer Society, pages 122-133, April 1980." 원문의 쪽은 122–134쪽이다(134쪽은 참고문헌 끝과 그림 1).

6. Ralph C. Merkle, "Protocols for Public Key Cryptosystems," *Proc. 1980 IEEE Symposium on Security and Privacy*, pp. 122–134 (DOI 10.1109/SP.1980.10006), 126쪽. 원문: "Fortunately, it is possible to selectively authenticate individual entries in the public file without having to know the whole public file by using Merkle's "tree authentication," [13]. The essence of tree authentication is to authenticate the entire public file by "divide and conquer."" 같은 쪽: "By applying this concept recursively, A can confirm Y_B in the public file knowing only R, $\log_2 n$ intermediate values, and Y_B itself." 127쪽: "For a more detailed discussion the reader is referred to [13]." 133쪽 참고문헌 [13]: "Merkle, R.C. A certified digital signature, to appear, CACM." 같은 제목의 논문은 CRYPTO '89에서 발표되어 1990년에 출판되었다 — R. C. Merkle, "A certified digital signature," in G. Brassard (ed.), *Advances in Cryptology — CRYPTO '89*, LNCS 435, pp. 218–238 (Springer, 1990). 이 서지는 바부시 외(2026) 참고문헌 [244]와 하버·스토네타(1991) 참고문헌의 같은 권 표기(부록 E)로 확인했고 원문은 열지 않았다. 그러므로 이 책은 1980년 논문을 트리가 처음 제시된 곳으로 쓰지 않는다. 저자 사이트 사본(스캔본, 텍스트 층은 OCR — 인용은 쪽 이미지로 대조했다).

7. Dave Bayer, Stuart Haber & W. Scott Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping," in R. Capocelli, A. De Santis & U. Vaccaro (eds.), *Sequences II: Methods in Communication, Security, and Computer Science*, pp. 329–334 (Springer, 1993), 저자 조판본 2쪽(§1). 원문: "We do this by building trees in place of the linked list of the linking solution, thus achieving an exponential increase in the number of observers. Such hash trees were previously used by Merkle [Merk 80] for a different purpose, to produce authentication certificates for a directory of public enciphering keys." (저자 조판본, 「March 1992」 표기, 쪽 1~6). 출판 연도는 1993이고(Crossref·OpenAlex·Springer·백서 [4] 일치), 1992를 뒷받침하는 출판 레코드는 없다.

8. Ralph C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in C. Pomerance (ed.), *Advances in Cryptology — CRYPTO '87*, LNCS 293, pp. 369–378 (Springer, 1988), 369쪽 초록. 원문: "The signature system can sign an unlimited number of messages, and the signature size increases logarithmically as a function of the number of messages signed." 학회 1987, 출판 1988.

9. Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *Journal of Cryptology* 3(2): 99–111 (1991), 99쪽 초록. 원문: "The problem is to time-stamp the data, not the medium. We propose computationally practical procedures for digital time-stamping of such documents so that it is infeasible for a user either to back-date or to forward-date his document, even with the collusion of a time-stamping service." 접수 1990-08-19, 개정 1990-10-26(99쪽 각주 1).

10. Haber & Stornetta (1991), 99쪽 §1. 원문: "For example, in intellectual property matters, it is sometimes crucial to verify the date an inventor first put in writing a patentable idea, in order to establish its precedence over competing claims."

11. Bayer, Haber & Stornetta (1993), 저자 조판본 1쪽 초록. 원문: "By using trees, we show how to achieve an exponential increase in the publicity obtained for each time-stamping event, while reducing the storage and the computation required in order to validate a given certificate." 서론 첫 문장(2쪽): "Causality fixes events in time."

12. Haber & Stornetta (1991), 110쪽 §8 Summary. 원문: "In the second solution, several members of the client pool must time-stamp the hash. The members are chosen by means of a pseudorandom generator that uses the hash of the document itself as seed. This makes it infeasible to choose deliberately which clients should and should not time-stamp a given hash. The second method could be implemented without the need for a centralized TSS at all."

13. Adam Back, "[ANNOUNCE] hash cash postage implementation," cypherpunks@toad.com, Fri, 28 Mar 1997 16:52:26 GMT (저자 사이트 사본 http://www.hashcash.org/papers/announce.txt, 2026-10-25 접근). 원문: "A

partial hash collision based postage scheme / I've been talking about a partial hash collision based postage scheme on the crypto lists for the last few days. The idea of using partial hashes is that they can be made arbitrarily expensive to compute (by choosing the desired number of bits of collision), and yet can be verified instantly." **날짜 불일치:** 2002년 논문은 초록과 본문에서 최초 제안을 「May 1997」로 적고 참고문헌 [1]을 "Adam Back. Hashcash, May 1997. Published at http://www.cypherspace.org/hashcash/"로 적는다 — 공지 메일의 1997-03-28과 두 달 어긋난다. 이 책은 「1997년 3월 cypherpunks 공지」로 쓰고 불일치를 여기 적어 둔다. 공지 사본은 저자 사이트 것이며 독립 아카이브와 대조하지 못했다.

14. Adam Back, "Hashcash — A Denial of Service Counter-Measure," technical report, 1 August 2002, 3쪽 §3. 원문: "Hashcash is a non-interactive, publicly auditable, trapdoor-free cost function with unbounded probabilistic cost." 1쪽 초록: "Hashcash was originally proposed as a mechanism to throttle systematic abuse of un-metered internet resources such as email, and anonymous remailers in May 1997." 같은 쪽: "At the time of publication of [1] the author was not aware of the prior work by Dwork and Naor in [2] who proposed a CPU pricing function for the application of combatting junk email." — 드워크·나오르(1992)의 원문은 이 책이 열지 않았고 백의 서술로만 적는다.

15. Back (2002), 7쪽 §7 Applications. 원문: "hashcash as a minting mechanism for Wei Dai's b-money electronic cash proposal, an electronic cash scheme without a banking interface [19]". 참고문헌 [19]: "Wei Dai. b-money. Published at http://www.eskimo.com/~weidai/bmoney.txt, Nov 1998." 백서 §4(3쪽): "we will need to use a proof-of-work system similar to Adam Back's Hashcash [6]".

16. Wei Dai, "b-money," http://www.weidai.com/bmoney.txt (1998; 2026-10-25 접근·축어 대조). 원문: "1. The creation of money. Anyone can create money by broadcasting the solution to a previously unsolved computational problem. The only conditions are that it must be easy to determine how much computing effort it took to solve the problem and the solution must otherwise have no value, either practical or intellectual."

17. 같은 글, Appendix A. 원문: "One of the more problematic parts in the b-money protocol is money creation. This part of the protocol requires that all of the account keepers decide and agree on the cost of particular computations. Unfortunately because computing technology tends to advance rapidly and not always publicly, this information may be unavailable, inaccurate, or outdated, all of which would cause serious problems for the protocol." 백서 2쪽 §2: "To accomplish this without a trusted party, transactions must be publicly announced [1] ...".

18. Nick Szabo, "Bit gold," *Unenumerated* (블로그), https://unenumerated.blogspot.com/2005/12/bit-gold.html (2026-10-25 접근·축어 대조). 원문: "The problem, in a nutshell, is that our money currently depends on trust in a third party for its value. As many inflationary and hyperinflationary episodes during the 20th century demonstrated, this is not an ideal state of affairs." / "Precious metals and collectibles have an unforgeable scarcity due to the costliness of their creation." / "What's worse, you can't pay online with metal. Thus, it would be very nice if there were a protocol whereby unforgeably costly bits could be created online with minimal dependence on trusted third parties, and then securely stored, transferred, and assayed with similar minimal trust." **게시일:** URL 경로는 2005/12이고 현재 화면 표시일은 「Saturday, December 27, 2008」이지만, Wayback 원본 스냅숏(web.archive.org/web/20060329122942id_/.../2005/12/bit-gold.html)의 날짜 머리글이 「Thursday, December 29, 2005」이고 본문 전문이 실려 있다. 이 책은 보존본의 표시일 2005-12-29를 게시일로 쓴다. 현재 페이지의 표시일이 2008-12-27로 달라진 경위는 확인하지 못했다.

19. 같은 글. 원문: "(3) The proof of work is securely timestamped. This should work in a distributed fashion, with several different timestamp services so that no particular timestamp service need be substantially relied on. (4) Alice adds the challenge string and the timestamped proof of work string to a distributed property title registry for bit gold. Here, too, no single server is substantially relied on to properly operate the registry. (5) The last-created string of bit gold provides the challenge bits for the next-created string." 그리고: "Note that Alice's control over her bit gold does not depend on her sole possession of the bits, but rather on her lead position in the unforgeable chain of title (chain of digital signatures) in the title registry."

20. 같은 글. 원문: "The main limits to the security of the scheme are how well trust can be distributed in steps (3) and (4), and the problem of machine architecture which will be discussed below."

21. Buterin (2014), 4쪽 「History」. 원문: "In 2005, Hal Finney introduced a concept of "reusable proofs of work", a system which uses ideas from b-money together with Adam Back's computationally difficult Hashcash puzzles to create a concept for a cryptocurrency, but once again fell short of the ideal by relying on trusted

computing as a backend." 사보의 글: "Hal Finney has implemented a variant of bit gold called RPOW (Reusable Proofs of Work). This relies on publishing the computer code for the "mint," which runs on a remote tamper-evident computer."

22. Satoshi Nakamoto, "Bitcoin P2P e-cash paper," cryptography@metzdowd.com, Fri Oct 31 14:10:00 EDT 2008, https://www.metzdowd.com/pipermail/cryptography/2008-October/014810.html (2026-10-25 접근·축어 대조). 원문: "I've been working on a new electronic cash system that's fully peer-to-peer, with no trusted third party." / "The main properties: Double-spending is prevented with a peer-to-peer network. No mint or other trusted parties. Participants can be anonymous. New coins are made from Hashcash style proof-of-work. The proof-of-work for new coin generation also powers the network to prevent double-spending." (Nakamoto Institute 사본과 본문 일치).

23. Nakamoto (2008), 9쪽 References(요약 전사 — URL·부제·학회 전체 이름은 줄였다). [1] W. Dai, "b-money," 1998; [2] H. Massias, X.S. Avila & J.-J. Quisquater, "Design of a secure timestamping service with minimal trust requirements," 20th Symposium on Information Theory in the Benelux, May 1999; [3] S. Haber & W.S. Stornetta, "How to time-stamp a digital document," *Journal of Cryptology* 3(2): 99-111, 1991; [4] D. Bayer, S. Haber & W.S. Stornetta, "Improving the efficiency and reliability of digital time-stamping," *Sequences II*, pages 329-334, 1993; [5] S. Haber & W.S. Stornetta, "Secure names for bit-strings," 4th ACM CCS, pages 28-35, April 1997; [6] A. Back, "Hashcash - a denial of service counter-measure," 2002; [7] R.C. Merkle, "Protocols for public key cryptosystems," 1980 Symposium on Security and Privacy, pages 122-133, April 1980; [8] W. Feller, "An introduction to probability theory and its applications," 1957. 이 가운데 [2]·[5]·[8]의 원문은 이 책이 열지 않았다.

24. 이 책의 동봉 데이터 `data/genesis_block.meta.json`·`data/README.md` (기준일 2026-10-25) — nTime 1231006505, 코인베이스 입력 스크립트의 69바이트 ASCII 문장 'The Times 03/Jan/2009 Chancellor on brink of second bailout for banks', Bitcoin Core v31.1 `src/kernel/chainparams.cpp` (커밋 9be056a8) L70·L134~L137과 일치. 12장 NB12-1~3. 시각 변환은 12장 NB12-1 출력.

25. Buterin (2014), 1쪽. 원문: "When Satoshi Nakamoto first set the Bitcoin blockchain into motion in January 2009, he was simultaneously introducing two radical and untested concepts. The first is the "bitcoin", a decentralized peer-to-peer online currency that maintains a value without any backing, intrinsic value or central issuer. ... However, there is also another, equally important, part to Satoshi's grand experiment: the concept of a proof of work-based blockchain to allow for public agreement on the order of transactions." / "And now, attention is rapidly starting to shift toward this second part of Bitcoin's technology, and how the blockchain concept can be used for more than just money." 이 책은 「가치를 유지한다」는 서술을 백서 저자의 표현으로만 옮기고 가격을 다루지 않는다.

26. Buterin (2014), 4쪽. 원문: "The innovation provided by Satoshi is the idea of combining a very simple decentralized consensus protocol, based on nodes combining transactions into a "block" every ten minutes creating an ever-growing blockchain, with proof of work as a mechanism through which nodes gain the right to participate in the system." 1쪽: "What Ethereum intends to provide is a blockchain with a built-in fully fledged Turing-complete programming language that can be used to create "contracts" that can be used to encode arbitrary state transition functions".

27. ethereum.org 공식 PDF 표지 「By Vitalik Buterin (2014)」; 본문 "as of April 2014"; ethereum.org/whitepaper/ 본문 "This whitepaper was published in 2014", 메타 설명 "published in 2013"; Wayback CDX 최초 캡처 vitalik.ca/ethereum.html 2013-12-19. 초고 본문은 열지 않았다.

28. SEC, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Release No. 81207 (July 25, 2017), 1~2쪽. 원문: "Based on the investigation, and under the facts presented, the Commission has determined that DAO Tokens are securities under the Securities Act of 1933 ("Securities Act") and the Securities Exchange Act of 1934 ("Exchange Act")." / "The Commission has determined not to pursue an enforcement action in this matter based on the conduct and activities known to the Commission at this time." / "The automation of certain functions through this technology, "smart contracts," or computer code, does not remove conduct from the purview of the U.S. federal securities laws." 매각 기간 "From April 30, 2016 through May 28, 2016"(2쪽). 공격·포크 날짜는 26장(이더리움 재단 공지 2016-06-17·07-20).

29. Hayden Adams, Noah Zinsmeister & Dan Robinson, "Uniswap v2 Core" (March 2020), 1쪽. 원문: "Uniswap v1 is an on-chain system of smart contracts on the Ethereum blockchain, implementing an automated liquidity protocol based on a "constant product formula" [1]."

0부 지도

3장 이 책을 읽는 법 — 표식·상자·위젯·노트북, 세 경로

> **고지**
>
> 먼저 분명히 해 둘 것이 하나 있습니다. **이 책은 무엇을 사라거나 팔라는 책이 아닙니다.** 블록체인이 어떻게 돌아가는지, 그 위에서 돈과 사람이 어떻게 움직이는지를 기술과 경제학의 말로 풀어 보는 공부책입니다. 그래서 이 책에는 가격 전망이 없고, 시세 차트도 원칙적으로 싣지 않았으며, 어느 거래소나 지갑이나 토큰이 좋다는 말도 없습니다. 이름이 나오는 곳은 이미 끝난 역사적 사실을 적을 때뿐입니다.
>
> 수치에는 전부 기준일과 출처를 붙였습니다. 이 분야는 빨리 바뀌므로 읽으시는 시점에는 달라졌을 수 있습니다. 코드는 원리를 눈으로 보기 위한 장난감이고, 실제 자산을 움직이도록 만들지 않았습니다. 그대로 쓰지 마십시오.
>
> 엮은이는 현직 법관입니다. 이 책은 엮은이 개인의 공부 기록이고 소속 기관과는 관계가 없습니다. 아직 끝나지 않은 사건이나 국회에 있는 법안에 관해서는 사실만 적고 제 생각을 적지 않았습니다. 법률 자문도 아닙니다.
>
> 고지 전문은 판권면(부록 F의 F.4)에 있습니다.

핵심 TAKEAWAY

이 책은 법학자가 고교 수학만 가지고도 끝까지 읽을 수 있게 썼습니다. 그 약속을 지키는 장치가 **표식 세 등급**([입문]·[중급]·[심화])과 **접히는 상자입니다** — [심화]를 다 건너뛰어도 본문의 논지는 끊기지 않습니다. 「법률가의 질문」 상자는 질문만 던지고 답은 5부로 넘기며, 위젯과 노트북은 본문의 식과 코드를 손으로 움직여 보는 자리입니다. 세 경로(입문·기술·경제와 법)가 장 번호로 적혀 있고, 수치에는 전부 기준일(2026-10-25)이 붙어 있으며, 모든 코드는 공부용이라 실제 자산에 쓰지 않습니다. 이 장에서 가져갈 것은 **「[심화]는 건너뛰어도 책이 끊기지 않는다」는 약속 자체입니다.**

학습목표

1. 표식 세 등급, 접히는 상자, 법률가의 질문 상자, 위젯·노트북의 쓰임을 안다.
2. 세 경로 가운데 자기 것을 고르고, 장 번호로 그 경로를 따라갈 수 있다.
3. 수치 기준일·「2026-10 기준」 표시·「원문 확인 필요」 표시가 무엇을 뜻하는지 안다.
4. NB00으로 자기 환경을 점검한다.

[입문]만 읽은 독자가 가져가는 것. 「[심화]는 건너뛰어도 책이 끊기지 않는다」는 약속 자체.

1. 표식 세 등급

[중급]·[심화] 문단에는 머리에 표식이 붙어 있습니다. 표식이 없는 문단이 [입문]입니다.

표식	전제	담는 것	건너뛰면
[입문]	고교 수학	직관, 그림, 비유, 법률가의 질문, 정리	건너뛰지 않습니다 — 이것이 본문입니다
[중급]	대수식, 확률 계산, 짧은 코드 읽기	식의 전개, 표, 파이썬 코드와 그 출력	결론은 [입문]에 다시 적혀 있어 논지가 끊기지 않습니다
[심화]	타원곡선 군 연산, 포아송·랜덤워크, 균형 도출, 증명	수학의 속, 원논문의 정정과 재현	접혀 있습니다 — 펼치지 않아도 됩니다

규칙은 하나입니다. **[심화] 상자 앞뒤의 [입문] 문장만으로 논지가 이어져야 한다.** 그래서 장마다 「[입문]만 읽은 독자가 가져가는 것」이 학습목표 아래에 한 줄로 적혀 있습니다. 그 한 줄을 모아 읽으면 책 전체의 뼈대가 됩니다. 수식에는 기호 풀이 한 줄과 수치 예 하나가 붙습니다 — 예컨대 9장의 공격 성공 확률 식 아래에는 「$q = 0.1$, $z = 6$ 이면 0.0002428」처럼 숫자가 하나 따라옵니다. 식을 읽지 않아도 그 숫자는 읽을 수 있습니다.

2. 상자 셋

접히는 상자. 화면본에서는 제목 줄을 눌러 펼치고, PDF에서는 회색 바탕으로 구별됩니다. [심화] 수식과 긴 코드가 여기 들어갑니다. 제목에 「[심화]」가 붙어 있으면 그 상자는 건너뛰어도 됩니다.

법률가의 질문. 「법률가의 질문」이라 표시된 상자(제목 앞에 동그라미 속 물음표가 있습니다)는 그 장의 기술·경제 사실에서 **법적 질문이 생기는 자리를** 적어 둔 것입니다. 질문만 적고 답은 적지 않습니다. 상자마다 끝에 5부의 절(27.2, 27.3, 28.1의 (a)~(g))로 가는 화살표가 있고, 그 절에서 이 책이 원문으로 확인한 1차 자료가 **어디까지 닿는지와 비어 있는 지점을** 적어 두었습니다. 스물세 개가 있고 (1·4·5·7·8·9·10·11·13·14·15·16·17·19·20·21·22·23·24·25·26·29·30장), 28.1 끝의 「상자와 쟁점의 대응표」가 전부를 한 표에 모읍니다. 이 상자가 이 책이 법률가에게 줄 수 있는 것의 본체입니다 — 법리 해설이 아니라, **기술·경제 사실이 법적 판단의 어느 지점에 들어가는지를 보여 주는 다리**(27.1).

핵심 Takeaway. 장 머리의 상자입니다. 「이 장을 읽으면 무엇을 할 수 있게 되는가」를 적습니다. 요약이 아니라 이 책의 독자 — 법률가 — 가 당장 가져갈 것입니다.

3. 위젯과 노트북

위젯은 화면본(HTML)에서 본문의 식 옆에 놓인 조작판입니다. 변수를 움직이면 도표가 바뀝니다. 처음 상태는 언제나 본문의 예제와 같아서, 본문의 숫자를 화면에서 먼저 확인하고 그 뒤에 변수를 움직이게 되어 있습니다. 변수는 **구조 변수입니다** — 해시파워 점유율, 확인 수, 담보율, 유동성 풀 크기, 상환 요구 비율. 자산의 시세나 투자 수익률을 넣는 변수는 없고 그런 위젯은 만들지 않았습니다. 하나 덧붙일 것은 20장의 공격 비용 계산(W6)입니다. 거기에는 공격에 드는 자원의 비용 단가(해시 단가)를 사용자가 직접 넣는 칸이 있고, 이 책은 그 값을 주지 않습니다. PDF에서는 위젯 자리에 처음 상태의 정지 그림이 들어갑니다.

위젯	장	움직이는 변수	처음 상태
W1 해시 눈사태	4	입력 문자열, 뒤집을 비트	「hello」에서 한 글자 변경
W4 머클 증명	6	거래 수 n, 증명할 잎	$n = 8$, 잎 5
W3 체인 변조 시연	7	변조할 블록 번호·내용	5블록 체인, 3번 블록 변조
W2 손 채굴기	8	난이도 비트 수	16비트
W5 이중지불 확률	9	q, z	$q = 0.1, z = 0 \ldots 10$ (백서 §11 표)
W12 블록 헤더 해부기	12	필드 선택, 논스	제네시스 블록
W7 AMM 슬리피지	15	풀 크기 x, y, 주문 Δx	1,000 : 1,000, 주문 100
W10 샌드위치 공격	17	피해 주문, 공격자 투입, 풀 크기, 허용치	15장의 풀, 피해자 100·공격자 100
W6 51% 공격 비용	20	점유율 q, 해시 단가, 지속 시간	점유율 60%, $k + e = 12$
W9 수수료 시장	21	블록별 가스 사용량	한도 60,000,000·목표 30,000,000
W8 스테이블코인 뱅크런	24	상환 요구 비율, 준비자산 구성, 담보율	유형 B′, 요구 40%
W11 클릭 연표	2·부록 D	클릭	1976~2023

노트북은 본문의 파이썬 코드를 그대로 옮긴 Colab 파일입니다. 장 번호와 같은 번호가 붙어 있고(NB04, NB05, ..., NB29), 본문에서 「NB09-2」라 적힌 코드 셀은 NB09 노트북의 둘째 셀입니다. 규칙 넷입니다. 본문의 「출력」 블록은 엮은이가 실제로 실행한 결과를 **그대로** 옮긴 것이고 손으로 다듬지 않았습니다. 설치 셀은 없습니다 — 표준 라이브러리만 쓰고, 예외는 25장의 수치 패키지 하나뿐이며 그 이유는 해당 장과 노트북 목록(부록 F)에 적혀 있습니다. 외부 API를 부르지 않습니다 — 공개 블록 데이터는 기준일에 받아 둔 스냅샷 파일(제네시스 블록, 해시율, 노출 출력 유형)만 읽고, 파일이 없으면 같은 값의 상수로 대신합니다. 그리고 **모든 코드는 공부용입니다.** 작은 소수 위의 장난감 타원곡선, 약한 난이도의 손 채굴기, 실제 자산을 움직이지 않는 상태 전이기입니다. 코드 주석에도 「실사용 금지」라고 적어 두었습니다. 실제 개인키를 만들거나 보관하거나 거래를 보내는 코드는 이 책에 없습니다.

아래가 환경 점검 셀(NB00)입니다. 이 책의 코드가 전제하는 것은 파이썬 3.13과 표준 라이브러리뿐입니다.

```
# NB00: 환경 점검 — 이 책의 코드가 전제하는 것은 Python 3.13 과 표준 라이브러리뿐이다 (공부용)
import sys, hashlib, secrets, json, struct, math, random, decimal

major, minor = sys.version_info[:2]
print(f"Python {major}.{minor}  →", "좋습니다(3.13 이상)" if (major, minor) >= (3, 13) else "3.13
이상을 권합니다 — 일부 셀의 출력 형식이 다를 수 있습니다")
print("SHA-256 동작 확인:", hashlib.sha256(b"").hexdigest()[:16], "… == e3b0c44298fc1c14 …",
hashlib.sha256(b"").hexdigest().startswith("e3b0c44298fc1c14"))
print("이중 SHA-256(12장이 쓰는 함수):",
hashlib.sha256(hashlib.sha256(b"abc").digest()).hexdigest()[:16], "…")
print("secrets 모듈(5장 장난감 서명의 비밀수 생성용):", "사용 가능" if secrets.randbits(1) in (0, 1)
else "?")
print("struct(12장 바이트 해석)·json(20·29장 스냅샷 읽기)·decimal(29장 수량 합산):", "사용 가능")
try:
    import numpy  # 25장 NB25 만 쓴다 — 없어도 다른 장은 전부 돈다
    print("numpy:", numpy.__version__, "(25장 NB25 에만 필요)")
except ImportError:
    print("numpy: 없음 — 25장 NB25 를 돌릴 때만 설치하면 됩니다. 다른 장은 전부 표준 라이브러리로 돕니다.")
print()
print("이 책의 코드는 전부 공부용입니다. 실제 자산·실제 개인키·실제 거래에 쓰지 않습니다(실사용 금지).")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
Python 3.13  → 좋습니다(3.13 이상)
SHA-256 동작 확인: e3b0c44298fc1c14 … == e3b0c44298fc1c14 … True
이중 SHA-256(12장이 쓰는 함수): 4f8b42c22dd3729b …
secrets 모듈(5장 장난감 서명의 비밀수 생성용): 사용 가능
struct(12장 바이트 해석)·json(20·29장 스냅샷 읽기)·decimal(29장 수량 합산): 사용 가능
numpy: 2.5.4 (25장 NB25 에만 필요)

이 책의 코드는 전부 공부용입니다. 실제 자산·실제 개인키·실제 거래에 쓰지 않습니다(실사용 금지).
```

4. 세 경로

전부 읽을 필요는 없습니다. 세 경로를 장 번호로 적습니다.

경로	장	전제
입문 경로	1·2·3 → 4·7·8·9·12 → 13·15 → 19·20·24 → 26 → 27·28 → 30	[입문] 문단만. 수식·코드 전부 건너뜀
기술 경로	1·3 → 4~12 전부 → 13~18 전부 → 20·21·23 → 29 → 부록 E	[중급]까지, 노트북 실행
경제·법 경로	1·2·3 → 4·8·9(개념만) → 15·17 → 19~25 전부 → 26 → 27·28 → 30 → 부록 B·C	[중급]까지, 25장은 [심화] 포함

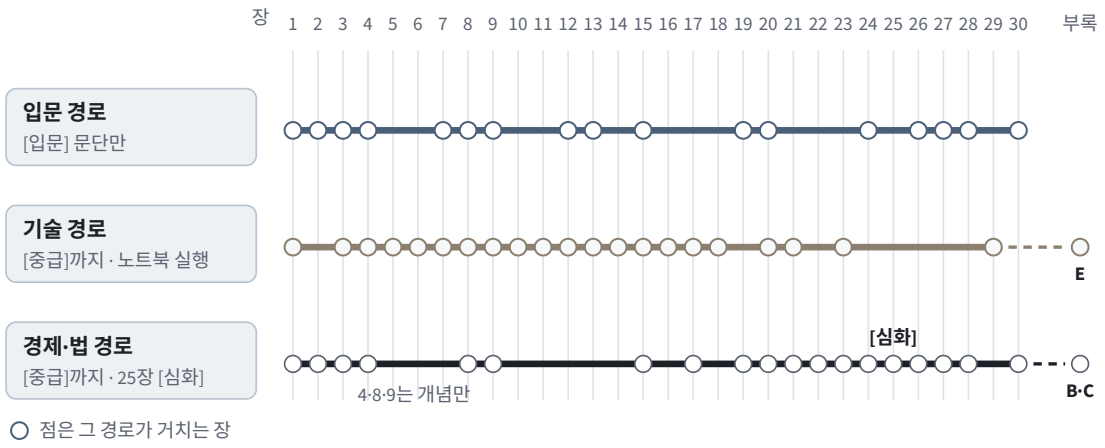


그림 3-1. 위 표의 세 경로를 장 번호를 역으로 둔 노선도로 옮겼다. 점은 그 경로가 거치는 장이고, 기술 경로는 부록 E로, 경제·법 경로는 부록 B·C로 이어진다.

입문 경로는 1부에서 다섯 장만 거칩니다 — 해시(4), 사슬(7), 작업증명(8), 확률적 최종성(9), 그리고 진짜 블록 하나(12). 그 다섯으로 「공개·단일한 역사·계산에 의한 다수」(1장)의 뜻이 닿습니다. 기술 경로는 1부와 2부를 다 읽고 6부의 양자 전환으로 가며, 부록 E의 수학 보충이 그 경로의 각주입니다. 경제·법 경로는 3부를 전부 읽습니다 — 25장만은 `[심화]`를 펼쳐야 하는데, 버블 계량검정의 요지가 임계값의 계산 방식 자체에 있기 때문입니다. 세 경로가 모두 거치는 장은 1·3·4·8·9·15·20입니다.

5. 일러두기 — 수치, 기준일, 표시

기준일. 이 책의 수치는 전부 기준일 2026년 10월 25일(한국 시각; 실제 관측은 2026년 10월 24일 UTC)입니다. 블록 높이, 난이도, 해시율, 상수, 노출 출력 유형, 조문의 시행 상태가 그 날의 것이고, 동봉한 스냅샷 파일(`data/`)이 그 원시값입니다. 하나만 다릅니다 — 29장의 노출 집계는 제3자가 2026년 10월 19일에 만든 스냅샷을 기준일에 받은 것입니다(29장 3절). 그 뒤에 바뀌었을 수 있습니다.

「2026-10 기준」. 본문에 낡기 쉬운 값이 불가피하게 들어간 자리에는 문장 안에 「2026-10 기준」이라고 적고, 부록 A 동향 노트에 같은 항목을 올려 두었습니다. 판을 바꿀 때는 부록 A만 갈아 끼웁니다(갱신 주기는 반기). 본문의 원리 문장은 바뀌지 않습니다 — 예컨대 「난이도는 2,016블록마다 조정된다」는 원리이고, 「기준일의 난이도는 약 1.4×10^{14}」는 부록 A-4입니다.

「원문 확인 필요」·「미확인」·「유보」. 이 책은 기억으로 서지를 적지 않았고, 원문을 열어 축어로 대조한 것만 각주에 인용했습니다. 확인하지 못한 것은 그렇다고 적었습니다. 각 장의 끝 「이 장의 출처」에는 쓴 자료와 함께 **쓰지 않은 것 · 확인하지 못한 것이** 적혀 있습니다. 「법원 문서가 판단하지 않은 것」, 「2차 전재로만 확인한 문구」, 「🟡 2차 집계」 같은 표시가 그것입니다. 그 표시는 약점이 아니라 이 책이 어디까지 믿을 수 있는지를 적은 눈금입니다.

각주. 각주에는 원문의 **축어 인용이** 영어·일본어·한국어 원문 그대로 들어 있습니다. 본문의 번역은 엮은이의 것이고, 번역이 미심쩍으면 각주의 원문을 보십시오. 조문은 번역하지 않고 원문만 적었습니다(30장) — 번역은 해석이기 때문입니다. 판결은 법원·선고일·사건번호를 함께 적었고, 판시사항·판결요지 같은 편집물은 인용하지 않았습니다.

이 책이 하지 않는 것. 가격 전망을 적지 않고(과거 가격 수치는 방법을 설명하는 데 꼭 필요한 곳에서만 기준일과 출처를 붙여 씁니다), 어느 자산·거래소·지갑·토큰도 추천하거나 비교 평가하지 않습니다. 사례로 이름이 나오는 곳은 확정된 역사적 사실 — 판결, 공식 조사, 도산 절차 문서, 표준 — 을 적을 때뿐입니다. 이 장 머리, 4부 첫머리, 판권면의 고지문 세 곳이 그 약속을 적은 것입니다.

저작권. 이 책의 글과 위젯 코드는 CC BY-NC 4.0으로 배포합니다 — 출처와 라이선스 링크를 밝히고 바꾼 곳이 있으면 그 사실을 표시하는 조건으로, 비영리 목적으로 복제·수정·재배포할 수 있습니다 (https://creativecommons.org/licenses/by-nc/4.0/). 각주에 인용한 원문의 저작권은 각 저작권자에게 있고, 동봉 스냅샷 데이터의 출처는 데이터 폴더의 README에 적혀 있습니다. 2부의 Solidity 읽기용 예시의 고지는 판권면(부록 F)에 둡니다.

6. 정리

표식이 깊이를 가르고, 상자가 길을 비키고, 질문 상자가 법의 자리를 표시하며, 위젯과 노트북이 식과 코드를 손에 쥐어 줍니다. 세 경로 가운데 하나를 고르고, 수치마다 기준일을 확인하고, 각주의 원문으로 번역을 되짚으십시오. 그리고 `[심화]`는 건너뛰어도 됩니다 — 그 약속이 지켜졌는지가 이 책이 잘 쓰였는지의 기준입니다. 다음 4장부터 1부입니다.

이 장의 출처

- 이 책의 집필 규칙(표식 세 등급, 접히는 상자, 법률가의 질문 상자, 위젯 변수의 범위, 코드의 「실사용 금지」, 기준일 표시, 동향 노트 갱신 주기, 라이선스)은 엮은이가 정한 것이다.
- 위젯 목록과 처음 상태는 각 장의 본문 예제(4·6·7·8·9·12·15·17·20·21·24장)와 같다. 노트북 목록은 부록 F.
- 이 책의 동봉 데이터 `data/README.md`(기준일 2026-10-25, 접근 2026-10-24 UTC) — 스냅샷 3종의 출처·산식.
- Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0), https://creativecommons.org/licenses/by-nc/4.0/.
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리만. 노트북 NB00.

1부 기술 기초

4장 해시 함수 — 지문을 찍는 일방향 함수

핵심 TAKEAWAY

해시 함수는 어떤 길이의 데이터든 고정 길이의 짧은 「지문」으로 바꾸는 함수입니다. 지문에서 원본으로 되돌아갈 수 없고(일방향), 같은 지문을 가진 다른 원본을 만들어 낼 수 없으며(충돌 저항), 원본의 한 글자가 바뀌면 지문 전체가 바뀝니다(눈사태). 블록체인이 「변조 불가」라고 불리는 근거의 절반이 여기서 나옵니다. 나머지 절반은 8장의 작업증명입니다. 이 장을 읽으면 해시값이 **무엇을 증명하고 무엇을 증명하지 못하는지를** 가를 수 있게 됩니다.

학습목표

이 장을 읽고 나면 다음을 할 수 있습니다.

1. 일방향성·충돌 저항·눈사태 효과를 서로 구별해 말한다.
2. Python 표준 라이브러리로 SHA-256을 직접 계산해, 입력의 한 글자 차이가 출력 전체를 바꾸는 것을 확인한다.
3. 「해시값만 보관한 문서」가 나중에 무엇을 증명할 수 있는지, 그 한계가 어디인지 기술 차원에서 설명한다.
4. [중급] 생일 역설로 256비트 해시의 충돌을 찾는 데 왜 2^{128} 번 시도가 필요한지 계산한다.

[입문]만 읽은 독자가 가져가는 것. 해시는 「되돌릴 수 없는 지문」이고, 블록체인이 「변조 불가」라고 불리는 근거의 절반은 여기서 나옵니다. 나머지 절반은 8장에 있습니다.

1. 이 기술은 무엇의 실패에서 나왔나

데이터가 중간에 바뀌지 않았는지 확인하는 가장 오래된 방법은 **검사합(checksum)입니다.** 바이트를 전부 더해 끝자리를 적어 두고, 나중에 다시 더해 같은지 봅니다. 전송 중 전파 잡음으로 비트 하나가 뒤집히는 **우연한** 오류를 잡는 데에는 충분합니다.

문제는 **고의입니다.** 더하기는 누구나 거꾸로 할 수 있습니다. 문서를 고친 사람은 끝자리를 맞추기 위해 바이트 하나를 조정하거나 덧붙이면 됩니다. 검사합은 「실수로 바뀌었나」에는 답하지만 「누가 일부러 바꿨나」에는 답하지 못합니다. 아래 3절의 코드가 이것을 눈으로 보여 줍니다.

그래서 암호학자들은 다른 종류의 함수를 찾았습니다. 계산은 쉽지만 거꾸로는 안 되고, 같은 값이 나오는 다른 입력을 일부러 만들어 낼 수도 없는 함수입니다. 머클(Ralph Merkle)은 1987년 논문에서 **일방향 함수(one-way function)를** 이렇게 정의했습니다. 「계산하기는 쉽지만 역을 구하기는 어려운 함수. x 가 주어지면 $y = F(x)$ 를 계산하기는 쉽다. y 가 주어지면 $F(x) = y$ 인 어떤 x 든 찾아내기는 어렵다.」 그리고

임의 길이 입력을 받아 짧은 고정 길이 출력을 내는 것을 **일방향 해시 함수(one-way hash function)라고** 불렀습니다.[1]

타임스탬프 연구자인 하버(Stuart Haber)와 스토네타(W. Scott Stornetta)는 1991년 논문에서 이와 비슷하되 더 강한 요구를 담은 **충돌 없는(collision-free) 해시 함수의** 족(family)을 정의했습니다. 임의 길이의 비트열을 고정 길이 l 비트로 압축하는 함수들로서, (1) 계산이 쉽고 족에서 하나를 무작위로 고르기도 쉬우며, (2) 주어진 h 에 대해 $h(x) = h(x')$ 인 서로 다른 x, x' 를 찾는 것이 계산상 불가능한 함수들입니다.[2] 이 두 사람의 논문이 블록체인 전사(前史)에서 왜 중요한지는 7장에서 봅니다.

2. 직관 — 지문, 그리고 지문이 하지 못하는 일

사람의 지문은 사람보다 훨씬 작은 정보이지만 사람을 가려냅니다. 지문에서 사람을 복원할 수는 없고, 다른 사람이 같은 지문을 갖도록 만들 수도 없습니다. 암호학적 해시 함수(cryptographic hash function)는 데이터의 지문입니다. 어떤 길이의 파일이든 넣으면 정해진 길이(SHA-256이면 256비트, 16진수로 64자)의 값이 나옵니다.

미국 표준기술연구소(NIST)의 보안 해시 표준(FIPS 180-4)은 이 함수들이 왜 「안전(secure)」하다고 불리는지를 두 문장으로 적습니다. 주어진 메시지 다이제스트에 대응하는 메시지를 찾는 것이 계산상 불가능하고, 같은 다이제스트를 내는 서로 다른 두 메시지를 찾는 것이 계산상 불가능하다. 그리고 메시지의 어떤 변경도 매우 높은 확률로 다른 다이제스트를 낳는다.[3] 표준의 문구는 원상 찾기 불가능과 충돌 찾기 불가능 둘을 들고 변경 시 다이제스트가 바뀐다고 덧붙입니다. 암호학 교과서들은 이것을 아래 세 성질로 나누어 부르며, 표는 이 책의 정리입니다.

성질	뜻	깨지면 생기는 일
원상 저항 (preimage resistance), 곧 일방향성	해시값 y 만 보고 $h(x) = y$ 인 x 를 찾을 수 없다	해시값만 공개한 비밀(비밀번호, 봉인된 문서)이 드러난다
제2 원상 저항 (second-preimage resistance)	주어진 x 와 같은 해시를 내는 다른 x' 를 찾을 수 없다	이미 공개된 문서를 같은 지문의 다른 문서로 바꿔 끼울 수 있다
충돌 저항 (collision resistance)	같은 해시를 내는 **아무** 두 입력 $x \neq x'$ 도 찾을 수 없다	처음부터 「두 얼굴」 문서를 만들어 둘 수 있다



그림 4-1. 해시 함수의 세 성질을 공격자에게 주어진 것과 찾아야 하는 것으로 나란히 놓았다. 오른쪽으로 갈수록 공격자가 고를 수 있는 것이 많아지며, 셋째인 충돌 저항이 가장 강한 요구다.

셋째가 가장 강한 요구입니다. 공격자가 두 입력을 모두 고를 수 있으니까요. 그래서 셋 중 셋째가 깨지기 가장 쉽고, 같은 비트 수에서 가장 먼저 위태로워집니다. 아래 4절의 생일 역설이 그 이유를 수로 보여 줍니다.

그리고 네 번째 성질이 있습니다. 표준 문구의 마지막 문장, 「어떤 변경도 다른 다이제스트를 낳는다」입니다. 실제로는 더 강해서, 입력의 비트 하나만 바꿔도 출력 비트의 절반쯤이 뒤집힙니다. 이것을 **눈사태 효과(avalanche effect)라고** 부릅니다. 지문이 「조금 닮은」 일은 없습니다. 같거나, 전혀 다릅니다.

해시가 하지 못하는 일

해시는 암호화가 아닙니다. 암호화는 열쇠가 있으면 되돌릴 수 있어야 합니다. 해시는 누구도 되돌릴 수 없습니다. 「암호화폐」라는 이름 때문에 비트코인이 거래 내용을 암호화한다고 생각하기 쉽지만, 비트코인의 거래와 원장은 암호화되어 있지 않습니다. 모두 공개되어 있고, 거기에 쓰이는 것은 해시와 서명입니다. 이 이야기는 5장에서 이어집니다.

해시는 또 **내용을 보증하지 않습니다.** 해시값은 「이 비트열이 그때 그 비트열과 같다」만 말합니다. 그 비트열이 진실인지, 누가 썼는지, 언제 만들어졌는지는 말하지 않습니다. 하버와 스토네타가 1991년에 이미 적어 둔 한계가 있습니다. 문서가 만들어진 뒤 타임스탬프를 받기까지 시간이 얼마든 지날 수 있으면, 어떤 방법도 문서 자체가 만들어진 시점을 앞쪽으로만 제한할 수 있다. 그러므로 타임스탬프는 일반적으로 문서가 **날짜를 거슬러 조작되지 않았다는** 증거로만 보아야 한다.[4] 해시를 체인에 묶어 「존재 증명」을 한다는 서비스는 많지만, 그 증명이 가리키는 것은 정확히 이 범위입니다.

> ? **법률가의 질문**
>
> 어떤 문서의 해시값만을 공개 원장에 적어 두었다가 수년 뒤 원본 파일과 함께 제출했습니다. 이 해시값은 무엇을 증명하고 무엇을 증명하지 못하는가요. 「그 시점에 그 비트열이 존재했다」와 「그 시점에 그 문서가 작성되었다」와 「그 문서의 내용이 진실이다」는 각각 다른 명제입니다. 증거법의 원본·사본·진정성립 개념은 이 셋 중 어디에 닿습니까. → 28장

3. 코드 — 한 글자가 전부를 바꾼다

[중급] Python 표준 라이브러리 `hashlib` 에 SHA-256이 들어 있습니다. 아래 코드는 전부 실제로 실행한 결과를 그대로 실었습니다. 실행 환경은 Python 3.13이고, 이 책의 모든 코드는 표준 라이브러리만 씁니다.

```
# NB04-1: SHA-256 지문 찍기 — 한 글자 차이
import hashlib

def sha256_hex(s: str) -> str:
    """문자열을 UTF-8 바이트로 바꿔 SHA-256 다이제스트를 16진수 64자로 돌려준다."""
    return hashlib.sha256(s.encode("utf-8")).hexdigest()

for s in ["hello", "hellp", "Hello", "hello "]:
    print(f"{s!r:10} -> {sha256_hex(s)}")

def differing_bits(h1: str, h2: str) -> int:
    """두 16진수 다이제스트에서 서로 다른 비트의 개수(해밍 거리)."""
    return bin(int(h1, 16) ^ int(h2, 16)).count("1")

print()
print("hello vs hellp : 다른 비트", differing_bits(sha256_hex("hello"), sha256_hex("hellp")), "/
256")
print("hello vs Hello : 다른 비트", differing_bits(sha256_hex("hello"), sha256_hex("Hello")), "/
256")
print("hello vs 'hello ': 다른 비트", differing_bits(sha256_hex("hello"), sha256_hex("hello ")),
"/ 256")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
'hello'    -> 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
'hellp'    -> fdd7585e08c4e2afd71dcabdb4636c89d557a3f42db9e2040c8bbd1708aa4ce7
'Hello'    -> 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
'hello '   -> 5e3235a8346e5a4585f8c58562f5052b8fe26a3bb122e1e96c76784964dfc461

hello vs hellp : 다른 비트 131 / 256
hello vs Hello : 다른 비트 125 / 256
hello vs 'hello ': 다른 비트 139 / 256
```

[중급] 네 입력은 사람 눈에 거의 같지만 지문은 전혀 다릅니다. 256비트 중 절반 가까이가 뒤집혔습니다. 이것이 눈사태 효과입니다. 우연이 아님을 확인하려면 비트 하나씩 뒤집으면서 평균을 내 봅니다.

비트 하나씩 뒤집기 — 눈사태의 평균

```
# NB04-2: 눈사태 효과 — 입력 비트를 하나씩 뒤집어 출력이 얼마나 바뀌는지 평균을 낸다
import hashlib

msg = b"The Times 03/Jan/2009 Chancellor on brink of second bailout for banks"
base = hashlib.sha256(msg).digest()

def flip_bit(data: bytes, i: int) -> bytes:
    """바이트열의 i번째 비트(0부터)를 뒤집는다."""
    b = bytearray(data)
    b[i // 8] ^= 1 << (i % 8)
    return bytes(b)

diffs = []
for i in range(len(msg) * 8):          # 입력의 모든 비트(69바이트 × 8 = 552개)를 한 번씩 뒤집는다
    h = hashlib.sha256(flip_bit(msg, i)).digest()
    diffs.append(sum(bin(a ^ b).count("1") for a, b in zip(base, h)))

print("뒤집은 입력 비트 수:", len(diffs))
print("출력에서 바뀐 비트 수 — 최소 %d, 최대 %d, 평균 %.1f (256비트 중)" % (min(diffs), max(diffs),
sum(diffs) / len(diffs)))
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
뒤집은 입력 비트 수: 552
출력에서 바뀐 비트 수 — 최소 101, 최대 150, 평균 127.5 (256비트 중)
```

[중급] 입력의 어느 비트를 뒤집어도 출력은 평균 128비트 안팎이 바뀝니다. 출력이 입력과 무관한 난수처럼 보인다는 뜻입니다. 이 입력 문장이 무엇인지는 12장에서 밝힙니다.

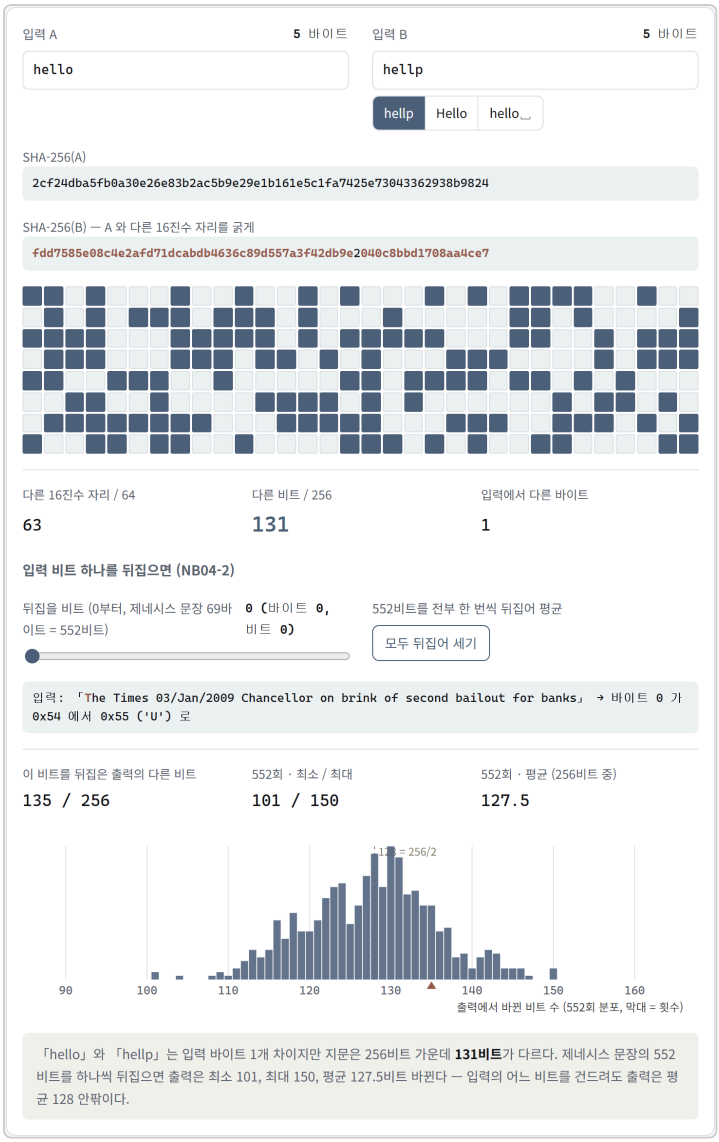


그림 4-2. 해시 눈사태. 「hello」와 「hellp」는 한 글자 차이지만 SHA-256 지문은 256비트 가운데 131비트가 다르다. 입력의 어느 비트를 뒤집어도 출력은 평균 128비트 안팎이 바뀐다(NB04-2: 552번 뒤집어 최소 101, 최대 150, 평균 127.5). — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 4장 3절).

검사합은 왜 안 되는가 — 위조 실습

[중급] 1절에서 한 말을 코드로 확인합니다. 바이트를 더해 끝자리(mod 256)를 적는 검사합과, SHA-256의 앞 16비트만 쓰는 「장난감 해시」를 비교합니다.

```
# NB04-3: 검사합 위조는 즉시, 해시 위조는 전수 탐색
import hashlib

def checksum8(data: bytes) -> int:
    """바이트 합의 끝자리(mod 256) — 우연한 오류용 검사합."""
    return sum(data) % 256

original = b"Pay to Alice 100"
target = checksum8(original)
forged = bytearray(b"Pay to Bob   100 ")          # 수취인을 바꾸고
forged.append((target - checksum8(forged)) % 256)   # 뒤에 바이트 하나를 붙여 끝자리를 맞춘다
print("원본 검사합  :", target)
print("위조문 검사합:", checksum8(forged), "| 위조문:", bytes(forged))

def toy_hash16(data: bytes) -> int:
    """SHA-256의 앞 16비트만 쓰는 장난감 해시(실사용 금지 — 설명용)."""
    return int.from_bytes(hashlib.sha256(data).digest()[:2], "big")

t = toy_hash16(original)
attempts = 0
while True:
    attempts += 1
    candidate = b"Pay to Bob   100 #" + str(attempts).encode()
    if toy_hash16(candidate) == t:
        break
print("16비트 장난감 해시를 맞추기까지 시도 횟수:", attempts, "(기대값 2^16 = 65,536)")
print("256비트 전체를 맞추려면 기대 시도 횟수는 2^256 ≈ %.1e" % 2**256)
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
원본 검사합  : 220
위조문 검사합: 220 | 위조문: b'Pay to Bob   100 k'
16비트 장난감 해시를 맞추기까지 시도 횟수: 27882 (기대값 2^16 = 65,536)
256비트 전체를 맞추려면 기대 시도 횟수는 2^256 ≈ 1.2e+77
```

검사합은 바이트 하나를 덧붙여 바로 맞춰졌습니다. 16비트짜리 장난감 해시는 수만 번을 시도해야 했고, 그 횟수는 비트 수가 하나 늘 때마다 두 배가 됩니다. 256비트면 기대 $2^{256} \approx 1.2 \times 10^{77}$ 번입니다. 2026년 비트코인 네트워크 전체의 추정 해시율 수준인 초당 10^{21} 번(8장)으로 10억 년을 돌려도 약 3×10^{37} 번, 필요한 횟수의 10^{40} 분의 1입니다. 「해시를 맞추는 다른 문서를 만든다」는 말이 왜 실무에서 「불가능」으로 취급되는지가 이 두 줄의 차이입니다.

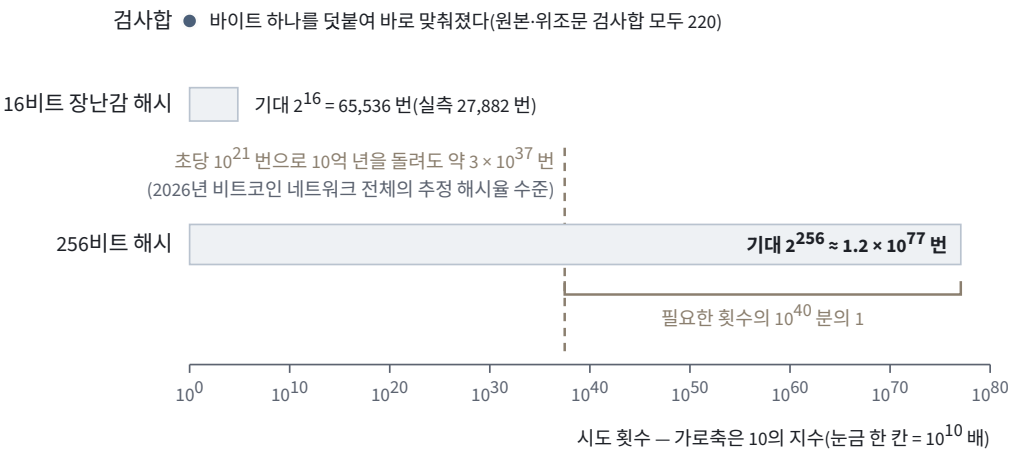


그림 4-3. 위조에 필요한 시도 횟수를 10의 지수 눈금 위에 놓았다. 수치는 NB04-3의 출력과 본문의 계산이며, 초당 10^{21} 번은 2026년 비트코인 네트워크 전체의 추정 해시율 수준이다.

4. [중급] 생일 역설 — 충돌 저항은 비트 수의 절반만큼만 강하다

[중급] 반에 23명이 있으면 생일이 같은 두 사람이 있을 확률이 절반을 넘습니다. 365일 중 23일밖에 안 되는데도 그렇습니다. 「누군가 둘」의 짝 수가 $\binom{23}{2} = 253$ 이기 때문입니다. 해시 충돌도 같은 셈입니다.

[중급] 출력 공간의 크기가 N 일 때 n 개를 무작위로 뽑으면 어느 두 개가 같을 확률은 근사적으로

$$P(\text{충돌}) \approx 1 - e^{-n^2 / (2N)}$$

입니다. 기호 풀이: N 은 가능한 해시값의 개수(SHA-256은 2^{256}), n 은 시도한 입력의 개수입니다. 이 확률이 1 / 2 가 되는 n 은 $n \approx 1.18\sqrt{N}$ 입니다. 수치 예: 16비트 장난감 해시($N = 65{,}536$)면 $n \approx 1.18 \times 256 \approx 302$ 개, SHA-256이면 $n \approx 1.18 \times 2^{128}$ 개입니다. 그래서 「256비트 해시의 충돌 저항은 128비트」라고 말합니다. 비트 수의 절반입니다.

[중급] 머클은 1987년 논문에서 이것을 「제곱근 공격(square root attack)」이라고 부르며 경고했습니다. 가장 뻔한(obvious) 해시 설계가 때로 이 공격에 약하다고요.[5] 아래 코드는 16비트 장난감 해시에서 첫 충돌이 나올 때까지 몇 개를 뽑아야 하는지 200번 재어 평균을 냅니다. 첫 충돌까지의 기대 개수는 $\sqrt{\pi N / 2} \approx 1.25\sqrt{N}$ 으로, $N = 65{,}536$ 이면 약 320.8개입니다.

```
# NB04-4: 생일 역설 — 16비트 장난감 해시에서 첫 충돌까지 뽑은 개수
import hashlib, math

def toy_hash16(data: bytes) -> int:
    return int.from_bytes(hashlib.sha256(data).digest()[:2], "big")

def draws_until_collision(run: int) -> int:
    """run번째 실험: 서로 다른 입력을 하나씩 넣어 처음 같은 해시가 나올 때까지의 개수."""
    seen = {}
    k = 0
    while True:
        k += 1
        h = toy_hash16(f"run{run}-item{k}".encode())
        if h in seen:
            return k
        seen[h] = k

results = [draws_until_collision(r) for r in range(200)]
N = 2**16
print("실험 횟수 200, 출력 공간 N = %d" % N)
print("첫 충돌까지 개수 — 평균 %.1f, 최소 %d, 최대 %d" % (sum(results) / len(results), min(results),
max(results)))
print("이론 기대값 sqrt(pi*N/2) = %.1f" % math.sqrt(math.pi * N / 2))
print("충돌 확률이 1/2이 되는 n ≈ 1.18*sqrt(N) = %.0f" % (1.18 * math.sqrt(N)))
print("SHA-256(N = 2^256)이면 1.18*sqrt(N) ≈ %.2e" % (1.18 * math.sqrt(2**256)))
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
실험 횟수 200, 출력 공간 N = 65536
첫 충돌까지 개수 — 평균 325.1, 최소 19, 최대 754
이론 기대값 sqrt(pi*N/2) = 320.8
충돌 확률이 1/2이 되는 n ≈ 1.18*sqrt(N) = 302
SHA-256(N = 2^256)이면 1.18*sqrt(N) ≈ 4.02e+38
```

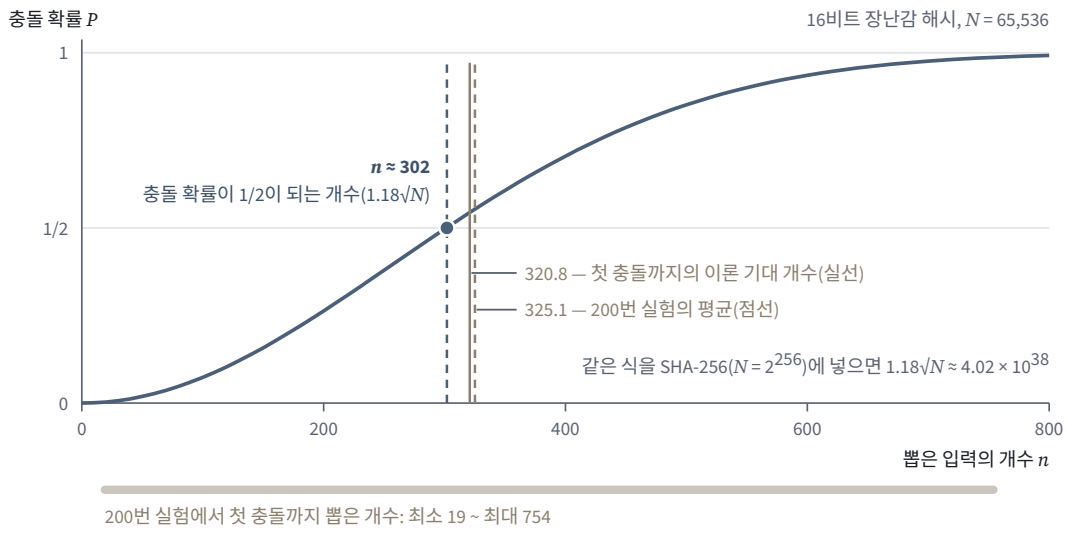


그림 4-4. 16비트 장난감 해시($N = 65{,}536$)에서 뽑은 개수 n에 따른 충돌 확률. 곡선은 본문 식 $1 - e^{-n^2/(2N)}$의 그래프이고, 세로선과 숫자는 NB04-4의 출력이다.

[중급] 평균이 이론값 근처에 옵니다. 16비트에서는 300여 개면 충돌이 납니다. 256비트에서는 10^{38} 개 규모입니다. 같은 식, 다른 지수입니다.

5. SHA-256은 무엇인가

비트코인이 쓰는 해시 함수는 SHA-256입니다. 미국 연방 표준 FIPS 180-4가 정한 알고리듬으로, 입력은 2^{64} 비트 미만의 어떤 메시지든 받고, 512비트 블록 단위로 처리하며, 32비트 워드로 계산해, 256비트 다이제스트를 냅니다.[6] 표준은 2012년판을 2015년 8월에 갱신한 것이고, NIST는 2023년에 이 표준의 개정 작업에 들어갔다고 밝혔습니다. 이 책이 참조한 현행본은 2015년판입니다.

비트코인은 SHA-256을 **두 번 겹쳐** 씁니다. 블록 헤더의 해시도, 거래의 식별자(txid)도 SHA256(SHA256(·)) 입니다. 주소를 만들 때는 공개키를 SHA-256으로 한 번 해시한 뒤 다시 RIPEMD-160이라는 160비트 해시로 줄입니다.[7] 왜 두 번인지에 관해 설계자가 남긴 설명은 없습니다. 인용한 논문은 「패딩 공격을 막기 위해 두 번 한다」는 이유를 괄호 안에 적지만 그것은 제3자의 설명이고, 이 책은 「두 번 겹친다」는 사실만 적습니다. 두 번 겹친 해시를 손으로 다시 계산해 공개된 블록 해시와 맞춰 보는 일은 12장에서 합니다.

▾ **[심화] SHA-256의 속 — 압축 함수를 이어 붙이는 구조**

[심화] 표준은 알고리듬을 두 단계로 나눕니다. **전처리**(메시지에 채움 비트를 붙이고, 512비트 블록으로 자르고, 초기 해시값을 정하는 일)와 **해시 계산**(블록마다 메시지 스케줄을 만들고 함수·상수·워드 연산으로 해시값을 반복 갱신하는 일)입니다. 마지막 해시값이 다이제스트가 됩니다.[8] 핵심은 「반복 갱신」입니다. 256비트 상태값 $H^{(i-1)}$과 512비트 메시지 블록 $M^{(i)}$를 압축 함수에 넣어 새 상태 $H^{(i)}$를 얻고, 그것을 다음 블록과 함께 다시 넣습니다. 상태값이 블록에서 블록으로 **사슬처럼** 넘어갑니다. 하버와 스토네타의 1991년 논문은 담고르(Damgård)가 일방향 「claw-free」 순열이 존재한다는 가정 아래 충돌 없는 해시 함수의 첫 정식 정의와 구성적 존재 증명을 주었다고 적고, 해시 함수의 「설계 원리」를 다룬 담고르의 1989년 논문도 함께 인용합니다.[9]

압축 함수를 사슬처럼 이어 붙이는 이 설계 원리의 이름과 자세한 전개는 부록 E로 보냅니다.

[심화] 아래는 SHA-256을 표준 명세대로 Python으로 다시 짠 것입니다. 60줄이 안 됩니다. `hashlib` 과 같은 값이 나오는지 확인하고, 두 블록짜리 메시지에서 상태값이 블록마다 어떻게 넘어가는지 중간값을 찍어 봅니다. **이 구현은 공부용입니다.** 실제 소프트웨어에서는 검증된 라이브러리를 쓰십시오(실사용 금지).

```
# NB04-5: SHA-256을 손으로 — 표준 명세의 반복 구조를 눈으로 본다 (공부용, 실사용 금지)
import hashlib, struct

def icbrt(n: int) -> int:
    """정수 세제곱근(내림) — 부동소수 오차 없이 상수를 만들기 위해 정수 연산만 쓴다."""
    x = 1 << ((n.bit_length() + 2) // 3)
    while True:
        y = (2 * x + n // (x * x)) // 3
        if y >= x:
            return x
        x = y

PRIMES =
[2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,83,89,97,101,103,107,109,113,

127,131,137,139,149,151,157,163,167,173,179,181,191,193,197,199,211,223,227,229,233,239,241,25
1,
          257,263,269,271,277,281,283,293,307,311]
# 상수 K: 처음 64개 소수의 세제곱근 소수부 앞 32비트 (FIPS 180-4 §4.2.2) — cbrt(p)·2^32 의 정수부를 2^32
로 나눈 나머지
K = [icbrt(p << 96) & 0xFFFFFFFF for p in PRIMES]
# 초기 해시값 H0: 처음 8개 소수의 제곱근 소수부 앞 32비트 (FIPS 180-4 §5.3.3)
import math
H0 = [math.isqrt(p << 64) & 0xFFFFFFFF for p in PRIMES[:8]]

def rotr(x, n): return ((x >> n) | (x << (32 - n))) & 0xFFFFFFFF

def compress(H, block):
    """압축 함수: 256비트 상태 H와 512비트 블록 하나 → 새 상태 (FIPS 180-4 §6.2.2)"""
    w = list(struct.unpack(">16I", block))
    for t in range(16, 64):
        s0 = rotr(w[t-15], 7) ^ rotr(w[t-15], 18) ^ (w[t-15] >> 3)
        s1 = rotr(w[t-2], 17) ^ rotr(w[t-2], 19) ^ (w[t-2] >> 10)
        w.append((w[t-16] + s0 + w[t-7] + s1) & 0xFFFFFFFF)
    a, b, c, d, e, f, g, h = H
    for t in range(64):
        S1 = rotr(e, 6) ^ rotr(e, 11) ^ rotr(e, 25)
        ch = (e & f) ^ (~e & g)
        T1 = (h + S1 + ch + K[t] + w[t]) & 0xFFFFFFFF
        S0 = rotr(a, 2) ^ rotr(a, 13) ^ rotr(a, 22)
        maj = (a & b) ^ (a & c) ^ (b & c)
        T2 = (S0 + maj) & 0xFFFFFFFF
        h, g, f, e, d, c, b, a = g, f, e, (d + T1) & 0xFFFFFFFF, c, b, a, (T1 + T2) &
0xFFFFFFFF
    return [(x + y) & 0xFFFFFFFF for x, y in zip(H, [a, b, c, d, e, f, g, h])]

def sha256_manual(msg: bytes, trace=False) -> str:
    L = len(msg) * 8
    padded = msg + b"\x80" + b"\x00" * ((55 - len(msg)) % 64) + struct.pack(">Q", L)   #
§5.1.1 채움
    H = H0[:]
    for i in range(0, len(padded), 64):
        H = compress(H, padded[i:i+64])
        if trace:
            print(f"  블록 {i//64 + 1} 처리 후 상태 H^({i//64 + 1}) = {''.join(f'{x:08x}' for x in
H)}")
    return "".join(f"{x:08x}" for x in H)

for m in [b"", b"abc", b"hello"]:
    ok = sha256_manual(m) == hashlib.sha256(m).hexdigest()
    print(f"{m!r:9} 손 구현 == hashlib ? {ok}")

two_blocks = b"a" * 100          # 100바이트 → 채움 뒤 512비트 블록 2개
print("100바이트 메시지(블록 2개)의 상태 변화:")
digest = sha256_manual(two_blocks, trace=True)
print("  최종 다이제스트            =", digest)
print("  hashlib                    =", hashlib.sha256(two_blocks).hexdigest())
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
b''       손 구현 == hashlib ? True
b'abc'    손 구현 == hashlib ? True
b'hello'  손 구현 == hashlib ? True
100바이트 메시지(블록 2개)의 상태 변화:
  블록 1 처리 후 상태 H^(1) = df5bb81ce81e0626fb45a8944fd40f31b25e6816d6d499c1ab90492900635e66
  블록 2 처리 후 상태 H^(2) = 2816597888e4a0d3a36b82b83316ab32680eb8f00f8cd3b904d681246d285a0e
  최종 다이제스트            = 2816597888e4a0d3a36b82b83316ab32680eb8f00f8cd3b904d681246d285a0e
  hashlib                    =
2816597888e4a0d3a36b82b83316ab32680eb8f00f8cd3b904d681246d285a0e
```

[심화] 블록 1을 처리한 상태값이 블록 2의 입력이 되고, 블록 2를 처리한 상태값이 그대로 다이제스트가 됩니다. 이 「상태를 넘겨 주는」 구조는 7장의 「이전 블록 해시를 다음 블록에 넣는」 구조와 모양이 같습니다. 블록체인은 어떤 뜻에서 해시 함수의 내부 구조를 네트워크 규모로 펼쳐 놓은 것입니다.

6. 정리 — 「변조 불가」의 절반

블록체인은 「변조 불가능한 장부」라고 흔히 소개됩니다. 이 장에서 본 것은 그 주장의 절반입니다. 해시 함수 덕분에 **변조는 반드시 드러납니다.** 한 글자를 바꾸면 지문이 바뀌고, 같은 지문을 내는 다른 내용을 만들어 끼울 수 없으니까요. 그러나 「드러난다」와 「막힌다」는 다릅니다. 지문이 바뀐 장부를 **새 지문과 함께** 통째로 다시 내놓으면 어떻게 될까요. 그것을 비싸게 만드는 장치가 8장의 작업증명이고, 비싸다는 것이 불가능하다는 뜻은 아니라는 점이 9장의 주제입니다. 부록 B의 오해 ①(「블록체인은 변조 불가능하다」)은 이 세 장을 다 읽은 뒤 판정합니다.

이 장의 출처

- NIST, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, August 2015. DOI 10.6028/NIST.FIPS.180-4. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf — 정의(iv쪽), 두 단계 구조(3쪽), 매개변수 표(Figure 1, 3쪽), 상수와 초기값(§4.2.2·§5.3.3), 채움(§5.1.1), SHA-256 계산(§6.2.2).
- Ralph C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in C. Pomerance (ed.), *Advances in Cryptology — CRYPTO '87*, LNCS 293, pp. 369–378. Springer, 1988. DOI 10.1007/3-540-48184-2_32 — 일방향 함수·일방향 해시 함수 정의와 제곱근 공격 경고(370쪽).
- Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *Journal of Cryptology* 3(2): 99–111, 1991. DOI 10.1007/BF00196791 — 충돌 없는 해시 함수의 정의(101–102쪽), 타임스탬프가 증명하는 범위(106쪽).
- Cyril Grunspan & Ricardo Pérez-Marco, "Double Spend Races," *International Journal of Theoretical and Applied Finance* 21(8): 1850053, 2018. DOI 10.1142/S021902491850053X; arXiv:1702.02867 — 이중 SHA-256(§2).
- Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin: Programming the Open Blockchain*, 3rd ed. O'Reilly, 2023 (CC BY-SA 4.0, GitHub 원고 커밋 94c9ba64) — 주소의 해시 구성(4장).
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `hashlib`·`struct`·`math` 만 사용. 노트북 NB04.

각주

1. Ralph C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in C. Pomerance (ed.), *Advances in Cryptology — CRYPTO '87*, LNCS 293 (Springer, 1988), 370쪽. 원문: "A one-way function F is a function that is easy to compute, but difficult to invert. Given x, computing y=F(x) is easy. Given y, determining any x such that F(x)=y is hard." 같은 쪽에서 「임의로 큰 입력을 받아 작은 고정 크기 출력을 내는 일방향 함수」를 one way hash function 이라고 부릅니다.

2. Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *Journal of Cryptology* 3(2): 99–111 (1991), 101–102쪽(§4.1 Hash).

3. NIST, *Secure Hash Standard (SHS)*, FIPS PUB 180-4 (August 2015), iv쪽(Announcing 절 3. Explanation). 원문: "The hash algorithms specified in this Standard are called secure because, for a given algorithm, it is computationally infeasible 1) to find a message that corresponds to a given message digest, or 2) to find two different messages that produce the same message digest. Any change to a message will, with a very high probability, result in a different message digest." DOI 10.6028/NIST.FIPS.180-4.

4. Haber & Stornetta (1991), 106쪽(§6.2 Time Constraints). 원문: "if any amount of time may pass between the creation of a document and when it is time-stamped, then no method can do more than forward-constrain the time at which the document itself was created. Thus, in general, time-stamping should only be considered as evidence that a document has not been back-dated."

5. Merkle (1988), 370쪽. 원문: "The design of one way hash functions should be approached with caution: the most obvious approaches are sometimes vulnerable to 'square root' attacks." 이어서 112비트를 64비트로 줄이는 소박한 설계가 약 2^{28} 번의 연산으로 깨지는 예를 듭니다.

6. FIPS 180-4, Figure 1(3쪽) "Secure Hash Algorithm Properties": SHA-256 — Message Size < 2^{64} bits, Block Size 512, Word Size 32, Message Digest Size 256. 개정 예정은 NIST CSRC의 FIPS 180-4 페이지 Planning Note(2023-03-07)에 따른 것으로, 2026-10-25 접근 시점 현행본은 2015년판이었습니다.

7. 이중 SHA-256: Cyril Grunspan & Ricardo Pérez-Marco, "Double Spend Races," *International Journal of Theoretical and Applied Finance* 21(8): 1850053 (2018), §2(arXiv:1702.02867v3 기준 5쪽) — "The mining algorithm consists in performing the double SHA-256 of the block header ...", 같은 문장은 괄호로 "(doubled to prevent “padding attacks”)" 라는 이유를 덧붙이고, 각주 1은 "a double hash SHA256(SHA256(header)) is computed" 라고 적습니다. 주소의 해시: Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed. (O'Reilly, 2023), 4장 — A = RIPEMD160(SHA256(K)) (GitHub 원고 커밋 94c9ba64, `ch04_keys.adoc`). 거래 식별자가 이중 SHA-256이라는 점은 12장에서 제네시스 블록 데이터로 직접 확인합니다.

8. FIPS 180-4, §1 Introduction(3쪽). 원문: "Each algorithm can be described in two stages: preprocessing and hash computation. Preprocessing involves padding a message, parsing the padded message into m-bit blocks, and setting initialization values to be used in the hash computation. The hash computation generates a message schedule from the padded message and uses that schedule, along with functions, constants, and word operations to iteratively generate a series of hash values."

9. Haber & Stornetta (1991), 102쪽: "Damgård gave the first formal definition, and a constructive proof of their existence, on the assumption that there exist one-way 'claw-free' permutations [4]." 참고문헌 [4]는 I. Damgård, "Collision-free hash functions and public-key signature schemes," *Eurocrypt '87*, LNCS 304 (1988), 203–217. 같은 논문 참고문헌 [5]가 I. Damgård, "A design principle for hash functions," *Crypto '89*, LNCS 435 (1990), 416–427 — 압축 함수를 반복하는 설계 원리를 다룬 쪽은 이것입니다(이 책은 두 논문의 원문을 확보하지 않았습니다).

1부 기술 기초

5장 공개키 서명 — 열쇠 둘로 권한을 나누기

핵심 TAKEAWAY

비트코인의 거래와 원장은 암호화되어 있지 않습니다. 모든 거래는 공개되어 있습니다. 비트코인이 지키는 것은 비밀이 아니라 **지출 권한이고**, 그 도구가 전자서명입니다. 개인키를 가진 사람만 서명을 만들 수 있고, 공개키를 아는 누구나 그 서명을 검증할 수 있습니다. 「코인을 가진다」는 말의 기술적 뜻은 「그 코인을 다음 사람에게 넘기는 서명을 만들 수 있다」는 것뿐입니다. 이 장을 읽으면 개인키·공개키·주소의 관계를 설명하고, 서명식 $s = k^{-1}(e + r\,d)$ 가 왜 그 모양인지 손으로 따라갈 수 있습니다.

학습목표

1. 개인키·공개키·주소의 관계를 그리고, 서명이 「내용 승인」이지 「암호화」가 아님을 설명한다.
2. [중급] ECDSA 서명 생성과 검증의 입출력을 표로 쓰고, 서명마다 새 비밀수 k 가 왜 필요한지 코드로 확인한다.
3. [심화] 작은 소수 위의 장난감 타원곡선에서 점 덧셈을 손으로 한 번 하고, 실제 secp256k1 곡선의 매개변수를 표준 문서에서 읽는다.
4. 개인키를 잃는다는 것이 기술적으로 무엇을 잃는 것인지 말한다.

[입문]**만 읽은 독자가 가져가는 것.** 암호화폐의 거래와 원장은 암호화되어 있지 않습니다. 지키는 것은 비밀이 아니라 **지출 권한입니다**.

1. 이 기술은 무엇의 실패에서 나왔나

전통적인 암호는 열쇠가 하나입니다. 같은 열쇠로 잠그고 같은 열쇠로 엽니다. 그래서 두 사람이 비밀 통신을 시작하려면 **먼저 그 열쇠를 안전하게 건네야** 합니다. 그런데 열쇠를 안전하게 건널 통로가 있다면 그 통로로 편지를 보내면 되지 않나요. 열쇠 배달 문제는 암호 자체보다 어려웠습니다. 디피(Whitfield Diffie)와 헬먼(Martin Hellman)은 1976년 논문 첫 문단에서 상업적 응용이 만든 두 가지 새 요구를 적었습니다. **안전한 열쇠 배달 통로의 필요를 최소화할 것**, 그리고 **손으로 쓴 서명에 해당하는 것을 제공할 것**.[1] 공개키 암호는 이 둘에 한꺼번에 답한 발명입니다.

생각의 핵심은 열쇠를 둘로 쪼개는 것입니다. 하나는 공개하고(공개키, public key) 하나는 숨깁니다(개인키, private key). 두 열쇠는 수학적으로 짝이지만, 공개키에서 개인키를 계산해 낼 수는 없습니다. 이것이 가능하려면 「한쪽으로는 쉽고 반대쪽으로는 어려운」 연산이 필요합니다. 4장의 일방향성과 같은 뿌리입니다. 다만 해시는 아무도 되돌릴 수 없는 반면, 서명 체계에서는 **개인키를 가진 사람만** 유효한 서명을 만들 수 있고 그 밖의 누구도 만들 수 없습니다. 개인키에서 공개키로는 쉽게 가지만 공개키에서

개인키로는 돌아갈 수 없는 연산이 그 바탕입니다.

2. 직관 — 서명은 승인이고, 암호화가 아니다

전자서명은 다음 세 동작으로 이루어집니다.

동작	누가	무엇을 넣어	무엇을 얻나
열쇠 생성	소유자	난수	개인키 d 와 공개키 Q 의 짝
서명	개인키를 가진 사람만	메시지 M, 개인키 d	서명 (r, s)
검증	공개키를 아는 누구나	메시지 M, 서명 (r, s), 공개키 Q	참 또는 거짓

메시지는 그대로 드러나 있습니다. 서명은 메시지를 감추는 것이 아니라, 「이 메시지를 개인키의 주인이 승인했다」는 증거를 **덧붙이는** 것입니다. 메시지의 비트 하나가 바뀌면 검증이 실패합니다. 그러니 서명은 두 가지를 한 번에 보증합니다. 누가 승인했는가(인증), 그리고 승인한 뒤 내용이 바뀌지 않았는가(무결성).

나카모토 백서는 코인을 이 서명의 사슬로 정의합니다. 「우리는 전자 코인을 디지털 서명의 사슬로 정의한다. 각 소유자는 직전 거래의 해시와 다음 소유자의 공개키를 전자서명하고 이를 코인 끝에 덧붙여 코인을 다음 사람에게 넘긴다. 수취인은 서명들을 검증해 소유권의 사슬을 확인할 수 있다.」[2] 코인은 물건이 아니라 **승인의 기록입니다**. 「코인을 가진다」는 것은 그 기록의 마지막 칸에 적힌 공개키에 짝이 되는 개인키를 가져서, 다음 칸을 쓸 수 있다는 뜻입니다.

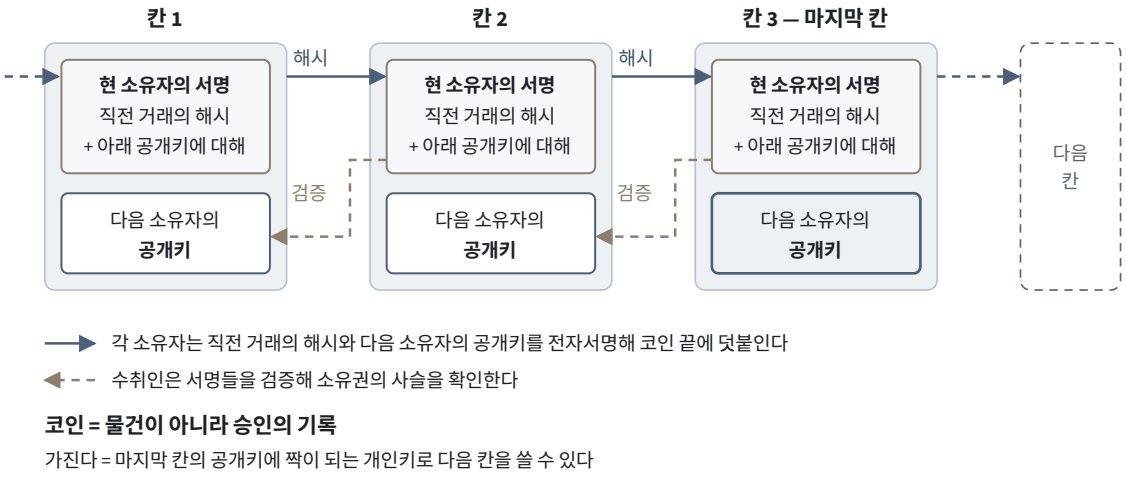


그림 5-1. 나카모토 백서 §2의 정의를 그 문장의 낱말로 새로 그렸다. 각 칸은 직전 거래의 해시와 다음 소유자의 공개키에 대한 현 소유자의 서명이고, 코인은 이 승인의 기록이다.

공개키에서 주소로

비트코인 「주소」는 공개키 그 자체가 아니라 공개키의 해시입니다. 공개키 K 를 SHA-256으로 해시하고 그 결과를 다시 RIPEMD-160이라는 160비트 해시로 줄입니다. A = RIPEMD160(SHA256(K)) 입니다.[3] 거래의 출력에는 이 20바이트가 적히고, 그 돈을 쓰려는 사람은 해시의 원본인 공개키와 서명을 함께 내놓아야 합니다. 그래서 이런 해시형 출력(P2PKH·P2WPKH)으로 돈을 받기만 한 주소는 공개키를 아직 드러내지 않습니다. 출력 유형이 다르면 사정이 다릅니다. 12장의 제네시스 출력(P2PK)에는 공개키가 그대로 적혀 있고, 아래 탭루트 출력(P2TR)도 공개키 하나를 바로 담습니다. 이 구별이 왜 중요한지는 29장에서 다시 나옵니다. 양자 컴퓨터가 위협하는 것은 공개키가 **드러난** 코인이기 때문입니다.

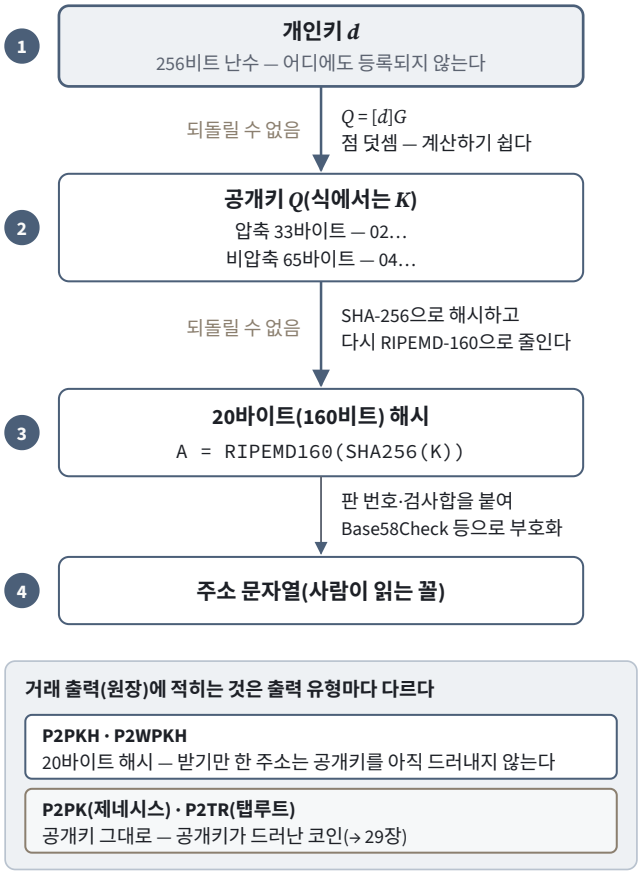


그림 5-2. 개인키에서 공개키로, 공개키에서 20바이트 해시로 가는 길은 한쪽으로만 간다. 마지막 부호화는 해시가 아니라 표기 방식이며, 거래 출력에 무엇이 적히는지는 출력 유형마다 다르다.

백서는 거래마다 새 열쇠 짝을 쓰라고 권합니다. 「추가 방화벽으로, 거래마다 새 열쇠 짝을 써서 거래들이 공통 소유자에게 연결되지 않게 해야 한다.」[4] 「지갑」이라고 부르는 소프트웨어가 하는 일의 대부분은 이 열쇠 짝들을 만들고 보관하는 것입니다. 이 책은 어느 지갑 제품도 이름을 들지 않습니다.

개인키를 잃으면

개인키는 큰 수 하나입니다. 어디에도 등록되어 있지 않고, 잊으면 되찾아 줄 사람이 없습니다. 개인키를 잃은 사람의 코인은 원장에 그대로 남아 있습니다. 누구도 그것을 쓸 수 없을 뿐입니다. 반대로 개인키가 유출되면, 유출된 키를 가진 누구나 그 코인을 쓰는 유효한 서명을 만들 수 있습니다. 원장은 서명의 유효성만 보고 사람을 보지 않습니다.

> **? 법률가의 질문**
>
> 개인키를 잃은 사람은 무엇을 잃은 것입니까. 코인은 원장에 그대로 적혀 있고, 그 사람의 주소에 있습니다. 잃은 것은 점유인가요, 소유인가요, 아니면 「접근 가능성」이라는 제3의 무엇인가요. 개인키를 훔친 사람이 코인을 옮기면 그는 무엇을 「취득」한 것입니까. 서명의 유효성만 보는 원장 위에서 「정당한 권리자」라는 개념은 어디에 자리합니까. → 27장

3. [중급] 서명식을 읽는다 — ECDSA

[중급] 비트코인이 처음부터 쓴 서명 방식은 ECDSA(Elliptic Curve Digital Signature Algorithm)입니다. 타원곡선 위에서 돌아가는 서명 알고리듬이며, 곡선은 secp256k1이라는 이름의 것을 씁니다. BIP-340 문서는 이 사실을 두 문장으로 적습니다. 「비트코인은 전통적으로 거래를 인증하기 위해 secp256k1 곡선 위의 ECDSA 서명과 SHA256 해시를 써 왔다. 이것들은 표준화되어 있다.」[5] 이 문장에서 「표준화」에 걸린 문서가 SEC 1(서명 알고리듬)이고, 「secp256k1 곡선」에 걸린 문서가 SEC 2(곡선 매개변수)입니다. 미국 연방 표준 FIPS 186-5도 같은 ECDSA 알고리듬을 정하지만, 곡선의 선택은 별도 문서(NIST SP 800-186)에 맡기며, secp256k1은 SEC 2가 정한 곡선입니다. 아래에서는 알고리듬의 단계를 FIPS 186-5의 서술로 읽습니다.

[중급] 기호를 먼저 정합니다. 곡선 위에 기준점 G가 있고, G를 정수 k번 「더한」 점을 $[k]G$라고 씁니다. G의 위수(order) n은 $[n]G$가 다시 「영점」이 되는 가장 작은 양수입니다. 개인키 d는 $1 \le d \le n-1$의 정수이고 공개키는 $Q = [d]G$입니다. d에서 Q를 계산하기는 쉽고(점 덧셈 수백 번), Q에서 d를 알아내는 것은 **타원곡선 이산대수 문제라고** 불리는 어려운 문제입니다. 이것이 이 서명의 일방향성입니다.

서명 생성 (FIPS 186-5 §6.4.1).[6]

단계	하는 일	수식
1	메시지를 해시한다	$H = \mathrm{Hash}(M)$
2	해시를 정수로 바꾼다	e — n의 비트 길이가 해시 길이 이상이면 H 전체, 아니면 H의 앞 $\log_2 n$ 비트(secp256k1과 SHA-256은 둘 다 256비트라 전체)
3	**메시지마다 새로운** 비밀수를 뽑는다	$k, 0 < k < n$
4	곡선 위의 점을 하나 만든다	$R = [k]G, r = x_R \bmod n$
5	서명의 둘째 성분을 계산한다	$s = k^{-1}(e + r\,d) \bmod n$
6	k와 k^{-1}을 즉시 파기한다	—
7	서명 출력	(r, s)

기호 풀이: d는 개인키, e는 메시지 해시를 정수로 본 것, k는 이번 서명에만 쓰는 일회용 비밀수, r은 $[k]G$의 x 좌표를 n으로 나눈 나머지, s는 그 셋을 묶은 값입니다. 수치 예는 아래 코드 NB05-2의 장난감 곡선($n = 79$)에서 봅니다.

서명 검증 (FIPS 186-5 §6.4.2).[7] 검증자는 d 도 k 도 모릅니다. 가진 것은 M, (r, s), Q 뿐입니다.

단계	하는 일	수식
1	r, s 가 $[1, n-1]$ 안인지 확인	—
2	메시지를 같은 방법으로 해시해 e 를 얻는다	e
3	두 계수를 만든다	$u = e\,s^{-1} \bmod n,\; v = r\,s^{-1} \bmod n$
4	점을 다시 만든다	$R_1 = [u]G + [v]Q$
5	그 점의 x 좌표가 r 과 같은지 본다	$r \stackrel{?}{=} x_{R_1} \bmod n$

[중급] 왜 이게 맞아 들어갈까요. $Q = [d]G$ 이므로 $R_1 = [u]G + [v\,d]G = [(e + r\,d)\,s^{-1}]G$ 입니다. 서명 생성에서 $s = k^{-1}(e + r\,d)$ 였으니 $(e + r\,d)\,s^{-1} = k$ 입니다. 따라서 $R_1 = [k]G = R$ 이고 x 좌표가 r 과 같습니다. 서명자만 아는 k 와 d 가 식 안에서 서로 지워지고, 검증자는 그 둘을 모른 채 등식만 확인합니다. 이것이 공개키 서명의 요체입니다.

비밀수 k 를 두 번 쓰면 개인키가 드러난다

[중급] 표준이 「k 는 서명마다 새로 만들고 보호하라」고 못 박는 이유는 식에서 바로 나옵니다. 같은 k 로 두 메시지 e_1, e_2 에 서명하면 r 이 같고, $s_1 - s_2 = k^{-1}(e_1 - e_2)$ 이므로 $k = (e_1 - e_2)(s_1 - s_2)^{-1}$ 이 공개 정보만으로 계산됩니다. k 를 알면 $d = (s_1 k - e_1)\,r^{-1}$ 로 개인키가 나옵니다. 아래 NB05-3이 장난감 곡선에서 이것을 실제로 해 보입니다. **서명의 안전은 개인키의 비밀만이 아니라 서명마다 새 난수의 품질에도 걸려 있습니다.**

ECDSA 서명은 두 모양을 가진다 — 가변성

[중급] ECDSA에는 또 하나의 성질이 있습니다. (r, s) 가 유효한 서명이면 $(r, n - s)$ 도 같은 메시지·같은 공개키에 대해 유효합니다. BIP-340은 이를 「ECDSA 서명은 본질적으로 가변적(malleable)이다. 비밀키 없이도 제3자가 주어진 공개키·메시지에 대한 유효한 서명을 같은 키·메시지에 유효한 **다른** 서명으로 바꿀 수 있다」고 적고, 비트코인이 네트워크 정책으로 둘 중 하나만 허용해 왔다고 덧붙입니다.[8] 서명이 바뀌면 그 서명을 담은 거래의 식별자(txid)도 바뀝니다. 「거래 가변성(transaction malleability)」이라고 불리는 문제의 한 갈래가 여기서 나오며, 26장의 Mt.Gox 서술에서 이 낱말을 다시 만납니다.

슈노르 서명과 탭루트

[중급] 2021년에 활성화된 탭루트(Taproot) 업그레이드로 비트코인에는 두 번째 서명 방식이 들어왔습니다. BIP-340이 정한 **슈노르(Schnorr) 서명입니다.** 64바이트 고정 길이이고, BIP-340은 ECDSA에 비해 세 가지 장점을 듭니다. 증명 가능한 안전성, 비가변성, 그리고 선형성(여러 참여자의 서명을 합쳐 공개키 합에 대한 하나의 서명으로 만들 수 있음)입니다.[9] BIP-341은 이 서명과 머클 가지를 쓰는 새 출력 유형(SegWit 버전 1)을 정했습니다.[10] 두 문서 모두 2026-10-25 현재 상태가 「Deployed」입니다. Bitcoin Core 소스는 탭루트 소프트포크의 활성화 매개변수로 신호 기간 2021-04-24~2021-08-11, 최소 활성화 높이 709,632를 적어 두었습니다.[11] ECDSA가 사라진 것은 아닙니다. 두 방식이 나란히 쓰입니다.

4. 코드 — 장난감 곡선 위에서 전부 해 본다

[중급] 실제 secp256k1의 수는 78자리라 손으로 따라가기 어렵습니다. 그래서 먼저 소수 $p = 97$ 위의 **장난감 곡선** $y^2 = x^3 + 7 \pmod{97}$ 에서 똑같은 연산을 합니다. 식의 모양($x^3 + 7$)은 secp256k1과 같고 크기만 작습니다. 아래 코드 전부는 **공부용이며 실사용 금지입니다.** 이 곡선의 점은 79개밖에 없어 개인키를 전수 탐색으로 1초 안에 찾을 수 있습니다.

▾ [심화] **타원곡선이란 — 점을 「더한다」는 말의 뜻**

[심화] 코블리츠(Neal Koblitz)는 1987년 논문에서 타원곡선을 이렇게 정의합니다. 표수가 2나 3이 아닌 체 K 위의 타원곡선 E_K 는 방정식 $y^2 = x^3 + ax + b$ ($a, b \in K$, 우변 삼차식에 중근 없음)의 해 $(x, y) \in K^2$ 의 집합에 「무한원점」을 하나 더한 것입니다.[1][2] 유한체 위에서는 이 「곡선」이 곡선처럼 보이지 않고 점의 흩어진 집합입니다. 그래도 두 점을 「더해」 셋째 점을 얻는 규칙이 있고, 그 규칙은 결합법칙·교환법칙·역원을 만족해 **군(group)을** 이룹니다. 무한원점 $\mathcal{O}$ 가 영(항등원)입니다.

[심화] 덧셈 규칙(소수체 $\mathbb{F}_p$ 위, 좌표는 모두 $\bmod p$):

- $P = (x_1, y_1)$, $Q = (x_2, y_2)$, $P \neq \pm Q$ 이면 기울기 $\lambda = (y_2 - y_1)(x_2 - x_1)^{-1}$, $x_3 = \lambda^2 - x_1 - x_2$, $y_3 = \lambda(x_1 - x_3) - y_1$.
- $P = Q$ (두 배)이면 $\lambda = (3x_1^2 + a)(2y_1)^{-1}$, 나머지는 같다.
- $Q = -P = (x_1, -y_1)$ 이면 $P + Q = \mathcal{O}$.

기호 풀이: $(\cdot)^{-1}$ 는 p 를 법으로 한 곱셈 역원(페르마 소정리로 $a^{p-2} \bmod p$), a 는 곡선 식의 일차항 계수(secp256k1과 장난감 곡선 모두 $a = 0$). 수치 예: 장난감 곡선에서 $G = (1, 28)$ 을 두 배 하면 $\lambda = 3 \cdot 1 \cdot (56)^{-1} \bmod 97$ 로 시작합니다. 아래 코드가 전부 계산합니다.

[심화] 스칼라 곱 $[k]G$ 는 G 를 k 번 더한 것인데, k 가 78자리면 k 번 더할 수 없습니다. 대신 k 를 이진수로 펼쳐 「두 배하고, 비트가 1이면 더하기」를 256번 하면 끝납니다. 계산은 이렇게 쉽습니다. 반대로 $[k]G$ 에서 k 를 찾는 빠른 방법은 알려져 있지 않습니다(고전 컴퓨터에서는 — 양자 컴퓨터는 29장).

```
# NB05-1: 장난감 타원곡선 y^2 = x^3 + 7 (mod 97) — 점 덧셈, 두 배, 스칼라 곱 (공부용, 실사용 금지)
p, a, b = 97, 0, 7          # secp256k1과 같은 모양(a=0, b=7), 크기만 작은 곡선
O = None                     # 무한원점(항등원)

def on_curve(P):
    if P is O: return True
    x, y = P
    return (y * y - (x ** 3 + a * x + b)) % p == 0

def inv(z): return pow(z, p - 2, p)              # 곱셈 역원 (페르마 소정리)

def add(P, Q):
    if P is O: return Q
    if Q is O: return P
    (x1, y1), (x2, y2) = P, Q
    if x1 == x2 and (y1 + y2) % p == 0: return O   # P + (-P) = O
    if P == Q: lam = (3 * x1 * x1 + a) * inv(2 * y1) % p
    else:      lam = (y2 - y1) * inv(x2 - x1) % p
    x3 = (lam * lam - x1 - x2) % p
    y3 = (lam * (x1 - x3) - y1) % p
    return (x3, y3)

def mul(k, P):                                    # 두 배하고-더하기 (double-and-add)
    R = O
    while k:
        if k & 1: R = add(R, P)
        P = add(P, P)
        k >>= 1
    return R

points = [(x, y) for x in range(p) for y in range(p) if on_curve((x, y))]
print("곡선 위의 점 개수(무한원점 포함) n =", len(points) + 1)
G = min(points)                                   # x가 가장 작은 점을 기준점으로
print("기준점 G =", G)
print("2G =", add(G, G), " 3G =", mul(3, G), " 79G =", mul(79, G), "(= 무한원점이면 None)")
d = 23                                            # 장난감 개인키 — 실사용 금지
Q = mul(d, G)
print("개인키 d = %d → 공개키 Q = [d]G = %s" % (d, Q))
found = next(k for k in range(1, 79) if mul(k, G) == Q)
print("전수 탐색으로 Q에서 d를 되찾음: d =", found, "(79개뿐이라 즉시 — 실제 곡선은 2^256 규모)")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
곡선 위의 점 개수(무한원점 포함) n = 79
기준점 G = (1, 28)
2G = (68, 81)  3G = (53, 38)  79G = None (= 무한원점이면 None)
개인키 d = 23 → 공개키 Q = [d]G = (61, 28)
전수 탐색으로 Q에서 d를 되찾음: d = 23 (79개뿐이라 즉시 — 실제 곡선은 2^256 규모)
```

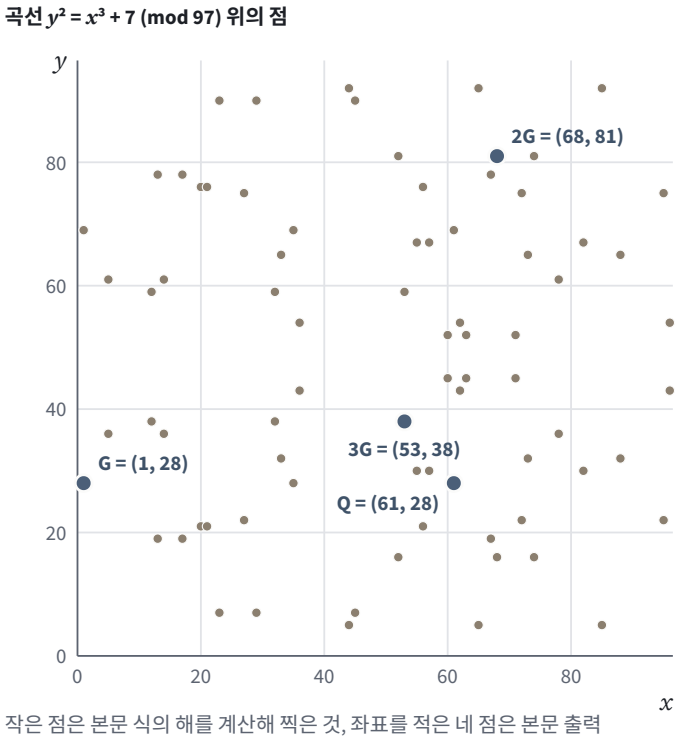


그림 5-3. 장난감 곡선 $y^2 = x^3 + 7 \pmod{97}$ 위의 점. 점은 본문 식의 해를 계산해 찍은 것이고, 라벨을 단 G·2G·3G·Q의 좌표와 점 개수 79(무한원점 포함)는 NB05-1 출력이다. 유한체 위에서 「곡선」은 흩어진 점의 집합이다.

[중급] 위수 79가 소수이므로 무한원점을 뺀 모든 점이 생성원이고 $[79]G = \mathcal{O}$ 입니다. 같은 코드가 숫자만 바꾸면 실제 곡선에서도 돕니다(NB05-4). 이제 이 군 위에서 ECDSA를 그대로 구현합니다.

```
# NB05-2: 장난감 ECDSA — 서명 생성·검증, 한 비트 바꾸면 실패, (r, n-s)도 유효 (공부용, 실사용 금지)
import hashlib
n = 79                                                  # 기준점 G의 위수

def inv_n(z): return pow(z, n - 2, n)

def msg_int(M: bytes) -> int:
    """메시지 해시를 정수로. n이 작으니 SHA-256 결과를 n으로 줄인다(설명용 단순화)."""
    return int.from_bytes(hashlib.sha256(M).digest(), "big") % n

def sign(d, M, k):
    e = msg_int(M)
    R = mul(k, G)
    r = R[0] % n
    s = inv_n(k) * (e + r * d) % n
    assert r != 0 and s != 0
    return (r, s)

def verify(Q, M, sig):
    r, s = sig
    if not (1 <= r <= n - 1 and 1 <= s <= n - 1): return False
    e = msg_int(M)
    u = e * inv_n(s) % n
    v = r * inv_n(s) % n
    R1 = add(mul(u, G), mul(v, Q))
    return R1 is not O and R1[0] % n == r

M = b"Pay 1 coin to Bob"
k = 17                                                  # 일회용 비밀수 — 설명을 위해 고정값(실제로는 안전한
난수)
sig = sign(d, M, k)
print("메시지:", M, "| 서명 (r, s) =", sig)
print("정상 검증             :", verify(Q, M, sig))
print("메시지 한 글자 변경    :", verify(Q, b"Pay 2 coin to Bob", sig))
print("다른 사람의 공개키로   :", verify(mul(24, G), M, sig))
r, s = sig
print("(r, n-s) =", (r, (n - s) % n), "도 유효한가:", verify(Q, M, (r, (n - s) % n)), " ← ECDSA
가변성")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
메시지: b'Pay 1 coin to Bob' | 서명 (r, s) = (17, 45)
정상 검증             : True
메시지 한 글자 변경    : False
다른 사람의 공개키로   : False
(r, n-s) = (17, 34) 도 유효한가: True  ← ECDSA 가변성
```

```
# NB05-3: 비밀수 k를 두 번 쓰면 — 공개된 서명 둘로 개인키를 계산한다 (공부용)
M1, M2 = b"Pay 1 coin to Bob", b"Pay 5 coins to Carol"
k_reused = 31
(r1, s1), (r2, s2) = sign(d, M1, k_reused), sign(d, M2, k_reused)
e1, e2 = msg_int(M1), msg_int(M2)
print("두 서명의 r이 같다:", r1 == r2, "(r =", r1, ")")
k_rec = (e1 - e2) * inv_n(s1 - s2) % n
d_rec = (s1 * k_rec - e1) * inv_n(r1) % n
print("되찾은 k =", k_rec, "| 되찾은 개인키 d =", d_rec, "| 실제 d =", d)
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
두 서명의 r이 같다: True (r = 52 )
되찾은 k = 31 | 되찾은 개인키 d = 23 | 실제 d = 23
```

[중급] 두 서명만으로 개인키가 복원되었습니다. 식이 그렇게 생겼기 때문이지, 곡선이 작아서가 아닙니다. 실제 곡선에서도 똑같이 됩니다.

실제 곡선 — secp256k1

[중급] 실제 비트코인 곡선의 매개변수는 SEC 2 문서 §2.4.1에 적혀 있습니다.[13] 소수 $p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$, 곡선 $y^2 = x^3 + 7$, 기준점 G 의 압축 표현 `02 79BE667E …`, 위수 n 은 `FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFE BAAEDCE6 AF48A03B BFD25E8C D0364141`, 여인수 $h = 1$. 아래 코드는 NB05-1의 함수에 이 수를 넣어 세 가지를 확인합니다. G 가 곡선 위에 있는가, $[n]G = \mathcal{O}$ 인가, 그리고 압축 표현 `02…` 에서 y 를 복원하면 비압축 표현 `04…` 와 같은가. 외부 패키지는 쓰지 않습니다. 수가 커졌을 뿐 연산은 같습니다.

```
# NB05-4: 실제 secp256k1 매개변수(SEC 2 §2.4.1) — 표준 라이브러리만으로 확인 (공부용, 실사용 금지)
import hashlib
p = 2**256 - 2**32 - 2**9 - 2**8 - 2**7 - 2**6 - 2**4 - 1
a, b = 0, 7
n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
G = (Gx, Gy)
print("p 는 16진수로", hex(p).upper()[2:])
print("G 가 곡선 위에 있는가:", on_curve(G))
print("[n]G 가 무한원점인가 :", mul(n, G) is O)
# 압축 표현 02||x 에서 y 복원: p ≡ 3 (mod 4) 이므로 y = (x^3+7)^((p+1)/4). 접두 02 는 y 가 짝수라는 뜻.
y_cand = pow((Gx**3 + 7) % p, (p + 1) // 4, p)
y_even = y_cand if y_cand % 2 == 0 else p - y_cand
print("압축 표현에서 복원한 y == 비압축 표현의 y :", y_even == Gy)

d_toy = 1                                          # 공부용 개인키 1 → 공개키는 G 자신. 실제 키 생성이
아니다.
Qx, Qy = mul(d_toy, G)
compressed = bytes([2 + (Qy & 1)]) + Qx.to_bytes(32, "big")
print("d=1 의 압축 공개키:", compressed.hex())
h160 = hashlib.new("ripemd160", hashlib.sha256(compressed).digest()).digest()
print("RIPEMD160(SHA256(K)) 20바이트:", h160.hex())
print("비압축 65바이트 공개키의 첫 바이트는 0x04 — 12장 제네시스 출력의 공개키가 이 꼴이다.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
p 는 16진수로 FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F
G 가 곡선 위에 있는가: True
[n]G 가 무한원점인가 : True
압축 표현에서 복원한 y == 비압축 표현의 y : True
d=1 의 압축 공개키: 0279be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798
RIPEMD160(SHA256(K)) 20바이트: 751e76e8199196d454941c45d1b3a323f1433bd6
비압축 65바이트 공개키의 첫 바이트는 0x04 — 12장 제네시스 출력의 공개키가 이 꼴이다.
```

[중급] 표준 문서의 숫자가 코드 안에서 서로 맞아 들어갑니다. 압축 공개키 33바이트에 두 번의 해시를 걸어 20바이트 주소 재료를 얻는 과정도 그대로 보였습니다. 개인키 1은 공부용입니다. 실제 지갑은 256비트 난수를 개인키로 쓰며, 이 책은 실제 키를 만들거나 보관하는 코드를 싣지 않습니다.

5. 정리

이 장의 요지는 세 줄입니다. 비트코인은 암호화하지 않고 서명합니다. 코인은 물건이 아니라 서명의 사슬이며, 「가진다」는 것은 「다음 서명을 만들 수 있다」는 뜻입니다. 그리고 서명의 안전은 개인키의 비밀과, 서명마다 새로 뽑는 비밀수의 품질에 걸려 있습니다. 다음 6장은 서명된 거래 수천 건을 해시 하나로 묶는 장치를 봅니다.

이 장의 출처

- Whitfield Diffie & Martin E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory* IT-22(6): 644–654, November 1976. DOI 10.1109/TIT.1976.1055638 — 열쇠 배달과 서명의 요구(644쪽).
- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. https://bitcoin.org/bitcoin.pdf — 코인 = 서명의 사슬(§2, 2쪽), 거래마다 새 열쇠(§10, 6쪽).
- NIST, *Digital Signature Standard (DSS)*, FIPS 186-5, February 2023. DOI 10.6028/NIST.FIPS.186-5 — ECDSA 서명 생성 §6.4.1(24쪽)·검증 §6.4.2(25쪽), *k* 보호 요구(24쪽).
- Certicom Research, *SEC 2: Recommended Elliptic Curve Domain Parameters*, Version 2.0, January 27, 2010. https://www.secg.org/sec2-v2.pdf — secp256k1 매개변수(§2.4.1, 9쪽).
- Neal Koblitz, "Elliptic Curve Cryptosystems," *Mathematics of Computation* 48(177): 203–209, 1987. DOI 10.1090/S0025-5718-1987-0866109-5; Victor S. Miller, "Use of Elliptic Curves in Cryptography," *CRYPTO '85*, LNCS 218, pp. 417–426, Springer, 1986. DOI 10.1007/3-540-39799-X_31 — 타원곡선 암호의 원전.
- Pieter Wuille, Jonas Nick & Tim Ruffing, "Schnorr Signatures for secp256k1," BIP 340; Pieter Wuille, Jonas Nick & Anthony Towns, "Taproot: SegWit version 1 spending rules," BIP 341. https://github.com/bitcoin/bips (상태 Deployed, 2026-10-25 접근) — ECDSA 가변성, 슈노르 서명의 장점, 탭루트 출력.
- Bitcoin Core v31.1, `src/kernel/chainparams.cpp` (커밋 9be056a8a72b624dae9623b2f7bded92c2a21c91) — 탭루트 활성화 매개변수.
- Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed., O'Reilly, 2023 (CC BY-SA 4.0, GitHub 원고 커밋 94c9ba64) — 주소 = RIPEMD160(SHA256(K))(4장).
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `hashlib` 만 사용. 노트북 NB05.

각주

1. Whitfield Diffie & Martin E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory* IT-22(6): 644–654 (November 1976), 644쪽. 원문: "such applications create a need for new types of cryptographic systems which minimize the necessity of secure key distribution channels and supply the equivalent of a written signature."

2. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), 2쪽(§2 Transactions). 원문: "We define an electronic coin as a chain of digital signatures. Each owner transfers the coin to the next by digitally signing a hash of the previous transaction and the public key of the next owner and adding these to the end of the coin. A payee can verify the signatures to verify the chain of ownership."

3. Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed. (O'Reilly, 2023), 4장 — "Starting with the public key K, we compute the SHA256 hash and then compute the RIPEMD-160 hash of the result, producing a 160-bit (20-byte) number: A = RIPEMD160(SHA256(K))". GitHub 원고 커밋 94c9ba64, `ch04_keys.adoc`. 사람이 읽는 주소 문자열은 이 20바이트에 판 번호와 검사합을 붙여 Base58Check 등으로 부호화한 것입니다(같은 장).

4. Nakamoto (2008), 6쪽(§10 Privacy). 원문: "As an additional firewall, a new key pair should be used for each transaction to keep them from being linked to a common owner."

5. Pieter Wuille, Jonas Nick & Tim Ruffing, "Schnorr Signatures for secp256k1," BIP 340, Motivation 절. 원문: "Bitcoin has traditionally used ECDSA signatures over the secp256k1 curve with SHA256 hashes for authenticating transactions. These are standardized, but have a number of downsides compared to Schnorr signatures over the same curve". github.com/bitcoin/bips, 파일 커밋 2885f13d(2026-01-12), 2026-10-25 접근. 「standardized」에 걸린 링크는 secg.org의 SEC 1 v2.0이고, 「secp256k1 curve」에 걸린 링크가 SEC 2 v2.0입니다. FIPS 186-5의 곡선 위임: "NIST-recommended curves for ECDSA are provided in SP 800-186"(FIPS 186-5 본문). 이 책은 SP 800-186의 곡선 목록을 대조하지 않았습니다.

6. NIST, *Digital Signature Standard (DSS)*, FIPS 186-5 (February 3, 2023), §6.4.1 "ECDSA Signature Generation Algorithm", 24쪽. 단계 9 원문: "Compute s = k^-1 · (e + r · d) mod n." 단계 3은 「per-message secret number k, 0 < k < n」. 같은 쪽: "A value k shall be generated at each invocation of the signature generation algorithm. The private key d and the per-message secret numbers k and k^-1 shall be protected from unauthorized disclosure and modification." DOI 10.6028/NIST.FIPS.186-5. 단계 2 원문: "If len(n) ≥ hashlen, set E = H. Otherwise, set E equal to the leftmost log2(n) bits of H." 표에서 세부 단계(정수 변환·실패 처리)는 생략했고 번호를 다시 매겼습니다.

7. FIPS 186-5, §6.4.2 "ECDSA Signature Verification Algorithm", 25쪽. 단계 5·6 원문: "Compute u = e · s^-1 mod n and v = r · s^-1 mod n." "Compute R1 = [u]G + [v]Q."

8. BIP 340, Motivation 절 Non-malleability 항목과 그 각주. 원문: "ECDSA signatures are inherently malleable; a third party without access to the secret key can alter an existing valid signature for a given public key and message into another signature that is valid for the same key and message." 각주: "If (r,s) is a valid ECDSA signature for a given message and key, then (r,n-s) is also valid for the same message and key. If ECDSA is restricted to only permit one of the two variants (as Bitcoin does through a policy rule on the network), it can be proven non-malleable under stronger than usual assumptions."

9. BIP 340, Abstract("This document proposes a standard for 64-byte Schnorr signatures over the elliptic curve secp256k1.")와 Motivation 절의 Provable security·Non-malleability·Linearity 항목. 헤더 Status: Deployed.

10. Pieter Wuille, Jonas Nick & Anthony Towns, "Taproot: SegWit version 1 spending rules," BIP 341, Abstract: "This document proposes a new SegWit version 1 output type, with spending rules based on Taproot, Schnorr signatures, and Merkle branches." Status: Deployed. github.com/bitcoin/bips, 파일 커밋 24e96e87, 2026-10-25 접근.

11. Bitcoin Core v31.1, `src/kernel/chainparams.cpp`(커밋 9be056a8), 메인넷 매개변수: `DEPLOYMENT_TAPROOT` nStartTime 1619222400 ("April 24th, 2021"), nTimeout 1628640000 ("August 11th, 2021"), min_activation_height 709632 ("Approximately November 12th, 2021"). 2026-10-25 접근.

12. Neal Koblitz, "Elliptic Curve Cryptosystems," *Mathematics of Computation* 48(177): 203–209 (January 1987), 203쪽(§2 Elliptic Curves). 코블리츠는 같은 쪽 서론 끝 감사의 말에서 밀러(V. S. Miller)가 독립적으로 「비슷한 착상(some similar ideas)」에 이르렀다고 적습니다. 밀러의 논문: Victor S. Miller, "Use of Elliptic Curves in Cryptography," *Advances in Cryptology — CRYPTO '85*, LNCS 218, pp. 417–426 (Springer, 1986). 밀러는 디피–헬먼 키 교환의 타원곡선 유사체를, 코블리츠는 유한체 곱셈군 위의 공개키 체계(Massey–Omura·ElGamal)의 타원곡선 유사체를 제안했습니다(각 논문 초록).

13. Certicom Research, *SEC 2: Recommended Elliptic Curve Domain Parameters*, Version 2.0 (Standards for Efficient Cryptography Group, January 27, 2010), §2.4.1 "Recommended Parameters secp256k1", 9쪽. p, $a = 0$, $b = 7$, G (압축·비압축), n, $h = 1$ 의 16진수 값. https://www.secg.org/sec2-v2.pdf, 2026-10-25 접근. 문서는 「Standards for Efficient Cryptography 2 (SEC 2)」로 출처를 표시하는 조건으로 복제를 허용합니다.

1부 기술 기초

6장 머클 트리 — 하나의 뿌리로 전체를 보증하기

핵심 TAKEAWAY

거래 수천 건을 둘씩 짝지어 해시하고, 그 해시들을 다시 둘씩 짝지어 해시하기를 반복하면 마지막에 해시 하나가 남습니다. 이 「뿌리」 하나가 거래 전체를 대신합니다. 어느 거래든 한 글자가 바뀌면 뿌리가 바뀝니다. 그리고 거래 하나가 그 안에 들어 있다는 것을 보이는 데에는 전체가 아니라 **해시 $\lceil \log_2 n \rceil$ 개만** 있으면 됩니다. 거래 4,000건이면 12개입니다. 블록 헤더 80바이트가 거래 전체를 대표할 수 있는 이유, 그리고 전체 장부를 내려받지 않은 가벼운 지갑이 「내 거래가 그 블록에 있다」를 확인할 수 있는 이유가 이 구조입니다.

학습목표

1. 거래 묶음을 뿌리 해시 하나로 요약하는 구조를 그린다.
2. [중급] 포함 증명(머클 증명)이 왜 $\lceil \log_2 n \rceil$ 개의 해시로 끝나는지 설명하고, 코드로 증명을 만들고 검증한다.
3. 비트코인이 쓰는 머클 트리의 두 가지 구현 특성(이중 SHA-256, 홀수 개일 때 마지막 복제)과 그로 인해 생긴 약점을 안다.
4. 「해시 하나로 전체를 보증한다」는 말이 무엇을 보증하고 무엇을 보증하지 않는지 가른다.

[입문]**만 읽은 독자가 가져가는 것.** 전체를 내려받지 않고도 「내 거래가 그 블록에 있다」를 확인할 수 있는 장치가 머클 트리입니다. 다만 그 확인은 「블록에 들어 있다」까지이고, 「그 블록이 진짜 체인에 있다」는 다른 문제입니다.

1. 이 기술은 무엇의 실패에서 나왔나

4장에 나온 하버와 스토네타의 1991년 타임스탬프 방식 가운데 「연결(linking)」 방식은 문서 해시들을 **한 줄로** 이었습니다. 각 인증서가 직전 요청의 정보를 담아서, 누구도 중간에 끼워 넣거나 빼낼 수 없게 한 것입니다. 그 사슬은 7장의 주제입니다. 그런데 한 줄로 잇는 방식에는 비용 문제가 있었습니다. 어떤 문서의 시각을 확인하려는 사람은 그 문서에서 자기가 믿는 인증서까지 사슬을 **하나씩** 따라가야 합니다. 믿을 만한 인증서가 문서 N 건마다 하나씩 있다면 검증에 최대 N 단계가 걸립니다.

베이어(Dave Bayer), 하버, 스토네타는 1993년 논문에서 이 문제를 이렇게 적었습니다. 「엄청나게 많은 평범한 거래들이 자기 타임스탬프를 역사 기록의 일부로 남기고 싶어 하는데, 역사는 관심이 없다면 어떻게 할 것인가?」 그들의 답은 **토너먼트였습니다.** 수많은 눈에 띄지 않는 타임스탬프 사건들을 참가자들이 벌이는 토너먼트로 하나의 눈에 띄는 사건으로 합친다. 우승자는 쉽게 널리 공표될 수 있고, 각 참가자는 상대 선수의 짧은 명단만 기억하면 토너먼트에 참가했음을 입증할 수 있다. 연결 리스트 자리에

트리를 세워 관찰자 수를 지수적으로 늘리는 것이다.[1] 그리고 그들은 이 「해시 트리」가 머클(Ralph Merkle)이 1980년에 **다른 목적으로**, 즉 공개 암호화 키 목록의 인증서를 만들기 위해 먼저 쓴 것이라고 밝힙니다.[2] 나카모토 백서도 머클 트리의 출처로 그 1980년 논문을 인용합니다.[3] 그래서 이 책은 머클 트리의 원전을 Merkle 1980으로 둡니다. 그 논문 자체는 공개 파일을 반씩 나눠 해시하는 「트리 인증」으로 한 항목을 뿌리와 $\log_2 n$ 개의 중간값만으로 확인하는 방법을 적고, 그 확인에 필요한 정보가 놓인 뿌리에서 항목까지의 길을 「인증 경로(authentication path)」라 부르되, 자세한 논의는 다른 원고 [13]으로 넘깁니다.[4] 이 책이 1980년 논문을 원전으로 두는 것은 백서와 베이어 외가 가리킨 출처가 그것이기 때문이지, 트리가 그 논문에서 처음 제시되었기 때문이 아닙니다. 머클이 1987년에 발표한 서명 논문은 같은 착상을 「무한한 일회용 서명 트리」로 발전시킨 것입니다. 각 노드가 왼쪽·오른쪽·메시지의 세 일회용 서명을 갖고, 그 셋의 공개값을 일방향 해시로 묶은 `HASH(i)` 로 노드를 인증하며, 검증자는 어느 노드에서 부모로 거슬러 뿌리까지 올라갑니다.[5]

2. 직관 — 토너먼트 대진표

거래 8건이 있다고 합시다. 각 거래의 해시를 구합니다(8개). 둘씩 짝지어 이어 붙인 뒤 다시 해시합니다(4개). 또 둘씩 짝지어 해시합니다(2개). 마지막으로 둘을 해시합니다(1개). 그 1개가 **머클 루트(Merkle root)**, 뿌리 해시입니다. 토너먼트 대진표를 거꾸로 세운 모양이라, 거래가 선수이고 뿌리가 우승자입니다.

이 뿌리는 거래 여덟 건 전체의 지문입니다. 어느 거래든 한 글자가 바뀌면 그 거래의 해시가 바뀌고, 그것과 짝지어진 위 단계 해시가 바뀌고, 결국 뿌리가 바뀝니다. 그래서 블록 헤더에는 거래를 전부 적는 대신 **뿌리 하나만** 적습니다. 백서는 이렇게 말합니다. 「거래들은 머클 트리로 해시되고, 뿌리만 블록의 해시에 포함된다.」[6] 블록 헤더가 거래 수와 무관하게 80바이트인 이유입니다(7장).

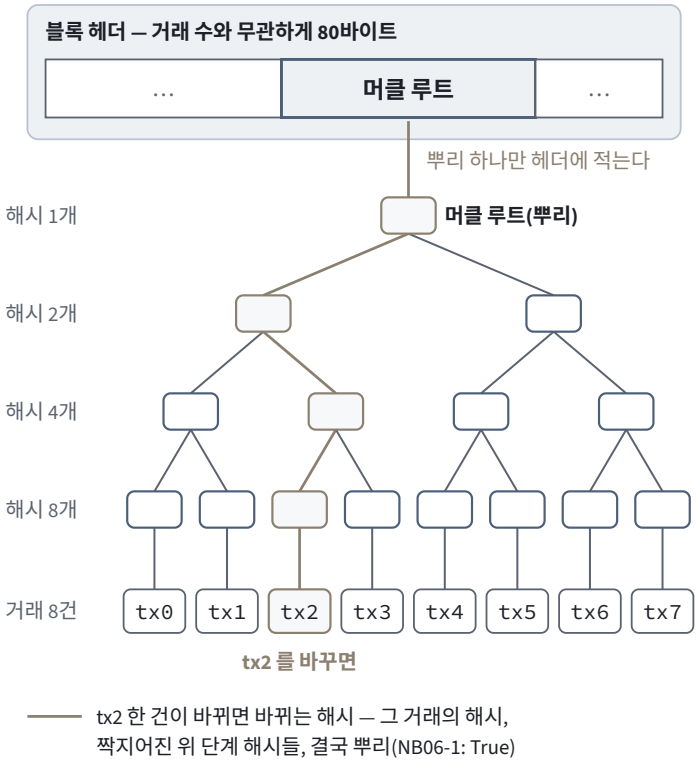


그림 6-1. 거래 8건의 머클 트리(NB06-1의 층별 해시 개수 8·4·2·1). tx2가 바뀌면 그 해시에서 뿌리까지가 모두 바뀌고, 80바이트 블록 헤더에는 그 뿌리 하나만 들어간다.

포함 증명 — 상대 선수 명단만 기억하면 된다

이제 거래 다섯 번째가 이 블록에 들어 있다는 것을 남에게 보여 주려 합니다. 거래 여덟 건을 전부 보여 줄 필요가 없습니다. 다섯 번째 거래의 해시가 뿌리까지 올라가는 길에서 **짝이 되었던 해시들만** 주면 됩니다. 여덟 건이면 세 개입니다. 받는 사람은 그 셋으로 뿌리를 다시 계산해 자기가 아는 뿌리와 같은지 봅니다. 베이어·하버·스토네타의 표현을 빌리면, 참가자는 자기 문서와 함께 「뿌리까지 가는 길에서 자기 문서의 해시값과 직접 결합된 $\lceil \log_2 N \rceil$ 개의 해시값」, 그리고 결합할 때 자기 값이 앞이었는지 뒤였는지를 나타내는 **좌우 표시를** 기록하면 되고, 검증은 「그 자료로 트리의 뿌리를 다시 계산하는 것」이 전부입니다.[7]

백서 §8의 「간이 지급 검증(Simplified Payment Verification, SPV)」이 바로 이것입니다. 전체 노드를 돌리지 않는 사용자는 블록 헤더만 보관하고, 「거래를 그 거래가 찍힌 블록에 연결하는 머클 가지(Merkle branch)」를 받아 자기 거래가 블록에 들어 있음을 확인합니다. 백서는 곧바로 한계도 적습니다. 그는 거래를 스스로 검증할 수는 없다. 블록에 연결되었음을 보고 「네트워크 노드가 그것을 받아들였다」는 것을 알 수 있을 뿐이고, 공격자가 네트워크를 압도하는 동안에는 공격자가 꾸며 낸 거래에 속을 수 있다.[8] 머클 증명은 「이 거래가 이 블록에 있다」를 보증합니다. 「이 블록이 올바른 체인에 있다」는 보증은 8장·9장의 몫입니다.

3. [중급] 증명의 길이 — 왜 $\log_2 n$ 인가

[중급] 트리의 높이는 잎의 수 n 을 둘씩 묶어 하나가 될 때까지 몇 번 묶느냐이므로 $\lceil \log_2 n \rceil$ 입니다. 포함 증명은 잎에서 뿌리까지 한 단계에 짝 해시 하나씩이므로 길이도 $\lceil \log_2 n \rceil$ 입니다.

$$\text{증명 길이} = \lceil \log_2 n \rceil \text{ 개의 해시} = 32 \lceil \log_2 n \rceil \text{ 바이트}$$

기호 풀이: n 은 블록 안 거래 수, $\lceil \cdot \rceil$ 은 올림, 32는 SHA-256 해시 한 개의 바이트 수입니다. 수치 예: $n = 4{,}000$ 이면 $\lceil \log_2 4000 \rceil = 12$, 증명은 해시 12개 384바이트입니다. 거래 4,000건을 통째로 받는 대신 384바이트로 확인이 끝납니다. 거래가 두 배로 늘어도 증명은 해시 하나만 늘어납니다.

뿌리·증명·검증

```
# NB06-1: 머클 트리 — 뿌리 계산, 포함 증명 만들기, 증명 검증 (일반형: SHA-256 한 번)
import hashlib, math

def H(b: bytes) -> bytes:
    return hashlib.sha256(b).digest()

def merkle_levels(leaves):
    """잎 해시 목록에서 각 층의 해시 목록을 아래층부터 위층 순서로 돌려준다.
    홀수 개면 마지막 것을 한 번 더 써서 짝을 만든다(비트코인과 같은 규칙 — 4절 참조)."""
    levels = [leaves]
    while len(levels[-1]) > 1:
        cur = levels[-1]
        if len(cur) % 2 == 1:
            cur = cur + [cur[-1]]
        levels.append([H(cur[i] + cur[i + 1]) for i in range(0, len(cur), 2)])
    return levels

def merkle_proof(levels, index):
    """index 번째 잎의 포함 증명: (짝 해시, 짝이 오른쪽인가) 의 목록 — 「좌우 표시」가 함께 간다."""
    proof = []
    for level in levels[:-1]:
        if len(level) % 2 == 1:
            level = level + [level[-1]]
        sibling = index ^ 1                          # 짝의 자리: 짝수면 오른쪽 이웃, 홀수면 왼쪽 이웃
        proof.append((level[sibling], sibling > index))
        index //= 2
    return proof

def verify_proof(leaf, proof, root):
    h = leaf
    for sibling, sibling_is_right in proof:
        h = H(h + sibling) if sibling_is_right else H(sibling + h)
    return h == root

txs = [f"tx{i}: Alice pays Bob {i} coin".encode() for i in range(8)]
leaves = [H(t) for t in txs]
levels = merkle_levels(leaves)
root = levels[-1][0]
print("거래 8건 → 층별 해시 개수:", [len(l) for l in levels], "→ 뿌리", root.hex()[:16] + "…")

idx = 4                                              # 다섯 번째 거래(0부터 세어 4)
proof = merkle_proof(levels, idx)
print("tx4 의 포함 증명 길이:", len(proof), "개의 해시 =", 32 * len(proof), "바이트")
print("증명 검증(원본 tx4)   :", verify_proof(leaves[idx], proof, root))
print("증명 검증(변조 tx4)   :", verify_proof(H(b"tx4: Alice pays Bob 40 coin"), proof, root))

tampered = leaves[:]; tampered[2] = H(b"tx2: Alice pays Bob 200 coin")
print("tx2 를 바꾸면 뿌리가 바뀌는가:", merkle_levels(tampered)[-1][0] != root)
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
거래 8건 → 층별 해시 개수: [8, 4, 2, 1] → 뿌리 23db046cc041f456…
tx4 의 포함 증명 길이: 3 개의 해시 = 96 바이트
증명 검증(원본 tx4)   : True
증명 검증(변조 tx4)   : False
tx2 를 바꾸면 뿌리가 바뀌는가: True
```



거래 수 n	사슬 따라가기(최대 단계)	머클 증명(해시 개수)	증명 바이트
1	1	0	0
2	2	1	32
8	8	3	96
100	100	7	224
1,000	1,000	10	320
4,000	4,000	12	384
10,000	10,000	14	448
100,000	100,000	17	544
1,000,000	1,000,000	20	640

거래 8건 → 층별 8·4·2·1 → 뿌리 23db046cc041f456⋯. tx4 의 포함 증명은 짝 해시 **3개 = 96바이트** (05f2ac⋯(오른쪽), 93790c⋯(오른쪽), ef3892⋯(왼쪽)). 거래가 두 배로 늘어도 증명은 해시 하나만 는다. 백만 건에서도 20개, 640 바이트.

그림 6-2. 머클 포함 증명. 거래 8건의 트리에서 다섯 번째 거래(tx4)의 포함 증명은 짝 해시 3개 96바이트다. 거래가 두 배로 늘어도 증명은 해시 하나만 는다. — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 6장 3절).

증명 길이 — 사슬과 트리

```
# NB06-2: 거래 수가 늘면 증명은 얼마나 느나 — 한 줄 사슬(N 단계)과 트리(log N)의 비교
import math
print(f"{'거래 수 n':>12} {'사슬 따라가기(최대 단계)':>22} {'머클 증명(해시 개수)':>20} {'증명
바이트':>10}")
for n in [1, 2, 8, 100, 1_000, 4_000, 10_000, 100_000, 1_000_000]:
    k = math.ceil(math.log2(n)) if n > 1 else 0
    print(f"{n:>12,} {n:>22,} {k:>20} {32*k:>10,}")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
      거래 수 n          사슬 따라가기(최대 단계)          머클 증명(해시 개수)     증명 바이트
              1                          1                     0            0
              2                          2                     1           32
              8                          8                     3           96
            100                        100                     7          224
          1,000                      1,000                    10          320
          4,000                      4,000                    12          384
         10,000                     10,000                    14          448
        100,000                    100,000                    17          544
      1,000,000                  1,000,000                    20          640
```

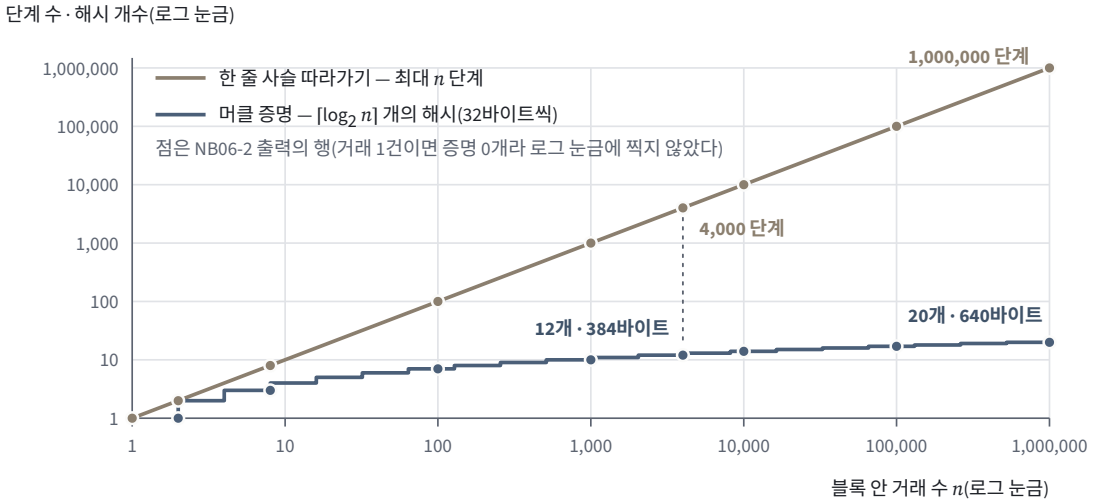


그림 6-3. 거래 수 n 에 따른 확인 단계 수. 두 축 모두 로그 눈금이고, 머클 선은 본문 식 $[\log_2 n]$ 의 그래프, 점은 NB06-2의 출력이다.

[중급] 백만 건에서도 증명은 해시 20개, 640바이트입니다. 베이어·하버·스토네타가 「N 에서 log$N$ 으로」라고 쓴 것이 이 표입니다. 백서 §7의 「가지치기」도 같은 구조에서 나옵니다. 오래된 거래를 버릴 때 트리의 가지를 「잘라 내고」 그 자리의 해시만 남기면 뿌리는 그대로이므로 블록 해시가 깨지지 않습니다.

4. 비트코인의 머클 트리 — 두 가지 구현 특성과 한 가지 약점

비트코인의 트리에는 두 가지 구현 특성이 있습니다. 해시를 SHA-256 **두 번으로** 하고(4장 5절 — 위 일반형 코드는 한 번), 어느 층에서든 해시 개수가 **홀수면 마지막 것을 복제해** 짝을 맞춥니다(위 일반형 코드도 이 규칙을 그대로 썼습니다).[9] 교과서는 이진 트리에 짝수 개의 잎이 필요하기 때문이라고 설명하는데, 이 복제 때문에 **서로 다른 거래 목록이 같은 뿌리를 내는** 경우가 생겼습니다. Bitcoin Core 소스의 긴 주석은 이것을 「심각한 결함」이라고 부르며 CVE-2012-2459라는 취약점 번호를 달아 두었습니다. 거래 목록 [1,2,3,4,5,6]과 [1,2,3,4,5,6,5,6]은 같은 뿌리를 냅니다. 마지막 두 개(5,6)를 복제한 목록은, 홀수 복제 규칙이 (F)와 (F,F)를 같은 값으로 묶기 때문입니다.[10] 공격자가 이런 중복 목록을 가진 블록을 보내면 노드는 「뿌리는 맞는데 거래가 틀린」 블록을 받게 되고, 그 블록을 영구히 무효로 표시해 버리면 진짜 블록까지 거부하게 됩니다. Core는 목록 끝에서 같은 해시 둘을 묶게 되는 경우를 감지해 막습니다.

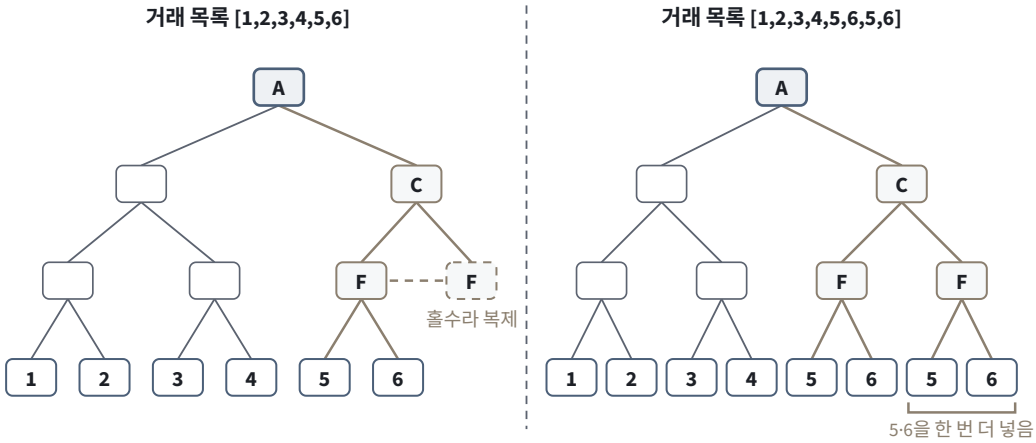


그림 6-4. CVE-2012-2459. 홀수 층에서 마지막 해시를 복제하는 규칙 때문에 (F)와 (F,F)의 해시가 모두 C가 되어, 서로 다른 두 거래 목록이 같은 뿌리 A를 낸다(NB06-3).

```
# NB06-3: 비트코인식 머클 트리 — 이중 SHA-256, 홀수 복제, 그리고 CVE-2012-2459 가 말하는 「같은 뿌리」
import hashlib

def H2(b: bytes) -> bytes:                          # 비트코인의 해시: SHA256(SHA256(x))
    return hashlib.sha256(hashlib.sha256(b).digest()).digest()

def bitcoin_merkle_root(txids):
    """txid 목록(내부 바이트 순서)에서 뿌리를 계산한다. 홀수면 마지막을 복제."""
    layer = list(txids)
    while len(layer) > 1:
        if len(layer) % 2 == 1:
            layer.append(layer[-1])
        layer = [H2(layer[i] + layer[i + 1]) for i in range(0, len(layer), 2)]
    return layer[0]

ids = [H2(bytes([i])) for i in range(1, 7)]         # 거래 1~6 의 가짜 txid
root_a = bitcoin_merkle_root(ids)
root_b = bitcoin_merkle_root(ids + ids[4:6])        # [1,2,3,4,5,6,5,6]
print("[1..6] 뿌리          :", root_a.hex())
print("[1..6,5,6] 뿌리      :", root_b.hex())
print("두 목록의 뿌리가 같은가:", root_a == root_b, " ← 거래 목록이 다른데 뿌리가 같다")

# 거래가 하나뿐인 블록(제네시스 블록이 그렇다)에서는 뿌리 = 그 거래의 txid
one = H2(b"only one transaction")
print("거래 1건이면 뿌리 == txid :", bitcoin_merkle_root([one]) == one, "(12장에서 진짜 데이터로 확인)")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
[1..6] 뿌리          : df316ddf151a6d7f65a868df97de2168ed674371119b038bb97c9926b3f7948d
[1..6,5,6] 뿌리      : df316ddf151a6d7f65a868df97de2168ed674371119b038bb97c9926b3f7948d
두 목록의 뿌리가 같은가: True  ← 거래 목록이 다른데 뿌리가 같다
거래 1건이면 뿌리 == txid : True (12장에서 진짜 데이터로 확인)
```

[중급] 또 하나의 약점은 **64바이트 거래입니다.** 머클 트리의 안쪽 노드는 32바이트 해시 둘을 이어 붙인 64바이트를 해시한 것인데, 비트코인 거래 중에도 직렬화하면 정확히 64바이트가 되는 것이 있습니다. 그러면 어떤 64바이트 거래는 「잎」으로도 「안쪽 노드」로도 읽힐 수 있어, 블록에 들어 있지 않은 거래의 포함 증명을 SPV 검증자에게 받아들이게 할 수 있습니다. 비트코인 개선 제안 BIP-54(「Consensus Cleanup」)는 64바이트 거래를 무효로 만들어 이 약점을 없애는 규칙을 담고 있습니다.[11] 이 제안의 상태는

2026-10-25 기준 「Complete」로, 채택 준비는 끝났으나 활성화되었다는 표시(「Deployed」)는 아닙니다. 활성화 여부는 부록 A 동향 노트에서 갱신합니다. 머클 증명을 법적 증거로 쓰려는 사람이 알아야 할 것은 이것입니다. **증명의 힘은 트리의 구현 세부에 달려 있고, 그 세부에는 알려진 허점이 있었으며(하나는 구현에서 막혔고, 하나는 수정안이 제안되어 있습니다) 바뀔 수 있습니다.**

5. 정리 — 뿌리가 보증하는 것

머클 루트 하나는 거래 전체의 지문입니다. 어떤 거래가 바뀌면 뿌리가 바뀌고, 어떤 거래가 들어 있음은 해시 $\lceil \log_2 n \rceil$ 개로 보일 수 있습니다. 이것이 보증하는 것은 「이 거래가 이 뿌리를 가진 블록에 들어 있다」입니다. 그 블록이 모두가 따르는 체인에 있는지, 그 체인이 뒤집히지 않을지는 뿌리가 말해 주지 않습니다. 다음 7장은 이 뿌리를 담은 80바이트 헤더들이 어떻게 사슬이 되는지를 봅니다.

이 장의 출처

- Dave Bayer, Stuart Haber & W. Scott Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping," in *Sequences II: Methods in Communication, Security, and Computer Science*, pp. 329–334. Springer, 1993. DOI 10.1007/978-1-4613-9323-8_24 — 토너먼트 비유와 머클 1980의 선례(330쪽), 트리 증명과 좌우 표시(§3, 저자 조판본 3~4쪽).
- Ralph C. Merkle, "Protocols for Public Key Cryptosystems," *Proc. 1980 IEEE Symposium on Security and Privacy*, pp. 122–134 (DOI 10.1109/SP.1980.10006) — 머클 트리의 원전으로 둔 문헌. 126·127·133·134쪽. 저자 사이트 사본(스캔본, 텍스트 층은 OCR — 인용은 쪽 이미지로 대조했다)(백서 참고문헌은 쪽을 122–133으로 적는다).
- Ralph C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," *CRYPTO '87*, LNCS 293, pp. 369–378. Springer, 1988. DOI 10.1007/3-540-48184-2_32 — 일회용 서명 트리(이후 발전).
- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — 머클 트리와 가지치기(§7, 4쪽), 간이 지급 검증(§8, 5쪽), 참고문헌 [7].
- Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed., O'Reilly, 2023 (CC BY-SA 4.0, GitHub 원고 커밋 94c9ba64) — 홀수 복제 규칙과 CVE-2012-2459 상자(11장).
- Antoine Poinsot & Matt Corallo, "Consensus Cleanup," BIP 54. https://github.com/bitcoin/bips/blob/master/bip-0054.md (상태 Complete, 2026-10-25 접근) — 64바이트 거래 약점.
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `hashlib`·`math` 만 사용. 노트북 NB06.

각주

1. Dave Bayer, Stuart Haber & W. Scott Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping," in R. Capocelli, A. De Santis & U. Vaccaro (eds.), *Sequences II: Methods in Communication, Security, and Computer Science* (Springer, 1993), pp. 329–334. 인용은 330쪽(§1 Introduction). 원문: "What if an immense flood of banal transactions want their time-stamps to become part of the historical record, but history just isn't interested? We propose to merge many unnoteworthy time-stamping events into one noteworthy event, using a tournament run by its participants. The winner can be easily and widely publicized. Each player, by remembering a short list of opponents, can establish participation in the tournament. We do this by building

trees in place of the linked list of the linking solution, thus achieving an exponential increase in the number of observers." 저자 조판본(March 1992 표기, 6쪽)은 2쪽.

2. 같은 글, 330쪽. 원문: "Such hash trees were previously used by Merkle [Merk 80] for a different purpose, to produce authentication certificates for a directory of public enciphering keys." [Merk 80] = R. C. Merkle, "Protocols for public key cryptosystems," *Proc. 1980 Symposium on Security and Privacy*, IEEE Computer Society (April 1980).

3. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), 4쪽(§7)·9쪽(참고문헌 [7]: R.C. Merkle, "Protocols for public key cryptosystems," In Proc. 1980 Symposium on Security and Privacy, IEEE Computer Society, pages 122–133, April 1980). 백서는 머클 트리에 [7][2][5]를 함께 인용합니다. 원문의 쪽은 122–134쪽입니다(134쪽은 참고문헌 끝과 그림 1).

4. Ralph C. Merkle, "Protocols for Public Key Cryptosystems," *Proc. 1980 IEEE Symposium on Security and Privacy*, pp. 122–134 (DOI 10.1109/SP.1980.10006), 126쪽. 원문: "Fortunately, it is possible to selectively authenticate individual entries in the public file without having to know the whole public file by using Merkle's "tree authentication," [13]. The essence of tree authentication is to authenticate the entire public file by "divide and conquer."" 같은 쪽: "By applying this concept recursively, A can confirm Y_B in the public file knowing only R, $\log_2 n$ intermediate values, and Y_B itself." 이어서: "The information needed to authenticate Y_B, given that R has already been authenticated, lies along the path from R to Y_B and will be called the authentication path." 127쪽: "For a more detailed discussion the reader is referred to [13]." 133쪽 참고문헌 [13]: "Merkle, R.C. A certified digital signature, to appear, CACM." 같은 제목의 논문은 CRYPTO '89에서 발표되어 1990년에 출판되었다 — R. C. Merkle, "A certified digital signature," in G. Brassard (ed.), *Advances in Cryptology — CRYPTO '89*, LNCS 435, pp. 218–238 (Springer, 1990). 이 서지는 바부시 외(2026) 참고문헌 [244]와 하버·스토네타(1991) 참고문헌의 같은 권 표기(부록 E)로 확인했고 원문은 열지 않았다. 그림 1(134쪽)은 잎 8개 트리에서 Y_5 의 인증 경로를 그린다. 저자 사이트 사본(스캔본, 텍스트 층은 OCR — 인용은 쪽 이미지로 대조했다).

5. Ralph C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," *CRYPTO '87*, LNCS 293 (Springer, 1988), 373–375쪽(절 제목 "AN INFINITE TREE OF ONE-TIME SIGNATURES"). 각 노드는 「a 'left' signature, a 'right' signature, and a 'message' signature」를 가지며(373쪽), 노드 i 의 「일방향 해시 총합」은 $\mathrm{HASH}(i) = F(F(y[i,\text{left},*]), F(y[i,\text{right},*]), F(y[i,\text{message},*]))$ 로 그 노드 자신의 세 서명 공개값을 묶은 것입니다(374쪽). 검증은 노드 i 에서 i / 2 로 거슬러 뿌리(노드 1, 공개 파일의 HASH(1))에 이를 때까지 진행합니다(375쪽). 이 논문은 「이후 발전」이며 비트코인의 거래 트리와는 쓰임이 다릅니다.

6. Nakamoto (2008), 4쪽(§7 Reclaiming Disk Space). 원문: "transactions are hashed in a Merkle Tree [7][2][5], with only the root included in the block's hash. Old blocks can then be compacted by stubbing off branches of the tree. The interior hashes do not need to be stored."

7. Bayer, Haber & Stornetta (1993), §3 Trees — 저자 조판본 3~4쪽(게재본으로는 331~332쪽에 해당하나 이 책은 게재본 쪽을 330쪽까지만 확인했으므로 저자 조판본 쪽으로 인용합니다). 원문: "To later establish priority, a participant need only record his own document, as well as the $\lceil \log_2 N \rceil$ hash values that were directly combined with the document's hash value along the path to the root of the tree. In addition, along with each combining hash value, the user needs to record its 'handedness,' indicating whether the newly computed value was placed before or after the combining hash value. Verification consists simply of recomputing the root of the tree from this data."

8. Nakamoto (2008), 5쪽(§8 Simplified Payment Verification). 원문: "A user only needs to keep a copy of the block headers of the longest proof-of-work chain ... and obtain the Merkle branch linking the transaction to the block it's timestamped in. He can't check the transaction for himself, but by linking it to a place in the chain, he can see that a network node has accepted it ... the simplified method can be fooled by an attacker's fabricated transactions for as long as the attacker can continue to overpower the network."

9. Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed. (O'Reilly, 2023), 11장 — "If there are an odd number of transactions to summarize, the last transaction hash will be duplicated to create an even number of leaf nodes ... Similarly, if there are an odd number of hashes to process at any level, the last hash is duplicated." GitHub 원고 커밋 94c9ba64, `ch11_blockchain.adoc`.

10. 같은 장, 상자 "A Design Flaw in Bitcoin's Merkle Tree" — Bitcoin Core 소스 주석을 약간 손질해 옮긴 것. 원문: "the following merkle tree algorithm has a serious flaw related to duplicate txids, resulting in a vulnerability (CVE-2012-2459). The reason is that if the number of hashes in the list at a given level is odd, the last one is

duplicated before computing the next level (which is unusual in merkle trees). This results in certain sequences of transactions leading to the same merkle root. ... The transaction lists [1,2,3,4,5,6] and [1,2,3,4,5,6,5,6] (where 5 and 6 are repeated) result in the same root hash A (because the hash of both of (F) and (F,F) is C)."

11. Antoine Poinsot & Matt Corallo, "Consensus Cleanup," BIP 54 (Layer: Consensus (soft fork), Status: Complete, Assigned 2025-04-11), Motivation 절. 원문: "In computing a block's Merkle root, a transaction with exactly 64 bytes of non-witness data can be interpreted both as an intermediate node in the tree and as a leaf in the tree. This makes it possible to trick an SPV verifier into accepting an inclusion proof for a transaction that is not part of a block, by pretending a 64-byte block transaction is actually an inner node." github.com/bitcoin/bips, 파일 커밋 927b6de9(2026-10-02), 2026-10-25 접근. 상태 「Complete」의 뜻은 BIP 3에 따릅니다.

1부 기술 기초

7장 블록과 체인 — 변조가 비싸지는 구조

핵심 TAKEAWAY

블록은 80바이트짜리 머리(헤더)와 거래 목록입니다. 헤더에는 **직전 블록 헤더의 해시가** 들어 있어, 블록들은 해시로 꿰어진 사슬이 됩니다. 과거의 어느 블록을 한 글자라도 바꾸면 그 블록의 해시가 바뀌고, 그것을 가리키는 다음 블록의 헤더가 틀리고, 그 다음도 틀립니다. 변조는 그 뒤의 모든 블록을 다시 만들어야 하는 일이 됩니다. 그러나 **다시 만드는 것 자체는 해시 사슬만으로는 공짜입니다.** 이 장의 장난감 체인이 그것을 보여 줍니다. 「다시 만들기」를 비싸게 하는 것은 다음 장의 작업증명이고, 그래서 블록체인의 불변성은 수학적 불가능이 아니라 **경제적 비용입니다.**

학습목표

1. 블록 헤더의 여섯 필드와 「이전 블록 해시」가 사슬을 만드는 방식을 설명한다.
2. 「변조 불가」가 아니라 「변조가 드러남」과 「변조 비용이 뒤로 갈수록 커짐」을 구별한다.
3. [중급] 장난감 체인을 만들어 중간 블록을 고치면 이후가 전부 깨지는 것, 그리고 작업증명 없이는 전부 다시 잇는 것이 즉시 가능하다는 것을 코드로 확인한다.
4. 해시 사슬이 원래 무엇을 위해 발명되었는지(타임스탬프) 안다.

[입문]**만 읽은 독자가 가져가는 것.** 블록체인의 불변성은 수학적 불가능이 아니라 **경제적 비용입니다.** 부록 B 오해 ①(「블록체인은 변조 불가능하다」)의 근거가 여기 있습니다.

1. 이 기술은 무엇의 실패에서 나왔나

어떤 문서가 「그때」 존재했다는 것을 어떻게 증명할까요. 하버와 스토네타는 1991년 논문에서 가장 소박한 답을 먼저 검토합니다. 「디지털 금고」입니다. 문서를 타임스탬프 서비스(TSS)에 보내면 서비스가 받은 시각을 기록하고 사본을 보관합니다. 그런데 이 방식에는 네 가지 걱정이 따랐습니다. 프라이버시(서비스가 문서를 봅니다), 대역폭과 저장(문서가 크면 비쌉니다), 무능(서비스가 실수하거나 사본을 잃을 수 있습니다), 그리고 **신뢰**. 「이 방식의 어떤 것도 TSS가 고객과 결탁해 실제와 다른 날짜·시각에 타임스탬프를 찍었다고 주장하는 것을 막지 못한다.」[1] 앞의 셋은 해시와 서명으로 풉니다(4장·5장). 넷째가 남습니다. 「그러나 누가 감시자를 감시할 것인가?」[2]

그들의 답이 **연결(linking)입니다.** 서비스가 발행하는 각 인증서에 직전 요청의 정보(시각·고객 번호·해시)와 직전 연결 정보의 해시를 함께 넣어 서명합니다. 이렇게 하면 서비스는 **실제보다 늦은 시각을 찍을 수 없습니다**(forward-date 불가). 그 시각 직전에 들어온 요청들의 비트가 인증서에 들어가야 하는데, 서비스는 그것을 아직 받지 못했기 때문입니다. **실제보다 이른 시각으로 소급해 찍을 수도 없습니다**(back-

date 불가). 그 이른 시각 바로 다음의 인증서들에는 그 문서의 비트가 들어가 있어야 하는데, 그 인증서들은 이미 그 비트 없이 발행되어 버렸기 때문입니다. 이미 발행된 흐름 사이에 새 문서를 끼워 넣으려면 해시 함수의 충돌을 계산해야 합니다.[3] 각 인증서가 직전 인증서를 가리키는 사슬. 이것이 블록체인의 뼈대이며, 백서 §3은 이 논문들을 인용하면서 같은 그림을 그립니다. 「각 타임스탬프는 직전 타임스탬프를 자기 해시 안에 포함해 사슬을 이루고, 추가되는 타임스탬프마다 앞의 것들을 강화한다.」[4]

베이어·하버·스토네타는 1993년 논문 서론의 첫 문장에서 이 착상의 바탕을 한 줄로 적었습니다. 「인과가 사건을 시간에 고정한다. 어떤 사건이 특정한 이전 사건들에 의해 결정되었고 특정한 이후 사건들을 결정한다면, 그 사건은 역사 속 제자리에 단단히 끼어 있다.」[5] 그들의 말로, 종이 문서에는 날짜를 정확히 매기고 사후 변조의 흔적을 살필 수 있는 법과학적 성질이 있지만, 디지털 문서는 매체에 그런 흔적을 남기지 않을 수 있습니다. 해시 사슬은 디지털 데이터에 **인과를 인위적으로 심는** 장치입니다.

2. 블록은 무엇으로 되어 있나

비트코인 블록은 두 부분입니다. **헤더(header)** 80바이트와 **거래 목록**. 거래 목록이 블록 크기의 대부분이고, 헤더는 거래가 몇 건이든 80바이트입니다.[6] 백서도 「거래 없는 블록 헤더는 약 80바이트」라고 적습니다.[7] 헤더의 여섯 필드는 다음과 같습니다.[8]

크기	필드	뜻
4바이트	버전(Version)	본래 판 번호. 쓰임이 바뀌어 왔다
32바이트	이전 블록 해시(Previous Block Hash)	직전(부모) 블록 헤더의 해시
32바이트	머클 루트(Merkle Root)	이 블록 거래들의 머클 트리 뿌리(6장)
4바이트	시각(Timestamp)	블록이 만들어진 대략의 시각(유닉스 시간)
4바이트	목표(Target, nBits)	작업증명 목표값의 압축 표현(8장)
4바이트	논스(Nonce)	작업증명에 쓰는 임의의 수

블록의 「이름」은 헤더의 해시입니다. 헤더 80바이트를 SHA-256으로 두 번 해시한 32바이트가 **블록 해시이고**, 더 정확히는 **블록 헤더 해시입니다**. 거래는 머클 루트를 통해 간접적으로만 들어갑니다.[9] 그리고 다음 블록의 헤더 둘째 칸에는 바로 이 해시가 적힙니다. 이것이 「체인」의 전부입니다. 블록 B_k의 헤더에 B_{k-1}의 해시가 들어 있고, B_{k-1}의 헤더에 B_{k-2}의 해시가 들어 있습니다. 제네시스 블록까지 거슬러 갑니다.

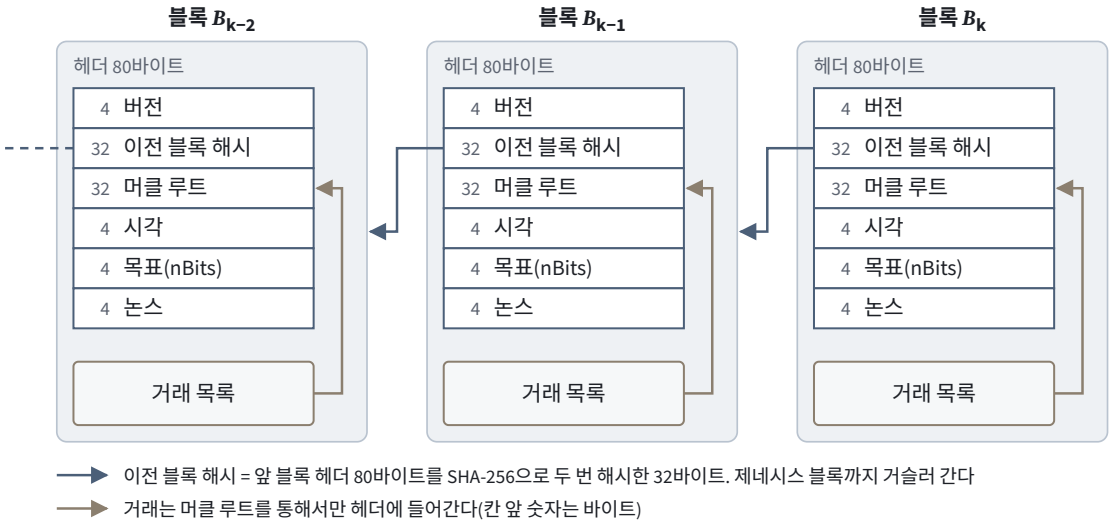


그림 7-1. 블록 세 개와 그 연결. 각 블록 헤더의 둘째 칸에 앞 블록 헤더의 해시가 들어 있고, 거래 목록은 머클 루트 한 칸으로만 헤더에 닿는다.

3. 변조는 왜 연쇄적으로 드러나는가

백서 §4는 두 문장으로 요약합니다. 「일단 CPU 노력이 들어가 작업증명을 만족하게 되면, 블록은 그 작업을 다시 하지 않고는 바뀔 수 없다. 그 뒤로 블록들이 사슬로 이어지면, 그 블록을 바꾸는 작업에는 그 뒤의 모든 블록을 다시 하는 일이 포함된다.」[10] 여기서 두 가지를 떼어 읽어야 합니다.

첫째, **드러남.** 블록 B_k 의 거래 한 글자를 고치면 그 머클 루트가 바뀌고, 헤더가 바뀌고, 헤더의 해시가 바뀝니다. 그런데 B_{k+1} 의 헤더에는 옛 해시가 적혀 있으니 B_{k+1} 은 더 이상 B_k 를 가리키지 않습니다. 사슬이 끊어진 것이 누구 눈에나 보입니다. 이것은 해시 사슬만으로 됩니다. 작업증명이 없어도 됩니다.

둘째, **비용.** 끊어진 사슬을 다시 이으려면 B_{k+1} 의 헤더에 새 해시를 적어야 하고, 그러면 B_{k+1} 의 해시도 바뀌니 B_{k+2} 도 고쳐야 합니다. 끝까지 전부입니다. 그런데 **해시 계산은 싸고 빠릅니다.** 블록 하나에 해시 한 번이면 되니, 해시 사슬만 있는 장부라면 변조자는 순식간에 전체를 다시 꿰어 「새 사슬」을 내놓을 수 있습니다. 그 사슬은 내부적으로 완벽히 일관됩니다. 드러나는 것은 「원래 사슬과 다르다」는 사실뿐이고, 둘 중 어느 것이 원래인지를 해시는 말해 주지 않습니다.

백서 인용문의 「CPU 노력」이 바로 이 틈을 메우는 장치입니다. 블록마다 헤더의 해시가 어떤 조건을 만족하도록 **논스를 바꿔 가며 천문학적인 횟수로 해시해야** 한다면(2026년 10월 기준 블록 하나에 기대 약 6×10^{23} 번, 8장), 다시 꿰는 데에 그만큼의 계산이 다시 듭니다. 뒤에 블록이 많이 쌓일수록 다시 꿸 양이 늘어납니다. 그래서 「변조 비용이 뒤로 갈수록 커진다」는 명제가 성립합니다. 이것은 다음 장의 주제이고, 그 비용이 얼마나 되는지, 그 비용을 낼 수 있는 자가 있으면 어떻게 되는지는 9장과 20장에서 봅니다.

「가장 긴 체인」이라는 규칙

사슬이 둘 이상 생기면 어느 것이 정본일까요. 백서 §5는 「노드들은 항상 가장 긴 체인을 올바른 것으로 여기고 그것을 늘리는 일을 계속한다」고 적습니다. 두 노드가 동시에 다른 블록을 내놓으면 각자 먼저 받은 것 위에 작업하되 다른 가지도 보관하고, 다음 작업증명이 나와 한 가지가 길어지면 다른 가지에서 일하던 노드들은 긴 쪽으로 옮깁니다.[11] 현행 구현은 「가장 긴」을 「누적된 작업증명이 가장 많은」으로 읽습니다. 로젠펠드(Meni Rosenfeld)는 2014년 논문에서 이를 「모든 노드는 자기가 아는 가장 긴 가지(더 정확히는 가장 많은 작업증명을 나타내는 가지)를 유효한 체인으로 여긴다」고 적습니다.[12] 난이도가 같은 구간에서는 둘이 같습니다.

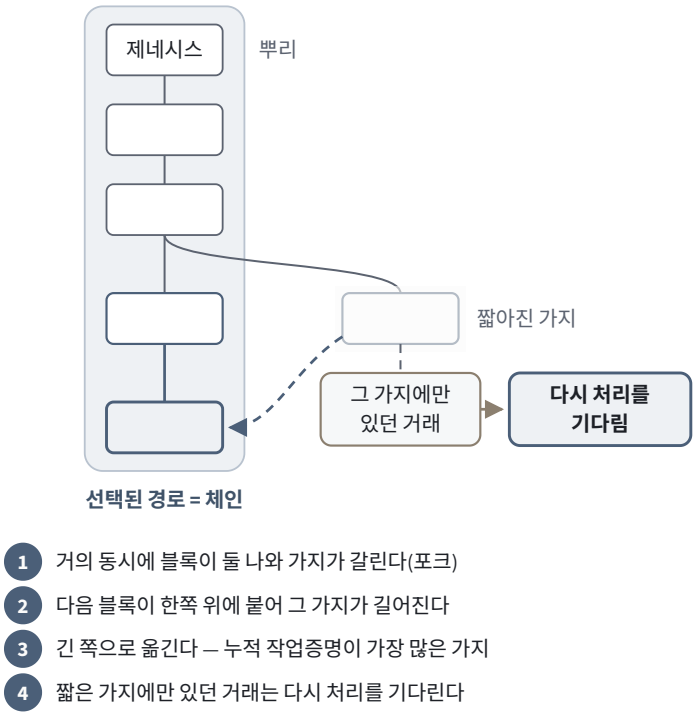


그림 7-2. 제네시스를 뿌리로 한 블록 트리에서 가지가 갈리고, 다음 블록이 붙어 길어진 쪽으로 노드들이 옮겨 가는 과정. 선택된 경로가 체인이고, 짧아진 가지에만 있던 거래는 다시 처리를 기다린다.

그러므로 블록체인의 「진짜」는 어디에 등록되어 있는 것이 아닙니다. 그것은 **규칙을 따르는 참가자들이 어느 가지 위에 계속 쌓는가로** 매 순간 결정됩니다. 이 점이 법률가에게 낯선 지점이고, 하드포크가 생기면 두 체인이 모두 「진짜」라고 주장하는 상황이 생기는 이유입니다(23장·26장).

> **? 법률가의 질문**
>
> 어떤 사람이 자기 개인정보가 담긴 거래가 블록체인에 기록되었다며 삭제를 구합니다. 체인은 「고칠 수 없게」 설계되어 있고, 고치려면 수많은 참가자가 그 뒤의 모든 블록을 다시 만들어야 합니다. 개인정보 삭제권은 지울 수 없는 원장에서 무엇을 요구하는 것이 됩니까. 삭제의 상대방은 누구이고, 「지운다」는 것은 기술적으로 어떤 행위를 가리킵니까. 처음부터 개인정보를 체인 밖에 두고 해시만 올리는 설계는 이 문제를 푸는가요, 미루는가요. → 28장

4. 코드 — 장난감 체인

변조와 다시 꿰기

[중급] 아래 장난감 체인은 작업증명이 없습니다. 헤더에 이전 해시·데이터·시각만 넣고 SHA-256으로 꿰기만 합니다. 이렇게 하면 「드러남」과 「공짜로 다시 꿰기」를 분리해서 볼 수 있습니다. 작업증명은 8장에서 이 클래스에 한 줄을 더해 붙입니다.

```
# NB07-1: 장난감 블록체인 — 작업증명 없는 해시 사슬. 변조는 드러나지만 다시 꿰는 것은 공짜다 (공부용)
import hashlib, json

def H(s: str) -> str:
    return hashlib.sha256(s.encode()).hexdigest()

class Block:
    def __init__(self, index, prev_hash, data, timestamp):
        self.index, self.prev_hash, self.data, self.timestamp = index, prev_hash, data,
timestamp
    def header(self) -> str:                        # 헤더 = 사슬에 들어가는 부분
        return json.dumps([self.index, self.prev_hash, H(self.data), self.timestamp])
    def hash(self) -> str:                          # 블록 해시 = 헤더의 해시 (데이터는 H(data)로만
들어간다 — 머클 루트 자리)
        return H(self.header())

def build_chain(datas):
    chain = [Block(0, "0" * 64, datas[0], 1231006505)]            # 제네시스
    for i, d in enumerate(datas[1:], start=1):
        chain.append(Block(i, chain[-1].hash(), d, 1231006505 + 600 * i))
    return chain

def validate(chain):
    """각 블록의 prev_hash 가 실제 직전 블록 해시와 같은지 검사. 틀린 첫 블록 번호를 돌려준다(없으면 None)."""
    for i in range(1, len(chain)):
        if chain[i].prev_hash != chain[i - 1].hash():
            return i
    return None

datas = ["genesis", "A→B 1", "B→C 2", "C→D 3", "D→E 4", "E→F 5"]
chain = build_chain(datas)
print("블록 수:", len(chain), "| 유효?", validate(chain) is None)
for b in chain:
    print(f"  #{b.index} prev={b.prev_hash[:10]}… hash={b.hash()[:10]}… data={b.data!r}")

print("\n--- 3번 블록의 거래를 '3' 에서 '300' 으로 바꾼다 ---")
chain[3].data = "C→D 300"
bad = validate(chain)
print("처음으로 끊어진 자리: 블록 #", bad, "(3번을 바꿨는데 4번이 틀렸다고 나온다 — 4번의 prev_hash 가 옛
해시)")
for b in chain[3:]:
    print(f"  #{b.index} prev={b.prev_hash[:10]}… 실제 직전 해시={chain[b.index-1].hash()[:10]}…
일치={b.prev_hash == chain[b.index-1].hash()}")

print("\n--- 변조자가 이후 블록의 prev_hash 를 전부 다시 적는다 ---")
for i in range(4, len(chain)):
    chain[i].prev_hash = chain[i - 1].hash()
print("다시 꿴 체인 유효?", validate(chain) is None, "← 작업증명이 없으면 전체를 다시 잇는 데 블록당 해시 한
번이면 끝난다")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
블록 수: 6 | 유효? True
  #0 prev=0000000000… hash=475e9da5b5… data='genesis'
  #1 prev=475e9da5b5… hash=e9668fba8f… data='A→B 1'
  #2 prev=e9668fba8f… hash=dcdec3cf8e… data='B→C 2'
  #3 prev=dcdec3cf8e… hash=daa1496cf4… data='C→D 3'
  #4 prev=daa1496cf4… hash=0e85befdef… data='D→E 4'
  #5 prev=0e85befdef… hash=3c35f3caa0… data='E→F 5'

--- 3번 블록의 거래를 '3' 에서 '300' 으로 바꾼다 ---
처음으로 끊어진 자리: 블록 # 4 (3번을 바꿨는데 4번이 틀렸다고 나온다 — 4번의 prev_hash 가 옛 해시)
  #3 prev=dcdec3cf8e… 실제 직전 해시=dcdec3cf8e…  일치=True
  #4 prev=daa1496cf4… 실제 직전 해시=a1b2cd168c…  일치=False
  #5 prev=0e85befdef… 실제 직전 해시=0e85befdef…  일치=True

--- 변조자가 이후 블록의 prev_hash 를 전부 다시 적는다 ---
다시 꿴 체인 유효? True ← 작업증명이 없으면 전체를 다시 잇는 데 블록당 해시 한 번이면 끝난다
```

[중급] 세 가지가 보입니다. 변조는 **바로 다음 블록에서** 드러납니다(3번을 고치면 4번의 연결이 끊깁니다). 끊어진 뒤를 전부 다시 적으면 체인은 다시 「유효」해집니다. 그리고 그 일은 즉시 끝납니다. 해시 사슬이 보증하는 것은 **일관성이지 역사가** 아닙니다. 두 개의 일관된 사슬이 있을 때 어느 쪽이 역사인지를 가르는 장치가 없으면, 「변조 불가」는 빈말입니다.



그림 7-3. 체인 변조 시연. 3번 블록의 거래를 「C→D 3」에서 「C→D 300」으로 바꾸면 4번 블록의 이전 해시가 어긋나 연결이 끊긴다. 작업증명이 없으면 끊어진 뒤를 다시 꿰는 데 블록당 해시 한 번이면 된다. — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 7장 4절).

두 사슬이 다 「유효」할 때

```
# NB07-2: 두 사슬이 다 「유효」할 때 — 해시만으로는 어느 쪽이 원본인지 알 수 없다
original = build_chain(datas)
forged = build_chain([d if i != 3 else "C→D 300" for i, d in enumerate(datas)])
print("원본 체인 유효?", validate(original) is None, "| 위조 체인 유효?", validate(forged) is None)
print("마지막 블록 해시 — 원본:", original[-1].hash()[:16], "… 위조:", forged[-1].hash()[:16], "…")
print("두 체인은 서로 다르지만 각각 내부적으로 완벽히 일관된다. 「어느 것이 진짜인가」는 해시가 답하지 못한다.")
print("이 질문에 답하려고 들어온 것이 8장의 작업증명(비용)과 「가장 많은 작업이 쌓인 체인」 규칙이다.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
원본 체인 유효? True | 위조 체인 유효? True
마지막 블록 해시 — 원본: 3c35f3caa070a93e … 위조: 63f530852e4eed45 …
두 체인은 서로 다르지만 각각 내부적으로 완벽히 일관된다. 「어느 것이 진짜인가」는 해시가 답하지 못한다.
이 질문에 답하려고 들어온 것이 8장의 작업증명(비용)과 「가장 많은 작업이 쌓인 체인」 규칙이다.
```

▾ [심화] **체인은 트리다 — 포크와 가지 선택**

[심화] 「체인」이라는 이름은 평상시 모습입니다. 각 블록이 직전 블록 해시 하나만 가리키므로 구조적으로는 **뿌리가 제네시스인 트리이고**, 체인은 그 트리에서 선택된 한 경로입니다. 로젠펠드의 서술이 정확합니다. 「블록들은 제네시스 블록을 뿌리로 하는 트리를 이루고, 각 블록은 자기가 참조하는 블록의 자식이다. 이 트리의 가지는 잎 블록에서 제네시스까지의 경로이며, 각 가지는 비트코인 거래 역사의 한 판본을 나타낸다. 각 가지는 내부적으로 일관되어야 하고 서로 충돌하는 두 거래를 담을 수 없다. 그러나 가지들끼리는 일관될 필요가 없다.」[13] 두 노드가 거의 동시에 블록을 찾으면 트리에 가지가 둘 생기고(포크), 다음 블록이 어느 한쪽 위에 붙으면 그 가지가 길어져 모두가 그쪽으로 옮깁니다. 짧아진 가지는 유효한 체인에서 빠지고, 그 가지에만 들어 있던 거래는 다시 처리를 기다리는 상태로 돌아갑니다. 이 되돌림이 얼마나 자주, 얼마나 깊게 일어날 수 있는지가 9장의 수학입니다.

[심화] 위 NB07-2의 두 체인은 트리로 보면 2번 블록에서 갈라진 두 가지입니다. 작업증명 체계에서는 두 가지의 누적 작업량을 비교해 큰 쪽을 택하고, 작업량은 각 블록의 난이도 목표에서 계산합니다. 「길이」가 아니라 「무게」입니다.

5. 정리

블록은 80바이트 헤더와 거래 목록이고, 헤더의 「이전 블록 해시」가 사슬을 만듭니다. 사슬 덕분에 과거의 변조는 반드시 드러납니다. 그러나 드러난 변조를 다시 꿰어 숨기는 데 해시 계산만 든다면 그 비용은 거의 0입니다. 「변조 불가」라는 말의 실제 내용은 「다시 꿰는 비용이 크고, 뒤로 갈수록 더 커진다」이며, 그 비용을 만드는 장치가 다음 장의 작업증명입니다. 이 장에서 기억할 한 문장은 이것입니다. **해시 사슬은 역사를 증명하지 않고 일관성을 증명한다. 역사는 비용이 증명한다.**

이 장의 출처

- Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *Journal of Cryptology* 3(2): 99–111, 1991. DOI 10.1007/BF00196791 — 소박한 해법의 네 걱정(101쪽), 연결 방식과 그 논증(103–104쪽).
- Dave Bayer, Stuart Haber & W. Scott Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping," *Sequences II*, pp. 329–334. Springer, 1993. DOI 10.1007/978-1-4613-9323-8_24 — 「인과가 사건을 시간에 고정한다」(330쪽).
- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — 타임스탬프 서버(§3, 2쪽), 변조에 드는 작업(§4, 3쪽), 가장 긴 체인 규칙(§5, 3쪽), 80바이트 헤더(§7, 4쪽).
- Meni Rosenfeld, "Analysis of Hashrate-Based Double Spending," arXiv:1402.2009, 2014 — 블록 트리와 가지 선택(§2, 2–3쪽).
- Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed., O'Reilly, 2023 (CC BY-SA 4.0, GitHub 원고 커밋 94c9ba64) — 헤더 80바이트와 여섯 필드, 블록 헤더 해시(11장).
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `hashlib`·`json` 만 사용. 노트북 NB07.

각주

1. Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *Journal of Cryptology* 3(2): 99–111 (1991), 101쪽(§3 A Naive Solution). 원문: "Trust. The fundamental problem remains: nothing in this scheme prevents the TSS from colluding with a client in order to claim to have time-stamped a document for a date and time different from the actual one."

2. 같은 글, 103쪽(§5 제사(題詞)). 유베날리스의 "Sed quis custodiet ipsos Custodes?"를 인용하고 "But who will guard the guards themselves?"로 옮깁니다.

3. 같은 글, 104쪽(§5.1 Linking). 원문: "the TSS cannot forward-date a document, because the certificate must contain bits from requests that immediately preceded the desired time, yet the TSS has not received them. The TSS cannot feasibly back-date a document by preparing a fake time-stamp for an earlier time, because bits from the document in question must be embedded in certificates immediately following that earlier time, yet these certificates have already been issued. Furthermore, correctly embedding a new document into the already-existing stream of time-stamp certificates requires the computation of a collision for the hash function H."

4. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), 2쪽(§3 Timestamp Server). 원문: "Each timestamp includes the previous timestamp in its hash, forming a chain, with each additional timestamp reinforcing the ones before it." 백서는 이 절에서 [2-5]를 인용하고, [3]·[4]가 Haber & Stornetta 1991과 Bayer·Haber·Stornetta 1993입니다.

5. Dave Bayer, Stuart Haber & W. Scott Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping," *Sequences II* (Springer, 1993), 330쪽(§1). 원문: "Causality fixes events in time. If an event was determined by certain earlier events, and determines certain subsequent events, then the event is sandwiched securely into its place in history."

6. Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed. (O'Reilly, 2023), 11장 — "The block header is 80 bytes, whereas the total size of all transactions in a block can be..." GitHub 원고 커밋 94c9ba64, `ch11_blockchain.adoc`.

7. Nakamoto (2008), 4쪽(§7). 원문: "A block header with no transactions would be about 80 bytes."

8. *Mastering Bitcoin*, 3rd ed., 11장, 표 "The structure of the block header" — Version 4 bytes / Previous Block Hash 32 / Merkle Root 32 / Timestamp 4 / Target 4 / Nonce 4. 각 필드의 바이트 배열 순서와 실제 값은 12장에서 제네시스 블록으로 확인합니다.

9. *Mastering Bitcoin*, 3rd ed., 11장(§Block Identifiers) — "The primary identifier of a block is its cryptographic hash … made by hashing the block header twice through the SHA256 algorithm. The resulting 32-byte hash is called the block hash but is more accurately the block header hash, because only the block header is used to compute it." 거래 목록은 머클 루트로만 헤더에 반영됩니다.

10. Nakamoto (2008), 3쪽(§4 Proof-of-Work). 원문: "Once the CPU effort has been expended to make it satisfy the proof-of-work, the block cannot be changed without redoing the work. As later blocks are chained after it, the work to change the block would include redoing all the blocks after it."

11. Nakamoto (2008), 3쪽(§5 Network). 원문: "Nodes always consider the longest chain to be the correct one and will keep working on extending it. If two nodes broadcast different versions of the next block simultaneously, some nodes may receive one or the other first. In that case, they work on the first one they received, but save the other branch in case it becomes longer. The tie will be broken when the next proof-of-work is found and one branch becomes longer; the nodes that were working on the other branch will then switch to the longer one."

12. Meni Rosenfeld, "Analysis of Hashrate-Based Double Spending," arXiv:1402.2009 (2014; 초고 2012-12-11), 3쪽(§2). 원문: "It is agreed that every node will consider the longest branch it is aware of as the valid chain (more precisely, the branch representing the most proof of work)."

13. Rosenfeld (2014), 2쪽(§2). 원문: "The blocks hence form a tree, with the genesis block as the root, and each block being a child of the block it references. A branch in this tree is a path from a leaf block to the genesis block; each such branch represents one version of the history of Bitcoin transactions. Each branch must be internally consistent and can never include two conflicting transactions; however, the branches need not be consistent with one another, and one branch can include a transaction which contradicts a transaction in another branch."

1부 기술 기초

8장 작업증명 — 난이도, 채굴, 가장 긴 체인

핵심 TAKEAWAY

채굴은 「퍼즐 풀기」가 아니라 **복권 긁기입니다.** 블록 헤더의 논스를 바꿔 가며 해시를 계산해, 해시값이 목표값 아래로 떨어지는 논스를 찾는 일입니다. 시도 한 번의 당첨 확률은 목표값이 정하고, 그 목표값은 2,016블록마다 「최근 블록들이 너무 빨리 나왔으면 낮추고 너무 늦었으면 높여서」 평균 10분에 한 블록이 나오도록 조정됩니다. 그래서 기계가 빨라져도 당첨 간격은 10분으로 되돌아갑니다. 보안에 뜻이 있는 변수는 어떤 참가자의 **절대 연산속도가** 아니라 전체 중 **점유율** q 입니다. 이 장을 읽으면 난이도 숫자를 목표값으로 바꿀 수 있고, 「컴퓨터가 빨라지면 블록체인이 뚫린다」가 왜 틀렸는지 설명할 수 있습니다.

학습목표

1. 작업증명이 「조건을 만족하는 해시를 찾는 복권」임을 설명하고, 성공 확률·기대 시도 횟수를 목표값에서 계산한다.
2. 블록 헤더의 nBits(압축 목표값)를 실제 목표값과 난이도로 바꾼다.
3. 2,016블록 조정 규칙을 Bitcoin Core 소스에서 읽고, 조정 폭 상하한(4배·¼배)이 왜 있는지 안다.
4. 「가장 긴(가장 무거운) 체인」 규칙과 「절대 속도가 아니라 점유율」이라는 명제를 연결한다.

[입문]만 읽은 독자가 가져가는 것. 채굴은 퍼즐이 아니라 복권이고, 기계가 빨라져도 당첨 간격은 10분으로 되돌려집니다. 부록 B 오해 ⑥(「컴퓨터가 빨라지면 블록체인이 뚫린다」)의 근거입니다.

1. 이 기술은 무엇의 실패에서 나왔나

작업증명은 돈을 위해 발명되지 않았습니다. **스팸** 때문이었습니다. 애덤 백(Adam Back)의 해시캐시(Hashcash)는 1997년 3월 사이퍼펑크 메일링리스트에 처음 공지되었고,[1] 2002년 기술보고서는 그 목적을 「이메일이나 익명 리메일러처럼 요금이 없는 인터넷 자원의 체계적 남용을 억제하는 장치」라고 적습니다.[2] 발상은 단순합니다. 메일 한 통을 보내려면 보내는 쪽 컴퓨터가 몇 초짜리 계산을 먼저 해야 한다. 한 통에 몇 초는 아무것도 아니지만 백만 통이면 수십 일이다. 받는 쪽은 그 계산이 되었는지를 해시 한 번으로 확인한다.

그 「계산」이 해시 찍기입니다. 백은 해시캐시 비용 함수를 「모두 0인 k 비트 문자열에 대한 부분 해시 충돌을 찾는 것」으로 정의합니다. 입력의 일부를 바꿔 가며 해시를 반복해, 출력의 앞 k 비트가 모두 0이 되는 입력을 찾습니다. 가장 빠른 방법은 무작정 시도하는 것(brute force)이고, 서버는 도전을 낼 필요가 없으며(비대화형), 누구나 토큰을 효율적으로 검증할 수 있고(공개 감사 가능), 서버라고 해서 토큰을 싸게 만들 수 있는 뒷문이 없습니다(trapdoor-free).[3] 백은 또 서버가 이미 쓴 토큰의 「이중 지출

데이터베이스」를 두어 같은 토큰의 재사용을 막아야 한다고 적습니다.[4] 「이중 지출」이라는 낱말이 2002년 스팸 방지 문서에 이미 들어 있습니다.

나카모토는 이 장치를 가져오면서 두 가지를 바꿨습니다. 첫째, 「메일 한 통」이 아니라 「블록 하나」에 비용을 붙였습니다. 둘째, 비용의 크기를 **고정하지 않고 네트워크가 스스로 조정하게** 했습니다. 백서 §4의 문장들입니다. 「P2P 방식으로 분산 타임스탬프 서버를 구현하려면 신문이나 유즈넷 게시물 대신 애덤 백의 해시캐시와 비슷한 작업증명 체계를 써야 한다.」 그리고 「하드웨어 속도의 증가와 노드 운영에 대한 관심의 변동을 보정하기 위해, 작업증명 난이도는 시간당 평균 블록 수를 목표로 하는 이동 평균으로 정해진다. 블록이 너무 빨리 생성되면 난이도가 올라간다.」[5] 해시캐시의 비대화형 비용 함수에는 작업량 k를 네트워크가 스스로 바꾸는 장치가 적혀 있지 않습니다. 보고서가 「서버 CPU 부하에 따라 작업 계수를 동적으로 조정」하는 안을 다루는 것은 서버가 도전을 내는 대화형 변종에서입니다.[6] 비트코인은 조정을 **합의 규칙** 안에 넣었습니다. 이 차이가 「기계가 빨라져도 10분」을 만듭니다.

2. 직관 — 복권

백서 §4의 정의입니다. 「작업증명은 어떤 값을 해시했을 때, 예컨대 SHA-256으로, 해시가 일정 개수의 0 비트로 시작하는 값을 찾는 일이다. 요구되는 평균 작업량은 요구되는 0 비트 수에 대해 지수적이고, 해시 한 번으로 검증할 수 있다.」[7] 채굴자는 블록 헤더의 논스(4바이트 임의 수)를 0, 1, 2, …로 바꾸면서 헤더를 해시합니다. 해시가 조건을 만족하면 그 헤더를 네트워크에 내놓습니다. 만족하지 않으면 다음 논스. 그게 전부입니다.

왜 「퍼즐」이 아니라 「복권」인가. 퍼즐은 머리를 쓰면 빨리 풉니다. 이 일은 머리를 쓸 곳이 없습니다. 4장에서 본 대로 SHA-256의 출력은 입력과 무관한 난수처럼 보이므로, 어떤 논스가 조건을 만족할지는 **해 보기 전에는 알 수 없습니다.** 시도 한 번은 복권 한 장이고, 당첨 확률은 매번 같습니다. 그런스판과 페레스마르코의 표현을 빌리면, 새 해시마다 작업은 처음부터 시작되므로 블록을 캐는 데 걸리는 시간은 **기억이 없습니다(memoryless).** 지금까지 10분을 허탕쳤다고 다음 1분의 당첨 확률이 오르지 않습니다.[8]

복권의 당첨 확률을 정하는 것이 **목표값(target)입니다.** 「앞에 0이 k 개」라는 말은 「해시값을 256비트 정수로 보았을 때 2^{256-k} 보다 작다」와 같습니다. 비트코인은 0의 개수 대신 목표값 자체를 헤더에 적어 두고(nBits 필드), 「헤더의 해시 ≤ 목표값」을 요구합니다. 목표값이 작을수록 당첨이 어렵습니다.

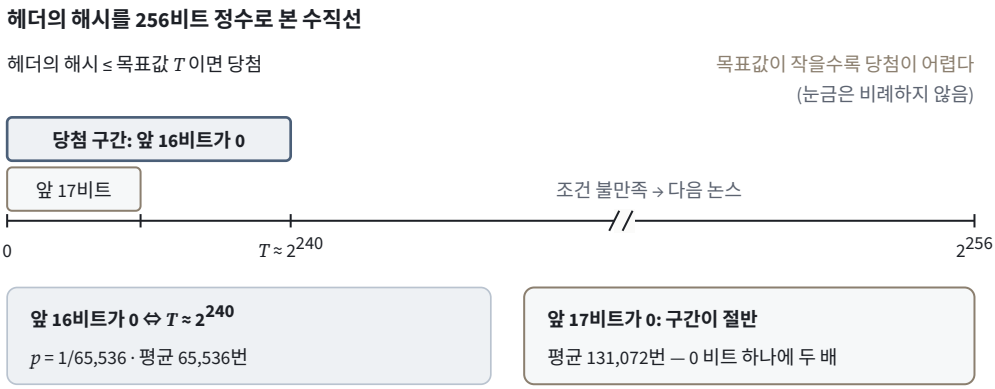


그림 8-1. 헤더의 해시를 256비트 정수로 본 수직선. 목표값 T 아래가 당첨 구간이고, 앞 0 비트가 하나 늘면 구간이 절반이 되어 기대 시도 횟수가 두 배가 된다. 눈금은 비례하지 않는다.

3. [중급] 수식 — 성공 확률, 기대 시도 횟수, 난이도

[중급] 해시 한 번이 조건을 만족할 확률은

$$p = \frac{T+1}{2^{256}} \approx \frac{T}{2^{256}}$$

입니다. 기호 풀이: T 는 목표값(256비트 정수), 2^{256} 은 가능한 해시값의 개수입니다. 수치 예: 「앞 16비트가 0」이면 $T \approx 2^{240}$, $p = 2^{-16} = 1/65{,}536$ 입니다. 시도마다 독립이고 확률이 같으니 첫 성공까지의 시도 횟수는 **기하분포를** 따르고, 기댓값은 $1/p$ 입니다. 앞 16비트면 평균 65,536번, 앞 17비트면 평균 131,072번입니다. 백서가 「지수적」이라고 한 것이 이것입니다. 0 비트 하나에 두 배.

[중급] **난이도(difficulty)** D 는 사람이 읽기 좋게 목표값을 뒤집은 수입니다. 비트코인의 가장 쉬운 목표값 $T_{\max}$(제네시스 블록의 nBits `0x1d00ffff` 가 가리키는 값)를 기준으로

$$D = \frac{T_{\max}}{T}$$

입니다. 수치 예: $T = T_{\max}/2$ 이면 $D = 2$. 「난이도 2」는 「제네시스 때보다 두 배 어렵다」는 뜻입니다. 난이도 1에서 블록 하나에 필요한 기대 해시 수는 $2^{256}/T_{\max}$ 로, 이것이 약 2^{32} 입니다(정확한 값은 아래 코드가 냅니다). 그래서 **난이도 D 에서 블록 하나당 기대 해시 수는 약 $D \cdot 2^{32}$** 이고, 블록이 평균 600초마다 나온다면 네트워크 전체의 해시율은 약 $D \cdot 2^{32} / 600$ 해시/초로 **추정할** 수 있습니다. 이것은 측정이 아니라 「600초 가정」 위의 추정입니다. 실제 블록 간격이 600초와 다르면 그만큼 다릅니다.

nBits — 4바이트에 256비트 수를 넣는 법

[중급] 헤더의 목표 필드는 4바이트뿐입니다. 256비트 수를 그대로 적을 수 없으니 **부동소수 비슷한 압축 표기를** 씁니다. 첫 바이트가 지수, 나머지 세 바이트가 가수(계수)입니다.

$$T = \text{계수} \times 256^{\,\text{지수}-3} = \text{계수} \times 2^{8(\text{지수}-3)}$$

기호 풀이: 계수는 nBits의 아래 3바이트를 정수로 본 것, 지수는 nBits의 첫 바이트입니다.[9] 수치 예: 제네시스의 `0x1d00ffff` 는 지수 `0x1d` = 29, 계수 `0x00ffff` = 65,535이므로 $T_{\max} = 65535 \times 256^{26}$ 입니다. 2026년 10월 25일 체인 끝 블록(높이 972,958)의 nBits는 `0x17020093` 이었고, 이것을 풀면 난이도 약 1.41×10^{14} 가 나옵니다(2026-10 기준, 부록 A 동향 노트).[10]

조정 규칙 — Bitcoin Core가 하는 일

[중급] 목표값은 2,016블록마다 한 번 조정됩니다. Bitcoin Core 소스가 규칙 그 자체입니다. 메인넷 매개변수는 목표 기간 `nPowTargetTimespan = 14 * 24 * 60 * 60` (2주)과 목표 간격 `nPowTargetSpacing = 10 * 60` (10분)이고, 조정 간격은 그 비 `DifficultyAdjustmentInterval() = nPowTargetTimespan / nPowTargetSpacing` = 2,016입니다.[11] 조정 함수 `CalculateNextWorkRequired` 는 직전 2,016블록 구간이 실제로 걸린 시간을 재고, 그 시간을 목표 기간의 ¼ 미만이면 ¼로, 4배 초과면 4배로 자른 뒤, 새 목표값을

$$T_{\text{new}} = T_{\text{old}} \times \frac{\text{실제 소요 시간}}{\text{목표 기간}}$$

으로 놓습니다. 결과가 가장 쉬운 목표값 `powLimit` 보다 크면 그 값으로 막습니다.[12] (`powLimit` 은

`0x00000000ffff…ffff` 로, 제네시스의 목표값 $T_{\max}$ 와는 compact 표기로 줄였을 때 같은 `0x1d00ffff` 가 됩니다. 이 책은 둘을 $T_{\max}$ 로 함께 부릅니다.) 기호 풀이: 목표 기간은 1,209,600초(14일), 실제 소요 시간은 구간 첫 블록과 끝 블록의 헤더 시각 차이입니다. 수치 예: 2주짜리 구간이 1주에 끝났으면 블록이 두 배 빨리 나온 것이니 $T_{\text{new}} = T_{\text{old}} / 2$, 난이도는 두 배가 됩니다. 상한 때문에 한 번의 조정으로 난이도가 4배 넘게 오르거나 ¼ 밑으로 내리지는 않습니다. 해시율이 갑자기 100배 늘어도 난이도는 4배씩 몇 차례에 걸쳐 따라갑니다.

[중급] 교과서가 지적하는 세부가 하나 있습니다. 「실제 소요 시간」은 구간의 첫 블록과 마지막 블록의 시각 차이인데, 2,016블록 구간에서 그 차이는 블록 **2,015개 분의 간격입니다**. 원래 소프트웨어의 이 어긋남 때문에 조정이 난이도를 높이는 쪽으로 0.05% 치우친다고 *Mastering Bitcoin* 은 적습니다.[13] 합의 규칙은 한번 정해지면 고치기가 매우 어려워서, 이런 사소한 어긋남도 그대로 남습니다.

조정 규칙의 알려진 약점 — 타임워프

[중급] 헤더의 시각은 채굴자가 적는 값이고 「대략의 시각」일 뿐입니다. 조정 규칙이 그 시각에 의존하므로, 시각을 교묘하게 적으면 조정을 속일 여지가 생깁니다. BIP-54(「Consensus Cleanup」)는 **타임워프(timewarp) 버그를** 이렇게 설명합니다. 「해시율 다수를 가진 공격자가 채굴 난이도를 임의로 낮추고, 따라서 블록 생성률을 임의로 높일 수 있게 한다. 최악의 경우 공격자는 공격 시작 뒤 38일 안에 난이도를 최솟값까지 끌어내릴 수 있다.」 제안은 조정 구간 첫 블록($N \bmod 2016 = 0$)의 시각이 직전 블록 시각보다 7,200초 넘게 **이른(과거의)** 값이 되지 못하게 하는 규칙($T_N \geq T_{N-1} - 7200$)과, 구간 마지막 블록의 시각이 구간 첫 블록 시각보다 이르지 못하게 하는 규칙을 함께 두어 이를 막습니다.[14] 2026-10-25 기준 BIP-54의 상태는 「Complete」이며 활성화(「Deployed」) 표시는 아닙니다. 부록 A에서 갱신합니다. 여기서 기억할 것은 하나입니다. 작업증명의 안전은 「해시가 어렵다」만이 아니라 **조정 규칙의 세부에도** 걸려 있고, 그 세부는 지금도 고쳐지는 중입니다.

4. 코드 — 손 채굴기

[중급] 7장의 장난감 체인에 한 줄을 더합니다. 「헤더 해시의 앞 k 비트가 0이 될 때까지 논스를 올린다.」 실행 시간은 기계마다 다르므로 싣지 않고, **시도 횟수만** 셉니다. 횟수는 하드웨어와 무관합니다.

```
# NB08-1: 손 채굴기 — 앞 k 비트가 0인 해시를 찾기까지 논스를 몇 번 바꾸는가 (공부용, 난이도는 장난감 수준)
import hashlib

def mine(header_prefix: bytes, bits: int):
    """논스를 0부터 올리며 SHA256(SHA256(prefix||nonce))의 앞 bits 비트가 0인 논스를 찾는다. (논스, 시도
횟수, 해시) 반환."""
    target = 1 << (256 - bits)                # 「앞 bits 비트가 0」 ⇔ 해시 < 2^(256-bits)
    nonce = 0
    while True:
        h = hashlib.sha256(hashlib.sha256(header_prefix + nonce.to_bytes(4,
"little")).digest()).digest()
        if int.from_bytes(h, "big") < target:
            return nonce, nonce + 1, h
        nonce += 1

prefix = b"block#1|prev=00..|merkle=ab..|time=1231006505|"
print(f"{'앞 0비트 k':>8} {'찾은 논스':>10} {'시도 횟수':>10} {'기대 2^k':>10} {'해시 앞 8자':>10}")
for bits in [4, 8, 12, 16, 20]:
    nonce, tries, h = mine(prefix, bits)
    print(f"{bits:>8} {nonce:>10,} {tries:>10,} {2**bits:>10,} {h.hex()[:8]:>10}")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
 앞 0비트 k      찾은 논스      시도 횟수      기대 2^k    해시 앞 8자
       4          3          4          16   04468f48
       8        329        330         256   004737bc
      12      6,665      6,666       4,096   000c395c
      16     24,541     24,542      65,536   00002bbf
      20    978,082    978,083   1,048,576   00000fc3
```

[중급] 시도 횟수는 2^k 근처에서 크게 흔들립니다. 운이 좋으면 기대값의 몇 분의 일, 나쁘면 몇 배입니다. 기하분포의 성질입니다. 같은 난이도로 여러 번 캐서 분포를 봅니다.

200번 캐기 — 시도 횟수의 분포

```
# NB08-2: 같은 난이도(k=12)로 200개 블록을 캐면 — 시도 횟수의 분포(기하분포)
import math
bits = 12
tries_list = []
for i in range(200):
    _, tries, _ = mine(f"block#{i}|".encode() + prefix, bits)
    tries_list.append(tries)
tries_list.sort()
n = len(tries_list)
mean = sum(tries_list) / n
print(f"k={bits}: 기대 시도 횟수 2^k = {2**bits:,}")
print(f"200회 — 평균 {mean:,.0f}, 중앙값 {tries_list[n//2]:,}, 최소 {tries_list[0]:,}, 최대
{tries_list[-1]:,}")
print(f"기하분포 이론: 평균 {2**bits:,}, 중앙값 ≈ ln2·2^k = {math.log(2)*2**bits:,.0f}, 표준편차 ≈ 2^k
= {2**bits:,}")
quick = sum(1 for t in tries_list if t <= 2**bits * 0.1)
slow = sum(1 for t in tries_list if t >= 2**bits * 3)
print(f"기대값의 1/10 안에 끝난 블록: {quick}개 ({quick/2:.0f}%) | 기대값의 3배 이상 걸린 블록: {slow}개
({slow/2:.0f}%)")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
k=12: 기대 시도 횟수 2^k = 4,096
200회 — 평균 4,061, 중앙값 3,058, 최소 27, 최대 25,448
기하분포 이론: 평균 4,096, 중앙값 ≈ ln2·2^k = 2,839, 표준편차 ≈ 2^k = 4,096
기대값의 1/10 안에 끝난 블록: 23개 (12%) | 기대값의 3배 이상 걸린 블록: 9개 (4%)
```

[중급] 평균이 2^k 근처이지만 흩어짐이 평균만큼 큽니다. 비트코인에서 「10분」은 평균이고, 실제 블록 간격은 같은 꼴로 크게 흩어집니다(심화 상자의 지수분포). 이 흩어짐은 결함이 아니라 복권의 본성입니다. 그리고 바로 이 흩어짐이 9장에서 공격자에게 기회를 줍니다.

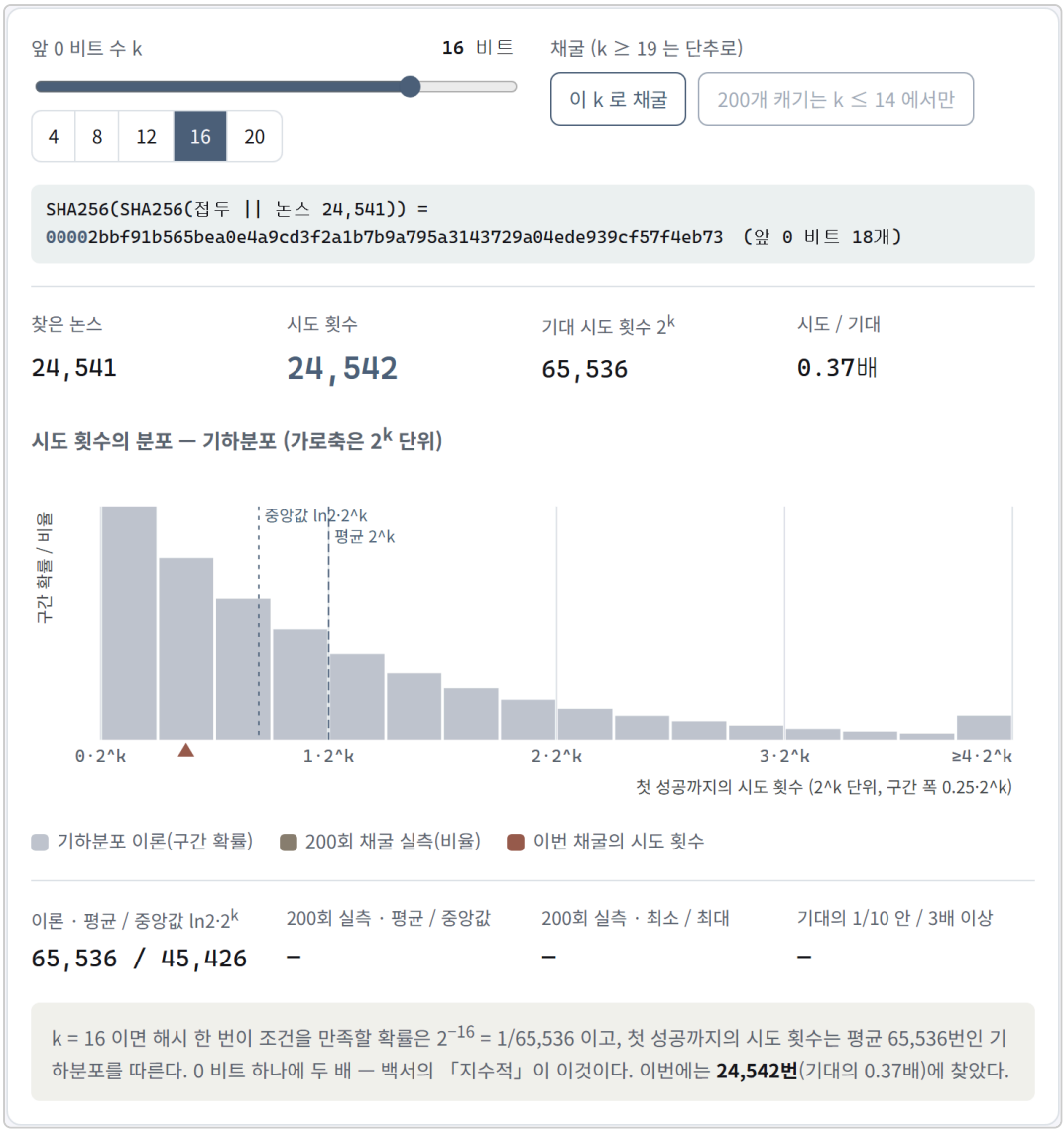


그림 8-2. 손 채굴기. 앞 16비트가 0인 해시를 찾기까지 논스를 0부터 올리면 24,541에서 찾는다(시도 24,542회, 기대 65,536회). 시도 횟수는 기하분포를 따르고 흩어짐은 평균만큼 크다. 실행 시간은 기계마다 다르므로 표시하지 않는다. — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 8장 4절).

nBits에서 목표값·난이도로

```
# NB08-3: nBits → 목표값 → 난이도 → 해시율 추정. 제네시스(0x1d00ffff)와 2026-10-25 체인 끝(0x17020093)
import math
def nbits_to_target(nbits: int) -> int:
    exponent, coefficient = nbits >> 24, nbits & 0x007FFFFF
    return coefficient * 256 ** (exponent - 3)

T_MAX = nbits_to_target(0x1d00ffff)              # 가장 쉬운 목표값(제네시스의 nBits; powLimit 을
compact 로 줄인 값과 같다)
print("T_max = 0x%064x" % T_MAX)
print("난이도 1에서 블록당 기대 해시 수 2^256/T_max = %.6f × 2^32" % (2**256 / T_MAX / 2**32))

for name, nbits in [("제네시스(2009-01-03)", 0x1d00ffff), ("높이 972,958 (2026-10-25 기준)",
0x17020093)]:
    T = nbits_to_target(nbits)
    D = T_MAX / T
    p = (T + 1) / 2**256
    print(f"\n{name}: nBits = 0x{nbits:08x}")
    print("  목표값 T = 0x%064x" % T)
    print("  해시 한 번의 성공 확률 p = %.3e  (앞 0비트 ≈ %.1f개)" % (p, 256 - math.log2(T + 1)))
    print("  난이도 D = T_max/T = %.1f" % D)
    print("  블록당 기대 해시 수 ≈ D·2^32 = %.3e" % (D * 2**32))
    H = D * 2**32 / 600
    print("  600초 가정 해시율 추정 D·2^32/600 = %.4e H/s" % H + (" = %.0f EH/s" % (H / 1e18) if H
>= 1e15 else " = %.1f MH/s" % (H / 1e6)))
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
T_max = 0x00000000ffff0000000000000000000000000000000000000000000000000000
난이도 1에서 블록당 기대 해시 수 2^256/T_max = 1.000015 × 2^32

제네시스(2009-01-03): nBits = 0x1d00ffff
  목표값 T = 0x00000000ffff0000000000000000000000000000000000000000000000000000
  해시 한 번의 성공 확률 p = 2.328e-10  (앞 0비트 ≈ 32.0개)
  난이도 D = T_max/T = 1.0
  블록당 기대 해시 수 ≈ D·2^32 = 4.295e+09
  600초 가정 해시율 추정 D·2^32/600 = 7.1583e+06 H/s = 7.2 MH/s

높이 972,958 (2026-10-25 기준): nBits = 0x17020093
  목표값 T = 0x0000000000000000000200930000000000000000000000000000000000000000
  해시 한 번의 성공 확률 p = 1.656e-24  (앞 0비트 ≈ 79.0개)
  난이도 D = T_max/T = 140577680051919.6
  블록당 기대 해시 수 ≈ D·2^32 = 6.038e+23
  600초 가정 해시율 추정 D·2^32/600 = 1.0063e+21 H/s = 1006 EH/s
```

[중급] 제네시스 때는 해시 한 번의 성공 확률이 2^{-32} 쯤이었고, 2026년 10월에는 그 10^{14} 분의 1입니다. 난이도로 환산하면 네트워크의 연산력이 10^{14} 배가 된 셈인데, 목표 간격은 그때도 지금도 10분입니다. 조정 규칙이 그렇게 만들었습니다(「600초 가정」의 추정이지 측정은 아닙니다).

난이도 조정 규칙

```
# NB08-4: 조정 규칙 시뮬레이션 — 해시율이 갑자기 16배가 되면 난이도는 몇 번의 조정으로 따라가나 (Core 규칙 그대로)
TIMESPAN, SPACING, INTERVAL = 1_209_600, 600, 2016      # chainparams.cpp·params.h 의 값
def retarget(T_old: int, actual_timespan: int) -> int:
    """pow.cpp CalculateNextWorkRequired: 소요 시간을 [timespan/4, timespan*4]로 자르고 비례 조정,
powLimit 상한."""
    actual = max(TIMESPAN // 4, min(TIMESPAN * 4, actual_timespan))
    T_new = T_old * actual // TIMESPAN
    return min(T_new, T_MAX)

T = T_MAX // 1000                       # 어떤 출발 난이도(1000)에서
H1 = 2**256 / (T + 1) / 600             # 그 난이도에서 정확히 600초/블록이 나오는 해시율(해시/초)을 기준 1로
둔다
hashrate = 1.0
print(f"{'조정 회차':>6} {'난이도':>10} {'구간 평균 블록 간격(초)':>22} {'다음 난이도':>12}")
for period in range(1, 7):
    if period == 2:
        hashrate = 16.0                 # 2회차 구간부터 해시율이 16배로 뛴다
    D = T_MAX / T
    expected_hashes_per_block = 2**256 / (T + 1)
    seconds_per_block = expected_hashes_per_block / (hashrate * H1)
    actual_timespan = int(seconds_per_block * INTERVAL)
    T_next = retarget(T, actual_timespan)
    print(f"{period:>6} {D:>10,.0f} {seconds_per_block:>22,.0f} {T_MAX / T_next:>12,.0f}")
    T = T_next
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
 조정 회차        난이도          구간 평균 블록 간격(초)       다음 난이도
     1      1,000                     600        1,000
     2      1,000                      38        4,000
     3      4,000                     150       16,000
     4     16,000                     600       16,000
     5     16,000                     600       16,000
     6     16,000                     600       16,000
```

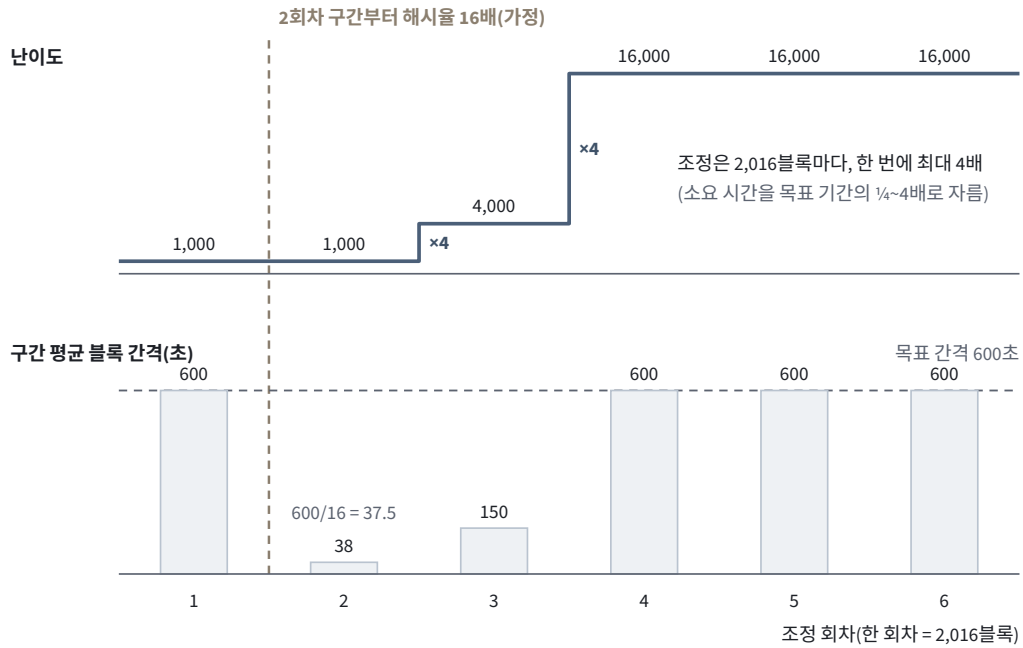


그림 8-3. Bitcoin Core 조정 규칙의 시뮬레이션(NB08-4 출력). 해시율 16배는 코드의 가정이고 실제 체인의 관측값이 아니다. 조정이 한 번에 4배까지만 오르므로 난이도가 두 번에 걸쳐 16배가 되고 블록 간격이 600초로 돌아온다.

[중급] 해시율이 한 번에 16배가 되면 블록이 약 38초(600/16 = 37.5초)마다 나오지만, 조정은 한 번에 4배까지만 올릴 수 있으므로 두 번의 조정(4배 × 4배)을 거쳐 난이도가 16배가 되고 간격이 600초로 돌아옵니다. 그 뒤로는 평형입니다. 상하한은 조정이 한 번의 우연한 구간에 과하게 반응하는 것을 막는 완충 장치이고, 그 대가로 급변에 대한 반응이 느립니다.

5. 「가장 긴 체인」과 점유율 q

작업증명은 두 가지 일을 합니다. 첫째, 7장에서 비어 있던 자리, **「두 사슬 중 어느 것이 역사인가」에** 답합니다. 백서 §4: 「작업증명은 다수결에서 대표성을 정하는 문제도 해결한다. 다수가 IP 주소 하나에 한 표로 정해진다면 많은 IP를 할당할 수 있는 누구에게나 뒤집힐 수 있다. 작업증명은 본질적으로 **CPU 하나에 한 표다**. 다수의 결정은 가장 많은 작업증명 노력이 투입된 가장 긴 체인으로 대표된다.」[15] 둘째, 변조를 비싸게 만듭니다. 「과거 블록을 바꾸려면 공격자는 그 블록과 그 뒤 모든 블록의 작업증명을 다시 하고, 그러고도 정직한 노드들의 작업을 따라잡고 앞질러야 한다.」[16]

여기서 보안 변수가 무엇인지가 드러납니다. 「정직한 노드들의 작업을 따라잡고 앞질러야 한다」는 조건은 **공격자의 해시율과 나머지의 해시율의 비에** 걸려 있습니다. 공격자 해시율이 전체의 q, 나머지가 $p = 1 - q$ 라고 하면, 다음 블록을 공격자가 찾을 확률이 q 입니다. 전체 해시율이 10배가 되어도 q 가 같으면 이 확률은 같고, 난이도 조정이 블록 간격을 10분으로 되돌리므로 「블록 수로 센 경주」의 모양도 같습니다. 절대 속도가 개입하는 유일한 경로는 조정 규칙이고, 조정은 q 를 바꾸지 않습니다. 그래서 「컴퓨터가 빨라지면 블록체인이 뚫린다」는 명제는 그 자체로는 틀렸습니다. 뚫리는 조건은 「누군가 전체의 큰 몫을 **혼자** 갖는가」입니다. 이 경주의 수학은 다음 장에서 전부 풉니다.

채굴자는 블록에 어떤 거래를 넣을지 고릅니다. 백서 §5의 다섯째 단계는 「노드는 블록 안의 모든 거래가 유효하고 이미 쓰이지 않은 것일 때에만 그 블록을 받아들인다」이고,[17] §6은 블록의 첫 거래로 새 코인을 블록 생성자에게 주는 「인센티브」와 거래 수수료를 설명합니다.[18] 채굴자는 수수료를 받고 거래를 실어 주지만, 어느 거래를 싣고 어느 거래를 미룰지는 규칙이 강제하지 않습니다. 이 사실이 17장과 21장의 출발점이고, 다음 질문의 배경입니다. 전기 요금·채굴 수익성 같은 계산은 이 책의 범위 밖입니다(3장 일러두기). 에너지 논쟁은 수치 없이 구조만 부록 A에서 다룹니다.

> **? 법률가의 질문**
>
> 채굴자는 누구의 대리인입니까. 거래를 보낸 사람의 대리인도, 받는 사람의 대리인도 아니며, 어떤 계약으로도 묶여 있지 않습니다. 유효한 거래를 블록에 넣지 않은 채굴자에게 의무가 있습니까. 특정 주소의 거래를 모두 거부하는 채굴자 집단이 있다면 그것은 누구에 대한 어떤 행위입니까. 「CPU 하나에 한 표」라는 다수결에 법적 의미가 있다면, 그 다수는 어떤 단체입니까. → 28장

▾ **[심화] 블록 도착은 포아송 과정이다**

[심화] 해시 한 번의 성공 확률 p 가 아주 작고 시도가 초당 아주 많이 일어나면, 「시간 t 안에 블록이 나올 확률」은 기하분포의 연속 극한인 **지수분포를** 따릅니다. 블록 간격 T 에 대해

$$P(T > t) = e^{-t\,/\,t_0}, \qquad t_0 = 600\text{초}$$

입니다. 기호 풀이: t_0 는 평균 블록 간격이고, 조정 규칙이 이것을 600초로 맞춥니다. 수치 예: 10분 안에 블록이 나올 확률은 $1 - e^{-1} \approx 63\%$, 30분 넘게 안 나올 확률은 $e^{-3} \approx 5\%$ 입니다. 기억 없음은 식에서 바로 나옵니다. $P(T > t_1 + t_2 \mid T > t_2) = P(T > t_1)$.[19] 간격이 지수분포이면 「주어진 시간 안의 블록 수」는 **포아송 분포를** 따르고, 「블록 z 개가 나오는 데 걸리는 시간」은 **감마 분포를** 따릅니다. 백서 §11은 공격자의 진척을 포아송으로 근사하면서 정직 체인이 「평균 시간에 정확히」 z 블록을 캤다고 단순화했는데, 그 단순화를 감마 분포로 풀어낸 것이 그런스판과 페레스마르코의 정확식입니다. 9장에서 두 결과를 나란히 계산합니다.

6. 정리

작업증명은 스팸 방지에서 왔고, 비트코인은 거기에 「네트워크가 스스로 조정하는 난이도」를 더했습니다. 채굴은 복권이라 시도마다 확률이 같고 기억이 없으며, 당첨 간격은 조정 규칙이 평균 10분으로 되돌립니다. 그래서 변조의 비용은 계산량으로 매겨지고, 안전은 절대 속도가 아니라 **점유율** q 에 걸립니다. 7장의 빈칸, 「두 사슬 중 어느 것이 역사인가」는 「더 많은 작업이 쌓인 쪽」으로 답해졌습니다. 그 답이 확률적이라는 것, 즉 「역사」가 시간이 갈수록 굳어지지만 결코 완전히 굳지는 않는다는 것이 다음 장입니다.

이 장의 출처

- Adam Back, "Hashcash — A Denial of Service Counter-Measure," 기술보고서, 2002-08-01. http://www.hashcash.org/papers/hashcash.pdf — 목적(1쪽), 비용 함수 정의와 이중 지출 데이터베이스(3쪽), 동적 조정(4쪽). 최초 공지: cypherpunks 1997-03-28, http://www.hashcash.org/papers/announce.txt.
- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — 작업증명 정의·CPU 하나에 한 표·난이도 조정(§4, 3쪽), 네트워크 단계(§5, 3쪽), 인센티브(§6, 4쪽).
- Bitcoin Core v31.1 (태그 커밋 9be056a8a72b624dae9623b2f7bded92c2a21c91): `src/kernel/chainparams.cpp`, `src/consensus/params.h`, `src/pow.cpp` — 조정 간격·목표 기간·상하한·powLimit. 독립 구현 대조: btcd 커밋 dd169fe0, `chaincfg/params.go`.
- Antoine Poinsot & Matt Corallo, "Consensus Cleanup," BIP 54 (Status: Complete, 2026-10-25 접근) — 타임워프 버그와 7,200초 규칙.
- Cyril Grunspan & Ricardo Pérez-Marco, "Double Spend Races," *IJTAF* 21(8): 1850053, 2018; arXiv:1702.02867 — 채굴 시간의 기억 없음(§2).
- Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed., O'Reilly, 2023 (CC BY-SA 4.0, GitHub 원고 커밋 94c9ba64) — nBits 변환식과 2,015블록 어긋남(12장).
- 동봉 데이터 `data/hashrate_2026-10-25.json` (기준일 2026-10-25): 블록 971,712·972,958 헤더, 난이도, 추정 해시율. 출처·산식·한계는 데이터 폴더의 README.
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `hashlib`·`math` 만 사용. 노트북 NB08.

각주

1. Adam Back, "[ANNOUNCE] hash cash postage implementation," cypherpunks 메일링리스트, 1997-03-28(저자 사이트 사본 http://www.hashcash.org/papers/announce.txt; 같은 사이트의 논문 목록은 「Mar 1997 - hashcash package 1st announced」). 2002년 보고서 본문은 최초 제안을 「May 1997」로 적어 두 달이 어긋납니다. 이 책은 공지 메일의 날짜를 따르고 불일치를 여기에 밝힙니다. 2026-10-25 접근.

2. Adam Back, "Hashcash — A Denial of Service Counter-Measure," 기술보고서, 2002-08-01, 1쪽. 원문: "Hashcash was originally proposed as a mechanism to throttle systematic abuse of un-metered internet resources such as email, and anonymous remailers in May 1997." http://www.hashcash.org/papers/hashcash.pdf.

3. Back (2002), 3쪽(§3 The Hashcash cost-function). 원문: "Hashcash is a non-interactive, publicly auditable, trapdoor-free cost function with unbounded probabilistic cost." "The hashcash cost-function is based on finding partial hash collisions on the all 0 bits k-bit string 0^k. The fastest algorithm for computing partial collisions is brute force."

4. Back (2002), 3쪽. 원문: "The server needs to keep a double spending database of spent tokens, to detect and reject attempts to spend the same token again."

5. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), 3쪽(§4 Proof-of-Work). 원문: "To implement a distributed timestamp server on a peer-to-peer basis, we will need to use a proof-of-work system similar to Adam Back's Hashcash [6], rather than newspaper or Usenet posts." "To compensate for increasing hardware speed and varying interest in running nodes over time, the proof-of-work difficulty is determined by a moving average targeting an average number of blocks per hour. If they're generated too fast, the difficulty increases."

6. Back (2002), 4쪽(§4.1 Dynamic throttling). 원문: "With interactive hashcash it becomes possible to dynamically adjust the work factor required for the client based on server CPU load." 비대화형(메일) 변종에 관해서는 보고서가 조정 장치를 서술하지 않으며(3쪽: 서버가 도전을 보낼 통로가 없다), 「조정 장치가 없다」는 이 책의 읽기입니다.

7. Nakamoto (2008), 3쪽. 원문: "The proof-of-work involves scanning for a value that when hashed, such as with SHA-256, the hash begins with a number of zero bits. The average work required is exponential in the number of zero bits required and can be verified by executing a single hash."

8. Cyril Grunspan & Ricardo Pérez-Marco, "Double Spend Races," *International Journal of Theoretical and Applied Finance* 21(8): 1850053 (2018), §2(arXiv:1702.02867v3, 5쪽). 원문: "For each new hash the work is started from scratch, therefore the random variable T measuring the time it takes to mine a block is memoryless ...".

9. Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed. (O'Reilly, 2023), 12장 — "target = coefficient × 2^(8 × (exponent − 3))". 예로 블록 277,316의 `0x1903a30c` 를 풀어 보입니다. GitHub 원고 커밋 94c9ba64, `ch12_mining.adoc`.

10. 이 책의 동봉 데이터 `data/hashrate_2026-10-25.json` (기준일 2026-10-25, 접근 UTC 2026-10-24 18:17). 블록 971,712(직전 조정 블록)와 972,958(관측 시점 체인 끝)의 헤더 원시 바이트를 두 탐색기(Blockstream Esplora·mempool.space)에서 받아 바이트 일치를 확인했고, nBits `0x17020093`, 난이도 140,577,680,051,919.6을 헤더에서 직접 다시 계산했습니다. 추정 해시율 $D \cdot 2^{32} / 600 \approx 1.006 \times 10^{21}$ H/s(약 1,006 EH/s). 같은 시각 mempool.space가 다른 방법으로 낸 비교값은 1.077×10^{21} H/s였고, 두 값 모두 참값이 아닙니다. 이 값들은 판마다 바뀌므로 본문에는 크기 수준만 적고 부록 A에서 갱신합니다.

11. Bitcoin Core v31.1(태그 커밋 9be056a8a72b624dae9623b2f7bded92c2a21c91), `src/kernel/chainparams.cpp` 메인넷: `consensus.nPowTargetTimespan = 14 * 24 * 60 * 60; // two weeks`, `consensus.nPowTargetSpacing = 10 * 60;`, `consensus.powLimit = uint256{"00000000ffff…ffff"}`; `src/consensus/params.h`: `int64_t DifficultyAdjustmentInterval() const { return nPowTargetTimespan / nPowTargetSpacing; }`. 2026-10-25 접근. 독립 구현 btcd(커밋 dd169fe0)의 `chaincfg/params.go` 도 TargetTimespan 14일·TargetTimePerBlock 10분·RetargetAdjustmentFactor 4로 같습니다.

12. 같은 판, `src/pow.cpp`, 함수 `CalculateNextWorkRequired`: `int64_t nActualTimespan = pindexLast->GetBlockTime() - nFirstBlockTime; if (nActualTimespan < params.nPowTargetTimespan/4) nActualTimespan = params.nPowTargetTimespan/4; if (nActualTimespan > params.nPowTargetTimespan*4) nActualTimespan = params.nPowTargetTimespan*4;` 이어(메인넷이 쓰는 `else` 분기 — `if (params.enforce_BIP94)` 의 Testnet4 분기는 생략) `bnNew.SetCompact(pindexLast->nBits); bnNew *=`

`nActualTimespan; bnNew /= params.nPowTargetTimespan; if (bnNew > bnPowLimit) bnNew = bnPowLimit;`. 조정 시점은 `GetNextWorkRequired` 가 `(pindexLast->nHeight+1) % params.DifficultyAdjustmentInterval() != 0` 이면 직전 nBits를 그대로 돌려주는 것으로 정해집니다.

13. *Mastering Bitcoin*, 3rd ed., 12장 — "While the target calibration happens every 2,016 blocks, because of an off-by-one error in the original Bitcoin software, it is based on the total time of the previous 2,015 blocks (not 2,016 as it should be), resulting in a retargeting bias toward higher difficulty by 0.05%." 같은 곳의 요약식 "New Target = Old Target * (20,160 minutes / Actual Time of Last 2015 Blocks)" 은 비율이 뒤집혀 있어 Core 코드(`bnNew *= nActualTimespan; bnNew /= nPowTargetTimespan`)와 반대입니다. 이 책은 코드를 정본으로 삼습니다.

14. Antoine Poinsot & Matt Corallo, "Consensus Cleanup," BIP 54 (Status: Complete), Motivation·Specification 절. 원문: "The timewarp bug makes it possible for a majority-hashrate attacker to arbitrarily lower mining difficulty, and therefore arbitrarily increase the block rate. In the worst case, an attacker can bring down the difficulty to its minimum within 38 days of starting the attack." 규칙: "if N % 2016 is equal to 0, the timestamp of the block must be set to a value higher than or equal to the value of the timestamp of block at height N-1 minus 7200"; 둘째 규칙은 N % 2016 = 2015 인 블록의 시각이 N−2015 블록의 시각 이상이어야 한다는 것(같은 Specification 절). github.com/bitcoin/bips 파일 커밋 927b6de9, 2026-10-25 접근.

15. Nakamoto (2008), 3쪽. 원문: "The proof-of-work also solves the problem of determining representation in majority decision making. If the majority were based on one-IP-address-one-vote, it could be subverted by anyone able to allocate many IPs. Proof-of-work is essentially one-CPU-one-vote. The majority decision is represented by the longest chain, which has the greatest proof-of-work effort invested in it."

16. Nakamoto (2008), 3쪽. 원문: "To modify a past block, an attacker would have to redo the proof-of-work of the block and all blocks after it and then catch up with and surpass the work of the honest nodes."

17. Nakamoto (2008), 3쪽(§5). 원문: "5) Nodes accept the block only if all transactions in it are valid and not already spent."

18. Nakamoto (2008), 4쪽(§6 Incentive). 원문: "By convention, the first transaction in a block is a special transaction that starts a new coin owned by the creator of the block." "The incentive can also be funded with transaction fees."

19. Grunspan & Pérez-Marco (2018), §2(arXiv v3, 5쪽). 원문: "P[T > t1 + t2 | T > t2] = P[T > t1]. Therefore we have P[T > t1 + t2] = P[T > t1 + t2|T > t2].P[T > t2] = P[T > t1].P[T > t2]."

1부 기술 기초

9장 최종성의 확률적 성격 — 「확인 수」의 뜻

핵심 TAKEAWAY

블록체인에는 「완료」의 순간이 없습니다. 어떤 거래가 블록에 들어간 뒤 그 위에 블록이 z 개 더 쌓이면, 그 거래를 되돌리려는 공격자가 성공할 확률이 z 에 대해 지수적으로 작아질 뿐입니다. 그 확률은 공격자의 해시율 점유율 q 와 z 만의 함수이고, 절대 연산속도나 기다린 **시간은** 들어가지 않습니다. 「6 확인」은 규칙이 아니라 관행입니다. $q = 0.1$ 일 때 6 확인이 뒤집힐 확률은 백서의 근사식으로 0.02%, 정확식으로 0.06%이며, q 가 0.5에 가까워질수록 같은 안전 수준에 필요한 확인 수가 끝없이 커지고, $q \geq 0.5$ 이면 어떤 확인 수로도 안전해지지 않습니다. 이 장을 읽으면 「지급은 언제 완료되었는가」라는 법률 질문이 기술적으로는 확률 하나로 번역된다는 것을 압니다.

학습목표

1. 백서 §11의 공격자 추월 확률을 유도 과정 없이도 표로 읽고, q 와 z 를 바꾸면 어떻게 변하는지 본다.
2. [중급] 도박꾼의 파산 식 $(q/p)^z$ 를 유도하고, 백서의 포아송 근사가 어디서 들어오는지 짚는다.
3. [심화] 후속 문헌(Rosenfeld 2014, Grunspan–Pérez-Marco 2018)이 백서의 근사를 어떻게 정정했는지, 그 정정이 안전 확인 수를 얼마나 늘리는지 계산한다.
4. 「6 확인」이 관행이지 규칙이 아님을, 그리고 보안 변수가 절대 속도가 아니라 점유율 q 임을 코드로 확인한다.

[입문]**만 읽은 독자가 가져가는 것.** 블록체인에는 「완료」의 순간이 없습니다. 되돌릴 확률이 작아질 뿐입니다. 부록 B 오해 ⑦(「6 확인이면 확정이다」)의 근거입니다.

1. 공격자는 무엇을 할 수 있고 무엇을 할 수 없나

먼저 범위를 정합니다. 해시율 다수를 가진 공격자라도 **마음대로 할 수는 없습니다.** 백서 §11의 첫 문단입니다. 「정직한 체인보다 빠르게 대체 체인을 만들려는 공격자를 생각하자. 이것이 성공한다고 해도 체제가 아무 변경에나 열리는 것은 아니다. 없는 가치를 만들어 내거나 공격자 소유가 아니었던 돈을 가져가는 것 같은 변경 말이다. 노드들은 유효하지 않은 거래를 지급으로 받아들이지 않을 것이고, 정직한 노드들은 그런 거래를 담은 블록을 결코 받아들이지 않을 것이다. 공격자가 시도할 수 있는 것은 **자기 자신의 거래 하나를 바꿔 최근에 쓴 돈을 되찾는 것뿐이다.**」[1] 서명 규칙(5장)은 해시율로 깨지지 않습니다. 해시율로 할 수 있는 일은 **역사의 순서를 바꾸는 것입니다.** 자기가 보낸 거래 대신 같은 코인을 자기에게 돌려보내는 거래를 담은 다른 가지를 만들어, 그 가지가 더 길어지게 하는 것. 이것이 **이중지불(double spending)** 공격입니다.

로젠펠드(Meni Rosenfeld)는 공격의 순서를 네 단계로 적습니다. 상인에게 지급하는 거래를 네트워크에 내놓는다. 그 거래가 블록에 들어가기 전의 최신 블록 위에, 같은 코인을 자기에게 보내는 거래를 담은 가지를 **비밀리에** 캔다. 상인이 충분한 확인 수를 보고 물건을 보낼 때까지 기다린다. 비밀 가지가 공개 가지보다 길어지면 공개한다. 더 긴 가지가 유효한 것으로 여겨지므로 상인에게 간 지급은 공격자에게 간 지급으로 바뀐다.[2] 상인은 물건도 코인도 잃고 공격자는 둘 다 갖습니다.

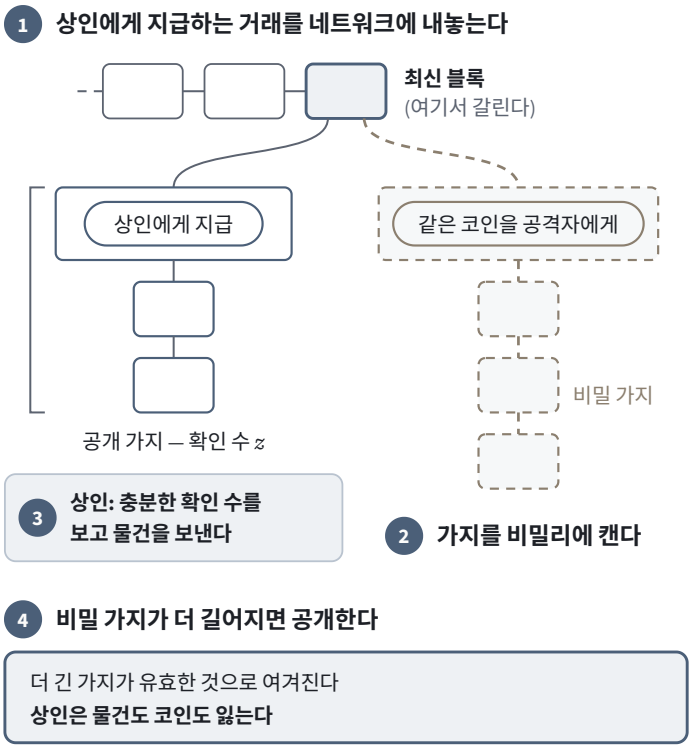


그림 9-1. 이중지불 공격의 네 단계(로젠펠드의 순서). 공개 가지에는 상인에게 지급하는 거래가, 비밀 가지에는 같은 코인을 공격자에게 보내는 거래가 들어 있다.

그러니 상인의 질문은 「이 거래가 확정되었는가」가 아니라 「**몇 블록을 기다리면 뒤집힐 확률이 충분히 작은가**」입니다. 거래가 들어 있는 블록을 포함해 그 위에 쌓인 블록의 수를 **확인 수(confirmations)**라고 부릅니다.[3] 백서 §11은 이 확률을 계산합니다.

2. [중급] 백서 §11의 식 — 두 단계

첫째 단계 — 도박꾼의 파산

[중급] 정직한 채굴자들이 다음 블록을 찾을 확률을 p, 공격자가 찾을 확률을 $q = 1 - p$ 라고 합니다. 블록 하나가 나올 때마다 두 체인의 길이 차이(격차)는 정직 쪽이 찾으면 +1, 공격자가 찾으면 −1 이 됩니다. 백서는 이것을 「이항 랜덤워크」라고 부르고, 공격자가 격차 z 를 따라잡을 확률을 「도박꾼의 파산」 문제에 빗댑니다.[4] 무한한 신용을 가진 도박꾼이 적자에서 출발해 언젠가 본전에 이를 확률입니다.

[중급] 격차 z 에서 언젠가 격차 0에 이를 확률을 q_z 라고 둡니다. $q_0 = 1$ 이고, 다음 블록이 누구 것인가로 조건을 나누면 $q_z = p\,q_{z+1} + q\,q_{z-1}$ 입니다. $q_z = r^z$ 꼴의 해를 넣으면 $pr^2 - r + q = 0$ 이고, 근은 $r = 1$ 과 $r = q/p$ 입니다. $q < p$ 일 때 $z \to \infty$ 에서 0으로 가는 해는 $(q/p)^z$ 뿐이므로

$$q_z = \begin{cases} 1 & p \le q \\ (q/p)^z & p > q \end{cases}$$

입니다.[5] 기호 풀이: q/p 는 공격자 대 정직 해시율의 비이고, z 는 공격자가 따라잡아야 할 블록 수입니다. 수치 예: $q = 0.1$ 이면 $q/p = 1/9$, $z = 6$ 에서 $q_z = 9^{-6} \approx 1.9 \times 10^{-6}$. 이것이 「51% 공격」의 수학입니다. $q \ge 1/2$ 이면 $q_z = 1$, 어떤 격차도 결국 따라잡힙니다. 로젠펠드의 말로 「공격자가 전체 해시율의 절반을 넘게 가지면 어떤 열세에서도 항상 따라잡는다」.[6]

둘째 단계 — 공격자는 이미 얼마나 캤는가

[중급] 상인이 z 확인을 기다리는 동안 공격자도 놀지 않았습니다. 상인이 물건을 보내는 순간 공격자의 비밀 가지가 이미 k 블록이라면 실제 격차는 $z - k$ 입니다. 백서는 k 를 이렇게 다룹니다. 「수취인은 거래가 블록에 추가되고 그 뒤에 z 블록이 이어질 때까지 기다린다. 그는 공격자가 얼마나 진척했는지 정확히 모르지만, 정직한 블록들이 블록당 평균 기대 시간만큼 걸렸다고 가정하면, 공격자의 잠재적 진척은 기댓값 $\lambda = zq/p$ 인 포아송 분포가 된다.」[7] 각 진척 k 의 포아송 확률에 그 지점에서 따라잡을 확률을 곱해 더하고, 무한 꼬리를 피하도록 정리하면

$$P = 1 - \sum_{k=0}^{z} \frac{\lambda^k e^{-\lambda}}{k!}\left(1 - (q/p)^{z-k}\right), \qquad \lambda = z\frac{q}{p}$$

입니다.[8] 기호 풀이: λ 는 정직 체인이 z 블록을 캐는 동안 공격자가 캤을 기대 블록 수, 합의 각 항은 「공격자가 k 개를 캤을 확률 × 그때 못 따라잡을 확률」입니다. 백서는 이 식을 C 코드 `AttackerSuccessProbability(q, z)` 로 옮기고 $q = 0.1$, $q = 0.3$ 의 표를 싣습니다. 수치 예: $q = 0.1$, $z = 6$ 에서 $P = 0.0002428$. 아래 코드가 이 표를 소수점 일곱째 자리까지 재현합니다.

3. 코드 — 백서 표의 재현과 그 너머

```
# NB09-1: 백서 §11 의 식 — C 코드를 그대로 옮겨 논문 표(q=0.1, 0.3)를 재현한다
import math

def nakamoto_P(q: float, z: int) -> float:
    """P = 1 - Σ_{k=0}^{z} λ^k e^{-λ}/k! · (1 - (q/p)^{z-k}),  λ = z·q/p   (Nakamoto 2008,
p.7)"""
    p = 1.0 - q
    lam = z * (q / p)
    total = 1.0
    for k in range(z + 1):
        poisson = math.exp(-lam)
        for i in range(1, k + 1):
            poisson *= lam / i
        total -= poisson * (1 - (q / p) ** (z - k))
    return total

def gamblers_ruin(q: float, z: int) -> float:
    """§11 첫 식 q_z: 공격자가 z 블록 뒤에서 언젠가 따라잡을 확률(포아송 진척을 무시한 값)"""
    p = 1.0 - q
    return 1.0 if p <= q else (q / p) ** z

for q, zs in [(0.1, range(0, 11)), (0.3, range(0, 51, 5))]:
    print(f"q={q}")
    for z in zs:
        print(f"  z={z:<3} P={nakamoto_P(q, z):.7f}   (q_z 도박꾼 파산만 = {gamblers_ruin(q,
z):.7f})")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
q=0.1
  z=0   P=1.0000000   (q_z 도박꾼 파산만 = 1.0000000)
  z=1   P=0.2045873   (q_z 도박꾼 파산만 = 0.1111111)
  z=2   P=0.0509779   (q_z 도박꾼 파산만 = 0.0123457)
  z=3   P=0.0131722   (q_z 도박꾼 파산만 = 0.0013717)
  z=4   P=0.0034552   (q_z 도박꾼 파산만 = 0.0001524)
  z=5   P=0.0009137   (q_z 도박꾼 파산만 = 0.0000169)
  z=6   P=0.0002428   (q_z 도박꾼 파산만 = 0.0000019)
  z=7   P=0.0000647   (q_z 도박꾼 파산만 = 0.0000002)
  z=8   P=0.0000173   (q_z 도박꾼 파산만 = 0.0000000)
  z=9   P=0.0000046   (q_z 도박꾼 파산만 = 0.0000000)
  z=10  P=0.0000012   (q_z 도박꾼 파산만 = 0.0000000)
q=0.3
  z=0   P=1.0000000   (q_z 도박꾼 파산만 = 1.0000000)
  z=5   P=0.1773523   (q_z 도박꾼 파산만 = 0.0144583)
  z=10  P=0.0416605   (q_z 도박꾼 파산만 = 0.0002090)
  z=15  P=0.0101008   (q_z 도박꾼 파산만 = 0.0000030)
  z=20  P=0.0024804   (q_z 도박꾼 파산만 = 0.0000000)
  z=25  P=0.0006132   (q_z 도박꾼 파산만 = 0.0000000)
  z=30  P=0.0001522   (q_z 도박꾼 파산만 = 0.0000000)
  z=35  P=0.0000379   (q_z 도박꾼 파산만 = 0.0000000)
  z=40  P=0.0000095   (q_z 도박꾼 파산만 = 0.0000000)
  z=45  P=0.0000024   (q_z 도박꾼 파산만 = 0.0000000)
  z=50  P=0.0000006   (q_z 도박꾼 파산만 = 0.0000000)
```

[중급] 백서 8쪽의 표와 일곱째 자리까지 같습니다. 오른쪽 열은 포아송 진척을 무시한 도박꾼의 파산만의 값입니다. 둘의 차이가 「상인이 기다리는 동안 공격자도 캤다」는 효과이고, 그 효과는 작지 않습니다. $q = 0.1, z = 6$ 에서 1.9×10^{-6} 이 2.4×10^{-4} 으로 백 배 넘게 커집니다.

「P < 0.1%」에 필요한 확인 수

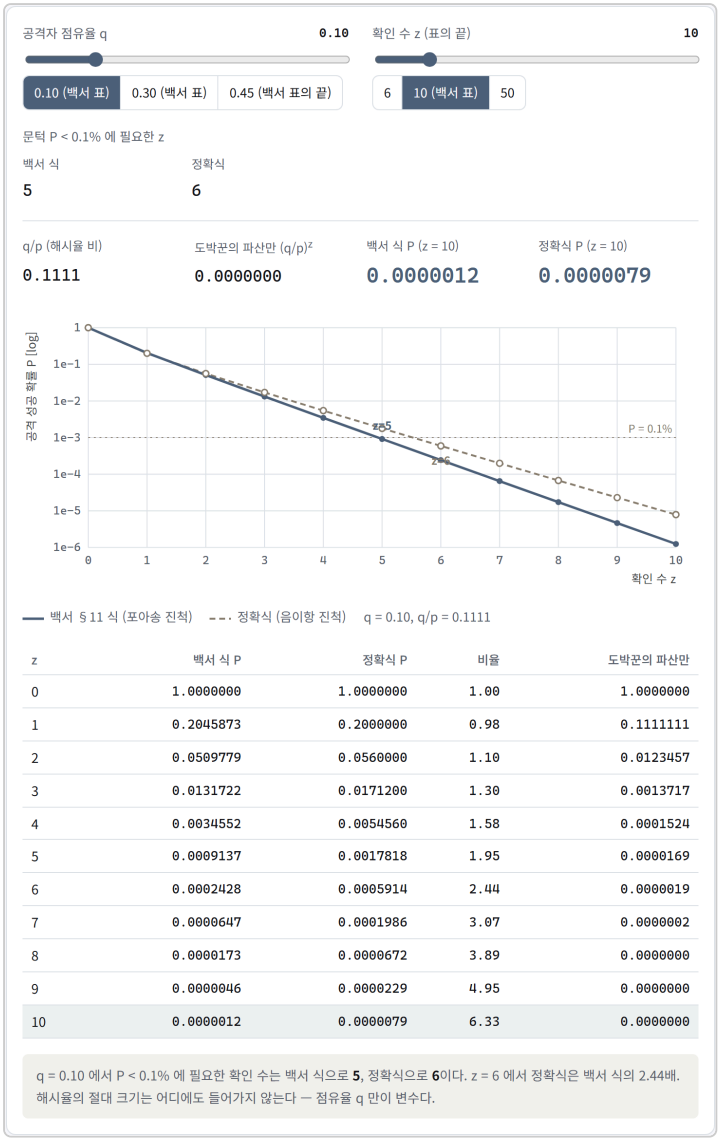
```
# NB09-2: 백서 8쪽 「P < 0.1%」 — 공격자 점유율 q 마다 필요한 최소 확인 수 z
def min_z(P_func, q, threshold=0.001):
    z = 0
    while P_func(q, z) > threshold:
        z += 1
    return z

print("q      z (Nakamoto 식)")
for q in (0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45):
    print(f"{q:.2f}   {min_z(nakamoto_P, q):>4}")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
q      z (Nakamoto 식)
0.10      5
0.15      8
0.20     11
0.25     15
0.30     24
0.35     41
0.40     89
0.45    340
```

[중급] 백서 8쪽의 「P < 0.001」 표(5, 8, 11, 15, 24, 41, 89, 340)와 같습니다. q 가 0.5에 다가가면 $q/p \to 1$ 이 되어 감쇠의 밑이 1에 가까워지고, 필요한 확인 수가 발산합니다. 「6 확인」이 어디서 왔는지도 이 표에 있습니다. $q = 0.1$ 행의 5(백서 식) 또는 6(아래 정확식)입니다. 로젠펠드의 평가는 이렇습니다. 「흔히 인용되는 기본값 6 확인에는 특별한 것이 없다. 그것은 공격자가 해시율의 10%를 넘게 모으기 어렵다는 가정과 0.1% 미만의 무시할 만한 위험을 받아들인다는 가정 위에서 택해졌다. 두

수치 모두 임의적이다. 6 확인은 가벼운 공격자에게는 과하고, 해시율 10%를 훨씬 넘는 작심한 공격자 앞에서는 무력하다.」[9]



z	백서 식 P	정확식 P	비율	도박꾼의 파산만
0	1.0000000	1.0000000	1.00	1.0000000
1	0.2045873	0.2000000	0.98	0.1111111
2	0.0509779	0.0560000	1.10	0.0123457
3	0.0131722	0.0171200	1.30	0.0013717
4	0.0034552	0.0054560	1.58	0.0001524
5	0.0009137	0.0017818	1.95	0.0000169
6	0.0002428	0.0005914	2.44	0.0000019
7	0.0000647	0.0001986	3.07	0.0000002
8	0.0000173	0.0000672	3.89	0.0000000
9	0.0000046	0.0000229	4.95	0.0000000
10	0.0000012	0.0000079	6.33	0.0000000

q = 0.10 에서 P < 0.1% 에 필요한 확인 수는 백서 식으로 **5**, 정확식으로 **6**이다. z = 6 에서 정확식은 백서 식의 2.44배. 해시율의 절대 크기는 어디에도 들어가지 않는다 — 점유율 q 만이 변수다.

그림 9-2. 이중지불 확률. 공격자 점유율 q = 0.1 에서 확인 수 z 에 따른 성공 확률 — 백서 §11의 식(실선)과 음이항 정확식(점선). z = 6 에서 백서 값 0.0002428, 정확식 0.0005914. — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 9장 3절).

[심화] 정정 — 백서의 근사는 낙관적이다

▾ [심화] 포아송이 아니라 음이항 — 정확식과 그 결과

[심화] 백서의 단순화는 「정직 블록 z 개가 평균 시간에 정확히 나왔다」는 가정입니다. 실제로는 그 시간이 흩어집니다(8장 심화 상자 — 감마 분포). 로젠펠드는 2012년 초고에서 이 가정을 버리고, 정직 체인이 n 블록을 캐는 동안 공격자가 캔 블록 수 m 을 **음이항 분포로** 둡니다. 「정직 네트워크의 n 번 실패 전에 공격자가 거둔 성공 횟수」이고 확률은 $P(m) = \binom{m+n-1}{m} p^n q^m$ 입니다.[10] 그런스판과 페레스마르코는 2018년 논문에서 같은 교정을 닫힌 꼴로 냈습니다. 「$0 < q < 1/2$ 를 공격자 집단의 상대 해시파워, $p = 1 - q$ 를 정직 채굴자의 것이라 하자. 정직 채굴자가 z 블록을 검증한 뒤 공격자가 성공할 확률은 $P(z) = I_{4pq}(z, 1/2)$ 이다. 여기서 $I_x(a, b)$ 는 정규화 불완전 베타 함수다.」 그리고 「일반적으로 $z \geq 2$ 에서 이 확률 $P(z)$ 는 나카모토가 얻은 $P_{SN}(z)$ 보다 크다. 비트코인 보안의 관점에서 이것은 나카모토가 제시한 것보다 더 큰 확인 수 z 가 필요함을 뜻한다.」[11]

[심화] 정확식은 특수 함수 없이도 계산할 수 있습니다. 공격자가 캔 블록 수 k 가 음이항이면

$$P(z) = \sum_{k=0}^{z-1} \binom{k+z-1}{k} p^z q^k \left(\frac{q}{p}\right)^{z-k} + \left(1 - \sum_{k=0}^{z-1} \binom{k+z-1}{k} p^z q^k\right)$$

입니다. 기호 풀이: 첫 합은 「공격자가 z 개 미만을 캤을 때 그 격차를 도박꾼의 파산으로 따라잡는 확률」, 둘째 항은 「이미 z 개 이상 캐서 따라잡은 확률」입니다. 수치 예: $q = 0.1$, $z = 6$ 에서 이 값은 0.000591로, 백서의 0.000243보다 2.4배 큽니다. 아래 코드는 이 유한합을 계산해 (1) 로젠펠드 논문 표 1의 $q = 10\%$ 행과, (2) 그런스판·페레스마르코 표 1의 「필요한 z」 행과 맞는지 봅니다.

```
# NB09-3: 정확식 — 음이항 유한합으로 계산해 Rosenfeld 표 1·Grunspan-Pérez-Marco 표 1과 대조 (외부 패키지 없음)
from math import lgamma, log, exp

def exact_P(q: float, z: int) -> float:
    """정직 체인이 z 블록을 캐는 동안 공격자 블록 수 k ~ NegBin; k<z 면 (q/p)^(z-k) 로 따라잡음, k>=z 면 이미 따라잡음.
    이항계수가 커지므로 로그로 계산한다: log C(k+z-1, k) = lgamma(k+z) - lgamma(k+1) - lgamma(z)."""
    if z == 0: return 1.0
    p = 1.0 - q
    if q >= p: return 1.0
    caught, mass = 0.0, 0.0
    for k in range(z):
        log_pk = lgamma(k + z) - lgamma(k + 1) - lgamma(z) + z * log(p) + k * log(q)
        pk = exp(log_pk)
        mass += pk
        caught += pk * (q / p) ** (z - k)
    return caught + (1.0 - mass)

print("Rosenfeld(2014) 표 1 의 q=10% 행: 20%, 5.600%, 1.712%, 0.546%, 0.178%, 0.059%, 0.020%, 0.007%, 0.002%, 0.001%")
print("이 책의 유한합        q=10% 행:", ", ".join(f"{100*exact_P(0.10, n):.3f}%" for n in range(1, 11)))
print()
print("Grunspan-Pérez-Marco(2018) 표 1: P<0.1% 에 필요한 z —  q: 0.10 0.15 0.20 0.25 0.30 0.35 0.40 0.45")
print("  논문의 z(정확식)      : 6 9 13 20 32 58 133 539")
print("  이 책의 유한합        :", " ".join(str(min_z(exact_P, q)) for q in (0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45)))
print("  논문의 z_SN(나카모토) : 5 8 11 15 24 41 81 340   ← q=0.40 의 81 은 백서 8쪽·위 NB09-2 의 89 와 다르다(아래 본문)")
print()
print(f"{'q':>5} {'z':>4} {'Nakamoto':>10} {'정확식':>10} {'비율':>6}")
for q, z in ((0.1, 1), (0.1, 3), (0.1, 6), (0.3, 5), (0.3, 10), (0.3, 24), (0.45, 40), (0.45, 340)):
    a, b = nakamoto_P(q, z), exact_P(q, z)
    print(f"{q:>5.2f} {z:>4} {a:>10.7f} {b:>10.7f} {b/a:>6.2f}")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
Rosenfeld(2014) 표 1 의 q=10% 행: 20%, 5.600%, 1.712%, 0.546%, 0.178%, 0.059%, 0.020%, 0.007%, 0.002%, 0.001%
이 책의 유한합        q=10% 행: 20.000%, 5.600%, 1.712%, 0.546%, 0.178%, 0.059%, 0.020%, 0.007%, 0.002%, 0.001%

Grunspan-Pérez-Marco(2018) 표 1: P<0.1% 에 필요한 z —  q: 0.10 0.15 0.20 0.25 0.30 0.35 0.40 0.45
  논문의 z(정확식)      : 6 9 13 20 32 58 133 539
  이 책의 유한합        : 6 9 13 20 32 58 133 539
  논문의 z_SN(나카모토) : 5 8 11 15 24 41 81 340   ← q=0.40 의 81 은 백서 8쪽·위 NB09-2 의 89 와 다르다(아래 본문)

    q    z   Nakamoto        정확식     비율
 0.10    1  0.2045873  0.2000000   0.98
 0.10    3  0.0131722  0.0171200   1.30
 0.10    6  0.0002428  0.0005914   2.44
 0.30    5  0.1773523  0.1976173   1.11
 0.30   10  0.0416605  0.0651067   1.56
 0.30   24  0.0008106  0.0040007   4.94
 0.45   40  0.3454201  0.3713860   1.08
 0.45  340  0.0009947  0.0089679   9.02
```

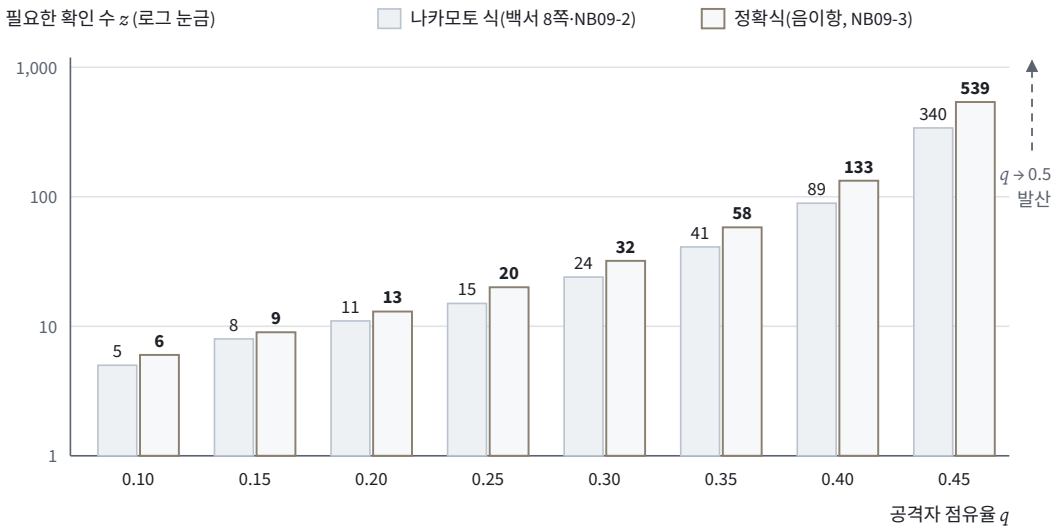


그림 9-3. 공격자 점유율 q 마다 $P < 0.1\%$ 에 필요한 확인 수 z 를 나카모토 식(NB09-2)과 정확식(NB09-3)으로 견준 막대(세로 로그 눈금). 모든 칸에서 정확식이 더 많은 확인을 요구하고, q 가 0.5에 다가가면 필요한 확인 수가 발산한다.

[심화] 유한합이 로젠펠드의 표와 그런스판·페레스마르코의 표를 모두 재현합니다. 두 논문의 모형은 겉보기 설정이 조금 다르지만(로젠펠드는 공격자가 블록 하나를 미리 캐 두고 「더 길어짐」을 성공으로 두고, 그런스판·페레스마르코의 식은 미리 캔 블록 없이 형식상 「동률」을 성공으로 셈합니다) 같은 수를 냅니다. 비율 열을 보면 $z = 1$ 에서는 백서 값이 오히려 크고, $z \geq 2$ 에서 정확식이 큽니다. 그런스판·페레스마르코가 적은 대로입니다. 비율은 z 가 클수록, q 가 클수록 벌어집니다. $q = 0.45$, $z = 340$ 에서 아홉 배입니다.

[심화] 한 가지 어긋남을 적어 둡니다. 그런스판·페레스마르코의 표 1은 나카모토 식으로 $q = 0.40$ 에서 필요한 z_{SN} 을 **81로** 적었는데, 백서 8쪽의 표와 이 책의 재계산(NB09-2)은 **89입니다**. 다른 일곱 칸은 모두 일치합니다. 이 책은 백서와 재계산을 따릅니다. 그런스판·페레스마르코의 z (정확식) 행은 이 책의 유한합과 전부 일치합니다.

```
# NB09-4: 몬테카를로 — 식을 믿지 말고 경주를 직접 해 본다 (시드 고정)
import random

def simulate(q: float, z: int, trials: int, seed: int = 20261025) -> float:
    """정직 체인이 z 블록을 캘 때까지 공격자도 캔다(블록마다 확률 q). 그 뒤 격차가 0 이하가 되면 성공, z+200 넘게
벌어지면 실패로 절단."""
    rng = random.Random(seed)
    p = 1.0 - q
    wins = 0
    for _ in range(trials):
        honest, attacker = 0, 0
        while honest < z:                          # 상인이 z 확인을 기다리는 동안
            if rng.random() < q: attacker += 1
            else: honest += 1
        gap = honest - attacker
        while 0 < gap < z + 200:                   # 그 뒤의 경주
            gap += -1 if rng.random() < q else 1
        if gap <= 0: wins += 1
    return wins / trials

for q, z, n in ((0.1, 3, 200_000), (0.1, 6, 200_000), (0.3, 10, 50_000)):
    mc, ex, na = simulate(q, z, n), exact_P(q, z), nakamoto_P(q, z)
    se = (ex * (1 - ex) / n) ** 0.5
    print(f"q={q} z={z:>2} n={n:>7,}: 몬테카를로 {mc:.5f} | 정확식 {ex:.5f} | 나카모토 {na:.5f} | MC
표준오차 ≈ {se:.5f}")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
q=0.1 z= 3 n=200,000: 몬테카를로 0.01728 | 정확식 0.01712 | 나카모토 0.01317 | MC 표준오차 ≈ 0.00029
q=0.1 z= 6 n=200,000: 몬테카를로 0.00061 | 정확식 0.00059 | 나카모토 0.00024 | MC 표준오차 ≈ 0.00005
q=0.3 z=10 n= 50,000: 몬테카를로 0.06414 | 정확식 0.06511 | 나카모토 0.04166 | MC 표준오차 ≈ 0.00110
```

[중급] 무작위 경주의 결과는 표준오차 범위 안에서 **정확식과** 맞고, 백서 값과는 그보다 멀리 떨어집니다. 식을 믿지 않아도 경주 자체가 같은 답을 냅니다.

```
# NB09-5: 절대 해시율 불변성 — 식에는 점유율 q 만 들어간다
print("공격자 해시율 a, 정직 해시율 h  →  q = a/(a+h),  P(z=6)")
for a, h in ((1, 9), (1e6, 9e6), (1e20, 9e20), (3e20, 7e20)):
    q = a / (a + h)
    print(f"  a={a:.0e} H/s, h={h:.0e} H/s → q={q:.2f}, 정확식 P(6)={exact_P(q, 6):.6f}, 나카모토
P(6)={nakamoto_P(q, 6):.6f}")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
공격자 해시율 a, 정직 해시율 h  →  q = a/(a+h),  P(z=6)
  a=1e+00 H/s, h=9e+00 H/s → q=0.10, 정확식 P(6)=0.000591, 나카모토 P(6)=0.000243
  a=1e+06 H/s, h=9e+06 H/s → q=0.10, 정확식 P(6)=0.000591, 나카모토 P(6)=0.000243
  a=1e+20 H/s, h=9e+20 H/s → q=0.10, 정확식 P(6)=0.000591, 나카모토 P(6)=0.000243
  a=3e+20 H/s, h=7e+20 H/s → q=0.30, 정확식 P(6)=0.156450, 나카모토 P(6)=0.132111
```

전체 해시율을 10^{20} 배 키워도 q 가 같으면 확률이 같습니다. 식에 절대 속도가 들어갈 자리가 없기 때문입니다. 로젠펠드가 적은 세 관찰이 이것을 요약합니다. 「성공 확률은 블록 수에 달려 있고 시간 상수 T_0 에는 달려 있지 않다.」 「확인을 더 기다리면 이중지불 성공 확률이 지수적으로 줄어든다. 감쇠율은 공격자의 상대 해시율에 달려 있다.」 「성공 확률은 확인 수에 달려 있지 기다린 시간의 양에 달려 있지 않다.」[12] 그리고 한 줄 더. 「어떤 확인 수도 성공률을 0으로 줄이지 못한다.」[13]

4. 「완료」는 확률이다

이제 「지급은 언제 완료되었는가」에 기술이 줄 수 있는 답을 적을 수 있습니다. 답은 시각이 아니라 **함수입니다.** 거래 위에 블록이 z 개 쌓인 시점에, 해시율 q 를 가진 공격자가 그 거래를 되돌릴 확률 $P(q, z)$. z 가 늘수록 지수적으로 작아지지만 0이 되지는 않고, q 가 1/2에 가까울수록 같은 수준까지 줄이는 데 필요한 z 가 끝없이 커지며, $q \geq 1/2$ 이면 어떤 z 로도 줄지 않습니다. 이것은 결제 시스템이 쓰는 「최종성(finality)」 — 규칙이 「이 시점부터는 취소할 수 없다」고 정하는 개념 — 과 다릅니다. 어느 시스템의 어느 규칙이 그 시점을 정하는지는 이 책이 1차 자료로 받치지 못했으므로 여기서는 개념만 적습니다(28장 1절 (c)). 비트코인에는 그런 시점이 프로토콜 안에 없고, 각 수취인이 자기 위험 선호에 따라 z 를 고릅니다. 10장의 지분증명은 이 자리에 「경제적 최종성」이라는 다른 종류의 답을 넣으려 합니다.

> **? 법률가의 질문**
>
> 지급은 언제 완료되었다고 볼 것입니까. 거래가 네트워크에 전파된 때, 블록에 들어간 때, 그 위에 z 개가 쌓인 때 중 어느 것인가요. 계약이나 약관이 「6 확인」을 완료 시점으로 정하면 그것이 기준이 됩니까. 그 뒤 체인이 재편되어 거래가 사라지면 이행은 처음부터 없었던 것이 됩니까, 있었다가 없어진 것이 됩니까. 압류·기수·이행지체 같은 시점 개념은 이 확률 함수의 어디에 걸어야 합니까. → 28장

▾ **[심화] 공격은 수지가 맞는가 — 로젠펠드의 경제 모형**

[심화] 위 확률은 「상대가 공격자라고 가정하는」 적대적 설정의 값입니다. 로젠펠드는 §6에서 더 현실적인 질문을 던집니다. 공격자는 수지가 맞을 때만 공격한다. 그는 공격을 상인 k 명에게 동시에 걸어 각각에게서 값 v 의 물건을 사고, 물건의 실제 가치는 $\alpha v (0 < \alpha \leq 1$, 환금성 손실)이며, 공격이 실패하면 물건값 kv 를 치르고 비밀 가지에서 캔 블록 o 개의 보상 oB 를 잃는다고 둡니다. 성공 확률이 r 이면 기대 이익은

$$kv(\alpha + r - 1) - (1 - r)\, oB$$

이고, 이것이 양이 되려면 $v > \dfrac{(1-r)\,oB}{k(\alpha + r - 1)}$ 이어야 합니다.[14] 기호 풀이: B 는 블록 보상, o 는 실패 시 버려지는 블록 수, $r = P(q, n)$ 은 위 확률입니다. 수치 예는 싣지 않습니다. 로젠펠드 자신이 「많은 모형 가정 때문에 이 값들은 걸러 들어야 한다」고 적었고,[15] 그의 표는 당시의 블록 보상과 임의의 k, α 를 전제합니다. 이 책에 남기는 것은 구조입니다. **거래 금액이 클수록 상인은 더 많은 확인을 요구해야 하고, 공격의 비용은 「실패했을 때 버려지는 블록 보상」으로 매겨진다.** 이 구조가 20장의 보안 예산 논의로 이어집니다.

5. 정리

해시율 다수도 서명을 위조하지는 못하고, 역사의 순서만 바꿀 수 있습니다. 그 순서 바꾸기가 성공할 확률은 q 와 z 만의 함수이며 z 에 지수적으로 줄지만 0이 되지 않고, $q \geq 1/2$ 이면 줄지 않습니다. 백서의 식은 낙관적이고, 정확식은 같은 안전 수준에 더 많은 확인을 요구합니다($q = 0.1$ 이면 5가 아니라 6, $q = 0.3$ 이면 24가 아니라 32). 「6 확인」은 관행이고, 「완료」는 확률입니다. 이 장의 모든 수치는 원논문의 식을 다시 계산하고 무작위 경주로 대조한 것입니다.

이 장의 출처

- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — §11 Calculations(6–8쪽): 공격 범위, 이항 랜덤워크, 도박꾼의 파산, 포아송 근사, C 코드, 표.
- Meni Rosenfeld, "Analysis of Hashrate-Based Double Spending," arXiv:1402.2009 [cs.CR], 2014-02-09(초고 2012-12-11) — 공격 단계(3–4쪽), 확인 수 정의(3쪽), 음이항 모형과 식 (1)(7쪽), 관찰(6·8–9쪽), 표 1(10쪽), 6 확인 평가(10쪽), 경제 모형(10–12쪽).
- Cyril Grunspan & Ricardo Pérez-Marco, "Double Spend Races," *International Journal of Theoretical and Applied Finance* 21(8): 1850053, 2018. DOI 10.1142/S021902491850053X; arXiv:1702.02867 (v3, 2020-05-06) — Theorem 1과 표 1(2쪽), Theorem 2(지수 감쇠의 증명, 2쪽).
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `math`·`random` 만 사용(특수 함수 패키지 없이 유한합으로 계산). 노트북 NB09. 원 재현 스크립트는 저자의 2026-09-28 연구 노트에서 가져와 다시 짰습니다.

각주

1. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), 6쪽(§11 Calculations). 원문: "We consider the scenario of an attacker trying to generate an alternate chain faster than the honest chain. Even if this is accomplished, it does not throw the system open to arbitrary changes, such as creating value out of thin air or taking money that never belonged to the attacker. Nodes are not going to accept an invalid transaction as payment, and honest nodes will never accept a block containing them. An attacker can only try to change one of his own transactions to take back money he recently spent."

2. Meni Rosenfeld, "Analysis of Hashrate-Based Double Spending," arXiv:1402.2009 (2014; 초고 2012-12-11), 3–4쪽(§3 Playing catch-up). 원문: "1. Broadcast to the network a transaction in which the attacked merchant is paid. 2. Secretly mine a branch which builds on the latest block at the time (before the transaction made it into a block), which includes instead a conflicting transaction which pays the attacker. 3. Wait until the transaction to the merchant receives enough confirmations and the merchant, confident in his payment, sends the product. 4. If necessary, continue extending the secret branch ... until it is longer than the public branch ... then broadcast it."

3. Rosenfeld (2014), 3쪽. 원문: "A transaction is said to have n confirmations if it is included in a block which is part of the valid chain, and there are n blocks in the path from that block to the leaf of the chain, inclusive."

4. Nakamoto (2008), 6쪽. 원문: "The race between the honest chain and an attacker chain can be characterized as a Binomial Random Walk. The success event is the honest chain being extended by one block, increasing its lead by +1, and the failure event is the attacker's chain being extended by one block, reducing the gap by -1. The probability of an attacker catching up from a given deficit is analogous to a Gambler's Ruin problem."

5. Nakamoto (2008), 6쪽. 백서는 q_z 를 「공격자가 z 블록 뒤에서 언젠가 따라잡을 확률」로 정의하고 위 식을 적습니다(유도는 참고문헌 [8] Feller를 가리킵니다). 위 유도는 이 책이 보충한 것입니다.

6. Rosenfeld (2014), 6쪽. 원문: "If the attacker controls more than half of the total network hashrate, he always succeeds in catching up, from any disadvantage."

7. Nakamoto (2008), 7쪽. 원문: "The recipient waits until the transaction has been added to a block and z blocks have been linked after it. He doesn't know the exact amount of progress the attacker has made, but assuming the honest blocks took the average expected time per block, the attacker's potential progress will be a Poisson distribution with expected value: λ = z q/p".

8. Nakamoto (2008), 7쪽. 「Rearranging to avoid summing the infinite tail of the distribution...」 뒤의 식과 C 코드. 8쪽에 $q = 0.1$·$q = 0.3$ 표와 「P < 0.001」 표.

9. Rosenfeld (2014), 10쪽. 원문: "There is nothing special about the default, often-cited figure of 6 confirmations. It was chosen based on the assumption that an attacker is unlikely to amass more than 10% of the hashrate, and that a negligible risk of less than 0.1% is acceptable. Both these figures are arbitrary, however; 6 confirmations are overkill for casual attackers, and at the same time powerless against more dedicated attackers with much more than 10% hashrate."

10. Rosenfeld (2014), 7쪽(§4 Waiting for confirmations). 원문: "We will not use this assumption, but rather model m more accurately as a negative binomial variable; it is the number of successes (blocks found by the attacker) before n failures (blocks found by the honest network), with a probability q of success." 로젠펠드의 식 (1)은 공격자가 공격 전에 블록 하나를 미리 캐 두었다고 가정하고 「더 길어짐」을 성공으로 보며, 이는 「미리 캔 블록 없이 동률을 성공으로 셈하는」 아래 식과 같은 값을 냅니다(그의 표 1 값으로 확인).

11. Cyril Grunspan & Ricardo Pérez-Marco, "Double Spend Races," *International Journal of Theoretical and Applied Finance* 21(8): 1850053 (2018), Theorem 1(arXiv:1702.02867v3, 2쪽). 원문: "Let 0 < q < 1/2, respectively p = 1 − q, be the relative hash power of the group of the attackers, respectively of honest miners. After z blocks have been validated by the honest miners, the probability of success of the attackers is P(z) = I_{4pq}(z, 1/2)". 이어서: "In general, for z ≥ 2, these probabilities P(z) are larger than those P_SN(z) obtained by Nakamoto. From the standpoint of bitcoin security, this shows than larger confirmation times z are necessary compared to those z_SN given by Nakamoto, ..."(문장 중간까지 인용).

12. Rosenfeld (2014), 6쪽·8쪽. 원문: "The probability of success depends on the number of blocks, and not on the time constant T0." "Waiting for more confirmations exponentially decreases the probability of double-spending success. The decay rate depends on the attacker's relative hashrate." "The probability of success depends on the number of confirmations and not on the amount of time waited."

13. Rosenfeld (2014), 9쪽. 원문: "No amount of confirmations will reduce the success rate to 0."

14. Rosenfeld (2014), 11–12쪽(§6 Economics of double-spending). 기대 이익 식: "kαv − (1 − r)(kv + oB) = kv(α + r − 1) − (1 − r)oB" 와 수지 조건 "v > (1 − r)oB / (k(α + r − 1))".

15. Rosenfeld (2014), 12쪽. 원문: "These values should be taken with a grain of salt, because of the many modeling assumptions made."

1부 기술 기초

10장 지분증명 — 에너지 대신 담보로

핵심 TAKEAWAY

작업증명은 「전기를 태운 사람」에게 블록을 쓸 권한을 주고, 지분증명(Proof of Stake, PoS)은 「보증금을 맡긴 사람」에게 줍니다. 보안의 원천이 바뀝니다. 작업증명에서 공격은 외부 자원(연산)을 사서 하는 것이고, 지분증명에서 공격은 체제 안에 맡긴 **자기 보증금을 잃을 각오로** 하는 것입니다. 그래서 지분증명에는 작업증명에 없는 장치가 하나 더 있습니다. 규칙을 어긴 검증자의 보증금을 깎는 **슬래싱(slashing)입니다**. 처벌이 보안의 일부입니다. 이 장을 읽으면 검증자·담보·증명·슬래싱의 뜻, 「3분의 2」 규칙이 어디서 오는지, 그리고 지분증명이 9장의 확률적 최종성 대신 무엇을 약속하는지를 압니다.

학습목표

1. 검증자(validator)·담보(stake)·증명(attestation)·슬래싱의 뜻을 설명하고, 작업증명과 지분증명이 「비용의 종류」에서 어떻게 다른지 말한다.
2. [중급] 캐스퍼(Casper FFG)의 두 가지 슬래싱 조건과 「3분의 2」 최종성 규칙을 읽고, 현행 이더리움 명세의 슬래싱 벌금 규칙을 계산한다.
3. [심화] 지분증명 특유의 공격(장거리 재편, 무위험 투표)과 그 방어(출금 지연·약한 주관성·비활성 누출)의 구조를 안다.
4. 「스테이킹 보상」을 수익이 아니라 메커니즘과 위험 구조로 설명한다.

[입문]만 읽은 독자가 가져가는 것. 지분증명은 「전기 대신 보증금」이고, 처벌(슬래싱)이 보안의 일부입니다. 부록 B 오해 ⑩(「PoS는 부자가 더 부자 되는 구조다」)은 이 장을 읽고 부분적으로만 판정합니다.

1. 이 기술은 무엇의 실패에서 나왔나

작업증명의 대가는 전기와 하드웨어입니다. 비트코인의 안전은 「공격자가 전체 해시율의 큰 몫을 혼자 갖기 어렵다」는 데 걸려 있고(9장), 그 어려움은 채굴 장비와 전력을 사 모으는 비용에서 옵니다. 그 비용은 체제 밖으로 나갑니다. 채굴자는 전기 요금을 전력 회사에 냅니다. 지분증명을 제안한 사람들이 내세운 것은 효율, 곧 그 자원 소모를 없애는 것이었습니다. 그리고 이 책의 정리로 그들의 설계가 작업증명에 더한 것은 **처벌입니다**. 작업증명에서 규칙을 어긴 채굴자가 잃는 것은 「그 블록에 들인 전기」뿐이고, 그 이상의 벌을 줄 손잡이가 체제 안에 없습니다.

부테린(Vitalik Buterin)과 그리피스(Virgil Griffith)의 2017년 캐스퍼(Casper) 논문은 지분증명을 이렇게 소개합니다. 「PoS 체제에서 블록체인은 체제 안의 코인을 가진 누구나 참여할 수 있는 절차로 새 블록을 덧붙이고 합의하며, 참여자의 영향력은 그가 가진 코인(「지분」)의 수에 비례한다. 이는 작업증명

「채
굴」보다 훨씬 효율적인 대안으로, 채굴의 높은 하드웨어·전기 비용 없이 블록체인이 돌아가게 한다.」[1] 그리고 핵심 주장을 적습니다. 「규칙을 어긴 검증자가 있으면 우리는 그 위반을 탐지하고 어느 검증자가 어겼는지 알 수 있다. 이 책임 추궁 가능성(accountability) 덕분에 악의적 검증자를 처벌할 수 있고, 체인 기반 PoS를 괴롭히는 「무위험 투표(nothing at stake)」 문제를 푼다. 규칙 위반의 벌은 검증자의 보증금 전부다.」[2] 처벌할 수 있다는 것, 이것이 지분증명이 작업증명에 더한 것입니다.

「무위험 투표」가 무엇인지는 작업증명과 대조하면 보입니다(캐스퍼 논문은 이 문제의 이름만 들고 설명하지 않으므로, 아래는 이 책의 풀이입니다). 작업증명에서 두 가지(포크)가 생기면 채굴자는 한쪽을 골라야 합니다. 해시율은 한 번에 한 곳에만 쓸 수 있으니까요. 서명은 공짜라서, 처벌 장치가 없는 지분증명이라면 검증자가 **두 가지 모두에** 서명해 두고 어느 쪽이 이기든 보상을 챙길 수 있습니다. 모두가 그렇게 하면 포크가 수렴하지 않습니다. 캐스퍼의 답은 「두 곳에 서명한 증거가 체인에 올라오면 보증금을 빼앗는다」입니다.

2. 직관 — 보증금, 투표, 그리고 처벌

지분증명 체제의 참가자를 **검증자(validator)라고** 부릅니다. 검증자가 되려면 코인을 체제에 **담보(stake, 보증금)로** 맡깁니다. 캐스퍼 논문의 정의: 「각 검증자는 보증금을 가진다. 검증자가 참여할 때 그 보증금은 맡긴 코인의 수이고, 참여한 뒤에는 보상과 벌금에 따라 오르내린다. 지분증명의 보안은 검증자의 **수가** 아니라 **보증금의 크기에서** 나오므로, 이 논문에서 「검증자의 3분의 2」라고 할 때 그것은 보증금 가중 비율을 뜻한다.」[3] 작업증명의 「CPU 하나에 한 표」가 「코인 하나에 한 표」로 바뀐 것입니다.

검증자는 두 가지 일을 합니다. 차례가 오면 블록을 **제안하고**, 다른 검증자가 제안한 블록에 「이것이 올바른 체인의 머리」라고 **투표(증명, attestation)합니다**. 캐스퍼는 블록마다가 아니라 일정 간격의 **체크포인트에** 투표하게 하고, 보증금 기준 3분의 2 이상이 같은 체크포인트 쌍에 투표하면 그 연결을 「초다수 연결(supermajority link)」이라 부릅니다. 초다수 연결이 닿은 체크포인트는 「정당화(justified)」되고, 정당화된 체크포인트에서 그 바로 다음 체크포인트로 초다수 연결이 생기면 앞의 것이 「**최종화(finalized)」 됩니다.**[4] 작업증명의 「블록이 쌓일수록 뒤집힐 확률이 지수적으로 준다」 대신, 「3분의 2가 두 단계로 도장을 찍으면 끝」이라는 **이산적인 최종성을** 두는 것입니다.

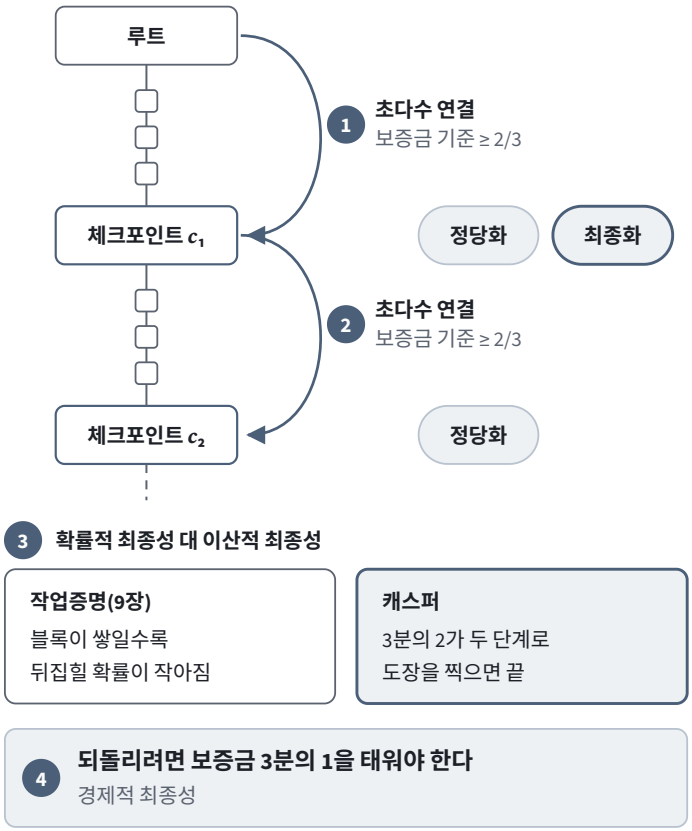


그림 10-1. 캐스퍼의 두 단계 도장. 보증금 기준 3분의 2 이상의 초다수 연결이 닿으면 체크포인트가 정당화되고, 그 바로 다음 체크포인트로 초다수 연결이 생기면 앞의 것이 최종화된다. 체크포인트 사이의 작은 네모는 블록이다.

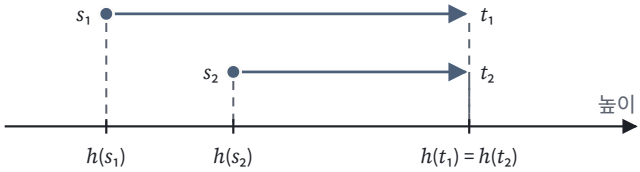
그 「끝」은 무엇을 뜻할까요. 논문의 핵심 정리는 이렇습니다. 「캐스퍼의 가장 주목할 성질은, 검증자의 3분의 1 이상이 두 슬래싱 조건 중 하나를 어기지 않는 한 **서로 충돌하는 두 체크포인트가 모두 최종화되는 것은 불가능하다는** 것이다.」[5] 뒤집으려면 보증금 총액의 3분의 1 이상이 규칙을 어기고 그 증거를 남겨야 하며, 그 증거가 체인에 올라오면 그들의 보증금이 사라집니다. 「되돌릴 수 없다」가 아니라 「되돌리려면 보증금 3분의 1을 태워야 한다」입니다. 이것을 **경제적 최종성이라고** 부릅니다. 9장의 확률적 최종성과 종류가 다릅니다. 하나는 확률이 작아지는 것이고, 하나는 가격표가 붙는 것입니다.

두 가지 금지 — 슬래싱 조건

캐스퍼의 「두 계명」은 간단합니다. 검증자 v 는 다음 두 투표를 동시에 하면 안 됩니다. (I) 같은 높이의 두 다른 체크포인트에 투표하는 것(이중 투표). (II) 한 투표의 출발점과 도착점 사이에 다른 투표의 출발점과 도착점이 완전히 들어가는 것(둘러싸기 투표).[6] 둘 중 하나라도 어긴 증거가 체인에 거래로 올라오면 「그 검증자의 보증금 전부를 빼앗고, 증거를 올린 사람에게 약간의 발견 수수료를 준다」.[7] 현행 이더리움 명세의 함수 `is_slashable_attestation_data` 는 이 둘을 그대로 코드로 적어 두었습니다. 「이중 투표: 두 증명 데이터가 다르면서 목표 에포크가 같다」, 「둘러싸기 투표: 한쪽의 출발 에포크가 다른 쪽의 출발 에포크보다 작고 도착 에포크는 더 크다」.[8]

(I) 이중 투표

같은 높이의 두 다른 체크포인트에 투표



명세: 두 증명 데이터가 다르면서 목표 에포크가 같다

(II) 둘러싸기 투표

한 투표의 출발점과 도착점 사이에 다른 투표가 완전히 들어감



명세: 한쪽의 출발 에포크가 더 작고 도착 에포크는 더 크다

그림 10-2. 한 검증자의 두 투표를 출발점 s 에서 도착점 t 로 가는 화살표로 그렸다. (I) 이중 투표는 도착 높이가 같고, (II) 둘러싸기 투표는 한 투표의 구간이 다른 투표를 완전히 감싼다. 아래 줄은 현행 명세 함수 is_slashable_attestation_data 의 조건이다.

3. 현행 규칙 — 숫자는 명세에서

[중급] 이더리움의 지분증명 「비콘 체인」은 최소 제네시스 시각을 2020년 12월 1일 12시(UTC)로 두고 출발했고,[9] 2022년 9월 6일 Bellatrix 포크(에포크 144,896)로 합의층을 준비한 뒤 같은 달 종단 총난이도에 도달하면서 실행층이 작업증명에서 지분증명으로 넘어갔습니다(설정 파일 주석의 추정일은 2022년 9월 15일). 그 뒤 Capella(2023-04), Deneb(2024-03), Electra(2025-05), Fulu(2025-12) 포크가 이어졌습니다.[10] 아래 숫자는 **2026-10-25 기준** 현행 명세(Fulu 활성 뒤)에서 읽은 것이고, 포크마다 바뀌어 왔으므로 부록 A 동향 노트에서 갱신합니다.[11]

항목	현행 값(2026-10 기준)	뜻
`MIN_ACTIVATION_BALANCE`	32 ETH	검증자로 활성화되는 데 필요한 최소 잔고
`MAX_EFFECTIVE_BALANCE_ELECTRA`	2,048 ETH	「compounding validator」(0x02 출금 자격 — 명세 용어) 하나가 가질 수 있는 유효 잔고 상한(Electra부터)
`MIN_SLASHING_PENALTY_QUOTIENT_ELECTRA`	4,096	슬래싱 즉시 벌금 = 유효 잔고 ÷ 4,096
`WHISTLEBLOWER_REWARD_QUOTIENT_ELECTRA`	4,096	신고자 보상 = 유효 잔고 ÷ 4,096
`PROPORTIONAL_SLASHING_MULTIPLIER_BELLATRIX`	3	상관 벌금의 배수(아래)

[중급] 슬래싱은 두 번 깎습니다. 먼저 **즉시 벌금**. 명세의 `slash_validator` 는 검증자를 퇴장 절차에 넣고,

슬래싱 표시를 하고, 「유효 잔고 ÷ `MIN_SLASHING_PENALTY_QUOTIENT_ELECTRA`」를 깎습니다.[12] 32 ETH 검증자면 32/4096 = 0.0078125 ETH입니다. 다음은 **상관 벌금(correlation penalty)**. 약 18일 뒤 (`EPOCHS_PER_SLASHINGS_VECTOR`의 절반 뒤) `process_slashings`가, 같은 기간에 슬래싱된 보증금의 합에 배수 3을 곱한 것(총 활성 잔고가 상한)을 총 활성 잔고로 나눈 비율만큼 **추가로** 깎습니다.[13] 혼자 어기면 둘째 벌금은 거의 0이고, 보증금의 3분의 1이 함께 어기면 $3 \times 1 / 3 = 1$, 곧 **전부를** 잃습니다. 캐스퍼 논문의 「보증금 전부」가 현행 규칙에서는 「많이 함께 어길수록 전부에 가까워지는」 비례 벌금으로 구현된 것입니다. 작은 실수(운영 사고로 한 검증자가 이중 서명)는 가볍게, 조직적 공격(3분의 1의 공모)은 치명적으로. 이 비례성이 설계의 요체입니다.

스테이킹 「보상」은 무엇의 대가인가

검증자는 블록을 제안하고 증명에 참여한 대가로 프로토콜이 새로 발행하는 코인을 받습니다(거래 수수료의 몫은 13장·21장). 흔히 「스테이킹 수익」이라고 부르지만, 이 책은 그 말을 쓰지 않습니다. 구조로 보면 그것은 (1) 보증금을 묶어 두는 기회비용, (2) 노드를 24시간 운영하는 노동과 설비, (3) 운영 실수나 공격 가담으로 보증금을 잃을 위험, 이 셋에 대한 **프로토콜의 지급입니다**. 받는 쪽에서는 보상이고, 체제 쪽에서는 보안 유지비입니다(20장). 그리고 위험은 추상적이지 않습니다. 같은 키로 두 노드를 동시에 돌리는 운영 사고 하나로 이중 서명이 생기고 슬래싱됩니다. 이 책은 어떤 스테이킹 서비스도 이름을 들지 않으며, 보상률을 적지 않습니다.

> ? **법률가의 질문**
>
> 스테이킹 보상은 무엇의 대가입니까. 검증이라는 **용역의** 대가인가요, 맡긴 코인이 낳은 **과실인가요**, 아니면 새 코인의 **원시취득인가요**. 보증금이 슬래싱으로 사라지면 그것은 벌금인가요, 위약금인가요, 담보의 실행인가요. 남의 코인을 받아 대신 스테이킹해 주는 사업자는 수탁자인가요, 채무자인가요. → 28장

4. 코드 — 담보 가중 선택과 슬래싱 산수

[중급] 실제 명세는 수천 줄이지만 두 가지 핵심 산수는 짧습니다. 담보에 비례해 제안자를 뽑는 것과, 슬래싱 벌금을 계산하는 것입니다. 전부 장난감이며 실사용 금지입니다.

```
# NB10-1: 담보 가중 무작위 선택 — 「코인 하나에 한 표」를 시뮬레이션으로 확인 (공부용, 시드 고정)
import random
rng = random.Random(20261025)

stakes = {"A": 32, "B": 32, "C": 64, "D": 128, "E": 256}      # 검증자별 담보(ETH 단위 장난감 값)
total = sum(stakes.values())
names, weights = list(stakes), list(stakes.values())

N = 100_000
picks = {n: 0 for n in names}
for _ in range(N):                                             # 슬롯마다 담보에 비례해 제안자 한 명을
뽑는다
    picks[rng.choices(names, weights=weights)[0]] += 1

print(f"{'검증자':>6} {'담보':>6} {'담보 비율':>9} {'선택 비율':>9}")
for n in names:
    print(f"{n:>6} {stakes[n]:>6} {stakes[n]/total:>9.3f} {picks[n]/N:>9.3f}")
print("담보가 두 배면 선택 횟수도 두 배 — 「코인 하나에 한 표」. 보상이 선택 횟수에 비례하면 담보에 비례한다.")
print("다만 유지되는 것은 비율이다 — 모두 같은 율로 늘면 절대 격차는 그 율만큼 커지고 비율은 그대로다. 부록 B ⑩은
이 점에서 「부분」 판정.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
   검증자      담보      담보 비율      선택 비율
     A      32      0.062      0.062
     B      32      0.062      0.061
     C      64      0.125      0.126
     D     128      0.250      0.253
     E     256      0.500      0.498
담보가 두 배면 선택 횟수도 두 배 — 「코인 하나에 한 표」. 보상이 선택 횟수에 비례하면 담보에 비례한다.
다만 유지되는 것은 비율이다 — 모두 같은 율로 늘면 절대 격차는 그 율만큼 커지고 비율은 그대로다. 부록 B ⑩은 이 점에서
「부분」 판정.
```

```
# NB10-2: 슬래싱 벌금 — 현행 명세(Electra 이후) 규칙으로 즉시 벌금과 상관 벌금을 계산 (명세의 정수 나눗셈 순서
그대로)
GWEI = 10**9
MIN_SLASHING_PENALTY_QUOTIENT_ELECTRA = 4096
WHISTLEBLOWER_REWARD_QUOTIENT_ELECTRA = 4096
PROPORTIONAL_SLASHING_MULTIPLIER_BELLATRIX = 3
EFFECTIVE_BALANCE_INCREMENT = 1 * GWEI                         # 1 ETH = 10^9 Gwei
(presets/mainnet/phase0.yaml)

def immediate_penalty(effective_balance_gwei: int) -> int:     # slash_validator
    return effective_balance_gwei // MIN_SLASHING_PENALTY_QUOTIENT_ELECTRA

def correlation_penalty(effective_balance_gwei: int, slashed_sum_gwei: int, total_active_gwei:
int) -> int:   # process_slashings
    increment = EFFECTIVE_BALANCE_INCREMENT
    adjusted = min(slashed_sum_gwei * PROPORTIONAL_SLASHING_MULTIPLIER_BELLATRIX,
total_active_gwei)
    per_increment = adjusted // (total_active_gwei // increment)
    return (effective_balance_gwei // increment) * per_increment

eb = 32 * GWEI                                                 # 32 ETH 검증자 하나
total_active = 1_000_000 * 32 * GWEI                           # 가정: 32 ETH 검증자 100만 명 =
3,200만 ETH 활성 (장난감 가정값)
print(f"즉시 벌금: {immediate_penalty(eb)/GWEI:.7f} ETH (= 32/4096), 신고자 보상: {eb //
WHISTLEBLOWER_REWARD_QUOTIENT_ELECTRA / GWEI:.7f} ETH")
print(f"{'같은 기간 슬래싱된 담보 비율':>18} {'상관 벌금(ETH)':>14} {'잔고 대비':>8}")
for frac in [0.0001, 0.01, 0.10, 1/3, 0.5]:
    slashed_sum = int(total_active * frac)
    pen = correlation_penalty(eb, slashed_sum, total_active)
    print(f"{frac:>18.4%} {pen/GWEI:>14.4f} {pen/eb:>8.1%}")
print("혼자 어기면 둘째 벌금은 0에 가깝고, 총 담보의 1/3이 함께 어기면 3 × 1/3 = 100% — 전부를 잃는다.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
즉시 벌금: 0.0078125 ETH (= 32/4096), 신고자 보상: 0.0078125 ETH
  같은 기간 슬래싱된 담보 비율      상관 벌금(ETH)    잔고 대비
           0.0100%          0.0096     0.0%
           1.0000%          0.9600     3.0%
          10.0000%          9.6000    30.0%
          33.3333%         32.0000   100.0%
          50.0000%         32.0000   100.0%
혼자 어기면 둘째 벌금은 0에 가깝고, 총 담보의 1/3이 함께 어기면 3 × 1/3 = 100% — 전부를 잃는다.
```

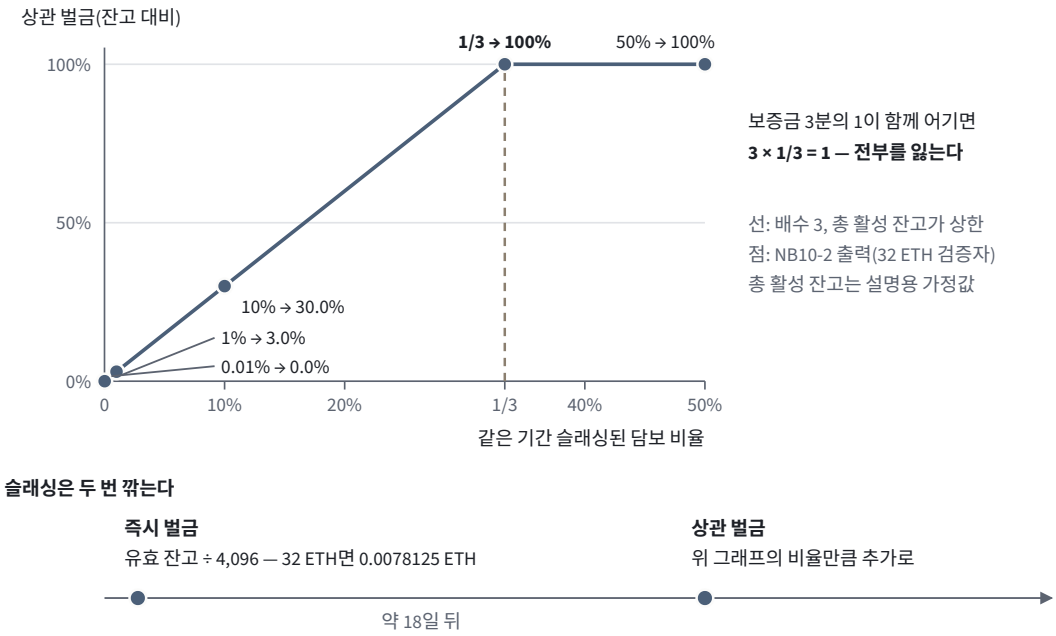


그림 10-3. 같은 기간 슬래싱된 담보 비율에 따른 상관 벌금(잔고 대비). 점은 NB10-2 출력이고 선은 배수 3과 총 활성 잔고 상한으로 그렸다. 총 활성 잔고는 NB10-2 의 설명용 가정값이다. 아래 띠는 즉시 벌금과 약 18일 뒤 상관 벌금의 순서다.

[중급] 벌금표의 모양이 설계 의도 그 자체입니다. 공격이 성공하려면 보증금의 3분의 1이 공모해야 하는데, 바로 그 지점에서 벌금이 100%가 됩니다. 「뒤집는 데 드는 비용 = 총 보증금의 3분의 1」이라는 경제적 최종성의 가격표는 이렇게 코드에 적혀 있습니다. 위 `total_active` 는 설명용 가정값입니다. 실제 총 활성 잔고는 매 에포크 바뀌며 이 책은 그 값을 적지 않습니다.

▾ [심화] 지분증명 특유의 공격 둘과 방어 — 장거리 재편, 대규모 이탈

[심화] 작업증명에 없는 공격이 둘 있습니다. 캐스퍼 논문 §4가 다룹니다.

장거리 재편(long range revision). 보증금을 이미 찾아간 옛 검증자들은 더 잃을 것이 없습니다. 「검증자 연합이 보증금을 인출하고 나면, 그 연합이 먼 과거에 보증금의 3분의 2를 넘게 가졌었다면, 그들은 과거의 초다수를 이용해 슬래싱될 두려움 없이 충돌하는 체크포인트들을 최종화할 수 있다(이미 돈을 찾았으므로).」[14] 작업증명에서는 과거를 다시 쓰려면 과거의 작업을 다시 해야 하지만, 서명은 공짜라 과거의 키로 과거를 다시 쓰는 데 비용이 없습니다. 방어는 두 가지입니다. 보증금을 찾는 데 **출금 지연을** 두고(논문은 「약 넉 달 분량의 블록」[15]), 클라이언트가 **최종화된 블록은 절대 되돌리지 않으며 일정 주기로(예: 1~2달에 한 번) 접속해 최신 체인을 확인한다는** 가정을 둡니다.[16] 둘째 가정에는 흔히 「약한 주관성(weak subjectivity)」이라는 이름이 붙는데, 그 명칭은 캐스퍼 논문과 이 책이 읽은 명세 사본에 나오지 않으므로 여기서는 뜻만

씁니다. 오래 떨어져 있던 노드는 어느 체인이 진짜인지 스스로 알 수 없고 바깥 정보(사회적 합의)에 기대야 합니다. 작업증명 노드는 제네시스부터 작업량만 세면 되었습니다. 이 차이는 「진짜 체인은 누가 정하는가」라는 23장의 질문에 새 층을 더합니다.

대규모 이탈(catastrophic crash). 검증자의 3분의 1 넘게가 한꺼번에 꺼지면(네트워크 분단, 고장, 또는 고의) 초다수 연결을 만들 수 없어 **아무것도 최종화되지 않습니다.** 체인은 자라지만 도장이 찍히지 않습니다. 논문의 처방은 「비활성 누출(inactivity leak)」입니다. 투표하지 않는 검증자의 보증금을 천천히 빼서, 남아서 투표하는 검증자들이 다시 3분의 2가 될 때까지 기다리는 것입니다.[17] 이 장치는 두 체인이 각각 다른 집단에 의해 최종화되는 상황을 만들 수 있고, 논문은 그때 「각 검증자는 자기가 먼저 본 최종화 체크포인트를 따르라」고 적은 뒤, 「이런 여러 공격에서 회복하는 정확한 알고리듬은 미해결 문제」라고 인정하면서 검증자들이 명백한 악의를 보고 「소수 소프트포크」를 수동으로 만들면 시장이 그 포크를 택할 것이라고 가정합니다.[18] 지분증명의 최종성은 평상시에는 이산적이고 깔끔하지만, 비상시에는 다시 사회적 조정으로 돌아갑니다.

5. 작업증명과 지분증명 — 비용의 종류가 다르다

두 체제를 한 표로 놓습니다. 어느 쪽이 낫다는 표가 아니라, **무엇이 다른지의** 표입니다.

	작업증명	지분증명
블록을 쓸 권한	해시 복권에 당첨된 자	담보에 비례해 뽑힌 검증자
한 표의 단위	연산(「CPU 하나에 한 표」)	담보(「코인 하나에 한 표」)
공격 비용의 성격	체제 **밖에서** 사는 자원(장비·전력)	체제 **안에** 맡긴 보증금의 상실
규칙 위반의 벌	그 블록에 들인 자원의 허비	슬래싱(보증금 차감, 공모 규모에 비례)
최종성	확률적 — 블록이 쌓일수록 뒤집힐 확률이 지수적으로 감소, 0은 아님	경제적 — 3분의 2의 두 단계 투표로 확정, 뒤집으려면 담보 3분의 1 이상이 슬래싱
오래 떨어졌던 노드	작업량을 세어 스스로 진짜 체인을 안다	최근 최종화 지점을 바깥에서 받아 와야 한다(주기적 접속 가정)
특유의 약점	51% 해시율 집중	장거리 재편, 대규모 이탈, 담보 집중

「PoS는 부자가 더 부자 되는 구조」라는 말은 어떻게 볼까요. 담보에 비례해 뽑히고 보상도 비례한다면 **비율은** 유지됩니다(위 NB10-1). 모두가 같은 율로 늘면 격차의 비율은 그대로입니다. 그러나 큰 참가자가 보상을 다시 담보로 넣는 복리 효과, 노드 운영의 규모의 경제, 최소 담보 32 ETH라는 문턱은 작은 참가자에게 불리하게 작용할 수 있습니다. 작업증명에서도 장비·전력의 규모의 경제가 같은 방향으로 작용합니다. 그래서 부록 B는 이 명제를 「부분 지지」로 둡니다. 비례 자체는 격차를 키우지 않고, 문턱과 규모의 경제가 키울 수 있습니다. 어느 쪽이 더 집중되었는지는 실측의 문제이고 이 책은 그 수치를 적지 않습니다.

6. 정리

지분증명은 「전기 대신 보증금」이고, 보증금이 있기에 처벌이 가능하며, 처벌이 있기에 「3분의 2가 도장을 찍으면 끝」이라는 경제적 최종성을 약속할 수 있습니다. 그 약속의 뒷면이 슬래싱 비례 벌금이고, 그 약속이 닿지 않는 곳(장거리 재편, 대규모 이탈)에서는 출금 지연·약한 주관성·비활성 누출이라는 보조 장치가 들어옵니다. 현행 숫자(32 ETH, 1/4096, 배수 3)는 포크마다 바뀌어 온 값이며 2026-10 기준입니다. 다음 11장은 합의와 별개로 「원장이 잔액을 적는 두 방식」을 봅니다.

이 장의 출처

- Vitalik Buterin & Virgil Griffith, "Casper the Friendly Finality Gadget," arXiv:1710.09437v4 [cs.CR], 2019-01-22 (v1 2017-10-25). https://arxiv.org/abs/1710.09437 — PoS 소개와 책임 추궁(1쪽), 보증금과 3분의 2(2쪽), 정당화·최종화(3쪽), 두 계명·슬래싱·Theorem 1(4쪽), 출금 지연(6쪽), 장거리 재편과 약한 주관성(6–7쪽), 비활성 누출과 미해결 문제(8쪽).
- ethereum/consensus-specs (master 커밋 b834fa8411cb2152563e8835c5b72c28f7503e7a, 2026-10-23): `specs/phase0/beacon-chain.md` (`is_slashable_attestation_data`), `specs/electra/beacon-chain.md` (Gwei values, Rewards and penalties, `slash_validator` , `process_slashings`), `configs/mainnet.yaml` (제네시스 시각, 포크 에포크), `presets/mainnet/*.yaml` (상수 이력). 공개 비콘 노드 `/eth/v1/config/spec` 응답과 일치 확인. 2026-10-25 접근.
- go-ethereum v1.17.10 (커밋 1df30b18), `params/chainspecs/mainnet.json` — 종단 총난이도.
- 이 책의 `widgets/data.json` `ethereum.staking` (as_of 2026-10-25) — 위 상수와 고정 링크.
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `random` 만 사용. 노트북 NB10.

각주

1. Vitalik Buterin & Virgil Griffith, "Casper the Friendly Finality Gadget," arXiv:1710.09437v4 [cs.CR] (2019-01-22; v1 2017-10-25), 1쪽(§1). 원문: "In a PoS system, a blockchain appends and agrees on new blocks through a process where anyone who holds coins inside of the system can participate, and the influence an agent has is proportional to the number of coins (or 'stake') it holds. This is a vastly more efficient alternative to proof of work (PoW) 'mining' and enables blockchains to operate without mining's high hardware and electricity costs." 학술지 게재본은 없고 arXiv 판(v4)을 인용합니다.

2. 같은 글, 1쪽(§1.1). 원문: "Accountability. If a validator violates a rule, we can detect the violation and know which validator violated the rule. Accountability allows us to penalize malfeasant validators, solving the 'nothing at stake' problem that plagues chain-based PoS. The penalty for violating a rule is a validator's entire deposit." 같은 문단은 「PoS 보안은 벌금(penalty)의 크기에 기초하고, 그 벌금을 채굴 보상 이득보다 훨씬 크게 둘 수 있으므로 PoS가 PoW보다 엄격히 강한 보안 유인을 준다」고 주장합니다. 이것은 저자들의 주장이며 이 책은 그 평가를 따라 말하지 않고 구조만 옮깁니다. 비용 비교는 20장.

3. Buterin & Griffith (2019), 2쪽(§2). 원문: "Each validator has a deposit; when a validator joins, its deposit is the number of deposited coins. After joining, each validator's deposit rises and falls with rewards and penalties. Proof of stake's security derives from the size of the deposits, not the number of validators, so for the rest of this paper, when we say '2/3 of validators', we are referring to the deposit-weighted fraction".

4. 같은 글, 3쪽(§2). 원문: "A supermajority link is an ordered pair of checkpoints (a, b) ... such that at least 2/3 of validators (by deposit) have published votes with source a and target b." "A checkpoint c is called justified if (1) it is the root, or (2) there exists a supermajority link c′ → c where checkpoint c′ is justified." "A checkpoint c

is called finalized if (1) it is the root or (2) it is justified and there is a supermajority link c → c′ where c′ is a direct child of c." 논문은 높이가 100의 배수인 블록을 체크포인트로 두고(2쪽 본문), 각주 2에서는 체크포인트 사이 간격을 「99 blocks」로 표현합니다. 현행 이더리움은 에포크 단위로 다른 매개변수를 씁니다.

5. 같은 글, 4쪽(§2). 원문: "The most notable property of Casper is that it is impossible for any two conflicting checkpoints to be finalized unless ≥ 1/3 of the validators violate one of the two Casper Commandments/slashing conditions". Theorem 1 (Accountable Safety, 4쪽): "Two conflicting checkpoints am and bn cannot both be finalized."

6. 같은 글, 4쪽(Figure 2 "The two Casper Commandments"). 원문: "AN INDIVIDUAL VALIDATOR v MUST NOT PUBLISH TWO DISTINCT VOTES ... SUCH THAT EITHER: I. h(t1) = h(t2). Equivalently, a validator must not publish two distinct votes for the same target height. OR II. h(s1) < h(s2) < h(t2) < h(t1). Equivalently, a validator must not vote within the span of its other votes."

7. 같은 글, 4쪽. 원문: "If a validator violates either slashing condition, the evidence of the violation can be included into the blockchain as a transaction, at which point the validator's entire deposit is taken away with a small 'finder's fee' given to the submitter of the evidence transaction."

8. ethereum/consensus-specs, `specs/phase0/beacon-chain.md`, 함수 `is_slashable_attestation_data`(master 커밋 b834fa84, 2026-10-23). 원문 주석: "Check if data_1 and data_2 are slashable according to Casper FFG rules." 코드: `(data_1 != data_2 and data_1.target.epoch == data_2.target.epoch)` (Double vote) `or (data_1.source.epoch < data_2.source.epoch and data_2.target.epoch < data_1.target.epoch)` (Surround vote). 2026-10-25 접근.

9. consensus-specs `configs/mainnet.yaml` (커밋 b834fa84): `MIN_GENESIS_TIME: 1606824000` 주석 "Dec 1, 2020, 12pm UTC". 이 값은 제네시스 시각의 하한이며 같은 파일에 `GENESIS_DELAY: 604800`(7일)이 있습니다. 실제 제네시스 시각은 이 책의 원자료에서 따로 확인하지 않았습니다.

10. 같은 파일(주석의 시각 부분은 생략): `ALTAIR_FORK_EPOCH: 74240 # Oct 27, 2021, …`, `BELLATRIX_FORK_EPOCH: 144896 # Sept 6, 2022, …`, `CAPELLA_FORK_EPOCH: 194048 # April 12, 2023, …`, `DENEB_FORK_EPOCH: 269568 # March 13, 2024, …`, `ELECTRA_FORK_EPOCH: 364032 # May 7, 2025, …`, `FULU_FORK_EPOCH: 411392 # December 3, 2025, …`. 실행층 전환(종단 총난이도 도달)은 같은 파일의 `TERMINAL_TOTAL_DIFFICULTY: 58750000000000000000000` 에 붙은 주석 "# Estimated on Sept 15, 2022" 로 추정일만 적혀 있습니다. go-ethereum v1.17.10 `params/chainspecs/mainnet.json` 도 같은 종단 총난이도를 적습니다.

11. 값의 출처는 consensus-specs `specs/electra/beacon-chain.md` (Gwei values·Rewards and penalties 표, `slash_validator`, `process_slashings`)와 공개 비콘 노드의 `/eth/v1/config/spec` 응답(둘이 일치)입니다. 이 책의 `widgets/data.json` `ethereum.staking` 에 고정 링크와 함께 적어 두었습니다. 접근 2026-10-25(UTC 2026-10-24).

12. consensus-specs `specs/electra/beacon-chain.md`, `slash_validator`: `initiate_validator_exit(state, slashed_index)`; `validator.slashed = Boolean(True)`; `validator.withdrawable_epoch = max(validator.withdrawable_epoch, epoch + EPOCHS_PER_SLASHINGS_VECTOR)`; `state.slashings[epoch % EPOCHS_PER_SLASHINGS_VECTOR] += validator.effective_balance`; `slashing_penalty = validator.effective_balance // MIN_SLASHING_PENALTY_QUOTIENT_ELECTRA`; `decrease_balance(state, slashed_index, slashing_penalty)`; 신고자 보상 `whistleblower_reward = validator.effective_balance // WHISTLEBLOWER_REWARD_QUOTIENT_ELECTRA`.

13. 같은 파일, `process_slashings`: `adjusted_total_slashing_balance = min(Gwei(sum(state.slashings)) * PROPORTIONAL_SLASHING_MULTIPLIER_BELLATRIX, total_balance)`; `penalty_per_effective_balance_increment = adjusted_total_slashing_balance // (total_balance // increment)`; 슬래싱된 검증자 중 `epoch + EPOCHS_PER_SLASHINGS_VECTOR // 2 == validator.withdrawable_epoch` 인 자에게 유효 잔고 증분 수에 비례한 벌금. `EPOCHS_PER_SLASHINGS_VECTOR` 는 8,192 에포크(`presets/mainnet/phase0.yaml`). 몫수의 이력: Phase0 128·512·1 → Altair 64·—·2 → Bellatrix 32·—·3 → Electra 4096·4096·3 (각 포크 preset 파일).

14. Buterin & Griffith (2019), 6쪽(§4.1). 원문: "Once a coalition of validators has withdrawn their deposits, if that coalition had more than 2/3 of deposits long ago in the past, they can use their historical supermajority to finalize conflicting checkpoints without fear of getting slashed (because they have already withdrawn their money). This is called the long-range revision attack ...".

15. 같은 글, 6쪽(§3). 원문: "At the start of the end dynasty, the validator's deposit is locked for a long period of time, called the withdrawal delay (approximately 'four months worth of blocks'), before the deposit is withdrawn."

16. 같은 글, 6쪽(§4.1). 원문: "In simple terms, long-range attacks are prevented by a fork choice rule to never revert a finalized block, as well as an expectation that each client will 'log on' and gain a complete up-to-date view of the chain at some regular frequency (e.g., once per 1–2 months)."

17. 같은 글, 8쪽(§4.2). 원문: "We can recover from this by instituting an 'inactivity leak' which slowly drains the deposit of any validator that does not vote for checkpoints, until eventually its deposit sizes decrease low enough that the validators who are voting are a supermajority."

18. 같은 글, 8쪽. 원문: "If this situation happens, then each validator should simply favor whatever finalized checkpoint it saw first. The exact algorithm for recovering from these various attacks remains an open problem."

1부 기술 기초

11장 UTXO와 계정 — 원장이 「잔액」을 적는 두 방식

핵심 TAKEAWAY

비트코인에는 「계좌 잔액」이 없습니다. 있는 것은 **아직 쓰이지 않은 거래 출력(UTXO)의** 더미이고, 「내 잔액」은 내 열쇠로 열 수 있는 출력들의 합을 지갑이 그때그때 더한 값입니다. 이더리움은 반대로 주소마다 **계정을** 두고 잔액과 일련번호(논스)를 적습니다. 두 모형은 같은 일을 다르게 합니다. 어느 쪽이 옳은 것이 아니라, 병렬 처리·프라이버시·프로그램 가능성·착오 이체의 모양이 다릅니다. 이 장을 읽으면 「코인을 잘못 보냈다」가 두 체계에서 서로 다른 사건이라는 것을 압니다.

학습목표

1. 비트코인의 미사용 출력(UTXO) 모형과 이더리움의 계정·논스 모형을 비교하고, 「잔액」이 각 모형에서 무엇인지 말한다.
2. [중급] 두 모형의 상태 전이 함수를 코드로 흉내 내어, 이중지불이 어디서 막히는지 본다.
3. 두 모형에서 「착오 이체」가 어떤 모양으로 생기는지 구별한다.
4. 거스름돈 출력과 주소 재사용이 프라이버시에 주는 함의를 안다.

[입문]만 읽은 독자가 가져가는 것. 비트코인에는 「계좌 잔액」이 없습니다. 쓰지 않은 거래 출력의 합이 있을 뿐입니다. 그래서 「착오 이체」가 두 체계에서 다르게 생깁니다.

1. 이 기술은 무엇의 실패에서 나왔나

은행 원장은 계좌 단위입니다. 계좌마다 잔액이 있고, 이체는 한 계좌에서 빼고 다른 계좌에 더합니다. 이 방식의 전제는 **원장을 쓰는 자가 하나라는** 것입니다. 두 창구가 같은 계좌에서 동시에 빼면 어떻게 되는지를 은행은 중앙 서버의 잠금으로 해결합니다. 그런데 블록체인에는 중앙 서버가 없습니다. 누구나 거래를 만들어 뿌리고, 누가 먼저인지는 블록에 들어가서야 정해집니다(9장). 그래서 나카모토는 잔액을 적는 대신 **코인 자체를 이어지는 서명의 사슬로** 정의했고(5장), 그 사슬의 끝에서 「아직 쓰이지 않은 출력」이 곧 가치가 되게 했습니다. 잔액이라는 상태를 두지 않으면 「같은 상태를 두 번 고친다」는 문제 자체가 사라집니다. 같은 출력을 두 번 쓰려는 거래가 둘 있으면, 둘 중 하나만 블록에 들어갈 수 있고 다른 하나는 「이미 쓰인 출력을 참조한다」는 이유로 무효가 됩니다.

이더리움은 다른 길을 갔습니다. 계약 코드가 상태를 읽고 쓰려면 「상태」가 명시적으로 있어야 하기 때문입니다. 부테린의 2014년 백서는 비트코인 UTXO 스크립트의 한계를 몇 가지 듭니다(출력에서 인출할 금액을 세밀하게 정할 수 없는 「가치 맹목」, 쓰였거나 안 쓰였거나 둘뿐이어서 여러 단계의

계약을 짤 수 없는 「상태 없음」 등).[1] 이더리움은 상태를 계정 단위로 명시적으로 둡니다. 두 모형은 서로 다른 목적에서 나왔습니다.

2. UTXO — 쓰지 않은 출력의 더미

비트코인 거래는 **입력들과 출력들로** 되어 있습니다. 백서 §9: 「가치를 쪼개고 합칠 수 있도록 거래는 여러 입력과 출력을 가진다. 보통은 더 큰 이전 거래에서 온 입력 하나이거나 작은 금액들을 합치는 여러 입력이고, 출력은 많아야 둘이다. 하나는 지급, 다른 하나는 거스름돈이 있으면 보내는 사람에게 돌려주는 것.」[2] 각 출력에는 금액과 「이것을 쓸 수 있는 조건」(공개키의 해시 또는 공개키 그 자체 — 5장·12장)이 적혀 있습니다. 각 입력은 **이전 거래의 어느 출력을** 쓰는지 가리키고, 그 출력의 조건을 푸는 서명을 담습니다.

한 번 입력으로 쓰인 출력은 영원히 「쓰인 것」이 됩니다. 쓰이지 않은 출력들의 집합이 **UTXO 집합 (Unspent Transaction Output set)이고**, 이것이 비트코인의 「상태」입니다. 부테린은 이것을 비트코인의 상태 전이로 깔끔하게 정리했습니다. 「비트코인의 「상태」는 발행되었으나 아직 쓰이지 않은 모든 코인(기술적으로는 「미사용 거래 출력」, UTXO)의 모음이며, 각 UTXO는 액면과 소유자(사실상 공개키인 20바이트 주소로 정의됨)를 가진다.」 그리고 상태 전이 함수 `APPLY(S, TX) → S'` 는 「각 입력이 가리키는 UTXO가 상태에 없으면 오류, 서명이 그 UTXO 소유자와 맞지 않으면 오류, 입력 액면 합이 출력 액면 합보다 작으면 오류, 그렇지 않으면 입력 UTXO를 전부 빼고 출력 UTXO를 전부 더한 상태를 돌려준다」입니다.[3]

이 정의에서 「잔액」이 어디 있는지 보십시오. 없습니다. 어떤 사람의 잔액은 「그 사람의 열쇠로 열 수 있는 UTXO의 액면 합」이고, 그것은 원장이 적어 두는 숫자가 아니라 **지갑이 UTXO 집합을 훑어 더하는 숫자입니다**. 거래 수수료도 같은 자리에서 나옵니다. 입력 합이 출력 합보다 크면 그 차이가 수수료이고, 백서 §6이 말한 대로 블록을 만든 채굴자가 가져갑니다.

거스름돈과 프라이버시

출력은 통째로만 쓸 수 있습니다. 10코인짜리 출력으로 3코인을 보내려면, 3코인은 받는 사람에게, 7코인은 **자기 자신의 새 주소로** 보내는 출력 둘을 만듭니다. 7코인 출력이 거스름돈입니다. 백서 §10은 거래마다 새 열쇠를 쓰라고 권하지만 곧 한계를 적습니다. 「여러 입력을 가진 거래에서는 그 입력들이 같은 소유자의 것이었음이 필연적으로 드러나므로 어느 정도의 연결은 피할 수 없다. 어떤 열쇠의 소유자가 드러나면 연결을 통해 같은 소유자의 다른 거래들도 드러날 위험이 있다.」[4] UTXO 모형에서 프라이버시는 「주소가 익명」이라는 데서 오는 것이 아니라 「주소를 바꿔 쓰는 습관」에서 오고, 그 습관은 입력 합치기와 거스름돈 패턴으로 깨집니다. 부록 B 오해 ⑧(「블록체인은 익명이다」)의 근거입니다.

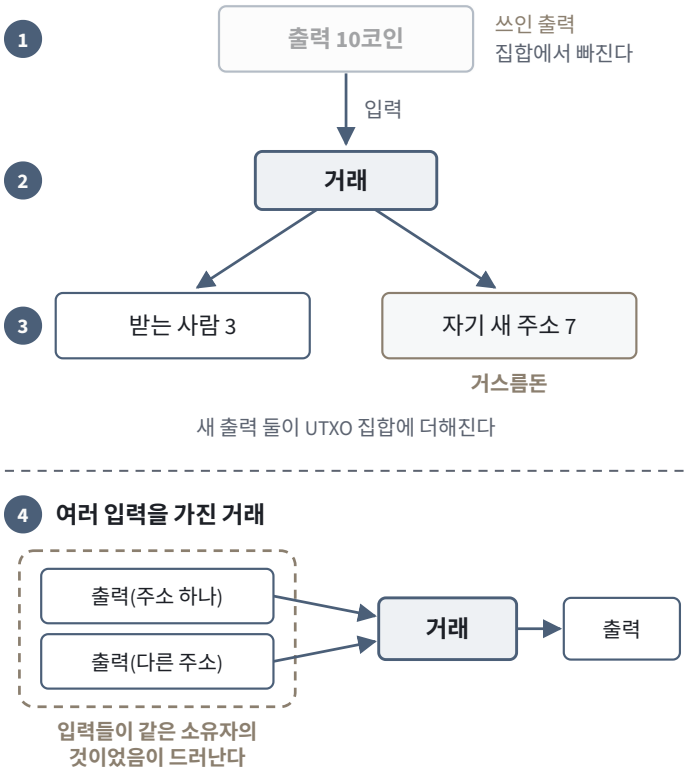


그림 11-1. 10코인 출력 하나가 입력으로 쓰여 받는 사람 3과 자기 새 주소 7(거스름돈)의 두 출력이 되고, 쓰인 출력은 UTXO 집합에서 빠진다. 아래는 여러 입력을 합친 거래가 그 입력들이 같은 소유자의 것이었음을 드러내는 모양(백서 §10).

3. 계정 — 주소마다 상태를 적는다

이더리움의 상태는 「계정(account)」들의 모임입니다. 부테린 백서: 「이더리움에서 상태는 「계정」이라는 객체들로 이루어지며, 각 계정은 20바이트 주소를 갖고, 상태 전이는 계정들 사이의 가치와 정보의 직접 이전이다. 이더리움 계정은 네 필드를 갖는다. 각 거래가 한 번만 처리되게 하는 계수기인 **논스**, 계정의 현재 이더 잔고, 계약 코드(있으면), 저장소(기본은 비어 있음).」[5] 황서(Yellow Paper)는 같은 네 필드를 정식으로 정의합니다. 세계 상태 σ 는 「주소(160비트 식별자)에서 계정 상태로 가는 사상」이고, 계정 상태 $\sigma[a]$ 는 nonce(「이 주소에서 보낸 거래 수, 또는 코드가 있는 계정이면 이 계정이 만든 계약 생성 수」), balance(「이 주소가 가진 Wei의 수」), storageRoot(저장소를 담은 머클 패트리샤 트리의 뿌리 해시), codeHash(EVM 코드의 해시)로 이루어집니다.[6] 계정에는 두 종류가 있습니다. 개인키로 움직이는 **외부 소유 계정**과 코드로 움직이는 **계약 계정입니다**.[7] 계약은 13장의 주제입니다.

여기서는 잔액이 **원장에 적힌 숫자입니다**. 이체는 보내는 계정의 잔고를 줄이고 받는 계정의 잔고를 늘립니다. 그러면 처음의 문제, 「같은 잔고에서 두 번 빼기」는 어떻게 막을까요. **논스입니다**. 계정의 논스는 그 계정이 보낸 거래 수이고, 거래에는 「이 거래는 내 n 번째 거래다」라는 논스가 적힙니다. 논스가 맞지 않는 거래는 무효입니다. 같은 서명된 거래를 두 번 내놓아도 두 번째는 논스가 이미 지나갔으므로 거부됩니다. UTXO 모형에서는 「출력이 이미 쓰였다」가 그 역할을 했고, 계정 모형에서는 「논스가 이미 쓰였다」가 합니다.

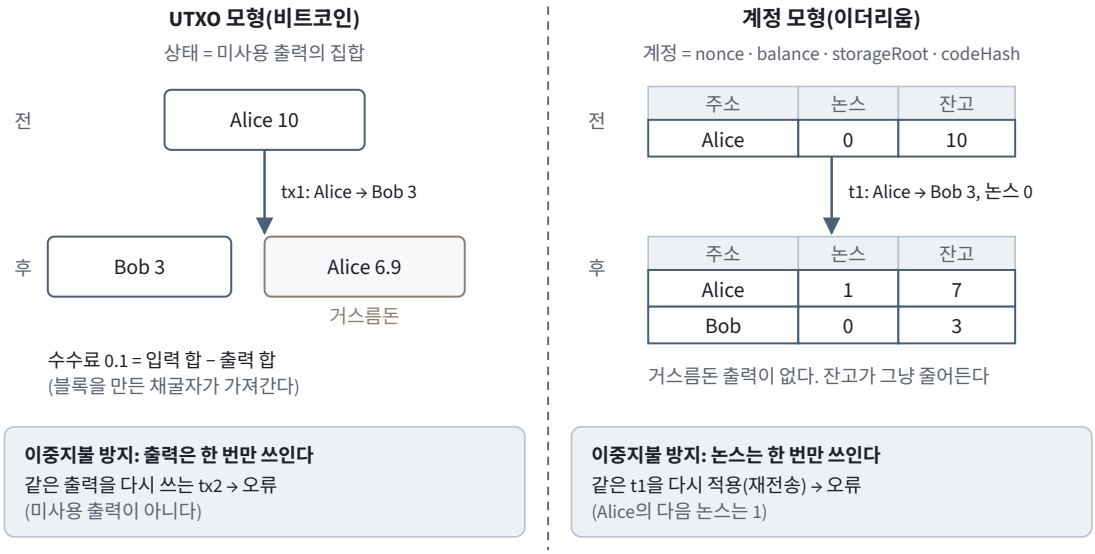


그림 11-2. 같은 지급(Alice → Bob 3)을 두 원장에 놓은 그림. 왼쪽은 NB11-1의 tx1로 수수료 0.1이 빠지고, 오른쪽은 NB11-2의 t1로 계정 쪽 예는 수수료를 생략했다. 이중지불을 막는 것은 각각 「출력은 한 번만」과 「논스는 한 번만」이다.

4. [중급] 코드 — 두 상태 전이 함수

[중급] 서명 검증은 5장에서 했으니 여기서는 「누가 열쇠를 가졌는가」를 이름으로 대신합니다. 두 함수 모두 부테린의 `APPLY(S, TX)` 서술을 그대로 옮긴 것입니다.

```
# NB11-1: UTXO 상태 전이 — 입력이 가리키는 출력을 집합에서 빼고 새 출력을 더한다 (공부용)
import hashlib, json

def txid(tx: dict) -> str:
    return hashlib.sha256(json.dumps(tx, sort_keys=True).encode()).hexdigest()[:12]

def apply_utxo(S: dict, tx: dict):
    """S: {(txid, index): {"owner":..., "value":...}}.  tx: {"inputs":[(txid, index, signer)],
"outputs":[(owner, value)]}
    부테린(2014)의 APPLY 세 단계 그대로. 성공하면 (새 상태, 수수료), 실패하면 오류 문자열."""
    S = dict(S)
    in_sum = 0
    for (prev_id, idx, signer) in tx["inputs"]:
        key = (prev_id, idx)
        if key not in S:
            return f"오류: 입력 {key} 는 미사용 출력이 아니다(이미 쓰였거나 없음)"
        if S[key]["owner"] != signer:
            return f"오류: 입력 {key} 의 소유자는 {S[key]['owner']} 인데 서명자가 {signer}"
        in_sum += S[key]["value"]
    out_sum = sum(v for _, v in tx["outputs"])
    if in_sum < out_sum:
        return f"오류: 입력 합 {in_sum} < 출력 합 {out_sum}"
    for (prev_id, idx, _) in tx["inputs"]:
        del S[(prev_id, idx)]
    tid = txid(tx)
    for i, (owner, value) in enumerate(tx["outputs"]):
        S[(tid, i)] = {"owner": owner, "value": value}
    return S, in_sum - out_sum

def balance(S, who):                        # 「잔액」은 원장에 없다 — 그때그때 더한다
    return sum(o["value"] for o in S.values() if o["owner"] == who)

coinbase = {"inputs": [], "outputs": [("Alice", 10)]}          # 블록 보상으로 Alice 에게 10
S = {(txid(coinbase), 0): {"owner": "Alice", "value": 10}}
print("처음 UTXO 집합:", {k: v for k, v in S.items()})

tx1 = {"inputs": [(txid(coinbase), 0, "Alice")], "outputs": [("Bob", 3), ("Alice", 6.9)]}   # 3
지급 + 6.9 거스름돈, 0.1 수수료
S, fee = apply_utxo(S, tx1)
print("tx1 적용 후 — 잔액 Alice:", balance(S, "Alice"), "Bob:", balance(S, "Bob"), "| 수수료:",
round(fee, 2))
print("UTXO 집합 크기:", len(S), "(10짜리 출력 하나가 사라지고 3, 6.9 두 개가 생겼다)")

tx2 = {"inputs": [(txid(coinbase), 0, "Alice")], "outputs": [("Carol", 10)]}                  #
같은 출력을 다시 쓰려는 이중지불
print("tx2(이중지불):", apply_utxo(S, tx2))
tx3 = {"inputs": [(txid(tx1), 0, "Mallory")], "outputs": [("Mallory", 3)]}                  # 남의
출력
print("tx3(남의 출력):", apply_utxo(S, tx3))
tx4 = {"inputs": [(txid(tx1), 0, "Bob")], "outputs": [("Carol", 3)]}                        # Bob
이 받은 3 을 Carol 에게
S, fee = apply_utxo(S, tx4)
print("tx4 적용 후 — Bob:", balance(S, "Bob"), "Carol:", balance(S, "Carol"))
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
처음 UTXO 집합: {('02c83c8a0105', 0): {'owner': 'Alice', 'value': 10}}
tx1 적용 후 — 잔액 Alice: 6.9 Bob: 3 | 수수료: 0.1
UTXO 집합 크기: 2 (10짜리 출력 하나가 사라지고 3, 6.9 두 개가 생겼다)
tx2(이중지불): 오류: 입력 ('02c83c8a0105', 0) 는 미사용 출력이 아니다(이미 쓰였거나 없음)
tx3(남의 출력): 오류: 입력 ('99736848c857', 0) 의 소유자는 Bob 인데 서명자가 Mallory
tx4 적용 후 — Bob: 0 Carol: 3
```

```
# NB11-2: 계정 상태 전이 — 잔고를 빼고 더하되, 논스로 「같은 거래 두 번」을 막는다 (공부용)
def apply_account(S: dict, tx: dict):
    """S: {address: {"nonce": n, "balance": b}}.  tx: {"from", "to", "value", "nonce"}"""
    S = {k: dict(v) for k, v in S.items()}
    sender = S.setdefault(tx["from"], {"nonce": 0, "balance": 0})
    if tx["nonce"] != sender["nonce"]:
        return f"오류: {tx['from']} 의 다음 논스는 {sender['nonce']} 인데 거래 논스가 {tx['nonce']}"
    if sender["balance"] < tx["value"]:
        return f"오류: 잔고 {sender['balance']} < 송금액 {tx['value']}"
    sender["nonce"] += 1
    sender["balance"] -= tx["value"]
    S.setdefault(tx["to"], {"nonce": 0, "balance": 0})["balance"] += tx["value"]
    return S

S = {"Alice": {"nonce": 0, "balance": 10}}
t1 = {"from": "Alice", "to": "Bob", "value": 3, "nonce": 0}
S = apply_account(S, t1)
print("t1 적용 후:", S)
print("같은 t1 을 다시 적용(재전송 공격):", apply_account(S, t1))
t2 = {"from": "Alice", "to": "Carol", "value": 3, "nonce": 1}
S = apply_account(S, t2)
print("t2 적용 후:", S, "— 거스름돈 출력이 없다. 잔고가 그냥 줄어든다")
t3 = {"from": "Alice", "to": "0x000…dead", "value": 4, "nonce": 2}      # 존재하지 않는(열쇠 없는) 주소로 — 착오 이체
S = apply_account(S, t3)
print("t3(착오 이체) 적용 후:", S, "— 받는 계정이 자동으로 생기고 잔고가 적힌다. 되돌릴 거래는 그 주소의 열쇠만 만들 수 있다")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
t1 적용 후: {'Alice': {'nonce': 1, 'balance': 7}, 'Bob': {'nonce': 0, 'balance': 3}}
같은 t1 을 다시 적용(재전송 공격): 오류: Alice 의 다음 논스는 1 인데 거래 논스가 0
t2 적용 후: {'Alice': {'nonce': 2, 'balance': 4}, 'Bob': {'nonce': 0, 'balance': 3}, 'Carol': {'nonce': 0, 'balance': 3}} — 거스름돈 출력이 없다. 잔고가 그냥 줄어든다
t3(착오 이체) 적용 후: {'Alice': {'nonce': 3, 'balance': 0}, 'Bob': {'nonce': 0, 'balance': 3}, 'Carol': {'nonce': 0, 'balance': 3}, '0x000…dead': {'nonce': 0, 'balance': 4}} — 받는 계정이 자동으로 생기고 잔고가 적힌다. 되돌릴 거래는 그 주소의 열쇠만 만들 수 있다
```

5. 두 모형의 비교 — 그리고 「착오 이체」의 두 모양

비교표입니다.

	UTXO 모형(비트코인)	계정 모형(이더리움)
상태	미사용 출력의 집합	주소 → (논스, 잔고, 저장소, 코드)
「잔액」	원장에 없음. 내 열쇠로 열리는 출력의 합을 지갑이 계산	원장에 적힌 숫자
거래	입력(이전 출력 참조 + 서명)들과 출력(금액 + 잠금 조건)들	받는 계정, 금액, 논스, 데이터, 가스 관련 필드와 서명(보내는 계정은 서명에서 복원) — 단순화
이중지불 방지	출력은 한 번만 쓰인다	논스는 한 번만 쓰인다
거스름돈	자기 주소로 가는 출력을 따로 만든다	없음. 잔고가 줄 뿐
병렬성	서로 다른 출력을 쓰는 거래는 순서와 무관하게 검증 가능	같은 계정의 거래는 논스 순서대로 처리해야 함
프라이버시	주소를 바꿔 쓸 수 있으나 입력 합치기·거스름돈으로 연결됨	한 주소에 역사가 쌓임
프로그램 가능성	출력의 잠금 조건(스크립트)에 한정	계약 계정의 코드와 저장소(13장)

이제 「코인을 잘못 보냈다」를 두 체계에서 그려 봅니다. **UTXO 모형에서** 착오 이체는 「잠금 조건이 잘못된 출력을 만든 것」입니다. 그 출력은 UTXO 집합에 그대로 있고, 그 조건을 풀 열쇠를 가진 사람만 쓸 수 있습니다. 열쇠가 아무에게도 없는 조건이면(오타로 생긴 주소 등) 그 출력은 영원히 집합 안에 남습니다. 누구도 「가져간」 것이 아닙니다. **계정 모형에서는** 「받는 계정의 잔고가 늘어난 것」입니다. 위 NB11-2의 t3처럼 받는 주소에 계정이 없었어도 그 순간 생깁니다. 되돌리는 거래는 그 주소의 열쇠만 만들 수 있습니다. 두 경우 모두 원장은 「누가 정당한 권리자인가」를 묻지 않습니다. 서명이 맞으면 유효한 거래입니다. 차이는 **무엇이 어디에 남아 있는가입니다**. 한쪽에는 「열리지 않은 출력」이, 다른 쪽에는 「늘어난 잔고」가 남습니다.

> ? **법률가의 질문**
>
> 착오로 보낸 코인은 누구의 것입니까. UTXO 모형에서는 「누구의 열쇠로도 열리지 않는 출력」이 생겼을 뿐이고, 계정 모형에서는 「어떤 주소의 잔고 숫자」가 늘었습니다. 「점유의 이전」이라는 말은 이 둘 중 어디에 맞습니까. 받은 사람이 그것을 자기 주소로 옮겼다면 그 행위는 무엇입니까. 그 사람이 열쇠를 잃어버려 옮길 수 없다면 반환 의무는 어떻게 됩니까. 우리 법원이 이 문제에서 어떤 틀을 썼는지는 5부에서 사건을 가리키기만 합니다. → 28장

6. 정리

비트코인은 잔액 대신 미사용 출력의 집합을 상태로 두고 「출력은 한 번만 쓰인다」로 이중지불을 막습니다. 이더리움은 계정마다 잔고와 논스를 두고 「논스는 한 번만 쓰인다」로 막습니다. 전자는 병렬성과 주소 바꾸기에 유리하고, 후자는 상태를 읽고 쓰는 프로그램에 유리합니다. 「잔액」이 원장에 있는가 없는가, 이 한 가지 차이가 착오 이체·압류·프라이버시의 모양을 전부 바꿉니다. 1부의 마지막 12장은 지금까지의 개념을 진짜 비트코인 블록 하나에서 바이트 단위로 확인합니다.

이 장의 출처

- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — 입력·출력과 거스름돈(§9, 5쪽), 다중 입력의 연결 위험(§10, 6쪽), 수수료(§6, 4쪽).
- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014(초고 2013). https://ethereum.org/whitepaper/ (공식 PDF, 2026-10-25 접근) — 비트코인 상태 전이와 UTXO 정의, 이더리움 계정의 네 필드와 두 종류, UTXO 스크립트의 한계.
- Gavin Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, Shanghai version efc5f9a, 2025-02-04. https://ethereum.github.io/yellowpaper/paper.pdf — 세계 상태와 계정 상태의 네 필드(§4.1, 4쪽).
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `hashlib`·`json` 만 사용. 노트북 NB11.

각주

1. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," ethereum.org 공식 PDF(2014; 초고 2013, 받은 판은 2014-04 이후 개정본), 「Scripting」 절 — UTXO 기반

스크립트의 한계로 Lack of Turing-completeness, Value-blindness, Lack of state, Blockchain-blindness 를 듭니다. 이 책은 그 중 「Lack of state — UTXO can either be spent or unspent; there is no opportunity for multi-stage contracts ...」와 「Value-blindness - there is no way for a UTXO script to provide fine-grained control over the amount that can be withdrawn」만 요지로 옮깁니다. 백서가 「그래서 계정 모형」이라고 인과를 직접 적는 것은 아닙니다. https://ethereum.org/whitepaper/ (2026-10-25 접근).

2. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), 5쪽(§9 Combining and Splitting Value). 원문: "To allow value to be split and combined, transactions contain multiple inputs and outputs. Normally there will be either a single input from a larger previous transaction or multiple inputs combining smaller amounts, and at most two outputs: one for the payment, and one returning the change, if any, back to the sender."

3. Buterin (2014), 「Bitcoin As A State Transition System」 절(PDF 5–6쪽). 원문: "The 'state' in Bitcoin is the collection of all coins (technically, 'unspent transaction outputs' or UTXO) that have been minted and not yet spent, with each UTXO having a denomination and an owner (defined by a 20-byte address which is essentially a cryptographic public key)". 상태 전이: "1. For each input in TX: i. If the referenced UTXO is not in S, return an error. ii. If the provided signature does not match the owner of the UTXO, return an error. 2. If the sum of the denominations of all input UTXO is less than the sum of the denominations of all output UTXO, return an error. 3. Return S with all input UTXO removed and all output UTXO added."

4. Nakamoto (2008), 6쪽(§10 Privacy). 원문: "Some linking is still unavoidable with multi-input transactions, which necessarily reveal that their inputs were owned by the same owner. The risk is that if the owner of a key is revealed, linking could reveal other transactions that belonged to the same owner."

5. Buterin (2014), 「Ethereum Accounts」 절. 원문: "In Ethereum, the state is made up of objects called 'accounts', with each account having a 20-byte address and state transitions being direct transfers of value and information between accounts. An Ethereum account contains four fields: The nonce, a counter used to make sure each transaction can only be processed once; The account's current ether balance; The account's contract code, if present; The account's storage (empty by default)".

6. Gavin Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Shanghai version efc5f9a (2025-02-04), §4.1 World State(4쪽). 원문: "The world state (state), is a mapping between addresses (160-bit identifiers) and account states ...". "nonce: A scalar value equal to the number of transactions sent from this address or, in the case of accounts with associated code, the number of contract-creations made by this account." "balance: A scalar value equal to the number of Wei owned by this address." "storageRoot: A 256-bit hash of the root node of a Merkle Patricia tree that encodes the storage contents of the account ...". "codeHash: The hash of the EVM code of this account ...". https://ethereum.github.io/yellowpaper/paper.pdf (2026-10-25 접근). 이 판은 Shanghai 포크 기준이며 그 뒤 포크의 변경은 반영되어 있지 않을 수 있습니다.

7. Buterin (2014), 같은 절. 원문: "In general, there are two types of accounts: externally owned accounts, controlled by private keys, and contract accounts, controlled by their contract code."

1부 기술 기초

12장 진짜 블록 해부 — 제네시스 블록 80바이트를 손으로 풀기

핵심 TAKEAWAY

1부에서 배운 것을 전부 진짜 데이터 하나에서 확인합니다. 비트코인의 첫 블록(제네시스 블록, 2009년 1월 3일)은 285바이트이고, 그 앞 80바이트가 헤더입니다. 이 장에서는 그 80바이트를 여섯 필드로 자르고, SHA-256을 두 번 걸어 블록 해시를 다시 계산해 공개된 값 `000000000019d668…` 과 맞추고, 유일한 거래(코인베이스)에 박힌 문장을 바이트에서 직접 읽어 냅니다. 블록은 신비한 것이 아니라 80바이트짜리 머리와 거래 목록이고, 누구나 집에서 다시 계산해 맞춰 볼 수 있습니다. 표준 라이브러리만 씁니다.

학습목표

1. 블록 헤더 80바이트를 버전·이전 해시·머클 루트·시각·비트·논스의 여섯 필드로 읽고, 바이트 순서(엔디안)를 다룬다.
2. 헤더의 이중 SHA-256을 계산해 공개된 블록 해시와 일치시키고, 그 해시가 nBits의 목표값 아래임을 확인한다.
3. [중급] 코인베이스 거래를 바이트 단위로 풀어 txid가 머클 루트와 같음을, 그리고 출력이 50 BTC의 P2PK임을 확인한다.
4. 「누구나 다시 계산할 수 있다」는 말이 블록체인의 검증 가능성에서 무엇을 뜻하는지 안다.

[입문]만 읽은 독자가 가져가는 것. 블록체인의 「블록」은 80바이트짜리 머리 + 거래 목록이고, 누구나 집에서 다시 계산해 맞춰 볼 수 있습니다.

1. 데이터는 어디서 왔나

이 장의 바이트는 이 책에 동봉한 스냅샷 파일에서 읽습니다. `genesis_block.hex`(285바이트 블록 전체)와 `genesis_header.hex`(앞 80바이트)입니다. 2026년 10월 24일(UTC)에 서로 다른 두 블록 탐색기의 공개 API에서 원시 바이트를 받아 **바이트 단위로 같음을** 확인했고, 그 값을 Bitcoin Core와 btcd 두 구현의 소스 코드에 박힌 제네시스 상수와 대조했습니다.[1] 이 책의 코드는 네트워크를 부르지 않고 이 파일만 읽습니다(파일이 없는 환경을 위해 같은 바이트를 코드 안에도 적어 두었습니다).

Bitcoin Core 소스에는 제네시스 블록이 **상수로** 들어 있습니다. 노드는 이 블록을 네트워크에서 받지 않고 자기 안에서 만듭니다. 그 코드는 시각 1231006505, 논스 2083236893, 비트 0x1d00ffff, 버전 1, 보상 50 코인을 넣어 블록을 만들고, 그 해시가 `000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f` 이고 머클 루트가

`4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b` 임을 단언(assert) 합니다.[2] 아래에서 우리는 이 단언을 밖에서 다시 합니다.

2. 80바이트를 여섯 칸으로

[중급] 먼저 바이트 순서 이야기를 해야 합니다. 비트코인은 수를 **리틀엔디안으로** 적습니다. 낮은 자리 바이트가 앞에 옵니다. 10진수 1231006505는 16진수로 `0x495FAB29` 인데, 헤더에는 `29 AB 5F 49` 순서로 들어갑니다. 해시값도 마찬가지로 내부에서는 바이트가 거꾸로 저장되고, 사람이 읽는 `000000000019d668…` 은 그것을 뒤집어 표시한 것입니다. 「해시가 0으로 시작한다」는 말은 표시 형식에서 참이고, 메모리 안에서는 0이 **끝에** 있습니다. 이 뒤집기를 빠뜨리면 아래 계산이 하나도 맞지 않습니다.

```
# NB12-1: 제네시스 블록 헤더 80바이트를 여섯 필드로 자른다 (동봉 스냅샷 파일, 없으면 같은 바이트의 상수)
import struct, hashlib, datetime, os

HEADER_HEX =
("01000000000000000000000000000000000000000000000000000000000000000000000003ba3edfd7a7b12b27ac72c
3e67768f617fc81bc3888a51323a9fb8aa4b1e5e4a29ab5f49ffff001d1dac2b7c")   # data/genesis_header.hex
와 같은 바이트

def load_hex(path, fallback):
    try:
        with open(path) as f: return f.read().strip()
    except OSError:
        return fallback

header = bytes.fromhex(load_hex("data/genesis_header.hex", HEADER_HEX))
print("헤더 길이:", len(header), "바이트")
assert header.hex() == HEADER_HEX, "동봉 파일과 상수가 다르다"

fields = [("버전(nVersion)", 0, 4), ("이전 블록 해시(hashPrevBlock)", 4, 32), ("머클 루트
(hashMerkleRoot)", 36, 32),
          ("시각(nTime)", 68, 4), ("목표(nBits)", 72, 4), ("논스(nNonce)", 76, 4)]
for name, off, size in fields:
    raw = header[off:off + size]
    print(f"[{off:>2}:{off+size:>2}] {name:<28} {raw.hex()}")

nVersion = struct.unpack("<I", header[0:4])[0]          # "<" = 리틀엔디안
hashPrev = header[4:36][::-1].hex()                     # 해시는 뒤집어서 표시
merkle   = header[36:68][::-1].hex()
nTime    = struct.unpack("<I", header[68:72])[0]
nBits    = struct.unpack("<I", header[72:76])[0]
nNonce   = struct.unpack("<I", header[76:80])[0]
print()
print("nVersion =", nVersion)
print("hashPrevBlock =", hashPrev, "(제네시스라 0 — 가리킬 부모가 없다)")
print("hashMerkleRoot =", merkle)
print("nTime =", nTime, "=", datetime.datetime.fromtimestamp(nTime,
datetime.timezone.utc).strftime("%Y-%m-%d %H:%M:%S UTC"))
print(f"nBits = 0x{nBits:08x}  ({nBits})")
print("nNonce =", nNonce, f"(0x{nNonce:08x})")
print("Core 소스의 CreateGenesisBlock(1231006505, 2083236893, 0x1d00ffff, 1, 50*COIN) 과
일치하는가:",
      (nTime, nNonce, nBits, nVersion) == (1231006505, 2083236893, 0x1d00ffff, 1))
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
헤더 길이: 80 바이트
[ 0: 4] 버전(nVersion)                  01000000
[ 4:36] 이전 블록 해시(hashPrevBlock)
0000000000000000000000000000000000000000000000000000000000000000
[36:68] 머클 루트(hashMerkleRoot)
3ba3edfd7a7b12b27ac72c3e67768f617fc81bc3888a51323a9fb8aa4b1e5e4a
[68:72] 시각(nTime)                     29ab5f49
[72:76] 목표(nBits)                     ffff001d
[76:80] 논스(nNonce)                    1dac2b7c

nVersion = 1
hashPrevBlock = 0000000000000000000000000000000000000000000000000000000000000000 (제네시스라 0 — 가리킬 부모가 없다)
hashMerkleRoot = 4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b
nTime = 1231006505 = 2009-01-03 18:15:05 UTC
nBits = 0x1d00ffff  (486604799)
nNonce = 2083236893 (0x7c2bac1d)
Core 소스의 CreateGenesisBlock(1231006505, 2083236893, 0x1d00ffff, 1, 50*COIN) 과 일치하는가: True
```

[중급] 여섯 필드가 Core 소스의 네 상수와 맞아떨어집니다. 시각 1231006505는 2009년 1월 3일 18시 15분 5초(UTC)입니다. 이전 블록 해시가 0인 것은 제네시스 블록만의 특징입니다. 7장의 사슬은 이 0에서 시작합니다.



그림 12-1. 블록 헤더 해부기. 제네시스 블록 헤더 80바이트를 여섯 필드로 색칠하고, 이중 SHA-256 을 다시 계산해 공개된 블록 해시 000000000019d668… 과 대조한다. 논스를 1만 바꿔도 조건이 깨진다. — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 12장 2절).

3. 해시를 다시 계산해 공개값과 맞춘다

[중급] 4장과 7장에서 말한 대로 블록 해시는 헤더 80바이트의 이중 SHA-256입니다. 계산하고, 뒤집고, 공개값과 비교합니다. 그리고 8장의 규칙대로 그 해시가 nBits가 가리키는 목표값 아래인지 봅니다.

```
# NB12-2: 이중 SHA-256 → 블록 해시. 공개값과 일치하는가, 목표값 아래인가, 논스를 바꾸면 어떻게 되는가
def sha256d(b: bytes) -> bytes:
    return hashlib.sha256(hashlib.sha256(b).digest()).digest()

KNOWN = "000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f"   # Core
chainparams.cpp 의 assert 값
block_hash = sha256d(header)[::-1].hex()
print("계산한 블록 해시:", block_hash)
print("공개된 블록 해시:", KNOWN)
print("일치:", block_hash == KNOWN)

def nbits_to_target(nbits: int) -> int:
    return (nbits & 0x007FFFFF) * 256 ** ((nbits >> 24) - 3)
target = nbits_to_target(nBits)
h_int = int(block_hash, 16)
print("목표값 T = 0x%064x" % target)
print("해시 ≤ 목표값 (유효한 작업증명):", h_int <= target)
print("해시의 앞 0비트 수:", 256 - h_int.bit_length(), "(목표값이 요구하는 최소는 32)")

# 논스를 1만 바꾸면
tampered = header[:76] + struct.pack("<I", nNonce + 1)
t_hash = sha256d(tampered)[::-1].hex()
print("논스를 2083236894 로 바꾼 헤더의 해시:", t_hash, "→ 목표값 아래?", int(t_hash, 16) <= target)
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
계산한 블록 해시: 000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
공개된 블록 해시: 000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
일치: True
목표값 T = 0x00000000ffff0000000000000000000000000000000000000000000000000000
해시 ≤ 목표값 (유효한 작업증명): True
해시의 앞 0비트 수: 43 (목표값이 요구하는 최소는 32)
논스를 2083236894 로 바꾼 헤더의 해시:
9b227a4a5daa0cbae6874144bc5d7797d0513e320aceadeb3b06304971a41b1c → 목표값 아래? False
```

세 줄로 요약됩니다. 계산한 해시가 공개된 해시와 같다. 그 해시는 목표값보다 작다(앞에 0이 32비트보다 많다). 논스를 1만 바꿔도 조건이 깨진다. 2009년 1월 3일에 누군가 이 논스 2083236893을 찾기 위해 평균 수십억 번의 해시를 계산했고(8장의 「난이도 1에서 블록당 약 2^{32} 번」), 우리는 지금 해시 두 번으로 그 사실을 검증했습니다. 만들기는 어렵고 검증은 쉽다. 작업증명의 비대칭이 이것입니다.

4. [중급] 거래를 풀어 본다 — 코인베이스와 그 안의 문장

[중급] 헤더 뒤에는 거래 수와 거래들이 옵니다. 제네시스 블록에는 거래가 하나뿐이고, 그것은 **코인베이스(coinbase)** 거래, 곧 「새 코인을 블록 생성자에게 주는 특별한 거래」입니다(8장의 백서 §6). 코인베이스 거래의 입력은 이전 출력을 가리키지 않고(해시 0, 인덱스 0xffffffff), 그 자리의 스크립트에 채굴자가 임의의 데이터를 넣을 수 있습니다. 제네시스 블록의 그 자리에 유명한 문장이 있습니다. 아래 코드는 그 문장을 **바이트에서 직접 읽습니다.** 이 책은 그 문장을 기억으로 쓰지 않습니다.

```
# NB12-3: 블록 전체 285바이트 — 거래 수, 코인베이스 거래의 입력·출력, 그 안의 문장, txid == 머클 루트
BLOCK_HEX = HEADER_HEX +
("0101000000010000000000000000000000000000000000000000000000000000000000000000ffffffff4d04ffff001d010445546865205469696d65732030332f4a616e2f32303039204368616e63656c6c6f72206f6e206272696e6b206f66207365636f6e64206261696c6f757420666f722062616e6b73ffffffff0100f2052a01000000434104678afdb0fe5548271967f1a67130b7105cd6a828e03909a67962e0ea1f61deb649f6bc3f4cef38c4f35504e51ec112de5c384df7ba0b8d578a4c702b6bf11d5fac00000000")
block = bytes.fromhex(load_hex("data/genesis_block.hex", BLOCK_HEX))
assert block.hex() == BLOCK_HEX and block[:80] == header
print("블록 길이:", len(block), "바이트 | 헤더 80 + 나머지", len(block) - 80)

pos = 80
tx_count = block[pos]; pos += 1                    # 거래 수(가변 길이 정수 — 여기서는 1바이트)
print("거래 수:", tx_count)
tx_start = pos
version = struct.unpack("<I", block[pos:pos+4])[0]; pos += 4
in_count = block[pos]; pos += 1
prev_txid = block[pos:pos+32][::-1].hex(); pos += 32
prev_index = struct.unpack("<I", block[pos:pos+4])[0]; pos += 4
script_len = block[pos]; pos += 1
script_sig = block[pos:pos+script_len]; pos += script_len
sequence = struct.unpack("<I", block[pos:pos+4])[0]; pos += 4
print(f"거래 버전 {version}, 입력 수 {in_count}, 이전 출력 = ({prev_txid[:8]}…, 0x{prev_index:08x}) ← 가리키는 출력 없음 = 코인베이스")
print("입력 스크립트(scriptSig) 길이:", script_len, "바이트")

# scriptSig 를 「푸시」 단위로 읽는다: 앞 바이트가 n(1~75)이면 그 뒤 n 바이트를 스택에 올린다
i, pushes = 0, []
while i < len(script_sig):
    n = script_sig[i]; pushes.append(script_sig[i+1:i+1+n]); i += 1 + n
for k, p in enumerate(pushes):
    print(f"  푸시 {k}: {len(p)}바이트  {p.hex() if len(p) <= 4 else p.hex()[:16] + '…'}")
print("  푸시 0 을 리틀엔디안 정수로:", int.from_bytes(pushes[0], "little"), "(= nBits 486604799 와 같은 수)")
print("  푸시 2 를 ASCII 로:", repr(pushes[2].decode("ascii")))

out_count = block[pos]; pos += 1
value = struct.unpack("<Q", block[pos:pos+8])[0]; pos += 8
spk_len = block[pos]; pos += 1
spk = block[pos:pos+spk_len]; pos += spk_len
locktime = struct.unpack("<I", block[pos:pos+4])[0]; pos += 4
print(f"출력 수 {out_count}, 금액 {value:,} 사토시 = {value/1e8} BTC, 잠금 스크립트 {spk_len}바이트, locktime {locktime}")
print("  잠금 스크립트 = [푸시 65바이트] 공개키(앞 바이트 0x%02x) … [0x%02x = OP_CHECKSIG]" % (spk[1], spk[-1]))
print("  공개키 65바이트 =", spk[1:66].hex())

tx_bytes = block[tx_start:pos]
print("거래 길이:", len(tx_bytes), "바이트 | 블록 끝까지 다 읽었는가:", pos == len(block))
txid = sha256d(tx_bytes)[::-1].hex()
print("txid = 이중 SHA-256(거래) =", txid)
print("헤더의 머클 루트        =", merkle)
print("거래가 하나뿐이면 머클 루트 == txid :", txid == merkle)
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
블록 길이: 285 바이트 | 헤더 80 + 나머지 205
거래 수: 1
거래 버전 1, 입력 수 1, 이전 출력 = (00000000…, 0xffffffff) ← 가리키는 출력 없음 = 코인베이스
입력 스크립트(scriptSig) 길이: 77 바이트
  푸시 0: 4바이트  ffff001d
  푸시 1: 1바이트  04
  푸시 2: 69바이트  5468652054696d65…
  푸시 0 을 리틀엔디안 정수로: 486604799 (= nBits 486604799 와 같은 수)
  푸시 2 를 ASCII 로: 'The Times 03/Jan/2009 Chancellor on brink of second bailout for banks'
출력 수 1, 금액 5,000,000,000 사토시 = 50.0 BTC, 잠금 스크립트 67바이트, locktime 0
  잠금 스크립트 = [푸시 65바이트] 공개키(앞 바이트 0x04) … [0xac = OP_CHECKSIG]
  공개키 65바이트 =
04678afdb0fe5548271967f1a67130b7105cd6a828e03909a67962e0ea1f61deb649f6bc3f4cef38c4f35504e51ec112
de5c384df7ba0b8d578a4c702b6bf11d5f
거래 길이: 204 바이트 | 블록 끝까지 다 읽었는가: True
txid = 이중 SHA-256(거래) = 4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b
헤더의 머클 루트          = 4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b
거래가 하나뿐이면 머클 루트 == txid : True
```

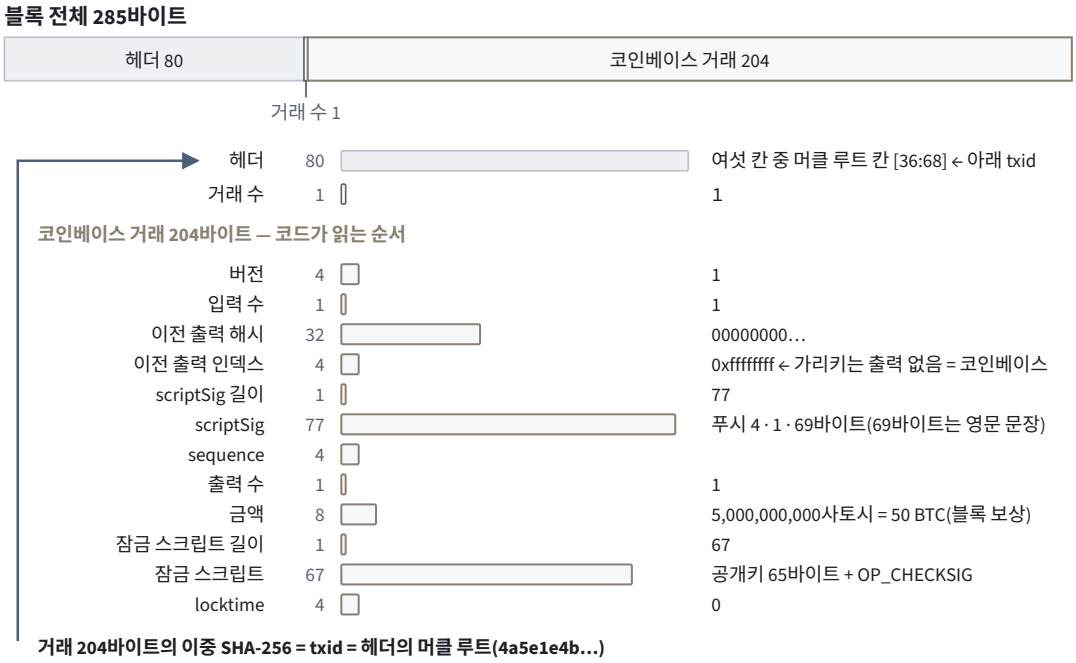


그림 12-2. 제네시스 블록 285바이트를 헤더 80, 거래 수 1, 코인베이스 거래 204로 가르고, 거래를 NB12-3이 읽는 순서대로 칸마다 펼친 바이트 지도. 막대 길이는 바이트 수에 비례하고, 거래 전체의 이중 SHA-256이 헤더의 머클 루트와 같다. 50 BTC는 제네시스 블록의 보상으로 적힌 금액이다.

바이트가 말한 것을 모읍니다. 거래는 하나이고, 입력은 아무것도 가리키지 않으며(코인베이스), 입력 스크립트에는 세 조각이 들어 있습니다. nBits와 같은 수 486604799, 숫자 4, 그리고 69바이트의 영문 문장. 출력은 하나, 금액은 5,000,000,000사토시 곧 50 BTC, 잠금 조건은 「이 65바이트 공개키로 서명하라」(P2PK, 5장). 거래 전체의 이중 SHA-256이 헤더의 머클 루트와 같습니다(6장의 「거래 하나면 뿌리 = txid」). 그리고 그 문장은 데이터에서 읽은 그대로 `The Times 03/Jan/2009 Chancellor on brink of second bailout for banks` 입니다. 이 문장은 Bitcoin Core 소스의 상수와도 글자 단위로 같습니다.[2]

[중급] 이 문장이 왜 거기 있는지에 관해 이 책은 추측하지 않습니다. 기술적으로 확인되는 것은 두 가지입니다. 첫째, 그것은 신문 제목과 같은 꼴의 문자열이고, 그 제목이 실제로 그 날짜의 신문에 실렸다면(이 책은 신문 원문을 대조하지 않았습니다) 예측할 수 없는 사건을 담은 것이므로 **그 블록이 그**

날짜 이전에 만들어졌을 수 없다는 증거가 됩니다(4장의 하버·스토네타가 말한 방법 — 예측할 수 없는 사건의 비트를 넣어 「이후」임을 보이는 것). 둘째, Bitcoin Core는 제네시스 블록의 코인베이스 출력을 **쓸 수 없는 것으로** 다룹니다. 소스의 주석대로 「제네시스 블록의 특수 처리 — 그 거래들의 연결을 건너뛴다(코인베이스가 쓸 수 없음)」.[3] 50 BTC는 원장에 적혀 있지만 누구도 쓸 수 없습니다.

5. 2026년의 블록과 견주면

제네시스 블록의 nBits `0x1d00ffff` 는 난이도 1입니다. 이 책의 기준일(2026-10-25, 관측은 2026-10-24 UTC)에 체인 끝이었던 블록(높이 972,958)의 nBits는 `0x17020093`, 난이도 약 1.4×10^{14} 였습니다(2026-10 기준, 8장과 부록 A).[4] 헤더의 모양은 17년 동안 바뀌지 않았습니다. 같은 80바이트, 같은 여섯 칸, 같은 이중 SHA-256. 바뀐 것은 nBits가 가리키는 목표값과, 그것을 맞추기 위해 네트워크가 쏟는 연산의 양입니다. 그 블록의 헤더 역시 이 장의 코드로 똑같이 풀립니다. 동봉 데이터에는 두 블록(971,712와 972,958)의 해시와 nBits가 있으니 nBits 변환은 바로 해 볼 수 있고, 80바이트 헤더 원시값은 동봉하지 않았으므로 공개 노드나 탐색기에서 직접 받아야 합니다.

6. 정리 — 1부를 닫으며

1부의 아홉 장이 제네시스 블록 285바이트 안에 다 들어 있습니다. 해시(4장)가 블록의 이름을 만들고, 서명과 공개키(5장)가 출력의 잠금 조건이며, 머클 루트(6장)가 거래를 헤더에 묶고, 이전 블록 해시(7장)가 사슬을 만들고, nBits와 논스(8장)가 작업증명이며, 그 위에 쌓인 블록 수(9장)가 「확정」의 정도이고, 출력의 집합(11장)이 상태입니다. 지분증명(10장)은 이 틀에서 「논스 찾기」를 「보증금과 투표」로 바꾼 것입니다. 이 모든 것을 **집에서 다시 계산할 수 있다는** 사실이 블록체인이 「신뢰 없이」 돌아간다고 말할 때의 실제 내용입니다. 누구의 말도 믿을 필요가 없고, 바이트와 해시 함수만 믿으면 됩니다. 2부는 이 원장 위에서 **코드가 실행되는** 세계로 갑니다.

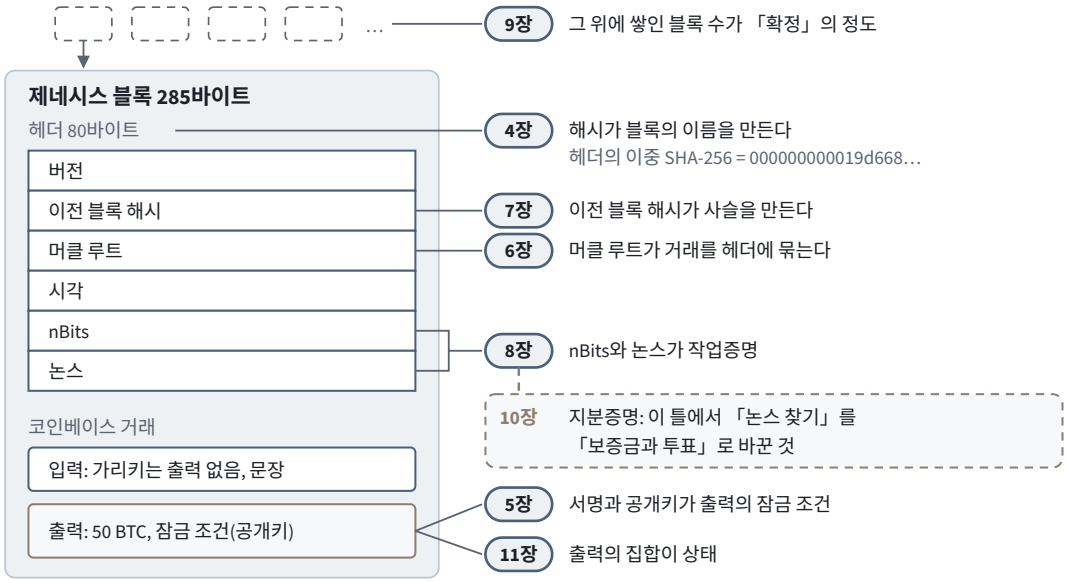


그림 12-3. 1부의 장들이 제네시스 블록 285바이트의 어느 자리에 들어 있는지 붙인 지도. 10장의 지분증명은 이 틀에서 「논스 찾기」를 「보증금과 투표」로 바꾼 곁가지로 표시했다.

이 장의 출처

- 이 책의 동봉 데이터 `data/genesis_block.hex`, `data/genesis_header.hex`, `data/genesis_block.meta.json`, `data/hashrate_2026-10-25.json` (기준일 2026-10-25) — 출처·접근 시각·대조 결과는 각 파일과 데이터 폴더 README에 있음.
- Bitcoin Core v31.1 (태그 커밋 9be056a8a72b624dae9623b2f7bded92c2a21c91): `src/kernel/chainparams.cpp` (제네시스 상수·문장·출력 스크립트·assert), `src/validation.cpp` (제네시스 코인베이스 특수 처리). 2026-10-25 접근.
- btcd (커밋 dd169fe0c32a56087d28b92fff3b8cf4fb43477a), `chaincfg/genesis.go` — 독립 구현의 제네시스 상수.
- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — 코인베이스(§6, 4쪽), 80바이트 헤더(§7, 4쪽).
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `struct`·`hashlib`·`datetime` 만 사용. 노트북 NB12.

각주

1. 이 책의 `data/genesis_block.meta.json` (기준일 2026-10-25). 출처 1: Blockstream Esplora `/api/block/000000000019d668…/raw` (2026-10-24T18:15:22Z), 출처 2: mempool.space 같은 경로(18:15:24Z). 두 응답 285바이트가 바이트 단위로 같고, 그 원시 블록 285바이트의 SHA-256은 `5299fac924b5a2fc19a88876a0042c19ac4d11fe69c3f66e47516e26185f9e99`. 소스 대조: Bitcoin Core v31.1 `src/kernel/chainparams.cpp` (커밋 9be056a8) L134–L137 `CreateGenesisBlock(1231006505, 2083236893, 0x1d00ffff, 1, 50 * COIN)` 와 해시·머클 루트 assert; btcd(커밋 dd169fe0) `chaincfg/genesis.go` Version 1·Timestamp 0x495fab29·Bits 0x1d00ffff·Nonce 0x7c2bac1d.

2. Bitcoin Core v31.1, `src/kernel/chainparams.cpp` 메인넷: `genesis = CreateGenesisBlock(1231006505, 2083236893, 0x1d00ffff, 1, 50 * COIN); consensus.hashGenesisBlock = genesis.GetHash(); assert(consensus.hashGenesisBlock == uint256{"000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f"}); assert(genesis.hashMerkleRoot == uint256{"4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b"});`. 같은 파일의 `CreateGenesisBlock` 은 `const char* pszTimestamp = "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks";` 와 출력 스크립트 `"04678afdb0fe…b6bf11d5f"_hex << OP_CHECKSIG` 를 씁니다. 2026-10-25 접근.

3. Bitcoin Core v31.1, `src/validation.cpp`: `// Special case for the genesis block, skipping connection of its transactions` `// (its coinbase is unspendable)` `if (block_hash == params.GetConsensus().hashGenesisBlock) {`. 2026-10-25 접근.

4. 이 책의 `data/hashrate_2026-10-25.json`. 블록 972,958의 해시 `00000000000000000000001857369fd717b186adc5c22960b54b87875029cc65ee`, nBits 0x17020093, 두 탐색기의 헤더 바이트 일치·이중 SHA-256 재계산 일치·해시 ≤ 목표값 확인.

2부 스마트계약과 응용

13장 이더리움 가상머신과 가스 — 코드를 실행하는 원장

핵심 TAKEAWAY

비트코인 원장은 「누가 얼마를 가졌나」를 적습니다. 이더리움 원장은 그 위에 **프로그램과 그 프로그램의 기억(저장소)을** 함께 적고, 거래가 들어오면 모든 노드가 같은 프로그램을 같은 순서로 실행해 같은 결과에 이릅니다. 이 실행기를 이더리움 가상머신(EVM)이라 부릅니다. 어떤 프로그램이든 돌릴 수 있는 기계는 멈추지 않는 프로그램도 돌릴 수 있으므로, 이더리움은 명령마다 값을 매긴 **가스(gas)를** 미리 사게 하고 가스가 바닥나면 실행을 멈추고 변경을 되돌립니다. 이 장을 읽으면 「스마트계약」이라는 말이 가리키는 것이 법적 의미의 계약이 아니라 **조건이 맞으면 스스로 실행되는 저장된 프로그램이고**, 가스가 그 프로그램을 멈추게 하는 안전장치라는 것을 압니다. 「계약인가」라는 질문은 이 장이 열고 5부가 받습니다.

학습목표

1. 「코드가 원장에서 실행된다」는 말의 뜻을 외부 소유 계정·계약 계정·거래·메시지의 네 낱말로 설명한다.
2. 가스가 왜 필요한지(정지 문제·자원 과금)를 말하고, 거래가 가스를 다 쓰면 무엇이 되돌려지고 무엇이 남는지 안다.
3. [중급] 거래의 가스 관련 필드(가스 한도, 최대 수수료, 우선 수수료)와 수수료의 몫(소각·검증자)을 읽는다. 금액 산식은 21장으로 넘긴다.
4. [중급] Solidity로 쓴 가장 작은 계약을 읽고, Python으로 상태 전이와 가스 소진을 흉내 낸다.

[입문]만 읽은 독자가 가져가는 것. 스마트계약은 「계약」이 아니라 **조건부 자동 실행 프로그램이고**, 가스는 그 프로그램을 멈추게 하는 안전장치입니다.

1. 이 기술은 무엇의 실패에서 나왔나

비트코인에도 프로그램은 있습니다. 출력마다 「이 코인을 쓰려면 이 조건을 풀어라」는 짧은 스크립트가 붙어 있고(11장), 그 조건은 대개 「이 공개키의 서명을 내라」입니다. 부테린(Vitalik Buterin)의 2014년 백서는 이 스크립트가 못 하는 일을 넷으로 꼽습니다. 반복(loop)이 없어 어떤 계산은 아예 적을 수 없고, 출력에서 얼마를 꺼낼지 세밀하게 정할 수 없으며, 출력은 「쓰였다/안 쓰였다」 둘뿐이어서 여러 단계로 진행되는 계약을 짤 수 없고, 스크립트가 블록 헤더 같은 체인 안의 다른 정보를 읽을 수 없다는 것입니다.[1] 11장은 이 중 가운데 둘을 받아 계정 모형을 설명했습니다. 이 장이 받는 것은 첫째와 넷째입니다.

그래서 백서가 제안한 것은 특정 응용이 아니라 **바닥 층입니다.** 「튜링 완전한 프로그래밍 언어가 내장된

블록체인으로, 누구나 스마트계약과 탈중앙 응용을 써서 소유권·거래 형식·상태 전이 함수에 관한 자기만의 규칙을 만들 수 있게 하는 것.」 그리고 스마트계약을 이렇게 정의합니다. 「값을 담고 있다가 특정 조건이 맞을 때에만 그것을 풀어 주는 암호학적 「상자」.」[2] 이 정의에 「당사자」나 「합의」라는 말이 없다는 점을 눈여겨보십시오. 상자와 조건과 값이 있을 뿐입니다. 이 장의 질문 상자는 여기서 나옵니다.

우드(Gavin Wood)의 황서(Yellow Paper)는 이 체계의 동기를 더 직설적으로 적습니다. 「이 프로젝트의 한 가지 핵심 목표는 서로를 신뢰할 방법이 달리 없는 동의한 개인들 사이의 거래를 가능하게 하는 것이다. 그 이유는 지리적 거리, 접속의 어려움, 또는 기존 법체계의 비호환성·무능·비협조·비용·불확실성·불편·부패일 수 있다.」 그리고 「풍부하고 모호하지 않은 언어로 상태 변화 체계를 명시하고, 합의가 자율적으로 집행될 것이라고 합리적으로 기대할 수 있게 체계를 설계함으로써」 그 목적을 이룬다고 씁니다.[3] 법률가가 읽으면 멈춰 서게 되는 문장입니다. 법체계의 결함을 대신하겠다는 선언이니까요. 이 책은 그 선언을 평가하지 않고, 그것이 **기술적으로 무엇을 뜻하는지만** 이 장에서 보입니다.

2. 직관 — 계정, 코드, 거래, 메시지

11장에서 이더리움 계정이 네 필드(논스·잔고·코드·저장소)를 갖고, 외부 소유 계정과 계약 계정 두 종류가 있다는 것을 보았습니다. 외부 소유 계정은 개인키로 움직이고 코드가 없습니다. 계약 계정은 코드가 있고 개인키가 없습니다. 계약 계정을 움직이는 유일한 방법은 **그 계정에 메시지를 보내는 것이고**, 메시지가 도착하면 그 계정의 코드가 실행됩니다.

백서는 「메시지」와 「거래」를 구별합니다. 「이더리움의 「메시지」는 비트코인의 「거래」와 어느 정도 비슷하지만 세 가지 중요한 차이가 있다. 첫째, 이더리움 메시지는 외부 주체가 만들 수도 있고 계약이 만들 수도 있다(비트코인 거래는 외부에서만 만들어진다). 둘째, 이더리움 메시지는 데이터를 담을 수 있다는 명시적 선택지가 있다. 마지막으로, 이더리움 메시지의 수신자가 계약 계정이면 응답을 돌려줄 수 있다. 이는 이더리움 메시지가 함수의 개념도 포괄한다는 뜻이다.」 그리고 「「거래」라는 용어는 외부 소유 계정에서 보내는 메시지를 담은 **서명된 데이터 묶음을** 가리키는 데 쓴다.」[4] 쉽게 말하면 거래는 사람이 시작하는 것이고, 메시지는 그 거래가 계약을 건드려 계약이 다시 다른 계약을 부를 때 생기는 안쪽의 호출까지 포함합니다. 계약이 계약을 부를 수 있다는 것, 이것이 백서가 「1급 시민」이라 부르는 성질입니다. 「계약은 외부 계정과 동등한 힘을 가지며, 메시지를 보내고 다른 계약을 만들 수도 있다.」[5]

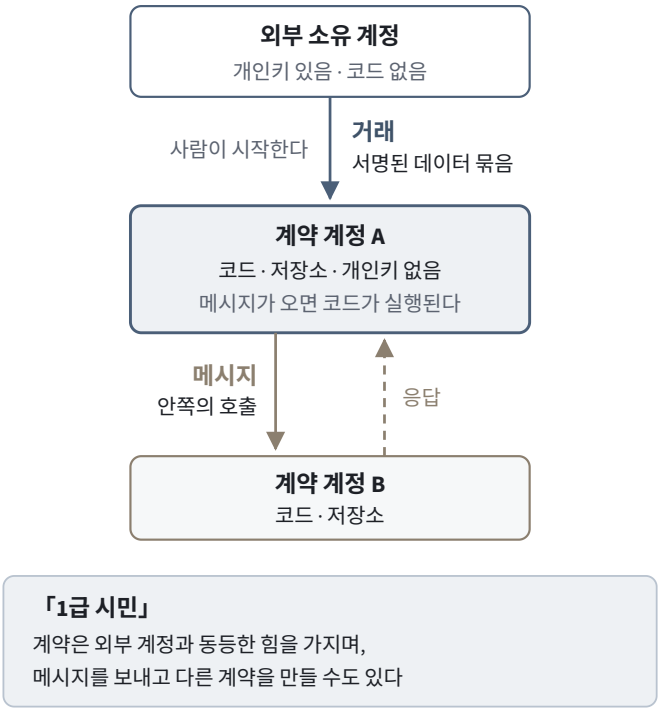


그림 13-1. 외부 소유 계정이 서명한 거래가 계약 계정 A의 코드를 실행시키고, A가 계약 B에 메시지를 보내 응답을 받는 구조. 거래는 사람이 시작하고, 메시지는 계약이 만드는 안쪽의 호출까지 포함한다(「1급 시민」).

거래 안에 무엇이 들어 있는지는 11장 비교표에서 「가스 관련 필드」라고만 적고 넘겼습니다. 여기서 풉니다. 백서의 거래는 「메시지의 수신자, 보내는 사람을 식별하는 서명, 보낼 이더의 양과 데이터, 그리고 STARTGAS와 GASPRICE라는 두 값」을 담습니다.[6] 황서(상하이판)는 이것을 정식 필드로 적습니다. 모든 거래에 공통인 것은 형식(type), 논스, **가스 한도(gasLimit)**, 수신자(to), 금액(value), 서명값 r·s이고, 런던 포크에서 들어온 2형 거래에는 **최대 가스당 수수료(maxFeePerGas)와 최대 우선 수수료(maxPriorityFeePerGas)가**, 그 전의 0형·1형 거래에는 **가스 가격(gasPrice)** 하나가 있습니다. 계약을 새로 만드는 거래에는 초기화 코드(init)가, 보통 거래에는 데이터(data)가 붙습니다.[7] 가스 한도에 황서가 붙인 설명이 이 장의 핵심입니다. 「이 거래를 실행하는 데 쓸 가스의 최대량. 이것은 어떤 계산이 이루어지기 전에 **미리 지불되고**, 나중에 늘릴 수 없다.」

거래 하나가 지나가는 길

백서는 상태 전이 함수 `APPLY(S, TX) → S'` 를 여섯 단계로 적습니다. 11장의 UTXO 판과 나란히 읽어 보십시오.[8]

1. 거래가 형식에 맞고, 서명이 유효하고, 논스가 보내는 계정의 논스와 맞는지 확인한다. 아니면 오류.
2. 거래 수수료를 STARTGAS × GASPRICE 로 계산하고, 서명에서 보내는 주소를 알아낸다. 그 계정에서 수수료를 빼고 논스를 올린다. 잔고가 모자라면 오류.
3. GAS = STARTGAS 로 두고, 거래의 바이트 수에 비례한 가스를 먼저 뗀다.
4. 금액을 보내는 계정에서 받는 계정으로 옮긴다. 받는 계정이 없으면 만든다. 받는 계정이 계약이면 **그 코드를 끝까지, 또는 가스가 바닥날 때까지 실행한다.**
5. 잔고 부족으로 송금이 실패했거나 코드 실행이 가스를 다 썼으면, **수수료 지급을 뺀 모든 상태 변화를 되돌리고**, 수수료는 채굴자 계정에 더한다.

6. 그렇지 않으면 남은 가스의 수수료를 보내는 사람에게 돌려주고, 쓴 가스의 수수료를 채굴자에게 보낸다.

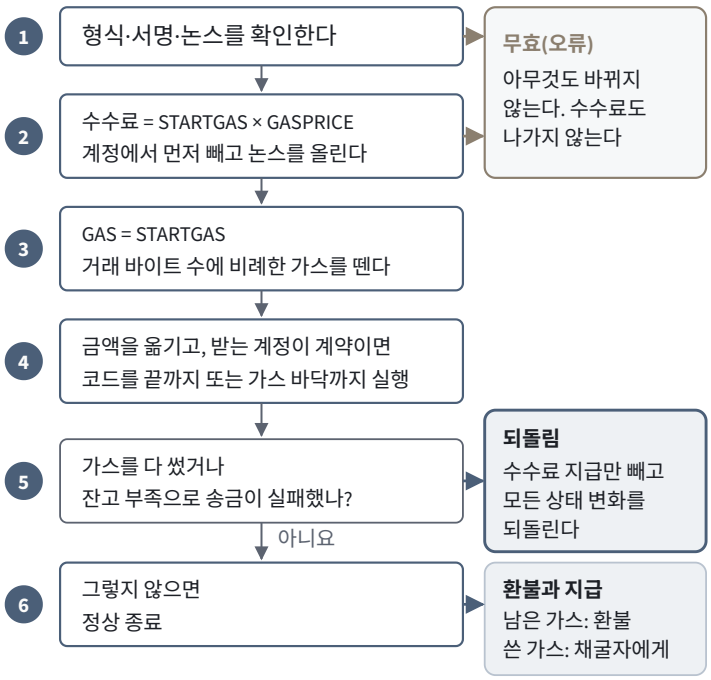


그림 13-2. 백서의 상태 전이 함수 APPLY(S, TX) → S′ 여섯 단계와 세 결말. 1·2단계에서 걸리면 무효로 아무것도 바뀌지 않고, 가스를 다 쓰면 수수료 지급만 남기고 되돌리며, 정상 종료면 남은 가스를 환불한다. 「채굴자」는 2014년 백서의 용어다.

5단계와 6단계가 가스의 두 얼굴입니다. 가스를 다 쓰면 **실행은 되돌려지지만 수수료는 남습니다.** 가스가 남으면 **남은 만큼은 돌아옵니다.** 백서의 한 문장으로는, 「거래 실행이 「가스를 다 쓰면」 모든 상태 변화는 되돌려진다 — 수수료 지급만 빼고. 그리고 가스가 남은 채 실행이 멈추면 남은 몫의 수수료는 보내는 사람에게 환불된다.」[9] 이 두 규칙이 왜 필요한지가 다음 절입니다.

3. 가스 — 멈추지 않는 프로그램을 멈추는 법

EVM은 반복과 재귀를 허용합니다. 백서는 그 결과를 숨기지 않습니다. 「이더리움 가상머신은 튜링 완전하다. 이는 EVM 코드가 생각할 수 있는 어떤 계산이든, 무한 루프까지 포함해 부호화할 수 있다는 뜻이다.」 그러면 「악의적인 사용자가 채굴자와 풀 노드를 무한 루프에 빠뜨려 사실상 멈추게 할 수 있는가」라는 문제가 생기고, 그 뿌리는 「컴퓨터과학에서 **정지 문제(halting problem)로** 알려진 문제, 곧 주어진 프로그램이 언제고 멈출지를 일반적으로는 알아낼 방법이 없다는 것」에 있습니다.[10] 노드가 거래를 받아 실행해 보기 전에는 그것이 끝날지 알 수 없고, 실행해 보는 것 자체가 비용입니다.

해법은 미리 알 수 없는 것을 알려 하지 않고 **상한을 미리 사게 하는 것입니다.** 「우리 해법은 거래가 취할 수 있는 계산 단계의 최대 수를 거래가 정하도록 요구하는 것이고, 실행이 그보다 오래 걸리면 계산은 되돌려지지만 수수료는 여전히 지불된다.」[11] 황서는 같은 것을 한 낱말로 적습니다. EVM은 「준(準) 튜링 완전 기계이며, 「준」이라는 한정은 계산이 가스라는 매개변수로 본질적으로 유계라는 사실에서 온다.」[12] 무한 루프를 돌리는 공격자는 자기가 산 가스만큼만 노드를 괴롭힐 수 있고, 그 가스값은 공격자가 냅니다(런던 포크 뒤로는 그중 기본 수수료 몫은 소각되고 우선 수수료 몫만 검증자에게 갑니다 — 아래 「수수료는 어디로 가는가」). 백서가 든 예 하나가 이 장치의 성격을 잘 보여 줍니다. 「공격자가 `send(A, contract.storage[A]); contract.storage[A] = 0` 꼴의 코드를 가진 계약을 보고, 첫

단계는 돌리되 둘째 단계는 못 돌릴 만큼만 가스를 넣어 거래를 보낸다(곧 인출은 하되 잔고는 줄지 않게). 계약 작성자는 이런 공격을 걱정할 필요가 없다. 실행이 중간에 멈추면 변경이 되돌려지기 때문이다.」[13] 가스 소진은 「실패했지만 반쯤 된 상태」를 남기지 않습니다. 전부 되거나, 전부 없던 일이 됩니다. 수수료만 빼고요.

가스는 무엇을 재는가

[중급] 가스는 돈이 아니라 **계산량의 단위입니다.** 황서 5절: 「네트워크 남용 문제를 피하고 튜링 완전성에서 불가피하게 따라오는 질문들을 비켜 가기 위해, 이더리움의 모든 프로그램 가능한 계산에는 수수료가 붙는다. 수수료 일람은 가스 단위로 명시된다. 따라서 프로그램 가능한 계산의 어떤 조각이든 (계약 생성, 메시지 호출, 계정 저장소의 사용과 접근, 가상머신 명령의 실행을 포함해) 가스로 표시된 **보편적으로 합의된 비용을** 갖는다.」[14] 명령마다 값이 정해져 있습니다. 황서 부록 G의 요금표에서 몇 줄만 옮기면 아래와 같습니다. 덧셈 같은 가벼운 명령은 3가스, 거래 한 건의 기본값은 21,000가스, 저장소에 0이 아닌 값을 새로 쓰는 것은 20,000가스입니다. 저장소 쓰기가 덧셈의 7,000배 가까이 비싼 이유는 황서가 적듯 「저장소 사용은 모든 노드의 상태 데이터베이스가 커지는 것에 직결되므로 저장소 사용을 최소화하도록 유인하기 위해」서이고, 같은 이유로 황서 §9.2는 저장소 칸을 지우는 명령에는 「요금을 면제할 뿐 아니라 환불을 준다」고 적습니다.[15] 다만 같은 판의 부록 G는 0으로 지울 때도 G_{sreset} 2,900을 받고 R_{sclear} 4,800을 환불 계수기에 더하도록 되어 있어, §9.2의 서술과 부록 G의 표가 서로 어긋납니다. 이 책의 표와 코드는 부록 G를 따릅니다.

항목(황서 부록 G 기호)	가스	뜻
$G_{verylow}$	3	ADD·SUB·PUSH·DUP 같은 가벼운 명령 한 번
G_{mid} / G_{high}	8 / 10	JUMP / JUMPI(조건 분기) 한 번
$G_{transaction}$	21,000	거래 한 건마다 내는 기본값
$G_{txdatazero}$ / $G_{txdatanonzero}$	4 / 16	거래 데이터의 0 바이트 / 0 아닌 바이트 하나당
$G_{coldsload}$ / $G_{warmaccess}$	2,100 / 100	이 거래에서 처음 건드리는 저장소 칸 / 이미 건드린 칸의 접근
G_{sset}	20,000	저장소 칸을 0에서 0 아닌 값으로 바꿀 때
G_{sreset}	2,900	0 아닌 값을 다른 값으로 바꾸거나 0으로 지울 때
R_{sclear}	4,800	0 아닌 값을 0으로 지웠을 때 환불 계수기에 더해지는 양
G_{create}	32,000	계약 하나를 새로 만드는 CREATE 명령

위 값은 황서 상하이판(2025-02-04) 부록 G에서 옮긴 것입니다.[16] 요금표는 포크마다 고쳐져 왔으므로(예: 런던 포크의 EIP-3529가 환불 상한을 2분의 1에서 5분의 1로 줄였습니다[17]) 이 책은 「현행 값」이라고 쓰지 않습니다. 이 장의 코드는 이 표의 값을 **그대로 상수로** 씁니다.

수수료는 어디로 가는가 — 10장에서 넘어온 질문

10장은 「거래 수수료의 몫」을 이 장으로 넘겼습니다. 답은 둘로 갈립니다. 황서 5절: 「런던 하드포크에서 EIP-1559가 도입된 뒤로, 블록에 포함되는 모든 거래는 **기본 수수료(base fee)를** 내야 한다. 기본 수수료는 소비한 가스 단위당 wei로 명시되며 한 블록 안의 모든 거래에 같다. 기본 수수료를 채우기 위해 낸 이더는 **소각된다(유통에서 빠진다).**」 그리고 「검증자가 거래를 포함하도록 유인하기 위해 **우선 수수료(priority fee)라는** 추가 수수료가 있다. … 우선 수수료를 채우는 이더는 수취 주소(beneficiary), 곧 보통 검증자가 지배하는 계정의 주소로 간다.」[18] 거래를 보내는 사람이 내는 총액은 「가스당 기본 수수료와 가스당 우선 수수료의 합에 소비한 가스 총량을 곱한 것」입니다.

그러니까 10장의 검증자가 받는 「수수료 몫」은 우선 수수료뿐이고, 기본 수수료는 **누구에게도 가지 않고 사라집니다.** 왜 그렇게 설계했는지(검증자가 수수료를 조작할 유인을 없애는 것 등)와 기본 수수료가 블록마다 어떻게 조정되는지, 그리고 「소각」을 둘러싼 경제학은 21장에서 다룹니다. 이 장은 수수료가 **가스 단위 × 가스당 가격이라는** 구조와, 그 가격이 두 조각으로 갈린다는 사실까지만 적습니다.

▾ [심화] 황서의 형식 — 거래 유효성 여덟 조건, 선지급, 환불

[심화] 황서 6절은 거래가 실행에 들어가기 전에 통과해야 하는 「내재적 유효성」 검사를 산문으로 여덟 가지 적습니다(형식 정의인 식 (69)에는 초기화 코드 길이 상한과 블록 가스 한도 잔량 조건이 더 들어 있습니다). 거래가 올바른 RLP 부호화이고, 서명이 유효하고, 논스가 보내는 계정의 현재 논스와 같고, 보내는 계정에 계약 코드가 없고(EIP-3607), **가스 한도가 내재 가스 g_0 보다 작지 않고**, 계정 잔고가 선지급액 v_0 이상이고, 2형 거래의 maxFeePerGas(0형·1형은 gasPrice)가 블록의 기본 수수료 H_f 이상이고, 2형 거래에서 maxPriorityFeePerGas가 maxFeePerGas 이하라는 것입니다.[19]

[심화] **내재 가스** g_0 는 코드를 한 줄도 돌리기 전에 드는 가스입니다. 거래 데이터의 바이트마다 0이면 $G_{txdatazero}$, 아니면 $G_{txdatanonzero}$ 를 더하고, 계약 생성 거래면 $G_{txcreate}$ 와 초기화 코드 길이에 비례한 양을 더하고, 모든 거래에 $G_{transaction}$ 을 더하고, 접근 목록 항목마다 정해진 양을 더합니다.[20] 수치 예: 데이터가 비어 있는 단순 송금이면 $g_0 = 21{,}000$ 입니다. 데이터가 0 아닌 바이트 4개와 0 바이트 28개라면 $21{,}000 + 4 \times 16 + 28 \times 4 = 21{,}176$ 입니다.

[심화] **유효 가스 가격** p 와 **우선 수수료** f 는 2형 거래에서 $f = \min(T_f, T_m - H_f)$, $p = f + H_f$ 입니다. 기호 풀이: T_f 는 거래가 적은 최대 우선 수수료, T_m 은 최대 가스당 수수료, H_f 는 블록 헤더의 기본 수수료입니다. 수치 예: 기본 수수료 10, 최대 우선 2, 최대 총액 15(단위는 가스당 wei, 장난감 값)이면 $f = \min(2, 15 - 10) = 2$, $p = 12$ 입니다. 기본 수수료가 14로 오르면 $f = \min(2, 1) = 1$, $p = 15$ 가 되어 우선 수수료가 깎입니다. **선지급액** $v_0 = T_g T_m + T_v$(가스 한도 × 최대 가스당 수수료 + 송금액)인데, 실제로 잔고에서 먼저 빠지는 것은 $T_g p$ 입니다.[21] 실행이 끝나면 남은 가스 g' 에 환불 계수기 A_r 를 「쓴 가스의 5분의 1」을 상한으로 더한 g^* 를 원래 가격 p 로 돌려줍니다(식 (82), 위 각주).

[심화] **기계 상태.** EVM은 256비트 워드의 스택 기계입니다. 「워드 크기(따라서 스택 항목의 크기)는 256비트다. Keccak-256 해시 체계와 타원곡선 계산을 쉽게 하려고 택했다. 메모리 모형은 단순한 워드 주소 바이트 배열이다. 스택의 최대 크기는 1024다. 기계는 또 독립된 저장소 모형을 갖는다. … 메모리는 휘발성이지만 저장소는 비휘발성이며 시스템 상태의 일부로 유지된다.」[22] 실행 중의 기계 상태 μ 는 「(사용 가능한 가스 g, 프로그램 카운터 pc, 메모리 내용, 메모리의 활성

워드 수, 스택 내용, 반환 데이터 버퍼)」의 여섯 묶음이고,[23] **예외 정지** 함수 Z 는 「가스가 모자라거나, 명령이 유효하지 않거나, 스택 항목이 모자라거나, JUMP/JUMPI의 목적지가 유효하지 않거나, 새 스택 크기가 1024를 넘거나, 정적 호출 중에 상태 변경을 시도하면」 참이 됩니다.[24] 예외 정지는 「상태 변화를 남기지 않는다. 기계는 즉시 멈추고 실행 주체에게 문제를 보고한다」(§9.1). 아래 코드는 이 중 「가스 부족」 하나만 흉내 냅니다.

4. 읽기용 Solidity — 가장 작은 계약

[중급] 계약은 보통 EVM 바이트코드로 직접 쓰지 않고 Solidity 같은 고급 언어로 써서 컴파일합니다. Solidity 공식 문서는 첫 예제로 「단순 저장소(SimpleStorage)」를 듭니다. 숫자 하나를 저장하는 칸, 그 칸에 쓰는 함수, 읽는 함수만 있는 계약입니다.[25] 아래는 **같은 구조로 이 책이 새로 쓴 코드이고, 읽기용입니다.** 이 책은 이 코드를 컴파일하거나 배포하지 않습니다(3장 일러두기). 머리의 `SPDX-License-Identifier` 줄은 라이선스를 적는 Solidity의 관례이며, 이 코드는 책의 다른 코드처럼 CC BY-NC 4.0을 따릅니다.

```
// SPDX-License-Identifier: CC-BY-NC-4.0
pragma solidity ^0.8.0;

// 읽기용 예시 — 이 책이 새로 쓴 코드입니다. 컴파일·배포하지 않습니다.
contract OneNumber {
    uint256 number;                                  // 상태 변수: 블록체인에 남는 칸 하나

    function set(uint256 newNumber) public {         // 누구나 불러 칸을 덮어쓸 수 있다
        number = newNumber;
    }

    function get() public view returns (uint256) {   // 칸을 읽기만 한다(view)
        return number;
    }
}
```

[중급] 문서가 자기 예제를 두고 한 설명이 이 코드에도 그대로 들어맞으므로, 그 설명을 따라 읽습니다(변수 이름만 이 책의 `number` 로 바꿔 읽으면 됩니다). 「Solidity에서 계약(contract)이란 이더리움 블록체인의 특정 주소에 머무는 코드(함수들)와 데이터(상태)의 모음이다.」 상태 변수 선언 한 줄은 「256비트 부호 없는 정수형의 상태 변수 하나를 선언하며, 데이터베이스의 칸 하나라고 생각하면 된다. 그 데이터베이스를 관리하는 코드의 함수를 불러 그 칸을 조회하고 바꿀 수 있다.」 그리고 이 계약이 하는 일에 관해 문서가 덧붙이는 말이 법률가에게 중요합니다. 「이 계약은 (이더리움이 만든 기반 덕분에) **누구든 숫자 하나를 저장할 수 있게 하고, 그 숫자는 세상 누구나 읽을 수 있으며, 당신이 그것을 공개하는 것을 (현실적으로) 막을 방법은 없다는** 것 말고는 아직 별일을 하지 않는다. 누구든 `set` 을 다른 값으로 다시 불러 당신의 숫자를 덮어쓸 수 있지만, 그 숫자는 블록체인의 역사에 여전히 저장되어 있다.」[26] 상태 변수 하나, 쓰는 함수 하나, 읽는 함수 하나. 「계약」이라는 이름이 붙어 있지만 여기에는 청약도 승낙도 당사자도 없습니다. 접근 제한을 더해 「나만 바꿀 수 있게」 만드는 것은 문서의 다음 예제 몫이고, 그것이 14장의 토큰으로 이어집니다.

> **? 법률가의 질문**
>
> 위 코드는 「계약(contract)」이라는 이름을 달고 있습니다. 백서의 정의로는 「조건이 맞을 때에만 값을 풀어 주는 상자」이고, 황서의 동기로는 「합의가 자율적으로 집행될 것을 합리적으로 기대할 수 있게 하는 체계」입니다. 여기에서 청약과 승낙은 어디에 있습니까. 코드를 배포한 행위가 청약인가요, 거래를 보내 함수를 부른 행위가 승낙인가요, 아니면 둘 다 의사표시가 아닌 사실행위인가요. 코드가 작성자의 의도와 다르게 움직였을 때, 「계약 내용」은 코드인가요 의도인가요. 가스가 바닥나 실행이 되돌려졌지만 수수료는 남았다면, 무엇이 이행되고 무엇이 불이행된 것인가요. 황서는 각주에서 거래를 만드는 도구가 「인간의 개시로부터 인과적으로 너무 멀어져 … 자율적 행위자로 간주되는 것이 옳을 시점이 올 수 있다」고까지 적습니다.[27] 그때 의사표시의 주체는 누구입니까. → 28장

5. 코드 — 장난감 EVM으로 상태 전이와 가스 소진을 흉내 낸다

[중급] 실제 EVM의 명령 집합(황서 부록 H)은 이보다 훨씬 크지만, 이 장의 요점을 보이는 데는 열한 개면 충분합니다. 아래는 황서 부록 G의 요금표 값을 그대로 상수로 쓰는 장난감 기계입니다. 스택과 저장소만 있고 메모리·호출·이벤트는 없습니다. 저장소 쓰기 비용은 부록 H 규칙 중 「처음 건드리는 칸의 추가 비용」과 「0에서 0 아닌 값으로 / 그 밖」 두 갈래만 흉내 냅니다. **전부 공부용이며 실사용 금지입니다.**

```
# NB13-1: 장난감 EVM — 바이트코드를 한 명령씩 실행하며 가스를 깎는다 (공부용, 실사용 금지)
# 요금표 값은 황서(상하이판) 부록 G에서 그대로 옮겼다. 명령 집합과 저장소 비용 규칙은 크게 단순화했다.
G_VERYLOW, G_MID, G_HIGH, G_JUMPDEST = 3, 8, 10, 1
G_COLDSLOAD, G_WARMACCESS, G_SSET, G_SRESET = 2100, 100, 20000, 2900
MOD = 2**256

OPS = {0x00: "STOP", 0x01: "ADD", 0x02: "MUL", 0x03: "SUB", 0x54: "SLOAD", 0x55: "SSTORE",
       0x56: "JUMP", 0x57: "JUMPI", 0x5b: "JUMPDEST", 0x60: "PUSH1", 0x80: "DUP1"}

class OutOfGas(Exception):
    def __init__(self, msg, steps=0):
        super().__init__(msg); self.steps = steps      # 멈추기 전까지 실행한 명령 수를 함께 담는다

def run(code: bytes, storage: dict, gas: int, trace: bool = False):
    """code 를 0번지부터 실행한다. 성공: (남은 가스, 새 저장소, 실행한 명령 수). 가스 부족: OutOfGas.
    저장소는 복사본에서 바꾸므로, 예외가 나면 호출자가 원래 저장소를 그대로 쓰면 「되돌리기」가 된다."""
    st = dict(storage); stack = []; pc = 0; touched = set(); steps = 0
    def charge(c):
        nonlocal gas
        if gas < c:
            raise OutOfGas(f"pc={pc}: {c} 가스 필요, 남은 가스 {gas}", steps)
        gas -= c
    while pc < len(code):
        name = OPS.get(code[pc])
        if name is None:
            raise OutOfGas(f"pc={pc}: 알 수 없는 명령 0x{code[pc]:02x}")   # 실제 EVM은 별도 예외.
여기서는 단순화
        steps += 1
        if trace:
            print(f"    pc={pc:>2} {name:<8} stack={stack} gas={gas}")
        if name == "STOP":
            break
        elif name == "PUSH1":
            charge(G_VERYLOW); stack.append(code[pc + 1]); pc += 2; continue
        elif name == "DUP1":
            charge(G_VERYLOW); stack.append(stack[-1])
        elif name in ("ADD", "MUL", "SUB"):
            charge(G_VERYLOW); b, a = stack.pop(), stack.pop()
            stack.append({"ADD": (a + b) % MOD, "MUL": (a * b) % MOD, "SUB": (a - b) % MOD}
[name])
        elif name == "SLOAD":
            key = stack.pop(); charge(G_WARMACCESS if key in touched else G_COLDSLOAD);
touched.add(key)
            stack.append(st.get(key, 0))
        elif name == "SSTORE":
            key, val = stack.pop(), stack.pop()
            cost = 0 if key in touched else G_COLDSLOAD
            touched.add(key)
            cost += G_SSET if (st.get(key, 0) == 0 and val != 0) else G_SRESET
            charge(cost)
            st[key] = val
        elif name == "JUMPDEST":
            charge(G_JUMPDEST)
        elif name == "JUMP":
            charge(G_MID); pc = stack.pop(); continue
        elif name == "JUMPI":
            charge(G_HIGH); dest, cond = stack.pop(), stack.pop()
            if cond != 0:
                pc = dest; continue
        pc += 1
    return gas, st, steps

# OneNumber.set(42) 의 핵심 저장소 연산을 흉내 낸 바이트코드: PUSH1 42, PUSH1 0, SSTORE, STOP  (칸 0 에
42 를 쓴다; ABI 해석·RETURN 은 뺐다)
set42 = bytes([0x60, 42, 0x60, 0, 0x55, 0x00])
gas_left, st, n = run(set42, {}, gas=30_000, trace=True)
print(f"빈 저장소에 42 쓰기: 명령 {n}개, 쓴 가스 {30_000 - gas_left} (= 3 + 3 + 2100 + 20000), 저장소
{st}")

set7 = bytes([0x60, 7, 0x60, 0, 0x55, 0x00])
gas_left2, st2, n2 = run(set7, st, gas=30_000)
```

```
print(f"42 를 7 로 덮어쓰기:  명령 {n2}개, 쓴 가스 {30_000 - gas_left2} (= 3 + 3 + 2100 + 2900), 저장소
{st2}")

# OneNumber.get() 의 핵심 연산을 흉내 냄: PUSH1 0, SLOAD, STOP  — 읽기는 쓰기보다 훨씬 싸다
get = bytes([0x60, 0, 0x54, 0x00])
gas_left3, _, _ = run(get, st2, gas=30_000)
print(f"칸 0 읽기:            쓴 가스 {30_000 - gas_left3} (= 3 + 2100)")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
    pc= 0 PUSH1    stack=[] gas=30000
    pc= 2 PUSH1    stack=[42] gas=29997
    pc= 4 SSTORE   stack=[42, 0] gas=29994
    pc= 5 STOP     stack=[] gas=7894
빈 저장소에 42 쓰기: 명령 4개, 쓴 가스 22106 (= 3 + 3 + 2100 + 20000), 저장소 {0: 42}
42 를 7 로 덮어쓰기:  명령 4개, 쓴 가스 5006 (= 3 + 3 + 2100 + 2900), 저장소 {0: 7}
칸 0 읽기:            쓴 가스 2103 (= 3 + 2100)
```

[중급] 저장소에 처음 값을 쓰는 데 22,106가스, 덮어쓰는 데 5,006가스, 읽는 데 2,103가스가 들었습니다. 명령 네 개짜리 프로그램인데도 비용의 거의 전부가 **저장소 접근에서** 나옵니다. 모든 노드가 영구히 들고 있어야 하는 것은 저장소이기 때문입니다. 이것이 「체인에 데이터를 올리는 것은 비싸다」는 말의 구체적 모양이고, 18장의 롤업이 「데이터는 올리되 실행은 밖에서」 하는 이유의 절반입니다.

[중급] 이제 이 기계를 11장의 계정 상태 전이기 안에 넣습니다. 황서 6절의 순서대로, 유효성 검사 → 선지급 → 실행 → (실패면 되돌리기) → 환불 → 수수료 배분입니다. 수수료 배분은 황서 5절대로 기본 수수료는 소각하고 우선 수수료는 수취 주소로 보냅니다.

```
# NB13-2: 거래 적용 — 유효성 검사, 선지급, 실행, 되돌리기, 환불, 기본 수수료 소각·우선 수수료 지급 (공부용)
G_TRANSACTION, G_TXDATAZERO, G_TXDATANONZERO = 21000, 4, 16

def intrinsic_gas(data: bytes) -> int:                          # 황서 식 (64) 중 보통 거래(계약 생성·
접근 목록 없음) 부분
    return G_TRANSACTION + sum(G_TXDATAZERO if b == 0 else G_TXDATANONZERO for b in data)

def apply_tx(state: dict, tx: dict, base_fee: int, beneficiary: str):
    """state: {addr: {"nonce","balance","code"(bytes 또는 None),"storage"(dict)}}.
    tx: {"from","to","value","data","gas_limit","max_fee","max_priority","nonce"}.  단위는 전부
장난감 값."""
    S = {a: {**acc, "storage": dict(acc.get("storage", {}))} for a, acc in state.items()}
    sender = S[tx["from"]]
    g0 = intrinsic_gas(tx["data"])
    # --- 내재적 유효성 (황서 §6 의 (3)(5)(6)(7)(8)) ---
    if tx["nonce"] != sender["nonce"]:                  return S, f"무효: 논스 {tx['nonce']} ≠
계정 논스 {sender['nonce']}"
    if tx["gas_limit"] < g0:                            return S, f"무효: 가스 한도
{tx['gas_limit']} < 내재 가스 {g0}"
    if tx["max_fee"] < base_fee:                        return S, f"무효: 최대 수수료
{tx['max_fee']} < 기본 수수료 {base_fee}"
    if tx["max_priority"] > tx["max_fee"]:              return S, "무효: 우선 수수료 > 최대 수수료"
    v0 = tx["gas_limit"] * tx["max_fee"] + tx["value"]
    if sender["balance"] < v0:                          return S, f"무효: 잔고
{sender['balance']} < 선지급액 {v0}"
    # --- 유효 가격 (식 (66)(67)) 과 비가역 선지급 (식 (73)(74)) ---
    priority = min(tx["max_priority"], tx["max_fee"] - base_fee)
    price = base_fee + priority
    sender["nonce"] += 1
    sender["balance"] -= tx["gas_limit"] * price
    gas = tx["gas_limit"] - g0
    # --- 실행 ---
    to = S.setdefault(tx["to"], {"nonce": 0, "balance": 0, "code": None, "storage": {}})
    status = "성공"
    try:
        if to["code"]:
            gas, new_storage, _ = run(to["code"], to["storage"], gas)
            to["storage"] = new_storage
        sender["balance"] -= tx["value"]; to["balance"] += tx["value"]
    except OutOfGas as e:
        status = f"가스 소진({e}) → 실행 되돌림"
        gas = 0                                         # 저장소·송금 변경은 버린다(원래 상태 유지).
선지급과 논스 증가는 남는다
    # --- 환불과 배분 (식 (82) 의 환불 계수기 Ar 는 이 장난감에서 0) ---
    gas_used = tx["gas_limit"] - gas
    sender["balance"] += gas * price                     # 안 쓴 가스는 산 값 그대로 돌려준다
    S.setdefault(beneficiary, {"nonce": 0, "balance": 0, "code": None, "storage": {}})
["balance"] += gas_used * priority
    burned = gas_used * base_fee
    return S, f"{status} | 쓴 가스 {gas_used} | 보낸 사람 부담 {gas_used * price} = 소각 {burned} +
검증자 {gas_used * priority}"

state = {"Alice": {"nonce": 0, "balance": 10_000_000, "code": None, "storage": {}},
         "Store": {"nonce": 1, "balance": 0, "code": set42, "storage": {}}}
BASE = 10                                               # 장난감 기본 수수료(가스당)
tx1 = {"from": "Alice", "to": "Store", "value": 0, "data": b"", "gas_limit": 50_000, "max_fee":
15, "max_priority": 2, "nonce": 0}
state, msg = apply_tx(state, tx1, BASE, "Val")
print("tx1 set(42):", msg)
print("   Store.storage =", state["Store"]["storage"], "| Alice 잔고", state["Alice"]["balance"],
"| Val 잔고", state["Val"]["balance"])

tx2 = {**tx1, "nonce": 1, "gas_limit": 21_500}            # 내재 가스 21000 은 넘지만 SSTORE 까지는
못 간다
state, msg = apply_tx(state, tx2, BASE, "Val")
print("tx2 가스 한도 21,500:", msg)
print("   Store.storage =", state["Store"]["storage"], "(그대로) | Alice 잔고", state["Alice"]
["balance"], "(수수료는 나갔다) | 논스", state["Alice"]["nonce"])

tx3 = {**tx1, "nonce": 2, "max_fee": 9}                   # 기본 수수료 10 보다 낮게 적은 거래
_, msg = apply_tx(state, tx3, BASE, "Val")
```

```
print("tx3 최대 수수료 9:", msg)
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
tx1 set(42): 성공 | 쓴 가스 43106 | 보낸 사람 부담 517272 = 소각 431060 + 검증자 86212
   Store.storage = {0: 42} | Alice 잔고 9482728 | Val 잔고 86212
tx2 가스 한도 21,500: 가스 소진(pc=4: 5000 가스 필요, 남은 가스 494) → 실행 되돌림 | 쓴 가스 21500 | 보낸
사람 부담 258000 = 소각 215000 + 검증자 43000
   Store.storage = {0: 42} (그대로) | Alice 잔고 9224728 (수수료는 나갔다) | 논스 2
tx3 최대 수수료 9: 무효: 최대 수수료 9 < 기본 수수료 10
```

[중급] tx1에서 보낸 사람이 낸 수수료는 쓴 가스 × (기본 10 + 우선 2)이고, 그중 기본 수수료 몫은 아무 계정에도 더해지지 않았습니다(소각). 검증자 계정 Val에는 우선 수수료 몫만 들어왔습니다. tx2는 가스 한도를 21,500으로 낮춰 보낸 거래입니다. 내재 가스 21,000은 넘으므로 유효한 거래이고 실행이 시작되지만, 저장소 쓰기(이미 42가 있으므로 2,100 + 2,900 = 5,000가스)에 닿기 전에 가스가 바닥납니다. 저장소는 그대로, 논스는 올라갔고, 잔고에서는 수수료가 빠졌습니다. 「되돌려지지만 수수료는 남는다」의 모양입니다. tx3은 기본 수수료보다 낮은 최대 수수료를 적어 **아예 블록에 들어갈 수 없는** 거래입니다. 유효성 검사에서 걸리고, 아무것도 바뀌지 않으며, 수수료도 나가지 않습니다. 「실행 중 실패」와 「무효 거래」는 다른 사건입니다.



그림 13-3. 가스 한도를 쓴 가스와 남은 가스(환불)로 가르고, 쓴 가스의 수수료를 기본 수수료 몫(소각)과 우선 수수료 몫(검증자)으로 다시 가른 두 단 막대. 수치는 5절 NB13-2 tx1의 장난감 값이다.

[중급] 마지막으로 정지 문제 그 자체입니다. 끝나지 않는 프로그램을 돌려 봅니다.

```
# NB13-3: 무한 루프 — 멈추지 않는 코드를 가스가 멈춘다 (공부용)
# JUMPDEST(0) ; PUSH1 0 ; JUMP  → 0번지로 영원히 되돌아간다. 한 바퀴에 1 + 3 + 8 = 12 가스.
forever = bytes([0x5b, 0x60, 0x00, 0x56])
for budget in [100, 1_000, 100_000]:
    try:
        run(forever, {}, gas=budget)
        print("끝났다?!")                                  # 이 줄은 실행되지 않아야 한다
    except OutOfGas as e:
        print(f"가스 {budget:>7}: 명령 {e.steps:>6}개 실행(약 {e.steps // 3:>5}바퀴) 뒤 멈춤 — {e}")

state2 = {"Alice": {"nonce": 0, "balance": 10_000_000, "code": None, "storage": {}},
          "Loop": {"nonce": 1, "balance": 0, "code": forever, "storage": {}}}
tx = {"from": "Alice", "to": "Loop", "value": 0, "data": b"", "gas_limit": 100_000, "max_fee":
15, "max_priority": 2, "nonce": 0}
state2, msg = apply_tx(state2, tx, BASE, "Val")
print("무한 루프 계약에 거래:", msg)
print("   Alice 잔고", state2["Alice"]["balance"], "— 가스 한도 전부를 산 값(100,000 × 12 =
1,200,000)이 그대로 나갔다. 노드는 100,000 가스어치만 일했다")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
가스     100: 명령     27개 실행(약     9바퀴) 뒤 멈춤 — pc=3: 8 가스 필요, 남은 가스 0
가스    1000: 명령    252개 실행(약    84바퀴) 뒤 멈춤 — pc=3: 8 가스 필요, 남은 가스 0
가스  100000: 명령  25002개 실행(약  8334바퀴) 뒤 멈춤 — pc=3: 8 가스 필요, 남은 가스 0
무한 루프 계약에 거래: 가스 소진(pc=3: 8 가스 필요, 남은 가스 0) → 실행 되돌림 | 쓴 가스 100000 | 보낸 사람 부담
1200000 = 소각 1000000 + 검증자 200000
   Alice 잔고 8800000 — 가스 한도 전부를 산 값(100,000 × 12 = 1,200,000)이 그대로 나갔다. 노드는 100,000
가스어치만 일했다
```

프로그램은 끝나지 않지만 기계는 멈춥니다. 가스 예산이 100이면 아홉 바퀴째에, 100,000이면 8,334바퀴째에 섭니다(한 바퀴 12가스). 공격자가 노드를 괴롭힐 수 있는 양은 자기가 미리 산 가스에 비례하고, 그 값은 공격자가 냅니다. 백서가 든 둘째 예 그대로입니다. 「공격자는 STARTGAS 값을 내야 하므로 채굴자는 계산이 지나치게 많은 단계를 밟을 것임을 미리 안다.」[28] 가스는 계산을 재는 단위이면서 동시에 **계산 요청의 상한을 거래 안에 적게 하는 장치입니다**.

6. 정리 — 「계약」이라는 이름의 프로그램

이더리움은 원장에 프로그램과 그 저장소를 올려 두고, 거래가 오면 모든 노드가 같은 프로그램을 돌려 같은 상태에 이르게 하는 기계입니다. 어떤 프로그램이든 돌릴 수 있으므로 멈추지 않는 프로그램도 돌릴 수 있고, 그래서 명령마다 가스 값을 매겨 미리 사게 하고, 바닥나면 수수료만 남기고 전부 되돌립니다. 수수료는 기본 수수료(소각)와 우선 수수료(검증자)로 갈립니다. 이 장에서 읽은 열 줄짜리 「계약」에는 당사자도 합의도 없었습니다. 그 위에 「잔고 장부」를 얹어 사람들이 토큰이라 부르는 것을 만드는 일이 다음 14장이고, 그 토큰들을 사람 없이 맞바꾸는 기계가 15장입니다. 부록 B 오해 ②(「스마트계약은 계약이다」)의 기술적 절반은 이 장이 주었고, 법적 절반은 28장이 받습니다.

이 장의 출처

- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014(초고 2013). https://ethereum.org/whitepaper/ (공식 PDF, 2026-10-25 접근) — 비트코인 스크립트의 네 한계(「Scripting」, PDF 11–12쪽), 튜링 완전 언어 내장과 「암호학적 상자」(13쪽), 메시지와 거래·1급 시민·가스 소진과 환불(14쪽), 상태 전이 함수 여섯 단계(15쪽), EVM 코드 실행 모형(17쪽), 정지 문제와 가스 해법·공격 예(28–29쪽).
- Gavin Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, Shanghai version efc5f9a, 2025-02-04. https://ethereum.github.io/yellowpaper/paper.pdf — 동기(§1.1), 거래 필드와 각주 3(§4.2, 4–5쪽), 가스와 지불·기본 수수료 소각·우선 수수료(§5, 8–9쪽), 유효성 여덟 조건·내재 가스·유효 가격·선지급·환불(§6, 9–10쪽), 실행 모형·기계 상태·예외 정지(§9, 13–16쪽), 요금표(부록 G, 28쪽). 이 판은 상하이 포크 기준이며 그 뒤 포크의 변경은 반영되어 있지 않을 수 있다.
- The Solidity Authors, *Solidity Documentation*, v0.8.38 (컴파일러 v0.8.38 릴리스 2026-10-22), 「Introduction to Smart Contracts」. https://docs.soliditylang.org/en/v0.8.38/introduction-to-smart-contracts.html (2026-10-25 접근; v0.8.37 판과 예제·설명 대조) — SimpleStorage 예제의 구조와 그 설명 문장. 4절의 Solidity 코드는 같은 구조로 이 책이 새로 쓴 것이고 문서의 예제 코드는 옮기지 않았다. 저장소 https://github.com/argotorg/solidity (GPL-3.0; 문서 페이지가 링크하는 옛 주소 github.com/ethereum/solidity 는 이 주소로 넘어온다).

- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리만 사용(외부 패키지 없음). 노트북 NB13. Solidity 코드는 읽기용이며 컴파일·배포하지 않았다.

각주

1. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," ethereum.org 공식 PDF(2014; 초고 2013, 받은 판은 2014-04 이후 개정본), 「Scripting」 절(PDF 11–12쪽). 네 한계의 원문 표제: "Lack of Turing-completeness", "Value-blindness", "Lack of state", "Blockchain-blindness". 첫째의 원문: "while there is a large subset of computation that the Bitcoin scripting language supports, it does not nearly support everything. The main category that is missing is loops." https://ethereum.org/whitepaper/ (2026-10-25 접근).

2. Buterin (2014), PDF 13쪽(「Ethereum」 절). 원문: "Ethereum does this by building what is essentially the ultimate abstract foundational layer: a blockchain with a built-in Turing-complete programming language, allowing anyone to write smart contracts and decentralized applications where they can create their own arbitrary rules for ownership, transaction formats and state transition functions. ... Smart contracts, cryptographic 'boxes' that contain value and only unlock it if certain conditions are met, can also be built on top of our platform, with vastly more power than that offered by Bitcoin scripting because of the added powers of Turing-completeness, value-awareness, blockchain-awareness and state." 1쪽 서론도 같은 취지를 적습니다: "What Ethereum intends to provide is a blockchain with a built-in fully fledged Turing-complete programming language that can be used to create 'contracts' that can be used to encode arbitrary state transition functions ...".

3. Gavin Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Shanghai version efc5f9a (2025-02-04), §1.1 Driving Factors(1쪽). 원문: "one key goal is to facilitate transactions between consenting individuals who would otherwise have no means to trust one another. This may be due to geographical separation, interfacing difficulty, or perhaps the incompatibility, incompetence, unwillingness, expense, uncertainty, inconvenience, or corruption of existing legal systems. By specifying a state-change system through a rich and unambiguous language, and furthermore architecting a system such that we can reasonably expect that an agreement will be thus enforced autonomously, we can provide a means to this end." https://ethereum.github.io/yellowpaper/paper.pdf (2026-10-25 접근).

4. Buterin (2014), 「Messages and Transactions」 절(PDF 14쪽). 원문: "'Messages' in Ethereum are somewhat similar to 'transactions' in Bitcoin, but with three important differences. First, an Ethereum message can be created either by an external entity or a contract, whereas a Bitcoin transaction can only be created externally. Second, there is an explicit option for Ethereum messages to contain data. Finally, the recipient of an Ethereum message, if it is a contract account, has the option to return a response; this means that Ethereum messages also encompass the concept of functions. The term 'transaction' is used in Ethereum to refer to the signed data package that stores a message to be sent from an externally owned account."

5. 같은 절, 같은 쪽. 원문: "An important consequence of the message mechanism is the 'first class citizen' property of Ethereum - the idea that contracts have equivalent powers to external accounts, including the ability to send message and create other contracts."

6. Buterin (2014), 같은 절. 원문: "Transactions contain the recipient of the message, a signature identifying the sender, the amount of ether and the data to send, as well as two values called STARTGAS and GASPRICE."

7. Wood (2025), §4.2 The Transaction(4–5쪽). 공통 필드 type·nonce·gasLimit·to·value·r·s; 1형·2형에 accessList·chainId·yParity; 2형에 maxFeePerGas("the maximum number of Wei to be paid per unit of gas for all computation costs incurred as a result of the execution of this transaction")·maxPriorityFeePerGas("the maximum number of Wei to be paid to the block's fee recipient as an incentive to include the transaction"); 0형·1형에 gasPrice. gasLimit의 원문: "A scalar value equal to the maximum amount of gas that should be used in executing this transaction. This is paid up-front, before any computation is done and may not be increased later". 거래 형식(0·1·2)은 EIP-2718이 도입했고 런던 판 기준 셋이라고 적습니다. 이 황서 판은 상하이 포크 기준이며 그 뒤 포크(블롭을 싣는 3형 거래 등)는 반영되어 있지 않을 수 있습니다.

8. Buterin (2014), 「Ethereum State Transition Function」 절(PDF 15쪽). 원문 여섯 단계: "1. Check if the transaction is well-formed (ie. has the right number of values), the signature is valid, and the nonce matches

the nonce in the sender's account. If not, return an error. 2. Calculate the transaction fee as STARTGAS * GASPRICE, and determine the sending address from the signature. Subtract the fee from the sender's account balance and increment the sender's nonce. If there is not enough balance to spend, return an error. 3. Initialize GAS = STARTGAS, and take off a certain quantity of gas per byte to pay for the bytes in the transaction. 4. Transfer the transaction value from the sender's account to the receiving account. If the receiving account does not yet exist, create it. If the receiving account is a contract, run the contract's code either to completion or until the execution runs out of gas. 5. If the value transfer failed because the sender did not have enough money, or the code execution ran out of gas, revert all state changes except the payment of the fees, and add the fees to the miner's account. 6. Otherwise, refund the fees for all remaining gas to the sender, and send the fees paid for gas consumed to the miner." 「채굴자」는 2014년 백서의 용어이고, 지분증명 전환 뒤에는 블록을 제안한 검증자의 수수료 수취 주소가 그 자리에 옵니다(10장).

9. Buterin (2014), 「Messages and Transactions」 절(PDF 14쪽). 원문: "If transaction execution 'runs out of gas', all state changes revert - except for the payment of the fees, and if transaction execution halts with some gas remaining then the remaining portion of the fees is refunded to the sender."

10. Buterin (2014), 「Computation And Turing-Completeness」 절(PDF 28쪽). 원문: "An important note is that the Ethereum virtual machine is Turing-complete; this means that EVM code can encode any computation that can be conceivably carried out, including infinite loops. ... This naturally leads to a problem: can malicious users essentially shut miners and full nodes down by forcing them to enter into an infinite loop? The issue arises because of a problem in computer science known as the halting problem: there is no way to tell, in the general case, whether or not a given program will ever halt."

11. 같은 절, 같은 쪽. 원문: "our solution works by requiring a transaction to set a maximum number of computational steps that it is allowed to take, and if execution takes longer computation is reverted but fees are still paid. Messages work in the same way."

12. Wood (2025), §9 Execution Model(13–14쪽). 원문: "It is a quasi-Turing-complete machine; the quasi qualification comes from the fact that the computation is intrinsically bounded through a parameter, gas, which limits the total amount of computation done."

13. Buterin (2014), PDF 29쪽. 원문: "An attacker sees a contract with code of some form like send(A,contract.storage[A]); contract.storage[A] = 0, and sends a transaction with just enough gas to run the first step but not the second (ie. making a withdrawal but not letting the balance go down). The contract author does not need to worry about protecting against such attacks, because if execution stops halfway through the changes get reverted."

14. Wood (2025), §5 Gas and Payment(8쪽). 원문: "In order to avoid issues of network abuse and to sidestep the inevitable questions stemming from Turing completeness, all programmable computation in Ethereum is subject to fees. The fee schedule is specified in units of gas (see Appendix G for the fees associated with various computation). Thus any given fragment of programmable computation (this includes creating contracts, making message calls, utilising and accessing account storage and executing operations on the virtual machine) has a universally agreed cost in terms of gas."

15. Wood (2025), §9.2 Fees Overview(14쪽). 원문: "Storage fees have a slightly nuanced behaviour—to incentivise minimisation of the use of storage (which corresponds directly to a larger state database on all nodes), the execution fee for an operation that clears an entry in the storage is not only waived, a qualified refund is given; in fact, this refund is effectively paid up-front since the initial usage of a storage location costs substantially more than normal usage."

16. Wood (2025), Appendix G. Fee Schedule(28쪽). 표의 원문: "Gverylow 3 Amount of gas to pay for operations of the set Wverylow. ... Gmid 8 ... Ghigh 10 ... Gwarmaccess 100 Cost of a warm account or storage access. ... Gcoldsload 2100 Cost of a cold storage access. Gsset 20000 Paid for an SSTORE operation when the storage value is set to non-zero from zero. Gsreset 2900 Paid for an SSTORE operation when the storage value's zeroness remains unchanged or is set to zero. Rsclear 4800 Refund given (added into refund counter) when the storage value is set to zero from non-zero. ... Gcreate 32000 Paid for a CREATE operation. ... Gtxdatazero 4 Paid for every zero byte of data or code for a transaction. Gtxdatanonzero 16 Paid for every non-zero byte of data or code for a transaction. Gtransaction 21000 Paid for every transaction." 저장소 쓰기의 실제 비용 규칙(부록 H의 SSTORE 항)은 「처음 건드리는 칸인가」와 「원래 값·현재 값·새 값」의 조합으로 더 세분되어 있고, 이 장의 코드는 그중 두 경우만 흉내 냅니다.

17. Wood (2025), §6.2(10쪽) 식 (82)와 각주 9. 원문: "The total refundable amount is the legitimately remaining gas g′, added to Ar, with the latter component being capped up to a maximum of one fifth (rounded down) of the total amount used Tg − g′." 각주 9: "The max refundable proportion of gas was reduced from one half to one fifth by EIP-3529 by Buterin and Swende [2021] in the London release".

18. Wood (2025), §5(8–9쪽). 원문: "Since the introduction of EIP-1559 by Buterin et al. [2019] in the London hard fork, every transaction included in a block must pay a base fee, which is specified as wei per unit of gas consumed and is constant for each transaction within a block. The ether that is paid to meet the base fee is burned (taken out of circulation). ... To incentivize validators to include transactions, there is an additional fee known as a priority fee, which is also specified as wei per unit of gas consumed. The total fee paid by the transactor therefore is the sum of the base fee per gas and the priority fee per gas multiplied by the total gas consumed. Ether used to satisfy the priority fee is delivered to the beneficiary address, the address of an account typically under the control of the validator."

19. Wood (2025), §6 Transaction Execution(9쪽). 원문: "(1) The transaction is well-formed RLP, with no additional trailing bytes; (2) the transaction signature is valid; (3) the transaction nonce is valid (equivalent to the sender account's current nonce); (4) the sender account has no contract code deployed (see EIP-3607 by Feist et al. [2021]); (5) the gas limit is no smaller than the intrinsic gas, g0, used by the transaction; (6) the sender account balance contains at least the cost, v0, required in up-front payment; (7) the maxFeePerGas, Tm, in the case of type 2 transactions, or gasPrice, Tp, in the case of type 0 and type 1 transactions, is greater than or equal to the block's base fee, Hf; and (8) for type 2 transactions, maxPriorityFeePerGas, Tf, must be no larger than maxFeePerGas, Tm."

20. Wood (2025), §6.2 Execution(9–10쪽) 식 (64). 원문: "We define intrinsic gas g0, the amount of gas this transaction requires to be paid prior to execution, as follows: g0 ≡ Σ (Gtxdatazero if i = 0, Gtxdatanonzero otherwise) + (Gtxcreate + R(‖ Ti ‖) if Tt = ∅, 0 otherwise) + Gtransaction + Σ (Gaccesslistaddress + ‖ TA[j]s ‖ Gaccessliststorage) ...".

21. Wood (2025), §6.2(10쪽) 식 (66)~(68): "p ≡ Tp if Tx = 0 ∨ Tx = 1; f + Hf if Tx = 2", "f ≡ Tp − Hf if Tx = 0 ∨ Tx = 1; min(Tf, Tm − Hf) if Tx = 2", "v0 ≡ TgTp + Tv if Tx = 0 ∨ Tx = 1; TgTm + Tv if Tx = 2". 실행 시작의 비가역 변경(식 (73)~(75)): "the nonce of the account of the sender, S(T), is incremented by one and the balance is reduced by part of the up-front cost, Tgp."

22. Wood (2025), §9.1 Basics(14쪽). 원문: "The EVM is a simple stack-based architecture. The word size of the machine (and thus size of stack items) is 256-bit. This was chosen to facilitate the Keccak-256 hash scheme and elliptic-curve computations. The memory model is a simple word-addressed byte array. The stack has a maximum size of 1024. The machine also has an independent storage model; this is similar in concept to the memory but rather than a byte array, it is a word-addressable word array. Unlike memory, which is volatile, storage is non volatile and is maintained as part of the system state." 예외 실행: "The machine can have exceptional execution for several reasons, including stack underflows and invalid instructions. Like the out-of-gas exception, they do not leave state changes intact. Rather, the machine halts immediately and reports the issue to the execution agent ...".

23. Wood (2025), §9.4.1 Machine State(15쪽). 원문: "The machine state μ is defined as the tuple (g, pc, m, i, s, o) which are the gas available, the program counter pc ∈ N256, the memory contents, the active number of words in memory (counting continuously from position 0), the stack contents, and the returndata buffer."

24. Wood (2025), §9.4.2 Exceptional Halting(15쪽) 식 (158) 아래 풀이. 원문: "This states that the execution is in an exceptional halting state if there is insufficient gas, if the instruction is invalid (and therefore its δ subscript is undefined), if there are insufficient stack items, if a JUMP/JUMPI destination is invalid, the new stack size would be larger than 1024 or state modification is attempted during a static call."

25. The Solidity Authors, *Solidity Documentation* v0.8.38, 「Introduction to Smart Contracts — A Simple Smart Contract — Storage Example」. https://docs.soliditylang.org/en/v0.8.38/introduction-to-smart-contracts.html (2026-10-25 접근; 문서 머리 표기는 「0.8.38-develop」). 같은 예제가 직전 고정판 v0.8.37 문서 (https://docs.soliditylang.org/en/v0.8.37/introduction-to-smart-contracts.html)에도 글자 단위로 같게 실려 있고, v0.8.37 문서의 머리 표기도 「0.8.37-develop」이어서 「-develop」은 이 문서 사이트의 제목 생성 방식이지 미완성판의 표시가 아닌 것으로 보입니다(2026-10-25 두 판 대조). 문서의 예제 코드는 GPL-3.0으로 배포되므로 이 책에 옮겨 싣지 않았습니다. 원래 코드는 위 주소에서 볼 수 있습니다. 문서 예제에서는 상태 변수 이름이 `storedData`, 함수 인자 이름이 `x` 입니다.

26. 같은 문서, 같은 절. 원문: "A contract in the sense of Solidity is a collection of code (its functions) and data (its state) that resides at a specific address on the Ethereum blockchain. The line uint storedData; declares a state variable called storedData of type uint (unsigned integer of 256 bits). You can think of it as a single slot in a database that you can query and alter by calling functions of the code that manages the database." "This contract does not do much yet apart from (due to the infrastructure built by Ethereum) allowing anyone to store a single number that is accessible by anyone in the world without a (feasible) way to prevent you from publishing this number. Anyone could call set again with a different value and overwrite your number, but the number is still stored in the history of the blockchain. Later, you will see how you can impose access restrictions so that only you can alter the number."

27. Wood (2025), §4.2 각주 3(4쪽). 원문: "Notably, such 'tools' could ultimately become so causally removed from their human-based initiation—or humans may become so causally-neutral—that there could be a point at which they rightly be considered autonomous agents. e.g. contracts may offer bounties to humans for being sent transactions to initiate their execution."

28. Buterin (2014), PDF 28–29쪽. 원문: "An attacker creates a very long infinite loop with the intent of forcing the miner to keep computing for such a long time that by the time computation finishes a few more blocks will have come out and it will not be possible for the miner to include the transaction to claim the fee. However, the attacker will be required to submit a value for STARTGAS limiting the number of computational steps that execution can take, so the miner will know ahead of time that the computation will take an excessively large number of steps."

2부 스마트계약과 응용

14장 토큰 표준 — ERC-20·721이 정하는 것과 정하지 않는 것

핵심 TAKEAWAY

토큰은 13장의 계약 계정 안에 든 **잔액 장부 하나입니다.** 「A에게서 X를 빼고 B에게 X를 더한다. 단 A에게 X가 있어야 하고 A가 승인해야 한다.」 이것이 전부이고, 부테린은 이것을 「연산 하나짜리 데이터베이스」라고 불렀습니다. 토큰 **표준**(ERC-20, ERC-721)은 그 장부를 다루는 **함수의 이름과 입출력을** 통일한 약속입니다. 지갑과 거래소가 처음 보는 토큰도 다룰 수 있게 하기 위해서입니다. 표준은 함수가 무엇을 하는지는 정하지만, 그 토큰이 세상에서 **무엇을 뜻하는지**(의결권인지, 수익 분배 청구권인지, 그림 파일 링크인지)는 정하지 않습니다. 그것은 표준 바깥에서 발행자가 한 약속이고, 그 약속의 법적 성질은 코드가 아니라 법이 정합니다. 이 장을 읽으면 「NFT를 샀다」는 문장이 기술적으로 무엇을 뜻하는지, 그리고 그 문장이 **말하지 않는 것이** 무엇인지 가를 수 있습니다.

학습목표

1. 코인(프로토콜 내장 자산)과 토큰(계약 안의 장부)을 구별하고, 토큰이 「계약 저장소의 사상(mapping) 하나」임을 설명한다.
2. [중급] ERC-20의 여섯 함수·두 이벤트와 ERC-721의 소유·이전·승인 함수를 읽고, 표준이 「MUST」로 정한 것과 「OPTIONAL」로 둔 것, 아예 침묵하는 것을 가른다.
3. 대체 가능(fungible) 토큰과 대체 불가(non-fungible) 토큰의 차이가 데이터 구조에서 어떻게 나타나는지 안다.
4. [중급] 「토큰이 가리키는 것」을 온체인 권리(코드가 집행하는 것)와 오프체인 약속(발행자가 말한 것)으로 나누어 읽는다.

[입문]**만 읽은 독자가 가져가는 것.** 토큰 표준은 함수 이름의 약속일 뿐이며, 그 토큰이 어떤 권리를 담는지는 표준이 아니라 **발행자의 바깥 약속이** 정합니다.

1. 이 기술은 무엇의 실패에서 나왔나

13장의 「단순 저장소」 계약은 숫자 하나를 저장했습니다. 숫자를 **주소마다** 하나씩 저장하면 어떻게 될까요. 부테린의 2014년 백서는 그것이 곧 화폐라고 말합니다. 「이해해야 할 핵심은, 통화 또는 토큰 체계란 근본적으로 **연산 하나를 가진 데이터베이스라는** 것이다. A에게서 X 단위를 빼고 B에게 X 단위를 준다. 단 (i) A가 거래 전에 X 단위 이상을 가졌어야 하고 (2) 거래가 A에 의해 승인되어야 한다. 토큰 체계를 구현하는 데 필요한 것은 이 논리를 계약에 넣는 것이 전부다.」[1] 백서는 그 논리를 여섯 줄로 적어 보이고, 용도로 「미국 달러나 금 같은 자산을 대리하는 하위 통화에서 회사 주식, 스마트 재산을 대리하는 개별 토큰, 위조 불가능한 쿠폰, 그리고 관습적 가치와 아무 연결이 없이 유인용 점수 체계로 쓰이는 토큰 체계까지」를 듭니다.[2] 이 목록을 기억해 두십시오. 같은 여섯 줄이 주식도 되고 쿠폰도 되고 게임 점수도 됩니다. 코드는 그 차이를 모릅니다.

문제는 그 여섯 줄을 **저마다 다르게** 쓴다는 데서 생겼습니다. 어떤 계약은 보내는 함수를 `send` 라 부르고 어떤 계약은 `transfer` 라 부르면, 지갑 프로그램은 토큰마다 따로 코드를 짜야 합니다. ERC-20의 동기 절은 한 문장입니다. 「표준 인터페이스는 이더리움 위의 어떤 토큰이든 다른 응용 — 지갑에서 탈중앙 거래소까지 — 이 다시 쓸 수 있게 한다.」[3] 2년 뒤 ERC-721은 같은 틀을 **서로 구별되는** 자산에 맞게 고쳤습니다. 「EIP-20은 NFT를 추적하는 데 불충분하다. 각 자산이 구별되는(대체 불가능한) 반면, 한 수량의 토큰은 각각이 동일(대체 가능)하기 때문이다.」[4] 둘 다 **합의된 함수 이름이** 없어서 생기는 비효율에서 나왔고, 둘 다 함수 이름을 통일하는 것으로 답했습니다. 그 이상은 하지 않았습니다.

2. 직관 — 코인과 토큰, 그리고 장부 하나

먼저 낱말을 가릅니다. **코인은** 프로토콜 자체가 아는 자산입니다. 비트코인의 BTC, 이더리움의 ETH(이더)가 그렇습니다. 이더는 계정의 `balance` 필드에 적히고(11장), 가스값을 내는 데 쓰입니다. 백서는 이더의 역할을 「다양한 디지털 자산 사이의 효율적 교환을 위한 1차 유동성 층을 제공하는 것과, 더 중요하게는 거래 수수료를 지불하는 수단을 제공하는 것」 둘로 적습니다.[5] **토큰은** 프로토콜이 모르는 자산입니다. 어떤 계약 계정의 저장소 안에 「주소 → 수량」 사상이 하나 있고, 그 계약의 함수가 그 사상을 고칩니다. 프로토콜이 보기에 토큰 이체는 그냥 「어떤 계약의 저장소 칸 두 개가 바뀐 것」입니다. 13장에서 본 저장소 쓰기 비용이 토큰 이체의 가스에서 큰 몫을 차지하는 이유입니다 — 그 몫은 받는 쪽 칸이 비어 있었는지(0→비0이면 20,000), 이미 값이 있었는지(2,900), 그 거래에서 처음 건드리는 칸인지(+2,100)에 따라 달라지고, 두 칸에 모두 값이 있던 보통의 이체에서는 기본 거래 비용 21,000보다 작습니다.



그림 14-1. 코인과 토큰. 이더는 계정의 balance 필드에 적히고, 토큰은 계약 계정 저장소 안의 「주소 → 수량」 사상 하나다. 사상에는 명단이 없고 모든 주소가 처음부터 0으로 읽힌다(잔액은 6절 NB14-1 출력).

[중급] 13장에서 인용한 Solidity 문서의 둘째 예제(「Subcurrency Example」)가 바로 이 장부입니다. 문서는 「다음 계약은 암호화폐의 가장 단순한 형태를 구현한다. 이 계약은 만든 사람만 새 코인을 만들 수 있게 한다(다른 발행 방식도 가능하다). 누구나 사용자 이름과 비밀번호로 등록할 필요 없이 서로 코인을 보낼 수 있다. 필요한 것은 이더리움 열쇠 쌍뿐이다」라고 소개합니다.[6] 그 코드의 핵심은 둘입니다. 주소마다 수량을 대응시키는 사상(mapping) 하나가 장부이고, 보내기 함수 안에서 보낸 사람의 잔액을 빼고 받는 사람의 잔액을 더하는 두 줄이 백서의 「연산 하나」입니다(같은 일을 하는 코드는 아래 6절에 이 책이 Python으로 새로 썼습니다). 문서가 사상(mapping)에 관해 덧붙인 말이 토큰의 성질을 잘 보여 줍니다. 「사상은 가능한 모든 열쇠가 처음부터 존재하고 바이트 표현이 전부 0인 값에 대응되도록 사실상 초기화된 해시 테이블로 볼 수 있다. 다만 사상의 모든 열쇠의 목록도, 모든 값의 목록도 얻을 수 없다.」[7] 그러니까 토큰 계약은 「누가 이 토큰을 가졌는가」의 **명단을 갖고 있지 않습니다.** 주소를 하나 대면 그 주소의 수량을 알려 줄 뿐입니다. 모든 주소의 잔액은 처음부터 0으로 「존재」합니다. 「보유자 명부」를 떠올리는 법률가에게 이 점은 작지 않습니다.

3. ERC-20 — 대체 가능 토큰이 약속하는 여섯 함수

[중급] ERC-20의 명세는 짧습니다. 함수 아홉 개와 이벤트 두 개가 전부이고, 그중 셋은 선택입니다. 표는 원문의 설명을 이 책이 요약한 것이며, 원문 문구는 각주에 있습니다.[8]

함수	뜻	표준의 요구 수준
`totalSupply()`	토큰 총량	필수
`balanceOf(owner)`	`owner` 의 잔액	필수
`transfer(to, value)`	호출자의 잔액에서 `to` 로 `value` 를 옮기고 `Transfer` 이벤트를 **반드시** 낸다. 잔액이 모자라면 실패해야 한다(SHOULD)	필수
`transferFrom(from, to, value)`	`from` 의 잔액에서 `to` 로 옮긴다. 「인출 흐름」용 — **계약이 당신 대신** 토큰을 옮길 수 있게 한다. `from` 이 호출자에게 미리 권한을 주지 않았으면 실패해야 한다(SHOULD)	필수
`approve(spender, value)`	`spender` 가 호출자의 계정에서 **여러 번에 걸쳐 합계** `value` **까지** 꺼내 가도록 허용한다. 다시 부르면 덮어쓴다	필수
`allowance(owner, spender)`	`spender` 가 `owner` 에게서 아직 꺼내 갈 수 있는 양	필수
`name()` · `symbol()` · `decimals()`	이름·기호·소수 자릿수	**선택** — 「인터페이스와 다른 계약은 이 값들이 있다고 기대해서는 안 된다(MUST NOT)」
`Transfer(from, to, value)` 이벤트	토큰이 옮겨질 때, **0 수량 이체를 포함해** 반드시 발생	필수
`Approval(owner, spender, value)` 이벤트	`approve` 가 성공할 때마다 발생	필수

[중급] 세 가지를 짚습니다. 첫째, `approve` / `transferFrom` 짝이 있는 이유입니다. 13장에서 계약 계정은 개인키가 없다고 했습니다. 그래서 계약은 당신의 토큰을 **당신 서명으로** 옮길 수 없습니다. 대신 당신이 먼저 `approve` 로 「이 계약이 내 토큰을 최대 얼마까지 가져가도 된다」고 적어 두면, 계약이 `transferFrom` 으로 가져갑니다. 15장의 거래소 계약이 토큰을 받는 방식이 이것입니다. 둘째, `approve` 에는 표준 자체가 경고를 붙여 두었습니다. 「공격 벡터를 막기 위해 클라이언트는 같은 수취인에 대한 허용량을 다른 값으로 바꾸기 전에 먼저 0으로 두도록 사용자 인터페이스를 만들어야 한다(SHOULD). 다만 계약 자체가 그것을 강제해서는 안 된다 — 이미 배포된 계약과의 호환성을 위해.」[9] 어떤 공격인지는 아래 코드에서 재연합니다. 거래 **순서가** 결과를 바꾸는 첫 사례이고, 17장의 예고편입니다. 셋째, 표준은 발행(minting)과 소각(burning)을 **정하지 않습니다.** 새 토큰이 생길 때 `_from` 을 0 주소로 한 `Transfer` 이벤트를 내라는 권고(SHOULD)가 있을 뿐, 누가 얼마를 어떤 조건으로 만들 수 있는지는 전부 발행자 몫입니다.

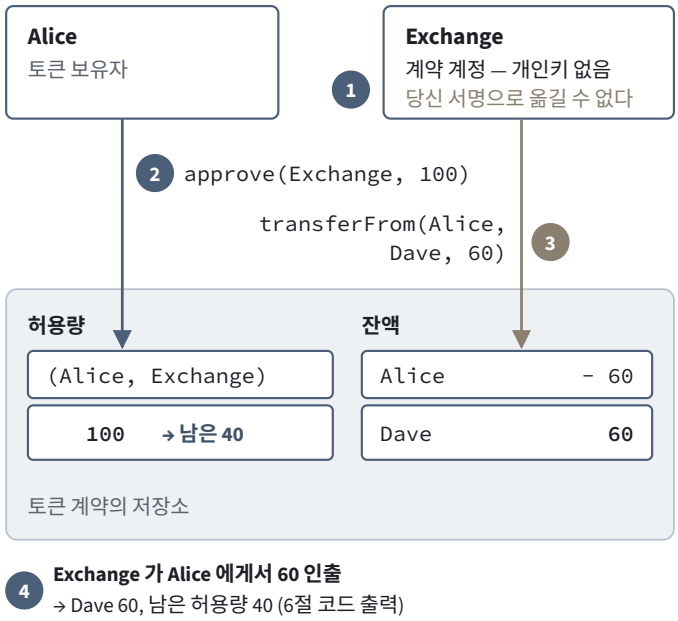


그림 14-2. approve 와 transferFrom 의 인출 흐름. 개인키가 없는 계약은 소유자가 미리 적어 둔 허용량 안에서만 토큰을 옮긴다. 수치는 6절 NB14-1 출력이다.

[중급] 표준이 「SHOULD」로 적은 자리는 구현마다 갈립니다. 유니스왑 v2 백서는 「ERC-20 표준은 `transfer()` 와 `transferFrom()` 이 성공·실패를 나타내는 불리언을 반환할 것을 요구한다. 그런데 일부 토큰(널리 쓰이는 것들을 포함해)의 구현은 반환값이 없다」고 적고, 그런 토큰을 다루기 위해 자기 쪽 코드를 고쳤다고 밝힙니다.[10] 「표준을 따른다」는 말은 「함수 이름이 같다」에 가깝고, 세부 동작은 계약마다 읽어야 합니다. 토큰이 같은 이름을 쓴다는 것과 같은 것이라는 것은 다릅니다.

4. ERC-721 — 하나씩 다른 것의 장부

[중급] 대체 불가능 토큰(non-fungible token, NFT)의 장부는 「주소 → 수량」이 아니라 「**토큰 번호 → 소유자 주소**」 입니다. ERC-721은 「모든 NFT는 ERC-721 스마트계약 안에서 고유한 `uint256` ID로 식별된다. 이 식별 번호는 계약의 생애 동안 바뀌어서는 안 된다(SHALL NOT). 그러면 (계약 주소, uint256 tokenId) 쌍이 이더리움 체인 위의 특정 자산에 대한 **전 세계적으로 고유하고 완전한 식별자가** 된다」고 적습니다.[11] 핵심 함수는 `ownerOf(tokenId)` (이 번호의 소유자는 누구인가), `balanceOf(owner)` (이 주소가 몇 개를 갖고 있는가), 그리고 이전·승인입니다. 이전에는 두 함수가 있습니다. `transferFrom` 은 「호출자가 수취인이 NFT를 받을 수 있는지 확인할 책임이 있으며, 그렇지 않으면 **영구히 잃을 수 있다**」고 대문자로 경고하고, `safeTransferFrom` 은 수취인이 계약이면 그 계약의 `onERC721Received` 를 불러 정해진 마법값이 돌아오지 않으면 실패합니다.[12] 11장에서 본 「열쇠 없는 주소로 간 코인」 문제를 토큰 수준에서 막으려는 장치입니다. 코드는 받을 준비가 된 계약인지 물어볼 수 있습니다. 사람에게는 물어볼 수 없습니다.

[중급] 승인도 둘입니다. `approve(approved, tokenId)` 는 토큰 하나에 대리인 하나를 두고, `setApprovalForAll(operator, approved)` 는 「제3자(「운영자」)가 호출자의 **모든** 자산을 관리할 수 있게 한다.」 표준은 이 운영자 개념을 「지갑·중개·경매 응용이 많은 수의 NFT를 빨리 다룰 수 있게 하는 강력한 도구」라고 설명합니다.[13] ERC-20의 `allowance` 가 없는 이유는 「모든 NFT가 고유하므로 수량이 없거나 하나」이기 때문입니다.[14] 표준은 또 자기가 **정하지 않는** 것을 명시합니다. 「NFT의 생성

(민
팅)과 소멸(소각)은 이 명세에 포함되지 않는다. 당신의 계약은 다른 수단으로 그것을 구현할 수 있다.」[15] 그리고 이전 함수들의 문서는 「거래가 **반드시** 실패해야 하는 조건만 명시」하며 구현은 「다른 상황에서도 실패할 수 있다(MAY)」고 하면서, 그 예로 「계약이 일시 정지되었을 때 이전 금지」, 「특정 주소의 수취 차단」, 「거래 양 당사자에게 수수료 부과」, 「읽기 전용 NFT 등기부 — 모든 이전·승인 함수에서 항상 실패」를 듭니다.[16] 표준을 따르는 NFT 계약이 **이전을 막을 수 있다는** 것이 표준 자체에 적혀 있습니다.

메타데이터 — 「그림」은 어디에 있는가

사람들이 「NFT」라고 부를 때 떠올리는 그림·음악·문서는 체인 위에 없습니다. ERC-721의 **메타데이터 확장은** 선택(OPTIONAL)이고, 그 내용은 `name()`, `symbol()`, 그리고 `tokenURI(tokenId)` 세 함수입니다. `tokenURI`는 「주어진 자산에 대한 고유한 URI」를 돌려주고, 「그 URI는 「ERC721 Metadata JSON Schema」를 따르는 JSON 파일을 가리킬 수 있다(may — 소문자).」 그 스키마에는 `name`, `description`, 그리고 `image` — 「이 NFT가 대리하는 자산을 나타내는 image/ *형식 자원을 가리키는 URI*」 — *세 항목이 있습니다.*[17] *그러니까 체인에 적힌 것은 주소 문자열** 하나이고, 그 주소가 가리키는 파일에 다시 그림의 주소가 적혀 있습니다. 표준은 이 설계를 택한 이유를 솔직하게 적습니다. 대안으로 「각 자산의 모든 메타데이터를 블록체인에 올리는 것」을 검토했으나 「너무 비싸다」는 이유로 버렸습니다.[18] 13장의 저장소 비용이 여기서 결정을 내렸습니다.

표준은 두 가지를 더 경고합니다. 「URI는 **가변일 수 있다(MAY be mutable)**(곧 때때로 바뀔 수 있다). 우리는 집의 소유권을 대리하는 NFT를 생각했는데, 이 경우 집에 관한 메타데이터(사진, 거주자 등)는 자연히 바뀔 수 있다.」 그리고 「어떤 스마트계약이든 **당신의** 계약과 같은 이름과 기호를 쓸 수 있다는 것을 모두에게 상기시킨다. 클라이언트가 어느 ERC-721 계약이 잘 알려진(정본) 것인지 판단하는 방법은 이 표준의 범위 밖이다.」[19] 토큰 번호와 소유자는 체인이 보증합니다. 그 토큰이 가리키는 파일의 내용, 그 파일이 계속 그 자리에 있을지, 그리고 그 계약이 「진짜」 그 작가의 계약인지는 보증하지 않습니다. 표준 작성자들이 스스로 그렇게 적었습니다.

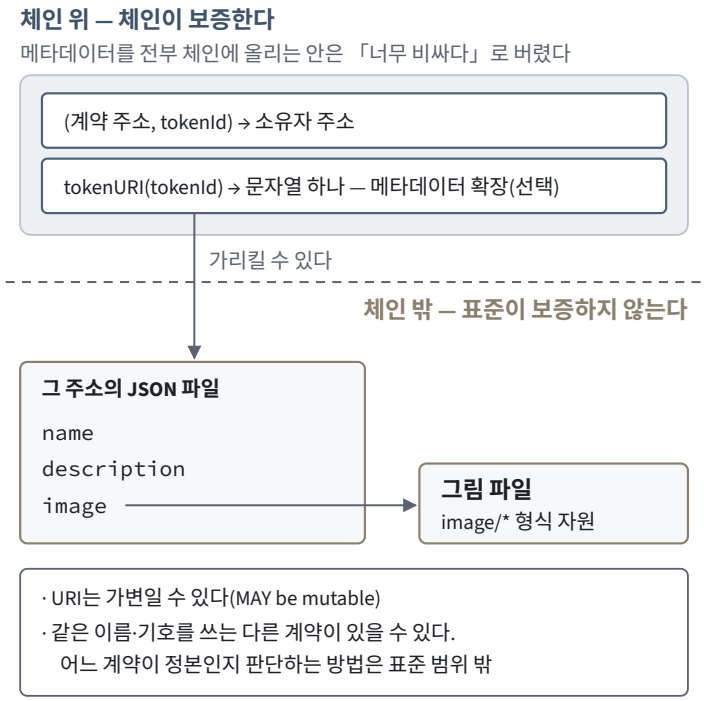


그림 14-3. NFT에서 체인이 보증하는 것과 보증하지 않는 것. 체인에는 (계약 주소, tokenId)의 소유자가 있고, 메타데이터 확장(선택)을 구현한 계약이면 tokenURI 문자열이 있다. 그 문자열이 가리킬 수 있는 JSON과 그림 파일, 어느 계약이 정본인지는 표준 밖에 있다.

> **법률가의 질문**
>
> 「NFT를 샀다」는 사람은 무엇을 산 것입니까. 체인에 적힌 것은 (계약 주소, 번호) 쌍의 소유자 칸에 그 사람의 주소가 들어갔다는 사실, 그리고 `tokenURI` 가 돌려주는 문자열 하나입니다. 그림 파일은 그 문자열이 가리키는 바깥 서버에 있고, 표준은 그 문자열이 바뀔 수 있다고 적습니다. 저작권은 어디에도 적혀 있지 않습니다. 그렇다면 매매의 목적물은 토큰인가요, 토큰이 가리키는 파일인가요, 파일에 관한 어떤 권리인가요, 아니면 「가리킨다」는 사실 그 자체인가요. 발행자가 서버를 닫아 그림이 사라지면 무엇이 멸실된 것인가요. 같은 이름과 기호로 다른 계약이 배포되었다면 어느 쪽이 「진품」인가요. 그리고 ERC-20 토큰에 발행자가 「수익 분배」나 「의결권」을 약속했다면, 그 약속은 코드에 있는 것인가요 바깥에 있는 것인가요. → 28장 · 27장

5. 토큰이 가리키는 것 — 온체인 권리와 오프체인 약속

이 장의 주장을 한 줄로 줄이면 이렇습니다. **코드가 집행하는 것과 발행자가 말한 것은** 다른 층에 있습니다. 코드가 집행하는 것은 「이 주소의 수량을 저 주소로 옮긴다」, 「이 번호의 소유자는 이 주소다」, 「이 조건이면 이전을 막는다」 같은 상태 전이입니다. 발행자가 말한 것은 「이 토큰은 우리 회사 이익의 몫이다」, 「이 토큰으로 투표할 수 있다」, 「이 토큰은 이 그림의 소유권이다」 같은 문장이고, 그 문장은 백서·웹사이트·게시판에 있습니다. 표준은 앞쪽만 다룹니다.

역사가 이 구별을 가장 또렷하게 보여 준 사건이 2016년의 「더 다오(The DAO)」입니다. 자세한 분석은 26장의 몫이고 여기서는 토큰과 약속의 관계만 봅니다. 미국 증권거래위원회(SEC)의 2017년 조사

보고서가 사실을 이렇게 적습니다. 「이더와 교환하여 The DAO는 (지불한 이더에 비례하는) DAO 토큰을 만들어 이더를 보낸 사람 또는 단체의 이더리움 블록체인 주소에 배정했다. DAO 토큰은 보유자에게 **일정한 의결권과 소유권을** 부여했다.」 토큰 보유자는 자금 지원 제안에 「자기가 지배하는 토큰 수로 가중된」 표를 던지고, 「백서가 「DAO가 자금을 댄 프로젝트에서 생겨 DAO가 받은 이더」로 정의한 「보상」을 받을 것으로」 홍보되었습니다.[20] 코드가 집행한 것은 토큰 수에 비례한 투표와 자금 이동이었습니다. 「보상」과 「소유권」은 백서와 웹사이트의 말이었습니다. 그리고 그 말의 법적 성질은 코드가 아니라 SEC가 정했습니다. 보고서의 결론은 「DAO 토큰은 1933년 증권법과 1934년 증권거래법상 증권이다」였고, 보고서는 서두(2쪽)에서 「이 기술, 「스마트계약」 또는 컴퓨터 코드를 통한 특정 기능의 자동화는 행위를 연방 증권법의 적용 범위에서 제외하지 않는다」고 썼습니다.[21] 이 결론이 옳은지, 다른 법역이라면 어떻게 볼지는 이 책의 범위 밖이고 5부도 서베이로만 다룹니다. 이 장이 가져가는 것은 구조입니다. 토큰 계약 안의 여섯 줄은 똑같아도, 그 토큰이 증권인지 쿠폰인지 게임 점수인지는 **코드 밖의 사실이** 정했습니다.

6. 코드 — 장부 둘을 흉내 내고, 표준이 적은 번호를 다시 계산한다

[중급] 두 장부를 Python 사전으로 흉내 냅니다. 서명 검증은 5장에서 했으므로 「누가 불렀는가」를 이름으로 대신하고, 이벤트는 목록에 적어 둡니다. 전부 공부용입니다.

```
# NB14-1: ERC-20 장부 흉내 — 잔액·허용량 사전, 여섯 함수, 두 이벤트, 그리고 approve 경쟁 조건 (공부용)
class ERC20Toy:
    """EIP-20 의 필수 함수만. 실패는 예외로 던진다(표준의 SHOULD throw). 이벤트는 self.events 에 적는다."""
    def __init__(self, name, symbol):
        self.name, self.symbol = name, symbol             # OPTIONAL 항목 — 있어도 되고 없어도 된다
        self.balances = {}                                 # 「모든 주소는 처음부터 0」 — 사전에 없으면
0으로 읽는다
        self.allowances = {}                               # (owner, spender) -> 남은 허용량
        self.events = []
    def totalSupply(self):          return sum(self.balances.values())
    def balanceOf(self, owner):     return self.balances.get(owner, 0)
    def allowance(self, o, s):      return self.allowances.get((o, s), 0)
    def mint(self, to, value):                             # 표준 밖 — 발행 규칙은 발행자 몫. 0x0 에서
온 Transfer 이벤트만 권고(SHOULD)
        self.balances[to] = self.balanceOf(to) + value
        self.events.append(("Transfer", "0x0", to, value))
    def transfer(self, caller, to, value):
        if self.balanceOf(caller) < value:
            raise ValueError(f"잔액 부족: {caller} 는 {self.balanceOf(caller)} 뿐")
        self.balances[caller] -= value
        self.balances[to] = self.balanceOf(to) + value
        self.events.append(("Transfer", caller, to, value))   # 0 수량이라도 이벤트는 낸다(MUST)
        return True
    def approve(self, caller, spender, value):
        self.allowances[(caller, spender)] = value             # 「다시 부르면 덮어쓴다」
        self.events.append(("Approval", caller, spender, value))
        return True
    def transferFrom(self, caller, frm, to, value):
        if self.allowance(frm, caller) < value:
            raise ValueError(f"허용량 부족: {frm} 이 {caller} 에게 허용한 양은 {self.allowance(frm,
caller)}")
        if self.balanceOf(frm) < value:
            raise ValueError("잔액 부족")
        self.allowances[(frm, caller)] -= value
        self.balances[frm] -= value
        self.balances[to] = self.balanceOf(to) + value
        self.events.append(("Transfer", frm, to, value))
        return True

tok = ERC20Toy("Toy Token", "TOY")
tok.mint("Alice", 1000)
tok.transfer("Alice", "Bob", 250)
print("잔액 — Alice:", tok.balanceOf("Alice"), "Bob:", tok.balanceOf("Bob"), "Carol(처음 보는
주소):", tok.balanceOf("Carol"), "| 총량:", tok.totalSupply())
try:
    tok.transfer("Bob", "Carol", 300)
except ValueError as e:
    print("Bob → Carol 300:", e)

# approve / transferFrom — 계약(여기서는 'Exchange')이 Alice 대신 토큰을 가져가는 인출 흐름
tok.approve("Alice", "Exchange", 100)
tok.transferFrom("Exchange", "Alice", "Dave", 60)
print("Exchange 가 Alice 에게서 60 인출 → Dave:", tok.balanceOf("Dave"), "| 남은 허용량:",
tok.allowance("Alice", "Exchange"))

# approve 경쟁 조건 — 표준이 「먼저 0 으로 두라」고 권고한 이유를 순서로 재연한다
tok2 = ERC20Toy("Toy", "TOY"); tok2.mint("Alice", 1000)
tok2.approve("Alice", "Spender", 100)                      # ① Alice: Spender 에게 100 허용
# Alice 가 「100 은 너무 많다, 50 으로 줄이자」며 approve(Spender, 50) 를 보낸다. 그런데 블록에 들어가는
순서는 보낸 순서가 아니다.
tok2.transferFrom("Spender", "Alice", "Spender", 100)      # ② Spender 가 Alice 의 거래보다 먼저
100 을 전부 꺼낸다
tok2.approve("Alice", "Spender", 50)                       # ③ 그 뒤 Alice 의 approve(50) 이
적용된다 — 「덮어쓰기」라 50 이 새로 열린다
tok2.transferFrom("Spender", "Alice", "Spender", 50)       # ④ Spender 가 50 을 더 꺼낸다
print("경쟁 조건 — Alice 는 50 만 허용할 뜻이었으나 Spender 가 가져간 양:", tok2.balanceOf("Spender"), "
(= 100 + 50)")
print("이벤트 수:", len(tok2.events), "| 마지막 셋:", tok2.events[-3:])
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
잔액 — Alice: 750 Bob: 250 Carol(처음 보는 주소): 0 | 총량: 1000
Bob → Carol 300: 잔액 부족: Bob 는 250 뿐
Exchange 가 Alice 에게서 60 인출 → Dave: 60 | 남은 허용량: 40
경쟁 조건 — Alice 는 50 만 허용할 뜻이었으나 Spender 가 가져간 양: 150 (= 100 + 50)
이벤트 수: 5 | 마지막 셋: [('Transfer', 'Alice', 'Spender', 100), ('Approval', 'Alice', 'Spender',
50), ('Transfer', 'Alice', 'Spender', 50)]
```

[중급] 마지막 재연이 「거래 순서」 문제의 첫 모습입니다. Alice가 허용량을 100에서 50으로 **줄이려** 보낸 거래가, Spender가 100을 먼저 꺼낸 뒤에 처리되면 50이 새로 열립니다. `approve` 가 「더하기」가 아니라 「덮어쓰기」이기 때문이고, 체인에 들어가는 순서를 Alice가 정하지 못하기 때문입니다. 누가 그 순서를 정하는지가 17장입니다.

```
# NB14-2: ERC-721 장부 흉내 — 번호 → 소유자, 승인·운영자, safeTransferFrom 의 수취인 검사, 가변 URI
(공부용)
MAGIC = "onERC721Received-magic"                              # 실제로는
bytes4(keccak256("onERC721Received(address,address,uint256,bytes)")) — 아래 NB14-3 에서 계산

class Wallet:                                                 # 「계약 수취인」 흉내. 받을 준비가 된 계약은
마법값을 돌려준다
    def __init__(self, ready): self.ready = ready
    def onERC721Received(self, operator, frm, tokenId, data): return MAGIC if self.ready else
"something else"

class ERC721Toy:
    def __init__(self):
        self.owner = {}            # tokenId -> owner
        self.approved = {}         # tokenId -> approved address
        self.operators = set()     # (owner, operator)
        self.uri = {}              # tokenId -> URI 문자열 (메타데이터 확장, OPTIONAL)
        self.events = []
    def ownerOf(self, tid):
        if tid not in self.owner: raise KeyError(f"유효하지 않은 NFT {tid}")   # 유효하지 않은 NFT(0
주소 배정 등)의 조회는 실패한다 — 원문 ownerOf 주석의 요지
        return self.owner[tid]
    def balanceOf(self, who):  return sum(1 for o in self.owner.values() if o == who)
    def mint(self, to, tid, uri):                              # 표준 밖 — 생성 규칙은 발행자 몫
        self.owner[tid] = to; self.uri[tid] = uri; self.events.append(("Transfer", "0x0", to,
tid))
    def tokenURI(self, tid):   self.ownerOf(tid); return self.uri[tid]
    def _authorized(self, caller, tid):
        o = self.ownerOf(tid)
        return caller == o or self.approved.get(tid) == caller or (o, caller) in self.operators
    def approve(self, caller, approved, tid):
        o = self.ownerOf(tid)
        if not (caller == o or (o, caller) in self.operators): raise PermissionError("소유자나
운영자만 승인할 수 있다")
        self.approved[tid] = approved; self.events.append(("Approval", o, approved, tid))
    def setApprovalForAll(self, caller, operator, ok):
        (self.operators.add if ok else self.operators.discard)((caller, operator))
        self.events.append(("ApprovalForAll", caller, operator, ok))
    def transferFrom(self, caller, frm, to, tid):
        if not self._authorized(caller, tid): raise PermissionError(f"{caller} 는 {tid} 를 옮길
권한이 없다")
        if self.owner[tid] != frm: raise ValueError("_from 이 현재 소유자가 아니다")
        if to == "0x0": raise ValueError("0 주소로는 보낼 수 없다")
        self.owner[tid] = to; self.approved.pop(tid, None)   # 「승인은 이전 시 초기화된다」
        self.events.append(("Transfer", frm, to, tid))
    def safeTransferFrom(self, caller, frm, to, tid, data=b"", contracts=None):
        self.transferFrom(caller, frm, to, tid)
        if contracts and to in contracts:                      # 수취인이 계약이면 받을 준비를 묻는다
            if contracts[to].onERC721Received(caller, frm, tid, data) != MAGIC:
                self.owner[tid] = frm; self.events.pop()       # 되돌림(실제로는 거래 전체가 revert)
                raise ValueError(f"수취인 {to} 가 마법값을 돌려주지 않아 이전을 되돌렸다")

nft = ERC721Toy()
nft.mint("Alice", 7, "https://example.invalid/meta/7.json")
print("7번 소유자:", nft.ownerOf(7), "| tokenURI:", nft.tokenURI(7))
contracts = {"ReadyContract": Wallet(ready=True), "DumbContract": Wallet(ready=False)}
try:
    nft.safeTransferFrom("Alice", "Alice", "DumbContract", 7, contracts=contracts)
except ValueError as e:
    print("safeTransferFrom → 준비 안 된 계약:", e, "| 소유자는 여전히", nft.ownerOf(7))
nft.transferFrom("Alice", "Alice", "DumbContract", 7)       # 「안전하지 않은」 이전은 묻지 않고 보낸다
— 표준의 대문자 경고가 이 자리
print("transferFrom → 같은 계약: 소유자", nft.ownerOf(7), "(꺼낼 코드가 없는 계약이면 영구히 잠긴다)")

nft.mint("Bob", 8, "https://example.invalid/meta/8.json")
nft.setApprovalForAll("Bob", "Marketplace", True)           # Bob 이 장터 계약을 운영자로 둔다
nft.transferFrom("Marketplace", "Bob", "Carol", 8)           # 운영자가 Bob 의 토큰을 Carol 에게
옮긴다
print("운영자 이전 뒤 8번 소유자:", nft.ownerOf(8), "| Bob 의 보유 수:", nft.balanceOf("Bob"), "|
Carol:", nft.balanceOf("Carol"))
```

```
nft.uri[8] = "https://example.invalid/meta/8-v2.json"       # 발행자가 URI 를 바꾼다 — 소유자 칸은 그대로
print("URI 를 바꾼 뒤 — 소유자:", nft.ownerOf(8), "| tokenURI:", nft.tokenURI(8), "(체인이 보증하는 것은 왼쪽, 바뀔 수 있는 것은 오른쪽)")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
7번 소유자: Alice | tokenURI: https://example.invalid/meta/7.json
safeTransferFrom → 준비 안 된 계약: 수취인 DumbContract 가 마법값을 돌려주지 않아 이전을 되돌렸다 | 소유자는 여전히 Alice
transferFrom → 같은 계약: 소유자 DumbContract (꺼낼 코드가 없는 계약이면 영구히 잠긴다)
운영자 이전 뒤 8번 소유자: Carol | Bob 의 보유 수: 0 | Carol: 1
URI 를 바꾼 뒤 — 소유자: Carol | tokenURI: https://example.invalid/meta/8-v2.json (체인이 보증하는 것은 왼쪽, 바뀔 수 있는 것은 오른쪽)
```

[중급] 두 장면이 이 장의 요지입니다. `safeTransferFrom` 은 받을 준비가 안 된 계약을 알아보고 되돌렸지만, `transferFrom` 은 묻지 않고 보냈고 토큰은 꺼낼 코드가 없는 계약에 잠겼습니다. 그리고 발행자가 URI 문자열을 바꾸어도 소유자 칸은 꿈쩍하지 않습니다. 체인은 「8번의 소유자는 Carol」을 보증하고, 「8번이 가리키는 것이 무엇인가」는 보증하지 않습니다.

▾ [심화] 표준이 적어 둔 번호를 다시 계산한다 — 함수 선택자와 인터페이스 식별자

[심화] ERC-721 원문에는 「이 인터페이스의 ERC-165 식별자는 `0x80ac58cd`」, 메타데이터 확장은 `0x5b5e139f`, 수취 마법값은 `bytes4(keccak256("onERC721Received(address,address,uint256,bytes)"))` 라고 적혀 있습니다. 이 번호들은 어디서 왔을까요. 이더리움은 함수를 이름으로 부르지 않고 **선택자(selector)**, 곧 함수 서명 문자열의 Keccak-256 해시 앞 4바이트로 부릅니다. ERC-165는 「인터페이스 식별자를 그 인터페이스의 모든 함수 선택자의 XOR로 정의한다」고 적습니다.[22] 12장에서 제네시스 블록의 해시를 다시 계산해 공개값과 맞춘 것처럼, 여기서는 표준이 적은 번호를 다시 계산해 맞춥니다.

[심화] 한 가지 걸림돌이 있습니다. 이더리움의 Keccak-256은 Python `hashlib` 의 `sha3_256` 과 **같지 않습니다.** 둘은 같은 스펀지 함수를 쓰지만 채움(padding) 바이트가 다릅니다(Keccak 원안은 `0x01`, NIST 표준 SHA-3는 `0x06`). 그래서 아래 코드는 Keccak-f[1600] 순열을 표준 라이브러리 없이 직접 적고, 채움 바이트를 `0x06` 으로 두었을 때 `hashlib.sha3_256` 과 같은 값이 나오는지로 구현을 **자기 검증한** 뒤, 채움 바이트를 `0x01` 로 바꿔 이더리움의 Keccak-256을 얻습니다. 공부용 구현이며 실사용 금지입니다.

```
# NB14-3: Keccak-256 을 손으로 — 채움 바이트 하나가 SHA3-256 과 Keccak-256 을 가른다. 그 다음 ERC-
165/721 의 식별자를 재계산 (공부용, 실사용 금지)
import hashlib
from functools import reduce

RC = [0x0000000000000001, 0x0000000000008082, 0x800000000000808A, 0x8000000080008000,
0x000000000000808B, 0x0000000080000001,
      0x8000000080008081, 0x8000000000008009, 0x000000000000008A, 0x0000000000000088,
0x0000000080008009, 0x000000008000000A,
      0x000000008000808B, 0x800000000000008B, 0x8000000000008089, 0x8000000000008003,
0x8000000000008002, 0x8000000000000080,
      0x000000000000800A, 0x800000008000000A, 0x8000000080008081, 0x8000000000008080,
0x0000000080000001, 0x8000000080008008]
ROT = [[0, 36, 3, 41, 18], [1, 44, 10, 45, 2], [62, 6, 43, 15, 61], [28, 55, 25, 21, 56], [27,
20, 39, 8, 14]]   # 회전량 r[x][y]
M64 = (1 << 64) - 1
def rol(v, n): return ((v << n) | (v >> (64 - n))) & M64 if n else v

def keccak_f(A):                                          # A[x][y]: 5×5 개의 64비트 레인.
24라운드 θ·ρ·π·χ·ι
    for rc in RC:
        C = [A[x][0] ^ A[x][1] ^ A[x][2] ^ A[x][3] ^ A[x][4] for x in range(5)]
        D = [C[(x - 1) % 5] ^ rol(C[(x + 1) % 5], 1) for x in range(5)]
        A = [[A[x][y] ^ D[x] for y in range(5)] for x in range(5)]                        # θ
        B = [[0] * 5 for _ in range(5)]
        for x in range(5):
            for y in range(5):
                B[y][(2 * x + 3 * y) % 5] = rol(A[x][y], ROT[x][y])                      # ρ,
π
        A = [[B[x][y] ^ ((~B[(x + 1) % 5][y]) & B[(x + 2) % 5][y]) for y in range(5)] for x in
range(5)]   # χ
        A[0][0] ^= rc                                                                    # ι
    return A

def sponge256(msg: bytes, pad_byte: int) -> bytes:        # 용량 512비트 → 속도(rate) 1088비트 =
136바이트, 출력 256비트
    rate = 136
    padded = bytearray(msg) + bytearray([pad_byte]) + bytearray((-len(msg) - 2) % rate) +
bytearray([0x80])
    if len(msg) % rate == rate - 1:                       # 한 바이트만 남는 경우: pad_byte | 0x80
한 바이트로 채운다
        padded = bytearray(msg) + bytearray([pad_byte | 0x80])
    A = [[0] * 5 for _ in range(5)]
    for off in range(0, len(padded), rate):
        block = padded[off:off + rate]
        for i in range(rate // 8):                         # 레인 i → (x, y) = (i % 5, i //
5), 리틀엔디안
            A[i % 5][i // 5] ^= int.from_bytes(block[8 * i:8 * i + 8], "little")
        A = keccak_f(A)
    return b"".join(A[i % 5][i // 5].to_bytes(8, "little") for i in range(4))

def sha3_256_manual(m): return sponge256(m, 0x06)         # NIST SHA-3: 채움 접미 0x06
def keccak256(m):       return sponge256(m, 0x01)         # 이더리움 Keccak-256: 채움 접미 0x01

for m in [b"", b"abc", b"a" * 200]:
    print(f"{m[:12]!r:16} 손 SHA3-256 == hashlib.sha3_256 ? {sha3_256_manual(m) ==
hashlib.sha3_256(m).digest()}")
print("keccak256(b'')    =", keccak256(b"").hex())
print("sha3_256(b'')     =", hashlib.sha3_256(b"").hexdigest(), "(채움 바이트 하나가 다를 뿐인데 전혀
다른 값)")

def selector(sig: str) -> int: return int.from_bytes(keccak256(sig.encode())[:4], "big")
print()
print("transfer(address,uint256) 선택자 =", f"0x{selector('transfer(address,uint256)'):08x}")
print("supportsInterface(bytes4) 선택자 =", f"0x{selector('supportsInterface(bytes4)'):08x}", "
(ERC-165 는 이 값을 식별자로 쓴다)")

erc721 = ["balanceOf(address)", "ownerOf(uint256)",
"safeTransferFrom(address,address,uint256,bytes)",
          "safeTransferFrom(address,address,uint256)",
"transferFrom(address,address,uint256)", "approve(address,uint256)",
          "setApprovalForAll(address,bool)", "getApproved(uint256)",
```

```
    "isApprovedForAll(address,address)"]
    erc721_meta = ["name()", "symbol()", "tokenURI(uint256)"]
    xor = lambda sigs: reduce(lambda a, b: a ^ b, (selector(s) for s in sigs))
    print(f"ERC-721 식별자(9 함수 XOR)        = 0x{xor(erc721):08x}   표준 원문: 0x80ac58cd → 일치
    {xor(erc721) == 0x80ac58cd}")
    print(f"ERC-721 Metadata 식별자(3 함수)   = 0x{xor(erc721_meta):08x}   표준 원문: 0x5b5e139f → 일치
    {xor(erc721_meta) == 0x5b5e139f}")
    print(f"onERC721Received 마법값           =
    0x{selector('onERC721Received(address,address,uint256,bytes)'):08x}   표준 원문: 0x150b7a02 →
    일치 {selector('onERC721Received(address,address,uint256,bytes)') == 0x150b7a02}")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
b''              손 SHA3-256 == hashlib.sha3_256 ? True
b'abc'           손 SHA3-256 == hashlib.sha3_256 ? True
b'aaaaaaaaaaaa'  손 SHA3-256 == hashlib.sha3_256 ? True
keccak256(b'')    = c5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470
sha3_256(b'')     = a7ffc6f8bf1ed76651c14756a061d662f580ff4de43b49fa82d80a4b80f8434a (채움 바이트
하나가 다를 뿐인데 전혀 다른 값)

transfer(address,uint256) 선택자 = 0xa9059cbb
supportsInterface(bytes4) 선택자 = 0x01ffc9a7 (ERC-165 는 이 값을 식별자로 쓴다)
ERC-721 식별자(9 함수 XOR)        = 0x80ac58cd   표준 원문: 0x80ac58cd → 일치 True
ERC-721 Metadata 식별자(3 함수)   = 0x5b5e139f   표준 원문: 0x5b5e139f → 일치 True
onERC721Received 마법값           = 0x150b7a02   표준 원문: 0x150b7a02 → 일치 True
```

[심화] 표준 문서에 적힌 세 번호가 모두 다시 계산됩니다. 셋째 값은 원문이 수취 인터페이스 `ERC721TokenReceiver` 의 ERC-165 식별자로 적은 것이고 마법값 자체는 `bytes4(keccak256(…))` 식으로만 적혀 있는데, 그 인터페이스의 함수가 하나뿐이라 두 값이 같습니다. 「인터페이스를 지원한다」는 것은 결국 이 4바이트 숫자들을 안다는 뜻이고, 지갑이 처음 보는 계약에 `supportsInterface(0x80ac58cd)` 를 물어 참이 돌아오면 「ERC-721 함수들이 있다」고 믿는 것입니다. 함수가 **있다는** 것이지 **기대대로 동작한다는** 것은 아닙니다. 4절에서 본 대로 표준은 이전을 막는 구현도 허용합니다.

7. 정리

토큰은 계약 저장소 안의 사상 하나이고, 표준은 그 사상을 다루는 함수 이름의 약속입니다. ERC-20은 「주소 → 수량」을, ERC-721은 「번호 → 소유자」를 다루고, 둘 다 발행·소각·의미를 정하지 않습니다. NFT의 「그림」은 체인 밖에 있고 그 주소는 바뀔 수 있다고 표준이 스스로 적습니다. 같은 여섯 줄이 주식도 쿠폰도 점수도 될 수 있으며, 어느 쪽인지는 코드 밖에서 정해집니다. 다음 15장은 이 토큰들을 사람 없이 맞바꾸는 계약을 봅니다. 그 계약이 토큰을 받는 방법이 바로 이 장의 `approve` 와 `transferFrom` 입니다. 부록 B 오해 ②(「스마트계약은 계약이다」)와 ⑫(「탈중앙이면 책임자가 없다」)에 이 장이 더하는 것은, 표준 작성자들조차 「누가 발행하고 무엇을 뜻하는가」를 표준 밖에 두었다는 사실입니다.

이 장의 출처

- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014(초고 2013). https://ethereum.org/whitepaper/ (공식 PDF, 2026-10-25 접근) — 토큰 체계의 정의와 용도(「Token Systems」, PDF 19–20쪽), 이더의 역할(「Currency And Issuance」, 30쪽).
- Fabian Vogelsteller & Vitalik Buterin, "ERC-20: Token Standard," EIP-20, Final, created 2015-11-19. https://eips.ethereum.org/EIPS/eip-20 ; 고정 링크 github.com/ethereum/ERCs 커밋 8358b94b (2026-10-25 접근) — 동기, 명세(함수·이벤트·OPTIONAL 표시·approve 경고).
- William Entriken, Dieter Shirley, Jacob Evans, Nastassia Sachs, "ERC-721: Non-Fungible Token Standard," EIP-721, Final, created 2018-01-24, requires 165. https://eips.ethereum.org/EIPS/eip-721 ; 고정 링크 github.com/ethereum/ERCs 커밋 49e372bc (2026-10-25 접근) — 동기, 명세(인터페이스 주석·메타데이터 확장·JSON 스키마), 근거(식별자·이전 메커니즘·메타데이터 선택).
- Christian Reitwießner 외, "ERC-165: Standard Interface Detection," EIP-165, Final, created 2018-01-23. https://eips.ethereum.org/EIPS/eip-165 ; 고정 링크 github.com/ethereum/ERCs 커밋 8dd085d1 (2026-10-25 접근) — 인터페이스 식별자 정의.
- The Solidity Authors, *Solidity Documentation* v0.8.38, 「Introduction to Smart Contracts — Subcurrency Example」. https://docs.soliditylang.org/en/v0.8.38/introduction-to-smart-contracts.html (2026-10-25 접근; v0.8.37 판과 대조) — Coin 예제의 설명 문장과 mapping 설명. 예제 코드는 옮기지 않았다.
- Hayden Adams, Noah Zinsmeister, Dan Robinson, "Uniswap v2 Core," March 2020, §3.3 — 비표준 ERC-20 구현의 존재.
- U.S. Securities and Exchange Commission, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Release No. 81207, July 25, 2017 — DAO 토큰의 권리 서술(5–7쪽), 결론과 「코드가 적용을 배제하지 않는다」(1–2쪽).
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `hashlib`·`functools` 만 사용. Keccak-256은 공부용으로 직접 구현해 `hashlib.sha3_256` 과 대조했다. 노트북 NB14.

각주

1. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," ethereum.org 공식 PDF(2014; 초고 2013), 「Token Systems」 절(PDF 19쪽). 원문: "The key point to understand is that all a currency, or token systen, fundamentally is is a database with one operation: subtract X units from A and give X units to B, with the proviso that (i) X had at least X units before the transaction and (2) the transaction is approved by A. All that it takes to implement a token system is to implement this logic into a contract." (원문의 「(i)」「(2)」 표기, 「X had」, 「systen」 오탈자는 그대로 옮겼다.) 20쪽의 Serpent 코드: `from = msg.sender / to = msg.data[0] / value = msg.data[1] / if contract.storage[from] >= value: contract.storage[from] = contract.storage[from] - value / contract.storage[to] = contract.storage[to] + value`. https://ethereum.org/whitepaper/ (2026-10-25 접근).

2. 같은 절, 19쪽. 원문: "On-blockchain token systems have many applications ranging from sub-currencies representing assets such as USD or gold to company stocks, individual tokens representing smart property, secure unforgeable coupons, and even token systems with no ties to conventional value at all, used as point

systems for incentivization."

3. Fabian Vogelsteller & Vitalik Buterin, "ERC-20: Token Standard," EIP-20 (Standards Track / ERC, status: Final, created 2015-11-19), 「Motivation」. 원문: "A standard interface allows any tokens on Ethereum to be re-used by other applications: from wallets to decentralized exchanges." https://eips.ethereum.org/EIPS/eip-20 ; 고정 링크 https://github.com/ethereum/ERCs/blob/8358b94b5ed4009473cb4c3040d9bba60e7bb786/ERCS/erc-20.md (2026-10-25 접근, 원문 다시 열어 대조).

4. William Entriken, Dieter Shirley, Jacob Evans, Nastassia Sachs, "ERC-721: Non-Fungible Token Standard," EIP-721 (Standards Track / ERC, status: Final, created 2018-01-24, requires: 165), 「Motivation」. 원문: "This standard is inspired by the ERC-20 token standard and builds on two years of experience since EIP-20 was created. EIP-20 is insufficient for tracking NFTs because each asset is distinct (non-fungible) whereas each of a quantity of tokens is identical (fungible)." https://eips.ethereum.org/EIPS/eip-721 ; 고정 링크 https://github.com/ethereum/ERCs/blob/49e372bc8e5fab0ba2f7e60da2f0c2895ea5b119/ERCS/erc-721.md (2026-10-25 접근, 원문 다시 열어 대조).

5. Buterin (2014), 「Currency And Issuance」 절(PDF 30쪽). 원문: "The Ethereum network includes its own built-in currency, ether, which serves the dual purpose of providing a primary liquidity layer to allow for efficient exchange between various types of digital assets and, more importantly, of providing a mechanism for paying transaction fees." 같은 절의 발행 계획(판매 가격 등)은 2014년 계획이며 이 책은 옮기지 않습니다.

6. The Solidity Authors, *Solidity Documentation* v0.8.38, 「Introduction to Smart Contracts — Subcurrency Example」. 원문: "The following contract implements the simplest form of a cryptocurrency. The contract allows only its creator to create new coins (different issuance schemes are possible). Anyone can send coins to each other without a need for registering with a username and password, all you need is an Ethereum keypair." 예제 코드(`contract Coin`)는 GPL-3.0으로 배포되므로 이 책에 옮겨 싣지 않았습니다. 원래 코드는 아래 주소에서 볼 수 있습니다. https://docs.soliditylang.org/en/v0.8.38/introduction-to-smart-contracts.html (2026-10-25 접근; v0.8.37 판과 동일함을 대조).

7. 같은 문서, 같은 절. 원문: "Mappings can be seen as hash tables which are virtually initialized such that every possible key exists from the start and is mapped to a value whose byte-representation is all zeros. However, it is neither possible to obtain a list of all keys of a mapping, nor a list of all values."

8. EIP-20, 「Specification」. 원문 발췌: "Callers MUST handle `false` from `returns (bool success)`. Callers MUST NOT assume that `false` is never returned!"; name·symbol·decimals 각각에 "OPTIONAL - This method can be used to improve usability, but interfaces and other contracts MUST NOT expect these values to be present."; transfer: "Transfers `_value` amount of tokens to address `_to`, and MUST fire the `Transfer` event. The function SHOULD `throw` if the message caller's account balance does not have enough tokens to spend. *Note* Transfers of 0 values MUST be treated as normal transfers and fire the `Transfer` event."; transferFrom: "The `transferFrom` method is used for a withdraw workflow, allowing contracts to transfer tokens on your behalf. This can be used for example to allow a contract to transfer tokens on your behalf and/or to charge fees in sub-currencies. The function SHOULD `throw` unless the `_from` account has deliberately authorized the sender of the message via some mechanism."; approve: "Allows `_spender` to withdraw from your account multiple times, up to the `_value` amount. If this function is called again it overwrites the current allowance with `_value`."; Transfer 이벤트: "MUST trigger when tokens are transferred, including zero value transfers. A token contract which creates new tokens SHOULD trigger a Transfer event with the `_from` address set to `0x0` when tokens are created."

9. EIP-20, 「approve」 항. 원문: "**NOTE**: To prevent attack vectors like the one described here and discussed here, clients SHOULD make sure to create user interfaces in such a way that they set the allowance first to `0` before setting it to another value for the same spender. THOUGH The contract itself shouldn't enforce it, to allow backwards compatibility with contracts deployed before". 「described here」「discussed here」는 원문에서 외부 문서와 GitHub 토론으로 가는 링크이며 이 책은 그 문서들을 확보하지 않았습니다. 공격의 구조는 아래 NB14-1에서 이 책이 재구성한 것입니다.

10. Hayden Adams, Noah Zinsmeister, Dan Robinson, "Uniswap v2 Core," March 2020, §3.3 Handling non-standard and unusual tokens(7–8쪽). 원문: "The ERC-20 standard requires that transferand transferFromreturn a boolean indicating the success or failure of the call [4]. The implementations of one or both of these functions on some tokens—including popular ones ...—instead have no return value." (원문이 예로 든 토큰 이름은 이 책의 규칙에 따라 생략했다.)

11. EIP-721, 「Rationale — NFT Identifiers」. 원문: "Every NFT is identified by a unique `uint256` ID inside the ERC-721 smart contract. This identifying number SHALL NOT change for the life of the contract. The pair `(contract address, uint256 tokenId)` will then be a globally unique and fully-qualified identifier for a specific asset on an Ethereum chain. ... callers SHALL NOT assume that ID numbers have any specific pattern to them, and MUST treat the ID as a 'black box'. Also note that NFTs MAY become invalid (be destroyed)."

12. EIP-721, 「Specification」 인터페이스 주석. transferFrom: "Transfer ownership of an NFT -- THE CALLER IS RESPONSIBLE TO CONFIRM THAT `_to` IS CAPABLE OF RECEIVING NFTS OR ELSE THEY MAY BE PERMANENTLY LOST"; safeTransferFrom: "When transfer is complete, this function checks if `_to` is a smart contract (code size > 0). If so, it calls `onERC721Received` on `_to` and throws if the return value is not `bytes4(keccak256("onERC721Received(address,address,uint256,bytes)"))`." 두 함수 모두 "Throws unless `msg.sender` is the current owner, an authorized operator, or the approved address for this NFT. Throws if `_from` is not the current owner. Throws if `_to` is the zero address. Throws if `_tokenId` is not a valid NFT."

13. EIP-721, 「Specification」 setApprovalForAll 주석 및 「Rationale — Transfer Mechanism」. 원문: "Enable or disable approval for a third party ('operator') to manage all of `msg.sender`'s assets"; "Additionally, an authorized operator may set the approved address for an NFT. This provides a powerful set of tools for wallet, broker and auction applications to quickly use a *large* number of NFTs."

14. EIP-721, 「Rationale — Transfer Mechanism」. 원문: "In ERC-721, there is no allowance because every NFT is unique, the quantity is none or one. Therefore we receive the benefits of ERC-20's original design without problems that have been later discovered."

15. 같은 절. 원문: "Creation of NFTs ('minting') and destruction of NFTs ('burning') is not included in the specification. Your contract may implement these by other means."

16. 같은 절. 원문: "The transfer and accept functions' documentation only specify conditions when the transaction MUST throw. Your implementation MAY also throw in other situations. This allows implementations to achieve interesting results: Disallow transfers if the contract is paused ... Blocklist certain address from receiving NFTs ... Disallow unsafe transfers ... Charge a fee to both parties of a transaction ... Read only NFT registry — always throw from `safeTransferFrom`, `transferFrom`, `approve` and `setApprovalForAll`".

17. EIP-721, 「Specification」 ERC721Metadata 인터페이스와 JSON 스키마. 원문: "The **metadata extension** is OPTIONAL for ERC-721 smart contracts (see 'caveats', below). This allows your smart contract to be interrogated for its name and for details about the assets which your NFTs represent."; tokenURI: "A distinct Uniform Resource Identifier (URI) for a given asset. ... The URI may point to a JSON file that conforms to the 'ERC721 Metadata JSON Schema'."; 스키마의 image: "A URI pointing to a resource with mime type image/* representing the asset to which this NFT represents."

18. EIP-721, 「Rationale — Metadata Choices」. 원문: "*Alternatives considered: put all metadata for each asset on the blockchain (too expensive), use URL templates to query metadata parts (URL templates do not work with all URL schemes, especially P2P URLs), multiaddr network address (not mature enough)*".

19. 같은 절. 원문: "The URI MAY be mutable (i.e. it changes from time to time). We considered an NFT representing ownership of a house, in this case metadata about the house (image, occupants, etc.) can naturally change."; "We also remind everyone that any smart contract can use the same name and symbol as *your* contract. How a client may determine which ERC-721 smart contracts are well-known (canonical) is outside the scope of this standard."

20. U.S. Securities and Exchange Commission, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Exchange Act Release No. 81207 (July 25, 2017), 5–7쪽. 원문: "In exchange for ETH, The DAO created DAO Tokens (proportional to the amount of ETH paid) that were then assigned to the Ethereum Blockchain address of the person or entity remitting the ETH. A DAO Token granted the DAO Token holder certain voting and ownership rights." (5쪽); "The various promotional materials disseminated by Slock.it's co-founders touted that DAO Token holders would receive 'rewards,' which the White Paper defined as, 'any [ETH] received by a DAO [Entity] generated from projects the DAO [Entity] funded.'" (6쪽); "DAO Token holders were to cast votes, which would be weighted by the number of tokens they controlled, for or against the funding of a specific proposal." (7쪽). 보고서 2쪽 각주 2는 이 문서가 「어떤 사실이나 쟁점에 대한 판정(adjudication)이 아니며 어떤 개인이나 단체의 위반을 인정하는 것도 아니다」라고 밝힙니다.

21. 같은 보고서, 1쪽·2쪽. 원문: "Based on the investigation, and under the facts presented, the Commission has determined that DAO Tokens are securities under the Securities Act of 1933 ('Securities Act') and the Securities Exchange Act of 1934 ('Exchange Act')." (1쪽); "The automation of certain functions through this technology, 'smart contracts,' or computer code, does not remove conduct from the purview of the U.S. federal securities laws." (2쪽). 공격(2016-06-17, 모금액의 약 3분의 1 이동)과 하드포크(2016-07-20)는 9쪽 — 26장에서 다룹니다.

22. Christian Reitwießner 외, "ERC-165: Standard Interface Detection," EIP-165 (status: Final, created 2018-01-23), 「How Interfaces are Identified」. 원문: "For this standard, an *interface* is a set of function selectors as defined by the Ethereum ABI. … We define the interface identifier as the XOR of all function selectors in the interface." https://eips.ethereum.org/EIPS/eip-165 ; 고정 링크 https://github.com/ethereum/ERCs/blob/8dd085d159cb123f545c272c0d871a5339550e79/ERCS/erc-165.md (2026-10-25 접근). ERC-721의 식별자 표기는 EIP-721 「Specification」의 인터페이스 주석 "Note: the ERC-165 identifier for this interface is 0x80ac58cd." / "0x5b5e139f" / "0x150b7a02".

2부 스마트계약과 응용

15장 탈중앙 거래소와 AMM — 호가창 없는 가격 결정

핵심 TAKEAWAY

거래소에서 가격은 사람이 부릅니다. 사는 쪽과 파는 쪽이 호가를 내고, 운영자가 맞춰 줍니다. 자동화된 시장 조성자(automated market maker, AMM)에는 호가도 운영자도 없습니다. 계약 하나가 두 토큰을 통에 담아 두고, 「두 수량의 곱은 줄지 않는다」는 규칙 하나로 모든 거래를 받습니다. 그러면 가격은 **통 안의 두 수량의 비율이** 되고, 큰 주문은 스스로 가격을 밀어 올립니다(슬리피지). 통에 토큰을 넣어 두는 사람(유동성 공급자)은 거래마다 수수료를 받지만, 두 토큰의 상대가격이 움직이면 「그냥 들고 있었을 때」보다 적게 가지게 됩니다(비영구 손실). 두 효과는 같은 저울의 양쪽입니다. 이 장을 읽으면 「거래소」라는 말이 전제하는 운영자·상대방·호가가 여기에는 없다는 것을 수식으로 확인하고, 그래서 5부의 「누구를 규제하는가」라는 질문이 왜 비어 보이는지 알게 됩니다.

학습목표

1. 호가창 거래소와 상수곱 AMM의 가격 결정을 대조하고, AMM에서 「가격」이 무엇의 비율인지 말한다.
2. [중급] 상수곱 $x \cdot y = k$ 에서 주문 크기 Δx 에 대한 수취량 Δy 와 실행 가격을 유도하고, 수수료가 어디서 빠지는지 안다.
3. [중급] 유동성 공급자의 지분 계산과 수수료 적립, 그리고 비영구 손실 $\frac{2\sqrt{r}}{1+r} - 1$ 을 유도해 **수수료 수입과 대칭으로** 읽는다.
4. [심화] 집중 유동성(가격 구간 공급)이 상수곱 곡선을 어떻게 잘라 쓰는지, 그리고 AMM 현물가가 오라클로 쓰이면 왜 조작되는지 안다.

[입문]만 읽은 독자가 가져가는 것. AMM에서 가격은 사람이 부르는 것이 아니라 **풀 안의 두 수량의 비율입니다.** 큰 주문은 스스로 가격을 밀어 올립니다.

1. 이 기술은 무엇의 실패에서 나왔나

전통적인 거래소는 **호가창(order book)으로** 돌아갑니다. 다이안(Philip Daian)과 공저자들의 2020년 논문이 깔끔하게 적습니다. 「고전적 거래소는 … 연속 지정가 호가창이라는 공통 설계를 공유한다. 호가창은 체계 안의 모든 매수·매도 미체결 주문의 목록이다. 매수 희망자는 자산을 사겠다는 최대 가격을 적은 지정가 매수 주문을 내고, 매도자도 마찬가지다. **중앙의 상대방인 거래소 운영자가** 매수자와 매도자를 짝지어, 매수 주문보다 낮은 가격의 매도 주문이 장부에 있으면 자동으로 거래를 성립시킨다.」[1] 이 구조를 블록체인 위에 그대로 옮기면 두 가지가 걸립니다. 첫째, 「체인 위에서 스마트계약이 중개하는 거래는 **느리다**.」 주문을 내고 취소하는 매 단계가 블록을 기다려야 하고, 그 사이에 이미 체결되었거나 취소된 주문이 살아 있는 것처럼 보입니다.[2] 둘째, 호가창을 누가 들고 있느냐입니다. 논문이 살핀 탈중앙 거래소들은 「호가창을 보통 체인 밖에서 유지」했고, 그래서 논문은 「「탈중앙」 거래소는 다소 잘못된 이름이다. 그런 체계 대부분은 중앙화된 부품을 갖고 있으며 … 더 정확히는 **비수탁(non-custodial)**, 곧 사용자가 자기 자금의 지배를 제3자에게 넘기지 않고 거래하는 것으로 분류되어야 한다」고 적습니다.[3]

그래서 나온 다른 설계가 호가창을 **없애는** 것입니다. 같은 논문의 묘사입니다. 「더 급진적인 DEX 설계인 자동화된 시장 조성자는 호가창을 완전히 건너뛴다. DEX는 토큰 또는 이더의 **준비금을 스스로 보유하는 스마트계약으로** 이루어진다. 두 자산 A와 B를 생각하자. 계약은 사용자가 언제든 자기 준비금을 상대방으로 삼아 정해진 비율로 A와 B를 맞바꿀 수 있게 한다. 사용자가 B로 A를 사면, 계약이 다음 거래에 제시하는 B 기준 A의 가격은 오른다. 반대로 A를 팔면 가격은 내린다. 이렇게 하면 한 사람이 상대방을 찾거나 짝짓기 없이 거래할 수 있다. 그 결과, 제시 비율을 시장 비율과 맞추기 위해 이 거래소와 다른 거래소 사이의 **끊임없는 차익거래가 필요하다**.」[4] 이 마지막 문장을 기억해 두십시오. AMM은 시장가격을 「알지」 못합니다. 차익거래자가 밖에서 와서 맞춰 줄 때까지 틀린 가격을 제시합니다. 그것이 16장과 17장의 씨앗입니다.

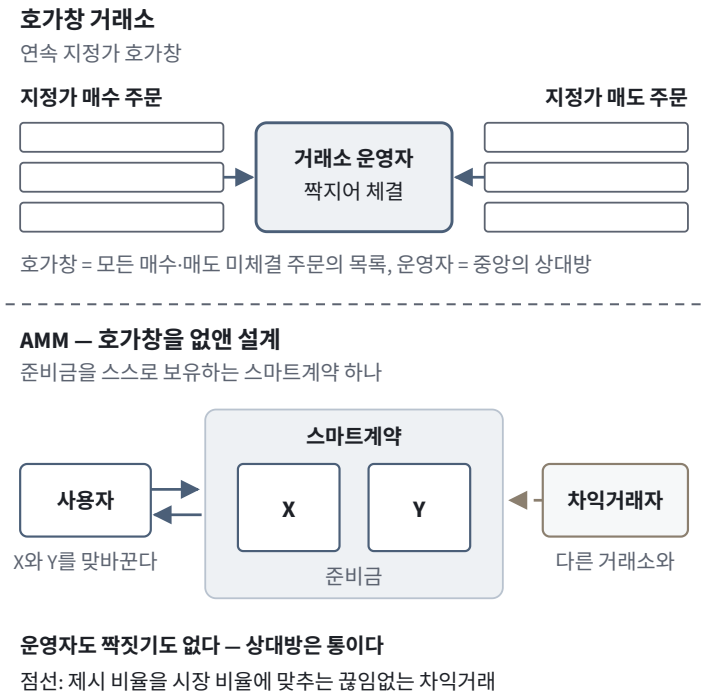


그림 15-1. 호가창 거래소와 AMM의 구조. AMM에는 호가창도 운영자도 없고, 제시 비율을 시장 비율에 맞추는 일은 바깥의 차익거래자가 한다.

2. 직관 — 통 두 개와 곱 하나

통에 토큰 X가 x 개, 토큰 Y가 y 개 들어 있습니다. 규칙은 하나입니다. **거래 뒤의 $x \cdot y$ 는 거래 전의 $x \cdot y$ 보다 작아질 수 없다.** 이것이 「상수곱(constant product)」 규칙이고, 유니스왑(Uniswap) v1·v2가 쓴 공식입니다. v2 백서의 서술: 「각 유니스왑 v1 짝(pair)은 두 자산의 준비금을 통에 담아 두 자산에 유동성을 제공하며, **준비금의 곱이 줄어들 수 없다는 불변량을** 유지한다. 거래자는 거래마다 30베이시스포인트의 수수료를 내고, 그것은 유동성 공급자에게 간다.」[5]

당신이 X를 Δx 개 넣고 Y를 꺼내 가려 합니다. 통의 X는 $x + \Delta x$ 가 되고, 곱을 지키려면 Y는 $k / (x + \Delta x)$ 까지만 남아야 합니다. 그러니 당신이 받는 Y는 그 차이입니다. 수치 예로 X 1,000개와 Y 1,000개가 든 통($k = 1{,}000{,}000$)에 X 100개를 넣으면, Y는 1,000,000 / 1,100 ≈ 909.1 개가 남아야 하므로 당신은 약 90.9개를 받습니다. 넣은 X 100개로 Y 90.9개를 받았으니 실행 가격은 X 하나에 Y 0.909개입니다. 그런데 거래 **직전의** 「가격」은 $y / x = 1$ 이었습니다. 실행 가격이 거래 직전 가격보다 9% 넘게 나빴습니다. 이것이 **슬리피지(slippage)이고**, 주문이 통에 비해 클수록 커집니다. 거래 뒤의 비율은 909.1 / 1,100 ≈ 0.826 입니다. 다음 사람은 X 하나에 Y 0.826개부터 시작합니다. 아무도 가격을 「정하지」 않았습니다. 비율이 바뀐 것뿐입니다.

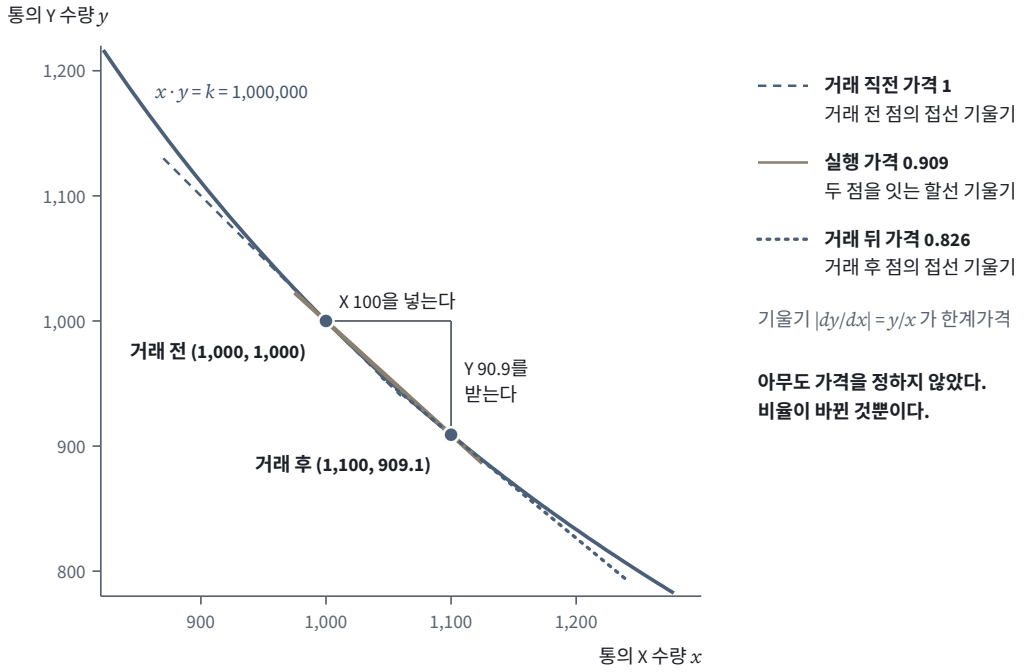


그림 15-2. X 1,000개와 Y 1,000개가 든 통에 X 100개를 넣는 거래. 곱은 그대로이고 비율만 바뀐다. 곡선은 본문 식 $xy = k$ 의 그래프이고, 세 직선의 기울기가 거래 직전 가격 1, 실행 가격 0.909, 거래 뒤 가격 0.826이다.

여기에 사람이 없다는 점을 다시 봅니다. 당신의 상대방은 통입니다. 통에 토큰을 넣어 둔 사람들(유동성 공급자)이 집합적으로 상대방 노릇을 하지만, 그들은 당신의 거래를 알지도 승낙하지도 않습니다. 운영자도 없습니다. v2 백서는 「계약은 일반적으로 업그레이드할 수 없다」고 적으면서도 「팩토리 계약의 변수 하나를 바꿔 체인 위 5베이시스포인트 수수료를 켤 수 있는 개인키가 하나 있다」고 밝힙니다.[6] 운영자는 없지만 **스위치 하나를 쥔 열쇠는** 있습니다. 「탈중앙」이라는 말이 어디까지인지 읽을 때 이런 문장을 찾아야 합니다.

> **? 법률가의 질문**
>
> 거래소 규제는 보통 「운영자」를 전제합니다. 등록하고, 상장 심사를 하고, 시세조종을 감시하고, 고객 자산을 분리 보관할 **누군가입니다**. 상수곱 AMM에서 그 누군가는 어디에 있습니까. 코드를 배포한 사람인가요, 통에 토큰을 넣어 둔 수천 명의 유동성 공급자인가요, 수수료 스위치의 열쇠를 쥔 주소인가요, 거버넌스 토큰의 다수인가요, 아니면 아무도 아닌가요. 당신이 통과 거래했을 때 「상대방」은 누구이고, 가격이 틀렸다면 누가 틀린 것인가요. → 27장 · 28장

3. 수식 — 상수곱, 수수료, 실행 가격

[중급] 기호를 둡니다. x, y 는 거래 전 준비금, $k = xy$, Δx 는 넣는 양, Δy 는 받는 양입니다. 수수료가 없으면

$$(x + \Delta x)(y - \Delta y) = k \quad \Longrightarrow \quad \Delta y = y - \frac{k}{x + \Delta x} = \frac{y\,\Delta x}{x + \Delta x}.$$

기호 풀이: 분자는 「넣은 비율만큼의 Y」, 분모는 「넣은 뒤의 X」입니다. 수치 예: $x = y = 1{,}000$, $\Delta x = 100$ 이면 $\Delta y = 1{,}000 \times 100 / 1{,}100 \approx 90.91$. 실행 가격 $\Delta y / \Delta x \approx 0.909$, 거래 직전의 한계가격 $y / x = 1$, 거래 뒤의 한계가격 $(y - \Delta y) / (x + \Delta x) \approx 0.826$ 입니다.

[중급] 한계가격. v2 백서는 「시각 t 에 유니스왑이 제시하는 (수수료 제외) 한계가격은 자산 a의 준비금을 자산 b의 준비금으로 나눠 계산할 수 있다」고 적습니다. $p_t = r_t^a / r_t^b$.[7] 수학적으로는 곡선 $y = k / x$ 의 기울기 $|dy / dx| = y / x$ 입니다. 「아주 작은 거래」의 가격이고, 실제 거래는 그보다 나쁜 가격을 받습니다. 그 차이가 슬리피지입니다. 슬리피지를 한계가격 대비 비율로 쓰면

$$\text{슬리피지} = 1 - \frac{\Delta y / \Delta x}{y / x} = 1 - \frac{x}{x + \Delta x} = \frac{\Delta x}{x + \Delta x}.$$

통 크기에 대한 주문 크기의 비율로만 정해집니다. 1,000짜리 통에 100을 넣으면 100 / 1,100 ≈ 9.1%, 10을 넣으면 10 / 1,010 ≈ 1.0%, 1,000을 넣으면 50%입니다. 통이 클수록 같은 주문의 슬리피지는 작습니다. 이것이 유동성 공급자가 왜 필요한지의 전부입니다.

[중급] 수수료. v1·v2는 넣는 양의 0.3%를 떼고 나머지로 곱을 지킵니다. 백서는 「v1의 거래 수수료는 상수곱 불변량을 강제하기 전에 계약에 지불된 양을 0.3% 줄이는 방식으로 적용된다. 계약은 암묵적으로 다음 식을 강제한다」라고 적고 $(x_1 - 0.003 \cdot x_{in}) \cdot y_1 \geq x_0 \cdot y_0$ 을 보인 뒤, v2에서는 두 자산이 동시에 들어올 수 있어 $(x_1 - 0.003 \cdot x_{in})(y_1 - 0.003 \cdot y_{in}) \geq x_0 y_0$ 으로 일반화합니다.[8] 기호 풀이: 아래 첨자 0은 거래 전, 1은 거래 후, x_{in} 은 들어온 X입니다. 수수료 $\gamma = 0.003$ 을 떼면 받는 양은

$$\Delta y = \frac{y\,(1 - \gamma)\,\Delta x}{x + (1 - \gamma)\,\Delta x}$$

가 됩니다. 수치 예: 같은 통에 100을 넣으면 $(1 - \gamma)\Delta x = 99.7$, $\Delta y = 1{,}000 \times 99.7 / 1{,}099.7 \approx 90.66$. 수수료가 없을 때의 90.91보다 0.25개 적게 받습니다. 그 0.3개(= 100 × 0.003)의 X는 어디로 갔을까요. **통에 남습니다.** 거래 뒤 통에는 X 1,100개, Y 909.34개가 있고, 그 곱은 약 1,000,273 으로 거래 전보다 **커졌습니다.** 수수료는 따로 어디로 송금되는 것이 아니라 k 를 키우는 모양으로 통 안에 쌓이고, 유동성 공급자는 자기 지분만큼 커진 통을 나중에 꺼내 갑니다. 이것이 다음 절의 열쇠입니다.

4. 유동성 공급 — 수수료 수입과 비영구 손실은 같은 저울의 양쪽

통에 토큰을 넣는 사람은 왜 넣을까요. 거래마다 떼이는 수수료가 통 안에 쌓이기 때문입니다. 그러면 공짜일까요. 아닙니다. 두 토큰의 상대가격이 움직이면, 통은 **오른 쪽 토큰을 팔고 내린 쪽 토큰을 사는 쪽으로** 자동으로 재조정됩니다. 차익거래자가 와서 비율을 맞추니까요. 그 결과 공급자는 「그냥 두 토큰을 들고 있었을 때」보다 적게 가지게 되고, 이 차이를 관행적으로 **비영구 손실(impermanent loss)이라** 부릅니다. v2 백서도 이 말을 씁니다. v1에서는 모든 짝에 이더가 끼어 있어 「모든 유동성 공급자가 이더에 노출되고, 다른 자산의 이더 대비 가격 변화에 따라 비영구 손실을 입는다」고요.[9] 「비영구」라는 이름은 가격이 원래 비율로 돌아오면 손실도 사라진다는 뜻이지, 손실이 가볍다는 뜻이 아닙니다. 이 책은 **공급을 「수익」이라 부르지 않습니다.** 수수료 수입과 비영구 손실을 같은 표에 놓고, 어느 쪽이 큰지는 거래량과 가격 변동에 달린 **실측의 문제로** 남깁니다.

[중급] 지분. 공급자는 넣은 몫에 비례해 「유동성 토큰(지분)」을 받습니다. 처음 넣는 사람은 $s = \sqrt{x_{deposited}\, y_{deposited}}$ 만큼 받고, 그 뒤 사람은 기존 준비금 대비 비율로 받습니다. $s_{minted} = \frac{x_{deposited}}{x_{starting}} s_{starting}$.[10] 꺼낼 때는 자기 지분 비율만큼 두 토큰을 **그때의 비율로** 받습니다. 그래서 거래 수수료로 k 가 커졌으면 같은 지분으로 더 많이 받고, 가격이 움직였으면 두 토큰의 구성이 달라져 있습니다. 백서는 수수료 적립량을 「$\sqrt{k}$ 의 성장」으로 잽니다. t_1 과 t_2 사이에 쌓인 수수료는 t_2 시점 통의 비율로 $f_{1,2} = 1 - \sqrt{k_1} / \sqrt{k_2}$ 입니다.[11]

[중급] 비영구 손실의 식. 상대가격이 p 에서 rp 로 바뀌었다고 합시다(r 은 가격 배율). 수수료가 없고 차익거래가 비율을 완전히 맞춘다고 가정하면, 통의 두 수량은 $x' = x / \sqrt{r}$, $y' = y\sqrt{r}$ 이 됩니다(곱은 그대로, 비율 $y'/x' = r\,y/x$). Y를 단위로 통의 가치를 재면 공급 쪽은 $V_{pool} = x' \cdot rp + y' = 2y\sqrt{r}$, 그냥 들고 있는 쪽은 $V_{hold} = x \cdot rp + y = y(1+r)$ 이므로

$$\frac{V_{pool}}{V_{hold}} = \frac{2\sqrt{r}}{1+r}, \qquad \text{비영구 손실} = \frac{2\sqrt{r}}{1+r} - 1 \le 0\,.$$

기호 풀이: $r = 1$ 이면 0, r 이 1에서 멀어질수록(어느 방향이든) 음수가 커집니다. 수치 예: $r = 2$ 면 $2\sqrt{2}/3 - 1 \approx -5.7\%$, $r = 4$ 면 $4/5 - 1 = -20\%$, $r = 1/4$ 도 -20% 입니다. 이 식은 이 책의 유도이고, 유도 과정은 아래 [심화] 상자와 NB15-2에 있습니다. 중요한 것은 모양입니다. **방향과 무관하게 손실이고, 변동이 클수록 크며, 수수료가 그것을 메우는가는 별개의 수치입니다.** 아래 코드가 두 수치를 한 표에 놓습니다.

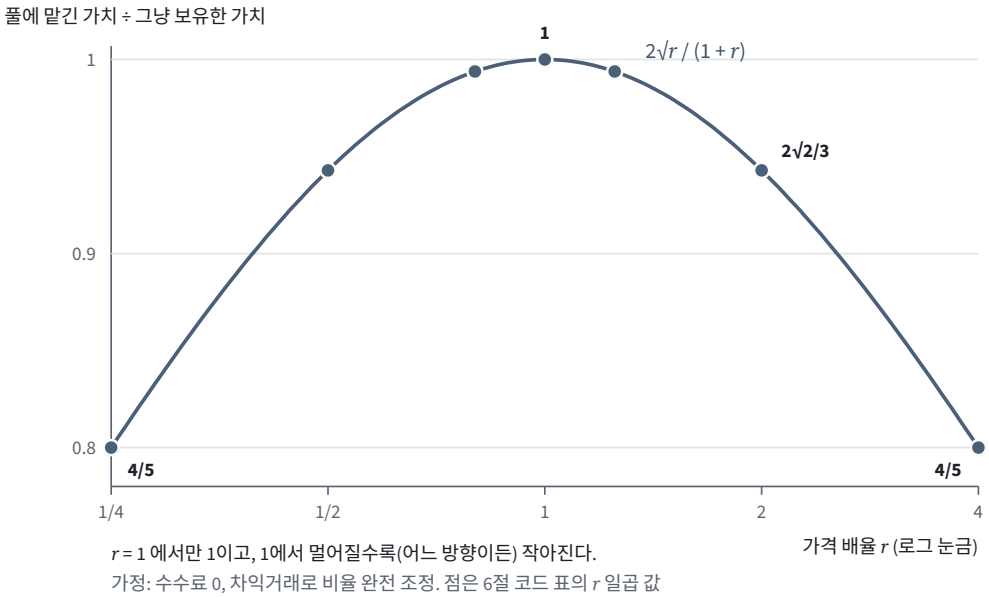


그림 15-3. 가격 배율 r 에 따라 풀에 맡긴 가치가 그냥 보유한 가치의 몇 배가 되는가. 곡선은 본문 식 $2\sqrt{r}\,/\,(1+r)$ 의 그래프이고, 가로축은 시간이 아니라 배율이다. 수수료 0과 차익거래의 완전 조정을 가정한다.

▾ [심화] 비영구 손실의 유도, 그리고 집중 유동성 — 곡선을 잘라 쓰기

[심화] 유도. 준비금 (x, y), $k = xy$, 거래 전 상대가격 $p = y/x$(X 하나의 Y 가격). 밖의 가격이 rp 가 되었습니다. 차익거래자는 통의 한계가격 y'/x' 가 rp 와 같아질 때까지 거래합니다. 두 조건 $x'y' = k$, $y'/x' = rp = ry/x$ 를 풀면 $x'^2 = k/(rp) = x^2/r$, 곧 $x' = x/\sqrt{r}$, $y' = y\sqrt{r}$. 통의 Y 기준 가치는 $x' \cdot rp + y' = (x/\sqrt{r})(ry/x) + y\sqrt{r} = 2y\sqrt{r}$. 보유 가치는 $x \cdot rp + y = ry + y$. 비율은 $2\sqrt{r}\,/\,(1+r)$ 이고, 산술–기하 평균 부등식 $\frac{1+r}{2} \geq \sqrt{r}$ 에 의해 1 이하입니다. 등호는 $r = 1$ 에서만 성립합니다. 수수료를 넣으면 k 가 거래마다 조금씩 자라므로 통의 가치에 그만큼이 더해집니다. 그래서 공급자의 최종 손익은 「수수료로 자란 $\sqrt{k}$」와 「$2\sqrt{r}\,/\,(1+r)$」의 곱으로 정해집니다.

[심화] 집중 유동성. 상수곱 곡선은 가격 $(0,\ \infty)$ 전 구간에 유동성을 펴 놓습니다. v3 백서는 이것이 「단순하고 유동성을 효율적으로 합칠 수 있지만, 통에 든 자산의 많은 부분이 **결코 쓰이지 않는다는** 뜻」이라고 적습니다.[12] 그래서 v3는 공급자가 가격 구간 $[p_a, p_b]$ 를 골라 그 구간에만 유동성을 두게 합니다. 「포지션은 자기 구간 안의 거래를 지탱할 만큼의 준비금만 유지하면 되므로, 그 구간 안에서는 **더 큰 준비금(가상 준비금)을 가진 상수곱 풀처럼** 행동할 수 있다.」 유동성의 크기를 $L = \sqrt{k}$ 로 재면 실제 준비금은

$$\left(x + \frac{L}{\sqrt{p_b}}\right)\left(y + L\sqrt{p_a}\right) = L^2$$

을 따릅니다.[13] 기호 풀이: 괄호 안의 덧셈 항이 「가상」 부분이고, 가격이 p_b 에 닿으면 X가, p_a 에 닿으면 Y가 바닥납니다. 「가격이 포지션의 구간을 벗어나면 그 포지션의 유동성은 더 이상 활성이 아니며 수수료를 벌지 않는다. 그 시점에 유동성은 한 자산으로만 이루어진다.」[14] 수치 예: 현재 가격 1, 구간 $[0.5, 2]$, $L = 1{,}000$ 이면 가상 준비금은 $x_v = L/\sqrt{P} = 1{,}000$, $y_v = L\sqrt{P} = 1{,}000$ 인데 실제로 넣어야 하는 양은 $x = L/\sqrt{P} - L/\sqrt{p_b} = 1{,}000 - 707.1 \approx 292.9$, $y = L\sqrt{P} - L\sqrt{p_a} = 1{,}000 - 707.1 \approx 292.9$ 입니다. 전 구간 풀의 29%만 넣고 같은 구간 안에서 같은 슬리피지를 제공합니다. 대가는 구간을 벗어나면 한 자산만 들고 있게 되는 것입니다. 그것이

전 구간 풀보다 비영구 손실을 더 가파르게 실현시킨다는 서술은 이 책의 설명이며, 원문은 「비활성·수수료 없음·단일 자산」까지만 적습니다. 수수료 단계도 셋(0.05%, 0.30%, 1%)으로 갈라 짝마다 여러 풀을 둘 수 있게 했습니다.[15] 이 책은 어느 설계가 낫다고 말하지 않습니다. 하나는 곡선을 전부 쓰고, 하나는 잘라 씁니다.

5. 현물가는 오라클이 아니다 — 16·17장으로 가는 다리

AMM의 비율은 누구나 읽을 수 있는 「가격」처럼 보입니다. 그래서 다른 계약이 그것을 가격 정보로 가져다 쓰고 싶어집니다. v2 백서는 그것이 왜 위험한지 공격 하나로 보입니다. 「어떤 다른 계약이 현재의 ETH-DAI 가격을 써서 파생상품을 정산한다고 하자. 측정되는 가격을 조작하려는 공격자는 ETH-DAI 짝에서 ETH를 사서(가격을 밀어 올리고), 파생상품 계약의 정산을 촉발해(부풀린 가격으로 정산되게 하고), 그 뒤 ETH를 짝에 되팔아 진짜 가격으로 되돌릴 수 있다. 이것은 **원자적 거래 하나로**, 또는 블록 안의 거래 순서를 지배하는 채굴자에 의해 이루어질 수도 있다.」[16] 2절에서 본 대로 큰 주문은 비율을 밀어 올립니다. 그 밀린 비율을 「가격」으로 읽는 계약이 있으면, 밀기 → 읽히기 → 되돌리기를 한 거래 안에 묶을 수 있습니다. 백서는 각주에서 「v1을 오라클로 쓴 것이 계약을 이런 공격에 취약하게 만든 실제 사례」로 외부 글 하나를 가리킵니다.[17] 이 책은 그 글을 확보하지 않았으므로 사례의 내용은 쓰지 않습니다.

v2의 대응은 **시간 가중 평균입니다**. 블록의 첫 거래 직전 가격을 누적해 두고, 두 시점의 누적값 차이를 경과 시간으로 나누면 그 구간의 평균 가격이 나옵니다. 「더 긴 기간을 고르면 공격자가 평균을 조작하는 비용이 커지지만, 가격은 덜 최신이 된다.」[18] 조작 비용과 신선도의 맞바꿈입니다. 그래도 백서는 블록 끝의 가격은 채굴자(또는 블록 하나를 가득 채울 만큼 가스를 쓰는 공격자)가 조작할 수 있다고 인정하면서, 「다음 블록까지 채굴하지 않는 한 되돌리는 차익거래에서 특별한 이점은 없을 수 있다」고 덧붙입니다. 「체인 밖 사실을 체인 안으로」라는 더 큰 문제는 16장이, 「거래 순서를 정하는 사람」은 17장이 받습니다.

6. 코드 — 상수곱 풀, 슬리피지 곡선, 그리고 저울의 양쪽

상수곱 풀과 슬리피지

[중급] 풀 클래스 하나를 만듭니다. 17장의 샌드위치 시뮬레이션이 이 클래스를 그대로 다시 씁니다. 단위는 전부 **풀 안의 토큰 개수이고**, 통화 단위는 없습니다.

```
# NB15-1: 상수곱 풀 — 스왑(수수료 포함), 유동성 추가·제거, 슬리피지 곡선 (공부용)
import math

class ConstantProductPool:
    """x·y = k 풀. fee 는 넣는 양에 매기는 비율(기본 0.003 = 30 bp). 지분은 처음 sqrt(x·y), 이후 비례."""
    def __init__(self, x: float, y: float, fee: float = 0.003):
        self.x, self.y, self.fee = float(x), float(y), fee
        self.shares = {"LP0": math.sqrt(x * y)}            # 첫 공급자의 지분 = 기하평균 (v2 식 (13))
    @property
    def k(self): return self.x * self.y
    @property
    def price_x_in_y(self): return self.y / self.x           # 한계가격: X 하나의 Y 가격 (v2 식 (1))
    def quote_x_for_y(self, dx: float) -> float:              # X 를 dx 넣으면 Y 를 얼마 받나 (수수료 뗀 뒤 곱을 지킴)
        eff = dx * (1 - self.fee)
        return self.y * eff / (self.x + eff)
    def swap_x_for_y(self, dx: float) -> float:
        dy = self.quote_x_for_y(dx)
        self.x += dx; self.y -= dy                            # 수수료 몫은 풀에 남아 k 가 자란다
        return dy
    def swap_y_for_x(self, dy: float) -> float:
        eff = dy * (1 - self.fee)
        dx = self.x * eff / (self.y + eff)
        self.y += dy; self.x -= dx
        return dx
    def add_liquidity(self, who: str, dx: float) -> float:    # 현재 비율대로 X dx 와 Y dx·(y/x) 를 넣는다 (v2 식 (12))
        dy = dx * self.y / self.x
        minted = dx / self.x * sum(self.shares.values())
        self.x += dx; self.y += dy
        self.shares[who] = self.shares.get(who, 0) + minted
        return minted
    def remove_liquidity(self, who: str):                     # 지분 비율만큼 두 토큰을 그때의 비율로 돌려준다
        s_total = sum(self.shares.values())
        frac = self.shares.pop(who) / s_total
        out_x, out_y = self.x * frac, self.y * frac
        self.x -= out_x; self.y -= out_y
        return out_x, out_y

pool = ConstantProductPool(1000, 1000)
print(f"처음: x={pool.x:.1f} y={pool.y:.1f} k={pool.k:,.0f} 한계가격(X 하나의 Y 가격)={pool.price_x_in_y:.4f}")
print(f"{'주문 Δx':>8} {'받는 Δy':>10} {'실행가격 Δy/Δx':>14} {'슬리피지':>9} {'거래 뒤 한계가격':>14}")
for dx in [1, 10, 100, 500, 1000]:
    p = ConstantProductPool(1000, 1000)
    before = p.price_x_in_y
    dy = p.swap_x_for_y(dx)
    exec_price = dy / dx
    print(f"{dx:>8} {dy:>10.3f} {exec_price:>14.4f} {1 - exec_price / before:>9.2%} {p.price_x_in_y:>14.4f}")

# 수수료는 어디로 갔나 — 100 을 넣은 뒤 k 를 본다
p = ConstantProductPool(1000, 1000); dy = p.swap_x_for_y(100)
print(f"\n100 을 넣은 뒤: 받은 Δy={dy:.3f} (수수료 없을 때 90.909), 풀 x={p.x:.1f} y={p.y:.3f}, k={p.k:,.1f} (거래 전 1,000,000 — 수수료 0.3 X 가 풀에 남아 k 가 자랐다)")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
처음: x=1000.0 y=1000.0 k=1,000,000 한계가격(X 하나의 Y 가격)=1.0000
   주문 Δx      받는 Δy     실행가격 Δy/Δx      슬리피지       거래 뒤 한계가격
        1       0.996          0.9960     0.40%          0.9980
       10       9.872          0.9872     1.28%          0.9803
      100      90.661          0.9066     9.34%          0.8267
      500     332.666          0.6653    33.47%          0.4449
     1000     499.249          0.4992    50.08%          0.2504

100 을 넣은 뒤: 받은 Δy=90.661 (수수료 없을 때 90.909), 풀 x=1100.0 y=909.339, k=1,000,272.8 (거래 전
1,000,000 — 수수료 0.3 X 가 풀에 남아 k 가 자랐다)
```

[중급] 표가 2절의 산수를 확인합니다. 주문이 통의 1%면 슬리피지는 약 1%(수수료 포함 1.28%), 10%면 약 9%(9.34%), 통만큼 넣으면 절반을 잃습니다. 이 표의 수치는 「풀 1,000:1,000」이라는 **구조 변수로만** 정해지고, 토큰이 무엇인지와 무관합니다. 이 장의 위젯 W7은 이 두 변수(풀 크기, 주문 크기)만 움직입니다.



그림 15-4. AMM 슬리피지. 풀 1,000 : 1,000 에 X 100 을 넣으면 Y 90.661 을 받고(수수료 0.3% 포함) 슬리피지는 9.34%다. 단위는 풀 안의 토큰 개수이고 통화가 아니다. — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 15장 6절).

저울의 양쪽 — 비영구 손실과 수수료

```
# NB15-2: 저울의 양쪽 — 가격 배율 r 에 따른 비영구 손실, 그리고 수수료로 자란 sqrt(k) 가 그것을 얼마나 메우는가 (공부용)
import math

def impermanent_loss(r: float) -> float:                    # 이 책의 유도: V_pool / V_hold − 1 = 2√r/(1+r) − 1
    return 2 * math.sqrt(r) / (1 + r) - 1

print("가격 배율 r 과 비영구 손실(수수료 0, 차익거래로 비율 완전 조정 가정):")
print(f"{'r':>6} {'보유 가치(Y 단위)':>16} {'풀 가치(Y 단위)':>16} {'비영구 손실':>10}")
x0 = y0 = 1000.0
for r in [0.25, 0.5, 0.8, 1.0, 1.25, 2.0, 4.0]:
    hold = x0 * r * (y0 / x0) + y0                              # X 를 새 가격 r·(y/x) 로 평가 + Y
    pool_val = 2 * y0 * math.sqrt(r)
    print(f"{r:>6.2f} {hold:>16.1f} {pool_val:>16.1f} {impermanent_loss(r):>10.2%}")

# 수수료 쪽 저울 — 거래가 왕복해 비율은 제자리로 돌아오지만 k 는 자란다. 거래량(풀 대비 배수)별로 sqrt(k) 성장을 센다.
print("\n수수료 적립: 크기 10 의 스왑을 X→Y, Y→X 번갈아 N 번 (비율은 거의 제자리, k 는 자란다)")
print(f"{'왕복 거래량/풀':>12} {'sqrt(k) 성장(=지분당 수수료 적립)':>30} {'V_pool = V_hold 가 되는 r(가정 하의 식 값)':>30}")
for n_pairs in [10, 100, 1000]:
    p = ConstantProductPool(1000, 1000)
    k0 = p.k
    for _ in range(n_pairs):
        p.swap_x_for_y(10); p.swap_y_for_x(10)
    growth = math.sqrt(p.k / k0) - 1                            # g = √(k2/k1) − 1. v2 식 (4) 의 f = 1 − √k1/√k2 와는 f = g/(1+g) 관계(기준 시점이 다르다)
    # 수수료 성장 g 가 비영구 손실을 정확히 상쇄하는 r: (1+g)·2√r/(1+r) = 1 → 2차방정식의 두 근
    a = 1 + growth
    disc = (2 * a) ** 2 - 4
    s1, s2 = (2 * a - math.sqrt(disc)) / 2, (2 * a + math.sqrt(disc)) / 2   # √r 의 두 근
    print(f"{2 * n_pairs * 10 / 1000:>11.1f}x {growth:>30.4%} {f'{s1**2:.3f} ≤ r ≤ {s2**2:.3f}':>30}")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
가격 배율 r 과 비영구 손실(수수료 0, 차익거래로 비율 완전 조정 가정):
     r      보유 가치(Y 단위)       풀 가치(Y 단위)     비영구 손실
  0.25           1250.0           1000.0    -20.00%
  0.50           1500.0           1414.2     -5.72%
  0.80           1800.0           1788.9     -0.62%
  1.00           2000.0           2000.0      0.00%
  1.25           2250.0           2236.1     -0.62%
  2.00           3000.0           2828.4     -5.72%
  4.00           5000.0           4000.0    -20.00%

수수료 적립: 크기 10 의 스왑을 X→Y, Y→X 번갈아 N 번 (비율은 거의 제자리, k 는 자란다)
    왕복 거래량/풀         sqrt(k) 성장(=지분당 수수료 적립) V_pool = V_hold 가 되는 r(가정 하의 식 값)
        0.2x                        0.0299%              0.952 ≤ r ≤ 1.050
        2.0x                        0.2985%              0.857 ≤ r ≤ 1.167
       20.0x                        2.9853%              0.614 ≤ r ≤ 1.628
```

[중급] 위 표가 이 장의 저울입니다. 왼쪽 표는 가격이 어느 쪽으로든 움직이면 공급자가 보유보다 못해진다는 것을, 오른쪽 표는 거래량이 풀 크기의 몇 배가 쌓이면 수수료가 어느 범위의 가격 변동까지 메우는지를 보입니다. 두 표의 수치는 전부 **풀 단위이고** 가정(수수료 0.3%, 차익거래 완전 조정, 왕복 거래)이 붙어 있습니다. 실제 풀에서 어느 쪽이 큰지는 그 풀의 거래량과 그 기간의 가격 변동으로 정해지는 실측의 문제이고, 이 책은 그 수치를 적지 않습니다. 「유동성 공급은 수익이 난다」도 「손실이 난다」도 이 장의 결론이 아닙니다. 결론은 **두 항이 한 식에 곱으로 들어 있다는** 것입니다.

7. 정리

AMM은 호가창과 운영자를 없애고 「곱은 줄지 않는다」는 규칙 하나로 거래를 받습니다. 가격은 두 수량의 비율이고, 큰 주문은 비율을 밀어 슬리피지를 낳으며, 수수료는 통 안에 남아 k를 키웁니다. 공급자는 그 성장을 지분으로 나눠 갖되 가격이 움직인 만큼 보유보다 못해집니다. 비율을 시장과 맞추는 것은 차익거래자이고, 그래서 현물 비율은 오라클로 쓰기에 위험합니다. 다음 16장은 「체인은 바깥을 모른다」는 더 근본적인 문제를, 17장은 이 장의 풀 클래스를 그대로 가져가 「누가 거래 순서를 정하는가」를 봅니다.

이 장의 출처

- Hayden Adams, Noah Zinsmeister, Dan Robinson, "Uniswap v2 Core," March 2020 (기술 백서, 10쪽). https://uniswap.org/whitepaper.pdf — 상수곱 불변량과 수수료(§1, 1쪽), 비영구 손실 용어(§2.1, 2쪽), 한계가격·오라클 조작·시간 가중 평균(§2.2, 2–3쪽), 수수료 적립 식 (4)(§2.4, 5쪽), 수수료 조정 식 (9)~(11)(§3.2.1, 7쪽), 지분 식 (12)·(13)(§3.4, 8쪽).
- Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, Dan Robinson, "Uniswap v3 Core," March 2021 (9쪽). https://uniswap.org/whitepaper-v3.pdf — 집중 유동성·가상 준비금·식 (2.2)·구간 이탈(§2, 2쪽), 수수료 단계(§1·§3.1), 거버넌스(§4, 3쪽).
- Philip Daian 외, "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability," *2020 IEEE Symposium on Security and Privacy*, pp. 910–927, DOI 10.1109/SP40000.2020.00040 — 호가창 거래소·DEX·AMM의 서술(§II-B·C, 912쪽), 「탈중앙 거래소는 잘못된 이름」(각주 1), 체인 위 거래의 느림(§I).
- 비영구 손실 식 $2\sqrt{r}\,/\,(1+r) - 1$과 집중 유동성 수치 예는 위 원전의 식에서 이 책이 유도·계산한 것이다.
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `math`만 사용. 노트북 NB15. 위젯 W7 `amm-slippage`의 처음 상태는 NB15-1의 풀 1,000:1,000·주문 100이다.

각주

1. Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, Ari Juels, "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability," *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 910–927, §II-B(912쪽). 원문: "Classic exchanges for trading stocks, commodities, and even cryptocurrencies generally share an accepted and common exchange design known as a continuous-limit orderbook. Such an orderbook consists of a list of all open offers from buyers and sellers in the system. Prospective buyers place a limit buy order, which specifies a maximum price at which they are willing to buy an asset; sellers correspondingly place limit sell orders. A centralized counterparty, the exchange operator, matches buyers and sellers, completing transactions automatically when there is a sell order on the books with a lower price than a buy order on the books."

2. 같은 글, §I(910쪽). 원문: "Despite their clear benefits, however, many DEXes come with a serious and fundamental weakness: on-chain, smart-contract-mediated trades are slow. Traders thus may attempt to take orders that have already been taken or canceled but appear active due to their views of messages sent on the network."

3. 같은 글, §II-C(912쪽) 및 각주 1(910쪽). 원문: "Order books are typically maintained off-chain."; "'Decentralized' exchange is something of a misnomer, as many such systems have centralized components; most systems we

call 'decentralized' exchanges could more accurately be classified as non-custodial: users trade without surrendering control of their funds to a third party in the process."

4. 같은 글, §II-C(912쪽). 원문: "A more radical DEX design, called an automated market maker [3], bypasses order books altogether. The DEX consists of a smart contract that itself holds a reserve of tokens and/or Ether. Consider two assets A and B. The contract allows a user to trade between A and B at any time using its reserves as a counterparty, at a set rate. If a user buys A using B, the price of A denominated in B offered by the smart contract for the next trade is increased. If a user instead sells A for B, the price decreases. In this way, single parties can trade without counterparty discovery or matching. Consequently, constant arbitrage between these and other exchanges is required to keep the rate offered in lockstep with the market rate for a commodity."

5. Hayden Adams, Noah Zinsmeister, Dan Robinson, "Uniswap v2 Core," March 2020, §1 Introduction(1쪽). 원문: "Uniswap v1 is an on-chain system of smart contracts on the Ethereum blockchain, implementing an automated liquidity protocol based on a 'constant product formula' [1]. Each Uniswap v1 pair stores pooled reserves of two assets, and provides liquidity for those two assets, maintaining the invariant that the product of the reserves cannot decrease. Traders pay a 30-basis-point fee on trades, which goes to liquidity providers. The contracts are non-upgradeable." 이 책은 유니스왑을 상수곱 AMM의 **문헌 출처로** 인용하며, 어떤 거래 서비스를 권하거나 비교하지 않습니다(3장 일러두기).

6. 같은 글, §1(1쪽). 원문: "While the contract is not generally upgradeable, there is a private key that has the ability to update a variable on the factory contract to turn on an on-chain 5-basis-point fee on trades. This fee will initially be turned off, but could be turned on in the future, after which liquidity providers would earn 25 basis points on every trade, rather than 30 basis points." v3 백서(2021) §4 Governance는 이 권한을 「팩토리에는 소유자가 있고, 처음에는 UNI 토큰 보유자가 지배한다. 소유자는 핵심 계약의 작동을 멈출 능력은 없다」고 적습니다 (`adams-2021-uniswap-v3-core.pdf`, 3쪽).

7. Adams 외 (2020), §2.2 Price oracle(2쪽) 식 (1). 원문: "The marginal price offered by Uniswap (not including fees) at time t can be computed by dividing the reserves of asset a by the reserves of asset b. pt = r^a_t / r^b_t (1)".

8. Adams 외 (2020), §3.2.1 Adjustment for fee(7쪽) 식 (9)~(11). 원문: "Uniswap v1's trading fee is applied by reducing the amount paid into the contract by 0.3% before enforcing the constant-product invariant. The contract implicitly enforces the following formula: (x1 − 0.003 · xin)) · y1 >= x0 · y0 (9) With flash swaps, Uniswap v2 introduces the possibility that xin and yin might both be non-zero ... the contract is written to enforce the following invariant: (x1 − 0.003 · xin) · (y1 − 0.003 · yin) >= x0 · y0 (10)". (원문 식 (9)의 괄호 불일치는 원문 그대로.) 5쪽 §2.3(플래시 스왑)의 각주 5는 수수료가 넣는 양에 매겨지므로 「꺼내는 양 기준으로는 약 0.3009%」라고 덧붙입니다.

9. Adams 외 (2020), §2.1 ERC-20 pairs(2쪽). 원문: "All liquidity providers have exposure to ETH, and suffer impermanent loss based on changes in the prices of other assets relative to ETH. When two assets ABC and XYZ are correlated—for example, if they are both USD stablecoins—liquidity providers on a Uniswap pair ABC/XYZ would generally be subject to less impermanent loss than the ABC/ETH or XYZ/ETH pairs."

10. Adams 외 (2020), §3.4 Initialization of liquidity token supply(8쪽) 식 (12)·(13). 원문: "s_minted = (x_deposited / x_starting) · s_starting (12)"; "Uniswap v2 initially mints shares equal to the geometric mean of the amounts deposited: s_minted = √(x_deposited · y_deposited) (13) This formula ensures that the value of a liquidity pool share at any time is essentially independent of the ratio at which liquidity was initially deposited."

11. 같은 글, §2.4 Protocol fee(5쪽) 식 (4). 원문: "The total collected fees can be computed by measuring the growth in √k (that is, √(x · y)) since the last time fees were collected. This formula gives you the accumulated fees between t1 and t2 as a percentage of the liquidity in the pool at t2: f1,2 = 1 − √k1/√k2 (4)".

12. Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, Dan Robinson, "Uniswap v3 Core," March 2021, §2 Concentrated Liquidity(2쪽). 원문: "In earlier versions, liquidity was distributed uniformly along the x · y = k reserves curve ... earlier versions were designed to provide liquidity across the entire price range (0, ∞). This is simple to implement and allows liquidity to be efficiently aggregated, but means that much of the assets held in a pool are never touched."

13. 같은 글, §2(2쪽) 식 (2.2). 원문: "A position only needs to maintain enough reserves to support trading within its

range, and therefore can act like a constant product pool with larger reserves (we call these the virtual reserves) within that range." "The amount of liquidity provided can be measured by the value L, which is equal to √k. The real reserves of a position are described by the curve: $(x + L/\sqrt{p_b})(y + L\sqrt{p_a}) = L^2$ (2.2)".

14. 같은 글, §2(2쪽). 원문: "When the price exits a position's range, the position's liquidity is no longer active, and no longer earns fees. At that point, its liquidity is composed entirely of a single asset, because the reserves of the other asset must have been entirely depleted. If the price ever reenters the range, the liquidity becomes active again."

15. 같은 글, §1·§3.1(1·3쪽). 원문: "The swap fee is no longer locked at 0.30%. Rather, the fee tier for each pool (of which there can be multiple per asset pair) is set on initialization (Section 3.1). The initially supported fee tiers are 0.05%, 0.30%, and 1%."

16. Adams 외 (2020), §2.2(2쪽). 원문: "However, Uniswap v1 is not safe to use as an on-chain price oracle, because it is very easy to manipulate. Suppose some other contract uses the current ETH-DAI price to settle a derivative. An attacker who wishes to manipulate the measured price can buy ETH from the ETH-DAI pair, trigger settlement on the derivative contract (causing it to settle based on the inflated price), and then sell ETH back to the pair to trade it back to the true price. This might even be done as an atomic transaction, or by a miner who controls the ordering of transactions within a block." 「ETH-DAI」는 원문의 예시 표기이며 이 책은 특정 토큰을 평가하지 않습니다.

17. 같은 쪽 각주 1. 원문: "For a real-world example of how using Uniswap v1 as an oracle can make a contract vulnerable to such an attack, see [3]." 참고문헌 [3]은 samczsun, "Taking undercollateralized loans for fun and for profit," Sept. 2019(블로그 글). 이 책은 이 글을 원자료로 확보하지 않았고 내용을 인용하지 않습니다.

18. 같은 글, §2.2(3쪽) 식 (2)·(3). 원문: "Uniswap v2 accumulates this price, by keeping track of the cumulative sum of prices at the beginning of each block in which someone interacts with the contract. Each price is weighted by the amount of time that has passed since the last block in which it was updated ..."; "Users of the oracle can choose when to start and end this period. Choosing a longer period makes it more expensive for an attacker to manipulate the TWAP, although it results in a less up-to-date price."; "A miner (or an attacker who uses enough gas to fill an entire block) could manipulate the price at the end of a block, but unless they mine the next block as well, they may not have a particular advantage in arbitraging the trade back."

2부 스마트계약과 응용

16장 오라클 문제 — 체인 밖 사실을 체인 안으로

핵심 TAKEAWAY

블록체인은 자기 안의 일만 압니다. 어느 주소에 얼마가 있는지, 어느 계약의 저장소 칸에 무엇이 적혀 있는지는 모든 노드가 똑같이 계산해 낼 수 있습니다. 그러나 「오늘 뉴욕의 기온」, 「어제 환율」, 「그 비행기가 연착했는가」는 체인 안에 없습니다. 그것을 체인 안으로 들여오는 장치를 **오라클(oracle)이라** 부르고, 오라클은 결국 **누군가가 적어 넣는 것입니다.** 그 누군가를 하나로 둘지, 여럿으로 나눌지, 틀리면 보증금을 잃게 할지는 설계의 문제이고, 어느 설계도 「믿어야 하는 사람」을 0으로 만들지는 못합니다. 이 장을 읽으면 「코드가 법」이라는 구호가 체인 안에서만 성립하는 이유와, 오라클이 틀린 값을 넣었을 때 계약이 **정확히 실행되면서 동시에 틀릴 수 있다는** 역설을 이해합니다.

학습목표

1. 블록체인이 바깥 사실을 스스로 가져올 수 없는 이유를 결정론(모든 노드가 같은 계산을 한다)으로 설명한다.
2. 오라클 설계의 세 갈래 — 단일 제공자, 다수 제공자의 집계, 경제적 담보 — 와 각각이 **무엇을 믿어야 하는지를** 말한다.
3. [중급] 「이상적 오라클」의 정의에서 출발해, 데이터 소스 오류·오라클 담합·시빌(Sybil)과 미러링·보고의 선행매매가 각각 어느 층의 실패인지 가른다.
4. AMM의 현물 비율을 오라클로 쓰는 것이 왜 조작되는지를 15장과 연결해 설명한다.

[입문]**만 읽은 독자가 가져가는 것.** 「코드가 법」이라는 구호는 체인 안에서만 성립하고, 체인 밖 사실은 결국 **누군가를 믿어야** 들어옵니다.

1. 이 기술은 무엇의 실패에서 나왔나

13장의 계약은 조건이 맞으면 값을 풀어 주는 상자였습니다. 그런데 사람들이 짜고 싶은 조건은 대개 바깥 세상에 있습니다. 부테린의 2014년 백서는 「금융 파생상품은 「스마트계약」의 가장 흔한 응용이자 코드로 구현하기 가장 쉬운 것 중 하나」라면서 곧바로 걸림돌을 적습니다. 「금융 계약을 구현할 때 주된 난제는 **대다수가 외부 가격 정보를 참조해야 한다는** 것이다. … 가장 단순한 방법은 특정 당사자(예: 나스닥)가 유지하는 「데이터 피드」 계약을 두어, 그 당사자가 필요할 때 계약을 갱신할 수 있게 하고, 다른 계약이 그 계약에 메시지를 보내 가격이 담긴 응답을 받을 수 있는 인터페이스를 제공하는 것이다.」[1] 신뢰할 제3자를 없애려고 만든 체계의 첫 응용이, **특정 당사자가 유지하는 계약을** 전제합니다. 이 긴장이 이 장의 전부입니다.

황서도 같은 패턴을 「여러 계약 공학 패턴 가운데 간단히 다루는 둘」(데이터 피드와 난수) 중 하나로 적어 두었습니다. 「데이터 피드 계약은 단일 서비스를 제공한다. 이더리움 안에서 외부 세계의 정보에 접근할 수 있게 한다. **이 정보의 정확성과 시의성은 보장되지 않으며**, 어느 한 데이터 피드에 얼마만큼의 신뢰를 둘 수 있는지 판단하는 것은 그 데이터 피드를 쓰는 계약 작성자의 일이다.」 패턴의 뒷부분은 「외부 서버가 이더리움 노드를 돌리면서, 새 블록을 발견하는 즉시 그 값을 갱신하는 유효한 거래를 만들어 계약에 보내는 것이고, 계약의 코드는 그 서버에 담긴 신원에서 온 갱신만 받아들인다.」[2] 요약하면 **서버 하나, 열쇠 하나, 그 열쇠가 적는 숫자 하나입니다**. 「탈중앙」이라는 말이 이 지점에서 가장 얇아집니다.

2. 왜 체인은 바깥을 모르는가

이유는 체인의 장점 그 자체에 있습니다. 13장에서 모든 노드가 같은 거래를 같은 순서로 실행해 같은 상태에 이른다고 했습니다. 그 「같음」이 합의의 조건입니다. 어떤 계약이 실행 중에 「지금 바깥 서버에서 기온을 읽어 오라」고 할 수 있다면, 서울의 노드와 상파울루의 노드가 다른 시각에 다른 값을 읽어 **다른 상태에** 이르고, 블록 검증은 노드마다 갈립니다. 그래서 계약 코드는 체인 안의 것만 읽을 수 있고, 바깥 값은 **누군가가 거래로 적어 넣은 뒤에야** 체인 안의 것이 됩니다. 이것은 이 책의 풀이이고, 2017년 체인링크(Chainlink) 백서는 같은 것을 결론으로 적습니다. 「기반이 되는 합의 프로토콜 때문에, 스마트계약이 돌아가는 블록체인은 외부 시스템과의 원래 그대로의(native) 통신을 지원할 수 없다. 오늘날 이 문제의 해법은 오라클이라는 새 기능을 도입해 바깥 세상과의 연결성을 제공하는 것이다. **기존 오라클은 중앙화된 서비스다.** 그런 서비스를 쓰는 어떤 스마트계약도 단일 실패점을 가지며, 전통적인 중앙 운영 디지털 합의보다 더 안전하지 않다.」[3]

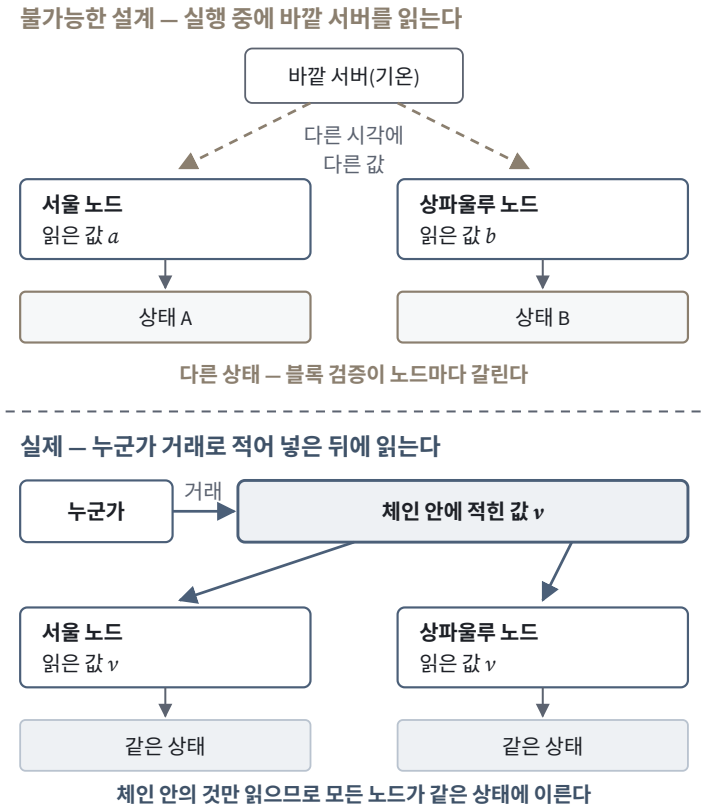


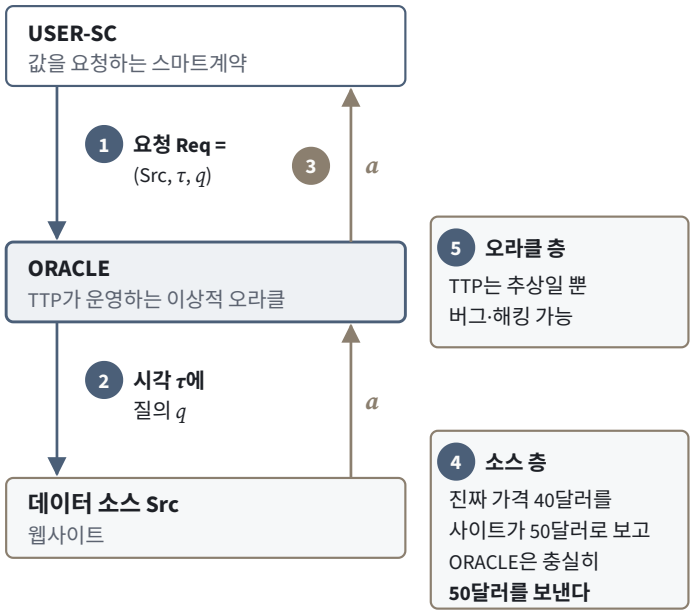
그림 16-1. 체인이 바깥을 모르는 이유. 계약이 실행 중에 바깥 서버를 읽으면 노드마다 상태가 갈리므로, 바깥 값은 누군가 거래로 적어 넣은 뒤에야 체인 안의 것이 된다.

같은 백서가 든 예가 법률가에게 익숙할 것입니다. 「증권 스마트계약(채권, 금리 파생상품 등)은 시장 가격과 참조 데이터를 보고하는 API에 접근해야 한다. 보험 스마트계약은 보험 사고에 관한 IoT 데이터 피드가 필요하다. 창고의 자기 문이 침입 시점에 잠겨 있었는가, 회사의 방화벽이 켜져 있었는가, 당신이 보험을 든 항공편이 정시에 도착했는가. 무역금융 스마트계약은 선적의 GPS 데이터, 공급망 ERP 데이터, 선적 물품의 통관 데이터가 필요하다.」[4] 보험금 지급 여부, 대금 지급 조건, 담보 처분 시점 — 계약법이 다루는 조건의 거의 전부가 체인 밖에 있습니다. 코드가 자동으로 집행하는 것은 「조건이 참이면 지급한다」까지이고, 「조건이 참인가」는 바깥에서 들어옵니다.

3. 이상적인 오라클, 그리고 그것이 불가능한 이유

[중급] 체인링크 백서 3절은 오라클의 「보안」을 정의하기 위해 사고실험을 하나 둡니다. 「지시를 글자 그대로 충실히 수행하는 이상적 존재인 신뢰할 제3자(TTP)가 오라클을 운영한다고 상상하자. … 그리고 그 TTP가 완벽히 신뢰할 수 있는 데이터 소스 Src에서 데이터를 얻는다고 하자.」 그 이상적 오라클 ORACLE에게 시킬 일은 셋뿐입니다. 「1. 요청 수락: 스마트계약 USER-SC에서 목표 데이터 소스 Src, 시각 또는 시각 범위 τ, 질의 q를 지정한 요청 Req = (Src, τ, q)를 받는다. 2. 데이터 획득: 시각 τ에 Src에 질의 q를 보낸다. 3. 데이터 반환: 답 a를 받으면 스마트계약에 a를 돌려준다.」 이것이 **무결성(integrity)입니다**. 「τ가 오후 4시, q가 「INTC 종목의 가격」이면 ORACLE의 무결성은 그 사이트에서 오후 4시에 질의한 INTC 가격을 정확히 USER-SC에 제공한다는 것을 보장한다.」[5]

[중급] 그리고 백서는 이 정의가 왜 현실에서 닿지 않는지를 두 층으로 적습니다. 첫째 층은 **데이터 소스입니다**. 「완벽히 신뢰할 수 있는 데이터 소스 Src는 물론 없다. 데이터는 결함 있는 웹사이트, 속이는 서비스 제공자, 정직한 실수 때문에 선의로든 악의로든 오염될 수 있다. Src가 신뢰할 수 없다면, ORACLE이 위 지시대로 TTP처럼 정확히 작동하더라도 우리가 원하는 보안 개념을 온전히 충족하지 못한다. … **인텔의 진짜 가격이 40달러인데 사이트가 50달러로 잘못 보고하면 ORACLE은 틀린 값 50달러를 USER-SC에 보낸다.** 단일 소스를 쓰는 한 이 문제는 피할 수 없다.」 둘째 층은 **오라클 자체입니다**. 「더 큰 문제는 우리의 TTP ORACLE이 추상에 그친다는 것이다. 어떤 서비스 제공자도 무조건 신뢰할 수 있지는 않다. 가장 선의인 자도 버그가 있거나 해킹될 수 있다.」[6] 오라클이 **지시대로 정확히 작동하면서도 틀릴 수 있다는** 것, 이것이 이 장의 질문 상자가 서는 자리입니다.



40·50달러는 백서의 설명용 가상 예시이며 시세 자료가 아니다

그림 16-2. 체인링크 백서가 정의한 이상적 오라클의 세 단계와, 그 정의가 현실에서 닿지 않는 두 층(데이터 소스, 오라클 자체). 40·50달러는 백서의 설명용 가상 예시다.

4. 설계의 세 갈래 — 무엇을 믿어야 하는가

오라클 설계는 「믿어야 하는 것」을 어디에 두느냐로 갈립니다. 표는 이 책의 정리이고, 각 칸의 근거는 아래 본문에 있습니다.

설계	무엇이 값을 적는가	믿어야 하는 것	실패 모양
단일 제공자(데이터 피드 계약)	서버 하나의 열쇠	그 운영자와 그 소스	운영자 실수·해킹·소스 오류가 그대로 체인에
다수 제공자의 집계	노드 n 개의 답을 다수결·중위값 등으로 모은 값	「정직한 노드가 충분히 많다」, 「소스들이 서로 독립이다」	소스가 서로 베끼면 다수결이 무의미, 한 주체가 여러 노드를 가장(시빌)
경제적 담보	위와 같되 노드가 보증금을 걸고 틀리면 잃음	「벌금이 조작 이익보다 크다」	조작으로 얻을 것이 보증금보다 큰 거래에서는 담보가 억제력이 아님
체인 안의 비율을 그대로 읽기(15장)	AMM 풀의 두 수량	「아무도 그 순간 비율을 밀지 않았다」	한 거래 안에서 밀기·읽히기·되돌리기

[중급] 소스 분산. 「결함 있는 단일 소스 Src를 다루는 단순한 방법은 여러 소스에서 데이터를 얻는 것, 곧 데이터 소스를 분산하는 것이다. … 예컨대 다수결. 과반의 소스가 같은 값 a를 돌려주면 agg는 a를 돌려주고, 아니면 오류를 돌려준다. 이 경우 과반(> k/2)의 소스가 정상으로 작동하는 한 ORACLE은 언제나 옳은 값 A를 돌려준다. … 또는 이상값(가장 큰 값과 가장 작은 값)을 버리고 나머지의 평균을 출력할 수도 있다.」 그러나 백서는 곧 한계를 붙입니다. 「물론 데이터 소스 사이의 결함이 **상관되어** 집계가 주는 보증을 약화시킬 수 있다. 사이트 Src1 = EchoEcho.com이 Src2 = TheHorsesMouth.com에서 데이터를 가져온다면, Src2의 오류는 언제나 Src1의 오류를 뜻한다.」[7] 다수결은 투표자들이 **서로 독립일 때만** 의미가 있습니다. 같은 통신사 기사를 받아 쓰는 신문 열 개가 같은

오보를 내는 것과 같습니다.

[중급] **오라클 분산.** 소스뿐 아니라 오라클 노드도 여럿 두고 「계약 안에서 집계」합니다. 「CHAINLINK-SC(체인 위 부분) 자체가 오라클 응답들을 집계한다. … A = Agg(A1, A2, …, An)을 계산해 결과 A를 USER-SC에 보낸다.」 장점으로 「개념적 단순성」, 「CHAINLINK-SC의 코드는 공개 검사할 수 있으므로 정확한 동작을 검증할 수 있다」는 **신뢰 가능성**, 유연성을 듭니다.[8] 집계 코드가 체인 위에 있으면 **집계 방식은** 믿지 않아도 됩니다. 믿어야 하는 것은 「노드 n 개 중 정직한 쪽이 과반」이라는 가정으로 옮겨 갑니다. 2021년의 후속 백서는 이것을 비잔틴 장애 허용(BFT)의 언어로 다시 적습니다. 「대부분의 BFT 알고리듬은 최소 $n = 3f + 1$ 개의 노드를 요구하며, f 는 결함 가능한 노드의 수이다. … 우리는 위원회 O가 이 요건을 충족하면, 곧 정직한 노드가 3분의 2를 넘으면 정직하다고 부른다.」[9] 10장의 「3분의 2」가 여기서 다시 나옵니다. 다만 2021년 백서는 두 체계의 보안이 **같은 자로 잴 수 없다고** 덧붙입니다. 「작업증명이나 지분증명 블록체인을 51% 공격으로 무너뜨리는 데는 공격자가 일시적으로, 어쩌면 익명으로 다수 자원을 얻어야 한다. … 반면 위원회 기반 체계를 무너뜨리는 것은 임계수(보통 3분의 1)의 노드를 부패시키는 것인데, 그 노드들은 공개적으로 알려진, 자원이 풍부한, 신뢰받는 주체일 수 있다.」[10] 어느 쪽이 더 안전한지는 **누가 공격자인가에** 달려 있습니다.

[중급] **경제적 담보와 평판.** 2017년 백서의 주문 매칭 설계에서 「오라클 서비스 제공자가 계약에 입찰할 때, 그는 SLA에 정해진 대로 **자기 위법행위 때문에 잃게 될 벌금 액수를 붙여** 그 계약에 구속된다.」[11] 그리고 평판 체계가 「배정된 요청 수, 완료한 요청 수, 동료 응답과 비교해 수용된 요청 수, 평균 응답 시간, 벌금 지불액」을 기록합니다.[12] 10장의 슬래싱과 같은 논리입니다. 틀리면 잃을 것이 있어야 정직이 합리적입니다. 그런데 백서 스스로 이 장치가 못 막는 것을 적어 두었습니다. **시빌 공격과 미러링입니다.** 「우리의 단순 집계와 계약 내 집계 프로토콜은 부정직한 노드가 정직한 노드의 답을 베끼는 무임승차를 막으려 한다. 그러나 어느 쪽도 시빌 공격을 막지 못한다. 그런 공격은 외견상 독립인 여러 오라클을 하나의 공격자가 지배하는 것이다. 이 공격자는 오라클 풀을 장악해 f 개보다 많은 오라클이 집계에 참여해 전략적 시점에, 예컨대 고가치 계약의 큰 거래에 영향을 주기 위해 거짓 데이터를 내도록 할 수 있다.」 그리고 「f 개를 넘는 오라클이 관여한 공격이나 결함은 **체인 위 행동만으로는 탐지할 수 없다는** 점에서 특히 치명적이다.」[13] 담보는 노드 하나의 유인을 바꿉니다. 노드 여럿이 사실은 하나라는 것은 담보가 알아내지 못합니다.

▾ [심화] **넷째 길 — 신뢰할 하드웨어, 그리고 그것이 옮기는 것**

[심화] 2017년 백서가 「장기 전략」으로 든 것은 분산이 아니라 **하드웨어입니다.** 「분산 오라클 네트워크는 결함 있는 오라클에 대해 높은 수준의 보호를 제공하려 한다. 대부분의 배치 시나리오에서 그것은 f 개의 비잔틴 결함에 맞서 올바른 응답을 얻으려 한다(단순 집계 프로토콜에서 $f < n / 2$). 신뢰할 하드웨어는 훨씬 더 많은 것을 줄 수 있다.」 인텔 SGX 같은 신뢰 실행 환경(enclave)에서 오라클을 돌리고 그 **증명(attestation)을** 배포하면 「오라클이 특정 응용을 실행하고 있다는 매우 강한 보증」을 얻고, 「HTTPS로 데이터 소스에 접속하는 엔클레이브 안의 오라클은 가져온 데이터가 조작되지 않았다는 강한 보증을 제공할 수 있다.」[14] 그리고 기밀성 — 분산 오라클은 질의를 공개 체인에 적으므로 「어느 시점에 어떤 서버는 질의를 데이터 소스 서버에 보내야 하고, 그래서 그 질의를 보아야 한다」는 한계를 엔클레이브 안에서 복호화해 처리함으로써 넘습니다. 백서는 부록 B에서 「인텔을 신뢰의 뿌리로 삼는 문제」를 따로 논합니다.[15] 신뢰가 사라진 것이 아니라 **칩 제조사와 그 증명 체계로 옮겨 간** 것입니다. 이 장의 표에 넷째 행을 더한다면 「믿어야 하는 것」 칸에는 「하드웨어 제조사의 증명과 엔클레이브 구현의

결함 없음」이 들어갑니다. 다른 길도 적혀 있습니다. 「데이터 소스가 자기가 내보내는 데이터에 디지털 서명을 한다면 오라클이 데이터를 조작하지 않을 것이라고 믿을 필요가 없어진다」는 인프라 변경(TLS-N)인데, 백서는 「데이터 소스들이 가까운 미래에 그것을 배치할 가능성은 낮다」고 적습니다.[16] 가장 깨끗한 해법은 **바깥 세상이 바뀌어야** 하는 해법입니다.

5. 조작은 어느 층에서 일어나는가

「오라클 조작」이라는 말은 여러 층을 한데 묶습니다. 이 책은 넷으로 가릅니다.

1. **소스의 오류·조작.** 3절의 「40달러를 50달러로 보고하는 사이트」. 오라클은 충실했고 소스가 틀렸습니다.
2. **오라클 노드의 담합·시빌.** 4절의 「f 를 넘는 노드가 전략적 시점에 거짓을 낸다」. 집계 코드는 충실했고 입력이 틀렸습니다.
3. **체인 안의 비율을 오라클로 읽은 것.** 15장 5절의 공격입니다. 유니스왑 v2 백서가 적은 대로 「공격자는 ETH를 사서 측정 가격을 밀어 올리고, 파생상품의 정산을 촉발하고, 되팔아 진짜 가격으로 되돌릴 수 있다. 이것은 원자적 거래 하나로, 또는 블록 안의 거래 순서를 지배하는 채굴자에 의해 이루어질 수 있다.」[17] 여기서는 **오라클이 따로 없습니다.** 어떤 계약이 다른 계약의 저장소 두 칸(두 준비금)을 「가격」으로 읽었을 뿐이고, 그 두 칸은 누구나 거래로 밀 수 있는 값이었습니다. 백서의 대응(시간 가중 평균)은 「조작 비용을 올리되 신선도를 잃는」 맞바꿈이었습니다.
4. **보고 자체의 선행매매.** 오라클이 정직하게 새 값을 올리는 거래가 **공개 대기열에** 보이면, 누군가 그보다 먼저 거래를 끼워 넣습니다. 2021년 백서의 서술: 「오라클을 써서 체인 위 거래소에 가격 피드를 공급하는 흔한 설계 패턴을 생각하자. 주기적으로(한 시간마다라고 하자) 오라클이 여러 자산의 가격을 모아 거래소 계약에 보낸다. 이 가격 데이터 거래는 명백한 차익거래 기회를 만든다. 예컨대 최신 보고가 어느 자산의 가격을 훨씬 높게 적었다면, 공격자는 오라클 보고를 **선행매매해** 자산을 사 모은 뒤 보고가 처리되는 즉시 되팔 수 있다.」[18] 오라클도 소스도 집계도 전부 옳았습니다. 틀린 것은 없고, 다만 **순서가** 돈이 되었습니다. 그것이 17장입니다.

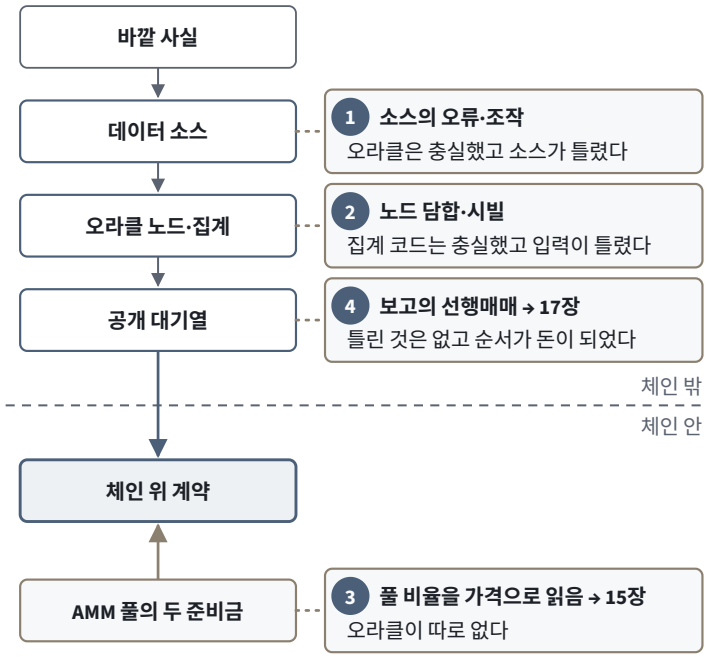


그림 16-3. 바깥 사실이 계약에 닿는 경로 위의 네 실패 지점. 번호는 본문 목록의 번호이고, 셋째는 오라클을 거치지 않는 옆길이다.

이 넷 중 셋째가 「DeFi 사고의 흔한 경로」라는 말을 자주 듣습니다. 이 책은 그 **빈도를 재지 않았습니다.** 유니스왑 v2 백서는 「실제 사례」로 블로그 글 하나를 가리킬 뿐이고(15장 각주), 이 책이 확보한 1차 문서 가운데 사고 유형별 건수나 피해 규모를 집계한 것은 없습니다. 그래서 이 장은 「조작의 **구조가** 이렇다」까지만 적고, 「흔하다」는 판단은 집계 자료가 확보될 때로 미룹니다(부록 A 동향 노트의 과제).

> **? 법률가의 질문**
>
> 오라클이 틀린 값을 넣었고, 계약은 그 값으로 정확히 실행되었습니다. 보험금이 지급되지 않았거나, 담보가 처분되었거나, 파생상품이 엉뚱한 가격으로 정산되었습니다. 코드는 한 줄도 어긋나지 않았습니다. 이것은 **이행입니까, 착오입니까.** 착오라면 누구의 착오인가요 — 값을 적어 넣은 오라클 운영자, 그 오라클을 믿기로 한 계약 작성자, 그 계약을 쓰기로 한 당사자. 틀린 값을 낸 것이 데이터 소스(사이트)였다면, 그리고 그 사이트는 누구와도 계약하지 않았다면 어떻게 됩니까. 다수 노드 중 3분의 1 넘게가 담합했다면 그들은 공동으로 무엇을 한 것입니까. 그리고 계약이 「다른 계약의 저장소 두 칸」을 가격으로 읽었을 뿐 아무도 「오라클」을 운영하지 않았다면, 그 값을 민 사람은 무엇을 조작한 것입니까. → 28장

6. 정리

체인은 모든 노드가 같은 계산을 하기 때문에 바깥을 읽을 수 없고, 바깥 값은 누군가가 거래로 적어 넣어야 들어옵니다. 그 누군가를 하나로 두면 그를 믿어야 하고, 여럿으로 두면 그들이 독립이라는 것을 믿어야 하고, 보증금을 걸게 하면 보증금이 조작 이익보다 크다는 것을 믿어야 하고, 체인 안의 비율을 읽으면 그 순간 아무도 비율을 밀지 않았다는 것을 믿어야 합니다. 「코드가 법」은 체인 안에서 성립하고, 조건이 참인지는 체인 밖에서 들어옵니다. 다음 17장은 이 장 마지막에 나온 「모두 옳았는데 순서가 돈이 되었다」는 문제를 봅니다. 부록 B 오해 ⑨(「코드가 법이다」)의 근거는 이 장과 26장이 나눠 맡습니다.

이 장의 출처

- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014(초고 2013). https://ethereum.org/whitepaper/ (2026-10-25 접근) — 데이터 피드 계약 패턴(「Financial derivatives and Stable-Value Currencies」, PDF 20쪽).
- Gavin Wood, Ethereum Yellow Paper, Shanghai version efc5f9a, 2025-02-04 — 데이터 피드 계약(§13.1, 17쪽).
- Steve Ellis, Ari Juels, Sergey Nazarov, "ChainLink: A Decentralized Oracle Network," 4 September 2017 (v1.0), 38쪽. https://research.chain.link/whitepaper-v1.pdf — 연결성 문제와 중앙화된 오라클(Abstract·§1), 이상적 오라클과 그 불가능(§3, 8–10쪽), 소스·오라클 분산(§4, 11–12쪽), 벌금(§2.1), 평판(§5.2), 시빌·미러링(§5.3), 신뢰할 하드웨어·TLS-N(§6, 21–25쪽). 공식 호스트 외의 독립 서지 레코드(DOI 등)는 없다.
- Lorenz Breidenbach 외 13인, "Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks," 15 April 2021 (v1.0), 136쪽. https://research.chain.link/whitepaper-v2.pdf — BFT 위원회($n = 3f + 1$)(§2.3), 두 신뢰 모형의 비교 불가(§2.4), 오라클 선행매매(§5.1.1).
- Hayden Adams 외, "Uniswap v2 Core," March 2020, §2.2 — AMM 현물가 조작과 시간 가중 평균.
- 이 장에는 코드가 없다(노트북도 없다). 「오라클 조작이 DeFi 사고의 흔한 경로」라는 빈도 주장은 1차 집계 자료가 없어 적지 않았다.

각주

1. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," ethereum.org 공식 PDF(2014; 초고 2013), 「Financial derivatives and Stable-Value Currencies」 절(PDF 20쪽). 원문: "Financial derivatives are the most common application of a 'smart contract', and one of the simplest to implement in code. The main challenge in implementing financial contracts is that the majority of them require reference to an external price ticker; ... The simplest way to do this is through a 'data feed' contract maintained by a specific party (eg. NASDAQ) designed so that that party has the ability to update the contract as needed, and providing an interface that allows other contracts to send a message to that contract and get back a response that provides the price." https://ethereum.org/whitepaper/ (2026-10-25 접근).

2. Gavin Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Shanghai version efc5f9a (2025-02-04), §13.1 Data Feeds(17쪽). 원문: "A data feed contract is one which provides a single service: it gives access to information from the external world within Ethereum. The accuracy and timeliness of this information is not guaranteed and it is the task of a secondary contract author—the contract that utilises the data feed—to determine how much trust can be placed in any single data feed. The general pattern involves a single contract within Ethereum which, when given a message call, replies with some timely information concerning an external phenomenon. ... the second part of the pattern would be for an external server to run an Ethereum node, and immediately on discovery of a new block, creates a new valid transaction, sent to the contract, updating said value in storage. The contract's code would accept such updates only from the identity contained on said server." https://ethereum.github.io/yellowpaper/paper.pdf (2026-10-25 접근).

3. Steve Ellis, Ari Juels, Sergey Nazarov, "ChainLink: A Decentralized Oracle Network," 4 September 2017 (v1.0), Abstract(1쪽). 원문: "Unfortunately, because of their underlying consensus protocols, the blockchains on which smart contracts run cannot support native communication with external systems. Today, the solution to this problem is to introduce a new functionality, called an oracle, that provides connectivity to the outside world. Existing oracles are centralized services. Any smart contract using such services has a single point of failure, making it no more secure than a traditional, centrally run digital agreement." 이 백서는 특정 사업자의 설계 문서이며, 이 책은 그 설계를 권하거나 비교하지 않고 오라클 문제의 **정의와 분류를** 적은 문헌으로만 인용합니다

(3장 일러두기).

4. 같은 글, §1 Introduction(3–4쪽). 원문: "Securities smart contracts such as bonds, interest rate derivatives, and many others will require access to APIs reporting market prices and market reference data, e.g. interest rates. Insurance smart contracts will need data feeds about IoT data related to the insurable event in question, e.g.: was the warehouse's magnetic door locked at the time of breach, was the company's firewall online, or did the flight you had insurance for arrive on time. Trade finance smart contracts will need GPS data about shipments, data from supply chain ERP systems, and customs data about the goods being shipped in order to confirm fulfillment of contractual obligations."

5. Ellis 외 (2017), §3 Oracle Security(8–9쪽). 원문: "Imagine that a trusted third party (TTP)—an ideal entity or functionality that always carries out instructions faithfully to the letter—were tasked with running an oracle. ... 1. Accept request: Ingest from a smart contract USER-SC a request Req = (Src, τ, q) that specifies a target data source Src, a time or range of times τ, and a query q; 2. Obtain data: Send query q to Src at time τ; 3. Return data: On receiving answer a, return a to the smart contract." "For example, if Src is https://www.FountOfKnowledge.com, τ is 4 p.m., and q = 'price for ticker INTC', the integrity of ORACLE guarantees that it will provide USER-SC with exactly the price of INTC as queried at 4 p.m. at https://www.FountOfKnowledge.com."

6. 같은 글, §3(10쪽). 원문: "There is, of course, no perfectly trustworthy data source Src. Data may be benignly or maliciously corrupted due to faulty web sites, cheating service providers, or honest mistakes. If Src isn't trustworthy, then even if ORACLE does operate exactly like a TTP as instructed above, it still doesn't completely meet the notion of security we want. ... If the true price of Intel is $40 and https://www.FountOfKnowledge.com misreports it as $50, for example, then ORACLE will send the incorrect value a = $50 to USER-SC. This problem is unavoidable when using a single source Src. ... A bigger issue, of course, is the fact that our TTP for ORACLE is just an abstraction. No service provider is unconditionally trustworthy. Even the best-intentioned may be buggy or hacked." (원문의 「$40」「$50」은 설명용 가상 예시이며 시세 자료가 아닙니다.)

7. Ellis 외 (2017), §4.1 Distributing sources(11쪽). 원문: "A simple way to deal with a faulty single source Src is to obtain data from multiple sources, i.e., distribute the data source. ... One, for example, is majority voting. If a majority of sources return the identical value a, the function agg returns a; otherwise it returns an error. In this case, provided that a majority (> k/2) sources are functioning correctly, ORACLE will always return a correct value A. ... For example, agg might discard outliers (e.g., the largest and smallest values ai) and output the mean of the remaining ones. Of course, faults may be correlated across data sources in a way that weakens the assurances provided by aggregation. If site Src1 = EchoEcho.com obtains its data from Src2 = TheHorsesMouth.com, an error at Src2 will always imply an error at Src1."

8. 같은 글, §4.2 Distributing oracles(11–13쪽). 원문: "Our initial proposed solution in ChainLink will be a simple one called in-contract aggregation. CHAINLINK-SC—which, again, denotes the on-chain part of ChainLink—will itself aggregate oracle responses. ... In other words, CHAINLINK-SC will compute A = Agg(A1, A2, . . . , An) for some function Agg (similar to agg, as described above), and send the result A to USER-SC. This approach is practical for small n, and has several distinct benefits: Conceptual simplicity ... Trustworthiness: As CHAINLINK-SC's code can be publicly inspected, its correct behavior can be verified. ... Flexibility ...".

9. Lorenz Breidenbach 외 13인, "Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks," 15 April 2021 (v1.0), §2.3 Notation(22쪽). 원문: "Most BFT algorithms require at least n = 3f + 1 nodes, where f is the number of potentially faulty nodes; remaining nodes are honest, in the sense that they follow the protocol exactly as specified. We refer to the committee O as honest if it meets this requirement, i.e., has greater than a 2/3-fraction of honest nodes." 저자 14인 중 학계 소속 7인은 첫 면 각주에서 각자의 자격을 밝힙니다 — Cachin·Koushanfar·Miller·Tramèr 4인은 「자문(advisor) 자격」, Juels는 「Chainlink Labs 수석과학자 자격」, Moroz·Zhang은 「Chainlink Labs 연구원 자격」.

10. 같은 글, §2.4 Note on Trust Models(22쪽). 원문: "It is important to recognize that the security of committee-based vs. permissionless decentralized systems is incommensurable. Compromising a PoW or a PoS blockchain via 51% attack requires that an adversary obtain majority resources ephemerally and potentially anonymously, for example by renting hash power in a PoW system. ... In contrast, compromising a committee-based system means corrupting a threshold number (typically one-third) of its nodes, where the nodes may be publicly known, well resourced, and trustworthy entities."

11. Ellis 외 (2017), §2.1 On-Chain Architecture(6쪽). 원문: "When an oracle service provider bids on a contract, they commit to it, specifically by attaching the penalty amount that would be lost due to their misbehavior, as defined in the SLA."

12. 같은 글, §5.2 Reputation System(18쪽). 원문의 다섯 지표: "Total number of assigned requests ... Total number of completed requests ... Total number of accepted requests ... Average time to respond ... Amount of penalty payments". 마지막 항목의 설명: "If penalty payments were locked in to assure a node operator's performance, the result would be a financial metric of an oracle provider's commitment not to engage in an 'exit scam' attack, where the provider takes users' money and doesn't provide services."

13. 같은 글, §5.3 Certification Service(19쪽). 원문: "Both our simple and in-contract aggregation protocols seek to prevent freeloading in the sense of dishonest nodes copying honest nodes' answers. But neither protects against Sybil attacks [9]. Such attacks involve an adversary that controls multiple, ostensibly independent oracles. This adversary can attempt to dominate the oracle pool, causing more than f oracles to participate in the aggregation protocol and provide false data at strategic times, e.g., in order to influence large transactions in high-value contracts. ... Attacks or faults involving > f oracles are especially pernicious in that they are undetectable from on-chain behavior alone. Additionally, to reduce operational costs, a Sybil attacker can adopt a behavior called mirroring, in which it causes oracles to send individual responses based on data obtained from a single data-source query."

14. Ellis 외 (2017), §6.1 Confidentiality(21–22쪽). 원문: "A distributed oracle network aims to offer a high degree of protection against faulty oracles. In most deployment scenarios, it seeks to attain a correct response in the face of f Byzantine faults (for f < n/2 in our simple aggregation protocol). Trusted hardware can offer much more and is proposed as a better approach to securing the ChainLink network." "Running an oracle in an enclave and distributing attestations can provide very strong assurance that the oracle is executing a particular application ... Additionally, an oracle running in an enclave that can connect to a data source via HTTPS can provide a strong assurance that the data it retrieves has not been tampered with."

15. 같은 글, §6.1(22–23쪽). 원문: "At some point a server needs to send a query to a target data source server. Thus it must see the query, irrespective of whatever confidentiality the query previously enjoyed." "We briefly discuss the issue of Intel as a root of trust in Appendix B."

16. 같은 글, §6.2 Infrastructure changes(25쪽). 원문: "Many of the challenges in constructing secure oracles arise from the fact that existing data sources don't digitally sign the data they serve. If they did, then oracles would not need to be trusted to refrain from tampering with data." "Unfortunately, until and unless TLS-N becomes a standard, data sources must expressly deploy it for clients to benefit. Few data sources are likely to in the near future."

17. Hayden Adams, Noah Zinsmeister, Dan Robinson, "Uniswap v2 Core," March 2020, §2.2(2쪽). 원문은 15장 각주에 전재.

18. Breidenbach 외 (2021), §5.1.1 Oracle Front-Running(45–46쪽). 원문: "Consider the common design pattern of using an oracle to supply various price feeds to an on-chain exchange: periodically (say every hour), the oracle collects price data for different assets and sends these to an exchange contract. These price-data transactions present obvious arbitrage opportunities: For example, if the newest oracle report lists a much higher price for some asset, an adversary could front-run the oracle report to buy up assets and immediately resell them once the oracle's report is processed." 같은 절은 대응책으로 「속도 방지턱」(거래를 몇 블록 대기시키기, 새 보고 도착 시 가격 소급 조정)과 「피기백」(사용자가 서명된 보고를 자기 거래에 붙여 보내기)을 들고, 각각의 한계를 적습니다.

2부 스마트계약과 응용

17장 MEV와 거래 순서 — 순서를 정하는 권력

핵심 TAKEAWAY

8장에서 채굴자가 어느 거래를 블록에 넣을지, 어떤 순서로 넣을지는 규칙이 강제하지 않는다고 했습니다. 15장에서 AMM의 가격은 거래 순서에 따라 바뀐다고 했습니다. 둘을 합치면 이렇게 됩니다. **블록을 만드는 사람은 거래 순서를 정할 수 있고, 순서를 정하는 것만으로 돈을 뽑을 수 있습니다.** 그 돈을 2019년 사전 인쇄본으로 처음 나와 2020년에 게재된 논문이 「채굴자 추출 가능 가치(miner extractable value, MEV)」라 이름 붙였고, 지분증명 전환 뒤에는 「최대 추출 가능 가치(maximum extractable value — ethereum.org PBS 페이지의 표기)」로 읽습니다. 가장 알기 쉬운 모양이 **샌드위치입니다.** 남의 매수 주문 앞에 내 매수를, 뒤에 내 매도를 끼우면 내 거래 두 번에 15장의 식이 적용되어 그 차이가 내게 남습니다. 전통 시장이 선행매매를 법으로 막는다면, 여기서는 그것을 막을 「중개인」이 없고 코드도 막지 않습니다. 이더리움이 택한 대응은 금지가 아니라 **분업입니다.** 블록을 짜는 사람(빌더)과 제안하는 사람(제안자)을 갈라, 순서를 정하는 권력이 소수 검증자에게 몰리지 않게 하려는 것입니다. 이 장을 읽으면 「탈중앙 시장에는 운영자가 없다」는 말 뒤에 「대신 순서를 정하는 사람이 있다」가 붙어야 한다는 것을 압니다.

학습목표

1. 거래가 블록에 들어가기 전 공개 대기열(mempool)에서 보인다는 사실과, 블록 생산자가 순서를 정한다는 사실에서 선행매매·후행매매·샌드위치가 어떻게 나오는지 설명한다.
2. [중급] 15장의 상수곱 식을 두 번 적용해 샌드위치 공격자의 이익과 피해자의 손실을 **풀 단위로** 계산하고, 피해자의 슬리피지 허용치가 공격 크기를 어떻게 제한하는지 본다.
3. [중급] 우선순위 가스 경매(PGA)가 「패자도 내는 경매」임을 읽고, MEV가 합의층 안정성(되감기 공격)에 왜 위협인지 안다.
4. 제안자–빌더 분리(PBS)가 무엇을 바꾸고 무엇을 바꾸지 않는지, 그리고 2026년 10월 기준으로 그것이 프로토콜 안에 있는지 밖에 있는지 안다.

[입문]**만 읽은 독자가 가져가는 것.** 블록체인에도 「순서를 정하는 사람」이 있고, 그 사람은 순서에서 돈을 뽑을 수 있습니다. 전통 시장의 선행매매 금지가 여기서는 **코드로 막히지 않습니다.**

1. 이 기술은 무엇의 실패에서 나왔나

거래는 블록에 들어가기 전에 네트워크를 떠돕니다. 다이안(Philip Daian)과 공저자들의 2020년 논문: 「이더리움 거래는 클라이언트 노드 소프트웨어가 P2P 가십 멀티캐스트 프로토콜로 중계한다. 이는 **모든 거래 정보가 이 네트워크의 모든 참여자에게 보인다는** 뜻이고, 가십 위상에서 유리한 자리에 있는 참여자에게는 더 일찍 보인다. 또 노드들은 현재 또는 예상되는 시스템 상태에서 모든 거래의 결과를 시뮬레이션할 수 있다.」[1] 당신이 AMM에 큰 매수 주문을 보내면, 그것이 블록에 들어가기 전에 다른 사람들이 그 주문을 읽고, 그 주문이 가격을 얼마나 밀 것인지 계산할 수 있습니다. 그리고 그 사람들은 「거래를 지켜본 뒤 더 높은 수수료를 붙여 자기 주문을 내어 먼저 채굴되게」 할 수 있습니다.[2] 이것이 **선행매매(front-running)입니다**.

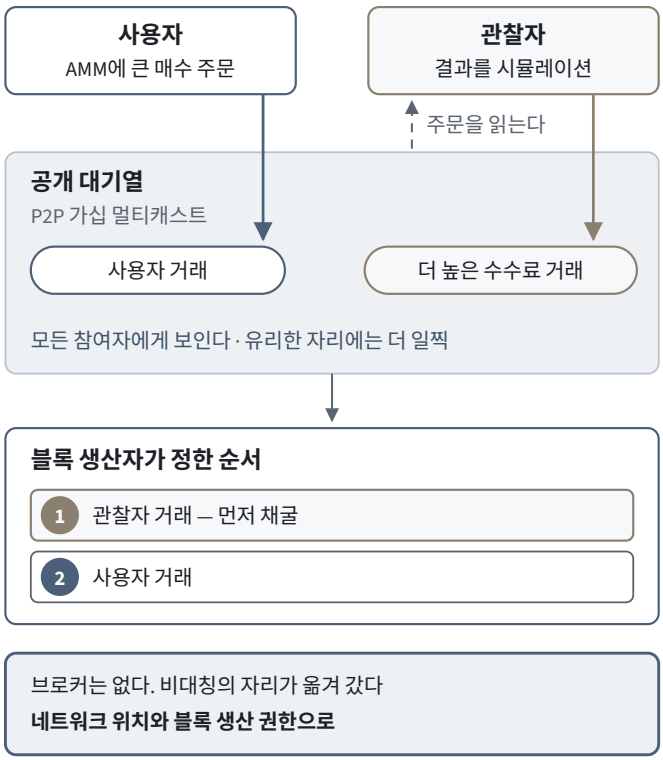


그림 17-1. 공개 대기열의 거래가 모든 참여자에게 보이는 동안 관찰자가 결과를 시뮬레이션하고, 더 높은 수수료를 붙인 자기 거래로 블록 생산자가 정한 순서의 앞자리를 얻는다. 브로커가 없는 곳에서 비대칭의 자리는 네트워크 위치와 블록 생산 권한으로 옮겨 갔다.

전통 시장에도 선행매매가 있고, 논문은 그것을 이렇게 대조합니다. 「전통 거래소는 선행매매라는 고전적 약탈 행위를 겪는다. 규제 시장에서 선행매매는 흔히 불법이며, 기소와 금융기관의 명예 손상을 낳았다. 선행매매는 보통 금융 구조 안의 권력 구조가 만든 정보 비대칭 — 예컨대 브로커가 사용자 정보에 특권적으로 접근하는 것 — 을 이용한다. **탈중앙 체계에는 브로커 역할을 하는 단일 당사자가 없으므로**, 정보 비대칭은 기반 인프라에서 유리한 자리에 있는 행위자에게 생길 수 있다.」[3] 규제가 겨냥하던 「브로커」가 없어졌는데 비대칭은 남았습니다. 비대칭의 자리가 **네트워크 위치와 블록 생산 권한으로** 옮겨 갔습니다.

논문이 짚는 더 깊은 차이는 **원자성입니다**. 「거래의 원자적 묶음 처리 때문에, 그리고 거래가 스마트계약에서 시작될 수 있기 때문에, 대리 계약을 통해 여러 거래소에 걸쳐 거래하는 봇을 만들 수

있다. 이 대리 계약은 한 거래 안에서 주문 묶음을 차례로 실행하고, 묶음의 어느 거래든 실패하면 예외를 던져 앞의 거래들을 되돌린다. 이는 차익거래자가 여러 거래소의 여러 거래를 **전부 아니면 전무의** 실패 모형으로 원자적으로 실행하는 거래 하나를 짤 수 있다는 뜻이다.」[4] 13장의 「가스 소진은 반쯤 된 상태를 남기지 않는다」가 여기서 공격 도구가 됩니다. 두 거래 중 하나만 성립할 위험이 없으니, 차익이 있으면 확정적으로 얻고 없으면 수수료만 잃습니다.

2. 직관 — 앞에 끼우기, 뒤에 끼우기, 양쪽에 끼우기

세 낱말을 정리합니다. 2021년 체인링크 2.0 백서의 정의가 간결합니다. 「선행매매는 거래 순서 공격이라는 부류의 한 예다. 후행매매(back-running)와 **샌드위치(선행매매 + 후행매매)** 같은 관련 공격이 여럿 있다.」[5]

- **선행매매.** 피해자의 주문이 가격을 올릴 것을 알고 **그 앞에** 같은 방향 주문을 넣습니다. 피해자가 가격을 올려 주면 나는 이미 산 것을 들고 있습니다.
- **후행매매.** 피해자의 주문 **뒤에** 반대 방향 주문을 넣습니다. 피해자가 밀어 올린 가격이 되돌아올 것을 아니, 그 되돌림을 내가 가져갑니다.
- **샌드위치.** 둘을 합칩니다. 앞에서 사고, 피해자가 가격을 올리면, 뒤에서 팝니다. 15장의 식이 세 번 적용되고, 그중 두 번이 내 것입니다.



그림 17-2. 선행매매·후행매매·샌드위치를 블록 안 자리로 견주었다. 피해자 주문은 세 경우 모두 서명한 그대로이고, 다른 것은 공격자 주문이 그 앞에 오는지 뒤에 오는지다.

샌드위치를 15장의 통으로 그리면 이렇습니다. 통에 X와 Y가 1,000개씩 있고, 피해자가 Y 100개로 X를 사려 합니다. 혼자였다면 15장의 식대로 X 약 90.7개를 받습니다. 공격자가 먼저 Y 100개로 X를 사면 통은 Y 1,100 / X 909.3이 되고 X 하나의 가격은 이미 올라 있습니다. 피해자는 이제 같은 Y 100개로 X 약 75.6개밖에 못 받습니다. 그 뒤 공격자가 자기 X 90.7개를 되팔면 Y를 100개보다 많이 받습니다. 공격자의 이익은 그 차이이고, 피해자의 손실은 「90.7 − 75.6」입니다. 단위는 전부 통 안의 토큰 개수입니다. 아래 코드가 이 산수를 그대로 돌립니다.

여기서 한 가지가 중요합니다. 공격자는 피해자의 주문을 **바꾸지 않았습니다.** 피해자의 거래는 피해자가

서명한 대로 정확히 실행되었습니다. 바뀐 것은 그 거래가 실행되는 **순간의 통 상태이고**, 그것을 바꾼 것은 순서입니다. 14장의 `approve` 경쟁 조건과 같은 모양입니다.

3. 누가 순서를 정하는가 — 경매와 생산자

[중급] 순서를 정하는 사람은 **블록을 만드는 사람입니다**. 그런데 블록 생산자가 직접 샌드위치를 만들지 않아도, 봇들이 생산자에게 **순서를 사려고** 경매를 벌입니다. 논문은 이것을 「우선순위 가스 경매(priority gas auction, PGA)」라 부릅니다. 「이더리움에서 우리가 관찰한 PGA 게임은 같은 (계정, 논스) 쌍으로 발행된 거래들로 이루어지며, 채굴자에게 내는 입찰을 표현한다. … 차익거래 기회에 입찰하려면 차익거래자는 가스 가격 g 를 붙인 스마트계약 거래를 발행한다. 뒤에 그 입찰을 올리고 싶으면 … 같은 논스에 더 높은 가스 가격 g' 를 붙여 거래를 다시 발행한다. 두 차익거래자가 서로 입찰하면 본질적으로 경매가 생긴다.」[6] 11장의 논스가 여기서 「같은 자리의 입찰 갱신」 장치로 쓰입니다. 그리고 이 경매에는 특이한 성질이 있습니다. **패자도 냅니다.** 「각 (계정, 논스) 쌍은 다음 블록에 거래 하나를 넣을 수 있으므로, 패자의 거래도 채굴되어 최종 블록에 포함된다. 채굴자는 실패한 거래도 포함할 유인이 있다. 그 거래들이 시도된 실행에 대해 지불하기 때문이다. … 게임이론 용어로 각 경매는 전부 지불 경매(all-pay auction)의 변형이다.」[7] 13장의 「가스를 다 쓰면 실행은 되돌려지지만 수수료는 남는다」가 패자의 비용이 되는 자리입니다.

[중급] 논문의 관측 몇 가지는 구조를 보여 주므로 옮깁니다(금액은 옮기지 않습니다). 봇들은 지연 시간을 줄이는 경쟁을 벌였고, 입찰 인상 폭은 시간이 가면서 당시 널리 쓰인 Parity 클라이언트가 허용하던 최소 인상 폭(12.5%)으로 수렴했으며, 논문은 이를 「협조적 균형」으로의 수렴으로 읽습니다.[8] 관찰된 경매의 대부분은 두세 명 사이의 게임이었고, 순수 수익 기회의 대부분에는 경쟁이 붙지 않았는데, 논문은 그 이유의 하나로 경쟁에서 계속 지는 봇이 낙담해 시장을 떠난다는 가설(부록 C)을 제시하고, 참가자가 늘수록 참여가 이익이 되려면 보상이 패배 비용에 비례해 커져야 한다는 산수도 덧붙입니다. 이 관측은 논문의 그림이 보여 주는 기간(가로축 2017-07~2019-03)에, 논문이 지원한 거래소들(당시 거래량 상위 다섯 DEX 포함)의 「순수 수익 기회」만 본 **하한이며**, 논문 스스로 측정 한계를 길게 적습니다.[9] 이 책이 가져가는 것은 수치가 아니라 모양입니다. 순서는 **경매로 팔리고**, 그 경매는 패자도 내는 경매이며, 경매 수입은 블록 생산자에게 갑니다.

MEV — 그리고 왜 그것이 합의층의 문제인가

[중급] 논문이 이름 붙인 개념이 여기서 나옵니다. 「순서 최적화(OO) 수수료는 **채굴자 추출 가능 가치(MEV)라는** 더 일반적인 계량 가능 가치의 한 경우다. MEV는 채굴자가 주어진 기간 안의 거래를 조작해 뽑아낼 수 있는 이더의 총량을 가리키며, 여러 블록 분량의 거래를 포함할 수 있다.」 봇들이 경매로 내는 돈은 생산자가 **직접 할 수도 있었던 일의** 값입니다. 「OO 수수료를 얻기 위해 채굴자는 사용자 거래를 재배열하고 자기 거래를 끼워 넣어 이익을 자기 계정으로 직접 거둘 수 있다. 예컨대 채굴자는 이 논문이 서술한 순수 수익 거래를 스스로 실행하면서, 같은 것을 시도하는 「패배한」 차익거래 봇들의 PGA 수수료까지 전부 챙길 수 있다.」[10]

[중급] 그리고 논문의 결론이 이 장을 1부와 묶습니다. 「안정된 블록체인에서는 블록 보상이 정직한 채굴자 행동을 유인한다. 그러나 … 순서 최적화 수수료는 **블록 보상을 넘을 수 있고**, 대신 포크 공격을 유인할 수 있다.」 두 공격을 듭니다. 하나는 「MEV가 큰 블록을 포크하는」 언더컷 공격(20장이 다루는 「보상 없는 비트코인의 불안정성」 논문이 먼저 분석한 것)이고, 다른 하나는 새로 이름 붙인 **타임밴딧(time-bandit) 공격입니다**. 「블록체인에 현재 블록 높이에서 끝나는 끝 구간(부분 사슬)이 있고, 거기서 훔칠 수

있는 가치가 블록 보상을 넘는다고 하자. 공격자는 그 구간의 시작으로 되감아, 거기서 나오는 MEV로, 그 구간의 끝까지 또는 그 너머로 포크를 채굴하는 수익성 있는 51% 공격을 **보조할** 수 있다.」[11] 9장은 블록이 쌓일수록 되감기가 비싸진다고 했습니다. 17장이 더하는 것은, 되감기로 **얻을 것이** 거래 순서 안에 들어 있다면 그 비용을 감당할 유인이 생긴다는 점입니다. 「우리 연구의 핵심 결과는 응용층의 보안이 합의층 보안에 현재적이고 직접적인 위협을 가한다는 것이다.」[12] 비용 쪽 계산은 20장, 전략 쪽은 23장으로 넘깁니다.

> ? **법률가의 질문**
>
> 샌드위치 공격자는 피해자의 주문을 한 글자도 바꾸지 않았습니다. 자기 주문 둘을 피해자 앞뒤에 놓았을 뿐이고, 셋 다 코드가 정한 대로 실행되었습니다. 이것은 시세조종입니까. 조종된 「시세」가 있습니까 — 통 안의 두 수량 비율을 시세라 부를 수 있습니까. 조종한 「행위자」는 누구입니까 — 주문을 넣은 봇 운영자, 그 순서를 받아 블록에 넣은 생산자, 그 순서를 경매로 판 중계자. 규제가 전제하는 「시장」과 「거래소」는 어디에 있습니까. 전통 시장에서 선행매매를 금지하는 근거(신인의무, 정보 비대칭의 악용)는 중개인이 없는 곳에서도 성립합니까. 그리고 생산자가 **직접** 샌드위치를 했다면, 그는 자기 블록에서 무엇을 한 것입니까. → 28장

4. 대응 — 금지가 아니라 분업: 제안자-빌더 분리

선행매매를 금지하는 법을 코드에 넣을 수는 없습니다. 어느 거래가 「피해자」이고 어느 거래가 「공격」인지 코드는 모르고, 순서를 정하는 권한은 블록 생산의 본질이기 때문입니다. 이더리움 연구자들이 겨냥한 것은 다른 문제, 곧 **집중이었습니다**. 부테린의 2021년 6월 연구 게시글: 「합의 네트워크의 지속적인 탈중앙화를 위협하는 주요 위험은 MEV를 둘러싼 경제학, 곧 다음 블록의 내용을 고를 능력에서 이익을 뽑아내는 정교한 기법들이다. … 보통의 PoS 보상은 합리적으로 평등해 단독 검증자도 큰 풀과 같은 수익률을 얻지만, 정교한 MEV 추출 기회를 찾는 데에는 **상당한 규모의 경제가** 있다. 10배 큰 풀은 MEV를 뽑을 기회가 10배 많을 뿐 아니라 기회마다 더 많이 뽑는 독점적 최적화에 훨씬 많은 노력을 들일 수 있다.」[13] 10장의 「PoS는 부자가 더 부자 되는 구조인가」가 여기서 새 근거를 얻습니다. 보상의 비례는 유지되지만, **순서에서 뽑는 돈은** 큰 쪽에 더 많이 갑니다.

제안은 역할을 가르는 것입니다. 「가장 잘 알려진 해법은 **제안자/블록 빌더 분리다**. 블록 제안자가 스스로 수익 최대화 블록을 만들려 애쓰는 대신, 블록 빌더라 부르는 바깥 행위자들이 완전한 블록 내용과 제안자에게 줄 수수료로 이루어진 묶음을 만드는 시장에 의존하고, 제안자는 수수료가 가장 높은 묶음을 고른다. 제안자의 선택은 가장 높은 수수료의 묶음을 고르는 것으로 줄어들며, 이는 너무 단순한 알고리듬이어서 탈중앙 풀 안에서는 속임을 막기 위해 MPC 안에서도 할 수 있다.」[14] 핵심은 **제안자가 블록 내용을 보지 못한 채 고른다는** 것입니다. 게시글이 바람직한 성질로 든 다섯 중 하나가 「묶음 탈취 불가능성: 제안자는 블록 빌더가 제안한 묶음에서 거래를 뽑아 자기 묶음을 만들 수 없어야 한다」입니다.[15] MEV를 없애는 것이 아니라, MEV를 뽑는 일을 **전문 빌더에게 몰아 주고** 그 대가를 모든 제안자가 경매로 받게 하는 설계입니다. 이더리움 공식 문서는 같은 취지를 「블록 빌더가 정교한 MEV 추출을 했을지라도 그 보상은 블록 제안자에게 간다. 그래서 소수의 전문 빌더가 MEV 추출을 지배하더라도 그 보상은 네트워크의 어느 검증자에게든, 개인 홈 스테이커에게도 갈 수 있다」고 적습니다.[16]

[중급] 이 설계가 **프로토콜 안에** 들어가기 전에, 밖에서 먼저 돌아가는 구현이 나왔습니다. 2021년 11월 플래시봇츠(Flashbots) 쪽 게시글이 설계를 적습니다. 「이 문서는 다가오는 이더리움 머지 포크와 호환되는 블록 구축 시장(흔히 블록 제안자/블록 빌더 분리 또는 PBS라 부르는 것)의 설계를 개요한다. 이 **신뢰 기반** 해법은 현재의 플래시봇츠 경매 설계와 매우 닮았되, 이더리움 합의를 바꾸지 않고 단독 스테이커가 참여할 수 있게 수정한 것이다.」 구조는 사용자·서처(searcher) → 빌더 → **릴레이** → 제안자이고, 「검증자는 릴레이에서 실행 페이로드 **헤더**(거래 내용을 뺀 것)를 받아 … 가장 가치 있는 헤더를 골라 서명한 뒤 릴레이에 돌려준다.」 「실행 페이로드 헤더에 거래 내용이 없으므로 검증자가 선행매매할 위험은 제거된다. … 그러나 이 설계는 검증자가 **릴레이를 신뢰해** 유효하지 않은 페이로드를 걸러 주고, 페이로드 가치를 정확히 보고하고, 헤더가 서명된 뒤 거래 내용을 공개하리라고 믿을 것을 요구한다.」[17] 소프트웨어 이름이 MEV-Boost이고, 그 저장소 README는 스스로를 「검증자가 경쟁적 블록 구축 시장에 접근하기 위해 돌리는 오픈소스 미들웨어 … 지분증명 이더리움을 위한 PBS의 **초기 구현**」이라 부르며, 「장래에 제안자/빌더 분리는 이더리움 프로토콜 자체에 내재화될(enshrined) 것」이라고 적습니다.[18]

그래서 **2026-10 기준** 그림은 이렇습니다. 프로토콜 안에는 제안자만 있고 빌더·릴레이는 없습니다. 분업은 프로토콜 밖의 미들웨어와 릴레이 사업자들이 돌리고 있으며, 이더리움 공식 로드맵 페이지는 「PBS는 연구가 진전된 단계에 있지만, 이더리움 클라이언트에서 시제품을 만들기 전에 풀어야 할 중요한 설계 질문이 남아 있다. **아직 확정된 명세는 없다**」고 적습니다.[19] 이 문장은 낡기 쉬우므로 부록 A 동향 노트에 올려 두고 판마다 다시 확인합니다. 같은 페이지는 PBS가 검열 저항에도 쓰일 수 있다고 적지만(제안자가 「포함 목록」으로 빌더의 거래 누락을 강제하는 설계), 그것 역시 「활발한 연구 영역」이라고 밝힙니다.

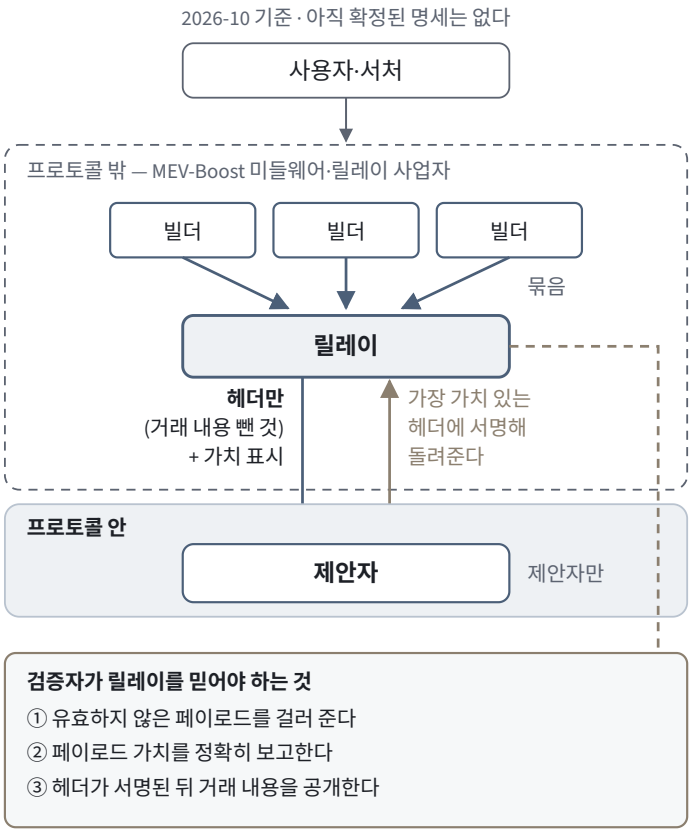


그림 17-3. 2026-10 기준 제안자–빌더 분리의 분업. 프로토콜 안에는 제안자만 있고 빌더와 릴레이는 프로토콜 밖(MEV-Boost 미들웨어)에서 돌며, 제안자는 거래 내용이 빠진 헤더에 서명한다. 아직 확정된 명세는 없다.

분업이 바꾸지 않는 것도 적어 둡니다. 샌드위치는 여전히 일어납니다. 바뀐 것은 **누가 그 이익을 나누는가입니다**. 빌더가 뽑고, 제안자가 경매로 받고, 사용자는 여전히 더 나쁜 가격을 받습니다. 사용자를 보호하는 다른 길들 — 거래를 공개 대기열에 내지 않고 빌더에게 직접 보내기, 거래 내용을 암호화한 대기열, 체인링크 2.0 백서가 제안한 「공정 순서 서비스」[20] — 은 각각 다른 누군가를 믿는 설계이고, 이 책은 어느 것도 평가하지 않습니다. 2층 롤업의 시퀀서가 자기 대기열을 비공개로 두어 이 문제를 다르게 다루는 것은 18장에서 봅니다.

5. 코드 — 샌드위치를 풀 단위로 돌려 본다

세 거래를 차례로 — 앞·피해자·뒤

[중급] 15장의 풀 클래스를 그대로 다시 적고(노트북이 독립적으로 돌아가야 하므로), 세 거래를 순서대로 넣습니다. 단위는 전부 풀 안의 토큰 개수이고, 「공격자 이익」은 **구조를 보이는 데에만** 씁니다. 이 수치는 어떤 실제 시장의 값이 아닙니다.

```
# NB17-1: 샌드위치 — 15장의 상수곱 풀을 그대로 가져와 「앞·피해자·뒤」 세 거래를 순서대로 넣는다 (공부용, 풀 단위)
import math

class ConstantProductPool:                                  # 15장 NB15-1 과 같은 클래스(스왑 부분만)
    def __init__(self, x, y, fee=0.003):
        self.x, self.y, self.fee = float(x), float(y), fee
    @property
    def price_x_in_y(self): return self.y / self.x
    def swap_x_for_y(self, dx):
        eff = dx * (1 - self.fee); dy = self.y * eff / (self.x + eff)
        self.x += dx; self.y -= dy; return dy
    def swap_y_for_x(self, dy):
        eff = dy * (1 - self.fee); dx = self.x * eff / (self.y + eff)
        self.y += dy; self.x -= dx; return dx

def sandwich(pool_x, pool_y, victim_y, attacker_y):
    """피해자는 Y victim_y 로 X 를 산다. 공격자는 그 앞에서 Y attacker_y 로 X 를 사고, 뒤에서 그 X 를 전부 되판다.
    반환: (공격자가 돌려받은 Y - 넣은 Y, 피해자가 받은 X, 피해자가 혼자였다면 받았을 X)"""
    alone = ConstantProductPool(pool_x, pool_y).swap_y_for_x(victim_y)
    p = ConstantProductPool(pool_x, pool_y)
    front_x = p.swap_y_for_x(attacker_y) if attacker_y > 0 else 0.0     # ① 앞: 공격자 매수
    victim_x = p.swap_y_for_x(victim_y)                                 # ② 피해자 매수 — 이미 오른 가격으로
    back_y = p.swap_x_for_y(front_x) if attacker_y > 0 else 0.0         # ③ 뒤: 공격자 매도
    return back_y - attacker_y, victim_x, alone

print("풀 X 1,000 / Y 1,000. 피해자는 Y 100 으로 X 를 산다.")
print(f"{'공격자 투입 Y':>12} {'공격자 순이익(Y)':>14} {'피해자가 받은 X':>14} {'혼자였다면 X':>12} {'피해자 손실 X':>12} {'피해자 손실 비율':>12}")
for a in [0, 50, 100, 200, 400, 800, 2000, 5000, 20000, 100000]:
    profit, vx, alone = sandwich(1000, 1000, 100, a)
    print(f"{a:>12} {profit:>14.3f} {vx:>14.3f} {alone:>12.3f} {alone - vx:>12.3f} {1 - vx / alone:>12.2%}")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
풀 X 1,000 / Y 1,000. 피해자는 Y 100 으로 X 를 산다.
    공격자 투입 Y     공격자 순이익(Y)      피해자가 받은 X      혼자였다면 X     피해자 손실 X    피해자 손실
비율
           0          0.000         90.661       90.661        0.000        0.00%
          50          9.353         82.601       90.661        8.060        8.89%
         100         17.368         75.570       90.661       15.091       16.65%
         200         30.322         63.957       90.661       26.704       29.45%
         400         48.047         47.526       90.661       43.135       47.58%
         800         66.972         29.196       90.661       61.466       67.80%
        2000         84.967         10.743       90.661       79.918       88.15%
        5000         92.174          2.731       90.661       87.930       96.99%
       20000         94.020          0.226       90.661       90.435       99.75%
      100000         94.020          0.010       90.661       90.651       99.99%
```

[중급] 표를 읽습니다. 공격자가 넣는 양이 클수록 피해자의 손실은 커지고, 공격자의 순이익은 **피해자가 넣은 Y 100개에 다가가되 그것을 넘지 못합니다.** 공격자가 가져가는 것은 결국 피해자가 낸 돈이고, 피해자 주문보다 훨씬 큰 자본을 넣어도 더 가져갈 것이 없습니다(표의 아래 줄들에서 순이익이 94 근처에서 멈추는 이유. 나머지는 수수료로 풀에 남습니다). 그러니 이 공격을 묶는 것은 공격자의 자본이 아닙니다. 피해자 주문 크기와 풀 크기라는 **구조 변수가** 상한을 정하고, 실제로 공격자를 가장 세게 묶는 것은 다음 셀의 변수입니다.



공격자 투입 Y	피해자가 받은 X	혼자였다면 X	피해자 손실 X	피해자 손실 비율	허용치 안?
0	90.661	90.661	0.000	0.00%	안
50	82.601	90.661	8.060	8.89%	안
100	75.570	90.661	15.091	16.65%	안
200	63.957	90.661	26.704	29.45%	안
400	47.526	90.661	43.135	47.58%	안
800	29.196	90.661	61.466	67.80%	안
2,000	10.743	90.661	79.918	88.15%	안
5,000	2.731	90.661	87.930	96.99%	안
20,000	0.226	90.661	90.435	99.75%	안
100,000	0.010	90.661	90.651	99.99%	안

앞 거래 100 이 가격을 1.0000 에서 1.2097 로 올려 놓아, 피해자는 혼자였다면 받았을 90.661 대신 **75.570** 을 받는다 — 손실 15.091 (16.65%). 허용치를 두지 않으면 공격자 투입이 커질수록 손실은 피해자 주문 전부에 다가간다 — 상한을 정하는 것은 공격자 자본이 아니라 풀 크기·피해자 주문·허용치라는 구조 변수다.

그림 17-4. 샌드위치 공격 — 피해자 쪽. 풀 1,000 : 1,000 에서 Y 100 으로 X 를 사는 피해자는 혼자였다면 90.661 을 받지만, 앞에 공격자 매수 100 이 끼면 75.570 을 받는다(손실 15.091, 16.65%). 출력은 피해자 손실뿐이다. — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 17장 5절).

슬리피지 허용치가 공격을 묶는다

```
# NB17-2: 피해자의 슬리피지 허용치가 공격 크기를 묶는다 — 「최소 수취량」 조건을 넣고 공격자의 최적 투입을 찾는다
(공부용, 풀 단위)
def best_attack(pool_x, pool_y, victim_y, tolerance, fee=0.003, grid=2000):
    """피해자가 「혼자였다면 받을 X 의 (1 - tolerance) 이상은 받아야 한다」는 조건을 거래에 붙였다고 가정.
    그 조건을 깨지 않는(=피해자 거래가 되돌려지지 않는) 공격자 투입 Y 가운데 순이익이 가장 큰 것을 격자 탐색으로
찾는다."""
    alone = ConstantProductPool(pool_x, pool_y, fee).swap_y_for_x(victim_y)
    min_out = alone * (1 - tolerance)
    best = (0.0, 0.0)                                          # (순이익, 투입)
    for i in range(1, grid + 1):
        a = pool_y * 2 * i / grid                              # 0 ~ 풀 Y 의 두 배까지 훑는다
        profit, vx, _ = sandwich(pool_x, pool_y, victim_y, a)
        if vx < min_out:                                       # 피해자 거래가 실패하면 샌드위치도
성립하지 않는다(앞 거래만 남아 공격자 손해)
            break
        if profit > best[0]:
            best = (profit, a)
    return best, alone, min_out

print("풀 X 1,000 / Y 1,000. 피해자 주문 Y 100. 피해자가 거래에 붙인 슬리피지 허용치별로 공격자의 최적 투입과
순이익:")
print(f"{'허용치':>8} {'피해자 최소 수취 X':>16} {'공격자 최적 투입 Y':>16} {'공격자 순이익(Y)':>14}")
for tol in [0.001, 0.005, 0.01, 0.02, 0.05, 0.10, 0.50]:
    (profit, a), alone, min_out = best_attack(1000, 1000, 100, tol)
    print(f"{tol:>8.1%} {min_out:>16.3f} {a:>16.1f} {profit:>14.3f}")
print("\n허용치가 좁을수록 끼울 수 있는 공격이 작아진다(이 1 Y 격자에서는 0.1%일 때 0). 순이익은 허용치에 거의
비례하고, 최적 투입은 그보다 빠르게 커진다.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
풀 X 1,000 / Y 1,000. 피해자 주문 Y 100. 피해자가 거래에 붙인 슬리피지 허용치별로 공격자의 최적 투입과 순이익:
     허용치       피해자 최소 수취 X       공격자 최적 투입 Y      공격자 순이익(Y)
    0.1%            90.570               0.0           0.000
    0.5%            90.208               2.0           0.404
    1.0%            89.754               5.0           1.004
    2.0%            88.848              10.0           1.992
    5.0%            86.128              27.0           5.234
   10.0%            81.595              56.0          10.380
   50.0%            45.331             434.0          50.317

허용치가 좁을수록 끼울 수 있는 공격이 작아진다(이 1 Y 격자에서는 0.1%일 때 0). 순이익은 허용치에 거의 비례하고, 최적
투입은 그보다 빠르게 커진다.
```

피해자가 거래에 「이것보다 적게 받으면 거래를 취소하라」는 조건(슬리피지 허용치)을 **좁게** 붙이면 공격자가 끼울 수 있는 공격이 그만큼 작아집니다(위 표의 1 Y 격자에서는 0.1%일 때 0으로 나왔지만, 더 잔 격자에서는 작은 공격이 성립합니다). 조건이 좁을수록 자기 거래가 실패할 확률도 커지니, 피해자는 실패 위험과 샌드위치 위험을 맞바꿉니다. 이 변수는 피해자 손에 있습니다. 이 장의 위젯 W10은 풀 크기·피해자 주문·공격자 투입·허용치 네 구조 변수만 움직이고, 통화 단위를 쓰지 않습니다.

6. 정리

거래는 블록에 들어가기 전에 공개되고, 블록을 만드는 사람이 순서를 정하며, 순서는 AMM의 가격을 바꾸므로 순서 자체가 돈이 됩니다. 봇들은 그 순서를 경매로 사고(패자도 내는 경매), 생산자는 직접 할 수도 있으며, 그 돈이 블록 보상을 넘으면 되감기 공격의 유인이 됩니다. 이더리움의 대응은 금지가 아니라 분업(PBS)이고, 2026-10 기준 그 분업은 프로토콜 밖의 미들웨어와 릴레이로 돌아가며 명세는 확정되지 않았습니다. 수수료 시장이 이 문제를 어떻게 다루었는지(기본 수수료 소각이 생산자의 수수료 조작 유인을 줄인다는 설계 의도)는 21장에서, 공격 비용의 계산은 20장에서, 전략의 균형은 23장에서 이어집니다. 2부의 마지막 18장은 거래를 아예 다른 층으로 옮기는 설계를 봅니다. 거기에도 순서를 정하는 사람(시퀀서)이 있습니다.

이 장의 출처

- Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, Ari Juels, "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability," *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 910–927, DOI 10.1109/SP40000.2020.00040 — 가십과 시뮬레이션(§III-C), 선행매매(§I·§II-D), 원자성(§III-A), PGA와 전부 지불 경매(§III-C), 관측과 한계(§IV·§VI·Appendix C), MEV 정의와 포크 공격(§VII). 금액 집계는 3장 일러두기의 원칙에 따라 옮기지 않았다.
- Lorenz Breidenbach 외, "Chainlink 2.0," 15 April 2021 — 선행매매·후행매매·샌드위치 용어(§5.1), 공정 순서 서비스(§5).
- Vitalik Buterin(vbuterin), "Proposer/block builder separation-friendly fee market designs," ethresear.ch, 2021-06-04. https://ethresear.ch/t/proposer-block-builder-separation-friendly-fee-market-designs/9725 ; 같은 저자, "Two-slot proposer/builder separation," 2021-10-10. https://ethresear.ch/t/two-slot-proposer-builder-separation/10980 (2026-10-25 접근, Discourse 원문 대조).
- 계정 thegostep, "MEV-Boost: Merge ready Flashbots Architecture," ethresear.ch, 2021-11-04. https://ethresear.ch/t/mev-boost-merge-ready-flashbots-architecture/11177 (2026-10-25 접근).
- flashbots/mev-boost, README.md, 커밋 7a60459f(2025-12-05). https://github.com/flashbots/mev-boost/blob/7a60459f1e61ee4d3b256978cd34bef5fe5e95c6/README.md
- ethereum.org, "Proposer-builder separation," Page last update 2026-06-24. https://ethereum.org/en/roadmap/pbs/ (소스 커밋 26bdbe8e; 2026-10-25 접근) — PBS와 MEV, 현재 진행 상황(「2026-10 기준」 문장, 부록 A 색인).
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `math` 만 사용. 노트북 NB17. 위젯 W10 `sandwich` 의 처음 상태는 NB17-1의 풀 1,000:1,000·피해자 주문 100·공격자 투입 100이다.

각주

1. Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, Ari Juels, "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability," *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 910–927, §III-C(914쪽). 원문: "Ethereum

transactions are routed in a peer-to-peer gossip multicast protocol by client node software. This means that all transaction information is available to all participants in this network, but earlier to participants with advantageous positions in the gossip topology. Additionally, nodes can simulate the outcome of every transaction given the current or expected system state." 2019년 arXiv 사전 인쇄본(arXiv:1904.05234)은 부제가 다르며, 이 책은 게재본을 인용합니다.

2. 같은 글, §I(910쪽). 원문: "Worse still, adversaries can frontrun orders, observing them and placing their own orders with higher fees to ensure they are mined first."

3. 같은 글, §II-D(912쪽). 원문: "Traditional exchanges experience a classic form of predatory market behavior called frontrunning [4]. In regulated markets, frontrunning is often illegal, and has resulted in prosecutions [26] and tarnished the reputations of financial institutions caught practicing it. Frontrunning generally exploits information asymmetries created by power structures within a financial structure, e.g., brokers having privileged access to user information. Because there is no single party playing the role of a broker in decentralized systems, information asymmetries can arise for actors in advantageous positions in underlying infrastructure."

4. 같은 글, §III-A(913쪽). 원문: "Because of the atomic batch-based processing of transactions, and because transactions can themselves be initiated by smart contracts, is possible to build bots that trade across exchanges through proxy contracts. These proxy contracts can execute batches of orders sequentially within a single transaction, reverting previous trades by throwing an exception if any trade in the batch fails. This means arbitrageurs have the opportunity to compose single transactions that execute multiple trades across multiple exchanges atomically, with an all-or-nothing failure model." (원문의 「is possible」은 그대로.)

5. Lorenz Breidenbach 외, "Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks," 15 April 2021 (v1.0), §5.1(45쪽). 원문: "Front-running exemplifies a class of transaction-ordering attacks: There are a number of related attacks such as back-running and sandwiching (front-running plus back-running) [237] that we don't cover here, but which FSS also helps address."

6. Daian 외 (2020), §III-C(914쪽). 원문: "In Ethereum, the PGA game we observe consists of transactions issued with the same (account, nonce) pair, expressing a bid to a miner, where the miner is paid gas fees as described in Section II-A. We call this interaction of issuing repeated bids a priority gas auction, or PGA. To place a bid on an arbitrage opportunity, an arbitrageur simply issues a smart contract transaction with associated gas price g that atomically bundles multiple trades (performing the arbitrage). If they later wish to increase this bid, ... they simply reissue the transaction with the same nonce and a higher gas price g′. If two arbitrageurs are bidding against each other, what emerges is essentially an auction ...".

7. 같은 글, 같은 쪽. 원문: "The transaction shown in the red square ... is also mined and included in the final block, as each (account, nonce) pair can include one transaction in the next block to be mined as per the protocol. Miners are incentivized to include even failed transactions, as these transactions pay for attempted execution. ... In game theoretic terms, each auction represents a variant of all pay auction, where instead of paying their full bid, the loser is forced to pay a percentage to the miner."

8. 같은 글, §VI-A·D(918–920쪽), Fig. 7·10. 원문: "we have observed empirically that players are consistently and substantially reducing their respective latencies over time (Figure 7)"; "Remarkably as shown in Figure 10, in our observed PGAs, players converge over time to a minimum bid raise of 12.5%, which is the minimum allowed raise in the Parity client." 「협조적 균형」: "We do claim, however, that players are converging to a more cooperative state, that is approximated by our model ...".

9. 같은 글, §IV-A1 Instrumentation Limitations(915쪽), Appendix C(924–925쪽). 원문: "Our supported exchanges include the top five DEXes by sustained volume at the time of infrastructure development."; "Because we aim to establish a lower bound (i.e. a potentially too conservative result), we feel this is acceptable."; "of the 2,792 PGAs we observe, 75% (2,096) are played between 2 or less players and 91% are played between 3 or less players." 측정 기간은 9개월(§IV-A), 지원 거래소에는 당시 거래량 상위 다섯 DEX가 포함. 금액 집계는 3장 일러두기의 원칙에 따라 옮기지 않았습니다.

10. Daian 외 (2020), §VII(920–921쪽). 원문: "OO fees represent one case of a more general quantifiable value we call miner-extractable value (MEV). MEV refers to the total amount of Ether miners can extract from manipulation of transactions within a given timeframe, which may include multiple blocks' worth of transactions."; "To capture OO fees, a miner can reorder users' transactions and potentially insert their own,

reaping profit in Ether directly to their account. For example, a miner could execute the pure revenue transactions described in this work themselves, while still claiming the PGA fees for all the 'losing' arbitrage bots attempting to do the same."

11. 같은 글, §VII(921쪽)·§VII-C(922쪽). 원문: "In a stable blockchain, block rewards incentivize honest miner behavior. As we show empirically in this section, though, order optimization (OO) fees, or implicit fees a miner is able to reap by leveraging their control of a consensus epoch, can exceed the block reward and instead incentivize forking attacks."; "Suppose a blockchain has a suffix (subchain) [height0, height1], with current block height height1, in which stealable value exceeds block rewards. An adversary can rewind to height0 and use the resulting MEV to subsidize a profitable 51% attack that mines a fork up to or past height1." 언더컷 공격의 원 출처는 논문 참고문헌 [11] Carlsten 외, CCS 2016 — 20장의 원자료.

12. 같은 글, §VII(920쪽). 원문: "In other words, a key result of our work is that application-layer security poses a current and direct threat to consensus-layer security."

13. Vitalik Buterin(계정 vbuterin), "Proposer/block builder separation-friendly fee market designs," ethresear.ch, 2021-06-04. 원문: "A major risk threatening the ongoing decentralization of consensus networks is the economics around miner extractable value (MEV), sophisticated tricks to extract profit from the ability to choose the contents of the next block. ... While normal PoS rewards are reasonably egalitarian, as single validators earn the same rate of return as powerful pools, there are significant economies of scale in finding sophisticated MEV extraction opportunities. A pool that is 10x bigger will have 10x more opportunities to extract MEV but it will also be able to spend much more effort on making proprietary optimizations to extract more out of each opportunity." https://ethresear.ch/t/proposer-block-builder-separation-friendly-fee-market-designs/9725 (2026-10-25 접근, Discourse 원문 다시 열어 대조).

14. 같은 글. 원문: "The best-known solution is proposer/block-builder separation. Instead of the block proposer trying to produce a revenue-maximizing block by themselves, they rely on a market where outside actors that we call block-builders produce bundles consisting of complete block contents and a fee for the proposer, and the proposer chooses the bundle with the highest fee. The proposer's choice is reduced to picking the highest-fee bundle, an algorithm so simple that in a decentralized pool it can even be done inside an MPC to prevent cheating."

15. 같은 글, 「Desired properties」. 원문: "Bundle un-stealability: proposers should not be able to take bundles proposed by block builders and extract transactions from them to make their own bundles, preventing the block builder from earning a profit (and possibly harming them even further)". 나머지 넷은 「untrusted proposer friendliness」, 「untrusted builder friendliness」, 「weak proposer friendliness」, 「consensus-layer simplicity and safety」. 같은 저자의 후속 설계 "Two-slot proposer/builder separation"(ethresear.ch, 2021-10-10, https://ethresear.ch/t/two-slot-proposer-builder-separation/10980)은 이를 두 슬롯 구조로 구체화합니다.

16. ethereum.org, "Proposer-builder separation," 로드맵 페이지(Page last update: June 24, 2026). 원문: "The block builders might have done sophisticated MEV extraction, but the reward for it goes to the block proposer. This means that even if a small pool of specialized block builders dominate MEV extraction, the reward for it could go to any validator on the network, including individual home stakers." https://ethereum.org/en/roadmap/pbs/ (2026-10-25 접근; 소스 저장소 ethereum/ethereum-org-website, 해당 파일 마지막 커밋 26bdbe8e, 2026-06-24).

17. 계정 `thegostep` (실명 미확인 — Discourse 표시명이 비어 있음), "MEV-Boost: Merge ready Flashbots Architecture," ethresear.ch, 2021-11-04. 원문: "This document outlines the design for a marketplace for block construction (often referred to as block proposer / block builder separation or PBS) compatible with the upcoming Ethereum merge fork. This trust based solution closely resembles the current Flashbots auction design with modifications to enable solo staker participation without introducing changes to Ethereum consensus."; "Validators receive execution payload headers from relays (execution payloads stripped of the transaction content). The relay must attach an indication of the payload value to each header. The validator selects the most valuable header, signs the payload, and returns it to the relay and escrow to be propagated to the network."; "Since the execution payload header does not include the transaction content, the risk of validators frontrunning is eliminated. ... However, this design also requires validators to trust the relay to filter invalid payloads, accurately report payload value, and reveal the transaction content once the header is signed." https://ethresear.ch/t/mev-boost-merge-ready-flashbots-architecture/11177 (2026-10-25 접근).

18. flashbots/mev-boost, README.md(커밋 7a60459f, 2025-12-05). 원문: "`mev-boost` is open source middleware

run by validators to access a competitive block-building market. MEV-Boost is an initial implementation of proposer-builder separation (PBS) for proof-of-stake (PoS) Ethereum."; "Relays aggregate blocks from **multiple** builders in order to select the block with the highest fees."; "In the future, proposer/builder separation will be enshrined in the Ethereum protocol itself to further harden its trust model." https://github.com/flashbots/mev-boost/blob/7a60459f1e61ee4d3b256978cd34bef5fe5e95c6/README.md (2026-10-25 접근).

19. ethereum.org, "Proposer-builder separation," 「Current progress」(Page last update: June 24, 2026). 원문: "PBS is in an advanced stage of research, but there are still some important design questions that need to be resolved before it can be prototyped in Ethereum clients. There is no finalized specification yet." 검열 저항 부분: "inclusion lists can be introduced so that when validators know about transactions but don't see them included in blocks, they can impose them as must-haves in the next block. … This is still an area of active research and the optimal configuration for the inclusion lists has not yet been determined." (2026-10-25 접근.)

20. Breidenbach 외 (2021), §5 Fair Sequencing Services(43–57쪽). 원문: "An overarching goal of the FSS in DONs is to empower DeFi creators to realize fair financial systems, that is, systems that don't advantage particular users (or miners) over others on the basis of speed, insider knowledge, or ability to perform technical manipulation. While a crisp, general notion of fairness is elusive, and perfect fairness in any reasonable sense is unachievable, FSS aims to provide developers with a powerful set of tools …". 이 책은 이 제안을 설계 유형의 하나로만 적습니다.

2부 스마트계약과 응용

18장 확장성 — 레이어 2와 롤업

핵심 TAKEAWAY

블록체인은 모든 노드가 모든 거래를 똑같이 다시 계산하기 때문에 안전합니다. 같은 이유로 느립니다. 노드를 늘려도 처리량은 늘지 않습니다. 모두가 같은 일을 하니까요. 블록을 키우면 처리량은 늘지만 노드를 돌리는 데 더 센 기계가 필요해져 참여자가 줄어듭니다. 이 긴장을 흔히 「확장성 트릴레마」라 부르는데, 그 이름은 **정리(定理)가 아니라 설계 휴리스틱입니다.** 이더리움이 택한 길은 **롤업입니다.** 거래의 **실행은** 바깥 체인(레이어 2)에서 하고, 그 거래 **데이터와** 결과의 **요약만** 본체(레이어 1)에 적습니다. 본점 장부에는 합계만 적고 세부는 지점이 처리하되, 분쟁이 생기면 본점에서 가립니다. 「가린다」는 방법이 둘입니다. 낙관적 롤업은 「틀렸다고 누군가 증명할 때까지 옳다고 치고」 이의 기간을 두며, 영지식 롤업은 「옳다는 수학적 증명을 매번 첨부」합니다. 이 장을 읽으면 레이어 2가 「더 빠른 블록체인」이 아니라 **본체에 데이터를 맡기고 실행을 떼어 낸 구조이고,** 그 구조 안에 다시 「순서를 정하는 사람」(시퀀서)이 있다는 것을 압니다.

학습목표

1. 처리량이 「블록당 용량 ÷ 블록 간격」으로 묶이는 이유와, 블록을 키우는 것이 왜 공짜가 아닌지 설명한다.
2. 롤업이 본체에 올리는 세 가지(거래 데이터, 상태 요약, 강제 포함 메시지)와 각각이 보증하는 것을 가른다.
3. [중급] 낙관적 롤업의 이의 기간·대화형 사기 증명(이분법)과 영지식 롤업의 유효성 증명이 「누가 정직해야 하는가」에서 어떻게 다른지 비교한다.
4. 「트릴레마」가 설계 휴리스틱임을 알고, 롤업이 신뢰를 없앤 것이 아니라 **어디로 옮겼는지**(시퀀서, 업그레이드 권한, 데이터 가용성 층)를 짚는다.

[입문]**만 읽은 독자가 가져가는 것.** 레이어 2는 「본점 장부에는 합계만 적고 세부는 지점에서」 하는 구조이며, 분쟁이 생기면 본점으로 돌아옵니다.

1. 이 기술은 무엇의 실패에서 나왔나

13장에서 모든 노드가 같은 거래를 같은 순서로 실행한다고 했습니다. 그 「모두가 같은 일을 한다」가 병목입니다. ZKsync 문서의 서술이 간결합니다. 「이더리움의 탈중앙 네트워크는 거래 처리량이 제한되어 있다. 그 용량이 검증자 수에 따라 늘지 않기 때문이다. 본질적으로 각 검증자는 처리되는 모든 거래를 검증하는 같은 일을 수행하며, 이것이 병목을 만든다. 반면 웹2에서는 서버를 더 붙이면 확장성이 개선된다. 요청을 처리하는 네트워크 용량이 새 서버마다 선형으로 늘기 때문이다.」[1] 보통의 서버 농장은 일을 **나누고**, 블록체인은 일을 **복제합니다**. 복제가 안전의 원천이고, 복제가 느림의 원천입니다.

처리량은 간단한 산수로 묶여 있습니다. 블록 하나에 들어갈 수 있는 양을 블록 사이의 시간으로 나눈 것이 상한입니다. 비트코인은 블록 가중치 4,000,000 단위를 목표 간격 600초로 나눕니다. 이더리움은 블록의 가스 한도를 슬롯 간격 12초로 나눕니다.[2] 가스 한도는 프로토콜 상수가 아니라 검증자들이 블록마다 조금씩 움직이는 값이어서, 이 책의 기준일 관측값은 60,000,000이었습니다(**2026-10 기준**, 21장과 부록 A).[3] 그러면 초당 5,000,000가스가 상한이고, 13장의 가장 단순한 거래(내재 가스 21,000)만 들어온다고 가정하면 초당 약 238건이 이론적 상한입니다. 이 계산은 이 책의 것이며 「최대로 단순한 거래만 가득 찼을 때」의 상한이지 실측 처리량이 아닙니다. 그 상한을 올리는 가장 쉬운 길은 가스 한도(블록 크기)를 키우는 것인데, 그러면 모든 노드가 블록마다 더 많이 계산하고 더 많이 저장해야 합니다. 이더리움 공식 문서가 적는 제약이 그것입니다. 「속도와 처리량이 중요하지만, 이 목표를 가능하게 하는 확장 해법이 **탈중앙성과 보안을 유지하는 것이** 필수적이다. 노드 운영자의 진입 장벽을 낮게 유지하는 것은 중앙화되고 안전하지 않은 컴퓨팅 파워로의 진행을 막는 데 결정적이다.」[4]

처리량·탈중앙성·보안의 이 긴장을 흔히 「**확장성 트릴레마**」**라** 부릅니다. 이름에 주의가 필요합니다. 이것은 「셋 중 둘만 가질 수 있다」는 **증명된 정리가 아니라**, 설계자들이 세 목표가 서로 당긴다는 경험을 요약한 **휴리스틱입니다**. 이 책이 원자료로 확인한 공식 문서 세 쪽(위 「Scaling」 페이지와 아래의 두 롤업 페이지)은 세 목표를 나열하되 「트릴레마」라는 낱말을 쓰지 않습니다. 어느 설계가 그 셋을 「동시에」 얻었다는 주장은 각 낱말을 어떻게 재느냐에 달린 것이고, 이 장은 그 측정을 하지 않습니다. 대신 이더리움이 택한 **구조만** 봅니다. 공식 문서는 그 선택을 이렇게 요약합니다. 「오랫동안 블록체인을 샤딩하는 것이 이더리움을 확장할 것으로 기대되었다. … 그러나 **레이어 2 롤업에 의한 확장이 주된 확장 기법으로 자리를 넘겨받았다.**」[5]

2. 롤업의 구조 — 실행은 밖, 데이터와 분쟁은 안

공식 문서의 한 줄 정의입니다. 「롤업은 거래 실행을 레이어 1 밖에서 수행하고, 그 뒤 데이터를 레이어 1에 게시하여 거기서 합의에 이른다. 거래 데이터가 레이어 1 블록에 포함되므로 롤업은 이더리움 고유의 보안으로 보호받을 수 있다.」[6] 부품은 둘입니다. 「체인 위 계약: 롤업의 운영은 이더리움 위에서 돌아가는 스마트계약이 통제한다. 롤업 블록을 저장하고, 롤업의 상태 갱신을 감시하고, 사용자 예치를 추적하는 계약들이다. … 체인 밖 가상머신: 롤업 프로토콜을 관리하는 계약은 이더리움에서 돌지만, 롤업 프로토콜은 계산과 상태 저장을 이더리움 가상머신과 분리된 다른 가상머신에서 수행한다. 체인 밖 VM이 응용이 사는 곳이고 상태 변화가 실행되는 곳이다.」[7]

롤업이 본체에 **무엇을** 올리는지를 가르는 것이 이 장에서 가장 중요합니다. 옵티미즘(Optimism) 쪽 OP Stack 문서의 표가 그것을 셋으로 나눕니다. 「세 가지 다른 것이 부모 체인에 간다. 이들을 섞어 생각하는 것이 롤업 보안에 관한 혼동의 가장 흔한 원천이다.」[8]

올리는 것	어디에	무엇을 사는가
거래 데이터	블롭 또는 calldata	누구나 시퀀서 없이 체인을 **재구성할** 수 있다(파생 가능)
예치(강제 포함 메시지)	입금 계약	본체가 거래를 레이어 2에 **강제로 넣을** 수 있다(검열 저항)
출력 루트(상태 요약)	분쟁 게임 계약	본체가 레이어 2 상태를 **인증해** 인출을 처리할 수 있다(정산 가능)



그림 18-1. 롤업이 레이어 1에 올리는 세 가지와 각각이 사는 것. 위 표의 세 행을 레이어 2에서 레이어 1로 내려가는 경로로 다시 그렸다.

같은 문서가 덧붙인 문장이 1부와 이어집니다. 「거래 데이터는 체인을 **파생 가능하게** 만드는 것이고, 출력 루트는 체인을 **정산 가능하게** 만드는 것이다. 데이터는 올리되 출력 루트를 올리지 않는 체인은 여전히 누구나 검증할 수 있는 체인이지만 아무도 인출할 수 없다. 출력 루트는 올리되 데이터를 올리지 않는 체인은 검증할 수 없고, 그 제안은 확인할 수 없다.」 6장의 머클 루트와 같은 모양입니다. 요약(루트)은 작고, 그 요약이 참인지 확인하려면 **원 데이터가 어딘가에 있어야** 합니다. 그래서 「데이터 가용성(data availability)」이 롤업 설계의 중심 낱말이 됩니다. ZKsync 문서: 「롤업의 상태가 이더리움 관찰자들에게 알려져 있지 않다면, 검증자(중앙이든 분산이든)가 처리를 멈추는 상황에서 신뢰할 검증자 없이는 상태 전이를 만드는 것이 불가능해진다. 그러나 데이터가 관찰자들에게 언제나 가용하다면, 신뢰받던 검증자가 운영을 그쳐도 상태를 복원해 네트워크 처리를 이어 갈 수 있다.」[9]

[중급] 데이터를 올리는 자리는 둘입니다. 거래의 `calldata`(13장의 거래 데이터 필드, 영구히 체인 역사에 남음)와 **블롭(blob)입니다**. 블롭은 2024년 3월 Dencun 포크(EIP-4844)가 더한 「실행층이 저장하지 않는」 데이터 공간으로, 별도의 수수료 시장에서 값이 매겨지며 「프로토콜은 네트워크가 블롭 데이터를 제공해야 하는 **최소 기간을** 대략 18일로만 정한다. 그것은 제공 의무이지 삭제 기한이 아니다. 블롭 데이터는 실행층 역사처럼 영구 보관되는 것이 아니며, 장기 접근은 보관 서비스에 달려 있다.」[10] 그 「대략 18일」은 합의층 설정의 `MIN_EPOCHS_FOR_BLOB_SIDECARS_REQUESTS: 4096` 에포크(에포크당

32슬롯 × 12초 = 6.4분, 4,096에포크 ≈ 18.2일)에서 옵니다(**2026-10 기준**, 부록 A).[11] 13장에서 저장소 쓰기가 왜 비싼지 보았습니다. 블롭은 「모든 노드가 영구히 들고 있지는 않는」 자리를 따로 만들어 롤업 데이터를 싸게 올리게 한 설계이고, 그 대가가 「18일 뒤에는 누가 들고 있는가」라는 새 질문입니다.

3. 낙관적 롤업 — 틀렸다고 증명될 때까지 옳다

첫째 방식은 「낙관」합니다. 「낙관적 롤업은 체인 밖 거래가 유효하다고 **가정하고**, 체인에 올리는 거래 묶음에 유효성 증명을 게시하지 않는다는 점에서 「낙관적」이라 불린다. … 대신 낙관적 롤업은 거래가 올바르게 계산되지 않은 경우를 탐지하는 **사기 증명(fraud proof)** 체계에 의존한다. 롤업 묶음이 이더리움에 제출된 뒤, 누구나 사기 증명을 계산해 롤업 거래의 결과에 이의를 제기할 수 있는 시간 창(**이의 기간**)이 있다.」 이의가 성공하면 「롤업 프로토콜은 거래를 다시 실행해 롤업의 상태를 갱신하고 … 잘못 실행된 거래를 블록에 넣은 시퀀서는 벌을 받는다.」 이의가 없이 기간이 지나면 「유효하다고 간주되어 이더리움에 받아들여진다.」[12] 10장의 슬래싱, 9장의 「시간이 지나야 굳는다」가 여기서 다시 쓰입니다. 다만 9장의 굳음은 확률이었고 여기서는 **이의 기간이라는 시계입니다**.

그 시계가 사용자에게 보이는 자리는 **인출입니다**. 「레이어 2에서 레이어 1로 자금을 인출하는 거래를 시작하면 사용자는 이의 기간 — **대략 7일** — 이 지날 때까지 기다려야 한다.」(**2026-10 기준**, 부록 A)[13] 레이어 2 안에서는 빠르지만, 본점으로 돌아가는 데에는 이의 기간이 붙습니다. 기다리지 않으려면 「유동성 공급자가 대기 중인 인출을 인수하고 수수료를 받고 레이어 1에서 사용자에게 지급」하는 길이 있는데, 그것은 또 다른 상대방을 믿는 일입니다.

사기 증명은 어떻게 작동하는가 — 이분법

[중급] 「틀렸다」를 어떻게 증명할까요. 거래 전부를 본체에서 다시 돌리면 비용이 너무 큽니다. 그래서 대화형 **이분법을** 씁니다. 아비트럼(Arbitrum) 문서의 설명이 가장 선명합니다. 「앨리스와 밥이 L1 계약이 중재하는 주고받기 프로토콜을 벌여, L1 계약이 해야 할 일을 최소로 하면서 분쟁을 해결한다는 것이 대화형 증명의 착상이다. 아비트럼의 접근은 분쟁을 **해부하는** 것이다. 앨리스의 주장이 실행 N 단계를 덮는다면, 그는 합치면 원래의 N 단계 주장이 되는 $N/2$ 크기의 주장 둘을 올리고, 밥은 그중 하나를 골라 이의한다. 이제 분쟁의 크기가 반으로 줄었다. 이 과정이 단계마다 반을 잘라 가며 계속되어, 둘이 **실행 한 단계에** 관해 다투게 될 때까지 이어진다. 지금까지 L1 중재자는 실행을 「본안으로」 생각할 필요가 없었다. 분쟁이 한 단계로 좁혀진 뒤에야 L1 중재자는 그 명령이 실제로 무엇을 하는지, 앨리스의 주장이 옳은지를 보고 분쟁을 해결한다.」[14] N 단계의 분쟁이 $\log_2 N$ 번의 왕복으로 한 단계로 줄어듭니다. 6장의 머클 증명이 $\log_2 n$ 개의 해시로 포함을 증명한 것과 같은 수학입니다.

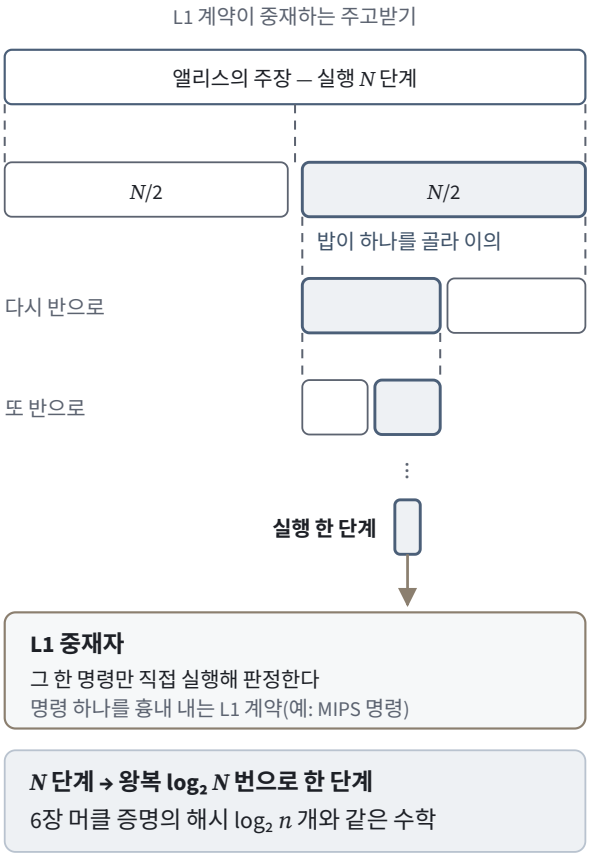


그림 18-2. 대화형 사기 증명의 이분법. 주장을 반씩 잘라 실행 한 단계에 이르면 L1 중재자가 그 한 명령만 실행한다. 왕복은 $\log_2 N$ 번이다.

[중급] OP Stack 명세는 같은 장치를 부품 셋으로 적습니다. 「사기 증명(fault proof)은 … 세 부품으로 이루어진다. 프로그램: 모든 롤업 입력(L1 데이터)에 대한 약정과 분쟁이 주어지면 분쟁을 무상태로 검증한다. VM: 무상태 프로그램과 그 입력이 주어지면 어떤 명령 단계든 추적하고 L1에서 증명한다. 대화형 분쟁 게임: 분쟁을 단일 명령으로 이분하고, 기저 사례를 VM으로 해결한다.」[15] 「L1에서 한 명령을 증명한다」는 것은, 레이어 2 가상머신의 명령 하나(예: MIPS 명령 하나)를 흉내 내는 **계약을 레이어 1에 두고** 그 한 단계만 본체에서 돌려 본다는 뜻입니다. 13장의 EVM 위에 다른 기계의 한 명령을 재현하는 계약이 올라가 있는 셈입니다.

[중급] 이 설계의 보안 명제는 하나로 요약됩니다. **정직한 사람이 한 명만 있으면 된다.** 아비트럼 문서: 「롤업 프로토콜의 핵심 보안 성질은 **정직한 검증자 한 명이** 체인의 올바른 실행이 확인되도록 강제할 수 있다는 것이다. 이는 아비트럼 체인을 실행하는 것이 이더리움 체인을 실행하는 것만큼 신뢰 불요라는 뜻이다. 당신, 그리고 당신 혼자(또는 당신이 고용한 누군가)가 당신의 거래가 올바르게 처리되도록 보장할 수 있다. 그리고 이것은 얼마나 많은 악의적 사람이 당신을 막으려 하든 참이다.」 그리고 BoLD라는 분쟁 프로토콜은 그 보증에 **시간 상한을** 붙였다고 적습니다. 「단일 정직한 당사자가 상대가 몇 명이든, 얼마를 담보로 걸든 **알려진 고정된 시간 안에** 올바른 결과의 확인을 강제할 수 있다.」[16] 공식 문서도 같은 것을 적습니다. 「체인의 유효성은 **정직한 노드 하나의** 존재에 의존한다. … 정직한 노드가 없으면 악의적 운영자가 유효하지 않은 블록과 상태 약정을 올려 자금을 훔칠 수 있다.」[17] 10장의 「3분의 2」와 대조해 보십시오. 낙관적 롤업은 다수결이 아니라 **한 사람의 이의로** 돌아갑니다. 그 한 사람이 실제로 깨어 있는지, 이의를 제기할 담보와 기술을 가졌는지가 가정입니다.

[중급] 한 가지 더. 사기 증명이 뒤집는 것은 **체인이 아니라 주장입니다**. OP Stack 문서: 「사기 증명 체계가 다루는 것은 셋째 행(출력 루트)만이다. 그것은 L2 상태에 관한 주장을 L1이 받아들일지 결정하며, L2 체인의 순서를 바꾸지 않는다. 패배한 제안은 체인에 관한 주장 하나를 제거한다. 체인을 되감지 않는다.」[18] 거래 데이터가 본체에 올라간 순간 순서는 굳었고(아비트럼 문서의 「하드 최종성」), 사기 증명은 그 데이터에서 **계산한 결과가** 맞는지만 다룹니다.

4. 영지식 롤업 — 옳다는 증명을 매번 첨부한다

둘째 방식은 기다리지 않습니다. 「영지식 롤업은 거래를 묶어 체인 밖에서 실행한다. … 롤업 운영자는 각 거래를 따로 보내는 대신 묶음의 모든 거래를 나타내는 데 필요한 **변경의 요약을** 제출한다. 또 그 변경이 올바르다는 것을 증명하는 **유효성 증명(validity proof)을** 만든다. 롤업의 상태는 이더리움에 배포된 스마트계약이 유지한다. 이 상태를 갱신하려면 롤업 노드는 검증을 위해 유효성 증명을 제출해야 한다. … 유효성 증명은 롤업이 제안하는 상태 변화가 정말로 주어진 거래 묶음을 실행한 결과라는 암호학적 보증이다.」[19] 본체의 **검증자 계약이** 증명을 확인하고 통과하면 새 상태 루트가 정본이 됩니다. 거의 모든 것을 다시 계산하는 대신 「계산이 옳았다는 짧은 증명」을 확인하는 것이고, 그래서 「영지식 롤업에서 이더리움으로 자금을 옮기는 데 지연이 없다. 롤업 계약이 유효성 증명을 검증하면 인출 거래가 실행된다.」[20]

이름에 관해 공식 문서가 적어 둔 경고를 옮깁니다. 「영지식」이라는 말 때문에 거래가 **비밀이** 된다고 오해하기 쉽습니다. 「영지식 롤업은 체인 밖 상태 전이가 올바르게 계산되었음을 확립하기 위해 **간결한(succinct)** 유효성 증명을 쓴다. 간결성은 증명을 작게, 검증을 기반 계산에 비해 효율적으로 유지해 이더리움이 모든 거래를 다시 실행하지 않고도 상태 전이를 검증할 수 있게 한다. 증명 체계는 추가로 영지식을 제공할 수 있는데, 이는 증명이 공개 명제에서 따라 나오는 것 외에 사적 입력에 관한 정보를 드러내지 않는다는 뜻이다. **이 둘은 별개의 성질이다. 확장을 위해 유효성 증명을 쓰는 것이 그 자체로 롤업의 거래를 기밀로 만들지는 않는다.**」[21] 롤업이 이 기술에서 쓰는 것은 「짧다」는 성질이고, 「숨긴다」는 성질은 덤이거나 쓰지 않습니다. 데이터는 2절대로 본체에 공개됩니다. 「게시된 데이터는 공개다. … 이 데이터를 게시하는 것이 무허가 검증을 가능하게 하는 것이며, 그것은 프라이버시 메커니즘이 아니다.」

[중급] 증명 체계는 크게 둘입니다. SNARK는 「증명 크기가 작고 검증 시간이 상수」이지만 「공통 참조 문자열(CRS)을 만드는 **신뢰 설정이** 필요하다. 공개 매개변수를 만드는 데 쓰인 정보가 악의적 행위자의 손에 들어가면 거짓 유효성 증명을 만들 수 있다.」 그래서 여러 참가자가 무작위성을 보태고 각자 자기 몫을 파기하는 의식(MPC ceremony)을 하고, 「한 명의 정직한 참가자라도 자기 입력을 파기하면 체계의 보안은 보장된다.」 STARK는 그 설정이 없이 「공개적으로 검증 가능한 무작위성」을 쓰고 양자 컴퓨터에 안전하다고 여겨지지만 「증명 크기가 더 커서 이더리움에서 검증하는 비용이 더 든다.」[22] 증명의 수학은 이 책의 범위 밖입니다(부록 E도 다루지 않습니다). 법률가가 가져갈 것은 하나입니다. 「옳다는 증명」에도 **믿어야 하는 전제가** 있다는 것 — 어떤 체계에서는 설정 의식의 참가자 중 한 명이 정직했다는 것, 모든 체계에서는 증명 회로가 실제 가상머신을 **정확히** 옮겼다는 것입니다. 공식 문서는 「범용 EVM 계산을 회로로 증명하는 것은 단순 계산을 증명하는 것보다 훨씬 어렵고 자원이 많이 든다」고 적습니다.[23]

[중급] ZKsync 문서는 그 부품을 넷으로 셉니다. 「노드 구현: 사용자에게서 거래를 받아 처리한다. 체인 밖 상태를 유지하고 거래를 묶음으로 모아 봉인된 묶음을 체인에 보낸다. ZK 회로: 검증 가능한 계산 논리를

나타내는 복잡한 수학적 구성물로, 무엇이 유효한 증명으로 검증될 수 있는지를 정한다. … 증명자(Prover): 체인 밖에서 처리된 거래의 올바름을 증언하는 암호학적 증명을 구성한다. … 스마트계약: zk롤업의 체인 위 부품으로, 증명자가 제출한 증명을 검증하고 이더리움 블록체인의 상태를 그에 따라 갱신한다.」[24] 넷 가운데 증명자는 무거운 기계가 필요합니다. 공식 문서의 단점 표: 「유효성 증명을 만드는 데 특수 하드웨어가 필요해 소수에 의한 체인 지배를 부추길 수 있다.」[25] 낙관적 롤업이 「누구나 이의할 수 있다」로 분산을 얻는 자리에서, 영지식 롤업은 「증명을 만들 수 있는 사람이 적다」는 다른 집중을 얻습니다.

5. 두 방식의 비교 — 누가 정직해야 하는가

표는 공식 문서 두 쪽의 장단점 표를 이 책이 「믿어야 하는 것」 기준으로 재배열한 것입니다.[26]

	낙관적 롤업	영지식 롤업
올바름의 근거	이의 기간 안에 아무도 사기 증명을 내지 않았다	검증자 계약이 유효성 증명을 통과시켰다
믿어야 하는 것	**정직한 노드 하나가** 깨어 있고 이의할 수 있다	증명 체계의 전제(설정 의식·회로의 정확성)와 그 수학
본체로 인출	이의 기간(대략 7일, 2026-10 기준)을 기다림	증명 검증 즉시
본체에 올리는 데이터	거래 데이터 전부(이의자가 재실행해야 하므로)	상태 재구성에 필요한 최소 데이터(원문: 「the minimal data required to rebuild the latest state」)
누가 체인을 전진시킬 수 있나	보통 노드 누구나(사기 증명 계산은 특수 하드웨어 불요)	증명을 만들 수 있는 자(특수 하드웨어)
EVM 호환	쉬움(EVM을 그대로 쓰거나 닮게)	어려움(EVM을 회로로 옮겨야)
공통의 약점	시퀀서가 거래 순서에 영향을 줄 수 있음	같음

6. 신뢰는 사라지지 않고 옮겨 간다

마지막 행이 이 장을 17장과 묶습니다. 두 방식 모두 거래를 받아 순서를 정하고 묶음을 만드는 **시퀀서가** 있습니다. OP Stack 문서는 「시퀀서의 대기열은 L1 이더리움의 공개 대기열과 달리 **비공개이며**, 이는 공개 대기 거래 풀이 만드는 MEV 기회를 제한한다」고 적습니다.[27] 17장의 선행매매는 거래가 공개 대기열에 보이는 데서 나왔습니다. 대기열을 한 사업자가 비공개로 들고 있으면 봇의 선행매매는 줄지만, **그 사업자가 순서를 정합니다.** 같은 문서가 신뢰의 경계를 그렇게 적습니다. 「신뢰 경계는 거래가 나이를 먹으면서 옮겨 간다. 데이터가 게시되기 전에 사용자는 시퀀서가 자기 거래를 재배열하거나 떨어뜨리지 않을 것을 믿고 있다. 그 뒤에는 데이터 가용성 층을 믿고 있다.」[28] 아비트럼 문서도 같은 것을 두 낱말로 가릅니다. 시퀀서가 실시간으로 내보내는 순서는 「시퀀서가 약속을 지키는 데 달려 있어」 **소프트 최종성이고**, 그 순서가 이더리움에 묶음으로 올라가 굳은 뒤가 **하드 최종성입니다.**[29] 9장의 「최종성」이 여기서 다시 두 겹이 됩니다.

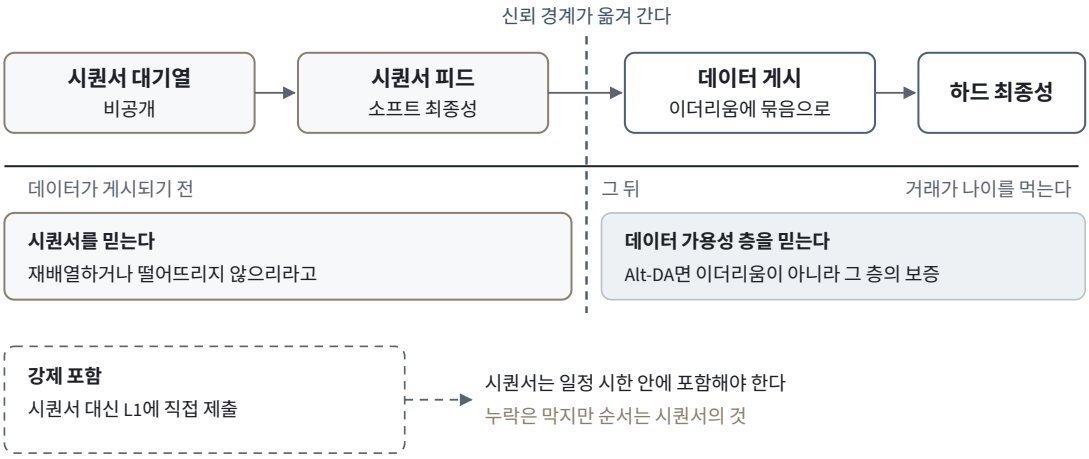


그림 18-3. 거래가 나이를 먹으며 옮겨 가는 신뢰 경계. 데이터가 이더리움에 게시되기 전에는 시퀀서를, 그 뒤에는 데이터 가용성 층을 믿는다. 강제 포함은 누락을 막지만 순서는 시퀀서가 정한다.

시퀀서가 거래를 **누락하는** 것에는 방어가 있습니다. 2절 표의 둘째 행, 강제 포함입니다. 「사용자는 시퀀서 대신 L1에 직접 거래를 제출할 수 있고, 그 경우 시퀀서는 유효한 블록을 계속 만들기 위해 일정 시한 안에 그 거래를 포함해야 한다.」[30] 누락은 막을 수 있지만 **순서는** 시퀀서의 것입니다. 그리고 데이터 가용성 층을 이더리움이 아닌 다른 것으로 바꾸는 선택(「Alt-DA」)을 하면 「체인의 보안은 이더리움의 것이 아니라 그 층의 가용성 보증의 함수가 된다」고 적고, 그 경우 「누구나 이더리움에서 체인을 재구성할 수 있다는 주장은 더 이상 성립하지 않는다」고 덧붙입니다.[31] 롤업은 신뢰를 0으로 만든 것이 아니라 **본체에 맡길 것과 사업자가 쥘 것을 나눈** 설계이고, 그 경계선은 롤업마다 다릅니다. 어느 롤업이 어디까지 본체에 맡겼는지, 업그레이드 열쇠는 누가 쥐었는지 같은 **현황은** 빨리 바뀌므로 본문에 적지 않고 부록 A 동향 노트의 몫으로 돌립니다(기준일과 출처를 붙여서).

7. 정리 — 2부를 닫으며

롤업은 실행을 밖으로 빼고 데이터와 상태 요약을 본체에 올립니다. 낙관적 롤업은 「정직한 한 사람의 이의」와 이의 기간으로, 영지식 롤업은 「옳다는 짧은 증명」과 그 증명 체계의 전제로 올바름을 담보합니다. 둘 다 시퀀서가 순서를 정하고, 둘 다 데이터가 어디에 있는가에 보안이 걸려 있습니다. 「트릴레마」는 그 긴장을 부르는 이름이지 정리가 아닙니다.

2부를 돌아봅니다. 13장은 코드를 실행하는 원장과 그것을 멈추는 가스를, 14장은 그 위의 장부(토큰)와 장부가 말하지 않는 것을, 15장은 사람 없이 토큰을 맞바꾸는 통을, 16장은 체인이 바깥을 모른다는 사실을, 17장은 순서를 정하는 권력을, 18장은 실행을 다른 층으로 옮기는 구조를 보았습니다. 여섯 장에 걸쳐 되풀이된 것은 하나입니다. 코드가 집행하는 것과 사람이 약속한 것은 **다른 층에** 있고, 「탈중앙」이라는 말이 지우지 못한 자리(운영자, 오라클, 시퀀서, 열쇠)가 그 둘 사이에 남습니다. 3부는 그 구조를 **경제학으로** 다시 읽습니다. 19장은 왜 아무것도 뒷받침하지 않는 토큰에 양의 가격이 성립할 수 있는지를, 20장은 이 모든 안전이 매 블록 얼마에 유지되는지를, 21장은 13장에서 미뤄 둔 수수료의 산식을 다룹니다.

이 장의 출처

- ethereum.org, "Scaling" (개발자 문서). https://ethereum.org/en/developers/docs/scaling/ ; 소스 github.com/ethereum/ethereum-org-website 커밋 31bb20f8(2026-10-22). 2026-10-25 접근 — 확장의 목표와 제약, 샤딩에서 롤업으로, 롤업 정의.
- ethereum.org, "Optimistic Rollups." https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/ ; 소스 커밋 3e3b0dec. 2026-10-25 접근 — 정의, 부품, 데이터 가용성·검열 저항, 이의 기간과 인출, 대화형 증명, 정직한 노드 하나, 장단점 표.
- ethereum.org, "Zero-knowledge rollups." https://ethereum.org/en/developers/docs/scaling/zk-rollups/ ; 소스 커밋 38c32517. 2026-10-25 접근 — 정의, 유효성 증명과 간결성·영지식의 구별, SNARK·STARK, zkEVM, 블롭 제공 기간, 장단점 표.
- Optimism Docs, "OP Stack architecture." https://docs.optimism.io/op-stack/architecture (2026-10-25 접근) — 세 가지 게시물과 그 뜻, 비공개 대기열, 신뢰 경계의 이동, 사기 증명은 체인을 되감지 않음, Alt-DA.
- OP Stack Specification, "Fault Proof," github.com/ethereum-optimism/specs 커밋 b733da80. https://github.com/ethereum-optimism/specs/blob/b733da804eeadc597c690f16601091e97af1fc68/specs/fault-proof/index.md — 사기 증명의 세 부품, 단일 명령 검증 계약.
- Arbitrum Docs, "Inside Arbitrum Nitro." https://docs.arbitrum.io/how-arbitrum-works/inside-arbitrum-nitro (2026-10-25 접근) — 대화형 증명(이분법), 정직한 검증자 하나·BoLD의 시간 상한, 소프트·하드 최종성.
- ZKsync Docs, "ZKsync protocol overview." https://docs.zksync.io/zksync-protocol/rollup (2026-10-25 접근) — 처리량 병목, 데이터 가용성 문제, 네 부품.
- ethereum/consensus-specs 커밋 b834fa84: `configs/mainnet.yaml` L74·L118·L194, `presets/mainnet/phase0.yaml` L34 — 슬롯 간격, 블롭 제공 최소 에포크, 에포크당 슬롯.
- 이 책의 `widgets/data.json` (as_of 2026-10-25) — 비트코인 블록 가중치·목표 간격, 이더리움 관측 가스 한도.
- 이 장에는 코드가 없다(노트북도 없다). 롤업별 현황·TVL 같은 수치는 본문에 적지 않고 부록 A 동향 노트로 보낸다. 세 문서 사이트(Optimism·Arbitrum·ZKsync)는 내용이 바뀔 수 있어 접근일을 붙였고, 축어 인용은 2026-10-25에 받은 본문에서 했다.

각주

1. ZKsync Docs, "ZKsync protocol overview," 「How and why it works?」. 원문: "Ethereum's decentralized network has limited transaction throughput because its capacity doesn't scale with the number of validators. In essence, each validator performs the same job of validating each processed transaction, creating a bottleneck. In contrast, in Web2 scalability improves with the addition of more servers, as the network's capacity to handle requests increases linearly with each new server." https://docs.zksync.io/zksync-protocol/rollup (2026-10-25 접근; 문서 사이트라 내용이 바뀔 수 있음). 이 책은 ZKsync 문서를 영지식 롤업 설계의 **문헌 출처로** 인용하며 어떤 서비스를 권하거나 비교하지 않습니다(3장 일러두기).

2. 비트코인: Bitcoin Core v31.1 `src/consensus/consensus.h` L15(`MAX_BLOCK_WEIGHT = 4000000`),

`src/kernel/chainparams.cpp` L97–L98(`nPowTargetSpacing = 10*60`) — 이 책의 `widgets/data.json bitcoin` 항목과 8장. 이더리움 슬롯 간격: ethereum/consensus-specs `configs/mainnet.yaml`(커밋 b834fa84, 2026-10-23) L74 `SLOT_DURATION_MS: 12000`. https://github.com/ethereum/consensus-specs/blob/b834fa8411cb2152563e8835c5b72c28f7503e7a/configs/mainnet.yaml (2026-10-25 접근). 같은 파일 L118에는 `SLOT_DURATION_MS_EIP8198: 10000` 이라는 값이 따로 적혀 있어 슬롯 간격을 바꾸는 개정이 준비되어 있는 것으로 보이나, 같은 파일 L69의 `EIP8198_FORK_EPOCH` 는 최댓값(2^64−1, 다른 미활성 포크와 같은 표기)으로 적혀 있어 기준일에는 활성 에포크가 지정되지 않은 것으로 보이며, 이 책은 그 내용을 확인하지 않았으므로 본문은 기준일에 적용 중인 12초로 씁니다(부록 A 색인).

3. 이 책의 `widgets/data.json` `ethereum.eip1559.observed_latest_block` : 블록 26,264,275(2026-10-24T18:23Z 관측) `gas_limit` 60,000,000, 공개 RPC 세 곳 응답 일치. 가스 한도는 「프로토콜 상수가 아니라 검증자가 블록마다 부모의 ±1/1024 범위에서 움직이는 값」(go-ethereum v1.17.10 `params/protocol_params.go` L26 `GasLimitBoundDivisor = 1024`). 상수 쪽 정본은 21장.

4. ethereum.org, "Scaling," 「Scaling overview」. 원문: "The main goal of scalability is to increase transaction speed (faster finality) and transaction throughput (higher number of transactions per second) without sacrificing decentralization or security."; "While speed and throughput are important, it is essential that scaling solutions enabling these goals remain decentralized and secure. Keeping the barrier to entry low for node operators is critical in preventing a progression towards centralized and insecure computing power." https://ethereum.org/en/developers/docs/scaling/ (소스 https://github.com/ethereum/ethereum-org-website/blob/31bb20f87637d0d5899bb5c871343c2981210f66/public/content/developers/docs/scaling/index.md , 마지막 커밋 2026-10-22; 2026-10-25 접근).

5. 같은 페이지, 「Onchain scaling」. 원문: "For a long time, sharding the blockchain was expected to scale Ethereum. This was going to involve splitting the blockchain into discrete pieces (shards) to be verified by subsets of validators. However, scaling by layer-2 rollups has taken over as the primary scaling technique. This is supported by the addition of a new cheaper form of data attached to Ethereum blocks that is specially designed to make rollups cheap for users."

6. 같은 페이지, 「Rollups」. 원문: "Rollups perform transaction execution outside layer 1 and then the data is posted to layer 1 where consensus is reached. As transaction data is included in layer 1 blocks, this allows rollups to be secured by native Ethereum security."

7. ethereum.org, "Optimistic Rollups," 「How do optimistic rollups interact with Ethereum?」. 원문: "**Onchain contracts**: The optimistic rollup's operation is controlled by smart contracts running on Ethereum. This includes contracts that store rollup blocks, monitor state updates on the rollup, and track user deposits. In this sense, Ethereum serves as the base layer or 'layer 1' for optimistic rollups. **Offchain virtual machine (VM)**: Although contracts managing the optimistic rollup protocol run on Ethereum, the rollup protocol performs computation and state storage on another virtual machine separate from the Ethereum Virtual Machine. The offchain VM is where applications live and state changes are executed; it serves as the upper layer or 'layer 2' for an optimistic rollup." https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/ (소스 커밋 3e3b0dec; 2026-10-25 접근).

8. Optimism Docs, "OP Stack architecture," 「Where data lives, and what you trust」. 원문: "Three different things go to the parent chain, and conflating them is the most common source of confusion about rollup security." 표: "Transaction data — Blobs or calldata at a DA address — Anyone can reconstruct the chain without the sequencer; Deposits — OptimismPortal — L1 can force transactions into L2; Output roots — DisputeGameFactory — L1 can authenticate L2 state for withdrawals." 이어서: "Transaction data is what makes the chain derivable. Output roots are what make it settleable. A chain that published data but no output roots would still be a chain anyone could verify, but no one could withdraw from it. A chain that published output roots but no data would be unverifiable, and its proposals uncheckable." https://docs.optimism.io/op-stack/architecture (2026-10-25 접근; 구 주소 /stack/rollup/overview 가 여기로 넘어옴. 문서 사이트라 내용이 바뀔 수 있음). 이 책은 이 문서를 낙관적 롤업 설계의 문헌 출처로만 인용합니다.

9. ZKsync Docs, 같은 페이지, 「The Data Availability (DA) Problem」. 원문: "If the state of the rollup is unknown to observers of Ethereum, then in scenarios where the validators (centralized or decentralized) stop processing, it becomes impossible to make state transitions without relying on a trusted validator. However, if the data is always available to observers, it's feasible to restore the state and continue processing the network even if the trusted validator ceases its operation."

10. ethereum.org, "Zero-knowledge rollups," 「How ZK-rollups publish transaction data on Ethereum」. 원문: "Since the Dencun upgrade introduced blob-carrying transactions (EIP-4844), rollups can also publish their data in blobs, which are priced in a separate fee market and are usually cheaper than `calldata`. Blobs are not stored by the execution layer, and the protocol defines only a minimum window of roughly 18 days over which the network must serve blob data. That is a serving obligation rather than a deletion deadline: blob data is not permanent archival storage the way execution-layer history is, and longer-term access depends on archival services. `calldata` remains available, and rollups differ in which they use." https://ethereum.org/en/developers/docs/scaling/zk-rollups/ (소스 커밋 38c32517; 2026-10-25 접근). Dencun 활성일(2024-03-13)은 10장의 포크 연표(consensus-specs `configs/mainnet.yaml` `DENEB_FORK_EPOCH` 주석).

11. ethereum/consensus-specs `configs/mainnet.yaml` (커밋 b834fa84) L194 `MIN_EPOCHS_FOR_BLOB_SIDECARS_REQUESTS: 4096`; `presets/mainnet/phase0.yaml` L34 `SLOTS_PER_EPOCH: 32`; L74 `SLOT_DURATION_MS: 12000`. 4,096 × 32 × 12초 = 1,572,864초 ≈ 18.2일(이 책의 계산). 2026-10-25 접근.

12. ethereum.org, "Optimistic Rollups," 「What is an optimistic rollup?」. 원문: "Optimistic rollups are considered 'optimistic' because they assume offchain transactions are valid and don't publish proofs of validity for transaction batches posted onchain. ... Optimistic rollups instead rely on a fraud-proving scheme to detect cases where transactions are not calculated correctly. After a rollup batch is submitted on Ethereum, there's a time window (called a challenge period) during which anyone can challenge the results of a rollup transaction by computing a fraud proof. If the fraud proof succeeds, the rollup protocol re-executes the transaction(s) and updates the rollup's state accordingly. The other effect of a successful fraud proof is that the sequencer responsible for including the incorrectly executed transaction in a block receives a penalty. If the rollup batch remains unchallenged (i.e., all transactions are correctly executed) after the challenge period elapses, it is deemed valid and accepted on Ethereum."

13. 같은 페이지, 「Exiting the rollup」. 원문: "If a user initiates an L2 > L1 transaction to withdraw funds escrowed on L1, they must wait until the challenge period—lasting roughly seven days—elapses."; "To avoid waiting a week before withdrawing funds to Ethereum, optimistic rollup users can employ a **liquidity provider** (LP). A liquidity provider assumes ownership of a pending L2 withdrawal and pays the user on L1 (in exchange for a fee)." 「대략 7일」은 2026-10-25 접근 시점의 서술이며 롤업마다·판마다 다를 수 있습니다.

14. Arbitrum Docs, "Inside Arbitrum Nitro," 「Resolving disputes using interactive fraud proofs — Interactive proving」. 원문: "The idea of interactive proving is that Alice and Bob will engage in a back-and-forth protocol, mediated by an L1 contract, to resolve their dispute with minimal work required from the L1 contract. Arbitrum's approach is to dissect the dispute. If Alice's claim covers N steps of execution, she posts two claims of size N/2, which combine to yield her initial N-step claim, then Bob picks one of Alice's N/2-step claims to challenge. Now the size of the dispute has been cut in half. This process continues, cutting the dispute in half at each stage, until they are disagreeing about a single step of execution. Note that so far, the L1 mediator hasn't had to think about execution 'on the merits.' It is only once the dispute is narrowed down to a single step that the L1 mediator needs to resolve the dispute by looking at what the instruction actually does and whether Alice's claim about it is correct." https://docs.arbitrum.io/how-arbitrum-works/inside-arbitrum-nitro (2026-10-25 접근; 문서 사이트). 이 책은 이 문서를 대화형 사기 증명의 문헌 출처로만 인용합니다.

15. OP Stack Specification, "Fault Proof," 「Overview」(커밋 b733da80). 원문: "A fault proof, also known as fraud proof or interactive game, consists of 3 components: Program: given a commitment to all rollup inputs (L1 data) and the dispute, verify the dispute statelessly. VM: given a stateless program and its inputs, trace any instruction step, and prove it on L1. Interactive Dispute Game: bisect a dispute down to a single instruction, and resolve the base-case using the VM." 「Fault Proof VM」 절: "a smart-contract to verify a single execution-trace step, e.g. a single MIPS instruction." https://github.com/ethereum-optimism/specs/blob/b733da804eeadc597c690f16601091e97af1fc68/specs/fault-proof/index.md (2026-10-25 접근).

16. Arbitrum Docs, 같은 페이지, 「Arbitrum Rollup protocol」. 원문: "The key security property of the rollup protocol is that any one honest validator can force the correct execution of the chain to be confirmed. This means that executing an Arbitrum chain is as trustless as executing an Ethereum chain. You, and you alone (or someone you hire), can ensure your transactions are processed correctly. And this is true no matter how many malicious people are trying to stop you. Under BoLD, this guarantee comes with a bounded delay: a single honest party can force confirmation of the correct outcome with a known, fixed amount of time, regardless of how many adversaries participate or how much they place as bond."

17. ethereum.org, "Optimistic Rollups," 「Why fraud proofs matter」 및 장단점 표. 원문: "This also relates to another security property of optimistic rollups: the validity of the chain relies on the existence of *one* honest node."; 표의 단점: "If there are no honest nodes a malicious operator can steal funds by posting invalid blocks and state commitments."; "Security model relies on at least one honest node executing rollup transactions and submitting fraud proofs to challenge invalid state transitions."

18. Optimism Docs, "OP Stack architecture," 「Where data lives, and what you trust」. 원문: "The fault proof system governs only the third row: it decides whether a claim about L2 state is accepted on L1, and it does not reorder the L2 chain. A defeated proposal removes a claim about the chain; it does not roll the chain back."

19. ethereum.org, "Zero-knowledge rollups," 「What are zero-knowledge rollups?」. 원문: "**Zero-knowledge rollups (ZK-rollups)** bundle (or 'roll up') transactions into batches that are executed offchain. ... ZK-rollup operators submit a summary of the changes required to represent all the transactions in a batch rather than sending each transaction individually. They also produce validity proofs to prove the correctness of their changes. The ZK-rollup's state is maintained by a smart contract deployed on the Ethereum network. To update this state, ZK-rollup nodes must submit a validity proof for verification. As mentioned, the validity proof is a cryptographic assurance that the state-change proposed by the rollup is really the result of executing the given batch of transactions."

20. 같은 페이지. 원문: "There are no delays when moving funds from a ZK-rollup to Ethereum because exit transactions are executed once the ZK-rollup contract verifies the validity proof. Conversely, withdrawing funds from optimistic rollups is subject to a delay to allow anyone to challenge the exit transaction with a fraud proof."

21. 같은 페이지, 「Validity proofs」. 원문: "ZK-rollups use succinct validity proofs to establish that offchain state transitions were computed correctly. Succinctness keeps proofs compact and verification efficient relative to the underlying computation, allowing Ethereum to verify state transitions without re-executing every transaction. A proof system may additionally provide zero knowledge, meaning its proofs reveal no information about private inputs beyond what follows from the public statement. These are separate properties: using validity proofs for scaling does not, by itself, make a rollup's transactions confidential." 데이터 공개: "Whichever mechanism a rollup uses, the published data is public. ... Publishing this data is what makes permissionless verification possible; it is not a privacy mechanism."

22. 같은 페이지, 「ZK-SNARKs」·「ZK-STARKs」. 원문: "For the ZK-SNARK protocol to work, creating a Common Reference String (CRS) is necessary: the CRS provides public parameters for proving and verifying validity proofs. The security of the proving system depends on the CRS setup; if information used to create public parameters fall into the possession of malicious actors they may be able to generate false validity proofs."; "As long as one honest participant destroys their input, the security of the ZK-SNARK system is guaranteed."; "ZK-SNARKs are popular for their small proof sizes and constant-time verification."; "ZK-STARKs are 'transparent', as they can work without the trusted setup of a Common Reference String (CRS). Instead, ZK-STARKs rely on publicly verifiable randomness"; "ZK-STARKs are also secure against quantum computers ... The downside to ZK-STARKs is that they produce larger proof sizes, which are more expensive to verify on Ethereum."

23. 같은 페이지, 「ZK-rollups and EVM compatibility」. 원문: "Unlike optimistic rollups, ZK-rollups are not readily compatible with the Ethereum Virtual Machine (EVM). Proving general-purpose EVM computation in circuits is more difficult and resource-intensive than proving simple computations (like the token transfer described previously)."

24. ZKsync Docs, 같은 페이지, 「ZKsync components」. 원문: "Node Implementation: This component is responsible for receiving transactions from users and processing them. It maintains the off-chain state and handles the aggregation of transactions into batches as well as sends sealed batches onchain. ZK Circuits: These circuits are intricate mathematical constructs that represent verifiable computation logic. They are responsible for determining what can be verified as a valid proof. ... Prover: The prover constructs the cryptographic proofs that attest to the correctness of the transactions processed off-chain. These proofs can be verified later on Ethereum, ensuring that only valid transactions are accepted. Smart Contracts: These contracts are the on-chain component of the zkRollup. They are responsible for verifying the proofs submitted by the prover and updating the Ethereum blockchain's state accordingly."

25. ethereum.org, "Zero-knowledge rollups," 장단점 표. 원문: "Producing validity proofs requires specialized hardware, which may encourage centralized control of the chain by a few parties."; "Hardware requirements

may reduce the number of participants that can force the chain to make progress, increasing the risk of malicious operators freezing the rollup's state and censoring users."

26. ethereum.org, "Optimistic Rollups," 「Pros and cons of optimistic rollups」; "Zero-knowledge rollups," 「Pros and cons of ZK-rollups」. 공통 약점의 원문: 낙관적 롤업 표 "Centralized rollup operators (sequencers) can influence transaction ordering."; 영지식 롤업 표 "Centralized operators (sequencers) can influence the ordering of transactions.". 낙관적 롤업의 장점: "Computing fraud proofs is open to regular L2 node, unlike validity proofs (used in ZK-rollups) that require special hardware."; 영지식 롤업의 장점: "Relies on trustless cryptographic mechanisms for security, not the honesty of incentivized actors as with optimistic rollups."

27. Optimism Docs, "OP Stack architecture," 「Life of a transaction」. 원문: "The sequencer's mempool is private, unlike L1 Ethereum's public one, which limits the MEV opportunities a public pending-transaction pool creates. Ordering is decided by the execution client, not the rollup node."

28. 같은 페이지, 「Where data lives, and what you trust」. 원문: "The trust boundary moves as a transaction ages. Before its data is published, a user is trusting the sequencer not to reorder or drop it. After, they are trusting the DA layer."

29. Arbitrum Docs, "Inside Arbitrum Nitro," 「How the Sequencer publishes the sequence」. 원문: "The feed represents the Sequencer's promise to record transactions in a particular order. If the Sequencer is honest and doesn't have a long downtime, this promise will be kept. ... This is 'soft finality' for transactions; it's 'soft' because it depends on the Sequencer keeping its promises."; "This is the final and official record of the transaction sequence. As soon as this Ethereum transaction reaches finality, the Layer 2 Nitro transactions it records will also have finality. ... This is 'hard finality.'"

30. ethereum.org, "Optimistic Rollups," 「Censorship resistance」. 원문: "Users can also submit their transactions on L1 instead of to the sequencer, in which case the sequencer has to include the transaction within a certain time limit to continue to produce valid blocks."

31. Optimism Docs, "OP Stack architecture," 「Topology options」. 원문: "Custom data availability. An Alt-DA layer instead of Ethereum blobs. Cheaper, and the chain's security becomes a function of that layer's availability guarantees rather than Ethereum's. The feature is experimental, and with it the claim that anyone can reconstruct the chain from Ethereum no longer holds: only a commitment reaches L1". 같은 절은 「표준 체인」을 「이더리움 데이터 가용성, 가스는 ETH, 거버넌스에 의한 업그레이드」로 적습니다.

3부 경제학

19장 화폐의 기능론과 「내재가치 0」 균형

핵심 TAKEAWAY

「내재가치가 없으니 결국 0이 된다」와 「발행량이 정해져 있으니 가치가 있다」는 둘 다 경제학의 결론이 아닙니다. 교환매개·가치척도·가치저장이라는 기능론은 점검표이지 가치의 증명이 아니고, 배당이 없는 물건의 양(+)의 가격은 「남들이 나중에 받아 줄 것」이라는 믿음이 스스로를 실현하는 **균형들 가운데 하나로** 성립합니다. 이 장에서 읽는 세 모형(Garratt–Wallace, Schilling–Uhlig, Biais 외)은 모두 가격 0인 균형을 전제하거나 적고(실링–울리히는 §7의 역방향 자기소거로) 양의 가격 균형이 그와 **나란히** 존재한다고 말하며, 양의 가격 균형 안에서도 가격 경로가 하나로 정해지지 않는다고 말합니다. 균형의 열거는 전망이 아닙니다. 이 장을 읽으면 판결문이나 논문에서 「가상자산은 내재가치가 없다」는 문장을 만났을 때, 그것이 경제학적으로 무엇을 말하고 무엇을 말하지 않는지 가를 수 있습니다.

학습목표

1. 화폐의 세 기능(교환매개·가치척도·가치저장)으로 암호자산을 점검하되, 그 점검이 가치의 유무를 판정하는 장치가 아님을 안다. 내부화폐·외부화폐의 구별을 쓸 수 있다.
2. [중급] 교환방정식 $MV = PT$ 가 무엇을 말하고, 유통속도 V 를 관측할 수 없을 때 왜 가격 설명 도구로 쓰기 어려운지 안다.
3. [심화] 세대중첩 모형(Garratt–Wallace)에서 가격 0 균형·일정 가격 균형·태양흑점 붕괴 균형이 왜 함께 존재하는지, 무한 생존 모형(Schilling–Uhlig)에서 「기본 가격식」이 왜 마팅게일 조건이 되는지, 거래편익 모형(Biais 외)에서 「기본가치」가 왜 가격 자체에 의존하는지 식으로 읽는다.
4. 세 모형이 공통으로 말하는 것(균형의 다수성·자기실현적 믿음)과 말하지 않는 것(어느 균형이 실현되는가, 가격의 방향)을 구별한다.

[입문]만 읽은 독자가 가져가는 것. 「내재가치가 없으니 사기」도 「희소하니 가치 있음」도 경제학의 결론이 아닙니다. 경제학이 말하는 것은 **양쪽 다 균형일 수 있다는** 것이고, 어느 쪽이 실현되는지는 모형이 정해 주지 않습니다.

1. 이 장이 답하려는 질문

1부에서 우리는 코인이 무엇인지 바이트 단위로 보았습니다. 서명으로 이어지는 출력의 사슬(5장·11장)이고, 그 사슬을 다시 쓰려면 작업증명을 다시 해야 합니다(7장·8장). 어디에도 「가치」는 없습니다. 발행자가 무엇을 돌려주겠다는 약속도 없습니다. 그런데 사람들은 그것을 재화와 바꿉니다. 왜 그럴 수 있는가, 그리고 그것이 언제까지 가능한가. 이 질문에 대한 흔한 두 답은 이렇습니다. 하나는 「배당도 상환도 없으니 기본가치가 0이고, 가격은 결국 거기로 간다」이고, 다른 하나는 「발행량 상한이 코드로 정해져 있으니 희소하고, 희소하니 가치가 있다」입니다.

경제학은 두 답 모두를 그대로 받지 않습니다. 첫째 답은 반만 맞습니다. 가격 0은 **하나의** 균형입니다. 그러나 양의 가격 균형도 있고, 둘 중 어느 것이 실현되는지를 이론이 정해 주지 않습니다. 둘째 답은 전제가 빠져 있습니다. 희소성은 「남들이 받아 준다면」 가치를 떠받치지만, 그 「받아 준다면」 자체를 만들어 주지는 않습니다. 희소한데 아무도 받지 않는 물건은 세상에 많습니다. 이 장은 이 두 답의 자리에 경제학이 실제로 내놓은 것, 곧 **균형의 열거를** 놓습니다. 그것은 전망이 아니고, 이 장은 어디에서도 가격을 말하지 않습니다.

2. 기능론 — 점검표이지 증명이 아니다

화폐를 정의할 때 경제학 교과서가 쓰는 틀은 기능 셋입니다. 고튼(Gary Gorton)과 장(Jeffery Zhang)의 2023년 논문은 이렇게 적습니다. 「화폐는 몇 가지 중요한 성질을 갖는데, 가장 흔히 꼽히는 셋은 **계산단위, 가치저장, 교환매개다**.」 그리고 넷째를 더합니다. 「가장 분명하면서도 명시되지 않는 성질은 화폐가 「**묻지 않고 받음(no-questions-asked, NQA)**」 **원칙도** 충족해야 한다는 것이다. 그 가치에 대한 실사 없이 거래에서 받아들여져야 한다.」[1] 법 쪽 문서도 같은 틀을 씁니다. 미국 SEC의 2017년 The DAO 보고서는 각주에서 FATF의 정의를 옮기는데, 「가상통화」를 「디지털로 거래될 수 있고 (1) 교환매개, 그리고/또는 (2) 계산단위, 그리고/또는 (3) 가치저장으로 기능하지만 어느 법역에서도 법정통화 지위를 갖지 않는 가치의 디지털 표상」으로 정의합니다.[2] 「그리고/또는」에 눈이 갑니다. 세 기능을 모두 갖추어야 한다는 것이 아니라, 하나라도 하면 된다는 정의입니다.

이 틀로 암호자산을 점검하면 결과는 뻔합니다. 교환매개로는 쓰이되 좁게 쓰이고, 가치척도로는 거의 쓰이지 않으며(가격이 다른 통화로 매겨집니다), 가치저장으로는 변동이 큽니다 — 정성적 요약이고, 그 실측은 아래 3절의 마카로프–쇼어·애시 외입니다. 2015년 논문 하나는 이 점검을 하고서 「비트코인이 화폐로 기능하려면 계산단위, 지급수단, 가치저장의 역할을 해야 한다」고 전제한 뒤 투기적 측면이 다른 기능을 압도해 가격이 시장 심리에 좌우되고, 그것이 계산단위 기능을 깎는다고 적습니다.[3] 그런데 이 점검이 끝나도 처음 질문은 남습니다. 기능을 잘 못한다는 것과 가격이 0이어야 한다는 것은 다른 명제입니다. 기능론은 **무엇을 화폐라 부를지** 정하는 점검표이지, 어떤 물건이 양의 가격을 가질 수 있는지를 판정하는 장치가 아닙니다. 가격의 질문에 답하려면 다른 도구가 필요합니다.

내부화폐와 외부화폐

그 도구로 가기 전에 구별 하나를 가져옵니다. 개럿(Rodney Garratt)과 월리스(Neil Wallace)의 2018년 논문은 비트코인을 어떻게 볼 것인가를 이 구별로 시작합니다. 「대부분의 경제학자는 내부화폐와 외부화폐를 구별한다. 내부화폐는 각 단위가 누군가의 자산이자 다른 누군가의 부채라는 뜻에서 경제 안에 있다. … 수표 계좌와 식료품점 쿠폰이 그 예다. 반대로 외부화폐는 대차대조표를 합쳐도 사라지지 않는다. 금본위제 아래의 금화와 1971년 8월 15일 닉슨이 금 창구를 닫은 뒤의 연방준비은행권이 그 예다. 이 이분법으로 보면 비트코인은 외부화폐로 보는 것이 가장 낫다. 특히 비트코인의 발행자는 그것을 다른 어떤 물건으로도 상환하겠다는 약속을 하지 않는다.」[4] 11장에서 본 것이 바로 이것입니다. UTXO는 누구의 부채도 아닙니다. 그래서 이 장의 질문은 「상환 약속 없는 물건이 왜 가격을 갖는가」가 되고, 그것은 금화와 불환지폐에 대해 경제학이 오래 물어 온 질문과 같은 질문입니다.

같은 논문이 모형의 전제를 한 문장으로 적습니다. 「사람들이 다른 것을 내주고 비트코인을 얻는 것은 오직 **미래에 다른 사람들도 그렇게 하리라고 생각하기 때문이다**. 다시 말해 비트코인의 소유는 피카소의 소유처럼 효용을 주지 않고, 농지·공장·3D 프린터처럼 다른 것의 생산에 투입되지도 않으며, 다른 가치 있는 물건의 배당 흐름을 받을 권리도 주지 않는다.」 그리고 덧붙입니다. 「비트코인에는 무시할 상품 가치조차 없다. 벽지로도 쓸 수 없다 — 1922–1923년 초인플레이션 때 독일 마르크로는 그렇게 했다는 이야기가 있지만.」[5] 흄과 수량설의 창시자들이 금은화폐의 상품 가치를 **무시하고** 분석했다면, 비트코인은 무시할 것이 없는 순수한 경우라는 것입니다. 이 전제 위에서 가격의 균형을 묻습니다.

3. [중급] 교환방정식과 그 한계

[중급] 가격을 수량으로 설명하는 가장 오래된 틀은 교환방정식입니다. 통화량 M, 유통속도 V, 물가 P, 거래량 T 에 대해

$$MV = PT$$

기호 풀이: 한 기간에 돈이 손을 바꾼 횟수(V)와 통화량의 곱은 그 기간에 거래된 금액(PT)과 같다는 **항등식입니다**. 수치 예: 코인 1,000개가 한 기간에 평균 두 번 쓰이고 거래된 재화가 4,000단위면, 재화 한 단위의 코인 가격은 $P = MV / T = 0.5$ 입니다. 이 식이 암호자산에 매력적인 이유는 분명합니다. M 은 프로토콜이 정하고(20장), T 는 원장에 적히니, 둘을 넣으면 가격이 나올 것 같습니다.

[중급] 실링(Linda Schilling)과 울리히(Harald Uhlig)의 2019년 논문이 이 식의 모형 안 모습을 보여 줍니다. 그들의 모형에서는 모든 돈이 매 기간 한 번씩 쓰이므로 「두 통화의 유통속도는 1이고, 통화가 하나뿐인 세계에서 고전적 수량설은 $y_t = D_t / P_t$ 를 준다」고 적고, 통화가 둘일 때 시장청산 조건은

$$y_t = \frac{D_t}{P_t} + \frac{Q_t}{P_t} B_t \qquad (9)$$

가 된다고 씁니다.[6] 기호 풀이: y_t 는 재화 산출, D_t 는 달러 통화량, P_t 는 달러 물가, B_t 는 비트코인 수량, Q_t 는 비트코인의 달러 가격입니다. 여기서 눈에 띄는 것은 미지수가 **둘**(D_t 와 Q_t)이라는 점입니다. 중앙은행이 P_t 를 목표로 D_t 를 움직이면, B_t 가 정해진 대로 늘어날 때 Q_t 가 어떻게 되는지는 이 식만으로 정해지지 않습니다. 논문이 이 식에서 끌어낸 정리 하나가 흥미롭습니다. 「비트코인 블록 보상은 비트코인 보유자에게 매기는 세금이 아니다. 달러 중앙은행이 매기는 달러 세금으로 조달된다.」 블록 보상이 늘어 B_t 가 커지면, 같은 P_t 와 Q_t 에서 식 (9)를 맞추려면 D_t 가 줄어야 하고, 그것은 달러 쪽의 일괄세로 나타난다는 논리입니다.[7] 20장이 다루는 「발행이 보유자를 희석한다」는 직관이 통화 경쟁

모형에서는 그대로 성립하지 않는다는 것입니다. 다만 이 결론은 모형의 전제(두 통화가 거래에서 완전히 대체 가능, 모든 돈이 매 기간 지출)에 매여 있습니다.

[중급] 교환방정식의 한계는 V 입니다. 항등식이므로 V 를 PT / M 으로 정의하면 언제나 성립하고, 그래서 설명력은 V 가 안정적이라는 **추가 가정에서** 나옵니다. 암호자산에서 그 가정을 검증하려면 「경제적으로 의미 있는 거래량」을 재야 하는데, 원장에 적힌 거래량은 그것이 아닙니다. 마카로프(Igor Makarov)와 쇼어(Antoinette Schoar)의 2021년 연구는 주소를 실체로 묶는 데이터베이스를 만들어 「비트코인 블록체인 거래량의 90%는 경제적으로 의미 있는 활동과 무관하며 프로토콜 설계와 익명성 선호의 부산물」이고, 「2015년부터 실제 비트코인 거래량의 75%는 거래소 또는 거래소 유사 기관과 연결되어 있다」고 보고합니다.[8] T 가 그렇게 흔들리면 V 는 측정값이 아니라 잔차입니다. 애시(Susan Athey)와 공저자들의 2016년 모형은 「정상 상태에서 환율은 거래량 대 비트코인 공급량의 비율로 정해진다」고 하면서도, 가격 자료와 대조한 뒤 「모형이 가격을 설명하는 능력에 대해 엇갈린 증거」를 제공한다고 스스로 적습니다. 그리고 사용 자료 분석은 「투자자와 드물게 쓰는 사용자가 비트코인의 대부분을 보유」한다는 집중과 투자자의 거래량 비중을 적고, 저자들은 그로부터 「환율은 현재 거래량 자체보다 비트코인의 미래에 대한 믿음에 더 민감할 가능성이 크다」고 — 「현재 거래량은 미래의 신호일 가능성이 크다」는 괄호 단서를 붙여 — 끌어냅니다.[9] 수량의 틀로 들어갔다가 믿음의 틀로 나온 셈입니다. 다음 절이 그 믿음의 모형입니다.

4. 「내재가치 0」의 균형 — 세 모형

세 논문을 읽습니다. 수학의 깊이는 다르지만 결론의 모양은 같습니다. 식 없이 요지만 먼저 적으면 이렇습니다.

모형	사람	결론의 모양
Garratt–Wallace (2018)	두 기간을 사는 세대가 겹친다	가격 0 균형이 항상 있다. 일정한 양의 가격 균형도 있을 수 있다. 「언젠가 0이 되되 그 날을 모르는」 균형도 있다. 붕괴의 날은 예측할 수 없다
Schilling–Uhlig (2019)	무한히 살며 소비와 생산을 번갈아 한다	두 통화가 함께 쓰이고 가격이 가격 커널과 무상관이면 코인의 달러 가격은 **마팅게일이다** (명제 1) — 상관이 있으면 그 부호가 기대 변화의 방향을 정한다(따름정리 1). 가격 변동은 교환매개 기능을 무효로 만들지 않는다
Biais 외 (2023)	두 기간을 사는 세대가 겹치고, 코인은 거래편익을 준다	「기본가치」는 미래 거래편익의 현재가치인데, 그 편익이 **가격에 의존하므로** 닻이 없다. 태양흑점이 가격에 들어오고 균형은 여럿이다

4.1 세대중첩 — 네 질문

[중급] 개럿과 윌리스는 새뮤얼슨의 2기간 세대중첩 모형에 외부화폐로서 비트코인 B 를 넣습니다. 매 기 N 명이 태어나 젊을 때 w^{young}, 늙어서 w^{old} 의 썩는 재화를 받고, 젊을 때 재화를 내고 코인을 사 두었다가 늙어서 다음 세대에 팝니다. 코인의 재화 가격을 p_t^B 라 하면 젊은이의 소비는 $c_t^{young} = w^{young} - p_t^B b_t$, 늙어서는 $c_t^{old} = w^{old} + p_{t+1}^B b_t$ 입니다(식 (1)·(2)). 저자들은 균형 전체를 그리는 대신 **네 질문을** 던집니다. 「(i) 비트코인의 가치가 일정하고 양(+)인 균형이 있는가? (ii) 가치가 항상 0인 균형이 있는가? (iii) 알려진 날짜 T 에 0이 되고 그 전 어느 날에는 양인 균형이 있는가? (iv) 가치가 다음처럼 무작위인 균형이 있는가 — 1, 2, …, $t-1$ 기에 양의 일정값이었다면 확률 π 로 t 기에도 그 값이고, 아니면 t 기와 그 뒤로 0인.」[10]

[중급] 답은 이렇습니다. **(i)는 「아마도」**. 효용함수와 소득 흐름이 젊을 때 저축할 유인을 주면(그림 1의 (a) 시나리오) 양의 일정 저축 s^{B*} 가 있고, 가격은 $s^{B*} = p^B(B / N)$ 로 정해집니다. 그렇지 않으면 없습니다. **(ii)는 「예」**. 「어느 t 에서든 젊은이들이 $t+1$ 에 비트코인이 무가치하리라고 믿는다고 하자. 그러면 각 젊은이는 어떤 양의 가격에서도 $b_t = 0$ 을 고른다(늙어서 팔 때 무가치할 자산을 얻으려고 젊을 때 소비할 수 있는 재화를 왜 내주겠는가?). 이 결론이 모든 t 에 성립하므로 모든 t 에서 $p_t^B = 0$ 인 균형이 있다.」 **(iii)은 「아니오」**. T 에 0이면 $T-1$ 에 아무도 사지 않고, 역진귀납으로 그 전에도 아무도 사지 않습니다. **(iv)는 조건부 「예」**. π 가 1에 충분히 가까우면 그런 균형이 있습니다.[11] 여기서 (ii)와 (iii)을 함께 읽어야 합니다. 가격 0은 언제나 균형이지만, 「알려진 날짜에 0이 된다」는 균형이 아닙니다. 네 질문 가운데 붕괴가 균형 안에 들어오는 길은 (iv), 곧 **매 기 작은 확률로** 오는 붕괴이고, 저자들은 결론에서 무작위성의 성격을 일반화하면 균형 집합이 더 커진다고 덧붙입니다.

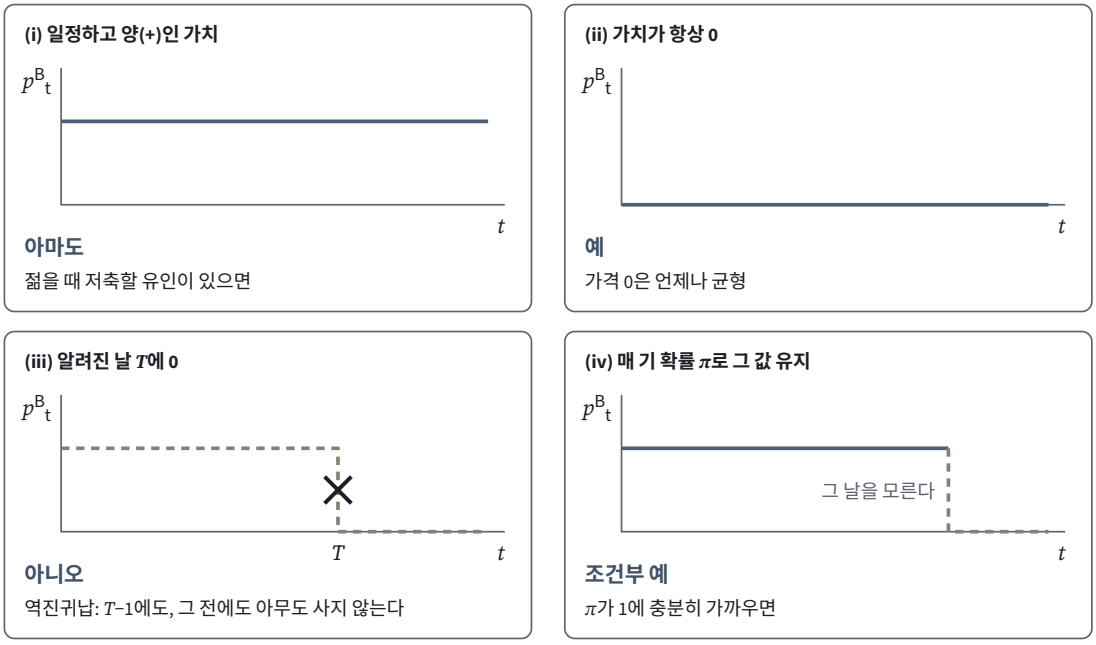


그림 19-1. 개럿-월리스의 네 질문과 답. 세로축은 모형 안의 코인의 재화 가격 p_t^B 이고 눈금이 없다. 가격 자료가 아니라 균형의 모양을 견주는 도식이다.

저자들이 (iv)에 붙인 예가 이 장의 요지를 숫자로 보여 줍니다. 「질문 (iv)의 답이 긍정이고 $\pi = 0.99$ 이며, 우리가 68기에 살고 있고, 앞선 모든 기에 앞면이 나왔으나 68기에 뒷면이 나왔다고 하자. 누군가는 67기에 화폐 가치의 붕괴를 예측하지 못했다고 경제학자를 비난하고 싶을 것이다. $t = 1$ 에 모형과 균형을 아는 사람이라면 처음 68기 안에 어느 때든 붕괴가 올 확률이 $(1 - 0.99^{68}) = 0.495$, 거의 절반이라고 말했을 것이다. 그러나 67기의 시점에서 68기의 붕괴는 확률 0.01로만 일어난다. 다시 말해 … 언젠가 붕괴가 올 가능성은 크지만, **언제** 올지 예측하는 것은 불가능하다.」[12] 아래 NB19-1이 이 산수를 그대로 돌립니다. 앞면·뒷면의 해석은 둘입니다. 순수한 외생적 **태양흑점**(Cass–Shell)일 수도 있고, 「더 나은 비트코인 2의 등장」이나 「프로토콜 해킹」, 「비트코인을 금지하는 법의 통과」 같은 **내재적** (intrinsic) 불확실성의 사건일 수도 있습니다. 저자들은 각주에서 2017년 8월 1일의 비트코인 캐시 출범을 그런 사건의 후보로 들면서 「시간이 말해 줄 것」이라고만 적습니다.[13]

[중급] 통화가 둘이면 문제는 더 커집니다. 저장비용이 없어 두 돈이 완전 대체재면, 일정 가격 균형에서 요구되는 것은 식 (11) $s^* = p^B B / N + p^M M / N$ 하나뿐이어서 「넓은 범위의 일정 가격 조합이 허용」됩니다. 「인구 중 α 만이 B 만 들려 하고 $1-\alpha$ 는 M 만 들려 한다」는 식의 강한 선호를 가정해야 비로소

식 (13) $p^B = \alpha N s / B$ 로 값이 정해지고, 「그래서 각 화폐의 가격은 그것을 쓰려는 사람의 비율에 따라 오르고 그 화폐의 재고에 따라 내린다.」 그리고 저자들은 이 분석이 「비트코인 1, 비트코인 2, ..., 비트코인 n 으로 쉽게 확장된다」고, 「동등한 외부화폐 사이에 강한 선호가 없으면 가장 단순한 일정 가격 균형에서조차 높은 정도의 가격 불확정성이 있다」고 적습니다.[14] 제목의 「비트코인 1, 비트코인 2, ...」가 여기서 나옵니다. 코드를 복제하는 데 비용이 거의 들지 않는다는 1부의 사실이, 경제학에서는 가치를 떠받칠 믿음을 더 얇게 만드는 사실로 읽힙니다. 마지막으로 이자입니다. 코인이 **다른 가치 있는 물건으로** 이자 $d > 0$ 를 지급한다면 「수익률 d / p 는 p 가 0으로 갈 때 무한대로 가므로」 가격 0 균형과 (iv)형 균형이 사라집니다.[15] 바꿔 말하면, 이 장의 다수성은 「배당 없음」에서 옵니다. 그 전제가 바뀌면 결론도 바뀝니다.

4.2 무한 생존 — 기본 가격식과 마팅게일

[중급] 실링과 울리히의 모형에서는 두 유형의 사람이 무한히 살며 홀수 기와 짝수 기에 소비와 생산을 번갈아 합니다. 그래서 매 기 파는 쪽과 사는 쪽이 갈리고, 돈이 필요합니다. 돈은 둘입니다. 중앙은행이 물가 목표에 맞춰 넣고 빼는 달러와, 양이 정해진 대로만 늘어나는 비트코인입니다. 두 돈이 **모두** 거래에 쓰이는 「기본적」 경우에 성립해야 하는 조건이 논문의 명제 1, **기본 가격식(fundamental pricing equation)입니다**. 두 기에 걸쳐 비트코인 거래가 있으면

$$\mathbb{E}_t\left[u'(c_{t+1})\frac{P_t}{P_{t+1}}\right] = \mathbb{E}_t\left[u'(c_{t+1})\frac{Q_{t+1}/P_{t+1}}{Q_t/P_t}\right] \qquad (1)$$

이고, 「$u'(c_{t+1}) / P_{t+1}$ 이 t 기 정보 조건부로 Q_{t+1} 과 상관이 없으면 비트코인 가격 과정 $\{Q_t\}$ 는 마팅게일이다: $Q_t = \mathbb{E}_t[Q_{t+1}]$ (2).」[16] 기호 풀이: $u'(c)$ 는 소비의 한계효용, P 는 달러 물가, Q 는 비트코인의 달러 가격입니다. 왼쪽은 달러를 들고 다음 기에 쓰는 실질 수익, 오른쪽은 비트코인을 들고 다음 기에 쓰는 실질 수익이고, 파는 쪽이 둘 다 받으려면 둘이 같아야 합니다. 수치 예: 상관이 없고 오늘 $Q_t = 100$(단위 없는 모형값)이면, 내일 Q_{t+1} 의 조건부 기대값은 100입니다. 오를 확률이 크면 그만큼 크게 내릴 가능성이 있어야 합니다. 저자들의 해설이 이 식의 뜻을 정확히 적습니다. 「재화 판매자는 위험 선호 조정 수익률이 더 높은 통화를 받고 팔 것이다. 통화 경쟁 때문에 비트코인과 달러 모두를 받고 판다는 것은 판매자가 무차별하다는 뜻이다. 따라서 위험 조정 수익률은 같아야 한다.」 그리고 이것이 「카레켄–윌리스(1981)의 유명한 결과의 한 판본」이며, 확률이 없는 경우로 줄이면 $Q_t = Q_{t+1}$, 곧 「어떤 Q_0 든 어떤 균형과 정합적이라는 환율 불확정성 결과」가 된다고 밝힙니다.[17]

[중급] 그다음이 이 논문의 고유한 부분입니다. 무한히 사는 사람은 코인을 **쓰지 않고 쥐고 있을** 수 있습니다. 명제 2는 그렇게 쥐고 있는 균형에서 성립해야 하는 「투기적 가격 조건(speculative price bound)」을 적는데, 요지는 코인을 쥐고 있으려면 가격이 기대값으로 **올라야** 한다는 것입니다. 그런데 명제 1은 두 돈이 함께 쓰이면 가격이 마팅게일이라고 했습니다. 둘을 나란히 놓고 「상호 조급함」(가정 3 — 오늘의 한계효용이 내일의 할인된 기대 한계효용보다 크다)을 더하면 정리 1이 나옵니다. 「모든 비트코인이 지출되고 (1)이 매 기 성립한다.」 저자들의 말로 「가격 변동성은 비트코인의 교환매개 기능을 무효로 만들지 않는다.」[18] 쥐고 있는 쪽은 오르기를 바라지만, 파는 쪽이 받아 주려면 내일의 가격이 너무 낮으면 안 되고, 그러려면 오늘의 가격이 높아야 하며, 오늘의 가격이 그렇게 높으면 쥐고 있을 이유가 사라진다는 순환입니다. 비확률 경제에서는 $Q_{t+1} \le Q_t \le \beta^2 Q_{t+2} / (\pi_{t+2}\pi_{t+1})$ 인데 $\beta < 1$, $\pi \ge 1$ 이니 모순이 바로 보입니다.[19]

[심화] 마팅게일은 「상관이 없으면」의 특수 경우입니다. 일반식인 따름정리 1은 $Q_t = \mathbb{E}_t[Q_{t+1}] + \kappa_t \cdot \mathrm{corr}_t(m_{t+1}, Q_{t+1})$ (식 (5))이고, 여기서 $m_t = u'(c_t) / P_t$ 는 명목 가격 커널, $\kappa_t > 0$

입니다. 「비트코인 가격과 가격 커널이 양의 상관이면 가격은 슈퍼마팅게일(기대값으로 하락)이고, 음의 상관이면 서브마팅게일(기대값으로 상승)」입니다.[20] 방향은 모형 안에서조차 **상관의 부호에** 달려 있고, 그 부호를 모형이 정해 주지 않습니다. 그래서 저자들은 균형을 「존재한다」고만 하지 않고 **만드는 법을** 줍니다. 식 (8) $Q_{t+1} = Q_t + \epsilon_{t+1} - \mathrm{cov}_t(\epsilon_{t+1}, m_{t+1}) / \mathbb{E}_t[m_{t+1}]$ 에 따라 「초기 비트코인 가격 Q_0 를 고르고, $\mathbb{E}_t[\epsilon_{t+1}] = 0$ 인 수열 ϵ_t 를 고른 뒤 … 시간을 따라 앞으로 Q_{t+1} 을 재귀적으로 구성한다.」[21] Q_0 도 ϵ 도 **고르는** 것입니다. 균형은 하나가 아니라 그렇게 고를 수 있는 만큼 많습니다. NB19-2가 이 구성을 몇 가지 ϵ 로 돌려 같은 Q_0 에서 갈라지는 경로들을 보여 줍니다. 그 경로들은 가격이 아니라 **모형 변수입니다**. 한편 발행 상한이 있고 달러 인플레이션이 양이면 「비트코인 전체 재고의 구매력은 시간이 가면서 0으로 줄어든다」는 명제 3(Real Bitcoin Disappearance)도 있습니다. 이것 역시 「위험 프리미엄이 충분히 높지 않으면」이라는 조건부 명제이고, 논문 7절의 자유 진입이 더해지면 「더 이상 성립하지 않을 수 있다」고 저자들이 적습니다.[22]

4.3 거래편익 — 닻이 없는 「기본가치」

[중급] 비아이스(Bruno Biais)와 공저자들의 2023년 논문은 코인에 **거래편익을** 줍니다. 「베네수엘라나 짐바브웨 시민은 자국 통화와 은행 체계가 무너졌을 때도 비트코인으로 거래할 수 있다. 전통 금융기관을 통한 이전이 높은 비용이나 정부 통제로 막힐 때 암호통화는 국경 간 이전에 쓰일 수 있다」는 편익과, 「제한된 태환성, 거래소 거래비용, 채굴 수수료, 붕괴 위험」이라는 비용입니다.[23] 그러면 코인은 주식과 비슷해집니다. 「거래편익은 주식에 배당이 하는 역할을 암호통화에 한다. 그러나 큰 차이가 하나 있다. 완전시장에서 배당은 주가에 의존하지 않고 평가의 실질적 닻을 준다. 반대로 암호통화가 주는 거래편익은 그 가격에 의존한다. 가격이 높을수록 표준 통화 대비 구매력이 강하고, 따라서 주는 거래편익이 크다.」[24] 명제 1의 식 (12)는 가격을 미래 거래편익 $T_{t+k}p_{t+k}$ 의 할인 합으로 적는데, 거기 p_{t+k} 가 들어 있습니다. 기본가치가 가격의 함수입니다. 그래서 「가격이 가격에 독립한 실질 변수가 아니라 미래 가격에 대한 믿음을 반영하므로, 균형은 외생적 태양흑점을 반영할 수 있다.」 저자들은 두 종류의 균형을 구성합니다. 붕괴 확률마다 하나씩 있는 **일정 가격 균형과**, 태양흑점이 매 기 가격을 흔드는 **변동 가격 균형입니다**. 「균형은 여럿이고 — 태양흑점의 표본 경로의 분포마다 하나씩 — 각 균형 안에 외생적 변동성이 있다. 기본 변수가 그대로여도 암호통화 가격은 태양흑점의 실현에 반응해 변한다.」[25]

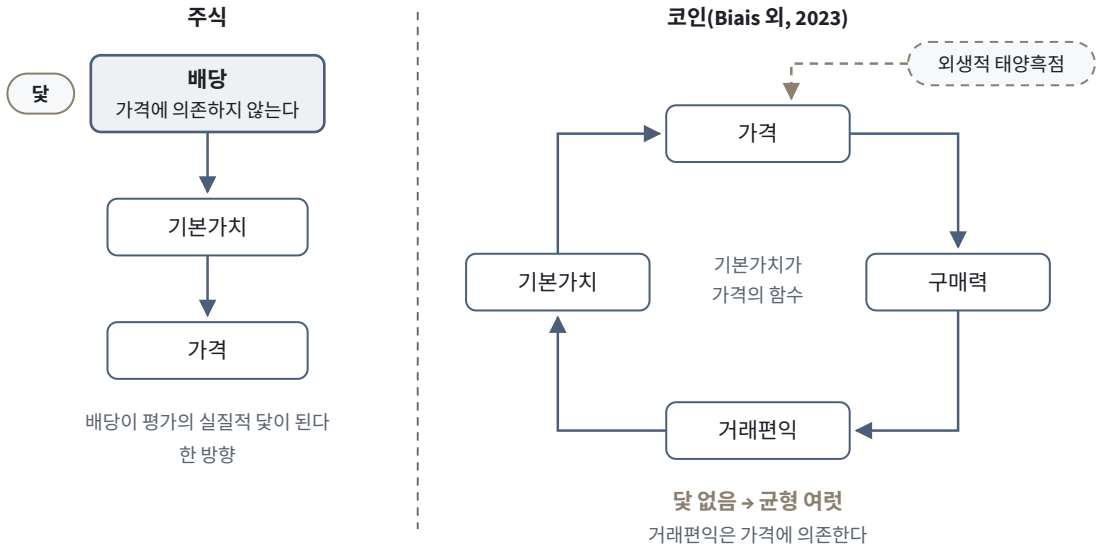


그림 19-2. 주식과 코인(Biais 외 2023)의 기본가치 비교. 배당은 가격에 의존하지 않아 닻이 되지만, 거래편익은 가격에 의존해 고리를 이루고 그 고리로 태양흑점이 들어온다.

[심화] 이 논문은 모형을 자료에 맞춰 보기도 합니다. 채굴 수수료, 거래소 개설·폐쇄 같은 사건, 가맹점 수용 같은 사건, 해킹 자료로 거래비용·편익 지수를 만들어 위험중립 가정 아래 보정합니다. 저자들은 「표본이 비교적 짧고 변수가 비정상일 가능성이 크므로 통계적 유의성이나 모수 추정을 주장하지 않는다」고 미리 적고, 결과를 이렇게 요약합니다. 「기본 변수의 변화는 비트코인 수익률 분산의 작은 몫(약 5%)만 설명한다. 모형이 바르게 설정되었고 대리변수가 정확하다는 가설 아래, 이는 비트코인 수익률 변동의 대부분이 기본 변수와 무관한 외생적 변동성을 반영한다는 뜻이다.」[26] 이 수치를 옮길 때 조건절을 떼면 안 됩니다. 「약 5%」는 모형과 대리변수가 맞다는 **가정 아래의** 보정값이고, 저자들 스스로 추정치라 부르지 않습니다. 이 장이 이 수치에서 가져가는 것은 크기가 아니라, 이 모형의 저자들조차 자료 대부분을 「기본 변수」로 설명하지 못했다는 고백입니다.

▾ [심화] 「버블」과 「폰지」라는 말 — 합리적 버블의 자리

[심화] 이 장의 다수성은 1980년대 합리적 버블 문헌의 결과와 같은 뿌리입니다. 블랑샤르(Olivier Blanchard)와 왓슨(Mark Watson)의 1982년 논문은 무재정거래 조건 $\mathbb{E}(p_{t+1} \mid \Omega_t) - p_t + x_t = r p_t$ (식 (1))의 해가 「기대 배당의 현재가치」 p_t^* 하나가 아니라, $\mathbb{E}(c_{t+1} \mid \Omega_t) = (1+r)c_t$ 를 만족하는 어떤 c_t 를 더한 $p_t = p_t^* + c_t$ (식 (3))도 전부 해임을 보입니다. 「시장가격은 재정거래 조건을 어기지 않고 시장 기본가치에서 벗어날 수 있다.」 그리고 「매 기 확률 π 로 남고 $1-\pi$ 로 터지는」 버블(식 (4))을 예로 듭니다 — 개럿-윌리스의 (iv)와 같은 모양입니다.[27] 티롤(Jean Tirole)의 1985년 논문은 세대중첩 경제에서 총량 버블이 존재할 필요충분조건을 주면서, 「새뮤얼슨이 소비대부 모형을 전개한 이래 화폐가 내재적으로 쓸모없는데도(곧 시장 기본가치가 0인데도) 양의 가치를 갖는 경제가 존재한다는 것은 잘 알려져 있다. 다시 말해 화폐 위에 버블이 존재할 수 있다 — 버블은 시장가격과 시장 기본가치의 차이로 정의된다」고 적습니다.[28] 이 문헌의 언어로는 불환지폐의 가치 자체가 「버블」입니다. 그 낱말은 여기서 경멸어가 아니라 정의어입니다. 「버블이었다」를 통계로 판정하는 문제는 25장에서, 판 오르트(Maarten van Oordt)의 2024년 모형이 「버블 균형」과 「폰지 등가 수익」을 어떻게 정의하고 발행 규칙(특히 수수료 소각)이 그 구별을 어떻게 바꾸는지는 21장·22장에서 다시 만납니다.[29]

5. 코드 — 균형의 열거를 숫자로

[중급] 아래 두 셀은 **가격을 다루지 않습니다**. 다루는 것은 모형 안의 변수와 확률입니다. 첫 셀은 개럿-윌리스의 「68기」 산수를, 둘째 셀은 실링-울리히의 식 (8)에 따른 균형 구성을 돌립니다.

```
# NB19-1: 개럿-윌리스 질문 (iv) — 매 기 확률 1-π 로 오는 붕괴. 「언젠가」의 확률과 「바로 다음 기」의 확률은 다르다 (공부용)
pi = 0.99                                                  # 한 기를 더 살아남을 확률(논문의 예)
print(f"π = {pi}: 다음 한 기에 붕괴할 조건부 확률 = {1 - pi:.2f}")
print(f"{'n(기)':>6} {'n기 안에 붕괴할 확률 1-π^n':>26} {'n기까지 살아남을 확률 π^n':>26}")
for n in [1, 10, 30, 68, 100, 300, 1000]:
    print(f"{n:>6} {1 - pi**n:>26.3f} {pi**n:>26.3f}")
print("\n68기 안에 붕괴할 확률은 거의 절반이지만, 67기에서 본 68기의 붕괴 확률은 여전히 0.01이다(논문 1891-1892쪽의 산수).")
print("어느 기에 붕괴하는가의 분포는 기하분포이고, 그 기대값은", round(1 / (1 - pi)), "기 — 평균은 있지만 날짜는 없다.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
π = 0.99: 다음 한 기에 붕괴할 조건부 확률 = 0.01
  n(기)          n기 안에 붕괴할 확률 1-π^n           n기까지 살아남을 확률 π^n
     1                       0.010                       0.990
    10                       0.096                       0.904
    30                       0.260                       0.740
    68                       0.495                       0.505
   100                       0.634                       0.366
   300                       0.951                       0.049
  1000                       1.000                       0.000

68기 안에 붕괴할 확률은 거의 절반이지만, 67기에서 본 68기의 붕괴 확률은 여전히 0.01이다(논문 1891-1892쪽의 산수).
어느 기에 붕괴하는가의 분포는 기하분포이고, 그 기대값은 100 기 — 평균은 있지만 날짜는 없다.
```

```
# NB19-2: 실링-울리히 식 (8) — 초기값 Q0 와 평균 0 교란 ε 를 「고르면」 균형 경로가 하나 만들어진다. 고를 수 있는
만큼 균형이 있다 (공부용)
# 주의: Q 는 모형 변수(단위 없음)다. 어떤 자산의 가격도 아니고, 아래 경로는 전망이 아니다.
import random
random.seed(19)

def construct_path(Q0, steps, eps_draw, cov_term=0.0):
    """식 (8): Q_{t+1} = Q_t + ε_{t+1} - cov_t(ε_{t+1}, m_{t+1}) / E_t[m_{t+1}].
    cov_term=0 이면 마팅게일(명제 1 식 (2)). 상관이 있으면 cov_term 만큼 기대 변화가 생긴다(따름정리 1)."""
    Q = [Q0]
    for _ in range(steps):
        Q.append(Q[-1] + eps_draw() - cov_term)
    return Q

Q0, steps = 100.0, 12
draws = {
    "ε≡0 (카레켄-월리스의 일정 가격)": lambda: 0.0,
    "ε ~ 균등(-5, 5)":               lambda: random.uniform(-5, 5),
    "ε ∈ {-20, +20} 각 1/2":          lambda: random.choice([-20.0, 20.0]),
}
print(f"Q0 = {Q0}, {steps}기. 같은 Q0 에서 출발한 세 균형 경로(마팅게일, cov 항 0):")
for name, d in draws.items():
    path = construct_path(Q0, steps, d)
    print(f"  {name:<32} → " + " ".join(f"{q:6.1f}" for q in path))

# 마팅게일 성질의 수치 확인: 많은 경로의 평균은 Q0 근처에 머문다(기대값으로 오르지도 내리지도 않는다)
N = 20000
end_values = [construct_path(Q0, steps, draws["ε ~ 균등(-5, 5)"])[-1] for _ in range(N)]
print(f"\n균등 교란 경로 {N:,}개의 {steps}기 말 평균: {sum(end_values)/N:.2f} (Q0 = {Q0}) — 평균은
그대로, 흩어짐만 커진다")
print(f"  {steps}기 말 최소 {min(end_values):.1f}, 최대 {max(end_values):.1f}")

# 따름정리 1: 가격 커널과의 상관이 양(+)이면 슈퍼마팅게일(기대값으로 하락), 음(-)이면 서브마팅게일(상승). 부호는
모형이 정하지 않는다.
for cov_term, label in [(1.0, "cov > 0 → 슈퍼마팅게일"), (-1.0, "cov < 0 → 서브마팅게일")]:
    mean_end = sum(construct_path(Q0, steps, draws["ε ~ 균등(-5, 5)"], cov_term)[-1] for _ in
range(N)) / N
    print(f"  {label}: {steps}기 말 평균 {mean_end:.2f}")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
Q0 = 100.0, 12기. 같은 Q0 에서 출발한 세 균형 경로(마팅게일, cov 항 0):
  ε≡0 (카레켄-월리스의 일정 가격)            →  100.0  100.0  100.0  100.0  100.0  100.0  100.0
100.0  100.0  100.0  100.0  100.0  100.0
  ε ~ 균등(-5, 5)                    →  100.0  101.8  104.6  104.8  104.9  103.9  108.8  106.7
103.2  100.8   98.4   96.7   94.4
  ε ∈ {-20, +20} 각 1/2              →  100.0   80.0  100.0  120.0  100.0   80.0   60.0   40.0
20.0   40.0   60.0   40.0   20.0

균등 교란 경로 20,000개의 12기 말 평균: 99.94 (Q0 = 100.0) — 평균은 그대로, 흩어짐만 커진다
  12기 말 최소 64.7, 최대 138.4
  cov > 0 → 슈퍼마팅게일: 12기 말 평균 87.89
  cov < 0 → 서브마팅게일: 12기 말 평균 112.12
```

[중급] 둘째 셀의 세 경로는 식 (8)의 조건(양의 가격, 따름정리 2의 상한)을 만족하는 한 **모두 균형입니다.** 어느 것이 「맞는」 경로인지 모형은 말하지 않습니다. 첫 경로는 카레켄–월리스의 일정 가격이고, 둘째와 셋째는 같은 Q_0 에서 교란의 크기만 달리 고른 것입니다. 2만 개 경로의 평균이 Q_0 에 머무는 것이 마팅게일의 뜻이고, 마지막 두 줄은 상관의 부호 하나로 평균의 방향이 바뀐다는 따름정리 1의 수치 판입니다. 이 셀에서 가져갈 것은 경로의 모양이 아니라 「고른다」는 동사입니다.

6. 모형이 말하는 것과 말하지 않는 것

세 모형을 겹쳐 놓으면 공통분모가 둘입니다. 첫째, **가격 0은 언제나 균형이다.** 개럿–월리스와 비아이스 외는 그 균형의 존재를 적고(비아이스 외는 각주에서 「$p_t = 0$ 인 균형은 항상 존재한다」고 적고 양의 가격 균형을 분석합니다), 실링–울리히는 §7에서 카레켄–월리스를 따라 「내일 통화의 가격이 0이면 오늘 가격도 0」이라는 역방향 자기소거를 적습니다.[30] 둘째, **양의 가격 균형은 믿음으로 지탱되고 하나로 정해지지 않는다.** 복제 코인이 많을수록(개럿–월리스), 거래편익이 가격 자체에 달려 있을수록(비아이스 외) 그 믿음의 닻은 얇아집니다. 그리고 공통으로 말하지 않는 것이 있습니다. 어느 균형이 실현되는가, 지금의 가격이 어느 균형 위에 있는가, 다음 기의 방향이 어디인가. 개럿–월리스는 결론에서 「이 논문에서 서술한 균형 가운데 어느 것도 비트코인 가치의 좋은 서술은 아니다. … 그래서 우리는 처음에 균형의 수가 **엄청나다고** 말했다」고 적습니다.[31] 이 장이 「가능성의 열거」라는 말을 고집하는 이유입니다.

그래서 두 상투구를 다시 봅니다. 「내재가치가 없으니 0이 된다」는 (ii)가 균형이라는 사실을 (iii)이 성립한다는 주장으로 바꾼 것입니다. 「언젠가」는 모형이 허용하지만 「언제」는 모형이 금지합니다. 「희소하니 가치 있다」는 식 (13)에서 B 가 분모에 있다는 사실만 보고 α(그것을 쓰려는 사람의 비율)가 분자에 있다는 사실을 뺀 것입니다. $\alpha = 0$ 이면 B 가 아무리 작아도 $p^B = 0$ 입니다. 두 상투구 모두 조건절을 지운 명제이고, 법률 문서에서 조건절이 지워진 명제는 사실인정의 자료가 되기 어렵습니다.

마지막으로, 모형들이 **법정통화에** 준 자리를 보고 다음 상자로 넘어갑니다. 비아이스 외는 표준 통화의 가격도 0으로 갈 수 있지만 「스타(1974)가 보였듯 정부가 그 통화로 내야 하는 세금을 매기면 이를 막을 수 있다. 다른 사람들이 표준 통화를 받지 않으리라고 예상하더라도, 정부가 세금 납부에 그것을 받으리라는 것을 아는 한 표준 통화에 대한 수요는 0에서 떨어진 가격에서 양(+)」이라고 적고, 그래서 표준 통화의 가격이 양인 균형만 분석합니다.[32] 개럿–월리스도 서두에서 카레켄–월리스의 불확정성 조건이 국가 통화에는 맞지 않는 이유로 「세금을 그 나라 통화로 내야 한다는 가정이나 그 통화로 매겨진 가격이 고정적이라는 가정」을 들었습니다.[33] 세 모형 어디에도 「법이 화폐라고 불러서」 가치가 생기는 장치는 없습니다. 있는 것은 **조세라는 수요의 바닥입니다.** 그것이 개럿–월리스의 서두와 비아이스 외에서 비트코인과 달러를 가르는 선입니다 — 실링–울리히에서는 물가 목표에 맞춘 중앙은행의 공급 조절이,

개럿–월리스 §IV에서는 법정화폐의 저장비용이 그 자리를 맡습니다. 조세라는 선은 기술이 아니라 법이 긋습니다.

> **법률가의 질문**
>
> 법정통화와 「화폐」는 같은 말입니까. 이 장의 모형에서 국가 통화를 비트코인과 가르는 것은 「법이 화폐라고 부른다」는 사실이 아니라 「세금을 그것으로 내야 한다」는 수요의 바닥이었습니다. 그렇다면 법이 어떤 물건을 「화폐」라고 부를 때 달라지는 것은 무엇입니까 — 채무의 변제 효력(법화), 계산단위로서의 지위, 형벌 규정(위조·통화 관련 범죄)의 객체, 과세의 틀, 그 밖의 무엇입니까. 반대로 법이 부르지 않아도 사람들이 받아 주는 물건은 어느 법 영역에서 「화폐」로 취급되고 어느 영역에서 「재산」으로 취급됩니까. 기능론의 세 기능 가운데 법이 요구하는 것은 몇 개입니까. 이 책은 답하지 않고 5부에서 각 법역의 정의 조문과 학설 구도를 가리킵니다. → 27장

7. 정리

기능론은 점검표입니다. 암호자산은 그 점검에서 좋은 점수를 받지 못하지만, 그것이 가격 0을 뜻하지는 않습니다. 가격의 질문에 경제학이 내놓은 답은 균형의 열거입니다. 배당 없는 외부화폐는 가격 0인 균형과 양의 가격 균형을 함께 가지며, 양의 가격 균형은 「남들이 받아 줄 것」이라는 믿음으로 지탱되고 하나로 정해지지 않습니다. 세대중첩 모형은 붕괴가 「언젠가」는 올 수 있어도 「언제」는 예측 불가능함을, 무한 생존 모형은 두 통화가 함께 쓰일 때 가격이 — 가격 커널과 무상관이면 — 마팅게일이고 상관의 부호가 기대 변화의 방향을 정함을, 거래편익 모형은 「기본가치」가 가격 자체에 의존해 닻이 없음을 보입니다. 이 장은 가격을 한 번도 쓰지 않았고, 다음 장들도 그렇습니다. 20장은 이 장이 「주어진 것」으로 둔 발행 규칙 B_t 가 보안의 재원이라는 것을, 21장은 그 재원의 다른 반쪽인 수수료가 어떻게 매겨지는지를 봅니다.

이 장의 출처

- Rodney Garratt & Neil Wallace, "Bitcoin 1, Bitcoin 2, ….: An Experiment in Privately Issued Outside Monies," *Economic Inquiry* 56(3): 1887–1897 (2018), DOI 10.1111/ecin.12569 — 내부·외부화폐(§II, 1887쪽), 모형 전제(1887–1888쪽), 네 질문과 답(§III.B, 1890–1891쪽), 68기 예와 태양흑점·내재적 불확실성 해석(1891–1892쪽), 두 통화와 복제 코인(§IV.A, 1893쪽), 이자(§V, 1895쪽), 결론(§VI, 1896쪽). (게재본). 확인 과정에서 적어 둔 WP판 위치(§3.1 식 (12)–(14)·(18))는 게재본 §IV.A 식 (11)–(13)·§IV.B 식 (18)에 대응한다.
- Linda Schilling & Harald Uhlig, "Some simple bitcoin economics," *Journal of Monetary Economics* 106: 16–26 (2019), DOI 10.1016/j.jmoneco.2019.07.002, CC BY — 명제 1 기본 가격식과 마팅게일(§3.1, 식 (1)·(2), 20쪽), 명제 2·가정 3·정리 1(§3.2, 20–21쪽), 따름정리 1 식 (5)·명제 3(§4, 21쪽), 식 (8) 균형 구성(22쪽), 식 (9)와 정리 2(§5, 22쪽), 자유 진입(§7, 25쪽). (게재본). 확인 과정에서 적어 둔 WP판 번호(식 (40)–(42), Proposition 1·2, Theorem 1)는 게재본에서 식 (1)–(3), 같은 명제·정리 번호로 대응한다.
- Bruno Biais, Christophe Bisière, Matthieu Bouvard, Catherine Casamatta & Albert J.

Menkveld, "Equilibrium Bitcoin Pricing," *Journal of Finance* 78(2): 967–1014 (2023), DOI 10.1111/jofi.13206 — 거래편익과 배당의 차이(968쪽), 균형의 다수성·외생적 변동성(969쪽), 보정 결과의 조건(970쪽), 조세와 표준 통화(975쪽), 모형(§I), 명제 1 식 (10)–(12)(§II.A, 977쪽), 각주 18(979쪽). (게재본).

- Susan Athey, Ivo Parashkevov, Vishnu Sarukkai & Jing Xia, "Bitcoin Pricing, Adoption, and Usage: Theory and Evidence," Stanford GSB Working Paper No. 3469 (2016-08; SSRN 2826674) — 정상 상태 환율과 엇갈린 증거(1·3쪽), 보유 집중과 믿음(3–4쪽).
- Igor Makarov & Antoinette Schoar, "Blockchain Analysis of the Bitcoin Market," NBER Working Paper 29396 (2021-10) — 허위 거래량 90%, 거래소 연결 75%(2쪽).
- Gary B. Gorton & Jeffery Y. Zhang, "Taming Wildcat Stablecoins," *University of Chicago Law Review* 90(3): 909–972 (2023) — 화폐의 세 기능과 NQA 원칙(911–912쪽).
- SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017-07-25) — FATF 「가상통화」 정의(3쪽 각주 5).
- Eng-Tuck Cheah & John Fry, *Economics Letters* 130: 32–36 (2015), 수락원고 — 기능론 점검(§2).
- Olivier J. Blanchard & Mark W. Watson, NBER Working Paper 945 (1982) — 무재정거래 조건과 버블 해(2–4쪽). Jean Tirole, *Econometrica* 53(5): 1071–1100 (1985) — 화폐 위의 버블 정의(1071쪽). Maarten R.C. van Oordt, Tinbergen Institute DP 2024-050/IV (2024) — 버블 균형·폰지 등가 수익(초록). 경로는 각주.
- 교환방정식 $MV = PT$ 자체는 교과서의 항등식으로 썼고 특정 원전을 인용하지 않았다. 모형 안의 판은 Schilling & Uhlig (2019) §5 식 (9).
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `random` 만 사용. 노트북 NB19. 두 셀 모두 가격 자료를 쓰지 않으며 출력은 모형 변수다.

각주

1. Gary B. Gorton & Jeffery Y. Zhang, "Taming Wildcat Stablecoins," *University of Chicago Law Review* 90(3): 909–972 (2023), 911–912쪽. 원문: "money has several important properties, with the three most commonly stated ones being a unit of account, a store of value, and a medium of exchange. The most obvious (yet not explicitly stated) property is that money must also satisfy the no-questions-asked (NQA) principle, which requires that the money be accepted in a transaction without due diligence on its value." 세 기능의 출처로 저자들은 Desan, *Making Money* (2014)를 인용합니다(각주 7).

2. SEC, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Release No. 81207 (2017-07-25), 3쪽 각주 5. 원문(FATF Report, *Virtual Currencies, Key Definitions and Potential AML/CFT Risks*, 2014-06 인용): "a digital representation of value that can be digitally traded and functions as: (1) a medium of exchange; and/or (2) a unit of account; and/or (3) a store of value, but does not have legal tender status (i.e., when tendered to a creditor, is a valid and legal offer of payment) in any jurisdiction."

3. Eng-Tuck Cheah & John Fry, "Speculative bubbles in Bitcoin markets? An empirical investigation into the fundamental value of Bitcoin," *Economics Letters* 130: 32–36 (2015), 수락원고 2쪽(§2). 원문: "In order to serve as money Bitcoin should fulfil the roles of unit of account (Dwyer, 2014), means of payments and store of value. In this regard as the speculative aspects of Bitcoin dominates its other functions Bitcoin prices appear to be at the mercy of nothing more that market sentiments … Hence, this undermines the role Bitcoin plays as a unit of account." 이 논문의 「기본가치 0」 추정은 25장에서 다룹니다.

4. Rodney Garratt & Neil Wallace, "Bitcoin 1, Bitcoin 2, ….: An Experiment in Privately Issued Outside Monies," *Economic Inquiry* 56(3): 1887–1897 (2018), 1887쪽(§II). 원문: "Most economists distinguish between inside and outside money. Inside money is inside the economy in the sense that each unit is someone's asset and

someone else's liability. … Examples of inside money are checking accounts and grocery-store coupons. Outside money, in contrast, does not disappear when balance sheets are consolidated. Examples are gold coins under a gold standard and Federal Reserve notes since August 15, 1971 when Nixon closed the gold window. In terms of that dichotomy, bitcoin is best viewed as an outside money. In particular, the issuer of bitcoin makes no promise to redeem it for any other object." (게재본).

5. Garratt & Wallace (2018), 1887–1888쪽. 원문: "people give up other things to get bitcoin only because they think that others in the future will do the same. In other words, ownership of bitcoin does not yield utility as might ownership of a Picasso, and is not an input into the production of other things as is farmland, a factory, or a 3D printer. Nor does such ownership entitle the owner to a dividend stream of other valuable objects."; "When it comes to bitcoin, there is no commodity value to ignore. You cannot even use it as wall paper—as some stories suggest was performed with German marks during the 1922–1923 hyperinflation."

6. Linda Schilling & Harald Uhlig, "Some simple bitcoin economics," *Journal of Monetary Economics* 106: 16–26 (2019), 22쪽(§5). 원문: "Since all Dollars and Bitcoins are spent once in every period, the velocity of both currencies equals one. In a world with only one currency, classical quantity theory yields $y_t = D_t / P_t$. … In our model instead, equilibrium market clearing implies $y_t = D_t / P_t + (Q_t / P_t)B_t$. (9) There are now two endogenous variables, the Dollar quantity D_t and the Bitcoin price Q_t." (게재본, CC BY). 확인 과정에서 적어 둔 WP판 번호(식 (40)–(42))는 게재본에서 (1)–(3)으로 바뀌었고, 이 책은 게재본 번호를 씁니다.

7. Schilling & Uhlig (2019), 22쪽, Theorem 2 (Bitcoin block rewards are financed by Dollar taxes). 원문(정리 뒤 설명): "The block rewards to miners … are financed not by deflating the Bitcoin currency but by the Dollar central bank, which has to accordingly decrease its Dollar supply. It does so per imposing Dollar lump-sum taxes on the population. The block rewards are not a tax on Bitcoin holders but are financed through Dollar taxes imposed by the Dollar central bank."

8. Igor Makarov & Antoinette Schoar, "Blockchain Analysis of the Bitcoin Market," NBER Working Paper 29396 (2021-10), 2쪽. 원문: "We first document that 90% of transaction volume on the Bitcoin blockchain is not tied to economically meaningful activities but is the byproduct of the Bitcoin protocol design as well as the preference of many participants for anonymity."; "Starting from 2015, 75% of real bitcoin volume has been linked to exchanges or exchange-like entities such as on-line wallets, OTC desks, and large institutional traders." 학술지 게재본은 찾지 못해 NBER WP로 인용합니다. 보유 집중의 수치는 22장.

9. Susan Athey, Ivo Parashkevov, Vishnu Sarukkai & Jing Xia, "Bitcoin Pricing, Adoption, and Usage: Theory and Evidence," Stanford GSB Working Paper No. 3469 (This Draft: August 2016; SSRN 2826674), 1쪽(초록)·3쪽. 원문: "In steady state, exchange rates are determined by the ratio of transaction volume to the supply of Bitcoins."; "Empirical evidence from Bitcoin prices and utilization provides mixed evidence about the ability of the model to explain prices."; "investors and infrequent users held the majority of Bitcoins"; "this implies that exchange rates are likely to be more sensitive to beliefs about the future of Bitcoin and less sensitive to current transaction volume for its own sake".

10. Garratt & Wallace (2018), 1890쪽(§III.B). 원문: "(i) Is there an equilibrium in which the value of bitcoin is constant and positive? (ii) Is there an equilibrium in which the value of bitcoin is always zero? (iii) Is there an equilibrium in which the value of bitcoin is zero at some known date T and is positive at some earlier dates? (iv) Is there an equilibrium in which the value of bitcoin is random in the following way: if it has been positive and constant at each date 1, 2, …, t − 1, then with probability π the value is equal to that constant at t; otherwise, it is zero at t and thereafter."

11. Garratt & Wallace (2018), 1890–1891쪽. (ii) 원문: "Suppose at any t, the young at t believe that bitcoin will be worthless at t + 1; that $p^B_{t+1} = 0$. Then, each young person chooses $b_t = 0$ at time t at any $p^B_t > 0$. (Why give up goods which can be consumed when young in order to acquire an asset that will be worthless when it is sold?) Because this conclusion holds for each t, there is an equilibrium in which $p^B_t = 0$ for all t." (iii) 원문: "Consider what happens at T − 1. As in our argument concerning question (ii), no one is willing to give up goods for money at T − 1. Then, by backward induction, no one gives up goods for money at any t < T. Hence, the answer to question (iii) is no." (iv)의 조건: "(b) if ∂f(0;1)/∂s^B > 0, then there exists π < *1 such that if* $\pi \in [\pi, 1]$, then the answer is yes."

12. Garratt & Wallace (2018), 1891–1892쪽. 원문: "Suppose the answer to question (iv) is positive, that π = 0.99, that we are living at date 68, that heads has been experienced at all earlier dates, but that tails has occurred at date 68. Someone might be tempted to criticize economists for not having predicted the collapse in the value of

money at date 67. At t = 1, someone who knows the model and the equilibrium would have said that the probability of a collapse at some time in the first 68 periods is (1–.99^68) = 0.495, almost one-half. However, from the vantage point of date 67, a collapse at date 68 happens only with probability 0.01. In other words, and consistent with our answer to question (iii), although a collapse at some time is likely, predicting when it will occur is not possible."

13. Garratt & Wallace (2018), 1892쪽과 각주 9. 원문: "One interpretation is that the uncertainty is purely extrinsic. That is, heads and tails represent the two outcomes of a publicly observed sunspot variable à la Cass and Shell (1983)."; "The appearance of tails could represent the appearance of a preferred version of bitcoin, say bitcoin 2 ... Or, we could imagine that at every date t there is an exogenous probability 1 − π that there is a disruption of platform for transacting bitcoin (someone hacks the protocol) or the passage of a law outlawing bitcoin."; 각주 9: "The August 1, 2017 launch of Bitcoin Cash, a new version of the bitcoin protocol with a larger block size, could have, at least in principle, led to the abandonment of the original bitcoin. Time will tell how this ultimately plays out."

14. Garratt & Wallace (2018), 1893쪽(§IV.A). 원문: "But this allows a wide array of constant money price combinations."; "So the price of each money is increasing in the fraction of people that prefer to use it and decreasing in the stock of each money."; "The above analysis applies equally well to the case of two privately issued outside monies: bitcoin 1 and bitcoin 2. Moreover, it extends easily to bitcoin 1, bitcoin 2, ..., bitcoin n. The key point is that absent strong preferences over equivalent types of outside money there would be a high degree of price indeterminacy, even in the simplest, constant price equilibrium."

15. Garratt & Wallace (2018), 1895쪽(§V). 원문: "If d denotes the interest payment (or dividend) and p the constant price, both in units of deposits at the central bank, then the yield on bitcoin is d/p, which for a given d > 0 goes to infinity as p goes to zero. Therefore, such an interest-bearing version of bitcoin cannot have a zero price, which, in turn, rules out the type (ii) and (iv) equilibria in Section III.B ...".

16. Schilling & Uhlig (2019), 20쪽(§3.1), Proposition 1 (Fundamental pricing equation). 원문: "Suppose that sales happen both in the Bitcoin-denominated consumption market as well as the Dollar-denominated consumption market at time t as well as at time t + 1, i.e. suppose that $x_t > 0$ and $x_{t+1} > 0$. Then [식 (1)]. If $u'(c_{t+1}) / P_{t+1}$ is uncorrelated with Q_{t+1}, conditional on time-t information, then the stochastic Bitcoin price process $\{Q_t\}_{t \geq 0}$ is a martingale $Q_t = E_t[Q_{t+1}]$. (2)"

17. Schilling & Uhlig (2019), 20쪽. 원문: "The goods seller will sell against the currency which offers a higher risk-preference adjusted return. Due to the currency competition, sales against both Bitcoins and Dollars implies seller indifference. The risk-adjusted returns, therefore, have to be the same."; "The result is a version of the celebrated result in Kareken and Wallace (1981). These authors did not consider stochastic fluctuations. Our martingale result then reduces to a constant Bitcoin price, $Q_t = Q_{t+1}$, and thus their 'exchange rate indeterminacy result' for time t = 0, that any Q_0 is consistent with some equilibrium, provided the Bitcoin price stays constant afterwards ...".

18. Schilling & Uhlig (2019), 20–21쪽, Proposition 2 (Speculative price bound)·Assumption 3 (Mutual Impatience)·Theorem 1 (No-Bitcoin-Speculation). 정리 1 원문: "Suppose that $B_t > 0$ and $Q_t > 0$ for all t. Impose Assumption 3. Then all Bitcoins are spent and (1) holds in every period t." 이어서: "Consequently, price volatility does not invalidate the medium-of-exchange function of Bitcoin."

19. Schilling & Uhlig (2019), 20–21쪽. 원문: "One can see this immediately, if the economy is non-stochastic. The two conditions of the two Propositions 1 and 2 would require $Q_{t+1} \leq Q_t \leq \beta^2 Q_{t+2} / (\pi_{t+2}\pi_{t+1})$, a contradiction, since β < 1 and $\pi_t \geq 1$ in all t per Assumption 1."

20. Schilling & Uhlig (2019), 21쪽(§4), Corollary 1 (Equilibrium Bitcoin Pricing Formula). 원문: "One immediate implication of Corollary 1 is that the Dollar denominated Bitcoin price process is a supermartingale (falls in expectation) if and only if in equilibrium the pricing kernel and the Bitcoin price are positively correlated for all t + 1 conditional on time t-information. Likewise, under negative correlation, the Bitcoin price process is a submartingale and increases in expectation."

21. Schilling & Uhlig (2019), 22쪽, 식 (8). 원문: "Let a sequence of Dollar goods prices P_t and pricing kernels m_t be given. Pick the initial Bitcoin price Q_0 and pick some sequence ϵ_t with $E_t[\epsilon_{t+1}] = 0$ and some chosen conditional covariance with the pricing kernels. Use Eq. (8) to recursively construct the sequence Q_{t+1}, going forward in time."

22. Schilling & Uhlig (2019), 21쪽 Proposition 3 (Real Bitcoin Disappearance)·22쪽·25쪽(§7). 원문: "In words, the purchasing power of the entire stock of Bitcoin shrinks to zero over time, if inflation is bounded below by a number strictly above one."; "Only a sufficiently high risk premium can avoid the Bitcoin disappearance."; §7: "The result on Bitcoin disappearance may no longer obtain. If over time new currencies enter faster than established currencies devalue, the total purchasing power of all cryptocurrencies may not vanish and can even increase."

23. Bruno Biais, Christophe Bisière, Matthieu Bouvard, Catherine Casamatta & Albert J. Menkveld, "Equilibrium Bitcoin Pricing," *Journal of Finance* 78(2): 967–1014 (2023), 968쪽. 원문: "For example, citizens of Venezuela or Zimbabwe can use Bitcoin to conduct transactions even when their national currencies and banking systems are in disarray. Cryptocurrencies can be used for cross-border transfers when high costs or government controls hinder transfers via traditional financial institutions. However, in addition to providing these transactional benefits, cryptocurrencies come with costs: limited convertibility into traditional currencies, transaction costs on exchanges, fees that agents must pay to have their transactions mined, and the risk of a crash of the cryptocurrency." (게재본).

24. Biais 외 (2023), 968쪽. 원문: "In this sense, transactional benefits are to cryptocurrencies what dividends are to stocks. There is however a major difference. In perfect markets, dividends do not depend on stock prices and provide a real anchor for valuation. In contrast, the transactional benefits provided by a cryptocurrency depend on its price: the higher the price of the cryptocurrency, the stronger is its purchasing power relative to the standard currency, and consequently the higher are the transactional benefits it delivers. Because cryptocurrency prices reflect beliefs about future prices rather than real variables independent of prices, equilibrium can reflect exogenous sunspots."

25. Biais 외 (2023), 969쪽; 모형은 §I(973–975쪽), 명제 1은 §II.A(977쪽, 식 (10)–(12)). 원문: "There are multiple equilibria, one for each possible distribution of the sample path of the sunspot. Within each equilibrium there is extrinsic volatility, since cryptocurrency prices change in response to realizations of the sunspot variable, even when fundamental variables remain constant." 식 (10) $1 + T_{t+1} = (1 + \theta_{t+1}) / (1 + \phi_t)$ 가 정의하는 변수 T_{t+1} 을 저자들은 「암호통화 한 단위당 순거래편익」으로 해석합니다(977쪽). 확인 과정에서 공란으로 둔 「거래편익 흐름」의 위치는 게재본 §II.A 명제 1, 식 (11)–(12), 977쪽입니다.

26. Biais 외 (2023), 970쪽. 원문: "That said, we do not claim statistical significance or estimation of population parameters because of the relatively short size of our sample and the likely non stationarity of our variables."; "Although our calibration quantifies the effect of fundamentals on required expected returns, it also shows that changes in fundamentals explain only a small share (around 5%) of the variance of Bitcoin returns. Under the hypothesis that our model is well-specified and our proxies accurate, this implies that most of Bitcoin return variation reflects extrinsic volatility unrelated to fundamentals." 자료는 §III(보정은 위험중립 가정, Liu and Tsyvinski (2021)를 근거로 든다, 970쪽).

27. Olivier J. Blanchard & Mark W. Watson, "Bubbles, Rational Expectations and Financial Markets," NBER Working Paper 945 (1982-07), 2–4쪽. 원문: "Thus the market price can deviate from its market fundamental value without violating the arbitrage condition."; 식 (4)의 설명: "In each period, the bubble will remain, with probability π, or crash, with probability 1–π. While the bubble lasts, the actual average return is higher than r, so as to compensate for the risk of a crash." (NBER 스캔).

28. Jean Tirole, "Asset Bubbles and Overlapping Generations," *Econometrica* 53(5): 1071–1100 (1985), 1071쪽. 원문: "Since Samuelson [30] developed his consumption loan model it has been well known that there exist economies in which money has a positive value in spite of the fact that it is intrinsically useless (i.e., its market fundamental is zero). In other words there can exist a bubble on money where a bubble is defined as the difference between the market price and the market fundamental." (JSTOR 스캔, 텍스트 층 없음 — 이미지로 직접 읽음). 권·호·쪽은 인쇄면 기준이며 DOI는 메타데이터가 어긋나 붙이지 않습니다.

29. Maarten R.C. van Oordt, "On Bubbles in Cryptocurrency Prices," Tinbergen Institute Discussion Paper TI 2024-050/IV (2024-08-01), 초록. 원문: "Investors in a bubble equilibrium hold the cryptocurrency because they expect its price to appreciate merely due to future investment inflows. Investors who participate in a bubble equilibrium for a cryptocurrency with non-negative money growth experience Ponzi-scheme equivalent payoffs in the aggregate. ... Cryptocurrencies with negative issuance (e.g., that burn transaction fees) may generate positive aggregate cash flows to investors even if their price path follows a bubble trajectory."

30. Biais 외 (2023), 979쪽 각주 18. 원문: "There always exists an equilibrium such that $p_t = 0$ $\forall$t. Here, we

investigate equilibria with strictly positive prices."

31. Garratt & Wallace (2018), 1896쪽(§VI). 원문: "Bitcoin currently has value and its value moves around. Admittedly, none of the equilibria described in this paper provide a good description of its value. However, the nature of the randomness described above could be generalized to enlarge the set of equilibria ... And we could go on and on. That is why we said at the outset that the number of equilibria is huge."

32. Biais 외 (2023), 975쪽. 원문: "In our model, the standard currency price could also go to zero. As Starr (1974) shows, however, this can be prevented when the government levies taxes that must be paid in that currency: even if agents anticipate that others will not to accept the standard currency, as long as they know that the government will accept it for the payment of taxes, demand for the standard currency is strictly positive at prices bounded away from zero. Accordingly, in our analyzes, we focus on equilibria in which the price of the standard currency is strictly positive at all times." (원문의 「will not to accept」는 그대로.)

33. Garratt & Wallace (2018), 1887쪽(§I). 원문: "Many would say that their sufficient conditions are not met by the currencies issued by countries. For example, they did not assume that the taxes levied by a country have to be paid in the form of that country's currency or that some prices denominated in the currency of a country are fixed or sticky. What about bitcoin? Bitcoin and its actual and potential rivals ... do seem to satisfy all the assumptions that Kareken and Wallace made to get exchange-rate indeterminacy."

3부 경제학

20장 보안 예산 — 보상에서 수수료로

핵심 TAKEAWAY

블록체인의 보안은 공짜가 아니라 **매 블록 지불되는 유지비입니다.** 채굴자(또는 검증자)가 매 블록 받는 보상이 정직한 참여자들의 지출을 유인하고, 그 지출의 크기가 곧 공격자가 넘어야 할 벽의 높이입니다. 부디시(Eric Budish)는 이것을 식 셋으로 적습니다. 영이윤 조건 $N^{*}c = p_{block}$, 유인양립 조건 $A^{*}N^{*}c \cdot t(A^{*}) > V_{attack}$, 그리고 둘을 합친 균형 제약 $p_{block} > V_{attack} \,/\, (A^{*}\, t(A^{*}))$. 말로 하면, **블록당 보상이 공격으로 얻을 가치에 견주어 언제나 커야** 합니다. 비트코인의 보상은 보조금(새 코인)과 수수료로 이루어지는데, 보조금은 21만 블록마다 반으로 줄어 0으로 가게 설계되어 있습니다. 그러면 그 자리를 수수료가 메워야 하는데, 칼스텐(Miles Carlsten)과 공저자들은 수수료만 남은 세계에서 채굴 게임 자체가 불안정해진다고 보입니다. 이 장을 읽으면 「51% 공격 비용」을 **어떻게 계산하는지**, 그 계산에 코인 가격이 아니라 어떤 변수가 들어가는지, 그리고 반감기가 왜 가격 사건이 아니라 **보안 예산의 사건인지** 압니다.

학습목표

1. 블록 보상이 보조금과 수수료로 이루어진다는 것, 비트코인의 보조금이 210,000블록마다 반감해 총량이 2,100만에 수렴한다는 것을 Core 소스의 상수로 확인한다.
2. [중급] 부디시의 세 식(영이윤·유인양립·균형 제약)을 읽고, 다수결 공격의 비용이 **흐름(flow)이지 재고(stock)가 아니라는** 뜻을 말한다. 공격 비용을 「해시율 점유율 × 기간 × 단가」로 계산하되, 입력이 해시율·단가·시간이고 코인 가격이 아님을 안다.
3. [중급] 이중지불 공격의 기대 지속시간 $t(A, e, k)$ 를 부디시 명제 3의 식으로 계산하고, 표 II를 재현한다. 17장의 타임밴딧 공격을 같은 틀(식 (3))에 놓되, 명제 3의 설정과 어디가 다른지 안다.
4. [심화] 보조금이 0이 된 세계의 채굴 게임(Carlsten 외) — 채굴 공백, 언더컷, 뒷문 열린 이기적 채굴 — 이 왜 블록 보상 세계와 다른지, 그리고 「전문 자본의 재고 손실」(Budish §V)이 비용 계산을 어떻게 바꾸는지 읽는다.

[입문]**만 읽은 독자가 가져가는 것.** 블록체인의 보안은 공짜가 아니라 **매 블록 지불되는 유지비이고**, 그 유지비의 재원이 새 코인에서 수수료로 바뀌고 있습니다.

1. 이 장이 답하려는 질문

8장에서 작업증명은 「블록 하나를 다시 쓰려면 그만큼의 계산을 다시 해야 한다」는 장치였고, 9장에서 공격자가 정직한 체인을 따라잡을 확률은 점유율 q 가 반을 넘지 않는 한 확인 수 z 와 함께 줄었습니다. 두 장 모두 「**공격자가 그만한 계산력을 갖지 못한다**」**는 전제를** 깔고 있었습니다. 그 전제는 어디서 옵니까. 계산력은 사고 빌릴 수 있는 것입니다. 정직한 참여자들이 지금 쓰고 있는 계산력보다 많은 계산력을 공격자가 모으지 못할 이유는 없습니다. 있는 것은 **비용뿐입니다**. 그러면 질문은 이렇게 바뀝니다. 정직한 참여자들은 왜 그만큼의 계산력을 쓰고 있는가, 그 계산력의 크기는 무엇이 정하는가, 그리고 그것을 넘어서는 비용은 얼마인가.

나카모토의 백서는 이 질문에 반 쪽을 할애했습니다. §6 「유인」입니다. 「관례상 블록의 첫 거래는 블록 생성자가 소유하는 새 코인을 시작하는 특별한 거래다. 이는 노드가 네트워크를 지원할 유인을 더하고, 발행할 중앙 권위가 없으므로 코인을 처음 유통에 넣는 방법을 준다. … 유인은 거래 수수료로도 조달할 수 있다. … 정해진 수의 코인이 유통에 들어가고 나면 유인은 전부 거래 수수료로 옮겨 갈 수 있고 완전히 인플레이션 없는 것이 될 수 있다.」 그리고 공격자에 대해 한 문장. 「탐욕스러운 공격자가 정직한 노드 전부보다 많은 CPU 능력을 모을 수 있다면, 그는 그것을 자기 지급을 되돌려 사람들을 속이는 데 쓸지 새 코인을 만드는 데 쓸지 골라야 할 것이다. 그는 규칙을 따르는 편 — 다른 모든 사람을 합친 것보다 많은 새 코인을 그에게 주는 규칙 — 이 체계와 자기 재산의 유효성을 무너뜨리는 것보다 더 이익임을 알게 될 것이다.」[1] 이 장은 이 반 쪽을 경제학으로 다시 씁니다. 결론부터 적으면, 백서의 마지막 문장은 **조건부로만** 성립하고, 그 조건은 「공격으로 얻을 것이 블록 보상에 견주어 작을 때」입니다.

2. 보상의 두 조각 — 보조금과 수수료

블록 하나를 만든 사람이 받는 것은 둘입니다. 프로토콜이 새로 만들어 주는 코인(**보조금**, block subsidy)과, 그 블록에 담긴 거래들이 낸 **수수료입니다**. 11장에서 본 대로 수수료는 입력 합과 출력 합의 차이이고, 보조금은 코인베이스 거래의 출력입니다(12장의 제네시스 블록에서 그 값은 50 BTC였습니다). 보조금의 규칙은 Bitcoin Core 소스에 상수로 적혀 있습니다. 반감 간격 `nSubsidyHalvingInterval = 210000`, 최초 보조금 `50 * COIN`, 그리고 `GetBlockSubsidy` 함수는 높이를 210,000으로 나눈 몫만큼 50 BTC를 오른쪽으로 비트 이동하며, 그 몫이 64 이상이면 0을 돌려줍니다.[2] 사토시 단위의 정수 50억을 비트 이동하므로 33번째 반감에서 보조금은 1 사토시 아래로 내려가 0이 되고, 총량은 2,100만 BTC에 **못 미치는** 값에 수렴합니다. 아래 NB20-1이 그 합을 정수 산술로 냅니다.

이 규칙이 뜻하는 것은 보상의 **구성비가 시간이 가면서 바뀐다는** 것입니다. 처음에는 거의 전부가 보조금이었고, 나카모토가 적은 대로 끝에는 전부가 수수료가 됩니다. 칼스텐과 공저자들의 2016년 논문은 이 전환을 「블록 보상 체제에서 거래 수수료 체제로」라고 부르고, 그 전환이 「채굴자가 블록 보상으로 25 비트코인을 받든 거래 수수료로 기대값 25 비트코인을 받든 블록체인의 보안에는 무관하다는 검토되지 않은 믿음」을 깬다고 예고합니다.[3] 6절에서 그 논증을 봅니다. 그 전에, 보상이 **얼마여야 하는지를** 정하는 틀이 필요합니다.

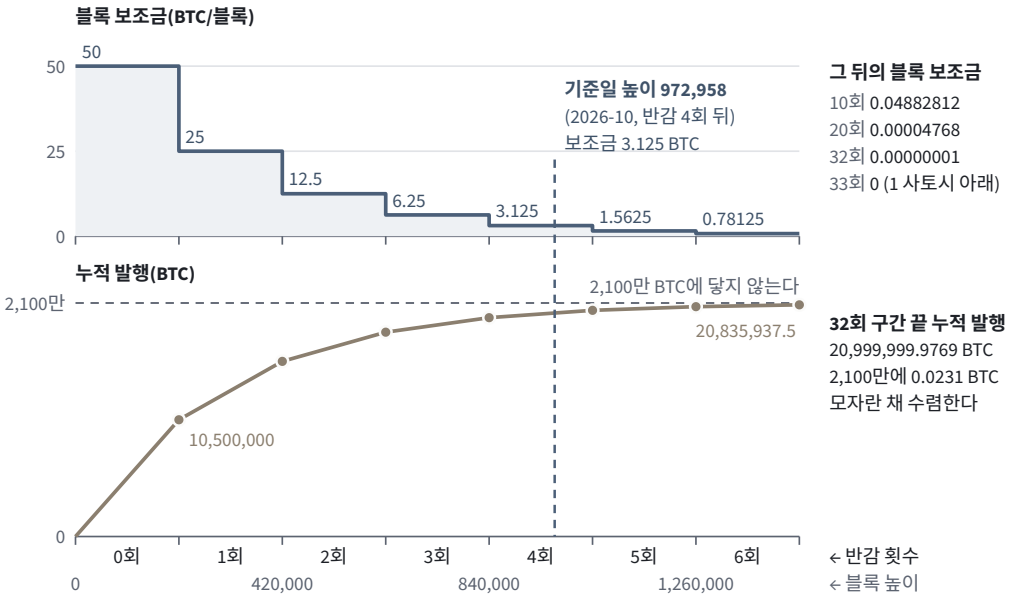
```
# NB20-1: 비트코인 보조금 일정 — Core 의 GetBlockSubsidy 를 정수 산술 그대로 옮긴다 (공부용)
COIN = 100_000_000                      # 1 BTC = 1e8 사토시 (consensus/amount.h)
HALVING = 210_000                       # nSubsidyHalvingInterval (chainparams.cpp)

def block_subsidy_sat(height: int) -> int:
    halvings = height // HALVING
    if halvings >= 64:                  # validation.cpp: 64번 이상 반감하면 0 (비트 이동 폭 보호)
        return 0
    return (50 * COIN) >> halvings      # 50 BTC 를 halvings 번 반으로 — 정수 비트 이동이라 1 사토시 아래는 버려진다

print(f"{'반감 횟수':>6} {'시작 높이':>10} {'블록 보조금(BTC)':>18} {'그 구간 끝 누적 발행(BTC)':>24}")
cum = 0
for h in range(0, 36):
    start = h * HALVING
    sub = block_subsidy_sat(start)
    cum += sub * HALVING
    if h <= 6 or sub == 0 or h in (10, 20, 32, 33, 34):
        print(f"{h:>6} {start:>10,} {sub / COIN:>18.8f} {cum / COIN:>24.8f}")
    if sub == 0:
        break
total = sum(block_subsidy_sat(h * HALVING) * HALVING for h in range(0, 64))
print(f"\n정수 산술로 낸 보조금 총합: {total:,} 사토시 = {total / COIN:.8f} BTC (2,100만에 {21_000_000 - total / COIN:.8f} BTC 모자란다)")
tip = 972_958                           # 2026-10-24 UTC 관측 체인 끝 높이 (data/hashrate_2026-10-25.json)
print(f"기준일 높이 {tip:,}: 반감 {tip // HALVING}회 뒤, 블록 보조금 {block_subsidy_sat(tip) / COIN} BTC, 다음 반감 높이 {(tip // HALVING + 1) * HALVING:,} (약 {((tip // HALVING + 1) * HALVING - tip) * 10 / 60 / 24:.0f}일 뒤, 블록당 10분 가정)")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
 반감 횟수       시작 높이         블록 보조금(BTC)         그 구간 끝 누적 발행(BTC)
     0           0        50.00000000        10500000.00000000
     1     210,000        25.00000000        15750000.00000000
     2     420,000        12.50000000        18375000.00000000
     3     630,000         6.25000000        19687500.00000000
     4     840,000         3.12500000        20343750.00000000
     5   1,050,000         1.56250000        20671875.00000000
     6   1,260,000         0.78125000        20835937.50000000
    10   2,100,000         0.04882812        20989746.09270000
    20   4,200,000         0.00004768        20999989.97250000
    32   6,720,000         0.00000001        20999999.97690000
    33   6,930,000         0.00000000        20999999.97690000

정수 산술로 낸 보조금 총합: 2,099,999,997,690,000 사토시 = 20999999.97690000 BTC (2,100만에 0.02310000 BTC 모자란다)
기준일 높이 972,958: 반감 4회 뒤, 블록 보조금 3.125 BTC, 다음 반감 높이 1,050,000 (약 535일 뒤, 블록당 10분 가정)
```



같은 수수료 수입이라면 반감 하나가 보조금 몫을 반으로 줄인다 — 벽을 그대로 두려면 수수료가 그 몫을 메워야 한다

그림 20-1. 보안 예산의 재원이 줄어드는 일정. 블록 보조금은 210,000블록마다 반으로 줄고, 누적 발행은 2,100만 BTC에 못 미친 채 수렴한다. 값은 NB20-1 출력이고 가격 축은 두지 않았다.

기준일(2026-10 기준) 비트코인은 네 번째 반감 뒤의 구간에 있고 블록 보조금은 3.125 BTC입니다.[4] 이 숫자는 부록 A에 올려 두었습니다. 다음 반감이 오면 1.5625 BTC가 됩니다. 이 장은 반감을 **보안 예산의 사건으로만** 다룹니다. 같은 수수료 수입이라면 반감 하나가 블록당 보상의 보조금 몫을 반으로 줄이고, 3절의 식 (1)이 성립하는 균형에서는 그만큼 공격 비용의 벽을 낮춥니다. 벽을 그대로 두려면 수수료가 그 몫을 메워야 합니다.

3. 부디시의 세 식 — 보안은 흐름이다

[중급] 부디시의 2025년 논문은 「핵심 논증은 식 셋」이라고 스스로 요약합니다.[5] 차례로 읽습니다. 기호는 논문 것을 그대로 씁니다. N 은 「신뢰 지원(trust support)」의 총량 — 작업증명이면 해시율, 지분증명이면 담보로 묶인 코인 — 이고, c 는 그 한 단위를 단위 시간 동안 공급하는 비용, p_{block} 은 블록 하나를 추가한 참여자가 받는 보상(보조금과 수수료를 합친 것)입니다.

첫째, 영이윤 조건. 누구나 들어오고 나갈 수 있으므로 참여자의 이윤은 0으로 경쟁됩니다. 참여자 i 가 다음 블록을 얻을 확률은 점유율 x_i / N 이고, 난이도 D 가 블록당 기대 신뢰지원·시간이면 블록은 평균 D / N 시간마다 나옵니다. 단위 신뢰지원당 단위 시간당 이윤은 $\frac{1}{N}\frac{D}{N}p_{block} - c$ 이고, 블록이 단위 시간에 하나씩 나오게 정규화하면($D^* = N^*$) 명제 1이 됩니다.

$$N^* c = p_{block} \qquad (1)$$

기호 풀이: 정직한 참여자 전체가 단위 시간에 쓰는 비용은 그 시간에 지급되는 블록 보상과 같습니다. 수치 예: 블록당 보상이 3.2 BTC 상당이면 정직한 채굴자 전체가 10분마다 쓰는 전기·장비 비용도 3.2 BTC 상당입니다. 부디시는 이것이 「지대추구 토너먼트의 표준 특성화 — 토너먼트의 상금 p_{block} 은 그 상금을 얻으려는 지출 $N^* c$ 로 소진된다」이며, 「Prat and Walter (2021)가 식 (1)이 비트코인 채굴 시장의 실제 균형 행동을 서술한다는 실증적 뒷받침을 준다」고 적습니다.[6] 8장의 난이도 조정이 여기서 하는 일을 보십시오. 해시율이 늘어도 블록 수는 늘지 않으므로(실링–울리히가 「해시율 증가는 총 비트코인 생산에

영향이 없다」고 적은 바로 그 성질[21]), 늘어난 지출은 보상을 나눠 가질 뿐이고, 그래서 이윤이 0이 될 때까지 지출이 늘거나 줄어듭니다.

둘째, 유인양립 조건. 정직한 참여자 전부가 아니라 **공격자** 하나를 더합니다. 그는 AN^* 단위($A > 1$)를 동원해 $\frac{A}{A+1}$ 의 다수를 얻고, 단위 시간에 AN^*c 를 씁니다. 공격이 끝나기까지 기대 시간을 $t(A)$ 라 하면 **총 공격비용은** $AN^*c \cdot t(A)$ 이고, 공격자는 $A \cdot t(A)$ 를 최소로 하는 A^* 를 고릅니다. 공격으로 얻는 가치를 V_{attack} 이라 하면

$$A^* N^* c \cdot t(A^*) > V_{attack} \qquad (2)$$

가 「총비용 기준으로 외부 공격자에 대해 유인양립」인 조건입니다.[7]

셋째, 균형 제약. N^*c 가 두 식에 다 들어 있으므로 (1)을 (2)에 넣으면 정리 1이 나옵니다.

$$p_{block} > \frac{V_{attack}}{A^* \cdot t(A^*)} \qquad (3)$$

「말로 하면, 블록체인의 신뢰 지원에 대해 참여자에게 지급되는 블록당 균형 보상은 블록체인을 공격해서 얻을 이익에 견주어 커야 한다.」[8] 기호 풀이: 분모 $A^* t(A^*)$ 는 「공격 한 번이 정직한 블록 몇 개 분량의 비용인가」를 재는 순수한 수(단위 없음)이고, 4절의 표 II가 그 값을 줍니다. 수치 예는 그 표를 보고 적겠습니다. 여기서 가져갈 것은 식의 모양입니다. 오른쪽에 **시간이** 들어 있습니다. 공격자는 공격이 지속되는 동안만 비용을 내고, 그 뒤에는 아무것도 잃지 않습니다. 부디시의 말로 「나카모토 신뢰는 기억이 없다 — 어느 순간의 보안은 그 순간의 신뢰 지원 크기만큼이다.」 보안이 **재고(stock)가 아니라 흐름(flow)입니다.** 그래서 「비자 네트워크의 사용자가 10분마다, 비자 네트워크를 성공적으로 공격해서 얻을 가치에 견주어 큰 수수료를 비자에 내야 한다고 상상해 보라. 브랜드가 광고에 대한 흐름 투자만큼만 신뢰받는다고 상상해 보라. 나라가 국경의 병사에 대한 흐름 지출만큼만 안전하다고 상상해 보라」는 비유가 따라옵니다.[9] 전통적 신뢰 — 법, 평판, 담보 — 와의 차이는 **규모의 경제입니다.** 법원은 처벌의 위협만으로 많은 범죄를 억제하고, 경찰은 한 은행 앞에 서 있지 않아도 여러 은행을 지킵니다. 나카모토 신뢰에는 그런 지렛대가 없습니다. 「공격이 1,000배 매력적이 되면 체계를 지키는 비용도 1,000배가 되어야 한다.」[10]

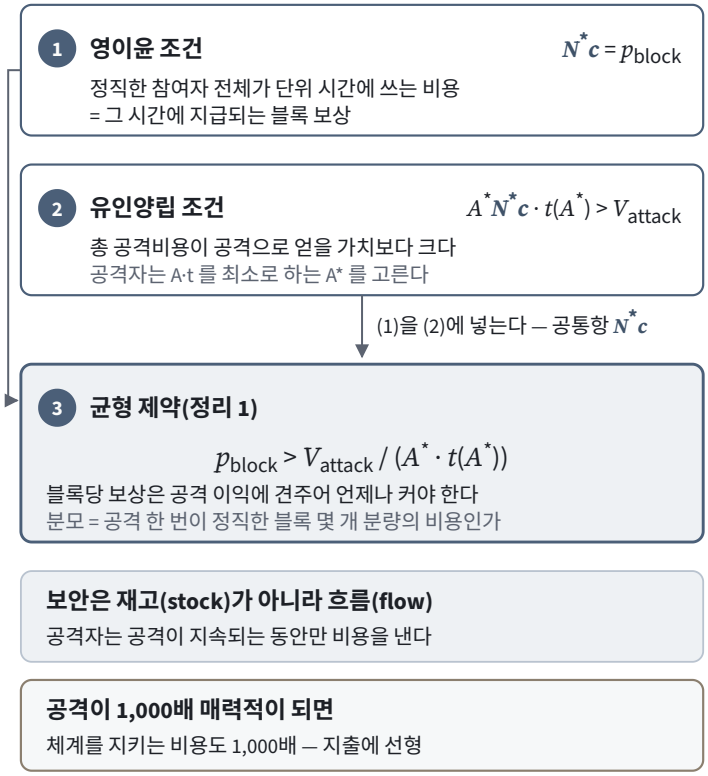


그림 20-2. 부디시의 세 식이 이어지는 모양. 영이윤 조건 (1)과 유인양립 조건 (2)가 공통항 $N^{*}c$ 로 합쳐져 균형 제약 (3)이 된다.

[중급] 여기서 10장과 이어지는 단서 하나를 적어 둡니다. 부디시는 이 세 식이 작업증명에 한정되지 않는다고 거듭 말합니다. 「신뢰 지원은 비트코인처럼 계산 작업을 제공하는 형태일 수도, … 이더리움 같은 지분증명 프로토콜에서 코인을 거는 형태일 수도 있다.」 지분증명이면 c 는 「단위 지분을 단위 시간 공급하는 기회비용」입니다. 다만 지분에는 작업과 다른 점이 하나 있습니다. 「지분은 계산 작업만큼 기억이 없지 않다. 지분은 담보처럼 일정 기간 체인에 잠겨 있고 … 그래서 공격자의 잠긴 지분을 몰수(슬래싱)해 처벌할 가능성이 열린다.」[11] 그것이 7절의 「재고 손실」입니다.

4. 공격은 얼마나 걸리고 얼마가 드는가 — 명제 3과 표 II

[중급] 식 (3)의 분모 $A^{*}t(A^{*})$ 를 채우려면 공격이 얼마나 걸리는지 알아야 합니다. 부디시는 이중지불 공격을 다섯 단계로 적습니다. 「(i) 공격자가 재화나 자산을 받고 암호통화를 보낸다 … (ii) 그 거래들이 통상대로 블록체인에 추가되게 둔다. (iii) 그 거래를 담지 않은 대안 최장 체인을 비밀리에 만든다 … (iv) 에스크로 기간이 지나기를 기다린다 … (v) 대안 최장 체인을 공개한다.」 그리고 9장의 확률 계산을 **시간의 기대값으로** 바꿉니다. 공격자가 거래를 심은 블록 수를 k, 에스크로(받는 쪽이 기다리는 확인 수)를 e 라 하고, 정직한 쪽이 단위 시간에 블록 하나, 공격자가 A 개를 포아송으로 만든다고 하면, 명제 3은

$$t(A, e, k) = (k+e) + \sum_{i=0}^{k+e} \left(\frac{i+1}{A-1}\right) \cdot \frac{(2(k+e)-1-i)!}{(k+e-i)!\,(k+e-1)!} \left(\frac{A}{1+A}\right)^{k+e-i} \left(\frac{1}{1+A}\right)^{k+e} \qquad (6)$$

입니다.[12] 기호 풀이: 첫 항 $k+e$ 는 공격자의 최선의 경우(정직한 쪽이 $k+e$ 블록을 만드는 사이 이미 앞서 있음)이고, 합의 각 항은 「그 시점에 공격자가 i 블록 뒤처져 있을 확률 × 그 뒤처짐을 따라잡는 기대 시간 $(i+1)/(A-1)$」입니다. $A \to \infty$ 면 $t \to k+e$, $A \to 1^{+}$ 면 $t \to \infty$ 입니다. 부디시는 「Grunspan and Pérez-Marco (2018, 2022)가 공격자의 해시파워가 50% 미만인 경우에 대해 밀접히 관련된 수학적

결과를 끌어냈다」고 적어 두는데, 9장에서 우리가 읽은 바로 그 논문입니다.[13]

```
# NB20-2: 부디시 명제 3 — 이중지불 공격의 기대 지속시간 t(A, e, k) 와 총비용 A·t 를 정직한 블록 보상 단위로
(공부용)
# 계산에 들어가는 것은 공격자 배수 A(점유율 A/(1+A))와 블록 수 k+e 뿐이다. 코인 가격·법정통화 단가는 없다.
from math import lgamma, log, exp

def attack_duration(A: float, ke: int) -> float:
    """식 (6). 계승은 로그감마로 — 큰 ke 에서 넘침을 피한다. 단위: 정직한 블록 시간(≈10분)."""
    t = float(ke)
    for i in range(0, ke + 1):
        log_comb = lgamma(2 * ke - i) - lgamma(ke - i + 1) - lgamma(ke)   # (2ke-1-i)! /
((ke-i)! (ke-1)!)
        log_prob = (ke - i) * log(A / (1 + A)) + ke * log(1 / (1 + A))
        t += (i + 1) / (A - 1) * exp(log_comb + log_prob)
    return t

rows = [1.05, 1.10, 1.20, 1.33, 1.50, 2.00, 5.00]
cols = [1, 6, 12, 36, 144]
print("기대 지속시간 t (단위: 정직한 블록 시간). 열 = k+e (이중지불 블록 수 + 에스크로)")
print(f"{'A (점유율)':>14}" + "".join(f"{c:>10}" for c in cols))
for A in rows:
    print(f"{A:>5.2f} ({A/(1+A):4.0%})" + "".join(f"{attack_duration(A, c):>10.2f}" for c in
cols))
print("\n총비용 A·t (단위: 블록당 정직한 신뢰지원 비용 N*c = 블록 보상 p_block)")
print(f"{'A (점유율)':>14}" + "".join(f"{c:>10}" for c in cols))
for A in rows:
    print(f"{A:>5.2f} ({A/(1+A):4.0%})" + "".join(f"{A * attack_duration(A, c):>10.2f}" for c in
cols))

# 공격자는 A·t 를 최소로 하는 A 를 고른다. k+e = 12(논문의 기본 사례)에서 격자 탐색
ke = 12
best = min(((A / 100) * attack_duration(A / 100, ke), A / 100) for A in range(101, 600))
print(f"\nk+e = {ke}: A·t 를 최소로 하는 A* ≈ {best[1]:.2f} (점유율 {best[1]/(1+best[1]):.0%}), 최소
A*·t(A*) ≈ {best[0]:.2f} 블록 보상")
print(f"→ 식 (3): p_block > V_attack / {best[0]:.1f}, 곧 블록당 보상이 공격 가치의 {1/best[0]:.2%} 를
넘어야 한다")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
기대 지속시간 t (단위: 정직한 블록 시간). 열 = k+e (이중지불 블록 수 + 에스크로)
       A (점유율)         1         6        12        36       144
 1.05 ( 51%)     25.51     42.43     56.59     97.91    229.61
 1.10 ( 52%)     13.02     22.88     31.73     60.14    167.31
 1.20 ( 55%)      6.79     13.24     19.69     43.10    146.79
 1.33 ( 57%)      4.34      9.60     15.35     38.04    144.18
 1.50 ( 60%)      3.08      7.84     13.43     36.47    144.00
 2.00 ( 67%)      1.89      6.45     12.20     36.01    144.00
 5.00 ( 83%)      1.12      6.00     12.00     36.00    144.00

총비용 A·t (단위: 블록당 정직한 신뢰지원 비용 N*c = 블록 보상 p_block)
       A (점유율)         1         6        12        36       144
 1.05 ( 51%)     26.78     44.56     59.42    102.81    241.09
 1.10 ( 52%)     14.32     25.16     34.90     66.16    184.04
 1.20 ( 55%)      8.14     15.89     23.63     51.72    176.15
 1.33 ( 57%)      5.78     12.77     20.41     50.59    191.76
 1.50 ( 60%)      4.62     11.76     20.14     54.71    216.01
 2.00 ( 67%)      3.78     12.89     24.39     72.02    288.00
 5.00 ( 83%)      5.59     30.02     60.00    180.00    720.00

k+e = 12: A·t 를 최소로 하는 A* ≈ 1.43 (점유율 59%), 최소 A*·t(A*) ≈ 20.00 블록 보상
→ 식 (3): p_block > V_attack / 20.0, 곧 블록당 보상이 공격 가치의 5.00% 를 넘어야 한다
```

[중급] 이 출력은 부디시의 표 II를 소수 둘째 자리까지 재현합니다(논문 36쪽). 읽는 법은 이렇습니다. 51.2% 점유율($A = 1.05$)의 공격자는 통계적으로는 결국 이기지만, $k + e = 12$ 에서 평균 56.6블록(약

9.5시간)이 걸리고 비용은 블록 보상 59.4개분입니다. 점유율을 60%로 올리면 13.4블록, 20.1개분으로 줄어듭니다. 부디시가 적은 대로 「51% 다수가 공격이 결국 성공함을 통계적으로 보장하기에 충분한 것은 맞지만, 비용을 최소화하는 공격자는 그보다 다소 큰 다수를 고를 것」입니다.[14] 마지막 줄이 식 (3)의 수치 예입니다. $k + e = 12$ 에서 $A^* \approx 1.43$, $A^* t(A^*) \approx 20.0$ 이고, 그래서 **블록당 보상이 공격 가치의 5.00%를 넘어야** 합니다. 부디시의 표 III이 이 5.00%에서 출발합니다. 「블록당 5%는 하루로 720%, 한 해로 약 263,000%가 된다. … 10억 달러 공격에 대해 체계를 지키려면 연 2.63조 달러의 보안 지출이 필요하다.」 반대로 에스크로가 길어지면 비용은 줄어, $e + k$ 가 한 주(1,008블록)면 블록당 0.09%입니다.[15] 이 달러 숫자들은 V_{attack} 을 가정한 **사고실험의 값이지** 어떤 자산의 가격이 아닙니다. 이 책이 가져가는 것은 비례 관계입니다. 지킬 가치가 열 배면 보안 예산도 열 배여야 하고, 그것은 사용자가 수수료로 내거나 보유자가 발행으로 부담해야 합니다.

순비용은 0일 수도 있다

[중급] 위 계산은 **총비용입니다**. 공격자는 공격 중에도 블록을 만들고, 그의 체인이 이기면 그 블록들의 보상을 받습니다. 부디시의 정리 2가 극단을 적습니다. 「공격자가 정직한 참여자에 견주어 어떤 비용 마찰도 겪지 않고($\kappa = 0$), 공격이 난이도 조정 없이 끝나며($D' = D^*$), 공격이 암호통화의 가치를 떨어뜨리지 않으면($\Delta_{attack} = 0$), **순 공격비용은 0이다**.」 증명은 한 줄입니다. 공격자의 비용 $At \cdot N^* c$ 와 그가 받는 보상 $At \cdot p_{block}$ 이 식 (1)에 의해 같습니다. 「직관은 이렇다. 정직한 참여자가 정직한 행동 아래 신뢰 지원 비용을 전부 보상받는 것과 같은 이유로 공격자도 신뢰 지원 비용을 전부 보상받는다. 허가 없는 합의는 사실상 공격자를 정직한 참여자처럼 대우한다. 다수가 진실을 정하기 때문이다.」[16] 저자 스스로 「마찰 0은 비현실적」이라고 적지만, 이 정리는 9장의 확률 계산이 놓친 것을 드러냅니다. 공격자에게 가장 비싼 것은 해시가 아니라 **마찰**(덜 효율적인 장비, 시작과 중단의 비용)과 **가치 하락입니다**.

17장의 타임밴딧은 같은 계산이다

[중급] 17장은 다이안과 공저자들의 **타임밴딧 공격을** 적어 두고 비용 계산을 이 장으로 넘겼습니다. 「블록체인에 끝 구간(부분 사슬) $[height_0, height_1]$ 이 있고 현재 블록 높이가 $height_1$ 이며, 거기서 훔칠 수 있는 가치가 블록 보상을 넘는다고 하자. 공격자는 $height_0$ 로 되감아, 거기서 나오는 MEV로, $height_1$ 까지 또는 그 너머로 포크를 채굴하는 수익성 있는 51% 공격을 보조할 수 있다.」[17] 부디시의 틀에 놓으면 V_{attack} 이 그 구간의 추출 가능 가치이고, 식 (3)의 논리 — 공격의 이익이 공격 기간의 블록 보상 지출을 넘는가 — 가 그대로 적용됩니다. 다만 명제 3의 설정과 같지는 않습니다. 명제 3의 공격자는 이중지불 시점부터 정직한 체인과 **나란히** 비밀 체인을 쌓지만, 타임밴딧은 이미 확정된 $height_1 - height_0$ 블록만큼 **뒤처진 채** 출발하므로 따라잡는 데 드는 기대 블록 수는 표 II의 값보다 큽니다(이 책은 그 식을 유도하지 않습니다). 다이안 외가 적은 조건은 「훔칠 수 있는 가치가 블록 보상을 넘는다」는 것뿐이고, 17장이 「응용층의 보안이 합의층 보안에 현재적이고 직접적인 위협」이라고 적은 문장은 이 비교의 모양을 가리킵니다. 그리고 그 문턱은 **블록 보상이 작아질수록** 낮아집니다. 21장이 다루는 이더리움의 기본 수수료 소각은 이 자리를 겨냥한 설계 가운데 하나입니다.

5. 공격 비용 계산기 — 입력은 해시율·단가·시간

[중급] 이제 설계가 요구한 계산기를 만듭니다. 입력은 셋입니다. 기준일의 네트워크 해시율 H(데이터 동봉), 공격자가 동원하는 추가 해시율의 배수 A(또는 점유율 $q = A/(1+A)$), 그리고 **사용자가 넣는** 단위 해시율의 단위 시간당 비용 c 입니다. 출력은 $A \cdot H \cdot c \cdot t(A, e, k)$ 입니다. 코인 가격은 어디에도 들어가지 않습니다. 비용의 **단위는** 사용자가 c 를 무엇으로 넣느냐가 정합니다. 전력 단가와 장비 임대료를 넣으면 법정통화가 되고, 아무것도 모르면 아래처럼 「블록 보상 단위」로 두면 됩니다. 식 (1)이 그것을 허락합니다. $N^* c = p_{block}$ 이므로 공격 비용은 $A \cdot t \cdot p_{block}$, 곧 「블록 보상 몇 개분」입니다.

```
# NB20-3: 공격 비용 계산기 — 입력은 해시율(스냅샷)·공격자 배수·단가·기간. 코인 가격은 입력이 아니다 (공부용)
import json, os

# 기준일 해시율 스냅샷(데이터 동봉; 파일이 없으면 같은 값의 상수로 대체 — Colab 에서 파일 없이 실행 가능)
try:
    with open(os.path.join("data", "hashrate_2026-10-25.json"), encoding="utf-8") as f:
        snap = json.load(f)
    H_network = float(snap["estimate"]["hashrate_H_per_s"])
except FileNotFoundError:
    H_network = 1.006294e21                 # H/s — 난이도 140,577,680,051,919.6 에서 D·2^32/600 으로 추정(600초 가정)
print(f"기준일 추정 해시율 H = {H_network:.3e} H/s = {H_network/1e18:,.0f} EH/s (2026-10-24 UTC, 600초 가정 추정 — 비교값보다 약 7% 작다)")

def attack_cost(A, ke, c_per_hash_second=None, p_block=None):
    """총비용. c_per_hash_second 를 주면 「단가 단위 × 해시·초」, p_block 을 주면 식 (1)로 「블록 보상 단위」."""
    t_blocks = attack_duration(A, ke)
    seconds = t_blocks * 600
    out = {"t_blocks": t_blocks, "hours": seconds / 3600, "attacker_hashrate_hs": A * H_network}
    if c_per_hash_second is not None:
        out["cost_in_price_units"] = A * H_network * c_per_hash_second * seconds
    if p_block is not None:
        out["cost_in_block_rewards"] = A * t_blocks            # 식 (1): N*c = p_block
        out["cost_in_coin_of_reward"] = A * t_blocks * p_block
    return out

p_block_subsidy_only = 3.125                 # 기준일 보조금(BTC). 수수료 몫 F 는 블록마다 다르므로 여기서는 0 으로 둔 하한
print(f"\n(가) 블록 보상 단위 — 수수료를 0 으로 둔 하한, p_block = {p_block_subsidy_only} BTC(보조금만)")
print(f"{'점유율':>6} {'A':>5} {'k+e':>4} {'기대 블록':>9} {'시간(h)':>8} {'블록 보상 개수':>12} {'BTC 상당(보조금만)':>18}")
for q in [0.55, 0.60, 0.67]:
    A = q / (1 - q)
    for ke in [6, 12, 144]:
        r = attack_cost(A, ke, p_block=p_block_subsidy_only)
        print(f"{q:>6.0%} {A:>5.2f} {ke:>4} {r['t_blocks']:>9.1f} {r['hours']:>8.1f} {r['cost_in_block_rewards']:>12.1f} {r['cost_in_coin_of_reward']:>18.1f}")

print("\n(나) 사용자 단가 단위 — c 는 해시 1회당 비용이 아니라 「1 H/s 를 1초 돌리는 비용」. 아래 값은 자리만 채운 가상의 단가다.")
c_demo = 1e-19                               # 가상 단가(단위 없음). 실제 전력·장비 단가는 사용자가 넣는다
for q in [0.55, 0.60]:
    A = q / (1 - q)
    r = attack_cost(A, 12, c_per_hash_second=c_demo)
    print(f"  점유율 {q:.0%}, k+e=12: 공격자 해시율 {r['attacker_hashrate_hs']:.2e} H/s × {r['hours']:.1f}시간 × 단가 → 비용 {r['cost_in_price_units']:,.0f} (단가 단위)")
print("\n같은 점유율·같은 기간에서 비용은 단가 c 에 비례한다. 단가를 반으로 넣으면 비용도 반 — 이 계산기가 말하는 것은 수준이 아니라 비례다.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
기준일 추정 해시율 H = 1.006e+21 H/s = 1,006 EH/s (2026-10-24 UTC, 600초 가정 추정 — 비교값보다 약 7% 작다)

(가) 블록 보상 단위 — 수수료를 0 으로 둔 하한, p_block = 3.125 BTC(보조금만)
   점유율      A  k+e     기대 블록    시간(h)     블록 보상 개수       BTC 상당(보조금만)
   55%  1.22    6      12.3      2.0         15.0               47.0
   55%  1.22   12      18.5      3.1         22.7               70.8
   55%  1.22  144     145.8     24.3        178.2              556.7
   60%  1.50    6       7.8      1.3         11.8               36.7
   60%  1.50   12      13.4      2.2         20.1               62.9
   60%  1.50  144     144.0     24.0        216.0              675.0
   67%  2.03    6       6.4      1.1         13.0               40.7
   67%  2.03   12      12.2      2.0         24.7               77.3
   67%  2.03  144     144.0     24.0        292.4              913.6

(나) 사용자 단가 단위 — c 는 해시 1회당 비용이 아니라 「1 H/s 를 1초 돌리는 비용」. 아래 값은 자리만 채운 가상의 단가다.
  점유율 55%, k+e=12: 공격자 해시율 1.23e+21 H/s × 3.1시간 × 단가 → 비용 1,368,343 (단가 단위)
  점유율 60%, k+e=12: 공격자 해시율 1.51e+21 H/s × 2.2시간 × 단가 → 비용 1,215,857 (단가 단위)

같은 점유율·같은 기간에서 비용은 단가 c 에 비례한다. 단가를 반으로 넣으면 비용도 반 — 이 계산기가 말하는 것은 수준이 아니라 비례다.
```

[중급] (가)의 「블록 보상 개수」 열에서 눈여겨볼 것은 「블록 보상 개수」가 **보상의 크기와 무관하다는** 점입니다. 점유율 60%, $k + e = 12$ 이면 20.1개분입니다. 보조금이 반감으로 반이 되면 그 20.1개분의 BTC 상당도 반이 됩니다 — 수수료를 0으로 둔 이 표에서는(수수료가 양이면 그만큼 덜 줄어듭니다). 이것이 「반감기는 보안 예산의 사건」이라는 말의 뜻입니다. (나)의 단가는 **자리만 채운 가상의 값입니다**. 실제 전력 단가나 장비 임대료를 이 책은 적지 않습니다. 이 장의 위젯 W6은 점유율·단가·기간 세 변수만 움직이고, 기준일 해시율은 동봉 스냅샷에서 읽으며, 가격 입력란을 두지 않습니다.

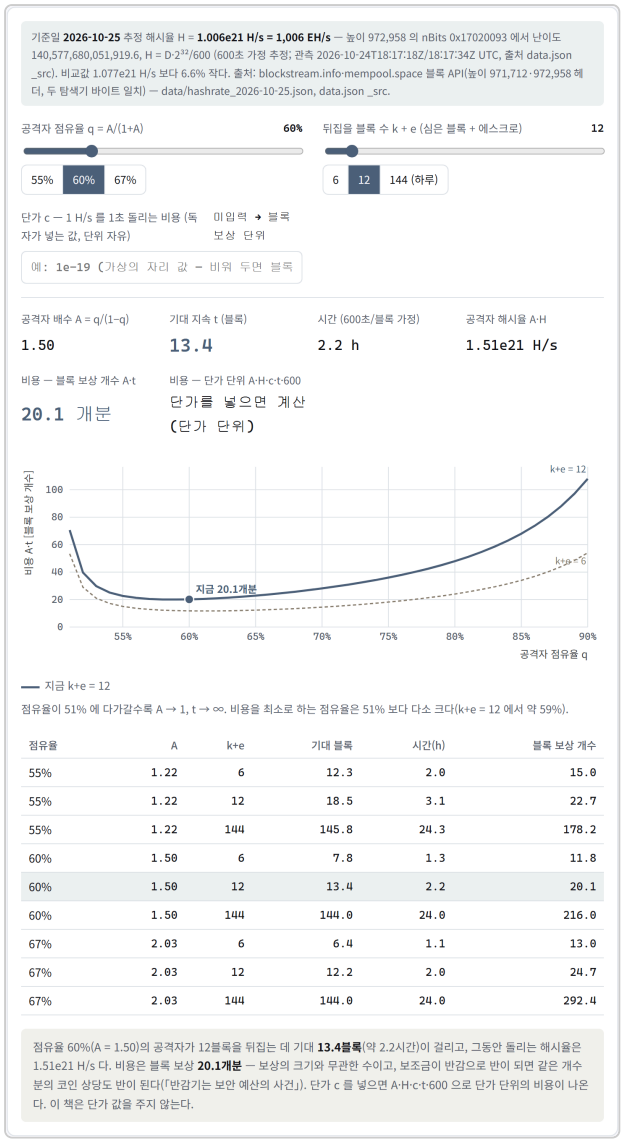


그림 20-3. 51% 공격 비용 계산기. 기준일(2026-10-25) 추정 해시율 1,006 EH/s 에서 점유율 60%, k + e = 12 이면 기대 13.4블록(2.2시간), 비용은 블록 보상 20.1개분이다. 해시·전력 단가는 독자가 넣고 이 책은 값을 주지 않는다. — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 20장 5절).

6. [심화] 보조금이 0이 된 세계 — Carlsten 외

▾ [심화] 수수료만 남으면 채굴 게임은 어떻게 달라지는가

[심화] 칼스텐과 공저자들은 「블록 보상이 0으로 줄어든 뒤의 비트코인」을 모형으로 삼습니다. 수수료는 단위 시간당 일정하게 쌓이고(길이 t 인 구간에 발표된 거래의 수수료 합이 t), 난이도는 단위 시간에 블록 하나가 나오게 맞춰지며, 지연은 없다고 둡니다. 저자들은 이 단순화를 일부러 택했다고 적습니다. 「우리의 결과는 단순화 가정으로 더 강해질 뿐이다. 이상하고 바람직하지 않은 결과가 단순화 가정을 허락하더라도 생긴다고 주장하는 것이기 때문이다.」[18]

[심화] 첫째, 채굴 공백(mining gap). 「블록 보상이 없으면, 블록이 발견된 직후에는 채굴의 기대 보상이 0인데 전기비용은 0이 아니어서 어떤 채굴자에게도 채굴이 이익이 되지 않는다.」 채굴기 하나가 전체 유효 해시파워의 a 를 차지하고 단위 시간 전기비가 p 라면 「블록이 발견된 뒤 $t = p / a$ 시간이 지나야 채굴이 다시 이익이 된다.」 그 결과 「채굴자들은 블록 사이 시간의 점점

작은 부분만 채굴하게 되고(난이도가 그에 맞춰 떨어진다), … 네트워크의 유효 해시파워가 떨어져 악의적 채굴자가 포크하기 쉬워진다.」[19] 블록 보상이 있을 때는 블록 직후에도 보상이 있어 이 공백이 없습니다.

[심화] **둘째, 언더컷(undercutting).** 보상이 블록마다 다르면 「어느 블록 위에 쌓을 것인가」가 전략이 됩니다. 포크가 났을 때 기본 클라이언트는 먼저 들은 블록을 잇지만, 「남은 거래 수수료가 가장 많은 블록을 잇는」 편이 엄밀히 낫습니다(PettyCompliant). 그런 채굴자가 일부라도 있으면, 「체인의 머리 블록이 남겨 둔 미청구 수수료보다 더 가치 있을 때 일부러 포크해 수수료를 조금 덜 가져가는」 전략(언더컷)이 열립니다. 저자들은 (i) 학습하는 비원자적 채굴자들이 시뮬레이션에서 점점 공격적인 언더컷으로 흘러가고, (ii) 그럼에도 모든 채굴자가 같은 함수형 전략을 쓰는 균형이 존재하며 — 정리 5.1, 람베르트 W 함수로 적힌 $f(x)$ — 「그 균형에서 n 시간 뒤 적체된 거래의 기대 수는 $\theta(\sqrt{n})$」, 곧 **처리되지 않은 거래가 끝없이 쌓인다는** 것을 보입니다. 그리고 (iii) 「채굴자의 66%가 기본 전략을 고수해도 언더컷은 이익」이라는 분석적 결과를 더합니다. 기본 전략의 비율이 y 일 때 언더컷의 기대 보상이 1을 넘는 조건이 $y \le 2/3$ 이기 때문입니다.[20]

[심화] **셋째, 이기적 채굴의 뒷문.** 23장에서 읽을 이기적 채굴(Eyal–Sirer)은 블록 보상 세계에서 점유율 문턱이 있습니다. 수수료 세계에서는 「선택적으로」 숨길 수 있습니다. 「블록 직후에 블록을 찾아 아직 새 거래가 없다면 그 블록은 말 그대로 무가치하다. 공개하는 대신 이기적 채굴에 써 보지 않을 이유가 있는가? 비용은 없고, 해시파워와 무관하게 2블록 앞설 확률은 양(+)이다. 반대로 … 한 시간 치 거래를 담은 블록은 「보통」 블록 여섯 개 값이니 왜 잃을 위험을 지겠는가?」 그래서 「작은」 블록만 숨기는 Selfish-Mine(β) 전략은 「모든 $\alpha, \gamma < 1$ 에서 기본 채굴과 고전적 이기적 채굴을 엄밀히 능가」하고, 「임의로 낮은 해시파워와 임의로 나쁜 연결」에서도 이익이며, 「배포 즉시 이익」입니다(고전적 이기적 채굴은 난이도 조정 뒤에야 이익). $\alpha = 1/3$ 에서 「기본 채굴과 이기적 채굴은 기대 보상 약 1/3이지만 최적 문턱의 이기적 채굴은 약 0.38, 13.6% 증가」입니다.[22]

[심화] 저자들이 서론 끝에 적은 것은 설계 권고입니다. 「나카모토는 블록 보상을 중앙 권위 없이 비트코인의 초기 배분을 이루기 위한 필요하지만 일시적인 악으로 본 듯하고, 거래 수수료 체제를 체계의 이상적이고 인플레이션 없는 정상 상태로 본 듯하다. 그러나 우리 연구는 거래 수수료 체제에서 순응하는 채굴자 행동을 유인하는 일이 블록 보상 체제에서보다 훨씬 더 벅찬 과제임을 보인다. 아마도 새 암호통화의 설계자들은 통화 인플레이션의 불가피성을 받아들이고 블록 보상을 영구적으로 두어야 할 것이다.」[23] 이더리움의 EIP-1559가 동기 절에서 든 네 문제 중 하나가 바로 이것입니다. 「발행이 없는 블록체인(비트코인과 Zcash 포함)은 장기적으로 채굴자 보상을 전적으로 거래 수수료로 전환하려 하는데, 알려진 문제들이 있어 큰 불안정을 낳을 가능성이 크다. 수수료를 훔치는 「자매 블록」 채굴을 유인하고, 훨씬 강한 이기적 채굴 공격 경로를 열며 … 현재로서는 좋은 완화책이 없다.」[24] 그 설계의 나머지는 21장입니다.

7. [심화] 재고가 걸려 있으면 — 전문 자본과 붕괴

▾ [심화] 공격자가 자본을 잃는다면 비용은 몇 자릿수 커지는가

[심화] 부디시의 §V는 3절의 비용 모형에 빠진 것을 더합니다. 「블록체인을 유지하는 데 쓰이는 자본이 전문화되어 있고(재용도가 불가능하고), 다수결 공격이 공격자의 자본 가치를 잃게 하거나 몰수되게 한다면, 공격자 비용 모형을 고쳐야 한다. 공격의 흐름 비용에 더해 전문 자본의 재고 가치 손실도 공격자에게 물려야 한다.」 비트코인의 ASIC(「SHA-256 해시를 많이 하는 것 말고는 쓸모가 없는」 칩)과 지분증명의 지분이 그런 자본입니다. 비용을 $c = rC + \eta$(자본의 임대비용 + 가변비용)로 쪼개면 식 (1)은 $N^{*}(rC + \eta) = p_{block}$ (7)이 되고, 자본을 전부 잃는 외부 공격자의 근사 조건은 $N^{*}C > V_{attack}$ (8), 자본 몫을 $\mu = rC \,/\, (rC + \eta)$ 라 하면 $N^{*}C = \mu p_{block} \,/\, r$ (9)이므로

$$p_{block} > \frac{r}{\mu} V_{attack} \qquad (10)$$

입니다. 「이것은 식 (3)보다 몇 자릿수 더 안전하다. r 이 단위 시간당 이자율이기 때문이다.」 부디시의 예시 계산은 $\mu = 0.4$, ASIC 연 할인율 50%로 블록당 $r \approx 0.0008\%$, $r \,/\, \mu = 0.002\%$ 이고, 이를 식 (3)의 5.0%와 견주면 「약 2,500배의 보안 비용 개선」입니다.[25]

[심화] 그러면 질문은 「공격자는 어떻게 자본을 잃는가」가 됩니다. 부디시는 셋을 듭니다. (1) 공격이 통화 가치를 무너뜨려 자본도 무너지는 경우 — 「그러나 붕괴에 취약하다는 것 자체가 심각한 문제이고 … 붕괴 자체를 노린 공격(사보타주)의 가능성을 연다.」 (2) 법적 처벌 — 「확실히 작동하겠지만 나카모토 신뢰가 법의 지배 없이는 실패한다는 것을 인정하는 셈이다.」 (3) 프로토콜이 알고리듬으로 공격자의 자본만 골라 몰수하는 것 — 동반 논문(Budish, Lewis-Pye & Roughgarden 2024)은 「이것이 비트코인을 포함하는 넓은 부류의 프로토콜에서는 불가능하고, 지분증명 이더리움을 포함하는 부류에서는 네트워크 신뢰성에 대한 강한 가정 아래 공격자가 총량의 $\rho = 2 \,/\, 3$ 미만일 때만 가능하다」고 보입니다.[26] 10장의 슬래싱이 이 자리에 있습니다. 2/3 미만의 공격자에게는 공격이 흐름이 아니라 재고를 걸게 만들 수 있지만, 그보다 큰 공격자에게는 외부의 신뢰 — 법 — 가 필요하다는 것이 부디시의 독법입니다.

? 법률가의 질문

보조금이 210,000블록마다 반감한다는 규칙은 코드에 적혀 있고, 코드는 다수가 바꿀 수 있습니다(23장). 이 장은 보조금이 보안 예산의 재원이고 그 재원이 수수료로 옮겨 가는 과정이 불안정할 수 있다고 적었습니다. 그렇다면 어느 날 다수의 노드와 채굴자가 「보조금을 영구적으로 두자」고 규칙을 바꾸면 — 칼스텐 외가 권고한 바로 그 방향으로 — 기존 보유자의 권리는 침해되는 것입니까. 보유자가 가진 것은 「2,100만 상한」에 대한 어떤 권리입니까, 아니면 UTXO의 잠금 조건을 풀 수 있는 능력뿐입니까. 규칙 변경에 동의하지 않은 소수가 옛 규칙의 체인에 남는다면 둘 중 어느 쪽이 「그」 비트코인입니까. 이 책은 답하지 않고 5부와 6부의 양자 전환 논의를 가리킵니다. 양자 전환 때 「취약 코인의 지출을 막는 소프트포크」가 같은 구조의 질문을 더 날카롭게 던집니다. → 27장 · 29장

8. 정리

블록 보상은 보조금과 수수료로 이루어지고, 비트코인의 보조금은 210,000블록마다 반으로 줄어 0으로 갑니다. 그 보상이 정직한 참여자의 지출을 끌어내고(영이윤), 그 지출이 공격자가 넘어야 할 벽이며(유인양립), 그래서 블록당 보상은 공격으로 얻을 가치에 견주어 언제나 커야 합니다(균형 제약). 보안은 재고가 아니라 흐름이고, 공격 비용은 「공격자 해시율 × 지속시간 × 단가」로 계산되며, 그 지속시간은 명제 3의 식이 줍니다. 입력은 해시율·단가·시간이지 코인 가격이 아닙니다. 반감기는 이 식의 p_{block} 을 줄이는 사건이고, 수수료가 그 자리를 메우지 못하면 벽이 낮아집니다. 수수료만 남은 세계의 채굴 게임은 블록 보상 세계와 달라서(채굴 공백·언더컷·뒷문 열린 이기적 채굴), 이더리움은 보상을 영구화하고 수수료의 구조를 바꾸는 길을 갔습니다(그 동기의 서술은 설계 문서의 몫이고, EIP-1559 동기 절은 이 문제를 네 항목 가운데 하나로 듭니다). 21장은 그 두 번째 길, 수수료가 어떻게 매겨지고 누구에게 가는가를 봅니다. 23장은 이 장이 「정직한 참여자」라고 부른 사람들이 정말 정직하게 행동할 유인이 있는지를 묻습니다.

이 장의 출처

- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), §6 Incentive(4쪽).
- Eric Budish, "Trust at Scale: The Economic Limits of Cryptocurrencies and Blockchains," *Quarterly Journal of Economics* 140(1): 1–62 (2025), DOI 10.1093/qje/qjae033, CC BY — 세 식(§III.A–C, 21–25쪽, 명제 1·정의 2·정리 1), 흐름 대 재고와 비유(7쪽·25쪽), 선형성(25–26쪽), 정리 2 순비용 0(§III.D, 26–28쪽), 지분증명(19쪽·22쪽), 이중지불 단계·명제 3 식 (6)·표 II(§IV.A–B, 33–36쪽), 표 III·IV와 5.00%(§IV.C, 37–39쪽), 전문 자본 식 (7)–(10)과 2,500배(§V.A, 41–43쪽), 자본을 잃는 세 길(10쪽·44쪽). WP판(NBER 24717, 2018)은 제목·내용이 다르므로 인용하지 않았다.
- Miles Carlsten, Harry Kalodner, S. Matthew Weinberg & Arvind Narayanan, "On the Instability of Bitcoin Without the Block Reward," *CCS '16*, pp. 154–167, DOI 10.1145/2976749.2978408 — 저자 공개 확장판으로 읽었고 절 번호(§1, §3.1–3.2, §5.1–5.5, §6.1–6.2, §7)만 인용했다. 그림·쪽 번호는 게재본과 다를 수 있어 쓰지 않았다.
- Philip Daian 외, "Flash Boys 2.0," *2020 IEEE S&P*, pp. 910–927 — 타임밴딧 공격(§VII-C, 922쪽).
- EIP-1559 (Final), Motivation 절 — 보상 없는 블록체인의 불안정. https://github.com/ethereum/EIPs/blob/e858e912ff55370eb8125873455af531e322d9b2/EIPS/eip-1559.md (2026-10-25 접근).
- Bitcoin Core v31.1 `chainparams.cpp` L84, `validation.cpp` L1843–L1854, `consensus/amount.h` L15; btcd `chaincfg/params.go` L330, `blockchain/validate.go` L44 — 반감 간격·최초 보조금·`GetBlockSubsidy`. 고정 링크는 `widgets/data.json` `bitcoin.subsidy._src`.
- `data/hashrate_2026-10-25.json` — 기준일 높이 972,958·난이도·추정 해시율(Blockstream·mempool.space 원시 응답, 600초 가정). 「2026-10 기준」 보조금 3.125 BTC는 부록 A 색인 A-16.
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `math`·`json`·`os` 만 사용. 노트북 NB20. NB20-2는 부디시 표 II를 재현하고, NB20-3의 (나) 단가는 가상의 자리 값이다. 위젯 W6

`attack-cost` 의 처음 상태는 NB20-3 (가)의 점유율 60%·$k + e = 12$(출력 20.1 블록 보상)이고 자산 가격 입력란은 두지 않는다(사용자가 넣는 것은 해시 단가다).

각주

1. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), 4쪽(§6 Incentive). 원문: "By convention, the first transaction in a block is a special transaction that starts a new coin owned by the creator of the block. This adds an incentive for nodes to support the network, and provides a way to initially distribute coins into circulation, since there is no central authority to issue them. ... The incentive can also be funded with transaction fees. ... Once a predetermined number of coins have entered circulation, the incentive can transition entirely to transaction fees and be completely inflation free."; "If a greedy attacker is able to assemble more CPU power than all the honest nodes, he would have to choose between using it to defraud people by stealing back his payments, or using it to generate new coins. He ought to find it more profitable to play by the rules, such rules that favour him with more new coins than everyone else combined, than to undermine the system and the validity of his own wealth."

2. Bitcoin Core v31.1(태그 커밋 9be056a8). `src/kernel/chainparams.cpp` L84 `consensus.nSubsidyHalvingInterval = 210000;` , `src/validation.cpp` L1843–L1854 `GetBlockSubsidy` (50 * COIN >> halvings, halvings ≥ 64이면 0), `src/consensus/amount.h` L15 `COIN = 100000000` . `widgets/data.json` `bitcoin.subsidy` 의 `_src` 에 줄 번호가 붙은 고정 링크가 있다(독립 출처 2: btcd `chaincfg/params.go` L330, `blockchain/validate.go` L44).

3. Miles Carlsten, Harry Kalodner, S. Matthew Weinberg & Arvind Narayanan, "On the Instability of Bitcoin Without the Block Reward," *CCS '16*, pp. 154–167 (2016), 저자 공개 확장판 1쪽(§1). 원문: "There has been an unexamined belief that in terms of the security of the block chain (including incentives of the mining game), it is immaterial whether miners receive (say) 25 bitcoins in each block as a block reward or 25 bitcoins in expectation as transaction fees." 저장본은 확장판이라 그림·쪽 번호는 인용하지 않고 절 번호만 적는다.

4. 높이 972,958(관측 2026-10-24T18:17Z, Blockstream·mempool.space 원시 응답 일치 — `data/hashrate_2026-10-25.json`)에서 972,958 ÷ 210,000 = 4(몫)이므로 보조금은 $50/2^4 = 3.125$ BTC. 부록 A 색인 A-16. 코인베이스 출력을 직접 확인한 것은 아니며 소스 코드의 규칙에서 계산한 값이다.

5. Eric Budish, "Trust at Scale: The Economic Limits of Cryptocurrencies and Blockchains," *Quarterly Journal of Economics* 140(1): 1–62 (2025), 1쪽(초록)·4쪽. 원문: "The core argument is three equations. A zero-profit condition on the quantity of honest blockchain 'trust support' (work, stake, etc.) and an incentive-compatibility condition on the system's security against majority attack (the Achilles heel of all forms of permissionless consensus) together imply an equilibrium constraint, which says that the 'flow' cost of blockchain trust has to be large at all times relative to the benefits of attacking the system." (게재본, CC BY). 2018년 NBER WP 24717은 제목이 다르다("The Economic Limits of Bitcoin and the Blockchain"); 이 책은 게재본 제목과 쪽으로 인용한다.

6. Budish (2025), 21–23쪽(§III.A), Proposition 1. 원문: "Let $N^* = \sum x_i^*$. In any zero-profit honest Nakamoto trust equilibrium, $N^* c = p_{block}$ (1) and $D^* = N^*$."; "Proposition 1 is widely known and is the standard characterization of a rent-seeking tournament: the prize in the tournament, p_{block}, is dissipated by expenditures aimed at winning the prize, $N^* c$. Prat and Walter (2021) provide empirical support that equation (1) describes actual equilibrium behavior in the Bitcoin mining market, with some additional nuances related to capital adjustment costs." 비용의 분해 $c = rC + \eta$(자본의 임대비용 + 전기 같은 가변비용)는 21쪽.

7. Budish (2025), 23–24쪽(§III.B), Definition 2. 원문: "This player can attack by choosing AN^* units of trust support, $A > 1$, for an $\frac{A}{A+1}$ majority at cost $AN^* c$ per unit time. Denote the expected duration of the attack by $t(A)$ ... Call $AN^* c \cdot t(A)$ the gross cost of attack. The attacker can choose A to minimize $A \cdot t(A)$."; "Nakamoto trust is incentive compatible (IC) against an outsider attack, on a gross-cost basis, if the gross cost of attack exceeds the benefits of attack: $A^* N^* c \cdot t(A^*) > V_{attack}$. (2)"

8. Budish (2025), 25쪽(§III.C), Theorem 1 (Equilibrium Constraint). 원문: "The zero-profit honest Nakamoto trust equilibrium condition (1) and the IC against outsider attack condition (2) together imply the equilibrium constraint: $p_{block} > \frac{V_{attack}}{A^* \cdot t(A^*)}$. (3) In words: the equilibrium per block payment to participants for providing trust support on the blockchain must be large relative to the benefits of attacking the blockchain."

9. Budish (2025), 7쪽·25쪽. 원문: "An intuition is that Nakamoto trust is memoryless—it is as secure at a moment in time as the amount of trust support at that moment."; "Imagine if users of the Visa network had to pay fees to Visa, every ten minutes, that were large relative to the value of a successful attack on the Visa network. Imagine a brand was only as trustworthy as its flow investment in advertising. Imagine that a country were only as secure as its flow expenditure on soldiers at the border."

10. Budish (2025), 25–26쪽. 원문: "From a computer security perspective, the key thing to note about constraint (3) is that the security of the blockchain is linear in the amount of expenditure on trust support. For instance, if attacking the system grows 1,000 times more attractive, then the cost of securing the system must grow 1,000 times as well. In contrast, in many other contexts investments in computer security yield convex returns, such as traditional uses of cryptography—analogously to how a lock on a door increases the security of a house by more than the cost of the lock." 규모의 경제 비유(은행·경찰·법원)는 7–8쪽과 그림 I.

11. Budish (2025), 4쪽·19쪽(§II.E.3). 원문: "Trust support can take the form of providing computational work as in Bitcoin ('proof of work'), providing other kinds of computational resources as in proof-of-storage or proof-of-memory protocols, or posting cryptocurrency coins in a proof-of-stake protocol, such as Ethereum."; "For proof of stake, c is the opportunity cost per unit time to supply one unit of stake."(22쪽); "The more important aspect of proof of stake for the purpose of my argument is that stakes are not as memoryless as computational work: stakes can be locked up on chain for a period of time, like collateral, and observably persist for the time they are locked, like reputation. This opens up the possibility of punishing attackers by confiscating their locked stakes (slashing), which makes attacks more expensive and hence the blockchain more secure."

12. Budish (2025), 33–35쪽(§IV.A–B), Proposition 3 및 식 (6). 다섯 단계 원문: "(i) The attacker sends cryptocurrency in exchange for goods or assets, potentially in many transactions over many blocks. (ii) The attacker allows those transactions to be added to the blockchain in the usual way … (iii) The attacker works in secret to create an alternative longest chain that does not include the transactions in (i). … (iv) The attacker waits for any escrow periods to elapse … (v) The attacker releases their alternative longest chain." 식의 해설: "In the attacker's best case, their attack takes k + e time. … Suppose, on the other hand, that the attacker is behind the honest chain by i ⩾ 0 blocks at the time the honest participants produce their k + e block. Given the Poisson arrival processes, it will take the attacker (i+1)/(A−1) of time in expectation to strictly surpass the honest chain. The last part of the expression gives the probability that the attacker's deficit is i blocks." 증명은 Online Appendix B.

13. Budish (2025), 34쪽. 원문: "Prior work by Grunspan and Pérez-Marco (2018, 2022) derives closely related mathematical results for the analysis of an attacker with less than 50% of the total hash power."

14. Budish (2025), 35–36쪽, Table II. 원문: "For example, if A = 1.05, which corresponds to a 51.2% attacker majority, the average duration is 56.6 blocks (9.5 hours) and the average gross cost is 59.4 times $N^{*}c$."; "It is true that a 51% majority is enough to ensure statistically that the attack will eventually succeed, but a cost-minimizing attacker will choose a somewhat larger majority."

15. Budish (2025), 37–39쪽, Table III·IV. 원문: "To keep the Nakamoto blockchain secure in this base case requires a per block cost that is 5.00% of the value secured against a double-spending attack. This follows directly from equation (3), rewritten as $p_{block} \,/\, V_{attack} \geq 1 \,/\, (A^{*} \cdot t(A^{*}))$, with $t(A^{*}) = 13.99$ at the attacker-optimal choice of $A^{*} = 1.43$ (or 59%)."; "A cost of 5% per block amounts to 720% of the value secured against attack per day, and about 263,000% of the value secured against attack per year. For example, to secure the system against a $1 billion attack requires $2.63 trillion of annual security expense."; "the cost declines to 0.09% of V_{attack} if e + k amounts to a week (e + k = 1,008)." 기본 사례는 e = 6, k = 6(37쪽 본문·각주 23).

16. Budish (2025), 26–28쪽(§III.D), Theorem 2 (Zero Net Attack Cost). 원문: "If the attacker does not face any cost frictions relative to the costs of honest participants (κ = 0), the attack concludes without any difficulty adjustment ($D^{'} = D^{*}$), and the attack does not cause the value of the cryptocurrency to fall ($\Delta_{attack} = 0$), then the net cost of attack is zero."; "The intuition behind this result is that the attacker is fully compensated for their trust-support costs for the same reason that honest participants are fully compensated for their trust-support costs under honest play. In effect, permissionless consensus treats the attacker as if they are an honest participant, because the majority determines the truth."; "To be clear, zero attack frictions seems unrealistic. But zero friction is often useful as a benchmark case …".

17. Philip Daian 외, "Flash Boys 2.0," *2020 IEEE S&P*, pp. 910–927, §VII-C(922쪽). 원문: "Suppose a blockchain has a

suffix (subchain) [height0, height1], with current block height height1, in which stealable value exceeds block rewards. An adversary can rewind to height0 and use the resulting MEV to subsidize a profitable 51% attack that mines a fork up to or past height1." 17장 각주 11 과 같은 자리.

18. Carlsten 외 (2016), §3.1. 원문: "we assume that for any time interval I of length t, the total sum of transaction fees for transactions announced during I is t"; "We also assume that the difficulty is set so that a hash puzzle is solved by someone in the network every one time unit in expectation ..."; "Put another way, our results are only made stronger by simplifying assumptions, because we are claiming that weird and undesirable consequences arise even if one is willing to grant simplifying assumptions."

19. Carlsten 외 (2016), §3.2. 원문: "Without a block reward, immediately after a block is found there is zero expected reward for mining but nonzero electricity cost, making it unprofitable for any miner to mine."; "If a is the fraction of the total (effective) hash power that a single rig generates, then a miner must wait t = p/a time steps after a block is found before mining becomes profitable again."; "it leads to miners mining for a smaller and smaller fraction of the time between the arrival of blocks (with the difficulty dropping to compensate). Clearly, this would have a negative impact for Bitcoin security, as the effective hash power in the network would drop, and it would become easier for a malicious miner to fork."

20. Carlsten 외 (2016), §5.1–5.5. PettyCompliant: "Miners can do strictly better by mining on the block that leaves the most transactions fees unclaimed."; 언더컷: "We call this the undercutting attack, where miners will actively fork the head of the chain and leave transactions unclaimed in the hope of incentivizing PettyCompliant miners to build on their block."; Theorem 5.1: "Then it is an equilibrium for every miner to use the strategy Function-fork(f) as long as: Every miner is non-atomic. Miners may only mine on top of chains of length H or H − 1. Furthermore, in any such equilibrium, the expected number of backlogged transactions after n time steps is $\Theta(\sqrt{n})$."; §5.5: "even if 66% of miners remain default compliant, undercutting is profitable."; "Therefore, whenever $y \leq 2/3$, the reward from FunctionFork(x) is at least one, and therefore it is a better choice than PettyCompliant ...".

21. Schilling & Uhlig (2019), 24쪽(§6). 원문: "This modeling choice captures the real-life feature, that the increase in the hash rate has no impact on the total Bitcoin production, see Huberman et al. (2017)."

22. Carlsten 외 (2016), §1(기여 3)·§6.1–6.2. 원문: "Intuitively, imagine you are mining and find yourself solving a new block immediately after a previous block was announced and before any new transactions have been announced. This block is literally worthless, so instead of publishing, why not use it to try and selfish mine? There is no cost, but a positive probability that you build a lead of two, no matter your hash power. Similarly, imagine instead that just by chance an hour goes by since the last block was found and you just solved a new block including all transactions that arrived during that period. This block is worth roughly six 'normal' blocks, so why risk losing it?"; "this strategy works for miners with arbitrarily low hash power and regardless of their connectedness in the Bitcoin network. Moreover, the attack is profitable as soon as it is deployed, whereas classic selfish mining only becomes profitable after a two-week difficulty adjustment period ..."; "when $\alpha = 1/3$, both selfish mining and default mining achieve expected reward of $\approx 1/3$, but selfish mining with the optimal cutoff achieves an expected reward of ≈.38, a 13.6% increase!"

23. Carlsten 외 (2016), §1 끝(「Related Work」 직전)·§7. 원문: "Nakamoto appears to have viewed the block reward as a necessary but temporary evil to achieve an initial allocation of bitcoins in the absence of a central authority, with the transaction fee regime being the ideal, inflation-free steady state of the system. But our work shows that incentivizing compliant miner behavior in the transaction fee regime is a significantly more daunting task than in the block reward regime. Perhaps instead, designers of new cryptocurrencies must resign themselves to the inevitability of monetary inflation and make the block reward permanent."(§1 끝의 같은 취지 문장과 §7 "We see the block reward as integral to the stability of the mining game.")

24. Vitalik Buterin 외, "EIP-1559: Fee market change for ETH 1.0 chain" (Final), Motivation 절. 원문: "Instability of blockchains with no block reward: In the long run, blockchains where there is no issuance (including Bitcoin and Zcash) at present intend to switch to rewarding miners entirely through transaction fees. However, there are known issues with this that likely leads to a lot of instability, incentivizing mining 'sister blocks' that steal transaction fees, opening up much stronger selfish mining attack vectors, and more. There is at present no good mitigation for this."
https://github.com/ethereum/EIPs/blob/e858e912ff55370eb8125873455af531e322d9b2/EIPS/eip-1559.md (2026-10-25 접근, raw 마크다운 대조).

25. Budish (2025), 41–43쪽(§V, §V.A), 식 (7)–(10). 원문: "If the capital used to maintain the blockchain is specialized (as opposed to repurposable), and a majority attack causes the attacker's capital to lose its value or be confiscated, then the attacker cost model needs to be modified. In addition to charging the attacker the flow cost of the attack, we also need to charge the attacker the stock value of their loss of specialized capital."; "This is several orders of magnitude more secure than equation (3) because r is the interest rate per unit time."; "Assume the capital share of mining is μ = 0.4 (De Vries 2018; Digiconomist 2022), and the annual discount rate for ASICs is 50% … which implies that the per unit time discount rate is r ≈ 0.0008%. This means r/μ = 0.002%. … If we use the base case value of 1/(A·*t*(*A*)) = 5.0%, we have a roughly 2,500-fold improvement in the cost of security."

26. Budish (2025), 10쪽·44쪽(§V.B). 원문: "One possibility is that the value of the cryptocurrency collapses because of the attack, and as a result the value of specialized capital collapses, too. But vulnerability to collapse is itself a serious problem—hardly a reassuring foundation for a novel economic system—and raises the possibility of an attack motivated by this collapse per se. A second possibility is legal punishment, which certainly would work but concedes that Nakamoto trust fails without support from rule of law. A third possibility is that the protocol could algorithmically confiscate or block the attacker's capital in a targeted punishment … we show that this approach (i) is impossible for a broad class of permissionless consensus protocols that includes Bitcoin's; (ii) is possible for a class of permissionless consensus protocols that includes proof-of-stake Ethereum's, but only under a strong assumption about network reliability and only if the attacker is less than proportion ρ = 2/3 of the total." 사보타주 공격의 최초 제기는 Rosenfeld (2014)라고 부디시가 적는다(44쪽 각주 30).

3부 경제학

21장 수수료 시장 — 경매에서 EIP-1559로

핵심 TAKEAWAY

블록 공간은 희소하고, 누가 그 공간을 쓸지 정하는 장치가 **수수료 시장입니다**. 비트코인과 초기 이더리움은 1차 가격 경매(pay-as-bid)를 썼습니다. 각자 입찰하고, 블록 생산자가 높은 입찰을 고르고, 들어간 사람은 자기 입찰액을 냅니다. 이 설계는 입찰자에게 「남들이 얼마를 쓸까」를 추측하게 만들고, 그 추측이 틀리면 과다 지불이 생깁니다. 이더리움이 2021년 8월 5일 런던 포크에서 들여온 EIP-1559는 수수료를 경매가 아니라 **공식으로** 정하게 바꿨습니다. 프로토콜이 블록마다 **기본 수수료를** 계산하고(직전 블록이 목표보다 찼으면 올리고 비었으면 내리며, 한 번에 최대 12.5%), 그 기본 수수료는 아무에게도 가지 않고 **소각되며**, 생산자는 사용자가 얹은 팁만 받습니다. 러프가든(Tim Roughgarden)은 이 설계가 세 가지 유인양립 조건 가운데 둘을 언제나, 하나를 보통 만족한다는 것을 증명했고, 「예약가격과 소각은 떼어 놓을 수 없다」는 것도 보였습니다. 소각은 생산자가 수수료를 조작할 유인을 없애는 장치이지, 코인 가격에 관한 명제가 아닙니다. 이 장을 읽으면 수수료가 「혼잡 통행료」라는 말의 정확한 뜻과, 그 통행료가 누구에게 가는지가 왜 설계의 핵심인지 압니다.

학습목표

1. 1차 가격 경매가 블록체인에서 왜 문제였는지(과다 지불·대기·변동성·보상 없는 체인의 불안정)를 EIP-1559의 동기 절과 러프가든의 네 가지 특이점으로 말한다.
2. [중급] EIP-1559의 구성 요소 — 기본 수수료, 이력 의존 조정, 소각, 가변 블록 크기(목표와 상한), 팁과 수수료 상한 — 를 읽고, 기본 수수료 조정식을 정수 산술로 계산한다. 상수(탄력 배수 2, 조정 분모 8)와 관측값(가스 한도)을 구별한다.
3. [중급] 세 유인양립 개념(DSIC·MMIC·OCA-proof)의 뜻과, 1차 가격 경매와 1559 메커니즘이 각각 어디에 해당하는지 표로 읽는다. 소각과 예약가격이 왜 함께 가야 하는지 안다.
4. [심화] 블롭 수수료 시장(EIP-4844)이 1559와 같은 원리를 다른 자원에 적용한 것임을 알고, 지수형 갱신식을 계산한다.

[입문]**만 읽은 독자가 가져가는 것.** 수수료는 「혼잡 통행료」이고, 1559는 통행료를 경매가 아니라 **공식으로** 정하게 바꾼 것입니다.

1. 이 장이 답하려는 질문

13장은 거래 수수료가 「가스 단위 × 가스당 가격」이고 그 가격이 두 조각(기본 수수료와 우선 수수료)으로 갈린다는 데서 멈췄습니다. 왜 두 조각인가, 기본 수수료는 누가 어떻게 정하는가, 왜 한 조각은 소각되는가가 이 장입니다. 17장은 봇들이 블록 생산자에게 순서를 사려고 벌이는 경매(PGA)를 보여 주었고 「기본 수수료 소각이 생산자의 수수료 조작 유인을 줄인다는 설계 의도」를 이 장으로 넘겼습니다. 18장은 블롭이라는 별도 자원의 수수료를 이 장으로 넘겼습니다. 20장은 수수료가 보안 예산의 재원이 되어 가는 과정이 불안정할 수 있다고 했습니다. 이 네 갈래가 모두 같은 질문으로 모입니다. **희소한 블록 공간을 누구에게, 얼마에, 누구의 수입으로 줄 것인가.**

2. 1차 가격 경매 — 무엇이 문제였나

러프가든의 2024년 논문은 첫 줄에서 문제를 정합니다. 「주요 블록체인의 부동산은 희소한 자원이다. … 수요는 훨씬 커서, 제출된 모든 거래의 풀에서 「온체인」에 넣을 부분집합을 고르는 메커니즘이 필요하다.」 그리고 역사를 적습니다. 「역사적으로 대부분의 블록체인 프로토콜은 입찰한 만큼 내는(pay-as-bid) 거래 수수료 메커니즘을 썼다. 모든 거래는 (블록체인의 고유 통화로) 입찰과 함께 제출되고, 블록의 채굴자가 어느 거래를 넣을지 정하며, 그 블록이 공개되면 포함된 각 거래의 입찰액이 생성자에게서 채굴자에게 이전된다. 우리는 블록체인 관례를 따라 이 메커니즘을 **1차 가격 경매(FPA)라** 부른다.」[1]

경매 이론의 눈으로 보면 블록체인은 보통의 경매장이 아닙니다. 러프가든이 든 네 가지 특이점입니다. 「(1) 블록의 채굴자는 그 내용에 대해 독재적 통제력을 가지며, 특히 프로토콜 설계자가 의도한 배분 규칙에서 벗어날 수 있다. (2) 블록의 채굴자는 실제 거래와 구별되지 않는 가짜 거래를 비용 없이 넣을 수 있다. (3) 지불은 「온체인」 자료에서 계산할 수 있어야 하는데, 온체인 자료는 보통 **탈락한 입찰에 대해 아무 정보도 주지 않는다**. (4) 채굴자와 사용자는 오프체인에서 쉽게 결탁해 거래 수수료 메커니즘을 조작할 수 있다.」 반대로 쓸 수 있는 것도 둘 있습니다. 「(5) 어느 블록의 메커니즘(예약가격 등)은 이전 블록들의 (공개된) 결과를 참고해 정할 수 있다. (6) 블록의 수입을 그 블록의 채굴자에게 직접 줄 필요가 없고, 예컨대 재단이나 미래 블록의 채굴자에게 돌릴 수 있다.」[2] 경매 교과서의 2차 가격 경매(비크리)가 여기서 바로 깨지는 이유가 (2)와 (3)입니다. 낙찰자가 「다음으로 높은 입찰」을 내게 하면, 채굴자는 가짜 거래로 그 입찰을 끌어올릴 수 있고, 탈락한 입찰은 체인에 없어 검증할 수도 없습니다. 러프가든의 예가 간단합니다. 자리가 셋인 블록에 상위 입찰이 10, 8, 3이면 정직한 2차 가격 경매의 수입은 3 × 3 = 9이고, 채굴자가 입찰 8짜리 가짜 거래를 넣으면 순수입은 2 × 8 = 16으로 뛴다는 것(원문 예 3.5의 요지).[3]

1차 가격 경매는 (1)·(2)에는 강합니다. 채굴자는 어차피 높은 입찰을 고르는 것이 이익이고 가짜 거래로 얻을 것이 없습니다. 문제는 **입찰자 쪽입니다**. EIP-1559의 동기 절이 네 가지를 듭니다. 첫째, 「거래 수수료 수준의 변동성과 거래의 사회적 비용 사이의 불일치」 — 「가스당 비용이 10 나노이더일 때 네트워크가 거래 하나를 더 받는 비용이 1 나노이더일 때보다 실제로 10배라고 하는 것은 터무니없다. 두 경우 모두 800만 가스와 802만 가스의 차이일 뿐이다.」 둘째, 「사용자의 불필요한 지연」 — 블록 가스 한도가 고정이어서 한 블록을 크게, 다음을 작게 만드는 「여유」 장치가 없습니다. 셋째, 「1차 가격 경매의 비효율」 — 「메커니즘 설계 문헌에서 매우 비효율적이라고 잘 알려져 있어 복잡한 수수료 추정 알고리듬이 필요한데, 그 알고리듬조차 잘 작동하지 않아 잦은 수수료 과다 지불을 낳는다.」 넷째, 20장에서 본 「블록 보상 없는 블록체인의 불안정」입니다.[4] 17장의 PGA가 셋째 문제의 실물입니다.

봇들이 같은 논스로 입찰을 거듭 올리고, 패자도 가스를 내는 경매였습니다.

3. EIP-1559 — 공식으로 정하는 통행료

EIP의 요지 절이 설계를 한 문단에 담습니다. 「프로토콜 안에 가스당 기본 수수료가 있고, 이것은 **부모 블록의 가스 사용량과 가스 목표(블록 가스 한도를 탄력 배수로 나눈 것)의 함수인 공식에 따라** 블록마다 오르거나 내린다. 블록이 가스 목표를 넘으면 기본 수수료가 오르고, 목표 아래면 내린다. 기본 수수료는 소각된다. 거래는 채굴자에게 포함을 유인하려고 줄 가스당 최대 수수료(우선 수수료)를 적고, 또 총액으로 낼 가스당 최대 수수료(최대 수수료)를 적는데, 후자는 우선 수수료와 블록의 네트워크 수수료(기본 수수료)를 모두 덮는다. 보내는 사람은 항상 자기 거래가 포함된 블록의 가스당 기본 수수료를 내고, 두 수수료의 합이 거래의 최대 수수료를 넘지 않는 한 거래에 적은 우선 수수료를 낸다.」[5] 러프가든은 같은 것을 「여덟 가지 핵심 착상」으로 나눕니다.

#	러프가든의 요약(§4)	이 책의 풀이
(1)	블록마다 프로토콜이 계산한 예약가격 「기본 수수료」가 있고, 그것을 내는 것이 포함의 전제다	통행료의 **하한이** 프로토콜 안에 있다
(2)	기본 수수료는 **이전 블록들만의** 함수이고 현재 블록의 거래에 의존하지 않는다	생산자가 자기 블록으로 통행료를 조작할 수 없다
(3)	기본 수수료 수입은 전부 소각된다	통행료가 누구의 수입도 아니다
(4)	최대 블록 크기를 두 배로($C_{max} = 2C$), 옛 최대 C 를 **목표** 크기로	「여유」 — 한 블록이 크고 다음이 작을 수 있다
(5)	직전 블록이 목표보다 크면 올리고 작으면 내린다	블록 크기가 **초과수요의 온체인 신호다**
(6)	사용자는 입찰 하나가 아니라 **팁과 수수료 상한** 둘을 적고, 상한이 기본 수수료 이상이어야 포함	13장의 maxPriorityFeePerGas·maxFeePerGas
(7)	크기 s, 팁 δ, 상한 c 인 거래가 기본 수수료 r 인 블록에 들어가면 $s\cdot\ \min\{r+\delta, c\}$ 를 낸다	13장 황서의 「유효 가격」
(8)	기본 수수료 몫($s\cdot r$)은 소각, 나머지($s\cdot\ \min\{\delta, c-r\}$)는 생산자에게	10장·13장의 「수수료 몫」

6

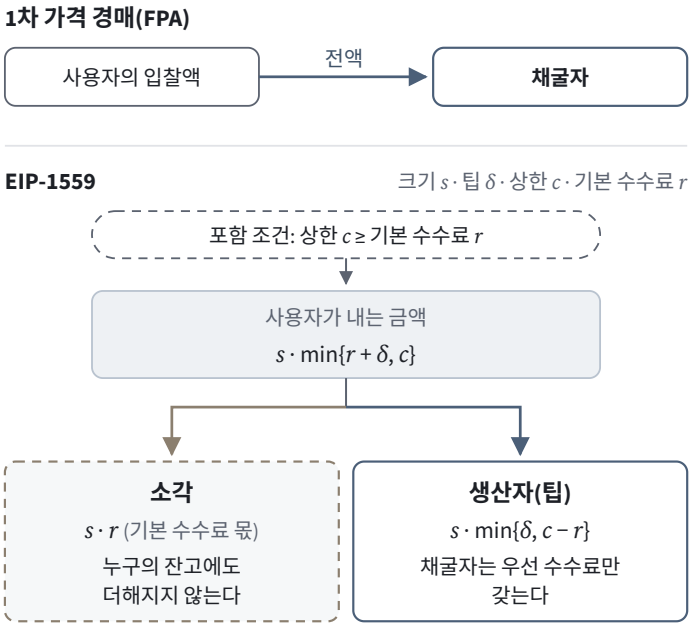


그림 21-1. 사용자가 낸 수수료가 어디로 가는가. 1차 가격 경매에서는 입찰액 전액이 채굴자에게 가고, EIP-1559에서는 기본 수수료 몫이 소각되고 팁만 생산자에게 간다(위 표의 (6)~(8)).

[중급] (5)의 「올리고 내린다」가 정확히 얼마인지는 EIP의 참조 구현에 있습니다. 상수 둘 — `BASE_FEE_MAX_CHANGE_DENOMINATOR = 8`, `ELASTICITY_MULTIPLIER = 2` — 과 정수 산술입니다. 부모 블록의 가스 목표는 가스 한도 ÷ 2이고, 부모가 목표를 넘었으면

$$b_{t+1} = b_t + \max\left(\left\lfloor \frac{b_t \cdot (g_t - g^*)}{g^*} \right\rfloor / 8,\ 1\right), \qquad \text{목표 아래면 } b_{t+1} = b_t - \left\lfloor \frac{b_t \cdot (g^* - g_t)}{g^*} \right\rfloor / 8$$

입니다(나눗셈은 모두 정수 나눗셈).[7] 기호 풀이: b_t 는 부모 블록의 기본 수수료, g_t 는 부모의 가스 사용량, g^* 는 부모의 가스 목표. 부모가 꽉 찼으면($g_t = 2g^*$) 정확히 1/8, 곧 12.5% 오르고, 비었으면 12.5% 내리며, 그 사이는 선형입니다. 러프가든의 각주가 같은 말을 합니다. 「정확히는, 빈 블록과 최대 크기 블록은 기본 수수료를 각각 12.5% 내리고 올리며, 다른 크기의 효과는 선형 보간으로 정해진다(특히 목표 크기와 같은 블록은 기본 수수료를 바꾸지 않는다).」[8] 수치 예는 역사에서 가져옵니다. 런던 포크의 첫 블록 12,965,000은 기본 수수료가 `INITIAL_BASE_FEE` 1 gwei(10^9 wei)로 시작했고, 가스 한도 30,029,122에 가스 사용량 30,025,257로 거의 꽉 찼습니다. 다음 블록의 기본 수수료는 $10^9 + \lfloor 10^9 \cdot (30{,}025{,}257 - 15{,}014{,}561) / 15{,}014{,}561 \rfloor / 8 = 1{,}124{,}967{,}822$ wei여야 하고, 블록 12,965,001의 실제 값이 그 값입니다.[9] 아래 NB21-1이 이 검산을 돌립니다.

```
# NB21-1: EIP-1559 기본 수수료 갱신 — 참조 구현의 정수 산술 그대로. 런던 포크 첫 블록으로 검산한 뒤 혼잡
시나리오를 돌린다 (공부용)
BASE_FEE_MAX_CHANGE_DENOMINATOR = 8      # EIP-1559 원문 상수
ELASTICITY_MULTIPLIER = 2
INITIAL_BASE_FEE = 1_000_000_000          # 1 gwei (wei 단위)

def next_base_fee(parent_base_fee: int, parent_gas_used: int, parent_gas_limit: int) -> int:
    """부모 블록만의 함수(러프가든의 착상 (2)). 나눗셈은 전부 정수 나눗셈."""
    target = parent_gas_limit // ELASTICITY_MULTIPLIER
    if parent_gas_used == target:
        return parent_base_fee
    if parent_gas_used > target:
        delta = parent_base_fee * (parent_gas_used - target) // target //
BASE_FEE_MAX_CHANGE_DENOMINATOR
        return parent_base_fee + max(delta, 1)
    delta = parent_base_fee * (target - parent_gas_used) // target //
BASE_FEE_MAX_CHANGE_DENOMINATOR
    return parent_base_fee - delta

# 검산 1: 런던 포크 첫 블록 12,965,000 (2021-08-05T12:33:42Z) → 12,965,001. 헤더 값은 공개 RPC 두 곳의
원시 응답(각주)
b_12965000, used, limit = INITIAL_BASE_FEE, 30_025_257, 30_029_122
calc = next_base_fee(b_12965000, used, limit)
print(f"블록 12,965,000: 기본 수수료 {b_12965000:,} wei, 가스 {used:,}/{limit:,} (목표 {limit //
2:,})")
print(f"계산한 12,965,001의 기본 수수료 {calc:,} wei — 체인에 적힌 값 1,124,967,822 과 일치: {calc ==
1_124_967_822}")

# 검산 2: 꽉 찬 블록은 정확히 +12.5%, 빈 블록은 −12.5%
print(f"꽉 찬 블록 뒤: {next_base_fee(10**9, 60_000_000, 60_000_000) / 10**9:.4f} gwei, 빈 블록 뒤:
{next_base_fee(10**9, 0, 60_000_000) / 10**9:.4f} gwei")

# 시나리오: 기준일 관측 가스 한도 60,000,000(목표 30,000,000)에서 혼잡(20블록 꽉 참) → 해소(20블록 목표의 1/4)
→ 정상(목표)
limit = 60_000_000                        # 2026-10-24 관측값(상수 아님, 부록 A-10)
scenario = [limit] * 20 + [limit // 8] * 20 + [limit // 2] * 5
b = 10**9
print(f"\n{'블록':>4} {'가스 사용':>12} {'기본 수수료(gwei)':>18} {'변화':>8}")
for i, g in enumerate(scenario, 1):
    nb = next_base_fee(b, g, limit)
    if i <= 3 or i in (10, 20, 21, 22, 23, 30, 40, 41, 45):
        print(f"{i:>4} {g:>12,} {nb / 10**9:>18.4f} {(nb / b - 1):>+8.2%}")
    b = nb
print("꽉 찬 블록이 20개 이어지면 기본 수수료는 (1.125)^20 ≈ {:.1f}배가 된다. 수요가 줄면 내려온다(빈 블록이면
블록마다 12.5%씩).".format(1.125 ** 20))
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
블록 12,965,000: 기본 수수료 1,000,000,000 wei, 가스 30,025,257/30,029,122 (목표 15,014,561)
계산한 12,965,001의 기본 수수료 1,124,967,822 wei — 체인에 적힌 값 1,124,967,822 과 일치: True
꽉 찬 블록 뒤: 1.1250 gwei, 빈 블록 뒤: 0.8750 gwei

  블록         가스 사용        기본 수수료(gwei)        변화
   1   60,000,000             1.1250  +12.50%
   2   60,000,000             1.2656  +12.50%
   3   60,000,000             1.4238  +12.50%
  10   60,000,000             3.2473  +12.50%
  20   60,000,000            10.5451  +12.50%
  21    7,500,000             9.5565   -9.37%
  22    7,500,000             8.6606   -9.37%
  23    7,500,000             7.8486   -9.37%
  30    7,500,000             3.9403   -9.37%
  40    7,500,000             1.4723   -9.37%
  41   30,000,000             1.4723   +0.00%
  45   30,000,000             1.4723   +0.00%
꽉 찬 블록이 20개 이어지면 기본 수수료는 (1.125)^20 ≈ 10.5배가 된다. 수요가 줄면 내려온다(빈 블록이면 블록마다
12.5%씩).
```

[중급] 기준일(2026-10 기준) 메인넷의 관측 가스 한도는 60,000,000이고 따라서 목표는 30,000,000입니다. 이것은 상수가 아닙니다. 생산자가 블록마다 부모의 ±1/1024 안에서 움직이는 값이고, 부록 A 색인 A-10에 올려 두었습니다. 두 상수(2와 8)는 기준일까지 어느 포크에서도 바뀌지 않았습니다.[10] 이 장의 위젯 W9는 이 셀의 시나리오를 움직이게 만든 것이고, 변수는 블록별 가스 사용량뿐입니다.



그림 21-2. EIP-1559 기본 수수료. 가스 한도 60,000,000(목표 30,000,000)에서 꽉 찬 블록 20개가 이어지면 기본 수수료는 1 gwei 에서 10.5451 gwei 로(블록마다 +12.5%), 1/8 블록 20개면 1.4723 gwei 로 내려오고 목표 블록에서는 그대로다. — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 21장 3절).

4. 세 가지 유인양립 — 누가 속일 수 있는가

[중급] 러프가든의 기여는 「이 설계가 좋은가」를 「누가 어떤 식으로 벗어날 수 있는가」로 바꾼 데 있습니다. 벗어날 수 있는 사람이 셋입니다. 사용자, 생산자, 그리고 둘의 결탁입니다. 각각에 대응하는 조건이 셋입니다.

- **DSIC(사용자).** 「채굴자가 의도된 배분 규칙을 수행한다고 가정할 때, 모든 사용자가 (평가액이 무엇이든) 우월전략 — 다른 사용자의 입찰이 무엇이든 자기 효용을 최대로 하는 입찰 — 을 갖는다.」 1차 가격 경매는 「물론 DSIC가 아니다.」[11] 남이 얼마를 쓸지에 따라 내 최적 입찰이 달라지기

때문입니다.

- **MMIC(근시안적 생산자).** 「모든 온체인 이력과 멤풀에 대해, 근시안적 채굴자가 가짜 거래를 만들지 않고 배분 규칙 x 의 제안을 따를 때 자기 효용이 최대가 된다.」 1차 가격 경매는 MMIC이고, 2차 가격 경매는 위의 예처럼 아닙니다.[12]
- **OCA-proof(오프체인 결탁).** 채굴자와 사용자들이 「온체인 입찰 벡터, 배분, 그리고 오프체인 이전」으로 이루어진 합의(OCA)를 맺는다고 할 때, 「모든 온체인 이력에 대해, … 어떤 OCA도 모든 거래 생성자의 효용과 채굴자의 효용을 **동시에** 엄밀히 높이지 못하는」 개별합리적 입찰 전략이 존재해야 합니다.[13]

[중급] 논문의 표 1이 결과를 한눈에 줍니다.

메커니즘	MMIC	DSIC	OCA-PROOF
1차 가격 경매(FPA)	예	아니오	예
2차 가격 경매(SPA)	아니오	거의	거의
수수료 일부(β)를 소각하는 FPA	예	아니오	**아니오**
1559	예	**보통**(정리 5.7)	예
기본 수수료 일부만 소각하는 1559	예	보통	**아니오**
팁 없는 메커니즘(tipless)	예	예	보통

[14]

[중급] 1559의 「보통」이 무엇인지 봅니다. 정리 5.7의 조건은 둘입니다. 「기본 수수료가 현재 수요곡선에 비해 **지나치게 낮지 않고**, 거래 생성자들이 개별합리적 입찰 전략을 쓴다.」 그러면 「1559 메커니즘은 예약가격에 채굴자의 포함 기회비용을 더한 가격의 게시가격 메커니즘처럼 작동」하고, 사용자의 「명백한 최적 입찰」은 상한을 자기 평가액으로, 팁을 한계비용 μ 로 두는 것입니다. 깨지는 때는 수요가 빠르게 늘어 기본 수수료가 아직 따라 올라오지 못한 때입니다. 기본 수수료를 내려는 거래가 최대 블록에도 다 들어가지 못하면, 러프가든은 「기본 수수료가 0인 특수 경우에 FPA로 되돌아가듯」 1559에서도 「입찰 전쟁을 예상할 수 있다」고 적습니다.[15] 공식이 경매를 **대체한** 것이 아니라, 평상시에는 공식이 일하고 급증기에는 경매가 되살아날 수 있는 설계입니다. 러프가든은 「세 성질을 모두 언제나 만족하는 메커니즘은 없다고 추측」했고, 각주에 그 추측이 뒤에 증명되었다고 덧붙여 두었습니다.[16]

```
# NB21-2: 1차 가격 경매 vs 1559 — 같은 사용자들, 같은 블록 공간. 「남이 얼마를 쓸까」 추측이 지불에 어떻게 들어오는지 (공부용)
# 평가액 v 는 가스당 지불 의사(단위 없는 모형값). 블록 용량 C = 4 거래, 전부 같은 크기 1. 채굴자 한계비용 μ = 0.
users = {"A": 10.0, "B": 8.0, "C": 7.5, "D": 7.0, "E": 6.0, "F": 3.0}      # 평가액(진실)
C = 4

def fpa(bids):
    """1차 가격 경매: 높은 입찰 C 개 포함, 각자 자기 입찰액을 내고 전부 채굴자에게."""
    order = sorted(bids.items(), key=lambda kv: -kv[1])[:C]
    return {u: b for u, b in order}, sum(b for _, b in order), 0.0

def mech1559(valuations, base_fee, tip=0.01):
    """1559: 상한 = 평가액, 팁 = μ 근처의 작은 값(정리 5.7의 「명백한 최적 입찰」). 기본 수수료는 소각, 팁만 채굴자에게."""
    eligible = sorted(((u, v) for u, v in valuations.items() if v >= base_fee), key=lambda kv: -kv[1])[:C]
    paid = {u: min(base_fee + tip, v) for u, v in eligible}
    burned = base_fee * len(eligible)
    return paid, sum(paid.values()) - burned, burned

print("(가) FPA — 사용자가 진실한 평가액을 그대로 입찰하면 과다 지불이 생긴다")
paid, miner, burned = fpa(users)
print(f"   포함: {paid}  채굴자 수입 {miner:.2f}, 소각 0")
print("   A 는 10 을 냈지만 6 을 조금 넘게만 냈어도 들어갔다(탈락 최고 입찰 E = 6) — 그래서 FPA 에서는 평가액대로 입찰하는 것이 우월전략이 아니다(DSIC 아님)")

print("\n(가') FPA — 모두가 「탈락 최고 입찰 근처」로 깎아 부르는 균형 비슷한 상태를 흉내 내면")
shaded = {u: round(v * 0.8, 2) for u, v in users.items()}
paid, miner, burned = fpa(shaded)
print(f"   포함: {paid}  채굴자 수입 {miner:.2f} — 깎는 폭은 남의 입찰에 대한 추측에 달려 있다")

for base_fee in [6.5, 7.2, 3.0]:
    paid, miner, burned = mech1559(users, base_fee)
    label = "적정" if base_fee == 6.5 else ("높음" if base_fee == 7.2 else "지나치게 낮음")
    print(f"\n(나) 1559 — 기본 수수료 {base_fee} ({label}): 포함 {paid}")
    n_elig = sum(1 for v in users.values() if v >= base_fee)
    if n_elig > C:
        print(f"   소각 {burned:.2f}, 채굴자 수입(팁) {miner:.2f}. 적격자 {n_elig}명 > 용량 {C} — 이 코드는 팁 경쟁을 계산하지 않는다(정리 5.7의 예외 자리, 아래 줄).")
    else:
        print(f"   소각 {burned:.2f}, 채굴자 수입(팁) {miner:.2f}. 사용자는 남의 입찰을 추측할 필요가 없다(상한 = 평가액).")
print("\n기본 수수료 3.0 처럼 지나치게 낮으면 기본 수수료를 낼 사용자가 용량보다 많아 팁 경쟁이 되살아난다 — 1559 가 FPA 로 되돌아가는 자리(정리 5.7의 조건).")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
(가) FPA — 사용자가 진실한 평가액을 그대로 입찰하면 과다 지불이 생긴다
   포함: {'A': 10.0, 'B': 8.0, 'C': 7.5, 'D': 7.0}  채굴자 수입 32.50, 소각 0
   A 는 10 을 냈지만 6 을 조금 넘게만 냈어도 들어갔다(탈락 최고 입찰 E = 6) — 그래서 FPA 에서는 평가액대로 입찰하는 것이 우월전략이 아니다(DSIC 아님)

(가') FPA — 모두가 「탈락 최고 입찰 근처」로 깎아 부르는 균형 비슷한 상태를 흉내 내면
   포함: {'A': 8.0, 'B': 6.4, 'C': 6.0, 'D': 5.6}  채굴자 수입 26.00 — 깎는 폭은 남의 입찰에 대한 추측에 달려 있다

(나) 1559 — 기본 수수료 6.5 (적정): 포함 {'A': 6.51, 'B': 6.51, 'C': 6.51, 'D': 6.51}
   소각 26.00, 채굴자 수입(팁) 0.04. 사용자는 남의 입찰을 추측할 필요가 없다(상한 = 평가액).

(나) 1559 — 기본 수수료 7.2 (높음): 포함 {'A': 7.21, 'B': 7.21, 'C': 7.21}
   소각 21.60, 채굴자 수입(팁) 0.03. 사용자는 남의 입찰을 추측할 필요가 없다(상한 = 평가액).

(나) 1559 — 기본 수수료 3.0 (지나치게 낮음): 포함 {'A': 3.01, 'B': 3.01, 'C': 3.01, 'D': 3.01}
   소각 12.00, 채굴자 수입(팁) 0.04. 적격자 6명 > 용량 4 — 이 코드는 팁 경쟁을 계산하지 않는다(정리 5.7의 예외 자리, 아래 줄).

기본 수수료 3.0 처럼 지나치게 낮으면 기본 수수료를 낼 사용자가 용량보다 많아 팁 경쟁이 되살아난다 — 1559 가 FPA 로 되돌아가는 자리(정리 5.7의 조건).
```

소각과 예약가격은 떼어 놓을 수 없다

[중급] 17장이 넘긴 질문 — 왜 기본 수수료를 생산자에게 주지 않는가 — 의 답이 표 1의 두 「아니오」에 있습니다. 러프가든의 따름정리 5.17과 5.18입니다. 수수료 일부를 소각하는 FPA는 OCA-proof가 아닙니다. 「FPA에서 수수료를 소각하면 채굴자와 사용자가 소각을 피하려고 오프체인에서 결탁할 촉매가 된다.」 반대로 기본 수수료의 일부라도 생산자에게 주는 1559도 OCA-proof가 아닙니다. 「예약가격 수입을 블록 채굴자에게 넘기면, 낮은 가치의 거래가 온체인에서는 예약가격을 내고 오프체인에서 채굴자에게 부분 환불을 받는 길이 열린다.」 그래서 「1559 메커니즘의 두 주요 혁신 — 예약가격과 소각된 수수료 — 는 **떼어 놓을 수 없이 묶여 있다**. 어느 한쪽만 바꾸면 OCA-proof가 아닌 메커니즘이 된다.」[17] EIP 자체도 같은 취지를 적습니다. 「이 수수료 체계의 중요한 측면은 채굴자가 우선 수수료만 갖는다는 것이다. 기본 수수료는 항상 소각된다(곧 프로토콜이 파괴한다). … 채굴자 추출 가능 가치(MEV)와 관련된 위험을 줄인다. … 끝으로 블록의 채굴자가 기본 수수료를 받지 않게 하는 것이 중요한데, 사용자에게서 더 많은 수수료를 뽑아내려고 수수료를 조작할 채굴자의 유인을 없애기 때문이다.」[18] 생산자가 기본 수수료를 받는다면, 그는 자기 블록을 일부러 채워 다음 블록들의 기본 수수료를 올릴 유인을 갖습니다. 소각은 그 유인을 끊는 장치입니다.



그림 21-3. 러프가든 표 1의 OCA-proof 열을 예약가격의 유무와 수수료의 귀속 두 축으로 다시 놓은 것. 어느 한쪽만 바꾼 두 칸이 「아니오」다.

[중급] 소각을 둘러싼 흔한 추론 하나를 여기서 끊어 둡니다. 「소각 → 공급 감소 → 가격 상승」은 이 장이 다루는 명제가 아니고, 이 설계의 근거도 아닙니다. EIP의 보안 고려 절은 반대로 조심스럽습니다. 「기본 수수료를 소각함으로써 우리는 더 이상 고정된 이더 공급을 보장할 수 없다. … 기본 수수료로 소각되는 양이 채굴 보상으로 만들어지는 양보다 많으면 ETH는 디플레이션이고, 그 반대면 인플레이션이다. 블록 공간에 대한 사용자 수요를 통제할 수 없으므로, 우리는 현재로서 ETH가 인플레이션이 될지 디플레이션이 될지 단언할 수 없고, 따라서 이 변경은 핵심 개발자들이 이더의 장기 수량에 대한 통제력을 일부 잃게 만든다.」[19] 러프가든은 소각의 게임이론적 대안으로 「기본 수수료 수입을 다음 ℓ 개 블록의 채굴자에게 똑같이 나누는」 선지급(pay-it-forward) 구현을 들고, 둘의 차이를 **누구에게 유리한가와 무엇이 변동하는가로** 적습니다. 「소각은 (디플레이션을 통해) 고유 통화 보유자에게 유리하고, 선지급은

(더 많은 수수료 수입을 통해) 채굴자에게 유리하다.」 그리고 「소각이면 블록체인의 인플레이션율이 블록마다 가변적이고 예측 불가능한 대신 채굴자 수입(사실상 블록체인의 보안을 지불하는 것)은 비교적 일정하고, 선지급이면 인플레이션율은 사실상 결정적이나 채굴자 보상(따라서 블록체인 보안)이 예측 불가능하다.」[20] 20장의 보안 예산이 여기서 다시 나옵니다. 소각은 보유자와 생산자 사이에서 수수료의 **귀속을** 정하는 선택이고, 그 선택이 보안 예산의 변동성을 바꿉니다. 가격은 이 서술의 어디에도 없습니다.

[중급] 남는 틈은 **여러 블록에 걸친** 조작입니다. 러프가든의 분석은 한 블록 시간 규모이고, 1559는 이력 의존 기본 수수료로 블록들을 얽어 놓았습니다. 「목표보다 작은 블록을 공개하면 다음 블록의 기본 수수료가 내려가 그 블록 채굴자의 수입이 늘 수 있다. … 채굴자 카르텔의 장기 조작은 반복 다인 죄수의 딜레마에서 결탁을 유지하는 문제와 닮았다. 모두가 협조하면 모두 나아지지만 각자는 일방적으로 이탈할 근시안적 유인을 갖는다.」 그리고 「주요 블록체인에서 지속적이고 해로운 채굴자 결탁은 아직 확정적으로 관측되지 않았다」고 적되, 「한 채굴자나 검증자가 전체 해시율이나 지분의 상당 부분(예컨대 30%)을 통제할 수 있는 체제」에서는 비근시안적 전략이 이익일 수 있다고 열어 둡니다.[21] 23장의 게임이론이 이 틈을 다룹니다.

5. [심화] 블롭 수수료 시장 — 같은 원리, 다른 자원

▾ [심화] EIP-4844의 블롭 가스와 지수형 갱신

[심화] 18장은 롤업이 데이터를 올리는 자리로 「블롭」을 소개하고 그 가격 결정을 이 장으로 넘겼습니다. EIP-4844(Shard Blob Transactions, status Final)는 블롭에 **별도의 가스를** 둡니다. 「우리는 블롭 가스라는 새 종류의 가스를 도입한다. 보통 가스와 독립이고, EIP-1559와 비슷한 자체 목표 규칙을 따른다. 헤더 필드 `excess_blob_gas` 에 블롭 가스 기본 수수료 계산에 필요한 영속 자료를 저장한다. 현재로서는 블롭만 블롭 가스로 가격이 매겨진다.」[22] 상수는 블롭 하나가 2^{17} 블롭 가스, 블록당 목표 393,216(블롭 3개), 상한 786,432(블롭 6개), 최소 기본 수수료 1 wei, 갱신 분수 3,338,477입니다. 갱신 규칙은 1559의 선형 보간이 아니라 **지수형입니다**. 「블록 N 이 X 블롭 가스를 쓰면 블록 $N+1$ 에서 `excess_blob_gas` 는 $X-$ 목표만큼 늘고, 블록 $N+1$ 의 블롭 가스 기본 수수료는 $e^{(X-\text{목표})\,/\,3{,}338{,}477}$ 배가 된다. … 기존 EIP-1559와 비슷한 효과를 내되, 같은 총 사용량에 대해 분포와 무관하게 같은 방식으로 반응한다는 뜻에서 더 「안정적」이다. … 갱신 분수는 블록당 최대 변화율 $e^{393{,}216\,/\,3{,}338{,}477} \approx 1.125$ 를 겨냥해 골랐다.」[23] 1559의 1/8과 같은 12.5%입니다. 블롭 수수료도 「거래 실행 전에 보내는 쪽 잔고에서 차감되어 **소각되며**, 거래가 실패해도 환불되지 않는다.」[24] 지수함수를 정수 산술로 흉내 내는 함수 `fake_exponential` 이 EIP에 있고, 아래 셀이 그것을 그대로 옮깁니다.

```
# NB21-3: EIP-4844 블롭 가스 기본 수수료 — 원문의 fake_exponential 과 상수를 그대로 옮긴다 (공부용)
MAX_BLOB_GAS_PER_BLOCK = 786_432          # 블롭 6개
TARGET_BLOB_GAS_PER_BLOCK = 393_216       # 블롭 3개
MIN_BASE_FEE_PER_BLOB_GAS = 1             # wei
BLOB_BASE_FEE_UPDATE_FRACTION = 3_338_477
GAS_PER_BLOB = 2**17

def fake_exponential(factor: int, numerator: int, denominator: int) -> int:
    """EIP-4844 Helpers: factor * e**(numerator/denominator) 의 테일러 전개 정수 근사(원문
그대로)."""
    i = 1
    output = 0
    numerator_accum = factor * denominator
    while numerator_accum > 0:
        output += numerator_accum
        numerator_accum = (numerator_accum * numerator) // (denominator * i)
        i += 1
    return output // denominator

def calc_excess_blob_gas(parent_excess: int, parent_blob_gas_used: int) -> int:
    if parent_excess + parent_blob_gas_used < TARGET_BLOB_GAS_PER_BLOCK:
        return 0
    return parent_excess + parent_blob_gas_used - TARGET_BLOB_GAS_PER_BLOCK

def base_fee_per_blob_gas(excess: int) -> int:
    return fake_exponential(MIN_BASE_FEE_PER_BLOB_GAS, excess, BLOB_BASE_FEE_UPDATE_FRACTION)

import math
print(f"상한까지 찬 블록(6블롭, 목표는 3) 하나당 변화율: e^(393216/3338477) =
{math.exp(TARGET_BLOB_GAS_PER_BLOCK / BLOB_BASE_FEE_UPDATE_FRACTION):.4f}")
# 시나리오: 30블록 연속 6블롭(상한) → 30블록 0블롭 → 목표(3블롭)
excess = 0
print(f"{'블록':>4} {'블롭 수':>6} {'excess_blob_gas':>16} {'블롭 가스당 기본 수수료(wei)':>26}")
for i, blobs in enumerate([6] * 30 + [0] * 30 + [3] * 3, 1):
    fee = base_fee_per_blob_gas(excess)
    if i in (1, 2, 3, 10, 20, 30, 31, 32, 40, 50, 60, 61, 63):
        print(f"{i:>4} {blobs:>6} {excess:>16,} {fee:>26,}")
    excess = calc_excess_blob_gas(excess, blobs * GAS_PER_BLOB)
print("상한 블록 30개 뒤 초과분은 30 × 393,216 = {:,} 이고 수수료는 e^(그 값/3,338,477) ≈ {:.0f}배. 빈
블록 30개면 초과분은 0 으로 돌아와 최소 1 wei 가 된다.".format(30 * 393_216, math.exp(30 * 393_216 /
3_338_477)))
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
상한까지 찬 블록(6블롭, 목표는 3) 하나당 변화율: e^(393216/3338477) = 1.1250
  블록   블롭 수  excess_blob_gas         블롭 가스당 기본 수수료(wei)
   1      6                0                            1
   2      6          393,216                            1
   3      6          786,432                            1
  10      6        3,538,944                            2
  20      6        7,471,104                            9
  30      6       11,403,264                           30
  31      0       11,796,480                           34
  32      0       11,403,264                           30
  40      0        8,257,536                           11
  50      0        4,325,376                            3
  60      0          393,216                            1
  61      3                0                            1
  63      3                0                            1
상한 블록 30개 뒤 초과분은 30 × 393,216 = 11,796,480 이고 수수료는 e^(그 값/3,338,477) ≈ 34배. 빈 블록
30개면 초과분은 0 으로 돌아와 최소 1 wei 가 된다.
```

[심화] 이 셀의 상수는 EIP-4844 원문의 값입니다. 확인 과정에서는 기준일 활성 포크(Fusaka) 뒤의 bpo1·bpo2 포크가 「블롭 매개변수만 바꾼다」고 적어 두었습니다. 그러므로 기준일 메인넷의 목표·상한 블롭 수는 원문의 3·6과 다를 수 있고, 이 책은 그 현행값을 확인하지 않았습니다. 부록 A 색인 A-17에 과제로 올립니다. 원리는 같습니다. 별도 자원, 별도 목표, 지수형 갱신, 소각.

> **법률가의 질문**
>
> 기본 수수료는 보내는 사람의 잔고에서 차감되어 **누구의 잔고에도 더해지지 않습니다**. 코드가 그렇게 적혀 있고, 그래서 러프가든은 「영구히 유통량에서 제거된다」고 표현합니다. 이 소각된 수수료는 누구에게 귀속된 것입니까. 사용자가 「네트워크」에 낸 대가입니까 — 그렇다면 네트워크는 수령의 주체입니까. 남은 보유자 전체에 「디플레이션을 통해」 돌아간 것입니까 — 그렇다면 그것은 이전입니까, 아니면 아무 일도 일어나지 않은 것입니까. 법은 「누구의 것도 아닌 가치의 소멸」을 어떤 틀로 봅니까 — 포기, 무주물, 소비, 아니면 그 어느 것도 아닌 새 범주입니까. 선지급 대안이었다면 같은 가치가 미래의 생산자에게 갔을 것이고, 그 선택은 프로토콜 개정으로 바뀔 수 있습니다. 이 책은 답하지 않고 5부의 학설 구도를 가리킵니다. → 27장

6. 정리

블록 공간은 희소하고 수수료 시장이 그것을 배분합니다. 1차 가격 경매는 생산자에게는 강하지만 사용자에게 「남이 얼마를 쓸까」를 추측하게 하고, 그 추측의 실패가 과다 지불과 변동성으로 나타났습니다. EIP-1559는 통행료의 하한을 프로토콜이 직전 블록의 혼잡도로 계산하게 하고(한 번에 최대 12.5%), 그 하한을 소각하며, 생산자에게는 팁만 줍니다. 러프가든의 한 블록 모형 안에서 이 설계는 생산자의 이탈과 오프체인 결탁에 언제나 강하고, 사용자의 전략적 입찰에는 수요 급증기를 빼면 강하며, 소각과 예약가격은 어느 하나만 떼면 결탁의 문이 열리는 한 쌍입니다. 소각은 수수료의 귀속을 보유자 쪽으로 정하는 선택이고 생산자의 조작 유인을 끊는 장치이지, 가격에 관한 명제가 아닙니다. 블롭 수수료 시장은 같은 원리를 다른 자원에 지수형 갱신으로 적용한 것입니다. 22장은 수수료가 아니라 **발행과 분배** 쪽 — 누가 얼마를 처음에 받고 누가 규칙을 바꾸는가 — 를 설계 변수로 읽습니다.

이 장의 출처

- Tim Roughgarden, "Transaction Fee Mechanism Design," *Journal of the ACM* 71(4), Article 30 (2024), DOI 10.1145/3674143 — 문제 설정과 FPA(30:1–30:2), 네 특이점·두 기회(30:2), 결과 요약(30:4–30:5), 정의 2.5 소각 규칙(30:9), DSIC·MMIC·OCA-proof 정의(§3, 30:10–30:13), 2차 가격 경매의 예(Example 3.5, 30:12), 1559의 여덟 착상과 정의 4.1·각주 24(§4, 30:13–30:15), 표 1(30:16), 따름정리 5.17(30:21)·5.18(30:22), §6 논의(30:22–30:23). (게재본). arXiv 2106.01340v3(심사판)과 2020년 보고서는 인용하지 않았다.
- EIP-1559, "Fee market change for ETH 1.0 chain" (Buterin·Conner·Dudley·Slipper·Norden·Bakhta; Final; created 2019-04-13) — Abstract, Motivation(네 문제와 소각의 의도), Specification(상수와 `validate_block` 참조 구현), Security Considerations(「ETH Burn Precludes Fixed Supply」). https://github.com/ethereum/EIPs/blob/e858e912ff55370eb8125873455af531e322d9b2/EIPS/eip-1559.md (2026-10-25 접근).
- EIP-4844, "Shard Blob Transactions" (Buterin 외 6인; Final; created 2022-02-25) — Parameters, Helpers(`fake_exponential`), Gas accounting, Rationale(갱신 규칙·처리량). https://github.com/ethereum/EIPs/blob/e858e912ff55370eb8125873455af531e322d9b2/EIPS/eip

-4844.md (2026-10-25 접근).

- go-ethereum v1.17.10 `params/chainspecs/mainnet.json` L17(`londonBlock` 12965000), `params/protocol_params.go` L157–L159, `params/config.go` L1016–L1023 — 커밋 1df30b18(`widgets/data.json` `_src`).
- 블록 12,964,999·12,965,000·12,965,001 헤더 — 공개 RPC https://eth.drpc.org(세 블록)·https://rpc.flashbots.net(12,964,999·12,965,000; 12,965,001은 pruned)의 `eth_getBlockByNumber` 원시 응답(2026-10-25 접근; 각주 9). 기준일 가스 한도 60,000,000은 `widgets/data.json` `ethereum.eip1559.observed_latest_block`(부록 A-10).
- Philip Daian 외, "Flash Boys 2.0" (2020) — 17장에서 인용한 PGA 관측(이 장에서는 17장을 가리키기만 한다).
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `math` 만 사용. 노트북 NB21. 위젯 W9 `fee-market` 의 처음 상태는 NB21-1의 시나리오(가스 한도 60,000,000·목표 30,000,000, 꽉 찬 블록 20개 → 1/8 블록 20개 → 목표 5개)이고, 변수는 블록별 가스 사용량뿐이다.

각주

1. Tim Roughgarden, "Transaction Fee Mechanism Design," *Journal of the ACM* 71(4), Article 30 (2024), 30:1–30:2. 원문: "Real estate on a major blockchain is a scarce resource. ... Demand for these blockchains is far larger, necessitating a mechanism that selects a subset of transactions to include 'on-chain' from the pool of all submitted transactions. Historically, most blockchain protocols have employed a pay-as-bid transaction fee mechanism. Every transaction is submitted with a bid (in the blockchain's native currency), the miner of a block decides which transactions should be included in it, and upon publication of that block, the bid of each included transaction is transferred from its creator to the miner. We follow blockchain convention and refer to this mechanism as a first-price auction (FPA)." (게재본). 2020년 보고서(arXiv:2012.00854)는 제목이 다르고("... for the Ethereum Blockchain: An Economic Analysis of EIP-1559"), 이 책은 JACM판을 인용한다.

2. Roughgarden (2024), 30:2. 원문: "(1) The miner of a block has dictatorial control over its contents and in particular may deviate from the allocation rule intended by the protocol designer. (2) The miner of a block can include fake transactions at no cost that are indistinguishable from real transactions. (3) Payments should be computable from 'on-chain' data, which typically discloses no information about losing bids. (4) Miners and users can easily collude off-chain to manipulate a transaction fee mechanism."; "(5) The choice of mechanism (such as a reserve price) for a given block could be informed by the (publicly visible) outcomes for previous blocks. (6) Revenue from a block need not be transferred directly to the block's miner and could instead be redirected, for example, to a foundation or to the miners of future blocks."

3. Roughgarden (2024), 30:12, Example 3.5 (Second-Price-Type Auctions Are Not MMIC). 원문: "Now imagine that the top three bids are 10, 8, and 3. If a miner honestly executes the auction, its revenue will be $3 \times 3 = 9$. If the miner instead submits a fake transaction with bid 8 and executes the auction (with the top two real transactions included along with the fake transaction), its net revenue jumps to $2 \times 8 = 16$."

4. Vitalik Buterin, Eric Conner, Rick Dudley, Matthew Slipper, Ian Norden & Abdelhamid Bakhta, "EIP-1559: Fee market change for ETH 1.0 chain" (Standards Track, Core, status Final, created 2019-04-13), Motivation 절. 원문: "Mismatch between volatility of transaction fee levels and social cost of transactions: ... It's absurd to suggest that the cost incurred by the network from accepting one more transaction into a block actually is 10x more when the cost per gas is 10 nanoeth compared to when the cost per gas is 1 nanoeth; in both cases, it's a difference between 8 million gas and 8.02 million gas."; "Needless delays for users: because of the hard per-block gas limit coupled with natural volatility in transaction volume, transactions often wait for several blocks before getting included ... no one significantly gains from the fact that there is no 'slack' mechanism"; "Inefficiencies of first price auctions: ... This is well-known in mechanism design literature to be highly inefficient, and so complex fee estimation algorithms are required. But even these algorithms often end up not working very well, leading to frequent fee overpayment."; 넷째 항목은 20장 각주 24. https://github.com/ethereum/EIPs/blob/e858e912ff55370eb8125873455af531e322d9b2/EIPS/eip-1559.md (2026-

10-25 접근, raw 마크다운을 다시 받아 축어 대조).

5. EIP-1559, Abstract 절. 원문: "There is a base fee per gas in protocol, which can move up or down each block according to a formula which is a function of gas used in parent block and gas target (block gas limit divided by elasticity multiplier) of parent block. The algorithm results in the base fee per gas increasing when blocks are above the gas target, and decreasing when blocks are below the gas target. The base fee per gas is burned. Transactions specify the maximum fee per gas they are willing to give to miners to incentivize them to include their transaction (aka: priority fee). Transactions also specify the maximum fee per gas they are willing to pay total (aka: max fee), which covers both the priority fee and the block's network fee per gas (aka: base fee). Senders will always pay the base fee per gas of the block their transaction was included in, and they will pay the priority fee per gas set in the transaction, as long as the combined amount of the two fees doesn't exceed the transaction's maximum fee per gas."

6. Roughgarden (2024), 30:13–30:14(§4). 원문 (1)–(3): "Each block has a protocol-computed reserve price (per unit size) called the base fee. Paying the base fee is a prerequisite for inclusion in a block. (2) The base fee is a function of the preceding blocks only and does not depend on the transactions included in the current block. (3) All revenues from the base fee are burned—that is, permanently removed from the circulating supply of the native currency." (4)–(5): "Double the maximum block size (i.e., define $C_{max} := 2C$), with the old maximum C serving as the new target block size ($C_{target} := C$). (5) Adjust the base fee upward or downward whenever the size of the latest block is bigger or smaller than the target block size, respectively." (6)–(8): "Rather than a single bid, the creator of a transaction is now responsible for specifying both a tip and a fee cap for it. A transaction will be included in a block only if its fee cap is at least the block's base fee. (7) If a size-s transaction with tip δ and fee cap c is included in a block with base fee r, the transaction creator pays a total of s · min{r + δ, c}. (8) Revenue from the base fee (i.e., s·r) is burned and the remainder (s·min{δ, c − r}) is transferred to the miner of the block." 러프가든은 (2)가 「어떻게 이전 블록에서 끌어내는가」가 불명확하다며 (4)·(5)를 그 답으로 적는다: "If miners publish a sequence of maximum-size blocks, how can the protocol distinguish whether the current base fee is too low or exactly right?"

7. EIP-1559, Specification 절의 참조 구현(`validate_block`). 원문(요지 부분): "parent_gas_target = self.parent(block).gas_limit // ELASTICITY_MULTIPLIER"; "elif parent_gas_used > parent_gas_target: gas_used_delta = parent_gas_used - parent_gas_target; base_fee_per_gas_delta = max(parent_base_fee_per_gas * gas_used_delta // parent_gas_target // BASE_FEE_MAX_CHANGE_DENOMINATOR, 1); expected_base_fee_per_gas = parent_base_fee_per_gas + base_fee_per_gas_delta"; "else: gas_used_delta = parent_gas_target - parent_gas_used; base_fee_per_gas_delta = parent_base_fee_per_gas * gas_used_delta // parent_gas_target // BASE_FEE_MAX_CHANGE_DENOMINATOR; expected_base_fee_per_gas = parent_base_fee_per_gas - base_fee_per_gas_delta". 상수: "INITIAL_BASE_FEE = 1000000000", "BASE_FEE_MAX_CHANGE_DENOMINATOR = 8", "ELASTICITY_MULTIPLIER = 2". 독립 출처 2는 go-ethereum v1.17.10 `params/protocol_params.go` L157–L159(`widgets/data.json` `ethereum.eip1559._src`). 같은 구현은 블록 가스 한도가 부모의 ±1/1024 안에서만 움직이고 5,000 이상이어야 한다고 적는다 — 가스 한도는 상수가 아니라 생산자가 신호하는 관측값이다.

8. Roughgarden (2024), 30:14 각주 24. 원문: "Precisely, empty and maximum-size blocks decrease and increase the base fee by 12.5%, respectively, with the effect of other block sizes then determined by linear interpolation. (In particular, blocks matching the target size do not alter the base fee)."

9. 런던 포크 활성 블록 번호는 go-ethereum v1.17.10 `params/chainspecs/mainnet.json` L17 `"londonBlock": 12965000`(커밋 1df30b18, 2026-10-25 접근). 블록 12,965,000의 헤더는 공개 RPC 두 곳(https://eth.drpc.org 와 https://rpc.flashbots.net 의 `eth_getBlockByNumber`, 2026-10-25 접근)에서 같은 값을 돌려주었고, 12,965,001은 eth.drpc.org 한 곳에서만 받았다(rpc.flashbots.net 은 이 블록에 「pruned history unavailable」, 코드 4444로 답했다): 12,965,000 — 해시 `0x9b83c12c…eee71`, 타임스탬프 1628166822(2021-08-05T12:33:42Z), `baseFeePerGas` 0x3b9aca00(10^9), `gasLimit` 30,029,122, `gasUsed` 30,025,257; 직전 블록 12,964,999의 `gasLimit` 은 15,029,237(탄력 배수 2가 포크 블록에서 한도를 두 배로 만든 것); 12,965,001 — `baseFeePerGas` 1,124,967,822, `gasLimit` 29,999,798, `gasUsed` 29,985,144. 흔히 쓰는 다른 공개 노드(publicnode)는 이 높이의 이력을 잘라 내 응답하지 않았다. 러프가든 게재본 30:2는 런던 포크를 「block number 12865000, which was mined on August 5, 2021」로 적는데, 블록 번호는 클라이언트 설정값 12,965,000과 한 자리가 다르다(오기로 보이며 이 책은 설정값을 따른다). 날짜는 두 자료가 같다.

10. `widgets/data.json` `ethereum.eip1559.observed_latest_block`(블록 26,264,275, 2026-10-24T18:23Z, RPC 3곳

일치: `gasLimit` 60,000,000, `gasUsed` 32,652,385)과 `_note`(go-ethereum v1.17.10 `params/config.go` L1016–L1023이 포크 분기 없이 두 상수를 돌려준다).

11. Roughgarden (2024), 30:11(§3.1). 원문: "A TFM is then dominant-strategy incentive compatible (DSIC) if, assuming that the miner carries out the intended allocation rule, every user (no matter what their valuation) has a dominant strategy—a bid that always maximizes the user's utility (3), no matter what the bids of the other users. FPAs are, of course, not DSIC."

12. Roughgarden (2024), 30:12(§3.2), Definition 3.4. 원문: "A TFM (x, p, q) is MMIC if, for every on-chain history H and mempool M, a myopic miner maximizes its utility (5) by creating no fake transactions (i.e., setting F = ∅) and following the suggestion of the allocation rule x (i.e., setting $B_k = x(H, M)$)."; "For example, FPAs are MMIC—the intended allocation rule maximizes miner net revenue, which the miner is happy to do ...". 「근시안적」은 현재 블록의 순수입만 보는 채굴자라는 뜻(30:11).

13. Roughgarden (2024), 30:12–30:13(§3.3), Definition 3.6·3.7. 원문: "A TFM (x, p, q) is OCA-proof if, for every on-chain history H, there exists an individually rational bidding strategy σ_H such that, for every possible set T of outstanding transactions and valuations v, there is no OCA under which the utility (3) of every transaction creator and the utility (5) of the miner is strictly higher than in the outcome $B_k = x(H, M(\sigma_H(v)))$ with on-chain bids $\sigma_H(v)$ and no off-chain transfers."

14. Roughgarden (2024), 30:16, Table 1 ("Which of the Six Listed TFMs are MMIC, DSIC, or OCA-Proof"). 표의 「거의(almost)」는 각주 28(낙찰 최저 입찰을 탈락 최고 입찰의 대리로 쓰는 SPA는 일반적으로 DSIC가 아니나 근사적으로 그렇다)과 Remark 5.13, 「보통(usually)」은 Theorem 5.7·Remark 5.9·Corollary 5.15 + Remark 5.16에 걸려 있다. 요약 문장: "Thus, if we assess these TFMs solely according to these three types of incentive guarantees, FPAs dominate β-burn FPAs with β > 0; the 1559 mechanism dominates these and β-burn 1559 mechanisms with β < 1; and the 1559 mechanism and the tipless mechanism are incomparable."

15. Roughgarden (2024), 30:4(§1의 결과 요약)·30:22(§6). 원문: "The 1559 mechanism reverts to an FPA in the special case of a zero reserve price, but Theorem 5.7 identifies two conditions that together are sufficient for the optimality of straightforward bidding: the base fee should not be excessively low for the current demand curve (Definition 5.6), and transaction creators should use individually rational bidding strategies. Under these conditions, the 1559 mechanism acts as a posted-price mechanism, with price equal to the reserve price plus the miner's opportunity cost for transaction inclusion."; "In a block with an excessively low base fee (presumably due to rapidly increasing demand), one might expect bidding wars in the 1559 mechanism (on account of the failure of DSIC)"; "The 'obvious optimal bid' in the 1559 mechanism (setting $c_t = v_t$ and $\delta_t = \mu$) is optimal only in blocks with neither an excessively low base fee nor overbidding."

16. Roughgarden (2024), 30:22와 각주 33. 원문: "We conjecture that there is no TFM that always satisfies all three properties."; 각주 33: "[This conjecture was recently proved by Chung et al. [17]]."

17. Roughgarden (2024), 30:4–30:5(§1 결과 요약)·30:22(Corollary 5.17·5.18). 원문: "Finally, OCA-proofness considerations show that two of the major innovations in the 1559 mechanism, relative to an FPA—a reserve price and burned fees—are inextricably linked, as making either change without the other leads to a non-OCA-proof transaction fee mechanism. Intuitively, burning fees in an FPA catalyzes a miner and users to collude off-chain to avoid the burn (Corollary 5.17) while passing revenue from a reserve price to a block's miner opens the door for low-value transactions to pay the reserve price on-chain while receiving a partial refund from the miner off-chain (Corollary 5.18)."

18. EIP-1559, Motivation 절 마지막 문단. 원문: "An important aspect of this fee system is that miners only get to keep the priority fee. The base fee is always burned (i.e. it is destroyed by the protocol). This ensures that only ETH can ever be used to pay for transactions on Ethereum, cementing the economic value of ETH within the Ethereum platform and reducing risks associated with miner extractable value (MEV). Additionally, this burn counterbalances Ethereum inflation while still giving the block reward and priority fee to miners. Finally, ensuring the miner of a block does not receive the base fee is important because it removes miner incentive to manipulate the fee in order to extract more fees from users." 이 책은 이 문단 가운데 메커니즘에 관한 부분(채굴자의 조작 유인 제거)만 가져가고, 「ETH의 경제적 가치를 공고히」 같은 서술은 설계자의 의도 표명으로만 둔다.

19. EIP-1559, Security Considerations 절 「ETH Burn Precludes Fixed Supply」. 원문: "By burning the base fee, we can no longer guarantee a fixed Ether supply. This could result in economic instability as the long term supply of ETH will no longer be constant over time. While a valid concern, it is difficult to quantify how

much of an impact this will have. If more is burned on base fee than is generated in mining rewards then ETH will be deflationary and if more is generated in mining rewards than is burned then ETH will be inflationary. Since we cannot control user demand for block space, we cannot assert at the moment whether ETH will end up inflationary or deflationary, so this change causes the core developers to lose some control over Ether's long term quantity."

20. Roughgarden (2024), 30:22–30:23(§6 "Alternatives to burning"). 원문: "One appealing alternative implementation, with the same incentive-compatibility properties, is to instead split the base fee revenues of a block equally among the miners of the next ℓ blocks."; "While burning the base fee revenue favors holders of the native currency (through deflation), this 'pay-it-forward' implementation favors miners (through more transaction fee revenue)."; "Thus, the blockchain's inflation rate would be variable and unpredictable from block to block, but miner revenue (which effectively pays for the blockchain's security [3, 8]) would stay relatively constant (modulo fluctuations in the market price of the native currency). With the pay-it-forward implementation, the inflation rate would be essentially deterministic but the miner rewards (and, hence, blockchain security) would be unpredictable (though never less than that with money burning)." 정의 2.5(30:9)는 소각을 「디플레이션을 통한 통화 보유자에 대한 일괄 환급, 자사주 매입과 비슷한 것」과 같다고 적는다 — 메커니즘의 서술이지 가격 명제가 아니다.

21. Roughgarden (2024), 30:23(§6 "Multi-block time scales"). 원문: "For example, publishing a smaller-than-target block decreases the base fee for the next block, potentially increasing the revenue of that block's miner. … Long-term manipulation by a cartel of miners thus resembles the challenge of sustaining collusion in a repeated multi-player Prisoner's Dilemma game: If all players cooperate … they are all better off, but each player has a myopic incentive to unilaterally deviate from this strategy …"; "Persistent and harmful miner collusion has not yet been definitively observed in a major blockchain such as Bitcoin or Ethereum."; "especially in a regime in which a single miner or validator might control a significant fraction (30%, say) of the overall hashrate or stake and therefore might plausibly benefit from non-myopic strategies."

22. Vitalik Buterin, Dankrad Feist, Diederik Loerakker, George Kadianakis, Matt Garnett, Mofi Taiwo & Ansgar Dietrichs, "EIP-4844: Shard Blob Transactions" (Standards Track, Core, status Final, created 2022-02-25, requires 1559·2718·2930·4895), Gas accounting 절. 원문: "We introduce blob gas as a new type of gas. It is independent of normal gas and follows its own targeting rule, similar to EIP-1559. We use the `excess_blob_gas` header field to store persistent data needed to compute the blob gas base fee. For now, only blobs are priced in blob gas." https://github.com/ethereum/EIPs/blob/e858e912ff55370eb8125873455af531e322d9b2/EIPS/eip-4844.md (2026-10-25 접근, raw 마크다운). 같은 커밋의 raw 파일로 확인했다.

23. EIP-4844, Parameters 표(`MAX_BLOB_GAS_PER_BLOCK` 786432, `TARGET_BLOB_GAS_PER_BLOCK` 393216, `MIN_BASE_FEE_PER_BLOB_GAS` 1, `BLOB_BASE_FEE_UPDATE_FRACTION` 3338477, `GAS_PER_BLOB` 217)과 Rationale 「Base fee per blob gas update rule」·「Throughput」 절. 원문: "If block N consumes X blob gas, then in block N+1 `excess_blob_gas` increases by X - TARGET_BLOB_GAS_PER_BLOCK, and so the `base_fee_per_blob_gas` of block N+1 increases by a factor of e((X - TARGET_BLOB_GAS_PER_BLOCK) / BLOB_BASE_FEE_UPDATE_FRACTION). Hence, it has a similar effect to the existing EIP-1559, but is more 'stable' in the sense that it responds in the same way to the same total usage regardless of how it's distributed."; "The parameter BLOB_BASE_FEE_UPDATE_FRACTION controls the maximum rate of change of the base fee per blob gas. It is chosen to target a maximum change rate of e** (TARGET_BLOB_GAS_PER_BLOCK / BLOB_BASE_FEE_UPDATE_FRACTION) ≈ 1.125 per block."; "The values for TARGET_BLOB_GAS_PER_BLOCK and MAX_BLOB_GAS_PER_BLOCK are chosen to correspond to a target of 3 blobs (0.375 MB) and maximum of 6 blobs (0.75 MB) per block. These small initial limits are intended to minimize the strain on the network … and are expected to be increased in future upgrades".

24. EIP-4844, Gas accounting 절. 원문: "The actual `blob_fee` as calculated via `calc_blob_fee` is deducted from the sender balance before transaction execution and burned, and is not refunded in case of transaction failure."

3부 경제학

22장 토큰 설계의 경제학 — 발행 규칙·거버넌스·인센티브

핵심 TAKEAWAY

14장에서 토큰 표준은 「함수 이름의 약속」이고 그 토큰이 어떤 권리를 담는지는 발행자의 바깥 약속이 정한다고 했습니다. 이 장은 그 바깥 약속을 **설계 변수로** 읽습니다. 셋입니다. 얼마나, 어떤 규칙으로 찍는가(발행). 처음에 누가 받는가(분배). 규칙을 누가 바꿀 수 있는가(거버넌스). 세 변수는 회사의 정관과 주식 발행 설계를 닮았지만 한 가지가 다릅니다. **집행자가 없습니다.** 약속을 어긴 발행자를 불러 세울 법원도, 정관 위반을 다툴 주주총회도 코드 안에는 없고, 있는 것은 다수결과 포크뿐입니다. 「토큰 경제학(tokenomics)」이라는 말은 흔히 이 세 변수를 가치의 근거처럼 포장하는 데 쓰입니다. 이 장은 반대로 그것을 **위험의 지도로** 읽습니다. 발행 규칙은 바뀔 수 있고, 분배는 집중되며, 거버넌스는 다수를 가진 쪽의 것입니다. 이 장을 읽으면 토큰 설계 문서를 받았을 때 무엇을 먼저 보아야 하는지, 그리고 「거버넌스 토큰 다수가 규칙을 바꾸면 소수는 무엇을 주장할 수 있는가」가 왜 법의 질문인지 압니다.

학습목표

1. 코인과 토큰의 구별, 발행 규칙의 세 유형(고정·감소, 영구·가변, 탄력)과 각각의 경제적 함의를 원전의 문장으로 말한다. 소각이 발행 규칙을 어떻게 바꾸는지 안다.
2. 분배의 네 통로(채굴·검증 보상, 사전 판매, 무상 배분, 팀·재단 할당)를 **메커니즘과 위험으로만** 읽는다. 보유 집중의 실측치를 표본·기간과 함께 옮긴다.
3. [중급] 투표 집중도를 지니 계수와 상위 k 보유 비율로 계산하고, 「주소 ≠ 사람」이라는 측정의 한계를 안다.
4. 거버넌스 토큰의 투표 구조를 The DAO의 공식 기록으로 읽고, 다수결·큐레이터·분리(split)가 각각 누구를 보호하도록 설계되었는지, 그리고 어디에 집행자가 없는지 안다.

[입문]**만 읽은 독자가 가져가는 것.** 토큰 설계는 회사 정관·주식 발행 설계와 구조가 닮았으나, **집행자가 없다는** 점에서 다릅니다.

1. 이 장이 답하려는 질문

19장은 아무것도 약속하지 않는 코인의 가격이 왜 양(+)일 수 있는지를, 24장은 「1달러로 돌려준다」는 약속이 붙은 토큰이 왜 런을 겪는지를 봅니다. 그 사이에 넓은 땅이 있습니다. 무엇인가를 약속하되 그 약속이 상환이 아닌 토큰들입니다. 프로토콜의 수수료를 나눠 받을 권리, 규칙 변경에 투표할 권리, 서비스에 접근할 권리, 또는 그저 「생태계의 성장에 참여」한다는 말. 이 토큰들의 설계 문서는 대개 「토큰 경제학」이라는 제목 아래 발행량·분배 일정·투표 구조를 적습니다. 이 장은 그 문서를 읽는 법입니다. 가격을 묻지 않고, 세 변수가 **누구에게 무엇을 할 수 있게 하는지**를 묻습니다.

먼저 낱말 하나. 우(Ke Wu)·휘틀리(Spencer Wheatley)·소르네트(Didier Sornette)의 2018년 논문은 「어떤 암호자산은 자기 자신의 독립된 네트워크에서 작동하는 「코인」이고, 다른 것은 코인 네트워크를 플랫폼으로 그 위에서 작동하는 「토큰」이라는 본질적 구별」을 둡니다.[1] 이 책도 그 구별을 씁니다. 코인(비트코인·이더)은 13장의 가스와 20장의 보상처럼 프로토콜 자체가 발행 규칙을 가지고, 토큰은 14장의 ERC-20 계약처럼 **누군가가 배포한 계약이** 발행 규칙을 가집니다. 규칙을 바꾸는 방법도 다릅니다. 코인의 규칙을 바꾸려면 23장의 포크가 필요하고, 토큰의 규칙은 계약을 쓴 사람이 어떻게 썼느냐에 달려 있습니다.

2. 발행 — 얼마나, 어떤 규칙으로

발행 규칙은 셋으로 나뉩니다.

유형	규칙	이 책에서 본 예	설계가 떠안는 것
고정·감소	총량 상한과 감소 일정이 코드에 있다	비트코인 — 210,000블록마다 반감, 2,100만 미만에 수렴(20장 NB20-1)	보조금이 0으로 가므로 보안 예산을 수수료가 메워야 한다(20장, Carlsten 외)
영구·가변	상한 없이 규칙에 따라 계속 찍되, 소각이 있으면 순발행은 음(−)일 수 있다	이더리움 — 블록 보상은 계속, 기본 수수료는 소각(21장)	순공급을 설계자가 통제하지 못한다(EIP-1559 보안 고려 절)
탄력	가격 목표에 맞춰 공급을 늘리고 줄인다	알고리듬형 스테이블코인(24장)	공급 조정이 두 번째 토큰의 수요에 기대므로 반사적 되먹임이 생긴다(24장 NB24-3)

셋째 유형은 24장에서 다루니 여기서는 첫째와 둘째를 봅니다. 고정 발행은 가장 단순하고 가장 자주 「가치의 근거」로 포장됩니다. 19장이 그 포장을 벗겼습니다. 개럿–월리스의 식 (13)에서 발행량 B 는 분모에 있지만 그것을 쓰려는 사람의 비율 α 가 분자에 있고, $\alpha = 0$ 이면 B 가 아무리 작아도 가격은 0입니다. 실링–울리히의 정리 2는 발행이 보유자를 희석한다는 직관조차 통화 경쟁 모형에서는 그대로 성립하지 않음을 보였습니다 — 블록 보상은 「비트코인 보유자에게 매기는 세금이 아니라 달러 세금으로 조달」됩니다.[2] 그리고 20장의 칼스텐 외는 고정 발행의 **설계 비용을** 적었습니다. 보조금이 0으로 가면 채굴 게임이 불안정해지므로 「새 암호통화의 설계자들은 통화 인플레이션의 불가피성을 받아들이고 블록 보상을 영구적으로 두어야 할지 모른다.」[3] 고정 발행은 보안 예산의 재원을 미래의 수수료 시장에 거는 선택이고, 그 선택의 값은 20장의 식 (3)이 매깁니다.

둘째 유형은 반대쪽에서 같은 문제를 만납니다. 이더리움의 백서는 이더에 「다양한 디지털 자산 사이의

효율적 교환을 위한 1차 유동성 층을 제공하고, 더 중요하게는 거래 수수료를 지불하는 메커니즘을 제공하는 이중 목적」을 주었고,[4] 2021년의 EIP-1559는 그 수수료의 일부를 태우면서 순공급의 경로를 **설계자가 통제할 수 없는 것으로** 만들었습니다. 21장에서 읽은 문장입니다. 「블록 공간에 대한 사용자 수요를 통제할 수 없으므로, 우리는 현재로서 ETH가 인플레이션이 될지 디플레이션이 될지 단언할 수 없고, 따라서 이 변경은 핵심 개발자들이 이더의 장기 수량에 대한 통제력을 일부 잃게 만든다.」[5] 여기서 「소각」을 둘러싼 흔한 서술을 다시 끊어 둡니다. 소각은 발행 규칙의 한 항이고, 21장의 러프가든이 적은 대로 수수료의 **귀속을** 보유자 쪽으로 정하는 선택입니다. 판 오르트의 2024년 모형은 그 선택이 버블 균형의 성질을 바꿀 수 있다고 적습니다 — 「순발행이 음(−)인 암호통화(예컨대 거래 수수료를 태우는)는 가격 경로가 버블 궤적을 따르더라도 투자자에게 양의 총현금흐름을 낳을 수 있다.」[6] 이것은 「소각하면 오른다」가 아니라 「소각이 있으면 이 모형의 어떤 균형이 폰지 등가에서 벗어난다」는 **모형 안의 명제이고**, 25장에서 「버블」이 무엇인지 정의한 뒤에 다시 읽을 수 있습니다.

토큰(계약이 발행하는 것)의 경우 발행 규칙은 계약 코드에 있고, 14장에서 본 대로 ERC-20 표준은 그것을 **정하지 않습니다**. 총량이 고정인지, 발행자가 더 찍을 수 있는지, 누가 태울 수 있는지는 전부 표준 밖의 함수입니다. 설계 문서가 「총량 고정」이라고 적어도 계약에 발행 함수가 남아 있으면 고정이 아닙니다. 그래서 토큰의 발행 규칙을 읽는 첫 질문은 문서가 아니라 **계약에 발행 권한이 누구에게 남아 있는가입니다**.

3. 분배 — 처음에 누가 받는가

발행된 단위는 누군가에게 갑니다. 통로는 넷입니다.

- **채굴·검증 보상.** 나카모토가 적은 대로 보상은 「발행할 중앙 권위가 없으므로 코인을 처음 유통에 넣는 방법」입니다.[7] 받는 사람은 자원(계산력·담보)을 넣은 사람이고, 분배의 기준은 자원의 비율입니다.
- **사전 판매.** 발행 전에 토큰을 팔아 자금을 모읍니다. 14장과 26장의 The DAO가 공식 기록이 있는 예입니다. 미국 SEC의 보고서에 따르면 「2016년 4월 30일부터 5월 28일까지 The DAO는 약 11억 5천만 DAO 토큰을 약 1,200만 이더와 교환으로 제안·판매했고」, 「토큰 가격은 구매 시점에 따라 100 DAO 토큰당 약 1~1.5 ETH 범위에서 움직였으며 … 제안되는 토큰 수, 구매자 수, 구매자의 전문성 수준에 아무 제한이 없었다.」[8]
- **무상 배분(에어드롭).** 조건(과거 사용, 보유, 특정 행위)을 만족한 주소에 토큰을 보냅니다. 이 책이 원전으로 확인한 공식 기록은 없으므로 메커니즘과 위험만 적습니다. 받은 쪽에 아무 비용이 들지 않았으므로 즉시 처분할 유인이 있고(덤핑), 조건을 자동화하면 같은 사람이 여러 주소로 조건을 채우는 일을 막기 어려우며(시빌), 배분이 넓어 보여도 실제 통제는 소수에 남을 수 있습니다(집중).
- **팀·재단·투자자 할당.** 발행량의 일정 몫을 개발팀·재단·초기 투자자에게 두고, 흔히 일정 기간 잠급니다(베스팅). 잠금이 풀리는 시점이 공급의 계단이 됩니다.

네 통로 어느 것도 그 자체로 좋거나 나쁘지 않습니다. 이 장이 묻는 것은 결과, 곧 **집중입니다**. 집중의 실측치는 적고, 그마저 측정이 어렵습니다. 11장에서 본 대로 주소는 사람이 아니고, 거래소의 콜드월렛 하나가 수많은 고객의 잔고를 담습니다. 마카로프–쇼어는 그 문제를 정면으로 다룹니다. 「소유 집중을 정하는 일은 가장 부유한 주소들의 잔고를 추적하는 것보다 복잡하다. 가장 큰 주소들 가운데 많은 수가 거래소와 온라인 지갑의 콜드월렛에 속하고, 그것은 많은 투자자를 대신해 비트코인을 보유한다.」 그래서 그래프 분석으로 주소를 개인과 중개기관으로 나눕니다.[9] 결과를 표본·기간과 함께 옮깁니다.

측정	값	표본·기준	출처·한계
1,000 BTC 이상 「부자 주소」	2,258개, 합계 790만 BTC(유통량의 거의 절반)	2020-12-31 잔고	주소 단위 — 저자들은 이 해석이 콜드월렛을 무시한다고 적는다
그 가운데 개인 / 중개기관 / 불명	1,013 / 1,154 / 47개	같은 기준	그래프 분석 분류. 세 범주의 합은 2,214개로 총계 2,258개보다 44개 적다 — 원문 28쪽의 숫자가 그대로 그렇고, 원문은 차이를 설명하지 않는다
중개기관 보유	550만 BTC, 유통량의 약 1/3	2020년 말	2014년부터 꾸준히 증가
개인 보유(부자 주소 중 개인 + 2020년 내내 거래 없는 1~1,000 BTC 묶음)	약 40만 묶음, 합계 850만 BTC	2020년 말	비활동 조건으로 중개기관 지갑을 걸러냄
개인 보유의 집중	상위 1,000묶음이 약 300만 BTC, 상위 10,000묶음이 500만 BTC 넘게(전체 발행량의 약 1/4)	2020년 말	**과소 추정이라고** 저자들이 명시 — 같은 주체의 여러 주소를 합치지 않았고, 초기 약 2만 주소를 한 사람(나카모토)으로 보지 않았다
채굴 집중	상위 10% 채굴자가 용량의 90%, 상위 0.1%(약 50) 가 약 50%	지난 5년(2021년 기준), 상위 20개 풀의 보상 분배 추적	집중은 반경기적 — 가격 급락 뒤·반감 뒤에 커진다

10

이 표에서 가져갈 것은 숫자보다 **측정의 교훈입니다**. 주소를 세면 집중이 과대평가되고(콜드월렛), 묶음을 세면 과소평가됩니다(같은 주체의 여러 묶음). 어느 쪽이든 「탈중앙」이라는 말이 분배의 평등을 뜻하지는 않습니다. 채굴 쪽의 집중은 23장의 문턱과 바로 이어집니다. 상위 0.1%가 용량의 절반을 쥔다는 것은 풀 뒤의 개별 채굴자 단위의 집중이고(저자들은 풀이 채굴자를 반드시 통제하지는 않는다고 적습니다), 이얄–시러의 문턱은 풀 단위·$\gamma = 0$ 일 때의 모형 값이므로 둘을 바로 잇지는 못합니다. 다만 결정 주체가 소수라는 사실은 23장의 문턱을 읽을 때 염두에 둘 것입니다.



그림 22-1. 마카로프–쇼어 표가 주는 측정의 교훈. 거래소 콜드월렛 주소 하나가 많은 고객의 잔고를 담으므로 주소를 세면 집중이 과대평가되고, 같은 주체의 여러 묶음을 따로 세면 과소평가된다. 분류 수치는 넣지 않고 편향의 방향만 그렸다.

[중급] 집중을 재는 두 수 — 지니 계수와 상위 k 비율

[중급] 보유량을 $x_1 \le x_2 \le \cdots \le x_n$ 으로 오름차순 정렬하면 **지니 계수는**

$$G = \frac{2\sum_{i=1}^{n} i\,x_i}{n\sum_{i=1}^{n} x_i} - \frac{n+1}{n}$$

입니다. 기호 풀이: 모두 같으면 0, 한 사람이 전부 가지면 $1 - 1/n$(큰 n 에서 1)입니다. **상위 k 보유 비율은** 가장 큰 k 개 보유량의 합을 전체로 나눈 것입니다. 수치 예: 보유자 다섯이 5, 5, 10, 20, 60을 가졌다면 $\sum x_i = 100$, $\sum i x_i = 1\cdot5 + 2\cdot5 + 3\cdot10 + 4\cdot20 + 5\cdot60 = 425$, $G = 2\cdot425 / (5\cdot100) - 6/5 = 1.7 - 1.2 = 0.5$ 이고, 상위 1 비율은 60%, 상위 2 비율은 80%입니다. 토큰 가중 투표에서 상위 1 비율이 50%를 넘으면 그 한 주소가 모든 안건을 혼자 정합니다. 두 수 모두 「주소 = 사람」을 전제하므로, 위 표의 교훈이 그대로 붙습니다. 같은 주체가 주소를 나누면 G 는 내려가고, 거래소가 고객 토큰으로 투표하면 상위 k 비율은 그 거래소의 것이 됩니다. 이 장은 설계대로 코드를 두지 않았습니다. 두 식은 손으로 계산할 수 있고, 어려운 것은 식이 아니라 분모입니다.

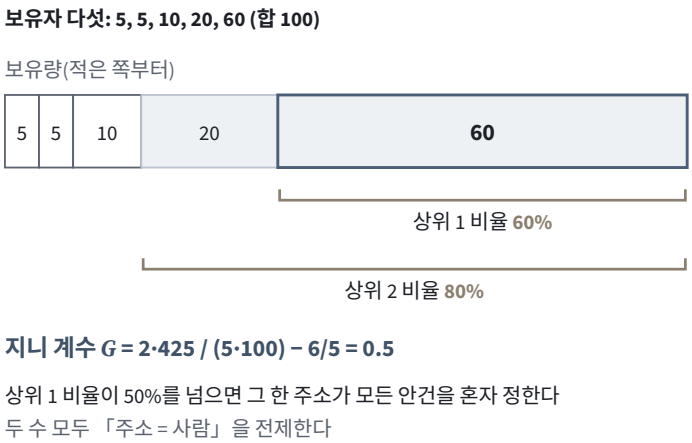


그림 22-2. 본문 수치 예(보유량 5, 5, 10, 20, 60)를 띠 하나에 놓았다. 상위 1·2 비율과 지니 계수 0.5는 본문 식으로 낸 값이다.

4. 거버넌스 — 규칙을 누가 바꾸는가

거버넌스 토큰은 「투표할 권리」를 담는 토큰입니다. 무엇에 대해 투표하는지는 계약마다 다릅니다. 유니스왑 v3 백서의 거버넌스 절이 간결한 예입니다. 「팩토리에는 소유자가 있고, 처음에는 UNI 토큰 보유자들이 통제한다. 소유자는 핵심 계약의 작동을 중단시킬 능력이 없다.」 소유자가 할 수 있는 것은 셋입니다. 「프로토콜 수수료를 켤 수 있고 … 스왑 수수료 가운데 프로토콜로 가는 비율을 $1/N$($4 \le N \le 10$) 또는 0 가운데 고를 수 있으며, 이 매개변수는 풀마다 정할 수 있다.」 「수수료 단계를 더할 수 있고 … 한번 더해진 수수료 단계는 제거할 수 없다.」 「소유권을 다른 주소로 이전할 권한이 있다.」[11] 15장에서 본 AMM의 수학은 그대로인데, 그 위에 「누가 수수료 스위치를 켜는가」라는 작은 정관이 얹혀 있습니다. 할 수 없는 것(중단)과 되돌릴 수 없는 것(단계 제거)이 명시되어 있다는 점이 중요합니다. 토큰 보유자의 권한은 코드가 열어 둔 만큼입니다.

가장 자세한 공식 기록은 The DAO입니다. 부테린의 2014년 백서는 「탈중앙 조직」을 「구성원 또는 주주의 집합을 가진 가상 실체로, 아마도 67% 다수로 실체의 자금을 쓰고 코드를 수정할 권리를 갖는 것. … 이는 본질적으로 전통적 회사나 비영리 조직의 법적 장치를 **암호학적 블록체인 기술만으로 집행하여** 복제한다」고 적었습니다.[12] 2016년의 The DAO가 그 첫 대규모 구현이었고, SEC 보고서가 그 투표 구조를 기록합니다. 「The DAO가 모은 ETH는 DAO 토큰 보유자의 다수결 투표로만 계약자에게 배분되어 제안을 지원할 수 있었다. DAO 토큰 보유자는 특정 제안의 지원에 찬성 또는 반대하는 투표를 하며, 투표는 **그들이 통제하는 토큰 수로 가중되었다.**」 그런데 그 투표에는 알려진 편향이 있었습니다. 「투표 절차는 왜곡된 투표 행동을 유인할 수 있어 DAO 토큰 보유자 다수의 합의를 정확히 반영하지 못하리라고 공개적으로 비판받았다. … 2016년 5월 27일 컴퓨터 보안 연구자들의 블로그 글이 지적했듯, The DAO의 구조는 「투표 뒤 사용자의 선택지를 제한하는 방식의 부작용으로 제안에 YES를 투표하고 NO 투표를 억누르는 강한 양(+)의 편향」을 가졌다.」 각주가 기제를 설명합니다. 투표한 토큰은 투표 주기가 끝날 때까지 묶이지만 기권한 토큰은 언제든 옮길 수 있어, 「DAO 토큰 보유자는 찬성하거나 기권하도록 유인되었다.」[13]

다수결 위에 **큐레이터가** 있었습니다. 「어떤 제안이든 DAO 토큰 보유자의 투표에 부쳐지기 전에 The DAO의 「큐레이터」 한 명 이상의 검토를 받아야 했다. The DAO 결성 당시 큐레이터는 Slock.it이 고른 개인들의 집단이었다. 백서에 따르면 … 큐레이터는 「상당한 권한」을 가졌다. 큐레이터는 결정적 보안 기능을 수행했고 어느 제안이 The DAO에 제출되고 투표되고 지원될 수 있는지에 대한 최종 통제력을

유지했다.」 큐레이터 가운데 한 사람은 공개적으로 「큐레이터는 화이트리스트에 대한 완전한 통제력 … 제안의 순서와 빈도를 통제할 명백한 능력」을 가졌다며 「큐레이터는 엄청난 권한을 가진다」고 말했고, 다른 큐레이터는 자기 개인 윤리를 포함한 주관적 기준을 공표했습니다. 큐레이터는 「정족수 요건을 격주로 50%씩 낮출 권한」도 가졌습니다.[14] 그리고 소수를 보호하는 장치가 하나 있었습니다. **분리(split) 입니다**. 각주에 따르면 「백서는 분리의 1차 목적이 소수 주주를 보호하고 흔히 「51% 공격」이라 부르는 것 — DAO 실체 토큰의 51%를 가진 공격자가 DAO 실체의 모든 자금을 자기에게 보내는 제안을 만드는 것 — 을 막는 데 있다」고 적었습니다. 토큰 보유자는 「약 46일이 걸리는 복잡한 다주차 절차」로 자기 토큰을 ETH로 바꿀 수 있었습니다.[15] 회사법의 언어로 옮기면, 다수결(주주총회), 의안 상정권을 쥔 소수의 선임자(이사회 비슷하나 선출되지 않은), 그리고 반대 주주의 탈퇴권(주식매수청구권 비슷한)이 코드로 적혀 있었던 셈입니다. 다만 그것들을 **다툴 곳이** 없었습니다. 공격자가 코드의 결함(SEC 보고서의 표현으로는 「The DAO 코드의 결함」)을 써서 자금의 약 1/3을 옮겼을 때, 남은 선택지는 법원이 아니라 하드포크였습니다. 그 사건은 26장에서 읽습니다.

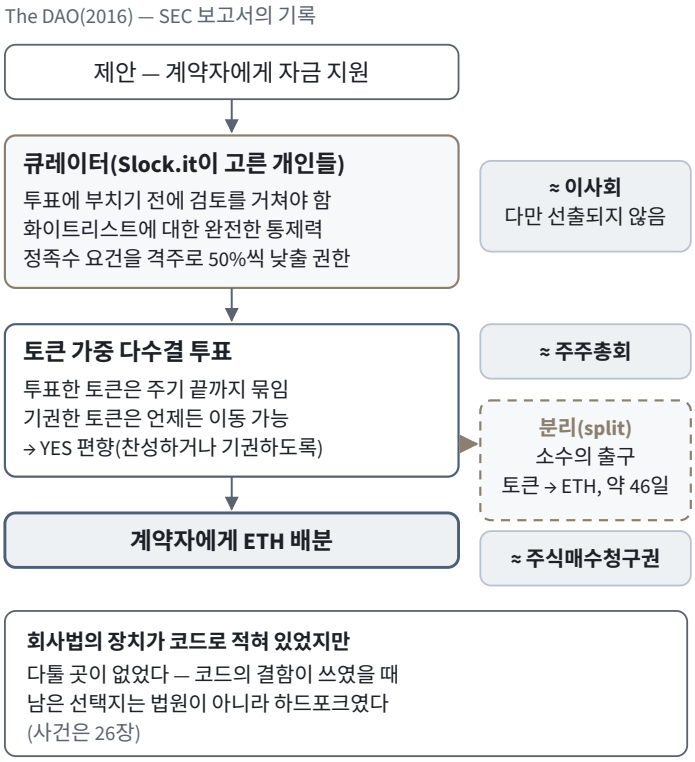


그림 22-3. SEC 보고서가 기록한 The DAO의 의사결정 경로와 회사법 대응. 큐레이터가 상정을, 토큰 가중 투표가 배분을 정했고, 소수의 출구는 분리였다.

세 변수를 합쳐 거버넌스 설계 문서를 읽는 질문을 적습니다. (1) 투표권은 토큰 수에 비례하는가, 그렇다면 상위 k 비율은 얼마인가. (2) 의안 상정에 문지기가 있는가, 그 문지기는 누가 뽑고 누가 바꾸는가. (3) 다수가 할 수 없게 코드가 막아 둔 것이 있는가(유니스왑의 「중단 불가」처럼). (4) 소수가 나갈 길이 있는가, 그 길은 얼마나 걸리는가. (5) 투표의 결과를 코드가 자동 집행하는가, 아니면 누군가의 서명이 필요한가(다중서명 키를 쥔 사람이 사실상의 집행자). 이 다섯 가운데 어느 것도 「토큰 경제학」 문서의 발행량 그래프에는 없습니다.

5. 「토큰 경제학」이 마케팅 용어로 쓰이는 점

설계 문서는 흔히 세 변수를 **가치의 서사로** 씁니다. 발행량 상한은 「희소성」으로, 소각은 「디플레이션」으로, 스테이킹 보상은 「수익」으로, 에어드롭은 「커뮤니티에 돌려줌」으로. SEC 보고서가 The DAO의 홍보 자료에서 인용한 문장이 그 어조의 표본입니다. 창립자는 참여자가 ETH를 보내 토큰을 사면 「투표하고 「보상」을 받을 수 있다」고 설명하면서 이것을 「**회사의 주식을 사서 … 배당을 받는 것**」**에 비유했고**, 웹사이트는 The DAO의 목적을 「어디에도 없으면서 동시에 어디에나 존재하고 오직 멈출 수 없는 코드의 강철 같은 의지로만 작동하며, 구성원의 이익을 위해 사업의 새 길을 개척하는 것」이라 적었습니다.[16] 이 장은 이 서사를 판정하지 않습니다. 다만 같은 변수가 **반대쪽으로도** 읽힌다는 것을 보였습니다. 희소성은 수요 없이는 가격이 아니고(19장), 소각은 수수료 귀속의 선택이며(21장), 보상은 메커니즘이고(10장·20장), 무상 배분은 덤핑과 시빌의 통로이며, 투표권은 상위 k 의 것입니다. 「토큰 경제학」이라는 말을 들으면 이 장이 권하는 번역은 하나입니다. 「**누가 무엇을 바꿀 수 있고, 그것을 막을 사람은 누구인가.**」

> **? 법률가의 질문**
>
> 거버넌스 토큰의 다수가 규칙을 바꿨습니다. 수수료 스위치를 켰거나, 소수가 반대한 제안을 통과시켰거나, 발행 규칙을 고쳤습니다. 투표는 코드가 정한 대로 집계되었고 결과는 자동으로 집행되었습니다. 소수는 무엇을 주장할 수 있습니까. 회사라면 주주평등, 충실의무, 소수주주권, 주식매수청구권이 있습니다. DAO는 조합입니까 — 그렇다면 구성원은 무한책임을 집니까. 법인입니까 — 어느 법에 따라 설립되었습니까. 아무것도 아닙니까 — 그렇다면 토큰 보유자 사이의 관계는 무엇이고, 큐레이터처럼 상정권을 쥔 사람의 의무는 누구에 대한 것입니까. The DAO 백서 자체가 「DAO 실체의 법적 지위는 활발하고 격렬한 논쟁의 대상」이라고 적었습니다 (SEC 보고서 3쪽 각주 10). 이 책은 답하지 않고 5부의 쟁점 절을 가리킵니다. → 28장

6. 정리

토큰 설계의 변수는 발행·분배·거버넌스 셋입니다. 발행 규칙은 고정·영구·탄력으로 갈리고 각각 보안 예산·순공급 통제·반사성이라는 값을 치르며, 토큰의 경우 표준이 아니라 계약이 그 규칙을 정합니다. 분배는 네 통로로 이루어지고 결과는 집중이며, 집중의 측정은 「주소 ≠ 사람」이라는 벽에 부딪힙니다 — 그래도 실측치는 상위 1만 묶음이 발행량의 약 1/4, 상위 0.1% 채굴자가 용량의 약 절반을 가리킵니다. 거버넌스 토큰은 코드가 열어 둔 만큼의 투표권이고, The DAO의 기록은 다수결·문지기·탈퇴권이 코드로 적혔으되 그것을 다툴 곳이 없었음을 보여 줍니다. 회사 정관과 닮았지만 집행자가 없다는 것, 그것이 이 장의 한 문장입니다. 23장은 규칙을 바꾸는 가장 큰 투표, 곧 포크가 게임이론에서 어떻게 보이는지를 다루고, 26장은 The DAO에서 그 투표가 실제로 어떻게 이루어졌는지를 읽습니다.

이 장의 출처

- Ke Wu, Spencer Wheatley & Didier Sornette, "Classification of cryptocurrency coins and tokens by the dynamics of their market capitalizations," *Royal Society Open Science* 5: 180381 (2018), DOI 10.1098/rsos.180381 — 코인·토큰의 구별(2쪽). 정의 한 문장만 썼다.
- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform" (2014; 초고 2013) — 「Decentralized Autonomous Organizations」(PDF 23쪽), 「Currency And Issuance」(PDF 30쪽).
- SEC, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Release No. 81207 (2017-07-25) — 판매(2–3쪽·6쪽), 홍보 문구(4–5쪽), 분리와 각주 21(6쪽), 투표와 각주 24(7쪽), 큐레이터(7–8쪽).
- Hayden Adams 외, "Uniswap v3 Core," March 2021 — §4 Governance(3쪽).
- Igor Makarov & Antoinette Schoar, NBER Working Paper 29396 (2021) — 소유 집중(5–6쪽, §5 26–29쪽), 채굴 집중(4쪽).
- Susan Athey 외, Stanford GSB WP 3469 (2016) — 소유 집중(3쪽). Satoshi Nakamoto (2008) §6. Linda Schilling & Harald Uhlig (2019) 정리 2. Miles Carlsten 외 (2016) §7. EIP-1559 Security Considerations. Maarten van Oordt (2024) 초록 — 각각 19·20·21장과 같은 자리.
- 지니 계수·상위 k 비율의 수치 예(5, 5, 10, 20, 60 → $G = 0.5$)는 이 책의 손계산이다. 설계대로 이 장에는 코드·노트북이 없다.
- 무상 배분(에어드롭)의 메커니즘·위험 서술은 원전 없이 이 책이 구조로 적은 것이며, 어떤 실제 배분 사례도 가리키지 않는다.

각주

1. Ke Wu, Spencer Wheatley & Didier Sornette, "Classification of cryptocurrency coins and tokens by the dynamics of their market capitalizations," *Royal Society Open Science* 5: 180381 (2018), 2쪽. 원문: "We make an essential distinction that some cryptos are 'coins' (which operate on their own independent network) and others 'tokens' (which operate on top of a coin network as a platform)." 이 논문의 분석 대상은 시가총액의 분포이므로 이 책은 구별의 정의만 가져가고 분포에 관한 결과는 옮기지 않는다(3장 일러두기).

2. Schilling & Uhlig (2019), 22쪽, Theorem 2. 19장 각주 7.

3. Carlsten 외 (2016), §1·§7. 20장 각주 23.

4. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform" (2014; 초고 2013), 「Currency And Issuance」 절(PDF 30쪽). 원문: "The Ethereum network includes its own built-in currency, ether, which serves the dual purpose of providing a primary liquidity layer to allow for efficient exchange between various types of digital assets and, more importantly, of providing a mechanism for paying transaction fees." 14장 각주 5 과 같은 자리. 같은 절이 적는 2014년의 발행 모형(판매 가격·초기 배분)은 그 뒤 포크들로 바뀌었으므로 이 책은 옮기지 않는다.

5. EIP-1559, Security Considerations 절 「ETH Burn Precludes Fixed Supply」. 21장 각주 19.

6. van Oordt (2024), 초록. 19장 각주 29. 원문: "Cryptocurrencies with negative issuance (e.g., that burn transaction fees) may generate positive aggregate cash flows to investors even if their price path follows a bubble trajectory."

7. Nakamoto (2008), 4쪽(§6). 20장 각주 1.

8. SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017-07-25), 2–3쪽·6쪽. 원문: "From April 30, 2016 through May 28, 2016, The DAO offered and sold approximately 1.15 billion DAO Tokens in exchange for a total of approximately 12 million Ether ('ETH')"; "The token price fluctuated in a range of approximately 1 to 1.5 ETH per 100 DAO Tokens, depending on when the tokens were purchased during the Offering Period. Anyone was eligible to purchase DAO Tokens (as long as they paid ETH). There were no limitations placed on the number of DAO Tokens offered for sale, the number of purchasers of DAO Tokens, or the level of sophistication of such purchasers." 보고서 3쪽은 모금 종료 시점의 달러 환산액도 적지만 이 책은 옮기지 않는다.

9. Makarov & Schoar (2021), 5쪽(서론). 원문: "Determining the concentration of ownership is more complicated than just tracking the holdings of the richest addresses, since many of the largest addresses belong to cold wallets of exchanges and online wallets, which hold Bitcoin on behalf of many investors. We develop a suite of algorithms based on graph analysis to classify addresses into those belonging to individual investors or those belonging to intermediaries."

10. Makarov & Schoar (2021), 4쪽·6쪽·26–29쪽(§5). 원문: "There were 2258 such addresses, which controlled 7.9 million bitcoins — almost half of all bitcoins in circulation. … The fact that so few addresses control almost half of the bitcoins in circulation is often taken as prima facie evidence of the high concentration of Bitcoin holdings. This view, however, neglects the fact that some of these addresses belong to cold wallets and therefore, represent holdings of a large number of people."(27쪽); "Overall, out of the total 2258 rich addresses, we classify 1013 as individual, 1154 as linked to intermediaries, and 47 as ambiguous."(28쪽); "By the end of 2020 is was equal to 5.5 million bitcoins, roughly one-third of Bitcoin in circulation at the time." (28쪽); "There are 400,000 of such clusters and they collectively control 8.5 million bitcoins by the end of 2020."(28쪽); "We see that only 1000 clusters control three million bitcoins and the top 10,000 own more than five million bitcoins which is about a quarter of all outstanding bitcoins. It is also important to note that this measurement of concentration most likely is an understatement since we cannot rule out that some of the largest addresses are controlled by the same entity. In particular, in the above calculations, we do not assign the ownership of early bitcoins, which are held in about 20,000 addresses, to one person (Satoshi Nakamoto) but consider them as belonging to 20,000 different individuals."(29쪽); 채굴: "The top 10% of miners control 90% and just 0.1% (about 50 miners) control close to 50% of mining capacity. Furthermore, this concentration of mining capacity is counter cyclical and varies with the Bitcoin price."(4쪽). 애시 외(2016)도 2015년 중반 기준으로 「비트코인 소유가 고도로 집중되어 있다」고 적는다(3쪽; 19장 각주 9).

11. Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer & Dan Robinson, "Uniswap v3 Core," March 2021, §4 Governance(3쪽). 원문: "The factory has an owner, which is initially controlled by UNI tokenholders. The owner does not have the ability to halt the operation of any of the core contracts. As in Uniswap v2, Uniswap v3 has a protocol fee that can be turned on by UNI governance. In Uniswap v3, UNI governance has more flexibility in choosing the fraction of swap fees that go to the protocol, and is able to choose any fraction 1/N where $4 \le N \le 10$, or 0. This parameter can be set on a per-pool basis. UNI governance also has the ability to add additional fee tiers. … Once a fee tier is added to the factory, it cannot be removed (and the tickSpacing cannot be changed). … Finally, UNI governance has the power to transfer ownership to another address." 15장과 같은 원전.

12. Buterin (2014), 「Decentralized Autonomous Organizations」 절(PDF 23쪽). 원문: "The general concept of a 'decentralized organization' is that of a virtual entity that has a certain set of members or shareholders which, perhaps with a 67% majority, have the right to spend the entity's funds and modify its code. The members would collectively decide on how the organization should allocate its funds. Methods for allocating a DAO's funds could range from bounties, salaries to even more exotic mechanisms such as an internal currency to reward work. This essentially replicates the legal trappings of a traditional company or nonprofit but using only cryptographic blockchain technology for enforcement."

13. SEC (2017), 7쪽과 각주 24. 원문: "ETH raised by The DAO was to be distributed to a Contractor to fund a proposal only on a majority vote of DAO Token holders. DAO Token holders were to cast votes, which would be weighted by the number of tokens they controlled, for or against the funding of a specific proposal. The voting process, however, was publicly criticized in that it could incentivize distorted voting behavior and, as a result, would not accurately reflect the consensus of the majority of DAO Token holders. Specifically, as noted in a May 27, 2016 blog post by a group of computer security researchers, The DAO's structure included a 'strong positive bias to vote YES on proposals and to suppress NO votes as a side effect of the way in which it restricts users' range of options following the casting of a vote.'"; 각주 24: "By voting on a proposal, DAO Token holders would 'tie up' their tokens until the end of the voting cycle. … If, however, a DAO Token holder

abstained from voting, the DAO Token holder could avoid these restrictions; any DAO Tokens not submitted for a vote could be withdrawn or transferred at any time. As a result, DAO Token holders were incentivized either to vote yes or to abstain from voting."

14. SEC (2017), 7–8쪽. 원문: "Before any proposal was put to a vote by DAO Token holders, it was required to be reviewed by one or more of The DAO's 'Curators.' At the time of the formation of The DAO, the Curators were a group of individuals chosen by Slock.it. According to the White Paper, the Curators of a DAO Entity had 'considerable power.' The Curators performed crucial security functions and maintained ultimate control over which proposals could be submitted to, voted on, and funded by The DAO."; "One member of the group chosen by Slock.it to serve collectively as the Curator stated publicly that the Curator had 'complete control over the whitelist … the order in which things get whitelisted, the duration for which [proposals] get whitelisted, when things get unwhitelisted … [and] clear ability to control the order and frequency of proposals,' noting that 'curators have tremendous power.' Another Curator publicly announced his subjective criteria for determining whether to whitelist a proposal, which included his personal ethics. Per the White Paper, a Curator also had the power to reduce the voting quorum requirement by 50% every other week."

15. SEC (2017), 6쪽과 각주 21. 원문: "DAO Token holders would also be permitted to redeem their DAO Tokens for ETH through a complicated, multi-week (approximately 46-day) process referred to as a DAO Entity 'split.'"; 각주 21: "According to the White Paper, the primary purpose of a split is to protect minority shareholders and prevent what is commonly referred to as a '51% Attack,' whereby an attacker holding 51% of a DAO Entity's Tokens could create a proposal to send all of the DAO Entity's funds to himself or herself." 결함이 분리 함수의 재귀 호출에 있었다는 기술적 서술은 SEC 보고서에 없고(1쪽은 "a flaw in The DAO's code"), 이더리움 재단 공지(2016-06-17)를 근거로 26장이 다룬다.

16. SEC (2017), 4–5쪽. 원문: "In November 2015, at an Ethereum Developer Conference in London, Christoph Jentzsch described his proposal for The DAO as a 'for-profit DAO [Entity],' where participants would send ETH (a virtual currency) to The DAO to purchase DAO Tokens, which would permit the participant to vote and entitle the participant to 'rewards.' Christoph Jentzsch likened this to 'buying shares in a company and getting … dividends.'"; "The DAO Website included a description of The DAO's intended purpose: 'To blaze a new path in business for the betterment of its members, existing simultaneously nowhere and everywhere and operating solely with the steadfast iron will of unstoppable code.'" SEC는 이 자료들을 근거의 하나로 DAO 토큰을 증권으로 판단했다(1쪽) — 그 법적 판단은 27장·28장.

3부 경제학

23장 게임이론 — 이기적 채굴과 균형

핵심 TAKEAWAY

블록체인의 규칙은 「따를 수밖에 없는」 것이 아니라 「**따르는 것이 이익이 되도록 설계된**」 것이고, 그 설계는 조건부로만 성립합니다. 백서는 공격자가 「규칙을 따르는 편이 더 이익임을 알게 될 것」이라고 적었고, 오랫동안 「정직한 채굴이 내시 균형」이라는 통념이 있었습니다. 이얄(Ittay Eyal)과 시러(Emin Gün Sirer)는 그 통념이 틀렸음을 보였습니다. 찾은 블록을 바로 공개하지 않고 숨기는 **이기적 채굴** 전략은 점유율 α 가 어떤 문턱을 넘으면 정직한 채굴보다 많은 몫을 얻고, 그 문턱은 네트워크 전파를 얼마나 장악하는가(γ)에 따라 1/3에서 0까지 내려갑니다. 그래서 「과반이 정직하면 안전하다」가 아니라 「3분의 2를 넘게 정직해야」 하고, 그조차 가장 낙관적인 가정에서입니다. 17장의 우선순위 가스 경매에서는 봇들이 협조적 상태로 수렴해 갔고(다이안 외의 모형이 근사하는), 20장의 수수료만의 체제에서는 칼스텐 외의 모형 안에서 언더컷이 균형이 됩니다. 이 장을 읽으면 「탈중앙 체계는 참여자의 선의 없이 돌아간다」는 말이 정확히 어떤 가정 위에 서 있는지, 그리고 하드포크 뒤 두 체인 중 어느 쪽이 「진짜」인지를 묻는 질문이 왜 게임이론의 질문인지 압니다.

학습목표

1. 백서 §6의 「정직이 이익」 논증을 2×2 이득표로 쓰고, 그것이 내시 균형 주장임을 안다. 부디시의 일회 게임이 「정직 균형의 필요조건」을 어떻게 적는지 읽는다.
2. [중급] 이기적 채굴 전략의 상태기계(선두 0, 0′, 1, 2, …)를 그리고, 몬테카를로로 풀의 몫을 재어 정직한 몫 α 와 비교한다.
3. [심화] 이얄–시러의 닫힌 식 R_{pool} 과 임계 점유율 $\frac{1-\gamma}{3-2\gamma} < \alpha < \frac{1}{2}$ 를 읽고, 시뮬레이션이 그 식을 재현하는지 확인한다. γ 가 왜 네트워크 전파의 변수인지, 저자들의 수정안이 왜 문턱을 1/4로 올리는지 안다.
4. 조정 게임의 틀로 포크 선택을 보고, 17장의 협조적 균형·20장의 언더컷 균형이 같은 틀의 다른 자리임을 안다.

[입문]만 읽은 독자가 가져가는 것. 블록체인의 규칙은 「따를 수밖에 없는」 것이 아니라 「따르는 것이 이익이 되도록 설계된」 것이고, 그 설계는 조건부로만 성립합니다.

1. 이 장이 답하려는 질문

7장과 8장에서 블록체인의 「불변성」은 수학이 아니라 비용이었습니다. 그 비용을 누가 왜 내는가는 20장의 질문이었고, 답은 블록 보상이었습니다. 그런데 보상을 받는 쪽은 사람입니다. 사람은 규칙이 시키는 대로가 아니라 **이익이 되는 대로** 움직입니다. 그러면 질문은 이렇습니다. 프로토콜이 적어 둔 행동(블록을 찾으면 바로 공개하고, 가장 긴 체인 위에 쌓고, 거래를 전부 담는다)이 정말 이익이 되는 행동인가. 이익이 된다면 어떤 조건에서인가. 되지 않는다면 무슨 일이 생기는가. 이것은 게임이론의 질문이고, 답의 모양은 **균형입니다**. 어떤 행동 조합이 균형이라는 것은 거기서 혼자 이탈해서 득 볼 사람이 없다는 뜻입니다. 그 이상도 이하도 아닙니다.

2. 백서의 논증을 이득표로

20장에서 읽은 백서 §6의 마지막 문장은 사실 게임이론의 주장입니다. 공격자에게는 두 전략이 있고(규칙을 따른다 / 체계를 무너뜨린다), 백서는 전자의 이득이 더 크다고 말합니다. 「그는 규칙을 따르는 편 — 다른 모든 사람을 합친 것보다 많은 새 코인을 그에게 주는 규칙 — 이 체계와 자기 재산의 유효성을 무너뜨리는 것보다 더 이익임을 알게 될 것이다.」[1] 가장 단순한 틀로 적으면 두 참여자(「나」와 「나머지 전부」), 두 전략(정직 / 공격)의 2×2 게임입니다.

나 ＼ 나머지	정직	공격
정직	내 점유율만큼의 보상, 체계 유지	나머지가 다수면 체계가 그들 것
공격	내가 다수면 V_{attack} − 비용, 소수면 비용만	체계 붕괴

이 표에서 「정직·정직」이 균형이려면 「내가 다수여도 공격의 순이익이 정직의 보상보다 작아야」 합니다. 그것이 20장 식 (3)의 내용이고, 부디시는 §III.E에서 같은 것을 **일회 게임의 내시 균형으로** 다시 적습니다. I 명이 각자 신뢰 지원량 x_i 와 자세(정직/공격)를 고르고, 누군가 $x_i > N/2$ 이면서 공격하면 그가 V_{attack} 을 갖고 나머지는 0, 아니면 각자 $\frac{x_i}{N}p$ 를 받습니다. 보조정리 1은 정직 균형이 있으면 $N^*c \le p$ (영이윤의 짝), 보조정리 2는 어떤 참여자가 0을 골랐다면 $N^*c \ge V_{attack}$(유인양립의 짝), 명제 2는 「정직 균형의 필요조건은 $p \ge V_{attack}/(1+1/I)$ 이고 $I \to \infty$ 에서 $p \ge V_{attack}$」입니다.[2] 백서의 「알게 될 것이다」가 성립하는 조건이 바로 이 부등식입니다. 그리고 부디시가 덧붙이듯 이 단순한 모형은 「정직한 행동이냐 다수결 공격이냐」만 봅니다. 「블록체인에 대한 많은 선행 게임이론 분석은 모든 참여자가 작다고 가정해 다수결 공격의 가능성을 없애는 대신 … 프로토콜을 정직하게 따르는 것 말고 더 풍부한 전략 집합을 허용한다 — 예컨대 Eyal and Sirer (2014).」[3] 이 장의 나머지가 그 풍부한 전략 집합입니다. 참여자가 **소수여도** 이탈할 수 있는가.

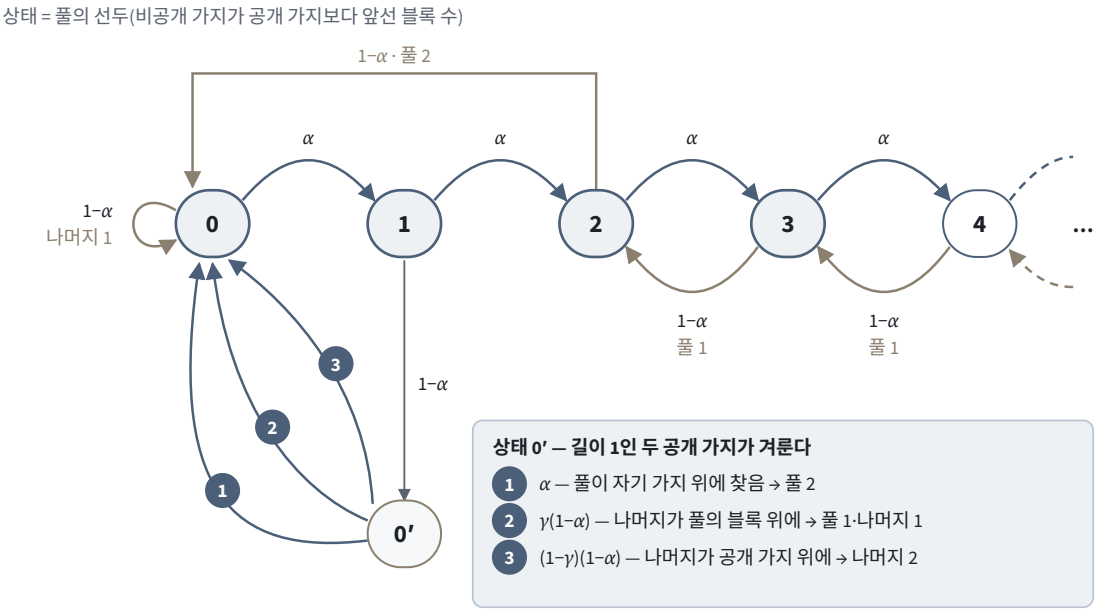
3. 통념 — 「정직한 채굴은 내시 균형이다」

이얄과 시러가 2013년에 공격한 것은 바로 그 통념이었습니다. 2018년 CACM 게재본의 서두입니다. 「통념은 채굴 프로토콜이 유인양립적이고 결탁하는 소수 집단에 대해 안전하다고, 곧 채굴자가 규정된 대로 프로토콜을 따르도록 유인한다고 주장한다. 우리는 비트코인 채굴 프로토콜이 유인양립적이지 **않음을** 보인다. 결탁하는 채굴자의 수입이 공정한 몫보다 커지는 공격을 제시한다.」[4] 통념의 근거는 단순했습니다. 「프로토콜이 채굴자에게 채굴 파워 비율에 비례해 보상한다고 믿었으므로, 큰 풀의 채굴자는 작은 풀에서와 같은 수입을 얻는다고 믿었다. 따라서 풀 운영의 고정비와 규모의 경제를 무시하면, 결탁하는 채굴자들이 점점 커지는 풀로 조직될 이점이 없다.」 관련 연구 절은 그 통념을 직접 인용합니다. 펠턴(Edward Felten)의 「모두가 유인대로 행동하면 결국 모두가 쓰인 대로 비트코인의 규칙을 따르게 된다는 뜻에서 비트코인 체계가 안정적이라는 민간 정리(folk theorem)가 있었다」와, 「해시파워의 X%를 가진 채굴자 연합은 총 채굴 수입의 X%를 넘게 얻을 수 없다는 … 증명된 적 없고 직관적 믿음으로 받아들여진 가정」입니다. 크롤(Kroll)·데이비(Davey)·펠턴의 2013년 분석은 「블록 보류 공격을 고려하지 않은 채 … 정직한 전략이 내시 균형을 이룬다고 논증」했습니다.[5]

4. 이기적 채굴 — 블록을 숨기는 전략

전략의 착상은 한 문단입니다. 「이기적 채굴이라 부르는 이 전략의 핵심 착상은 풀이 발견한 블록을 비공개로 두어 **일부러 체인을 포크하는** 것이다. 정직한 노드들은 공개 체인에서 계속 채굴하고, 풀은 자기 비공개 가지에서 채굴한다. 풀이 더 많은 블록을 찾으면 공개 체인보다 더 길게 앞서 가고 새 블록들을 계속 비공개로 둔다. 공개 가지가 풀의 비공개 가지에 길이로 다가오면, 이기적 채굴자들은 비공개 체인의 블록을 공개한다. 이 전략은 비트코인 프로토콜을 따르는 정직한 채굴자들이 결국 아무 쓸모 없는 암호퍼즐에 자원을 낭비하게 만든다. 우리의 분석은 정직한 쪽과 이기적인 쪽 모두 자원을 낭비하지만 **정직한 채굴자가 비례적으로 더 많이 낭비하며**, 이기적 풀의 보상이 네트워크 채굴 파워 몫을 넘는다는 것을 보여 준다.」[6] 8장의 「가장 긴 체인」 규칙이 여기서 무기가 됩니다. 규칙은 「가장 긴 것을 따르라」고만 했지 「찾는 즉시 공개하라」를 강제할 수 없기 때문입니다. 블록을 언제 공개할지는 찾은 사람만 압니다.

[중급] 전략은 풀의 **선두**(비공개 가지가 공개 가지보다 몇 블록 앞서는가)에 따라 정해집니다. 모형은 이렇습니다. 풀의 채굴 파워 비율이 α, 나머지가 $1-\alpha$, 블록 발견은 지수분포 간격입니다.[7] 상태 $s = 0, 1, 2, \ldots$ 에서 풀이 블록을 찾으면(빈도 α) 선두가 하나 늘고, $s \geq 3$ 에서 나머지가 찾으면(빈도 $1-\alpha$) 하나 줄어듭니다. 선두가 2일 때 나머지가 찾으면 풀은 비공개 가지를 전부 공개해 선두 0으로 돌아가며 그 두 블록을 모두 얻습니다. 선두가 1일 때 나머지가 찾으면 **상태 0′** — 길이 1인 두 공개 가지가 겨루는 상태 — 가 되고, 여기서 세 갈래가 모두 상태 0으로 갑니다. 「(1) 풀이 자기의 옛 비공개 가지 위에 블록을 찾는다(빈도 α), (2) 나머지가 옛 비공개 가지 위에 찾는다(빈도 $\gamma(1-\alpha)$), (3) 나머지가 공개 가지 위에 찾는다(빈도 $(1-\gamma)(1-\alpha)$).」 여기서 γ 는 「풀의 블록 위에서 채굴하기로 한 정직한 채굴자의 비율」입니다.[8] γ 는 전략 변수가 아니라 **네트워크 변수입니다**. 정직한 채굴자는 「먼저 들은 가지」를 잇는데, 풀이 자기 블록을 네트워크에 얼마나 빨리 퍼뜨릴 수 있느냐가 γ 를 정합니다.



화살표 옆 위 줄은 빈도, 아래 줄은 그 전이로 풀·나머지가 얻는 블록 수(최종 체인에 남는 것만).
α = 풀의 채굴 파워 비율, γ = 풀의 블록 위에서 채굴하기로 한 정직한 채굴자의 비율.
풀이 찾을 때(α)는 선두만 늘고 보상은 나중에 정해진다.

그림 23-1. 이기적 채굴의 상태기계. 상태는 풀의 비공개 선두이고, 화살표 옆에 빈도와 그 전이로 풀·나머지가 얻는 블록 수를 적었다. Eyal & Sirer (2018) 그림 1을 바탕으로 다시 그림.

[중급] 보상은 **최종 체인에 남는** 블록에만 붙습니다. 저자들은 전이마다 누가 몇 블록을 얻는지 (a)~(h)로 적습니다. 요지만 옮기면, 상태 0′에서 풀이 찾으면 풀이 2(자기 두 블록 모두 확정), 나머지가 풀의 블록 위에 찾으면 풀 1·나머지 1, 나머지가 자기 블록 위에 찾으면 나머지 2. 선두 2에서 나머지가 찾으면 풀이 2. 선두가 2보다 클 때 나머지가 찾으면 「나머지의 새 블록은 풀이 가지를 전부 공개하면 체인 밖에 남으므로 나머지는 아무것도 얻지 못하고, 풀은 i 번째 블록을 공개해 1을 얻는다.」[9] 그리고 중요한 한 문장. 「예상대로, 이기적 채굴이 가져오는 고의적 분기는 정직한 채굴자들이 체인 밖으로 떨어질 블록에서 일하게 만든다. 이는 다시 총 블록 생성률의 하락으로 이어져 $r_{pool} + r_{others} < 1$ 이 된다. 프로토콜은 주 체인의 채굴률이 평균 10분당 한 블록이 되도록 난이도를 조정할 것이다. 따라서 각 행위자의 실제 수입률은 **수입률의 비율**, 곧 주 체인 블록 가운데 자기 블록의 비율이다.」[10] 풀의 몫 $R_{pool} = r_{pool} / (r_{pool} + r_{others})$ 가 이 장의 기준량입니다. 정직하게 채굴하면 이 값은 α 입니다. 아래 셀이 상태기계를 그대로 돌려 R_{pool} 을 재고, 닫힌 식과 견줍니다.

```
# NB23-1: 이기적 채굴 몬테카를로 — 이얄-시러의 상태기계(선두 0, 0′, 1, 2, …)를 사건 단위로 돌려 풀의 몫 R_pool
을 잰다 (공부용)
import random

def selfish_mining_share(alpha: float, gamma: float, n_events: int, seed: int = 23) -> float:
    """블록 발견 사건을 n_events 번 뽑는다(풀이 찾을 확률 α). 돌려주는 값: 주 체인에 남은 블록 중 풀의 비율."""
    rng = random.Random(seed)
    lead = 0                 # 풀의 비공개 선두(블록 수)
    tie = False              # 상태 0′: 길이 1인 두 공개 가지가 겨루는 중
    pool_blocks = others_blocks = 0
    for _ in range(n_events):
        pool_found = rng.random() < alpha
        if tie:                                  # 상태 0′ — 세 갈래 모두 상태 0 으로
            if pool_found:                       # (b) 풀이 자기 가지 위에 찾음 → 풀 2
                pool_blocks += 2
            elif rng.random() < gamma:           # (c) 나머지가 풀의 가지 위에 찾음 → 풀 1, 나머지 1
                pool_blocks += 1; others_blocks += 1
            else:                                # (d) 나머지가 자기 가지 위에 찾음 → 나머지 2
                others_blocks += 2
            tie = False; lead = 0
            continue
        if pool_found:                           # (a) 풀이 찾음 → 선두 +1 (보상은 나중에)
            lead += 1
            continue
        # 나머지가 찾음
        if lead == 0:                            # (e) 비공개 가지 없음 → 나머지 1
            others_blocks += 1
        elif lead == 1:                          # (f) 선두 1 → 풀이 숨긴 블록을 공개, 겨루기 시작
            tie = True; lead = 0
        elif lead == 2:                          # (g) 선두 2 → 풀이 두 블록 공개, 나머지 블록은 버려짐
→ 풀 2
            pool_blocks += 2; lead = 0
        else:                                    # (h) 선두 > 2 → 풀이 한 블록 공개(풀 1), 나머지
블록은 결국 버려짐
            pool_blocks += 1; lead -= 1
    return pool_blocks / (pool_blocks + others_blocks)

def R_pool_closed_form(a: float, g: float) -> float:
    """CACM판 식 (3) = arXiv v5 식 (8): R_pool = [α(1-α)²(4α+γ(1-2α)) - α³] / [1 - α(1+(2-α)α)],
0 ≤ α ≤ 1/2"""
    return (a * (1 - a) ** 2 * (4 * a + g * (1 - 2 * a)) - a ** 3) / (1 - a * (1 + (2 - a) * a))

N = 400_000
print(f"사건 {N:,}번. 정직한 채굴의 몫은 α 그 자체. 아래는 이기적 채굴의 몫(시뮬레이션 / 닫힌 식).")
print(f"{'α':>5} {'γ':>5} {'시뮬 R_pool':>12} {'식 R_pool':>10} {'정직 α':>8} {'이기적이
유리?':>12}")
for g in [0.0, 0.5, 1.0]:
    for a in [0.10, 0.20, 0.25, 0.30, 0.34, 0.40, 0.45]:
        sim = selfish_mining_share(a, g, N)
        cf = R_pool_closed_form(a, g)
        print(f"{a:>5.2f} {g:>5.1f} {sim:>12.4f} {cf:>10.4f} {a:>8.2f} {'예' if cf > a else
'아니오':>12}")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
사건 400,000번. 정직한 채굴의 몫은 α 그 자체. 아래는 이기적 채굴의 몫(시뮬레이션 / 닫힌 식).
    α     γ    시뮬 R_pool   식 R_pool     정직 α     이기적이 유리?
 0.10   0.0       0.0358     0.0356     0.10          아니오
 0.20   0.0       0.1300     0.1297     0.20          아니오
 0.25   0.0       0.1948     0.1951     0.25          아니오
 0.30   0.0       0.2713     0.2731     0.30          아니오
 0.34   0.0       0.3446     0.3463     0.34            예
 0.40   0.0       0.4813     0.4837     0.40            예
 0.45   0.0       0.6488     0.6518     0.45            예
 0.10   0.5       0.0724     0.0724     0.10          아니오
 0.20   0.5       0.1830     0.1824     0.20          아니오
 0.25   0.5       0.2501     0.2500     0.25          아니오
 0.30   0.5       0.3254     0.3269     0.30            예
 0.34   0.5       0.3953     0.3970     0.34            예
 0.40   0.5       0.5235     0.5256     0.40            예
 0.45   0.5       0.6778     0.6806     0.45            예
 0.10   1.0       0.1097     0.1092     0.10            예
 0.20   1.0       0.2363     0.2352     0.20            예
 0.25   1.0       0.3048     0.3049     0.25            예
 0.30   1.0       0.3792     0.3806     0.30            예
 0.34   1.0       0.4456     0.4476     0.34            예
 0.40   1.0       0.5652     0.5674     0.40            예
 0.45   1.0       0.7065     0.7094     0.45            예
```

[중급] 표를 읽습니다. $\gamma = 0$(정직한 채굴자가 겨루기에서 언제나 자기 쪽 블록을 먼저 듣는 경우)이면 $\alpha = 0.30$ 에서 이기적 채굴이 손해이고 0.34 부터 이익입니다. $\gamma = 0.5$ 면 0.25 근처가 경계이고, $\gamma = 1$ 이면 **아무리 작은 풀도** 이익입니다. 시뮬레이션과 닫힌 식은 몬테카를로 오차(사건 40만 번에서 약 ±0.005) 안에서 맞습니다. 그 닫힌 식과 경계가 다음 절입니다.

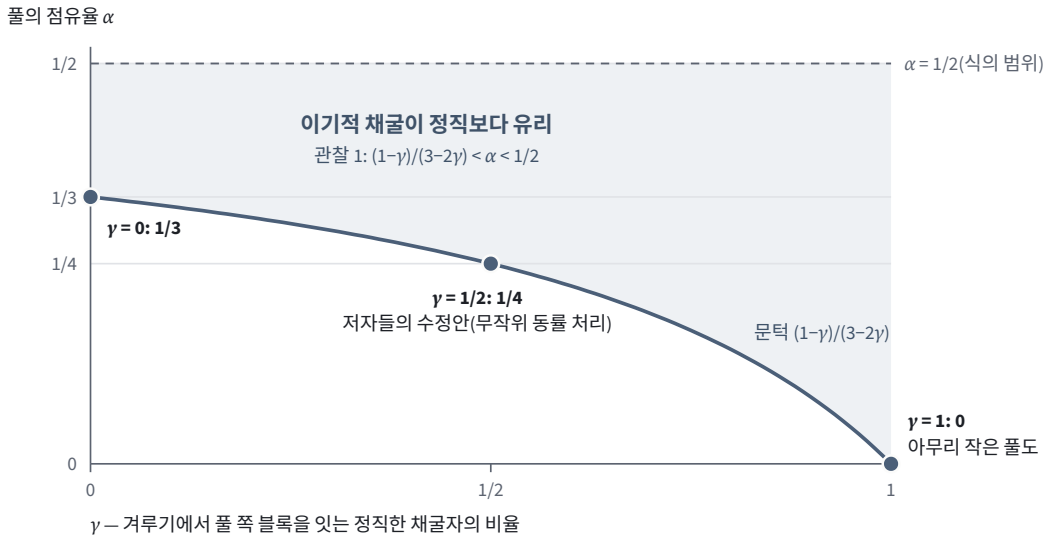


그림 23-2. 이기적 채굴이 정직보다 유리한 점유율의 범위(관찰 1). 곡선은 본문 식 $\frac{1-\gamma}{3-2\gamma}$ 의 그래프이고, 세 점은 본문의 1/3·1/4·0이다.

▾ [심화] 닫힌 식, 임계 점유율, 그리고 왜 1/3·1/4·0인가

[심화] 상태기계의 정상 확률을 풀면(CACM판은 「상세는 전체 보고서에」라고 적고, arXiv v5가 식 (1)(방정식계)과 (2)~(5)(해)로 적습니다) 상태 0, $0'$, 1, $k \geq 2$ 의 확률이 α 의 유리함수로 나오고, 그것을 전이별 보상에 곱해 더하면 풀의 몫이 됩니다. CACM판의 식 (3), arXiv v5의 식 (8)입니다.

$$R_{pool} = \frac{\alpha(1-\alpha)^2(4\alpha + \gamma(1-2\alpha)) - \alpha^3}{1-\alpha(1+(2-\alpha)\alpha)}, \qquad 0 \le \alpha \le \frac{1}{2}$$

기호 풀이: α 는 풀의 채굴 파워 비율, γ 는 겨루기에서 풀 쪽 블록을 잇는 정직한 채굴자의 비율. 수치 예: $\alpha = 0.4, \gamma = 0$ 이면 $R_{pool} \approx 0.484$ 로 점유율 0.4보다 큽니다. 이 식이 α 보다 커지는 범위를 풀면 **관찰 1이** 나옵니다. 「주어진 γ 에 대해 크기 α 의 풀은 다음 범위에서 자기 상대적 크기보다 큰 수입을 얻는다: $\frac{1-\gamma}{3-2\gamma} < \alpha < \frac{1}{2}$.」[11] $\gamma = 0$ 이면 하한이 1 / 3, $\gamma = 1$ / 2 면 1 / 4, $\gamma = 1$ 이면 0 입니다. 저자들의 설명이 왜 그런지를 말해 줍니다. 「풀이 위험에 처하는 것은 비밀 블록을 정확히 하나 들고 있을 때, 정직한 채굴자가 그것과 겨룰 블록을 공개할 수 있을 때뿐이다. $\gamma = 1$ 이면 풀은 나머지가 자기 가지를 찾아도 자기의 한 블록 가지를 빨리 퍼뜨릴 수 있어 모든 정직한 채굴자가 여전히 풀의 블록 위에서 채굴한다. 이 경우 풀은 이기적 채굴을 따를 때 아무 위험도 지지 않고 수입은 언제나 정직한 알고리듬을 따를 때보다 낫다. 그래서 문턱은 0이고, 어떤 크기의 풀도 이기적 채굴로 이익을 볼 수 있다. 반대 극단 $\gamma = 0$ 에서는 정직한 채굴자가 언제나 자기 블록을 먼저 공개·전파하고 문턱은 1/3이다.」[12] 그리고 **관찰 2**: 「이기적 채굴을 하는 풀에서, 문턱보다 큰 풀의 각 구성원의 수입은 풀 크기와 함께 증가한다.」 수입이 점유율에 대해 기울기 1보다 가파르게 올라가기 때문이고, 그래서 「합리적 채굴자는 수입을 늘리려 이기적 풀에 우선적으로 가담하고 … 풀은 어떤 장치의 저지도 받지 않고 과반을 향해 커진다.」[13] 통념의 「X%는 X%」가 깨지는 순간 풀의 크기에 상한을 두는 힘이 사라진다는 것입니다.

[심화] γ 를 풀이 **올릴 수 있다는** 것이 공격을 현실적으로 만듭니다. 「영리한 풀 운영자는 채굴 파워가 0인 채굴자를 많이 비트코인 채굴자 네트워크에 더해 정직한 채굴자에 대한 시빌 공격을 할 수 있다. 이 가상 채굴자들은 자료 전파에 참여해 전방 센서 노릇을 하되 새 블록을 채굴하지는 않는다. … 블록 X를 들으면 무시하고 블록 P를 전파하기 시작한다. … 충분한 가상 노드를 더하면 풀 운영자는 γ 를 올릴 수 있다. 그 결과(관찰 1 참조)는 0에 가까운 문턱이다.」 저자들의 수정안은 그래서 정직한 쪽의 **동률 처리 규칙을** 바꾸는 것입니다. 「채굴자가 같은 길이의 경쟁 가지들을 알게 되면 전부 전파하고 어느 것 위에서 채굴할지 균등하게 무작위로 고른다. … 이는 $\gamma = 1$ / 2 를 낳고, 다시 1/4의 문턱을 낳는다.」 「다른 채굴자의 채택과 무관하게」 동작하므로 「하드포크가 필요 없고」(하위 호환·점진적 도입은 같은 쪽의 다른 문장), 「현재 두 가지의 길이가 같을 때 각 채굴자의 선택은 사실상 네트워크 위상과 지연으로 정해지는 임의의 것이므로 … 새 취약점을 들이지 않는다」는 것입니다.[14] 다만 저자들은 그 수정안으로도 「체계는 여전히 취약하다. 이미 25% 문턱을 넘는 풀이 있고, 때로는 33%의 이론적 한계마저 넘는다」고 적습니다.[15] 그리고 결론은 통념의 수치를 바꿉니다. 「이기적 채굴을 막으려면 네트워크의 적어도 2/3가 정직해야 한다. 단순 과반으로는 충분하지 않다.」[16]

이 공격의 두 가지 성질을 적어 둡니다. 첫째, **발각이 어렵습니다**. 「이기적 채굴 행동을 숨기기는 쉽고 금지하기는 어렵다. 이기적 풀은 서로 다른 비트코인 주소와 IP 주소를 쓰고 블록 생성 시각을 위조해 자기 크기를 드러내지 않을 수 있다.」 징후는 둘 — 버려진 블록의 증가와 블록 간격 분포의 왜곡 — 이지만 「어느 쪽도 확정적으로 재기 어렵다.」[17] 둘째, 「붕괴」가 가치 0을 뜻하지 않습니다. 저자들이 「붕괴」라 부르는 것은 「단일 주체가 통화 체계 전체를 통제하는 상태」이고, 「그런 붕괴가 곧바로 비트코인의 가치가 0으로 떨어진다는 뜻은 아니다. 통제 주체는 수수료만을 위해서라도 대부분의 거래를 받아들일 유인이 있고 … 은밀히 남아 자기가 통화 전체를 통제할 수 있다는 사실을 숨기는 쪽을 택할 수도 있다.」[18] 탈중앙의 상실은 바깥에서 보이지 않을 수 있다는 것입니다. 5부의 「탈중앙이면 책임자가 없다」는 명제

(부록 B 오해 ⑫)가 여기서 한 번 더 흔들립니다. 책임자가 **없는** 것과 **보이지 않는** 것은 다릅니다.

5. 같은 틀의 다른 자리 — 조정 게임, 협조적 균형, 언더컷

이기적 채굴은 「채굴자가 블록을 언제 공개할지 고를 수 있다」는 자유에서 나왔습니다. 채굴자의 자유는 그것만이 아닙니다. 20장에서 본 칼스텐 외의 전략 언어가 셋을 듭니다. 「어느 블록을 이을 것인가, 가용 거래 가운데 얼마를 담을 것인가, 찾은 블록을 공개할 것인가.」[19] 세 자유 각각에 균형의 질문이 붙습니다.

[중급] 어느 블록을 이을 것인가 — 조정 게임. 두 가지가 같은 길이로 겨룰 때 채굴자에게 중요한 것은 「어느 쪽이 옳은가」가 아니라 「다른 사람들이 어느 쪽을 이을 것인가」입니다. 다수가 잇는 쪽이 이기고, 진 쪽에 쌓은 블록은 버려집니다. 이것은 교과서의 조정 게임이고, 균형이 둘(모두 A / 모두 B)인 게임입니다. 프로토콜이 「먼저 들은 쪽」이라는 **초점(focal point)을** 주는 이유가 그것이고, 이얄–시러의 γ 와 수정안(무작위 동률 처리)은 그 초점을 공격자가 조작하지 못하게 하려는 것이었습니다. 하드포크는 이 조정 게임의 큰 판입니다. 규칙이 바뀐 체인과 옛 체인이 갈릴 때 어느 쪽이 「비트코인」 또는 「이더리움」인지를 정하는 것은 코드가 아니라 **참여자들이 어디로 조정하는가입니다.** 14장에서 본 SEC의 The DAO 보고서가 적듯, 2016년 7월 20일의 하드포크 뒤 「소수 집단은 블록체인은 불변이라는 개념에 반한다는 이유로 하드포크로 만들어진 새 이더리움 블록체인을 채택하지 않기로 하고 옛 판본을 계속 썼는데, 그것이 지금 이더리움 클래식으로 알려져 있다.」[20] 두 체인은 모두 살아남았고, 어느 쪽이 「진짜」인지는 조정의 결과이지 증명의 결과가 아닙니다. 26장이 이 사건을 다룹니다.

[중급] 얼마를 담을 것인가 — 언더컷 균형. 20장 6절에서 읽었습니다. 보상이 수수료뿐이면 「남은 수수료가 많은 블록을 잇는」 전략이 엄밀히 낮고, 그 위에서 「일부러 포크해 수수료를 덜 가져가는」 언더컷이 균형이 되며, 그 균형에서는 처리되지 않은 거래가 끝없이 쌓입니다(정리 5.1). 칼스텐 외의 결론을 이 장의 언어로 옮기면, **보상이 일정할 때 균형이었던 전략이 보상이 변동하면 균형이 아니게 된다는** 것입니다. 「비원자적 채굴자라면 블록 보상 모형에서 기본 순응 전략은 자명하게 균형이고 이 결과는 일반적인 지연 모형에서도 견고하다. … 그런데 거래 수수료 모형에서는 채굴자가 비원자적이어도 균형 행동이 복잡하고 바람직하지 않다.」[21]

[중급] 순서를 두고 경쟁하는 봇들 — 협조적 균형. 17장이 넘긴 것입니다. 다이안과 공저자들은 우선순위 가스 경매의 봇들이 서로 입찰을 올리는 경쟁에서 **번갈아 최소 폭으로 올리는** 협조 상태로 수렴하는 것을 관찰하고, 그것을 반복 게임의 그림 트리거(grim trigger) 전략으로 모형화했습니다. 「두 참여자가 다음 전략으로 조정한다고 하자. 정해진 간격으로 참여자들이 번갈아 입찰하되 매번 최소 인상 폭으로 올린다. 어느 참여자든 너무 일찍 입찰하거나 정해진 것보다 높게 올려 이탈하면, 다른 참여자는 게임이 더 이상 이익이 되지 않을 만큼 입찰을 올려 응수한다.」 관찰 3은 「그림 트리거 협조적 PGA에서 최적의 입찰은 $W[0] = s, W[i] = W[i-1] \times (1+\iota)$」, 곧 매번 최소 폭이고, 정리 1은 「이더리움 PGA와 정합적인 매개변수에 대해, 두 참여자 모두 D, W 협조 전략을 따르는 2인 PGA의 그림 트리거 내시 균형이 존재한다」입니다. 저자들은 조심스럽습니다. 「우리는 완전한 협조가 일어나고 있다거나 우리의 정확한 균형이 작동 중이라고 주장하지 않는다. 다만 참여자들이 우리 모형이 근사하는 더 협조적인 상태 — 시간에 걸친 자기 기대 이익을 최대로 하려고 다른 참여자에게 기회를 허락하는 상태 — 로 수렴하고 있다고 주장한다.」[22] 17장 각주에 적은 12.5%(패리티 클라이언트의 최소 인상 폭)로의 수렴이 그 증거였습니다. 「경쟁」으로 설계된 경매가 참여자 수가 적으면 **암묵적 담합으로** 갈 수 있다는 것은 경매 설계의 오랜 경계이고, 체인 위에서 그와 닮은 수렴이 관찰되었습니다 — 저자들의 단서대로 「정확히 그 균형」은 아닐지라도.

세 자리를 겹쳐 보면 이 장의 명제가 보입니다. 백서의 「정직이 이익」은 참여자가 작고, 보상이 일정하고, 공개가 즉시이고, 순서에서 뽑을 것이 없을 때의 균형입니다. 네 조건 가운데 어느 하나가 풀릴 때마다 — 큰 풀, 수수료 보상, 보류 가능한 블록, MEV — 다른 균형이 나타났습니다. 규칙은 여전히 그대로입니다. 바뀐 것은 규칙을 따르는 것이 **이익인가였습니다**.

```
# NB23-2: 2×2 이득표와 「문턱」 — 작은 참여자 둘의 공개/보류 게임을 이기적 채굴의 몫으로 채워 넣어 균형을 읽는다 (공부용)
# 두 풀 P(점유율 a), Q(점유율 b)가 각각 「정직(즉시 공개)」 또는 「보류(이기적 채굴)」를 고른다. 나머지(1-a-b)는 정직.
# 단순화: 보류하는 풀이 하나뿐이면 그 몫은 R_pool(α, γ); 둘 다 보류하면 서로 상대의 숨긴 블록을 모르므로 각자 R_pool 을 쓰되 상대도 보류하는 효과는 무시한다(한계 명시).
def payoff(a, b, P_selfish, Q_selfish, gamma):
    rp = R_pool_closed_form(a, gamma) if P_selfish else a
    rq = R_pool_closed_form(b, gamma) if Q_selfish else b
    return rp, rq

for gamma in [0.0, 0.5]:
    print(f"\nγ = {gamma}: 문턱 (1-γ)/(3-2γ) = {(1-gamma)/(3-2*gamma):.3f}")
    for a, b in [(0.20, 0.20), (0.30, 0.30), (0.40, 0.20)]:
        print(f"  P={a:.2f}, Q={b:.2f}  (행: P의 선택, 열: Q의 선택; 칸: P의 몫, Q의 몫)")
        print(f"  {'':>10}{'Q 정직':>18}{'Q 보류':>18}")
        for ps, pl in [(False, "P 정직"), (True, "P 보류")]:
            cells = []
            for qs in [False, True]:
                rp, rq = payoff(a, b, ps, qs, gamma)
                cells.append(f"({rp:.3f}, {rq:.3f})")
            print(f"  {pl:>10}{cells[0]:>18}{cells[1]:>18}")
        best_P = "보류" if R_pool_closed_form(a, gamma) > a else "정직"
        best_Q = "보류" if R_pool_closed_form(b, gamma) > b else "정직"
        print(f"  → P의 우월전략 {best_P}, Q의 우월전략 {best_Q}  (점유율이 문턱 아래면 정직이, 위면 보류가 우월하다)")
print("\n이 표는 「상대가 무엇을 하든 내 몫이 문턱으로 정해진다」는 단순화 위에 있다. 두 풀이 동시에 보류하는 경우의 상호작용은 모형 밖이다.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로

γ = 0.0: 문턱 (1-γ)/(3-2γ) = 0.333
  P=0.20, Q=0.20  (행: P의 선택, 열: Q의 선택; 칸: P의 몫, Q의 몫)
                           Q 정직               Q 보류
        P 정직     (0.200, 0.200)    (0.200, 0.130)
        P 보류     (0.130, 0.200)    (0.130, 0.130)
  → P의 우월전략 정직, Q의 우월전략 정직  (점유율이 문턱 아래면 정직이, 위면 보류가 우월하다)
  P=0.30, Q=0.30  (행: P의 선택, 열: Q의 선택; 칸: P의 몫, Q의 몫)
                           Q 정직               Q 보류
        P 정직     (0.300, 0.300)    (0.300, 0.273)
        P 보류     (0.273, 0.300)    (0.273, 0.273)
  → P의 우월전략 정직, Q의 우월전략 정직  (점유율이 문턱 아래면 정직이, 위면 보류가 우월하다)
  P=0.40, Q=0.20  (행: P의 선택, 열: Q의 선택; 칸: P의 몫, Q의 몫)
                           Q 정직               Q 보류
        P 정직     (0.400, 0.200)    (0.400, 0.130)
        P 보류     (0.484, 0.200)    (0.484, 0.130)
  → P의 우월전략 보류, Q의 우월전략 정직  (점유율이 문턱 아래면 정직이, 위면 보류가 우월하다)

γ = 0.5: 문턱 (1-γ)/(3-2γ) = 0.250
  P=0.20, Q=0.20  (행: P의 선택, 열: Q의 선택; 칸: P의 몫, Q의 몫)
                           Q 정직               Q 보류
        P 정직     (0.200, 0.200)    (0.200, 0.182)
        P 보류     (0.182, 0.200)    (0.182, 0.182)
  → P의 우월전략 정직, Q의 우월전략 정직  (점유율이 문턱 아래면 정직이, 위면 보류가 우월하다)
  P=0.30, Q=0.30  (행: P의 선택, 열: Q의 선택; 칸: P의 몫, Q의 몫)
                           Q 정직               Q 보류
        P 정직     (0.300, 0.300)    (0.300, 0.327)
        P 보류     (0.327, 0.300)    (0.327, 0.327)
  → P의 우월전략 보류, Q의 우월전략 보류  (점유율이 문턱 아래면 정직이, 위면 보류가 우월하다)
  P=0.40, Q=0.20  (행: P의 선택, 열: Q의 선택; 칸: P의 몫, Q의 몫)
                           Q 정직               Q 보류
        P 정직     (0.400, 0.200)    (0.400, 0.182)
        P 보류     (0.526, 0.200)    (0.526, 0.182)
  → P의 우월전략 보류, Q의 우월전략 정직  (점유율이 문턱 아래면 정직이, 위면 보류가 우월하다)

이 표는 「상대가 무엇을 하든 내 몫이 문턱으로 정해진다」는 단순화 위에 있다. 두 풀이 동시에 보류하는 경우의 상호작용은
모형 밖이다.
```

[중급] 둘째 셀은 교과서의 이득표에 이 장의 숫자를 넣어 본 것입니다. 점유율이 문턱 아래인 풀에게는 정직이, 위인 풀에게는 보류가 우월합니다 — 이 단순화가 각 풀의 몫을 상대의 선택과 **독립으로 두었기** 때문에 생기는 결과이지 발견이 아닙니다. 그래서 이 표 안에서는 작은 풀만 있는 세계에서 「모두 정직」이 균형이고, 문턱을 넘는 풀이 하나라도 생기면 그 풀에게는 「보류」가 균형 전략입니다. 셀의 주석이 밝히듯 두 풀이 동시에 보류하는 경우는 이 단순화 밖입니다. 그 경우의 분석은 이 책의 범위를 넘습니다.

> **? 법률가의 질문**
>
> 하드포크 뒤 두 체인이 모두 살아남았습니다. 같은 역사를 공유하고, 어느 블록부터 규칙이 갈렸으며, 양쪽 모두 자기가 「그」 원장이라고 합니다. 이 장은 어느 쪽이 이기는지를 정하는 것이 참여자들의 조정이라고 적었습니다. 그렇다면 「진짜」 원장은 누가 정합니까 — 해시파워의 다수입니까, 노드 운영자의 다수입니까, 거래소가 어느 쪽에 그 이름표를 붙이는가입니까, 개발자들입니까, 아니면 아무도 아닙니까. 포크 전에 코인을 가졌던 사람은 포크 뒤 두 체인 모두에 같은 잔고를 갖습니다. 그는 하나를 가졌습니까, 둘을 가졌습니까. 계약서에 「비트코인 1개를 지급한다」고 적혀 있었다면 어느 체인의 1개입니까. 이기적 채굴자처럼 규칙 안에서 이탈한 사람은 무엇을 위반한 것입니까 — 규칙입니까, 기대입니까, 아무것도 아닙니까. 이 책은 답하지 않고 5부의 두 절을 가리킵니다. → 27장 · 28장

6. 정리

백서의 「규칙을 따르는 편이 이익」은 하나의 균형 주장이고, 그 균형은 보상이 공격 가치에 견주어 클 때(20장), 참여자가 작을 때, 보상이 일정할 때, 블록이 즉시 공개될 때, 순서에서 뽑을 것이 없을 때 성립합니다. 이얄–시러는 블록을 숨기는 전략이 점유율 1/3(전파를 장악하면 0)부터 정직보다 많은 몫을 주고, 그 몫이 풀 크기와 함께 늘어 풀을 과반으로 밀어 간다고 논증했습니다. 그래서 「과반이 정직하면 안전」이 아니라 「3분의 2를 넘게 정직해야」이고, 그조차 낙관적 가정에서입니다. 수수료만 남으면 언더컷이 균형이 되고(20장), 순서 경매의 봇들은 암묵적 협조로 수렴하며(17장), 포크 선택은 조정 게임이어서 「진짜」 체인은 증명이 아니라 조정이 정합니다. 규칙은 그대로인데 균형이 바뀌는 것, 그것이 이 장이 「조건부」라고 부른 것의 뜻입니다. 24장은 같은 「믿음이 결과를 정하는 다수 균형」을 다른 자리 — 상환 약속이 있는 토큰의 뱅크런 — 에서 봅니다.

이 장의 출처

- Ittay Eyal & Emin Gün Sirer, "Majority Is Not Enough: Bitcoin Mining Is Vulnerable," *Communications of the ACM* 61(7): 95–102 (2018), DOI 10.1145/3212998 — 통념과 주장(95쪽), 모형(§3.1, 97쪽), 전략과 상태기계·전이별 보상·식 (3)(§3.2–4.1, 97–99쪽), 관찰 1·2와 γ(§4.3, 99–100쪽), 풀 형성(§5, 100쪽), 시빌과 수정안(§6, 100쪽), 관련 연구의 통념 인용(§7, 101쪽), 붕괴·탐지·은폐(§8, 101–102쪽), 결론(§9, 102쪽). arXiv:1311.0243v5 — 상태 확률 식 (1)–(5)(8쪽), 수입 식 (6)–(8)(10쪽), 관찰 1 식 (9)(11쪽). FC 2014 학회판(LNCS 8437, pp. 436–454)은 받지 않아 쪽을 인용하지 않았다.
- Satoshi Nakamoto (2008), §6 Incentive(4쪽).
- Eric Budish, "Trust at Scale," *QJE* 140(1): 1–62 (2025) — 일회 게임의 보조정리 1·2와 명제 2(§III.E, 28–30쪽), 작은 참여자 문헌에 대한 주석(30쪽).
- Miles Carlsten 외, "On the Instability of Bitcoin Without the Block Reward," *CCS '16* — 세 결정(§3.3), 균형의 비교(§5.4). 확장판, 절 번호만.
- Philip Daian 외, "Flash Boys 2.0," *2020 IEEE S&P*, pp. 910–927 — 협조 전략·관찰 3·정리 1(§VI-D, 919–920쪽).
- SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017) — 이더리움 클래식(9쪽 각주 32).
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `random` 만 사용. 노트북 NB23. NB23-1은 이얄–시러의 상태기계를 사건 단위로 돌려 CACM판 식 (3)과 대조하고, NB23-2는 그 식으로 2×2 이득표를 채운 공부용 단순화다(두 풀이 동시에 보류하는 상호작용은 모형 밖).

각주

1. Nakamoto (2008), 4쪽(§6). 20장 각주 1 과 같은 자리.
2. Eric Budish, "Trust at Scale," *QJE* 140(1): 1–62 (2025), 28–30쪽(§III.E), Lemma 1·2, Proposition 2. 원문: "If there is an honest equilibrium, then $N^* c \leq p$ (analog of equation (1))."; "If there is an honest equilibrium with $x_i^* = 0$ for some player i, then $N^* c \geq V_{attack}$ (analog of equation (2))."; "A necessary condition for an honest equilibrium is $p \geq \frac{V_{attack}}{1+1/I}$. In the limit as I → ∞ this is $p \geq V_{attack}$ (analog of equation (3))."
3. Budish (2025), 30쪽. 원문: "Note that this simple model purposefully restricts attention to honest play or majority attack. Many prior game-theoretic analyses of blockchains assume that all players are small, eliminating the possibility of majority attack, but allow for a richer set of strategies for such small players

than just honestly following the protocol (e.g., Eyal and Sirer 2014; Carlsten et al. 2016; Biais et al. 2019; Saleh 2021)."

4. Ittay Eyal & Emin Gün Sirer, "Majority Is Not Enough: Bitcoin Mining Is Vulnerable," *Communications of the ACM* 61(7): 95–102 (2018), 95쪽(초록). 원문: "Conventional wisdom asserts that the mining protocol is incentive-compatible and secure against colluding minority groups, that is, it incentivizes miners to follow the protocol as prescribed. We show that the Bitcoin mining protocol is not incentive-compatible. We present an attack with which colluding miners' revenue is larger than their fair share." (게재본). 원 논문은 FC 2014(LNCS 8437, pp. 436–454), 사전 인쇄본은 arXiv:1311.0243(v5, 2013-11-15). 이 책은 CACM판의 쪽·번호를 쓰고, arXiv v5의 식 번호는 각주에 적는다.

5. Eyal & Sirer (2018), 95쪽·101쪽(§7). 원문: "Because the protocol was believed to reward miners in proportion to their ratio of the mining power, a miner in a large pool was believed to earn the same revenue as it would in a small pool. Consequently, if we ignore the fixed cost of pool operation and potential economies of scale, there is no advantage for colluding miners to organize into ever-increasing pools."; "Felten notes that 'there was a folk theorem that the Bitcoin system was stable, in the sense that if everyone acted according to their incentives, the inevitable result would be that everyone followed the rules of Bitcoin as written.' Others have claimed that 'the well-known argument – never proven, but taken on intuitive faith – that a minority of miners can't control the network is a special case of a more general assumption: that a coalition of miners with X% of the network's hash power can make no more than X% of total mining revenues.'"; "Kroll, Davey and Felten's work provides a game-theoretic analysis of Bitcoin, without taking into account block withholding attacks such as selfish mining, and argues that the honest strategy constitutes a Nash equilibrium, implying incentive-compatibility."

6. Eyal & Sirer (2018), 95쪽(§1). 원문: "The key idea behind this strategy, called Selfish Mining, is for a pool to keep its discovered blocks private, thereby intentionally forking the chain. The honest nodes continue to mine on the public chain, while the pool mines on its own private branch. If the pool discovers more blocks, it develops a longer lead on the public chain, and continues to keep these new blocks private. When the public branch approaches the pool's private branch in length, the selfish miners reveal blocks from their private chain to the public. This strategy leads honest miners that follow the Bitcoin protocol to waste resources on mining cryptopuzzles that end up serving no purpose. Our analysis demonstrates that, while both honest and selfish parties waste some resources, the honest miners waste proportionally more, and the selfish pool's rewards exceed its share of the network's mining power".

7. Eyal & Sirer (2018), 97쪽(§3.1). 원문: "The system is comprised of a set of miners 1, . . ., n. Each miner i has mining power m_i ... Each miner chooses a chain head to mine, and finds a subsequent block for that head after a time interval that is exponentially distributed ... We assume that miners are rational; that is, they try to maximize their revenue, and may deviate from the protocol to do so."; "The expected relative revenue, or simply the revenue of a pool is the expected fraction of blocks that were mined by that pool out of the total number of blocks in the longest chain."

8. Eyal & Sirer (2018), 98쪽(§4, Figure 1 상태기계). 원문: "For state s = 0, 1, 2, . . ., with frequency α, the pool mines a block and the lead increases by one to s + 1. In states s = 3, 4, . . ., with frequency (1 − α), the honest miners mine a block and the lead decreases by one to s − 1. If the others mine a block when the lead is two, the pool publishes its private branch, and the system drops to a lead of 0. If the others mine a block with the lead is 1, we arrive at the aforementioned state 0'. From 0', there are three possible transitions, all leading to state 0 with total frequency 1: (1) the pool mines a block on its previously private branch (frequency α), (2) the others mine a block on the previously private branch (frequency γ(1 − α)), and (3) the others mine a block on the public branch (frequency (1 − γ)(1 − α))."; γ의 정의: "We denote by γ the ratio of honest miners that choose to mine on the pool's block, and the other (1−γ) of the non-pool miners mine on the other branch."

9. Eyal & Sirer (2018), 98–99쪽(§4.1 (a)~(h)). (h) 원문: "Lead was more than 2, others find a block. The others decrease the lead, which remains at least two. The new block (say with number i) will end outside the chain once the pool publishes its entire branch, therefore the others obtain nothing. However, the pool now reveals its i'th block, and obtains a revenue of one for its i'th block."

10. Eyal & Sirer (2018), 99쪽. 원문: "As expected, the intentional branching brought on by selfish mining leads the honest miners to work on blocks that end up outside the blockchain. This, in turn, leads to a drop in the total block generation rate with $r_{pool} + r_{others} < 1$. The protocol will adapt the mining difficulty such that the

mining rate at the main chain becomes one block per 10min on average. Therefore, the actual revenue rate of each agent is the revenue rate ratio; that is, the ratio of its blocks out of the blocks in the main chain."

11. Eyal & Sirer (2018), 99쪽(§4.3 "The effect of α and γ"), Observation 1(범위의 식 자체는 게재본에서 그림으로 조판되어 텍스트 층에 없고, arXiv v5 §4.4 식 (9)에 $\frac{1-\gamma}{3-2\gamma} < \alpha < \frac{1}{2}$ 로 적혀 있다). CACM 원문: "When the pool's revenue given in Equation 3 is larger than α, the pool will earn more than its relative size by using the Selfish-Mine strategy. ... Observation 1. For a given γ, a pool of size α obtains a revenue larger than its relative size for α in the range [식]." R_{pool} 의 닫힌 식은 CACM판 식 (3)(99쪽), arXiv v5 식 (8)(10쪽): "$R_{pool} = \frac{r_{pool}}{r_{pool}+r_{others}} = \cdots = \frac{\alpha(1-\alpha)^2(4\alpha+\gamma(1-2\alpha))-\alpha^3}{1-\alpha(1+(2-\alpha)\alpha)}$." arXiv v5의 상태 확률은 식 (2)~(5)(8쪽). 확인 과정에서 적어 둔 「Theorem 1」은 게재본 §5 본문이 「(Theorem 1)」이라고 가리키는 결과로, 두 판 모두에서 관찰 1·2와 같은 절의 내용이다.

12. Eyal & Sirer (2018), 99–100쪽. 원문: "Note that the pool is only at risk when it holds exactly one block secret, and the honest miners might publish a block that would compete with it. For γ = 1, the pool can quickly propagate its one-block branch if the others find their own branch, so all honest miners would still mine on the pool's block. In this case, the pool takes no risk when following the Selfish-Mine strategy and its revenue is always better than when following the honest algorithm. The threshold is therefore zero, and a pool of any size can benefit by following Selfish-Mine. In the other extreme, γ = 0, the honest miners always publish and propagate their block first, and the threshold is at 1/3. With γ = 1/2 the threshold is at 1/4. Figure 3 shows the threshold as a function of γ."

13. Eyal & Sirer (2018), 100쪽(§4.3·§5). 원문: "Observation 2. For a pool running the Selfish-Mine strategy, the revenue of each pool member increases with pool size for pools larger than the threshold."; "At this point, rational miners will preferentially join the selfish pool to increase their revenues. Moreover, the pool's members will want to accept new members, as this would increase their own revenue (Observation 2). The selfish pool would therefore increase in size, unopposed by any mechanism, towards a majority."

14. Eyal & Sirer (2018), 100쪽(§6.1–6.2). 원문: "But a savvy pool operator can perform a sybil attack on honest miners by adding a significant number of zero-power miners to the Bitcoin miner network. These virtual miners act as advance sensors by participating in data dissemination, but do not mine new blocks. ... once they hear of block X, they ignore it and start propagating block P. ... By adding enough virtual nodes, the pool operator can thus increase γ. The result (see Observation 1), is a threshold close to zero."; "Specifically, when a miner learns of competing branches of the same length, it should propagate all of them, and choose which one to mine on uniformly at random. ... This yields γ = 1/2, which in turn yields a threshold of 1/4."; "This improvement is independent of the adoption of the change at other miners, therefore it does not require a hard fork. This change to the protocol does not introduce new vulnerabilities to the protocol: Currently, when there are two branches of equal length, the choice of each miner is arbitrary, effectively determined by the network topology and latency."

15. Eyal & Sirer (2018), 102쪽(§8.4). 원문: "Even with our proposed fix that raises the threshold to 25%, the system remains vulnerable: there already exist pools whose mining power exceeds the 25% threshold, and at times, even the 33% theoretical hard limit. Responsible miners should therefore break off from large pools until no pool exceeds the threshold size." 이 문장의 「이미 존재한다」는 2013년 10월의 풀 통계(참고문헌 18)를 가리킨다.

16. Eyal & Sirer (2018), 102쪽(§9)·101쪽(§7). 원문: "We also show that at least 2/3 of the network needs to be honest to thwart selfish mining; a simple majority is not enough."; "Over 2/3 of the participants need to be honest to protect against selfish mining, under the most optimistic of assumptions."

17. Eyal & Sirer (2018), 101–102쪽(§8.2–8.3). 원문: "There are two telltale network signatures of selfish mining that can be used to detect when selfish mining is taking place, but neither are easy to measure definitively."; "It is easy to hide Selfish-Mine behavior, and difficult to ban it. A selfish pool may never reveal its size by using different Bitcoin addresses and IP addresses, and by faking block creation times."

18. Eyal & Sirer (2018), 101쪽(§8.1). 원문: "We therefore refer to a state in which a single entity controls the entire currency system as a collapse of Bitcoin. Note that such a collapse does not immediately imply that the value of a Bitcoin drops to 0. The controlling entity will have an incentive to accept most transactions, if only to reap their fees, and because if it mines all Bitcoins, it has strong motivation that they maintain their value. It may also choose to remain covert, and hide the fact that it can control the entire currency."

19. Carlsten 외 (2016), §3.3. 원문: "At each instant, every miner makes several distinct decisions: Which block to extend. How much of the available transactions (and associated fees) to include in the block they are solving. For each unpublished block, whether or not to publish." (확장판, 절 번호만 인용).

20. SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017-07-25), 9쪽 각주 32. 원문: "A minority group, however, elected not to adopt the new Ethereum Blockchain created by the Hard Fork because to do so would run counter to the concept that a blockchain is immutable. Instead they continued to use the former version of the blockchain, which is now known as 'Ethereum Classic.'" 포크 활성 블록(1,920,000)과 「포크는 프로토콜을 바꾸지 않았다」는 서술은 EIP-779(26장).

21. Carlsten 외 (2016), §5.4. 원문: "To understand the impact of Theorem 5.1, first consider the block reward model. With non-atomic miners, DefaultCompliant is trivially an equilibrium, and this result is robust to general models of latency (proof in Appendix D). But as we move to atomic miners, strategies like selfish mining arise and equilibria get messy (if they exist at all). Now, in the transaction-fee model, even when miners are non-atomic, equilibrium behavior is complex and undesirable, as we have just shown."

22. Philip Daian 외, "Flash Boys 2.0," *2020 IEEE S&P*, pp. 910–927, §VI-D(919–920쪽). 원문: "Consider two players who coordinate with the following strategies: at set intervals, players alternate their bids, each time bidding the minimum increase. If any player deviates either by bidding too soon or by raising the bid higher than they are supposed to, the other player will respond by raising their bid such that the game is no longer profitable."; "Observation 3. In a grim-trigger cooperative PGA, for all $i \le i_{max}$, the optimal choice for bids is $W[0] = s$ and $W[i] = W[i-1] \times (1 + \iota)$."; "Theorem 1. For parameters consistent with Ethereum PGAs, there exist a grim-trigger Nash equilibria for a 2 player PGA where both players follow a D, W cooperative strategy."; "We stress that we don't claim that perfect cooperation is occurring or even that our exact equilibrium is on play. We do claim, however, that players are converging to a more cooperative state, that is approximated by our model, in which they allow opportunities for other players in an effort to maximize their own expected profit over time." 형식적 처리는 논문 Appendix E.

3부 경제학

24장 스테이블코인 — 설계 유형과 뱅크런 구조

핵심 TAKEAWAY

19장의 코인은 아무것도 약속하지 않는 외부화폐였습니다. 스테이블코인은 반대로 「**1개 = 1달러로 돌려준다**」는 **약속이** 붙은 토큰이고, 그 순간 문제는 통화 경제학에서 **은행의 경제학으로** 옮겨 갑니다. 고튼과 장의 말로 「기술은 새롭지만 경제학은 수백 년 되었고」, 발행자는 「규제받지 않는 은행」입니다. 약속을 받치는 자산이 무엇이냐에 따라 법정화폐 담보·암호자산 초과담보·알고리듬형·합성형으로 갈리지만, 네 유형 모두 같은 저울 위에 있습니다. 상환 요구가 몰리면 준비자산을 팔아야 하고, 팔 수 없거나 제값에 팔 수 없으면 1달러 약속이 깨지며(디페그), 깨질 것 같다는 믿음 자체가 상환 요구를 부릅니다. 다이아몬드와 딥비그가 1983년에 적은 뱅크런의 구조 그대로입니다. 「런에 걸리지 않는 요구불예금 계약은 유동성 서비스를 제공하지 않는다.」 이 장을 읽으면 스테이블코인의 「안정」이 가격의 성질이 아니라 **대차대조표와 상환 절차의 성질임을** 알고, 2023년 3월의 런이 어디까지 교과서의 그림과 닮았고 어디서 달랐는지(2차 시장) 봅니다. 특정 코인의 안전성을 비교하지 않습니다. 유형의 구조만 봅니다.

학습목표

1. 스테이블코인을 「상환 약속이 붙은 토큰」으로 정의하고, 법정화폐 담보·암호자산 초과담보·알고리듬형·합성형의 대차대조표를 그린다. 19장의 외부화폐와 무엇이 다른지 안다.
2. [중급] 담보율과 상환 가능 비율을 정의하고, 준비자산의 유동성 구성이 상환 요구 집중 때 무엇을 정하는지 계산한다. 「묻지 않고 받음(NQA)」 원칙과 정보민감성이 왜 핵심인지 안다.
3. [심화] 다이아몬드–딥비그 모형의 두 균형(정상·런)과 그 선택이 왜 「거의 무엇에든」 달릴 수 있는지, 태환 정지와 예금보험이 각각 무엇을 바꾸는지 식으로 읽는다. 알고리듬형의 반사적 피드백을 단순 모형으로 본다.
4. 2022년 5월과 2023년 3월의 런을 실증 연구의 수치로 읽고(표본·기간·추정치), 어느 부분이 교과서와 같고 어느 부분이 다른지(2차 시장의 존재) 구별한다.

[입문]**만 읽은 독자가 가져가는 것.** 스테이블코인은 은행 예금과 MMF 사이 어딘가에 있고, 그래서 **은행이 겪는 런을 그대로 겪습니다.**

1. 이 장이 답하려는 질문

19장은 개럿-윌리스를 따라 비트코인을 외부화폐 — 「각 단위가 누군가의 자산이자 다른 누군가의 부채」가 **아닌** 돈 — 로 놓았습니다.[1] 스테이블코인은 그 정의의 반대쪽에 있습니다. 발행자가 「이 토큰 하나를 가져오면 1달러를 준다」고 약속하므로, 토큰은 보유자의 자산이자 발행자의 부채입니다.
내부화폐입니다. 그리고 내부화폐의 가치는 믿음이 아니라 **발행자가 약속을 지킬 수 있는가에** 달려 있습니다. 그 질문은 은행업이 수백 년 동안 다뤄 온 질문입니다. 고튼(Gary Gorton)과 장(Jeffery Zhang)의 2023년 논문 제목이 「들고양이 스테이블코인 길들이기(Taming Wildcat Stablecoins)」인 이유입니다. 「암호통화의 기반 기술은 새롭지만 경제학은 수백 년 되었다. … 이 경우 새 기술은 19세기에 여러 나라에서 유통되던 은행권과 비슷한 유통 사적 화폐를 스테이블코인의 형태로 재창조했다. 그 함의는 스테이블코인 발행자가 **규제받지 않는 은행이라는** 것이다.」[2]

이 장은 네 질문을 차례로 묻습니다. 약속을 무엇이 받치는가(유형). 약속이 지켜지려면 대차대조표가 어떤 조건을 만족해야 하는가(담보율과 유동성). 조건이 만족되어도 왜 런이 일어날 수 있는가(다이아몬드-딥비그). 그리고 실제 런은 그 그림과 얼마나 같았는가(실증). 특정 코인의 이름은 확정된 역사적 사건을 가리킬 때만 쓰고, 어느 코인이 더 안전한지는 묻지 않습니다. 이 장의 단위는 **유형과 구조입니다**.

2. 네 유형 — 약속을 무엇이 받치는가

국제결제은행(BIS)의 2022년 연차경제보고서는 둘로 나눕니다. 「스테이블코인의 두 주요 유형은 자산 담보형과 알고리듬형이다. 자산 담보형 스테이블코인 … 은 보통 중앙화된 중개기관이 운영하며, 그 기관이 기초 담보를 투자하고 코인의 상환과 생성을 조정한다. 자산은 국채, 단기 회사채, 은행 예금, 또는 다른 암호통화로 보유될 수 있다. 반대로 알고리듬형 스테이블코인 … 은 목표 통화나 자산에 대한 가치를 유지하려고 공급을 자동으로 재조정하는 복잡한 알고리듬에 의존한다. 법정통화 의존을 피하려고, 흔히 다른 암호통화에 대한 차익거래 기회를 사용자에게 제공하는 방식으로 그렇게 한다.」[3] 자산 담보형을 담보의 종류로 다시 나누고, 2026년 뉴욕 연방준비은행 블로그가 다룬 네 번째 유형을 더하면 아래 표가 됩니다. 각 행의 대차대조표는 이 책이 유형의 **구조를** 적은 것이지 어느 코인의 실제 장부가 아닙니다.

유형	자산(준비)	부채	「1달러」가 깨지는 길
법정화폐 담보	현금·은행 예금, 단기 국채, 환매조건부채권, (때로) 기업어음 등	발행 토큰 = 액면 상환 청구권	준비자산을 제때 제값에 팔지 못함(예금 동결, 어음 할인), 발행자 자체의 지급능력 의심
암호자산 초과담보	다른 암호자산(변동성이 크므로 액면보다 많이 잠금), 또는 다른 스테이블코인	발행 토큰	담보 가치 급락으로 초과분 소진, 청산 연쇄, 담보로 잡은 다른 스테이블코인의 디페그 전염
알고리듬형	발행자 자신의 **두 번째 토큰으로의 교환 약속** (실물 준비 없음)	발행 토큰	두 번째 토큰의 수요가 사라지면 「1달러어치」로 바꿔 줄 것이 없음 — 반사적 나선
합성형	스테이킹한 암호자산 롱 + 무기한 선물 숏(델타 헤지)	발행 토큰	자금조달 금리가 음(−)으로 돌아서거나 증거금 비용이 오르면 헤지 유지가 손실 — 상환과 청산의 되먹임

셋째 유형의 기제는 BIS 보고서의 Box A가 2022년 5월의 사건을 들어 적습니다. 「알고리듬형

스테이블코인인 그것은 테라 블록체인 위의 다른 암호통화 루나와의 자동화된 차익거래 전략으로 공급을 조정해 가치를 유지했다. UST는 1달러어치의 루나로, 그 역으로도 태환될 수 있게 함으로써 미 달러에 일대일 페그를 유지하려 했다. 예컨대 테라가 99센트로 떨어지면 사용자는 거래소에서 UST를 99센트에 사서 테라 플랫폼에서 1달러어치의 새 루나로 바꿀 수 있었다. 이 장치의 결정적 측면은 **루나의 시가총액이 UST의 시가총액을 넘을 때만** 사용자가 UST를 루나로 바꾸려 하리라는 것이었다.」[4] 넷째 유형은 뉴욕 연준의 2026년 6월 글이 설명합니다. 「합성 스테이블코인은 … 금융공학으로 달러 페그를 겨냥한다. 암호통화(보통 스테이킹 풀에 맡긴 이더)의 롱 포지션과 그것을 상쇄하는 무기한 선물 계약의 숏 포지션을 결합한다. … 이 구조는 실제 달러 준비금 없이 「합성」 달러 포지션을 만든다.」 그리고 「핵심 취약점은 헤지가 파생상품 시장의 지속적 유동성과 일관되게 양(+)인 자금조달 금리에 의존한다는 것이다. 자금조달이 음(−)으로 돌아서거나 증거금 비용이 오르면 합성 달러의 유지가 비싸지고 상환이 시작된다.」[5] 테라의 붕괴 자체는 26장에서 다룹니다. 이 장에서는 기제만 가져갑니다.

담보율과 상환 가능 비율

[중급] 유형이 달라도 약속이 지켜지는 조건은 같은 두 수로 적을 수 있습니다. 발행 토큰 수를 S, 준비자산의 현재 가치를 A 라 하면 **담보율은**

$$\mathrm{CR} = \frac{A}{S}$$

이고, 상환 요구가 발행량의 비율 q 만큼 몰렸을 때 **상환 가능 비율은** 그 요구를 맞추기 위해 **지금 당장 팔 수 있는** 자산의 처분 가치를 요구액으로 나눈 것

$$\mathrm{RR}(q) = \frac{\sum_i \ell_i A_i \,(1 - h_i)}{qS}$$

입니다. 기호 풀이: A_i 는 자산 종류 i 의 가치, $\ell_i \in [0,1]$ 은 그 가운데 당장 처분 가능한 몫(현금은 1, 동결된 예금은 0), h_i 는 급히 팔 때의 할인율입니다. 수치 예: 발행 100, 준비 100(현금 10, 국채 60(할인 0.5%), 은행 예금 25, 어음 5(할인 5%))이고 예금이 동결되지 않았다면 $q = 0.3$ 에서 처분 가능 가치는 $10 + 59.7 + 25 + 4.75 = 99.45$ 로 요구 30을 충분히 넘습니다. 그런데 **예금 25가 동결되고** $q = 0.8$ 이면 처분 가능 가치는 74.45, 요구는 80이어서 $\mathrm{RR} = 0.93$, 액면 상환이 불가능합니다. 담보율은 1.0 그대로인데 말입니다. 담보율은 지급능력의 수이고 상환 가능 비율은 유동성의 수이며, 런은 두 번째 수에서 시작합니다. 아래 NB24-2가 이 산수를 동적으로 돌립니다.

발행자 대차대조표

자산(준비)			부채	
현금		10	발행 토큰	100
국채	할인 0.5%	60		
은행 예금		25		
어음	할인 5%	5		
합		**100**		

담보율 CR = A/S = 100/100 = 1.0
— 지급능력의 수

상환 가능 비율 RR(q) = 처분 가능 가치 / 요구
— 유동성의 수. 런은 이 수에서 시작한다

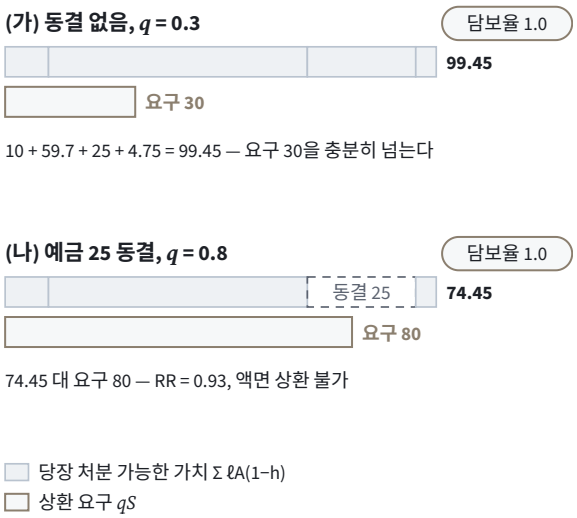


그림 24-1. 본문 수치 예의 발행자 대차대조표와 두 경우. 담보율은 둘 다 1.0이지만 당장 처분 가능한 가치가 상환 요구를 덮는지는 다르다.

3. 왜 런이 생기는가 — 「묻지 않고 받음」의 경제학

고튼과 장은 스테이블코인이 아직 화폐가 아니라고 하면서 그 이유를 19장에서 본 NQA 원칙으로 댑니다. 「대상이 교환매개로 쓰이리라고 그냥 가정할 수는 없다. 그러려면 그 대상이 NQA 원칙을 충족해야 한다. 교환매개의 가치가 당사자들이 그 가치에 대해 **서로 다르게 정보를 갖고 있기 때문에** 변동한다면 재화와 용역을 사거나 계약을 맺기가 얼마나 어려울지 상상해 보라. … 그래서 사적 화폐의 생산자는 정보에 둔감한 도구를 설계하려 한다. 거래의 어느 당사자도 그 가치에 대해 실사를 하지 않을 것인데, 그러기에는 너무 비싸기 때문이다. 그리고 모든 당사자가 이것을 알기 때문에 그 도구를 액면으로 받아들인다.」 고튼과 장은 그 둔감함을 얻는 최선의 길이 「부채를 부채로 받치는」 것 — 「사적 은행권을 국채로, 요구불예금을 대출 포트폴리오로, 환매조건부채권을 특정 채권으로」 — 이라고 적습니다(당·고튼·홈스트룀·오르도네스의 주장을 옮긴 것).[6] 그러면 런은 무엇입니까. 「가격이 고정되어 (다른 가격들이 수요 변화에 반응하듯) 변하지 않아야 한다면, 때때로 조정되어야 하는 것은 **수량이다**. 경제적 불확실성이 높아진 시기에는 그 수량이 뱅크런으로 매우 빠르게 0으로 조정될 수 있다. 화폐의 뒷받침이 의심스러워지는 상황이다.」[7] 가격을 못 박았으니 충격이 수량으로 빠져나가고, 수량의 조정은 「먼저 온 사람이 먼저 받는」 줄서기가 된다는 것입니다.

스테이블코인 발행자는 여기서 은행과 다른 자리에 있습니다. 「발행자는 불투명성과 투명성 사이의 상충에 직면한다. 한편으로는 아무도 뒷받침 자산에 대해 정보를 생산하는 것이 이익이 되지 않을 만큼 뒷받침이 불투명한 것이 최선이다. 다른 한편으로 뒷받침이 신뢰할 만하지 않다면 시장은 그에 대해 정보를 생산하려 할 것이다. 스테이블코인 발행자는 규제받지 않고 은행 검사관에 기댈 수 없어(따라서 불투명할 수 없어) 투명성이 최선이라는 견해를 가질 수 있다.」[8] 은행은 예금보험과 검사로 「묻지 않아도 되게」 만들었고, 스테이블코인은 월별 증명 보고서로 「물어보면 답하겠다」고 합니다. 두 해법은 같지 않습니다. 물어볼 수 있다는 것은 물어볼 이유가 생기면 모두가 동시에 묻는다는 뜻이기도 합니다. 뉴욕 연준의 2023년 보고서는 그 「물음」의 실물을 보고합니다. 「스테이블코인은 MMF보다 더 넓은 범위의 위험 프로필을 보인다. 어떤 스테이블코인은 현금과 미 국채 같은 안전자산으로 뒷받침된다고 보고하고 … 다른 것들은 회사채나 심지어 다른 암호자산 같은 더 위험한 담보로 뒷받침된다고 보고된다.」 그리고 「보고의 질은 발행자마다 다르다. 표준화되거나 규제된 보고 요건이 없기 때문이다.」

[9] 6절에서 이 보고서의 수치를 봅니다.

4. [심화] 다이아몬드–딥비그 — 런은 균형이다

1983년의 논문은 은행이 왜 존재하고 왜 런에 취약한지를 한 모형으로 답합니다. 저자들의 요약입니다. 「우리 모형은 세 가지 중요한 점을 보인다. 첫째, 요구불예금을 발행하는 은행은 서로 다른 무작위 시점에 소비해야 하는 사람들 사이에 더 나은 위험 분담을 제공함으로써 경쟁시장을 개선할 수 있다. 둘째, 이 개선을 제공하는 요구불예금 계약에는 바람직하지 않은 균형(뱅크런)이 있어, 모든 예금자가 공황에 빠져 즉시 인출한다 — 은행 파산을 걱정하지 않았다면 예금을 두었을 사람들까지. 셋째, 뱅크런은 실질적 경제 문제를 일으킨다. 건전한 은행도 파산할 수 있어 대출 회수와 생산적 투자의 중단을 낳기 때문이다.」[10] 바꿔 말하면 **유동성 서비스와 런 취약성은 같은 것의 두 면입니다.** 「유동성 없는 청구권을 유동성 있는 청구권으로 바꾸는 바로 그 변환이 은행이 제공하는 유동성 서비스와 런에 대한 취약성 모두의 원인이다.」[11]

▾ [심화] 모형의 뼈대, 두 균형, 그리고 두 처방

[심화] 기간은 셋($T = 0, 1, 2$), 재화는 하나입니다. 생산 기술은 0기에 1을 넣으면 2기에 $R > 1$ 을 돌려주되, 1기에 중단하면 원금 1만 돌려줍니다 — 「장기 투자는 어느 정도 비가역적」이라는 뜻입니다. 모든 사람은 0기에 같고, 1기에 자기 유형을 **남몰래** 알게 됩니다. 유형 1은 1기에만 소비하려 하고(비율 t), 유형 2는 2기에 소비합니다.[12] 유형이 공개 정보라면 최적 보험 계약은 유형 1에게 $c_1^* > 1$, 유형 2에게 $c_2^* < R$ 을 주어 둘 사이에 위험을 나눕니다. 그런데 유형은 사적 정보라 시장이 그 계약을 쓸 수 없고, 은행이 **요구불예금으로** 그것을 흉내 냅니다. 「요구불예금 계약은 1기에 인출하는 각 예금자에게 0기 예금 단위당 고정액 r_1 을 준다. 인출 요청은 은행 자산이 바닥날 때까지 무작위 순서로 **순차적으로** 처리된다.」[13] 1기에 r_1 을 받는 사람의 몫 V_1 과, 기다려서 2기에 받는 사람의 몫 V_2 는

$$V_1(f_j, r_1) = \begin{cases} r_1 & f_j < r_1^{-1} \\ 0 & f_j \geq r_1^{-1} \end{cases}, \qquad V_2(f, r_1) = \max\left\{\frac{(1 - r_1 f)R}{1 - f},\ 0\right\}$$

입니다(식 (6)·(7)). 기호 풀이: f_j 는 나보다 앞서 처리된 인출의 비율, f 는 1기 총 인출 비율. $r_1 f$ 가 1을 넘으면 은행의 자산이 바닥나 뒷줄은 0을 받고, 기다린 사람들은 남은 $(1 - r_1 f)$ 를 R 배로 불려 나눕니다.[14]

[심화] **두 균형.** $r_1 = c_1^*$ 로 두면 「유형 1은 1기에 인출하고 유형 2는 기다리는 것이 균형이다 — 그것이 예상되는 한. 이 좋은 균형은 최적 위험 분담을 이룬다. 또 다른 균형(뱅크런)은 모든 사람이 공황에 빠져 1기에 예금을 인출하려 하는 것이다. 이것이 예상되면 모든 사람이 1기 인출을 선호할 것이다. 예금의 액면가가 은행 자산의 청산 가치보다 크기 때문이다.」 그리고 「$r_1 > 1$ 인 모든 경우에 런은 균형이다. $r_1 = 1$ 이면 … 은행은 자산의 직접 보유를 흉내 낼 뿐이므로 … 런에 걸리지 않는 요구불예금 계약은 유동성 서비스를 제공하지 않는다.」[15] 어느 균형이 실현되는가는 모형 밖입니다. 「균형의 선택이 경제에서 공통으로 관측되는 어떤 무작위 변수에 달려 있다면 그럴 수 있다. 나쁜 실적 보고, 다른 은행에서 공통으로 관측된 런, 부정적 정부 전망, 심지어 태양흑점일 수도 있다. … 관측되는 변수가 은행의 상태에 대해 근본적인 어떤 것도 전할 필요가 없다. 문제는 일단 예금이 들어오고 나면 런을 예상하게 만드는 **무엇이든** 런으로 이어진다는 것이다.」[16] 19장의

태양흑점이 여기서 다시 나옵니다. 거기서는 가치 0으로의 붕괴를, 여기서는 런을 고르는 장치로.

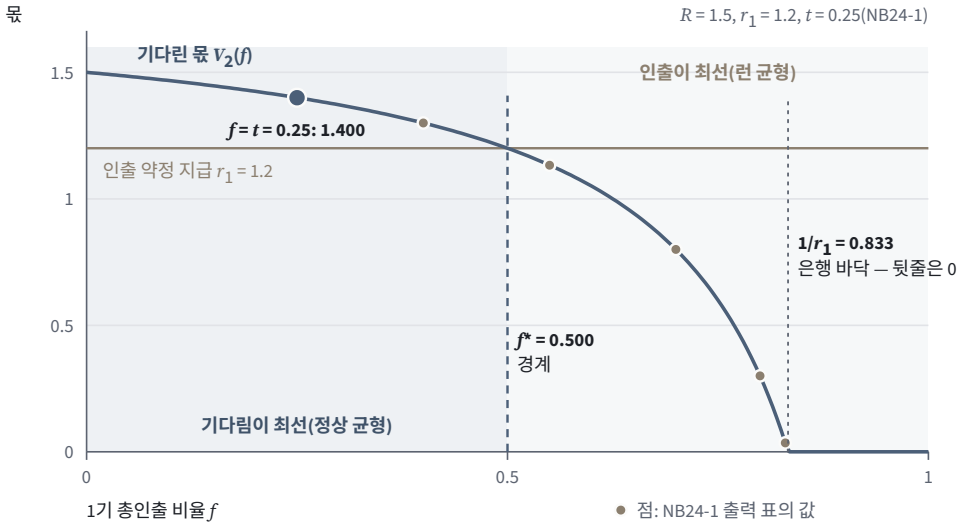


그림 24-2. 다이아몬드–딥비그 모형에서 기다린 사람의 몫과 인출 약정 지급 r_1. 인출은 순차로 처리되어 $r_1 f$ 가 1을 넘으면 은행 자산이 바닥나고 뒷줄은 0을 받는다. 곡선은 본문 식 (7)의 그래프($R = 1.5, r_1 = 1.2$)이고, 점과 숫자는 NB24-1의 출력이다.

[심화] **두 처방.** 첫째는 **태환 정지입니다.** 「은행이 1기에 인출이 너무 많을 때 태환을 정지할 수 있다면, 이 정책에 대한 예상이 유형 2의 조기 인출 유인을 없애 런을 막는다.」 정지점 $\hat{f}$ 를 두면 「유형 2는 다른 사람들의 인출을 어떻게 예상하든 2기까지 기다리는 쪽이 더 많이 받으므로 … 유일한 내시 균형 $f = t$ 가 있고, 사실 이것은 우월전략 균형이다.」 다만 「이 계약은 정상 인출량 t 가 알려져 있고 확률적이지 않은 경우에만 완벽히 작동한다.」[17] t 가 무작위이면 명제 1 — 「순차 처리 제약을 지켜야 하는 은행 계약은 t 가 확률적일 때 최적 위험 분담을 이룰 수 없다」 — 이 성립하고, 태환 정지는 런을 막되 일부 유형 1을 사후적으로 묶어 두는 비효율을 남깁니다. 둘째가 **예금보험입니다.** 「정부 예금보험이 있는 요구불예금 계약은 정부가 예금보험 재원을 위한 최적 조세를 부과하면 제약 없는 최적을 유일한 내시 균형(사실은 우월전략 균형)으로 달성한다.」 핵심은 정부가 **인출이 끝난 뒤에** 과세할 수 있다는 점입니다. 「정부는 이미 인출한 사람에게도 세금을 매길 수 있으므로 실현된 1기 총인출 f 에 세금을 걸 수 있다. 이것은 순차 처리를 해야 하고 인출이 이루어진 뒤에는 그 금액을 줄일 수 없는 은행과 뚜렷이 대조된다.」 그리고 「모든 가능한 예상 아래 런에 가담하는 것이 결코 이익이 되지 않으므로 … 신뢰에 관한 전략적 문제가 생기지 않는다.」[18] 최후의 대부자(할인창구)는 기술이 무위험일 때 예금보험과 같은 일을 할 수 있다고 저자들은 덧붙입니다.[19]

```
# NB24-1: 다이아몬드–딥비그의 두 균형 — 요구불예금 r1 > 1, 1기 총인출 비율 f 에 따라 「기다림」의 몫 V2 가
「인출」 r1 보다 큰가 (공부용)
R, r1, t = 1.5, 1.2, 0.25          # 2기 수익 R, 1기 약정 지급 r1(> 1), 유형 1(꼭 1기에 써야 하는 사람)
의 비율 t

def V2(f):                          # 식 (7): 기다린 사람의 몫
    return max((1 - r1 * f) * R / (1 - f), 0.0) if f < 1 else 0.0

print(f"R = {R}, r1 = {r1}, 유형 1 비율 t = {t}. 은행이 바닥나는 인출 비율 1/r1 = {1 / r1:.3f}")
print(f"{'1기 총인출 f':>12} {'기다린 몫 V2(f)':>16} {'인출 몫 r1':>10} {'유형 2의 최선':>12}")
for f in [t, 0.40, 0.55, 0.70, 0.80, 0.83, 0.90, 1.00]:
    v = V2(f)
    print(f"{f:>12.2f} {v:>16.3f} {r1:>10.2f} {'기다림' if v > r1 else '인출(런)':>12}")
# 기다림이 인출보다 못해지는 경계: (1 - r1 f)R/(1 - f) = r1  →  f* = (R - r1)/(r1 (R - 1))
f_star = (R - r1) / (r1 * (R - 1))
print(f"\n경계 f* = (R - r1)/(r1(R - 1)) = {f_star:.3f}. 남들이 이보다 많이 인출하리라 믿으면 나도
인출하는 것이 최선이다 — 믿음이 결과를 정한다.")
print(f"f = t = {t} (정상)에서 유형 2는 기다리고 {V2(t):.3f} 를 받는다(> r1). f = 1 (런)에서는 모두가 평균
1 을 나눠 갖고 뒷줄은 0 — 은행이 없었을 때보다 모두 나쁘다.")
print("\n태환 정지(f 가 정지점에 닿으면 1기 지급 중단)를 넣으면 기다린 사람의 몫이 r1 아래로 떨어질 수 없어 유형
2 는 기다리는 것이 우월전략이 된다 — t 를 알 때.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
R = 1.5, r1 = 1.2, 유형 1 비율 t = 0.25. 은행이 바닥나는 인출 비율 1/r1 = 0.833
    1기 총인출 f      기다린 몫 V2(f)    인출 몫 r1     유형 2의 최선
        0.25            1.400       1.20          기다림
        0.40            1.300       1.20          기다림
        0.55            1.133       1.20        인출(런)
        0.70            0.800       1.20        인출(런)
        0.80            0.300       1.20        인출(런)
        0.83            0.035       1.20        인출(런)
        0.90            0.000       1.20        인출(런)
        1.00            0.000       1.20        인출(런)

경계 f* = (R - r1)/(r1(R - 1)) = 0.500. 남들이 이보다 많이 인출하리라 믿으면 나도 인출하는 것이 최선이다 —
믿음이 결과를 정한다.
f = t = 0.25 (정상)에서 유형 2는 기다리고 1.400 를 받는다(> r1). f = 1 (런)에서는 모두가 평균 1 을 나눠 갖고
뒷줄은 0 — 은행이 없었을 때보다 모두 나쁘다.

태환 정지(f 가 정지점에 닿으면 1기 지급 중단)를 넣으면 기다린 사람의 몫이 r1 아래로 떨어질 수 없어 유형 2 는
기다리는 것이 우월전략이 된다 — t 를 알 때.
```

5. 스테이블코인에 옮겨 놓기 — 같은 것과 다른 것

모형의 자리마다 스테이블코인의 대응물을 놓습니다. 은행 = 발행자. 비유동 자산 = 준비자산 가운데 당장 팔 수 없거나 할인 없이 팔 수 없는 것(2절의 ℓ_i, h_i). 요구불예금 = 액면 상환 청구권. 순차 처리 = 1차 시장(발행자 창구)의 상환 대기열. 두 균형과 태양흑점 선택은 그대로 옮겨집니다. 다른 것이 둘 있습니다.

첫째, **예금보험이 없습니다.** 고튼과 장의 표 1이 선택지를 늘어놓습니다. 현상 유지와 「보험 없는 은행 유사 규제」는 런도 NQA도 해결하지 못하고, 「예금보험 대상 은행의 규제 테두리 안에서 발행」·「국채나 중앙은행 준비금으로 1:1 뒷받침」·「중앙은행 디지털 통화로 대체」 셋만 둘 다 해결한다고 저자들은 적습니다.[20] 1:1 뒷받침이 왜 런을 없애는지는 다이아몬드–딥비그에서 바로 보입니다. $r_1 = 1$ 이고 자산이 전부 당장 처분 가능하면 런은 균형이 아닙니다 — 그리고 그 대가로 유동성 변환이라는 은행업의 핵심도 사라집니다. 이 책은 어느 선택지가 옳은지 말하지 않습니다. 법역별로 어느 쪽을 택했는지는 27장의 서베이 몫입니다.

둘째, **2차 시장이 있습니다.** 은행 예금은 양도되지 않지만 스테이블코인은 거래소에서 거래됩니다. 연준 이사회의 2025년 12월 노트가 이 차이를 2023년 3월의 사건에서 짚습니다. 「은행 예금과 달리 스테이블코인은 2차 시장의 거래소에서 활발히 거래된다. 이는 1차 시장에서 상환을 정지해도 은행을

잠시 닫는 것처럼 런을 멈출 수는 없다는 뜻이다.」[21] 다이아몬드–딥비그의 첫 처방(태환 정지)이 이 자리에서 다르게 작동합니다. 1차 시장의 창구를 닫으면 상환 압력이 2차 시장의 가격으로 빠져나갑니다(연준 노트는 창구가 닫힌 동안 「상환 압력이 쌓였다」고 적습니다). 가격 하락이 다시 상환 요구를 부른다는 되먹임은 노트가 보고한 것이 아니라 **이 책의 모형 가정이고**, 아래 NB24-2는 그것을 단순한 규칙으로 넣습니다.

준비자산 구성과 상환 요구

```
# NB24-2: 준비자산 구성 × 상환 요구 → 디페그 되먹임 — 유형 A·B·C 는 구조의 예시이지 어느 코인의 장부가 아니다 (공부용)
# 되먹임 규칙(이 책의 단순화): 상환 가능 비율 RR < 1 이거나 발행자가 창구를 닫으면 2차 시장 가격이 RR 쪽으로 내려가고,
# 가격이 1 아래면 추가 보유자가 상환에 나선다(아래 '민감도' 만큼). 가격·수량은 전부 모형 변수다.
def run_dynamics(reserves, q0, sensitivity=1.5, rounds=6, window_open=True):
    """reserves: [(이름, 비중, 처분가능몫 ℓ, 급매 할인 h)], 비중 합 = 1(담보율 1.0). q0: 처음 상환 요구 비율."""
    S = 1.0                                      # 발행량 1 로 정규화
    q, price, history = q0, 1.0, []
    redeemed = 0.0
    for r in range(1, rounds + 1):
        liquid = sum(w * l * (1 - h) for _, w, l, h in reserves)        # 지금 당장 처분 가능한 가치 (발행량 1 기준)
        demand = min(q, 1.0) - redeemed
        rr = liquid / demand if demand > 0 else float("inf")
        if window_open and rr >= 1:
            redeemed += demand; price = 1.0
            history.append((r, q, round(rr, 2), price, "액면 상환 완료"))
            break
        price = max(min(rr, 1.0), 0.0) if window_open else max(price - 0.05, 0.0)   # 창구가 닫히면 가격이 미끄러진다(규칙)
        history.append((r, round(q, 3), round(rr, 2), round(price, 3), "창구 열림" if window_open else "창구 닫힘"))
        q = min(1.0, q + sensitivity * (1 - price) * (1 - q))          # 가격이 1 아래면 더 많은 보유자가 상환에 나선다
    return history

types = {
    "A 현금·국채 위주":            [("현금", 0.20, 1.0, 0.0), ("단기국채", 0.75, 1.0, 0.005), ("은행예금", 0.05, 1.0, 0.0)],
    "B 예금·어음 섞임":            [("현금", 0.05, 1.0, 0.0), ("단기국채", 0.40, 1.0, 0.005), ("은행예금", 0.30, 1.0, 0.0), ("기업어음", 0.25, 1.0, 0.05)],
    "B′ B에서 은행예금 전부 동결": [("현금", 0.05, 1.0, 0.0), ("단기국채", 0.40, 1.0, 0.005), ("은행예금(동결)", 0.30, 0.0, 0.0), ("기업어음", 0.25, 1.0, 0.05)],
    "C 암호자산 초과담보(처음 담보율 1.5 → 담보 45% 급락 뒤 0.83)": [("담보 암호자산 — 급락 45% 뒤 잔존 가치, 급매 할인 15%", 1.5 * 0.55, 1.0, 0.15)],
}
for name, res in types.items():
    cr = sum(w for _, w, _, _ in res)
    print(f"\n[{name}] 담보율 {cr:.2f}")
    for q0 in [0.10, 0.40, 0.80]:
        hist = run_dynamics(res, q0)
        last = hist[-1]
        print(f"  처음 요구 {q0:.0%}: {len(hist)}라운드 → 요구 {last[1]:.0%}, 상환가능비율 {last[2]}, 가격 {last[3]} ({last[4]})")

print("\n[B′ 에서 발행자가 창구를 닫은 경우] — 2차 시장이 있으면 창구를 닫아도 런은 가격으로 이어진다")
for row in run_dynamics(types["B′ B에서 은행예금 전부 동결"], 0.40, window_open=False):
    print("  라운드 {} 요구 {} RR {} 가격 {} {}".format(*row))
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로

[A 현금·국채 위주] 담보율 1.00
  처음 요구 10%: 1라운드 → 요구 10%, 상환가능비율 9.96, 가격 1.0 (액면 상환 완료)
  처음 요구 40%: 1라운드 → 요구 40%, 상환가능비율 2.49, 가격 1.0 (액면 상환 완료)
  처음 요구 80%: 1라운드 → 요구 80%, 상환가능비율 1.25, 가격 1.0 (액면 상환 완료)

[B 예금·어음 섞임] 담보율 1.00
  처음 요구 10%: 1라운드 → 요구 10%, 상환가능비율 9.86, 가격 1.0 (액면 상환 완료)
  처음 요구 40%: 1라운드 → 요구 40%, 상환가능비율 2.46, 가격 1.0 (액면 상환 완료)
  처음 요구 80%: 1라운드 → 요구 80%, 상환가능비율 1.23, 가격 1.0 (액면 상환 완료)

[B' B에서 은행예금 전부 동결] 담보율 1.00
  처음 요구 10%: 1라운드 → 요구 10%, 상환가능비율 6.85, 가격 1.0 (액면 상환 완료)
  처음 요구 40%: 1라운드 → 요구 40%, 상환가능비율 1.71, 가격 1.0 (액면 상환 완료)
  처음 요구 80%: 6라운드 → 요구 97%, 상환가능비율 0.7, 가격 0.704 (창구 열림)

[C 암호자산 초과담보(처음 담보율 1.5 → 담보 45% 급락 뒤 0.83)] 담보율 0.83
  처음 요구 10%: 1라운드 → 요구 10%, 상환가능비율 7.01, 가격 1.0 (액면 상환 완료)
  처음 요구 40%: 1라운드 → 요구 40%, 상환가능비율 1.75, 가격 1.0 (액면 상환 완료)
  처음 요구 80%: 6라운드 → 요구 96%, 상환가능비율 0.73, 가격 0.726 (창구 열림)

[B' 에서 발행자가 창구를 닫은 경우] — 2차 시장이 있으면 창구를 닫아도 런은 가격으로 이어진다
  라운드 1 요구 0.4 RR 1.71 가격 0.95 창구 닫힘
  라운드 2 요구 0.445 RR 1.54 가격 0.9 창구 닫힘
  라운드 3 요구 0.528 RR 1.3 가격 0.85 창구 닫힘
  라운드 4 요구 0.634 RR 1.08 가격 0.8 창구 닫힘
  라운드 5 요구 0.744 RR 0.92 가격 0.75 창구 닫힘
  라운드 6 요구 0.84 RR 0.82 가격 0.7 창구 닫힘
```

[중급] 표를 읽을 때 유형 이름에 걸리지 마십시오. A·B·C는 준비자산의 **구성이** 다를 뿐이고, 어느 유형이 실제 어느 코인인지 이 책은 말하지 않습니다. 보이는 것은 세 가지입니다. 담보율이 1.0으로 같아도 처분 가능 몫 ℓ 과 급매 할인 h 가 결과를 가르고(B와 B′) — 이것이 유동성의 문제입니다. 처음 담보율이 1.5여도 담보가 45% 급락하면 담보율 자체가 1 아래(0.83)로 내려가므로(C) 그때의 액면 붕괴는 유동성이 아니라 **지급능력의** 문제입니다 — 초과담보형에서는 두 문제가 담보 가격 하나로 이어져 있습니다. 그리고 창구를 닫으면 가격이 미끄러진다는 **규칙을 셀에 넣으면**, 창구 폐쇄는 은행과 달리 런을 멈추지 못합니다 — 이 셋째는 셀이 보여 준 결과가 아니라 2차 시장이 있다는 가정의 귀결이고, 그 가정의 근거는 아래 연준 노트입니다. 이 장의 위젯 W8은 이 셀의 세 변수 — 상환 요구 비율, 준비자산의 유동성 구성, 담보율 — 만 움직이고 코인 이름을 쓰지 않습니다.

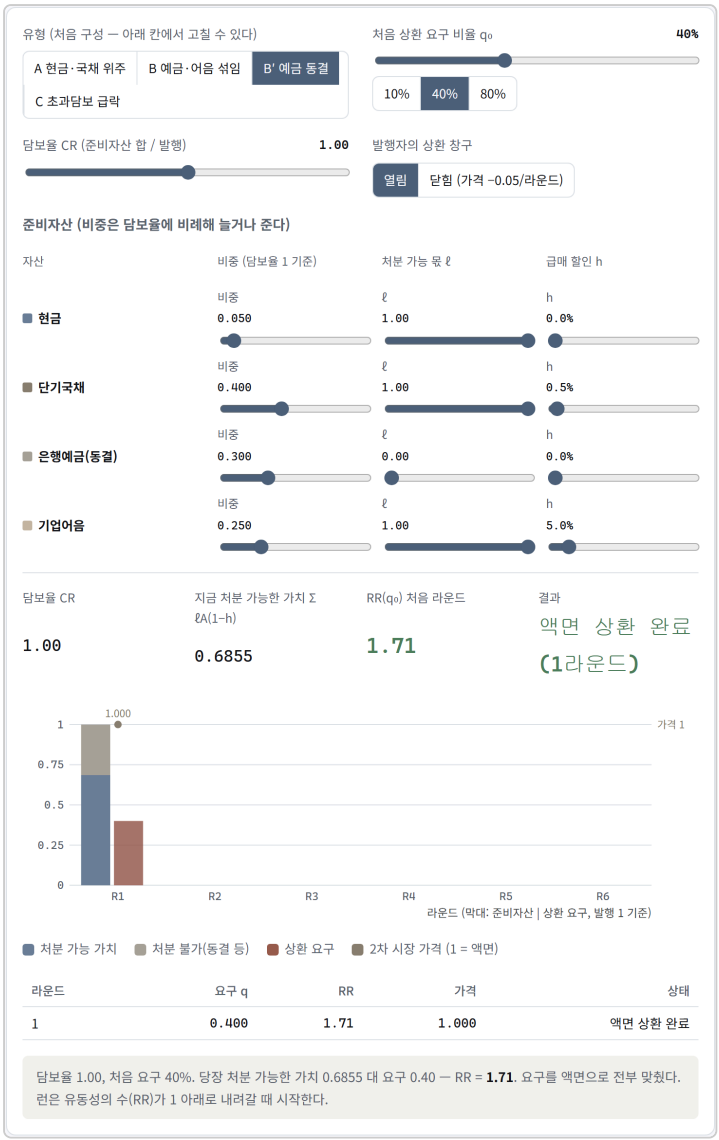


그림 24-3. 스테이블코인 뱅크런. 유형 B′(은행예금 30% 동결)에서 상환 요구 40%는 상환 가능 비율 1.71 로 액면 상환되지만, 80%면 여섯 라운드 뒤 요구 97%·가격 0.704 로 미끄러진다. 유형은 구조의 예시이지 어느 코인의 장부가 아니다. — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 24장 5절).

알고리듬형 — 준비자산 대신 교환 약속

▾ [심화] 알고리듬형의 반사적 되먹임 — 단순 모형

[심화] 알고리듬형은 준비자산 대신 **두 번째 토큰으로의 교환 약속으로** 1달러를 받칩니다. BIS가 적은 기제를 모형 변수로 옮기면 이렇습니다. 스테이블 토큰 S 와 변동 토큰 L 이 있고, S 하나는 언제나 「1달러어치의 L」로 바꿀 수 있습니다. L 의 가격이 p_L 이면 S 하나를 태우고 $1 / p_L$ 개의 L 을 새로 찍습니다. 이 장치가 작동하려면 L 을 받아 줄 수요가 있어야 하고, BIS의 표현대로 「L 의 시가총액이 S 의 시가총액을 넘을 때만」 사용자가 교환하려 합니다. 수요가 흔들려 p_L 이 내리면, 같은 1달러어치를 주려고 더 많은 L 을 찍어야 하고, 늘어난 공급이 p_L 을 더 내리며, 그것이 더 많은 상환을 부릅니다. 이것이 BIS 가 「고전적 런 동학」이라 부른 것의 기제이고,[22] 2022년 5월에 실제로 무엇이 어떤 순서로 일어났는지는 26장에서 읽습니다. 아래 셀은 이 되먹임을 수요 함수 하나로 흉내 냅니다. 숫자는 전부 모형값이고 어떤 자료에도 맞추지 않았습니다.

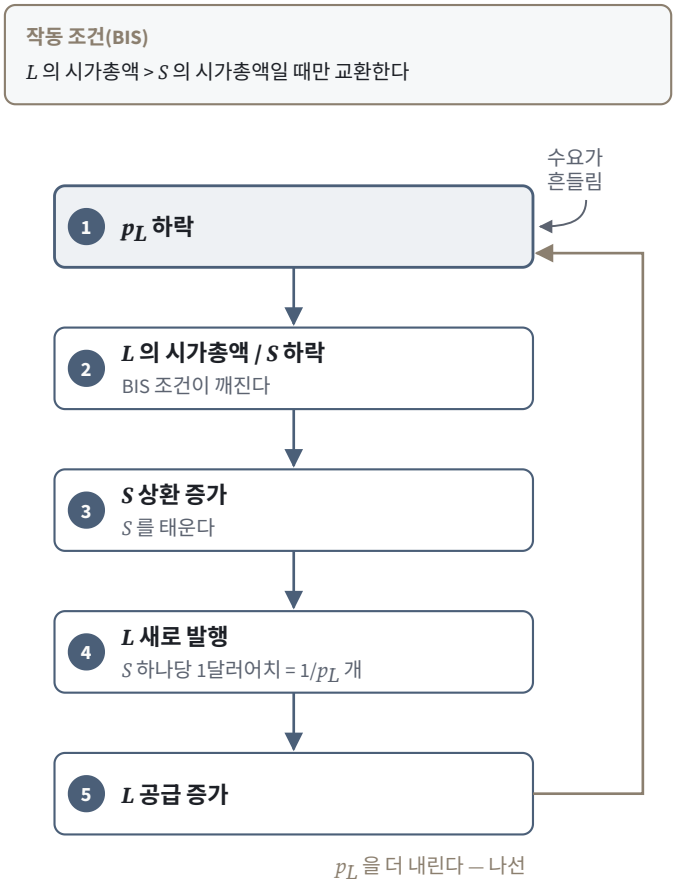


그림 24-4. 알고리듬형 스테이블코인의 반사적 되먹임. 본문이 BIS의 기제를 모형 변수로 옮긴 순서를 고리로 그렸고, 숫자는 넣지 않았다.

```
# NB24-3: 알고리듬형의 반사적 되먹임 — 「S 하나 = 1달러어치 L」 교환 약속, L 의 수요곡선, 뒷받침이 깨지면 커지는 상환 셋만으로 나선이 생기는지 본다 (공부용, 모형 변수)
def algorithmic_spiral(S0, L0, p0, eps=1.0, rounds=10, base_redeem=0.05, panic=0.6, shock_round=3, shock=0.3):
    """L 의 수요곡선: p_L = D / L**(1+eps). eps > 0 이면 공급이 늘 때 가격이 비례 이상으로 내린다(수요가 비탄력적; eps=1 이면 L 시가총액 = D/L).
    매 라운드 S 보유자 일부가 S 를 태우고 1/p_L 개의 L 을 받는다. 뒷받침(L 시가총액/S)이 1 아래면 상환 비율이 panic 만큼 커진다.
    shock_round 에 수요 수준 D 가 shock 배로 준다. 모든 숫자는 모형값이다."""
    S, L = float(S0), float(L0)
    D = p0 * L ** (1 + eps)
    print(f"{'라운드':>4} {'S 발행량':>9} {'L 발행량':>11} {'p_L':>8} {'L 시가총액/S':>12} {'이번 상환 비율':>12}")
    for r in range(1, rounds + 1):
        if r == shock_round:
            D *= shock                                          # 수요 충격(모형 변수)
        p_L = D / L ** (1 + eps)
        backing = L * p_L / S                                   # S 하나당 L 전체로 돌려줄 수 있는 달러어치
        share = base_redeem + (panic * (1 - backing) if backing < 1 else 0.0)
        share = min(share, 0.9)
        print(f"{r:>4} {S:>9,.0f} {L:>11,.0f} {p_L:>8.4f} {backing:>12.2f} {share:>12.2f}")
        burn = S * share
        S -= burn
        L += burn / p_L                                         # 태운 S 만큼 「1달러어치」의 L 을 새로 찍는다
    return S, L, D / L ** (1 + eps)

S_end, L_end, p_end = algorithmic_spiral(S0=1_000, L0=2_000, p0=1.0)
print(f"\n끝: S {S_end:,.0f}, L {L_end:,.0f}, p_L {p_end:.4f}.")
print("수요 충격 뒤 L 의 시가총액이 S 아래로 떨어지면(BIS 의 조건이 깨지면) 상환이 L 을 더 찍고, 늘어난 공급이 p_L 을 더 내리며, 그것이 상환을 더 부른다 — 나선.")
print("이 표의 숫자는 어떤 자료에도 맞추지 않은 모형값이다. 보이는 것은 모양뿐이다: 뒷받침이 「다른 토큰의 수요」일 때 그 수요가 사라지면 뒷받침도 사라진다.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
 라운드      S 발행량        L 발행량       p_L      L 시가총액/S      이번 상환 비율
   1     1,000       2,000   1.0000         2.00         0.05
   2       950       2,050   0.9518         2.05         0.05
   3       902       2,100   0.2721         0.63         0.27
   4       659       2,996   0.1337         0.61         0.29
   5       471       4,400   0.0620         0.58         0.30
   6       328       6,699   0.0267         0.55         0.32
   7       222      10,663   0.0106         0.51         0.35
   8       145      17,966   0.0037         0.46         0.37
   9        91      32,602   0.0011         0.40         0.41
  10        54      65,409   0.0003         0.34         0.45

끝: S 30, L 151,116, p_L 0.0001.
수요 충격 뒤 L 의 시가총액이 S 아래로 떨어지면(BIS 의 조건이 깨지면) 상환이 L 을 더 찍고, 늘어난 공급이 p_L 을 더 내리며, 그것이 상환을 더 부른다 — 나선.
이 표의 숫자는 어떤 자료에도 맞추지 않은 모형값이다. 보이는 것은 모양뿐이다: 뒷받침이 「다른 토큰의 수요」일 때 그 수요가 사라지면 뒷받침도 사라진다.
```

6. 실증 — 두 번의 런

[중급] 실증 연구 셋이 2022년 5월과 2023년 3월의 런을 자료로 적었습니다. 이 책은 그 가운데 **구조에 관한 수치만** 옮기고, 각 수치에 표본과 기간을 붙입니다.

연구	자료·기간	발견	한계
Anadu 외, NY Fed Staff Report 1073 (2023/2024)	스테이블코인 일별 자금 흐름·가격, 암호시장 스트레스 사건(비트코인 가격 급락일로 식별)	**안전으로의 도피**: 스트레스 일에 발행자 보고 기준 「위험한」 준비자산의 코인에서 「안전한」 코인으로 순유출. **디페그 문턱**: 가격이 1달러 이상이면 흐름이 가격과 무관하나, 1달러 아래로 떨어지면 일별 유출률이 34bp 뛴다(MMF의 「95센트」 문턱과 비교). 2022년 5월 런은 알고리듬형·위험 담보형에서, 2023년 3월 런은 그 전까지 「안전」으로 여겨진 코인에서 시작해 그 코인을 담보로 잡은 다른 코인으로 번짐. 도피는 **같은 블록체인 안에서** 일어나는 경향	「안전·위험」 분류가 발행자의 자기 보고에 의존(각주 6). 문턱 추정의 식별은 가격 1달러 전후의 불연속 — 인과가 아니라 관측된 반응
Du·Sonawane·Watsky, FEDS Note (2025-12-17)	2023-03-09~14의 시간별 1차 시장 순발행·2차 시장 가격·거래량(데이터 제공자 Allium·CryptoCompare)	발행자가 **총 준비금의 약 8%인** 33억 달러를 파산한 은행에 예치하고 있었다고 공시 → 상환 요구 급증 → 주말 1차 시장 사실상 정지 → 2차 시장 최저 86센트 → 당국의 예금 전액 보호 발표 뒤 회복, 월요일 상환 재개 뒤 페그 복귀. 1차 시장이 닫히자 2차 시장 거래가 급증(시간당 약 20억 달러)했고 **탈중앙 거래소가 거래의 대부분을** 차지. 다른 코인이 1:1 교환 장치로 이 코인을 담보로 잡고 있어 전염	사건 하나의 기술(記述)이며 반사실 없음(저자들 명시). 정책 논의는 범위 밖
Anadu 외, Liberty Street Economics (2026-07-31)	같은 발행자의 준비자산 구성(MMF의 가중평균만기·환매조건부채권 비중·은행 예금 비중), 2023–2026 월별·분기별	사건 뒤 준비 MMF의 가중평균만기가 국채 전용 MMF 분포의 5백분위수 아래로 내려가 머물렀고, 환매조건부채권 비중이 0에서 90% 넘게 뛰었다가 69%로, 은행 예금은 사건 전 수준 아래로 줄고 대형(GSIB) 은행에 집중 → 「금리 위험에서 상대방 위험으로」 위험의 **종류가 바뀜**	발행자 한 곳의 자발적 공시 자료

23 24 25

[중급] 세 연구를 겹치면 교과서와 닮은 것과 다른 것이 갈립니다. 닮은 것은 「먼저 온 사람이 먼저 받는」 상환 대기열과 전염(한 코인을 담보로 잡은 다른 코인으로)입니다. 다른 것이 둘입니다. 첫째, 방아쇠는 태양흑점이 아니라 **근본 정보였습니다** — 준비금의 약 8%가 묶였다는 공시이고, 연준 노트도 「발행자에 대한 지급능력 우려가 작용했을 것」이라고 적습니다.[27] 둘째, 2차 시장입니다. 창구가 닫힌 주말 동안 거래가 2차 시장으로 몰렸고 가격이 저점을 찍었습니다. 회복은 두 단계였습니다. 당국의 예금 보호 발표 뒤 「급반등」, 발행자가 상환을 재개한 월요일에 「완전 회복」 — 노트는 반사실을 다루지 않으므로 어느 쪽이 런을 멈췄는지는 이 자료로 말할 수 없습니다. 말할 수 있는 것은 순서뿐이고, 다이아몬드–딥비그의 둘째 처방(예금보험)이 스테이블코인 자체가 아니라 그 준비자산이 놓인 은행에 적용되었다는 **구조입니다**.

[심화] 한 걸음 더 나가면 준비자산 쪽의 되먹임도 있습니다. BIS 워킹페이퍼 1270은 「달러 담보 스테이블코인 흐름이 단기 미 국채 수익률에 미치는 영향을 2021년 1월부터 2026년 3월까지의 일별 자료로」 살피고, 「국소 투영과 토큰–블록체인 쌍 수준의 특이 변동을 이용한 그래늘러 도구변수로, 35억 달러(2표준편차) 유입이 3개월 국채 수익률을 즉시 0.71bp, 10일 안에 최대 4bp 낮추며 장기물로의 파급은 제한적」이고 「효과는 국채시장 스트레스 아래서, 그리고 스테이블코인 부문의 규모와 함께

커진다」고 보고합니다.[26] 이 추정은 2026년 6월 개정판의 것이고, 식별 전략(도구변수)이 명시되어 있어 — 도구의 배제 제약이 성립한다는 전제 아래 — 상관이 아니라 효과로 읽을 수 있습니다. 저자들은 추정치를 「통계적으로 유의」하다고 적고 불확실성은 본문 그림의 신뢰구간으로 제시하므로 이 책은 점추정치만 옮깁니다. 함의는 런의 **반대 방향이고**, 그것은 저자들 자신의 말입니다. 「우리의 추정치는 유출 효과의 하한일 가능성이 크다 — 발행자가 매입 시점을 고를 수 있는 장기 시장 성장기에서 얻은 것이기 때문이다.」 상환이 몰려 준비 국채를 팔아야 하면 같은 통로로 수익률이 움직일 수 있다는 것이고, 연준 노트가 「런이 다르게 전개되었다면 상환 압력이 발행자에게 뒷받침 자산(예컨대 미 국채)의 청산을 강요해 전통 금융시장에 파급 효과를 낳았을 수 있다」고 적은 것이 그 자리입니다.[28] 기준일의 부문 규모·국채 보유액은 부록 A 동향 노트에 둡니다(A-18).

> **? 법률가의 질문**
>
> 스테이블코인 보유자는 무엇을 가진 사람입니까. 고튼과 장은 「스테이블코인이 요구불예금인지를 법적으로 판단하려면 먼저 보유자와 발행자 사이의 계약이 지분 계약인지 채무 계약인지 정해야 한다」고 묻고, 그 질문이 1970년대 MMF가 「예금」인지를 두고 벌어진 논쟁과 같은 모양이라고 적습니다.[29] 보유자는 예금자입니까(그렇다면 예금보험과 은행 규제의 대상입니까), 무담보 채권자입니까(그렇다면 발행자 도산 시 일반 채권자와 같은 순위입니까), 아니면 준비자산에 대한 신탁의 수익자입니까(그렇다면 준비자산은 도산재단에서 분리됩니까). 2023년 3월에 발행자는 「회사 자원으로, 필요하면 외부 자본으로 부족분을 메우겠다」고 공언했습니다(연준 노트 §1, 각주 27). 그 공언은 보유자에게 어떤 권리를 줍니까. 1:1 뒷받침을 법이 요구한다면 그 「뒷받침」의 법적 구조 — 분리 보관, 우선권, 신탁 — 는 무엇이어야 합니까. 이 책은 답하지 않고 5부의 법역별 서베이와 쟁점 절을 가리킵니다. → 27장 · 28장

7. 정리

스테이블코인은 상환 약속이 붙은 토큰이고, 그래서 외부화폐의 경제학이 아니라 은행의 경제학에 속합니다. 약속을 받치는 것이 법정화폐 자산이든, 초과담보로 잡은 암호자산이든, 두 번째 토큰으로의 교환이든, 헤지된 파생상품 포지션이든, 1달러가 깨지는 길은 같습니다. 상환이 몰릴 때 제때 제값에 팔 수 없다는 것이고, 깨질 것 같다는 믿음이 상환을 부른다는 것입니다. 담보율은 지급능력의 수이고 상환 가능 비율은 유동성의 수이며, 런은 둘째 수에서 시작합니다. 다이아몬드–딥비그는 런이 좋은 균형과 나란히 존재하는 또 하나의 균형이고, 그 선택이 근본과 무관한 신호에 달릴 수 있으며, 태환 정지와 예금보험이 각각 무엇을 바꾸는지를 보였습니다. 스테이블코인에는 예금보험이 없고 2차 시장이 있어, 창구를 닫는 것이 런을 멈추지 못합니다. 2023년 3월의 런은 이 그림과 닮은 동학을 보였고(방아쇠는 근본 정보, 회복의 순서는 자료가 말해 주는 데까지만), 2022년 5월의 알고리듬형은 유동성 변환이 없는 구조여서 대응이 다릅니다 — 그 사건은 26장이 읽습니다. 이 장은 어느 코인이 안전한지 말하지 않았습니다. 「안정」은 가격의 성질이 아니라 대차대조표와 절차의 성질입니다. 25장은 반대쪽 끝, 아무 약속도 없는 자산의 가격에 「버블」이라는 말을 통계로 붙일 수 있는지를 묻습니다. 2022년 5월의 붕괴는 26장에서 실패의 구조로 읽습니다.

이 장의 출처

- Gary B. Gorton & Jeffery Y. Zhang, "Taming Wildcat Stablecoins," *University of Chicago Law Review* 90(3): 909–972 (2023) — 초록(909쪽), 수량 조정과 런(912쪽), 표 1(913쪽)과 단점(914쪽), NQA와 부채-위-부채(918쪽), 불투명성 상충(919쪽), 요구불예금인가(920쪽).
- Douglas W. Diamond & Philip H. Dybvig, "Bank Runs, Deposit Insurance, and Liquidity," *Journal of Political Economy* 91(3): 401–419 (1983) — *Federal Reserve Bank of Minneapolis Quarterly Review* 24(1): 14–23 (2000) 재수록본으로 읽었고 인용 쪽은 재수록본 쪽이다(세 요점 15쪽, 모형 16쪽, 계약과 식 (6)·(7) 17–18쪽, 두 균형·태양흑점 18쪽, 태환 정지 19쪽, 명제 1 20쪽, 명제 2 21쪽, 할인창구 22쪽). JPE 원본 쪽은 대조하지 못했다.
- Bank for International Settlements, "III. The future monetary system," *BIS Annual Economic Report 2022*, pp. 75–115 — 두 유형(81쪽), Box A(82쪽).
- Kenechukwu Anadu 외 9인, "Runs and Flights to Safety: Are Stablecoins the New Money Market Funds?," Federal Reserve Bank of New York Staff Reports No. 1073 (2023-09, rev. 2024-04) — 초록·서론(1–3쪽).
- Chuan Du, Ria Sonawane & Cy Watsky, "In the Shadow of Bank Runs," *FEDS Notes* (2025-12-17), DOI 10.17016/2380-7172.3958 — Summary, §1–1.2. HTML(2026-10-25 접근).
- Pablo D. Azar & Jeff Garofano, "Synthetic Stablecoins and Financial Stability," *Liberty Street Economics* (2026-06-23), DOI 10.59576/lse.20260623 — 합성형의 구조와 취약점. HTML(2026-10-25 접근).
- Kenechukwu Anadu 외 8인, "Stablecoins and (Non)Crypto Shocks: A 2026 Update," *Liberty Street Economics* (2026-07-31) — 준비자산 구성의 변화. HTML(2026-10-25 접근).
- Rashad Ahmed & Iñaki Aldasoro, "Stablecoins and safe asset prices," BIS Working Papers No 1270 (2025-05, rev. 2026-06) — 초록.

- Rodney Garratt & Neil Wallace (2018), 1887쪽 — 내부·외부화폐(19장과 같은 자리).
- 담보율·상환 가능 비율의 식과 NB24-2·NB24-3의 되먹임 규칙은 이 책의 단순화이며 어느 코인의 장부에도 맞추지 않았다.
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리만 사용. 노트북 NB24. 위젯 W8 `stablecoin-run` 의 처음 상태는 NB24-2의 유형 B′·처음 요구 40%이고, 변수는 상환 요구 비율·준비자산 유동성 구성·담보율 셋이며 코인 이름을 쓰지 않는다.

각주

1. Garratt & Wallace (2018), 1887쪽. 19장 각주 4.

2. Gary B. Gorton & Jeffery Y. Zhang, "Taming Wildcat Stablecoins," *University of Chicago Law Review* 90(3): 909–972 (2023), 909쪽(초록). 원문: "While the technology underlying cryptocurrencies is new, the economics is centuries old. ... In this case, the new technology has recreated circulating private money in the form of stablecoins, which are similar to the banknotes that circulated in many countries during the nineteenth century. The implication is that stablecoin issuers are unregulated banks. Based on lessons learned from economic theory and financial history, we argue that circulating private money is not an effective medium of exchange because it is not always accepted at par and its issuers are vulnerable to destabilizing bank runs."

3. Bank for International Settlements, "III. The future monetary system," *BIS Annual Economic Report 2022*, pp. 75–115 (2022-06-26), 81쪽. 원문: "The two main types of stablecoin are asset-backed and algorithmic. Asset-backed stablecoins, such as Tether, USD Coin and Binance USD, are typically managed by a centralised intermediary who invests the underlying collateral and coordinates the coins' redemption and creation. Assets can be held in government bonds, short-term corporate debt or bank deposits, or in other cryptocurrencies. In contrast, algorithmic stablecoins, such as TerraUSD before its implosion, rely on complex algorithms that automatically rebalance supply to maintain their value relative to the target currency or asset. To avoid reliance on fiat currency, they often do so by providing users with an arbitrage opportunity relative to another cryptocurrency." 원문이 예로 든 코인 이름은 이 책의 본문에서 뺐다.

4. BIS (2022), 82쪽, Box A "The collapse of the TerraUSD stablecoin". 원문: "An algorithmic stablecoin, it maintained value by adjusting supply in an automated arbitrage trading strategy with another cryptocurrency, Luna, on the Terra blockchain. UST aimed to keep a one-for-one peg to the US dollar by being convertible into one dollar's worth of Luna, and vice versa. For example, should Terra fall to 99 cents, a user could purchase UST on an exchange for 99 cents and then exchange their UST for $1 worth of new units of Luna on the Terra platform. A crucial aspect of this arrangement was that users would only be willing to exchange UST into Luna if Luna's market capitalisation exceeded that of UST." 붕괴의 경과와 규모는 26장.

5. Pablo D. Azar & Jeff Garofano, "Synthetic Stablecoins and Financial Stability," Federal Reserve Bank of New York *Liberty Street Economics*, 2026-06-23. 원문: "synthetic stablecoins like Ethena (USDe) target their dollar peg through financial engineering: they combine a long position in cryptocurrency (typically ether staked in a staking pool like Lido) with an offsetting short position in perpetual futures contracts. ... This structure creates a 'synthetic' dollar position without requiring actual dollar reserves."; "The key fragility, however, is that the hedge depends on continuous liquidity in derivatives markets and a consistently positive funding rate. If funding turns negative or margin costs rise, the synthetic dollar becomes expensive to maintain and redemptions begin." https://libertystreeteconomics.newyorkfed.org/2026/06/synthetic-stablecoins-and-financial-stability/ (HTML, 2026-10-25 다시 열어 축어 대조). 원문이 든 프로토콜 이름은 본문에서 뺐다.

6. Gorton & Zhang (2023), 918쪽. 원문: "But it cannot just be assumed that an object will be used as a medium of exchange. For that to happen, the object must satisfy the NQA principle. Imagine attempting to purchase goods and services, or enter into contractual arrangements, when the value of the medium of exchange fluctuates because parties are differentially informed about its value. Engaging in such transactions would be incredibly difficult. Private producers of money therefore try to design an instrument that is information insensitive: no party to a transaction would engage in due diligence on its value because doing so would be too expensive. And because all parties to the transaction know this, they accept the instrument at par."; "And this is best accomplished by backing the debt with debt—for example, by backing private banknotes with

state bonds, by backing demand deposits with portfolios of loans, or by backing sale and repurchase agreements (repos) with specific bonds."

7. Gorton & Zhang (2023), 912쪽. 원문: "However, if the price is to stay fixed and not vary (as other prices usually do in response to changes in demand), then it is the quantity that must sometimes adjust. During times of heightened economic uncertainty, the quantity can adjust very quickly to zero in a bank run, a situation in which the backing of the money becomes suspect. Stablecoin issuers are vulnerable to these bank runs, which can spill over into the broader economy."

8. Gorton & Zhang (2023), 919쪽. 원문: "Stablecoin issuers therefore face a trade-off with respect to opacity and transparency. On one hand, it would be best if the backing for their stablecoins were so opaque that nobody would find it profitable to produce information about the backing assets. On the other hand, if the backing were not credible, then the market would want to produce information about the backing. Stablecoin issuers may take the view that transparency is best, because they are not regulated and cannot rely on bank examiners (and, thus, cannot be opaque)."

9. Kenechukwu Anadu, Pablo D. Azar, Marco Cipriani, Thomas M. Eisenbach, Catherine Huang, Mattia Landoni, Gabriele La Spada, Marco Macchiavelli, Antoine Malfroy-Camine & J. Christina Wang, "Runs and Flights to Safety: Are Stablecoins the New Money Market Funds?," Federal Reserve Bank of New York Staff Reports No. 1073 (2023-09, 2024-04 개정), 1–2쪽과 각주 6. 원문: "In fact, stablecoins exhibit an even wider range of risk profiles than MMFs. Some stablecoins report that they are backed by safe assets, such as cash and US Treasuries, whose value remains stable or even increases during times of stress. Others, instead, are reportedly backed by riskier collateral, such as corporate debt or even other crypto assets ..."; 각주 6: "Our characterization of a stablecoin as 'safe' or 'risky' is based on the composition of its assets (also known as 'reserves') as reported by the stablecoin issuer. Reporting quality, however, varies across stablecoin issuers, because there is no standardized or regulatory reporting requirement." (2024-04 개정판).

10. Douglas W. Diamond & Philip H. Dybvig, "Bank Runs, Deposit Insurance, and Liquidity," *Journal of Political Economy* 91(3): 401–419 (1983). 이 책은 *Federal Reserve Bank of Minneapolis Quarterly Review* 24(1): 14–23 (2000) 재수록본으로 읽었고, 쪽은 **재수록본 쪽이다**(재수록본 15쪽). 재수록본은 머리 각주에 「JPE 1983, vol. 91, no. 3, pp. 401–19에서 허락을 받아 재수록했으며 ... 출판을 위해 편집되었다」고 밝힌다. 원문: "Our model demonstrates three important points. First, banks issuing demand deposits can improve on a competitive market by providing better risk-sharing among people who need to consume at different random times. Second, the demand deposit contract providing this improvement has an undesirable equilibrium (a bank run) in which all depositors panic and withdraw immediately, including even those who would prefer to leave their deposits in if they were not concerned about the bank failing. Third, bank runs cause real economic problems because even healthy banks can fail, causing the recall of loans and the termination of productive investment."

11. Diamond & Dybvig (1983/2000), 재수록본 18쪽. 원문: "It is precisely the transformation of illiquid claims into liquid claims that is responsible both for the liquidity service provided by banks and for their susceptibility to runs."

12. Diamond & Dybvig (1983/2000), 재수록본 16쪽. 원문: "long-term capital investments are somewhat irreversible"; "The model has three periods (T = 0, 1, 2) and a single homogeneous good. The productive technology yields R > 1 units of output in period 2 for each unit of input in period 0. If production is interrupted in period 1, the salvage value is just the initial investment."; "Each faces a privately observed, uninsurable risk of being of type 1 or of type 2. In period 1, each agent's type is determined and revealed privately to the agent. Type 1 agents care only about consumption in period 1, and type 2 agents care only about consumption in period 2."

13. Diamond & Dybvig (1983/2000), 재수록본 17쪽. 원문: "In our model, the demand deposit contract gives each agent withdrawing in period 1 a fixed claim of r_1 per unit deposited in period 0. Withdrawal tenders are served sequentially in random order until the bank runs out of assets."; 순차 처리 제약: "a bank's payoff to any agent can depend only on the agent's place in line and not on future information about agents later in line." (18쪽)

14. Diamond & Dybvig (1983/2000), 재수록본 18쪽, 식 (6)·(7). 원문: "$V_1(f_j, r_1) = r_1$ if $f_j < r_1^{-1}$; 0 if $f_j \geq r_1^{-1}$" and "$V_2(f, r_1) = \max\{(1 - r_1 f)R / (1 - f), 0\}$ where f_j is the quantity of withdrawers' deposits serviced before agent j and f is the total quantity of demand deposits withdrawn, both as fractions of total demand deposits."

15. Diamond & Dybvig (1983/2000), 재수록본 18쪽. 원문: "If this contract is in place, it is an equilibrium for type 1 agents to withdraw at T = 1 and for type 2 agents to wait, provided this is what is anticipated. This good equilibrium achieves optimal risk-sharing. Another equilibrium (a bank run) has all agents panicking and trying to withdraw their deposits at T = 1: if this is anticipated, all agents will prefer to withdraw at T = 1. This is because the face value of deposits is larger than the liquidation value of the bank's assets."; "For all $r_1 > 1$, runs are an equilibrium. ... but if $r_1 = 1$, the bank simply mimics direct holding of the assets, and the bank is therefore no improvement on simple competitive-claims markets. A demand deposit contract which is not subject to runs provides no liquidity services."

16. Diamond & Dybvig (1983/2000), 재수록본 18쪽. 원문: "This could happen if the selection between the bank run equilibrium and the good equilibrium depended on some commonly observed random variable in the economy. This could be a bad earnings report, a commonly observed run at some other bank, a negative government forecast, or even sunspots. ... The observed variable need not convey anything fundamental about the bank's condition. The problem is that once agents have deposited, anything that causes them to anticipate a run will lead to a run."

17. Diamond & Dybvig (1983/2000), 재수록본 19쪽. 원문: "If banks can suspend convertibility when withdrawals are too numerous at T = 1, anticipation of this policy prevents runs by removing the incentive of type 2 agents to withdraw early."; "Given this contract, no type 2 agent will withdraw at T = 1 because no matter what the agent anticipates about others' withdrawals, the agent receives higher proceeds by waiting until T = 2 to withdraw ... Therefore, there is a unique Nash equilibrium which has f = t. In fact, this is a dominant strategy equilibrium ..."; "This contract works perfectly only in the case where the normal volume of withdrawals, t, is known and not stochastic." 명제 1(20쪽): "Bank contracts (which must obey the sequential service constraint) cannot achieve optimal risk-sharing when t is stochastic and has a nondegenerate distribution."

18. Diamond & Dybvig (1983/2000), 재수록본 21쪽, Proposition 2. 원문: "Demand deposit contracts with government deposit insurance achieve the unconstrained optimum as a unique Nash equilibrium (in fact, a dominant strategies equilibrium) if the government imposes an optimal tax to finance the deposit insurance."; "As the government can impose a tax on an agent who has withdrawn, the government can base its tax on f, the realized total value of T = 1 withdrawals. This is in marked contrast to a bank, which must provide sequential service and cannot reduce the amount of a withdrawal after it has been made."; "Furthermore, deposit insurance prevents runs because, for all possible anticipated withdrawal policies of other agents, participating in a bank run never pays. As a result, no strategic issues of confidence arise."

19. Diamond & Dybvig (1983/2000), 재수록본 22쪽. 원문: "Through its discount window, the Federal Reserve can, as a lender of last resort, provide a service similar to deposit insurance. ... The identity of deposit insurance and discount window services occurs because the technology is riskless."

20. Gorton & Zhang (2023), 913쪽, Table 1 "Options to Address Stablecoins" — 열: Eliminate Runs? / Achieve NQA?; 행: Status quo (No/No), Implement bank-like regulations on stablecoin issuers, but no insurance (No/No), Issue stablecoins from within the insured-bank regulatory perimeter (Yes/Yes), Require stablecoins to be backed one-for-one by Treasuries or central bank reserves (Yes/Yes), Replace stablecoins with a central bank digital currency (Yes/Yes). 저자들은 뒤의 셋에도 각각 단점이 있다고 적는다(914쪽, 국채 1:1의 과소 발행 문제 등).

21. Chuan Du, Ria Sonawane & Cy Watsky, "In the Shadow of Bank Runs: Lessons from the Silicon Valley Bank Failure and Its Impact on Stablecoins," *FEDS Notes*, Board of Governors of the Federal Reserve System, 2025-12-17, §1.2. 원문: "Unlike bank deposits, stablecoins are actively traded on exchanges in the secondary markets. This means that the suspension of redemption on the primary market cannot arrest a run, as temporarily closing a bank would." https://www.federalreserve.gov/econres/notes/feds-notes/in-the-shadow-of-bank-run-lessons-from-the-silicon-valley-bank-failure-and-its-impact-on-stablecoins-20251217.html (HTML, 2026-10-25 다시 열어 축어 대조; 2026-01-08 정정 이력 있음).

22. BIS (2022), 82쪽, Box A. 원문: "As UST dropped below its peg, a classic run dynamic took hold as investors sought to redeem their funds." 그 뒤의 경과 서술은 26장에서 인용한다.

23. Anadu 외 (2023/2024), 2–3쪽. 원문: "we isolate stress events in crypto markets associated with large declines in the price of Bitcoin and study the response of stablecoin investors to such events. We find that, during these episodes, investors run from riskier stablecoins to safer ones ..."; "while a stablecoin priced $1.00 or above experiences flows that are uncorrelated with prices, once its price drops below the $1 threshold, its daily outflow rate jumps by 34 basis points. This suggests that investors consider a stablecoin to have depegged

when its price drops below $1, similar to the 95-cent 'break-the-buck' threshold for MMFs."; "The most significant episode so far occurred when Terra, an algorithmic stablecoin and the fourth-largest stablecoin at the time, suffered a run and subsequently collapsed in early May 2022. Following a classic flight-to-safety dynamic, this run had negative spillover effects on other algorithmic stablecoins and stablecoins backed by risky assets, whereas those backed by relatively safer assets experienced net inflows."; "Unlike the May 2022 Terra run, which occurred among the riskier stablecoins, the March 2023 run had at its epicenter USDC, which had previously been considered among the safest stablecoins due to its portfolio of Treasury securities and bank deposits. The run spilled over into Dai and Frax, both of which were partially collateralized by USDC."; 초록: "as flight-to-safety flows occur within MMF families, stablecoin flows tend to happen within blockchains."

24. Du·Sonawane·Watsky (2025), §1–1.2. 원문: "At 10 p.m. on Friday evening, Circle announced that it was unable to withdraw $3.3 billion of USDC reserves from SVB (around 8% of total reserves at the time), sparking a wave of redemption requests for the stablecoin."; "At its trough, USDC traded at 86 cents to the dollar."; "USDC's price on the secondary market recovered sharply after the backstop announcement on Sunday and fully recovered once Circle began processing redemptions on Monday, March 13."; "Hourly trading volume shot up to nearly $2 billion on March 11"; "during the crisis, trading on DEXs rose dramatically and accounted for most of the activity."; 전염: "a crypto-collateralized stablecoin, Dai, was operating one-to-one exchange facilities — called Peg Stability Modules ('PSMs') — against USDC and other stablecoins. Over the weekend, these PSMs were rapidly drained of liquidity. Dai, and other stablecoins with otherwise no exposure to SVB, also lost their peg to the dollar."; 범위: "Policy discussions relating to the design of stablecoin regulations are outside of the scope of our study. Future research should explore relevant counterfactuals". 이 표의 「86센트」는 원문이 적은 저점 **가격 수치다**(2023-03-11 주말 중 저점; 원문 각주 9는 시간별 시가 차트에서는 약 90센트로 보인다고 밝힌다). 3장 일러두기의 예외(사건의 규모를 방법론 설명에 쓰는 과거 자료)로 출처·기준일과 함께 한 번만 적는다.

25. Kenechukwu Anadu, Pablo D. Azar, Sean Baker, Marco Cipriani, Thomas M. Eisenbach, Mattia Landoni, Gabriele La Spada, Marco Macchiavelli & J. Christina Wang, "Stablecoins and (Non)Crypto Shocks: A 2026 Update," *Liberty Street Economics*, 2026-07-31. 원문: "On March 11, 2023, Circle reported that approximately 8 percent of USDC's reserves were held at SVB"; "Following SVB's failure, the CRF's WAM fell below that of the median TO fund and has remained below the 5th percentile of the WAM distribution of TO MMFs."; "the CRF's holdings of repurchase agreements (repos) spiked from zero to over 90 percent of its net assets. This share has since retreated some but still stands at 69 percent"; "USDC's bank deposits rose briefly after SVB's failure but have since remained below the levels that prevailed before SVB's failure. ... after SVB's collapse, USDC's bank deposits became, and have remained, concentrated in GSIBs."; "Overall, the SVB event triggered a change in the type of risk held by one of the largest stablecoin issuers, away from interest-rate risk and toward counterparty risk." https://libertystreeteconomics.newyorkfed.org/2026/07/stablecoins-and-noncrypto-shocks-a-2026-update/ (HTML, 2026-10-25 접근·축어 대조). 두 글을 모두 썼다(2026-06-23 글은 합성형, 2026-07-31 글은 준비자산 구성).

26. Rashad Ahmed & Iñaki Aldasoro, "Stablecoins and safe asset prices," BIS Working Papers No 1270 (2025-05, 2026-06 개정), 초록. 원문: "This paper examines the impact of dollar-backed stablecoin flows on short-term US Treasury yields using daily data from January 2021 to March 2026. Using local projections and a granular instrumental variable exploiting idiosyncratic variation in stablecoin growth at the token-blockchain pair level, we find a $3.5 billion (2-standard deviation) inflow lowers 3-month Treasury bill yields by 0.71 basis points on impact, and up to 4 basis points within 10 days, with limited spillovers to longer tenors. Effects are state-dependent, increasing under Treasury market stress and with the scale of the stablecoin sector." (2026-06 개정판). 불확실성은 본문의 충격반응 그림(신뢰구간)으로 제시되고 표의 수치로는 없어, 이 책은 초록의 점추정치만 옮기고 식별 전략을 함께 적는다. 하한 문장 원문(결론부): "our estimates likely constitute a lower bound for outflow effects, as they are identified from a period of secular market growth that affords issuers discretion over the timing of purchases."

27. Du·Sonawane·Watsky (2025), §1.1. 원문: "Solvency concerns around Circle are likely to have played a part."; 회복 두 단계: "USDC's price on the secondary market recovered sharply after the backstop announcement on Sunday and fully recovered once Circle began processing redemptions on Monday, March 13."; 상환 압력: "Pent up redemption pressure"; 발행자 공언: "Public reassurances from Circle that it would 'stand behind USDC and cover any shortfall using corporate resources, involving external capital if necessary,' may have helped to shore up the price of USDC during the weekend, but were not sufficient to restore the peg."

28. Du·Sonawane·Watsky (2025), Summary. 원문: "But had the run unfolded differently, the pressure for USDC

redemptions might have forced Circle to liquidate backing assets (for example, U.S.Treasury securities), with potential knock-on effects on traditional financial markets."

29. Gorton & Zhang (2023), 920쪽. 원문: "In order for the stablecoins to be considered a deposit—let alone a demand deposit—one must determine whether the underlying contract between the holder of stablecoins and the stablecoin issuer is an equity contract or a debt contract. The distinction between equity contracts and debt contracts arose during the 1970s, as critics of money market funds alleged that their business practice of redeeming shares for cash was essentially deposit taking and therefore a violation of the Glass-Steagall Act."

3부 경제학

25장 버블의 계량검정 — 무엇을 재고 무엇을 못 재는가

핵심 TAKEAWAY

「비트코인은 버블이다」라는 문장은 법정에서도 논문에서도 자주 나오지만, 그것이 **검정 가능한 명제가** 되려면 두 가지를 먼저 정해야 합니다. 버블이 무엇과의 차이인가(기초가치), 그리고 그 차이가 어떤 통계적 모양을 띠는가(폭발적 성장). 계량경제학이 내놓은 표준 도구인 PWY·PSY의 우측 단위근 검정(SADF·GSADF)은 둘째 질문에만 답합니다. 가격이 **잠깐 동안 단위근보다 빠르게** 자랐는가를 재고, 그 구간의 시작과 끝에 날짜를 찍습니다. 첫째 질문은 검정이 아니라 **가정입니다**. 같은 자산의 가격을 놓고도 연구들이 — 확률적 버블 모형의 「버블이 가격의 48.7%」(치아–프라이), PWY 검정의 「1% 수준 기각은 비트코인뿐」(하프너), 「기초가치(채굴 한계비용)에서 벗어난 폭발은 없음」(포도르스키) — 으로 갈리는 이유가 거기 있습니다(거래소·기간도 서로 다릅니다). 변동성이 시간에 따라 변하면 교과서 임계값은 믿을 수 없게 되고 — 분산이 커지는 쪽에서는 너무 자주 기각하며, 종목에 따라 반대 방향도 있습니다(하프너) — 기초가치를 어떻게 두느냐에 따라 같은 가격이 버블이기도 아니기도 합니다(포도르스키). 이 장은 실제 가격 자료를 쓰지 않습니다. 폭발 구간을 심은 합성 자료에 검정을 돌려 **검정이 무엇을 잡고 무엇을 놓치는지를** 정답을 알고 봅니다. 「버블 판정」을 증거로 받았을 때 물어야 할 것 — 표본, 기간, 기초가치 가정, 임계값, 변동성 처리 — 이 이 장의 결론입니다.

학습목표

1. 합리적 버블의 정의(가격 − 기초가치)와 그것이 검정에 요구하는 두 가정을 안다. 「버블」이라는 말이 법적 판단에서 무엇을 뜻할 수 있고 무엇을 뜻할 수 없는지 구별한다.
2. [중급] ADF 회귀와 폭발 계수($\beta > 0$, 곧 $\rho > 1$)를 읽고, 우측 단위근 검정이 무엇을 귀무가설로 두는지 안다. 합성 자료로 SADF·GSADF 통계량을 계산해 PSY의 임계값과 견준다.
3. [심화] GSADF 통계량의 이중 재귀, 최소 창 규칙, 날짜 찍기(BSADF)를 읽고, 몬테카를로 임계값과 와일드 부트스트랩 임계값이 왜 다른지 시변 변동성 아래의 크기 왜곡으로 확인한다.
4. 문헌의 버블 판정 6건(철회 1건은 별도)을 표본·기간·기초가치 가정·임계값과 함께 표로 읽고, 결론이 갈리는 자리를 짚는다. 철회된 논문을 인용 계보에서 어떻게 다루는지 안다.

[입문]**만 읽은 독자가 가져가는 것.** 「버블 판정」은 가격이 **잠깐 단위근보다 빠르게 자랐다는** 통계이고, 기초가치가 무엇인가는 검정이 아니라 가정입니다.

1. 이 장이 답하려는 질문

19장은 아무것도 약속하지 않는 자산의 가격이 왜 0이 아닐 수 있는지를 모형으로 보았고, 그 모형들은 가격의 궤적이 하나가 아니라 여럿임을 — 균형의 다수성을 — 말했습니다. 그 가운데 어떤 궤적을 「버블」이라 부릅니다. 블랑샤르와 왓슨은 1982년에 정의를 분명히 했습니다. 「합리적 행동과 합리적 기대는 자산 가격이 기초가치와 같아야 함을 함축하지 않는 경우가 많다. 바꿔 말해 가격이 그 가치에서 **합리적으로** 벗어날 수 있다 — 합리적 버블이다.」 그리고 두 번째 질문을 바로 붙였습니다. 「특정 시장에 버블이 존재하는지를 통계적으로 탐지할 수 있는가. 가격과 수익 양쪽의 자료가 있으면 과제가 훨씬 쉽다. … 금처럼 비금전적 요소 때문에 수익을 관측하기 어려운 시장에서는 과제가 더 어렵다. 우리는 「연(run) 검정」과 「꼬리 검정」의 사용을 검토하고, 그것들이 기껏해야 **정황 증거를** 준다고 결론짓는다.」[1] 비트코인은 수익(배당)이 없는 자산이므로 블랑샤르–왓슨의 「더 어려운 쪽」에 있습니다. 이 장은 그 어려운 쪽에서 40년 뒤의 계량경제학이 무엇을 할 수 있게 되었는지, 그리고 여전히 무엇을 가정해야 하는지를 봅니다.

법률가에게 이 질문이 중요한 이유는 하나입니다. 「버블이었다」는 판정이 사기·시세조종·설명의무 위반 사건에서 **증거로** 제출될 수 있기 때문입니다. 그 판정이 무엇을 재고 무엇을 가정했는지를 모르면, 통계가 아니라 수사(修辭)를 증거로 받게 됩니다.

2. 버블의 정의가 요구하는 두 가정

티롤은 1985년에 버블을 「시장가격과 기초가치의 차이」로 정의하고, 「새뮤얼슨이 소비대부 모형을 개발한 이래, 화폐가 본질적으로 무용함에도(곧 기초가치가 0임에도) 양의 가치를 갖는 경제가 존재한다는 것은 잘 알려져 있다. 바꿔 말해 화폐에 버블이 존재할 수 있다」고 적었습니다.[2] 이 정의에서 검정의 두 가정이 나옵니다.

가정	내용	비트코인에서의 문제
기초가치 가정	버블은 차이이므로, 빼는 쪽(기초가치)이 있어야 한다	배당이 없다. 연구마다 다른 것을 뺀다 — 모형 안에서 추정하니 0과 구별되지 않음(치아–프라이), 채굴 한계비용(포도르스키), 활성 사용자 수의 멱함수(휘틀리 외), 또는 아예 빼지 않고 가격 자체를 본다(PSY 계열 대부분)
통계적 모양 가정	버블 구간에서 가격은 **단위근보다 빠르게**(폭발적으로) 자란다	기초가치 자체가 폭발적이면(채굴 난이도처럼) 가격의 폭발은 버블의 증거가 아니다

둘째 가정이 왜 그런지는 19장의 식에서 바로 나옵니다. 블랑샤르–왓슨의 식 (1)에서 버블 항은 $\mathbb{E}(c_{t+1} \mid \Omega_t) = (1+r)\,c_t$ 를 만족해야 합니다 — 기대값이 매기 $(1+r)$ 배로 자라야 보유자가 그것을 들고 있을 유인이 생깁니다. 단위근 과정은 기대값이 그대로이고($\mathbb{E}(y_{t+1}) = y_t$), 버블은 기대값이 **커집니다**. 그래서 버블 검정은 「계수가 1인가」가 아니라 「계수가 1보다 **큰가**」를 묻는 **우측** 단위근 검정이 됩니다. PSY의 표현으로 「우측 단위근 검정은 … 자료의 온건한 폭발 또는 하위마팅게일 행동에 대해 정보를 주며, 따라서 시장 진단이나 경보의 한 형태로 유용하다.」[3]

3. [중급] ADF 회귀와 폭발 계수

[중급] 출발점은 보통의 단위근 회귀입니다. PSY의 식 (4)는 표본의 r_1 분위에서 r_2 분위까지의 창에서

$$\Delta y_t = \hat{\alpha}_{r_1, r_2} + \hat{\beta}_{r_1, r_2} y_{t-1} + \sum_{i=1}^{k} \hat{\psi}^{i}_{r_1, r_2} \Delta y_{t-i} + \hat{\varepsilon}_t$$

를 추정하고, 그 창의 ADF 통계량 $ADF_{r_1}^{r_2}$ 을 β 의 t 비율로 둡니다.[4] 기호 풀이: y_t 는 (로그) 가격, $\Delta y_t = y_t - y_{t-1}$, k 는 단기 동학의 시차 수, $r_w = r_2 - r_1$ 은 창의 너비(표본 비율). 귀무가설은 $\beta = 0$ (단위근, 약한 표류 허용), 대립가설은 $\beta > 0$(폭발)입니다. 수준으로 적으면 $y_t = \alpha + \rho y_{t-1} + \cdots$ 에서 $\rho = 1 + \beta$ 이고 $\rho = 1$ 대 $\rho > 1$ 의 검정과 같은 t 값입니다. 보통의 ADF 검정이 **왼쪽**($\beta < 0$, 정상성)을 보는 것과 반대입니다. 수치 예: $\rho = 1.04$ 인 구간이 30기 이어지면 기대값은 $1.04^{30} \approx 3.24$ 배가 됩니다 — 「온건한 폭발」이 이런 크기입니다.

[중급] PWY(2011)의 검정은 창의 **시작을 표본 처음에 고정하고 끝을 늘려 가며** ADF 통계량을 계산해 그 최댓값을 취합니다.

$$SADF(r_0) = \sup_{r_2 \in [r_0, 1]} ADF_0^{r_2}$$

r_0 은 첫 창의 너비입니다.[5] 창을 늘려 가는 이유는 간단합니다. 표본 전체로 한 번만 회귀하면 짧은 폭발 구간은 긴 단위근 구간에 묻혀 보이지 않지만, 폭발 구간이 끝나는 자리에서 끝나는 창은 그것을 잡습니다. 그런데 한 가지 문제가 남습니다. 창의 시작이 표본 처음에 고정되어 있으므로, **첫 번째 버블이 꺼진 뒤의 두 번째 버블은** 그 앞의 붕괴 때문에 잘 잡히지 않습니다. 붕괴를 포함한 표본은 「가짜 정상성」을 띠기 때문입니다. PSY 2015의 기여가 그 문제를 푼 것이고, 다음 절입니다.

[중급] 아래 세 셀은 **방법론 시연입니다** — 폭발 구간을 심은 합성 자료에 검정을 돌려 무엇을 잡고 무엇을 놓치는지를 정답을 알고 봅니다. 실제 시세 자료는 쓰지 않습니다.

```
# NB25-1: 합성 자료에 PWY/PSY 검정 — 폭발 구간을 심은 가짜 가격에 SADF·GSADF 를 계산한다 (방법론 시연. 실제
시세 자료는 쓰지 않는다)
import numpy as np

rng = np.random.default_rng(25)      # 시드 고정 — 같은 결과가 재현된다

def make_series(T=200, bubble=(100, 130), beta=1.04, sigma=1.0, collapse_to=None):
    """표류 없는 랜덤워크에 [bubble[0], bubble[1]) 구간만 y_t = beta·y_{t-1} + e_t 로 자라게 심고, 끝나면
심기 전 수준 근처로 떨어뜨린다."""
    y = np.empty(T); y[0] = 30.0
    e = rng.normal(0.0, sigma, T)
    for t in range(1, T):
        if bubble[0] <= t < bubble[1]:
            y[t] = beta * y[t - 1] + e[t]
        elif t == bubble[1]:
            y[t] = (collapse_to if collapse_to is not None else y[bubble[0] - 1]) + e[t]
        else:
            y[t] = y[t - 1] + e[t]
    return y

def adf_grid(y, min_win):
    """모든 창 [s, e] 에 대한 시차 0 ADF t-통계량(Δy_t 를 상수와 y_{t-1} 에 회귀, y_{t-1} 계수의 t 비율)을
한꺼번에 계산한다.
    돌려주는 값: (T-1)×(T-1) 배열 A, A[s, e] = 창이 관측 s+1 … e+1 의 Δy 를 쓰는 회귀의 t. 너무 짧은 창은
NaN."""
    x = y[:-1]; z = np.diff(y)                    # x_t = y_{t-1}, z_t = Δy_t, t = 1 … T-1
    n = len(z)
    def ps(a):                                    # 앞에 0 을 붙인 누적합 — 구간합을 뺄셈 하나로
        return np.concatenate(([0.0], np.cumsum(a)))
    Px, Pz, Pxx, Pxz, Pzz = ps(x), ps(z), ps(x * x), ps(x * z), ps(z * z)
    s = np.arange(n)[:, None]; e = np.arange(n)[None, :]
    cnt = (e - s + 1).astype(float)
    Sx = Px[e + 1] - Px[s]; Sz = Pz[e + 1] - Pz[s]
    Sxx = Pxx[e + 1] - Pxx[s]; Sxz = Pxz[e + 1] - Pxz[s]; Szz = Pzz[e + 1] - Pzz[s]
    with np.errstate(invalid="ignore", divide="ignore"):
        den = cnt * Sxx - Sx * Sx
        beta = (cnt * Sxz - Sx * Sz) / den
        alpha = (Sz - beta * Sx) / cnt
        rss = Szz - alpha * Sz - beta * Sxz
        s2 = rss / (cnt - 2)
        t = beta / np.sqrt(s2 * cnt / den)
    t[(cnt < min_win) | (e < s)] = np.nan
    return t

def sadf_gsadf(y, r0):
    T = len(y); w = int(np.floor(r0 * T))
    A = adf_grid(y, w)
    sadf = np.nanmax(A[0, :])                     # 시작 고정(0), 끝을 늘림
    bsadf = np.nanmax(A, axis=0)                  # 끝 r2 마다 시작을 전부 훑은 최댓값 — 날짜 찍기용
    gsadf = np.nanmax(bsadf)                      # 모든 (r1, r2) 의 최댓값
    return sadf, gsadf, bsadf, A

T = 200
r0 = 0.01 + 1.8 / np.sqrt(T)                      # PSY 의 최소 창 규칙 → 0.137 (T = 200)
y = make_series(T)
sadf, gsadf, bsadf, A = sadf_gsadf(y, r0)
PSY_T200 = {"SADF 95%": 1.40, "SADF 99%": 1.90, "GSADF 95%": 2.08, "GSADF 99%": 2.70}   # PSY
(2015) Table 1, 유한표본, T=200, r0=0.137
print(f"합성 자료: T = {T}, 폭발 구간 t ∈ [100, 130), β = 1.04, 그 뒤 붕괴. 최소 창 r0 = {r0:.3f} →
{int(np.floor(r0*T))} 관측")
print(f"SADF  = {sadf:.3f}   (PSY 표 1 임계값 95% {PSY_T200['SADF 95%']:.2f}, 99% {PSY_T200['SADF
99%']:.2f})")
print(f"GSADF = {gsadf:.3f}   (PSY 표 1 임계값 95% {PSY_T200['GSADF 95%']:.2f}, 99%
{PSY_T200['GSADF 99%']:.2f})")
print(f"전체 표본 한 번의 ADF(시작 0, 끝 T) = {A[0, -1]:.3f}  ← 폭발 구간이 긴 단위근 구간에 묻혀 보이지
않는다")
peak = int(np.nanargmax(bsadf)) + 1
print(f"BSADF 가 최대인 끝점 t = {peak} (심은 구간의 끝 130 직전이면 잘 잡은 것)")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
합성 자료: T = 200, 폭발 구간 t ∈ [100, 130), β = 1.04, 그 뒤 붕괴. 최소 창 r0 = 0.137 → 27 관측
SADF  = 11.918   (PSY 표 1 임계값 95% 1.40, 99% 1.90)
GSADF = 12.361   (PSY 표 1 임계값 95% 2.08, 99% 2.70)
전체 표본 한 번의 ADF(시작 0, 끝 T) = -2.466  ← 폭발 구간이 긴 단위근 구간에 묻혀 보이지 않는다
BSADF 가 최대인 끝점 t = 129 (심은 구간의 끝 130 직전이면 잘 잡은 것)
```

[중급] 첫 셀이 보여 주는 것은 셋입니다. 표본 전체로 한 번 돌린 ADF는 폭발을 보지 못합니다. 창을 늘려 가는 SADF와 모든 창을 훑는 GSADF는 PSY의 유한표본 임계값(T = 200, $r_0 = 0.137$ 에서 GSADF 95% 2.08, 99% 2.70)을 넘습니다.[6] 그리고 BSADF가 최대인 끝점이 심은 구간의 끝 근처에 있습니다. 이 모든 것은 **정답을 알고** 본 것입니다. 실제 자료에서는 정답이 없고, 그래서 다음 두 절이 필요합니다.

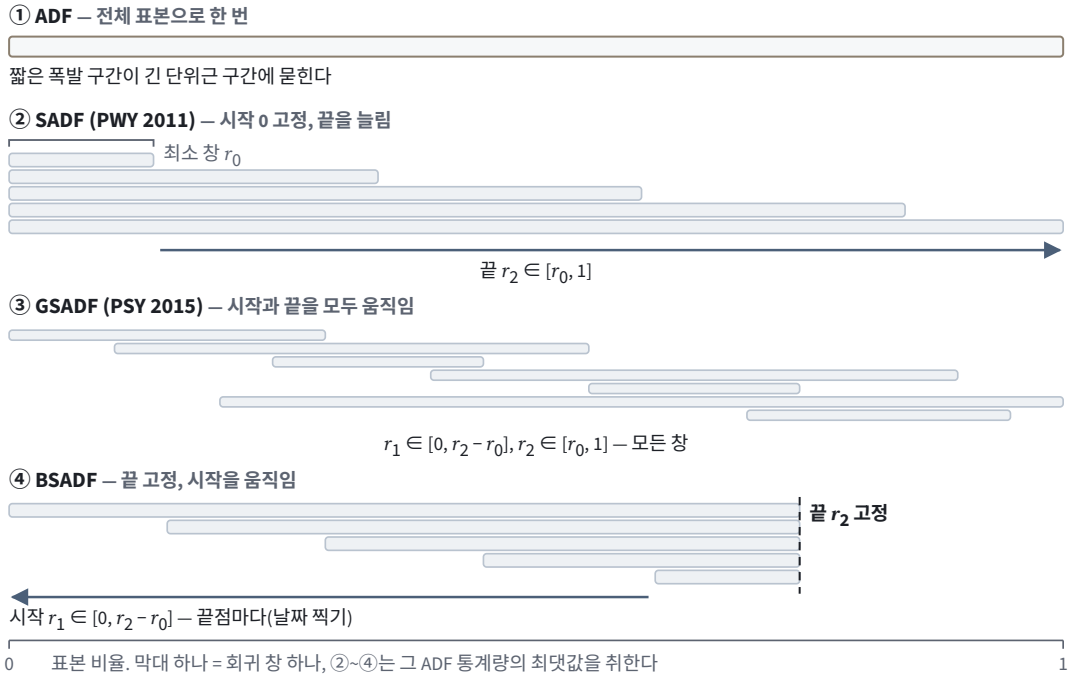


그림 25-1. 네 통계량이 회귀 창을 놓는 방식. 가로축은 표본 비율이고 막대 하나가 회귀 창 하나다. 막대의 위치는 배치를 보이는 예시다.

4. [심화] GSADF, 최소 창, 날짜 찍기, 그리고 임계값

▾ [심화] 이중 재귀와 날짜 찍기

[심화] PSY의 통계량은 창의 시작과 끝을 **모두** 움직입니다.

$$GSADF(r_0) = \sup_{\substack{r_2 \in [r_0, 1] \\ r_1 \in [0,\, r_2 - r_0]}} ADF_{r_1}^{r_2}$$

정리 1은 귀무가설(점근적으로 무시할 수 있는 표류를 가진 랜덤워크) 아래 이 통계량의 극한분포를 위너 과정의 범함수로 주고, 그 분포는 최소 창 r_0 에 의존합니다.[7] 그래서 임계값 표가 r_0 별로 있고, r_0 이 작아질수록 임계값이 커집니다 — 「r_0 이 0.190에서 0.055로 줄면 GSADF의 95% 점근 임계값이 1.89에서 2.30으로 오른다.」 창을 더 많이 훑을수록 우연히 큰 값이 나올 기회도 많아지기 때문입니다. 같은 이유로 GSADF의 임계값은 SADF보다 큽니다(「$r_0 = 0.10$ 에서 95% 점근 임계값은 GSADF 2.19, SADF 1.49」).[8]

[심화] 날짜 찍기. 「이 관측이 버블 구간에 속하는가」를 실시간으로 묻는 것이 규제자의 질문이고, PSY는 그것을 **후향 SADF로** 답합니다. 끝점 r_2 를 고정하고 시작점을 0 부터 $r_2 - r_0$ 까지 움직여 ADF 통계량의 최댓값을 취한 것이 $BSADF_{r_2}(r_0)$ 이고, PWY의 재귀 ADF(시작 고정)는 그 특수 사례입니다.[9] 시작일은 $BSADF_{r_2}$ 가 임계값을 처음 넘는 관측, 끝일은 그 뒤 L_T 만큼 지나서 처음 임계값 아래로 내려가는 관측입니다. $L_T = \log(T)$ 같은 최소 지속 조건은 「추정된 자기회귀 계수의 짧은 깜빡임을 제외하기 위한」 것이고, 자료 빈도에 맞춰 조정합니다.[10] 둘째 셀이 몬테카를로로 임계값 열을 만들고 날짜를 찍습니다.

```
# NB25-2: 몬테카를로 임계값과 날짜 찍기 — 귀무(랜덤워크)를 많이 만들어 GSADF 의 95%·99% 분위수와 BSADF 임계값
열을 얻고, 합성 자료에 날짜를 찍는다 (방법론 시연)
def mc_critical_values(T, r0, reps=2000, seed=2025):
    g = np.random.default_rng(seed)
    w = int(np.floor(r0 * T))
    gs = np.empty(reps); bs = np.empty((reps, T - 1))
    for i in range(reps):
        yn = 30.0 + np.cumsum(g.normal(0.0, 1.0, T))           # 귀무: 표류 없는 랜덤워크
        A = adf_grid(yn, w)
        bs[i] = np.nanmax(A, axis=0); gs[i] = np.nanmax(bs[i])
    return gs, bs

gs_null, bs_null = mc_critical_values(T, r0)
cv95, cv99 = np.quantile(gs_null, [0.95, 0.99])
print(f"귀무 랜덤워크 {len(gs_null)}회 → GSADF 임계값 95% {cv95:.2f}, 99% {cv99:.2f}   (PSY 표 1:
2.08, 2.70 — 이 셀은 반복 2,000회·시차 0·표류 없음, PSY 는 약한 표류 d=η=1)")
print(f"합성 자료의 GSADF {gsadf:.3f} → 95% 기준 {'기각(폭발 있음)' if gsadf > cv95 else '기각 못
함'}, 99% 기준 {'기각' if gsadf > cv99 else '기각 못 함'}")

# 날짜 찍기: 끝점 t 마다 귀무 BSADF_t 의 95% 분위수를 임계값으로 쓴다(PSY 는 [T r2] 관측 기준 SADF 임계값을
쓰며, 여기서는 같은 몬테카를로에서 직접 얻은 열로 대신한다)
bs_cv95 = np.nanquantile(bs_null, 0.95, axis=0)
LT = int(np.ceil(np.log(T)))                        # 최소 지속 log(T) ≈ 5.3 → 6 관측
flag = bsadf > bs_cv95
episodes, start = [], None
for t, f in enumerate(flag):
    if f and start is None: start = t
    if (not f or t == len(flag) - 1) and start is not None:
        end = t if not f else t + 1
        if end - start >= LT: episodes.append((start + 1, end + 1))   # +1: Δy 인덱스 → 원 시계열
시점
        start = None
print(f"임계값 열(끝점별 BSADF 95%)로 찍은 폭발 구간(최소 지속 {LT} 관측): {episodes}")
print(f"심은 구간: [100, 130). 시작은 몇 관측 늦게, 끝은 붕괴 직후에 잡히는 것이 보통이다 — 검정은 「자라는 중」
을 보지 「자라기 시작했다」를 보지 못한다.")
print("심은 구간 밖에 잡힌 구간이 있으면 그것은 거짓 양성이다 — 끝점마다 5% 를 허용하면 170여 개 끝점 가운데
어딘가는 우연히 넘는다(올다니 외가 말한 다중성 문제).")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
귀무 랜덤워크 2000회 → GSADF 임계값 95% 2.11, 99% 2.65   (PSY 표 1: 2.08, 2.70 — 이 셀은 반복 2,000회·
시차 0·표류 없음, PSY 는 약한 표류 d=η=1)
합성 자료의 GSADF 12.361 → 95% 기준 기각(폭발 있음), 99% 기준 기각
임계값 열(끝점별 BSADF 95%)로 찍은 폭발 구간(최소 지속 6 관측): [(49, 63), (102, 130)]
심은 구간: [100, 130). 시작은 몇 관측 늦게, 끝은 붕괴 직후에 잡히는 것이 보통이다 — 검정은 「자라는 중」을 보지
「자라기 시작했다」를 보지 못한다.
심은 구간 밖에 잡힌 구간이 있으면 그것은 거짓 양성이다 — 끝점마다 5% 를 허용하면 170여 개 끝점 가운데 어딘가는
우연히 넘는다(올다니 외가 말한 다중성 문제).
```

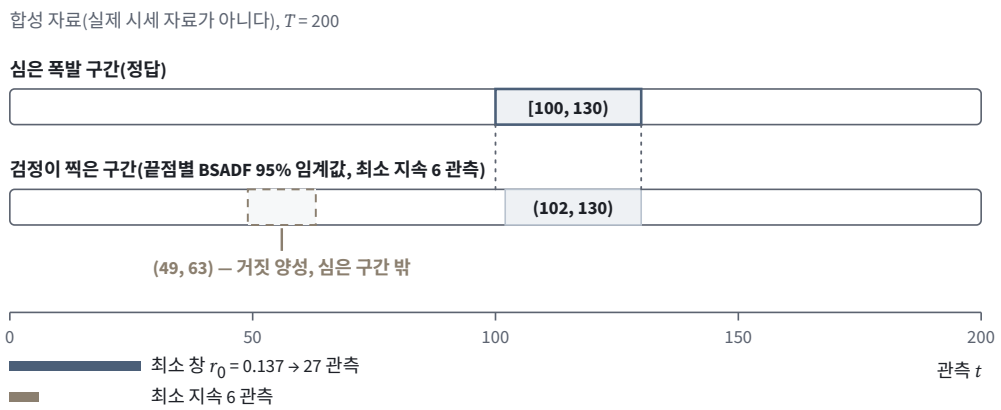


그림 25-2. 합성 자료(실제 시세 자료가 아님)에서 심은 폭발 구간과 검정이 찍은 구간. 구간과 관측 수는 NB25-1·NB25-2의 출력이고, 가격 곡선은 그리지 않았다.

▾ [심화] 변동성이 변하면 — 와일드 부트스트랩

[심화] 교과서 임계값은 오차의 분산이 일정하다는 가정에서 나옵니다. 암호자산 가격은 그렇지 않습니다. 하프너는 2020년 논문에서 이 점을 정면으로 다룹니다. 「하비(Harvey) 외(2016)가 보인 대로, PWY의 원래 검정은 비정상 변동성이 있으면 편향된다.」 하비 외는 「PWY 검정의 크기를 교정하는 와일드 부트스트랩 절차를 제안하고 그것이 좋은 검정력 성질을 가짐을 보였고」, 하프너는 그 접근을 확률적 변동성까지 넓혀 암호자산 11종목과 CRIX 지수에 적용합니다. 결과는 **방향이 하나가 아닙니다**. 「전역 이분산을 고려하면 0에 매우 가까웠던 p값들이 커지는 효과가 있다. … 바꿔 말해 투기적 버블을 지지하는 증거는 이 효과를 무시했을 때보다 훨씬 덜 뚜렷하다」 — 그러나 「모든 p값이 오르는 것은 아니며, 특히 큰 것들은 교정 뒤 오히려 내려가는 경향이 있다.」 비트코인은 PWY 통계량 5.0347, 교과서 p값 0.0001이 와일드 부트스트랩 p값 **0.0032로** — 여전히 1%에서 기각되지만 — 올라가고, 「1% 수준에서 투기적 버블을 지지해 귀무가설을 기각하는 것은 비트코인 하나뿐」이며 5%에서는 12계열(11종목 + CRIX) 중 9 — 종목만 세면 11 중 8 — 로 늘어납니다.[11] 올다니와 공저자들은 2025년에 PSY의 임계값이 「두 추론 문제 — ADF 오차의 시변 무조건 이분산, 그리고 (검정하는 가설의 수와 함께 거짓 양성 확률이 커지는) 다중성 문제 — 에 강건하지 않다」고 적고, 필립스–시(2020)의 복합 와일드 부트스트랩으로 임계값을 만듭니다.[12] 셋째 셀이 이 크기 왜곡을 합성 자료로 보여 줍니다.

```
# NB25-3: 시변 변동성 아래의 크기 왜곡과 와일드 부트스트랩 — 버블이 없는데 분산만 커지는 귀무 자료에 교과서
임계값을 쓰면 얼마나 자주 「버블」이라 하는가 (방법론 시연)
def null_with_vol_break(T, g, s1=1.0, s2=3.0, brk=None):
    brk = T // 2 if brk is None else brk
    sig = np.where(np.arange(T) < brk, s1, s2)
    return 30.0 + np.cumsum(g.normal(0.0, 1.0, T) * sig)

g = np.random.default_rng(2526); reps = 400; w = int(np.floor(r0 * T))
rej_const = rej_break = 0
for i in range(reps):
    y0 = 30.0 + np.cumsum(g.normal(0.0, 1.0, T))                        # 분산 일정
    y1 = null_with_vol_break(T, g)                                       # 중간에 분산 9배(표준편차
3배)
    rej_const += np.nanmax(adf_grid(y0, w)) > cv95
    rej_break += np.nanmax(adf_grid(y1, w)) > cv95
print(f"귀무 자료 {reps}개씩, 몬테카를로 95% 임계값 {cv95:.2f} 사용 — 명목 크기 5%")
print(f"  분산 일정:             기각률 {rej_const / reps:.1%}")
print(f"  중간에 표준편차 3배:  기각률 {rej_break / reps:.1%}   ← 버블이 하나도 없는데 이만큼 「버블」이라
한다")

# 와일드 부트스트랩(하프너·필립스-시 방식의 뼈대): 관측 자료의 잔차에 ±1 또는 정규 가중을 곱해 다시 만든 귀무
표본에서 임계값을 얻는다 — 분산의 시간 변화가 그대로 보존된다
def wild_bootstrap_cv(y, w, B=199, seed=7):
    gb = np.random.default_rng(seed)
    resid = np.diff(y) - np.mean(np.diff(y))                             # 귀무(단위근) 아래의 잔차:
Δy 에서 평균을 뺀 것
    stats = np.empty(B)
    for b in range(B):
        eb = resid * gb.normal(0.0, 1.0, len(resid))                     # 와일드: 각 잔차에 독립
N(0,1) 을 곱한다
        yb = y[0] + np.concatenate(([0.0], np.cumsum(eb)))
        stats[b] = np.nanmax(adf_grid(yb, w))
    return np.quantile(stats, 0.95)

# 교과서 임계값으로는 「버블」로 판정되는 귀무 자료를 하나 고른다(시드 0 부터 차례로 — 재현 가능)
for sd in range(200):
    y_break = null_with_vol_break(T, np.random.default_rng(sd))
    stat = np.nanmax(adf_grid(y_break, w))
    if stat > cv95: break
cv_wb = wild_bootstrap_cv(y_break, w)
print(f"\n분산이 변하는 귀무 자료 하나(시드 {sd}): GSADF {stat:.2f}. 교과서 임계값 {cv95:.2f} → {'기각'
if stat > cv95 else '기각 못 함'}; 와일드 부트스트랩 95% 임계값 {cv_wb:.2f} → {'기각' if stat > cv_wb
else '기각 못 함'}")
print("부트스트랩 임계값은 자료의 변동성 구조를 따라 움직인다. 같은 통계량이 한 임계값으로는 「버블」, 다른
임계값으로는 「아님」이 될 수 있다.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
귀무 자료 400개씩, 몬테카를로 95% 임계값 2.11 사용 — 명목 크기 5%
  분산 일정:             기각률 4.8%
  중간에 표준편차 3배:  기각률 34.0%   ← 버블이 하나도 없는데 이만큼 「버블」이라 한다

분산이 변하는 귀무 자료 하나(시드 0): GSADF 2.32. 교과서 임계값 2.11 → 기각; 와일드 부트스트랩 95% 임계값 3.45
→ 기각 못 함
부트스트랩 임계값은 자료의 변동성 구조를 따라 움직인다. 같은 통계량이 한 임계값으로는 「버블」, 다른 임계값으로는
「아님」이 될 수 있다.
```

5. 기초가치를 무엇으로 두는가 — 세 답

지금까지는 둘째 가정(폭발적 모양)의 검정이었습니다. 첫째 가정(무엇을 빼는가)은 검정이 아니라 선택이고, 문헌은 세 가지를 골랐습니다.

추정하니 0과 구별되지 않는다. 치아(Eng-Tuck Cheah)와 프라이(John Fry)의 2015년 논문은 확률적 버블 모형에서 붕괴 크기 매개변수 v를 추정하고 「$v = 0$」을 귀무가설로 검정합니다. 2013년 1월 1일부터 11월 30일까지의 자료에서 $v = 0.546$(표준오차 0.090, $t = 6.06$), 장기 기초가치의 성장률

$\tilde{\mu} = 0.00166$(표준오차 0.00195, $p = 0.394$)이었고, 저자들은 「버블 동안 가격 상승이 너무 극적이어서 추정된 장기 기초가치가 0과 통계적으로 다르지 않다」며 「식 (16)에 따르면 버블이 관측 가격의 약 48.7%를 차지한다」고 적었습니다. 결론 셋째 항은 「비트코인의 기초가치는 0이다」입니다.[13] 이 결론은 모형 안의 명제입니다. 기초가치의 **성장률이** 0과 구별되지 않는다는 추정으로부터 기초가치 **수준이** 0이라는 결론으로 가는 데는 모형의 식 (12)·(17)이 필요하고, 그 식은 가정입니다.

채굴 한계비용으로 둔다. 포도르스키(Andrea Podhorsky)의 2024년 논문은 20장의 세계에서 기초가치를 끌어냅니다. 프로토콜이 난이도를 조정해 공급을 관리하므로 「목표 공급량을 채굴하는 한계비용」이 모형이 함의하는 기초가치이고, 그 기초가치 자체가 **폭발적일 수 있습니다**. 「비트코인의 변동성 크고 폭발적인 가격 경로는 비트코인 프로토콜의 공급 관리 체계의 귀결이다.」 실증은 세 단계입니다. 가격과 기초가치 모두 단위근(ADF −2.51, $p = .36$; −1.90, $p = .62$), 둘은 공적분(엥글–그레인저 −2.74, $p = .071$, 10% 수준), 그리고 PSY 검정을 **가격에** 돌리면 SADF 3.7·GSADF 3.9로 1% 임계값(2.2·2.9)을 넘지만, 가격에서 기초가치를 뺀 **잔차에** 돌리면 −.08·1.52로 10% 임계값(1.3·2.2)에도 못 미칩니다. 「원가격 자료는 버블의 존재를 가리키지만 잔차는 그렇지 않으므로 … 비트코인 가격의 외견상 버블은 기초가치에 돌릴 수 있다.」[14] 같은 가격, 다른 기초가치, 반대 결론입니다.

네트워크 크기로 둔다. 휘틀리(Spencer Wheatley)와 공저자들(소르네트 포함)은 2019년에 「일반화된 메트칼프 법칙」 — 시가총액이 활성 사용자 수의 멱함수 — 을 기초가치로 두고, 거기서 크게 벗어난 네 구간을 버블로 식별한 뒤 LPPLS(로그주기 멱법칙 특이점) 모형으로 붕괴 시점의 구간을 예측하려 했습니다. 추정된 지수는 2가 아니라 약 1.69(「각 사용자가 평균적으로 $N^{2/3}$ 명과 연결된 더 성긴 네트워크」)이고, 네 버블은 2012년 5–8월(84일), 2013년 1–4월(98일), 2013년 10–11월(47일), 2015년 6월–2017년 12월(924일)입니다(표의 시가총액·성장 배수 열은 옮기지 않습니다).[15] 저자들 스스로 모형 불확실성을 길게 적습니다 — 적합 창을 바꾸면 「예측된 수용 능력이 4자릿수 차이」가 나고, 「단일 모형에 기초한 정밀한 추론은 … 오도적이며 장기 예측은 사실상 무의미하다.」[16] 이 책은 그들의 시가총액 수치와 「현재 과대평가」 판단을 옮기지 않습니다(3장 일러두기). 가져가는 것은 구조입니다. 기초가치를 사용자 수로 두면, 버블은 「사용자 수가 정당화하는 것보다 비싼 구간」이 되고, 그 판단은 멱함수의 지수 하나에 걸립니다.

6. 문헌의 판정표 — 표본·기간·가정·임계값을 함께

[중급] 이 장이 읽은 실증 연구의 판정을 한 표에 둡니다. **판정만 떼어 옮기지 않습니다.** 표본·기간·기초가치 가정·임계값이 다르면 같은 「기각」도 다른 명제입니다. 수치는 모두 **과거 자료에 대한 검정 통계량이지** 가격이 아니며, 이 책은 가격 수준·수익률·시가총액을 옮기지 않았습니다.

연구	자료·표본·기간	기초가치 가정	방법·임계값	판정	이 책의 주석
Cheah & Fry (2015)	CoinDesk 지수 일별, 2010-07-18~2014-07-17; 검정은 2013-01-01~11-30	0 (모형 안에서 추정 → 0과 구별 안 됨)	확률적 버블 모형, $H_0{:}v = 0$	$v = 0.546$ (SE 0.090), 버블이 가격의 약 48.7%	「기초가치 0」은 모형의 식 (12)·(17)에 걸린 결론
Cheung·Roca·Su (2015)	Mt. Gox USD 일별, 2010-07-17~2014-02-18, N = 1,307	없음(로그가격 자체)	PSY GSADF, $r_0 \approx 2\%$(20 관측), 99% 임계값	99% 임계값 초과 33회; 큰 버블 셋 — 2011-04-24~07-03(논문 보고 66일; 달력으로는 양끝 포함 71일, 차이의 원인 미확인), 2013-01-27~04-15(79일), 2013-11-05~2014-02-18(106일)	거래소 한 곳의 가격; 셋째는 표본 끝(거래소 폐쇄)에서 끊김
Hafner (2020)	2017-12-31 기준 상위 11종목 + CRIX, BTC 는 2013-04-28부터 1,709 관측	없음	PWY SADF, $r_0 = 0.1$; 몬테카를로 p값 vs 와일드 부트스트랩(5,000회)	BTC 5.0347: p 0.0001 → WB p 0.0032 (1%에서 기각되는 유일한 종목); 5%에서는 12계열(11종목 + CRIX) 중 9	시변 변동성을 넣으면 증거가 「훨씬 덜 뚜렷」 — 단 큰 p값은 오히려 내려가기도
Phillips·Shi·Yu (2015)	일별 1,764 관측 (기간은 본문 §4), 최소 창 93	목표 공급량의 채굴 한계비용 (가격과 공적분, EG −2.74, p .071)	PSY SADF·GSADF, 몬테카를로 2,000회	가격: 3.7·3.9 > 1% (2.2·2.9). 잔차: −.08·1.52 < 10% (1.3·2.2)	같은 가격이 기초가치를 바꾸면 「버블 아님」
Oldani·Bruno·Signorelli (2025)	BTC·ETH·XRP 일별 로그가격, 2018-01-01~2024-06-14, T = 2,357, $\lfloor r_0 T \rfloor = 110$	없음	PSY GSADF, 필립스–시 복합 와일드 부트스트랩(B = 499, T_b = 183일)	BTC 3.709, ETH 2.289, XRP 3.966 — 모두 1% 부트스트랩 임계값 (2.086·2.124·2.104) 초과. BTC 네 구간(2018-11, 2019-05~06, 2020-12~2021-03, 2024-02~03)	XRP 는 통계량은 넘지만 「짧은 단일 구간」(2020-11-23~25)뿐이고, 시차 7에서는 「전혀 지지하지 않으며」, 주별 자료에서는 10%에 그쳐, 초록은 「XRP 의 과열을 지지하는 검정은 없다」고 쓴다 — 본문·표·초록 사이의 이 간극은 독자가 직접 확인할 자리
Wheatley 외 (2019)	Bitstamp USD 시간별, 2012-01-01~2018-01-08; 활성 사용자 수	일반화된 메트칼프 법칙(지수 ≈ 1.69)	기초가치 대비 괴리 + LPPLS	버블 넷(84·98·47·924일; 배수는 옮기지 않음)	기초가치가 멱함수 지수 하나에 걸림; 저자들이 모형 불확실성 명시
Corbet·Lucey·Yarovaya (2018)	—	—	—	**철회됨** (철회 공지 FRL 91: 109483, 2026-03)	이 책은 근거로 쓰지 않는다. 철회 사유 미확인

17 18 19

[중급] 표에서 가져갈 것은 판정의 합이 아니라 **갈리는 자리입니다.** 같은 비트코인 가격에 대해 (가) 기초가치를 빼지 않으면 대부분의 연구가 폭발 구간을 찾고, (나) 채굴 한계비용을 빼면 찾지 못하며, (다) 변동성을 교정하면 증거가 약해지되 비트코인은 남고, (라) 거래소·빈도·기간이 바뀌면 구간의 수와

길이가 바뀝니다. 이 넷 가운데 어느 것도 「버블인가」에 대한 답이 아니라 「무엇을 버블이라 부르기로 했는가」에 대한 답입니다.

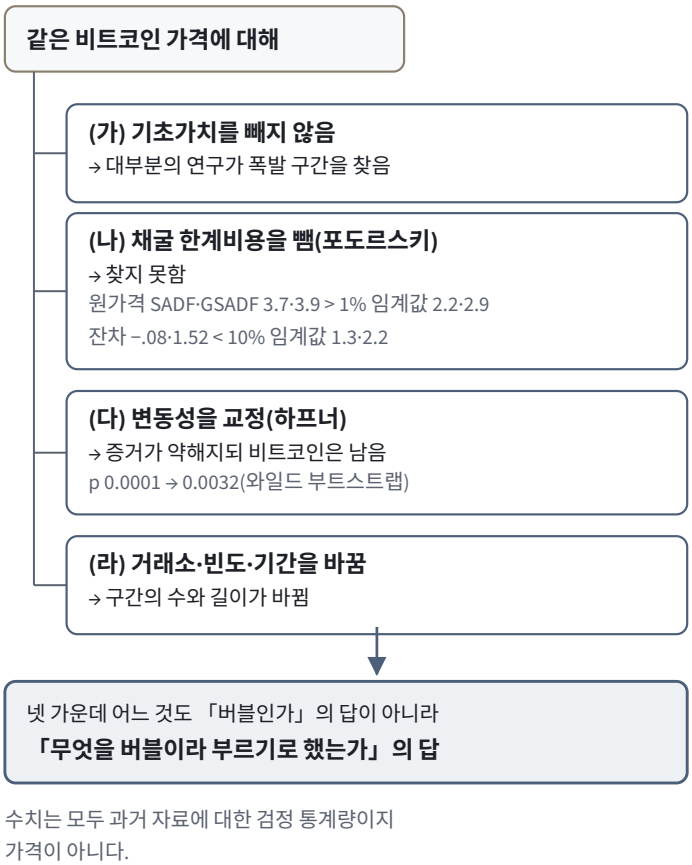


그림 25-3. 같은 비트코인 가격에 대해 기초가치·변동성·자료의 가정이 바뀌면 판정이 갈리는 자리. 연구마다 거래소·기간은 서로 다르다. 그림 속 수치는 모두 검정 통계량이다.

7. 말하는 것과 말하지 않는 것

우측 단위근 검정이 **말하는 것**: 어떤 구간에서 (로그)가격의 자기회귀 계수가 1보다 유의하게 컸다. 그 구간의 시작과 끝을 날짜로 추정할 수 있다. 복수의 구간도 잡을 수 있다(PSY). 변동성이 변하면 임계값을 자료에서 다시 만들어야 한다(하프너·필립스–시).

말하지 않는 것: 왜 그랬는가(수요 충격인지, 기초가치의 변화인지, 조작인지). 누가 그랬는가(검정은 가격 계열 하나만 본다). 그 뒤에 무엇이 오는가(PSY 자신은 이 방법을 「사전적(ex ante) 조기 경보」라 부르지만, 그 경보의 내용은 「지금 자라고 있다」이지 「곧 꺼진다」가 아니다). 그리고 **기초가치가 무엇인가** — 이것은 검정 밖의 가정입니다. 블랑샤르–왓슨이 1982년에 쓴 「기껏해야 정황 증거」는 도구가 정교해진 뒤에도 질문의 구조가 바뀌지 않았다는 점에서 여전히 맞는 말입니다.

한 가지를 더 적어 둡니다. 19장의 모형들은 버블이 **합리적일 수** 있다고 했습니다. 티롤은 버블이 존재할 조건을 주었고, 블랑샤르–왓슨은 「폭주하는 자산 가격과 시장 붕괴 같은 현상이 합리적 버블과 정합적」이라고 했습니다. 그렇다면 「버블이었다」는 통계적 판정은 그 자체로 누군가의 비합리성이나 기망을 함축하지 않습니다. 함축하려면 추가 전제가 필요하고, 그 전제는 통계가 아니라 사실인정의 몫입니다.

> **법률가의 질문**
>
> 전문가 증인이 「해당 기간 가격은 GSADF 검정에서 1% 수준으로 폭발적이었으므로 버블이었다」는 의견서를 냈습니다. 이 장을 읽은 뒤 물을 것은 정해져 있습니다. 어떤 자료(거래소·빈도·기간)인가. 기초가치를 무엇으로 두었는가, 아니면 두지 않았는가. 임계값은 교과서 표인가, 자료에서 만든 부트스트랩인가. 변동성의 변화는 어떻게 처리했는가. 최소 창과 최소 지속 조건은 무엇인가. 그리고 그 판정이 — 통계적으로 맞다 해도 — 피고의 **어떤 행위와** 어떻게 연결되는가. 폭발적 구간의 존재는 시세조종의 증거입니까, 사기적 기망의 증거입니까, 아니면 다수 균형 가운데 하나가 실현되었다는 관측입니까. 거꾸로 「버블이 아니었다」는 판정(이를테면 채굴 한계비용과 공적분)은 피고를 돕습니까. 이 책은 답하지 않고 5부의 쟁점 절을 가리킵니다. → 28장

8. 정리

버블은 가격과 기초가치의 차이이고, 그 검정은 두 가정 위에 섭니다. 무엇을 빼는가, 그리고 차이가 폭발적 모양을 띠는가. 우측 단위근 검정(PWY의 SADF, PSY의 GSADF)은 둘째만 다룹니다. 합성 자료에서 보았듯 창을 움직이는 검정은 심은 폭발 구간을 잡고 날짜도 대체로 찍지만, 시변 변동성 아래서는 교과서 임계값이 너무 자주 기각하고, 부트스트랩 임계값은 자료를 따라 움직입니다. 첫째 가정에 대한 문헌의 답은 셋(0·채굴 한계비용·네트워크 크기)이고, 답이 바뀌면 같은 가격이 버블이기도 아니기도 합니다. 그래서 「버블 판정」을 받으면 표본·기간·기초가치·임계값·변동성 처리를 함께 묻습니다. 3부는 여기서 끝납니다. 19장의 「가치는 어디서 오는가」에서 출발해 보안 예산·수수료·토큰 설계·게임이론·스테이블코인을 거쳐 「버블을 잴 수 있는가」까지 왔고, 공통의 답은 **조건부였습니다**. 모형은 가능성을 열거하고, 검정은 모양을 재며, 어느 쪽도 현실의 한 경로를 예언하지 않습니다. 26장은 그 조건들이 실제로 깨진 네 사건을 읽습니다.

이 장의 출처

- Peter C. B. Phillips, Shuping Shi & Jun Yu, "Testing for Multiple Bubbles: Historical Episodes of Exuberance and Collapse in the S&P 500," *International Economic Review* 56(4): 1043–1078 (2015), DOI 10.1111/iere.12132 — 우측 검정의 뜻(1047쪽), 귀무 모형 식 (3)·회귀 식 (4)(1047–1048쪽), SADF(1048쪽), GSADF와 정리 1(1049쪽), 최소 창 규칙과 Table 1(1050쪽), BSADF(1051쪽), 날짜 찍기(1052쪽). (게재본).
- Olivier J. Blanchard & Mark W. Watson, NBER Working Paper 945 (1982) — 초록·서론. Jean Tirole, *Econometrica* 53(5): 1071–1100 (1985) — 1071쪽. 모두 19장과 같은 자료.
- Christian M. Hafner, *Journal of Financial Econometrics* 18(2): 233–249 (2020) — 서론(235쪽), 자료 Table 1(236쪽), §4 Table 6(245–246쪽).
- Andrea Podhorsky, *International Review of Financial Analysis* 93: 103158 (2024), CC BY — 초록, §4 날짜 찍기와 식 (14)(11쪽), §5 Table 2·3(11–12쪽).
- Chiara Oldani, Giovanni S. F. Bruno & Marcello Signorelli, *Journal of Economic Asymmetries* 31: e00420 (2025), CC BY — §4 자료와 부트스트랩 절차(5–6쪽), Table 1·2와 BTC 구간(6쪽),

ETH·XRP(7쪽), 초록(1쪽).

- Eng-Tuck Cheah & John Fry, *Economics Letters* 130: 32–36 (2015) — 저자 수락원고의 4–6쪽·9쪽 (Table 2); 게재본 쪽은 대조하지 못했다. Adrian Cheung, Eduardo Roca & Jen-Je Su, *Applied Economics* 47(23): 2348–2358 (2015) — 게재본 2353·2356쪽.
- Spencer Wheatley 외, *Royal Society Open Science* 6: 180538 (2019), CC BY — 5–7쪽, Table 1.
- Corbet·Lucey·Yarovaya, *Finance Research Letters* 26: 81–88 (2018) — **철회**(공지 FRL 91: 109483, 2026). 본문을 받지 않았고 인용하지 않았다.
- 코드 실행: Python 3.13 (conda 환경 `bcbook`), `numpy` 2.5.4 — 3부에서 유일하게 쓴 수치 패키지. 모든 창의 ADF 통계량을 누적합으로 한꺼번에 계산하는 데 썼고(순수 파이썬 이중 루프로는 몬테카를로가 수십 분 걸린다), 선정 이유는 `notebooks/README.md` · `requirements.txt`. 노트북 NB25. **합성 자료만 썼다** — 시드 25·2025·2526·0(탐색)·7, 폭발 구간 t ∈ [100, 130), β = 1.04. 실제 시세 자료는 쓰지 않았다.

각주

1. Olivier J. Blanchard & Mark W. Watson, "Bubbles, Rational Expectations and Financial Markets," NBER Working Paper 945 (July 1982), 초록과 서론. 원문: "Rationality of both behavior and of expectations often does not imply that the price of an asset be equal to its fundamental value. In other words, there can be rational deviations of the price from this value, rational bubbles."; "In the last two sections, we consider whether the presence of bubbles in a particular market can be detected statistically. The task is much easier if there are data on both prices and returns. ... In markets in which returns are difficult to observe, possibly because of a nonpecuniary component, such as gold, the task is more difficult. We consider the use of both 'runs tests' and 'tail tests' and conclude that they give circumstantial evidence at best." 19장 각주 27 과 같은 자료.

2. Jean Tirole, "Asset Bubbles and Overlapping Generations," *Econometrica* 53(5): 1071–1100 (September 1985), 1071쪽(서론). 원문: "Since Samuelson [30] developed his consumption loan model it has been well known that there exist economies in which money has a positive value in spite of the fact that it is intrinsically useless (i.e., its market fundamental is zero). In other words there can exist a bubble on money where a bubble is defined as the difference between the market price and the market fundamental." (JSTOR 스캔, 텍스트 층 없음 — 이미지로 읽어 대조). 19장 각주 28 과 같은 자료(서지는 소유자가 받은 스캔의 표지로 확인).

3. Peter C. B. Phillips, Shuping Shi & Jun Yu, "Testing for Multiple Bubbles: Historical Episodes of Exuberance and Collapse in the S&P 500," *International Economic Review* 56(4): 1043–1078 (November 2015), 1047쪽. 원문: "Right-sided unit root tests, as discussed in PWY, are informative about mildly explosive or submartingale behavior in the data and are therefore useful as a form of market diagnostic or warning alert." (게재본). PWY는 Phillips, Wu & Yu, *IER* 52 (2011), 201–226.

4. PSY (2015), 1047–1048쪽, 식 (4). 원문: "The empirical regression model can then be written as $\Delta y_t = \hat{\alpha}_{r_1,r_2} + \hat{\beta}_{r_1,r_2} y_{t-1} + \sum_{i=1}^{k} \hat{\psi}^{i}_{r_1,r_2} \Delta y_{t-i} + \hat{\varepsilon}_t$, where k is the (transient) lag order. The number of observations in the regression is $T_w = \lfloor T r_w \rfloor$ ... The ADF statistic (t-ratio) based on this regression is denoted by $ADF_{r_1}^{r_2}$." 귀무가설의 모형은 식 (3) $y_t = dT^{-\eta} + \theta y_{t-1} + \varepsilon_t$, $\theta = 1$ (1047쪽).

5. PSY (2015), 1048쪽. 원문: "The PWY test relies on repeated estimation of the ADF model on a forward expanding sample sequence, and the test is obtained as the sup value of the corresponding ADF statistic sequence. ... The PWY test is then a sup statistic based on the forward recursive regression and is simply defined as $SADF(r_0) = \sup_{r_2 \in [r_0, 1]} ADF_0^{r_2}$."

6. PSY (2015), 1050쪽, Table 1 "The asymptotic and finite sample critical values of the SADF and GSADF tests against an explosive alternative". 유한표본 행 "T = 200, r0 = 0.137": SADF 90/95/99% = 1.12/1.40/1.90, GSADF 90/95/99% = 1.84/2.08/2.70. 주석: "The finite sample critical values are obtained from Monte Carlo simulation with 2000 replications." 최소 창 규칙(같은 쪽): "we recommend a rule for choosing r0 that is based on a lower bound of 1% of the full sample and has the simple functional form $r_0 = 0.01 + 1.8 / \sqrt{T}$". 이 셀의 ADF 는 시차 0 이고 PSY 의 유한표본 임계값은 $d = \eta = 1$ 의 귀무 모형(약한 표류)에서 얻은 것이며 시차는 밝히지 않는다. 난수·반복 수

·표류가 다르므로 두 수가 정확히 같을 이유는 없다 — 다음 셀이 직접 잰다.

7. PSY (2015), 1049쪽. 원문: "we denote this statistic by GSADF(r0). That is, $GSADF(r_0) = \sup_{r_2 \in [r_0, 1],\, r_1 \in [0, r_2 - r_0]} ADF_{r_1}^{r_2}$."; Theorem 1: "When the regression model includes an intercept and the null hypothesis is a random walk with an asymptotically negligible drift (i.e., $dT^{-\eta}$ with η > 1/2 and constant d) as in (3), the limit distribution of the GSADF test statistic is [위너 과정의 범함수] ...". 극한분포의 식은 본문 1049쪽.

8. PSY (2015), 1050쪽. 원문: "First, as the minimum window size r0 decreases, critical values of both test statistics increase. For instance, when r0 decreases from 0.190 to 0.055, the 95% asymptotic critical value of the GSADF statistic rises from 1.89 to 2.30. Second, critical values for the GSADF statistic are larger than those of the SADF statistic. For instance, when r0 = 0.10, the 95% asymptotic critical value of the GSADF statistic is 2.19 whereas that of the SADF statistic is 1.49."

9. PSY (2015), 1051쪽. 원문: "In particular, the backward SADF test performs a sup ADF test on a backward expanding sample sequence where the endpoint of each sample is fixed at r2 ... and the start point varies from 0 to r2 − r0 ... The backward SADF statistic is then defined as the sup value of the ADF statistic sequence over this interval, viz., $BSADF_{r_2}(r_0) = \sup_{r_1 \in [0, r_2 - r_0]} ADF_{r_1}^{r_2}$. The PWY procedure (i.e., the recursive ADF test) for bubble identification is a special case of the backward sup ADF test in which r1 = 0 ...". 왜 PWY 방식이 두 번째 버블을 놓치는지(가짜 정상성, Evans 1991의 비판)도 같은 쪽.

10. PSY (2015), 1052쪽. 원문: "The origination date of a bubble ⌊Tre⌋ is calculated as the first chronological observation whose ADF statistic exceeds the critical value. ... The estimated termination date of a bubble ⌊Trf⌋ is the first chronological observation after ⌊Tre⌋ + LT whose ADF statistic goes below the critical value. PWY impose a condition that for a bubble to exist its duration must exceed a slowly varying (at infinity) quantity such as LT = log(T). This requirement helps to exclude short lived blips in the fitted autoregressive coefficient and, as discussed below, can be adjusted to take into account the data frequency." 식 (6)·(7)(1052쪽)은 BSADF를 $\lfloor Tr_2 \rfloor$ 관측 기준 SADF 임계값 scv 와 비교한다. NB25-2 는 끝점별 몬테카를로 분위수를 임계값으로 쓰고 L_T 보다 짧은 구간을 버리는 방식이라 식 (7)의 처리(시작 + $\delta \log T$ 뒤 처음 내려가는 관측)와 조금 다르다 — 시연용 단순화.

11. Christian M. Hafner, "Testing for Bubbles in Cryptocurrencies with Time-Varying Volatility," *Journal of Financial Econometrics* 18(2): 233–249 (2020), 235쪽(서론)·245–246쪽(§4, Table 6). 원문: "As shown by Harvey et al. (2016), the original test of PWY is biased in the presence of non-stationary volatility. ... Harvey et al. (2016) suggest a wild bootstrap procedure to correct the size of the PWY test and show that it has good power properties."; 초록: "This paper extends this approach to the case where volatility is time varying ..."; "Taking into account global heteroskedasticity has the effect of increasing the p-values that were found to be very close to zero ... In other words, the evidence in favor of speculative bubbles is much less pronounced than when this effect is ignored."; Table 6 BTC 행: PWY 5.0347, p-value 0.0001, WB 5% 2.8349, WB 1% 4.2929, WB p-value 0.0032; "not all p-values increase, in particular the larger ones tend to decrease under the correction, which is the case for BCH, ETH, XMR, and DASH"; "Only one p-value is smaller than 1% (BTC with 0.0032), meaning that at the 1% level one would reject the null hypothesis in favor of a speculative bubble only for the Bitcoin. At the 5% level, this number increases to 9 (all cryptos except IOTA, BCH, and XLM)" — 9는 CRIX 를 포함한 수(종목만 세면 11 중 8). 자료: 2017-12-31 기준 시가총액 상위 11종목과 CRIX 지수, BTC 는 2013-04-28부터 1,709 관측(Table 1, 236쪽); $r_0 = 0.1$, 와일드 부트스트랩 5,000회.

12. Chiara Oldani, Giovanni S. F. Bruno & Marcello Signorelli, "Collapsing bubbles in the prices of cryptocurrencies," *Journal of Economic Asymmetries* 31: e00420 (June 2025), 5–6쪽(§4.1). 원문: "The Monte Carlo CVs for SADF and GSADF tabulated in the original PSY paper ... are not robust to two potential inference issues: 1) time-varying unconditional heteroscedasticity in the ADF errors and 2) the multiplicity issue (the probability of a false positive increases with the number of hypotheses tested) ... Phillips and Shi (2020) (PS) suggest a composite wild bootstrap procedure for computing robust CVs ...". 절차 다섯 단계(5–6쪽)와 부트스트랩 창 $T_b = 183$ 일, $B = 499$. 본문은 최소 창을 「$r_0 = 0.1 + 1.8 / \sqrt{T}$」로 적지만 표 2의 $\lfloor r_0 T \rfloor = 110$ 은 PSY 규칙 $0.01 + 1.8 / \sqrt{2357}$ 로 계산한 값과 일치하므로 본문의 0.1은 오기로 보인다(이 책의 판단, 원문 확인 필요).

13. Eng-Tuck Cheah & John Fry, "Speculative bubbles in Bitcoin markets? An empirical investigation into the fundamental value of Bitcoin," *Economics Letters* 130: 32–36 (May 2015). 이 책은 저자 수락원고(AAM)로 읽었고 쪽은 그 판의 쪽이다. 자료: "daily closing prices for the Bitcoin Coindesk Index from July 18th 2010 to July 17th 2014"(5쪽), 검정 구간 "January 1st 2013-November 30th 2013"(5쪽). 결과(6쪽): "We reject the null hypothesis (11) and have significant evidence of a bubble. Further, the parameter μ̃ is not statistically different from zero

suggesting that equation (17) applies. Thus, during the bubble price rises are so dramatic that the estimated long-term fundamental value is not statistically different from zero. … Equation (16) suggests that the bubble accounts for around 48.7% of observed prices."; Table 2(9쪽): v 0.546 (E.S.E. 0.090, t 6.060, p 0.000), μ̃ 0.00166 (0.00195, 0.852, 0.394); 결론(6쪽): "Thirdly, the fundamental value of Bitcoin is zero." 귀무·대립가설은 식 (11) $H_0{:}v = 0$, $H_1{:}v > 0$ (4쪽).

14. Andrea Podhorsky, "Bursting the bitcoin bubble: Do market prices reflect fundamental bitcoin value?," *International Review of Financial Analysis* 93: 103158 (May 2024), 초록·11–12쪽(§5). 원문: "demonstrates that the bitcoin's volatile and explosive price path is a consequence of the Bitcoin protocol's system of supply management"(초록); "The ADF test on the price of the bitcoin p_{bt} resulted in a test statistic of −2.51 and a p-value of .36, and the ADF test on the fundamental value … −1.90 and a p-value of .62 …"; "the Engle–Granger cointegration test … resulted in a test statistic of −2.74 and a p-value of .071, indicating that we can reject the null hypothesis of no cointegration at the 10% level of significance."; "For the price data, the SADF and GSADF statistics are 3.7 and 3.9, respectively, which both exceed their 1% right-tailed critical values (3.7 > 2.2 and 3.9 > 2.9)"; "Next, the summary SADF and GSADF tests are applied to the residuals v̂t. The SADF and GSADF statistics are −.08 and 1.52, respectively, which fall well below their 10% right-tailed critical values (−.08 < 1.3) and (1.5 < 2.2)"; "Since the raw price data indicates the presence of bubbles while the residuals v̂t do not, from (14) it follows that we can attribute the apparent bubbles in the price of the bitcoin to the fundamental value …". 표본 1,764 일별 관측, 최소 창 93($r_0 = .01 + 1.8 / \sqrt{1764}$), 임계값은 2,000회 몬테카를로(11쪽). CC BY.

15. Spencer Wheatley, Didier Sornette, Tobias Huber, Max Reppen & Robert N. Gantner, "Are Bitcoin bubbles predictable? Combining a generalized Metcalfe's Law and the Log-Periodic Power Law Singularity model," *Royal Society Open Science* 6: 180538 (2019), 5–7쪽. 지수: 각주 11 "Our finding of a smaller exponent b ≈ 1 + 2/3 expresses a more sparsely connected network in which each user is on average linked to ≈N^{2/3} other users …"; OLS 계수 (a, b) = (1.51, 1.69)(그림 2 설명, 6쪽). Table 1(7쪽) "Bubble statistics. Columns: start, end (time of peak value, prior to correction), duration in days, starting and peak market cap, growth factor … and average daily return": 1 — 25 May 2012 / 18 Aug 2012 / 84일; 2 — 3 Jan 2013 / 11 Apr 2013 / 98일; 3 — 7 Oct 2013 / 23 Nov 2013 / 47일; 4 — 8 June 2015 / 18 Dec 2017 / 924일(시가총액 열과 성장 배수 열은 옮기지 않는다 — 3장 일러두기). 자료: "The price data for Bitcoin is from Bitstamp, in USD, hourly from 1 January 2012 to 8 January 2018; the Bitcoin circulating supply comes from blockchain.info." 시가총액 열은 옮기지 않았다. CC BY.

16. Wheatley 외 (2019), 5쪽. 원문: "More generally, within the sample, the fitted curves are similar, but, beyond the sample, differences explode such that there are 4 orders of magnitude difference between the predicted carrying capacities. Here, model uncertainty dominates uncertainty of estimated parameters. … Therefore, precise inference based on a single model—notably omitting any limitation imposed by the physical Bitcoin network—is misleading, and long-term predictions effectively meaningless. However, smoothing of past values is not problematic, and short-term projections may be reasonable."

17. Adrian (Wai-Kong) Cheung, Eduardo Roca & Jen-Je Su, "Crypto-currency bubbles: an application of the Phillips–Shi–Yu (2013) methodology on Mt. Gox bitcoin prices," *Applied Economics* 47(23): 2348–2358 (2015), 게재본. 자료(2353쪽): "most actively traded bitcoins (i.e., MT.GOX) over the period 17 July 2010 to 18 February 2014. They are all denominated in the US dollar. There are 1307 daily observations in the sample period."; 최소 창(2353쪽): "Following Phillips et al. (2013a), we set r0 to approximately 2% of the data (i.e., 20)"; 결과(2356쪽): "The GSADF test statistic … exceeds its corresponding 99% critical value … 33 times over the sample period, suggesting that 33 episodes of bubbles are identified. … The duration of the bubbles ranges from 1 day only to 106 days. … there were three big bubbles … The start date and the end date of the first big bubble are 24 April 2011 and 3 July 2011, respectively, covering 66 days in between while the start date and the end date of the second big bubble are 27 January 2013 and 15 April 2013, respectively, with a lifespan of 79 days. The third (and the longest) bubble has a history of 106 days, covering the period from 5 November 2013 to 18 February 2014 (i.e., the end of the sample period)". 첫 버블의 「66 days」는 게재본 2356쪽 그대로다. 그러나 2011-04-24부터 07-03까지는 양끝을 포함해 71일이고(둘째·셋째 구간의 79·106일은 양끝 포함 계산과 맞는다), 66이 결측일을 뺀 관측치 수인지 오기인지 논문은 설명하지 않는다. 이 책은 논문의 수를 고치지 않고 그대로 옮기되 이 차이를 적어 둔다.

18. Oldani 외 (2025), 6–7쪽, Table 2 "GSADF Tests with bootstrap CVs and fixed lag order = 0 (daily data 01/01/2018–14/06/2024)": BTC 3.709 ***(RTBS90/95/99 = 1.246/1.500/2.086), ETH 2.289*** (1.210/1.513/2.124), XRP 3.966*** (1.340/1.769/2.104); "Number of obs. T = 2357; initializing window size ⌊r0T⌋ = 110; RTBS: right-tail Bootstrap CVs … based on wild bootstrap with 499 replications and bootstrap window Tb = 183." BTC 구간(6쪽): "The first bubble episode starts on 2018-11-24 and terminates on 2018-12-08 with a break in between. The second starts on 2019-05-11 and terminates on 2019-06-28 … The third is the longest interval, spanning four months, from

2020-12-19 to 2021-03-13 … The last starts on 2024-02-28 and terminates on 2024-03-15 …". XRP(7쪽): "for XRP ln prices the GSADF test with zero lag order rejects unit root at the 1 % level (Table 2), but only a short-lived single bubble is found, from 2020-11-23 to 2020-11-25"; 8쪽: "With a fixed lag order of 7, GSADF does not support exuberant behavior at all (Table 4)."; 초록(1쪽): "No test supports the exuberance of XRP that is traded on a centralized ledger …". 저자들이 BTC 2018·2019 구간을 반감기와, 2020–21 구간을 팬데믹 뒤 과열과 연결한 서술은 선행 연구의 해석을 옮긴 것이어서 이 책은 가져가지 않는다(20장 — 반감기는 가격 사건으로 쓰지 않는다).

19. Shaen Corbet, Brian Lucey & Larisa Yarovaya, "Datestamping the Bitcoin and Ethereum bubbles," *Finance Research Letters* 26: 81–88 (September 2018), DOI 10.1016/j.frl.2017.12.006 — Crossref 레코드 제목에 "RETRACTED:" 표기. 철회 공지: "Retraction notice to 'Datestamping the Bitcoin and Ethereum Bubbles' [Finance Research Letters 26 (2018) 81-88]," *Finance Research Letters* 91: 109483 (March 2026), DOI 10.1016/j.frl.2026.109483. 확인 과정에서는 Retraction Watch 기록일(2025-12-18)과 공지 DOI 등록일(2026-02-06)이 다르고 **철회 사유를 확인하지 못했다고** 적는다. 이 책은 본문을 받지 않았고 어떤 수치도 인용하지 않는다. 이 논문을 인용한 뒤의 문헌(올다니 외 포함)을 읽을 때는 그 인용이 철회 전의 것임을 염두에 둔다.

4부 사건으로 읽기

26장 네 사건 — Mt.Gox · The DAO · Terra-Luna · FTX

> **고지**
>
> 이제부터 네 사건을 읽습니다. 미리 말해 두면, 이 부는 「누가 얼마를 벌고 잃었나」를 좇지 않습니다. 묻는 것은 하나입니다 — **무엇이 무너졌는가.** 기술인가, 경제 설계인가, 거버넌스인가, 아니면 처음부터 사기였는가. 이름과 수치는 판결문과 공식 조사 문서에 적힌 것만 썼습니다.
>
> 고지 전문은 판권면(부록 F의 F.4)에 있습니다.

핵심 TAKEAWAY

이 장은 네 사건을 **같은 진단표** 위에 놓습니다. 열은 넷입니다. 무엇이 무너졌는가(기술·경제 설계·거버넌스·사기), 사고가 체인 안에서 났는가 밖에서 났는가, 코드가 한 일과 사람이 한 일은 무엇인가, 그 뒤에 무엇이 따라왔는가(도산절차·하드포크·형사). 사실은 판결·결정(확정 여부는 각주에 적습니다)·공식 조사·도산 절차 문서에 적힌 것만 쓰고, 금액은 그 문서에 적힌 것을 한 번씩만 씁니다. 네 사건에서 깨진 층은 각각 다릅니다. Mt.Gox는 **거래소 장부와 체인의 불일치**이고(장부 조작은 형사 1심·항소심 판결이 인정했고, 2014년 2월 소실의 원인은 어느 법원 문서도 판단하지 않았습니다), The DAO는 **계약 코드의 결함과 그 뒤의 거버넌스 결정이며**, Terra는 **설계의 반사성과 발행자의 진술이고**, FTX는 **고객 자산의 유용과 내부통제의 부재입니다.** 1부에서 배운 합의 규칙과 원장 무결성이 외부 공격으로 깨진 사건은 이 넷에 없습니다. 다만 그 문장은 「블록체인 자체」가 무엇인지 먼저 정해 놓고 읽어야 하고, 이 장은 1절에서 그 범위를 셋으로 나눠 정의한 뒤에만 그렇게 말합니다. 법률가가 가져갈 것은 사건 서사가 아니라 **진단의 틀입니다.** 어느 층이 무너졌는지를 가려야 어느 법이 들어올 자리인지가 보입니다.

학습목표

1. 네 사건을 같은 진단표 — ① 무너진 층 ② 체인 안팎 ③ 코드가 한 일과 사람이 한 일 ④ 사후 대응 — 에 올려 나란히 읽는다.
2. 「블록체인 자체」라는 말을 세 층(합의 규칙·원장 무결성 / 프로토콜의 성질 / 그 위에 얹힌 장부·계약·발행자)으로 나누어 쓰고, 사건마다 어느 층의 이야기인지를 가린다.
3. 각 사건에서 1차 문서(판결·공식 조사·도산 절차 문서)가 **무엇을 판단했고 무엇을 판단하지 않았는지를** 구별한다. 거래소의 공지와 보도는 1차 문서가 아니다.
4. 사건마다 법률가의 질문이 어디에서 생기는지를 적어 두고, 5부의 쟁점 절로 넘긴다.

[입문]**만 읽은 독자가 가져가는 것.** 네 사건에서 깨진 것은 각각 다르고, 그 가운데 **비트코인·이더리움의 합의 규칙이 외부 공격으로 깨져 원장이 뒤집힌 사건은 없습니다.** 무너진 자리는 체인 밖의 장부, 체인 위에 얹힌 계약 코드, 발행자의 설계와 진술, 그리고 사람의 손이었습니다.

1. 이 장이 읽는 방식 — 진단표와 「블록체인 자체」의 범위

사건을 한 편씩 이야기로 쓰면 「누가 얼마를 잃었나」가 축이 됩니다. 이 책은 그 축을 쓰지 않습니다(3장 일러두기). 대신 네 사건을 같은 표에 올립니다. 열은 넷입니다.

열	묻는 것
① 무너진 층	기술(합의 규칙·원장·암호)인가, 경제 설계(토큰·담보·수수료 구조)인가, 거버넌스(누가 규칙을 바꾸는가)인가, 사기(진술과 실제의 차이)인가. 하나가 아닐 수 있다
② 체인 안팎	사고가 원장 위의 상태 전이로 일어났는가, 원장 밖의 장부·은행 계좌·회사 안에서 일어났는가
③ 코드와 사람	코드가 적힌 대로 한 일과, 사람이 손으로 한 일을 가른다. 「코드대로 실행되었다」는 말이 책임을 지우는 말이 아니라는 점이 여기서 보인다
④ 사후 대응	도산 절차·하드포크·형사·행정 조사 가운데 무엇이 따라왔고, 그 문서가 무엇을 판단했는가

둘째 열을 쓰려면 「블록체인 자체」라는 말의 범위를 먼저 정해야 합니다. 이 말은 세 가지 다른 것을 가리키는 데 쓰입니다.

층	뜻	이 책에서 다룬 곳
(가) 합의 규칙과 원장 무결성	블록이 이어지는 규칙, 가장 긴(무거운) 체인 선택, 서명 검증, 거래 유효성 규칙. 「변조 비용」이 지키는 것	7장·8장·9장·23장
(나) 프로토콜의 성질	규칙이 허용하는 행동 가운데 설계자가 의도하지 않았거나 응용층을 괴롭히는 것. 거래 가변성(서명을 바꿔 거래 식별자를 바꾸는 성질), 순서 결정 권한(MEV) 등	5장·17장
(다) 그 위에 얹힌 것	거래소의 내부 장부, 스마트계약의 코드, 스테이블코인 발행자의 대차대조표와 진술. 체인이 보증하지 않는다	13장·14장·24장

이 장이 「블록체인 자체는 깨지지 않았다」고 쓸 때 가리키는 것은 **(가)** 입니다. (나)는 사건에 따라 관여했을 수 있고 — Mt.Gox가 그렇게 주장했습니다 — (다)는 네 사건 모두의 무대입니다. 세 층을 섞어 「블록체인은 안전하다」거나 「블록체인은 위험하다」고 말하는 것은 어느 쪽도 이 장이 하지 않는 일입니다.



그림 26-1. 「블록체인 자체」의 세 층과 네 사건이 무너진 자리. 「깨지지 않았다」는 문장이 가리키는 것은 (가) 층이며, 점선은 Mt.Gox 거래소가 원인으로 든 것으로 법원은 판단하지 않았다.

사실의 출처는 다음 셋으로 한정합니다. 확정되었거나 확정 여부를 밝혀 적은 **판결·결정**, 규제기관의 **공식 조사 보고서**, 그리고 **도산 절차에서 법원·관재인·채무자가 낸 문서입니다**. 거래소의 보도자료는 「당사자가 그렇게 말했다」는 사실의 출처일 뿐 그 내용이 참이라는 출처가 아니고, 신문 보도는 쓰지 않습니다. 두 가지를 덧붙입니다. The DAO 결함의 기술 서술은 판결·조사 문서가 적지 않으므로 이더리움 재단 공지와 EIP를 씁니다(22장이 이 장으로 넘긴 것). 그리고 Mt.Gox의 2014년 2월 7일 인출 중단 공지는 원문 아카이브를 열지 못해 2차 전재본으로 인용하고 그 사실을 밝힙니다.

2. Mt.Gox (2014) — 장부와 체인의 불일치

무엇이 적혀 있는가

비트코인 거래소 Mt.Gox를 운영한 회사(이하 판결의 표기대로 「M사」)는 2014년 2월 28일 도쿄지방재판소에 민사재생 절차 개시를 신청했습니다. 그날 회사가 낸 공지는 재무 상황을 이렇게 적었습니다. 자산 총액 3,841,866,163엔, 유동부채 총액 6,501,119,371엔. 그리고 유동부채가 늘어난 배경에 「ビットコインの消失，預り金の消失という原因が考えられ」라고 적었습니다.[1] 같은 공지는 경위를 이렇게 적습니다. 2014년 2월 초 「ビットコインのシステムのバグを悪用した不正アクセスにより」 송금이 정상 완료되지 않는 거래가 늘었고, 조사 결과 이용자 보유분 약 75만 BTC와 자사 보유분 약 10만 BTC의 거의 전부가 없어졌음이 2월 24일경까지 판명되었으며, 같은 날 이용자 예탁금 총액과 금융기관 예금 잔고 사이에 최대 약 28억엔의 부족이 확인되었고, 2월 25일 정오 사이트 접속을 전면 정지했다는 것입니다. 원인에 관해서는 「第三者によるハッキングによる被害を含めて，複数の原因があると見込まれる」라고 적었습니다. 여기에 「거래 가변성」이라는 낱말은 없습니다.

3주 뒤인 3월 20일 회사는 2011년 6월 이전에 쓰던 구식 지갑을 다시 스캔해 약 20만 BTC(199,999.99 BTC)가 남아 있음을 확인했다고 공지하고, 소실 추정치를 약 85만 BTC에서 약 65만 BTC로 낮췄습니다. 그 공지도 「ＢＴＣの消失の原因や消失したＢＴＣの数量は現在調査中」이라고 적습니다.[2] 4월 16일 도쿄지방재판소는 민사재생 개시 신청을 기각하고 보전관리명령을 냈으며, 4월 24일 파산절차 개시결정을 했습니다. 결정의 이유는 한 줄입니다. 「一件記録によれば，債務者が支払不能の状態にあることが認められる。」[3] 파산관재인은 2014년 7월 첫 채권자집회 보고서에서 소실 조사를 외부 회계법인에 맡겼고 「the completion date is undetermined」라고 적었으며, 2016년 5월 보고서에서는 「Due to the circumstances, such as the limited amount of information that exists, I believe it will be difficult to understand the full picture」라고 적었습니다. 2018년 3월 보고서는 미국 법무부가 2017년 7월 다른 거래소 운영자를 기소하면서 공소사실에 「he obtained funds by hacking MtGox」를 넣었다는 사실과 관재인이 정보를 요청했다는 사실을 적습니다.[4] 2017년 11월 일부 채권자의 신청으로 2018년 6월 22일 민사재생 절차가 개시되어 파산절차는 정지되었습니다.[5] 관재인 사이트의 문서 가운데 소실 원인을 확정해 발표한 것은 찾지 못했습니다.

거래소는 무엇을 원인으로 들었고, 실증은 무엇을 보았나

[중급] 2월 28일 공지보다 앞서, 회사는 2월 7일 비트코인 인출을 전부 중단한다고 공지했습니다. 그 공지는 「the increase in the flow of withdrawal requests has hindered our efforts on a technical level」이라며 「a temporarily pause on all withdrawal requests」가 필요하다고 적었고, 원인은 적지 않았습니다.[6] 2월 10일 공지가 처음으로 원인을 들었습니다. 「Addressing Transaction Malleability」라는 제목 아래 「A bug in the bitcoin software makes it possible for someone to use the Bitcoin network to alter transaction details to make it seem like a sending of bitcoins to a bitcoin wallet did not occur when in fact it did occur」라고 적은 것입니다.[7] 5장에서 본 **거래 가변성입니다**. 서명의 모양을 바꾸면 거래 식별자(txid)가 바뀌고, 식별자로만 거래의 성공·실패를 추적하는 거래소 소프트웨어는 실제로 송금된 거래를 「실패」로 읽어 다시 보낼 수 있습니다. 위 세 층으로 말하면 (나)의 성질이 (다)의 장부를 속인 경로입니다.

[중급] 이 주장은 네트워크를 관측하면 검증할 수 있습니다. 데커(Christian Decker)와 바텐호퍼(Roger Wattenhofer)는 2013년 1월부터 비트코인 네트워크에 노드를 두고 자기 노드에 전달된 거래를 기록해 둔 상태였고(연결된 피어는 도달 가능한 노드의 약 20%), 그 자료에서 가변성 공격의 흔적 — 같은 입력을 쓰되 서명 인코딩만 다른 거래 쌍 — 을 셌습니다. 결론은 이렇습니다. 「while MtGox claimed to have lost 850,000 bitcoins due to malleability attacks, we merely observed a total of 302,000 bitcoins ever being involved in malleability attacks. Of these, only 1,811 bitcoins were in attacks before MtGox stopped users from withdrawing bitcoins. Even more, 78.64% of these attacks were ineffective. As such, barely 386 bitcoins could have been stolen using malleability attacks from MtGox or from other businesses.」[8] 가변성 공격은 2월 10일 공지 **뒤에** 급증했고(공지가 공격 방법을 알린 뒤), 인출이 중단되기 전의 양은 소실 주장의 약 470분의 1입니다. 이 논문은 「MtGox needs to explain the whereabouts of 849,600 bitcoins」로 끝납니다. 실증이 말하는 것은 「가변성 공격으로 설명되는 양이 매우 작다」까지이고, 그 밖의 양이 어디로 갔는지는 말하지 않습니다.

형사 판결이 인정한 것 — 장부 위의 가공 잔고

대표이사는 2015년 체포·기소되었고, 도쿄지방재판소는 2019년 3월 15일 私電磁的記録不正作出·同供用죄로 懲役2年6月·執行猶予4年을 선고하면서 業務上横領과 会社法違反(특별배임) 부분은 무죄로 했습니다.[9] 도쿄고등재판소는 2020년 6월 11일 피고인의 항소를 기각했고, 무죄 부분은 검찰이 항소하지 않아 항소심의 심판 대상이 아니었습니다.[10] 상고 여부와 확정일은 이 책이 확인하지 못했습니다(2026-10 기준). 아래는 두 판결 본문이 인정한 사실과 판단입니다. 판례 데이터베이스의 「判示事項」 요약은 편집물이므로 인용하지 않습니다.

1심 판결은 먼저 거래소의 구조를 적습니다. 이용자가 돈을 회사 지정 은행계좌로 보내거나 비트코인을 회사 관리 주소로 보내면, 시스템이 이를 자동으로 확인해 이용자 계정의 잔고를 늘립니다. 그 뒤 이용자끼리 거래가 성립해도 「本件取引システム上の口座残高に増減が生じるだけで，…
ビットコインも同様に，本件取引システム上の口座残高に変動は生じるが，
ブロックチェーン上に変動は生じない」 — 즉 거래소 안의 매매는 장부 위의 숫자 이동일 뿐이고, 블록체인이 움직이는 것은 이용자가 반환·송부를 청구해 회사가 실제로 보낼 때뿐입니다.[11] 이것이 11장과 12장에서 본 원장과, 그 위에 얹힌 (다)층 장부의 관계입니다. 판결은 또 비트코인의 사법상 성질에 대해 이렇게 유보합니다. 「ビットコインは「有体物」（民法８５条）ではなく，
その帰属や利用者間での取引がどのような私法上の性質を有するかも一義的に明らかとはいえないが，本件の判断に影響のない限り，ビットコインの保有・所有や利用者間の売買などと表現し，日常的な意味合いで記載する。」[12] 5부 27.2의 학설 구도가 받는 문장입니다.

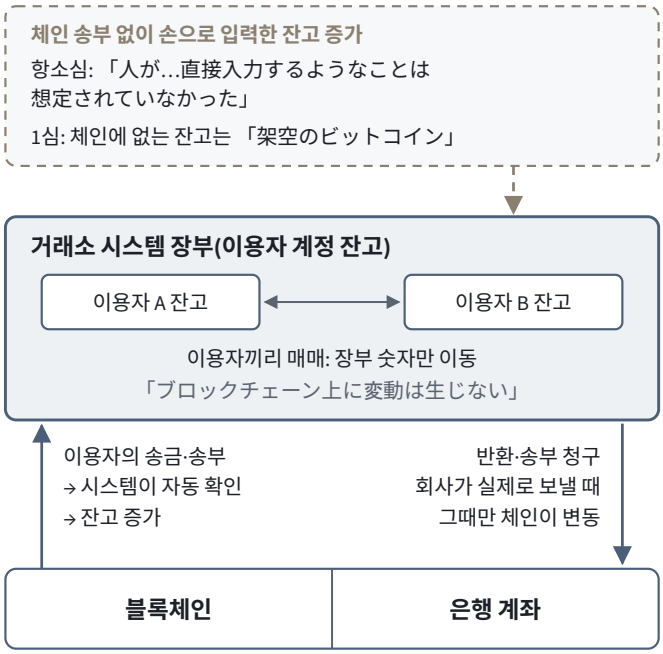


그림 26-2. 1심 판결이 적은 거래소 구조. 이용자끼리의 매매는 장부 위 숫자만 움직이고, 체인 송부 없이 손으로 입력한 잔고를 판결은 「架空のビットコイン」이라 했다.

[중급] 인정된 사실의 뼈대는 세 번의 「잔고 증가 행위」입니다.[13] 첫째, 2011년 8월 피고인의 모회사(1심 판결의 D社, 항소심 판결의 T社)가 인수한 폴란드의 다른 거래소가 시스템 결함으로 1만 7천 BTC를 잃어 「ブロックチェーン上に存在しない１万７０００ＢＴＣの返還債務」를 떠안자, 피고인은 자기 계정의 미국 달러 잔고를 15만 달러 늘리는 정보를 입력하고 그 달러로 이용자들로부터 약 1만 8천 BTC를 사들인 뒤 자기 계정의 비트코인 잔고를 1만 7,500 BTC 삭제했습니다(「１５万米ドル増加行為」). 둘째, 2011년 11월 회사가 쓰던 프랑스 은행계좌가 일시 동결되어 약 1억엔을 꺼낼 수 없게 되자, 피고인은 2011년 11월부터 2012년 4월까지 8회에 걸쳐 비트코인을 회사 주소로 보내지 않은 채 자기 계정의 비트코인 잔고를 30만 BTC 늘리는 정보를 입력하고, 2013년 2월까지 이용자들에게 합계 약 30만 BTC를 팔아 약 130만 달러를 얻었습니다(「３０万ＢＴＣ増加行為」). 셋째, 그 결과 「ブロックチェーン上には存在しないビットコイン３０万ＢＴＣが利用者間で取引されることとなった」 상태를 거두어들이려고, 2013년 2월부터 9월까지 21회에 걸쳐 자기 계정의 달러 잔고를 합계 3,350만 달러 늘리는 정보를 입력하고 이용자들로부터 약 2,977만 달러에 약 30만 BTC를 사들인 뒤 그 비트코인 잔고를 삭제했습니다(「本件各行為」). 기소된 것은 셋째 행위이고, 법원은 셋째가 둘째를 은폐하는 수단이므로 둘을 한 덩어리로 보아야 한다고 했습니다.

[중급] 쟁점은 이 입력이 「不正に」 전자적 기록을 만든 것인가였습니다. 변호인은 피고인이 회사의 유일한 포괄적 업무집행권자이므로 피고인의 행위는 곧 회사의 의사라고 다퉜습니다. 법원은 회사의 의사를 피고인의 내심과 떼어 **이용규약과 시스템의 성질에서** 읽었습니다. 핵심 문장입니다.
「ビットコインの存在根拠及びその経済的価値の根拠は，取引されるビットコインが，
ブロックチェーン上に存在することにかかっているのであって，
ブロックチェーン上に存在しないビットコインは，仮に，
本件取引システム上のビットコイン口座に残高が表示されたとしても，
架空のビットコインといわざるを得ない。」 이용규약은 매도인이 「売却オファーの目的となる…ビットコインは，実際のビットコインに対応することを保証」한다고 적었고, 신용거래를 허용하는 조항이나 화면 표시는 없었습니다. 그래서 회사의 의사는

「B社管理のビットコインアドレスにおけるビットコインの受送付等による裏付けがある場合に限って，本件取引システム上の利用者の口座残高を増加させるべきである」는 것이고 피고인의 권한도 그 범위에 한정된다는 것입니다.[14] 항소심은 이 판단을 「概ね同旨をいうものとして是認できる」고 하면서, 전자적 기록의 증명 기능은
「それが作出されるべきシステムの設置運営主体の意思に基づき,
正当に作出されることが当然の前提」라고 덧붙였습니다.[15]

[중급] 반면 **금전에** 관해서는 다른 결론이 났습니다. 이용자가 회사 지정 계좌로 보낸 돈은 회사에 귀속되고, 당시 분별관리를 명하는 법령도 규약도 없었으므로, 장부의 달러 잔고에 상당하는 돈이 회사 은행계좌에 있어야 한다고는 할 수 없다는 것입니다.[16] 그래서 회사 계좌에서 피고인 명의 계좌 등으로 보낸 송금은 이용자의 돈을 횡령한 것(주위적 소인)이 아니고, 회사의 돈을 횡령한 것(예비적 소인 1)도 — 단기대여금으로 회계처리되었고 반환의 현실적 가능성이 있었다는 이유로 — 아니며, 특별배임(예비적 소인 2)도 아니라고 했습니다. 두 결론을 나란히 놓으면 이 사건의 구조가 보입니다. **비트코인 쪽에서는 「체인 위에 있어야 한다」는 뒷받침 요구가 회사의 의사로 인정되었고, 돈 쪽에서는 「은행에 있어야 한다」는 요구가 인정되지 않았습니다.** 그 차이를 만든 것은 비트코인의 「존재」가 블록체인 기록으로 정의된다는 1절 (가)층의 성질과, 당시 분별관리 법령이 없었다는 법제의 사정입니다. 한국의 현행 가상자산이용자보호법이 예치금의 분리 예치·신탁(제6조)과 가상자산의 분리 보관 및 「동일한 종류와 수량의 가상자산을 실질적으로 보유」할 의무(제7조 제2항)를 둔 것은 이 두 자리에 해당하는 의무를 둔 것입니다 — 그 입법 목적이 이 사건이었는지는 입법 자료를 대조하지 않았으므로 적지 않습니다.[17] 그 조문이 어떤 성질의 권리를 전제하는지는 28장에서 묻습니다.

진단

① 무너진 층은 **거래소 장부와 내부통제입니다** — (다)층입니다. 법원이 유죄로 판단한 부분에서는 체인 송부 없이 장부 위에 체인에 없는 잔고가 생겨 거래되었습니다. 2014년 2월의 소실이 어느 층의 일인지는 **법원 문서가 판단하지 않았습니다.** 거래소는 (나)층의 성질(거래 가변성)을 들었고, 뒤의 2월 28일 공지에서는 해킹 가능성도 적었으며, 실증은 가변성으로 설명되는 양이 매우 작다고 보았고, 관재인은 전모 파악이 어렵다고 했습니다. ② 체인 밖입니다. ③ 코드가 한 일은 이용자 송금을 자동 확인해 잔고를 올리는 것이었고, 항소심이 적은 대로
「人が個々の取引やその履歴のデータ等を直接入力するようなことは想定されていなかった」
자리에 사람의 손이 들어갔습니다. ④ 민사재생 신청 → 기각·보전관리 → 파산 → 채권자 신청에 의한 민사재생(2018)이라는 도산 절차와, 전자적 기록 부정작출의 유죄·횡령의 무죄라는 형사 판단이 따라왔습니다. 상고·확정 여부는 미확인입니다.

3. The DAO (2016) — 계약 코드의 결함과 그 뒤의 결정

무엇이 적혀 있는가

미국 증권거래위원회(SEC)의 2017년 7월 25일 조사 보고서(Release No. 81207)가 사실관계를 가장 정돈된 형태로 적습니다. The DAO는 「a term used to describe a "virtual" organization embodied in computer code and executed on a distributed ledger or blockchain」의 한 사례이고, 독일 회사 Slock.it과 그 공동창업자들이 만들었습니다. 2016년 4월 30일부터 5월 28일까지 약 11억 5천만 DAO 토큰이 약 1,200만 ETH와 교환되어 팔렸고, 토큰 보유자는 제안에 투표하고 수익을 나누기로 되어 있었습니다.[18] 제안은 「큐레이터」가 화이트리스트에 올려야 투표에 부쳐졌고, 백서는 큐레이터가 「considerable power」를 가진다고 적었습니다.[19] 그리고 보고서 9쪽: 「On June 17, 2016, an unknown individual or group (the "Attacker") began rapidly diverting ETH from The DAO, causing approximately 3.6 million ETH—1/3 of the total ETH raised by The DAO offering—to move from The DAO's Ethereum Blockchain address to an Ethereum Blockchain address controlled by the Attacker (the "Attack"). Although the diverted ETH was then held in an address controlled by the Attacker, the Attacker was prevented by The DAO's code from moving the ETH from that address for 27 days.」 그 뒤 「Slock.it's co-founders and others endorsed a "Hard Fork"」, 「On July 20, 2016, after a majority of the Ethereum network adopted the necessary software updates, the new, forked Ethereum Blockchain became active.」 각주 32는 소수가 「a blockchain is immutable」이라는 개념에 반한다는 이유로 포크를 채택하지 않고 옛 체인을 계속 썼으며 그것이 「Ethereum Classic」이라 불린다고 적습니다.[20]

결함의 **기술적** 서술은 SEC 보고서가 아니라 공격 당일 이더리움 재단 블로그가 적었습니다(22장은 SEC가 「a flaw in The DAO's code」라고만 적었다는 점을 밝히고 이 서술을 이 장으로 넘겼습니다). 「The attack is a recursive calling vulnerability, where an attacker called the "split" function, and then calls the split function recursively inside of the split, thereby collecting ether many times over in a single transaction.」 그리고 같은 글의 한 문장이 1절의 층 구분을 그대로 말합니다. 「This is an issue that affects the DAO specifically; Ethereum itself is perfectly safe.」[21] 13장에서 본 EVM의 호출 모형으로 옮기면 — 이것은 이 책의 풀이이고, 1차 문서는 「split 함수의 재귀 호출」까지만 적습니다 — 계약이 잔고를 갱신하기 **전에** 외부 호출로 이더를 보냈고, 받는 쪽 계약이 그 호출 안에서 다시 같은 함수를 불러 아직 갱신되지 않은 잔고를 되풀이해 빼낸 모양입니다. 코드는 한 줄도 어긋나지 않고 적힌 대로 실행되었습니다. 어긋난 것은 작성자의 의도와 코드 사이였습니다.

재단 블로그는 처음에 소프트포크(The DAO와 그 자식 계약 — 같은 코드 해시를 가진 계정 — 의 잔고를 줄이는 거래를 무효로 하는 규칙)가 제안되었다고 적었고, 결국 하드포크가 되었습니다. 2016년 7월 20일 글은 이렇게 적습니다. 「Block 1920000 contained the execution of an irregular state change which transferred ~12 million ETH from the "Dark DAO" and "Whitehat DAO" contracts into the WithdrawDAO recovery contract. The fork itself took place smoothly, with roughly 85% of miners mining on the fork」. 옛 체인을 따르려는 사용자는 클라이언트를 갱신하되 `--oppose-dao-fork` 플래그로 실행하라고도 적었습니다.[22] 이 포크를 사후에 문서화한 EIP-779는 한 문장으로 성격을 못 박습니다. 「Unlike other hard forks, the DAO Fork did not change the protocol; all EVM opcodes, transaction format, block structure, and so on remained the same. Rather, the DAO Fork was an "irregular state change" that transferred ether balances from a list of accounts ("child DAO"

contracts) into a specified account (the "WithdrawDAO" contract).」[23]

진단

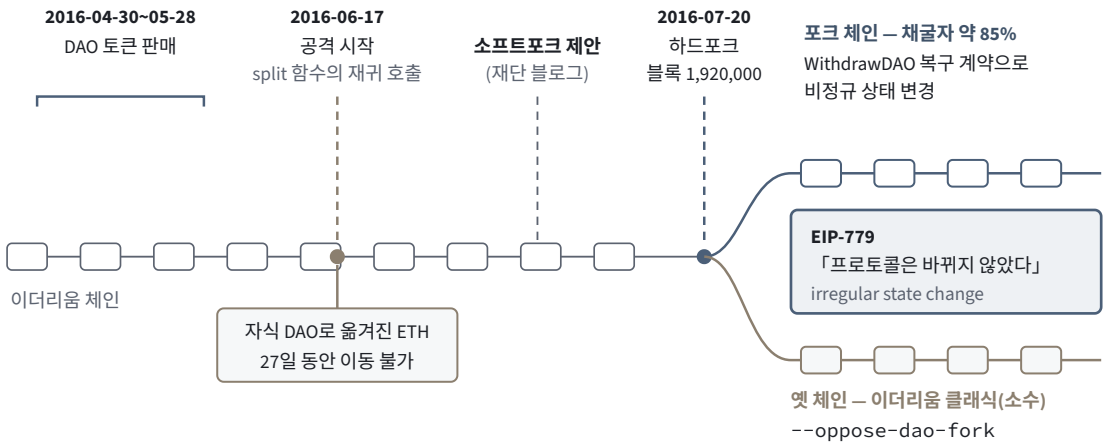


그림 26-3. The DAO 사건의 경과와 2016-07-20 블록 1,920,000에서의 분기. 날짜 사이 간격은 비례하지 않으며 ETH 수량은 넣지 않았다.

① 무너진 층은 둘입니다. 하나는 **응용층 계약 코드의 결함이고** — (다)층 — 다른 하나는 **거버넌스입니다**. 결함 자체는 「DAO에만 영향을 미치고 이더리움은 안전」했습니다. 그런데 그 뒤의 결정, 곧 노드 다수가 소프트웨어를 갱신해 특정 계정들의 잔고를 다른 계정으로 옮기는 「비정규 상태 변경」을 받아들인 것은 (가)층의 **규칙을 바꾸지 않고 원장의 상태를 바꾼** 일입니다. EIP-779의 문장이 정확히 그 둘을 가릅니다. 합의 규칙이 깨진 것이 아니고, 참여자들이 규칙 안에서(클라이언트 갱신이라는 조정) 상태를 고쳐 쓴 것입니다. 소수는 따르지 않았고 옛 체인은 살아남았습니다 — 23장이 「누가 진짜 원장을 정하는가」를 조정 게임으로 본 자리입니다. ② 공격은 체인 **안에서**, 계약 코드가 정한 상태 전이로 일어났습니다. ③ 코드는 적힌 대로 했고, 사람은 (가) 소프트포크를 제안하고 (나) 하드포크를 채택하거나 거부하는 결정을 했습니다. ④ 하드포크와 두 체인의 병존, 그리고 1년 뒤 SEC의 조사 보고서가 따라왔습니다. SEC는 「DAO Tokens are securities」라고 판단하면서도 집행 조치는 하지 않기로 했고, 「The automation of certain functions through this technology, "smart contracts," or computer code, does not remove conduct from the purview of the U.S. federal securities laws」라고 적었습니다.[24] 그 판단의 틀(Howey)은 27장에서, 증권성 자체는 28장에서 받습니다.

> **? 법률가의 질문**
>
> The DAO의 하드포크는 무엇이었습니까. 결함으로 빠져나간 자금을 되찾은 **구제**입니까 — 그렇다면 누구의 권한으로, 어떤 절차로, 어떤 기준에 따른 구제입니까. 공격자가 「코드대로」 취득한 것을 다수의 소프트웨어 갱신으로 거두어 간 **몰수**입니까 — 그렇다면 공격자에게 적법절차는 있었습니까, 그리고 「코드가 법」이라던 플랫폼 위에서 코드가 허용한 결과를 되돌린 것은 무엇을 위반한 것입니까. 아니면 참가자 다수가 자기들 사이의 규칙을 바꾼 **계약 변경**입니까 — 그렇다면 반대한 소수가 옛 체인에 남은 것은 계약의 해제입니까, 분할입니까. EIP-779는 「프로토콜은 바뀌지 않았고 비정규 상태 변경이 있었다」고 적습니다. 그 구별이 법적으로 의미가 있습니까. 이 책은 답하지 않고 5부의 쟁점 절을 가리킵니다. → 28장

4. Terra-Luna (2022) — 설계의 반사성과 발행자의 진술

무엇이 적혀 있는가

기제는 24장 2절이 국제결제은행의 2022년 보고서로 적었습니다. 알고리듬형 스테이블코인 UST는 「1달러어치의 루나」로 태환되게 함으로써 페그를 유지하려 했고, 「users would only be willing to exchange UST into Luna if Luna's market capitalisation exceeded that of UST」가 그 장치의 결정적 조건이었습니다.[25] 그 조건이 깨지면 상환이 루나를 더 찍고 늘어난 공급이 루나의 값을 내려 상환을 더 부르는 되먹임이 생긴다는 것이 24장 NB24-3이 모형으로 보인 나선입니다. 이 장은 그 기제를 되풀이하지 않고, 2022년 5월 뒤에 법원 문서가 무엇을 적었는지만 봅니다.

미국 연방뉴욕남부지방법원의 *SEC v. Terraform Labs Pte. Ltd. and Do Hyeong Kwon* 사건(No. 1:23-cv-01346-JSR)에서, 법원은 2023년 12월 28일 피고들이 미등록 증권을 모집·판매해 증권법 제5조를 위반했다는 SEC의 주장에 대해 약식판결을 했고, 2024년 4월 5일 배심은 사기 금지 조항 위반에 대해 평결했습니다. 평결지는 다음과 같습니다. 증권법 제17조 청구에서 Terraform은 「Liable」이고 주관적 요건은 「Recklessly」, Kwon은 「Liable」이고 「Intentionally」. 증권거래법 Rule 10b-5의 고의·무모한 위반 청구에서 둘 다 「Liable」. Terraform의 위반에 대한 지배인(control person) 책임에서 Kwon은 「Liable」.[26] 2024년 6월 12일의 최종판결은 피고들이 「consented to entry of this Final Judgment; waived findings of fact and conclusions of law; and waived any right to appeal」 했음을 적고, 금지명령과 함께 금전 제재를 정했습니다. Terraform에 부당이득 반환 3,586,875,883달러와 판결 전 이자 466,952,423달러, 민사제재금 420,000,000달러(합계 4,473,828,306달러는 델라웨어 파산법원의 도산 사건에서 확정되는 챕터 11 계획에 따라 지급하되, Terraform은 그 금액을 SEC의 일반 무담보 채권으로 다루는 계획의 인가를 구하기로 했다 — 7쪽), Kwon에 부당이득 반환 110,000,000달러(Terraform과 연대)와 이자 14,320,196달러, 민사제재금 80,000,000달러입니다.[27] 최종판결은 또 피고들이 「any crypto asset securities, including but not limited to UST, MIR, LUNA, wLUNA, and LUNA 2.0」의 매매에 관여하는 것을 금지하고, Terraform이 보유한 해당 토큰 지갑의 개인키를 「burn or destroy」하도록 했습니다.

형사 사건은 같은 법원의 *United States v. Do Hyeong Kwon*(No. 1:23-cr-00151-PAE)입니다. 이 장은 판결문(Judgment in a Criminal Case, ECF No. 63)만으로 씁니다. 피고인은 공소장(S1)의 제1s 소인 「18 U.S.C. § 371 Conspiracy to Defraud」과 제4s 소인 「18 U.S.C. § 1343 and 2 Wire Fraud」에 유죄를 인정했고(「pleaded guilty」), 범행 종료일은 두 소인 모두 2023년 3월 23일로 적혀 있습니다. 형은 합계 180개월입니다(제1s 소인 60개월, 제4s 소인 180개월, 동시 집행 — 선고 녹취록 기준이며, 판결문의 소인별 기재는 이와 다릅니다[29]). 그리고 2024년 12월 31일 미국 도착 이후의 구금 기간과 몬테네그로에서의 인도 전 구금 17개월 8일을 산입하도록 했습니다. 석방 후 감독(supervised release)은 부과하지 않았고, 특별부과금 200달러, 몰수는 별도 명령(Dkt. No. 47)에 따릅니다. 판결 선고일은 2025년 12월 11일, 담당 판사는 Paul A. Engelmayer입니다.[28] 판결문 2쪽에는 「the Court encourages any court in the Republic of Korea that may sentence the defendant to take this Court's sentence into account」와, 형기의 절반을 마친 뒤 한국으로의 이송 신청을 지지한다는 권고가 있습니다.

진단

① 무너진 층은 **경제 설계와 진술입니다**. 설계의 반사성은 24장이 모형으로 보였고, 법원 문서가 더한 것은 발행자와 그 대표의 진술이 증권법의 사기 금지 조항에 걸렸다는 **판단입니다** — 배심은 Terraform을 「Recklessly」, Kwon을 「Intentionally」로 가렸고, 형사 법원은 사기 공모와 전신 사기의 유죄 인정을 받았습니다. 무엇을 어떻게 진술했는지는 이 장의 1차 문서 범위(평결지·판결문)에 적혀 있지 않으므로 쓰지 않습니다. ② 체인 안과 밖이 겹칩니다. 교환 모듈은 체인 위에서 설계대로 돌았고, 진술은 체인 밖에서 이루어졌습니다. ③ 코드는 설계대로 했습니다 — 조건이 깨졌을 때 나선을 그리는 것까지 설계의 일부였습니다. 사람이 한 일은 설계를 택하고 그것을 설명한 일입니다. ④ 민사(약식판결·배심 평결·동의에 의한 최종판결), 형사(유죄 인정·180개월), 도산(델라웨어 파산법원)이 따라왔습니다. 「스테이블코인 보유자는 무엇을 가진 사람인가」는 24장이 던진 질문이고, 28장와 27장의 스테이블코인 열이 받습니다.

5. FTX (2022) — 고객 자산의 유용과 내부통제의 부재

무엇이 적혀 있는가

2022년 11월 11일(West Realm Shires Inc.는 11월 14일) FTX 그룹이 델라웨어 파산법원에 챕터 11을 신청하면서 새로 최고경영자가 된 John J. Ray III는 11월 17일 「Declaration of John J. Ray III in Support of Chapter 11 Petitions and First Day Pleadings」를 냈습니다. 그는 Enron을 비롯한 대형 도산을 맡아 온 사람인데, 5항에서 이렇게 적었습니다. 「Never in my career have I seen such a complete failure of corporate controls and such a complete absence of trustworthy financial information as occurred here. From compromised systems integrity and faulty regulatory oversight abroad, to the concentration of control in the hands of a very small group of inexperienced, unsophisticated and potentially compromised individuals, this situation is unprecedented.」[30] 선언서는 그룹을 네 「사일로」(FTX US 계열, Alameda, 벤처, FTX.com 계열)로 나누고, 「Each of the Silos was controlled by Mr. Bankman-Fried」라고 적습니다.[31] 그리고 통제의 부재를 항목별로 적습니다. 지출 승인은 「an on-line 'chat' platform where a disparate group of supervisors approved disbursements by responding with personalized emojis」로 이루어졌고,[32] 디지털 자산에 관해서는 「the use of an unsecured group email account as the root user to access confidential private keys and critically sensitive data」, 「the absence of daily reconciliation of positions on the blockchain」, 「the use of software to conceal the misuse of customer funds」, 「the secret exemption of Alameda from certain aspects of FTX.com's auto-liquidation protocol」, 「the absence of independent governance as between Alameda … and the Dotcom Silo」가 있었다고 적습니다.[33] 신청일 당일 최소 3억 7,200만 달러의 무단 이전이 있었고, 그 뒤 약 3억 달러 상당의 FTT 토큰이 「an unauthorized source」에 의해 추가 발행되었다는 것도 선언서가 적는 사실입니다.[34]

형사 사건은 미국 연방뉴욕남부지방법원의 *United States v. Samuel Bankman-Fried*(No. 1:22-cr-00673-LAK)입니다. 판결문(Judgment in a Criminal Case, ECF No. 424)은 피고인이 (S6) 공소장의 제1~7 소인 전부에 대해 「was found guilty」했음을 적고, 죄명을 전신 사기(제1·3 소인, 18 U.S.C. §§ 1343 and 2), 전신 사기 공모(제2·4 소인, § 1349), 증권 사기 공모(제5 소인, § 371), 상품 사기 공모(제6 소인, § 371), 자금세탁 공모(제7 소인, § 1956(h))로 적습니다. 선고일은 2024년 3월 28일, 판사는 Lewis A. Kaplan입니다.[35] 미국 연방제2항소법원은 2026년 6월 12일 원심을 확인(affirm)했습니다. 항소심

의견서는 1심 선고를 「25 years' imprisonment, to be followed by three years of supervised release」와 「a forfeiture of approximately $11 billion」으로 요약하고, 정부의 소추 이론을 이렇게 적습니다. 「The government's theory at trial was that Bankman-Fried secured investments in FTX by promising customers that their funds would be secure on the platform and used only for cryptocurrency transactions on FTX. Instead, Bankman-Fried freely transferred investor funds from FTX customer accounts to Alameda and elsewhere where he used those funds for purposes investors had not authorized, including for his own personal benefit.」 그리고 증거를 「conservatively stated, robust」라고 평가하면서, 「while Bankman-Fried assured customers, investors, regulators and the public that FTX customer funds were safe, he transferred billions of dollars from FTX customer accounts to Alameda and elsewhere」라고 적습니다.[36] 상고허가신청 여부는 이 책이 확인하지 않았습니다(2026-10 기준).

진단

① 무너진 층은 **사기와 내부통제입니다**. 기술도, 경제 설계도, 체인의 거버넌스도 아닙니다. 선언서가 적은 항목 — 공유 이메일 계정의 개인키 접근, 체인 상 포지션의 일일 대사 부재, 고객 자금 유용을 감추는 소프트웨어, 특정 계정의 자동 청산 면제 — 은 전부 (다)층의 **회사 안에서** 일어난 일입니다. ② 체인 밖입니다. 거래소 안의 「잔고」는 Mt.Gox와 같은 종류의 장부 숫자였고, 체인이 그것을 보증하지 않는다는 점도 같습니다. ③ 코드가 한 일이 있다면 그것은 사람이 시킨 일입니다 — 「the use of software to conceal the misuse of customer funds」. 사람이 한 일이 전부입니다. ④ 챕터 11, 7개 소인 전부의 유죄 평결과 25년형, 항소심의 확인이 따라왔습니다. 두 거래소 사건이 8년을 사이에 두고 같은 자리에서 무너졌다는 점은, 1절의 층 구분이 기술의 문제가 아니라 **장부를 누가 어떻게 지키는가의** 문제임을 보여 줍니다. 한국 가상자산이용자보호법 제6조·제7조가 의무를 둔 자리가 여기임은 2절에서 적었습니다.

6. 네 사건을 한 표에

아래 표는 2~5절의 진단을 모은 것입니다. 「무너진 층」의 괄호는 1절의 (가)(나)(다)입니다.

	MT.GOX (2014)	THE DAO (2016)	TERRA-LUNA (2022)	FTX (2022)
① 무너진 층	거래소 장부·내부통제(다). 2014-02 소실의 층은 법원 미판단 — 거래소는 (나) 거래 가변성을 들었고 2월 28일 공지에서는 해킹 가능성도 적음; 실증은 가변성으로 설명되는 양이 매우 작다고 봄	계약 코드의 결함(다) + 거버넌스(규칙은 바꾸지 않고 상태를 바꾼 다수의 결정)	경제 설계의 반사성(24장) + 발행자·대표의 진술(증권사기 평결, 사기 공모·전신 사기 유죄 인정)	사기 + 내부통제 부재(다)
② 체인 안팎	밖(장부 조작). 2014-02 소실의 층은 법원 미판단	안(계약 코드의 상태 전이) → 포크도 원장 상태의 변경	안(교환 모듈)과 밖(진술)	밖(회사 장부·은행·개인키 관리). 신청일의 무단 이전은 체인 위의 이전이나 그 경위는 선언서가 조사 중이라 적음
③ 코드와 사람	코드: 송금 자동 확인 → 잔고 갱신. 사람: 뒷받침 없는 잔고를 손으로 입력(「人が…直接入力するようなことは想定されていなかった」)	코드: 적힌 대로 재귀 호출 실행. 사람: 소프트포크 제안·하드포크 채택/거부	코드: 설계대로(조건이 깨지면 나선). 사람: 설계 선택과 진술	코드: 유용을 감추는 소프트웨어(사람의 지시). 사람: 전부
④ 사후 대응	민사재생 신청(2014-02-28) → 기각·보전관리(04-16) → 파산 개시(04-24) → 채권자 신청 민사재생 개시(2018-06-22). 형사: 1심 유죄(부정작출)·무죄(횡령), 항소 기각. 상고·확정 미확인	하드포크(블록 1,920,000, 2016-07-20) · 두 체인 병존 · SEC 조사 보고서(2017, 증권 판단·집행 조치 없음)	SEC 민사: 약식판결(2023-12-28)·배심 평결(2024-04-05)·동의 최종판결(2024-06-12). 형사: 유죄 인정·180개월(2025-12-11). 도산(Bankr. D. Del.)	챕터 11(2022-11-11, WRS는 11-14). 형사: 7개 소인 유죄·25년(2024-03-28), 항소심 확인(2026-06-12). 상고허가신청 미확인
1차 문서	東京地裁 2019. 3. 15.·東京高裁 2020. 6. 11. 판결, 파산개시 결정, 회사 공지, 관재인 보고서	SEC Release 81207, EIP-779, 재단 공지 2건	ECF 229·273(민사), ECF 63(형사)	Ray 선언서 D.I. 24, ECF 424, 2d Cir. 의견서

7. 「블록체인 자체는 깨지지 않았다」의 범위

이제 1절에서 유보한 문장을 범위를 정해 씁니다. **(가) 합의 규칙과 원장 무결성이 외부 공격으로 깨져 비트코인이나 이더리움의 원장이 뒤집힌 사건은 이 넷에 없습니다.** Mt.Gox와 FTX에서 문서가 인정한 것은 체인 밖 장부와 회사 안의 일이었고, Terra에서 법원이 판단한 것은 진술이었으며, The DAO의 공격은 응용층 계약의 일이었습니다. 이 책의 1부가 「변조 비용」으로 설명한 성질이 깨졌다고 적은 1차 문서는 네 사건 어디에도 없습니다 — 다만 Terra는 비트코인·이더리움이 아닌 자기 체인 위의 일이고, 그 체인의 합의 층을 이 장의 1차 문서는 다루지 않습니다.

다만 두 가지를 덧붙여야 합니다. 첫째, Mt.Gox에 관해서는 더 좁게 써야 합니다. 거래소는 당시 (나)층의 성질인 거래 가변성을 원인으로 들었으나 뒤의 2월 28일 공지에서는 해킹 가능성도 적었고, 실증 분석은 가변성 공격으로 설명되는 양이 매우 작다고 보았으며, 관재인은 전모 파악이 어렵다고 했고, 형사 1심·항소심 판결은 소실 원인을 판단하지 않았습니다. 그러니 「블록체인이 깨지지 않았다」는 문장을

Mt.Gox의 소실에 대해 **적극적으로** 쓰는 것은 법원 판단의 뒷받침이 없습니다. 이 장이 말할 수 있는 것은 「(가)층이 깨졌다는 문서가 없다」까지입니다. 둘째, The DAO의 하드포크는 (가)층이 깨진 것은 아니지만 **(가)층의 원장 상태가 다수의 결정으로 바뀐** 사건입니다. 「합의 규칙은 그대로였다」는 EIP-779의 문장과 「상태를 고쳐 썼다」는 사실은 모순이 아니라 한 쌍이고, 그 쌍이 바로 위 질문 상자가 묻는 것입니다. 변조 비용이 지키는 것은 「규칙을 따르지 않는 사람」으로부터의 원장이지, 「규칙을 바꾸기로 한 다수」로부터의 원장이 아닙니다 — 7장이 불변성을 수학이 아니라 비용이라고 적은 뜻이 여기서 한 번 더 드러납니다.

8. 정리

네 사건을 같은 표에 올리면 서사가 사라지고 구조가 남습니다. 두 거래소는 체인 밖 장부에서 무너졌고, 그 둘 사이의 차이는 장부를 거짓으로 쓴 사람이 대표 한 사람이었는가와 조직이었는가, 그리고 법제가 분별관리를 요구했는가였습니다. 토큰 발행자는 설계의 조건이 깨졌을 때 코드가 설계대로 나선을 그렸고, 법원이 판단한 것은 설계가 아니라 진술이었습니다. 계약 코드는 적힌 대로 실행되어 작성자의 의도를 배반했고, 그것을 되돌린 것은 코드가 아니라 사람들의 조정이었습니다. 어느 사건에서도 1부의 합의 규칙이 외부 공격에 깨졌다는 1차 문서는 없고, 그 문장은 1절의 범위 안에서만 참입니다. 법률가가 가져갈 것은 이 표의 **열입니다**. 어느 층이 무너졌는지를 가린 뒤에야 재산법·증권법·도산법·형법 가운데 어느 것이 들어올 자리인지가 보이고, 그 자리들을 27장과 28장이 서베이합니다. 그리고 이 네 사건이 「가상자산」·「crypto-asset」·「digital asset」·「暗号資産」·「cryptoasset」 가운데 어느 이름으로 불리는가에 따라 어느 법이 들어오는지가 달라진다는 것이 30장의 주제입니다.

이 장의 출처

- 東京地方裁判所 平成31年3月15日 判決(平成27年(刑わ)第2274号·第2842号, 平成28年(刑わ)第2248号, 私電磁的記**録**不正作出，同供用，業務上**横**領被告事件) — 主文, 「罪となるべき事実」, 「事実認定の補足**説**明」 第１~第６, 「量刑の理由」. (판례 DB HTML 전문). 「判示事項」란은 DB 편집물이라 인용하지 않았다. 상고 여부·확정일 미확인.
- 東京高等裁判所第４刑事部 令和2年6月11日 宣告 判決(令和元年(う)第948号) — 主文, 第１~第４. (courts.go.jp, 14쪽).
- 東京地方裁判所民事第２０部 平成26年4月24日 破産手続開始決定(平成２６年（フ）第３８３０号). (스캔본, OCR).
- 株式会社ＭＴＧＯＸ 공지 — 「民事再生手続開始の申立てに関するお知らせ」(2014-02-28, 일·영), 「当社保有ビットコインの**残**高に関するお知らせ」(2014-03-20), 「民事再生手続開始申立ての棄却、保全管理命令のお知らせ」(2014-04-16). 채무자의 진술로만 썼다.
- 破産管財人·再生管財人 小林信明 보고서·공지 — 제1회(2014-07-23)·제6회(2016-05-25)·제10회(2018-03-07) 채권자집회 보고서 영문부, 「Announcement of Commencement of Civil Rehabilitation Proceedings」(2018-06-22).
- MtGox Team, 2014-02-07 인출 중단 공지 — Business Insider 2014-02-07 기사 전재본으로 대조(**2차 전재**); 2014-02-10 보도자료 「Addressing Transaction Malleability」 — Decker & Wattenhofer (2014) 6쪽의 전재 문장으로 인용(원 URL 소멸, 웨이백 스냅숏 20140210122955 로 위치를 확인했다).

- Christian Decker & Roger Wattenhofer, "Bitcoin Transaction Malleability and MtGox," arXiv:1403.6676 (2014); *ESORICS 2014*, LNCS, pp. 313–326, DOI 10.1007/978-3-319-11212-1_18 — §3 Timeline(6~7쪽), §4.1(9쪽), §6 Conclusion(12쪽). (arXiv판 쪽).
- SEC, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Exchange Act Release No. 81207 (July 25, 2017) — 1~3·7~9·11~16쪽, 각주 2·32.
- Vitalik Buterin, "CRITICAL UPDATE Re: DAO Vulnerability" (2016-06-17); "Hard Fork Completed" (2016-07-20), Ethereum Foundation Blog. HTML(2026-10-25 다시 열어 축어 대조).
- Casey Detrio, "EIP-779: Hardfork Meta: DAO Fork" (created 2017-11-26, Final). https://eips.ethereum.org/EIPS/eip-779 (HTML, 2026-10-25 축어 대조).
- Bank for International Settlements, *Annual Economic Report 2022*, ch. III, Box A(82쪽) — 24장과 같은 자리.
- *SEC v. Terraform Labs Pte. Ltd. and Do Hyeong Kwon*, No. 1:23-cv-01346-JSR (S.D.N.Y.) — Verdict, ECF No. 229 (2024-04-05); Final Judgment, ECF No. 273 (2024-06-12, 스캔본 OCR). `sdny-2024-sec-v-terraform-final-judgment.pdf` (RECAP). SEC 보도자료 2024-73·Litigation Release 는 봇 차단으로 열지 못해 쓰지 않았다.
- *United States v. Do Hyeong Kwon*, No. 1:23-cr-00151-PAE (S.D.N.Y.), Judgment in a Criminal Case, ECF No. 63 (imposed 2025-12-11, filed 2025-12-12). (PACER, 스캔본 OCR).
- *United States v. Do Hyeong Kwon*, No. 1:23-cr-00151-PAE (S.D.N.Y.), Transcript of Proceedings re: Sentence held on 12/11/2025, ECF No. 67 (filed 2025-12-30), 163쪽. (CourtListener RECAP 사본, 167쪽).
- Declaration of John J. Ray III, *In re FTX Trading Ltd., et al.*, Case No. 22-11068 (JTD) (Bankr. D. Del.), D.I. 24 (2022-11-17) — ¶¶1·4~10·62~71. (RECAP).
- *United States v. Samuel Bankman-Fried*, No. 1:(S6)22-CR-673-001 (LAK) (S.D.N.Y.), Judgment in a Criminal Case, ECF No. 424 (imposed 2024-03-28). (RECAP, 스캔본 OCR).
- *United States v. Bankman-Fried*, No. 24-961-cr (2d Cir. June 12, 2026), DktEntry 73.1 — 1~11쪽. (RECAP). 상고허가신청 여부 미확인.
- 「가상자산 이용자 보호 등에 관한 법률」(현행 법률 제21857호, 시행 2026. 10. 2.) 제6조·제7조 — 국가법령정보센터(Open API, 2026-10-25 조회).
- 외국 법원의 한국어 표기는 법원 「외국 법원 및 사법관련 국제기구 번역 가이드라인」(검토의견판,) 을 따랐다 — 도쿄지방재판소·도쿄고등재판소, 미국 연방뉴욕남부지방법원·미국 연방제2항소법원·미국 연방델라웨어파산법원.
- 이 장은 코드·노트북이 없다(설계). 가격·시가총액·수익률 자료는 쓰지 않았고, 금액은 판결·결정·채무자 공지에 적힌 것을 한 번씩만 적었다.

각주

1. 株式会社ＭＴＧＯＸ, 「民事再生手続開始の申立てに関するお知らせ」(平成26年2月28日), 1~2쪽 「１. 当社の財務状況と申立てに至った経緯及び理由」. 원문: 「資産総額　3,841,866,163 円 / 流動負債総額 6,501,119,371 円」; 「当社の流動負債が増大した背景には，ビットコインの消失，預り金の消失という原因が考えられ」; 「平成２６年２月初め頃，ビットコインのシステムのバグを悪用した不正アクセスにより，ビットコインの送金（ビットコインの引出）が正常に完了しない取引が増え」;

「ユーザーの取引履歴上のビットコイン保有高であるおよそ７５万ビットコイン，及び，
当社自身の取引履歴上の保有ビットコインである約１０万ビットコインのほぼ全てがなくなっていることが判明しました」；「金額は調査中であり変動する可能性が高いが最大約２８億円程度」；
「第三者によるハッキングによる被害を含めて，複数の原因があると見込まれるため」. 사건번호 平成２６年（再）第１２号. (일본어 원문), `-en.pdf` (회사가 낸 영문 번역, 「[Translation from the Japanese]」 표시). 이 문서는 채무자 자신의 진술이고, 그 내용이 참이라는 법원의 판단이 아니다.

2. 株式会社ＭＴＧＯＸ, 「当社保有ビットコインの残高に関するお知らせ」(平成26年3月20日), 1쪽. 원문:
「平成２６年３月７日（金），当社が平成２３年６月以前に利用していた旧式ウォレット内に約２０万ＢＴＣ（１９万９９９９．９９ＢＴＣ）が残存していることを確認いたしました」；
「消失したＢＴＣの数量は約６５万ＢＴＣとなるものと見込んでおります（ただし，
ＢＴＣの消失の原因や消失したＢＴＣの数量は現在調査中ですので，
調査の結果により数量が変動する可能性があります。)」.

3. 東京地方裁判所民事第２０部 平成26年4月24日 破産手続開始決定(平成２６年（フ）第３８３０号), 1쪽. 원문:
「債務者株式会社MTGOXについて破産手続を開始する。／理由／一件記録によれば，
債務者が支払不能の状態にあることが認められる。」 파산관재인 小林信明. 스캔본이라 OCR로 읽었고(사건번호는 영문 번역면 OCR 「88380」이 아니라 일본어면 OCR과 2014년 관재인 보고서 머리의 텍스트층 「３８３０」을 따랐다), 기각·보전관리명령은 회사 공지 「民事再生手続開始申立ての棄却、保全管理命令のお知らせ」(平成26年4月16日). `mtgox-2014-rehabilitation-dismissal-notice.pdf`.

4. 파산관재인 보고서 영문부. 제1회 채권자집회 보고서(2014-07-23) 「V. Investigation of disappearance of BTC, etc.」: "I began investigation of the existing assets of the bankrupt entity, background behind the disappearance of BTC, etc. by delegating such work to Deloitte Touche Tohmatsu LLC and ReEx Accounting Firm. Such investigation still continues now and the completion date is undetermined as of today." 제6회 보고서(2016-05-25) 「III. 1. Status of Investigation」: "Due to the circumstances, such as the limited amount of information that exists, I believe it will be difficult to understand the full picture; however, I am planning to continue these investigations, as much as possible." 제10회 보고서(2018-03-07) 「IV.」: "On July 26, 2017, the U.S. Department of Justice announced that Alexander Vinnik, manager of a cryptocurrency exchange, was arrested and indicted for involvement in money laundering of more than 4 billion dollars. The allegations in the indictment include the fact that he obtained funds by hacking MtGox. I have contacted the U.S. Department of Justice through a U.S. law firm and have made a request for information." `mtgox-trustee-2016-sixth-creditors-meeting-report.pdf`, `mtgox-trustee-2018-tenth-creditors-meeting-report.pdf`. 미국 형사 절차의 원문은 대조하지 않았고, 이 책은 관재인이 그 사실을 보고했다는 것만 적는다.

5. Civil Rehabilitation Trustee Nobuaki Kobayashi, "Announcement of Commencement of Civil Rehabilitation Proceedings" (2018-06-22, 2018-10-26 갱신), 1쪽. 원문: "some of the creditors of MTGOX Co., Ltd. ('MTGOX') filed a petition for commencement of civil rehabilitation proceedings against MTGOX with the Tokyo District Court on November 24, 2017."; "at 5:00 p.m. on June 22, 2018, the Tokyo District Court issued an order of the commencement of civil rehabilitation proceedings for MTGOX. As a result, the previously ongoing bankruptcy proceedings were stayed." 사건 표시 「Tokyo District Court 2017 (sai) no. 35」.

6. MtGox Team, 2014-02-07 공지. 원문 아카이브(web.archive.org 스냅숏 20140207054536)는 이 판에서는 열지 못했고, Business Insider 기사(Joe Weisenthal, "Bitcoin Plunges As Major Exchange Mt. Gox Halts All Withdrawals," 2014-02-07)가 전재한 본문으로 대조했다 — **2차 전재다**. 전재된 원문: "In our efforts to resolve the issue being encountered by various bitcoin withdrawals, it was determined that the increase in the flow of withdrawal requests has hindered our efforts on a technical level. To understand the issue thoroughly, the system needs to be in a static state. In order for our team to resolve the withdrawal issue it is necessary for a temporarily pause on all withdrawal requests to obtain a clear technical view of the current processes." (https://www.businessinsider.com/mtgox-halts-withdrawals-2014-2 , 2026-10-25 접근). Decker & Wattenhofer (2014), 6쪽도 이 공지가 「거래 가변성을 언급하지 않는다」고 적는다("Specifically [10] does not mention transaction malleability").

7. MtGox Team, press release (2014-02-10), 「Addressing Transaction Malleability」. 원 URL `https://www.mtgox.com/press_release_20140210.html` 은 소멸했고, 웨이백 스냅숏 20140210122955 로 위치를 확인했다. 이 장의 인용 문장은 Christian Decker & Roger Wattenhofer, "Bitcoin Transaction Malleability and MtGox," arXiv:1403.6676 (2014), 6쪽이 전재한 것이다: "Addressing Transaction Malleability: MtGox has detected unusual activity on its Bitcoin wallets and performed investigations during the past weeks. This confirmed the presence of transactions which need to be examined more closely. Non-technical Explanation: A bug in

the bitcoin software makes it possible for someone to use the Bitcoin network to alter transaction details to make it seem like a sending of bitcoins to a bitcoin wallet did not occur when in fact it did occur. Since the transaction appears as if it has not proceeded correctly, the bitcoins may be resent. MtGox is working with the Bitcoin core development team and others to mitigate this issue." 같은 쪽은 2월 10일 공지가 가변성을 「sole cause」로 지목했다고 정리한다. (arXiv판; 게재본은 *ESORICS 2014*, LNCS, pp. 313–326, DOI 10.1007/978-3-319-11212-1_18).

8. Decker & Wattenhofer (2014), 12쪽 §6 Conclusion. 원문은 본문 인용과 같고 마지막 문장: "Even if all of these attacks were targeted against MtGox, MtGox needs to explain the whereabouts of 849,600 bitcoins." 측정 방법은 §4(네트워크 관측 노드가 수집한 충돌 거래 집합에서 서명 인코딩만 다른 쌍을 가변성 공격으로 분류). 자료는 2013년 1월부터의 네트워크 관측이고, 저자들은 노드가 모든 거래를 보았다고 보장하지 않는다. 이 책은 이 수치를 「가변성 공격의 관측 상한」으로 읽고, 소실의 원인을 적극적으로 특정하는 데 쓰지 않는다.

9. 東京地方裁判所 平成31年3月15日 判決(平成27年(刑わ)第2274号·第2842号, 平成28年(刑わ)第2248号, 刑事第６部 裁判長裁判官 中山大行), 主文. 원문: 「被告人を懲役２年６月に処する。／
この裁判が確定した日から４年間その刑の執行を猶予する。／…／本件公訴事実中，各業務上横領，
平成２９年３月１０日付け予備的訴因並びに罪名及び罰条追加請求書記載の各業務上横領（予備的訴因１）
及び各会社法違反（予備的訴因２）の点については，被告人は無罪。」 판결문 말미의 작성일 표기는
「平成３１年３月１８日」이고 선고일은 3월 15일이다(求刑 懲役１０年). 원문: (판례 DB HTML 전문; 「判示事項」란은 DB 편집물이라 쓰지 않았다).

10. 東京高等裁判所第４刑事部 令和2年6月11日 宣告 判決(令和元年(う)第948号 私電磁的記録不正作出, 同供用被告事件), 1쪽. 원문: 「本件控訴を棄却する。」; 「なお原審では，被告人は，…業務上横領（予備的訴因会社法違反）の罪にも問われ，原判決はこれらの罪については無罪としたが,
この点につき検察官は控訴しなかった。」 판결문 말미의 날짜 표기는 「令和２年６月１８日」이다. (courts.go.jp 공개 PDF, 14쪽).

11. 東京地裁 平成31. 3. 15. 判決, 「事実認定の補足説明」 第１の２(５)ウ. 원문: 「本件取引システムにおいては,
利用者間でビットコイン取引が行われても，本件取引システム上の口座残高に増減が生じるだけで,
本件各銀行口座等で管理されている金銭残高に実際の変動が生じることはなく，また，ビットコインも同様に,
本件取引システム上の口座残高に変動は生じるが，ブロックチェーン上に変動は生じない。そして，利用者が,
Ｂ社に対し，金銭の返金やビットコインの送付を請求し，Ｂ社が，金銭を利用者の銀行口座に送金し，あるいは,
ビットコインをＢ社が管理するビットコインアドレスから利用者のビットコインアドレスに送付した段階で,
実際の変動が生じる。」 같은 곳(４): 이용자 송금액과 자사 금전이 은행계좌에서 분별관리되지 않았고 경비도 그 계좌에서 지급되었다(「利用者が送金した金銭と自社の金銭が分別して管理されておらず，かつ,
本件各銀行口座等から，Ｂ社の経費等が支払われていた」).

12. 東京地裁 平成31. 3. 15. 判決, 第１の２(３) 괄호 안. 원문은 본문 인용과 같다. 같은 절은 비트코인을 「ブロックチェーン上に記録されている状態を「存在」という」라고 정의해 쓴다(第１の２(１)).

13. 東京地裁 平成31. 3. 15. 判決, 第２の２(２)~(４). 원문(발췌): 「Ｅ社を買収したＤ社は，利用者に対し,
ブロックチェーン上には存在しない１万７０００ＢＴＣの返還債務を引き受けることとなった」;
「残高約１３米ドルのＣ口座の米ドル口座に係る口座履歴情報に,
残高を１５万米ドル増加させる旨の情報を入力し…合計約１５万米ドルで合計約１万８０００ＢＴＣを購入した。
被告人は，同月２０日，Ｃ口座のビットコイン口座の残高を１万７５００ＢＴＣ削除した」;
「平成２３年１１月１４日から平成２４年４月１３日までの間，合計８回にわたって,
ビットコインをＢ社が管理するビットコインアドレスに送付することなく，…
残高を３０万ＢＴＣ増加させる旨の情報を入力し…利用者との間で，合計約３０万ＢＴＣを売却し,
その対価として，Ｃ口座の米ドル口座に約１３０万米ドルを得た」;「３０万ＢＴＣ増加行為等の結果,
本件取引システム上では,
ブロックチェーン上には存在しないビットコイン３０万ＢＴＣが利用者間で取引されることとなった。そこで,
被告人は，同３０万ＢＴＣを回収する目的で，…平成２５年２月１４日から同年９月２７日までの間,
合計２１回にわたり，…残高を合計３３５０万米ドル増加させる旨の情報を入力し…，利用者から,
代金約２９７７万米ドルで，合計約３０万ＢＴＣを購入した後,
Ｃ口座のビットコイン口座の残高を約３０万ＢＴＣ削除した」. 항소심 판결 2~3쪽은 같은 사실을 「原判決が認定した前提事実等の骨子」로 되풀이한다(판결 표기는 「Ｄ社」·「Ｂ社」 대신 「Ｔ社」·「Ｍ社」, 「Ｃ口座」 대신 「Ａ口座」). 본문의 금액·수량은 판결에 적힌 것이고, 이 장은 그것을 한 번씩만 적는다.

14. 東京地裁 平成31. 3. 15. 判決, 第２の３(３)ア(ア)·(イ)·(オ). 원문은 본문 인용과 같고, (オ) 전문:

「以上を総合すれば，本件取引システム上では，
ブロックチェーン上に存在しないビットコインが取引されることは想定されておらず，
利用規約及び利用者間のビットコインの取引の実態並びにビットコインの性質等からすれば，
本件取引システムの口座残高は，利用者に対し，
ブロックチェーン上に存在するビットコインに裏付けられたものであることを保証しており，
利用者もそのように認識していたと認められる。したがって，B社の意思は，
本件取引システム上の利用者の口座残高に相当するビットコインがB社管理のブロックチェーン上にも存在すべきであり，B社管理のビットコインアドレスにおけるビットコインの受送付等による裏付けがある場合に限って，
本件取引システム上の利用者の口座残高を増加させるべきであるというものであり，
被告人に与えられた権限もこの範囲に限定されていたと認められる。」 이용규약 조항의 영문 원문: "Seller warrants that the Bitcoins offered to sell or to transfer correspond to actual Bitcoins."

15. 東京高裁 令和2. 6. 11. 判決, 8~9쪽(第３の１)·10~11쪽(第３の２(３)). 원문: 「…とした原判決の判断は，概ね同旨をいうものとして是認できる」; 「本件取引システムは，…極めて大量の取引を迅速かつ正確に行うための自動プログラムであり，被告人が行ったような，人が個々の取引やその履歴のデータ等を直接入力するようなことは想定されていなかったと認められる」; 「電磁的記録の証明機能は，それが作出されるべきシステムの設置運営主体の意思に基づき，正当に作出されることが当然の前提とされているもので，このように正当に作られたものであることの信頼を確保する必要があるという観点から解釈すべきと解される」.

16. 東京地裁 平成31. 3. 15. 判決, 第２の３(２)イ·第３の２·第４·第５. 원문: 「利用者がB社の指定する銀行口座に送金した金銭は，B社に帰属すると認められるのであり，金銭に関しては，本件取引システム上の利用者の口座残高に相当する金銭がB社の本件各銀行口座等にも存在すべきであるとはいえず」; 「本件当時頃，前記（ア）の管理方法を禁止する旨の法令はなく，利用規約上も，分別管理を前提とする規定…はなかった」; 「本件各振込送金は，被告人に対する貸付けとして行われたものであり，返済の現実的可能性があると認めるのが相当である」. 送金의 명세(2013년 9~12월, 피고인 명의 계좌 등으로 500만엔·500만엔·1,000만엔, 가구 구입대금 600만 9,245엔, 사업양수대금 3억 1,500만엔)는 第３の１. 이 장은 무죄 판단의 근거 구조만 옮기고 그 당부를 평가하지 않는다.

17. 「가상자산 이용자 보호 등에 관한 법률」(법률 제19563호로 제정, 현행 법률 제21857호, 시행 2026. 10. 2.) 제6조 제1항·제7조 제2항. 원문: 「가상자산사업자는 이용자의 예치금(…)을 고유재산과 분리하여 「은행법」에 따른 은행 등 대통령령으로 정하는 공신력 있는 기관(이하 "관리기관"이라 한다)에 대통령령으로 정하는 방법에 따라 예치 또는 신탁하여 관리하여야 한다.」; 「가상자산사업자는 자기의 가상자산과 이용자의 가상자산을 분리하여 보관하여야 하며, 이용자로부터 위탁받은 가상자산과 동일한 종류와 수량의 가상자산을 실질적으로 보유하여야 한다.」 법제처 국가법령정보센터(Open API, MST 290735, 2026-10-25 조회). 이 조문이 Mt.Gox를 염두에 두고 입법되었다는 서술은 하지 않는다 — 입법 자료를 대조하지 않았다.

18. SEC, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Exchange Act Release No. 81207 (July 25, 2017), 1~3쪽. 원문: "The DAO is one example of a Decentralized Autonomous Organization, which is a term used to describe a 'virtual' organization embodied in computer code and executed on a distributed ledger or blockchain."(1쪽); "From April 30, 2016 through May 28, 2016, The DAO offered and sold approximately 1.15 billion DAO Tokens in exchange for a total of approximately 12 million Ether ('ETH')"(2~3쪽). 보고서 각주 2는 이 문서가 「does not constitute an adjudication of any fact or issue」라고 밝힌다 — 사실인정이 아닌 조사 보고다. 달러 환산액(3쪽)은 옮기지 않는다.

19. SEC (2017), 7~8쪽. 원문: "Before any proposal was put to a vote by DAO Token holders, it was required to be reviewed by one or more of The DAO's 'Curators.' … According to the White Paper, the Curators of a DAO Entity had 'considerable power.' The Curators performed crucial security functions and maintained ultimate control over which proposals could be submitted to, voted on, and funded by The DAO."

20. SEC (2017), 9쪽 「4. Security Concerns, The 'Attack' on The DAO, and The Hard Fork」와 각주 32. 원문은 본문 인용과 같고, 각주 32: "A minority group, however, elected not to adopt the new Ethereum Blockchain created by the Hard Fork because to do so would run counter to the concept that a blockchain is immutable. Instead they continued to use the former version of the blockchain, which is now known as 'Ethereum Classic.'" 같은 쪽: "The Hard Fork had the effect of transferring all of the funds raised (including those held by the Attacker) from The DAO to a recovery address, where DAO Token holders could exchange their DAO Tokens for ETH."

21. Vitalik Buterin, "CRITICAL UPDATE Re: DAO Vulnerability," Ethereum Foundation Blog (2016-06-17). 원문은

본문 인용과 같고, 이어서: "The leaked ether is in a child DAO ...; even if no action is taken, the attacker will not be able to withdraw any ether at least for another ~27 days (the creation window for the child DAO)."; "A software fork has been proposed, (with NO ROLLBACK; no transactions or blocks will be 'reversed') which will make any transactions that make any calls/callcodes/delegatecalls that reduce the balance of an account with code hash 0x7278d050... lead to the transaction ... being invalid". https://blog.ethereum.org/2016/06/17/critical-update-re-dao-vulnerability (HTML, 2026-10-25 다시 열어 축어 대조). 자금이 옮겨진 자식 DAO 주소는 이 글이 적고 EIP-779 목록에도 있으나, 공격 거래 해시는 어느 1차 문서로도 대조하지 않았으므로 본문에 적지 않는다.

22. Vitalik Buterin, "Hard Fork Completed," Ethereum Foundation Blog (2016-07-20). 원문은 본문 인용과 같고, 이어서: "The recovery contract is already returning DAO token holders' ether; about 4.5 million ETH has been sent to DAO token holders, and about 463000 ETH is in the control of the curator"; "If any users continue to be interested in following the non-fork chain, they should still update, but run with the --oppose-dao-fork flag enabled, though they should beware of transaction replay attacks". https://blog.ethereum.org/2016/07/20/hard-fork-completed (HTML, 2026-10-25 축어 대조). 「~12 million ETH」는 SEC 보고서의 「approximately 3.6 million ETH」(공격자가 옮긴 양)와 다른 양이다 — 포크가 옮긴 것은 공격자 보유분을 포함한 The DAO 계열 계약의 잔고 전부다.

23. Casey Detrio, "EIP-779: Hardfork Meta: DAO Fork," Ethereum Improvement Proposals (created 2017-11-26, Status Final), Abstract. 원문은 본문 인용과 같다. Specification: "Codename: DAO Fork / Activation: Block == 1,920,000 on Mainnet"; 블록 1,880,000 에서 The DAO(0xbb9bc244d798123fde783fcc1c72d3bb8c189413)·extraBalance·자식 DAO 들의 목록 L 을 정하고 그 잔고를 WithdrawDAO(0xbf4ed7b27f1d666546e30d74d50d173d20bca754)로 옮긴다. https://eips.ethereum.org/EIPS/eip-779 (HTML, 2026-10-25 축어 대조).

24. SEC (2017), 1~2쪽. 원문: "The Commission has determined not to pursue an enforcement action in this matter based on the conduct and activities known to the Commission at this time."; "Based on the investigation, and under the facts presented, the Commission has determined that DAO Tokens are securities under the Securities Act of 1933 ('Securities Act') and the Securities Exchange Act of 1934 ('Exchange Act')."(1쪽); "The automation of certain functions through this technology, 'smart contracts,' or computer code, does not remove conduct from the purview of the U.S. federal securities laws."(2쪽). 보고서의 Howey 적용(11~15쪽)은 큐레이터와 Slock.it의 「undeniably significant」 노력(12쪽)과 토큰 보유자 투표권의 한계(13~15쪽)를 근거로 든다.

25. Bank for International Settlements, *Annual Economic Report 2022*, ch. III, 82쪽 Box A "The collapse of the TerraUSD stablecoin". 원문은 24장 각주 4 에 전재했다. BIS 보고서는 중앙은행 간 기구의 분석 보고서로, 이 책의 1차 문서 기준(판결·공식 조사·도산 문서) 가운데 「공식 조사」에 넣지 않고 기제 설명의 출처로만 쓴다.

26. *SEC v. Terraform Labs Pte. Ltd. and Do Hyeong Kwon*, No. 1:23-cv-01346-JSR (S.D.N.Y.), Verdict, ECF No. 229 (filed 2024-04-05), 1~2쪽. 원문: "1. On the SEC's first claim, under Section 17, we the jury find defendant Terraform Labs Pte. Ltd. ('Terraform'): Liable √"; "2. ... we the jury find that Terraform acted ...: Recklessly √"; "3. ... we the jury find defendant Do Hyeong Kwon ('Kwon'): Liable √"; "4. ... Kwon acted ...: Intentionally √"; "5. On the SEC's second claim, for intentional or reckless violation of Rule 10b-5, we the jury find defendant Terraform: Liable √"; "6. ... Kwon: Liable √"; "7. On the claim for control person liability regarding Terraform's violation of Rule 10b-5, we the jury find defendant Kwon: Liable √". (RECAP 사본). 2023-12-28 약식판결(Dkt. No. 149)은 최종판결 1~2쪽의 경과 서술에서 확인했다: "On December 28, 2023, the Court granted the SEC's Motion for Summary Judgment and found that Defendants engaged in unregistered offers and sales of securities in violation of Securities Act Sections 5(a) and 5(c) ...".

27. 같은 사건, Final Judgment Against Defendants Terraform Labs Pte Ltd. and Do Hyeong Kwon, ECF No. 273 (filed 2024-06-12), 2·5~8쪽(스캔본, OCR로 읽고 금액은 자릿수 단위로 재확인). 원문: "Defendants having entered a general appearance; consented to the Court's jurisdiction over Defendants and the subject matter of this action; consented to entry of this Final Judgment; waived findings of fact and conclusions of law; and waived any right to appeal from this Final Judgment."(2쪽); "Terraform is liable for disgorgement totaling $3,586,875,883, ...together with prejudgment interest thereon in the amount of $466,952,423. Of these amounts, Kwon is jointly and severally liable with Terraform in the amount of $110,000,000 of disgorgement, plus prejudgment interest thereon in the amount of $14,320,196. ... the Court further imposes a civil penalty on Terraform of $420,000,000. The $4,473,828,306 in total monetary remedies against Terraform shall be payable as provided for in the confirmed Chapter 11 Plan in the Bankruptcy Case"(6쪽); "Kwon is liable for disgorgement totaling $110,000,000, together with prejudgment interest thereon in the amount of $14,320,196.

... the Court further imposes a civil penalty on Kwon of $80,000,000."(8쪽); 도산 사건 표시 "In re: Terraform Labs Pte. Ltd., No. 24-10070 (BLS) (Bankr. D. Del.)"(5쪽); "Defendants shall otherwise burn or destroy private keys in Terraform's possession to wallets or blockchain addresses holding Terraform Crypto Assets"(5쪽). 이 장은 금액을 판결문에서 읽었고 SEC 보도자료(2024-73)는 봇 차단으로 열지 못해 쓰지 않았다. 「누가 얼마를 벌었나」가 아니라 법원이 명한 반환·제재의 크기를 한 번 적는 것이다.

28. *United States v. Do Hyeong Kwon*, No. 1:23-cr-00151-PAE (S.D.N.Y.), Judgment in a Criminal Case, ECF No. 63 (Date of Imposition 12/11/2025, filed 12/12/25), 1~5쪽(5쪽 스캔본, OCR). 원문: "pleaded guilty to count(s) 1s & 4s of the S1 Indictment"; "18 U.S.C. § 371 Conspiracy to Defraud 3/23/2023 1s / 18 U.S.C. § 1343 and 2 Wire Fraud 3/23/2023 4s"(1쪽); "One hundred eighty (180) months. Sixty (60) months on count 1 and one hundred (120) months on count 4, these terms to run concurrently. The defendant shall be given credit for time served for both (1) his time in custody since December 31, 2024, when he arrived in the United States; and (2) 17 months and 8 days that the defendant was in pre-extradition custody in Montenegro."; "Separately, the Court encourages any court in the Republic of Korea that may sentence the defendant to take this Court's sentence into account. The Court supports any application supported by the United States Attorney's Office with respect to a transfer of the defendant to the Republic of Korea after completion of half of his custodial sentence."(2쪽); "The Court declines to impose a term of supervised release."(3쪽); "Assessment $ 200.00"(4쪽); "See Order of Forfeiture filed separately at Dkt. No. 47."(5쪽). (PACER 사본). 유죄 인정의 사실관계는 판결문에 적혀 있지 않으므로 이 장은 쓰지 않는다. 대한민국에서의 절차는 계속 중일 수 있어 사실 서술도 하지 않는다. 소인별 형기 기재는 선고 녹취록과 다르다 — 다음 각주.

29. 같은 사건, Transcript of Proceedings re: Sentence held on 12/11/2025, ECF No. 67 (filed 12/30/25), 163쪽(녹취록 쪽 169). 원문: "it is the judgment of the Court that you are to serve a sentence of 15-years imprisonment, 180 months in the custody of the Bureau of Prisons." / "As to the particular counts, I impose a sentence of five years, 60 months, on Count One, which is the maximum sentence available, and a sentence of 15 years, 180 months, on Count Four, the terms to run concurrently." 판결문(ECF No. 63) 2쪽의 「Sixty (60) months on count 1 and one hundred (120) months on count 4, these terms to run concurrently」는 이 구두 선고와 다르다(스캔 이미지로 확인한 원문 그대로이며 OCR 오류가 아니다). 구두 선고와 서면 판결이 어긋날 때 어느 쪽이 우선하는지는 이 책이 원문을 확인하지 않았으므로 적지 않는다. CourtListener RECAP 도킷(2026-10-27 조회) 기준 ECF No. 63 뒤에 정정 판결(amended judgment) 기재는 없다 — RECAP은 올라온 문서만 보이므로 도킷 전체의 부존재 확인은 아니다. (CourtListener RECAP 사본, 167쪽).

30. Declaration of John J. Ray III in Support of Chapter 11 Petitions and First Day Pleadings, *In re FTX Trading Ltd., et al.*, Case No. 22-11068 (JTD) (Bankr. D. Del.), D.I. 24 (filed 2022-11-17), ¶¶1·4~5. 원문은 본문 인용과 같고, ¶4: "I have over 40 years of legal and restructuring experience. I have been the Chief Restructuring Officer or Chief Executive Officer in several of the largest corporate failures in history. I have supervised situations involving allegations of criminal activity and malfeasance (Enron)." (RECAP 사본, 30쪽). 선언서는 선서 진술(「under penalty of perjury」)이고 법원의 사실인정이 아니다. ¶7은 「all facts set forth in this declaration ... are based on my personal knowledge, my review of relevant materials in the Debtors' files or my opinion」이라고 밝힌다.

31. Ray Declaration, ¶¶9~10. 원문: "I have identified four groups of businesses, which I refer to as 'Silos.' These Silos include: (a) ... (the 'WRS Silo'), which includes the businesses known as 'FTX US,' ...; (b) a group composed of Debtor Alameda Research LLC and its Debtor subsidiaries (the 'Alameda Silo'); (c) ... (the 'Ventures Silo'); and (d) a group composed of Debtor FTX Trading Ltd. and its Debtor and non-Debtor subsidiaries (the 'Dotcom Silo'), including the exchanges doing business as 'FTX.com' ..."; "Each of the Silos was controlled by Mr. Bankman-Fried." 각 사일로의 대차대조표 수치(¶¶18~19 등)는 Ray 자신이 「I do not have confidence in it」이라고 적었으므로 이 장은 옮기지 않는다.

32. Ray Declaration, ¶62. 원문: "The Debtors did not have the type of disbursement controls that I believe are appropriate for a business enterprise. For example, employees of the FTX Group submitted payment requests through an on-line 'chat' platform where a disparate group of supervisors approved disbursements by responding with personalized emojis."

33. Ray Declaration, ¶65. 원문: "The FTX Group did not keep appropriate books and records, or security controls, with respect to its digital assets. Mr. Bankman-Fried and Mr. Wang controlled access to digital assets of the main businesses in the FTX Group (with the exception of LedgerX, regulated by the CFTC, and certain other regulated and/or licensed subsidiaries). Unacceptable management practices included the use of an unsecured group email account as the root user to access confidential private keys and critically sensitive data for the FTX Group companies around the world, the absence of daily reconciliation of positions on the

blockchain, the use of software to conceal the misuse of customer funds, the secret exemption of Alameda from certain aspects of FTX.com's auto-liquidation protocol, and the absence of independent governance as between Alameda (owned 90% by Mr. Bankman-Fried and 10% by Mr. Wang) and the Dotcom Silo (in which third parties had invested)."

34. Ray Declaration, ¶66. 원문: "These balances exclude cryptocurrency not currently under the Debtors' control as a result of (a) at least $372 million of unauthorized transfers initiated on the Petition Date, ..., (b) the dilutive 'minting' of approximately $300 million in FTT tokens by an unauthorized source after the Petition Date and (c) the failure of the co-founders and potentially others to identify additional wallets believed to contain Debtor assets." ¶71: "One of the most pervasive failures of the FTX.com business in particular is the absence of lasting records of decision-making."

35. *United States v. Samuel Bankman-Fried*, No. 1:(S6)22-CR-673-001 (LAK) (S.D.N.Y.), Judgment in a Criminal Case, ECF No. 424 (Date of Imposition 3/28/2024), 1~3쪽(스캔본, OCR). 원문: "was found guilty on count(s) (S6)1 through (S6)7 after a plea of not guilty."; "18 USC §§ 1343 and 2 Wire Fraud ... (S6) 1&3 / 18 U.S.C. § 1349 Wire Fraud Conspiracy ... (S6) 2&4 / 18 U.S.C. § 371 Securities Fraud Conspiracy ... (S6) 5 / 18 U.S.C. § 371 Commodities Fraud Conspiracy ... (S6)6 / 18 USC § 1956(h) Money Laundering Conspiracy ... (S6)7"; "Hon. Lewis A. Kaplan, U.S.D.J." (RECAP 사본). 형기 합계는 판결문 3쪽의 동시·순차 집행 구조 대신 항소심 의견서의 요약(25년)으로 적었다. 배심 평결일 2023-11-02는 RECAP 도켓 항목의 확인이고 이 장은 판결문·항소심 의견서에 적힌 것만 본문에 썼다.

36. *United States v. Bankman-Fried*, No. 24-961-cr (2d Cir. June 12, 2026) (Parker, Lee, Kahn, JJ.; opinion by Parker, J.), 2·6~7·9쪽. 원문: "We AFFIRM the judgment of the district court."(2쪽); "Following a four-week trial, he was convicted on all counts. The district court sentenced him to 25 years' imprisonment, to be followed by three years of supervised release. The district court also imposed a forfeiture of approximately $11 billion."(6쪽); 소추 이론과 「robust」 평가는 본문 인용과 같다(6~7·9쪽). 의견서 8쪽은 피고인의 「투자는 건전했고 결국 손실이 없었을 것」이라는 항변이 *Kousisis v. United States*, 605 U.S. 114 (2025)의 「fraudulent-inducement theory」에 의해 「essentially been rejected」되었다고 적는다. (DktEntry 73.1, 42쪽). 항소심 의견서는 「draw the facts from the evidence presented at trial, viewed in the light most favorable to the government」(6쪽)라는 기준으로 사실을 적으므로, 이 장이 옮긴 사실 서술은 그 기준 아래의 것이다.

5부 법제도

27장 기술과 법의 다리 (1) — 깊이 다루지 않은 이유, 학설 구도, 법역별 서베이

핵심 TAKEAWAY

5부는 답이 아니라 **질문의 지도입니다**. 1~4부가 던진 「법률가의 질문」 스물세 개가 각각 어느 법적 쟁점에 닿는지를 적고, 판결·조문을 원문으로 찾아갈 1차 자료의 위치를 적습니다. 이 장은 세 절입니다. 27.1은 이 책이 법을 깊이 다루지 않은 다섯 가지 이유를 밝힙니다. 27.2는 「가상자산은 무엇인가」라는 사법상 성질의 학설을 **대립 구도로** 늘어놓고, 각 견해가 1~3부의 어떤 **기술 사실**(개인키의 배타적 통제, 대체가능성, 복제 불가성, 최종성의 확률성, 발행자의 부재, 기록의 공유)에 기대고 어떤 사실에 되받아쳐지는지를 대응표로 적습니다 — 이것이 이 책만 줄 수 있는 몫이고, 어느 견해로도 단정하지 않습니다. 27.3은 한국·EU·미국·일본·영국 다섯 법역을 같은 틀(정의어·주무 기관·발행과 유통·스테이블코인·사법상 성질)의 표로 비교하고 축마다 기준일을 붙입니다. 조문은 법령 원문으로 확인한 것만, 판결은 법원·선고일·사건번호와 함께 본문을 확인한 것만 적었습니다. 시행 전 조문(한국 소득세법의 가상자산 과세, 일본 金商法 정의 이관, 영국 2026년 규정의 본격 시행, 미국 GENIUS법)은 시행 전이라고 적었습니다.

학습목표

1. 이 책이 법을 서베이 수준으로만 다루는 이유 다섯 가지를 말하고, 1~4부의 기술·경제 사실이 법적 판단의 **어느 지점에** 들어가는지를 가리킬 수 있다.
2. 가상자산의 사법상 성질에 관한 견해들을 대립 구도로 놓고, 견해마다 기대는 기술 사실과 그 사실이 되받아치는 자리를 짝지을 수 있다. 어느 견해가 옳은지 말하지 않는다.
3. 다섯 법역의 정의어·주무 기관·발행과 유통 규제·스테이블코인 규율·사법상 성질을 같은 표에서 읽고, 축마다 기준일이 왜 필요한지 안다.
4. 「시행 중」과 「공포되었으나 시행 전」과 「계류 중」을 구별해 쓴다.

[입문]만 읽은 독자가 가져가는 것. 이 부는 답이 아니라 **질문의 지도입니다**. 같은 기술 사실이 법역마다, 견해마다 다른 결론의 근거로 쓰인다는 것, 그래서 기술을 아는 것이 법적 판단의 자리를 찾는 첫걸음이라는 것입니다.

27.1 이 책이 법을 깊이 다루지 않은 이유

이 책은 법학자가 읽을 것을 전제로 쓰였지만 법을 깊이 다루지 않습니다. 이유는 다섯입니다.

1. **이 분야의 법은 빠르게 바뀝니다.** 깊이 쓴 법리 해설은 이 책의 다른 부보다 먼저 낡습니다. 해시 함수의 성질(4장)이나 뱅크런의 구조(24장)는 10년 뒤에도 그대로이지만, 아래 27.3의 표는 이 책이 인쇄되는

사이에도 바뀝니다. 한국의 가상자산 과세 조문은 공포된 뒤 세 번 시행이 미뤄졌고, 일본은 정의 조문을 한 법률에서 다른 법률로 옮기는 개정을 공포해 놓고 시행을 기다리고 있으며, 영국은 새 규제 체계의 본격 시행일을 2027년으로 정해 두었습니다. 그래서 법제도 서술에는 전부 기준일을 붙이고, 낡기 쉬운 값은 부록 A 동향 노트에 두었습니다.

2. **법역마다 체계가 다릅니다.** 깊이 들어가려면 법역별로 책 한 권이 필요합니다. 같은 물건을 한국은 「가상자산」, EU는 「crypto-asset」, 미국은 맥락에 따라 「security」·「commodity」·「property」·「digital asset」, 일본은 「暗号資産」, 영국은 「cryptoasset」이라 부르고, 그 이름이 어느 기관의 관할과 어느 법의 적용을 부르는지가 다릅니다(30장). 다섯 법역을 한 표에 놓는 것은 비교의 출발점일 뿐 어느 법역의 해설도 아닙니다.
3. **독자가 법률가입니다.** 이 책이 줄 수 있는 것은 법리 해설이 아니라 **기술·경제 사실이 법적 판단의 어느 지점에 들어가는지 보여 주는 다리입니다.** 「개인키를 잃은 사람은 무엇을 잃었나」(5장)에 답하려면 서명 검증만 보는 원장 위에서 「정당한 권리자」가 어디에 자리하는지를 알아야 하고, 「지금은 언제 완료되었나」(9장)에 답하려면 최종성이 확률이라는 사실을 알아야 합니다. 그 사실을 1~3부가 주었으니, 그것을 법리에 꽂는 일은 독자의 몫입니다. 27.2의 대응표가 그 꽂는 자리를 보여 주는 장치입니다.
4. **엮은이는 현직 법관입니다.** 국내에서 계속 중인 사건, 국회에 계류 중인 법안, 대법원이 정리하지 않은 쟁점에 대해서는 사실만 적고 평가하지 않습니다. 한국법을 외국법보다 얕게 다룬 것은 이 때문입니다. 한국의 판결은 본문을 확인한 세 건만 적고, 그 판결이 옳은지, 어느 학설이 타당한지는 쓰지 않습니다.
5. **더 깊이 읽을 곳을 가리킵니다.** 28장에 법역별 1차 자료와 개설서를 모았습니다. 서지는 원자료 확인에서 확정된 것만 적었고, 확정되지 않은 것은 비워 두었습니다.

외국법 서베이의 범위는 다섯 법역입니다 — 한국, EU, 미국, 일본, 영국. 각각 성문의 포괄 규제(EU MiCA), 기존 법의 적용과 특별법의 병존(미국), 결제·금융상품 양쪽 법의 분업(일본), 보통법의 재산 범주 확장과 새 규제 체계의 도입(영국), 그리고 이용자 보호와 불공정거래 규제를 먼저 입법한 한국이라는 서로 다른 길을 걸었기 때문입니다. 비교 축은 여섯이고(정의어·주무 기관·발행과 유통·스테이블코인·사법상 성질·기준일), 각 칸에는 조문 번호와 법원·선고일·사건번호를 적되 해설은 하지 않습니다.

27.2 법적 성질의 학설 구도 — 견해와 기술 사실의 대응

「비트코인은 무엇인가」라는 사법상의 질문에 각 법역의 학설과 판결은 서로 다른 답을 내놓았습니다. 물건이다, 물건은 아니지만 물권에 준하는 지배권의 객체다, 채권은 아니다, 재산권이 아니라 사실상태다, 제3의 재산 범주다, 통제할 수 있는 전자기록이다, 화폐다 또는 아니다, 증권이다 또는 아니다, 상품이다, 재산상 이익이다. 이 절은 그 견해들을 **옳고 그름이 아니라 근거의 구조로** 늘어놓습니다. 각 견해는 1~3부가 다룬 기술 사실 가운데 어떤 것에 기대고, 어떤 것에 되받아쳐집니까. 그것을 대응표로 적는 것이 이 절의 일이고, 어느 견해로도 단정하지 않습니다.

출발점으로 삼은 문헌은 사법정책연구원의 2022년 연구보고서(이혜정, 『가상자산에 대한 민사집행 연구』)입니다. 이 보고서는 민사집행의 목적물을 정하기 위해 한국·미국·영연방·독일·일본의 성질 논의(EU는 규제 개관)를 한자리에 모았고, 「가상자산을 물건으로 보고 그에 대한 권리를 인정할지에 대해 견해가 통일되어 있지 않아, 가상자산을 취득한 자가 '가상자산을 소유하고 있다'고 단정 짓기 힘들고」 그래서 「'소유' 대신 '보유'」라는 낱말을 쓴다고 밝힙니다.[1] 이 책도 같은 이유로 「보유」를 씁니다. 외국의 1차 자료는 영국 법률위원회의 2023년 최종보고서, 2025년의 영국 재산법, UNIDROIT의 2023년 원칙, 미국 통일상법전 제12편(2022년 개정), 유럽연합 사법재판소의 2015년 판결, 미국의 1946년·2018년

·2023년 판결과 2017년 SEC 보고서, 일본 도쿄지방재판소의 2019년 판결, 한국 대법원의 세 판결입니다.

대응표

열은 넷입니다. 견해(어느 법역의 누가), 핵심 주장, 그 주장이 기대는 기술 사실(이 책의 어느 장), 그리고 같은 기술 사실 가운데 그 주장을 되받아치는 자리입니다. 넷째 열이 비어 있는 행은 없습니다 — 기술 사실은 어느 견해에도 일방적으로 유리하지 않습니다.

견해	핵심 주장	기대는 기술 사실 (장)	되받아치는 기술 사실 (장)
① **물건(동산)설** — 한국 긍정설[2]	개인키를 가진 자만 송금을 지시할 수 있고(배타적 지배가능성), 승인된 송금은 과반 해킹 없이 이중지불될 수 없으므로(관리가능성) 민법 제98조의 「관리할 수 있는 자연력」에 해당한다	서명이 지출 권한을 통제한다(5장). 소비된 UTXO는 다시 쓸 수 없다(11장). 변조 비용(7·8장)	개인키는 복제 가능한 정보다(5장). 이전은 블록에 들어가야 완결되고 그 최종성은 확률이다(9장). 비트코인에서 에너지를 추출할 수 없다 — 「자연력」의 문언
② **물건 부정, 물권 유사의 지배권설** — 한국[3]	보유자는 재산적 가치에 대해 독점적·배타적 지배를 하고 누구에게나 주장할 수 있으므로 법률의 근거가 없는 「준물권적 구조를 가진 권리」다	서명 없이는 아무도 옮길 수 없다(5장)	기록은 모든 참가자가 공유하므로 개인키가 있어도 「독점적 지배」라 하기 어렵다(6·7·12장 — 원장은 누구나 내려받는다). 일본 견해 ②~④에 대한 물권법정주의(민법 제175조) 지적도 같은 자리
③ **채권설 부정** — 한국[4]	송금 지시를 블록에 넣는 급부는 네트워크의 프로그램이 이행하고 참여자 개개인은 자원을 조금씩 제공할 뿐이라 「채무자」가 없다. 네트워크는 책임재산도 중앙기관도 없어 법인·단체로 볼 수 없다	채굴자·검증자는 누구의 대리인도 아니다(8장 질문 상자). 발행자가 없다(1·19장)	거래소에 맡긴 코인은 거래소에 대한 **채권**(가상자산이전청구권)으로 볼 여지가 있다 — 장부와 체인이 다른 층이라는 사실(26장 Mt.Gox). 채권의 **객체는** 될 수 있다
④ **사실상태설** — 일본 견해 ⑦·⑧[5]	발행자가 없고 소유권의 객체는 유체물에 한정되므로 보유는 물권·채권·사원권·지적재산권 어느 재산권도 수반하지 않는 사실상태다. (이 견해를 지지하는 다른 의견: 그 결과 물권적 청구가 배제되어 「소유와 점유의 일치」라는 금전 유사의 취급이 확보된다)	원장은 서명의 유효성만 본다 — 「정당한 권리자」를 모른다(5장). 발행자 부재(1장)	사실상태라면 착오로 이전된 코인·도난 코인의 반환은 무엇에 근거하는가(11장·28.1(a)). 참가자 전원의 「규범적 합의」가 프로토콜과 구별되어 존재하는지 불분명하다(23장 조정 게임)
⑤ **사원권·조합권설** — 일본 견해 ⑤, 한국 조합권 견해[6]	보유자는 네트워크라는 단체의 구성원이고, 「프로그램 시스템」을 계약 내용으로·「블록체인의 내부적 거래」를 공동사업으로·「취득」을 출자로 읽으면 민법 제703조의 조합이다	규칙은 참여자의 조정으로 유지되고 바뀐다(23장). 하드포크는 「투표」다(22·26장)	단체의 성립·가입·탈퇴를 위한 법령이나 의사표시가 없다. 노드 운영자·채굴자·보유자의 집합이 일치하지 않는다(8·10장). 조합권설 자신도 그 조합권이 「이익분배에 대한 기대권이며, 실체적 권리가 아니라 조합원의 지위권」이라 한다
⑥ **지적재산설** — 일본 견해 ⑥[5]	저작권으로 보호되는 저작물일 가능성	코드는 저작물이다(13장의 계약 코드)	채굴로 「사상이 창작적으로 표현」된다고 평가하기 어렵다. 해시값은 함수의 출력이다(4장)
⑦ **「무형의 이익」** — 독일 학설 일치[7]	민법상 물건은 아니지만 그 상위개념인 목적물로서 무형의 이익이다. 채권이 아니라는 점(83쪽), 유체성이 없다는 점(81쪽)에 학설이 일치한다	발행자·채무자 부재(1장). 데이터는 유체물이 아니다	「무형의 이익」은 집행·귀속의 규칙을 바로 주지 않는다 — 어느 절차로 압류하는가(28.1(a))
⑧ **제3범주 재산설** — 영국 법률위원회, Property (Digital Assets etc) Act 2025[8][9]	어떤 디지털 자산은 점유의 대상(thing in possession)도 소권의 대상(thing in action)도 아니지만 인적 재산권의 객체가 될 수 있다. 입법은 「그 둘 어느 것도 아니라는 사실이 재산권의 객체임을 막지 않는다」고 확인만 하고, 경계는 보통법이 그린다	**경합성(rivalrousness)** — 한 사람의 사용이 다른 사람의 사용을 배제한다(11장 UTXO 소비). 체계 밖에서 독립적으로 존재한다(1·7장)	거래소 장부 위의 「잔고」는 경합적이지 않을 수 있다(26장 Mt.Gox의 가공 잔고). 보고서 자신이 「composed of data」를 기준에서 뺐다

견해	핵심 주장	기대는 기술 사실 (장)	되받아치는 기술 사실 (장)
⑨ **통제(control) 기반** — UNIDROIT 원칙, 미국 UCC 제12편[10][11]	「digital asset」은 「통제의 대상이 될 수 있는 전자기록」이고, 「controllable electronic record」는 「전자적 매체에 저장되어 제12-105조에 따라 통제될 수 있는 기록」이다. **통제를** 정의와 이전·담보 규칙의 연결점으로 삼되, 누가 재산권을 갖는가 자체는 「other law」에 맡긴다(UNIDROIT Principle 3(3)(a))	개인키 = 지출 권한(5장). 통제의 배타성과 이전 가능성(11장)	통제가 곧 권리는 아니다 — 훔친 열쇠도 통제한다(5장 질문 상자). 여러 서명(다중 서명)·보관업자 경유 통제는 어떻게 세는가(26장 FTX의 개인키 관리)
⑩ **화폐성** — 부정설·긍정설(한국), EU 부가가치세, 미국 세법, 한국 대법원[12][13][14][15]	부정: 가치척도 기능을 못 하고 발행총량이 고정되어 있다. 긍정: 장래 수용 가능성. EU: 「bitcoin」은 「no purpose other than to be a means of payment」이고 환전은 부가가치세 면제. 미국: 세법상 「property」이고 「not … currency」. 한국 대법원: 「형법을 적용하면서 법정화폐와 동일하게 보호해야 하는 것은 아니다」	교환매개·가치척도·가치저장 기능의 점검(19장). 세금 수요의 바닥이 법정화폐를 가른다(19장 모형)	19장이 보인 것은 「내재가치 0의 양의 가격」이 균형일 수 있다는 것이지 화폐라는 것이 아니다. 「법이 화폐라고 부를 때 달라지는 것」은 법역마다 다르다(19장 질문 상자)
⑪ **증권성** — 한국 유가증권 부정설, 미국 Howey·SEC·Ripple, GENIUS 법[16][17][18][19][20]	한국: 발행인이 없고 체인 기록은 배서가 아니므로 유가증권으로 보기 어렵다는 견해가 대체적. 미국: 「투자계약」은 「a contract, transaction or scheme whereby a person invests his money in a common enterprise and is led to expect profits solely from the efforts of the promoter or a third party」이고, DAO 토큰은 증권이며(2017), 같은 토큰이라도 기관 판매는 투자계약이고 거래소의 익명 매매는 아니다(2023). 결제 스테이블코인은 법으로 증권에서 뺐다(2025)	토큰 표준은 함수 이름의 약속일 뿐 권리를 담지 않는다(14장). 큐레이터와 발행자의 「노력」(22·26장). 2차 시장에서 매수인은 매도인이 누군지 모른다(15장 AMM·17장)	「형식이 아니라 경제적 실질」(Howey)을 보면 코드가 아니라 **바깥의 약속이** 증권성을 정한다(14장) — 그래서 같은 토큰이 거래 경로에 따라 다르게 판단된다
⑫ **상품성** — 미국 CFTC·McDonnell[21]	가상통화는 「'goods' exchanged in a market for a uniform quality and value」이고 상품거래법의 「commodity」에 들어가며, CFTC의 관할은 다른 기관과 **경합**한다	대체가능성(11장 UTXO·14장 ERC-20). 선물·파생상품의 기초자산	「security」와 「commodity」가 같은 물건에 겹친다 — 명명이 관할을 가른다(30장)
⑬ **재산상 이익·무형재산** — 한국 대법원[22][23][15]	비트코인은 「경제적인 가치를 디지털로 표상하여 전자적으로 이전, 저장과 거래가 가능하도록 한 가상자산의 일종으로 사기죄의 객체인 재산상 이익」(2021. 11. 11.)이고, 「재산적 가치가 있는 무형의 재산」으로 몰수할 수 있으며(2018. 5. 30.), 원인 없이 이체받은 자는 부당이득반환의무를 질 수 있으나 「타인의 사무를 처리하는 자」는 아니다(2021. 12. 16.)	전자적 이전·저장·거래(11·12장). 「보관되었던 전자지갑의 주소만을 확인할 수 있을 뿐 그 주소를 사용하는 사람의 인적사항을 알 수 없고, 거래 내역이 분산 기록되어 있어 다른 계좌로 보낼 때 당사자 이외의 다른 사람이 참여해야 하는」 성질(판결이 적은 기술 사실)	형사법의 「재산상 이익」 판단이 민사법의 귀속을 정하지는 않는다 — 판결 자신이 「부당이득반환청구를 할 수 있는 주체가 피해자인지 아니면 거래소인지 명확하지 않다」고 적음(26장 장부와 체인)

견해	핵심 주장	기대는 기술 사실 (장)	되받아치는 기술 사실 (장)
⑭ **예금·채무·지분** — 스테이블코인(Gorton & Zhang)[24]	스테이블코인이 요구불예금인지를 판단하려면 보유자와 발행자 사이의 계약이 「equity contract or a debt contract」인지를 먼저 정해야 한다 — 1970년대 MMF 논쟁과 같은 모양	상환 약속이 붙은 토큰은 내부화폐다(24장). 대차대조표와 상환 절차(24장)	2차 시장이 있어 창구를 닫아도 런이 멈추지 않는다(24장) — 예금과 다른 자리. 발행자 「공언」의 법적 효력(24장 질문 상자)

표를 읽는 법

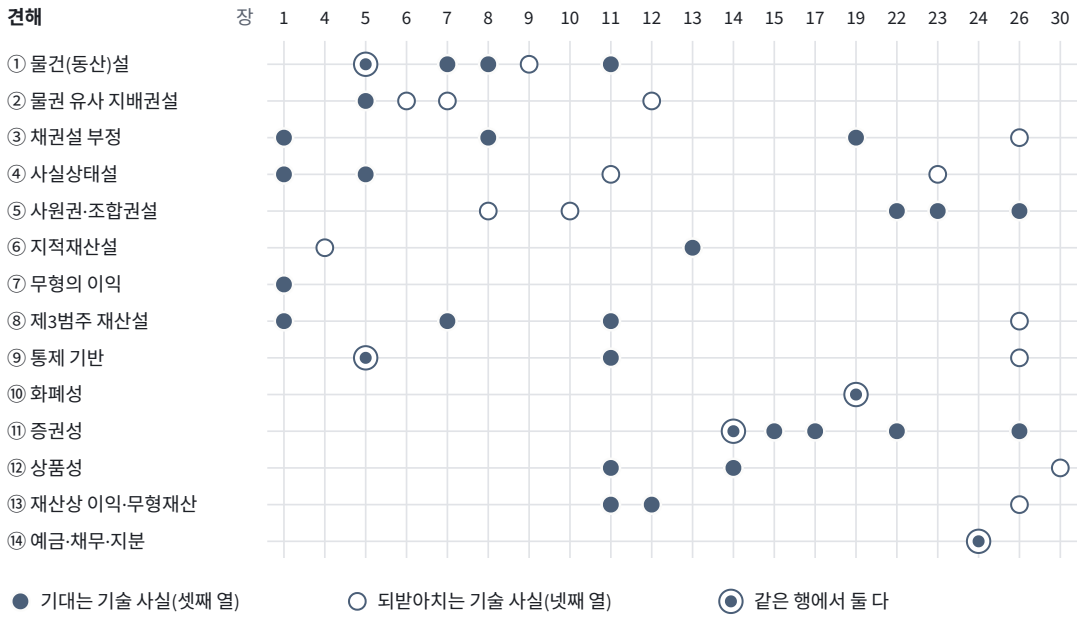


그림 27-1. 27.2 대응표의 열네 견해가 기대는 장(채운 점, 셋째 열)과 되받아쳐지는 장(빈 점, 넷째 열). 괄호 속 절 참조(28.1(a) 등)는 장이 아니어서 점을 찍지 않았다.

표의 셋째 열과 넷째 열은 같은 장을 가리키는 때가 많습니다. 5장의 「서명이 지출 권한을 통제한다」는 사실은 물건설(①)과 지배권설(②)과 통제 기반(⑨)의 근거이면서, 「훔친 열쇠도 통제한다」·「개인키는 복제 가능한 정보다」라는 반론의 근거이기도 합니다. 9장의 「최종성은 확률이다」는 사실은 물건의 관리가능성을 흔드는 자리이면서, 「언제 완료되었는가」(28.1(c))를 묻는 자리이기도 합니다. 그래서 어느 견해가 「기술에 맞다」고 말할 수 없습니다. 기술 사실은 견해를 고르는 심판이 아니라, 각 견해가 **어느 사실을 전제하고 어느 사실을 설명해야 하는지를** 보이는 거울입니다.

두 가지를 덧붙입니다. 첫째, 법원은 이 질문을 피해 갈 수 있을 때 피해 갔습니다. 도쿄지방재판소의 2019년 판결은 「ビットコインは「有体物」（民法８５条）ではなく，
その**帰属**や利用者間での取引がどのような私法上の性質を有するかも一義的に明らかとはいえないが，本件の判断に影響のない限り，ビットコインの保有・所有や利用者間の売買などと表現し，日常的な意味合いで記載する」고 적고 사건을 판단했습니다.[25] 한국 대법원의 2021년 판결도 「재산상 이익」이라는 형사법의 범주로 답하면서 민사상 귀속은 이 사건 비트코인의 이체 경위가 불분명하다는 이유로 「부당이득반환청구를 할 수 있는 주체가 피해자인지 아니면 거래소인지 명확하지 않다」고 적는 데 그쳤습니다 — 사안에 매인 판단이지 귀속 일반에 대한 답이 아닙니다. 둘째, 입법은 **정의로** 답했지 **성질로** 답하지 않았습니다. 한국 가상자산이용자보호법 제2조 제1호의 「경제적 가치를 지닌 것으로서 전자적으로 거래 또는 이전될 수 있는 전자적 증표(그에 관한 일체의 권리를 포함한다)」, EU MiCA 제3조 제1항 제5호의 「a digital representation of a value or of a right」는 규제의 대상을 정하는 문장이고,

보유자가 물권을 갖는지 채권을 갖는지를 정하는 문장이 아닙니다.[26] 영국의 2025년 법이 두 조문으로 「재산권의 객체가 될 수 있다」만 확인하고 경계를 보통법에 남긴 것이 그 점을 가장 분명히 보여 줍니다. 30장이 이 정의 조문들을 나란히 놓습니다.

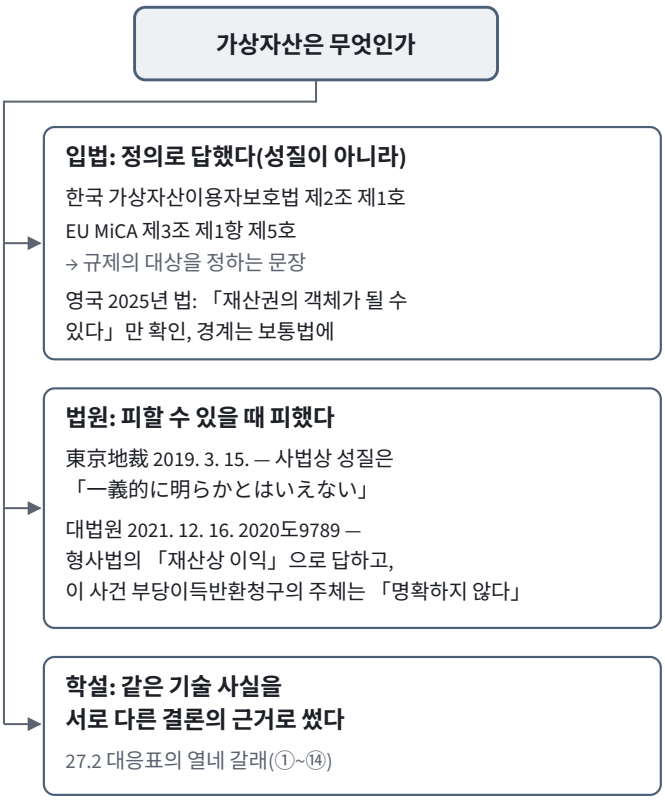


그림 27-2. 「가상자산은 무엇인가」에 입법은 정의로 답했고, 법원은 피할 수 있을 때 피했으며, 학설은 같은 기술 사실을 서로 다른 결론의 근거로 썼다.

이 절로 연결된 질문 상자

1~3부와 6부의 상자 가운데 이 절을 가리키는 것은 여섯입니다. 각각 표의 어느 행이 받는지 적어 둡니다.

상자	질문의 요지	이 절에서 받는 자리
5장	개인키를 잃은 사람은 무엇을 잃었나 — 점유인가, 소유인가, 접근 가능성인가. 서명의 유효성만 보는 원장에서 「정당한 권리자」는 어디에 있나	①·②·⑨가 「개인키 = 배타적 통제」에 기대고, ④가 「원장은 권리자를 모른다」에 기댄다. 영국 보고서의 「경합성」(⑧)은 재산의 객체가 되는 표지로, UNIDROIT의 「통제」(⑨)는 정의와 이전 규칙의 연결점으로 접근 가능성을 끌어들인 답이고(귀속 자체는 둘 다 정하지 않는다), 한국 대법원(⑬)은 「주소만 확인할 수 있을 뿐 그 주소를 사용하는 사람의 인적사항을 알 수 없」는 성질을 적었다
19장	법정통화와 「화폐」는 같은 말인가 — 법이 「화폐」라 부를 때 달라지는 것은 무엇인가	⑩. EU 부가가치세는 「지급수단」으로, 미국 세법은 「재산」으로, 한국 대법원은 「형법을 적용하면서 법정화폐와 동일하게 보호해야 하는 것은 아니」라고 답했다(형법 적용에 한정한 판단). 27.3 과세 행
20장	다수가 발행 규칙을 바꾸면 보유자의 권리는 침해되나 — 보유자가 가진 것은 「2,100만 상한」에 대한 권리인가, UTXO를 풀 능력뿐인가	④(사실상태)·⑤(조합권)·⑨(통제)가 각각 다른 답을 함의한다. 사실상태설이면 침해할 권리가 없고, 조합권설이면 다수결의 한계가 문제되며, 통제 기반이면 규칙 변경은 통제의 대상 자체를 바꾼다. 29장의 취약 코인 지출 제한이 같은 구조를 더 날카롭게 묻는다 — 28.1(g)
21장	소각된 수수료는 누구에게 귀속되나 — 포기·무주물·소비·새 범주	어느 견해도 「누구의 것도 아닌 가치의 소멸」을 바로 설명하지 않는다. 물건설이면 소유권의 포기 또는 멸실, 사실상태설이면 상태의 소멸, 통제 기반이면 통제 대상의 소멸이다. 이 책은 어느 쪽도 고르지 않는다
23장	하드포크 뒤 두 체인 중 「진짜」 원장은 누가 정하나 — 포크 전 보유자는 하나를 가졌나 둘을 가졌나	⑧의 경합성 기준은 포크 뒤 두 체인의 코인이 서로 경합하지 않으므로 둘을 각각 재산으로 볼 여지를 주고, ⑨의 통제 기준도 같은 개인키가 두 체인에서 통제를 준다. 「계약서의 비트코인 1개」가 어느 체인의 것인지는 계약 해석의 문제로 남는다 — 28.1(g)
30장	정의 조문이 기술을 따라가지 못할 때 — 유추인가, 입법 공백인가	27.3 정의어 행과 30장의 조문 대역. 입법은 성질이 아니라 정의로 답했으므로, 정의가 비는 자리에서 성질 논의(이 표)가 다시 호출된다

27.3 법역별 서베이

아래 표는 다섯 법역을 여섯 축으로 비교합니다. 칸에 든 것은 조문 번호와 법원·선고일·사건번호이고, 해설은 없습니다. **기준일은 2026-10-25이고**, 축마다 「시행 중」·「공포, 시행 전」·「미확인」 을 가립니다. 표 아래에 법역별 주석을 붙였습니다.

축	한국	EU	미국	일본	영국
정의어 (조문)	「가상자산」 — 가상자산이용자 보호법 §2(1) (법률 제19563호 제정, 현행 제21857호; 정의 불변); 특정금융정보법 §2(3)은 이를 준용	「crypto-asset」 — MiCA Art. 3(1)(5); 「asset-referenced token」(6), 「e-money token」(7), 「utility token」(9); 고유·대체불가 자산 제외 Art. 2(3), 금융상품 제외 Art. 2(4)(a)	단일 정의 없음. 「security」 (Securities Act §2(a)(1) 「investment contract」 — *Howey*), 「commodity」(상품거래법 — *McDonnell*이 「Title 7 U.S.C. § 1(a)(9)」로 인용), 「property」 (IRS Notice 2014-21), 「digital asset」·「payment stablecoin」(GENIUS Act §2(6)·(22)), 「controllable electronic record」 (UCC §12-102)	「暗号資産」 — 資金決済法 §2(14)(令和元年法律第28号로 「仮想通貨」에서 개명, 시행 2020-05-01; 2023-06-01부터 第14項); 「電子記録移転権利」 — 金商法 §2(3); 「電子決済手段」 — 資金決済法 §2(5)	「cryptoasset」 — FSMA 2000 s. 417(1)(FSMA 2023 s. 69로 삽입, 시행 2023-08-29); 「qualifying cryptoasset」·「qualifying stablecoin」 — RAO art. 88F·88G(SI 2026/102로 삽입)
주무 기관	금융위원회 (가상자산법 §14 조사·조치, §13 감독·검사); 금융정보분석원장 (특정금융정보법 §7 신고)	회원국 관할 당국(Art. 93); 유의미한 ART·EMT 발행자는 EBA 감독 (Art. 117)	SEC·CFTC의 경합 관할(*McDonnell*); 결제 스테이블코인은 통화감독청 등 「primary Federal payment stablecoin regulator」(GENIUS §2(25); 비은행 연방 허가 발행자(§2(11))는 §2(25)(D)에 따라 통화감독청); FinCEN(MSB 등록 — SEC DAO 보고서 각주 20)	内閣総理大臣 등록(資金決済法 §63の2) — 금융청	FCA(SI 2026/102 reg. 1(3))
발행·유통 규제	발행·공시 규정 없음(법 22개조는 이용자 자산 보호·불공정거래 금지·감독). 사업자는 신고(특정금융정보법 §7), 예치금 분리(가상자산법 §6), 가상자산 분리보관·실질 보유(§7)	백서 작성·공시 의무(Title II, Art. 4), 암호자산서비스업자 인가 (Art. 59), 시장남용 금지 (Title VI, Art. 86~92)	증권이면 §5 등록 또는 면제(SEC DAO 보고서); 상품이면 CEA 사기 금지(*McDonnell*)	暗号資産交換業 등록제(§63の2)	2026년 규정이 「qualifying cryptoasset」의 공모·거래소 상장을 지정 활동으로, 발행·보관 등을 규제 활동으로 — **본격 시행 2027-10-25**

축	한국	EU	미국	일본	영국
스테이블코인	별도 범주 없음(가상자산법 §2(1) 안에서; 「전자화폐」·「선불전자지급수단」·한국은행 발행 전자적 화폐는 제외 — 다·사목)	ART(Title III, Art. 16 인가)·EMT(Title IV, Art. 48 — 신용기관·전자화폐기관만) — **적용 2024-06-30**	GENIUS Act(2025-07-18 제정): 허가 발행자, 증권·상품 아님(§17) — **시행일은 §20에 따라 늦어도 2027-01-18, 규칙 공표에 따라 더 이를 수 있음(미확인)**	電子決済手段(資金決済法 §2(5))	「qualifying stablecoin」(RAO art. 88G) — 2027-10-25
사법상 성질	대법원: 재산상 이익(2021. 11. 11. 2021도9855), 무형재산·몰수 가능(2018. 5. 30. 2018도3619), 원인 없는 이체 수령자는 배임의 주체 아님(2021. 12. 16. 2020도9789). 민법 §98 물건 정의 아래 학설 분립(27.2). 강제징수는 국세징수법 §55③·§66②(2)·지방세징수법 §61③	MiCA는 규제 규정 — 사법상 성질은 이 책이 읽은 조문 범위에서 정하지 않음(미확인). 부가가치세: 교환은 면제(CJEU 2015. 10. 22. C-264/14)	세법상 property(2014-21); UCC 제12편(2022 모델법 — 주 채택 미확인); 증권·상품 판단은 거래 단위	東京地裁 2019. 3. 15.: 「有体物」 아님, 사법상 성질은 「一義的に明らかとはいえない」 유보	Property (Digital Assets etc) Act 2025 s. 1(2025-12-02 시행); Law Com No 412; 개발자의 신인의무 가능성은 「심리할 중대한 쟁점」(*Tulip Trading v van der Laan* [2023] EWCA Civ 83, 2023-02-03)
과세	소득세법 §21①(27) 가상자산소득 — **공포 2020-12-29, 시행 2027-01-01(세 번 유예), 2026-10-25 미시행**	부가가치세 면제(*Hedqvist*)	재산 거래 과세 원칙(2014-21)	(이 책 범위 밖)	(이 책 범위 밖)
기준일	2026-10-25	2026-10-25	2026-10-25	2026-10-25	2026-10-25

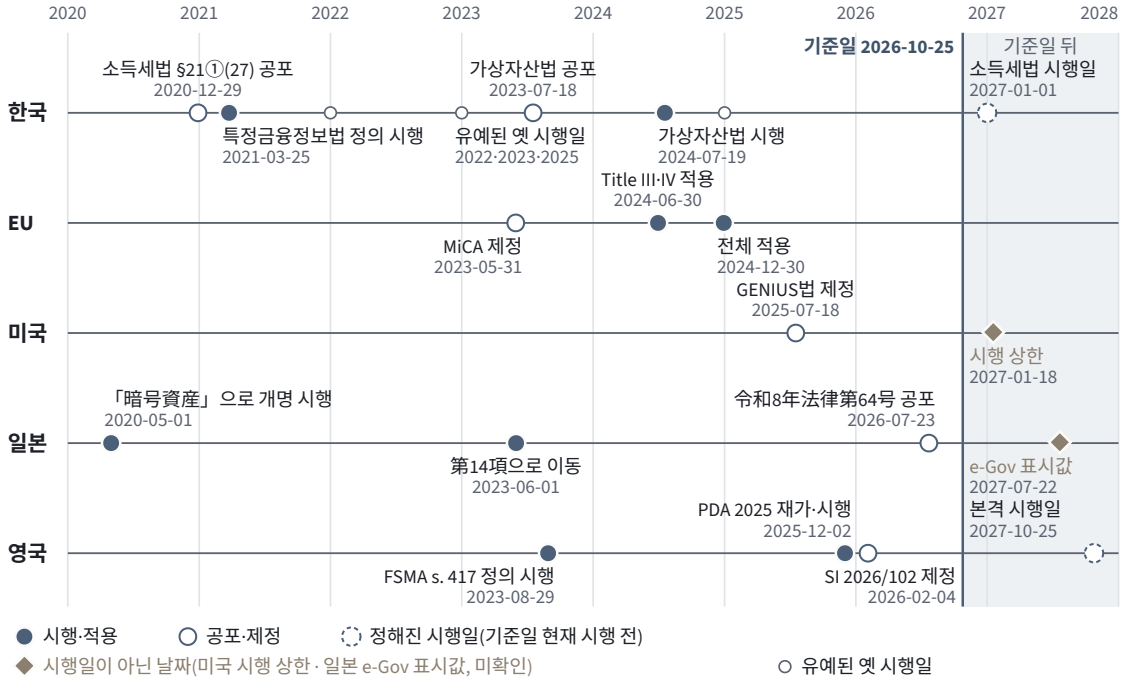


그림 27-3. 다섯 법역의 정의·규제 조문이 공포·시행된 날과 이 책의 기준일 2026-10-25(세로선). 보조색 마름모는 시행일이 아닌 날짜(미국 GENIUS법 시행 상한, 일본 e-Gov 표시값 — 정령 미확인)다.

한국

「가상자산 이용자 보호 등에 관한 법률」은 2023년 7월 18일 법률 제19563호로 제정되어 2024년 7월 19일 시행되었고, 현행 법률은 2026년 8월 4일 공포·10월 2일 시행된 제21857호(형사소송법 개정에 따른 타법개정)입니다. 제2조의 정의는 제정 이후 바뀌지 않았습니다.[27] 법은 22개 조문으로 이용자 예치금의 보호(제6조), 가상자산의 분리 보관과 실질 보유(제7조), 불공정거래행위의 금지(제10조 — 미공개중요정보 이용, 시세조종, 부정거래, 사업자의 자기발행 가상자산 거래 금지)와 손해배상(제10조 제6항), 금융위원회의 감독·조사(제13·14조), 과징금·벌칙(제17·19조)을 두고, 발행이나 공시를 규율하는 조문은 두지 않았습니다. 사업자의 진입은 「특정 금융거래정보의 보고 및 이용 등에 관한 법률」 제7조의 금융정보분석원장에 대한 신고이고, 그 법 제2조 제3호의 「가상자산」 정의는 가상자산법 제2조 제1호를 가리킵니다.[28] 과세는 소득세법 제21조 제1항 제27호가 「가상자산을 양도하거나 대여함으로써 발생하는 소득」을 기타소득으로 정했으나, 이 조문은 2020년 12월 29일 공포된 뒤 시행일이 2022년 → 2023년 → 2025년 → 2027년 1월 1일로 세 번 미뤄져 **기준일 현재 시행 전입니다.** 법제처 국가법령정보센터의 「현행」 본문에는 이 호가 시행 중인 것처럼 표시되므로 그대로 인용하면 틀립니다.[29] 강제징수에서는 국세징수법 제55조 제3항이 세무서장이 가상자산을 압류할 때 체납자 또는 보관 사업자에게 「해당 가상자산의 이전을 문서로 요구할 수 있」게 하고, 제66조 제2항 제2호가 「가상자산사업자를 통해 거래되는 가상자산」의 직접 매각을 허용하며, 지방세징수법 제61조 제3항도 비슷한 이전 요구를 둡니다(다만 지방세는 「요구하여야 한다」는 의무이고, 문서 요건은 제3자에게 요구할 때만 붙습니다).[30] 민사집행법에는 가상자산 조문이 없고, 사법정책연구원 보고서는 「그 밖의 재산권」으로 보아 압류·이전명령·매각명령의 절차를 입법안으로 제안했습니다.[31] 그 뒤의 입법 경과는 이 책이 확인하지 않았습니다(2026-10 기준, 부록 A).

EU

암호자산시장규정(Regulation (EU) 2023/1114, MiCA)은 2023년 5월 31일 제정되어 관보 L 150(2023년 6월 9일)에 실렸고, 제149조에 따라 **2024년 12월 30일부터 적용**되되 자산준거토큰(Title III)과 전자화폐토큰(Title IV)은 **2024년 6월 30일부터** 적용됩니다.[32] 정의는 제3조 제1항입니다. 「crypto-asset」은 「a digital representation of a value or of a right that is able to be transferred and stored electronically using distributed ledger technology or similar technology」(제5호)이고, 그 아래 자산준거토큰(제6호)·전자화폐토큰(제7호)·유틸리티토큰(제9호)이 있습니다. 적용일에는 제149조 제4항의 추가 예외(일부 조항은 발효일부터)가 있습니다. 「unique and not fungible」한 암호자산(제2조 제3항)과 금융상품 등(제2조 제4항)은 적용 범위 밖입니다.[33] 유통은 백서 작성·공시(제4조), 자산준거토큰 발행 인가(제16조 — 또는 제17조를 준수하는 신용기관), 전자화폐토큰은 신용기관·전자화폐기관만 발행(제48조), 암호자산서비스업자 인가(제59조)로 짜여 있고, Title VI(제86~92조)가 거래 플랫폼 안팎의 내부자거래와 시장조작을 금지합니다.[34] 감독은 회원국이 지정하는 관할당국(제93조)이 맡고, 「significant」로 분류된 자산준거토큰·전자화폐토큰의 발행자는 유럽은행감독청(EBA)의 감독 아래 있습니다(제117조). 사법상 성질은 이 책이 읽은 조문 범위에서 정하지 않습니다 — 규정 전체를 통독한 것은 아니므로 「미확인」으로 적습니다. 부가가치세에 관해서는 유럽연합 사법재판소가 2015년 10월 22일 *Hedqvist* 사건에서 비트코인과 전통 통화의 환전이 부가가치세 지침 제135조 제1항 (e)의 면제 거래라고 판시했습니다.[13]

미국

미국에는 단일한 정의가 없습니다. 같은 토큰이 거래의 모양에 따라 증권법의 「security」가 되고, 상품거래법의 「commodity」가 되고, 세법의 「property」가 됩니다. 증권성의 기준은 1946년 연방대법원의 *Howey* 판결이 정한 투자계약의 정의이고, SEC는 2017년 The DAO 조사 보고서에서 그 기준으로 DAO 토큰을 증권으로 판단했습니다(26장). 2023년 7월 13일 미국 연방뉴욕남부지방법원은 *SEC v. Ripple Labs* 사건의 약식판결에서 「XRP, as a digital token, is not in and of itself a "contract, transaction[,] or scheme"」이라 하면서, 기관투자자에게 서면 계약으로 판 것은 투자계약이고 거래소에서 익명의 매수인에게 판 것은 아니라고 가렸습니다 — 토큰이 아니라 **거래가** 판단 단위입니다.[19] 상품성은 2018년 3월 6일 미국 연방뉴욕동부지방법원이 *CFTC v. McDonnell* 사건의 가처분 결정에서 가상통화가 상품거래법(결정의 표기로 「Title 7 U.S.C. § 1(a)(9)」)의 「commodity」에 들어가고 CFTC의 관할이 다른 기관과 경합한다고 판단했습니다.[21] 세법에서는 2014년 국세청 고시(Notice 2014-21)가 「virtual currency is treated as property」라 했고, 2023년 고시(Notice 2023-34)는 「법정통화인 법역이 없다」는 배경 서술만 고치고 문답의 결론은 그대로 두었습니다.[14] 결제 스테이블코인에 관해서는 2025년 7월 18일 GENIUS법(Pub. L. 119-27)이 제정되어 「payment stablecoin」을 정의하고(§2(22)), 허가받은 발행자가 발행한 것은 증권도 상품도 아니라고 명시했으며(§17), 시행일은 제정 후 18개월 또는 연방 감독기관의 최종 규칙 공표 후 120일 가운데 이른 날입니다(§20) — 기준일 현재 규칙 제정 상황을 이 책은 확인하지 않았습니다.[20] 사법상 귀속에 관해서는 통일법위원회와 미국법률협회가 2022년 통일상법전 제12편 「Controllable Electronic Records」를 승인했습니다. 모델법이고 각 주의 채택 여부는 확인하지 않았습니다.[11]

일본

일본은 결제법과 금융상품법이 분업합니다. 「資金決済に関する法律」(平成21年法律第59号)은 「仮想通貨」라는 정의어를 令和元年法律第28号(공포 2019-06-07, 시행 2020-05-01)로 「暗号資産」으로 바꿨고, 현행 정의는 제2조 제14항에 있습니다(2023-06-01 시행 판본부터 제5항에서 제14항으로 이동).[35] 같은 법 제63조の2는 「暗号資産交換業は、内閣総理大臣の登録を受けた者でなければ、行ってはならない」고 하여 등록제를 두고, 제2조 제15항이 교환업의 범위(매매·교환, 그 매개·취차·대리, 그에 관한 금전 관리, 타인을 위한 암호자산 관리)를 정합니다.[36] 법정통화 표시 자산인 스테이블코인은 「電子決済手段」(제2조 제5항)으로 따로 정의됩니다. 투자성이 있는 토큰은 「金融商品取引法」 제2조 제3항 괄호 안의 「電子記録移転権利」로 금융상품법의 규율을 받습니다 — 암호자산 정의의 단서가 「金融商品取引法第二十九条の二第一項第八号に規定する権利を表示するもの」를 제외하는 것과 짝입니다.[37] 2026년 7월 23일 공포된 令和8年法律第64号는 암호자산의 정의를 資金決済法에서 金商法 제2조 제49항으로 옮기는 개정인데, **공포·일부 시행되었으나 정의 이관 부분은 기준일 현재 시행 전입니다.** e-Gov 법령검색은 그 판본을 2027-07-22 시행 예정으로 표시하나 실제 시행일을 정하는 정령은 확인하지 못했습니다.[38] 사법상 성질에 관해서는 2019년 3월 15일 도쿄지방재판소 판결이 「有体物」이 아니고 사법상 성질은 「一義的に明らかとはいえない」고 유보한 것(26장)과, 학설 여덟 갈래(27.2 표 ④~⑥)가 있습니다.

영국

영국은 재산법과 규제법 양쪽에서 입법했습니다. 규제법 쪽의 정의어 「cryptoasset」은 Financial Services and Markets Act 2023 제69조가 FSMA 2000 제417조 제1항에 삽입한 것으로, 「any cryptographically secured digital representation of value or contractual rights that— (a) can be transferred, stored or traded electronically, and (b) that uses technology supporting the recording or storage of data (which may include distributed ledger technology)」이고 2023년 8월 29일 시행되었습니다.[39] 그 위에 2026년 2월 4일 제정된 The Financial Services and Markets Act 2000 (Cryptoassets) Regulations 2026(SI 2026/102)이 「qualifying cryptoasset」(대체가능·이전가능·가치나 계약상 권리의 기록에 그치지 않는 것, 전자화폐·법정통화·한정된 네트워크 안의 토큰 제외)과 「qualifying stablecoin」(특정 법정통화에 대해 안정적 가치를 겨냥하고 그 유지를 위해 법정통화나 다른 자산을 보유하는 것)을 규제 활동의 대상으로 정했습니다. 이 규정의 **본격 시행일(full commencement day)은 2027년 10월 25일이고**, 기준일 현재는 FCA의 규칙·지침 제정, 인가 신청의 접수·결정 등 준비 목적의 부분만 발효되어 있습니다(reg. 1(3)).[40] 재산법 쪽에서는 법률위원회가 2023년 6월 최종보고서(Law Com No 412)에서 「제3범주」를 확인하는 입법을 권고했고, Property (Digital Assets etc) Act 2025가 2025년 12월 2일 국왕 재가를 받아 같은 날 시행되었습니다. 조문은 둘이고, 「a thing in possession」도 「a thing in action」도 아니라는 사실만으로 인적 재산권의 객체임이 막히지 않는다고 적을 뿐 무엇이 재산인지는 정하지 않습니다(27.2 ⑧).[8][9] 판례로는 영국 항소법원(Civil Division)의 *Tulip Trading Ltd v van der Laan and others* [2023] EWCA Civ 83(2023년 2월 3일 선고, Lewison·Popplewell·Birss LJJ)이 있습니다. 개인키를 해킹으로 잃었다고 주장하는, 보유자라고 주장하는 회사가 비트코인 네트워크의 개발자들에게 신인의무(fiduciary duty)를 주장한 사건에서, 항소법원은 「The conclusion is not that there is a fiduciary duty in law in the circumstances alleged by Tulip, only that the case advanced raises a serious issue to be tried」라고 하여 국외 송달 허가를 취소한 1심(High Court, Chancery Division, Falk J, [2022] EWHC 667 (Ch))의 판단을 뒤집었습니다. Birss LJ는 「If the decentralised governance of bitcoin really is a myth, then in my

judgment there is much to be said for the submission that bitcoin developers, while acting as developers, owe fiduciary duties to the true owners of that property」라고 덧붙였습니다.[41] 8장의 「채굴자는 누구의 대리인인가」와 23장의 「누가 진짜 원장을 정하는가」가 법정에 들어온 모양이고, 28.1(g)가 받습니다.

이 절로 연결된 질문 상자

이 절을 가리키는 상자는 셋입니다. 14장의 「NFT를 산 사람은 무엇을 샀나」는 정의어 행이 받습니다 — EU는 「unique and not fungible」한 암호자산을 MiCA 적용 범위에서 뺐고(Art. 2(3)), 영국의 2026년 규정은 「fungible」을 「qualifying cryptoasset」의 요건으로 두어 대체불가 토큰을 제외했으며, 한국은 법 제2조 제1호 아목의 위임을 받은 시행령(시행 2026. 10. 2.) 제2조 제4호가 「단일하게 존재하여 다른 전자적 증표로 대체할 수 없는 전자적 증표」를 빼되 「특정 재화나 서비스의 지급수단으로 사용될 수 있는 전자적 증표 등 금융위원회가 정하여 고시하는 전자적 증표」는 다시 넣는 단서를 두었습니다(원문은 30장). 같은 상자의 「발행자의 수익 분배 약속은 코드에 있는가 바깥에 있는가」는 미국의 증권성 판단(Howey·SEC·Ripple)이 바깥의 약속과 거래의 모양을 본다는 데에서 답의 자리를 찾습니다. 15장의 「AMM에 운영자가 있는가」는 주무 기관과 발행·유통 행이 받습니다 — 네 법역의 규제 조문은 「a person」(MiCA Art. 59), 「者」(資金決済法 第63条の2), 「영업으로 하는 자」(가상자산법 제2조 제2호), 「person」(GENIUS §2(7))을 수범자로 전제하고(영국 규정의 수범자 조문은 이 판에서는 대조하지 않았습니다), 상수곱 식 하나가 돌아가는 계약에는 그 수범자가 비어 있습니다. GENIUS법 제2조 제7호 (B)(v)가 「participating in a liquidity pool or other similar mechanism for the provisioning of liquidity for peer-to-peer transactions」를 「digital asset service provider」에서 명시적으로 뺀 것은 그 빈자리를 입법이 인식했다는 사실이고, 그것이 옳은 선택인지는 이 책이 말하지 않습니다.[42] 24장의 「스테이블코인 보유자는 예금자인가 채권자인가 수익자인가」는 스테이블코인 행이 받습니다 — EU는 전자화폐토큰의 발행을 신용기관·전자화폐기관에만 허용했고(Art. 48), 미국은 결제 스테이블코인을 증권·상품에서 빼고 허가 발행자 체계를 두었으며(GENIUS §17), 일본은 「電子決済手段」으로 따로 정의했고, 영국은 「qualifying stablecoin」 발행을 규제 활동으로 정했으며, 한국은 별도 범주를 두지 않았습니다. 어느 법역도 보유자의 사법상 지위(예금자·채권자·수익자)를 그 정의 조문에서 바로 정하지는 않았습니다 — 그 질문은 28장.

표에 적지 않은 것

두 가지를 밝힙니다. 첫째, 싱가포르·스위스는 이 판에 넣지 않았습니다(애초 계획은 「필요하면 한 줄씩」이었고, 이 책의 원자료에 그 두 법역의 1차 자료가 없었습니다). 둘째, 각 법역의 **계류 중인 법안은** 적지 않았습니다. 이 책의 원칙상 사실만 적을 수 있는 대상이지만, 이 책의 원자료에 법안 원문이 없어 사실 자체를 확인하지 못했기 때문입니다. 일본의 金商法 이관은 「계류」가 아니라 「공포 완료·시행 전」이므로 표에 넣었습니다. 미국 GENIUS법도 제정되었으므로 넣었고, 시행일만 미확정으로 적었습니다.

정리

이 장은 세 가지를 했습니다. 법을 깊이 다루지 않는 다섯 이유를 밝혔고, 사법상 성질의 열네 갈래 견해를 1~3부의 기술 사실과 짝지었으며, 다섯 법역을 여섯 축으로 비교했습니다. 표 두 장을 겹쳐 읽으면 한 가지가 보입니다. **입법은 「무엇을 규제하는가」를 정의로 답했고, 법원은 「무엇인가」라는 성질의 질문을 피할 수 있을 때 피했으며, 학설은 같은 기술 사실을 서로 다른 결론의 근거로 썼습니다.** 그래서 기술 사실을 아는 것은 어느 견해를 고르는 일이 아니라, 각 견해가 **무엇을 설명해야 하는지를** 아는 일입니다. 개인키를 잃은 사람이 무엇을 잃었는지(5장), 최종성이 확률인 세계에서 지급은 언제 완료되는지(9장), 발행자가 없는 자산에 채무자가 있는지(19장), 다수가 규칙을 바꾸면 소수는 무엇을 주장하는지(22·23·26장) — 이 질문들은 28장의 일곱 쟁점 절이 받습니다. 그리고 다섯 법역이 같은 물건을 다섯 이름으로 부르는 것이 무엇을 가르는지는 30장이 정의 조문을 나란히 놓고 봅니다.

이 장의 출처

- 이혜정, 『가상자산에 대한 민사집행 연구』(사법정책연구원 연구총서 2022-12, 2022) — 제2장 제3절 「가상자산의 성질」(74~103쪽: 미국 74~75, 독일 80~84, 일본 84~85(가.), 국내 학계 88~98쪽, IV. 103쪽), 제3장(민사집행 설계, 요약 xviii~xix·제4절 210~212쪽), 국세징수법 개정 경위(166~173쪽). (422쪽; 인용 쪽은 인쇄 쪽). 보고서가 정리한 학설을 옮겼고 개별 논문 원문은 대조하지 않았다.
- 대법원 2018. 5. 30. 선고 2018도3619 판결; 대법원 2021. 11. 11. 선고 2021도9855 판결(공2022상, 56); 대법원 2021. 12. 16. 선고 2020도9789 판결 — 국가법령정보센터 판례 전문(Open API, 2026-10-25 조회). 본문을 읽었고 판시사항·판결요지는 인용하지 않았다.
- 「가상자산 이용자 보호 등에 관한 법률」(제정 법률 제19563호, 현행 제21857호) 제2·6·7·10·13·14조·목차; 「특정 금융거래정보의 보고 및 이용 등에 관한 법률」 제2조 제3호·제7조; 소득세법 제21조 제1항 제27호와 부칙 유예 이력(법률 제17757·18578·19196·20615호); 국세징수법 제55조 제3항·제66조 제2항; 지방세징수법 제61조 제3항; 민법 제98조·제703조 — 국가법령정보센터(Open API, 2026-10-25 조회). 소득세법 조문은 「현행」 표시와 달리 미시행이다(부칙 원문 확인).
- Regulation (EU) 2023/1114 (MiCA), OJ L 150, 9.6.2023, p. 40 — Art. 2(3)~(4), 3(1)(5)~(9), 4, 16, 48, 59, 86~92, 93, 117, 149.
- CJEU, *Skatteverket v David Hedqvist*, C-264/14, ECLI:EU:C:2015:718 (22 October 2015) — ¶¶24·42·52, 주문. Cellar XHTML(2026-10-25 접근).
- *SEC v. W. J. Howey Co.*, 328 U.S. 293 (1946) — 298~300쪽.
- SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017-07-25) — 1·11~15쪽.
- *SEC v. Ripple Labs, Inc.*, No. 1:20-cv-10832 (AT) (S.D.N.Y. July 13, 2023), ECF No. 874, 682 F. Supp. 3d 308 — 15~23·34쪽. 항소 경과의 SEC 공표문(Litigation Release)은 봇 차단으로 원문을 열지 못해 쓰지 않았다.
- *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. Mar. 6, 2018) — 3·24쪽.
- IRS Notice 2014-21, 2014-16 I.R.B. 938; Notice 2023-34, 2023-19 I.R.B. 837. `irs-2023-notice-2023-34-modification-2014-21.pdf`.
- GENIUS Act, Pub. L. No. 119-27, 139 Stat. 419 (July 18, 2025) — §2(6)·(11)·(22)·(25), §17, §20.
- *Uniform Commercial Code Amendments (2022)*, Article 12, §12-102 — 주석판 235쪽.

- 資金決済に関する法律 第2条第5項·第14項·第15項, 第63条の2; 金融商品取引法 第2条第3項; 令和八年法律第六十四号 반영 미시행 판본 金商法 第2条第49項 — e-Gov 법령 API v2(2026-10-25 조회).
- 東京地方裁判所 平成31年3月15日 判決(平成27年(刑わ)第2274号 등) — 「事実認定の補足説明」 第１の２(３).
- Financial Services and Markets Act 2000, s. 417(1); Financial Services and Markets Act 2023, s. 69 — legislation.gov.uk(2026-10-25 접근). The Financial Services and Markets Act 2000 (Cryptoassets) Regulations 2026, SI 2026/102 — reg. 1, RAO art. 88F·88G.
- Law Commission, *Digital Assets: Final Report* (Law Com No 412, 2023) — ¶¶2.45~2.61. Property (Digital Assets etc) Act 2025 (c. 29).
- *Tulip Trading Ltd v van der Laan and others* [2023] EWCA Civ 83 (3 February 2023) — ¶¶1~12·91~93.
- UNIDROIT, *Principles on Digital Assets and Private Law* (2023) — Principle 2, 서론 0.6.
- Gorton & Zhang (2023), 920쪽 — 24장과 같은 자리.
- 외국 법원·기관의 한국어 표기는 법원 「외국 법원 및 사법관련 국제기구 번역 가이드라인」(검토의견판,)을 따랐다 — 미국 연방대법원·미국 연방뉴욕남부지방법원·미국 연방뉴욕동부지방법원, 도쿄지방재판소, 영국 항소법원, 유럽연합 사법재판소(*Hedqvist*를 선고한 것은 Court of Justice(Fifth Chamber) — 가이드라인은 CJEU에 「유럽연합 사법재판소」 용례를 적고 「유럽연합 법원」 안을, ECJ에 「유럽연합 대법원·최고법원」 안을 검토 중이라 적는다). 「High Court」는 가이드라인이 안을 셋 두고 정하지 않아 원어로 적었다.
- 이 장은 코드·노트북이 없다(설계). 가격·시가총액 자료는 없다. 계류 법안은 원문을 확인하지 못해 적지 않았다.

각주

1. 이혜정, 『가상자산에 대한 민사집행 연구』(사법정책연구원 연구총서 2022-12, 2022-08-05 발행), 103쪽(제2장 제3절 IV.). 원문: 「한편, 가상자산을 물건으로 보고 그에 대한 권리를 인정할지에 대해 견해가 통일되어 있지 않아, 가상자산을 취득한 자가 '가상자산을 소유하고 있다'고 단정 짓기 힘들고, 이러한 고민은 가상자산 관련 다수의 논문과 판결에서 '소유' 대신 '보유'라는 단어를 사용하고 있는 것에서도 확인할 수 있다.」 (PDF 127쪽). 아래 각주의 쪽은 인쇄 쪽이다.

2. 이혜정(2022), 88~89쪽(제2장 제3절 III. 1. 가.). 긍정설의 논거: 「비트코인은 '비트코인 주소'에서 관리되고, 개인키를 가진 자에 의하여 기술적으로 통제받으므로 민법 제98조의 자연력에 해당한다는 견해」(각주 417, 김이수), 「당해 가상자산지갑의 개인키를 가진 자만이 송금지시를 내릴 수 있고(배타적 지배가능성), 그러한 송금지시 내역이 분산원장에 빠짐없이 기록되며, 한 번 송금이 완료된...비트코인은 전 세계 비트코인 네트워크 참여자의 과반수를 동시에 해킹하지 않는 이상 현실적으로 이중지불 될 수 없으므로(관리가능성), 비트코인은 민법상 물건 중 동산이라는 견해」(각주 418, 전승재·권헌영). 부정설의 논거(89쪽): 「개인키는 단순한 정보에 불과하고 복제가 가능할 뿐만 아니라, 정보에 대한 '점유'나 '보관'이라는 개념도 명확치 않아 엄격한 의미의 '배타성'이 있다고 보기 어렵다」(각주 419, 기노성), 「네트워크 참여자에 의한 승인이 없는 한 비트코인의 이전이 완결되지 않으므로 배타적 지배가능성이 다소 불완전하다」(각주 420, 정영수), 「비트코인에서 에너지를 추출할 수 없으므로 전통적인 의미의 '자연력'으로 보기 어렵다」(각주 421). 민법 제98조 원문: 「본법에서 물건이라 함은 유체물 및 전기 기타 관리할 수 있는 자연력을 말한다.」(국가법령정보센터, 2026-10-25 조회). 이 책은 보고서가 정리한 학설을 옮겼고 개별 논문 원문은 대조하지 않았다.

3. 이혜정(2022), 89~90쪽(III. 1. 나.). 원문: 「비트코인 보유자는 비트코인의 재산적 가치에 대해 독점적이고 배타적인 지배권을 행사하고, 거래의 상대방뿐만 아니라 누구에게나 비트코인에 대한 권리를 주장할 수 있으므로 물권과 유사한 권리를 갖는다고 보아야 하나, 이는 법률에 따라 부여된 권리가 아니므로 준물권과 유사한 성격의 '준물권적 구조를 가진 권리'로 본다」(각주 422, 이나래); 반대: 「블록체인의 기록은 비트코인 네트워크의 모든 참가자에 의해 공유되고 있으므로 설령 개인키를 가지고 있어도 비트코인을 독점적으로 지배하고 있다고 말하기는 쉽지 않다」(각주 423,

정영수). 일본 견해 ②~④에 대한 물권법정주의 지적은 85쪽(각주 402).

4. 이혜정(2022), 90쪽(III. 1. 다.·라.). 원문: 「① 송금지시의 목적이 되는 '당해 송금지시를 포함한 블록의 생성'이라는 급부는 블록체인 네트워크상에서 실행되는 컴퓨터 프로그램에 의해 이행되고 참여자 개개인은 여기에 필요한 컴퓨팅 자원을 각자 조금씩 제공할 뿐이므로 참여자 개개인을 '채무자'로 보기 힘들고, ② 비트코인 네트워크는 책임재산도 없고 사람에 의해 운영되는 중앙 통제기관도 없으므로 법인·단체로 취급하기도 곤란하며, ③ 비트코인 네트워크를 움직이는 컴퓨터 프로그램 자체를 채무자의 지위에 놓고 법을 따르라고 할 방법도 없다」(각주 424~425, 전승재·권헌영); 채권의 객체가 될 수 있다는 견해(각주 426). 거래소에 대한 「가상자산이전청구권」은 보고서 제3장의 집행 설계가 전제하는 것이다(xviii~xix쪽).

5. 이혜정(2022), 84~85쪽(II. 5. 가.). 견해 ①~⑧의 요지와 각 견해에 대한 지적을 옮겼다. 원문(견해 ⑦): 「비트코인에는 발행자가 존재하지 않는 점이나 소유권의 객체는 유체물에 한정되는 점 등을 근거로 비트코인의 보유는 물권·채권·사원권·지적재산권이라는 재산권을 수반하는 것이 아니라 사실상태에 불과하다는 견해」(각주 400, 片岡義広·武内斉史); 견해 ⑦에 대한 평가: 「이 견해에 따르면 물권적 청구의 우려가 배제되어 '소유와 점유의 일치'를 원칙으로 하는 금전과 유사한 취급이 확보되기 때문에 비트코인의 지불수단으로서의 유용성 확보라는 점에서 타당하다는 의견」(각주 405); 견해 ⑧에 대한 비판: 「합의의 법적 성질이나 법적 효과가 명확하지 않고, 프로토콜과는 구별되는 참가자 전원의 어떤 규범적 합의가 있다고 보기 어렵다」(각주 406); 견해 ⑥에 대한 평가: 「비트코인의 채굴에 의해 사상이 창작적으로 표현되어 있다고 평가하기 곤란하다」(각주 404). 일본 문헌 원문은 대조하지 않았다.

6. 이혜정(2022), 85쪽(견해 ⑤와 그 비판: 「단체의 성립·가입·탈퇴를 위한 법령이나 의사표시가 없는 이상 법적인 의미에서의 단체가 존재하지 않는다고 볼 수밖에 없으므로」, 각주 403), 93쪽(III. 1. 사.·아.: 사원권 부정 견해(각주 439), 조합권 견해 — 「'프로그램 시스템'을 계약의 내용으로, '블록체인의 내부적 거래'를 공동사업으로, '비트코인의 취득'을 상호 출자로 이해하여, 블록체인이 특수한 조합이고, 비트코인은 조합권이 될 수 있다는 견해가 있는데, 단, 이 견해도 비트코인이라는 조합권은 향후 예상되는 이익분배에 대한 기대권이며」). 민법 제703조 제1항 원문: 「조합은 2인 이상이 상호출자하여 공동사업을 경영할 것을 약정함으로써 그 효력이 생긴다.」(2026-10-25 조회).

7. 이혜정(2022), 81·83~84쪽(II. 4.). 원문: 「유체성을 가지고 있지 못하다는 점에 대해서는 학설상 다툼이 없으므로」(81쪽, 각주 372); 「가상자산이 채권이 아니라는 점에 대해서 학설이 일치한다」(각주 388); 「결국 가상자산은 민법상 물건은 아니지만 그 상위개념인 목적물로서 무형의 이익에 속한다는 점에 관해 학설이 일치하고 있다」(각주 393: Bachert 2021, Badstuber 2019, Boehm/Bruns 2017, Kütük/Sorge 2014, Spindler/Bille 2014). 독일 문헌 원문은 대조하지 않았다.

8. Law Commission, *Digital Assets: Final Report* (Law Com No 412, HC 1486, 2023-06-27 인쇄 명령, 2023-06-28 공개), 19쪽 ¶2.45, 22쪽 ¶2.53, 23쪽 ¶¶2.57~2.58·2.60. 원문: "we conclude that some digital assets are neither things in possession nor things in action, but that nonetheless the law of England and Wales treats them as capable of being things to which personal property rights can relate."(¶2.45); "We recommend legislation confirming that the fact that a thing is neither a thing in possession nor a thing in action does not prevent it from being a thing to which personal property rights can relate. Such things include crypto-tokens like bitcoin or ether which are not rights or claims in themselves and which can be used and enjoyed independently of whether any rights or claims in relation to them are enforceable by action."(¶2.53); 기준(이제는 「indicia」): "(1) it is composed of data ...; (2) it exists independently of persons and exists independently of the legal system; and (3) it is rivalrous."(¶2.57), "we conclude that 'composed of data' need not be a criterion in itself"(¶2.58); 경합성의 정의(각주 82): "A thing is rivalrous if the use or consumption of the thing by one person, or a specific group of persons, necessarily prejudices the use or consumption of that thing by one or more other persons." (304쪽; 인용 쪽은 인쇄 쪽). 보고서 각주 62는 「Our report does not consider regulatory matters」라고 밝힌다.

9. Property (Digital Assets etc) Act 2025 (c. 29), Royal Assent 2 December 2025, s. 1·s. 2. 원문: "1 Objects of personal property rights / A thing (including a thing that is digital or electronic in nature) is not prevented from being the object of personal property rights merely because it is neither— (a) a thing in possession, nor (b) a thing in action."; "2 ... (1) This Act extends to England and Wales and Northern Ireland. (2) This Act comes into force on the day on which it is passed." (legislation.gov.uk 인쇄본). 조문은 둘뿐이다.

10. UNIDROIT, *Principles on Digital Assets and Private Law* (Rome, 2023), Principle 2(1)·(2), 11쪽. 원문: "(1) 'Electronic record' means information which is (i) stored in an electronic medium and (ii) capable of being retrieved. (2) 'Digital asset' means an electronic record which is capable of being subject to control." 서론 0.6은 「control」이 보통법 법역의 'control'이나 대륙법의 '점유'와 같지 않다고 적는다: "The concept of control used in these Principles, for instance, is not intended to be understood as 'control' as used in certain common law jurisdictions. Also, while being akin to the concept of 'possession' as used in certain civil law jurisdictions, control as used in these Principles must not be understood to be identical to such possession"(2쪽). 집행이사회

제102차 회기(2023-05) 채택.

11. *Uniform Commercial Code Amendments (2022)*, Article 12 「Controllable Electronic Records」, § 12-102(a)(1), 주석판 235쪽. 원문: "'Controllable electronic record' means a record stored in an electronic medium that can be subjected to control under Section 12-105. The term does not include a controllable account, a controllable payment intangible, a deposit account, an electronic copy of a record evidencing chattel paper, an electronic document of title, electronic money, investment property, or a transferable record." ALI 2022년 5월·ULC 2022년 7월 연차총회 승인(서문). 통일법은 모델법이고 각 주의 채택 여부·시기는 이 책이 확인하지 않았다.

12. 이혜정(2022), 91~92쪽(III. 1. 마.). 원문(부정설): 「가치척도의 기준이 되기 위해서는 그 가치가 일정해야 하고 발행수량을 유동적으로 조절하여 상황에 따라 가치기준을 유지할 수 있어야 하는데, 비트코인은 시장가격이 급등락하고, 발행총량이 2100만 BTC로 한정되어 있기 때문에 화폐로서의 기능을 할 수 없다는 것이다」(각주 431); (긍정설): 「현재보다는 미래에 화폐로 수용될 가능성을 긍정적으로 전망하고 있다」(각주 432~434). 이 책은 양쪽을 옮기되 전망에는 가담하지 않는다(3장 일러두기).

13. Judgment of the Court (Fifth Chamber) of 22 October 2015, *Skatteverket v David Hedqvist*, Case C-264/14, ECLI:EU:C:2015:718(Cellar 메타데이터로 확인), ¶¶24·42·52·57, 주문 1·2. 원문: "the 'bitcoin' virtual currency with bidirectional flow, which will be exchanged for traditional currencies in the context of exchange transactions, cannot be characterised as 'tangible property' within the meaning of Article 14 of the VAT Directive, given that, as the Advocate General has observed in point 17 of her Opinion, that virtual currency has no purpose other than to be a means of payment."(¶24); "The 'bitcoin' virtual currency, being a contractual means of payment, cannot be regarded as a current account or a deposit account, a payment or a transfer."(¶42); "In the case in the main proceedings, it is common ground that the 'bitcoin' virtual currency has no other purpose than to be a means of payment and that it is accepted for that purpose by certain operators."(¶52); 주문 2: "Article 135(1)(e) of Directive 2006/112 must be interpreted as meaning that the supply of services such as those at issue in the main proceedings, which consist of the exchange of traditional currencies for units of the 'bitcoin' virtual currency and vice versa, ..., are transactions exempt from VAT, within the meaning of that provision". Cellar XHTML(`publications.europa.eu/resource/cellar/bb73ad87-78a7-11e5-86db-01aa75ed71a1.0006.01/DOC_2` , 2026-10-25 접근). 판결은 부가가치세 지침의 해석이고 사법상 성질의 판단이 아니다.

14. IRS Notice 2014-21, 2014-16 I.R.B. 938, §4 Q&A-1·Q&A-2, 2쪽. 원문: "Q-1: How is virtual currency treated for federal tax purposes? A-1: For federal tax purposes, virtual currency is treated as property. General tax principles applicable to property transactions apply to transactions using virtual currency."; "Q-2: ... A-2: No. Under currently applicable law, virtual currency is not treated as currency that could generate foreign currency gain or loss for U.S. federal tax purposes." IRS Notice 2023-34, 2023-19 I.R.B. 837, §3: "The Department of the Treasury and the Internal Revenue Service are aware that certain foreign jurisdictions have enacted laws that characterize Bitcoin as legal tender. Thus, the sentence in the Background section of Notice 2014-21 stating that virtual currency does not have legal tender status in any jurisdiction is no longer accurate as to Bitcoin."; §3 끝: "This change to the Background section of Notice 2014-21 does not affect the answers to the FAQs set forth in section 4 of Notice 2014-21 ...". `irs-2023-notice-2023-34-modification-2014-21.pdf` .

15. 대법원 2021. 12. 16. 선고 2020도9789 판결(원심 수원고등법원 2020. 7. 2. 선고 2020노171 판결, 파기환송), 이유 3. 다. 원문: 「가상자산은 국가에 의해 통제받지 않고 블록체인 등 암호화된 분산원장에 의하여 부여된 경제적인 가치가 디지털로 표상된 정보로서 재산상 이익에 해당한다(대법원 2021. 11. 11. 선고 2021도9855 판결 참조). 가상자산은 보관되었던 전자지갑의 주소만을 확인할 수 있을 뿐 그 주소를 사용하는 사람의 인적사항을 알 수 없고, 거래 내역이 분산 기록되어 있어 다른 계좌로 보낼 때 당사자 이외의 다른 사람이 참여해야 하는 등 일반적인 자산과는 구별되는 특징이 있다. 이와 같은 가상자산에 대해서는 현재까지 관련 법률에 따라 법정화폐에 준하는 규제가 이루어지지 않는 등 법정화폐와 동일하게 취급되고 있지 않고 그 거래에 위험이 수반되므로, 형법을 적용하면서 법정화폐와 동일하게 보호해야 하는 것은 아니다.」; 이유 3. 가.: 「가상자산을 이체받은 자는 가상자산의 권리자 등에 대한 부당이득반환의무를 부담하게 될 수 있다. 그러나 이는 당사자 사이의 민사상 채무에 지나지 않고 ... 피고인은 어떠한 경위로 이 사건 비트코인을 이체받은 것인지 불분명하여 부당이득반환청구를 할 수 있는 주체가 피해자인지 아니면 거래소인지 명확하지 않다.」 국가법령정보센터 판례 전문(Open API, 판례 일련번호 219775, 2026-10-25 조회). 판결 본문을 읽었고 판시사항·판결요지는 인용하지 않았다.

16. 이혜정(2022), 92~93쪽(III. 1. 바.). 원문: 「비트코인을 '유가증권'으로 보기 힘들다는 견해가 대체적인바, 그 논거로 ① 비트코인은 발행인이 없고, 블록체인 상에 트랜잭션이 기재되는 것을 유가증권의 배서로 보기 힘들므로, 발행행위가 필요하고 배서·교부에 의해 이전하는 '유가증권'에 비트코인이 포함된다고 보기 힘든 점, ② ... 비트코인 그 자체는 교환의

매개물일 뿐 별도의 권리나 청구권이 화체되어 있지 않다는 점 등이 제시된다」(각주 436~438). 자본시장법상 금융투자상품성에 관한 논의는 같은 보고서 95~97쪽.

17. *SEC v. W. J. Howey Co.*, 328 U.S. 293 (1946) (decided May 27, 1946), 298~299쪽. 원문: "an investment contract for purposes of the Securities Act means a contract, transaction or scheme whereby a person invests his money in a common enterprise and is led to expect profits solely from the efforts of the promoter or a third party, it being immaterial whether the shares in the enterprise are evidenced by formal certificates or by nominal interests in the physical assets employed in the enterprise."; "It embodies a flexible rather than a static principle, one that is capable of adaptation to meet the countless and variable schemes devised by those who seek the use of the money of others on the promise of profits."(299쪽); "the arrangements whereby the investors' interests are made manifest involve investment contracts, regardless of the legal terminology in which such contracts are clothed."(300쪽). (U.S. Reports 원본, 미국 의회도서관).

18. SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017-07-25), 1·11~15쪽. 원문: "the Commission has determined that DAO Tokens are securities under the Securities Act of 1933 … and the Securities Exchange Act of 1934"(1쪽); "In analyzing whether something is a security, 'form should be disregarded for substance,' …, 'and the emphasis should be on economic realities underlying a transaction, and not on the name appended thereto.'"(11쪽); "Their efforts, not those of DAO Token holders, were the 'undeniably significant' ones, essential to the overall success and profitability of any investment into The DAO."(15쪽). 26장 3절과 같은 자리.

19. *SEC v. Ripple Labs, Inc.*, No. 1:20-cv-10832 (AT) (S.D.N.Y. July 13, 2023) (Torres, J.), Order, ECF No. 874, 682 F. Supp. 3d 308, 15·23·34쪽. 원문: "XRP, as a digital token, is not in and of itself a 'contract, transaction[,] or scheme' that embodies the Howey requirements of an investment contract."(15쪽); 기관 판매: Howey 세 요건 충족(16~22쪽); 프로그램 매매: "Ripple's Programmatic Sales were blind bid/ask transactions, and Programmatic Buyers could not have known if their payments of money went to Ripple, or any other seller of XRP."(23쪽); "the SEC's motion for summary judgment is GRANTED as to the Institutional Sales, and otherwise DENIED. Defendants' motion for summary judgment is GRANTED as to the Programmatic Sales, the Other Distributions, and Larsen's and Garlinghouse's sales"(34쪽). (RECAP). 보고서 쪽수 682 F. Supp. 3d 308은 같은 법원의 2024-08-07 명령(ECF 973)의 인용으로 확인. 그 뒤 항소 경과를 알리는 SEC 공표문은 봇 차단으로 원문을 열지 못해 이 책은 적지 않는다.

20. Guiding and Establishing National Innovation for U.S. Stablecoins Act (GENIUS Act), Pub. L. No. 119-27, 139 Stat. 419 (July 18, 2025), §2(22)·§17·§20. 원문: "(22) PAYMENT STABLECOIN.—The term 'payment stablecoin'— (A) means a digital asset— (i) that is, or is designed to be, used as a means of payment or settlement; and (ii) the issuer of which— (I) is obligated to convert, redeem, or repurchase for a fixed amount of monetary value, not including a digital asset denominated in a fixed amount of monetary value; and (II) represents that such issuer will maintain, or create the reasonable expectation that it will maintain, a stable value relative to the value of a fixed amount of monetary value; and (B) does not include a digital asset that— (i) is a national currency; (ii) is a deposit …; or (iii) is a security, …, except that, for the avoidance of doubt, no bond, note, evidence of indebtedness, or investment contract that was issued by a permitted payment stablecoin issuer shall qualify as a security solely by virtue of its satisfying the conditions described in subparagraph (A), consistent with section 17 of this Act."(139 Stat. 421~422); §17 제목 "AMENDMENTS TO CLARIFY THAT PAYMENT STABLECOINS ARE NOT SECURITIES OR COMMODITIES AND PERMITTED PAYMENT STABLECOIN ISSUERS ARE NOT INVESTMENT COMPANIES"; §20: "This Act, and the amendments made by this Act, shall take effect on the earlier of (1) the date that is 18 months after the date of enactment of this Act; or (2) the date that is 120 days after the date on which the primary Federal payment stablecoin regulators issue any final regulations implementing this Act." (govinfo). 제정일 2025-07-18 기준 상한은 2027-01-18이고, 그보다 이른 시행은 최종 규칙 공표에 달려 있다 — 2026-10-25 현재 규칙 공표 여부는 이 책이 확인하지 않았다(부록 A). §17(a)의 「The term 'security' does not include a payment stablecoin …」 문장은 투자자문업법 §202(a)(18)에 추가되는 것이고, 증권법·증권거래법 쪽은 같은 조의 다른 항이다.

21. *CFTC v. McDonnell*, No. 18-CV-361 (JBW), 287 F. Supp. 3d 213 (E.D.N.Y. Mar. 6, 2018) (Weinstein, J.), Memorandum & Order, 3·24쪽. 원문: "Until Congress clarifies the matter, the CFTC has concurrent authority, along with other state and federal administrative agencies, and civil and criminal courts, over dealings in virtual currency."(3쪽); "Virtual currencies can be regulated by CFTC as a commodity. Virtual currencies are 'goods' exchanged in a market for a uniform quality and value. … They fall well-within the common definition of 'commodity' as well as the CEA's definition of 'commodities' as 'all other goods and articles . . . in which contracts for future delivery are presently or in the future dealt in.' Title 7 U.S.C. § 1(a)(9)."(24쪽); "The jurisdictional authority of CFTC to regulate virtual currencies as commodities does not preclude other

agencies from exercising their regulatory power when virtual currencies function differently than derivative commodities."(24쪽). (govinfo). 가처분 단계의 결정이다.

22. 대법원 2021. 11. 11. 선고 2021도9855 판결(원심 서울고등법원 2021. 7. 9. 선고 2020노357 판결, 상고기각), 이유 2. 가. 원문: 「비트코인은 경제적인 가치를 디지털로 표상하여 전자적으로 이전, 저장과 거래가 가능하도록 한 가상자산의 일종으로 사기죄의 객체인 재산상 이익에 해당한다.」 참조조문 형법 제347조. 국가법령정보센터 판례 전문(Open API, 판례 일련번호 222659)(2026-10-25 조회). 공보: 공2022상, 56(2020도9789 판결의 참조판례 표시에서).

23. 대법원 2018. 5. 30. 선고 2018도3619 판결(원심 수원지방법원 2018. 1. 30. 선고 2017노7120 판결, 상고기각), 이유 1. 가. 3)·5). 원문: 「범죄수익은닉규제법에 정한 중대범죄에 해당하는 범죄행위에 의하여 취득한 것으로 재산적 가치가 인정되는 무형재산도 몰수할 수 있다.」; 「피고인이 범죄수익은닉규제법에 정한 중대범죄에 해당하는 정보통신망법 위반(음란물유포)죄와 도박개장방조죄에 의하여 취득한 비트코인은 재산적 가치가 있는 무형의 재산이라고 보아야 한다. 그 이유는 다음과 같다. ① 비트코인은 경제적인 가치를 디지털로 표상하여 전자적으로 이전, 저장 및 거래가 가능하도록 한, 이른바 '가상화폐'의 일종이다. ② 피고인은 …비트코인을 대가로 지급받아 재산적 가치가 있는 것으로 취급하였다.」 국가법령정보센터 판례 전문(Open API, 판례 일련번호 197020)(2026-10-25 조회). 2018년 판결은 「가상화폐」, 2021년 두 판결은 「가상자산」이라는 낱말을 쓴다 — 30장.

24. Gary B. Gorton & Jeffery Y. Zhang, "Taming Wildcat Stablecoins," *University of Chicago Law Review* 90(3): 909–972 (2023), 920쪽. 원문은 24장 각주 29 에 전재했다: "In order for the stablecoins to be considered a deposit—let alone a demand deposit—one must determine whether the underlying contract between the holder of stablecoins and the stablecoin issuer is an equity contract or a debt contract."

25. 東京地方裁判所 平成31年3月15日 判決(平成27年(刑わ)第2274号 등), 「事実認定の補足説明」 第１の２(３). 26장 각주 12 와 같은 자리.

26. 「가상자산 이용자 보호 등에 관한 법률」 제2조 제1호(법률 제19563호 제정 문언, 현행 법률 제21857호에서 불변), Regulation (EU) 2023/1114 Art. 3(1)(5). 원문은 30장에 전재한다.

27. 법제처 국가법령정보센터, 「가상자산 이용자 보호 등에 관한 법률」 연혁(Open API, MST 290735, 법령 ID 014474 조문 연혁, 2026-10-25 조회). 제정 법률 제19563호(공포 2023. 7. 18., 부칙 제1조 「공포 후 1년이 경과한 날」 시행 → 2024. 7. 19.); 현행 법률 제21857호(공포 2026. 8. 4., 시행 2026. 10. 2.). 조문 목차: 제1조 목적 ~ 제22조 과태료(22개 조문). 제2조 제1호 원문은 30장에 전재.

28. 「특정 금융거래정보의 보고 및 이용 등에 관한 법률」(현행 법률, 공포 2026. 8. 4., 시행 2026. 10. 2.) 제2조 제3호·제7조 제1항. 원문: 「3. "가상자산"이란 「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산을 말한다.」; 「① 가상자산사업자(이를 운영하려는 자를 포함한다. 이하 이 조에서 같다)는 대통령령으로 정하는 바에 따라 다음 각 호의 사항을 금융정보분석원장에게 신고하여야 한다.」(국가법령정보센터 Open API(MST 290725, 제2조·제7조), 2026-10-25 조회). 가상자산 정의가 이 법에 처음 들어온 것은 법률 제17113호(공포 2020. 3. 24., 시행 2021. 3. 25.)이고, 2024. 7. 19. 가상자산법 부칙으로 현행 준용 문언이 되었다(두 판본 원문은 30장).

29. 소득세법 제21조 제1항 제27호(법률 제17757호로 신설, 공포 2020. 12. 29.). 원문: 「27. 「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산(이하 "가상자산"이라 한다)을 양도하거나 대여함으로써 발생하는 소득(이하 "가상자산소득"이라 한다)」. 시행일 유예 이력(각 개정 법률의 부칙 개정문 원문): 법률 제17757호 부칙 제1조 제2호 「2022년 1월 1일」 → 법률 제18578호(2021. 12. 8.) 「2023년 1월 1일」 → 법률 제19196호(2022. 12. 31.) 「2025년 1월 1일」 → 법률 제20615호(2024. 12. 31.) 「2027년 1월 1일」. 국가법령정보센터 「현행」 본문(MST 280405)은 이 호를 시행 중인 것처럼 보여 주므로, 이 책은 부칙 원문의 확인을 따라 「미시행」으로 적는다(2026-10-25 조회).

30. 국세징수법(현행, 시행 2026. 10. 2.) 제55조 제3항·제66조 제2항 제2호; 지방세징수법(현행, 시행 2026. 10. 2.) 제61조 제3항. 원문: 「③ 관할 세무서장은 제2항에 따라 「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산(…)을 압류하려는 경우 체납자[같은 법 제2조제2호에 따른 가상자산사업자(…) 등 제3자가 체납자의 가상자산을 보관하고 있을 때에는 그 제3자를 말한다]에게 대통령령으로 정하는 바에 따라 해당 가상자산의 이전을 문서로 요구할 수 있고, 요구받은 체납자 또는 그 제3자는 이에 따라야 한다. <신설 2021.12.21, 2023.7.18>」; 「② … 2. 가상자산사업자를 통해 거래되는 가상자산: 가상자산사업자를 통한 매각 <개정 2021.12.21>」; 지방세징수법 제61조 「③ … 해당 가상자산의 이전을 요구하여야 한다. 이 경우 제3자에게 이전을 요구하는 경우에는 문서로 하여야 한다. <신설 2022.1.28, 2023.7.18>」(Open API, 2026-10-25 조회). 이 조문들의 입법 경위는 이혜정(2022) 166~173쪽.

31. 이혜정(2022), xviii~xix쪽(요약)·210~212쪽(제3장 제4절 IV. 2.). 원문: 「이때 민사집행법상 가상자산의 성질을 '그 밖의 재산권'으로 보고, 이를 기초로 채권집행, 동산집행 규정 등을 적절히 조화하여 가상자산에 대한 집행 규정안을 제시하였다」; 「① 채무자가 그의 가상자산을 관리할 경우에는 '그 밖의 재산권'인 가상자산 자체를 압류하고, '가상자산 이전명령'에 의해 집행관이 채무자의 가상자산을 이전받으면, 매각명령에 의해 환가된 금전을 집행법원이 배당하는

절차로 집행안을 구상하였고, ② 제3자가 가상자산을 관리하고 채무자가 제3자에 대해 가상자산이전청구권을 가지는 경우에는 '가상자산이전청구권' 자체를 '그 밖의 재산권'으로서 압류한 후」. 유체동산으로 볼 때의 난점(블록체인에 압류 표시를 할 기술이 없고, 지갑에 표시해도 압류 대상이 특정되지 않는다)은 211~212쪽.

32. Regulation (EU) 2023/1114 of the European Parliament and of the Council of 31 May 2023 on markets in crypto-assets, OJ L 150, 9.6.2023, p. 40, Art. 149. 원문: "2. This Regulation shall apply from 30 December 2024. 3. By way of derogation from paragraph 2, Titles III and IV shall apply from 30 June 2024."(L 150/183). (Cellar PDF).

33. MiCA Art. 3(1)(5)~(7)·(9), Art. 2(3)~(4), L 150/62~63. 원문: "(5) 'crypto-asset' means a digital representation of a value or of a right that is able to be transferred and stored electronically using distributed ledger technology or similar technology; (6) 'asset-referenced token' means a type of crypto-asset that is not an electronic money token and that purports to maintain a stable value by referencing another value or right or a combination thereof, including one or more official currencies; (7) 'electronic money token' or 'e-money token' means a type of crypto-asset that purports to maintain a stable value by referencing the value of one official currency; … (9) 'utility token' means a type of crypto-asset that is only intended to provide access to a good or a service supplied by its issuer"; "3. This Regulation does not apply to crypto-assets that are unique and not fungible with other crypto-assets. 4. This Regulation does not apply to crypto-assets that qualify as one or more of the following: (a) financial instruments; (b) deposits, including structured deposits; (c) funds, except if they qualify as e-money tokens; …".

34. MiCA Art. 4(1)(L 150/67), Art. 16(1)(L 150/76), Art. 48(1)(L 150/107), Art. 59(1), Title VI Art. 86·89·91(L 150/140~143), Art. 93(1)(L 150/144), Art. 117(1)(L 150/161). 원문(발췌): "A person shall not make an offer to the public of a crypto-asset other than an asset-referenced token or e-money token in the Union unless that person: (a) is a legal person; (b) has drawn up a crypto-asset white paper in respect of that crypto-asset in accordance with Article 6"(Art. 4); "A person shall not make an offer to the public, or seek the admission to trading, of an asset-referenced token, within the Union, unless that person is the issuer of that asset-referenced token and is: (a) a legal person or other undertaking that is established in the Union and has been authorised in accordance with Article 21 by the competent authority of its home Member State; or (b) a credit institution that complies with Article 17."(Art. 16); "A person shall not make an offer to the public or seek the admission to trading of an e-money token, within the Union, unless that person is the issuer of such e-money token and: (a) is authorised as a credit institution or as an electronic money institution"(Art. 48); "A person shall not provide crypto-asset services, within the Union, unless that person is: (a) a legal person or other undertaking that has been authorised as crypto-asset service provider in accordance with Article 63"(Art. 59); "This Title shall also apply to any transaction, order or behaviour concerning crypto-assets as referred to in paragraph 1, irrespective of whether such transaction, order or behaviour takes place on a trading platform."(Art. 86(2)); "No person shall engage in or attempt to engage in market manipulation."(Art. 91(1)); "Member States shall designate the competent authorities responsible for carrying out the functions and duties provided for in this Regulation."(Art. 93(1)); "Where an asset-referenced token has been classified as significant in accordance with Article 43 or 44, the issuer of such asset-referenced token shall carry out its activities under the supervision of EBA."(Art. 117(1)).

35. 資金決済に関する法律(平成二十一年法律第五十九号) 第二条第十四項(e-Gov 법령 API v2, 현행 시행 판본 `421AC0000000059_20260812_508AC0000000064`, 2026-10-25 조회). 원문은 30장에 전재. 개명 개정: 令和元年法律第二十八号 「情報通信技術の進展に伴う金融取引の多様化に対応するための資金決済に関する法律等の一部を改正する法律」, 公布 2019-06-07, 施行 2020-05-01(2020-04-30 판본 第2条第5項 「仮想通貨」 → 2020-05-01 판본 「暗号資産」 → 2023-06-01 판본부터 第14項). 第5項에서 第14項으로 옮긴 2023-06-01 시행 판본의 e-Gov 판본 식별자는 `421AC0000000059_20230601_504AC0000000061`(令和四年法律第六十一号)이다 — 개정문 자체는 대조하지 않았다.

36. 資金決済法 第六十三条の二·第二条第十五項(같은 판본). 원문: 「(暗号資産交換業者の登録) 第六十三条の二 暗号資産交換業は、内閣総理大臣の登録を受けた者でなければ、行ってはならない。」; 「この法律において 「暗号資産交換業」とは、次に掲げる行為のいずれかを業として行うことをいい、… 一 暗号資産の売買又は他の暗号資産との交換 二 前号に掲げる行為の媒介、取次ぎ又は代理 三 その行う前二号に掲げる行為に関して、利用者の金銭の管理をすること。 四 他人のために暗号資産の管理をすること(…)。」 第二条第五項 「電子決済手段」 정의도 같은 판본에서 확인했다.

37. 金融商品取引法(昭和二十三年法律第二十五号) 第二条第三項(e-Gov 현행 판본 `323AC0000000025_20260812_508AC0000000064`, 2026-10-25 조회). 괄호 안 원문:

「(電子情報処理組織を用いて移転することができる財産的価値
(電子機器その他の物に電子的方法により記録されるものに限る。) に表示される場合
(流通性その他の事情を勘案して内閣府令で定める場合を除く。) に限る。以下「電子記録移転権利」という。)」.
資金決済法 第二条第十四項 단서: 「ただし、
金融商品取引法第二十九条の二第一項第八号に規定する権利を表示するものを除く。」

38. 令和八年法律第六十四号 「金融商品取引法及び資金決済に関する法律の一部を改正する法律」, 公布 2026-07-23. e-Gov 법령 API v2 개정 이력과 미시행 판본(`323AC0000000025_20270722_508AC0000000064` , `current_revision_status: UnEnforced` , 2026-10-25 조회): 金融商品取引法 第二条第四十九項
「この法律において「暗号資産」とは、次に掲げるものをいう。ただし、
第二十九条の二第一項第八号に規定する権利を表示するものを除く。」; 같은 날짜의 資金決済法 미시행 판본 第二条에서는 暗号資産 정의가 빠지고 第十四項은 「電子決済手段サービス仲介業」 정의가 된다. 시행 단계 2026-07-23·08-12(시행 완료), 2027-03-31·04-01·07-22·10-01, 2028-01-01·06-13(미시행); 2027-07-22 판본의 시행 주석은 「公布の日から起算して一年を超えない範囲内において政令で定める日」. 정령은 미확인. 「계류」가 아니라 「공포 완료·정의 부분 시행 전」이다 — 부록 A.

39. Financial Services and Markets Act 2000, s. 417(1) 「cryptoasset」 정의(Financial Services and Markets Act 2023 (c. 29), s. 69(4)(a)로 삽입). 원문은 본문 인용과 같다. 시행: "I2 S. 69 in force at 29.8.2023 by S.I. 2023/779, reg. 4(uu)". legislation.gov.uk `ukpga/2000/8/section/417` · `ukpga/2023/29/section/69` (HTML, 2026-10-25 접근·축어 대조). s. 69(4)(b)는 재무부가 규정으로 정의를 바꿀 수 있게 한다("(5) The Treasury may by regulations amend the definition of 'cryptoasset' in subsection (1).").

40. The Financial Services and Markets Act 2000 (Cryptoassets) Regulations 2026, SI 2026/102, made 4th February 2026, reg. 1(2)~(3), 그리고 이 규정이 Regulated Activities Order(SI 2001/544)에 삽입하는 art. 88F·88G(36~37쪽). 원문: "(2) Subject to paragraph (3) these Regulations come into force on 25th October 2027 (the 'full commencement day'). (3) These Regulations come into force at the end of the period of 21 days beginning with the day after the day on which they are made, for the following purposes— (a) enabling the FCA—"; "88F.—(1) A qualifying cryptoasset. (2) A 'qualifying cryptoasset' means a cryptoasset which is— (a) fungible, (b) transferable, (c) not solely a record of value or contractual rights, including rights in another cryptoasset, and (d) not excluded by paragraph (4)."; "(4) A cryptoasset does not fall within paragraph (1) if it is— (a) a specified investment cryptoasset, ...; (b) electronic money; (c) currency of the United Kingdom or any other country or territory, including a central bank digital currency; (d) a cryptoasset that— (i) cannot be transferred or sold in exchange for money or other cryptoassets, except by way of redemption with the issuer, and (ii) can only be used by the holder— (aa) to acquire goods or services from the issuer, or (bb) to acquire goods or services within a limited network"; "88G.—(1) A qualifying stablecoin. (2) A 'qualifying stablecoin' means a qualifying cryptoasset where— (a) that cryptoasset seeks or purports to maintain a stable value in relation to a particular fiat currency ('the referenced fiat currency'), and (b) fiat currency ... or other assets are held for the purpose of maintaining a stable value". (legislation.gov.uk). FCA 최종 규칙 PS26/9~13(2026-06-30 공표)은 목록 위치만 확인했고 이 책은 인용하지 않는다.

41. *Tulip Trading Ltd (a Seychelles company) v van der Laan and others* [2023] EWCA Civ 83 (CA-2022-001050 등), Court of Appeal (Civil Division), judgment handed down 3 February 2023, Lewison, Popplewell and Birss LJJ, ¶¶1~2·5·91~93. 원문: "The question in this appeal is whether the developers who look after bitcoin may arguably owe fiduciary duties or duties in tort to an owner of that cryptocurrency."(¶1); "The conclusion was that Tulip had not established a serious issue to be tried because there was no realistic prospect of establishing that the facts pleaded amount to a breach of fiduciary or tortious duty owed by the defendants to Tulip."(¶5, 1심 요약); "I would allow this appeal. The conclusion is not that there is a fiduciary duty in law in the circumstances alleged by Tulip, only that the case advanced raises a serious issue to be tried. The time to decide on the duty in this case is once the facts are established. ... If the decentralised governance of bitcoin really is a myth, then in my judgment there is much to be said for the submission that bitcoin developers, while acting as developers, owe fiduciary duties to the true owners of that property."(¶91, Birss LJ; Popplewell LJ·Lewison LJ 동의 ¶¶92~93). (judiciary.uk). 사건명은 항소심 표지대로 「v van der Laan」이다(제1피고 Bitcoin Association for BSV는 항소심 피항소인 대리 목록에 없다). 이 판결은 관할 단계의 판단(심리할 쟁점의 존재)이고 본안 판단이 아니다. 「High Court」의 한국어 표기는 법원 번역 가이드라인이 안을 셋 두고 정하지 않아 원어로 적었다.

42. GENIUS Act §2(7), 139 Stat. 419~420. 원문: "(7) DIGITAL ASSET SERVICE PROVIDER.—The term 'digital asset service provider'— (A) means a person that, for compensation or profit, engages in the business in the United States ... of— (i) exchanging digital assets for monetary value; (ii) exchanging digital assets for other digital assets; (iii) transferring digital assets to a third party; (iv) acting as a digital asset custodian; or (v)

participating in financial services relating to digital asset issuance; and (B) does not include— (i) a distributed ledger protocol; (ii) developing, operating, or engaging in the business of developing distributed ledger protocols or self-custodial software interfaces; (iii) an immutable and self-custodial software interface; (iv) developing, operating, or engaging in the business of validating transactions or operating a distributed ledger; or (v) participating in a liquidity pool or other similar mechanism for the provisioning of liquidity for peer-to-peer transactions."

5부 법제도

28장 기술과 법의 다리 (2) — 쟁점별 개관, 더 읽을 곳

핵심 TAKEAWAY

이 장은 1~4부가 던진 「법률가의 질문」 상자 가운데 이 장을 가리키는 열여섯 개를 일곱 쟁점 — (a) 착오이체와 형사책임 (b) 토큰·NFT·스테이킹의 성질 (c) 스마트계약의 계약성과 이행 시점 (d) 개인정보 삭제권과 불변성 (e) 시장 규제 (f) 증거 (g) DAO·하드포크·거버넌스 — 로 받습니다. 쟁점마다 한두 쪽입니다. 하는 일은 셋입니다. 상자가 던진 질문을 그대로 되받고, 그 질문이 기대는 **기술 사실이** 1~3부의 어느 장에 있는지를 가리키고, 원문을 확인한 1차 자료(판결은 법원·선고일·사건번호, 조문은 법률 번호·시행일)가 그 질문의 **어느 지점에** 닿는지를 적습니다. 답은 쓰지 않습니다. 한국 대법원이 「원인불명으로 재산상 이익인 가상자산을 이체받은 자」를 배임죄로 처벌하는 것은 죄형법정주의에 반한다고 한 것, SEC가 「코드로 자동화되었다는 사실이 행위를 증권법의 범위 밖으로 빼내지 않는다」고 한 것, 영국 항소법원이 개발자의 신인의무를 「심리할 중대한 쟁점」이라 한 것은 모두 그 지점을 보여 주는 사례이지 결론이 아닙니다. 28.2는 원자료 확인에서 서지가 확정된 1차 자료와 개설서만 적었고, 모자라는 자리는 비워 두었습니다.

학습목표

1. 열여섯 개 질문 상자가 일곱 쟁점의 어디에 닿는지 지도를 가진다.
2. 쟁점마다 「기술 사실이 들어가는 지점」과 「1차 자료가 이미 답한 지점」과 「아직 비어 있는 지점」을 가린다.
3. 한국·외국 법원의 판결을 법원·선고일·사건번호로, 조문을 법률 번호·시행일로 찾아갈 수 있다.
4. 「통계적 판정」(25장)·「코드대로 실행」(13·16장)·「다수의 결정」(22·23·26장)이 각각 어떤 법적 요건의 **증거일** 수는 있어도 그 요건 **자체는** 아니라는 구별을 쓴다.

[입문]만 읽은 독자가 가져가는 것. 일곱 쟁점 어디에서도 기술은 답을 정하지 않습니다. 기술이 정하는 것은 **질문이 어디서 생기는가이고**, 그 자리를 아는 것이 이 장의 전부입니다.

28.1 쟁점별 개관

(a) 착오이체와 형사책임 — 그리고 강제집행

상자. 11장은 물었습니다. 착오로 보낸 코인은 누구의 것인가. UTXO 모형에서는 「누구의 열쇠로도 열리지 않는 출력」이 생겼을 뿐이고 계정 모형에서는 「어떤 주소의 잔고 숫자」가 늘었다. 「점유의 이전」이라는 말은 어디에 맞는가. 받은 사람이 그것을 자기 주소로 옮겼다면 그 행위는 무엇인가. 열쇠를 잃어 옮길 수 없다면 반환 의무는 어떻게 되는가.

기술 사실이 들어가는 지점. 11장이 보인 대로 비트코인에는 「잔고」가 없고 지출되지 않은 출력의 합이 있으며, 이더리움에는 잔고가 있습니다. 어느 쪽이든 원장은 송금의 **원인을** 적지 않습니다 — 매매대금인지 착오인지 원장은 모릅니다(5장: 원장은 서명의 유효성만 봅니다). 그리고 「되돌리기」가 없습니다. 보낸 사람이 거래를 취소할 수 없고(9장: 최종성), 받은 사람이 돌려보내는 **새 거래만** 있습니다. 은행 송금의 착오와 다른 점은 중간에 환급을 집행할 기관이 없다는 것이고, 같은 점은 받은 사람의 손에 「법률상 원인 없는」 재산이 있다는 것입니다.

1차 자료가 닿는 지점. 한국 대법원은 2021년 12월 16일 선고 2020도9789 판결에서 바로 이 자리를 다뤘습니다. 피고인은 「알 수 없는 경위로 피해자의 … 거래소 가상지갑에 들어 있던 199.999비트코인 … 을 자신의 계정으로 이체받」은 뒤 그 대부분을 자기 다른 계정으로 옮겼고, 원심은 착오송금의 횡령 법리를 유추해 배임죄를 인정했습니다. 대법원은 파기환송했습니다. 「가상자산 권리자의 착오나 가상자산 운영 시스템의 오류 등으로 법률상 원인관계 없이 다른 사람의 가상자산 전자지갑에 가상자산이 이체된 경우, 가상자산을 이체받은 자는 가상자산의 권리자 등에 대한 부당이득반환의무를 부담하게 될 수 있다. 그러나 이는 당사자 사이의 민사상 채무에 지나지 않고 이러한 사정만으로 가상자산을 이체받은 사람이 신임관계에 기초하여 가상자산을 보존하거나 관리하는 지위에 있다고 볼 수 없다.」 그리고 「원인불명으로 재산상 이익인 가상자산을 이체받은 자가 가상자산을 사용·처분한 경우 이를 형사처벌하는 명문의 규정이 없는 현재의 상황에서 착오송금 시 횡령죄 성립을 긍정한 판례 … 를 유추하여 신의칙을 근거로 피고인을 배임죄로 처벌하는 것은 죄형법정주의에 반한다.」[1] 판결은 11장의 질문에 세 가지로 닿습니다. 첫째, 판결은 가상자산을 「재산상 이익」이라 했고 「재물」인지는 판단하지 않았습니다 — 횡령(주위적 공소사실)은 배임 부분과 「동일체의 관계」에 있다는 이유로 함께 파기되었을 뿐이고, 재물성은 학설이 갈리는 쟁점으로 남아 있습니다. 둘째, 「부당이득반환청구를 할 수 있는 주체가 피해자인지 아니면 거래소인지 명확하지 않다」 — 26장이 본 장부와 체인의 두 층이 여기서 다시 나타납니다. 셋째, 판결은 가상자산이 「법정화폐와 동일하게 취급되고 있지 않고 그 거래에 위험이 수반되므로, 형법을 적용하면서 법정화폐와 동일하게 보호해야 하는 것은 아니다」라고 적었습니다. 이 판결이 「민사상 채무」라고 한 반환의무가 **무엇의** 반환인지 — 같은 수량의 코인인지, 그 가액인지 — 는 판결의 판단 범위 밖이고, 27.2의 성질 논의가 그 자리를 채워야 합니다.

같은 재판부의 2021년 11월 11일 선고 2021도9855 판결은 「비트코인은 … 사기죄의 객체인 재산상 이익에 해당한다」고 했고, 2018년 5월 30일 선고 2018도3619 판결은 범죄수익은닉규제법상 「재산적 가치가 인정되는 무형재산도 몰수할 수 있다」면서 압수된 비트코인 가운데 범죄수익에 해당하는 수량을 특정해 몰수한 원심을 유지했습니다.[2] 세 판결을 겹치면 형사법은 비트코인을 **얻은 쪽**(사기의 객체, 몰수의 대상)에서는 재산으로 다루었고, **원인 없이 받은 쪽**(배임의 주체인가)에서는 신임관계의 부재를 들어 처벌을 부정했습니다. 이 비대칭이 옳은지는 이 책이 말하지 않습니다.

강제집행. 민사집행법에는 가상자산 조문이 없습니다. 사법정책연구원 보고서는 가상자산을 「그 밖의 재산권」으로 보고 — 유체동산으로 보면 「블록체인에 채무자의 가상자산이 압류되었다는 표시를 할 기술은 현재 그 구현이 불가능」해 압류의 효력 발생 요건을 갖출 수 없다는 이유 등으로 — 채무자가 스스로 관리하는 경우에는 가상자산 자체를, 거래소가 관리하는 경우에는 「가상자산이전청구권」을 압류하고 집행관에게 이전시켜 매각하는 입법안을 제안했습니다.[3] 강제징수 쪽은 이미 입법되었습니다. 국세징수법 제55조 제3항은 세무서장이 가상자산을 압류할 때 체납자 또는 그것을 보관하는 사업자에게 「해당 가상자산의 이전을 문서로 요구할 수 있」고 요구받은 자는 「이에 따라야 한다」고 정했고(2021. 12. 21. 신설), 제66조 제2항 제2호는 「가상자산사업자를 통해 거래되는 가상자산」을 사업자를 통해 직접 매각할 수 있게 했습니다.[4] 두 조문이 전제하는 기술 사실은 하나입니다 — **체인 위의 코인은 개인키 없이는 옮길 수 없으므로, 집행은 열쇠를 가진 사람에게 「옮기라」고 명하는 형태를 취할 수밖에 없다**

(5장). 11장의 상자가 마지막에 물은 「열쇠를 잃어 옮길 수 없다면」은 바로 이 요구가 닿지 않는 자리이고, 어느 조문도 그 경우를 정하지 않았습니다.

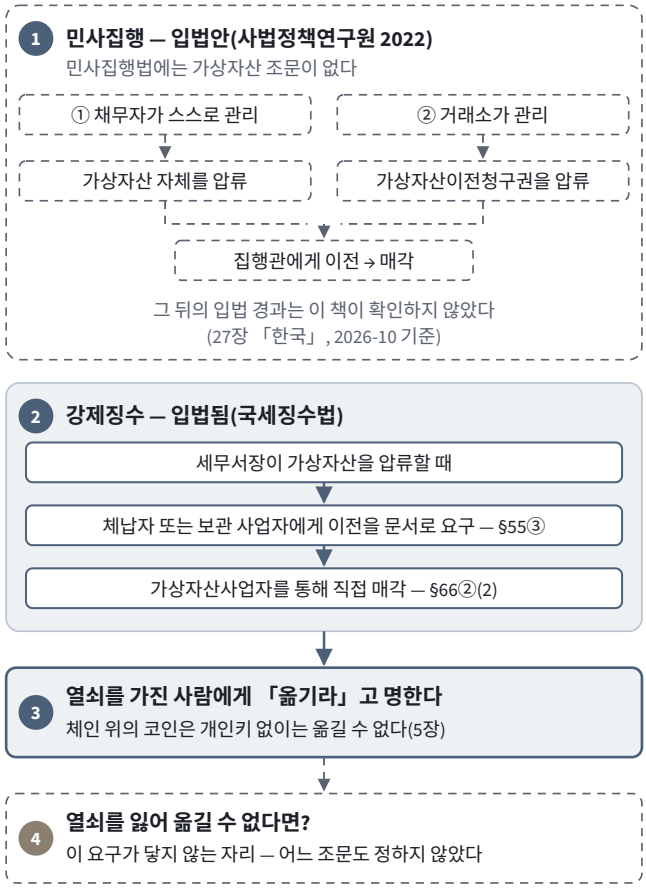


그림 28-1. 가상자산 강제집행의 세 층. 민사집행은 사법정책연구원 보고서의 입법안(점선)이고, 국세 강제징수는 국세징수법 제55조 제3항·제66조 제2항 제2호로 이미 입법되었다. 시행 중인 조문은 열쇠를 가진 사람에게 옮기라고 명하는 형태이고, 열쇠를 잃은 경우는 어느 조문도 정하지 않았다

(b) 토큰·NFT·스테이킹의 성질 — 과세를 짧게

상자. 10장은 물었습니다. 스테이킹 보상은 용역의 대가인가, 맡긴 코인이 낳은 과실인가, 새 코인의 원시취득인가. 슬래싱으로 사라진 보증금은 벌금인가 위약금인가 담보의 실행인가. 남의 코인을 받아 대신 스테이킹해 주는 사업자는 수탁자인가 채무자인가. 14장은 물었습니다. NFT를 산 사람은 토큰을 샀나, 파일을 샀나, 권리를 샀나, 「가리킨다」는 사실을 샀나. 발행자가 서버를 닫으면 무엇이 멸실되나. 발행자가 수익 분배를 약속했다면 그 약속은 코드에 있나 바깥에 있나. 24장은 물었습니다. 보유자는 예금자인가, 무담보 채권자인가, 신탁의 수익자인가. 「회사 자원으로 부족분을 메우겠다」는 공언은 어떤 권리를 주는가.

기술 사실이 들어가는 지점. 세 상자의 공통 사실은 14장의 것입니다 — **토큰 표준은 함수 이름의 약속일 뿐이고, 토큰이 무엇을 담는지는 발행자의 바깥 약속이 정합니다.** ERC-20 토큰의 잔고, ERC-721의 (계약 주소, 번호) 쌍과 `tokenURI` 문자열, 스테이블코인의 상환 약속, 스테이킹 보상의 발행 규칙은 전부 체인 위의 상태이지만, 그것이 「권리」인지는 체인이 정하지 않습니다. 10장의 사실은 스테이킹 보상이 프로토콜의 발행 규칙에서 나오고 슬래싱이 같은 규칙의 일부라는 것, 그리고 보증금은 「처벌이 보안의 일부」라는 설계라는 것입니다.

1차 자료가 닿는 지점. 세 갈래입니다.

첫째, **증권성은** 바깥의 약속과 거래의 모양을 봅니다. SEC는 2017년 DAO 토큰을 투자계약으로 판단하면서 「form should be disregarded for substance … and the emphasis should be on economic realities underlying a transaction, and not on the name appended thereto」라는 연방대법원 판례를 인용했고, 토큰 보유자의 투표권이 있어도 「DAO Token holders were substantially reliant on the managerial efforts of Slock.it, its co-founders, and the Curators」라고 했습니다.[5] 2023년 *Ripple* 약식판결은 같은 토큰의 **기관 판매는** 투자계약이고 거래소의 **익명 매매는** 아니라고 가렸습니다 — 「Programmatic Buyers could not have known if their payments of money went to Ripple, or any other seller of XRP」.[6] 14장 상자의 마지막 질문에 대한 미국법의 답은 「약속은 바깥에 있고, 그 약속이 누구에게 어떤 경로로 전달되었는가가 증권성을 정한다」입니다. 미국 의회는 2025년 GENIUS법으로 허가받은 발행자의 결제 스테이블코인을 증권·상품에서 뺐습니다(§17).[7] 한국 가상자산이용자보호법은 「주식·사채 등의 전자등록에 관한 법률」의 전자등록주식등을 가상자산에서 제외하고(제2조 제1호 라목), 자본시장법상 증권성 판단은 이 책 범위 밖입니다.

둘째, **NFT는** 세 법역의 정의가 갈립니다. EU MiCA는 「crypto-assets that are unique and not fungible with other crypto-assets」에 적용되지 않고(Art. 2(3)), 영국의 2026년 규정은 「fungible」을 「qualifying cryptoasset」의 요건으로 두어 대체불가 토큰을 규제 활동의 대상에서 뺐으며, 한국은 가상자산법 제2조 제1호 아목의 위임을 받은 시행령 제2조 제4호가 「단일하게 존재하여 다른 전자적 증표로 대체할 수 없는 전자적 증표」를 빼되 「특정 재화나 서비스의 지급수단으로 사용될 수 있는 전자적 증표 등 금융위원회가 정하여 고시하는 전자적 증표는 제외」한다는 단서를 두었습니다.[8] 14장 상자의 「무엇을 샀나」에 대해 정의 조문이 말하는 것은 「규제가 들어오는가」까지이고, 매매의 목적물이 토큰인지 파일인지 권리인지는 어느 조문도 정하지 않습니다. 27.2의 ⑨(통제 기반)는 「(계약 주소, 번호) 쌍의 소유자 칸을 바꿀 수 있는 능력」을 귀속의 단위로 보는 답을 함의하고, 그 답에서는 서버가 닫혀 그림이 사라져도 「통제의 대상」은 그대로 남습니다 — 그것이 상자가 물은 「무엇이 멸실되었나」에 대한 한 가지 가능한 대답이지 이 책의 대답은 아닙니다.

셋째, **스테이블코인 보유자의 지위는** 24장이 고른과 장의 말로 물었습니다 — 「equity contract or a debt contract」. 네 법역(한국은 별도 범주를 두지 않았습니다 — 27.3)의 입법은 발행자의 **자격과 준비자산을** 정했습니다. EU는 전자화폐토큰을 신용기관·전자화폐기관만 발행하게 했고(MiCA Art. 48), 미국 GENIUS법은 허가 발행자가 「obligated to convert, redeem, or repurchase for a fixed amount of monetary value」할 것을 정의에 넣었으며(§2(22)(A)(ii)(I)), 일본은 「電子決済手段」의 첫 유형(제1호)을 「通貨建資産」에 한정해 정의했고, 영국은 「fiat currency … or other assets are held for the purpose of maintaining a stable value」를 「qualifying stablecoin」의 요건으로 두었습니다.[9] 어느 조문도 보유자가 예금자인지 채권자인지 수익자인지를 그 자리에서 정하지 않습니다. 발행자 도산 시의 순위, 준비자산의 분리, 「공언」의 효력은 각 법역의 도산법·신탁법으로 넘어가고, 그 지점이 24장 상자가 가리킨 곳입니다.

과세를 짧게. 10장 상자의 「스테이킹 보상은 무엇의 대가인가」는 과세에서 가장 먼저 답을 요구받는 질문인데, 이 책의 원자료에는 그 답을 적은 1차 자료가 없습니다. 적을 수 있는 것은 두 가지입니다. 미국 국세청은 2014년 고시에서 가상통화를 「property」로 다루고 재산 거래의 일반 원칙을 적용한다고 했고, 2023년 고시는 그 결론을 그대로 두었습니다.[10] 한국 소득세법 제21조 제1항 제27호는 「가상자산을 양도하거나 대여함으로써 발생하는 소득」을 기타소득으로 정했으나 **2026-10-25 현재 시행 전(시행 2027. 1. 1.)**이고, 그 문언은 「양도·대여」만 적어 스테이킹 보상이 어디에 들어가는지는 조문에서 바로 읽히지 않습니다.[11] 이 책은 그 해석에 들어가지 않습니다.

(c) 스마트계약의 계약성과 이행 시점

상자. 13장은 물었습니다. 열 줄짜리 「contract」에서 청약과 승낙은 어디에 있나. 코드 배포가 청약이고 함수 호출이 승낙인가, 둘 다 사실행위인가. 코드가 작성자의 의도와 다르게 움직였을 때 「계약 내용」은 코드인가 의도인가. 가스가 바닥나 실행이 되돌려졌지만 수수료는 남았다면 무엇이 이행되고 무엇이 불이행되었나. 황서가 말한 「자율적 행위자」가 되면 의사표시의 주체는 누구인가. 16장은 물었습니다. 오라클이 틀린 값을 넣었고 코드는 한 줄도 어긋나지 않았다 — 이행인가 착오인가, 누구의 착오인가, 아무도 오라클을 운영하지 않았다면 값을 민 사람은 무엇을 조작했나. 9장은 물었습니다. 지급은 언제 완료되었나 — 전파된 때, 블록에 들어간 때, 그 위에 z 개가 쌓인 때. 약관이 「6 확인」을 정하면 그것이 기준인가. 체인이 재편되어 거래가 사라지면 이행은 처음부터 없었던 것이 되나.

기술 사실이 들어가는 지점. 세 상자는 세 가지 사실에 기댑니다. 첫째, 13장 — 스마트계약은 「조건이 맞을 때에만 값을 풀어 주는 상자」이고, 상태 전이는 거래가 블록에 들어갈 때 일어나며, 가스 소진 시 상태는 되돌려지되 수수료는 남습니다. 둘째, 16장 — 체인은 바깥 사실을 스스로 알 수 없고, 바깥 사실은 누군가를 믿어야 들어옵니다. 셋째, 9장 — 완료의 순간은 없고 되돌릴 확률이 작아질 뿐이며, 「6 확인」은 관행이지 규칙이 아닙니다. 법이 「의사표시」·「이행」·「완료」라는 **시점과 주체가 있는** 개념을 쓸 때, 기술은 그 셋 가운데 시점은 확률로, 주체는 비워서 줍니다.

1차 자료가 닿는 지점. 계약성에 관해 이 책이 원문을 확인한 1차 자료는 SEC의 2017년 보고서가 유일합니다. 보고서는 「The automation of certain functions through this technology, "smart contracts," or computer code, does not remove conduct from the purview of the U.S. federal securities laws」라고 적고, 각주에서 「smart contract」를 Nick Szabo의 정의 — 「a computerized transaction protocol that executes terms of a contract」 — 로 소개합니다.[12] 이 문장이 닿는 지점은 「코드가 실행했다」는 사실이 **행위자를** 지우지 않는다는 것이지, 코드가 계약인지는 말하지 않습니다. 26장의 The DAO가 그 예입니다 — 코드는 적힌 대로 실행되었고, SEC는 그 코드를 배포하고 홍보한 사람들의 「노력」을 보았습니다. 16장 상자의 「값을 민 사람은 무엇을 조작했나」에는 시장 규제의 조문이 닿을 수 있고, 그것은 (e)에서 봅니다.

비어 있는 지점. 9장 상자의 「지급은 언제 완료되는가」에 대해 이 책은 1차 자료를 적지 못했습니다. 9장은 결제 시스템의 최종성을 「규칙이 정한 취소 불가 시점」이라는 개념으로만 적고 어느 시스템의 어느 규칙인지는 특정하지 않았으며, 5부에서 결제 최종성 법제의 1차 자료로 받치겠다고 했는데, 이 책의 원자료에 그 법제의 원문(결제 최종성에 관한 법령·지침)이 없어 이 판에서 받치지 못했습니다. 유보로 남기고 보고합니다. 적을 수 있는 것은 구조입니다 — 결제 시스템의 최종성이 **규칙이 정한 시점이라면**, 블록체인의 최종성은 **확률이 작아지는 과정이어서**, 약관이 「6 확인」을 정하는 것은 그 과정 위에 규칙의 시점을 **덧씌우는** 일입니다. 그 덧씌움이 당사자 사이에서는 기준이 되더라도 체인이 재편되면 거래 자체가 사라질 수 있다는 것(9장 4절)이 상자의 마지막 질문이고, 그 자리에 어느 법역의 1차 자료가 있는지는 다음 판의 과제입니다.

(d) 개인정보 삭제권과 불변성

상자. 7장은 물었습니다. 자기 개인정보가 담긴 거래가 체인에 기록되었다며 삭제를 구한다. 체인은 고칠 수 없게 설계되어 있고, 고치려면 수많은 참가자가 그 뒤의 모든 블록을 다시 만들어야 한다. 삭제권은 지울 수 없는 원장에서 무엇을 요구하는가. 삭제의 상대방은 누구이고 「지운다」는 것은 기술적으로 어떤 행위인가. 처음부터 개인정보를 체인 밖에 두고 해시만 올리는 설계는 문제를 푸는가, 미루는가.

기술 사실이 들어가는 지점. 7장이 보인 것은 불변성이 수학적 불가능이 아니라 **변조 비용이라는**

것입니다. 한 블록의 한 글자를 바꾸면 그 뒤 모든 블록의 해시가 어긋나고, 어긋남을 없애려면 그 뒤를 전부 다시 캐야 합니다(7·8장). 그러니 「지운다」는 것은 기술적으로 (가) 그 블록 이후를 다시 만드는 일 — 다수의 조정이 필요하고, 공개 메인넷에서 그렇게 과거 기록을 지운 사례는 이 책의 원자료에 없습니다. 26장의 The DAO 포크는 과거 블록을 다시 캔 것이 아니라 블록 1,920,000에서 합의 규칙에 예외적 상태 변경을 넣어 잔고를 옮긴 것이어서(EIP-779) 이 유형이 아닙니다 — 이거나, (나) 각 노드가 자기 사본에서 그 데이터를 지우는 일 — 다른 노드의 사본에는 남습니다 — 이거나, (다) 체인에는 해시만 있고 원문은 밖에 있어 밖의 원문을 지우는 일입니다. (다)에서 체인에 남는 해시는 4장이 보인 대로 원문을 되돌릴 수 없는 지문이지만, 원문이 다른 곳에 남아 있으면 「그 원문이 그때 있었다」는 증명으로 되살아납니다 — 상자가 「푸는가, 미루는가」라고 물은 이유입니다.

1차 자료가 닿는 지점. 한국 개인정보 보호법 제36조는 정보주체가 「개인정보처리자에게 그 개인정보의 정정 또는 삭제를 요구할 수 있」고(제1항 — 다른 법령이 수집 대상으로 명시한 개인정보에는 단서가 있습니다), 처리자는 「지체 없이 그 개인정보를 조사하여 정보주체의 요구에 따라 정정·삭제 등 필요한 조치」를 해야 하며(제2항 — 다른 법령에 특별한 절차가 있으면 제외), 「삭제할 때에는 복구 또는 재생되지 아니하도록 조치하여야 한다」(제3항)고 정합니다.[13] EU 일반개인정보보호규정(GDPR) 제17조는 제1항이 든 사유 가운데 하나가 있을 때(그리고 제3항의 예외에 해당하지 않을 때) 정보주체가 「the erasure of personal data concerning him or her without undue delay」를 받을 권리와 관리자의 「obligation to erase personal data without undue delay」를 정하고, 관리자가 그 정보를 공개했고 제1항에 따라 지울 의무를 지는 경우에는 「taking account of available technology and the cost of implementation, shall take reasonable steps, including technical measures, to inform controllers which are processing the personal data」라고 정합니다.[14] 두 조문이 7장의 상자에 닿는 지점은 세 낱말입니다. 「개인정보처리자」·「controller」 — 공개 블록체인에서 그것은 누구인가(8장: 채굴자는 누구의 대리인도 아니다). 「복구 또는 재생되지 아니하도록」 — 다른 노드의 사본과 해시가 남는 구조에서 그것은 가능한가. 「available technology and the cost of implementation」 — 변조 비용이 바로 그 「cost」인가. 어느 법역의 법원 판결도 이 책의 원자료에는 없고, 이 책은 판단하지 않습니다.

(e) 시장 규제 — 거래소·시세조종·MEV

상자. 8장은 물었습니다. 채굴자는 누구의 대리인인가. 유효한 거래를 블록에 넣지 않은 채굴자에게 의무가 있나. 특정 주소의 거래를 모두 거부하는 채굴자 집단은 누구에 대한 어떤 행위를 한 것인가. 15장은 물었습니다. 거래소 규제가 전제하는 「운영자」 — 등록하고, 상장을 심사하고, 시세조종을 감시하고, 고객 자산을 분리 보관할 누군가 — 는 상수곱 AMM의 어디에 있나. 17장은 물었습니다. 샌드위치 공격자는 피해자의 주문을 한 글자도 바꾸지 않았다. 이것은 시세조종인가. 조종된 「시세」가 있나, 조종한 「행위자」는 누구인가, 규제가 전제하는 「시장」은 어디에 있나. 생산자가 직접 샌드위치를 했다면 그는 자기 블록에서 무엇을 한 것인가. 25장은 물었습니다. 「GSADF 검정에서 1% 수준으로 폭발적이었다」는 의견서는 시세조종의 증거인가, 기망의 증거인가, 다수 균형 가운데 하나가 실현되었다는 관측인가.

기술 사실이 들어가는 지점. 네 상자는 서로 다른 사실에 기댑니다. 8장 — 블록 생산자는 어떤 계약으로도 묶여 있지 않고, 「CPU 하나에 한 표」의 다수는 단체가 아닙니다. 15장 — AMM의 가격은 풀 안 두 수량의 비율이고 호가도 운영자도 상대방도 없습니다. 17장 — 순서를 정하는 권한이 있고, 그 권한은 코드로 막히지 않으며, 제안자–구축자 분리 뒤에는 순서가 경매로 팔립니다. 25장 — 폭발적 단위근 검정은 기초가치 가정에 따라 판정이 뒤집히고, 「버블이었다」는 통계 명제이지 행위 명제가 아닙니다.

1차 자료가 닿는 지점. 시장 규제는 한국·EU의 조문과 미국의 판결을 원문으로 확인했습니다. 한국 가상자산이용자보호법 제10조는 미공개중요정보 이용(제1항), 통정매매·가장매매(제2항), 「가상자산의 매매를 유인할 목적으로 가상자산의 매매가 성황을 이루고 있는 듯이 잘못 알게 하거나 그 시세를 변동 또는 고정시키는 매매 또는 그 위탁이나 수탁」(제3항), 「부정한 수단, 계획 또는 기교를 사용하는 행위」 등(제4항), 사업자의 자기발행 가상자산 거래(제5항)를 금지하고, 위반자에게 손해배상책임을 지웁니다(제6항). 「누구든지」로 시작하는 제2~4항은 수범자를 사업자에 한정하지 않습니다.[15] EU MiCA Title VI는 「crypto-assets that are admitted to trading or in respect of which a request for admission to trading has been made」에 관해 「any transaction, order or behaviour ..., irrespective of whether such transaction, order or behaviour takes place on a trading platform」에 적용되고, 내부자거래(Art. 89)와 시장조작(Art. 91)을 금지합니다.[16] 미국에서는 *McDonnell* 결정이 상품거래법의 사기 금지 조항을 가상통화의 현물 거래에 적용할 CFTC의 권한을 인정했습니다.[17] 거래소 진입은 한국의 신고(특정금융정보법 제7조), EU의 인가(MiCA Art. 59), 일본의 등록(資金決済法 第63条の2), 영국의 2027년 시행 예정 규제 활동(SI 2026/102)입니다(27.3).

조문이 상자에 닿는 자리와 닿지 않는 자리. 한국 제10조 제3항의 「시세를 변동 또는 고정시키는 매매」와 MiCA 제91조의 「market manipulation」은 17장의 샌드위치가 들어갈 **문언의 자리를** 줍니다 — 「누구든지」·「any person」이고, 「시세」라는 문언에 특정 거래소로 한정하는 말은 없습니다(그것이 어디까지 미치는지는 해석의 몫이고 이 책은 들어가지 않습니다). 그러나 두 조문은 서로 다른 자리에서 멈춥니다. MiCA는 「admitted to trading」이거나 상장 신청된 암호자산을 전제하고, 한국 제10조 제3항은 「매매를 유인할 목적」을 요구합니다. 15장 상자의 「운영자」와 8장 상자의 「대리인」은 어느 조문도 채우지 않습니다 — 조문은 행위를 금지할 뿐 AMM의 배포자·유동성 공급자·거버넌스 다수 가운데 누가 「운영자」인지, 블록 생산자가 거래를 거부하는 것이 「행위」인지를 말하지 않습니다. GENIUS법이 유동성 풀 참여를 「digital asset service provider」에서 명시적으로 뺀 것(27.3)은 그 빈자리를 입법이 알고 비워 둔 한 사례입니다. 25장 상자에 대해서는 조문이 아니라 구조로 답할 수 있습니다. 어느 조문의 요건도 「폭발적 구간」이 아닙니다 — 한국 제10조는 「목적」과 「매매」를, MiCA 제91조는 「activities」를, 상품거래법은 「fraud」를 요건으로 삼습니다. 통계 판정은 그 요건의 **증거가** 될 수는 있어도 요건 자체가 아니고, 25장이 보인 대로 그 판정은 기초가치 가정과 임계값 선택에 따라 뒤집힙니다. 「버블이 아니었다」는 판정도 같은 이유로 피고를 바로 돕지 않습니다.

(f) 증거 — 해시·타임스탬프·원본

상자. 4장은 물었습니다. 문서의 해시값만 공개 원장에 적어 두었다가 수년 뒤 원본 파일과 함께 제출했다. 이 해시값은 무엇을 증명하고 무엇을 증명하지 못하는가. 「그 시점에 그 비트열이 존재했다」와 「그 시점에 그 문서가 작성되었다」와 「그 문서의 내용이 진실이다」는 각각 다른 명제다. 증거법의 원본·사본·진정성립 개념은 이 셋 중 어디에 닿는가. 1장은 물었습니다 — 원장이 여럿이면 어느 사본이 원본인가.

기술 사실이 들어가는 지점. 4장이 보인 대로 해시는 되돌릴 수 없는 지문이고, 같은 입력에서 같은 출력이 나오며, 한 글자가 다르면 전체가 바뀝니다. 그래서 해시가 체인에 있고 원본이 제출되었다면, 해시를 다시 계산해 맞춰 보는 것으로 「이 비트열이 그 블록 시각 이전에 존재했다」는 명제는 증명됩니다 — **단, 블록 시각의 뜻을 정확히 읽어야 합니다.** 7장이 하버와 스토네타의 원문으로 적은 대로, 타임스탬프 체인이 보증하는 것은 「늦은 시각으로 찍을 수 없다 / 이른 시각으로 소급할 수 없다」는 두 방향의 **경계이고,** 비트코인 헤더의 시각은 채굴자가 적는 「대략의 시각」이고, 프로토콜은 그것이 앞뒤 블록과 크게 어긋나지 않게만 제한합니다(8장). 그리고 해시는 「작성」과 「진실」에 대해서는 아무것도 말하지 않습니다 — 거짓 문서도 지문이 있습니다. 12장이 보인 제네시스 블록의 신문 제목 디코딩이 좋은

예입니다. 블록에 그 문구가 있다는 사실은 그 블록이 그 문구가 공개된 **뒤에** 만들어졌다는 것까지만 말하고, 그 문구가 실제 신문 제목이었는지는 블록이 증명하지 않습니다(12장은 그래서 조건부로 썼습니다). 1장의 「어느 사본이 원본인가」는 6·7장이 답의 자리를 줍니다 — 사본은 전부 같은 뿌리 해시를 가지므로 어느 것도 「원본」이 아니면서 어느 것이나 원본과 같습니다.

1차 자료가 닿는 지점. 한국 민사소송법 제357조는 「사문서는 그것이 진정한 것임을 증명하여야 한다」, 제358조는 「사문서는 본인 또는 대리인의 서명이나 날인 또는 무인(拇印)이 있는 때에는 진정한 것으로 추정한다」고 정하고, 전자서명법 제3조는 전자서명이 「전자적 형태라는 이유만으로 서명, 서명날인 또는 기명날인으로서의 효력이 부인되지 아니한다」고 정합니다.[18] 세 조문이 4장 상자에 닿는 지점은 「진정성립」이 **작성자의** 문제라는 것입니다 — 서명·날인은 「누가 썼는가」를 추정시키고, 해시는 「무엇이 있었는가」를 증명합니다. 둘은 다른 명제입니다. 5장이 보인 디지털 서명은 개인키 보유자가 그 비트열을 승인했다는 것을 증명하고, 그 개인키가 누구의 것인지는 체인 밖의 사실입니다.
도쿄고등재판소의 2020년 판결은 전자적 기록의 증명 기능에 관해 이렇게 적었습니다.
「電磁的記録の証明機能は，それが作出されるべきシステムの設置運営主体の意思に基づき，正当に作出されることが当然の前提とされているもので」 — 전자적 기록은 「それを使用することが予定されている一定のシステムやプログラムのもとで用いられて初めて本来の証明機能を果たし得る」.[19] 형사 조문의 해석이지만, 거래소 장부의 숫자가 「무엇을 증명하는가」를 **그 시스템이 무엇을 하기로 되어 있었는가에서** 읽은 판단이고, 4장 상자의 「해시는 무엇을 증명하는가」도 같은 방식으로 — 그 체인이 무엇을 하기로 되어 있었는가(7장: 존재의 시각 경계)에서 — 답이 시작됩니다.

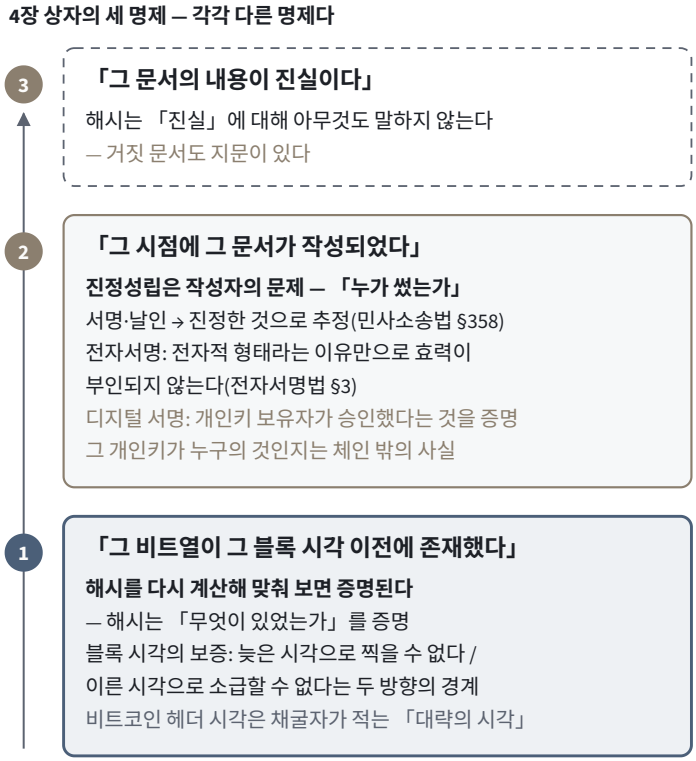


그림 28-2. 4장 상자의 세 명제와 각 명제에 닿는 수단. 해시는 비트열이 블록 시각 이전에 존재했다는 것을 증명하고, 서명·날인은 누가 썼는가를 추정시키며, 내용의 진실에 대해서는 해시가 아무것도 말하지 않는다

(g) DAO·하드포크·거버넌스의 법적 지위

상자. 넷입니다. 22장 — 거버넌스 토큰의 다수가 규칙을 바꿨다. 소수는 무엇을 주장할 수 있나. DAO는 조합인가(구성원은 무한책임인가), 법인인가(어느 법에 따라 설립되었나), 아무것도 아닌가(큐레이터의 의무는 누구에 대한 것인가). 23장 — 하드포크 뒤 두 체인이 모두 살아남았다. 「진짜」 원장은 누가 정하나 — 해시파워 다수, 노드 다수, 거래소, 개발자, 아무도. 포크 전 보유자는 하나를 가졌나 둘을 가졌나. 「비트코인 1개를 지급한다」는 계약은 어느 체인의 1개인가. 이기적 채굴자는 무엇을 위반했나. 26장 — The DAO의 하드포크는 구제인가, 몰수인가, 다수결에 의한 계약 변경인가. 「프로토콜은 바뀌지 않았고 비정규 상태 변경이 있었다」는 구별이 법적으로 의미가 있나. 그리고 6부의 29장이 묻습니다 — 취약 코인의 지출을 소프트포크로 막는 것은 수용인가, 질서유지인가, 무주물 처리인가, 결정 주체는 누구의 대리인인가.

기술 사실이 들어가는 지점. 23장이 보인 대로 규칙은 참여자들의 조정으로 유지되고, 어느 체인이 「이기는가」를 정하는 것도 조정입니다. 22장이 보인 대로 거버넌스 토큰의 투표는 코드가 집계하고 결과는 자동으로 집행됩니다. 26장이 보인 대로 하드포크는 「규칙을 바꾸지 않고 상태를 바꾼」 일일 수 있고, 소수는 옛 체인에 남을 수 있습니다. 8장이 보인 대로 채굴자·노드·보유자의 집합은 일치하지 않습니다. 법이 「단체」·「구성원」·「대표」·「결의」라는 개념을 쓸 때, 기술은 겹치지 않는 여러 집합과 코드가 집계하는 투표를 줍니다.

1차 자료가 닿는 지점. 셋입니다. 첫째, SEC의 2017년 보고서는 The DAO를 「an unincorporated organization」이라 부르고, 증권법의 「issuer」 정의가 「person」에 「any unincorporated organization」을 포함한다는 조문을 들어 The DAO가 증권의 발행인이라고 했습니다.[20] 보고서는 또 The DAO 백서 자신이 「the legal status of [DAO Entities] remains the subject of active and vigorous debate and discussion」이라고 적었음을 각주에 옮겼습니다.[21] 22장 상자의 「아무것도 아닌가」에 대한 미국 증권법의 답은 「발행인이 되기에는 충분한 무엇」입니다. 둘째, 영국 항소법원의 2023년 *Tulip Trading* 판결은 비트코인 네트워크의 개발자들이 보유자에게 신인의무를 지는지가 「a serious issue to be tried」라고 했고, 「If the decentralised governance of bitcoin really is a myth, then … there is much to be said for the submission that bitcoin developers, while acting as developers, owe fiduciary duties to the true owners of that property」라고 적었습니다.[22] 23장 상자의 「개발자인가」에 대해 영국 법원은 「사실이 확정되면 그럴 수 있다」는 지점까지 갔습니다 — 본안의 결론은 아닙니다. 셋째, EIP-779는 The DAO 포크를 「did not change the protocol … Rather, the DAO Fork was an "irregular state change"」로 문서화했고, 재단은 옛 체인을 따르려는 사용자에게 「--oppose-dao-fork」 플래그를 안내했습니다.[23] 26장 상자의 「그 구별이 법적으로 의미가 있나」에 대해 기술 문서가 주는 것은 구별 자체이고, 의미는 법의 몫입니다. 한국법에서는 민법 제703조 제1항이 「조합은 2인 이상이 상호출자하여 공동사업을 경영할 것을 약정함으로써 그 효력이 생긴다」고 정하고, 27.2 ⑤의 조합권설이 그 조문에 네트워크를 끼워 읽으려 했으며, 「단체의 성립·가입·탈퇴를 위한 법령이나 의사표시가 없다」는 반론이 있습니다.[24] 29장의 질문에 닿는 1차 자료는 이 책의 원자료에 없습니다 — 29장 5절은 비트코인 쪽 논의의 세 갈래(그대로 둔다·소각한다·속도를 제한한다)와 한 백서가 열거한 정책 선택지를 소개만 하고, 이 절은 그 질문에 닿는 법령·판결이 없다는 사실을 적어 둡니다.

비어 있는 지점. 22장 상자의 「DAO는 법인인가 — 어느 법에 따라」에 대해, DAO에 법인격을 부여하는 입법이 있는 법역이 있는지를 이 책은 1차 자료로 확인하지 않았습니다. 23장 상자의 「계약서의 비트코인 1개는 어느 체인의 1개인가」에 대해서도 판결·조문을 찾지 않았습니다. 두 자리는 다음 판의 과제로 남기고, 유보로 보고합니다.

상자와 쟁점의 대응표

이 장과 27장이 받은 상자 스물세 개를 한 표에 모읍니다. 0부·6부의 상자(1·29·30장)도 각 장의 상자 문안으로 적습니다.

장	질문의 요지	받는 절
1	여러 원장 사본 중 「원본」은 무엇인가	28.1(f)
4	해시값 보관은 무엇을 증명하는가	28.1(f)
5	개인키 상실은 무엇의 상실인가	27.2
7	삭제권은 지울 수 없는 원장에 무엇을 요구하는가	28.1(d)
8	채굴자는 누구의 대리인인가	28.1(e)
9	지급은 언제 완료되는가	28.1(c) — 1차 자료 유보
10	스테이킹 보상은 무엇의 대가인가	28.1(b) — 과세 1차 자료 없음
11	착오 이체 코인은 누구 것인가	28.1(a)
13	스마트계약은 계약인가	28.1(c)
14	NFT 매수인은 무엇을 샀나	28.1(b)·27.3
15	AMM에 「운영자」가 있는가	27.3·28.1(e)
16	오라클 오류로 실행된 계약은 착오인가	28.1(c)·(e)
17	샌드위치는 시세조종인가	28.1(e)
19	법정통화와 「화폐」는 같은 말인가	27.2
20	발행 규칙 변경은 권리 침해인가	27.2·29장
21	소각된 수수료는 누구에게 귀속되나	27.2
22	거버넌스 다수결에 소수는 무엇을 주장하나	28.1(g)
23	하드포크 뒤 「진짜」 원장은 누가 정하나	27.2·28.1(g)
24	스테이블코인 보유자는 예금자인가 채권자인가	27.3·28.1(b)
25	버블 판정은 사기의 증거가 되나	28.1(e)
26	The DAO 포크는 구제인가 몰수인가	28.1(g)
29	취약 코인 지출 제한은 수용인가	27.2·28.1(g) — 1차 자료 없음
30	정의 조문이 기술을 못 따라갈 때	27.2

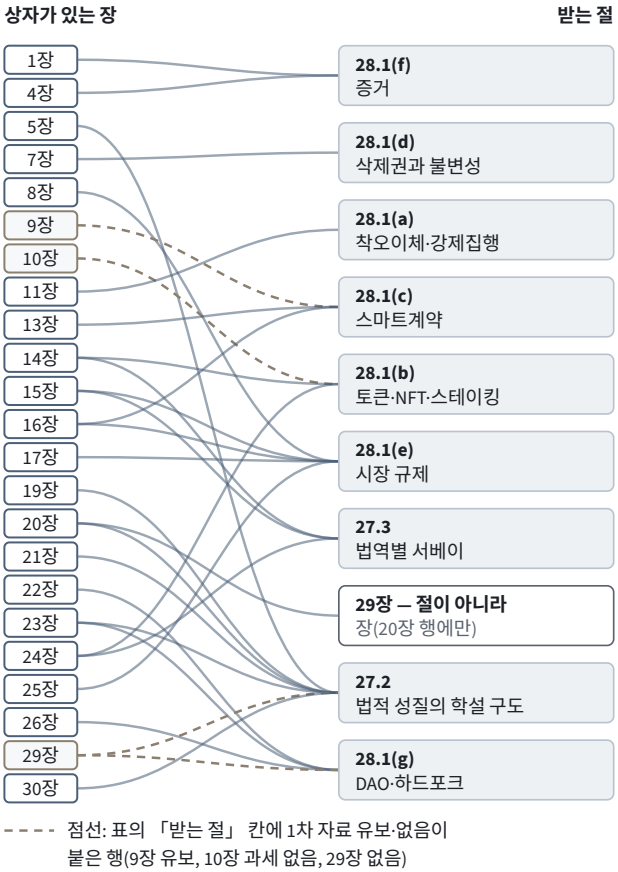


그림 28-3. 위 대응표를 선으로 옮긴 것. 왼쪽은 질문 상자가 있는 23개 장, 오른쪽은 그 상자를 받는 절이다. 점선은 표에 1차 자료 유보·없음이 붙은 행(9·10·29장)이고, 오른쪽 상자는 선이 덜 엇갈리도록 절 번호 순이 아닌 순서로 놓았다

28.2 더 읽을 곳

아래는 원자료 확인에서 서지가 확정되었고 이 책이 원문을 열어 본 1차 자료와 개설서입니다. 5부가 하지 않은 일 — 법리 해설 — 을 하려는 독자가 먼저 갈 곳입니다. 확정되지 않은 서지는 적지 않았고, 모자라는 법역은 비워 두었습니다.

국제·비교 - UNIDROIT, *Principles on Digital Assets and Private Law* (Rome, 2023). 19개 원칙과 해설. 「통제」를 귀속의 기준으로 세운 법역 중립적 설계. - FATF, *Updated Guidance for a Risk-Based Approach to Virtual Assets and Virtual Asset Service Providers* (Paris, 28 October 2021). 「virtual asset」·「VASP」의 국제 정의와 자금세탁방지 체계. 공표면은 2025년의 권고 1 개정 등을 반영하지 않았다고 경고한다. - Gary B. Gorton & Jeffery Y. Zhang, "Taming Wildcat Stablecoins," *University of Chicago Law Review* 90(3): 909–972 (2023). 스테이블코인을 은행사·화폐사의 맥락에 놓은 법학 논문.

한국 - 이혜정, 『가상자산에 대한 민사집행 연구』(사법정책연구원 연구총서 2022-12, 2022). 성질 논의의 국내외 정리(제2장 제3절)와 민사집행 입법안(제3장). - 대법원 2018. 5. 30. 선고 2018도3619 판결; 대법원 2021. 11. 11. 선고 2021도9855 판결(공2022상, 56); 대법원 2021. 12. 16. 선고 2020도9789 판결 — 국가법령정보센터 판례 전문. - 「가상자산 이용자 보호 등에 관한 법률」(법률 제19563호 제정, 현행 제21857호); 「특정 금융거래정보의 보고 및 이용 등에 관한 법률」 제2조 제3호·제7조; 소득세법 제21조 제1항 제27호(시행 2027. 1. 1.); 국세징수법 제55조·제66조; 지방세징수법 제61조 — 국가법령정보센터.

EU - Regulation (EU) 2023/1114 (MiCA), OJ L 150, 9.6.2023, p. 40. - CJEU, *Skatteverket v David Hedqvist*, C-264/14, ECLI:EU:C:2015:718 (22 October 2015). - Regulation (EU) 2016/679 (GDPR), OJ L 119, 4.5.2016, p. 1, Art. 17.

미국 - *SEC v. W. J. Howey Co.*, 328 U.S. 293 (1946). - SEC, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Release No. 81207 (July 25, 2017). - *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. 2018). - *SEC v. Ripple Labs, Inc.*, 682 F. Supp. 3d 308 (S.D.N.Y. 2023). - IRS Notice 2014-21, 2014-16 I.R.B. 938; Notice 2023-34, 2023-19 I.R.B. 837. - GENIUS Act, Pub. L. No. 119-27, 139 Stat. 419 (2025). - *Uniform Commercial Code Amendments (2022)*, Article 12 (ULC·ALI, 주석판). - 미국 로스쿨 강의계획서 한 편: Pepe, *Reading Group: Blockchain Law and Policy* (GW Law, Fall 2022) — 증권법·상품법·세법·FinCEN·FATF를 한 학기에 배열한 범위의 예.

일본 - 資金決済に関する法律(平成21年法律第59号) 第2条·第63条の2; 金融商品取引法(昭和23年法律第25号) 第2条第3項; 令和8年法律第64号 — e-Gov 法令検索. - 東京地方裁判所 平成31年3月15日 判決(平成27年(刑わ)第2274号 등); 東京高等裁判所 令和2年6月11日 判決(令和元年(う)第948号) — courts.go.jp(항소심), 판례 DB(1심).

영국 - Law Commission, *Digital Assets: Final Report* (Law Com No 412, HC 1486, 2023). - Property (Digital Assets etc) Act 2025 (c. 29). - The Financial Services and Markets Act 2000 (Cryptoassets) Regulations 2026, SI 2026/102. - *Tulip Trading Ltd v van der Laan and others* [2023] EWCA Civ 83.

한국의 단행본 — 국회도서관 소장 레코드로 서지만 확인한 것. 이 책은 아래 책들의 본문을 열지 않았고, 서지는 국회도서관 국가학술정보 레코드(2026-10-25 조회)의 제목·저자·출판사·연도대로 적습니다. 내용의 요약·평가는 하지 않습니다. - 안현수, 『가상자산법』(박영사, 2024). 국회도서관 제어번호 MONO12024000082483. - 김홍기, 『자본시장법 = 금융사지배구조법 | 금융소비자보호법 | 가상자산법』(박영사, 2024) — 서명이 가상자산법을 함께 적은 교과서. 제어번호 MONO12024000019710. - 이정엽·이석준·김성인, 『가상자산 판례백선. 민사·신청편』(박영사, 2023). 제어번호 MONO12023000016475. - 이정엽·이석준·김성인·장민석·한웅희, 『가상자산 판례백선. 형사·행정편』(박영사, 2024). 제어번호 MONO12024000019412.

비워 둔 자리. 일본·영국·EU의 개설서(교과서·주석서)는 OpenAlex·국회도서관 레코드로 서지를 확정하지 못해 적지 않았습니다(영국의 한 편저는 서평 레코드로 제목·편자만 확인되어 서지가 미완입니다). 결제 최종성 법제(28.1(c)), 스테이킹 과세(28.1(b)), DAO 법인격 입법(28.1(g))의 1차 자료도 없습니다. 다음 판에서 채울 자리입니다.

정리

일곱 쟁점을 지나며 같은 모양이 반복되었습니다. 상자가 던진 질문은 기술 사실에서 생겼고 — 원장은 원인을 모른다, 토큰은 권리를 담지 않는다, 완료의 순간이 없다, 지우는 데에 비용이 든다, 순서를 정하는 권한이 있다, 해시는 존재만 증명한다, 투표는 코드가 집계한다 — 1차 자료는 그 질문의 **한 지점에** 닿았습니다. 한국 대법원은 착오 이체의 수령자를 배임의 주체로 보지 않았고, SEC는 코드의 자동 실행이 행위자를 지우지 않는다고 했고, 영국 항소법원은 개발자의 신인의무를 심리할 쟁점으로 남겼으며, 다섯 법역의 입법은 사업자를, 그 가운데 넷은 스테이블코인 발행자를 정의했습니다. 어느 자료도 상자의 질문 전부에 답하지 않았고, 비어 있는 지점 — 결제 최종성, 스테이킹 과세, DAO 법인격, 포크 뒤 계약의 목적물 — 은 그렇다고 적었습니다. 법률가가 이 장에서 가져갈 것은 답이 아니라, 1~3부의 어느 장이 어느 쟁점의 **전제 사실인지를** 적은 이 대응표입니다. 다음 장은 그 쟁점들 전부에 걸려 있는 한 가지 — 같은 물건을 다섯 법역이 다섯 이름으로 부르고, 그 이름이 관할·과세·형사 적용을 가른다는 사실 — 를 정의 조문 원문으로 봅니다(30장).

이 장의 출처

- 대법원 2021. 12. 16. 선고 2020도9789 판결; 대법원 2021. 11. 11. 선고 2021도9855 판결; 대법원 2018. 5. 30. 선고 2018도3619 판결 — 국가법령정보센터 판례 전문(Open API, 2026-10-25 조회). 본문을 읽었고 판시사항·판결요지는 인용하지 않았다. 2010도891 판결은 2020도9789 판결의 인용으로만.
- 「가상자산 이용자 보호 등에 관한 법률」 제2·10·14조; 「특정 금융거래정보의 보고 및 이용 등에 관한 법률」 제7조; 소득세법 제21조 제1항 제27호(미시행); 국세징수법 제55조 제3항·제66조 제2항; 지방세징수법 제61조 제3항; 개인정보 보호법 제36조; 민사소송법 제357조·제358조; 전자서명법 제3조; 민법 제703조 — 국가법령정보센터(Open API, 2026-10-25 조회).
- 이혜정, 『가상자산에 대한 민사집행 연구』(사법정책연구원 2022) — xviii~xix·85·93·210~212쪽.
- SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017-07-25) — 1~4·11·13·15~16쪽.
- *SEC v. Ripple Labs, Inc.*, ECF No. 874 (S.D.N.Y. July 13, 2023) — 15·23·34쪽.
- *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. Mar. 6, 2018) — 3·24쪽.
- GENIUS Act, Pub. L. No. 119-27 (2025) — §2(7)·(22), §17, §20.
- IRS Notice 2014-21; Notice 2023-34. `irs-2023-notice-2023-34-modification-2014-21.pdf`.
- Regulation (EU) 2023/1114 (MiCA) — Art. 2(3), 48, 59, 86, 89, 91.
- Regulation (EU) 2016/679 (GDPR), OJ L 119, 4.5.2016, p. 1 — Art. 17(1)~(2). Cellar XHTML(2026-10-25 접근·축어 대조).
- 資金決済に関する法律 第2条第5項, 第63条の2 — e-Gov 法令 API v2(2026-10-25 조회).
- SI 2026/102 — RAO art. 88F·88G.
- *Tulip Trading Ltd v van der Laan and others* [2023] EWCA Civ 83 (3 February 2023) — ¶¶1·91.
- 東京高等裁判所 令和2年6月11日 判決(令和元年(う)第948号) — 10~11쪽.
- EIP-779; Ethereum Foundation Blog 2016-07-20 — 26장과 같은 자리.
- 28.2의 서지는 원자료 확인에서 서지가 확정된 항목과, 국회도서관 국가학술정보 API 레코드(2026-10-25 조회, 제어번호 기재)로 서지만 확인한 한국 단행본 4종이다. 단행본의 본문은 열지 않았다. 일본·

영국·EU 개설서·결제 최종성 법제·스테이킹 과세·DAO 법인격 입법은 확정 서지가 없어 비워 두었다.

- 이 장은 코드·노트북이 없다(설계). 가격·시가총액 자료는 없다. 계류 법안은 적지 않았다.

각주

1. 대법원 2021. 12. 16. 선고 2020도9789 판결(특정경제범죄가중처벌등에관한법률위반(횡령)[인정된죄명: 특정경제범죄가중처벌등에관한법률위반(배임)]; 원심 수원고등법원 2020. 7. 2. 선고 2020노171 판결, 파기환송; 대법관 노정희(재판장) 김재형(주심) 안철상 이흥구). 이유 1.(예비적 공소사실의 요지), 3. 가.·다.·라. 원문은 본문 인용과 같다. 참조판례: 대법원 2010. 12. 9. 선고 2010도891 판결(착오송금 횡령), 대법원 2020. 2. 20. 선고 2019도9756 전원합의체 판결(「타인의 사무를 처리하는 자」), 대법원 2021. 11. 11. 선고 2021도9855 판결. 국가법령정보센터 판례 전문(Open API, 판례 일련번호 219775, 2026-10-25 조회). 2010도891 판결은 이 책이 본문을 확인하지 않았으므로 「2020도9789 판결이 인용한 판결」로만 적는다. 환송 뒤의 경과는 확인하지 않았다.

2. 대법원 2021. 11. 11. 선고 2021도9855 판결, 이유 2. 가.; 대법원 2018. 5. 30. 선고 2018도3619 판결, 이유 1. 가. 3)·5)·6)·나. 2). 원문(2018도3619 나. 2)): 「피고인으로부터 임의제출받아 압수한 216.1249474비트코인(BTC) 중 191.32333418비트코인(BTC)은 범죄수익은닉규제법에 정한 중대범죄인 이 사건 음란사이트의 운영과 관련한 범죄행위에 의하여 취득한 것이므로, 범죄수익에 해당하여 이를 몰수한다.」 27장 각주 22·23 과 같은 자리.

3. 이혜정, 『가상자산에 대한 민사집행 연구』(사법정책연구원 2022), xviii~xix·210~212쪽. 원문: 「블록체인에 채무자의 가상자산이 압류되었다는 표시를 할 기술은 현재 그 구현이 불가능하고, 가상자산지갑은 가상자산 관련 정보를 관리하는 것으로서 … 가상자산지갑 자체에 압류표시를 하는 것이 압류 대상인 가상자산이 무엇인지를 명확히 한다고 보기 어려우므로」(211쪽); 「'가상자산을 유체동산'으로 보면 가상자산에 대한 압류 자체가 불가능 또는 무효인 결과로 연결될 수 있는 점」(212쪽). 입법안 요지는 27장 각주 31. 그 뒤 민사집행법 개정 여부는 미확인(부록 A).

4. 국세징수법(현행, 시행 2026. 10. 2.) 제55조 제3항·제66조 제2항 제2호; 지방세징수법(현행) 제61조 제3항. 원문은 27장 각주 30.

5. SEC, Release No. 81207 (2017-07-25), 11·13쪽. 원문: "In analyzing whether something is a security, 'form should be disregarded for substance,' Tcherepnin v. Knight, 389 U.S. 332, 336 (1967), 'and the emphasis should be on economic realities underlying a transaction, and not on the name appended thereto.' United Housing Found., 421 U.S. at 849."(11쪽); "Although DAO Token holders were afforded voting rights, these voting rights were limited. DAO Token holders were substantially reliant on the managerial efforts of Slock.it, its co-founders, and the Curators."(13쪽). 인용 안의 연방대법원 판례 둘은 SEC 보고서가 인용한 것이고 이 책이 원문을 확인한 것이 아니다.

6. *SEC v. Ripple Labs, Inc.*, No. 1:20-cv-10832 (AT) (S.D.N.Y. July 13, 2023), ECF No. 874, 682 F. Supp. 3d 308, 15·23·34쪽. 원문은 27장 각주 19.

7. GENIUS Act, Pub. L. No. 119-27 (July 18, 2025), §2(22)·§17 — 27장 각주 20. §17(a)(투자자문업법 §202(a)(18)에 추가되는 문장): "The term 'security' does not include a payment stablecoin issued by a permitted payment stablecoin issuer, as such terms are defined in section 2 of the GENIUS Act." 시행일(§20)은 미확정.

8. Regulation (EU) 2023/1114 Art. 2(3): "This Regulation does not apply to crypto-assets that are unique and not fungible with other crypto-assets."; SI 2026/102가 삽입한 RAO art. 88F(2)(a) "fungible"; 「가상자산 이용자 보호 등에 관한 법률」 제2조 제1호 아목과 같은 법 시행령(대통령령, 공포 2026. 9. 29., 시행 2026. 10. 2.) 제2조 제4호. 원문: 「4. 수집을 주된 목적으로 하는 전자적 증표, 거래 당사자 간의 거래 확인만을 목적으로 하는 전자적 증표 등 단일하게 존재하여 다른 전자적 증표로 대체할 수 없는 전자적 증표. 다만, 특정 재화나 서비스의 지급수단으로 사용될 수 있는 전자적 증표 등 금융위원회가 정하여 고시하는 전자적 증표는 제외한다.」(국가법령정보센터 Open API(MST 290171, 제2조), 2026-10-25 조회). 고시는 확인하지 않았다. 원문은 30장. 27장 각주 33·40.

9. MiCA Art. 48(1)(a); GENIUS Act §2(22)(A)(ii)(I); 資金決済法 第2条第5項 「電子機器その他の物に電子的方法により記録されている通貨建資産に限り」; RAO art. 88G(2)(b). 원문은 27장 각주 34·20·36·40.

10. IRS Notice 2014-21, §4 Q&A-1; Notice 2023-34 §3 — 27장 각주 14.

11. 소득세법 제21조 제1항 제27호(법률 제17757호 신설, 시행 2027. 1. 1.로 유예) — 27장 각주 29. 「2026-10 기준」 미시행.

12. SEC, Release No. 81207 (2017-07-25), 2쪽 본문과 각주 3. 원문: "The automation of certain functions through this technology, 'smart contracts,' or computer code, does not remove conduct from the purview of the U.S. federal securities laws."; 각주 3: "Computer scientist Nick Szabo described a 'smart contract' as: a computerized transaction protocol that executes terms of a contract. The general objectives of smart contract design are to satisfy common contractual conditions (such as payment terms, liens, confidentiality, and even enforcement), minimize exceptions both malicious and accidental, and minimize the need for trusted intermediaries." Szabo 1994년 글의 원문은 이 책이 확인하지 않았다.

13. 「개인정보 보호법」(현행, 공포 2026. 3. 10., 시행 2026. 9. 11.) 제36조 제1~3항. 원문: 「① 제35조에 따라 자신의 개인정보를 열람한 정보주체는 개인정보처리자에게 그 개인정보의 정정 또는 삭제를 요구할 수 있다. 다만, 다른 법령에서 그 개인정보가 수집 대상으로 명시되어 있는 경우에는 그 삭제를 요구할 수 없다. ② 개인정보처리자는 제1항에 따른 정보주체의 요구를 받았을 때에는 개인정보의 정정 또는 삭제에 관하여 다른 법령에 특별한 절차가 규정되어 있는 경우를 제외하고는 지체 없이 그 개인정보를 조사하여 정보주체의 요구에 따라 정정·삭제 등 필요한 조치를 한 후 그 결과를 정보주체에게 알려야 한다. ③ 개인정보처리자가 제2항에 따라 개인정보를 삭제할 때에는 복구 또는 재생되지 아니하도록 조치하여야 한다.」(국가법령정보센터 Open API(MST 283839, 제36조), 2026-10-25 조회). 같은 법의 2027년 시행 예정 개정 둘(법률 제21910호 시행 2027. 3. 9., 제21445호 시행 2027. 7. 1.)의 시행예정본에서도 제36조는 바뀌지 않는다(국가법령정보센터 시행예정본, 2026-10-26 조회).

14. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 (General Data Protection Regulation), OJ L 119, 4.5.2016, p. 1, Art. 17(1)~(2). 원문: "1. The data subject shall have the right to obtain from the controller the erasure of personal data concerning him or her without undue delay and the controller shall have the obligation to erase personal data without undue delay where one of the following grounds applies: (a) the personal data are no longer necessary ...; (b) the data subject withdraws consent ...; (c) the data subject objects ...; (d) the personal data have been unlawfully processed; ..."; "2. Where the controller has made the personal data public and is obliged pursuant to paragraph 1 to erase the personal data, the controller, taking account of available technology and the cost of implementation, shall take reasonable steps, including technical measures, to inform controllers which are processing the personal data that the data subject has requested the erasure by such controllers of any links to, or copy or replication of, those personal data." Cellar XHTML(`publications.europa.eu/resource/celex/32016R0679` , 2026-10-25 접근·축어 대조). 제3항의 예외(표현의 자유, 법적 의무, 공익 등)는 옮기지 않았다.

15. 「가상자산 이용자 보호 등에 관한 법률」(현행 법률 제21857호, 시행 2026. 10. 2.) 제10조 제1~6항. 원문(제3항·제4항 제1호·제6항): 「③ 누구든지 가상자산의 매매를 유인할 목적으로 가상자산의 매매가 성황을 이루고 있는 듯이 잘못 알게 하거나 그 시세를 변동 또는 고정시키는 매매 또는 그 위탁이나 수탁을 하는 행위를 하여서는 아니 된다.」; 「④ 누구든지 가상자산의 매매, 그 밖의 거래와 관련하여 다음 각 호의 행위를 하여서는 아니 된다. 1. 부정한 수단, 계획 또는 기교를 사용하는 행위」; 「⑥ 제1항부터 제5항까지를 위반한 자는 그 위반행위로 인하여 이용자가 그 가상자산의 매매, 그 밖의 거래와 관련하여 입은 손해를 배상할 책임이 있다.」(국가법령정보센터 Open API(MST 290735, 제10조), 2026-10-25 조회). 조사·조치는 제14조(금융위원회), 과징금은 제17조, 벌칙은 제19조.

16. Regulation (EU) 2023/1114, Title VI "PREVENTION AND PROHIBITION OF MARKET ABUSE INVOLVING CRYPTO-ASSETS", Art. 86(1)~(2), Art. 89(1), Art. 91(1), L 150/140~143. 원문: "1. This Title shall apply to acts carried out by any person concerning crypto-assets that are admitted to trading or in respect of which a request for admission to trading has been made. 2. This Title shall also apply to any transaction, order or behaviour concerning crypto-assets as referred to in paragraph 1, irrespective of whether such transaction, order or behaviour takes place on a trading platform."(Art. 86); "For the purposes of this Regulation, insider dealing shall be deemed to arise where a person possesses inside information and uses that information by acquiring or disposing of, for its own account or for the account of a third party, directly or indirectly, crypto-assets to which that information relates."(Art. 89(1)); "No person shall engage in or attempt to engage in market manipulation."(Art. 91(1)).

17. *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. Mar. 6, 2018), 3·24쪽 — 27장 각주 21. 「(2) whether the amendments to the CEA under the Dodd-Frank Act permit the CFTC to exercise its jurisdiction over fraud that does not directly involve the sale of futures or derivative contracts. Both questions are answered in the affirmative.」(3쪽).

18. 민사소송법(현행, 시행 2026. 10. 2.) 제357조·제358조; 전자서명법(현행, 시행 2022. 10. 20.) 제3조. 원문: 「제357조(사문서의 진정의 증명) 사문서는 그것이 진정한 것임을 증명하여야 한다.」; 「제358조(사문서의 진정의 추정) 사문서는 본인 또는 대리인의 서명이나 날인 또는 무인(拇印)이 있는 때에는 진정한 것으로 추정한다.」; 「제3조(전자서명의 효력) ① 전자서명은 전자적 형태라는 이유만으로 서명, 서명날인 또는 기명날인으로서의 효력이 부인되지

아니한다. ② 법령의 규정 또는 당사자 간의 약정에 따라 서명, 서명날인 또는 기명날인의 방식으로 전자서명을 선택한 경우 그 전자서명은 서명, 서명날인 또는 기명날인으로서의 효력을 가진다.」(Open API, 2026-10-25 조회). 전자문서의 증거능력·증거조사 방법에 관한 그 밖의 조문은 이 책이 확인하지 않았다.

19. 東京高等裁判所第４刑事部 令和2年6月11日 宣告 判決(令和元年(う)第948号), 10~11쪽(第３の２(３)). 원문: 「電磁的記録には，以下のような文書偽造罪等における文書とは異なる特質がある。すなわち，
作成過程については，可視性，可読性がないだけでなく，入力されたデータが既存のデータとともに加工，処理され新たに作り出されることが少なくなく，その作出に複数の者の意思や行為が関わることがある。また，
利用過程については，
それを使用することが予定されている一定のシステムやプログラムのもとで用いられて初めて本来の証明機能を果たし得るものである。そうであると，電磁的記録の証明機能は，
それが作出されるべきシステムの設置運営主体の意思に基づき，
正当に作出されることが当然の前提とされているもので，
このように正当に作られたものであることの信頼を確保する必要があるという観点から解釈すべきと解される。」 26장 각주 15 과 같은 자리.

20. SEC, Release No. 81207 (2017-07-25), 1·15~16쪽. 원문: "The United States Securities and Exchange Commission's ... Division of Enforcement ... has investigated whether The DAO, an unincorporated organization; Slock.it UG ('Slock.it'), a German corporation; Slock.it's co-founders; and intermediaries may have violated the federal securities laws."(1쪽); "The definition of 'issuer' is broadly defined to include 'every person who issues or proposes to issue any security' and 'person' includes 'any unincorporated organization.' 15 U.S.C. § 77b(a)(4)." (15쪽); "The DAO, an unincorporated organization, was an issuer of securities, and information about The DAO was 'crucial' to the DAO Token holders' investment decision."(16쪽).

21. SEC (2017), 3~4쪽 각주 10. 원문: "The White Paper contained the following statement: A word of caution, at the outset: the legal status of [DAO Entities] remains the subject of active and vigorous debate and discussion. Not everyone shares the same definition. Some have said that [DAO Entities] are autonomous code and can operate independently of legal systems; others have said that [DAO Entities] must be owned or operate[d] by humans or human created entities. ... This paper does not speculate about the legal status of [DAO Entities] worldwide." 22장 질문 상자가 인용한 자리와 같다.

22. *Tulip Trading Ltd v van der Laan and others* [2023] EWCA Civ 83 (3 February 2023), ¶¶1·91 — 27장 각주 41.

23. EIP-779 Abstract; Vitalik Buterin, "Hard Fork Completed" (2016-07-20) — 26장 각주 23·22.

24. 민법 제703조 제1항(Open API, 2026-10-25 조회); 이혜정(2022), 85·93쪽 — 27장 각주 6.

6부 열린 문제

29장 양자 전환 — 서명은 깨지고 해시는 버틴다, 그리고 취약 코인

핵심 TAKEAWAY

양자 컴퓨터가 위협하는 것은 「비트코인」이 아니라 **특정한 서명 방식입니다**. 쇼어(Shor)의 알고리듬은 타원곡선 위의 이산로그를 다항 시간에 풀어 공개키에서 개인키를 뽑아내고, 그로버(Grover)의 알고리듬은 해시 함수의 역상 탐색을 제곱근만큼만 앞당깁니다. 그래서 5장의 서명은 깨지고 4·8장의 해시와 작업증명은 버팁니다. 공격에 필요한 기계의 크기는 2017년의 2,330 논리 큐비트에서 2026년의 1,200 논리 큐비트 안팎으로 내려왔고, 물리 큐비트 추정은 가정에 따라 수십만에서 십억 단위까지 갈리고, 한 논문 안에서도 목표 시간 하나로 1,300만에서 19억까지 움직입니다 — 추정치는 관측이 아니라 가정의 함수입니다. 위험에 놓인 것은 **공개키가 이미 드러난 코인이고**, 기준일 스냅샷의 2차 집계는 그 범위를 발행량의 약 3분의 1로 읽습니다. 비트코인 쪽 전환 제안(BIP-360·361)은 2026-10 기준 Draft이고 활성화된 양자내성 요소는 없으며, 이더리움 공식 페이지는 2029년을 「바뀔 수 있는 계획 목표」로 적은 로드맵을 공개했습니다. 법률가가 가져갈 것은 기술이 아니라 **마지막 질문입니다** — 취약 코인의 지출을 소프트포크로 막는 일은 수용인가, 질서유지인가, 무주물 처리인가, 그리고 그것을 결정하는 다수는 누구의 대리인인가. 이 장은 그 질문을 던지고 답하지 않습니다.

학습목표

1. 쇼어의 알고리듬이 서명(ECDLP)을, 그로버의 알고리듬이 해시를 각각 어떻게 위협하는지와 그 비대칭(서명은 깨지고 해시는 버틴다)을 설명한다.
2. [중급] 논리 큐비트와 물리 큐비트를 구별하고, 자원 추정치가 어떤 가정(오류율·코드 사이클·반응 시간·연결성·목표 시간)에 따라 수백 배 움직이는지 표로 읽는다.
3. 공개키가 체인에 드러나는 세 자리(스크립트 유형·재사용·멤풀)를 가리고, 기준일 스냅샷의 유형별 수량을 2차 집계라는 단서와 함께 읽는다.
4. 전환 제안(BIP-360·BIP-361, NIST FIPS 203~205, 이더리움의 네 영역)이 무엇을 바꾸고 무엇을 바꾸지 않는지 안다.
5. 「취약 코인 지출 제한」 논쟁의 세 갈래(그대로 둔다·소각한다·속도를 제한한다)를 구조로 적고, 그것이 어느 법적 질문으로 이어지는지 5부의 자리로 넘긴다.

[입문]**만 읽은 독자가 가져가는 것.** 양자컴퓨터가 깨는 것은 「비트코인」이 아니라 **특정 서명 방식이고**, 진짜 어려운 문제는 기술이 아니라 **누가 어떤 권한으로 전환을 결정하고 남은 코인을 어떻게 할 것인가입니다**. 부록 B 오해 ⑤(「양자컴퓨터가 나오면 비트코인은 끝난다」)의 근거가 이 장에 있습니다.

1. 두 알고리듬, 두 표적 — 비대칭

1부가 쓴 암호 도구는 둘이었습니다. 4장의 해시 함수와 5장의 공개키 서명입니다. 양자 컴퓨터는 이 둘을 **다르게** 건드립니다. 미국 표준기술연구소(NIST)의 2016년 보고서는 그 차이를 한 표로 적었습니다 — 대칭키 암호(AES)는 「더 긴 키가 필요」, 해시 함수(SHA-2·SHA-3)는 「더 긴 출력이 필요」, 그리고 RSA·타원곡선 서명(ECDSA)·타원곡선 키 교환(ECDH)·유한체 서명(DSA)은 「더 이상 안전하지 않음」.[1] 비트코인의 서명은 secp256k1 곡선 위의 ECDSA이고(5장), 2021년부터 같은 곡선 위의 슈노르 서명도 씁니다. 둘 다 세 번째 줄에 속합니다. 비트코인의 작업증명은 SHA-256을 두 번 돌리는 것이고(8장), 그것은 두 번째 줄에 속합니다.

왜 다를까요. 보고서는 두 알고리듬의 이름을 적습니다. 1994년 벨 연구소의 피터 쇼어(Peter Shor)가 양자 컴퓨터로 소인수분해와 이산로그 문제를 **효율적으로** 풀 수 있음을 보였고, 그래서 「그런 가정에 기댄 모든 공개키 암호체계를 무력하게」 만든다는 것입니다.[2] 반면 그로버(Lov Grover)의 탐색 알고리듬이 주는 것은 「구조 없는 탐색 문제에서 제곱근만큼의 가속」이고, 보고서는 「그런 가속은 암호 기술을 쓸모없게 만들지 않고, 대칭키의 경우에도 더 긴 키를 요구하는 효과를 가질 뿐」이라고 적습니다.[3] 5장에서 서명의 안전은 「$[k]G$ 에서 k 를 찾는 빠른 방법이 알려져 있지 않다」는 데 걸려 있었습니다. 고전 컴퓨터에서는 그렇습니다. 양자 컴퓨터에서는 쇼어의 알고리듬이 바로 그 k 를 찾습니다. 4장에서 해시의 안전은 「역상을 찾는 데 기대 2^{256} 번의 시도가 든다」는 데 걸려 있었고, 그로버의 알고리듬은 그것을 2^{128} 번 안팎으로 줄입니다 — 여전히 천문학적인 수이고, 앞의 표가 적은 대로 「더 긴 출력」으로 되돌릴 수 있는 종류의 가속입니다.



그림 29-1. 쇼어의 알고리듬과 그로버의 알고리듬이 비트코인의 서명과 해시를 건드리는 두 갈래. 왼쪽은 구조가 있는 이산로그 문제가 다항 시간으로 떨어져 서명의 문제 자체를 바꿔야 하는 길, 오른쪽은 구조 없는 탐색에 제곱근 가속만 생겨 출력 길이로 되살리는 길이다. 따옴표 문구는 NIST 2016 보고서 표

비트코인의 양자 위협을 정면으로 다룬, 이 장이 연 문헌 가운데 가장 이른 동료심사 논문이 이 비대칭을 숫자로 적었습니다. 아가왈(Divesh Aggarwal) 등 다섯 저자의 2018년 논문입니다. 초록의 결론은 두 문장입니다. 「우리는 비트코인이 쓰는 작업증명이 향후 10년 동안 양자 컴퓨터에 의한 상당한 가속에

비교적 저항적이라는 것을 발견했는데, 주된 이유는 특화된 ASIC 채굴기가 가까운 장래의 양자 컴퓨터에 대해 추정되는 클록 속도에 비해 극도로 빠르기 때문이다. 반면 비트코인이 쓰는 타원곡선 서명 방식은 훨씬 더 위험하며, 가장 낙관적인 추정으로는 이르면 2027년에 양자 컴퓨터에 의해 완전히 깨질 수 있다.」[4] 뒷문장의 연도는 2017년 시점의 「가장 낙관적인」 외삽이고, 이 책은 그 외삽에 가담하지 않습니다(2절). 앞문장의 구조는 그 뒤 문헌이 되풀이 확인했습니다. 2026년 구글 양자 AI 팀 등의 백서는 그로버 기반 채굴 공격이 「다음 수십 년 동안 걱정할 것이 아니다」라고 적고, 그 이유 둘을 듭니다 — 제곱근 가속은 양자 오류정정의 비용에 거의 다 잡아먹히고, 그로버의 알고리듬은 병렬화가 잘 되지 않는데 채굴은 하드웨어 가속과 대규모 병렬화의 세계라는 것입니다.[5]

[중급] 숫자로 보면 이렇습니다. 아가왈 외는 그로버 채굴기에 필요한 논리 큐비트 수가 난이도와 무관하게 2,402개로 고정된다고 계산하고, 초전도 회로의 당시 최대 게이트 속도 66.7 MHz와 물리 게이트 오류율 5×10^{-4}, 난이도 10^{12} 를 가정하면 양자 채굴기 한 대의 유효 해시율이 13.8 GH/s(물리 큐비트 약 440만 개)라고 적습니다 — 「기성품 ASIC 장치가 내는 14 TH/s보다 천 배 넘게 느리다」는 것입니다.[6] 2026년 백서는 더 극단적인 가정으로 같은 결론을 냅니다. 「양자 컴퓨터가 SHA-256을 1마이크로초 오류정정 사이클 한 번에 계산한다는 환상적인 가정」 아래에서도 양자 채굴기의 해시율은 0.25 TH/s로, 흔한 ASIC 채굴기 S19 Pro의 110 TH/s보다 두 자릿수 아래라고 적습니다.[7] 8장이 보인 대로 작업증명의 안전은 절대 속도가 아니라 **점유율에** 걸려 있고, 양자 채굴기가 점유율을 뒤집을 만큼 빠르다는 추정은 이 장이 연 문헌 어디에도 없습니다.

▾ [심화] **왜 비대칭인가 — 구조가 있는 문제와 없는 문제**

[심화] 5장 심화 상자에서 타원곡선 위의 점 G 와 $Q = [k]G$ 가 주어졌을 때 k 를 찾는 문제(ECDLP)를 보았습니다. 이 문제에는 **대수적 구조가** 있습니다. 점들은 군(群)을 이루고, 스칼라 곱은 그 군 위의 주기적인 함수입니다. 쇼어의 알고리듬은 바로 그 주기성을 양자 푸리에 변환으로 읽어 냅니다. 뢰텔러(Martin Roetteler) 외 마이크로소프트 연구진의 2017년 논문은 이렇게 요약합니다. 쇼어는 「다항 시간 양자 알고리듬 둘 — 하나는 정수 소인수분해, 다른 하나는 소수 위수 유한체의 이산로그 계산 — 을 제시했다. 쇼어는 후자가 다른 체로 일반화된다고 적는다. 그것은 타원곡선의 경우로도 일반화된다.」[8] 반면 해시 함수의 역상 찾기에는 그런 구조가 없습니다. 입력을 하나씩 넣어 보는 것 말고 지름길이 없는 「구조 없는 탐색」이고, 그로버의 알고리듬이 거기서 얻는 이득은 N 을 $\sqrt{N}$ 으로 줄이는 것이 전부이며, NIST 보고서가 인용한 1997년 결과대로 탐색 문제에 지수적 가속은 **불가능**합니다(보고서 참고문헌 [3], Bennett·Bernstein·Brassard·Vazirani 1997). 그래서 같은 「양자」라는 말 아래에서 하나는 다항 시간으로 떨어지고 다른 하나는 제곱근만 줄어듭니다. 4장의 세 성질(역상·제2역상·충돌 저항)은 출력 길이를 늘리는 것으로 되살릴 수 있지만, 5장의 서명은 **문제 자체를 바꿔야** 합니다 — 그것이 4절의 「양자내성 서명」입니다.

[심화] 한 가지를 덧붙입니다. 아가왈 외는 해시 기반 작업증명 가운데서도 **충돌 찾기를** 요구하는 변형(Momentum)이 그로버보다 더 적은 양자 이득만 허용함을 보입니다. 충돌 찾기의 양자 하한이 $|S|^{2/3}$ 이라는 결과(Aaronson–Shi 2004)를 써서, 최적 고전 시간 T 에 대해 양자 시간이 적어도 $T^{2/3}$ 임을 논증합니다 — 「현재 비트코인의 작업증명과 달리, 이 제안에서는 S 가 상수 크기를 넘는 순간 양자 컴퓨터가 제곱근 이득을 얻지 못한다」는 것입니다.[9] 이 책은 그 제안을 평가하지 않고, 「해시 쪽은 설계로 더 버티게 만들 여지가 있다」는 사실만 적어 둡니다.

2. 자원 추정 — 큐비트 둘, 가정 다섯

「양자 컴퓨터가 서명을 깬다」는 명제는 참입니다. 「그런 양자 컴퓨터가 언제 만들어지는가」는 다른 질문이고, 이 책은 그 질문에 답하지 않습니다(3장 일러두기 — 전망을 쓰지 않는다). 대신 그 질문을 **어떻게 세는지를** 보입니다. 두 단위를 먼저 갈라야 합니다. **논리 큐비트는** 오류가 정정된, 알고리듬이 실제로 쓰는 추상적 큐비트이고, **물리 큐비트는** 하드웨어의 실제 소자입니다. 논리 큐비트 하나를 만들려면 물리 큐비트 수백에서 수천 개가 들고, 그 배수는 소자의 오류율·오류정정 부호·목표 실행 시간에 따라 바뀝니다. 그래서 「논리 1,200개」와 「물리 50만 개」는 같은 기계를 두 단위로 적은 것이고, 논리 수는 알고리듬의 개선으로, 물리 수는 거기에 더해 하드웨어 가정으로 움직입니다.

[중급] 256비트 소수체 타원곡선의 ECDLP를 쇼어의 알고리듬으로 푸는 데 필요한 자원의 추정은 아래처럼 내려왔습니다. 이 책이 원문을 연 네 편만 적습니다.

연도	논문	대상 곡선	논리 큐비트	비클리퍼드 게이트(TOFFOLI)	물리 큐비트 · 시간	가정
2017	Roetteler 외	NIST P-256	2,330	약 1.26×10^{11}	(논리 수준만)	일반식 $9n + 2\lceil \log_2 n \rceil + 10$ 큐비트, $448n^3 \log_2 n + 4090n^3$ 게이트
2018	Aggarwal 외	비트코인 서명 (Roetteler 입력)	2,334(보조 4 포함)	1.28×10^{11}	1.7×10^6 · 6.49일 / 485,550 · 30분	표면 부호; 오류 5×10^{-4} ·66.6 MHz / 오류 10^{-5}·10 GHz
2022	Webber 외	「비트코인 네트워크의 256비트 타원곡선」(선행 회로 입력)	2,871	5.76×10^9 (T)	1,300만 · 1일 / 3억 1,700만 · 1시간 / 19억 · 10분	표면 부호; 오류 10^{-3}·코드 사이클 1 μs·반응 10 μs
2026	Babbush 외	secp256k1	≤1,200 / ≤1,450	≤9,000만 / ≤7,000만	50만 미만 · 18~23분 (준비 상태에서 9~12분)	표면 부호(yoke 저장); 오류 10^{-3}·평면 연결·반응 10 μs

[중급] 첫 행의 일반식은 뢰텔러 외가 회로를 **실제로 시뮬레이션해** 얻은 상한입니다. 「n 비트 소수체 위의 타원곡선 이산로그는 최대 $9n + 2\lceil \log_2(n) \rceil + 10$ 큐비트와 최대 $448n^3 \log_2(n) + 4090n^3$ 토폴리 게이트로 계산할 수 있다」, 그리고 P-256에서 「2330 논리 큐비트와 약 $1.26 \cdot 10^{11}$ 토폴리 게이트」.[10] 아가왈 외는 이 회로를 입력으로 받아 비트코인 서명에 대입했고, 두 가지 하드웨어 가정으로 물리 큐비트와 시간을 냈습니다. 「표면 부호와 물리 게이트 오류율 $p_g = 5 \times 10^{-4}$ 에서 … 66.6 MHz 클록으로 문제를 푸는 시간은 1.7×10^6 물리 큐비트를 써서 6.49일이다. 성능 개선을 내다보면, 10 GHz 클록과 10^{-5} 오류율에서 서명은 485550 큐비트로 30분 만에 깨진다.」[11] 웨버(Mark Webber) 외는 다른 선행 회로(2,871 논리 큐비트, T 게이트 5.76×10^9)를 입력으로 받아, 같은 하드웨어 가정 안에서 **목표 시간만 바꾸면** 물리 큐비트 수가 얼마나 움직이는지를 보였습니다 — 하루면 1,300만, 한 시간이면 3억 1,700만, 10분이면 19억.[12] 바부시(Ryan Babbush) 외는 회로를 공개하지 않고 영지식 증명으로 자원 수만 검증 가능하게 공개했는데, 「≤1200 논리 큐비트와 ≤9천만 토폴리 게이트, 또는 ≤1450 논리 큐비트와 ≤7천만 토폴리 게이트」이고, 초전도 아키텍처에서 「50만 개 미만의 물리 큐비트로 수 분 안에」 실행된다고 적습니다.[13]

[중급] 아래 코드는 첫 행의 일반식에 $n = 256$ 을 넣어 보고, 네 편의 논리 자원을 한 표에 놓습니다. 물리 큐비트는 계산하지 않습니다 — 그것은 식 하나로 나오는 수가 아니라 가정 다섯 개의 함수이기 때문입니다.

```
# NB29-1: 쇼어 알고리듬의 ECDLP 자원 — 뢰텔러 외(2017) 일반식에 n=256 대입, 네 편의 논리 자원 비교 (공부용)
import math

def roetteler_bound(n):
    """Roetteler et al. (2017) 초록의 상한식 — 논리 큐비트와 Toffoli 게이트 수."""
    qubits = 9 * n + 2 * math.ceil(math.log2(n)) + 10
    toffoli = 448 * n**3 * math.log2(n) + 4090 * n**3
    return qubits, toffoli

for n in (192, 224, 256, 384, 521):
    q, t = roetteler_bound(n)
    print(f"n={n:>3}비트: 논리 큐비트 상한 {q:>5},  Toffoli 상한 {t:.3e}")

q256, t256 = roetteler_bound(256)
print()
print("n=256 — 식의 상한 Toffoli", f"{t256:.4e}", "| 논문이 시뮬레이션으로 보고한 값 약 1.26e11 (식은
상한이라 조금 크다)")
print("아가왈 외(2018)가 대입한 값 1.28e11 은 식의 상한을 그대로 쓴 것 →", f"{t256/1.28e11:.3f}", "배")

papers = [  # (연도, 논문, 논리 큐비트, Toffoli 또는 T 게이트 수)
    (2017, "Roetteler 외 (P-256)",            2330, 1.26e11),
    (2018, "Aggarwal 외 (보조 4 포함)",       2334, 1.28e11),
    (2022, "Webber 외 입력 회로 (T 게이트)",  2871, 5.76e9),
    (2026, "Babbush 외 — 저큐비트 회로",      1200, 9.0e7),
    (2026, "Babbush 외 — 저게이트 회로",      1450, 7.0e7),
]
print()
print(f"{'연도':<6}{'논문':<34}{'논리 큐비트':>10}{'게이트 수':>12}{'2017 대비 게이트':>16}")
for y, name, lq, gates in papers:
    ratio = "단위 다름(T)" if "T 게이트" in name else f"{1.26e11/gates:.0f}배"
    print(f"{y:<6}{name:<34}{lq:>10,}{gates:>12.2e}{ratio:>16}")
print()
print("물리 큐비트는 여기서 계산하지 않는다 — 오류율·코드 사이클·반응 시간·연결성·목표 시간의 함수다(본문 2절
심화).")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
n=192비트: 논리 큐비트 상한  1754,  Toffoli 상한 5.300e+10
n=224비트: 논리 큐비트 상한  2042,  Toffoli 상한 8.528e+10
n=256비트: 논리 큐비트 상한  2330,  Toffoli 상한 1.287e+11
n=384비트: 논리 큐비트 상한  3484,  Toffoli 상한 4.494e+11
n=521비트: 논리 큐비트 상한  4719,  Toffoli 상한 1.150e+12

n=256 — 식의 상한 Toffoli 1.2875e+11 | 논문이 시뮬레이션으로 보고한 값 약 1.26e11 (식은 상한이라 조금 크다)
아가왈 외(2018)가 대입한 값 1.28e11 은 식의 상한을 그대로 쓴 것 → 1.006 배

연도    논문                                  논리 큐비트        게이트 수      2017 대비 게이트
2017  Roetteler 외 (P-256)                  2,330    1.26e+11              1배
2018  Aggarwal 외 (보조 4 포함)               2,334    1.28e+11              1배
2022  Webber 외 입력 회로 (T 게이트)           2,871    5.76e+09        단위 다름(T)
2026  Babbush 외 — 저큐비트 회로              1,200    9.00e+07           1400배
2026  Babbush 외 — 저게이트 회로              1,450    7.00e+07           1800배

물리 큐비트는 여기서 계산하지 않는다 — 오류율·코드 사이클·반응 시간·연결성·목표 시간의 함수다(본문 2절 심화).
```

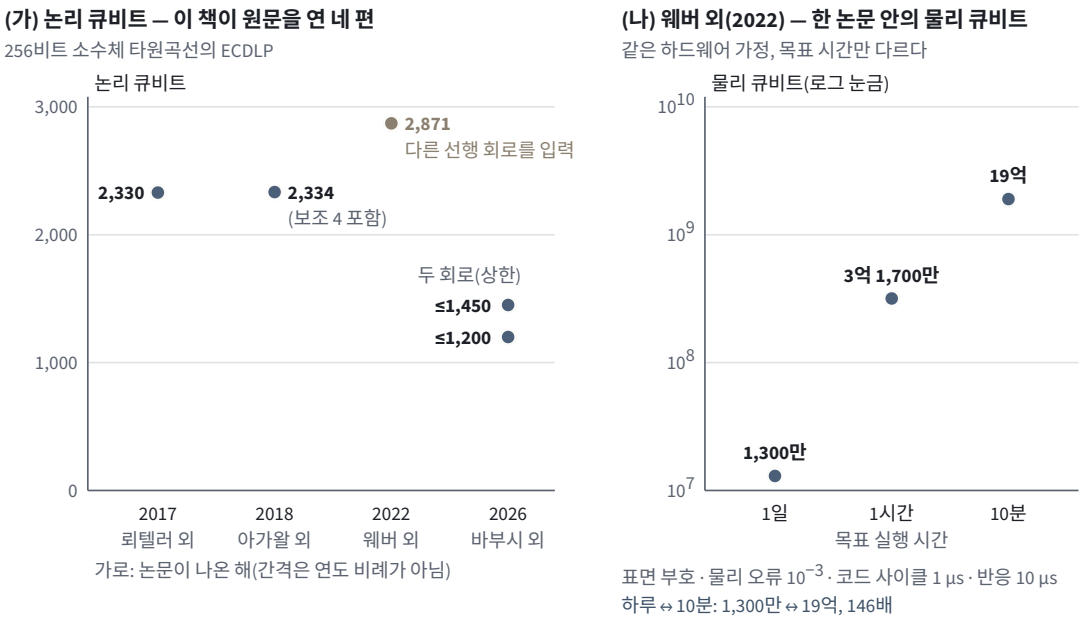


그림 29-2. 위 표의 자원 추정을 두 면에 점으로만 옮겼다. (가)는 네 편의 논리 큐비트이고 2022년 값은 다른 선행 회로를 입력으로 한 것이다. (나)는 웨버 외 한 논문 안에서 하드웨어 가정을 고정하고 목표 시간만 바꾼 물리 큐비트(로그 눈금)다. 가정이 달라 점을 잇지 않았고 추세선·연장선을 긋지 않았다

표에서 읽을 것은 **방향이지** 수치가 아닙니다. 논리 큐비트는 9년 사이 거의 절반으로(2,330 → 1,200 안팎), 게이트 수는 천 배 넘게 줄었습니다. 그것은 하드웨어가 아니라 **알고리듬과 오류정정 이론이** 움직인 결과이고, 바부시 외는 그래서 「물리 큐비트 수를 세는 단순한 모형은 기술 진보를 제대로 잡아내지 못한다」며 임계값 모형을 권합니다 — 진보는 연속이 아니라 「새 내부 능력의 개발과 규모 확장 난관의 극복에 대응하는 불연속적 도약」으로 온다는 것입니다.[14] 이 책이 시점 예측을 쓰지 않는 이유가 여기에도 있습니다. 2017년 논문의 「이르면 2027년」은 그 시점의 하드웨어 추세 외삽이었고, 2026년 백서는 바로 그런 외삽 방식 자체에 회의를 적습니다. 전문가 설문의 수치(2026년 3월 공표, 26명)는 본문이 아니라 부록 A에 적어 두었습니다 — 의견은 관측이 아니기 때문입니다.

▾ [심화] 가정 다섯 — 같은 가정끼리만 비교할 수 있다

[심화] 물리 큐비트 추정이 수백 배 갈리는 이유를 가정별로 나눕니다. (i) **물리 오류율** — 아가왈 외의 두 시나리오는 5×10^{-4} 와 10^{-5} 로 50배 다르고, 두 시나리오는 클록도 66.6 MHz와 10 GHz로 150배 다르므로, 오류율과 클록을 함께 바꾼 차이가 물리 큐비트를 170만에서 48만으로, 시간을 6.49일에서 30분으로 바꿉니다 — 시간 차이의 대부분은 클록에서 옵니다. (ii) **코드 사이클과 반응 시간** — 웨버 외의 결과는 사이클 1 µs·반응 10 µs 가정이고, 느린 플랫폼(이온 트랩 등)은 사이클이 수백 µs라 같은 논리 회로를 돌리는 데 훨씬 오래 걸립니다. 바부시 외는 그래서 「빠른 클록」 아키텍처(초전도·광자)와 「느린 클록」 아키텍처(중성 원자·이온 트랩)를 갈라 두 시나리오를 둡니다 — 전자가 먼저 오면 지출 중인 거래까지 공격 가능한 「on-spend」 공격이 함께 오고, 후자가 먼저 오면 오래 노출된 공개키만 공격 가능한 「at-rest」 공격이 먼저 온다는 것입니다.[15] (iii) **연결성** — 바부시 외는 평면 격자의 차수 4 연결을 가정하고, 더 공격적인 연결을 가정한 다른 부호들은 「10만 개에 가까운」 수를 내지만, 차수 7 부호는 「아직 실제 초전도 소자에서 시연되지 않았다」고, 차수 10의 Pinnacle 구조(RSA-2048 기준)는 더 까다로운 소자 사양에 「한 달 가까운 실행 시간」이 든다고 따로 적습니다.[16] (iv) **목표 실행 시간** — 웨버 외의 한

논문 안에서 하루·한 시간·10분은 1,300만·3억 1,700만·19억으로, 146배입니다. 비트코인에서 목표 시간이 중요한 이유는 다음 절에서 봅니다 — 지출 중인 거래의 공개키는 블록에 들어가기까지의 **짧은 창에만** 노출되기 때문입니다. (v) **논리 회로 자체** — 2,330에서 1,200으로의 감소. 그래서 「웨버 3억 1,700만 ↔ 바부시 50만 미만」은 600배 차이지만, 둘은 입력 회로·오류정정 저장 방식·목표 시간이 모두 다릅니다. 같은 가정끼리만 비교할 수 있고, 어느 수치도 참값이 아닙니다.

3. 무엇이 노출되어 있나 — 공개키의 세 자리

쇼어의 알고리듬이 필요로 하는 입력은 **공개키입니다**. 공개키가 보이지 않으면 뽑아낼 개인키도 없습니다. 5장에서 비트코인 「주소」가 공개키 그 자체가 아니라 그 해시라는 것, 그래서 해시형 출력(P2PKH·P2WPKH)으로 돈을 받기만 한 주소는 공개키를 아직 드러내지 않는다는 것, 그리고 12장의 제네시스 출력(P2PK)과 탭루트 출력(P2TR)은 공개키를 그대로 담는다는 것을 보았습니다. 그 구별이 이 장의 중심입니다. 공개키가 드러나는 자리는 셋입니다.

1. **스크립트 자체에 키가 있는 출력.** P2PK(초기 채굴 보상, 제네시스 포함), 맨 다중서명(P2MS), 그리고 2021년의 P2TR. 받는 순간부터 노출됩니다.
2. **재사용된 해시형 주소.** P2PKH·P2WPKH 주소에서 한 번이라도 지출하면 그 입력에 공개키가 공개되고, 같은 주소에 다시 돈을 받아 두면 그 잔고는 노출된 키로 잠겨 있는 셈입니다. P2SH·P2WSH는 스크립트가 공개됩니다.
3. **멤풀.** 어느 유형이든 지출하는 순간 공개키가 네트워크에 뿌려지고, 블록에 들어가 확인이 쌓일 때까지의 몇 분에서 몇 시간이 노출 창입니다.

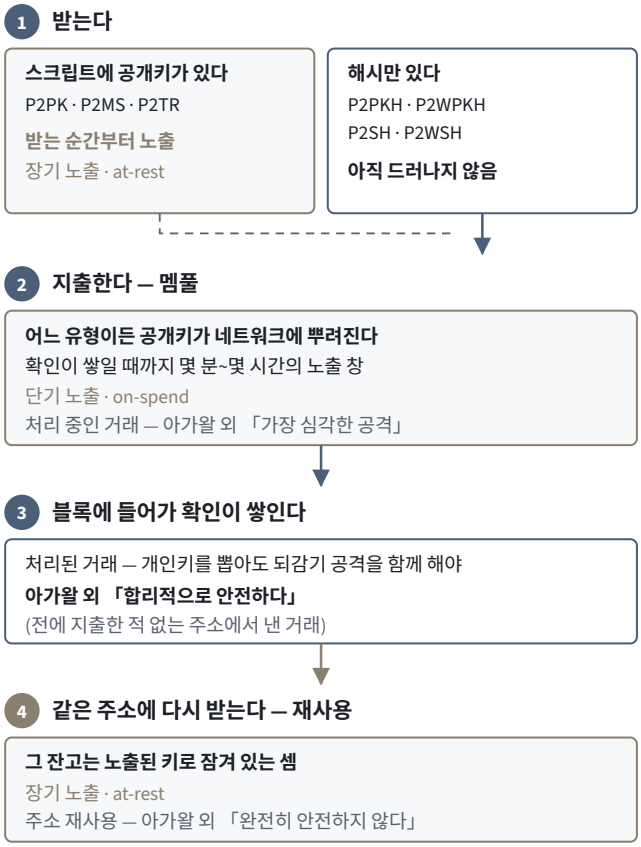


그림 29-3. 한 출력의 생애에서 공개키가 드러나는 자리. 스크립트에 키가 있는 출력은 받는 순간부터, 해시형 출력은 지출하는 순간 멤풀에서, 재사용한 주소는 다시 받은 잔고가 노출된 키로 잠긴다. 따옴표 문구는 아가왈 외의 세 상황 구분이다

BIP-360은 앞의 둘을 「장기 노출 공격(long exposure attack)」, 셋째를 「단기 노출 공격(short exposure attack)」이라 부르고, 유형별 취약성을 한 표로 적었습니다.[17] 바부시 외는 같은 구별을 「at-rest」와 「on-spend」라 부르며, 체인 밖의 노출 — 다른 체인에서의 지출, 제3자 앱과의 공유, 확장 공개키(xpub)의 공유 — 도 셉니다.[18]

출력 유형	장기 노출에 취약한가(BIP-360 표)	왜
P2PK	예	잠금 스크립트에 공개키가 그대로 있다
P2PKH	아니오*	해시만 있다. 지출 뒤 재사용하면 취약
P2MS	예	잠금 스크립트에 공개키들이 그대로 있다
P2SH	아니오*	스크립트 해시만 있다. 재사용하면 취약
P2WPKH	아니오*	위와 같음(세그윗)
P2WSH	아니오*	위와 같음(세그윗)
P2TR	예	조정된(tweaked) 공개키가 스크립트에 그대로 있다
P2MR(제안)	아니오*	스크립트 트리의 머클 루트만 있다(4절)

2021년의 탭루트가 「예」 열에 있다는 사실을 눈여겨봐 두십시오. 5장이 적은 대로 탭루트는 키 경로 지출을 위해 공개키 하나를 출력에 바로 담습니다. 바부시 외는 이것을 「양자 보안의 관점에서 보안 후퇴

(security regression)」라고 부릅니다.[19] 2009년의 P2PK가 이미 겪은 노출을 2021년의 설계가 다른 이유로 되살린 셈이고, 4절의 BIP-360이 하려는 일이 바로 그 키 경로를 떼어 내는 것입니다.

[중급] 그러면 얼마나 노출되어 있을까요. 이 책은 기준일(2026-10-25)에 **2차 집계** 하나를 동봉했습니다. 공개 SQL을 공개한 제3자(Project Eleven)가 BigQuery의 공개 비트코인 데이터셋에서 블록 972,132(2026-10-19 생성)까지를 집계한 값이고, 이 책은 그 SQL을 다시 돌리지 않았습니다 — 출처 신뢰도는 🟡 이고, 1차 재현값은 다음 판의 과제입니다.[20] 단위는 BTC 수량이고, 분모는 그 높이까지의 일정 발행량 20,100,415.625 BTC(쓸 수 없는 제네시스 출력과 미수령 보조금을 포함하는 상한)입니다.

범주	BTC	일정 발행량 대비
P2PK(스크립트에 키)	1,715,077.0	8.53%
P2TR(스크립트에 키)	217,063.2	1.08%
맨 P2MS(스크립트에 키)	416.6	0.002%
P2PKH 재사용(지출 이력 있음)	1,185,672.2	5.90%
P2WPKH 재사용	2,098,570.4	10.44%
P2SH 재사용(스크립트 공개)	1,199,814.7	5.97%
P2WSH 재사용	705,381.4	3.51%
합계	**7,121,995.4**	**35.43%**

[중급] 교차 확인은 시점과 방법이 달라 **크기 수준만** 맞습니다. mempool research의 2025년 5월 보고서는 블록 892,385(2025-04-14)의 UTXO 집합을 스크립트 유형별로만 나눠, P2PK가 UTXO **개수로는** 0.03%이지만 **금액으로는** 8.70%(1,720,270 BTC), P2TR은 개수로 34.22%이지만 금액으로 0.75%(147,912 BTC)라고 적습니다.[21] 두 집계가 겹치는 범주(P2PK)에서 171만과 172만으로 맞고, 이것이 이 수치를 믿을 수 있는 유일한 근거입니다. 업계의 추정들도 같은 자리에 모입니다 — 이 책은 그것들을 보조 출처로만 씁니다. 체인코드 연구소의 2025년 5월 보고서는 2023년 쿼리를 인용해 P2PK가 「UTXO의 약 0.025%이지만 비트코인 가치의 약 8.68%(약 1,720,747 BTC)를 잠근다」고 적고, 재사용까지 넣은 범위를 「유통량의 약 20~50%(400만~1,000만 BTC)」로, 이 장의 주 출처와 같은 기관(Project Eleven)의 이른 시점 값을 약 626만 BTC로 옮깁니다(원문은 「mid January 2015」라 적었으나 문맥상 2025의 오기로 보입니다 — 독립된 교차 확인은 아닙니다).[22] 딜로이트 분석(본문에 작성일 없음, 수치 시점 불명)은 P2PK와 재사용 P2PKH만으로 「400만 BTC 이상(전체의 약 25%)」, 코인셰어스의 2026년 2월 글은 P2PK만으로 「약 160만 BTC — 공급의 약 8%」를 적습니다.[23] BIP-361의 동기 절은 「2026년 3월 1일 기준 전체 비트코인의 34% 이상이 체인에 공개키를 드러냈다」고 적되 산출 방법은 적지 않습니다.[24] 바부시 외는 자체 집계로 「P2PK로 잠긴 170만 BTC 남짓(전체의 9% 가까이)」과 「모든 유형을 합쳐 약 690만 BTC」를 적습니다.[25] 폭의 원인은 셋입니다. 재사용을 세는가, P2TR·P2SH를 넣는가, 스냅샷이 언제인가. **P2PK 약 170만 BTC**는 직접 집계한 네 출처(스냅샷·mempool·Chaincode·바부시 외)가 170만대에 모이고 업계 둘은 160만~200만으로 적으므로, 1절의 비대칭과 함께 이 장에서 가장 단단한 숫자입니다.

```
# NB29-2: 기준일 스냅샷(2차 집계)의 유형별 노출 수량 — 동봉 파일을 읽고 소계·비율을 다시 계산한다 (공부용)
import json, os
from decimal import Decimal

FALLBACK = {  # data/exposed_output_types_2026-10-25.json 과 같은 값 (파일이 없는 환경용)
    "height": 972132, "created": "2026-10-19T10:28:55Z", "issued": "20100415.625",
    "rows": [("p2pk_script_type", "1715076.99695260"), ("taproot_script_type",
"217063.18329201"),
             ("multisig_script_type", "416.60645555"), ("pubkeyhash_reuse", "1185672.15392868"),
             ("witness_v0_keyhash_reuse", "2098570.39816002"), ("scripthash_reuse",
"1199814.71613438"),
             ("witness_v0_scripthash_reuse", "705381.38634311"), ("witness_unknown_reuse",
"0.00000000")]}

try:
    with open(os.path.join("data", "exposed_output_types_2026-10-25.json"), encoding="utf-8") as
f:
        snap = json.load(f)
    prim = snap["primary"]
    height, created = prim["snapshot_block_height"], prim["snapshot_created_at_utc"][:19]
    issued = Decimal(snap["denominator"]["issued_supply_btc"])
    rows = [(r["category"], r["btc"]) for r in prim["by_category"]]
    src = "동봉 파일"
except FileNotFoundError:
    height, created, issued = FALLBACK["height"], FALLBACK["created"],
Decimal(FALLBACK["issued"])
    rows, src = FALLBACK["rows"], "상수(파일 없음)"

print(f"출처: {src} | 스냅샷 블록 {height:,} ({created}) | 2차 집계(제3자 SQL, 재실행 안 함) | 분모 =
일정 발행량 {issued:,} BTC(상한)")
print(f"{'범주':<30}{'BTC':>18}{'발행량 대비':>12}")
script_type = Decimal(0); reuse = Decimal(0)
for cat, btc in rows:
    v = Decimal(btc)
    if cat.endswith("script_type"): script_type += v
    else: reuse += v
    print(f"{cat:<30}{v:>18,.3f}{v / issued * 100:>11.3f}%")
total = script_type + reuse
print("-" * 60)
print(f"{'소계: 스크립트에 키가 있는 출력':<30}{script_type:>18,.3f}{script_type / issued *
100:>11.3f}%")
print(f"{'소계: 재사용으로 키·스크립트 공개':<30}{reuse:>18,.3f}{reuse / issued * 100:>11.3f}%")
print(f"{'합계':<30}{total:>18,.3f}{total / issued * 100:>11.3f}%")
print()
print("읽는 법: 분모가 유통량의 상한이므로 비율은 하한 쪽이다. 재사용 범주는 키 없는 스크립트도 노출로 세어 과대
쪽이다.")
print("체인 밖 노출(xpub·서명 메시지)과 멤풀 체류 중 노출은 세지 않는다. 가격 환산은 하지 않는다.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
출처: 동봉 파일 | 스냅샷 블록 972,132 (2026-10-19T10:28:55) | 2차 집계(제3자 SQL, 재실행 안 함) | 분모 =
일정 발행량 20,100,415.625 BTC(상한)
범주                                          BTC      발행량 대비
p2pk_script_type                    1,715,076.997      8.533%
taproot_script_type                   217,063.183      1.080%
multisig_script_type                      416.606      0.002%
pubkeyhash_reuse                    1,185,672.154      5.899%
witness_v0_keyhash_reuse            2,098,570.398     10.440%
scripthash_reuse                    1,199,814.716      5.969%
witness_v0_scripthash_reuse           705,381.386      3.509%
witness_unknown_reuse                       0.000      0.000%
------------------------------------------------------------
소계: 스크립트에 키가 있는 출력                 1,932,556.787      9.615%
소계: 재사용으로 키·스크립트 공개                5,189,438.655     25.818%
합계                                  7,121,995.441     35.432%

읽는 법: 분모가 유통량의 상한이므로 비율은 하한 쪽이다. 재사용 범주는 키 없는 스크립트도 노출로 세어 과대 쪽이다.
체인 밖 노출(xpub·서명 메시지)과 멤풀 체류 중 노출은 세지 않는다. 가격 환산은 하지 않는다.
```

[중급] 유형 분류는 바이트 몇 개로 합니다. 잠금 스크립트(scriptPubKey)의 앞머리를 보면 어느 유형인지, 그래서 **받는 순간 키가 드러나는지가** 결정됩니다. 아래 코드는 12장의 제네시스 출력 스크립트(67바이트, P2PK)를 실제 바이트로 분류하고, 나머지 유형은 모양만 맞춘 가짜 바이트로 분류 규칙을 보입니다. 가짜 바이트는 주소가 아니고 어디에도 돈을 보낼 수 없습니다(실사용 금지 — 공부용 분류기입니다).

```
# NB29-3: 잠금 스크립트의 앞머리로 출력 유형과 「받는 순간 공개키 노출 여부」를 가린다 (공부용 — 가짜 바이트, 실사용
금지)
GENESIS_SPK = bytes.fromhex(  # 12장 제네시스 코인베이스 출력의 scriptPubKey (67바이트): PUSH65 <공개키>
OP_CHECKSIG

"4104678afdb0fe5548271967f1a67130b7105cd6a828e03909a67962e0ea1f61deb649f6bc3f4cef38c4f35504e51ec
112de5c384df7ba0b8d578a4c702b6bf11d5fac")

def classify(spk: bytes):
    """표준 잠금 스크립트 유형 → (유형, 받는 순간 키 노출?, 설명). BIP-360 표의 '장기 노출' 열과 같은 판정."""
    n = len(spk)
    if n in (67, 35) and spk[0] in (0x41, 0x21) and spk[-1] == 0xAC:
        return "P2PK", True, "공개키(65 또는 33바이트)가 스크립트에 그대로 있다"
    if n == 25 and spk[:3] == b"\x76\xa9\x14" and spk[-2:] == b"\x88\xac":
        return "P2PKH", False, "공개키의 HASH160(20바이트)만 있다 — 지출 때 키 공개, 재사용하면 취약"
    if n == 23 and spk[:2] == b"\xa9\x14" and spk[-1] == 0x87:
        return "P2SH", False, "스크립트의 HASH160만 있다 — 지출 때 스크립트 공개"
    if n == 22 and spk[:2] == b"\x00\x14":
        return "P2WPKH", False, "세그윗 v0, 키 해시 20바이트"
    if n == 34 and spk[:2] == b"\x00\x20":
        return "P2WSH", False, "세그윗 v0, 스크립트 해시 32바이트"
    if n == 34 and spk[:2] == b"\x51\x20":
        return "P2TR", True, "세그윗 v1(탭루트), 조정된 x-only 공개키 32바이트가 그대로 있다"
    if n == 34 and spk[:2] == b"\x52\x20":
        return "P2MR(제안, BIP-360 Draft)", False, "세그윗 v2 제안 — 스크립트 트리의 머클 루트 32바이트만,
키 경로 없음"
    if n >= 3 and 0x51 <= spk[0] <= 0x60 and spk[-1] == 0xAE:
        return "P2MS(맨 다중서명)", True, "OP_m <키들> OP_n OP_CHECKMULTISIG — 공개키들이 그대로 있다"
    return "비표준/미분류", None, "이 분류기는 표준 유형만 안다"

samples = [
    ("제네시스 출력(실제 바이트)", GENESIS_SPK),
    ("P2PKH 모양(가짜)",  b"\x76\xa9\x14" + bytes(20) + b"\x88\xac"),
    ("P2SH 모양(가짜)",   b"\xa9\x14" + bytes(20) + b"\x87"),
    ("P2WPKH 모양(가짜)", b"\x00\x14" + bytes(20)),
    ("P2WSH 모양(가짜)",  b"\x00\x20" + bytes(32)),
    ("P2TR 모양(가짜)",   b"\x51\x20" + bytes(32)),
    ("P2MR 모양(가짜)",   b"\x52\x20" + bytes(32)),
    ("P2MS 1-of-2 모양(가짜)", b"\x51" + b"\x21" + bytes(33) + b"\x21" + bytes(33) +
b"\x52\xae"),
]
print(f"{'표본':<26}{'바이트':>6}  {'유형':<28}{'받는 순간 키 노출':<16}설명")
for name, spk in samples:
    kind, exposed, why = classify(spk)
    flag = {True: "예", False: "아니오*", None: "?"}[exposed]
    print(f"{name:<26}{len(spk):>6}  {kind:<28}{flag:<16}{why}")
print()
print("* 아니오 = 받기만 하면 안전. 지출하면 키·스크립트가 공개되므로 같은 주소를 다시 쓰면 취약해진다(BIP-360 표의
No* 주석).")
print("제네시스 출력의 공개키 앞 바이트:", GENESIS_SPK[1:2].hex(), "(04 = 비압축 65바이트) — 2009년부터
체인에 그대로 적혀 있다.")
```

```
# 출력 — bcbook(Python 3.13), 실행 결과 그대로
표본                              바이트  유형                                  받는 순간 키 노출       설명
제네시스 출력(실제 바이트)              67  P2PK                                  예                  공개키(65 또는
33바이트)가 스크립트에 그대로 있다
P2PKH 모양(가짜)                    25  P2PKH                                 아니오*             공개키의
HASH160(20바이트)만 있다 — 지출 때 키 공개, 재사용하면 취약
P2SH 모양(가짜)                     23  P2SH                                  아니오*             스크립트의
HASH160만 있다 — 지출 때 스크립트 공개
P2WPKH 모양(가짜)                   22  P2WPKH                                아니오*             세그윗 v0, 키
해시 20바이트
P2WSH 모양(가짜)                    34  P2WSH                                 아니오*             세그윗 v0,
스크립트 해시 32바이트
P2TR 모양(가짜)                     34  P2TR                                  예                  세그윗 v1(탭루트),
조정된 x-only 공개키 32바이트가 그대로 있다
P2MR 모양(가짜)                     34  P2MR(제안, BIP-360 Draft)            아니오*              세그윗 v2 제안
— 스크립트 트리의 머클 루트 32바이트만, 키 경로 없음
P2MS 1-of-2 모양(가짜)              71  P2MS(맨 다중서명)                      예                   OP_m <키들>
OP_n OP_CHECKMULTISIG — 공개키들이 그대로 있다

* 아니오 = 받기만 하면 안전. 지출하면 키·스크립트가 공개되므로 같은 주소를 다시 쓰면 취약해진다(BIP-360 표의 No* 주석).
제네시스 출력의 공개키 앞 바이트: 04 (04 = 비압축 65바이트) — 2009년부터 체인에 그대로 적혀 있다.
```

「노출」과 「도난」은 다릅니다. 아가왈 외는 세 상황을 갈랐습니다. 주소 재사용(노출된 키가 잔고를 잠그고 있음 — 「완전히 안전하지 않다」), 처리된 거래(지출한 뒤 블록이 쌓인 거래 — 개인키를 뽑아도 9장의 이중지불 공격을 함께 해야 하므로 「양자 공격에 대해 합리적으로 안전하다」), 그리고 처리 중인 거래(멤풀의 공개키를 보고 블록에 들어가기 전에 개인키를 뽑아 다른 거래를 먼저 넣는 것 — 「우리는 이것을 가장 심각한 공격으로 본다」).[26] 셋째가 되려면 2절의 「목표 시간」이 블록 간격 안에 들어와야 하고, 바부시 외는 자기 추정(약 9분)을 비트코인의 10분 평균 간격에 대입해 성공 확률을 「41%에 조금 못 미친다」(단순화 가정 아래)고 그렸으며, 블록 간격이 짧은 체인에서는 그 확률이 급격히 떨어진다고 적습니다.[27] 그 수치는 모형의 출력이지 관측이 아닙니다. 이 책이 가져가는 것은 모양입니다 — **공개키가 드러난 코인은 기계가 오면 열리고, 드러나지 않은 코인은 지출하는 몇 분이 위험하다.** 그래서 전환 제안은 「드러내지 않는 출력」과 「드러나도 안전한 서명」 두 갈래로 갑니다.

4. 전환 제안 — 출력 유형, 서명, 일몰

4.1 BIP-360 — 키 경로를 떼어 낸 탭루트

BIP-360은 새 출력 유형 「Pay-to-Merkle-Root(P2MR)」를 소프트포크로 넣자는 제안입니다. 2026-10 기준 상태는 **Draft**(판 0.12.1)이고, 활성화되지 않았습니다.[28] 하는 일은 하나입니다. 「P2MR 출력은 P2TR(Pay-to-Taproot) 출력과 거의 같은 기능으로 동작하되, 키 경로 지출을 제거한다.」 탭루트 출력이 조정된 공개키를 담던 자리에 **스크립트 트리의 머클 루트**(6장)만 담습니다. 공개키는 스크립트 안에 있어도 되지만 그 스크립트는 지출 때까지 트리 뒤에 숨어 있고, 그래서 받는 순간의 노출이 사라집니다.[29] 세그윗 버전 2를 쓰고 주소는 `bc1z` 로 시작합니다. 제안자들은 이 BIP가 **하지 않는 것을** 분명히 적습니다. 「P2MR 출력은 타원곡선 암호에 대한 "장기 노출 공격"에만 저항한다」, 멤풀의 단기 노출을 막으려면 「비트코인에 양자내성 서명을 도입해야 할 수 있다」, 그리고 「이 제안은 양자내성 서명 방식의 도입을 포함하지 않는다.」[30] 비용도 적습니다 — 깊이 1인 트리의 P2MR 증인은 135바이트로, 탭루트 키 경로 지출의 66바이트보다 크지만, 같은 스크립트 경로 지출의 탭루트보다는 32바이트 작습니다.[31] 그리고 결론에서 「이 BIP는 어떤 특정한 양자 컴퓨팅 시간표에 대해서도 입장을 취하지 않는다」고 적습니다.[32]

4.2 양자내성 서명 — 세 표준과 크기의 문제

1절의 비대칭이 가리키는 출구는 「문제를 바꾼 서명」입니다. NIST는 2024년 8월 13일 세 표준을 공표했습니다. FIPS 203(ML-KEM, 격자 기반 키 캡슐화), FIPS 204(ML-DSA, 격자 기반 서명), FIPS 205(SLH-DSA, 해시 기반 서명)입니다.[33] 서명에 해당하는 것은 뒤의 둘이고, FIPS 204는 ML-DSA가 「대규모 양자 컴퓨터를 가진 공격자에 대해서도 안전하다고 믿어진다」고, FIPS 205는 SLH-DSA가 「대규모 양자 컴퓨터의 공격에 대한 저항을 제공하도록 설계되었다」고 각각 적습니다. 비트코인과 이더리움에 이것을 넣을 때의 문제는 안전성이 아니라 **크기입니다**. 아가왈 외가 2017년에 이미 적은 대로, 「블록체인 맥락에서 서명 방식의 가장 중요한 매개변수는 서명과 공개키의 길이다 — 거래를 완전히 검증하려면 어떤 식으로든 저장해야 하기 때문이다.」[34] 표로 봅니다.

방식	공개키	서명	출처
슈노르(secp256k1, 비트코인 현행)	32바이트	64바이트	BIP-340(5장); BIP-360 증인 계산의 「signature (1 + 64 bytes)」
ML-DSA-44	1,312바이트	2,420바이트	FIPS 204 Table 2
ML-DSA-65	1,952바이트	3,309바이트	FIPS 204 Table 2
ML-DSA-87	2,592바이트	4,627바이트	FIPS 204 Table 2
SLH-DSA-128s(SHA2·SHAKE)	32바이트	7,856바이트	FIPS 205 Table 2
SLH-DSA-128f	32바이트	17,088바이트	FIPS 205 Table 2
SLH-DSA-256s	64바이트	29,792바이트	FIPS 205 Table 2

격자 기반 서명은 현행 서명의 **수십 배**, 해시 기반 서명은 **백 배 넘게** 큽니다. 18장에서 블록 공간이 유한한 자원이고 21장에서 그 자원에 값이 매겨진다는 것을 보았습니다. 서명이 40배 커지면 같은 블록에 들어가는 거래 수가 그만큼 줄고, 그 비용을 누가 어떻게 부담하느냐는 설계 문제입니다. 아가왈 외는 해시 기반 방식이 「해시 함수가 랜덤 오라클처럼 행동한다는 가정 아래 증명 가능한 안전성」을 가지는 대신 그로버 때문에 양자 안전 수준이 고전 수준의 절반이 되고, 격자 기반은 같은 양자 안전 수준에서 「서명과 공개키 길이의 합에 약간의 이점」이 있다고 적었습니다. 선택은 아직 열려 있고, BIP-360도 「ML-DSA나 SLH-DSA 같은 양자내성 서명 알고리듬은 … 사용 전에 면밀히 검토되어야 한다」고만 적습니다.

4.3 BIP-361 — 옛 서명의 일몰

BIP-360이 새 출력 유형을 **더하는** 제안이라면, BIP-361은 옛 서명을 **거두는** 제안입니다. 2026-10 기준 Draft이고, 유형은 Informational, 그리고 「Requires: TBD Post Quantum Signature BIP」 — 아직 존재하지 않는 양자내성 서명 BIP를 전제로 합니다.[35] 두 단계입니다. 「A단계: 양자 취약 주소로의 송금을 금지해 양자내성 주소 유형의 채택을 앞당긴다. B단계: ECDSA/슈노르 지출을 양자 안전 구제 프로토콜로 묶어 제한함으로써 양자 취약 UTXO의 도난을 막는다. 이것은 활성화 5년 뒤의 널리 공지된 기준일(flag-day)로 촉발된다.」 명세 표는 A단계를 「BIP-361 활성화 뒤 160,000블록(약 3년)」, B단계를 「A단계 활성화 뒤 2년」으로 적습니다.[36] 「구제 프로토콜」이란 양자 공격자가 알 수 없고 진짜 소유자만 아는 지식 — 예컨대 BIP-32 지갑의 상위 비밀키 — 을 증명하게 하는 장치이고, 제안자들은 「그런 구제 프로토콜이 비트코인 공급의 대부분을 덮을 수 있다면 ECDSA/슈노르 검증을 제한하는 것은 기껏해야 **가볍게 몰수적**(at most mildly confiscatory)일 것」이라고 적습니다.[37] P2PK에는 그런 지식의 비대칭이 없어 구제 프로토콜을 만들 수 없다고 보고, 그 코인에 대해서는 별도 제안(Hourglass — 5절)을 지지한다고 적습니다.

이 제안의 성격을 한 줄로 적으면, **아직 없는 서명 BIP를 전제로 한 일정표입니다.** 그 일정표가 「합의(consensus)」로 채택되려면 7장과 23장이 보인 대로 노드와 채굴자 다수가 새 규칙을 돌려야 하고, 그 다수가 B단계에서 하는 일은 특정 UTXO의 지출 조건을 **사후에** 바꾸는 것입니다. 20장의 질문 상자가 「발행 규칙을 다수가 바꾸면 보유자의 권리는 침해되는가」를 물었습니다. 여기서는 발행 규칙이 아니라 **지출 가능성 자체를** 바꿉니다. 5절이 그 질문으로 갑니다.

4.4 이더리움 — 네 영역과 계정 추상화

이더리움의 노출은 비트코인과 구조가 다릅니다. 11장의 계정 모형에서 한 번이라도 거래를 보낸 계정은 공개키가 드러나 있고, 계정을 계속 쓰는 것이 기본 사용 방식입니다. 거기에 10장의 지분증명이 쓰는 검증자 서명(BLS)과 18장의 데이터 가용성이 쓰는 KZG 커밋먼트가 모두 타원곡선 페어링에 기대고 있습니다. 이더리움 공식 사이트의 로드맵 보안 페이지(소스 커밋 290d753c, 2026-07-12)는 네 영역을 적습니다 — 검증자 합의 서명(BLS), 데이터 가용성의 커밋먼트(KZG), 계정 서명(ECDSA), 롤업의 영지식 증명 체계.[38] 같은 페이지의 굵은 글씨 문장은 「오늘날 어떤 양자 컴퓨터도 이더리움의 암호를 깰 수 없다」이고, 같은 페이지는 2026년 3월의 1,200 논리 큐비트 추정(2절)을 「이전 추정보다 약 20배 적다」고 인용합니다 — 바부시 외의 「20배」는 물리 큐비트(50만 미만) 쪽이고 논리 큐비트는 2절 표대로 약 1.9배 줄었습니다. 하위 페이지(quantum-resistance, 커밋 643a7a45, 2026-10-23)는 「현재의 양자 하드웨어는 이 규모에서 멀리 떨어져 있고, 수천 개의 잡음 있는 물리 큐비트로 동작한다」고 적습니다(2026-10 기준, 부록 A).[39]

대응의 모양은 비트코인과 **거꾸로입니다.** 비트코인 제안이 「출력 유형 하나를 더하고 옛 서명을 거두는」 프로토콜 변경이라면, 이더리움은 계정 서명에서 **계정별 선택을** 택하려 합니다(2026-10 기준 계획). 「단일한 프로토콜 전체 이전 대신, 이더리움은 계정 추상화(구체적으로 EIP-8141, 2026년 하반기 Hegotá에 고려 중)를 써서 사용자에게 서명 민첩성(signature agility)을 주려 한다. 개별 계정이 프로토콜 전체가 바뀌기를 기다리지 않고 양자내성 서명 방식으로 바꿀 수 있다.」[40] 같은 사이트의 상위 로드맵 페이지(2026-07-12 소스)는 Hegotá를 「2027년 목표」로 적어, 두 공식 페이지의 연도가 서로 다릅니다. 이 책은 어느 한쪽을 고르지 않고 둘을 그대로 적습니다(부록 A-29). 검증자 서명 쪽은 해시 기반 서명(leanXMSS)으로 바꾸되, 그 서명이 BLS의 96바이트에 비해 「대략 3,000바이트」로 커지는 문제를 영지식 증명으로 **집계해** 푼다는 것이 공식 페이지의 설계입니다(2026-10 기준 계획).[41] 재단의 양자 대응 페이지(2026-06-27 갱신)는 「우리는 암호학적으로 유의미한 양자 컴퓨터가 임박했다고 믿지 않는다」고

적으면서도, 「우리 팀의 현재 평가에 따르면 L1 프로토콜 업그레이드는 2029년까지 완료될 수 있다」고 적습니다(2026-10 기준 계획 목표, 부록 A).[42] 그리고 휴면 자산에 대해 — 다음 절의 문제 — 「비트코인에서는 공급의 대략 5%가 널리 버려졌다고 여겨지는 초기 주소 형식과 결부되어 있는 반면, 이더리움의 장기 휴면 노출은 0.1%에 가깝다(재단 추정)」고 적고, 「아무것도 하지 않거나, 취약 코인을 동결하거나」 두 시나리오가 있으며 「궁극적으로 커뮤니티 거버넌스의 문제」라고 씁니다.[43]

5. 취약 코인 — 세 갈래와 한 질문

전환이 아무리 잘 되어도 **옮겨 가지 못하는 코인이** 남습니다. 열쇠를 잃은 코인, 주인이 세상을 떠난 코인, 그리고 3절의 P2PK 약 170만 BTC 가운데 2009년 이래 한 번도 움직이지 않은 것들. 바부시 외는 이 부분을 「휴면 디지털 자산」이라 부르고 두 사실을 적습니다. P2PK 170만 BTC의 「다수는 2009년 이래 거래된 적이 없다」는 것, 그리고 「사토시 나카모토가 소유했다고 믿어지는 계정들이 약 100만 비트코인을 이 P2PK 스크립트 뒤에 보유한다」는 것 — 뒤의 것은 귀속 **추정이고** 이 책은 그 귀속에 가담하지 않습니다.[44] 이 코인들은 소프트웨어 갱신으로 옮길 수 없습니다. 열쇠를 쥔 사람이 없기 때문입니다. 그래서 기계가 오면 누군가 가져가거나, 아무도 못 가져가게 규칙을 바꾸거나, 둘 중 하나가 됩니다.

비트코인 쪽 논의는 세 갈래로 정리되어 있습니다. 바부시 외의 요약을 그대로 옮깁니다. 「**그대로 둔다**(Do Nothing): 프로토콜을 바꾸지 않아 양자 공격자가 휴면 자산을 가져가게 한다. **소각한다**(Burn): 특정 날짜 뒤 휴면 자산을 지출 불가능하게 프로토콜을 고치며, 사실상 P2PK 스크립트로 잠긴 모든 자산을 몰수한다. **모래시계**(Hourglass): 휴면 자산을 지출할 수 있는 속도를 제한하도록 프로토콜을 고친다.」[45] BIP-361의 작성자들은 같은 세 갈래를 「누구나 가져가게 한다 / 속도를 제한해 가져가게 한다 / 아무도 못 가져가게 한다」로 적고, 앞의 둘을 「재분배 딜레마」를 만드는 제안으로 분류합니다.[46] 바부시 외는 2025년 한 회의의 비공식 투표에서 세 갈래가 「대략 같은 지지」를 받았다고 적고, 「합의는 여전히 잡히지 않는다」고 씁니다.[47] 소각과 모래시계는 기술적으로 소프트포크로 가능합니다 — 특정 UTXO의 지출을 금지하거나 제한하는 것은 규칙을 **좁히는** 일이기 때문입니다(7장·23장). 그대로 두기는 규칙 변경이 없습니다. 어려운 것은 기술이 아닙니다.

세 갈래가 각각 무엇을 **전제하는지를** 적어 둡니다. 「그대로 둔다」는 열쇠를 아는 자가 소유자라는 비트코인의 소유 모형을 기계가 와도 유지한다는 것이고 — 바부시 외는 「CRQC는 어떤 공개키에서든 개인키를 빠르게 뽑아내므로 비트코인 소유 모형의 토대를 깬다」고 적습니다[48] — 「소각한다」는 열쇠를 아는 자가 소유자라는 모형을 다수결로 **정지시키는** 것이며, 「모래시계」는 그 사이에서 속도를 조절하는 것입니다. 규칙을 바꾸는 쪽을 택한다면 결정 주체는 **노드와 채굴자의 다수이고**, 그 다수가 누구를 대리하는지는 아래 상자가 묻습니다. 바부시 외의 백서는 거기서 더 나아가 정부 쪽의 선택지 — 버려진 재산의 「디지털 인양(digital salvage)」, 미국의 통일 미청구재산법(RUUPA 2016)과 그 「보유자(holder)」 요건의 부적합, 권리 위에 잠자는 자(laches) 법리, 국가안보 대응, 커뮤니티와의 대화 — 를 열거하고, 「휴면 자산의 규제된 파괴는 실행 가능한 정책이 아니다」라는 논증(블록을 지우려면 체인 전체를 다시 캐야 하고, 채굴자에게 거래 검열을 강제하면 채굴이 국외로 옮겨 가거나 하드포크가 난다)을 폅니다.[49] 이 책은 그 논증을 **소개만** 합니다. 평가하지 않는 이유는 둘입니다. 이 문제를 다룬 법령·판결은 이 책의 원자료에 없고(28.1(g)), 그 백서 자체가 이해관계 있는 기관의 문서이기 때문입니다.

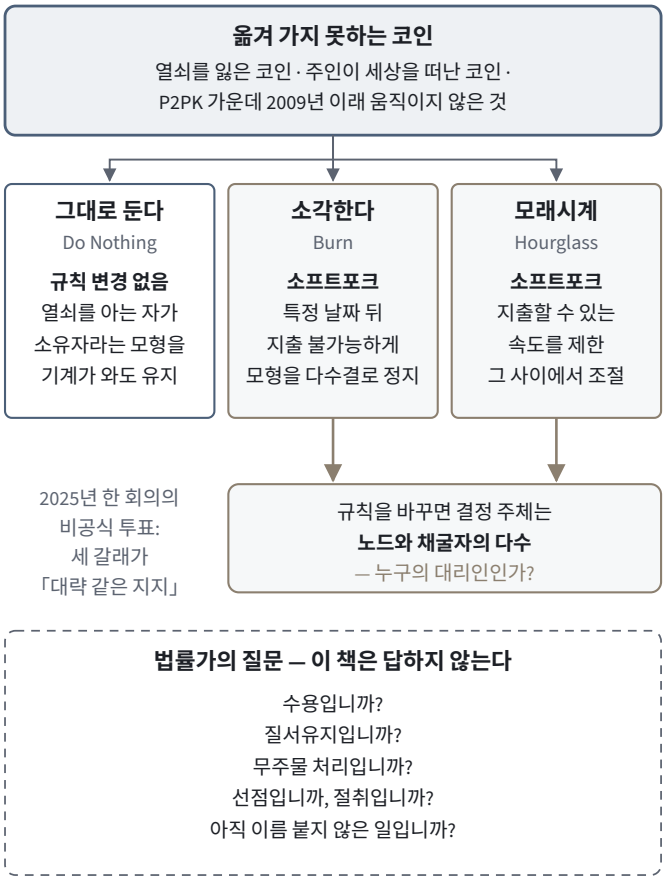


그림 29-4. 옮겨 가지 못하는 코인을 두고 비트코인 쪽에서 논의되는 세 갈래와 각 갈래의 전제. 규칙을 바꾸는 두 갈래의 결정 주체는 노드와 채굴자의 다수다. 아래 물음은 질문 상자의 문안이고 어느 갈래에도 짝짓지 않았다

? 법률가의 질문

어느 날 노드와 채굴자의 다수가 「공개키가 드러난 옛 출력은 더 이상 지출할 수 없다」는 규칙을 소프트포크로 넣었다고 합시다(4.3의 B단계, 또는 「소각」). 그 순간 열쇠를 잃지 않은 소수 보유자의 코인도 함께 잠깁니다. 이것은 **수용입니까** — 그렇다면 수용의 주체는 누구이고 보상은 누가 합니까. **질서유지입니까** — 기계가 오면 어차피 누군가 가져갈 재산을 아무도 못 가져가게 막은 것이니. 아니면 열쇠를 아는 자가 아무도 없어진 재산에 대한 **무주물 처리입니까** — 그렇다면 열쇠를 아직 아는 소수는 어떻게 됩니까. 「모래시계」로 속도만 제한하면 성질이 달라집니까. 반대로 「그대로 두어」 양자 컴퓨터를 가진 자가 먼저 가져가게 하면, 그 취득은 선점입니까, 절취입니까, 아니면 어느 법역의 어느 조문도 아직 이름 붙이지 않은 일입니까. 그리고 이 모든 것을 **결정하는 다수** — 7장의 「규칙을 따르는 참가자」들 — 는 코인 보유자의 대리인입니까, 네트워크의 대리인입니까, 자기 자신의 대리인입니까. 20장 상자가 발행 규칙으로 물은 것을 이 장은 지출 가능성으로 다시 묻습니다. 이 책은 답하지 않고 5부로 넘깁니다. 27.2의 열네 갈래(사실상태설·조합권설·통제 기반 등) 가운데 어느 것을 택하느냐에 따라 이 질문의 답이 달라지고, 28.1(g)는 이 질문에 닿는 1차 자료가 이 책의 원자료에 없다고 적어 두었습니다. → 27장 · 28장

6. 「2026-10 기준」 — 이 장에서 낡을 것들

3장 일러두기의 갱신 규칙에 따라 이 장이 쓴 현재 상황 문장을 모읍니다. 전부 부록 A에 올려 두었고, 판을 바꿀 때 거기만 고칩니다.

- BIP-360은 Draft(판 0.12.1), BIP-361은 Draft, BIP-54는 Complete이되 배포 절이 없어 활성화로 쓰지 않는다 — 2026-10 기준(부록 A-27).
- 이 장이 검토한 양자내성 서명 전환 제안(BIP-360·361, EIP-8141·leanXMSS)은 비트코인·이더리움 어느 쪽 메인넷에도 활성화되지 않았다(제안·계획 단계) — 2026-10 기준(부록 A-27·A-29). 이는 해시 기반 보호처럼 이미 있는 성질이 없다는 뜻이 아니다.
- 하드웨어: 이더리움 공식 페이지는 「수천 개의 잡음 있는 물리 큐비트」라 적고, 2026년 백서의 추정은 「50만 개 미만」이다 — 2026-10 기준(부록 A-28). 전문가 26명 설문(2026년 3월 공표)의 수치도 부록 A에만 둔다.
- 이더리움 로드맵: EIP-8141은 Hegotá 업그레이드에 「고려 중」, 핵심 양자내성 이정표의 목표는 「대략 2029년」이며 「계획 목표이고 바뀔 수 있다」 — 2026-10 기준(부록 A-29).
- 노출 수량: 3절의 표는 2026-10-19 블록 972,132의 2차 집계이고 1차 재현값은 없다 — 2026-10 기준(부록 A-30).
- ECDSA 폐기 일정 — 이더리움 페이지는 「NIST가 2030년 사용 중단·2035년 금지를 예상한다」고 문서명 없이 적고, BIP-360은 2030년을 CNSA 2.0 일정으로, 2035년을 NIST IR 8547로 적는다. 이 책은 NIST IR 8547 원문을 열지 않았다 — 과제(부록 A-31).

7. 정리

양자 컴퓨터는 1부의 두 도구를 다르게 건드립니다. 쇼어의 알고리듬은 서명의 바탕인 이산로그 문제를 다항 시간으로 떨어뜨리고, 그로버의 알고리듬은 해시의 역상 탐색을 제곱근만큼만 앞당깁니다. 그래서 위협은 「비트코인」이 아니라 **공개키가 드러난 서명에** 있고, 작업증명은 ASIC의 속도와 병렬화 앞에서 버팁니다. 그 기계의 크기는 알고리듬과 오류정정 이론의 진보로 2,330에서 1,200 논리 큐비트 안팎으로 내려왔고, 물리 큐비트 추정은 가정 다섯(오류율·코드 사이클·반응 시간·연결성·목표 시간)에 따라 수십만에서 십억 단위까지 갈리므로 같은 가정끼리만 비교할 수 있습니다. 드러난 공개키는 세 자리 — 스크립트 유형, 재사용, 멤풀 — 에 있고, 기준일의 2차 집계는 발행량의 약 3분의 1을 노출로 셉니다(P2PK 약 170만 BTC는 직접 집계한 출처들이 모이는 가장 단단한 수). 전환 제안은 드러내지 않는 출력(BIP-360), 드러나도 안전한 서명(FIPS 204·205 — 수십에서 수백 배 큰 서명), 옛 서명의 일몰(BIP-361), 계정별 선택(이더리움)으로 갈리고, 2026-10 기준 어느 것도 활성화되지 않았습니다. 그리고 옮겨 가지 못하는 코인이 남습니다. 그대로 둘 것인가, 소각할 것인가, 속도를 제한할 것인가 — 세 갈래 모두 기술로는 가능하고, 어느 것도 기술이 정하지 않습니다. 법률가가 이 장에서 가져갈 것은 그 사실입니다. 열쇠를 아는 자가 소유자라는 모형이 깨지는 날, 그 모형을 **정지시키는 결정의 성질과 결정하는 다수의 지위를** 어느 법이 어떻게 이름 붙일지는 비어 있는 자리입니다. 다음 30장은 그 「이름 붙이기」 자체 — 같은 비트열을 다섯 법역이 다섯 이름으로 부르는 일 — 를 조문 원문으로 봅니다.

이 장의 출처

- NIST, *Report on Post-Quantum Cryptography*, NISTIR 8105 (April 2016), DOI 10.6028/NIST.IR.8105 — Table 1(2쪽), 쇼어·그로버 서술(1·3쪽), 해시 기반 서명(4~5쪽).
- Divesh Aggarwal, Gavin K. Brennen, Troy Lee, Miklos Santha & Marco Tomamichel, "Quantum Attacks on Bitcoin, and How to Protect Against Them," *Ledger* 3 (2018), DOI 10.5195/ledger.2018.127 — 저장본 arXiv:1710.10377v1: 초록(1쪽), 채굴 공격(4~8쪽), 서명 공격 세 상황과 자원(8~9쪽), Momentum(11~12쪽), 서명 방식 비교 Table II(13쪽). 쪽은 arXiv판.
- Martin Roetteler, Michael Naehrig, Krysta M. Svore & Kristin Lauter, "Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms," *ASIACRYPT 2017*, LNCS, pp. 241–270, DOI 10.1007/978-3-319-70697-9_9 — 저장본 arXiv:1706.06752v3: 일반식(1쪽), 쇼어 서술(2쪽), P-256 값(4쪽).
- Mark Webber, Vincent Elfving, Sebastian Weidt & Winfried K. Hensinger, "The Impact of Hardware Specifications on Reaching Quantum Advantage in the Fault Tolerant Regime," *AVS Quantum Science* 4(1): 013801 (2022), DOI 10.1116/5.0073075 — 저장본 arXiv:2108.12371v3: 초록(1쪽), 비트코인 절(13쪽).
- Ryan Babbush 외, "Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations," arXiv:2603.28846v2 (2026-04-15), 프리프린트 — 초록·4쪽·8쪽(자원·시간), 10~11쪽(연결성·시나리오·임계값 모형), 12~15쪽(노출·공격 기제), 19~20쪽(xpub·작업증명), 36~42쪽(휴면 자산·정책 선택지). Google Quantum AI·Ethereum Foundation 소속 저자 포함.
- NIST FIPS 203·204·205 (Published August 13, 2024) — 초록; FIPS 204 Table 2(16쪽); FIPS 205 §11 Table 2. `nist-2024-fips-204-module-lattice-signature.pdf`, `nist-2024-fips-205-stateless-hash-signature.pdf`.
- BIP-360 "Pay-to-Merkle-Root (P2MR)" (Draft, v0.12.1), BIP-361 "Post Quantum Migration and Legacy Signature Sunset" (Draft) — github.com/bitcoin/bips 커밋 100ba872c808946155b1eeb8c4568058f2eea0a7 (`bip-0360.mediawiki`, `bip-0361.mediawiki`, `README.mediawiki` L1286~1298), 2026-10-25 접근·축어 대조. BIP-54 상태는 커밋 927b6de9915c9262615a6399de51b200f81e5aa4 `bip-0054.md` (부록 A-3).
- ethereum.org, "A more secure Ethereum"(소스 커밋 290d753c, 2026-07-12)·"Post-quantum cryptography on Ethereum"(소스 커밋 643a7a45, 2026-10-23) — 2026-10-25 접근. Ethereum Foundation Post-Quantum team, https://pq.ethereum.org/ (Last updated June 27, 2026) — 2026-10-25 접근.
- 이 책의 동봉 데이터 `data/exposed_output_types_2026-10-25.json` (기준일 2026-10-25; 주 출처 Project Eleven 집계 블록 972,132, 비교 출처 mempool research UTXO Set Report 블록 892,385) — 2차 집계(🟡), 출처·한계는 `data/README.md`.
- 보조 출처(업계, 🟡): Milton & Shikhelman, *Bitcoin and Quantum Computing* (Chaincode Labs, May 2025); Deloitte, "Quantum computers and the Bitcoin blockchain"(작성일 미기재); CoinShares, "Quantum Vulnerability in Bitcoin: A Manageable Risk"(2026-02-06). 2026-10-25 접근. 달러 환산·시장 서술은 옮기지 않았다.

- 코드 실행: Python 3.13 (conda 환경 `bcbook`), 표준 라이브러리 `math`·`json`·`decimal` 만 사용. NB29-2는 동봉 `data/exposed_output_types_2026-10-25.json` 을 읽고 파일이 없으면 같은 값의 상수를 쓴다. 노트북 NB29.
- 이 장은 가격·시가총액·수익률 자료를 쓰지 않았고, 「양자 위협으로 가격이 …」 류의 서술을 하지 않았다. 양자 컴퓨터의 등장 시점을 예측하지 않았다. 쇼어(1994/1997)·그로버(1996)의 원 논문, Häner 외(2020), NIST IR 8547, 미국 통일 미청구재산법 원문은 이 책이 열지 않았다.

각주

1. Lily Chen, Stephen Jordan, Yi-Kai Liu, Dustin Moody, Rene Peralta, Ray Perlner & Daniel Smith-Tone, *Report on Post-Quantum Cryptography*, NISTIR 8105 (April 2016), 2쪽, Table 1 "Impact of Quantum Computing on Common Cryptographic Algorithms". 행 원문: "AES — Symmetric key — Encryption — Larger key sizes needed / SHA-2, SHA-3 — Hash functions — Larger output needed / RSA — Public key — Signatures, key establishment — No longer secure / ECDSA, ECDH (Elliptic Curve Cryptography) — Public key — Signatures, key exchange — No longer secure / DSA (Finite Field Cryptography) — Public key — Signatures, key exchange — No longer secure".

2. NISTIR 8105, 1쪽. 원문: "In 1994, Peter Shor of Bell Laboratories showed that quantum computers, a new technology leveraging the physical properties of matter and energy to perform calculations, can efficiently solve each of these problems, thereby rendering all public key cryptosystems based on such assumptions impotent [1]." 보고서의 [1]은 P. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," *SIAM J. Comput.* 26(5): 1484–1509 (1997) — 이 책은 쇼어의 원문을 열지 않았고 NIST와 아가왈 외(아래)의 인용으로만 쓴다.

3. NISTIR 8105, 1쪽. 원문: "In particular, Grover's search algorithm proffers a quadratic speedup on unstructured search problems. While such a speedup does not render cryptographic technologies obsolete, it can have the effect of requiring larger key sizes, even in the symmetric key case." 3쪽: "We don't know that Grover's algorithm will ever be practically relevant, but if it is, doubling the key size will be sufficient to preserve security. Furthermore, it has been shown that an exponential speed up for search algorithms is impossible, suggesting that symmetric algorithms and hash functions should be usable in a quantum era [3]."

4. Divesh Aggarwal, Gavin K. Brennen, Troy Lee, Miklos Santha & Marco Tomamichel, "Quantum Attacks on Bitcoin, and How to Protect Against Them," *Ledger* 3 (2018), DOI 10.5195/ledger.2018.127; 이 책의 저장본은 arXiv:1710.10377v1(2017-10-28), 1쪽 초록. 원문: "We find that the proof-of-work used by Bitcoin is relatively resistant to substantial speedup by quantum computers in the next 10 years, mainly because specialized ASIC miners are extremely fast compared to the estimated clock speed of near-term quantum computers. On the other hand, the elliptic curve signature scheme used by Bitcoin is much more at risk, and could be completely broken by a quantum computer as early as 2027, by the most optimistic estimates." 쪽은 arXiv판 기준.

5. Ryan Babbush, Adam Zalcman, Craig Gidney, Michael Broughton, Tanuj Khattar, Hartmut Neven, Thiago Bergamaschi, Justin Drake & Dan Boneh, "Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations," arXiv:2603.28846v2 (2026-04-15), 20쪽 §III.D. 원문: "However, we maintain that this is not something to worry about in the next several decades. Grover-based attacks on Bitcoin mining are not practically relevant for two reasons. First, the quadratic quantum speedup from Grover's algorithm is all but consumed by the overheads of quantum error correction [154]. Second, Grover's algorithm does not parallelize well [155]. In Bitcoin mining, hardware acceleration and massive parallelization are a much greater advantage than the modest speedup from Grover's algorithm." 저자 소속에 Google Quantum AI와 Ethereum Foundation(Drake)이 있다 — 이해관계가 있는 기관의 문서임을 적어 둔다. 프리프린트이고 게재 정보는 없다.

6. Aggarwal 외 (2018), 6~7쪽 §III.A. 원문: "The number of logical qubits needed in the Grover algorithm is fixed at 2402, independent of the difficulty." / "Assuming maximum gate speeds attainable on current devices of s = 66.7MHz [KKL+17] and assuming an experimentally challenging, but not implausible, physical gate error rate of pg = 5×10−4, and close to current difficulty D = 10^12, the overheads are cτ = 538.6 and cnQ = 1810.7, implying an effective hash rate of hQC = 13.8GH/s using nQ = 4.4 × 10^6 physical qubits. This is more than one

thousand times slower than off the shelf ASIC devices which achieve hash rates of 14TH/s ...". 2017년 난이도 가정값이고, 기준일 난이도(약 1.4×10^{14}, 8장·부록 A-4)와 다르다.

7. Babbush 외 (2026), 20쪽. 원문: "Under the fantastical assumption that a CRQC could compute SHA-256 in a single 1 microsecond error correction cycle, a quantum miner could achieve a hashrate of 0.25 TH/s which is more than two orders of magnitude below 110 ± 3% TH/s of the popular ASIC miner S19 Pro [156]. Under realistic assumptions, rather than fantastical ones, quantum miner's hashrate plummets by over 10 orders of magnitude [157]." 같은 쪽: "But for now, quantum mining remains science fiction more than a concrete threat."

8. Martin Roetteler, Michael Naehrig, Krysta M. Svore & Kristin Lauter, "Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms," *ASIACRYPT 2017*, LNCS, pp. 241–270, DOI 10.1007/978-3-319-70697-9_9; 이 책의 저장본은 arXiv:1706.06752v3(2017-10-31), 2쪽. 원문: "In his famous paper [44], Peter Shor presented two polynomial-time quantum algorithms, one for integer factorization and another one for computing discrete logarithms in a finite field of prime order. Shor notes that the latter algorithm can be generalized to other fields. It also generalizes to the case of elliptic curves. Hence, given the prerequisite that a large enough general purpose quantum computer can be built, the algorithms in Shor's paper completely break all current crypto systems based on the difficulty of factoring or computing discrete logarithms." 쪽은 arXiv판 기준.

9. Aggarwal 외 (2018), 11~12쪽 §IV.A. 원문: "As the classical running time is m|S|, we see that unlike for the current proof of work in Bitcoin, with this proposal a quantum computer would not be able to achieve a quadratic advantage as soon as S is more than constant size. In particular, since √m|S|^{2/3} is minimized also when S = √(2^{n+ℓ}/t), the running time of even the fastest quantum algorithm is at least T^{2/3}, which is substantially larger than T^{1/2}."

10. Roetteler 외 (2017), 1쪽 초록·4쪽. 원문: "We conclude that elliptic curve discrete logarithms on an elliptic curve defined over an n-bit prime field can be computed on a quantum computer with at most 9n + 2⌈log2(n)⌉+ 10 qubits using a quantum circuit of at most 448n^3 log2(n) + 4090n^3 Toffoli gates." / "For example, our simulation of the point addition quantum circuit for the NIST standardized curve P-256 needs 2330 logical qubits and the full Shor algorithm would need about 1.26 · 10^11 Toffoli gates." 시뮬레이션 대상은 NIST 곡선이고 secp256k1은 서론에서 사용처로만 언급된다("the Bitcoin curve secp256k1 from the SEC2 [8] standard", 1쪽) — 같은 256비트 소수체 곡선이라 규모는 같다.

11. Aggarwal 외 (2018), 8~9쪽 §III.B. 원문: "Bitcoin uses n = 256 bit signatures so the number of Toffoli gates is 1.28 × 10^11, which can be slightly parallelized to depth 1.16 × 10^11." / "nQ = 2334 × cnQ(pg), where the first factor is the number of logical qubits including 4 logical ancilla qubits" / "Using a surface code with a physical gate error rate of pg = 5 × 10−4, the overhead factors are cτ = 291.7 and cnQ = 735.3, and the time to solve the problem at 66.6 MHz clock speed is 6.49 days using 1.7 × 10^6 physical qubits. Looking forward to performance improvements, for 10GHz clock speed and error rate of 10−5, the signature is cracked in 30 minutes using 485550 qubits."

12. Mark Webber, Vincent Elfving, Sebastian Weidt & Winfried K. Hensinger, "The Impact of Hardware Specifications on Reaching Quantum Advantage in the Fault Tolerant Regime," *AVS Quantum Science* 4(1): 013801 (2022), DOI 10.1116/5.0073075; 이 책의 저장본은 arXiv:2108.12371v3(2021-11-11), 1쪽 초록·13쪽. 원문: "It would require approximately 317 million physical qubits to break the encryption within one hour using the surface code, a code cycle time of 1 µs, a reaction time of 10 µs, and physical gate error of 10−3. To break the encryption instead within one day it would require 13 million physical qubits." / "To break it within 10 minutes with the same code cycle time it would require 1.9 billion physical qubits whereas to break it in 1 day would require only 13 million physical qubits." 입력 회로는 "the depth optimized approach provided by Haner et al[48] for 256 bit encryption, which corresponds to 5.76 · 10^9 T gates, 2871 logical qubits"(13쪽) — 그 선행 논문의 원문은 이 책의 원자료에 없다.

13. Babbush 외 (2026), 1쪽 초록·4쪽·8쪽. 원문: "We demonstrate that Shor's algorithm for this problem can execute with either ≤1200 logical qubits and ≤90 million Toffoli gates or ≤1450 logical qubits and ≤70 million Toffoli gates. In the interest of responsible disclosure, we use a zero-knowledge proof to validate these results without disclosing attack vectors. On superconducting architectures with 10−3 physical error rates and planar connectivity, those circuits can execute in minutes using fewer than half a million physical qubits." / 4쪽: "(nearly a 20 fold reduction over prior estimates [44])" / 8쪽: "If we assume a control system reaction time

of 10 microseconds (a standard figure for superconducting qubits [100]), and a 50% overhead per Toffoli gate, then 70 or 90 million Toffoli gates can be resolved in 18 or 23 minutes on a superconducting CRQC — close to the 10 minute average Bitcoin block time." / "we should estimate the time required to launch an on-spend attack starting from this primed state at the moment the public key is learned to be roughly either 9 minutes or 12 minutes."

14. Babbush 외 (2026), 11쪽 §II.C. 원문: "By contrast, quantum computing, with its broad variety of hardware platforms, is still in the "era of ferment" where simple models, such as counting physical qubits, fail to adequately capture technological progress. Instead, progress comes in discrete jumps corresponding to development of new internal capabilities and overcoming scaling challenges, e.g., by getting device error rates below the threshold for an error-correcting code [125] or implementing coherent interconnects for modular architectures [126–128]. Therefore, progress in quantum computing is better understood using a threshold model rather than in terms of the number of physical qubits."

15. Babbush 외 (2026), 10~11쪽 §II.C. 원문: "Scenario 1: On-spend attacks become viable about the same time as at-rest attacks. This is a possibility if the first CRQCs are fast-clock devices. / Scenario 2: At-rest attacks become viable meaningfully sooner than on-spend attacks. This is a possibility if slow-clock quantum platforms advance more rapidly than fast-clock ones." 4쪽: "We delineate how superconducting, silicon, and photonic platforms, with their fast gates and short error correction cycle times, pose a distinct threat to active transactions that slower architectures like neutral atoms and ion traps likely cannot match."

16. Babbush 외 (2026), 10쪽. 원문: "More aggressive hardware assumptions — such as the "bicycle" architecture used for 2-gross qLDPC codes [115, 117] — could drop qubit counts closer to one hundred thousand physical qubits, but this approach requires non-local degree-seven connectivity that has yet to be demonstrated in actual superconducting qubit devices." / "Consequently, while a Pinnacle-style device would theoretically require even fewer than one-hundred thousand superconducting qubits to break 256-bit ECDLP, a planar degree-four device with half a million qubits remains the better understood and also likely more feasible engineering path."

17. BIP-360 "Pay-to-Merkle-Root (P2MR)" (Hunter Beast, Ethan Heilman, Isabel Foxen Duke; Status: Draft; Version 0.12.1), github.com/bitcoin/bips 커밋 100ba872c808946155b1eeb8c4568058f2eea0a7 `bip-0360.mediawiki` (2026-10-25 접근). 「Long Exposure vs Short Exposure Attacks」 절. 원문: "A long exposure attack is an attack performed on exposed blockchain data, such as exposed public keys, or the scripts of spent outputs. These are likely to be the earliest quantum attacks made possible on Bitcoin, because attackers will have ample time — as much time as vulnerable keys are exposed — to carry out quantum key recovery. / Short exposure attacks, however, require faster quantum computers, because they must occur within the relatively short time that a transaction is unconfirmed in the mempool." 표의 「No*」 주석: "Funds in P2PKH, P2SH, P2WPKH, P2WSH, and P2MR outputs can become vulnerable to long exposure quantum attacks anytime their script reveals a public key." 그리고 "Note: Extended public keys, commonly known as "xpubs," and wallet descriptors also reveal quantum vulnerable public key information."

18. Babbush 외 (2026), 12쪽 §III.A·17쪽 §III.B(4) Offchain Exposure·19쪽. 원문: "Since the public key is disclosed directly in the locking script, it remains visible to everyone from the moment the bitcoin is received. Therefore, bitcoin locked by a P2PK script is vulnerable to both at-rest and on-spend attacks." / "However, spending such bitcoin does entail disclosure of the public key in the unlocking script, so it remains vulnerable to on-spend attacks." / "In the absence of CRQCs, it is safe to share extended public keys and use normal derivation to obtain keys for cryptocurrency transactions. ... Therefore, in the presence of CRQCs, reuse or disclosure of an extended public key constitutes a pernicious form of Offchain Exposure vulnerability that enables an attacker with a CRQC to compromise multiple private keys at the cost of one."

19. Babbush 외 (2026), 13쪽. 원문: "However, P2TR stores the tweaked public key directly in the locking script without hiding it behind a hash value which brings back the quantum vulnerability of P2PK and P2MS addresses and therefore, from the standpoint of quantum security, constitutes a security regression [30, 137]."

20. 이 책의 `data/exposed_output_types_2026-10-25.json` (기준일 2026-10-25, 접근 2026-10-24T18:19:27Z). 주 출처: Project Eleven Bitcoin Risq List metrics endpoint(스냅샷 블록 972132, 2026-10-19T10:28:55Z 생성), 방법은 공개 SQL(github.com/p-11/bitcoin-risq-list-query 커밋 bb3177f8 `btc_at_quantum_risk_query.sql`) — "(1) outputs whose script type puts a key in the locking script (P2PK, bare multisig, P2TR); (2) unspent balances of

P2PKH/P2WPKH/P2SH/P2WSH addresses that have previously been spent from (key or script revealed)". 분모는 Bitcoin Core `GetBlockSubsidy` 규칙으로 계산한 일정 발행량 20,100,415.625 BTC. 응답에 있던 환율 필드는 옮기지 않았다(3장 일러두기). 한계(`data/README.md`): 제3자 집계이고 노드에서 재계산하지 않았다; P2SH·P2WSH 재사용 범주는 공개된 스크립트에 키가 없어도 노출로 센다; 체인 밖 노출과 멤풀 체류 중 노출은 세지 않는다; 분모는 유통량의 상한이다.

21. mempool research, "UTXO Set Report" (orangesurf, 2025-05-18), https://research.mempool.space/utxo-set-report/ (2026-10-24 접근) — 블록 892385(2025-04-14) chainstate를 `bitcoin-utxo-dump`로 분류. 이 책의 `data/exposed_output_types_2026-10-25.json` `comparison` 항목에 옮긴 값: p2pk count 45,257(0.03%)·value 1,720,270 BTC(8.70%); p2tr count 59,262,426(34.22%)·value 147,912 BTC(0.75%). 보고서 스스로 "will be updated to fix false identification of <37 byte scripts as P2MS outputs"라고 적었다.

22. Anthony Milton & Clara Shikhelman, *Bitcoin and Quantum Computing: Current Status and Future Directions* (Chaincode Labs, May 2025), https://chaincode.com/bitcoin-post-quantum.pdf (2026-10-25 접근, 55쪽). 원문: "P2PK, one of Bitcoin's original output types used for early mining rewards [BSE18], represents only ~0.025% of current UTXOs but locks a disproportionate ~8.68% of Bitcoin's value (~1,720,747 BTC) [Erh23]." / "Approximately 20-50% of all Bitcoin in circulation (4-10 million BTC) is potentially vulnerable to CRQC attacks." / "Project 11 reported that, as of mid January 2015, there were ~11.1 million BTC addresses with a non-zero balance and exposed public key for ~6.26 million BTC [PE25]." 업계 보고서(🟡)이며 이 책은 보조 출처로만 쓴다. 달러 환산액은 옮기지 않았다.

23. Deloitte(Itan Barmes, Bram Bosch, Olaf Haalstra), "Quantum computers and the Bitcoin blockchain," https://www.deloitte.com/nl/en/services/risk-advisory/perspectives/quantum-computers-and-the-bitcoin-blockchain.html (2026-10-25 접근; 본문에 작성일이 없고 페이지 메타데이터의 datePublished 는 2024-10-26 — 수치의 시점은 불명). 원문: "Nevertheless, there are still over 4 million BTC (about 25% of all Bitcoins) which are potentially vulnerable to a quantum attack." P2PK "circa 2M", 재사용 P2PKH "2.5M". CoinShares, "Quantum Vulnerability in Bitcoin: A Manageable Risk" (Published on Feb 6th, 2026), https://coinshares.com/insights/research-data/quantum-vulnerability-in-bitcoin-a-manageable-risk/ (2026-10-25 접근). 원문: "The exposure is limited to addresses where public keys are visible, primarily legacy Pay-to-Public-Key (P2PK) outputs, which hold approximately 1.6 million btc—about 8% of the total supply." 둘 다 업계 자료(🟡, 후자는 자산운용사)이고 보조 출처로만 쓴다. 두 글의 달러 환산과 시장 영향 서술은 옮기지 않았다.

24. BIP-361 "Post Quantum Migration and Legacy Signature Sunset" (Jameson Lopp 외 5인; Status: Draft; Type: Informational; Assigned 2026-02-11), 같은 커밋 `bip-0361.mediawiki`, Motivation 절. 원문: "As of March 1, 2026, over 34% of all bitcoin have revealed a public key on-chain; those UTXOs could be stolen by an attacker with a sufficiently powerful quantum computer." 산출 방법은 본문에 없다.

25. Babbush 외 (2026), 12쪽·14쪽(Figure 5 설명). 원문: "A little over 1.7 million bitcoin (nearly 9% of all bitcoin) is secured by P2PK locking scripts." / "At the time of writing ~6.9M total bitcoin across all protocols are vulnerable. Plot generated using data from bigquery-public-data.crypto bitcoin [139]." 재사용 판정 기준: "(Addresses that have appeared in an Input) AND (Currently hold a balance in Unspent Outputs)".

26. Aggarwal 외 (2018), 8쪽 §III.B. 원문: "1. (Reusing addresses) ... Any address that has bitcoin and for which the public key has been revealed is completely insecure. / 2. (Processed transactions) If a transaction is made from an address which has not been spent from before, and this transaction is placed on the blockchain with several blocks following it, then this transaction is reasonably secure against quantum attacks. ... / 3. (Unprocessed transactions) After a transaction has been broadcast to the network, but before it is placed on the blockchain it is at risk from a quantum attack. ..." / "We view item (3) as the most serious attack."

27. Babbush 외 (2026), 15쪽. 원문: "The timing and success probability of on-spend attacks are depicted in Figure 6. The probability of success is slightly less than 41% under the idealized assumptions stated in the caption." / "Reducing block interval makes on-spend attacks significantly harder. For example, we estimate the success probability of an on-spend attack using a superconducting qubit CRQC against Litecoin, a Bitcoin derivative which targets 2.5 minutes between blocks, to be less than 3%." 같은 쪽은 공격자와 소유자의 수수료 입찰 경쟁(RBF)과 「초토화(scorched earth)」 시나리오도 적지만, 이 책은 그 서술을 옮기지 않는다.

28. BIP-360 헤더(같은 커밋). 원문: "BIP: 360 / Layer: Consensus (soft fork) / Title: Pay-to-Merkle-Root (P2MR) / Authors: Hunter Beast, Ethan Heilman, Isabel Foxen Duke / Status: Draft / Type: Specification / Assigned: 2024-12-18 / License: BSD-3-Clause / Version: 0.12.1 / Requires: 340, 341, 342". 저장소 색인(README.mediawiki 같은 커밋 L1286~1291)도 Draft.

29. BIP-360 Abstract·Design 절. 원문: "This document proposes a new output type: Pay-to-Merkle-Root (P2MR), via a soft fork. P2MR outputs operate with nearly the same functionality as P2TR (Pay-to-Taproot) outputs, but with the key path spend removed." / "In other words, P2MR outputs commit to the Merkle root of a script tree without committing to an internal key. The script(s) being committed to, however, may contain a key or key-hash." / Address Format: "P2MR outputs use SegWit version 2, resulting in mainnet addresses that start with bc1z, following BIP 173."

30. BIP-360 Abstract·Security Considerations 절. 원문: "It is worth noting that proposed P2MR outputs are only resistant to "long exposure attacks" on elliptic curve cryptography; that is, attacks on keys exposed for time periods longer than needed to confirm a spending transaction. / Protection against more sophisticated quantum attacks, including protection against private key recovery from public keys exposed in the mempool while a transaction is waiting to be confirmed (a.k.a. "short exposure attacks"), may require the introduction of post-quantum signatures in Bitcoin." / "While this proposal does not include the introduction of post-quantum signature schemes, we think it's worth commenting on security considerations related to this possibility. Quantum-resistant signature algorithms (e.g. ML-DSA or SLH-DSA) offer different levels of protection and should be scrutinized before use."

31. BIP-360 「Transaction Size and Fees」 절. 원문: "P2MR witness for a depth-1 tree (135 bytes)" / "P2TR key path spend witness (66 bytes)" / "Thus, the P2MR witness would be 135 - 66 = 69 bytes larger than a P2TR key path spend witness." / "For this reason, a P2MR witness will be 32 bytes smaller than an equivalent P2TR script path spend witness, except at m = 7, where the two control blocks fall on opposite sides of the compact size boundary and the saving is 34 bytes."

32. BIP-360 Conclusion 절. 원문: "In this proposal, we adopt a "prepared not scared" approach to the possible advancement of quantum computing and offer Bitcoin users an option for increased protection if they so choose. This BIP does not take a position on any specific quantum computing timeline, but rather proposes a flexible and unobtrusive option for users that wish to mitigate this risk according to their own estimate of the timeline."

33. NIST, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203 (Published August 13, 2024), DOI 10.6028/NIST.FIPS.203, 초록: "This standard specifies a key-encapsulation mechanism called ML-KEM. The security of ML-KEM is related to the computational difficulty of the Module Learning with Errors problem. At present, ML-KEM is believed to be secure, even against adversaries who possess a quantum computer."; NIST, *Module-Lattice-Based Digital Signature Standard*, FIPS 204 (August 13, 2024), DOI 10.6028/NIST.FIPS.204, 초록: "This standard specifies ML-DSA, a set of algorithms that can be used to generate and verify digital signatures. ML-DSA is believed to be secure, even against adversaries in possession of a large-scale quantum computer."; NIST, *Stateless Hash-Based Digital Signature Standard*, FIPS 205 (August 13, 2024), DOI 10.6028/NIST.FIPS.205, 초록: "SLH-DSA is based on SPHINCS+, which was selected for standardization as part of the NIST Post-Quantum Cryptography Standardization process." 크기: FIPS 204 Table 2 "Sizes (in bytes) of keys and signatures of ML-DSA"(16쪽) — ML-DSA-44 private 2560 / public 1312 / signature 2420; ML-DSA-65 4032 / 1952 / 3309; ML-DSA-87 4896 / 2592 / 4627. FIPS 205 §11 Table 2 "SLH-DSA parameter sets" — SLH-DSA-SHA2-128s·SHAKE-128s pk 32 / sig 7 856; 128f pk 32 / sig 17 088; 192s pk 48 / sig 16 224; 192f pk 48 / sig 35 664; 256s pk 64 / sig 29 792. `nist-2024-fips-204-module-lattice-signature.pdf`, `nist-2024-fips-205-stateless-hash-signature.pdf`.

34. Aggarwal 외 (2018), 13쪽 §IV.B. 원문: "In the blockchain context the most important parameters of a signature scheme are the signature and public key lengths, as these must be stored in some capacity to fully verify transactions, and the time to verify the signature. Looking at Table II, with respect the sum of signature and public key lengths, the only reasonable options are hash and lattice based schemes." 같은 쪽: "Hash based schemes like XMSS have the advantage of having provable security, at least assuming the chosen hash function behaves like a random oracle. The generic quantum attack against these schemes is to use Grover's algorithm which means that their quantum security level is half of the classical security level."

35. BIP-361 헤더(같은 커밋). 원문: "BIP: 361 / Layer: Consensus (soft fork) / Title: Post Quantum Migration and Legacy Signature Sunset / Authors: Jameson Lopp, Christian Papathanasiou, Ian Smith, Joe Ross, Steve Vaile, Pierre-Luc Dallaire-Demers / Status: Draft / Type: Informational / Assigned: 2026-02-11 / License: BSD-3-Clause / Requires: TBD Post Quantum Signature BIP". 색인 README.mediawiki L1293~1298도 Draft.

36. BIP-361 Abstract·Specification 절. 원문: "Phase A: Disallows sending of any funds to quantum-vulnerable

addresses, hastening the adoption of PQ address types. / Phase B: Restricts ECDSA/Schnorr spends by encumbering them with a quantum-safe rescue protocol, preventing theft of funds in quantum-vulnerable UTXOs. This is triggered by a well-publicized flag-day five years after activation." 표: "A — Permitted sends are from legacy scripts to PQ scripts. — Everyone holding or accepting BTC. — 160,000 blocks (~3 years) after BIP-361 activation. / B — At a predetermined block height, nodes tighten requirements on verification of ECDSA/Schnorr. — Everyone holding or accepting BTC. — 2 years after Phase A activation."

37. BIP-361 Rationale 「Minimizes loss of access to funds」 절·Backward Compatibility 절. 원문: "Such rescue protocols rely on asymmetry of knowledge between a quantum attacker and the authentic coin-holder. Any context where an asymmetry exists in favor of the authentic holder can theoretically be turned into a rescue protocol." / "It remains to be seen how much of the legacy Bitcoin supply can be theoretically covered by such rescue protocols. If one or more rescue protocols can be designed to cover the majority of the Bitcoin supply, then restricting ECDSA/Schnorr verification will be at most mildly confiscatory, and will come at little expense to the overall integrity of the Bitcoin network." / "BIP-361 authors support Hourglass for P2PK because it's not currently believed possible to construct a rescue protocol for P2PK UTXOs, as no knowledge asymmetry is known."

38. ethereum.org, "A more secure Ethereum" (roadmap/security), 소스 `public/content/roadmap/security/index.md` 커밋 290d753cb2ffa47ce0cfc5e65c1539f367b82cde(2026-07-12), 「Quantum resistance」 절(2026-10-25 접근). 원문: "Four areas have been identified as requiring post-quantum upgrades: validator consensus signatures (BLS), the commitment schemes used for data availability (KZG), account signatures (ECDSA), and the ZK-proof systems used by rollups." 하위 페이지 "Post-quantum cryptography on Ethereum", 소스 `public/content/roadmap/security/quantum-resistance/index.md` 커밋 643a7a456984eacc8f07105e752a67256ac6bb96(2026-10-23).

39. ethereum.org roadmap/security(커밋 290d753c), 원문: "**No quantum computer can break Ethereum's cryptography today.** The hardware required does not yet exist at scale. But recent research suggests the gap is closing faster than previously expected. In March 2026, Google Quantum AI published a paper estimating that breaking 256-bit elliptic curve cryptography (the type Ethereum uses for account signatures) could require roughly 1,200 logical qubits, about 20 times fewer than earlier estimates." quantum-resistance 페이지(커밋 643a7a45): "Current quantum hardware is far from this scale, operating with a few thousand noisy physical qubits. Logical qubits (which correct for errors and perform reliable computation) require many physical qubits each."

40. ethereum.org quantum-resistance 페이지(커밋 643a7a45), 「3. Account signatures: ECDSA」 절. 원문: "Rather than a single protocol-wide migration, Ethereum plans to use account abstraction (specifically EIP-8141, being considered for Hegotá in second half of 2026) to give users **signature agility**. Individual accounts could switch to a post-quantum signature scheme without waiting for the entire protocol to change." roadmap/security 페이지(커밋 290d753c): "EIP-8141 is being considered for the Hegotá upgrade. Core post-quantum infrastructure milestones target completion by approximately 2029. These are planning targets and may shift."

41. ethereum.org quantum-resistance 페이지(커밋 643a7a45), 「1. Consensus-layer BLS signatures」 절. 원문: "leanXMSS: Ethereum will replace BLS signatures with leanXMSS, a hash-based signature scheme for validators. Hash-based signatures are considered quantum-safe because they rely only on the security of hash functions, which quantum computers weaken but do not break." / "Because hash-based signatures are significantly larger (roughly 3,000 bytes compared to 96 bytes for BLS), switching to leanXMSS would produce significantly more data per slot. To solve this, leanVM acts as an aggregation engine, compressing the data by 250x."

42. Ethereum Foundation Post-Quantum team, "Post-Quantum Ethereum," https://pq.ethereum.org/ (Last updated: June 27, 2026; 2026-10-25 접근). 원문: "We do not believe a cryptographically relevant quantum computer is imminent. But migrating a decentralized, global protocol takes years of coordination, engineering, and formal verification. The work must begin well before the threat arrives." / FAQ: "No single fixed date. Based on our team's current assessment, L1 protocol upgrades could be completed by 2029, with full execution-layer migration taking additional years beyond that." FAQ 서두의 면책: "The Foundation does not set protocol direction … Protocol direction and upgrades are determined through open community governance processes (e.g., All Core Devs), not by any single team or organization."

43. pq.ethereum.org FAQ 「What about long-dormant funds and abandoned wallets?」. 원문: "Accounts that have never revealed a public key are not directly exposed. However, the question of what should be done about funds that cannot or will not migrate is ultimately a community governance question. Two natural scenarios exist: do nothing, or freeze vulnerable coins. Unlike Bitcoin, where roughly 5% of the supply is associated with early address formats widely believed to be abandoned (including Satoshi's ~1M BTC), Ethereum's equivalent long-dormant exposure is closer to 0.1% (an EF estimate)."

44. Babbush 외 (2026), 36쪽 §VII·§VIII.A. 원문: "The P2PK locking scripts have seen little use since 2009 and a majority of the 1.7 millions bitcoin have not been transacted since 2009 despite the fact that such coins are known to be vulnerable to at-rest quantum attacks. This leads to the widespread assumption that (in most cases), the private keys of these Bitcoin UTXOs have been lost. For example, accounts believed to have been owned by Satoshi Nakamoto hold about 1 million bitcoin [293, 294] behind these P2PK scripts ...". 같은 쪽: "Thus, there is no obvious way to update Bitcoin in a fashion that would keep lost P2PK coins and other dormant assets spendable by their owners and also protected against quantum computers."

45. Babbush 외 (2026), 37쪽 §VIII.B.1. 원문: "The Bitcoin community is considering three general categories of approaches to dealing with dormant assets [26]: Do Nothing: the protocol is kept unchanged allowing quantum attackers to acquire the dormant assets. Burn: the protocol is modified to render dormant assets unspendable after a certain date, effectively confiscating all assets locked by P2PK scripts. Hourglass [303]: the protocol is modified to limit the rate at which dormant assets may be spent." 같은 절은 네 번째 안("Bad Sidechain" — 체인 밖 소유 증명을 받아 주는 전용 사이드체인)을 저자들 자신의 제안으로 덧붙인다(39쪽). 이 책은 그 제안을 평가하지 않는다.

46. BIP-361 Rationale 절. 원문: "Any proposal that allows for the quantum theft of "lost" bitcoin is creating a redistribution dilemma. There are 3 types of proposals: 1. Allow anyone to steal vulnerable coins, benefiting those who reach quantum capability earliest. 2. Allow throttled theft of coins, which leads to RBF battles and ultimately miners subsidizing their revenue from quantum recovered coins. 3. Allow no one to steal vulnerable coins." BIP-360 「Other Methods」 절은 소각을 "burn vulnerable coins as proposed by Jameson Lopp and others in QBIP"로, 속도 제한을 "members of our team have separately proposed Hourglass"로 적는다.

47. Babbush 외 (2026), 38~39쪽. 원문: "Consensus remains elusive. Indeed, different members of the Bitcoin community have different philosophical beliefs and financial incentives." / "Indeed, an informal poll at 2025 Presidio Bitcoin Quantum Summit in San Francisco saw roughly equal support for each of the three categories of solutions [26]." 같은 쪽: "From a technical perspective, both Burn and Hourglass proposals could be implemented as a soft fork since banning or limiting transactions that spend dormant coins is a narrowing of the protocol rules. However, this requires broad support in the community."

48. Babbush 외 (2026), 41쪽 §VIII.C.2. 원문: "Bitcoin's ownership model (as well as that of many other cryptocurrencies) is based on the principle that the owner of any given asset is anyone possessing the knowledge of the private key corresponding to the public key to which the asset is locked on the ledger. This principle rests on the assumption that the private key cannot be computed by anyone else. However, a CRQC can quickly derive a private key from any public key, thus breaking the foundation of Bitcoin's model of ownership."

49. Babbush 외 (2026), 39~42쪽 §VIII.C. 「디지털 인양」: "One public policy option treats dormant assets as effectively abandoned property subject to regulated digital salvage (akin to "sunken treasure"). ... Furthermore, principles such as escheatment [320], treasure trove, adverse possession, and maritime salvage offer general precedent for regulated recovery of abandoned property." RUUPA: "In particular, RUUPA deals with assets in possession and control of a business entity called a "holder" and distinct from the assets' owner. However, no party involved in the operation of the Bitcoin blockchain clearly meets the legal requirements to be the "holder" of the dormant coins." laches: "Another general legal principle that may apply is the doctrine of laches [321]; if an owner of dormant coins has known for years that their assets are at risk and has failed to transact them to a post-quantum address, then they may be deemed to have failed to assert their rights through inaction." 「파괴 불가」: "Due to the cryptographic relationships between the blocks on the chain, rewriting the contents of a block requires changing all the blocks that follow it. ... The computational cost of doing so is equivalent to the computational cost of re-mining more than a decade of Bitcoin's history." / "If a government banned mining transactions spending dormant assets, then foreign miners outside cooperating jurisdictions could continue to include them in their blocks." 이 책은 이 논증들을 옮기기만 하고 평가하지 않는다. 미국 통일 미청구재산법·각 법리의 원문은 이 책이 열지 않았다.

6부 열린 문제

30장 명명 문제 — 가상자산·crypto-asset·digital asset·暗号資産·cryptoasset

핵심 TAKEAWAY

같은 비트열을 한국은 「가상자산」, EU는 「crypto-asset」, 미국은 맥락에 따라 「security」·「commodity」·「property」·「digital asset」, 일본은 「暗号資産」, 영국은 「cryptoasset」이라 부릅니다. 이 장은 다섯 법역의 정의 조문을 **원문 그대로** 나란히 놓고, 조문마다 법률 번호·시행일·기준일을 붙입니다. 그리고 그 이름이 무엇을 가르는지를 셋으로 봅니다 — **관할**(어느 기관이 보는가), **과세**(무엇으로 세금을 매기는가), **형사 적용**(어느 죄의 객체가 되는가). 기술은 이름을 정하지 않습니다. 1부가 보인 것은 80바이트 헤더와 서명과 상태 전이이고, 그것을 「통화」로 부르느냐 「자산」으로 부르느냐 「증권」으로 부르느냐는 전부 법의 선택입니다. 그 선택은 한 법역 안에서도 바뀌었습니다 — 일본은 「仮想通貨」를 「暗号資産」으로 고쳤고, 한국 대법원은 2018년에 「가상화폐」, 2021년에 「가상자산」이라 썼으며, 미국 국세청은 「법정통화인 법역이 없다」는 문장을 2023년에 지웠습니다. 이 장은 부록 C 대역표의 본문판이고, 27.3의 표와 수치·조문이 어긋나지 않게 썼습니다.

학습목표

1. 다섯 법역의 정의 조문을 원문으로 읽고, 각 조문이 **무엇을 넣고 무엇을 빼는지**(제외 목록)를 비교한다.
2. 「통화/화폐」·「자산」·「증권」·「상품」·「재산」이라는 다섯 범주어가 각 법역에서 어느 기관의 관할, 어느 세법의 틀, 어느 죄의 객체를 부르는지 가린다.
3. 한 법역 안에서 이름이 바뀐 세 사례(일본 2020, 한국 대법원 2018→2021, 미국 IRS 2023)를 조문·판결 원문으로 확인한다.
4. 정의가 기술을 따라가지 못하는 자리 — 대체불가 토큰, 자동화된 시장, 발행자 없는 자산 — 를 조문의 문언에서 찾는다.

[입문]**만 읽은 독자가 가져가는 것.** 같은 물건을 「통화」로 부르느냐 「자산」으로 부르느냐가 그 물건의 법적 운명을 바꿉니다. **기술은 이름을 정하지 않습니다.**

1. 왜 이름이 문제인가

12장에서 제네시스 블록 80바이트를 손으로 풀었습니다. 그 안에는 버전·이전 해시·머클 루트·시각·비트·논스가 있었고, 「통화」라는 글자도 「자산」이라는 글자도 없었습니다. 11장의 UTXO는 「쓰지 않은 거래 출력」이고 13장의 계정 잔고는 상태 트리의 숫자입니다. 이 책이 1~3부에서 보인 것은 전부 그런 것들 — 비트열, 서명, 상태 전이, 발행 규칙, 균형 — 이고, 그것들에 이름을 붙인 것은 법입니다. 그런데 법마다 다른 이름을 붙였고, 그 이름이 어느 기관의 관할·어느 세법·어느 죄를 부르는지를 가릅니다. 26장의 네 사건에서 그것이 보였습니다. 같은 토큰이 미국에서는 「security」로 판단되어 증권법이 들어오고(The DAO·Terra), 「commodity」로 판단되어 상품거래법이 들어오며, 한국 형사법에서는 「재산상 이익」이어서 사기의 객체가 되지만, 횡령의 객체인 「재물」인지는 대법원이 판단하지 않았습니다. 이 장은 그 이름들의 **조문 원문을** 나란히 놓습니다. 부록 C의 대역표가 행과 열로 압축한 것을 여기서는 문장으로 읽습니다.

조문을 읽는 규칙 셋입니다. 첫째, **원문 그대로** 옮기고 번역은 붙이지 않습니다 — 번역은 해석이고 이 책은 해석하지 않습니다(이 책의 원칙: 영역본으로 조문을 단정하지 않는다). 둘째, 조문마다 법률 번호와 시행일을 붙이고 기준일(2026-10-25) 현재 **시행 중인지를** 적습니다. 셋째, 정의 조문의 **제외 목록을** 본문과 같은 무게로 읽습니다 — 무엇을 빼는가가 그 법역이 이 물건을 어디에 세우는지를 말합니다.

2. 다섯 법역의 정의 조문 — 원문

2.1 한국 — 「가상자산」

「가상자산 이용자 보호 등에 관한 법률」 제2조 제1호. 법률 제19563호로 제정(공포 2023. 7. 18., 시행 2024. 7. 19.)된 문언이고, 현행 법률 제21857호(시행 2026. 10. 2.)에서도 바뀌지 않았습니다. **2026-10-25 시행 중.**[1]

> 1. "가상자산"이란 경제적 가치를 지닌 것으로서 전자적으로 거래 또는 이전될 수 있는 전자적 증표(그에 관한 일체의 권리를 포함한다)를 말한다. 다만, 다음 각 목의 어느 하나에 해당하는 것은 제외한다. 가. 화폐 · 재화 · 용역 등으로 교환될 수 없는 전자적 증표 또는 그 증표에 관한 정보로서 발행인이 사용처와 그 용도를 제한한 것 나. 「게임산업진흥에 관한 법률」 제32조제1항제7호에 따른 게임물의 이용을 통하여 획득한 유 · 무형의 결과물 다. 「전자금융거래법」 제2조제14호에 따른 선불전자지급수단 및 같은 조 제15호에 따른 전자화폐 라. 「주식 · 사채 등의 전자등록에 관한 법률」 제2조제4호에 따른 전자등록주식등 마. 「전자어음의 발행 및 유통에 관한 법률」 제2조제2호에 따른 전자어음 바. 「상법」 제862조에 따른 전자선하증권 사. 「한국은행법」에 따른 한국은행(이하 "한국은행"이라 한다)이 발행하는 전자적 형태의 화폐 및 그와 관련된 서비스 아. 거래의 형태와 특성을 고려하여 대통령령으로 정하는 것

아목의 위임을 받은 시행령 제2조는 다섯 호의 제외 목록을 두고, 그 제4호가 대체불가 토큰에 닿습니다(3절·각주 15).

같은 문언이 먼저 들어온 곳은 「특정 금융거래정보의 보고 및 이용 등에 관한 법률」 제2조 제3호였습니다. 법률 제17113호(공포 2020. 3. 24., 시행 2021. 3. 25.)가 정의 본문을 직접 두었고, 2024. 7.

19. 가상자산법 부칙으로 지금의 준용 문언이 되었습니다.[2]

> (법률 제17113호 신설 당시, 시행 2021. 3. 25. ~ 2024. 7. 18.) 3. "가상자산"이란 경제적 가치를 지닌 것으로서 전자적으로 거래 또는 이전될 수 있는 전자적 증표(그에 관한 일체의 권리를 포함한다)를 말한다. 다만, 다음 각 목의 어느 하나에 해당하는 것은 제외한다. 가. ~ 바. (위와 같음) 사. 거래의 형태와 특성을 고려하여 대통령령으로 정하는 것
>
> (현행) 3. "가상자산"이란 「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산을 말한다.

과세 조문은 소득세법 제21조 제1항 제27호입니다. 법률 제17757호(공포 2020. 12. 29.)로 신설되었고 시행일이 세 번 미뤄져 **2027. 1. 1. 시행 — 2026-10-25 미시행입니다.**[3]

> 27. 「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산(이하 "가상자산"이라 한다)을 양도하거나 대여함으로써 발생하는 소득(이하 "가상자산소득"이라 한다)

2.2 EU — 「crypto-asset」

Regulation (EU) 2023/1114 (MiCA), Art. 3(1), points (5)~(9). 관보 L 150(2023. 6. 9.) 63쪽. 제149조에 따라 **2024-12-30부터 적용**(Title III·IV는 2024-06-30부터). **2026-10-25 적용 중.**[4]

> (5) 'crypto-asset' means a digital representation of a value or of a right that is able to be transferred and stored electronically using distributed ledger technology or similar technology; (6) 'asset-referenced token' means a type of crypto-asset that is not an electronic money token and that purports to maintain a stable value by referencing another value or right or a combination thereof, including one or more official currencies; (7) 'electronic money token' or 'e-money token' means a type of crypto-asset that purports to maintain a stable value by referencing the value of one official currency; (8) 'official currency' means an official currency of a country that is issued by a central bank or other monetary authority; (9) 'utility token' means a type of crypto-asset that is only intended to provide access to a good or a service supplied by its issuer;

제외 목록은 제2조에 있습니다.

> 3. This Regulation does not apply to crypto-assets that are unique and not fungible with other crypto-assets.
> 4. This Regulation does not apply to crypto-assets that qualify as one or more of the following: (a) financial instruments; (b) deposits, including structured deposits; (c) funds, except if they qualify as e-money tokens; (d) securitisation positions …; (e) non-life or life insurance products …; (f) pension products …; (g) officially recognised occupational pension schemes …; (h) individual pension products …; (i) a pan-European Personal Pension Product …; (j) social security schemes ….

2.3 미국 — 단일 정의 없음

미국에는 「암호자산」에 해당하는 단일 정의 조문이 없고, 같은 물건이 어느 법의 어느 범주어에 들어가는지를 법원과 기관이 거래의 모양에 따라 판단합니다. 원문으로 확인한 범주어는 다섯입니다.

(가) 「investment contract」 — 증권. 증권법은 「security」에 「investment contract」를 넣고, 연방대법원은 1946년 그 뜻을 이렇게 정했습니다. **판례법, 2026-10-25 유효.**[5]

> an investment contract for purposes of the Securities Act means a contract, transaction or scheme whereby a person invests his money in a common enterprise and is led to expect profits solely from the efforts of the promoter or a third party, it being immaterial whether the shares in the enterprise are evidenced by formal certificates or by nominal interests in the physical assets employed in the enterprise.

(나) 「commodity」 — 상품. 상품거래법의 정의 문언을 2018년 미국 연방뉴욕동부지방법원이 이렇게 옮기고 적용했습니다. **지방법원 결정, 2026-10-25 기준 — 상급심 경과 미확인.**[6]

> They fall well-within the common definition of "commodity" as well as the CEA's definition of "commodities" as "all other goods and articles . . . in which contracts for future delivery are presently or in the future dealt in." Title 7 U.S.C. § 1(a)(9).

(다) 「property」 — 세법상 재산. 국세청 고시 2014-21(2014-16 I.R.B. 938). **행정 해석, 2023-34로 배경 문장만 수정, 2026-10-25 유효.**[7]

> Virtual currency is a digital representation of value that functions as a medium of exchange, a unit of account, and/or a store of value. In some environments, it operates like "real" currency -- i.e., the coin and paper money of the United States or of any other country that is designated as legal tender, circulates, and is customarily used and accepted as a medium of exchange in the country of issuance -- but it does not have legal tender status in any jurisdiction. (2014-21 Background 원문; 2023-34가 세 번째 문장을 아래로 고침)
>
> In certain contexts, virtual currency may serve one or more of the functions of "real" currency -- i.e., the coin and paper money of the United States or of any other country that is designated as legal tender, circulates, and is customarily used and accepted as a medium of exchange in the country of issuance -- but the use of virtual currency to perform "real" currency functions is limited. (2023-34 §3)
>
> Q-1: How is virtual currency treated for federal tax purposes? A-1: For federal tax purposes, virtual currency is treated as property. General tax principles applicable to property transactions apply to transactions using virtual currency. (2014-21 §4)

(라) 「digital asset」·「payment stablecoin」 — 연방 성문법. GENIUS Act, Pub. L. 119-27, 139 Stat. 419 (July 18, 2025), §2(6)·(22). **제정 완료, 시행일은 §20에 따라 늦어도 2027-01-18이고 최종 규칙 공표에**

따라 더 이를 수 있음 — 2026-10-25 현재 규칙 공표 여부 미확인.[8]

> (6) DIGITAL ASSET.—The term "digital asset" means any digital representation of value that is recorded on a cryptographically secured distributed ledger. (22) PAYMENT STABLECOIN.—The term "payment stablecoin"— (A) means a digital asset— (i) that is, or is designed to be, used as a means of payment or settlement; and (ii) the issuer of which— (I) is obligated to convert, redeem, or repurchase for a fixed amount of monetary value, not including a digital asset denominated in a fixed amount of monetary value; and (II) represents that such issuer will maintain, or create the reasonable expectation that it will maintain, a stable value relative to the value of a fixed amount of monetary value; and (B) does not include a digital asset that— (i) is a national currency; (ii) is a deposit (as defined in section 3 of the Federal Deposit Insurance Act (12 U.S.C. 1813)), including a deposit recorded using distributed ledger technology; or (iii) is a security, as defined in section 2 of the Securities Act of 1933 (15 U.S.C. 77b), section 3 of the Securities Exchange Act of 1934 (15 U.S.C. 78c), or section 2 of the Investment Company Act of 1940 (15 U.S.C. 80a–2), except that, for the avoidance of doubt, no bond, note, evidence of indebtedness, or investment contract that was issued by a permitted payment stablecoin issuer shall qualify as a security solely by virtue of its satisfying the conditions described in subparagraph (A), consistent with section 17 of this Act.

(마) 「controllable electronic record」 — 사법(私法) 모델법. Uniform Commercial Code Amendments (2022), §12-102(a)(1). **모델법, 주별 채택 여부 미확인.**[9]

> "Controllable electronic record" means a record stored in an electronic medium that can be subjected to control under Section 12-105. The term does not include a controllable account, a controllable payment intangible, a deposit account, an electronic copy of a record evidencing chattel paper, an electronic document of title, electronic money, investment property, or a transferable record.

2.4 일본 — 「暗号資産」

資金決済に関する法律(平成21年法律第59号) 第2条第14項. 「仮想通貨」였던 정의어를 令和元年法律第28号(公布 2019-06-07, 施行 2020-05-01)가 「暗号資産」으로 바꿨고, 2023-06-01 시행 판본부터 第5項에서 第14項으로 옮겨졌습니다. **2026-10-25 시행 중.**[10]

> この法律において「暗号資産」とは、次に掲げるものをいう。ただし、
> 金融商品取引法第二十九条の二第一項第八号に規定する権利を表示するものを除く。 一
> 物品等を購入し、若しくは借り受け、又は役務の提供を受ける場合に、
> これらの代価の弁済のために不特定の者に対して使用することができ、かつ、
> 不特定の者を相手方として購入及び売却を行うことができる財産的価値
> （電子機器その他の物に電子的方法により記録されているものに限り、本邦通貨及び外国通貨、
> 通貨建資産並びに電子決済手段（通貨建資産に該当するものを除く。）を除く。
> 次号において同じ。）であって、電子情報処理組織を用いて移転することができるもの 二
> 不特定の者を相手方として前号に掲げるものと相互に交換を行うことができる財産的価値であって、電子情報処理組織を用いて移転することができるもの

명칭 변경 전후의 첫 문장입니다.

> (2020-04-30 판본, 第2条第5項) この法律において「仮想通貨」とは、次に掲げるものをいう。
> (2020-05-01 판본, 第2条第5項) この法律において「暗号資産」とは、次に掲げるものをいう。
> ただし、金融商品取引法（昭和二十三年法律第二十五号）
> 第二条第三項に規定する電子記録移転権利を表示するものを除く。

단서가 제외하는 「電子記録移転権利」는 金融商品取引法 第2条第3項 괄호 안에 있습니다.

> …電子情報処理組織を用いて移転することができる財産的価値
> （電子機器その他の物に電子的方法により記録されるものに限る。）に表示される場合
> （流通性その他の事情を勘案して内閣府令で定める場合を除く。）に限る。以下
> 「電子記録移転権利」という。）…

그리고 令和8年法律第64号(公布 2026-07-23)는 이 정의를 金商法 第2条第49項으로 옮기는 개정입니다. **공포·일부 시행되었으나 정의 이관 부분은 2026-10-25 미시행.** 미시행 판본의 첫 문장입니다.

> (金融商品取引法 第2条第49項, 미시행 판본) この法律において「暗号資産」とは、
> 次に掲げるものをいう。ただし、
> 第二十九条の二第一項第八号に規定する権利を表示するものを除く。

2.5 영국 — 「cryptoasset」

Financial Services and Markets Act 2000, s. 417(1). Financial Services and Markets Act 2023 (c. 29), s. 69(4)(a)로 삽입, **2023-08-29 시행(SI 2023/779, reg. 4(uu)). 2026-10-25 시행 중.**[11]

> "cryptoasset" means any cryptographically secured digital representation of value or contractual rights that— (a) can be transferred, stored or traded electronically, and (b) that uses technology supporting the recording or storage of data (which may include distributed ledger technology).

규제 활동의 대상이 되는 범위는 2026년 규정이 Regulated Activities Order에 삽입한 두 조문이 정합니다. The Financial Services and Markets Act 2000 (Cryptoassets) Regulations 2026, SI 2026/102, made 2026-02-04. **본격 시행일(full commencement day) 2027-10-25 — 2026-10-25 현재 FCA 규칙 제정을 위한 부분만 발효.**[12]

> 88F.—(1) A qualifying cryptoasset. (2) A "qualifying cryptoasset" means a cryptoasset which is— (a) fungible, (b) transferable, (c) not solely a record of value or contractual rights, including rights in another cryptoasset, and (d) not excluded by paragraph (4). (3) For the purposes of paragraph (2)(b), the circumstances in which a cryptoasset is to be treated as "transferable" include where it confers transferable rights. (4) A cryptoasset does not fall within paragraph (1) if it is— (a) a specified investment cryptoasset, other than one specified by— (i) article 74A (electronic money); and (ii) article 88F (qualifying cryptoassets); (b) electronic money; (c) currency of the United Kingdom or any other country or territory, including a central bank digital currency; (d) a cryptoasset that— (i) cannot be transferred or sold in exchange for money or other cryptoassets, except by way of redemption with the issuer, and (ii) can only be used by the holder— (aa) to acquire goods or services from the issuer, or (bb) to acquire goods or services within a limited network of service providers which have direct commercial agreements with the issuer. 88G.—(1) A qualifying stablecoin. (2) A "qualifying stablecoin" means a qualifying cryptoasset where— (a) that cryptoasset seeks or purports to maintain a stable value in relation to a particular fiat currency ("the referenced fiat currency"), and (b) fiat currency (which may be referenced fiat currency referred to in sub-paragraph (a)) or other assets are held for the purpose of maintaining a stable value, in accordance with sub-paragraph (a). (3) Where a cryptoasset seeks or purports to maintain a stable value in relation to an asset other than a fiat currency, the cryptoasset is not to be regarded as falling within paragraph (2)(a) even if the asset is expressed in terms of a fiat currency. [(4)·(5) 생략]

재산법 쪽의 조문은 정의가 아니라 **확인입니다**. Property (Digital Assets etc) Act 2025 (c. 29), s. 1. **2025-12-02 국왕 재가·같은 날 시행(England and Wales, Northern Ireland).**[13]

> A thing (including a thing that is digital or electronic in nature) is not prevented from being the object of personal property rights merely because it is neither— (a) a thing in possession, nor (b) a thing in action.

2.6 국제 — 「virtual asset」

다섯 법역 밖의 참조점으로 자금세탁방지기구(FATF)의 정의를 둡니다. 2021년 10월 28일 갱신 지침의 용어집입니다. **권고 — 법적 구속력 없음.**[14]

> A virtual asset is a digital representation of value that can be digitally traded, or transferred, and can be used for payment or investment purposes. Virtual assets do not include digital representations of fiat currencies, securities and other financial assets that are already covered elsewhere in the FATF Recommendations.

3. 조문을 겹쳐 읽기 — 넣은 것과 뺀 것

여섯 정의를 겹치면 공통 뼈대가 하나 보입니다. 「**가치(또는 권리)의 디지털 표현**」 + 「**전자적 이전 가능성**」. 한국은 「경제적 가치를 지닌 것으로서 전자적으로 거래 또는 이전될 수 있는 전자적 증표」, EU는 「a digital representation of a value or of a right that is able to be transferred and stored electronically」, 미국 GENIUS법은 「any digital representation of value that is recorded on a cryptographically secured distributed ledger」, 일본은 「財産的価値…であって、電子情報処理組織を用いて移転することができるもの」, 영국은 「digital representation of value or contractual rights that … can be transferred, stored or traded electronically」, FATF는 「digital representation of value that can be digitally traded, or transferred」. 1부가 보인 기술 가운데 조문이 집어 든 것은 **이전 가능성**뿐이고, 해시·서명·합의 규칙·최종성은 어느 정의에도 없습니다. EU와 영국과 미국 GENIUS법만이 「distributed ledger technology」·「cryptographically secured」라는 기술 낱말을 넣었고, 한국과 일본은 기술 중립입니다.

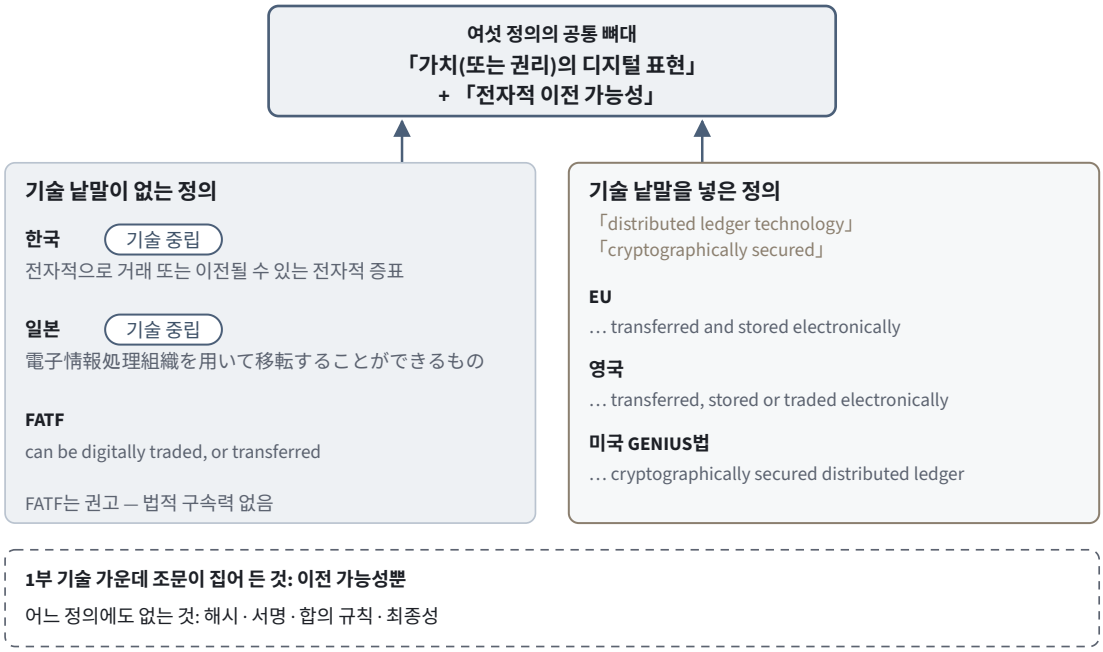


그림 30-1. 여섯 정의를 겹친 공통 뼈대와 기술 낱말의 유무. 기술 낱말을 넣은 것은 EU·영국·미국 GENIUS법뿐이고, 1부 기술 가운데 조문이 집어 든 것은 이전 가능성 하나다. 각 법역 아래의 구절은 본문이 인용한 조문의 일부다

갈라지는 자리는 **제외 목록입니다.**

뺀 것	한국	EU	미국 (GENIUS)	일본	영국 (88F)
법정통화·중앙은행 디지털 화폐	사목(한국은행 발행 전자적 화폐)	(8) 「official currency」는 별개 개념(정의에 제외 조항 없음)	(B)(i) national currency — payment stablecoin 정의에만; (6) digital asset에는 제외 없음	本邦通貨及び外国通貨, 通貨建資産	(4)(c) currency … including a central bank digital currency
전자화폐·선불수단	다목	(4)(c) funds(단, EMT는 포함)	— (예금은 (B)(ii)로 따로 뺌)	電子決済手段(第5項으로 분리)	(4)(b) electronic money
증권·금융상품	라목(전자등록주식등)·마목·바목	(4)(a) financial instruments	(B)(iii) security	단서: 金商法 第29条の2第1項第8号의 권리(電子記録移転権利)	(4)(a) specified investment cryptoasset
용도 한정 토큰	가목(발행인이 사용처·용도를 제한)	— (유틸리티토큰은 포함, (9))	—	—	(4)(d) 발행자 상환만 가능·한정 네트워크
게임 결과물	나목	—	—	—	—
대체불가(NFT)	아목 → 시행령 §2(4)(금융위원회가 고시하는 것 — 예: 지급수단으로 사용될 수 있는 증표 — 은 단서로 다시 포함)	Art. 2(3) unique and not fungible	—	—	(2)(a) fungible 요건
위임	아목(대통령령)	—	—	—	—

이 표에서 세 가지가 읽힙니다. 첫째, **「통화」는 어디에서나 밖에 있습니다.** 한국·일본·영국은 법정통화와 그 디지털 형태를 정의에서 명시적으로 빼고, 미국은 「payment stablecoin」 정의에서 「national currency」를 빼며, EU는 「official currency」를 별개 개념으로 둡니다. 일본은 이름 자체를 「仮想通貨」에서 「暗号資産」으로 바꿨습니다. 19장이 모형으로 보인 「법정통화를 가르는 것은 세금 수요의 바닥」이라는 명제와 짝을 이루는 입법의 선택입니다 — 법은 자기가 발행하는 것에만 「통화」라는 이름을 남겼습니다. 둘째, **증권과의 경계는 모두 긋되 방법이 다릅니다.** 한국·EU·영국·GENIUS법은 「증권이면 이 법 밖」이라고 적고, 일본은 「電子記録移転権利」라는 금융상품법의 개념을 단서로 끌어와 뺍니다. 미국의 증권법 자체는 거꾸로 **「투자계약」이면 이 법 안**이라고 하여, 같은 토큰이 거래의 모양에 따라 증권이 되기도 안 되기도 합니다(*SEC v. Ripple Labs, Inc.*, No. 1:20-cv-10832 (AT) (S.D.N.Y. July 13, 2023), ECF No. 874, 15·23·34쪽 — 27장 각주 19). 셋째, **대체불가 토큰은 세 법역이 뺐되 층이 다릅니다.** EU는 규정 본문(Art. 2(3))에서, 영국은 규제 활동 정의(art. 88F(2)(a) 「fungible」)에서 뺐고, 한국은 법 제2조 제1호 아목의 위임을 받은 시행령 제2조 제4호가 「단일하게 존재하여 다른 전자적 증표로 대체할 수 없는 전자적 증표」를 빼되 「특정 재화나 서비스의 지급수단으로 사용될 수 있는 전자적 증표 등 금융위원회가 정하여 고시하는 전자적 증표」는 다시 넣는 단서를 두었습니다.[15] 14장이 보인 ERC-721의 (계약 주소, 번호) 쌍이 「지급수단으로 사용될 수 있는」 것으로서 금융위원회 고시에 드는지는 그 토큰이 무엇을 가리키는가와 고시에 달려 있고 — 「정의 조문이 기술을

따라가지 못할 때」라고 설계가 적은 자리의 한 예입니다.

4. 이름이 가르는 것 — 관할·과세·형사

관할

이름은 먼저 **누가 보는가를** 정합니다. 한국에서 「가상자산」은 금융위원회의 감독·조사(가상자산법 제13·14조)와 금융정보분석원장의 신고(특정금융정보법 제7조)를 부르고, 「전자등록주식등」이면 그 법 밖으로 나가 자본시장 쪽으로 갑니다. EU에서 「crypto-asset」은 회원국 관할당국(MiCA Art. 93)과 — 유의미한 자산준거·전자화폐토큰이면 — EBA(Art. 117)를 부르고, 「financial instrument」면 MiCA 밖입니다. 일본에서 「暗号資産」은 내각총리대신(금융청) 등록(資金決済法 第63条の2)을, 「電子記録移転権利」는 金商法을 부릅니다. 영국에서 「qualifying cryptoasset」은 2027년부터 FCA의 규제 활동을 부릅니다. 미국에서는 이름을 **기관이 고릅니다** — 「security」면 SEC, 「commodity」면 CFTC이고, 2018년 결정은 그 둘이 「concurrent」라고 했습니다.[16] 26장의 Terra는 SEC가 「security」로 소추해 배심 평결을 받은 사례이고, The DAO는 SEC가 「security」로 판단하고 집행 조치는 하지 않은 사례입니다. 15장의 AMM과 17장의 MEV가 보인 「운영자 없는 시장」은 이 어느 이름에도 수범자가 없다는 점에서 관할의 빈자리이고(28.1(e)), GENIUS법 제2조 제7호가 유동성 풀 참여를 「digital asset service provider」에서 명시적으로 뺀 것은 입법이 그 빈자리를 알고 있었다는 사실입니다.

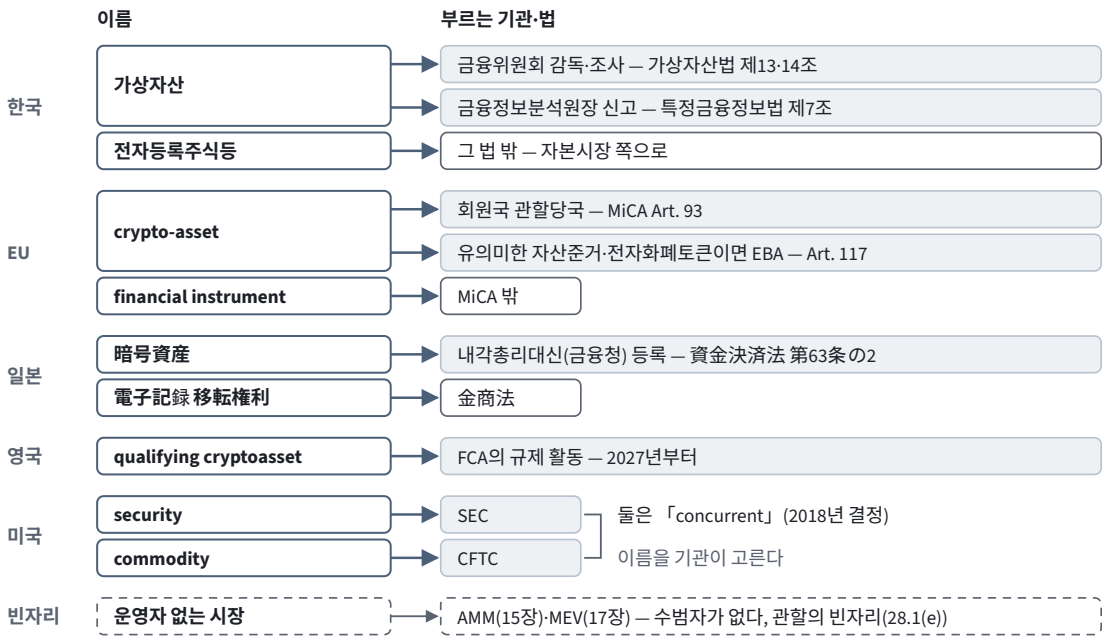


그림 30-2. 법역별로 이름이 부르는 감독 기관과 법. 이름이 먼저 누가 보는가를 정하고, 미국에서는 이름을 기관이 고른다. 이름을 짚으면 그 이름이 부르는 곳만 남는다. 운영자 없는 시장은 어느 이름에도 수범자가 없는 빈자리다

과세

이름은 다음으로 **무엇으로 세금을 매기는가를** 정합니다. 미국 국세청은 2014년 「virtual currency」를 「property」로 다루기로 했고, 그래서 재산 거래의 일반 원칙 — 취득가액과 양도가액의 차이 — 이 적용되며, 「currency」가 아니므로 외환 손익의 틀은 들어오지 않습니다(Q&A-2).[17] 이름을 「통화」로 두었다면 다른 조문이 들어왔을 것입니다. EU 부가가치세에서는 거꾸로 「통화」 쪽으로 읽었습니다 — 유럽연합 사법재판소는 2015년 비트코인이 「no purpose other than to be a means of payment」이고 「tangible property」가 아니라고 하면서, 전통 통화와의 환전을 「currency, bank notes and coins used as legal tender」에 관한 거래의 면제 조항(Art. 135(1)(e))에 넣었습니다. 판결은 환전을 용역의 공급(주문 1)으로 보고 그 면제 여부를 판단했습니다.[18] 한국은 「가상자산소득」을 기타소득의 한 호로 두었고(소득세법 제21조 제1항 제27호), 그 조문은 시행 전입니다. 같은 비트열이 미국에서는 재산, EU 부가가치세에서는 지급수단, 한국에서는 (시행되면) 기타소득의 원천입니다. 기술은 셋 가운데 어느 것도 고르지 않았습니다.

형사 적용

이름은 마지막으로 **어느 죄의 객체가 되는가를** 정합니다. 한국 대법원은 비트코인을 「재산상 이익」이라 했고(2021. 11. 11. 선고 2021도9855), 그래서 사기죄의 객체가 되지만 「재물」인지는 판단하지 않았으며, 원인 없이 이체받은 사람을 착오송금 판례를 유추해 신의칙을 근거로 배임죄로 처벌하는 것은 「명문의 규정이 없는 현재의 상황에서」 죄형법정주의에 반한다고 했습니다(2021. 12. 16. 선고 2020도9789).[19] 몰수에서는 「재산적 가치가 있는 무형의 재산」이라는 이름으로 범죄수익은닉규제법의 몰수 대상이 되었습니다(2018. 5. 30. 선고 2018도3619). 같은 물건이 「재산상 이익」·「무형재산」이라는 두 이름으로 사기·몰수의 자리를 채우고, 배임의 자리에서는 「재물」인지가 판단되지 않은 채 「타인의 사무를 처리하는 자」가 아니라는 이유로 유죄 판단이 파기되었고, 횡령 부분은 그와 동일체라는 이유로 함께 파기·환송되었습니다. 일본의 Mt.Gox 형사 판결은 다른 모양을 보여 줍니다. 1심 판결(2019)은 법이 「暗号資産」으로 개명하기 전이어서 「仮想通貨」라 썼고, 항소심 판결(2020)은 개명 뒤여서 「暗号資産（仮想通貨）」라 썼습니다.[20] 그리고 두 판결이 유죄로 인정한 죄는 비트코인을 객체로 하는 죄가 아니라 **거래소 장부의 전자적 기록을** 객체로 하는 私電磁的記録不正作出·同供用이었고, 이용자 돈의 횡령은 「利用者が…送金した金銭は，B社に帰属する」는 이유로 무죄였습니다(26장). 거기서 법원이 정한 것은 비트코인의 이름이 아니라 「ブロックチェーン上に存在しないビットコインは…架空のビットコイン」이라는 **존재의 기준**이었고, 그 기준은 1부의 기술 사실 — 비트코인은 블록체인에 기록된 상태로만 존재한다 — 에서 왔습니다. 미국의 두 형사 판결(26장)은 「wire fraud」·「securities fraud conspiracy」·「commodities fraud conspiracy」를 소인으로 했고, 「commodities fraud」와 「securities fraud」가 한 공소장에 나란히 선 것은 미국에서 같은 물건이 두 이름을 동시에 가진다는 사실의 형사 쪽 모습입니다.[21]

5. 한 법역 안에서 이름이 바뀐 세 사례

이름은 법역 사이에서만 다른 것이 아니라 한 법역 안에서도 바뀌었고, 세 사례를 원문으로 확인했습니다.

법역	전	후	바꾼 문서	바뀐 것
일본	「仮想通貨」(資金決済法 第2条第5項, 2020-04-30 판본)	「暗号資産」(2020-05-01 판본; 현행 第14項)	令和元年法律第28号 (公布 2019-06-07, 施行 2020-05-01)	정의어와 함께 電子記録移転権利 제외 단서가 들어왔다. 「통화」라는 글자가 이름에서 빠졌다
한국 (대법원)	「이른바 '가상화폐'의 일종」(2018. 5. 30. 선고 2018도3619)	「가상자산의 일종」(2021. 11. 11. 선고 2021도9855); 「가상자산은 … 재산상 이익」(2021. 12. 16. 선고 2020도9789)	사이에 특정금융정보법 제17113호(시행 2021. 3. 25.)가 「가상자산」을 법률 용어로 들였다	판결의 낱말이 바뀌었다. 판단의 틀(재산적 가치 있는 무형재산 / 재산상 이익)이 그 때문에 바뀌었다고는 판결이 적지 않았다
미국 (IRS)	「it does not have legal tender status in any jurisdiction」(Notice 2014-21, 2014)	「the use of virtual currency to perform "real" currency functions is limited」(Notice 2023-34, 2023)	Notice 2023-34 §3	배경 서술만 고치고 「property」라는 결론(Q&A-1·2)은 그대로 두었다

셋 가운데 어느 것도 **기술이 바뀌어서** 일어난 일이 아닙니다. 비트코인의 프로토콜은 2019년과 2021년과 2023년 사이에 이름을 바꿀 만한 변화를 겪지 않았습니다. 바뀐 것은 법이 그것을 어디에 세우기로 했는가이고, 그 결정이 이름에 적힌 것입니다. 이것이 설계가 이 장에 적어 둔 한 줄 — 「기술은 이름을 정하지 않는다」 — 의 뜻입니다.

> **? 법률가의 질문**
>
> 정의 조문이 기술을 따라가지 못할 때 법률가는 무엇을 합니까. 한국 가상자산법 제2조 제1호의 「전자적 증표」에 ERC-721의 (계약 주소, 번호) 쌍이 들어갑니까 — EU·영국은 「대체불가」를 명시적으로 뺐고 한국은 시행령 제2조 제4호가 빼되 금융위원회가 고시하는 것(예: 「지급수단으로 사용될 수 있는」 증표)은 다시 넣는 단서를 두었는데, 그 단서가 ERC-721에 어떻게 적용됩니까. 「발행인이 사용처와 그 용도를 제한한 것」(가목)에서 「발행인」이 없는 토큰은 어떻게 됩니까. 「전자적으로 거래 또는 이전될 수 있는」에서 거래소 장부 위의 잔고 — 체인에 없는 30만 BTC(26장) — 는 「이전될 수 있는 전자적 증표」입니까, 발행자(거래소)에 대한 채권입니까. 일본 조문의 「不特定の者を相手方として購入及び売却を行うことができる」는 상대방이 없는 AMM(15장)에서 어떻게 읽힙니까. 이것은 유추로 메울 틈입니까, 입법이 메워야 할 공백입니까, 아니면 정의 조문이 **그 물건의 성질을 정하지 않는다는** 사실을 받아들이고 27.2의 성질 논의로 돌아가야 하는 신호입니까. 이 책은 답하지 않고 5부의 학설 구도를 가리킵니다. → 27장

6. 정리

다섯 법역의 정의 조문을 나란히 놓으면 공통 뼈대 — 가치의 디지털 표현과 전자적 이전 가능성 — 와 갈라지는 제외 목록이 보입니다. 통화는 모두 밖에 두되 층이 다르고, 증권과의 경계는 모두 긋되 방법이 다르며, 대체불가 토큰은 셋이 뺐되 층이 다릅니다. 그 이름은 관할(누가 보는가)·과세(무엇으로 매기는가)·형사(어느 죄의 객체인가)를 가르고, 한 법역 안에서도 바뀌었습니다. 1부에서 3부까지 이 책이 보인 비트열과 서명과 상태 전이와 균형은 그 어느 이름도 자기 안에 담고 있지 않습니다. 법률가가 이 장에서 가져갈 것은 이름을 읽을 때 두 가지를 함께 보는 습관입니다. 그 이름이 **어느 법의 어느 조문에** 적혀 있고 **언제부터 시행**되었는가, 그리고 그 조문이 **무엇을 뺐는가**. 부록 C의 대역표는 이 장을 행과 열로 압축한 것이고, 27.3의 서베이 표는 이름 뒤의 기관과 절차를 적은 것입니다. 셋을 겹쳐 읽으면 어느 법역에서 어느 질문이 비어 있는지가 보입니다.

이 장의 출처

- 「가상자산 이용자 보호 등에 관한 법률」 제2조 제1호(법률 제19563호 제정 문언, 현행 제21857호); 「특정 금융거래정보의 보고 및 이용 등에 관한 법률」 제2조 제3호(2021. 3. 25. 시행 판본 MST 215971과 현행 판본 MST 290725); 소득세법 제21조 제1항 제27호(시행 2027. 1. 1., 미시행) — 국가법령정보센터(Open API, 2026-10-25 조회). 조문은 원문 그대로 옮겼고 줄바꿈만 목 단위로 넣었다.
- Regulation (EU) 2023/1114 (MiCA), OJ L 150, 9.6.2023, p. 40 — Art. 2(3)~(4), 3(1)(5)~(9), 149.
- *SEC v. W. J. Howey Co.*, 328 U.S. 293, 298~299 (1946).
- *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. 2018), 3·24쪽.
- IRS Notice 2014-21, 2014-16 I.R.B. 938; Notice 2023-34, 2023-19 I.R.B. 837. `irs-2023-notice-2023-34-modification-2014-21.pdf`.
- GENIUS Act, Pub. L. No. 119-27, 139 Stat. 419 (2025) — §2(6)·(7)·(22), §17, §20.
- *Uniform Commercial Code Amendments (2022)*, §12-102.
- 資金決済に関する法律 第2条第5項·第14項, 第63条の2; 金融商品取引法 第2条第3項; 令和八年法律第六十四号 반영 미시행 판본 金商法 第2条第49項 — e-Gov 법령 API v2(2026-10-25 조회). 2020-04-30·2020-05-01 판본 第2条第5項 첫 문장은 `asof=` 조회 기록.
- Financial Services and Markets Act 2000, s. 417(1); Financial Services and Markets Act 2023, s. 69 — legislation.gov.uk(2026-10-25 접근). SI 2026/102 — reg. 1, RAO art. 88F·88G. Property (Digital Assets etc) Act 2025 (c. 29), s. 1~2.
- FATF, *Updated Guidance for a Risk-Based Approach to Virtual Assets and VASPs* (2021-10-28), Glossary. SEC Release 81207 각주 5(2014년 FATF 정의 전재).
- CJEU, *Skatteverket v David Hedqvist*, C-264/14 (22 October 2015), ¶¶24·49·52, 주문.
- 대법원 2018. 5. 30. 선고 2018도3619 판결; 대법원 2021. 11. 11. 선고 2021도9855 판결; 대법원 2021. 12. 16. 선고 2020도9789 판결 — 국가법령정보센터 판례 전문.
- 東京地方裁判所 平成31年3月15日 判決(平成27年(刑わ)第2274号 등); 東京高等裁判所 令和2年6月11日 判決(令和元年(う)第948号) — 26장과 같은 파일.

- *United States v. Bankman-Fried*, ECF No. 424 (S.D.N.Y. 2024-03-28); *United States v. Kwon*, ECF No. 63 (S.D.N.Y. 2025-12-11) — 26장과 같은 파일.
- 이 장은 코드·노트북이 없다(설계). 조문의 번역은 붙이지 않았다. 「仮想通貨」→「暗号資産」 개명의 입법 이유, 미국 증권법·상품거래법 정의 조문의 원문, 각 주의 UCC 제12편 채택, GENIUS법 시행 규칙의 공표 여부는 확인하지 않았다.

각주

1. 「가상자산 이용자 보호 등에 관한 법률」(법률 제19563호, 공포 2023. 7. 18., 시행 2024. 7. 19.; 현행 법률 제21857호, 공포 2026. 8. 4., 시행 2026. 10. 2.) 제2조 제1호. 국가법령정보센터(Open API, MST 290735, 제2조, 2026-10-25 조회). 제2조의 개정 이력은 제정 1건뿐이다(법령 ID 014474 조문 연혁). 정의어는 「가상자산」이고 법은 가상자산을 「통화」·「화폐」라고 부르지 않는다 — 다목·사목이 「전자화폐」·「전자적 형태의 화폐」를 **제외**한다.

2. 「특정 금융거래정보의 보고 및 이용 등에 관한 법률」 제2조 제3호. 2021. 3. 25. 시행 판본(법률 제17113호, MST 215971, `get_law_text(mst=215971, jo=제2조, efYd=20210325)`)과 현행 판본(MST 290725, 시행 2026. 10. 2.)을 국가법령정보센터에서 각각 열어 대조했다(2026-10-25). 두 판본의 차이는 둘이다 — 현행 가상자산법 제2조 제1호에 「사. 「한국은행법」에 따른 한국은행(...)이 발행하는 전자적 형태의 화폐 및 그와 관련된 서비스」가 끼어 들어 옛 사목이 아목이 되었고, 특금법은 정의를 가상자산법으로 넘기고 준용 문언만 남겼다. 2021년 판본 제2조 제1호 하목은 「가상자산사업자」를 「가상자산과 관련하여 다음 1)부터 6)까지의 어느 하나에 해당하는 행위를 영업으로 하는 자」로 정의했다.

3. 소득세법 제21조 제1항 제27호(법률 제17757호 신설; 부칙 제1조 제2호의 시행일이 법률 제18578호·제19196호·제20615호로 「2022년 1월 1일」→「2023년 1월 1일」→「2025년 1월 1일」→「2027년 1월 1일」로 개정). 국가법령정보센터 「현행」 본문(MST 280405)은 이 호를 시행 중처럼 보여 주지만 부칙 원문으로 확인한 시행일은 2027. 1. 1.이다(27장 각주 29). 조문 자신이 「양도하거나 대여」만 적는다.

4. Regulation (EU) 2023/1114 of the European Parliament and of the Council of 31 May 2023 on markets in crypto-assets, OJ L 150, 9.6.2023, p. 40, Art. 3(1)(5)~(9)(L 150/63), Art. 2(3)~(4)(L 150/62~63), Art. 149(2)~(3)(L 150/183). (Cellar PDF). 정의어는 「crypto-asset」이고 「currency」는 「official currency」(제8호)에만 쓴다 — 암호자산은 「value or right」의 「digital representation」이다.

5. *SEC v. W. J. Howey Co.*, 328 U.S. 293, 298~299 (1946) (decided May 27, 1946). 증권법 제2조 (a)(1)(15 U.S.C. § 77b(a)(1))의 「security」 정의 문언 자체는 이 책이 원문을 열지 않았고, SEC 보고서(2017, 11쪽)가 「Under Section 2(a)(1) of the Securities Act and Section 3(a)(10) of the Exchange Act, a security includes 'an investment contract.'」라고 적은 것으로 위치만 확인했다.

6. *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. Mar. 6, 2018), 24쪽. 7 U.S.C. § 1a(9)의 조문 원문 자체는 이 책이 열지 않았고 결정이 인용한 문언을 옮겼다(결정은 「§ 1(a)(9)」로 표기한다).

7. IRS Notice 2014-21, 2014-16 I.R.B. 938, Section 2 Background(1쪽)·Section 4 Q&A-1(2쪽); IRS Notice 2023-34, 2023-19 I.R.B. 837, Section 3(2~3쪽). `irs-2023-notice-2023-34-modification-2014-21.pdf`. 2023-34 §3의 이유: "The Department of the Treasury and the Internal Revenue Service are aware that certain foreign jurisdictions have enacted laws that characterize Bitcoin as legal tender." 어느 법역인지는 고시가 적지 않았고 이 책도 적지 않는다.

8. GENIUS Act, Pub. L. No. 119-27, §2(6)(139 Stat. 419), §2(22)(139 Stat. 421~422), §20(effective date). (govinfo). §17은 허가 발행자의 결제 스테이블코인을 「security」·「commodity」에서 제외하는 개정을 담는다(27장 각주 20).

9. *Uniform Commercial Code Amendments (2022)*, Article 12, §12-102(a)(1), 주석판 235쪽. 27장 각주 11.

10. 資金決済に関する法律 第二条第十四項(e-Gov 법령 API v2, 현행 판본 `421AC0000000059_20260812_508AC0000000064`); 2020-04-30·2020-05-01 판본 第二条第五項(`asof=` 조회로 확인한 문언); 金融商品取引法 第二条第三項(현행 판본 `323AC0000000025_20260812_508AC0000000064`); 令和八年法律第六十四号 반영 미시행 판본 金商法 第二条第四十九項 (`323AC0000000025_20270722_508AC0000000064`, `current_revision_status: UnEnforced`). 모두 2026-10-25 조회. 法定通貨 표시 자산은 같은 법 第二条第五項 「電子決済手段」(「通貨建資産に限り」)로 따로 정의된다. 「仮想通貨」→「暗号資産」 개명의 입법 이유는 이 책이 1차 자료로 확인하지 않았다.

11. Financial Services and Markets Act 2000, s. 417(1) 「cryptoasset」; Financial Services and Markets Act 2023, s. 69(4) 및 Commencement Information "I2 S. 69 in force at 29.8.2023 by S.I. 2023/779, reg. 4(uu)". legislation.gov.uk(HTML, 2026-10-25 접근·축어 대조). 27장 각주 39.

12. SI 2026/102, reg. 1(2)~(3); 이 규정이 Regulated Activities Order(SI 2001/544)에 삽입하는 art. 88F·88G(36~37쪽). 27장 각주 40.

13. Property (Digital Assets etc) Act 2025 (c. 29), s. 1·s. 2(1)~(2). 27장 각주 9.

14. FATF, *Updated Guidance for a Risk-Based Approach to Virtual Assets and Virtual Asset Service Providers* (Paris, 28 October 2021), Glossary(문서 끝 용어집). 공표면은 이 지침이 2025년의 권고 1 개정 등을 반영하지 않았다고 경고한다. 2014년 FATF 보고서의 「virtual currency」 정의(SEC 2017 보고서 각주 5가 전재)는 "(1) a medium of exchange; and/or (2) a unit of account; and/or (3) a store of value"의 기능론으로 적혀 있었고, 2021년 용어집은 「payment or investment purposes」로 바뀌었다 — 기능론에서 용도론으로의 이동이고, 그 뜻은 이 책이 평가하지 않는다.

15. 「가상자산 이용자 보호 등에 관한 법률 시행령」(대통령령, 공포 2026. 9. 29., 시행 2026. 10. 2.) 제2조(가상자산에서 제외되는 범위) 제4호. 원문: 「4. 수집을 주된 목적으로 하는 전자적 증표, 거래 당사자 간의 거래 확인만을 목적으로 하는 전자적 증표 등 단일하게 존재하여 다른 전자적 증표로 대체할 수 없는 전자적 증표. 다만, 특정 재화나 서비스의 지급수단으로 사용될 수 있는 전자적 증표 등 금융위원회가 정하여 고시하는 전자적 증표는 제외한다.」 같은 조 제1호(전자채권)·제2호(모바일 상품권)·제3호(한국은행 네트워크의 예금 등)·제5호(금융위원회 고시)도 제외 목록이다. 국가법령정보센터 Open API(MST 290171, 제2조)(2026-10-25 조회). 금융위원회 고시는 확인하지 않았다.

16. *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. 2018), 3·24쪽 — 27장 각주 21. 관할 조문의 위치는 27.3의 표와 각주.

17. IRS Notice 2014-21, §4 Q&A-1·Q&A-2 — 27장 각주 14.

18. CJEU, *Skatteverket v David Hedqvist*, C-264/14 (22 October 2015), ¶¶24·49·52, 주문 2. 원문: "Transactions involving non-traditional currencies, that is to say, currencies other than those that are legal tender in one or more countries, in so far as those currencies have been accepted by the parties to a transaction as an alternative to legal tender and have no purpose other than to be a means of payment, are financial transactions."(¶49) — 27장 각주 13.

19. 대법원 2021. 11. 11. 선고 2021도9855 판결, 이유 2. 가.; 대법원 2021. 12. 16. 선고 2020도9789 판결, 이유 3. 라.; 대법원 2018. 5. 30. 선고 2018도3619 판결, 이유 1. 가. 5) — 27장 각주 22·15·23. 2018년 판결은 「이른바 '가상화폐'의 일종」, 2021년 두 판결은 「가상자산의 일종」·「가상자산은 … 재산상 이익에 해당한다」라고 쓴다 — 특금법 제17113호(시행 2021. 3. 25.)가 「가상자산」을 법률 용어로 들인 뒤의 판결들이다. 이 책은 그 낱말 선택이 판단에 영향을 주었다고 쓰지 않는다 — 판결은 그 이유를 적지 않았다.

20. 東京地方裁判所 平成31年3月15日 判決(平成27年(刑わ)第2274号 등), 「事実認定の補足説明」 第１の２(１): 「ビットコインは，インターネット上で流通する，いわゆる仮装通貨である。」(판결 원문의 표기 그대로); 東京高等裁判所 令和2年6月11日 判決(令和元年(う)第948号), 1쪽: 「本件は，暗号資産（仮想通貨）であるビットコインの取引所を運営していた会社の代表取締役であった被告人が」. 26장 각주 9·10. 「架空のビットコイン」 문장은 1심 第２の３(３)ア(ア)(26장 각주 14).

21. *United States v. Samuel Bankman-Fried*, No. 1:(S6)22-CR-673-001 (LAK) (S.D.N.Y.), Judgment, ECF No. 424 (2024-03-28), 1~2쪽: 「18 U.S.C. § 371 Securities Fraud Conspiracy … (S6) 5 / 18 U.S.C. § 371 Commodities Fraud Conspiracy … (S6)6」; *United States v. Do Hyeong Kwon*, No. 1:23-cr-00151-PAE (S.D.N.Y.), Judgment, ECF No. 63 (2025-12-11): 「18 U.S.C. § 371 Conspiracy to Defraud … 1s / 18 U.S.C. § 1343 and 2 Wire Fraud … 4s」. 26장 각주 35·28.

부록

부록 A 동향 노트 · 색인 (2026-10 기준 — 1~6부·0부 기재분)

> 본문에 「2026-10 기준」을 쓴 문장은 쓰는 즉시 여기 올린다(3장 일러두기의 갱신 규칙). 판을 바꿀 때 이 표의 항목만 갱신한다. 기준일 2026-10-25. 가격·시가총액 항목은 두지 않는다.

#	장·절	본문의 원리 문장	현재 상황(2026-10 기준)	1차 출처	다음 판에서 할 일
A-1	4장 5절	SHA-256은 FIPS 180-4가 정한 알고리듬	현행본 2015-08판. NIST는 2023-03-07 Planning Note로 개정 결정	https://csrc.nist.gov/pubs/fips/180-4/upd1/final	개정판이 나오면 판·연도 갱신
A-2	5장 3절	비트코인은 ECDSA와 슈노르 서명을 나란히 쓴다	BIP-340·341 Status Deployed	github.com/bitcoin/bips(bip-0340 커밋 2885f13d, bip-0341 커밋 24e96e87)	확인만
A-3	6장 4절·8장 3절	머클 트리 64바이트 거래 약점·타임워프는 BIP-54가 고치려 한다	BIP-54 「Consensus Cleanup」 Status **Complete**(배포 절 없음, 활성화 아님)	github.com/bitcoin/bips bip-0054.md 커밋 927b6de9	Deployed로 바뀌면 6·8장 문장 수정
A-4	8장 3절 ·12장 5절	난이도는 2,016블록마다 조정된다	블록 972,958(관측 2026-10-24 UTC) nBits 0x17020093, 난이도 140,577,680,051,919.6, 600초 가정 추정 해시율 ≈1.006×10^21 H/s(비교값 1.077×10^21)	`data/hashrate_2026-10-25.json` (Blockstream·mempool.space 원시 응답)	반기마다 스냅샷 교체, 본문은 크기 수준만
A-5	8장 5절	에너지 논쟁은 수치 없이 구조만	(본문 미기재)	—	동향 노트 본문에 추정 방법 둘(해시율 역산 vs 하드웨어 효율 가정)만 서술, 수치는 출처·기준일과 함께
A-6	10장 3절	지분증명 상수는 포크마다 바뀐다	Fulu 활성(2025-12-03, 에포크 411392): MIN_ACTIVATION_BALANCE 32 ETH, MAX_EFFECTIVE_BALANCE_ELECTRA 2,048 ETH, 슬래싱 몫 1/4096, 신고자 1/4096, 비례 배수 3. Gloas 미예정	consensus-specs master b834fa84; `widgets/data.json`	다음 포크 활성 시 상수 재확인
A-7	10장 3절	이더리움은 2022년 9월 PoS로 전환했다	포크 연표: Altair 2021-10-27 → Bellatrix 2022-09-06 → (TTD 도달 추정 2022-09-15) → Capella 2023-04-12 → Deneb 2024-03-13 → Electra 2025-05-07 → Fulu 2025-12-03	consensus-specs configs/mainnet.yaml	변동 없음, 확인만

#	장·절	본문의 원리 문장	현재 상황(2026-10 기준)	1차 출처	다음 판에서 할 일
A-8	5장 3절	탭루트는 2021년 활성화	Core chainparams 최소 활성화 높이 709,632(「Approximately November 12th, 2021」)	Bitcoin Core v31.1 chainparams.cpp	변동 없음
A-9	17장 4절	제안자–빌더 분리는 프로토콜 밖의 미들웨어·릴레이로 돌아간다	ethereum.org PBS 로드맵 페이지(갱신 2026-06-24): 「확정된 명세는 없다」, 포함 목록은 연구 중; mev-boost README: 「장래에 프로토콜에 내재화될 것」	https://ethereum.org/en/roadmap/pbs/ (소스 커밋 26bdbe8e); https://github.com/flashbots/mev-boost/blob/7a60459f1e61ee4d3b256978cd34bef5fe5e95c6/README.md	명세 확정·배포 시 17장 4절 문장과 같은 절의 「분업이 바꾸지 않는 것」 문단 수정
A-10	18장 1절	처리량 상한 = 블록당 용량 ÷ 블록 간격	관측 가스 한도 60,000,000(블록 26,264,275, 2026-10-24T18:23Z, RPC 3곳 일치), 슬롯 12,000ms(`SLOT_DURATION_MS`); 같은 설정 파일에 `SLOT_DURATION_MS_EIP8198: 10000` 값이 따로 있고 L69 `EIP8198_FORK_EPOCH` 는 최댓값(2^64−1, 미지정)으로 적혀 있음(내용 미확인)	`widgets/data.json` ; consensus-specs b834fa84 `configs/mainnet.yaml` L74·L118	가스 한도 반기 재관측; 슬롯 10초가 활성되면 18장 1절 상한 계산·21장 재검토
A-11	18장 2절	블롭 데이터는 최소 제공 기간만 보장되고 영구 보관되지 않는다	`MIN_EPOCHS_FOR_BLOB_SIDECARS_REQUESTS: 4096` → 4,096 × 32 × 12초 ≈ 18.2일(ethereum.org 「대략 18일」)	consensus-specs b834fa84 `configs/mainnet.yaml` L194; https://ethereum.org/en/developers/docs/scaling/zk-rollups/	상수나 슬롯 간격이 바뀌면 일수 재계산
A-12	18장 3절	낙관적 롤업에서 본체로 인출하려면 이의 기간을 기다린다	ethereum.org: 「대략 7일」(일반 서술, 롤업별 상이 가능)	https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/ (소스 커밋 3e3b0dec)	동향 노트 본문에 롤업별 실제 이의 기간을 출처·기준일과 함께
A-13	18장 6절	롤업마다 신뢰 경계(시퀀서·데이터 가용성 층·업그레이드 권한)가 다르다	(본문 미기재 — 과제)	—	동향 노트 본문에 롤업별 현황(시퀀서 집중, 사기 증명 허가 여부, DA 선택, 업그레이드 열쇠)을 1차 출처·기준일과 함께. **TVL·시가총액은 가격 자료라 쓰지 않는다**
A-14	16장 5절	오라클 조작은 소스·노드 담합·현물가 읽기·보고 선행매매 네 층에서 일어난다	(빈도 미기재 — 과제)	—	사고 유형별 1차 집계 자료(건수·기간·방법 명시)가 확보되면 「흔하다」 판단과 함께 동향 노트에. 2차 집계는 🟡 로 표시

#	장·절	본문의 원리 문장	현재 상황(2026-10 기준)	1차 출처	다음 판에서 할 일
A-15	13장 3절	가스 요금표는 포크마다 바뀐다	원고 값은 황서 상하이판(2025-02-04) 부록 G 기준; 3형(블롭) 거래 등 뒤 포크 미반영	https://ethereum.github.io/yellowpaper/paper.pdf	현행 EIP 목록과 대조해 바뀐 값·새 거래 형식을 동향 노트에; 황서 신판이 나오면 13장 각주 판 표기 갱신
A-16	20장 2절	블록 보조금은 210,000블록마다 반감한다 (Core `GetBlockSubsidy`, 정수 시프트)	관측 높이 972,958(2026-10-24T18:17Z) ÷ 210,000 = 4 → 보조금 3.125 BTC(소스 코드 규칙에서 계산, 코인베이스 출력 직접 확인 아님). 다음 반감 높이 1,050,000	`data/hashrate_2026-10-25.json` (Blockstream·mempool.space 원시 응답); Bitcoin Core v31.1 `src/kernel/chainparams.cpp` L84	높이 1,050,000 뒤 1.5625 BTC 로 갱신. 보상의 법정 통화 환산은 본문에 두지 않는다
A-17	21장 5절	블롭 수수료 시장은 별도 자원·별도 목표·지수형 갱신·소각이다	원고 상수는 EIP-4844 원문 값(목표 3·상한 6 블롭, 393,216/786,432 블롭 가스, 갱신 분모 3,338,477). 확인 과정에서는 기준일 활성 포크(Fusaka) 뒤 bpo1·bpo2 가 블롭 매개변수만 바꾼다고 적었고, **기준일 메인넷의 현행 목표·상한 블롭 수는 이 판에서는 확인하지 않았다**(과제)	https://eips.ethereum.org/EIPS/eip-4844	consensus-specs·execution-specs 의 현행 블롭 일정(BLOB_SCHEDULE) 값을 1차 출처로 확인해 21장 각주에 「원문값 / 현행값」 병기
A-18	24장 6절	상환 압력은 준비 국채 매각을 통해 단기 국채시장으로 전해질 수 있다	(부문 규모·발행자 국채 보유액 미기재 — 과제) BIS WP1270 2026-06 개정판: 2021-01~2026-03 일별 자료, 35억 달러(2σ) 유입당 3개월물 수익률 즉시 −0.71bp, 10일 내 최대 −4bp, 국채시장 스트레스 아래·부문 규모와 함께 효과 증가	(초록)	부문 규모는 **발행 잔액**(시가총액이라는 말을 쓰지 않는다)으로, 발행자 국채 보유액은 발행자 준비금 공시·재무부 자료로 기준일과 함께 동향 노트 본문에. 가격 자료는 쓰지 않는다
A-19	26장 2절·6절	Mt.Gox 형사 사건은 1심(東京地裁 2019. 3. 15.)·항소심(東京高裁 2020. 6. 11., 항소 기각)까지 판결 본문을 확인했다	상고 여부·확정일 **미확인**(2026-10 기준). 미국 형사 절차(관재인 보고서가 소개한 Vinnik 기소)는 원문 미대조		최고재판소 결정이 확인되면 26장 2절과 6절 표의 「상고·확정 미확인」을 고친다
A-20	26장 5절·6절	FTX 형사 사건은 1심 판결(S.D.N.Y. 2024-03-28)과 항소심 확인(2d Cir. 2026-06-12)까지	상고허가신청(certiorari) 여부 **미확인**(2026-10 기준)		연방대법원 처분이 확인되면 26장 표 갱신
A-21	27.3·30장 2.1	한국 소득세법 제21조 제1항 제27호(가상자산소득)는 공포되었으나 시행 전	시행 2027. 1. 1.(법률 제20615호로 세 번째 유예; 2026-10 기준 미시행). 국가법령정보센터 「현행」 본문은 시행 중처럼 표시	국가법령정보센터 Open API(MST 280405) + 부칙 개정문	2027-01-01 시행 뒤 「미시행」 표기를 「시행 중」으로; 추가 유예 법률이 공포되면 날짜 갱신

#	장·절	본문의 원리 문장	현재 상황(2026-10 기준)	1차 출처	다음 판에서 할 일
A-22	27.3·30장 2.4	일본 暗号資産 정의는 資金決済法 第2条第14項에 있다	令和8年法律第64号(公布 2026-07-23)가 정의를 金商法 第2条第49項으로 옮기는 부분은 **미시행**(e-Gov 표시 2027-07-22, 시행 정령 미확인; 2026-10 기준)	e-Gov API v2 `law_data/323AC0000000025_20270722_508AC0000000064`	시행되면 27.3 정의어 행·30장 2.4 를 金商法 第2条第49項으로 바꾸고 資金決済法 第2条第14項은 「電子決済手段サービス仲介業」 정의로
A-23	27.3·30장 2.3	미국 GENIUS Act(Pub. L. 119-27, 2025-07-18)는 제정되었다	시행일은 §20에 따라 「제정 후 18개월(상한 2027-01-18) 또는 최종 규칙 공표 후 120일 중 이른 날」 — **규칙 공표 여부 미확인**(2026-10 기준)	§20	연방 감독기관 최종 규칙 공표일이 확인되면 시행일 확정
A-24	27.3·30장 2.5	영국 SI 2026/102(made 2026-02-04)는 FCA 규칙 제정 부분만 발효	본격 시행일(full commencement day) **2027-10-25**(2026-10 기준 시행 전)	reg. 1(2)	2027-10-25 뒤 「시행 전」을 「시행 중」으로; FCA 최종 규칙(PS26/9~13)은 이 판 미인용
A-25	27.3 한국	민사집행법에는 가상자산 조문이 없다(2022년 보고서 시점)	그 뒤 개정 여부 **미확인**(2026-10 기준 과제)	이혜정(2022)	민사집행법 연혁을 국가법령정보센터에서 확인해 조문이 생겼으면 27.3·28.1(a) 수정
A-26	27.2 ⑨·27.3 미국	UCC 제12편(2022)은 모델법	주별 채택 여부 **미확인**(2026-10 기준 과제)		ULC 채택 현황표를 1차 출처로 확인해 동향 노트 본문에
A-27	29장 4절·6절	비트코인의 양자 전환은 새 출력 유형(BIP-360)·옛 서명의 일몰(BIP-361)·양자 내성 서명의 세 갈래 제안이다	BIP-360 「Pay-to-Merkle-Root (P2MR)」 Status **Draft**(v0.12.1, Assigned 2024-12-18), BIP-361 「Post Quantum Migration and Legacy Signature Sunset」 Status **Draft**(Informational, Assigned 2026-02-11, Requires 「TBD Post Quantum Signature BIP」), BIP-54 Complete(배포 절 없음 — A-3). 이 전환 제안들은 어느 메인넷에도 활성화되지 않음(2026-10 기준; 이미 있는 해시 기반 보호 성질과는 다른 문제)	github.com/bitcoin/bips 커밋 100ba872 `bip-0360.mediawiki` · `bip-0361.mediawiki` · `README.mediawiki` L1286~1298	상태가 Final/Deployed/Rejected로 바뀌면 29장 4.1·4.3·6절 문장과 부록 B ⑤ 「실제 구조」 열 갱신

#	장·절	본문의 원리 문장	현재 상황(2026-10 기준)	1차 출처	다음 판에서 할 일
A-28	29장 2절·6절	쇼어 알고리듬의 ECDLP 자원은 논리 큐비트·게이트 수로 추정하고 물리 큐비트는 가정 다섯의 함수다	최신 추정 ≤1,200 논리 큐비트·≤9,000만 Toffoli, 초전도 가정에서 물리 50만 개 미만(Babbush 외 2026-04, 프리프린트, Google·EF 소속 저자); 이더리움 공식 페이지(소스 커밋 643a7a45, 2026-10-23)는 「현재 하드웨어는 수천 개의 잡음 있는 물리 큐비트」. 전문가 설문(GRI·evolutionQ 「Quantum Threat Timeline Report 2025」, 2026-03 공표, 26명): RSA-2048을 24시간 안에 깨는 기계가 10년 안에 나올 가능성에 대해 19/26이 5% 초과, 13/26이 약 50% 이상이라 답함 — 의견이지 관측이 아님	; ethereum.org quantum-resistance 페이지; https://globalriskinstitute.org/mp-files/pdf-quantum-threat-timeline-report-2025.pdf/ (2026-10-25 접근)	동료심사 게재본이 나오거나 논리 큐비트 1,000개 규모의 시연이 보고되면 29장 2절 표 갱신. 설문은 매년 갱신되므로 최신 판으로 교체
A-29	29장 4.4·6절	이더리움은 네 영역(BLS·KZG·ECDSA·ZK)을 양자내성으로 바꾸되 계정 서명은 계정 추상화로 계정별 선택	EIP-8141은 Hegotá 업그레이드에 「고려 중」(목표 연도는 roadmap/security 「2027」·quantum-resistance 「2026년 하반기」로 두 페이지가 다름 — 이 책은 어느 쪽도 채택하지 않음); 핵심 양자내성 이정표 목표 「대략 2029년」(「계획 목표이고 바뀔 수 있다」); EF 양자 대응 팀 2026-01 신설, pq.ethereum.org 로드맵 I~*M*(2026-06-27 갱신)	ethereum.org roadmap/security(소스 커밋 290d753c, 2026-07-12)·quantum-resistance(커밋 643a7a45, 2026-10-23); https://pq.ethereum.org/	포크에 포함·활성화되면 29장 4.4 문장 갱신; 2029 목표가 바뀌면 연도 갱신
A-30	29장 3절	공개키가 드러난 코인(스크립트 유형·재사용·멤풀)만 쇼어의 표적이다	기준일 스냅샷(2차 집계 🟡, 블록 972,132, 2026-10-19): 노출 7,121,995 BTC = 일정 발행량의 35.4%(P2PK 1,715,077·P2TR 217,063·재사용 5,189,439). 1차 재현값 없음. 보조: Chaincode 2025-05 P2PK 1,720,747(8.68%)·Project Eleven 2025-01 6.26M; Deloitte(무일자) 4M+(25%); CoinShares 2026-02-06 ~1.6M(8%); BIP-361 「2026-03-01 기준 34% 이상」; Babbush 2026 ~6.9M	`data/exposed_output_types_2026-10-25.json` · `data/README.md`	반기마다 스냅샷 교체; 공개 노드에서 유형별 UTXO를 직접 집계한 1차 재현값을 만들면 🟡 표시를 걷어낸다
A-31	29장 6절	(본문 미기재 — 과제) NIST의 ECDSA 폐기 일정	이더리움 페이지(「NIST anticipates deprecating ECDSA by 2030 and disallowing it by 2035」)와 BIP-360(「according to NIST IR 8547, Elliptic Curve Cryptography (ECC) is planned to be disallowed within the US federal government after 2035」)이 NIST IR 8547을 인용 — 이더리움 페이지는 문서명을 들지 않고, BIP-360의 2030은 CNSA 2.0 일정·2035만 NIST IR 8547 — 이 책은 그 문서의 원문을 열지 않았다	(미확인)	NIST IR 8547 원문을 받아 에넣고 29장 4.2에 한 줄 추가

#	장·절	본문의 원리 문장	현재 상황(2026-10 기준)	1차 출처	다음 판에서 할 일
A-32	2장 11절	백서 이후의 이정표는 각 담당 장이 원문으로 다룬다	2024년 이후 「제도 변화」로만 적은 항목 — EU MiCA 적용 2024-12-30(30장), 한국 가상자산이용자보호법 시행 2024-07-19(27·30장), 미국 GENIUS Act 제정 2025-07-18(A-23), 영국 Property (Digital Assets etc) Act 2025 국왕 재가 2025-12-02·SI 2026/102 made 2026-02-04(A-24), 일본 令和8年法律第64号 공포 2026-07-23(A-22), NIST FIPS 203~205 공표 2024-08-13(29장), BIP-360·361 Draft(A-27). **현물 상장지수펀드의 승인·상장은 이 책이 1차 자료(승인 명령·상장 공지)를 확보하지 못해 어느 장에도 적지 않았다** — 과제	각 항목의 1차 자료는 해당 장·A-22~A-24·A-27	2장 표와 부록 D·W11은 2023년까지로 닫혀 있다. 이 뒤 항목은 이 색인에만 더한다. 상장지수펀드는 규제기관 공식 문서(승인 명령 원문)를 확보한 뒤에만 적되, 규모·가격은 쓰지 않는다

부록

부록 B 흔한 오해 판정표

블록체인을 두고 자주 들리는 명제 열두 개를 **판정 / 왜 그렇게 보이나 / 실제 구조 / 근거 장** 네 열로 적습니다. 판정은 세 등급입니다 — **사실**(명제가 그대로 참), **부분**(명제의 한 조각은 참이고 다른 조각은 거짓이어서 그대로 쓰면 틀림), **기각**(명제가 참이 아님). 「실제 구조」 열은 해당 장이 원문으로 확인해 적은 문장을 그대로 옮기거나 요약한 것이고, 근거의 원전은 그 장의 각주에 있습니다. 이 표는 새 자료를 더하지 않습니다. 명제 하나가 「기각」이라는 말은 그 반대 명제가 참이라는 뜻이 아니라, 그 명제를 그대로 쓸 수 없다는 뜻입니다.

#	명제	판정	왜 그렇게 보이나	실제 구조	근거 장
①	블록체인은 변조 불가능하다	**부분**	과거 블록 한 글자를 바꾸면 그 뒤의 모든 블록이 어긋나므로 「고칠 수 없다」고 느껴진다. 「불변의 장부」라는 소개 문구가 그 느낌을 굳힌다	변조는 보존해 둔 해시·사슬과 **비교하면 드러난다**(해시 사슬) — 그러나 해시 사슬 자체는 두 역사 가운데 어느 쪽이 원본인지 가려 주지 않고, 「드러난다」와 「막힌다」도 다르다. 작업증명이 없는 해시 사슬은 블록당 해시 한 번으로 전부 다시 꿸 수 있고(7장 NB07-1), 작업증명이 있어도 변조는 불가능이 아니라 **비용이다**. 7장: 「블록체인의 불변성은 수학적 불가능이 아니라 **경제적 비용입니다**.」 그 비용은 뒤로 갈수록 커지되 유한하고, 점유율 q 가 1/2을 넘으면 「막힌다」가 성립하지 않는다(9장)	7·8·9
②	스마트계약은 계약이다	**부분**	「contract」라는 낱말이 코드의 자료형 이름이고, 「코드가 자동으로 이행한다」는 설명이 계약의 이행처럼 들린다	기술 쪽: 13장이 읽은 열 줄짜리 「계약」에는 「당사자도 합의도 없었습니다」 — 그것은 조건이 맞을 때 값을 풀어 주는 프로그램이다. 법 쪽: 이 책이 원문을 확인한 1차 자료(SEC Release No. 81207, 2017-07-25, 2쪽)는 「이 기술, "스마트계약" 또는 컴퓨터 코드를 통한 특정 기능의 자동화는 행위를 미국 연방 증권법의 적용 범위에서 벗어나게 하지 않는다」고 할 뿐 코드가 계약인지는 말하지 않는다(28.1(c)). 계약의 **이행 수단이** 될 수는 있고, 계약인지는 코드 밖에서 정해진다. 「계약 내용은 코드인가 의도인가」는 13장 상자의 열린 질문이다	13·28.1(c)
③	비트코인은 폰지다	**부분**	뒤에 들어온 사람의 돈으로 앞사람이 이익을 보는 모양이 유통시장의 매매와 겹쳐 보이고, 가격이 신념에 기댄다는 점이 사기의 냄새로 읽힌다	19장의 경제학이 보인 것은 「배당 없는 외부화폐는 가격 0인 균형과 양의 가격 균형을 함께 가지며, 양의 가격 균형은 「남들이 받아 줄 것」이라는 믿음으로 지탱되고 하나로 정해지지 않습니다」라는 **균형의 열거이고**, 그 문헌의 언어에서 불환지폐의 가치 자체가 「버블」(정의어)이다 — 경멸어가 아니다. 같은 문헌(19장 심화 상자의 van Oordt 2024)은 통화 증가율이 음이 아닌 암호자산의 버블 균형 참가자가 「합계로는 폰지 사기와 같은 수익(Ponzi-scheme equivalent payoffs in the aggregate)」을 경험한다고 적는다 — 명제의 한 조각은 모형 안에서 참이다. 다른 조각은 다르다. 22장이 보인 발행 규칙은 코드에 적힌 것이고 「회사 정관과 닮았지만 집행자가 없다」 — 수익을 약정하고 자금을 모아 지급을 통제하는 **운영 주체가** 없다. 26장의 Terra에서 배심이 인정한 것은 증권법 사기 금지 조항 위반이고, 그 진술은 프로토콜이 아니라 그 위의 발행자에게서 나왔다. 가격에 반복적 투기 국면이 있었는지는 25장의 통계 문제이고 별개다	19·22·25·26
④	비트코인은 아무것도 암호화하지 않는다	**사실**	「암호화폐」라는 이름이 거래 내용이 암호화되어 있다는 인상을 준다 — 그래서 이 명제는 오해가 아니라 오해를 **바로잡는** 명제다	5장: 「비트코인의 거래와 원장은 암호화되어 있지 않습니다.」 쓰이는 것은 해시(4장)와 서명(5장)이지 암호화가 아니다. 모든 거래는 공개되어 있고(1장·11장), 12장이 제네시스 블록의 모든 바이트를 그대로 읽었다. 다만 범위는 「거래와 원장」이다	4·5·12
⑤	양자컴퓨터가 나오면 비트코인은 끝난다	**부분**	쇼어의 알고리듬이 서명을 깬다는 것은 사실이고, 그 사실이 「비트코인 전체가 무너진다」로 넘어간다	깨지는 것은 **서명**(공개키가 드러난 코인)이고, 해시와 작업증명은 그로버의 제곱근 가속에 버틴다 — 29장 1절. 노출된 코인은 기준일 2차 집계로 모든 유형을 합쳐 발행량의 약 3분의 1이고, 그 가운데 가장 단단한 수가 P2PK의 약 170만 BTC(전체의 9% 가까이)이며, 노출되지 않은 코인도 지출 거래를 내보내 확인을 기다리는 동안에는 표적이 될 수 있다 — 그 시간은 몇 분에서 몇 시간이고 고정되어 있지 않다(29장 3절). 양자내성 서명 표준(FIPS 204·205)은 2024년에 공표되었으나 비트코인의 전환 제안(BIP-360·361)은 2026-10 기준 Draft이고 두 메인넷 어디에도 활성화되지 않았다(29장 4절·부록 A-27). 「끝난다」가 아니라 **전환과 남는 코인의 처리가** 문제이고, 그 처리는 기술이 아니라 거버넌스와 법의 문제다(29장 5절)	29

#	명제	판정	왜 그렇게 보이나	실제 구조	근거 장
⑥	컴퓨터가 빨라지면 블록체인이 뚫린다	**기각**	해시율이 2009년 이래 열네 자릿수 가까이 늘었으니(8장 NB08-3) 「언젠가 다 뚫린다」로 읽힌다	8장: 「채굴은 퍼즐이 아니라 복권이고, 기계가 빨라져도 당첨 간격은 10분으로 되돌려집니다」 — 난이도가 2,016블록마다 조정된다. 8장: 다음 블록을 공격자가 찾을 확률은 점유율 q 이고 「전체 해시율이 10배가 되어도 q 가 같으면 이 확률은 같고, 난이도 조정이 블록 간격을 10분으로 되돌리므로 「블록 수로 센 경주」의 모양도 같습니다」; 9장: 추격 성공 확률은 q 와 확인 수 z 만의 함수다. 뚫리는 변수는 절대 속도가 아니라 **점유율이다**. 해시 함수 자체의 안전(역상·충돌)은 속도가 아니라 알고리듬의 문제이고(4장), 양자 가속은 ⑤에서 따로 본다	4·8·9
⑦	「6 확인」이면 확정이다	**기각**	거래소·지갑이 6확인을 기다렸다가 「완료」로 표시하므로 규칙처럼 보인다	9장: 「「6 확인」은 규칙이 아니라 관행입니다.」 되돌릴 확률은 z 에 지수적으로 줄지만 0이 되지 않고, $q \geq 1/2$ 이면 줄지 않는다. 백서의 식은 낙관적이어서 정확식은 같은 안전 수준에 더 많은 확인을 요구한다($q = 0.1$ 이면 5가 아니라 6, $q = 0.3$ 이면 24가 아니라 32). 「완료」의 순간은 없고 확률이 작아질 뿐이다 — 법의 「이행」·「완료」 개념과 어긋나는 자리는 28.1(c)	9·28.1(c)
⑧	블록체인은 익명이다	**기각**	주소가 사람 이름이 아니라 문자열이어서 「누군지 모른다」로 읽힌다	모든 거래는 **공개되어** 있다(1장 — 「거래가 공개적으로 알려져야」; 12장은 제네시스 블록의 바이트를 다 읽었다). 11장: 「UTXO 모형에서 프라이버시는 「주소가 익명」이라는 데서 오는 것이 아니라 「주소를 바꿔 쓰는 습관」에서 오고, 그 습관은 입력 합치기와 거스름돈 패턴으로 깨집니다.」 익명이 아니라 **가명이고**, 백서 §10 자신이 「여러 입력을 가진 거래에서는 … 어느 정도의 연결은 피할 수 없다」고 적는다. 공개키가 체인에 그대로 드러나는 출력 유형이 있다는 사실(29장 3절)은 그 반대쪽의 증거다	1·11·12
⑨	코드가 법이다	**부분**	스마트계약은 적힌 대로 실행되고 누구도 중간에서 멈출 수 없으므로 「코드가 곧 규칙」으로 보인다	16장: 「「코드가 법」은 체인 안에서 성립하고, 조건이 참인지는 체인 밖에서 들어옵니다」 — 오라클이 틀린 값을 넣어도 코드는 한 줄도 어긋나지 않는다. 26장(The DAO): 「계약 코드는 적힌 대로 실행되어 작성자의 의도를 배반했고, 그것을 되돌린 것은 코드가 아니라 사람들의 조정이었습니다」 — 코드가 허용한 결과를 다수가 소프트웨어 갱신으로 거두었다. 그리고 1차 자료는 「이 기술, "스마트계약" 또는 컴퓨터 코드를 통한 특정 기능의 자동화는 행위를 미국 연방 증권법의 적용 범위에서 벗어나게 하지 않는다」(SEC Release No. 81207, 2017-07-25, 2쪽; 28.1(c))고 적는다. 체인 안의 실행 규칙으로서는 참, 체인 밖 법의 적용을 막는 규범으로서는 거짓	16·26·28.1(c)
⑩	지분증명은 부자가 더 부자 되는 구조다	**부분**	담보에 비례해 뽑히고 보상도 비례하므로 「많이 가진 자가 더 많이 받는다」	10장: 비례 자체는 격차의 **비율을** 키우지 않는다(NB10-1 — 모두가 같은 율로 늘면 비율은 그대로). 그러나 보상을 다시 담보로 넣는 복리 효과, 노드 운영의 규모의 경제, 최소 담보 32 ETH라는 문턱은 작은 참가자에게 불리하게 작용할 수 있고, 작업증명에서도 장비·전력의 규모의 경제가 같은 방향으로 작용한다. 「어느 쪽이 더 집중되었는지는 실측의 문제이고 이 책은 그 수치를 적지 않습니다.」 그래서 **부분**	10

#	명제	판정	왜 그렇게 보이나	실제 구조	근거 장
⑪	스테이블코인은 안정적이다	**기각**	이름이 「안정」을 약속하고 1달러 근방의 가격이 오래 유지된다	24장: 「스테이블코인은 상환 약속이 붙은 토큰이고, 그래서 외부화폐의 경제학이 아니라 은행의 경제학에 속합니다.」 「「안정」은 가격의 성질이 아니라 대차대조표와 절차의 성질입니다.」 담보율은 지급능력의 수, 상환 가능 비율은 유동성의 수이고 런은 둘째 수에서 시작하며, 예금보험이 없고 2차 시장이 있어 창구를 닫는 것이 런을 멈추지 못한다. 2022년 5월(알고리듬형)과 2023년 3월(법정화폐 담보형)의 두 사건은 구조가 다르고, 어느 코인이 안전한지는 이 책이 말하지 않는다. 이름이 안정을 보장하지 않으므로 명제 그대로는 기각	24·26
⑫	탈중앙이면 책임자가 없다	**기각**	회사도 대표도 없는 네트워크에서 「누구에게 책임을 묻는가」가 비어 보인다	책임자가 **없는** 것과 **보이지 않는** 것은 다르다(23장 — 이기적 채굴자는 숨을 수 있다). 17장: 「블록을 만드는 사람이 순서를 정하며」, 18장의 롤업에는 「순서를 정하는 사람(시퀀서)」과 업그레이드 권한이 있다. 22장: 거버넌스 토큰과 The DAO의 기록은 다수결·큐레이터·분리가 코드로 적혔으되 「집행자가 없다」는 것 — 그러나 다중서명 키를 쥔 사람은 「사실상의 집행자」다. 1차 자료: SEC(Release No. 81207, 2017-07-25)는 The DAO를 「an unincorporated organization」이라 부르며 증권의 발행인이라 했고, 영국 항소법원(*Tulip Trading Ltd v van der Laan* [2023] EWCA Civ 83, 2023-02-03)은 비트코인 개발자가 보유자에게 신인의무를 지는지가 「a serious issue to be tried」라고 했다(28.1(g)). 책임자가 없는 것이 아니라 **어느 자리의 누구인지가 열려 있다**	17·18·22·23·28.1(g)

열두 명제 가운데 **사실은** 하나(④), **기각은** 다섯(⑥·⑦·⑧·⑪·⑫), **부분은** 여섯(①·②·③·⑤·⑨·⑩)입니다. 「부분」이 많은 것은 이 명제들이 대개 참인 기술 사실 하나를 틀린 결론으로 넘기는 모양이기 때문입니다 — 변조는 드러난다(참) → 변조 불가능(거짓), 서명은 깨진다(참) → 비트코인은 끝난다(거짓), 코드는 적힌 대로 돈다(참) → 코드가 법이다(거짓). 법률가가 이 표에서 가져갈 것은 판정 자체보다 **어느 조각이 참인지를 가르는 습관입니다**. 소송에서 「블록체인은 변조 불가능하므로 이 기록은 진정하다」는 주장이 들어오면, 참인 조각(변조는 드러난다)과 거짓인 조각(불가능)을 갈라 읽는 것이 28.1(f)의 출발점입니다.

출처

- 각 행의 「실제 구조」는 해당 장의 본문·정리·코드 출력에서 옮겼고, 그 원전(나카모토 백서, FIPS 180-4, 하버·스토네타, 이얄–시러, 다이아몬드–딥비그, SEC 2017 보고서, *Tulip Trading*, BIP-360·361, 아가왈 외, 바부시 외 등)은 각 장의 각주에 있다. 이 부록은 새 자료를 더하지 않았다.
- 판정 틀은 집필 전 연구 회차의 가설 판정표(「판정 / 핵심 근거 / 근거 위계 / 반대 근거 / 판정이 바뀌는 관측」 다섯 열)를 독자용으로 네 열로 줄인 것이다. 그 회차의 판정은 참고만 했고, 근거는 전부 본문 장과 그 원전으로 다시 확인했다.
- 가격·시가총액·수익률 자료는 어느 행에도 없다.

부록

부록 C 기술 용어 ↔ 법령 용어 대역표

30장이 문장으로 읽은 것을 행과 열로 압축한 표입니다. 조문 원문과 법률 번호·시행 상태는 30장 2절·3절의 것을 **글자 그대로** 옮겼고, 이 부록은 새 조문·새 번역을 더하지 않았습니다(번역은 해석이므로 조문은 원문만 둡니다 — 30장 1절의 규칙). 기준일은 2026-10-25이고, 시행 상태가 바뀌는 조문은 부록 A(A-21~A-24)가 추적합니다. 세 표입니다 — C.1 다섯 법역의 정의어, C.2 제외 목록, C.3 기술 용어가 각 법역의 어느 낱말에 닿는가.

C.1 다섯 법역의 정의어 (30장 2절)

법역	정의어	조문	법률 번호 · 시행 상태 (2026-10-25)	정의의 뼈대(원문 발췌)
한국	「가상자산」	「가상자산 이용자 보호 등에 관한 법률」 제2조 제1호	법률 제19563호 제정(공포 2023. 7. 18., 시행 2024. 7. 19.), 현행 제21857호(시행 2026. 10. 2.) — **시행 중**	「경제적 가치를 지닌 것으로서 전자적으로 거래 또는 이전될 수 있는 전자적 증표(그에 관한 일체의 권리를 포함한다)」
한국 (준용)	「가상자산」	「특정 금융거래정보의 보고 및 이용 등에 관한 법률」 제2조 제3호	법률 제17113호(공포 2020. 3. 24., 시행 2021. 3. 25.)가 정의 본문을 두었고, 2024. 7. 19.부터 준용 문언	「「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산을 말한다」
한국 (과세)	「가상자산소득」	소득세법 제21조 제1항 제27호	법률 제17757호 신설(공포 2020. 12. 29.), 시행 2027. 1. 1. — **미시행**	「「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산(이하 "가상자산"이라 한다)을 양도하거나 대여함으로써 발생하는 소득(이하 "가상자산소득"이라 한다)」
EU	「crypto-asset」	Regulation (EU) 2023/1114 (MiCA), Art. 3(1)(5)	관보 L 150(2023. 6. 9.), 제149조에 따라 2024-12-30부터 적용(Title III·IV는 2024-06-30부터) — **적용 중**	"a digital representation of a value or of a right that is able to be transferred and stored electronically using distributed ledger technology or similar technology"
EU (하위 유형)	「asset-referenced token」·「e-money token」·「utility token」	MiCA Art. 3(1)(6)·(7)·(9)	위와 같음	(6) "purports to maintain a stable value by referencing another value or right or a combination thereof, including one or more official currencies"; (7) "purports to maintain a stable value by referencing the value of one official currency"; (9) "only intended to provide access to a good or a service supplied by its issuer"

법역	정의어	조문	법률 번호 · 시행 상태(2026-10-25)	정의의 뼈대(원문 발췌)
미국 (증권)	「investment contract」	Securities Act §2(a)(1)의 「security」 — 연방대법원 *Howey* 판결의 정의	판례법(*SEC v. W. J. Howey Co.*, 328 U.S. 293, 298~299 (1946)), 유효	"a contract, transaction or scheme whereby a person invests his money in a common enterprise and is led to expect profits solely from the efforts of the promoter or a third party"
미국 (상품)	「commodity」	7 U.S.C. § 1a(9) — *CFTC v. McDonnell* 결정이 인용	지방법원 결정(E.D.N.Y. 2018), 상급심 경과 미확인	"all other goods and articles . . . in which contracts for future delivery are presently or in the future dealt in"
미국 (세법)	「virtual currency」 → 「property」	IRS Notice 2014-21 §4 Q&A-1; Notice 2023-34 §3	행정 해석, 2023-34로 배경 문장만 수정, 유효	"For federal tax purposes, virtual currency is treated as property."
미국 (연방 성문법)	「digital asset」·「payment stablecoin」	GENIUS Act §2(6)·(22)	Pub. L. 119-27, 139 Stat. 419 (2025-07-18) 제정; 시행일은 §20에 따라 늦어도 2027-01-18, 규칙 공표 여부 미확인	(6) "any digital representation of value that is recorded on a cryptographically secured distributed ledger"; (22)(A) "a digital asset— (i) that is, or is designed to be, used as a means of payment or settlement; and (ii) the issuer of which— (I) is obligated to convert, redeem, or repurchase for a fixed amount of monetary value, not including a digital asset denominated in a fixed amount of monetary value; and (II) represents that such issuer will maintain, or create the reasonable expectation that it will maintain, a stable value …"; (B) 제외 목록은 C.2
미국 (사법 모델법)	「controllable electronic record」	UCC Amendments (2022) §12-102(a)(1)	모델법, 주별 채택 여부 미확인	"a record stored in an electronic medium that can be subjected to control under Section 12-105. The term does not include …"(제외 목록은 30장 2.3 (마))
일본	「暗号資産」	資金決済に関する法律 第2条第14項	平成21年法律第59号; 「仮想通貨」→「暗号資産」 개명 令和元年法律第28号(施行 2020-05-01); 第5項→第14項 이동 2023-06-01 — **시행 중**. 金商法 第2条第49項으로 옮기는 令和8年法律第64号(公布 2026-07-23)는 공포·일부 시행, **정의 이관 부분은 2026-10-25 미시행**	「物品等を購入し、若しくは借り受け、又は役務の提供を受ける場合に、これらの代価の弁済のために不特定の者に対して使用することができ、かつ、不特定の者を相手方として購入及び売却を行うことができる財産的価値（電子機器その他の物に電子的方法により記録されているものに限り、本邦通貨及び外国通貨、通貨建資産並びに電子決済手段（通貨建資産に該当するものを除く。）を除く。次号において同じ。）であって、電子情報処理組織を用いて移転することができるもの」

법역	정의어	조문	법률 번호 · 시행 상태(2026-10-25)	정의의 뼈대(원문 발췌)
영국 (정의)	「crypto asset」	Financial Services and Markets Act 2000, s. 417(1)	FSMA 2023 s. 69(4)(a)로 삽입, 2023-08-29 시행 — **시행 중**	"any cryptographically secured digital representation of value or contractual rights that— (a) can be transferred, stored or traded electronically, and (b) that uses technology supporting the recording or storage of data (which may include distributed ledger technology)"
영국 (규제 활동)	「qualifying cryptoasset」· 「qualifying stablecoin」	RAO art. 88F·88G (SI 2026/102가 삽입)	SI 2026/102 made 2026-02-04; 본격 시행일 2027-10-25 — **시행 전**(FCA 규칙 제정 부분만 발효)	88F(2) "a cryptoasset which is— (a) fungible, (b) transferable, (c) not solely a record of value or contractual rights, including rights in another cryptoasset, and (d) not excluded by paragraph (4)"; 88G(2) "a qualifying cryptoasset where— (a) that cryptoasset seeks or purports to maintain a stable value in relation to a particular fiat currency ('the referenced fiat currency'), and (b) fiat currency … or other assets are held for the purpose of maintaining a stable value, in accordance with sub-paragraph (a)"
영국 (재산법)	(정의가 아니라 확인)	Property (Digital Assets etc) Act 2025 (c. 29), s. 1	2025-12-02 국왕 재가·같은 날 시행	"A thing (including a thing that is digital or electronic in nature) is not prevented from being the object of personal property rights merely because it is neither— (a) a thing in possession, nor (b) a thing in action."
국제 (참조)	「virtual asset」	FATF 2021 갱신 지침 용어집	권고, 법적 구속력 없음	"a digital representation of value that can be digitally traded, or transferred, and can be used for payment or investment purposes. Virtual assets do not include …" (제외 문장은 30장 2.6)

C.2 제외 목록 (30장 3절의 표)

뺀 것	한국	EU	미국 (GENIUS)	일본	영국 (88F)
법정통화·중앙은행 디지털 화폐	사목(한국은행 발행 전자적 화폐)	(8) 「official currency」는 별개 개념(정의에 제외 조항 없음)	(B)(i) national currency — payment stablecoin 정의에만; (6) digital asset에는 제외 없음	本邦通貨及び外国通貨, 通貨建資産	(4)(c) currency … including a central bank digital currency
전자화폐·선불수단	다목	(4)(c) funds(단, EMT는 포함)	— (예금은 (B)(ii)로 따로 뺌)	電子決済手段(第5項으로 분리)	(4)(b) electronic money
증권·금융상품	라목(전자등록주식등)·마목·바목	(4)(a) financial instruments	(B)(iii) security	단서: 金商法 第29条の2第1項第8号의 권리(電子記録移転権利)	(4)(a) specified investment cryptoasset
용도 한정 토큰	가목(발행인이 사용처·용도를 제한)	— (유틸리티토큰은 포함, (9))	—	—	(4)(d) 발행자 상환만 가능·한정 네트워크
게임 결과물	나목	—	—	—	—
대체불가(NFT)	아목 → 시행령 §2(4)(금융위원회가 고시하는 것 — 예: 지급수단으로 사용될 수 있는 증표 — 은 단서로 다시 포함)	Art. 2(3) unique and not fungible	—	—	(2)(a) fungible 요건
위임	아목(대통령령)	—	—	—	—

C.3 기술 용어는 어느 낱말에 닿는가

행은 이 책의 1~3부가 쓴 기술 용어, 열은 그 용어가 다섯 법역의 어느 정의어·어느 제외 항목에 닿는지입니다. 「—」는 그 법역의 조문이 그 기술 용어에 해당하는 낱말을 두지 않았다는 뜻이고, 30장이 적지 않은 것은 여기도 적지 않았습니다. 마지막 열은 30장이 짚은 **어긋나는 지점입니다**.

기술 용어(장)	한국	EU	미국	일본	영국	어긋나는 지점(30장)
코인·토큰 일반 — UTXO의 출력값, 계정의 잔고(11장)	「가상자산」 = 「전자적으로 거래 또는 이전될 수 있는 전자적 증표」	「crypto-asset」 = "digital representation of a value or of a right that is able to be transferred and stored electronically"	「digital asset」 = "digital representation of value that is recorded on a cryptographically secured distributed ledger" (GENIUS); 과세는 「property」; 증권법·상품법은 거래의 모양에 따라	「暗号資産」 = 「財産的価値…であって、電子情報処理組織を用いて移転することができるもの」	「crypto asset」 = "digital representation of value or contractual rights that … can be transferred, stored or traded electronically"	「1부가 보인 기술 가운데 조문이 집어 든 것은 **이전 가능성뿐이고**, 해시·서명·합의 규칙·최종성은 어느 정의에도 없습니다. EU와 영국과 미국 GENIUS법만이 「distributed ledger technology」·「cryptographically secured」라는 기술 낱말을 넣었고, 한국과 일본은 기술 중립입니다.」
블록체인·분산 원장(7장)	— (기술 중립)	"distributed ledger technology or similar technology"	"cryptographically secured distributed ledger" (GENIUS §2(6))	— (「電子情報処理組織」)	"technology supporting the recording or storage of data (which may include distributed ledger technology)"	30장 3절: 「EU와 영국과 미국 GENIUS법만이 … 기술 낱말을 넣었고」 — 세 조문의 문언은 왼쪽 그대로
법정통화·중앙은행 디지털 화폐(19장)	사목이 **제외**	「official currency」(8)는 별개 개념	「national currency」는 payment stablecoin 정의에서 제외	「本邦通貨及び外国通貨」 제외	(4)(c) 제외	「「통화」는 어디에서나 밖에 있습니다. … 법은 자기가 발행하는 것에만 「통화」라는 이름을 남겼습니다.」

기술 용어(장)	한국	EU	미국	일본	영국	어긋나는 지점(30장)
스테이블코인(24장)	(30장에 정의 조문 없음 — 27.3 서베이표)	「asset-referenced token」(6)·「e-money token」(7)	「payment stablecoin」 §2(22)	法定通貨表示 자산은 「電子決済手段」(第2条第5項, 「通貨建資産に限り」)로 따로 정의	「qualifying stablecoin」 88G(2) — "seeks or purports to maintain a stable value … and (b) … assets are held"	세 법역의 정의어가 잡는 자리가 다르다 — EU (6)·(7)은 가치 안정의 **표방**, 영국 88G(2)는 표방에 준비자산 보유를 더한 것, GENIUS §2(22)(A)(ii)(I)만이 **상환 의무**(24장의 「돌려준다는 약속」)를 정의에 넣었고, 일본은 통화 표시 자산을 아예 다른 낱말로 뺐다(문언은 30장 2절)
대체불가 토큰 — ERC-721의 (계약 주소, 번호) 쌍(14장)	시행령 §2(4)가 빼되 「특정 재화나 서비스의 지급수단으로 사용될 수 있는 전자적 증표 등 금융위원회가 정하여 고시하는 전자적 증표」는 다시 넣음	Art. 2(3) "unique and not fungible" 제외	—	—	88F(2)(a) 「fungible」 요건	「대체불가 토큰은 세 법역이 뺐되 층이 다릅니다.」 ERC-721 쌍이 「지급수단으로 사용될 수 있는」 것으로서 금융위원회 고시에 드는지는 그 토큰이 무엇을 가리키는가와 고시에 달려 있다
토큰 표준이 정하지 않는 것 — 발행·의미(14장)	가목 「발행인이 사용처와 그 용도를 제한한 것」 제외	「utility token」(9) "access to a good or a service supplied by its issuer"	—	—	(4)(d) 발행자 상환만 가능·한정 네트워크 제외	조문은 **발행인을** 전제한다 — 30장 질문 상자: 「「발행인이 사용처와 그 용도를 제한한 것」(가목)에서 「발행인」이 없는 토큰은 어떻게 됩니까.」
증권성 — 토큰 매각과 투자계약(14·22·26장)	라목 「전자등록주식등」 제외	(4)(a) 「financial instruments」 제외	「investment contract」(*Howey*) 면 증권법 **안**; GENIUS (B)(iii) 「security」 제외	단서 「電子記録移転権利」 제외	(4)(a) 「specified investment cryptoasset」 제외	「한국·EU·영국·GENIUS법은 「증권이면 이 법 밖」이라고 적고, 일본은 「電子記録移転権利」라는 금융상품법의 개념을 단서로 끌어와 뺍니다. 미국의 증권법 자체는 거꾸로 **「투자계약」이면 이 법 안이라고** 하여, 같은 토큰이 거래의 모양에 따라 증권이 되기도 안 되기도 합니다」

기술 용어(장)	한국	EU	미국	일본	영국	어긋나는 지점(30장)
자동화된 시장 — 운영자 없는 AMM(15장)	—	—	GENIUS §2(7)이 유동성 풀 참여를 「digital asset service provider」에서 명시적으로 뺌	「不特定の者を相手方として購入及び売却を行うことができる」	—	「15장의 AMM과 17장의 MEV가 보인 「운영자 없는 시장」은 이 어느 이름에도 수범자가 없다는 점에서 관할의 빈자리이고 (28.1(e))」; 30장 질문 상자: 일본 조문의 「不特定の者を相手方として」는 상대방이 없는 AMM에서 어떻게 읽히는가
거래소 장부의 잔고 — 체인에 없는 코인(26장)	「전자적으로 거래 또는 이전될 수 있는 전자적 증표」인가	—	—	東京地裁: 「ブロックチェーン上に存在しないビットコインは…架空のビットコイン」(26장)	—	30장 질문 상자: 「거래소 장부 위의 잔고 — 체인에 없는 30만 BTC(26장) — 는 「이전될 수 있는 전자적 증표」입니까, 발행자(거래소)에 대한 채권입니까.」
「통화」라는 이름 자체(19·30장)	법은 가상자산을 「통화」·「화폐」라 부르지 않음(다목·사목은 「전자화폐」·「전자적 형태의 화폐」를 **제외**)	「currency」는 「official currency」(8)에만	IRS: 「virtual currency」라 부르되 「property」로 과세; 2023-34는 「it does not have legal tender status in any jurisdiction」 문장을 지움	「仮想通貨」→「暗号資産」(令和元年法律第28号)	—	「그 선택은 한 법역 안에서도 바뀌었습니다 — 일본은 「仮想通貨」를 「暗号資産」으로 고쳤고, 한국 대법원은 2018년에 「가상화폐」, 2021년에 「가상자산」이라 썼으며, 미국 국세청은 「법정통화인 법역이 없다」는 문장을 2023년에 지웠습니다.」

세 표를 겹쳐 읽으면 30장의 결론이 그대로 보입니다. 공통 뼈대는 「가치(또는 권리)의 디지털 표현」 + 「전자적 이전 가능성」이고, 갈라지는 자리는 제외 목록이며, 기술 용어 가운데 조문에 들어간 것은 이전 가능성 하나뿐입니다. 어느 열에 「—」가 많은지가 그 법역에서 어느 질문이 비어 있는지를 보여 줍니다.

출처

- 30장 2절(정의 조문 원문)·3절(제외 목록 표)·4절(관할)·5절(이름의 변경)·질문 상자 — 조문 원문·법률 번호·시행일·판결 서지는 전부 30장의 각주와 「이 장의 출처」에 있다. 이 부록은 새 조문·새 번역·새 자료를 더하지 않았다.
- 27.3 서베이 표(법역별 기관·절차)는 이 부록의 범위 밖이다 — 이름 뒤의 기관은 27장, 이름 자체는 30장과 이 부록.

부록

부록 D 연표 — 1976~2023 (클릭 연표 W11의 정지 판)

2장 1절의 표와 같은 항목·같은 서지로 만든 정지 판입니다(2장 표가 한 행에 묶은 사건은 여기서 날짜별로 풀었고, 장 칸은 그 사건을 원문으로 다루는 장 전부입니다). 화면본의 클릭 연표(W11)는 이 표의 데이터(`widgets/data.json` 의 `timeline`)로 그려지고, 사건을 누르면 해당 장으로 갑니다. 연도 표기 규칙은 2장 1절과 같습니다 — 학회명과 함께 쓰면 학회 연도, 서지로 쓰면 출판 연도; 웹 문서는 게시일. 확정하지 못한 날짜는 단정하지 않고 그 사실을 적었습니다. 2024년 이후 항목은 낡기 쉬우므로 부록 A에만 둡니다(A-32). 가격·시가총액 축은 없습니다.

1. 29개 이정표

날짜	이정표	1차 자료(이 책이 연 것)	장
1976-11	디피·헬먼, 「New Directions in Cryptography」 — 공개키 암호와 「서면 서명에 상당하는 것」	*IEEE Trans. Inform. Theory* IT-22(6): 644–654	5
1980-04	머클, 「Protocols for Public Key Cryptosystems」 — 공개키 목록을 인증하는 「트리 인증」(자세한 논의는 다른 원고 [13]으로 넘김)	*Proc. 1980 IEEE Symposium on Security and Privacy*, pp. 122–134 — 126–127쪽	6
1982-07	램포트·쇼스택·피즈, 「The Byzantine Generals Problem」	*ACM TOPLAS* 4(3): 382–401	1·8
1982 (학회) / 1983 (출판)	차움, 「Blind Signatures for Untraceable Payments」 — 은닉 서명 전자현금	*Advances in Cryptology: Proceedings of Crypto 82*, pp. 199–203 (Springer, 1983)	1·2
1987 (학회) / 1988 (출판)	머클, 「A Digital Signature Based on a Conventional Encryption Function」 — 트리 서명	*CRYPTO '87*, LNCS 293, pp. 369–378 (Springer, 1988)	6
1991	하버·스토네타, 「How To Time-Stamp a Digital Document」 — 타임스탬프 사슬	*J. Cryptology* 3(2): 99–111	4·7
1993	베이어·하버·스토네타, 「Improving the Efficiency and Reliability of Digital Time-Stamping」 — 사슬을 트리로	*Sequences II*, pp. 329–334 (Springer, 1993); 저자 조판본은 「March 1992」 표기	6·7
1997-03-28	백, 해시캐시 공지(cypherpunks) — 계산으로 비용을 매기는 우표	hashcash.org/papers/announce.txt; 2002년 논문은 「May 1997」로 적음(불일치는 2장 각주)	8
1998	다이, 「b-money」 — 계산으로 돈을 만들고 모두가 장부를 갖는 그림	weidai.com/bmoney.txt(문서에 날짜 없음; 1998은 백서 [1] 기재, 월·일 미기재)	1·8
2005-12-29	사보, 「Bit gold」 — 작업증명·타임스탬프·소유권 등기	unenumerated.blogspot.com/2005/12/bit-gold.html(Wayback 2006-03-29 스냅숏으로 게시일 확정; 현 화면 표시일 2008-12-27)	8
2008-10-31	나카모토, 「Bitcoin P2P e-cash paper」 — 백서 공개	cryptography@metzdowd.com 014810; 백서 PDF	1·7·8·9

날짜	이정표	1차 자료(이 책이 연 것)	장
2009-01-03	제네시스 블록(nTime 1231006505 = 18:15:05 UTC)	`data/genesis_block.hex`; Bitcoin Core v31.1 `chainparams.cpp`	12
2014 (초고 2013)	부테린, 이더리움 백서(초고 2013, 공식 판 2014)	ethereum.org 공식 PDF; Wayback 최초 캡처 2013-12-19(2장 각주)	13
2014-02-28	Mt.Gox 민사재생 신청(東京地裁 平成26年(再)第12号)	채무자 공지(일·영)	26
2014-04-24	Mt.Gox 파산절차 개시결정(東京地裁 平成26年(フ)第3830号)	결정 원문	26
2016-04-30 ~ 05-28	The DAO 토큰 매각	SEC Release No. 81207, 2쪽	14·22·26
2016-06-17	The DAO 공격(재귀 호출 결함)	이더리움 재단 공지 2016-06-17; SEC Release No. 81207, 9쪽	26
2016-07-20	이더리움 하드포크(블록 1,920,000, 「irregular state change」)	이더리움 재단 공지 2016-07-20; EIP-779; SEC Release No. 81207, 9쪽	26
2017-07-25	SEC, 「Report of Investigation … The DAO」 — 「DAO Tokens are securities」	Release No. 81207	26·27
2019-03-15	東京地裁, Mt.Gox 대표 형사 1심 판결	판결 전문(HTML)	26
2020-03	애덤스·진스마이스터·로빈슨, 「Uniswap v2 Core」 — 상수곱 자동화 시장	백서 PDF	15
2020-06-11	東京高裁, Mt.Gox 대표 형사 항소심 판결(항소 기각)	판결 PDF(courts.go.jp)	26
2021-08-05	이더리움 런던 포크(EIP-1559, 블록 12,965,000, 12:33:42 UTC)	블록 헤더(공개 RPC)·go-ethereum `london` `Block`	21
2021-11	비트코인 탭루트 활성화(최소 활성화 높이 709,632)	Bitcoin Core v31.1 `chainparams.cpp`(부록 A-8)	5·29
2022-05	Terra 붕괴	BIS AER 2022 Box A(기제); 판결·평결은 26장	24·26
2022-09-15 (추정)	이더리움 지분증명 전환(The Merge; TTD 도달 추정일)	consensus-specs `configs/mainnet.yaml`(부록 A-7)	10
2022-11-11	FTX 챕터 11 신청(「Petition Date」; West Realm Shires는 11-14)	Ray 선언서 ¶40	26
2023-02-03	영국 항소법원, *Tulip Trading Ltd v van der Laan* [2023] EWCA Civ 83	판결 PDF	27·28
2023-06-09	EU MiCA 관보 게재(OJ L 150; 적용 2024-12-30)	Cellar PDF	27·30

이 표의 날짜는 모두 각 담당 장의 각주가 가리키는 1차 자료에서 왔고, 이 부록은 새 자료를 더하지 않았습니다. 「추정」·「미기재」·「불일치」 표시는 그 장의 서술과 같습니다.

2. 클릭 연표 (화면본)

위 표를 시간 축에 놓은 그림입니다. 화면본에서는 항목을 눌러 출처를 보고, 「n장 →」 단추로 그 사건을 다루는 장으로 갑니다. 인쇄본에는 처음 상태의 정지 그림을 실었습니다.



그림 D-1. 클릭 연표 1976~2023 — 위 표의 29개 이정표를 시간 축에 놓은 것. 항목을 누르면 출처가 보이고, 「n장 →」 단추로 그 사건을 원문으로 다루는 장으로 간다. 날짜는 확인된 정밀도(연·월·일)대로 적고 추정일은 따로 표시한다. 가격 축은 없다. — 화면본에서는 이 자리에서 값을 바꿔 볼 수 있다(화면본 부록 D 2절).

출처

- 2장 「이 장의 출처」와 각 담당 장(1·5·6·7·8·10·12·13·14·15·21·22·24·26·27·28·30장)의 각주. 새 자료 없음.
- `widgets/data.json` `timeline` (기준일 2026-10-25) — 이 표와 같은 항목·같은 출처.

부록

부록 E 수학 보충

이 부록은 본문의 [심화] 상자가 「부록 E로 보낸다」고 적은 것만 씁니다. 본문 전체를 훑어 그 자리를 모았더니 하나였습니다. 설계 단계에서 후보로 적어 두었던 다른 주제(모듈러 산술과 유한체, 타원곡선 군, 포아송 과정과 랜덤워크, 2×2 게임과 내시 균형, 단위근과 폭발 검정)는 집필하면서 각 장의 [심화] 상자가 자기 안에서 다 풀었습니다 — 5장(타원곡선과 점 덧셈), 9장(음이항·정확식), 23장(닫힌 식과 임계 점유율), 25장(이중 재귀·와일드 부트스트랩). 그래서 여기에 되풀이하지 않습니다.

대응표 — 어느 상자가 여기를 가리키는가

장·상자	가리키는 문장	이 부록의 절
4장 「[심화] SHA-256의 속 — 압축 함수를 이어 붙이는 구조」	「압축 함수를 사슬처럼 이어 붙이는 이 설계 원리의 이름과 자세한 전개는 부록 E로 보냅니다.」	E.1
18장 4절의 [중급] 문단(증명 체계)	「증명의 수학은 이 책의 범위 밖입니다(부록 E도 다루지 않습니다).」 — 요구가 아니라 **범위 밖 선언**. 이 부록도 다루지 않는다	—

E.1 머클–담고르 구성 — 압축 함수를 이어 붙이기

[심화] 4장은 FIPS 180-4가 SHA-256을 두 단계 — 전처리(채움, 블록 나누기, 초기값)와 해시 계산(블록마다 상태값을 갱신 — 이 부록이 압축 함수라 부르는 단계; FIPS 180-4 자체는 이 낱말을 쓰지 않는다) — 로 적는다는 것을 보였고,[1] NB04-5에서 그 상태값이 블록에서 블록으로 넘어가는 중간값을 찍었습니다. 그 설계 원리에는 이름이 있습니다. **머클–담고르 구성(Merkle–Damgård construction)** 입니다. 이름의 두 사람은 1989년 같은 학회(CRYPTO '89)에 각각 논문을 냈고 — 담고르의 「A design principle for hash functions」와 머클의 「One-way hash functions and DES」 — 하버·스토네타(1991)의 참고문헌 [5]·[16]이 그 둘을 적습니다.[2] 이 책은 두 논문의 원문을 열지 않았으므로 구성 자체는 이 책이 원문을 연 교과서 초고의 서술로 적습니다. 나라야난 외는 이렇게 씁니다. 「우리는 해시 함수가 임의 길이의 입력에서 동작하기를 요구한다. 다행히 **고정 길이** 입력에서 동작하는 해시 함수를 만들 수 있기만 하면, 그것을 임의 길이 입력에서 동작하는 해시 함수로 바꾸는 일반적 방법이 있다. 그것을 머클–담고르 변환이라 부른다. SHA-256은 이 방법을 쓰는 널리 쓰이는 해시 함수 가운데 하나다. 흔한 용어로 바탕이 되는 고정 길이 충돌저항 해시 함수를 압축 함수라 부른다. 바탕 압축 함수가 충돌저항이면 전체 해시 함수도 충돌저항임이 증명되어 있다.」[3]

[심화] 구성을 기호로 적으면 이렇습니다. 압축 함수 $f : \{0,1\}^n \times \{0,1\}^b \to \{0,1\}^n$ 이 있습니다 — n 비트 상태와 b 비트 블록을 받아 n 비트 상태를 냅니다. SHA-256에서는 $n = 256$, $b = 512$ 입니다. 메시지 M 에 채움 비트와 길이를 붙여 b 비트 블록 $M^{(1)}, \ldots, M^{(N)}$ 으로 자르고, 정해진 초기값 $H^{(0)}$ 에서 시작해

$$H^{(i)} = f\left(H^{(i-1)},\ M^{(i)}\right), \qquad i = 1, \ldots, N$$

을 되풀이한 뒤 마지막 $H^{(N)}$ 을 출력으로 삼습니다. 기호 풀이: $H^{(i)}$ 는 i 번째 블록까지 먹은 뒤의 상태값, f 는 4장 NB04-5의 64라운드 압축 함수입니다. 수치 예: 4장에서 두 블록짜리 메시지를 넣었을 때 NB04-5가

찍은 $H^{(1)}$ 과 $H^{(2)}$ 가 바로 이 식의 두 단계이고, $H^{(2)}$ 가 `hashlib.sha256` 의 값과 같았습니다.

[심화] 왜 충돌저항이 올라가는지는 짧은 역추적 논증으로 보입니다. 서로 다른 메시지 $M \neq M'$ 이 같은 출력을 냈다고 합시다. 길이가 다르면 마지막 블록(길이를 담은 블록)이 다르므로 마지막 단계 $f(H^{(N-1)}, M^{(N)}) = f(H'^{(N'-1)}, M'^{(N')})$ 에서 입력이 다른데 출력이 같은 압축 함수의 충돌이 바로 나옵니다. 길이가 같으면 블록 수가 같고, 마지막 단계부터 거슬러 올라가며 처음으로 입력이 갈리는 단계를 찾습니다 — 어느 단계에서든 $(H^{(i-1)}, M^{(i)}) \neq (H'^{(i-1)}, M'^{(i)})$ 인데 $H^{(i)} = H'^{(i)}$ 이면 그것이 압축 함수의 충돌입니다. 메시지가 다르므로 그런 단계는 반드시 있습니다. 그래서 「전체 함수의 충돌 하나」에서 「압축 함수의 충돌 하나」가 기계적으로 나오고, 압축 함수가 충돌저항이면 전체도 그렇습니다. 길이를 채움에 넣는 것(FIPS 180-4의 전처리가 메시지 길이 64비트를 붙이는 이유)이 이 논증의 첫 갈래를 받칩니다.

[심화] 나라야난 외는 한 가지를 더 짚습니다. 「블록체인 구성이 앞 절에서 본 머클–담고르 구성과 닮았다는 것을 눈치챘을 것이다. 실제로 둘은 꽤 닮았고 같은 안전성 논증이 둘 다에 적용된다.」[4] 7장의 사슬 — 각 블록이 직전 블록 헤더의 해시를 담는다 — 은 「상태값이 다음 단계로 넘어가는」 같은 모양이고, 나라야난 외가 말하는 것은 그 모양 덕에 같은 안전성 논증(끝 해시의 충돌이 어딘가의 블록 충돌로 환원된다)이 둘 다에 적용된다는 점까지입니다. 중간 블록을 바꾸면 그 뒤가 어긋난다는 7장의 관찰은 그 논증의 결과이지 논증 자체는 아닙니다. 다만 7장이 보였듯 그 모양만으로는 「다시 꿰기」가 공짜이고, 비싸게 만드는 것은 8장의 작업증명입니다.

출처

- NIST FIPS 180-4 (2015) §1 — 4장과 같은 자리.
- Haber & Stornetta (1991) 참고문헌 [5]·[16] — 담고르(CRYPTO '89; 1990)·머클(CRYPTO '89; 1990)의 서지(LNCS 435, Springer 1990). 두 원문은 미확보.
- Narayanan 외, *Bitcoin and Cryptocurrency Technologies*, 저자 공개 초고(2016-02-09), 1장 — 머클–담고르 변환의 서술과 블록체인과의 유사성.
- 역추적 논증은 위 교과서의 「증명되어 있다」는 서술을 이 책이 풀어 쓴 것이다. 코드 없음.

각주

1. NIST, *Secure Hash Standard (SHS)*, FIPS PUB 180-4 (August 2015), §1 Introduction(3쪽). 4장 각주 8의 원문과 같다.

2. Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *Journal of Cryptology* 3(2): 99–111 (1991), 110~111쪽 References. [5]: "I. Damgård. A design principle for hash functions. In Advances in Cryptology—Crypto '89 (ed. G. Brassard), pp. 416-427. Lecture Notes in Computer Science, vol. 435, Springer-Verlag, Berlin, 1990." [16]: "R. C. Merkle. One-way hash functions and DES. In Advances in Cryptology—Crypto '89 (ed. G. Brassard), pp. 428-446. Lecture Notes in Computer Science, vol. 435, Springer-Verlag, Berlin, 1990." 두 논문의 원문은 이 책의 원자료에 없다(1부 유보 항목). 4장 각주 9는 같은 참고문헌의 4와 [5]를 적는다.

3. Arvind Narayanan, Joseph Bonneau, Edward Felten, Andrew Miller & Steven Goldfeder, *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction* (Princeton University Press, 2016), 저자 공개 초고(「Draft — Feb 9, 2016」), 1장. 원문: "Recall that we require that our hash functions work on inputs of arbitrary length. Luckily, as long as we can build a hash function that works on fixed-length inputs, there's a generic method to convert it into a hash function that works on arbitrary-length inputs. It's called the Merkle-Damgard transform. SHA-256 is one of a number of commonly used hash functions that make use of this method. In common terminology, the underlying fixed-length collision-resistant hash function is called the compression function. It has been proven that if the underlying compression function is collision resistant,

then the overall hash function is collision resistant as well." 출판본 쪽은 미대조.

4. Narayanan 외 (2016 초고), 1장. 원문: "You may have noticed that the block chain construction is similar to the Merkle-Damgard construction that we saw in the previous section. Indeed, they are quite similar, and the same security argument applies to both of them."

부록

부록 F 용어집 · 인터랙티브 그림 목록 · 노트북 목록 · 판권

F.1 용어집

본문에서 첫 등장 때 영문을 병기한 낱말을 모았습니다. 뜻은 이 책이 그 낱말을 쓰는 뜻이고, 자세한 정의는 괄호의 장에 있습니다. 법령 정의어는 부록 C에 따로 있습니다.

용어	영문	이 책의 뜻	장
이중지불	double spending	같은 디지털 돈을 두 번 쓰는 것. 디지털 현금의 핵심 난제	1
신뢰받는 제3자	trusted third party	장부를 쓰고 「먼저」를 정하는 한 사람(은행·조폐소)	1
시빌 공격	Sybil attack	한 공격자가 가짜 참가자를 많이 만들어 다수를 차지하는 것	1
비잔틴 장군 문제	Byzantine Generals Problem	배신자가 섞인 집단이 합의에 이르는 조건을 묻는 문제	1·8
은닉 서명	blind signature	서명자가 내용을 보지 않고 서명하는 방식 — 차움의 전자현금	1·2
해시캐시	Hashcash	계산으로 비용을 매기고 해시 한 번으로 검증하는 우표	2·8
해시 함수	(cryptographic) hash function	임의 길이 입력을 고정 길이 지문으로 바꾸는 일방향 함수	4
역상 저항·제2역상 저항·충돌 저항	preimage / second-preimage / collision resistance	해시 함수의 세 성질	4
눈사태 효과	avalanche effect	입력 한 비트가 바뀌면 출력 절반쯤이 바뀌는 성질	4
이중 SHA-256	double SHA-256	SHA-256을 두 번 적용 — 비트코인의 블록 해시·거래 식별자	4·12
머클–담고르 구성	Merkle–Damgård construction	압축 함수를 사슬처럼 이어 임의 길이 입력을 처리하는 설계	4·부록 E
공개키·개인키	public key / private key	열쇠 둘로 권한을 나누는 암호 — 공개키로 검증, 개인키로 서명	5
디지털 서명	digital signature	개인키로 만들고 공개키로 검증하는 승인 표시	5
타원곡선 이산로그 문제	ECDLP	$Q = [k]G$에서 k를 찾는 문제 — 서명 안전의 바탕	5·29
secp256k1	—	비트코인·이더리움이 쓰는 타원곡선	5

용어	영문	이 책의 뜻	장
ECDSA	Elliptic Curve Digital Signature Algorithm	비트코인·이더리움 계정의 서명 알고리듬	5
슈노르 서명	Schnorr signature	탭루트(2021)부터 비트코인이 함께 쓰는 서명(BIP-340)	5
거래 가변성	transaction malleability	승인된 거래의 내용과 유효성은 그대로 둔 채 표현을 바꿔 거래 식별자(txid)를 바꿀 수 있던 성질 — 서명 (r, s) 를 $(r, n - s)$ 로 바꾸는 것이 한 갈래	5·26
주소	address	대개 공개키(또는 스크립트)의 해시를 부호화한 것(P2TR는 공개키 자체)	5
P2PK · P2PKH · P2SH · P2WPKH · P2WSH · P2TR · P2MS	—	비트코인 출력(잠금 스크립트)의 표준 유형	5·12·29
P2MR	Pay-to-Merkle-Root	키 경로를 뗀 탭루트 — BIP-360이 제안한 출력 유형(Draft)	29
머클 트리·머클 루트·머클 증명	Merkle tree / root / proof	거래들을 해시로 묶어 뿌리 하나로 보증하고, 가지 몇 개로 포함을 증명	6
블록 헤더	block header	80바이트 — 버전·이전 해시·머클 루트·시각·목표·논스	7·12
논스	nonce	작업증명에서 바꿔 가며 해시하는 임의의 수(계정 모형에서는 거래 순번)	8·11
목표값·nBits·난이도	target / nBits / difficulty	해시가 넘지 말아야 할 값과 그 압축 표현, 그 역수 척도	8
작업증명	proof of work	목표값 아래의 해시를 찾는 계산 — 「CPU 하나당 한 표」	8
해시율	hash rate	네트워크가 초당 시도하는 해시 수(난이도에서 추정)	8·20
난이도 조정	difficulty adjustment	2,016블록마다 블록 간격을 10분으로 되돌리는 규칙	8
채굴·채굴자	mining / miner	작업증명을 풀어 블록을 만드는 일과 사람	8
코인베이스	coinbase	블록의 첫 거래 — 새 코인과 수수료를 채굴자에게	12·20
블록 보조금·반감	block subsidy / halving	블록당 새 코인, 210,000블록마다 절반	20
가장 긴 체인·누적 작업	longest chain / most accumulated work	정본 사슬을 고르는 규칙	7·8
포크(소프트·하드)	soft fork / hard fork	옛 노드도 따르는 규칙 변경(규칙을 좁히는 일) / 옛 노드와 갈라서는 규칙 변경	7·23·26·29
확인 수	confirmations	거래가 들어 있는 블록을 포함해 그 위에 쌓인 블록의 수	9
확률적 최종성	probabilistic finality	되돌릴 확률이 작아질 뿐 「완료」의 순간이 없는 성질	9
도박꾼의 파산	gambler's ruin	백서 §11이 공격자의 추격 확률에 쓴 확률론 모형	9
이기적 채굴	selfish mining	블록을 숨겼다 내놓아 몫을 늘리는 전략	23
지분증명	proof of stake	담보(스테이크)와 투표로 블록을 만드는 합의	10
검증자	validator	지분증명에서 담보를 걸고 블록을 제안·증명하는 참가자	10
슬래싱	slashing	규칙 위반 검증자의 담보를 깎는 처벌	10

용어	영문	이 책의 뜻	장
UTXO	unspent transaction output	아직 쓰이지 않은 거래 출력 — 비트코인의 상태 단위	11
계정 모형	account model	주소마다 잔고와 논스를 두는 상태 모형 — 이더리움	11
이더리움 가상머신	EVM	모든 노드가 같은 프로그램을 돌리는 상태 기계	13
가스·가스 한도	gas / gas limit	명령마다 매긴 계산 값과 거래가 미리 사는 상한	13
기본 수수료·우선 수수료·소각	base fee / priority fee / burning	EIP-1559의 수수료 두 몫 — 전자는 소각, 후자는 검증자에게	13·21
스마트계약	smart contract	조건이 맞을 때 값을 풀어 주는 체인 위의 프로그램	13
ERC-20 · ERC-721 · ERC-165	—	토큰·대체불가 토큰·인터페이스 식별의 함수 이름 약속	14
대체불가 토큰	NFT, non-fungible token	(계약 주소, 번호) 쌍으로 식별되는 토큰	14
자동화 시장조성자	AMM, automated market maker	호가창 없이 공식(상수곱)으로 가격을 정하는 계약	15
상수곱·슬리피지·비영구 손실	constant product / slippage / impermanent loss	AMM의 식, 주문 크기에 따른 가격 악화, 유동성 공급자의 손익 구조	15
유동성 공급자	liquidity provider	풀에 두 토큰을 넣고 수수료를 받는 참가자	15
오라클	oracle	체인 밖 사실을 체인 안으로 들여오는 장치	16
최대 추출 가능 가치	MEV, maximum extractable value(옛 이름 miner extractable value)	거래 순서·포함·배제를 정하는 권한에서 뽑는 가치	17
선행매매·후행매매·샌드위치	front-running / back-running / sandwich	순서를 이용한 거래 전략	17
제안자–빌더 분리	PBS, proposer–builder separation	블록을 만드는 사람과 제안하는 사람을 나누는 설계	17
멤풀	mempool	블록에 들어가기 전 거래가 기다리는 대기열	17·29
롤업(낙관적·영지식)	optimistic / zero-knowledge rollup	실행을 떼어 내고 데이터를 본체에 맡기는 2층 설계	18
시퀀서	sequencer	롤업에서 순서를 정하는 사람	18
데이터 가용성·블롭	data availability / blob	데이터가 있음을 보장하는 문제와 그 임시 저장 단위	18
화폐의 세 기능	functions of money	교환 매개·계산 단위·가치 저장	19
마팅게일	martingale	조건부 기댓값이 현재값과 같은 과정 — 19장의 가격 명제	19
합리적 버블	rational bubble	무재정거래 조건을 어기지 않고 기본가치에서 벗어난 가격	19·25

용어	영문	이 책의 뜻	장
보안 예산	security budget	정직한 참가자가 보안에 쓰는 흐름 비용 — 보상이 그 재원	20
EIP-1559	—	2021년 런던 포크의 수수료 시장 설계	21
거버넌스 토큰·DAO	governance token / decentralized autonomous organization	투표권을 담은 토큰과 코드로 적힌 조직	22
스테이블코인	stablecoin	상환 약속이 붙은 토큰 — 법정화폐 담보·암호자산 초과담보·알고리듬형·합성형	24
뱅크런·담보율	bank run / collateral ratio	상환이 몰리는 사태와 지급능력의 수	24
폭발적 근·GSADF	explosive root / generalized sup ADF	버블 구간을 찾는 계량검정	25
논리 큐비트·물리 큐비트	logical / physical qubit	오류정정된 추상 큐비트 / 하드웨어 소자	29
쇼어·그로버 알고리듬	Shor's / Grover's algorithm	이산로그를 다항 시간에 / 탐색을 제곱근으로	29
양자내성 암호	PQC, post-quantum cryptography	양자 컴퓨터에도 안전하다고 믿어지는 암호 — FIPS 203·204·205	29
ML-DSA · SLH-DSA	—	격자 기반 서명 표준(FIPS 204) · 해시 기반 서명 표준(FIPS 205)	29
장기 노출·단기 노출(at-rest·on-spend)	long / short exposure attack	드러난 공개키에 대한 공격 / 지출 중 멤풀의 공개키에 대한 공격	29
결제 최종성	settlement finality	규칙이 정한 취소 불가 시점. 비트코인 작업증명에는 확인 수만으로 재편 확률이 0이 되는 시점이 없고(9장), 지분증명의 「최종화」는 프로토콜 규칙상의 최종성이지 법적 결제 최종성과 같은 것은 아니다(10장)	9·10·28.1(c)
가상자산 · crypto-asset · digital asset · 暗号資産 · cryptoasset	—	다섯 법역의 정의어	30·부록 C

F.2 인터랙티브 그림(위젯) 목록

화면본의 위젯 열두 개입니다. 처음 상태는 본문 예제와 같고, 변수는 전부 구조 변수입니다(가격·수익률 없음). PDF에서는 처음 상태의 정지 그림이 들어갑니다.

ID	이름	장 • 자리	변수	처음 상태
W1	해시 눈사태	4장, NB04-2 뒤	입력 문자열, 뒤집을 비트	「hello」에서 한 글자 변경
W4	머클 증명	6장, NB06-1 뒤	거래 수 n, 증명할 잎	$n = 8$, 잎 5
W3	체인 변조 시연	7장, NB07-1 뒤	변조할 블록 번호·내용	5블록 체인, 3번 블록 변조
W2	손 채굴기	8장, NB08-2 뒤	난이도 비트 수	16비트
W5	이중지불 확률	9장, NB09-2 뒤	q, z	$q = 0.1, z = 0 \ldots 10$
W12	블록 헤더 해부기	12장, NB12-1 뒤	필드 선택, 논스	제네시스 블록
W7	AMM 슬리피지	15장, NB15-1 뒤	풀 크기 x, y, 주문 Δx	1,000 : 1,000, 주문 100
W10	샌드위치 공격	17장, NB17-1 뒤	풀 크기, 피해 주문, 공격자 투입, 슬리피지 허용치	15장의 풀, 피해자 100·공격자 100
W6	51% 공격 비용	20장, NB20-3 뒤	점유율 q, 해시 단가, 지속 시간	점유율 60%, $k + e = 12$
W9	수수료 시장	21장, NB21-1 뒤	블록별 가스 사용량	한도 60,000,000 · 목표 30,000,000
W8	스테이블코인 뱅크런	24장, NB24-2 뒤	상환 요구 비율, 준비자산 구성, 담보율	유형 B′, 요구 40%
W11	클릭 연표	2장 · 부록 D	클릭	1976~2023

F.3 노트북 목록

본문의 파이썬 코드를 그대로 옮긴 Colab 노트북입니다. 출력은 엮은이가 Python 3.13(conda 환경 `bcbook`)에서 실행한 결과를 그대로 실었고, 같은 실행에서 본문의 「출력」 블록이 나왔습니다. 설치 셀은 없고 외부 API를 부르지 않습니다. 동봉 데이터가 필요한 셀은 `data/` 의 스냅샷을 읽되 파일이 없으면 같은 값의 상수로 대신합니다. 모든 코드는 공부용입니다 — 실제 자산·실제 개인키·실제 거래에 쓰지 않습니다.

노트북	장	셀	내용	외부 패키지 • 데이터	열기
NB00	3	1	환경 점검 — 파이썬 판, `hashlib` · `secrets` · `struct` · `json` · `decimal` 동작	없음	Colab
NB04	4	5	SHA-256 지문, 눈사태, 검사합 위조 vs 해시, 생일 역설, 손으로 짠 SHA-256	없음	Colab

노트북	장	셀	내용	외부 패키지 • 데이터	열기
NB05	5	4	장난감 곡선($p = 97$) 군 연산, 장난감 ECDSA·가변성, k 재사용 키 복원, secp256k1 매개변수 확인	없음	Colab
NB06	6	3	머클 트리·증명·검증, 증명 길이 표, 비트코인식 트리와 CVE-2012-2459	없음	Colab
NB07	7	2	장난감 체인 변조·재연결, 두 일관된 체인	없음	Colab
NB08	8	4	손 채굴기, 시도 횟수 분포, nBits → 목표값 → 난이도, 조정 규칙 시뮬레이션	없음	Colab
NB09	9	5	백서 식·표 재현, $P < 0.1\%$ 표, 정확식(음이항 유한합), 몬테카를로, 해시율 불변성	없음	Colab
NB10	10	2	담보 가중 선택, 슬래싱 즉시·상관 벌금	없음	Colab
NB11	11	2	UTXO 상태 전이, 계정 상태 전이	없음	Colab
NB12	12	3	제네시스 헤더 파싱, 이중 SHA-256 대조, 코인베이스 거래 해부	`data/genesis_header.hex` · `genesis_block.hex`	Colab
NB13	13	3	장난감 EVM(황서 부록 G 요금표), 거래 적용, 무한 루프와 가스 소진	없음	Colab
NB14	14	3	ERC-20 장부 흉내와 approve 경쟁 조건, ERC-721 장부 흉내, Keccak-256 직접 구현과 식별자 재계산	없음	Colab
NB15	15	2	상수곱 풀(스왑·유동성·슬리피지 표), 비영구 손실 표와 수수료 성장 표	없음	Colab
NB17	17	2	샌드위치 시뮬레이션, 피해자 슬리피지 허용치별 공격자 최적 투입	없음	Colab
NB19	19	2	개럿–월리스 붕괴 확률, 실링–울리히 가격 경로 구성과 마팅게일 확인	없음	Colab
NB20	20	3	비트코인 보조금 일정, 부디시 명제 3·표 II 재현, 공격 비용 계산기(단위는 블록 보상)	`data/hashrate_2026-10-25.json`	Colab
NB21	21	3	EIP-1559 정수 갱신식과 런던 포크 블록 검증, 1차 가격 경매 vs 1559 비교, EIP-4844 블롭 수수료 경로	없음	Colab
NB23	23	2	이기적 채굴 상태기계 몬테카를로, 2×2 이득표와 문턱	없음	Colab
NB24	24	3	다이아몬드–딥비그 두 균형, 준비자산 구성 × 상환 요구 → 디페그 되먹임, 알고리듬형 반사 나선	없음	Colab
NB25	25	3	합성 자료 SADF·GSADF, 몬테카를로 임계값·BSADF 날짜 찍기, 시변 변동성과 와일드 부트스트랩	`numpy` 2.5.4 — 이 책의 유일한 외부 패키지(이유는 25장)	Colab
NB29	29	3	쇼어 알고리듬 자원 일반식과 네 편 비교, 스냅샷의 유형별 노출 수량 재계산, 잠금 스크립트 유형 분류기	`data/exposed_output_types_2026-10-25.json`	Colab

1·2·16·18·22·26·27·28·30장과 부록은 설계대로 노트북이 없습니다(개념·서베이·조문 장). 13·14장의 Solidity 예시는 읽기용이며 노트북에 넣지 않았습니다.

F.4 판권

『법률가를 위한 블록체인 — 기술과 경제학으로 읽기』

- 엮은이: 권보원 (대구지방법원 부장판사) / Bowon Kwon
- 판: 1.0판 · 2026. 11. 14. 발행 (수치 기준일 2026-10-25)
- 저장소: AI-Assisted-Handbooks/blockchain-for-lawyers (https://github.com/BowonKwon/AI-Assisted-Handbooks)

- 라이선스: 이 책의 글과 위젯 코드는 **Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0)** 으로 배포합니다. https://creativecommons.org/licenses/by-nc/4.0/ — 출처를 밝히면 비영리 목적으로 복제·수정·재배포할 수 있습니다. 각주에 인용한 원문의 저작권은 각 저작권자에게 있습니다.

- 동봉 데이터(`data/`)의 출처·기준일·산식은 데이터 폴더의 README에 있습니다. 블록 데이터는 공개 노드·탐색기의 원시 응답이고, 노출 출력 유형 집계는 제3자의 2차 집계입니다(29장).
- 13장의 Solidity 읽기용 예시는 Solidity 공식 문서 첫 예제의 구조를 따라 이 책이 새로 쓴 코드이며, 다른 코드와 같이 CC BY-NC 4.0을 따릅니다. Solidity 문서의 예제 코드(GPL-3.0)는 싣지 않았습니다.
- 이 책은 엮은이 개인의 견해이며 소속 기관과 무관합니다.
- AI 활용: 이 책은 엮은이가 생성형 AI(Claude)와 함께 만들었습니다. 장마다 원문과 대조하는 검증을 여러 차례 거쳤지만 틀린 서술이 남아 있을 수 있습니다. 수치와 인용은 원문으로 다시 확인해 주십시오. 이 책은 앞으로도 계속 고쳐 나가겠습니다.
- 틀린 곳: 원문과 다른 서술을 찾으면 저장소의 Issues(https://github.com/BowonKwon/AI-Assisted-Handbooks/issues)로 알려 주십시오. 고친 내용은 판과 날짜를 올려 반영합니다.

고지

이 책은 학술 자습서입니다. 블록체인의 기술과 경제학을 공부하기 위해 썼고, 특정 암호자산·토큰·금융상품을 사거나 팔거나 보유하라고 권하지 않습니다. 이 책의 어떤 문장도 투자 권유, 투자 자문, 가격 전망으로 읽혀서는 안 됩니다.

이 책에 나오는 거래소·프로젝트·토큰의 이름은 확정된 역사적 사실을 서술하기 위해서만 썼으며, 추천이나 비교 평가가 아닙니다. 수치는 모두 기준일과 출처를 붙였고, 그 뒤 바뀌었을 수 있습니다. 코드는 원리를 보이기 위한 것이며 실제 자산을 다루도록 만들지 않았습니다. 이 책의 코드와 설명을 실제 자산 관리에 쓰지 마십시오.

엮은이는 현직 법관이지만 이 책은 엮은이 개인의 학습 기록이며, 소속 기관의 견해와 무관합니다. 계속 중인 사건이나 계류 중인 법안에 관해서는 사실만 적고 평가하지 않았습니다. 이 책은 법률 자문이 아닙니다.

출처

- F.1의 뜻풀이는 각 장의 본문에서 왔다. 새 정의를 더하지 않았다.
- F.2는 각 장의 위젯 삽입 자리에서, F.3은 노트북 폴더의 목록에서 옮겼다.
- F.4의 엮은이 표기·소속·라이선스·소속 무관 문장·고지문은 엮은이가 정한 것이다. 고지문은 이 판권면이 전문이고, 3장 머리와 26장 첫머리에 짧은 판을 두었다. AI 활용 고지와 「틀린 곳」 문장은 같은 저장소의 다른 자습서와 같은 문안이다.